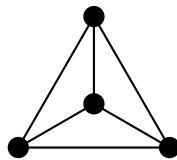


Void

Conditions of Formulability

A Formal Kernel for a Theory of Formulability

Johannes Michael Wielsch



Built with AI
Machine-verified in Agda

May 2026
DOI: 10.5281/zenodo.19491035

For Julia, Lara, Lia, and Lukas.

May you always question, and may the questions be beautiful.

Abstract

This volume begins with a licensing problem. A formal system may say that there are two positions. What inside the system licenses that claim as two, rather than one position named twice or a larger carrier with unmentioned points? *Void* names the pre-formal boundary at which this question arises. It is not an empty set, not an empty type, and not a first object; it is the absence of the formal field in which such objects could already be stated.

The entry pressure is Leibniz's principle of the identity of indiscernibles, used here in a modest form: if two alleged positions cannot be distinguished by anything expressible in the system, the system has no internal ground for holding them apart as two. Leibniz does not prove the project, and the principle is not itself an Agda theorem. It supplies the collapse pressure. *Void and Form* asks what explicit data are needed to stabilize two positions against that collapse in intensional Martin-Löf type theory as checked by Agda.

The formal answer is the record *Distinction*. It contains a carrier *S*, two distinguished positions ℓ and r , a separation witness $\ell \neq r$, and a cover showing that every carrier point is one of those two positions. The separation field is anti-collapse data. The cover field is exhaustion, or anti-surplus data. Two named points alone do not define a two-point structure in MLTT: without separation, the names may collapse; without cover, further points may remain in the carrier.

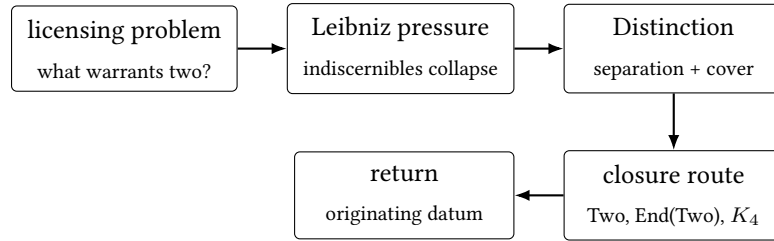
Given the *Distinction* data, the checked route is conditional and precise. Every distinction has normal form *Two*; the endomorphism space $\text{Two} \rightarrow \text{Two}$ has exactly four cases, the two constants, identity, and swap; under the stated representation conditions of case separation, complete realisation, and no surplus, the corresponding simple graph closure is K_4 ; and the closure is read back to the originating *Distinction* datum. The broader theory of formulability is the architectural reading of this dependency chain, not an additional Agda theorem.

The remainder of the volume keeps that staging. The early chapters make the licensing problem and the Leibniz pressure explicit. The Agda development then checks the formal consequences of the supplied record under `--safe` and `--without-K`. Later arithmetic, categorical, spectral, and interpretive material is

downstream of this threshold. It does not license the threshold from behind.

Reading Key

The route of the book is short at its core:



The first movement is not a theorem. It names the pressure that any formal theory already lives under when it uses distinguishable positions. The second movement is the formal grant: the record fields that make the claim of two explicit. From that point onward, the mathematical claims are checked terms.

Licensing problem. The book does not begin from K_4 , from a completed truth-value object, or from an interpretive reading. It begins from the question of what licenses a system's claim that two positions are present.

Leibniz pressure. If nothing expressible distinguishes the two alleged positions, the system has no internal ground for holding them apart. This pressure motivates the need for explicit anti-collapse data.

Formal threshold. A Distinction supplies the data used by the file: a carrier, two distinguished positions, a separation witness, and a cover. Separation prevents collapse. Cover prevents hidden surplus.

Checked kernel. Given that record, Agda checks normal form Two, four-case endomorphism classification, K_4 closure under the representation contract, uniqueness up to the stated equivalence, and return to the originating datum.

Downstream structure. The later internal build of arithmetic, categorical interfaces, spectral statements, and interpretive material belongs after the threshold. Those layers may use the kernel; they do not serve as evidence for it.

The proposed theory of formulability is the reading of this ordered route. If the pre-formal pressure is ignored, the code loses its reason for beginning with distinction. If the checked chain fails, the reading loses its formal spine. If later interpretation is treated as premise, the claim overreaches. The project is strongest when these statuses remain visibly separated.

Placement

The entry pressure is Leibnizian, but not as an authority argument. The identity of indiscernibles supplies a clear logical irritation: two positions cannot be maintained as two when no expressible difference keeps them apart. The present file asks how that pressure is represented as data inside MLTT/Agda.

Constructive type theory supplies the execution environment. Brouwer, Heyting, Curry, Howard, Martin-Löf, and Voevodsky belong to the background because this volume uses witnesses, proof terms, and the `--without-K` discipline rather than informal assertion. Type theory is not the origin of the distinction; it is the chosen formal language in which the consequences of the displayed data are checked.

The finite closure has a graph-theoretic face. Euler, Cayley, and Birkhoff belong to the background of counting, symmetry, and canonical representation. In this file, the relevant statement is local: under the representation contract for the four endomorphism cases of `Two`, the four-case carrier has complete simple adjacency and no room for either identification or surplus.

Lawvere and Tarski belong to the later semantic and categorical orientation: structures are studied by the maps and interpretations they admit. Those perspectives become useful only after the entry datum and the finite closure are in place.

What is new here is the execution of a precise chain. The book makes the dependency from licensing problem, to explicit distinction data, to normal form, to four-case closure, to return datum visible enough that the formal segment can be checked rather than merely asserted.

The Licensing Threshold

Chapter 1

The Licensing Problem

What The Question Asks

A formal system may say that there are two positions. The primitive question of this volume is what licenses that statement inside the system. Are there really two positions, or only one position named twice? Are there exactly two, or a carrier with further points that have not been accounted for?

The book begins here because the question is smaller and sharper than a claim about origins. It does not begin with K_4 , with a truth-value object, or with a later interpretation. It begins with the local licensing problem of two.

The word *Void* names the boundary before a formal carrier has been supplied. It is not a hidden object, not an empty type, and not an empty set. It marks the absence of the domain in which such objects could already be introduced. Once a carrier, positions, equality, and cover data are available, the formal development can begin. Before that point, the problem is to say what must be supplied.

The Boundary Before Reading

A reading already distinguishes what is read from the reading by which it is available. A judgement distinguishes the term, the type, and the act of judging. An equation distinguishes its sides even when it proves them equal. A map distinguishes source and target. None of these operations creates the first availability of distinguishable positions; each uses it.

The boundary is therefore not a theorem inside the later calculus. It is the point at which the later calculus receives the data it needs in order to operate. The first checkable step is not the word *Void*. The first checkable step is the explicit record that answers the licensing problem.

Chapter 2

Leibniz Pressure

Indiscernibles Collapse

The entry pressure is the identity of indiscernibles. In the form needed here, it says: if two alleged positions cannot be distinguished by anything expressible in the system, the system has no internal ground for holding them apart as two.

This is not an appeal to Leibniz as authority, and it is not yet an Agda theorem. It is the collapse pressure that makes the formal datum necessary. To stabilize two positions, a system must carry data that prevents their collapse. To stabilize exactly two positions, it must also carry data that prevents hidden surplus.

Later, after a carrier and equality have been fixed, the file contains a small Agda term named `leibniz-collapse`. That term is one precise execution of the pressure: if every predicate true of one element transfers to another, equality follows. Agda does not create the thought that indiscernibles collapse. It checks one instance of it after the formal threshold has been crossed.

The Status Of The Opening Claim

The opening claim is deliberately modest in its formal status and severe in its consequence. It says that formalisation uses distinguishable positions before it can analyse them. The claim can be tested by trying to remove those positions. Remove the distinction between two sides of an equation, and equality has nothing to compare. Remove source and target, and no map remains. Remove premise and conclusion, and no inference remains.

For that reason the reader is asked to keep five statuses separate throughout the book.

1. **Boundary arguments** name what formalisation already presupposes. They

are not Agda theorems.

2. **Formal representations** translate the boundary into explicit data that a selected proof assistant can manipulate.
3. **Machine-checked theorems** prove what follows from those data under the displayed rules.
4. **The formulability claim** is the architectural thesis that the checked chain exposes the first stable skeleton of formulability itself.
5. **Interpretive readings** belong after the formal record has closed. They are not premises of the kernel.

Confusing these statuses weakens the project. Treat the boundary argument as a theorem, and the beginning overclaims. Treat the checked theorems as metaphor, and the execution is lost. Treat later readings as premises, and the formal kernel is obscured. The book is held together by the order of these layers.

Chapter 3

Representation Requires Distinguishable Positions

Presence Is Not Representation

Presence alone does not represent. A single undifferentiated occurrence, with no position from which it can be read and no position to which it can be related, carries no representation. Representation begins only when something can appear *as* something: a mark as a sign, a term as an inhabitant, a side as equal to another side, a map as connecting source to target.

This “as” is already a distinction of positions. It separates what is presented from the presentation through which it is available. Even the attempt to collapse them requires a relation of collapse, and that relation again has sides. The collapse cannot be stated without the distinction it denies.

The first formal datum must therefore do more than name points. It must make the positions usable: separated enough not to collapse, and covered enough that the carrier contains no unaccounted remainder.

Sign, Object, And Reading

Every representation contains a minimum structure: something stands, something is stood for, and some relation lets the standing count as such. Collapsing the positions destroys representation rather than purifying it. If the sign is nothing but the signed, there is no sign. If the relation vanishes, the standing-as vanishes with it.

This is why the formal threshold is stated as data rather than imagery. A carrier supplies the domain. Two distinguished positions supply the places to be held apart.

Separation prevents their collapse. Cover prevents the carrier from hiding a third position.

Chapter 4

Why The First Formal Datum Is Binary

Why One Is Not Enough

One named point does not answer the licensing problem of two. It carries no separation witness and gives no relation with two sides. If all positions in a carrier are identified with one point, there is no internal ground for a two-count.

The formal development later makes this precise: singleton and contractible carriers cannot host a *Distinction*. That theorem belongs after the record has been defined. The pre-formal point is only that a single undifferentiated position cannot license two.

Why Three Is Not The Entry Datum

Three named positions do not remove the binary problem. They add it several times. To hold three positions apart, the system must already distinguish each relevant pair and must also account for how the third position is covered, ordered, labelled, or selected. That additional data may be useful later, but it is not the minimal answer to the licensing problem of two.

The later formal treatment of higher arities is local to the record format used in this file. It does not claim that no other framework can choose a ternary primitive. It shows that, for this route, the binary record is the first formal datum that carries separation without surplus.

The Minimal Candidate

The binary datum is not selected because two is symbolically attractive. It is selected because it is the smallest candidate that can answer both parts of the licensing problem. It has enough structure to block collapse, and with a cover it has enough discipline to block hidden surplus.

This does not yet prove `two-normal-form`. It prepares the ground on which that theorem becomes legible. The theorem says, inside the selected Agda fragment, that any separated and covered binary carrier is boundary-preservingly isomorphic to `Two`.

Chapter 5

Separation And Exhaustion

Separation Is Anti-Collapse

The field later named $\ell \neq r$ is the formal anti-collapse clause. It is not decorative inequality. It is the data that prevents the two distinguished positions from being identified inside the carrier.

This is where Leibniz pressure enters the record. If the two positions were indiscernible in the relevant formal sense, the system would have no ground for holding them apart. The separation field is the explicit formal answer to that pressure at the threshold.

Cover Is Anti-Surplus

Two named points do not prove that the carrier has only those two points. MLTT does not infer absence from a lack of constructors in the prose. A carrier can contain further points unless a cover supplies a case for every element.

The cover field is therefore the anti-surplus clause. It says that each carrier element is equal to the left boundary or to the right boundary. Only with this field can the later eliminator inspect an arbitrary carrier point and reduce it to one of the two displayed cases.

Chapter 6

The Field Before Theory

Position Before Measurement

Measurement begins later than distinguishability. To measure something, a procedure must already be able to return one value rather than another, compare a mark with a scale, and keep the measured position apart from the act of measuring it. Before any unit can be introduced, the field must already admit retrievable positions.

This field is informational only in the primitive sense: something can be recovered as this rather than that. It is the minimum readability needed before a database, medium, observer, measurement, or equation can be specified. The present volume formalizes the first record that can carry that readability without collapse or surplus.

Theory Enters An Already Cut Field

Every formal theory enters through some discipline of distinction. It may choose different primitives, different equality principles, different universes, different tactics, or different computational kernels; but if it can state anything at all, it must distinguish statement from non-statement, rule from application, term from context, and success from failure.

This is not a rival formalism. It is the pressure that motivates the formal grant. Once a particular carrier is selected, the differences between proof assistants matter intensely. Before selection, the shared question is simpler: what licenses the system's use of distinguishable positions?

Chapter 7

Formalisations Are Plural

No Formal System Generates The Cut

The boundary motivates the need for a carrier; it does not force the concrete name of the carrier. Agda does not create the cut. MLTT does not create the cut. Coq, Lean, cubical systems, set-theoretic foundations, and other disciplined calculi would not create it either. At most, a formal system can receive the pressure as explicit data and then expose which consequences survive inside its rules.

This distinction matters because otherwise the book would confuse two claims. The licensing problem is pre-formal. Agda is the chosen execution environment in which the consequences of one formal answer are checked. The machine-checked theorems begin only after the answer has been supplied as data.

What A Carrier Must Discipline

A proof assistant can carry this development only if it can keep the relevant data visible. It must let the formal representative state a carrier, two boundary positions, a separation witness, and a coverage principle. It must make case analysis explicit enough that hidden residue cannot pass as absence. It must distinguish construction from postulate, and it must let failed alternatives close by contradiction rather than rhetoric.

For the present development, the carrier must also discipline equality, totality, and exhaustivity. Equality cannot identify too much too soon. Functions cannot be partial procedures masquerading as proofs. Coverage cannot be inferred from the fact that only two names were written down. The formal language has to keep these gates visible while the eliminative chain runs.

MLTT As One Disciplined Carrier

Intensional Martin-Löf type theory provides such a carrier for this book. It makes constructions explicit, treats propositions as types, and requires proofs to appear as terms. Agda then exposes that discipline through a concrete checker. Under `--safe`, no postulate can be smuggled into the file. Under `--without-K`, equality is not flattened by uniqueness of identity proofs.

This is why MLTT and Agda enter here: not as metaphysical origin, and not as the only conceivable formal clothing of the argument, but as the expression in which the present execution is carried out. A proof that another assistant can carry the same structure would be a further theorem of translation or reconstruction. It is not assumed here, and it is not needed in order for the present file to run.

The formulability claim does not depend on Agda being metaphysically privileged. It depends on the fact that this execution is explicit enough to expose the dependency chain. Another carrier could strengthen the claim by reconstructing the same chain under different rules; it could also reveal exactly where the present chain depends on MLTT.

What Remains Open

It remains open, in this volume, whether other proof assistants can carry the route with the same fidelity. A successful reconstruction would have to reproduce the boundary representative, the binary normal form, the endomorphism classification, the failure of under-representation, the closure at K_4 under the stated representation contract, and the return from the closure record to distinction without importing stronger principles that change the result.

Until such reconstructions are written, the honest statement is exact: this book mechanises the route in MLTT as implemented by Agda. It shows what follows there. It does not claim that Agda is the only possible witness.

Chapter 8

The Formal Threshold

The Tool: Agda as Executioner

The machine enters only after the threshold has been crossed. Agda can verify that a term has the type it claims to have. It can verify that a case split is exhaustive, that a contradiction has no branch, that a construction uses no postulate, and that an alleged equality reduces to `refl`. It cannot verify, from nowhere, the pre-formal licensing problem itself. That problem is what asks for the data the checker will then inspect.

This division is the contract that keeps the work honest. Before the threshold, the book argues that formalisation uses distinguishable positions. After the threshold, every claimed consequence is submitted to the type checker. The pre-formal argument does not pretend to be code. The formal development does not pretend to have generated its own condition of possibility.

This is where the theory of formulability becomes testable. The method receives distinction as explicit data. The checker then decides whether normal form, four-case classification, representation closure, and return to the datum actually follow. If the chain breaks, the architectural reading loses its formal spine.

The compiler flags used in this work, `--safe` and `--without-K`, restrict the formal environment further. No postulates can be introduced, and no appeal to the uniqueness of identity proofs is permitted. These flags do not create the structure; they make the execution inspectable.

From Cut To Distinction

At the threshold, the licensing problem receives a formal answer. The record Distinction is not *Void* itself and not the origin of difference. It is the MLTT

presentation of the explicit data needed to stabilize a two-point datum:

1. a carrier S : Set, so there is a domain in which positions can be addressed;
2. two boundary points ℓ and r , so the cut has sides;
3. a separation witness $\ell \neq r$, the anti-collapse datum;
4. a cover, the exhaustion datum, so every element of the carrier is classified by one of the two sides.

The cover is crucial. MLTT does not allow the inference from “only two points have been named” to “there are no other points.” Such an inference would confuse names with coverage. If the formal carrier is to be exactly binary for the purposes of elimination, coverage has to be part of the data. This is why the record is precise: it states exactly what the formal machine needs.

Reader Contract

After this point the text changes status. The prose may still motivate and orient, but the mathematical claims are tied to the Agda development that follows. A construction must be an inhabitant. An equality must reduce or be proved. A contradiction must have no branch. A classification must cover all cases. A uniqueness claim must state the relevant equivalence and close it.

The first code block therefore does not begin the ontology. It begins the execution. The threshold has already identified the need for anti-collapse and anti-surplus data; the formal file now receives those data and lets the checker eliminate whatever cannot survive them. This is the spine of the project: licensing problem, explicit data, execution, closure, and later interpretation must remain connected without being confused.

Part I

Foundation

The formal development begins by fixing the minimal logical infrastructure on which the later combinatorial and arithmetic arguments depend. Equality, pairing, negation, and the natural numbers are often treated as background. Here they are introduced explicitly so that later uniqueness claims rest on a fully visible formal basis.

This choice is methodological rather than polemical. Before any graph-theoretic or spectral argument can be evaluated, the ambient logic has to be specified carefully enough that no hidden axiom or informal convention can influence the outcome.

Chapter 9

Minimal Logical Infrastructure

The first task is to specify the formal environment itself. In Agda, constructive work depends not only on the definitions one writes but also on the background principles permitted by the language.

The flag `--safe` excludes postulates and ensures that no external axiom is introduced silently. The flag `--without-K` avoids the additional identification of all equality proofs and keeps the treatment of identity compatible with standard homotopy-theoretic practice. Universe levels provide the usual stratification that prevents impredicative self-reference.

None of this is peculiar to the present text. These are standard structural restrictions, recorded here because the later arguments use them repeatedly and because the opening code block is easiest to read once its ambient assumptions have been stated explicitly.

```
-- § Strict Environment Constraint: No axioms, no postulates, no Axiom K.
module Void where

-- § Stratification primitives prevent self-reference paradoxes.
open import Agda.Primitive using (Level; lzero; lsuc; _⊔_; Setω)
```

Propositional Equality

A formal development needs an internal notion of equality before it can state substitution, transport, or congruence. In the Martin-Löf type theory used here for the formal execution, this role is played by the identity type.

The definition used here is the standard one: an inductive family generated by reflexivity. From that single constructor one obtains symmetry, transitivity, congru-

ence, and substitution by straightforward pattern matching on `refl`. No additional extensional principle is introduced in this development.

```
-- § Propositional equality: the identity type with unique constructor refl.
infix 4 _≡_

data _≡_ {ℓ : Level} {A : Set ℓ} (x : A) : A → Set ℓ where
  refl : x ≡ x
{-# BUILTIN EQUALITY _≡_ #-}

-- § Symmetry is obtained by elimination on refl.
sym : {ℓ : Level} {A : Set ℓ} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

-- § Transitivity is obtained by elimination on the first proof.
trans : {ℓ : Level} {A : Set ℓ} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
trans refl q = q

-- § Congruence: equality is invariant under function application.
cong : {ℓ1 ℓ2 : Level} {A : Set ℓ1} {B : Set ℓ2} (f : A → B) {x y : A} → x ≡ y → f x ≡ f y
cong f refl = refl

-- § Substitution: transport along an equality proof.
subst : {ℓ1 ℓ2 : Level} {A : Set ℓ1} (P : A → Set ℓ2) {x y : A} → x ≡ y → P x → P y
subst P refl px = px

-- § Leibniz pressure executed after a carrier and equality are fixed.
leibniz-collapse :
  {S : Set} (a b : S) →
  ((P : S → Set) → P a → P b) →
  a ≡ b
leibniz-collapse a b ind = ind (λ x → a ≡ x) refl
```

The term `leibniz-collapse` is the promised formal echo of the opening pressure. Once a carrier and equality are fixed, transfer of all predicates from a to b yields $a \equiv b$ by choosing the predicate $\lambda x \rightarrow a \equiv x$. The lemma does not turn the pre-formal boundary into an Agda theorem; it records the collapse principle inside the formal setting after the setting has been supplied.

The preceding definitions also yield a small uniqueness lemma. If $p : x \equiv y$ and $q : y \equiv x$ compose with p to the trivial loop, then q is propositionally equal to `sym p`. This is simply the reflexivity computation written as a separate statement, but it will be convenient later when inverse paths have to be identified canonically.

```
-- § Symmetry is the unique inverse closing the trivial loop.
sym-unique :
  {ℓ : Level} {A : Set ℓ} {x y : A}
  (p : x ≡ y) (q : y ≡ x) →
  trans p q ≡ refl →
  q ≡ sym p
sym-unique refl q eq = eq
```

Universe Lifting

Universe stratification prevents self-reference, but it also creates a practical issue: data living at one level cannot simply be reused at the next. The standard remedy is `Lift`, which places a value in a higher universe without changing its content.

The accompanying lemmas record the two properties that matter later. First, the embedding is faithful: equality between lifted values reflects equality between the originals. Second, `Lift` satisfies the expected universal property, so any wrapper with exact round-trip behaviour is canonically isomorphic to it under that requirement.

```
-- § Lift embeds a type into the next universe level.
data Lift {ℓ : Level} (A : Set ℓ) : Set (lsuc ℓ) where
  lift : A → Lift A

-- § Lower extracts the embedded value.
lower : {ℓ : Level} {A : Set ℓ} → Lift A → A
lower (lift x) = x

-- § Lift is injective by construction.
lift-injective : {ℓ : Level} {A : Set ℓ} {x y : A} → lift x ≡ lift y → x ≡ y
lift-injective refl = refl

-- § A type isomorphism: mutually inverse maps with both round-trips trivial.
record SetIso {ℓ₁ ℓ₂ : Level} (A : Set ℓ₁) (B : Set ℓ₂) : Set (ℓ₁ ⊔ ℓ₂) where
  field
    to   : A → B
    from : B → A
    to-from : (b : B) → to (from b) ≡ b
    from-to : (a : A) → from (to a) ≡ a
open SetIso public

-- § Universal property of Lift: every faithful wrapper is canonically isomorphic.
Lift-universal :
  {ℓ : Level} {A : Set ℓ}
  (W : Set (lsuc ℓ))
  (wrap : A → W) (unwrap : W → A) →
  ((a : A) → unwrap (wrap a) ≡ a) →
  ((w : W) → wrap (unwrap w) ≡ w) →
  SetIso W (Lift A)
Lift-universal {ℓ} {A} W wrap unwrap unwrap-wrap wrap-unwrap = record
  { to   = λ w → lift (unwrap w)
  ; from = λ { (lift a) → wrap a }
  ; to-from = λ { (lift a) → cong lift (unwrap-wrap a) }
  ; from-to = λ w → wrap-unwrap w
  }
```

Universe-Polymorphic Equality

Some later statements quantify over all universe levels and therefore live in Set_{ω} . For that reason the text also introduces an identity type at the top universe.

The definition mirrors ordinary propositional equality exactly: there is a single constructor `reflω`, and the standard derived operations are obtained in the same way. The lemma below records its universal role among reflexive relations at Set_ω .

```
-- § Universe-polymorphic equality at Setω.
infix 4 _≡ω_

data _≡ω_ {A : Setω} (x : A) : A → Setω where
  reflω : x ≡ω x

-- § Symmetry at Setω.
symω : {A : Setω} {x y : A} → x ≡ω y → y ≡ω x
symω reflω = reflω

-- § Transitivity at Setω.
transω : {A : Setω} {x y z : A} → x ≡ω y → y ≡ω z → x ≡ω z
transω reflω q = q

-- § Congruence at Setω.
congω : {A B : Setω} (f : A → B) {x y : A} → x ≡ω y → f x ≡ω f y
congω f reflω = reflω

-- § Substitution at Setω.
substω : {A : Setω} (P : A → Setω) {x y : A} → x ≡ω y → P x → P y
substω P reflω px = px

-- § ≡ω is initial among reflexive relations at Setω.
≡ω-initial :
  (R : {A : Setω} → A → A → Setω) →
  ({A : Setω} {x : A} → R x x) →
  {A : Setω} {x y : A} → x ≡ω y → R x y
≡ω-initial R r reflω = r _
```

Products and Dependent Pairs

So far, every structure holds a single piece of information. But the moment a statement like “there exists an element with a certain property” must be expressed, two things must be held at once: the element and the proof of its property. Without a way to bundle these together, existential statements are inexpressible inside this constructive setting. The system can talk about individual objects but cannot witness claims about them.

The product type ($A \times B$) is the simplest response: it holds two independent values side by side. But the more consequential structure is the dependent pair ($\Sigma A B$), where the type of the second component depends on the value of the first. This is what makes it possible to say “there exists an a in A such that $B(a)$ holds”—the a and its proof travel together as a single witness. Without dependent pairs, the development would have universal statements but no internal form of existential witness.

The formal apparatus is correspondingly lean. The record `_×_` handles the non-dependent case of two pieces of data carried together, while `Σ` expresses the genuinely existential case in which the second component depends on the first. From this point on, existential claims in the book are not verbal gestures but inhabitants of `Σ`-types, with ordinary products as the constant special case.

```
-- § Non-dependent product of two types.
infixr 3 _×_

record _×_ (A B : Set) : Set where
  constructor _,_
  field
    fst : A
    snd : B

open _×_ public

-- § Dependent pair: the type of witnesses.
record Σ (A : Set) (B : A → Set) : Set where
  constructor _,_
  field
    fst : A
    snd : B fst

open Σ public
```

Absurdity and Negation

If the filter described in the introduction is to have any teeth, it must be possible to express that something *cannot* exist. A system that can only affirm—never deny—has no elimination power. Under such a regime, incompatible candidates would remain indistinguishable from admissible ones.

The empty type ($\perp$) is the formal expression of impossibility: a type with no constructors, and therefore no inhabitants. If one were to hold a value of type $\perp$, a contradiction would already have been reached, and from that contradiction anything follows. This is $\perp$ -elim, the principle of *ex falso quodlibet*: from the impossible, derive whatever you wish.

Negation falls out immediately. To say “ A is false” is to say “ A leads to a contradiction”—formally, $\neg A$ is the function type $A \rightarrow \perp$. No separate negation primitive is added. Double negation and inequality are then derived forms, not new axioms.

The code reflects exactly this economy. The type $\perp$ has no constructors, $\perp$ -elim is its eliminator, and negation is defined directly as the function type $A \rightarrow \perp$. Refutation is therefore not an external meta-principle but part of the same internal calculus as construction.

```

-- § The empty type: no constructors, no inhabitants.
data ⊥ : Set where

-- § Ex falso: the empty type eliminates into any type.
⊥-elim : {ℓ : Level} {A : Set ℓ} → ⊥ → A
⊥-elim ()

-- § Ex falso at the top universe.
⊥-elimω : {A : Setω} → ⊥ → A
⊥-elimω ()

-- § Negation: the unique map into absurdity.
¬_ : {ℓ : Level} → Set ℓ → Set ℓ
¬ A = A → ⊥

-- § Double negation.
¬¬_ : {ℓ : Level} → Set ℓ → Set ℓ
¬¬ A = ¬ (¬ A)

-- § Inequality: negation of propositional equality.
infix 2 _≠_
_≠_ : {ℓ : Level} {A : Set ℓ} → A → A → Set ℓ
x ≠ y = ¬ (x ≡ y)

```

Disjoint Union

Pairs express “both A and B.” But logic also requires “either A or B”—and it requires knowing *which one*. An untagged alternative—where the two cases are merged without a label—destroys the ability to distinguish them. Case analysis becomes impossible.

The disjoint union ($A \uplus B$) solves this by tagging each side: inj_1 marks a witness from the left, inj_2 from the right. Pattern matching can then determine which case holds. The Set_ω variant ($_ \uplus _$) extends this to the top universe level, for the same reason equality was extended to all levels: the later arguments quantify at that height and need the same tagged alternatives there.

Again the formal mechanism is minimal. The datatype $_ \uplus _$ stores the two tagged alternatives, and $_ \uplus _$ repeats the same construction at the top universe. Every later binary split in the book, including the coverage data of *Distinction*, is simply an instance of pattern matching on this tagged sum.

```

-- § Disjoint union (coproduct) of two types.
infixr 2 _⊔_

data _⊔_ {ℓ : Level} (A B : Set ℓ) : Set ℓ where
  inj₁ : A → A ⊔ B
  inj₂ : B → A ⊔ B

-- § Disjoint union at Setω.

```

```

infixr 2 _⊔ω_

data _⊔ω_ (A B : Setω) : Setω where
inj₁ω : A → A ⊔ω B
inj₂ω : B → A ⊔ω B

```

Stabilized Contact

Once products, dependent pairs, equality, and tagged alternatives are available, contact can be stated without adding a new primitive. Given two evaluations into a common carrier, contact is the dependent witness that one origin from the first source and one origin from the second source share an image. It is not a metaphysical bond between completed objects; it is synchronized retrievability carried as data.

Self-contact is the decisive test. An evaluation is stable only if its self-contact does not hide a collapse: whenever two origins become equal after evaluation, the origins were already equal. In that case the self-contact carries no surplus over the origin. The diagonal is not postulated from outside; it is recovered as the residue of non-collapse.

Agreement zones and stable relations are the first two shapes obtained from this discipline. An agreement zone records the positions at which two evaluations cease to differ. A stable relation is a pair of evaluations whose combined imprint does not identify distinct origins. These constructions occur after the formal threshold. They do not prove that the pre-formal boundary is itself a type; they show what becomes available once a disciplined carrier, equality, and dependent witnesses are already in force.

```

-- § Contact of two sources through a common carrier.
Contact : {A B C : Set} → (A → C) → (B → C) → Set
Contact {A} {B} f g = Σ A (λ a → Σ B (λ b → f a ≡ g b))

-- § Self-contact: two origins meeting under the same evaluation.
SelfContact : {A B : Set} → (A → B) → Set
SelfContact i = Contact i i

-- § Non-collapse: equal images create no new identification.
NonCollapsing : {A B : Set} → (A → B) → Set
NonCollapsing {A} i = (x y : A) → i x ≡ i y → x ≡ y

-- § Diagonal: two origins together with their equality.
Diagonal : Set → Set
Diagonal A = Σ A (λ x → Σ A (λ y → x ≡ y))

-- § A diagonal witness always induces self-contact.
diagonal-self-contact : {A B : Set} (i : A → B) → Diagonal A → SelfContact i
diagonal-self-contact i (x, (y, same)) = x, (y, cong i same)

-- § No-surplus self-contact: self-contact returns to the diagonal.
SelfContactNoSurplus : {A B : Set} → (A → B) → Set

```

```

SelfContactNoSurplus {A} i = SelfContact i → Diagonal A

-- § A non-collapsing self-contact reduces to equality of origins.
self-contact-collapses :
  {A B : Set} (i : A → B) →
  NonCollapsing i →
  SelfContact i →
  Diagonal A
self-contact-collapses i nc (x, (y, same)) = x, (y, nc x y same)

-- § Non-collapse is exactly the no-surplus discipline for self-contact.
self-contact-no-surplus :
  {A B : Set} (i : A → B) →
  NonCollapsing i →
  SelfContactNoSurplus i
self-contact-no-surplus i nc = self-contact-collapses i nc

-- § Agreement zone of two evaluations from one source.
Agreement : {A B : Set} → (A → B) → (A → B) → Set
Agreement {A} f g =  $\Sigma$  A (λ x → f x ≡ g x)

-- § Agreement is contact with the same origin read twice.
agreement-contact :
  {A B : Set} (f g : A → B) →
  Agreement f g → Contact f g
agreement-contact f g (x, same) = x, (x, same)

-- § Explicit constructor for non-dependent pairs.
pair : {A B : Set} → A → B → A × B
pair x y = record { fst = x ; snd = y }

-- § Pair equality from component equalities.
pair-eq :
  {A B : Set} {x y : A} {u v : B} →
  x ≡ y → u ≡ v → pair x u ≡ pair y v
pair-eq refl refl = refl

-- § Joint imprint of two evaluations.
pairMap : {X Y Z : Set} → (X → Y) → (X → Z) → X → Y × Z
pairMap f g x = pair (f x) (g x)

-- § Stable relation: the joint imprint is non-collapsing.
StableRelation : {X Y Z : Set} → (X → Y) → (X → Z) → Set
StableRelation f g = NonCollapsing (pairMap f g)

-- § Component agreement in a stable relation forces origin equality.
stable-relation-components :
  {X Y Z : Set} (f : X → Y) (g : X → Z) →
  StableRelation f g →
  (x y : X) →
  f x ≡ f y →
  g x ≡ g y →
  x ≡ y
stable-relation-components {Y = Y} {Z = Z} f g stable x y same-f same-g =
  stable x y (pair-eq {A = Y} {B = Z} same-f same-g)

```

Ontological Primitives

Before distinction itself is defined, three auxiliary predicates fix the local vocabulary. A contractible type has a center to which every inhabitant is equal, and therefore carries no internal separation. A type with a distinct pair contains two provably different elements. Non-triviality is simply the negation of contractibility.

These notions provide the vocabulary for the next stage of the argument. They make precise what it means for a carrier to admit a genuine boundary and prepare the non-contractibility results used shortly afterward.

```
-- § Contractibility: a single center absorbs all inhabitants.
isContr : Set → Set
isContr A =  $\Sigma$  A ( $\lambda$  c  $\rightarrow$  ( $x : A$ )  $\rightarrow$  c  $\equiv$  x)

-- § A type has a distinct pair if two inhabitants are provably unequal.
HasDistinctPair : Set → Set
HasDistinctPair A =  $\Sigma$  A ( $\lambda$  a  $\rightarrow$   $\Sigma$  A ( $\lambda$  b  $\rightarrow$  a  $\neq$  b))

-- § Non-triviality: the type is not contractible.
NonTrivial : Set → Set
NonTrivial A =  $\neg$  (isContr A)
```

The Distinction Record

Formally, a distinction is the answer this file gives to the licensing problem. It consists of a carrier, two designated elements of that carrier, a proof that these elements are different, and a cover asserting that every element of the carrier is equal to one of them. The proof $\ell \neq r$ is anti-collapse data. The cover is anti-surplus data. Together they say that the two displayed positions neither collapse into one nor leave an unclassified remainder in the carrier.

The cover is not an ornamental extra. Without it one would have two marked points but no uniform way to classify arbitrary elements of the carrier, and the later elimination arguments would lose their basic case split. The counterexample below makes that dependence explicit rather than rhetorical.

```
-- § Binary distinction data: carrier, boundary points, anti-collapse, and cover.
record Distinction : Set1 where
  field
    S   : Set
     $\ell$  : S
    r   : S
     $\ell \neq r$  :  $\ell \neq r$ 
    cover : ( $x : S$ )  $\rightarrow$  ( $x \equiv \ell$ )  $\sqcup$  ( $x \equiv r$ )

open Distinction public
```

Necessity of Coverage

The separation proof and the cover serve different purposes. It is natural to ask whether the cover might be recovered from the two designated boundary points alone. The answer is negative under the displayed data. On a carrier with more than two distinguishable elements, an eliminator based only on the boundary witnesses fails, because an off-boundary element need not satisfy a predicate determined at those two points.

The four-element vertex type $K_4\text{Vertex}$, introduced later in Chapter 57, provides a concrete witness to this failure. Lemma 57 records the argument formally. The cover is therefore part of the definition for operational reasons: it is what makes global two-case elimination possible.

Minimal presentation in MLTT

The five fields of `Distinction` – the carrier S , the two boundary points ℓ and r , the separation proof $\ell \neq r$, and the cover – are stated here as the minimal presentation sufficient for the elimination framework developed in this text. Each field is operationally indispensable: the carrier alone has no second point to compare to, the boundary points without separation may collapse, and the cover cannot be derived from the separated pair. Lemma 57 refutes any such derivation by exhibiting $K_4\text{Vertex}$ as a four-element counterexample.

What this presentation does *not* claim is the stronger reading sometimes attached to “minimality”: that naming only ℓ and r as inhabitants of S would, by itself, make S contain no third element. Standard Martin-Löf type theory does not validate that step; it admits types with elements not produced by any named constructor and provides no internal mechanism that turns “not introduced” into “does not exist”. The cover field is therefore not a workaround. It is the exact two-class coverage data that the type theory cannot supply for free.

Chapter 10

Non-Vacuity

The record `Distinction` has now been defined, but a definition alone does not produce an inhabitant. If `Distinction` were empty, then every universal statement about distinctions would hold vacuously, and the subsequent development would lose substantive force. The purpose of this chapter is to block that collapse in the weakest form needed at the outset.

Under `--safe`, no inhabitant of `Distinction` is simply postulated. The statement used here is the double-negation form $\neg\neg \text{Distinction}$. It does not yet supply a witness, but it excludes stable denial and is exactly the weaker non-vacuity guard needed before the later constructive witness appears.

Stronger results appear later. Chapter 12 constructs `Two-distinction` without premises, and Chapter 63 provides an independent route through measurement. By contrast, the stronger claim that every inhabited type already contains a distinction is false; the unit type $\top$ is the standard counterexample. The present chapter therefore uses the weaker formulation because it is both correct and sufficient for what follows.

```
-- § Non-vacuity demands that Distinction cannot be refuted.
record NonVacuityLaw : Set1 where
  field
    nonvacuity :  $\neg\neg \text{Distinction}$ 
open NonVacuityLaw public
```

Law 0.0: Distinction Is Irrefutable

The first law simply restates the field of `NonVacuityLaw`. Its role is foundational: without it, every later theorem quantified over distinctions would hold for purely vacuous reasons.

```
-- § Law 0.0: Distinction is irrefutable.
law0-0-nonvacuity : (nv : NonVacuityLaw)  $\rightarrow \neg\neg \text{Distinction}$ 
law0-0-nonvacuity = nonvacuity
```

Law 0.1: Non-Vacuity Is Non-Eliminability

This is the same content in a slightly sharper form. The negation of *Distinction* is itself unstable, even though no explicit witness has yet been constructed.

```
-- § Law 0.1: NonVacuityLaw is non-eliminability of Distinction.
law0-1-nonvacuity-is-non-eliminability :
  NonVacuityLaw → ¬ (¬ Distinction)
law0-1-nonvacuity-is-non-eliminability = nonvacuity
```

Law 0.2: Distinction Carrier Is Inhabited

Once a distinction record is given, its carrier is inhabited, witnessed immediately by the left boundary point ℓ .

```
-- § Law 0.2: Distinction carrier is inhabited.
law0-2-inhabited : (d : Distinction) → S d
law0-2-inhabited d = ℓ d
```

Law 0.3: Distinction Carrier Has A Distinct Pair

The record also provides a distinct pair directly, namely the two boundary points together with their inequality proof.

```
-- § Law 0.3: Distinction carrier has a distinct pair.
law0-3-has-distinct-pair : (d : Distinction) → HasDistinctPair (S d)
law0-3-has-distinct-pair d = (ℓ d, (r d, ℓ ≠ r d))
```

Law 0.4: Distinction Carrier Is Non-Contractible

The carrier of a distinction cannot be contractible. Under the record fields, a contraction center would identify ℓ and r , contradicting the separation proof $\ell \neq r$.

```
-- § Law 0.4: Distinction carrier is non-contractible.
law0-4-not-contractible : (d : Distinction) → NonTrivial (S d)
law0-4-not-contractible d (c, collapse) =
  ℓ ≠ r d (trans (sym (collapse (ℓ d))) (collapse (r d)))
```

Law 0.5: Distinct Pair Forces Non-Contractibility

The previous argument does not depend on the full distinction record. Any type equipped with a distinct pair is non-contractible for the same reason: a contraction center would force the two elements to be equal.

```
-- § Law 0.5: Distinct pair forces non-contractibility.
law0-5-distinct-pair-forces-nontrivial :
```



```

{A : Set} → HasDistinctPair A → NonTrivial A
law0-5-distinct-pair-forces-nontrivial (a, (b, a ≠ b)) (c, collapse) =
a ≠ b (trans (sym (collapse a)) (collapse b))

```

Law 0.6: Distinction Is Reconstructible From Its Data

The distinction record contains no hidden structure beyond its displayed fields. Given a carrier, two elements, a proof that they differ, and a cover, one can reconstruct a Distinction directly.

```
-- § Law 0.6: Distinction is reconstructible from its data.
```

```
fromDistinctCovered :
```

```
{S₀ : Set} (a b : S₀) →
```

```
a ≠ b →
```

```
((x : S₀) → (x ≡ a) ⊔ (x ≡ b)) →
```

```
Distinction
```

```
fromDistinctCovered {S₀} a b a ≠ b cov = record
```

```
{ S = S₀ ; ℓ = a ; r = b ; ℓ ≠ r = a ≠ b ; cover = cov }
```

```
law0-6-reconstruction :
```

```
{S₀ : Set} (a b : S₀) →
```

```
(a ≠ b : a ≠ b) →
```

```
(cov : (x : S₀) → (x ≡ a) ⊔ (x ≡ b)) →
```

```
Distinction
```

```
law0-6-reconstruction = fromDistinctCovered
```

Law 0.7: Reconstruction Is Exact

This reconstruction is exact in the strongest available sense. If one extracts the fields of a distinction and feeds them back into the constructor, the result is definitionally equal to the original term, so the proof is simply `refl`.

```
-- § Law 0.7: Reconstruction is exact by definitional equality.
```

```
law0-7-reconstruction-exact :
```

```
(d : Distinction) →
```

```
fromDistinctCovered (ℓ d) (r d) (ℓ ≠ r d) (cover d) ≡ d
```

```
law0-7-reconstruction-exact d = refl
```

The record may therefore be unpacked and reassembled without loss. Later arguments use this observation when moving between the record itself and its component data.

Chapter 11

Elimination and Classification

Once a distinction is fixed, the associated endomorphism space is severely constrained. Because the cover identifies every element of the carrier with either ℓ or r , an endomorphism is determined entirely by its values on those two points. The boundary-output table therefore has at most four entries.

This chapter makes that observation precise and uses it to derive the four-function configuration that later reappears as K_4 .

Two-Class Coverage

Law 1.1: Carrier Coverage Is Two-Class

The first classification lemma is just the cover field restated for later use. Its importance is practical rather than subtle: every subsequent case distinction on the carrier depends on the fact that the cover reduces all elements to these two classes.

```
-- § Law 1.1: Carrier coverage is two-class.  
law1-1-cover : (d : Distinction) → (x : S d) → (x ≡ ℓ d) ⊔ (x ≡ r d)  
law1-1-cover = cover
```

Two-class coverage is the first structural restriction on the carrier. Figure 11.1 records this dichotomy as a two-node distinction diagram.

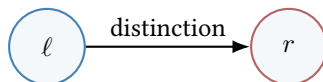


Figure 11.1: Binary distinction data: two boundary witnesses forced by two-class coverage.

Law 1.2: Elimination Is Forced By Coverage

Coverage yields a canonical eliminator for predicates on the carrier. If a property P is known at the two boundary points ℓ and r , then it is known everywhere, because the cover reduces every input to one of those two cases. The uniqueness lemma below records the corresponding rigidity: any eliminator satisfying the same boundary equations agrees with the canonical one everywhere.

Formally, the point is straightforward. Any eliminator that agrees with `Distinction-elim` on the two boundary witnesses must inspect `cover d x`, and this case split leaves only the same two branches. The boundary equations then force the values to coincide, so the alleged alternative reduces to pointwise equality with the canonical eliminator. The dependent eliminator inherits the same rigidity through transport.

-- § The canonical eliminator for a distinction carrier.

`Distinction-elim :`

$(d : \text{Distinction}) \rightarrow$

$\{P : \mathbf{S} \, d \rightarrow \mathbf{Set}\} \rightarrow$

$P \, (\ell \, d) \rightarrow$

$P \, (r \, d) \rightarrow$

$(x : \mathbf{S} \, d) \rightarrow$

$P \, x$

`Distinction-elim` $d \{P\} p\ell \, pr \, x$ **with** `cover d x`

... | `inj1` $x \equiv \ell = \text{subst } P \, (\text{sym } x \equiv \ell) \, p\ell$

... | `inj2` $x \equiv r = \text{subst } P \, (\text{sym } x \equiv r) \, pr$

-- § Law 1.2: Elimination is forced by coverage.

`law1-2-elim :`

$(d : \text{Distinction}) \rightarrow$

$\{P : \mathbf{S} \, d \rightarrow \mathbf{Set}\} \rightarrow$

$P \, (\ell \, d) \rightarrow$

$P \, (r \, d) \rightarrow$

$(x : \mathbf{S} \, d) \rightarrow$

$P \, x$

`law1-2-elim` = `Distinction-elim`

-- § The non-dependent eliminator is pointwise unique on its boundary equations.

`Distinction-elim-unique :`

$(d : \text{Distinction}) \{B : \mathbf{Set}\}$

$(e_1 \, e_2 : B \rightarrow B \rightarrow \mathbf{S} \, d \rightarrow B) \rightarrow$

$((b\ell \, br : B) \rightarrow e_1 \, b\ell \, br \, (\ell \, d) \equiv b\ell) \rightarrow$

$((b\ell \, br : B) \rightarrow e_1 \, b\ell \, br \, (r \, d) \equiv br) \rightarrow$

$((b\ell \, br : B) \rightarrow e_2 \, b\ell \, br \, (\ell \, d) \equiv b\ell) \rightarrow$

$((b\ell \, br : B) \rightarrow e_2 \, b\ell \, br \, (r \, d) \equiv br) \rightarrow$

$(b\ell \, br : B) (x : \mathbf{S} \, d) \rightarrow e_1 \, b\ell \, br \, x \equiv e_2 \, b\ell \, br \, x$

`Distinction-elim-unique` $d \, e_1 \, e_2 \, e_1 \ell \, e_1 r \, e_2 \ell \, e_2 r \, b\ell \, br \, x$ **with** `cover d x`

... | `inj1` $p =$

`trans` (`cong` $(e_1 \, b\ell \, br) \, p$)

`(trans` $(e_1 \ell \, b\ell \, br)$

```

      (trans (sym (e2 ℓ bℓ br))
        (cong (e2 bℓ br) (sym p))))
... | inj2 p =
  trans (cong (e1 bℓ br) p)
    (trans (e1 r bℓ br)
      (trans (sym (e2 r bℓ br))
        (cong (e2 bℓ br) (sym p)))))

```

Duality

Once the carrier has been reduced to the two boundary classes, there is a canonical nontrivial self-map obtained by exchanging them. This map, called *duality*, sends ℓ to r and r to ℓ . Later results close the classification absolutely: up to pointwise equality, it is the only non-identity automorphism of a distinction.

Its basic property is involutivity. Because every element of the carrier is identified by the cover with one of the two boundary points, applying the swap twice returns the original element after a short case analysis.

```

-- § The dual map: swap boundary cases.
Distinction-dual : (d : Distinction) → S d → S d
Distinction-dual d x with cover d x
... | inj1 _ = r d
... | inj2 _ = ℓ d

-- § Duality is involutive by exhaustive case analysis.
Distinction-dual-involutive :
  (d : Distinction) →
  (x : S d) →
  Distinction-dual d (Distinction-dual d x) ≡ x
Distinction-dual-involutive d =
  Distinction-elim d proof-ℓ proof-r
where
  proof-ℓ : Distinction-dual d (Distinction-dual d (ℓ d)) ≡ ℓ d
  proof-ℓ with cover d (ℓ d)
... | inj2 ℓ≡r = ⊥-elim ((ℓ≠r d) ℓ≡r)
... | inj1 _ with cover d (r d)
... | inj1 r≡ℓ = ⊥-elim ((ℓ≠r d) (sym r≡ℓ))
... | inj2 _ = refl

  proof-r : Distinction-dual d (Distinction-dual d (r d)) ≡ r d
  proof-r with cover d (r d)
... | inj1 r≡ℓ = ⊥-elim ((ℓ≠r d) (sym r≡ℓ))
... | inj2 _ with cover d (ℓ d)
... | inj2 ℓ≡r = ⊥-elim ((ℓ≠r d) ℓ≡r)
... | inj1 _ = refl

```

Law 1.3: Duality Is An Involution

The previous construction is summarized in theorem form here: duality is an involution.

```
-- § Law 1.3: Duality is an involution.
law1-3-dual-involutive :
  (d : Distinction) →
  (x : S d) →
  Distinction-dual d (Distinction-dual d x) ≡ x
law1-3-dual-involutive = Distinction-dual-involutive
```

Pointwise Equality

To compare endomorphisms, the natural relation is pointwise equality. Two functions are identified when they agree at every argument. On a distinction this relation is especially effective, because the cover reduces every input to one of the two boundary cases.

The universal property below states this explicitly. Any relation on endomorphisms that already determines agreement on the two boundary points refines to pointwise equality on the whole carrier under the cover.

```
-- § Pointwise equality of functions.
infix 4 _≐_
_≐_ : {ℓ1 ℓ2 : Level} {A : Set ℓ1} {B : Set ℓ2} → (A → B) → (A → B) → Set (ℓ1 ⊔ ℓ2)
_≐_ {A = A} f g = (x : A) → f x ≡ g x

-- § The identity function.
id : {A : Set} → A → A
id x = x

-- § Pointwise equality is initial among boundary-respecting comparisons of endomorphisms.
≐-universal :
  (d : Distinction)
  (R : (S d → S d) → (S d → S d) → Set) →
  ({f g : S d → S d} → R f g → f (ℓ d) ≡ g (ℓ d)) →
  ({f g : S d → S d} → R f g → f (r d) ≡ g (r d)) →
  {f g : S d → S d} → R f g → f ≐ g
≐-universal d R agree-ℓ agree-r {f} {g} rfg x with cover d x
... | inj1 p = trans (cong f p) (trans (agree-ℓ rfg) (cong g (sym p)))
... | inj2 p = trans (cong f p) (trans (agree-r rfg) (cong g (sym p)))
```

Four-Case Endomorphism Classification

An endomorphism $f : S \rightarrow S$ is determined by the pair of values $f(\ell)$ and $f(r)$. Since each of these values must be either ℓ or r , exactly four candidate combinations arise:

- $f(\ell) = \ell$ and $f(r) = \ell$: the constant-left map.
- $f(\ell) = r$ and $f(r) = r$: the constant-right map.
- $f(\ell) = \ell$ and $f(r) = r$: the identity.
- $f(\ell) = r$ and $f(r) = \ell$: the dual.

The soundness proof below turns these four candidates into an exhaustive classification of the endomorphism space. The datatype `EndoCase` is introduced simply to name them once and for all.

```
-- § The four-case classification of endomorphisms.
data EndoCase : Set where
  case-constL : EndoCase
  case-constR : EndoCase
  case-id      : EndoCase
  case-dual    : EndoCase
```

Figure 11.2 illustrates the basic obstruction behind the later appearance of K_4 : a three-vertex closure cannot accommodate four distinct endomorphism cases without omission.

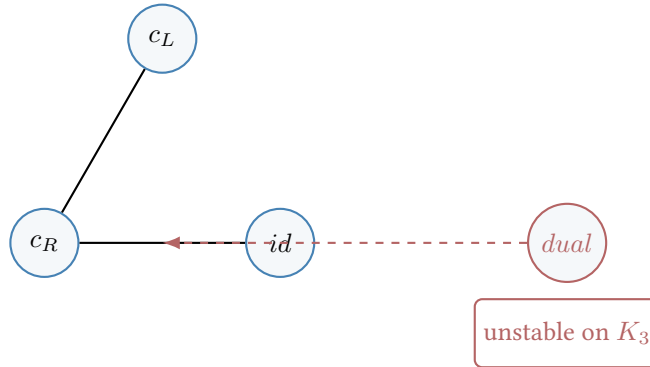


Figure 11.2: The failure of K_3 : the four mathematically distinct endomorphism cases cannot close over three vertices. One function is structurally forced into non-closure.

The K_4 Classification Module

The four cases are only names until they are related back to actual endomorphisms of the carrier. The K_4 module packages the corresponding constructions: the endomorphisms themselves, pointwise equality, the four representatives, and the maps that interpret and classify them. The parameter d keeps the whole discussion relative to a fixed distinction.

Step 1: Endo type and pointwise equivalence. The endomorphism type and its equivalence relation are fixed first.

```
-- § K4 endomorphism algebra: classify, interpret, and verify.
module K4 (d : Distinction) where
  Endo : Set
  Endo = S d → S d

  ⋅-refl : {f : Endo} → f ⋅ f
  ⋅-refl x = refl

  ⋅-sym : {f g : Endo} → f ⋅ g → g ⋅ f
  ⋅-sym p x = sym (p x)

  ⋅-trans : {f g h : Endo} → f ⋅ g → g ⋅ h → f ⋅ h
  ⋅-trans p q x = trans (p x) (q x)
```

Step 2: The four canonical endomorphisms. The four representative endomorphisms are then defined. The only slightly nontrivial one is the dual map, whose behaviour on the boundary points is verified by case analysis on cover. Their canonical status is established by the classification laws that follow, not assumed at the moment of definition.

```
-- § Constant-left endomorphism.
constL : Endo
constL _ = ℓ d

-- § Constant-right endomorphism.
constR : Endo
constR _ = r d

-- § The dual endomorphism.
dual : Endo
dual = Distinction-dual d

-- § Dual sends ℓ to r.
dual-ℓ : dual (ℓ d) ≡ r d
dual-ℓ with cover d (ℓ d)
... | inj1 _ = refl
... | inj2 ℓ ≡ r = ⊥-elim ((ℓ ≠ r d) ℓ ≡ r)

-- § Dual sends r to ℓ.
dual-r : dual (r d) ≡ ℓ d
dual-r with cover d (r d)
... | inj1 r ≡ ℓ = ⊥-elim ((ℓ ≠ r d) (sym r ≡ ℓ))
... | inj2 _ = refl
```

Step 3: Interpretation and classification. Each case label is realized as one of the four representative endomorphisms. Conversely, every endofunction is classified by inspecting its values at the two boundary points.


```

-- § Interpret a case label as an endofunction.
interpret : EndoCase → Endo
interpret case-constL = constL
interpret case-constR = constR
interpret case-id     = id
interpret case-dual   = dual

-- § Classify an endofunction by its boundary outputs.
classify : Endo → EndoCase
classify f with cover d (f (ℓ d)) | cover d (f (r d))
... | inj1 _ | inj1 _ = case-constL
... | inj2 _ | inj2 _ = case-constR
... | inj1 _ | inj2 _ = case-id
... | inj2 _ | inj1 _ = case-dual

```

Step 4: Soundness of classification. Classification followed by interpretation recovers the original endomorphism at the two boundary points. The full pointwise statement then follows from the carrier eliminator.

```

-- § Soundness at ℓ: interpretation recovers the value at ℓ.
sound-at-ℓ : (f : Endo) → interpret (classify f) (ℓ d) ≡ f (ℓ d)
sound-at-ℓ f with cover d (f (ℓ d)) | cover d (f (r d))
... | inj1 fl≡ℓ | inj1 _ = sym fl≡ℓ
... | inj2 fl≡r | inj2 _ = sym fl≡r
... | inj1 fl≡ℓ | inj2 _ = sym fl≡ℓ
... | inj2 fl≡r | inj1 _ = trans dual-ℓ (sym fl≡r)

-- § Soundness at r: interpretation recovers the value at r.
sound-at-r : (f : Endo) → interpret (classify f) (r d) ≡ f (r d)
sound-at-r f with cover d (f (ℓ d)) | cover d (f (r d))
... | inj1 fr≡ℓ | inj1 _ = sym fr≡ℓ
... | inj2 fr≡r | inj2 _ = sym fr≡r
... | inj1 fr≡ℓ | inj2 _ = sym fr≡r
... | inj2 fr≡r | inj1 _ = trans dual-r (sym fr≡ℓ)

-- § Soundness: classification followed by interpretation recovers behavior.
classify-sound : (f : Endo) → interpret (classify f) ≡ f
classify-sound f x = Distinction-elim d (sound-at-ℓ f) (sound-at-r f) x

```

Step 5: Boundary determination and injectivity. Endofunctions are determined by their boundary values, again through the eliminator. The interpretation map is injective because different case labels produce different behaviour at ℓ or r .

```

-- § Endofunctions are determined by their boundary values.
endo-determined :
(f g : Endo) →
f (ℓ d) ≡ g (ℓ d) →
f (r d) ≡ g (r d) →
f ≡ g
endo-determined f g eqℓ eqr x = Distinction-elim d eqℓ eqr x

```

```

-- § Interpretation is injective: distinct cases produce distinct behavior.
interpret-injective :
  (c c' : EndoCase) →
  interpret c ≐ interpret c' →
  c ≐ c'
interpret-injective case-constL case-constL _ = refl
interpret-injective case-constL case-constR p = ⊥-elim ((ℓ ≠ r d) (p (ℓ d)))
interpret-injective case-constL case-id p = ⊥-elim ((ℓ ≠ r d) (p (r d)))
interpret-injective case-constL case-dual p =
  ⊥-elim ((ℓ ≠ r d) (trans (p (ℓ d)) dual-ℓ))

interpret-injective case-constR case-constL p =
  ⊥-elim ((ℓ ≠ r d) (sym (p (ℓ d))))
interpret-injective case-constR case-constR _ = refl
interpret-injective case-constR case-id p =
  ⊥-elim ((ℓ ≠ r d) (sym (p (ℓ d))))
interpret-injective case-constR case-dual p =
  ⊥-elim ((ℓ ≠ r d) (sym (trans (p (r d)) dual-r))))

interpret-injective case-id case-constL p =
  ⊥-elim ((ℓ ≠ r d) (sym (p (r d))))
interpret-injective case-id case-constR p = ⊥-elim ((ℓ ≠ r d) (p (ℓ d)))
interpret-injective case-id case-id _ = refl
interpret-injective case-id case-dual p =
  ⊥-elim ((ℓ ≠ r d) (trans (p (ℓ d)) dual-ℓ))

interpret-injective case-dual case-constL p =
  ⊥-elim ((ℓ ≠ r d) (sym (trans (sym (dual-ℓ)) (p (ℓ d)))))
interpret-injective case-dual case-constR p =
  ⊥-elim ((ℓ ≠ r d) (trans (sym (dual-r)) (p (r d)))))
interpret-injective case-dual case-id p =
  ⊥-elim ((ℓ ≠ r d) (sym (trans (sym (dual-ℓ)) (p (ℓ d)))))
interpret-injective case-dual case-dual _ = refl

```

Step 6: Classification uniqueness. Soundness and injectivity together show that the label assigned by `classify` is unique. Here the absolute statement is local and proved: once a case label interprets to the same endomorphism, it must be the classifier's label.

```

-- § Classification is unique: the label is forced by soundness + injectivity.
classify-unique : (f : Endo) → (c : EndoCase) → interpret c ≐ f → c ≐ classify f
classify-unique f c c ≐ f =
  interpret-injective c (classify f) (≐-trans c ≐ f (≐-sym (classify-sound f)))

```

Step 7: The classifier is a left-inverse of interpretation. Applying the uniqueness statement to $f := \text{interpret } c$ shows that classification is a left inverse to interpretation.

```

-- § Classify followed by interpret followed by classify recovers the label.
classify-interpret : (c : EndoCase) → classify (interpret c) ≐ c
classify-interpret c = sym (classify-unique (interpret c) c ≐-refl)

```

Step 8: Endomorphisms modulo $\cong$ form a four-element set. Taken together, the preceding lemmas identify `EndoCase` with the quotient of `Endo` by pointwise equivalence. The record below packages the corresponding maps together with their round-trip and uniqueness properties.

```
-- § The bijection between EndoCase and (Endo /  $\cong$ ).
record EndoCase-bijection : Set where
  field
    to      : Endo → EndoCase
    from    : EndoCase → Endo
    to-from : (c : EndoCase) → to (from c)  $\equiv$  c
    from-to : (f : Endo) → from (to f)  $\cong$  f
    to-unique : (f : Endo) → (c : EndoCase) → from c  $\cong$  f → c  $\equiv$  to f

k4-bijection : EndoCase-bijection
k4-bijection = record
{ to      = classify
; from    = interpret
; to-from = classify-interpret
; from-to = classify-sound
; to-unique = classify-unique
}
```

Mutual Distinctness of Cases

The four case labels are pairwise distinct. Since they are constructors of an inductive datatype, each inequality is verified in Agda by the impossible pattern `()`. This is an absolute constructor fact, not a rhetorical exclusion.

```
-- § All four EndoCase constructors are pairwise distinct.
EndoCase-distinct :
  (case-constL  $\equiv$  case-constR →  $\perp$ ) ×
  (case-constL  $\equiv$  case-id    →  $\perp$ ) ×
  (case-constL  $\equiv$  case-dual  →  $\perp$ ) ×
  (case-constR  $\equiv$  case-id    →  $\perp$ ) ×
  (case-constR  $\equiv$  case-dual  →  $\perp$ ) ×
  (case-id  $\equiv$  case-dual      →  $\perp$ )
EndoCase-distinct =
  ( $\lambda$  ()) ,
  (( $\lambda$  ()),
   (( $\lambda$  ()),
    (( $\lambda$  ()),
     (( $\lambda$  ()),
      ( $\lambda$  ()))))))
```

Top-Level Classification Witnesses

The constructions inside the `K4` module are now repackaged as top-level lemmas. No additional hypotheses are introduced; the point is simply to make soundness and

uniqueness available in a form that can be cited later without reopening the module.

```
-- § Top-level K4 soundness witness.
k4-classification-sound :
  (d : Distinction) →
  (f : S d → S d) →
  Σ EndoCase (λ c → K4.interpret d c ≐ f)
k4-classification-sound d f = K4.classify d f , K4.classify-sound d f

-- § Top-level K4 uniqueness witness.
k4-classification-unique :
  (d : Distinction) →
  (f : S d → S d) →
  (c1 c2 : EndoCase) →
  K4.interpret d c1 ≐ f →
  K4.interpret d c2 ≐ f →
  c1 ≡ c2
k4-classification-unique d f c1 c2 p1 p2 =
  K4.interpret-injective d c1 c2 (K4.≐-trans d p1 (K4.≐-sym d p2))
```

Law 1.4: Endo(S) Classifies Into Exactly Four Cases

This law records the classifier introduced above. The word “exactly” is justified by the soundness and uniqueness witnesses already proved inside the module.

```
-- § Law 1.4: Endo(S) classifies into exactly four cases.
law1-4-classify : (d : Distinction) → (S d → S d) → EndoCase
law1-4-classify d = K4.classify d
```

Law 1.5: Classification Is Sound

Soundness says that interpreting the label assigned to an endomorphism recovers that endomorphism pointwise.

```
-- § Law 1.5: Classification is sound.
law1-5-classify-sound : (d : Distinction) → (f : S d → S d) → K4.interpret d (K4.classify d f) ≐ f
law1-5-classify-sound d f = snd (k4-classification-sound d f)
```

Law 1.6: Endo Is Determined By Boundary Values

On a two-class carrier, agreement at the two boundary points already implies agreement everywhere.

```
-- § Law 1.6: Endo is determined by boundary values.
law1-6-endo-determined :
  (d : Distinction) →
  (f g : S d → S d) →
```

$$f(\ell d) \equiv g(\ell d) \rightarrow$$

$$f(r d) \equiv g(r d) \rightarrow$$

$$f \overset{\circ}{=} g$$

law1-6-endo-determined $d = K_4.\text{endo-determined } d$

Law 1.7: Classification Is Unique

If a case label c interprets to an endomorphism pointwise equal to f , then c is forced to be the label assigned to f by the classifier.

```
-- § Law 1.7: Classification is unique.
law1-7-classify-unique :
  (d : Distinction) →
  (f : S d → S d) →
  (c : EndoCase) →
  K4.interpret d c  $\overset{\circ}{=} f \rightarrow$ 
  c  $\equiv$  K4.classify d f
law1-7-classify-unique d f c p =
  k4-classification-unique d f c (fst (k4-classification-sound d f)) p (snd (k4-classification-sound d f))
```

Law 1.8: Classification Is a Bijection

The preceding laws identify EndoCase with endomorphisms modulo pointwise equality. The present statement records the left-inverse half of that bijection explicitly.

```
-- § Law 1.8: classify  $\circ$  interpret = id at the case-label level.
law1-8-classify-interpret :
  (d : Distinction) →
  (c : EndoCase) →
  K4.classify d (K4.interpret d c)  $\equiv$  c
law1-8-classify-interpret d = K4.classify-interpret d
```

At this point the four-case closure is explicit. The four labels are exhaustive and mutually distinct, so Figure 11.3 records the closure as the complete graph K_4 .

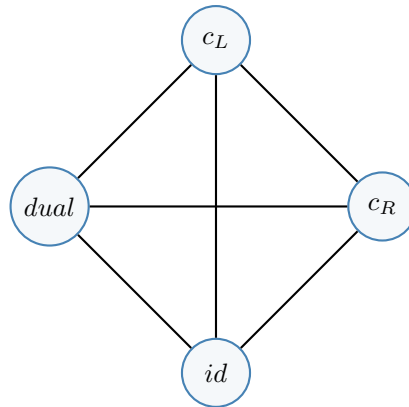


Figure 11.3: Exhaustive four-case endomorphism classification closes as the complete graph K_4 .

Chapter 12

Two Normal Form

The endomorphism space has now been classified completely, but the carrier S of a distinction is still abstract. Coverage reduces it to two boundary classes; what is not yet fixed is the explicit normal form of that carrier. This chapter supplies that normal form and proves that every distinction reaches it.

The cover yields maps in both directions between an arbitrary distinction carrier and the type `Two`. These maps are inverse up to the expected equalities, so under the conditions already fixed every distinction carrier is forced into isomorphism with a two-element type. At that point the remaining variation is not hidden structure; it is orientation, whether ℓ corresponds to L or to R.

The Two Type

The datatype `Two` is the explicit two-element object used to display that normal form. Its two constructors are distinct, and its cover is immediate by pattern matching. The later isomorphism theorem, not naming convention, makes it the normal form for arbitrary distinction carriers.

```
-- § The canonical two-element type.
data Two : Set where
  L : Two
  R : Two

-- § L and R are distinct.
Two-L≠R : L ≠ R
Two-L≠R ()

-- § Exhaustive two-class coverage for Two.
Two-cover : (x : Two) → (x ≡ L) ⊔ (x ≡ R)
Two-cover L = inj₁ refl
Two-cover R = inj₂ refl
```

```

-- § Two forms a canonical Distinction.
Two-distinction : Distinction
Two-distinction = record
{ S    = Two
; ℓ    = L
; r    = R
; ℓ≠r  = Two-L≠R
; cover = Two-cover
}

```

This supplies the constructive witness whose existence was deferred in Chapter 10. The object `Two-distinction` inhabits `Distinction` without further hypotheses, and it will be used throughout the remainder of the text whenever a concrete distinction is required.

Marked Subcarriers

Once `Two` has appeared, a carrier can be read by the two sides of the normal form. Such a reading is only a marking, not a hidden principle that classifies every predicate automatically. A predicate becomes a mark only when a constructive decision procedure accompanies it; without that decision, no side can be assigned without adding information not present in the predicate itself.

The marked subcarriers are therefore fibers of a displayed map into `Two`. The left fiber contains exactly the points marked `L`; the right fiber contains exactly the points marked `R`. The cover of `Two` then gives a two-way reading for each marked point, while the predicate construction records the honest direction: decidability produces a marker, not arbitrary truth.

```

-- § A constructive decision for a proposition.
Decidable : Set → Set
Decidable P = P ⊔ ¬ P

-- § A marker reads a carrier into the two normal-form sides.
Marker : Set → Set
Marker A = A → Two

-- § The fiber selected by a marker at a Two side.
MarkedSubcarrier : {A : Set} → Marker A → Two → Set
MarkedSubcarrier {A} marker side = Σ A (λ x → marker x ≡ side)

-- § The left selected subcarrier.
MarkedLeft : {A : Set} → Marker A → Set
MarkedLeft marker = MarkedSubcarrier marker L

-- § The right selected subcarrier.
MarkedRight : {A : Set} → Marker A → Set
MarkedRight marker = MarkedSubcarrier marker R

```



```

-- § Every marker classifies each input into one of the two sides.
marker-cover : {A : Set} (marker : Marker A) → (x : A) → (marker x ≡ L) ⊔ (marker x ≡ R)
marker-cover marker x = Two-cover (marker x)

-- § A decidable predicate determines a marker.
predicate-marker : {A : Set} → (P : A → Set) → ((x : A) → Decidable (P x)) → Marker A
predicate-marker P decide x with decide x
... | inj1 _ = L
... | inj2 _ = R

-- § Positive evidence sends the predicate marker to L.
predicate-marker-positive :
{A : Set} (P : A → Set) (decide : (x : A) → Decidable (P x)) →
(x : A) → P x → predicate-marker P decide x ≡ L
predicate-marker-positive P decide x px with decide x
... | inj1 _ = refl
... | inj2 notPx = ⊥-elim (notPx px)

-- § Negative evidence sends the predicate marker to R.
predicate-marker-negative :
{A : Set} (P : A → Set) (decide : (x : A) → Decidable (P x)) →
(x : A) → ¬ (P x) → predicate-marker P decide x ≡ R
predicate-marker-negative P decide x notPx with decide x
... | inj1 px = ⊥-elim (notPx px)
... | inj2 _ = refl

-- § The left fiber of any marker is decidable.
marker-left-decision : {A : Set} (marker : Marker A) → (x : A) → Decidable (marker x ≡ L)
marker-left-decision marker x with marker-cover marker x
... | inj1 p = inj1 p
... | inj2 p = inj2 (λ q → Two-L≠R (trans (sym q) p))

-- § Identity reading on Two for contact with a displayed mark.
idTwo : Two → Two
idTwo side = side

-- § Each marked point has contact with its displayed Two side.
marked-contact : {A : Set} (marker : Marker A) → A → Contact marker idTwo
marked-contact marker x = x , (marker x , refl)

```

Classifier Objects

The binary marker is not the last possible form of classification. A richer formal environment may carry a truth carrier with more structure than `Two`. What cannot change is the skeleton of classification: there must be a carrier of marks, an acknowledged truth point, a map from the object being read into that carrier, and a selected fiber over the truth point.

This does not import a categorical axiom into the development. The code below records the exact data that any such classifier must display before it can classify anything at all. The binary classifier is then recovered as the first case: `Two` with `L` as

the truth point. Thus the richer truth object is not identified with `Two`; it is a later generalization of the same marking discipline.

```
-- § A classifier object: truth carrier with a distinguished truth point.
record ClassifierObject : Set1 where
  field
    truthCarrier : Set
    truthPoint : truthCarrier

-- § A marker into an arbitrary classifier object.
ClassifierMarker : ClassifierObject → Set → Set
ClassifierMarker classifier A = A → ClassifierObject.truthCarrier classifier

-- § The truth fiber selected by the distinguished truth point.
TruthFiber : (classifier : ClassifierObject) {A : Set} → ClassifierMarker classifier A → Set
TruthFiber classifier {A} marker =  $\Sigma$  A ( $\lambda x \rightarrow$  marker  $x \equiv$  ClassifierObject.truthPoint classifier)

-- § A selected carrier classified by a truth fiber.
record ClassifiedSubcarrier (classifier : ClassifierObject) (A : Set) : Set1 where
  field
    characteristic      : ClassifierMarker classifier A
    selectedCarrier     : Set
    to-truth-fiber      : selectedCarrier → TruthFiber classifier characteristic
    from-truth-fiber    : TruthFiber classifier characteristic → selectedCarrier
    to-from-truth-fiber : (fiber : TruthFiber classifier characteristic) →
      to-truth-fiber (from-truth-fiber fiber)  $\equiv$  fiber
    from-to-truth-fiber : (selected : selectedCarrier) →
      from-truth-fiber (to-truth-fiber selected)  $\equiv$  selected

-- § Every marker classifies its own truth fiber.
truth-fiber-classified :
  (classifier : ClassifierObject) {A : Set} →
  (marker : ClassifierMarker classifier A) →
  ClassifiedSubcarrier classifier A
truth-fiber-classified classifier marker = record
{ characteristic = marker
; selectedCarrier = TruthFiber classifier marker
; to-truth-fiber =  $\lambda$  fiber  $\rightarrow$  fiber
; from-truth-fiber =  $\lambda$  fiber  $\rightarrow$  fiber
; to-from-truth-fiber =  $\lambda$  fiber  $\rightarrow$  refl
; from-to-truth-fiber =  $\lambda$  fiber  $\rightarrow$  refl
}

-- § A truth-fiber point contacts the classifier's truth point.
truth-fiber-contact :
  (classifier : ClassifierObject) {A : Set} →
  (marker : ClassifierMarker classifier A) →
  (fiber : TruthFiber classifier marker) →
  Contact marker ( $\lambda$  ( $\_$  : TruthFiber classifier marker)  $\rightarrow$  ClassifierObject.truthPoint classifier)
truth-fiber-contact classifier marker (x , proof) = x , ((x , proof) , proof)

-- § The binary classifier: Two with L as truth.
TwoClassifier : ClassifierObject
TwoClassifier = record
```

```

{ truthCarrier = Two
; truthPoint = L
}

-- § The binary classifier marker is exactly the earlier Marker.
two-classifier-marker : {A : Set} → ClassifierMarker TwoClassifier A ≡ Marker A
two-classifier-marker = refl

-- § The truth fiber of the binary classifier is exactly the left marked subcarrier.
two-classifier-truth-fiber : {A : Set} (marker : Marker A) → TruthFiber TwoClassifier marker ≡ MarkedLeft marker
two-classifier-truth-fiber marker = refl

-- § The left marked subcarrier is classified by the binary classifier.
two-classifier-marked-left : {A : Set} (marker : Marker A) → ClassifiedSubcarrier TwoClassifier A
two-classifier-marked-left marker = truth-fiber-classified TwoClassifier marker

-- § Binary truth-fiber contact is contact with the L side of Two.
two-truth-contact : {A : Set} (marker : Marker A) → MarkedLeft marker → Contact marker idTwo
two-truth-contact marker (x , proof) = x , (L , proof)

```

Distinction Cover as Binary Fiber

The cover field of a distinction now has a direct reading in the marking language. It is not a classifier for arbitrary predicates on the carrier. It is the displayed binary reading already present in the record: each point of the carrier is read as belonging to the left boundary class or to the right boundary class.

Under this marker the two fibers recover exactly the two boundary classes. The left fiber is the class of points equal to ℓ ; the right fiber is the class of points equal to r . Thus the cover does not hover as an external condition over the carrier. Once Two is available, it is the first fiber presentation of the distinction itself.

```

-- § The cover of a distinction determines its boundary marker.
distinction-marker : (d : Distinction) → Marker (S d)
distinction-marker d x with cover d x
... | inj1 _ = L
... | inj2 _ = R

-- § Equality with the left boundary marks a point as L.
distinction-marker-left :
(d : Distinction) (x : S d) →
x ≡ ℓ d →
distinction-marker d x ≡ L
distinction-marker-left d x x ≡ ℓ with cover d x
... | inj1 _ = refl
... | inj2 x ≡ r = ⊥-elim (ℓ ≠ r d (trans (sym x ≡ ℓ) x ≡ r))

-- § Equality with the right boundary marks a point as R.
distinction-marker-right :
(d : Distinction) (x : S d) →

```

```

 $x \equiv r \ d \rightarrow$ 
distinction-marker  $d \ x \equiv R$ 
distinction-marker-right  $d \ x \equiv r$  with cover  $d \ x$ 
... | inj1  $x \equiv \ell = \perp$ -elim ( $\ell \neq r \ d \ (\text{trans} (\text{sym } x \equiv \ell) \ x \equiv r)$ )
... | inj2  $\_ = \text{refl}$ 

-- § The left fiber contains only the left boundary class.
distinction-left-fiber-to-boundary :
( $d : \text{Distinction}$ )  $\rightarrow$ 
MarkedLeft (distinction-marker  $d$ )  $\rightarrow$ 
 $\Sigma (S \ d) (\lambda \ x \rightarrow x \equiv \ell \ d)$ 
distinction-left-fiber-to-boundary  $d \ (x, \text{marked-left})$  with cover  $d \ x$ 
... | inj1  $x \equiv \ell = x, \ x \equiv \ell$ 
... | inj2  $\_ = \perp$ -elim (Two-L $\neq$ R (sym marked-left))

-- § The left boundary class lies in the left fiber.
distinction-boundary-to-left-fiber :
( $d : \text{Distinction}$ )  $\rightarrow$ 
 $\Sigma (S \ d) (\lambda \ x \rightarrow x \equiv \ell \ d) \rightarrow$ 
MarkedLeft (distinction-marker  $d$ )
distinction-boundary-to-left-fiber  $d \ (x, \ x \equiv \ell) =$ 
 $x, \text{distinction-marker-left } d \ x \equiv \ell$ 

-- § The right fiber contains only the right boundary class.
distinction-right-fiber-to-boundary :
( $d : \text{Distinction}$ )  $\rightarrow$ 
MarkedRight (distinction-marker  $d$ )  $\rightarrow$ 
 $\Sigma (S \ d) (\lambda \ x \rightarrow x \equiv r \ d)$ 
distinction-right-fiber-to-boundary  $d \ (x, \text{marked-right})$  with cover  $d \ x$ 
... | inj1  $\_ = \perp$ -elim (Two-L $\neq$ R marked-right)
... | inj2  $x \equiv r = x, \ x \equiv r$ 

-- § The right boundary class lies in the right fiber.
distinction-boundary-to-right-fiber :
( $d : \text{Distinction}$ )  $\rightarrow$ 
 $\Sigma (S \ d) (\lambda \ x \rightarrow x \equiv r \ d) \rightarrow$ 
MarkedRight (distinction-marker  $d$ )
distinction-boundary-to-right-fiber  $d \ (x, \ x \equiv r) =$ 
 $x, \text{distinction-marker-right } d \ x \equiv r$ 

-- § The original cover appears as a cover by marked fibers.
distinction-fiber-cover :
( $d : \text{Distinction}$ )  $\rightarrow$ 
( $x : S \ d$ )  $\rightarrow$ 
MarkedLeft (distinction-marker  $d$ )  $\uplus$  MarkedRight (distinction-marker  $d$ )
distinction-fiber-cover  $d \ x$  with marker-cover (distinction-marker  $d$ )  $x$ 
... | inj1 marked-left = inj1 ( $x, \text{marked-left}$ )
... | inj2 marked-right = inj2 ( $x, \text{marked-right}$ )

-- § The left boundary fiber is classified by the binary classifier.
distinction-classified-left :
( $d : \text{Distinction}$ )  $\rightarrow$ 
ClassifiedSubcarrier TwoClassifier ( $S \ d$ )
distinction-classified-left  $d = \text{two-classifier-marked-left } (\text{distinction-marker } d)$ 

```

```

-- § The cover packaged as the binary fiber reading of a distinction.
record DistinctionFiberReading (d : Distinction) : Set1 where
field
  boundary-marker      : Marker (S d)
  left-boundary-marked  : boundary-marker (ℓ d) ≡ L
  right-boundary-marked : boundary-marker (r d) ≡ R
  left-fiber-to-boundary : MarkedLeft boundary-marker → Σ (S d) (λ x → x ≡ ℓ d)
  boundary-to-left-fiber : Σ (S d) (λ x → x ≡ ℓ d) → MarkedLeft boundary-marker
  right-fiber-to-boundary : MarkedRight boundary-marker → Σ (S d) (λ x → x ≡ r d)
  boundary-to-right-fiber : Σ (S d) (λ x → x ≡ r d) → MarkedRight boundary-marker
  fiber-cover           : (x : S d) → MarkedLeft boundary-marker ⊔ MarkedRight boundary-marker
  classified-left-fiber  : ClassifiedSubcarrier TwoClassifier (S d)

-- § Every distinction carries its own binary fiber reading.
distinction-fiber-reading :
  (d : Distinction) →
  DistinctionFiberReading d
distinction-fiber-reading d = record
{ boundary-marker      = distinction-marker d
; left-boundary-marked = distinction-marker-left d (ℓ d) refl
; right-boundary-marked = distinction-marker-right d (r d) refl
; left-fiber-to-boundary = distinction-left-fiber-to-boundary d
; boundary-to-left-fiber = distinction-boundary-to-left-fiber d
; right-fiber-to-boundary = distinction-right-fiber-to-boundary d
; boundary-to-right-fiber = distinction-boundary-to-right-fiber d
; fiber-cover           = distinction-fiber-cover d
; classified-left-fiber  = distinction-classified-left d
}

```

Stable Classified Contact

The point of contact with richer mathematical languages is now visible. A later theory may speak of relations, characteristic maps, truth values, or classified subobjects. Those names are harmless when their data are displayed. What matters is that relation, stability, and classification arrive together: a joint imprint must not collapse origins, and the selected part of the origin carrier must be the truth fiber of a displayed marker.

This is an interface of union, not rivalry. It does not deny later languages their own constructions. It asks them to expose the contact data by which those constructions become readable: two evaluations, a non-collapsing joint imprint, a classifier object, and a characteristic map into that classifier. Once those are present, the selected subcarrier is no longer a free predicate hovering over the system; it is the fiber forced by the truth anchor.

```

-- § Stable classified contact: relation, stability, and classification together.
record StableClassifiedContact (classifier : ClassifierObject) (X Y Z : Set) : Set1 where
field

```

```

scc-leftEval  :  $X \rightarrow Y$ 
scc-rightEval :  $X \rightarrow Z$ 
scc-stableRelation : StableRelation scc-leftEval scc-rightEval
scc-marker    : ClassifierMarker classifier X

-- § The selected truth fiber forced by a stable classified contact.
scc-classified-subcarrier :
  (classifier : ClassifierObject) {X Y Z : Set} →
  StableClassifiedContact classifier X Y Z →
  ClassifiedSubcarrier classifier X
scc-classified-subcarrier classifier contact =
  truth-fiber-classified classifier (StableClassifiedContact.scc-marker contact)

-- § Stability still forces component agreement back to origin equality.
scc-stable-components :
  (classifier : ClassifierObject) {X Y Z : Set} →
  (contact : StableClassifiedContact classifier X Y Z) →
  (x y : X) →
  StableClassifiedContact.scc-leftEval contact x ≡ StableClassifiedContact.scc-leftEval contact y →
  StableClassifiedContact.scc-rightEval contact x ≡ StableClassifiedContact.scc-rightEval contact y →
  x ≡ y
scc-stable-components classifier contact x y same-left same-right =
  stable-relation-components
  (StableClassifiedContact.scc-leftEval contact)
  (StableClassifiedContact.scc-rightEval contact)
  (StableClassifiedContact.scc-stableRelation contact)
  x y same-left same-right

-- § The classifier marker of a stable contact selects a truth-fiber point by contact.
scc-truth-fiber-contact :
  (classifier : ClassifierObject) {X Y Z : Set} →
  (contact : StableClassifiedContact classifier X Y Z) →
  (fiber : TruthFiber classifier (StableClassifiedContact.scc-marker contact)) →
  Contact (StableClassifiedContact.scc-marker contact)
  (λ ( _ : TruthFiber classifier (StableClassifiedContact.scc-marker contact)) →
    ClassifierObject.truthPoint classifier)
scc-truth-fiber-contact classifier contact =
  truth-fiber-contact classifier (StableClassifiedContact.scc-marker contact)

```

The interface is now closed in the modest sense appropriate here. A stable classified contact is not recovered from a selected fiber alone: the fiber remembers classification, not the two evaluations whose joint imprint was stable. But once those evaluations, their stability witness, and the marker are displayed, no further datum is needed to assemble the interface. The binary classifier is also not a metaphor for the general case; it is the definitional first case of the same pattern. Finally, stability may be transported only through maps that do not collapse information. This last condition is essential: arbitrary postcomposition can identify values and destroy stability.

```

-- § A stable classified contact is assembled from exactly its displayed data.
scc-from-data :
  (classifier : ClassifierObject) {X Y Z : Set} →

```

```

(leftEval : X → Y) →
(rightEval : X → Z) →
StableRelation leftEval rightEval →
ClassifierMarker classifier X →
StableClassifiedContact classifier X Y Z
scc-from-data classifier leftEval rightEval stable marker = record
{ scc-leftEval   = leftEval
; scc-rightEval  = rightEval
; scc-stableRelation = stable
; scc-marker    = marker
}

-- § Reassembling a contact from its displayed fields returns the same interface.
scc-fields-reconstruct :
(classifier : ClassifierObject) {X Y Z : Set} →
(contact : StableClassifiedContact classifier X Y Z) →
scc-from-data classifier
(StableClassifiedContact.scc-leftEval contact)
(StableClassifiedContact.scc-rightEval contact)
(StableClassifiedContact.scc-stableRelation contact)
(StableClassifiedContact.scc-marker contact)
≡ contact
scc-fields-reconstruct classifier contact = refl

-- § The binary classifier reconstruction bundles the definitional first case.
record BinaryClassifierReconstruction (A : Set) (marker : Marker A) : Set1 where
field
marker-is-binary : ClassifierMarker TwoClassifier A ≡ Marker A
truth-fiber-is-left : TruthFiber TwoClassifier marker ≡ MarkedLeft marker
classified-left : ClassifiedSubcarrier TwoClassifier A
truth-contact-is-left : MarkedLeft marker → Contact marker idTwo

-- § Two is the first classifier interface, with the left fiber as truth fiber.
binary-classifier-reconstruction :
{A : Set} (marker : Marker A) →
BinaryClassifierReconstruction A marker
binary-classifier-reconstruction marker = record
{ marker-is-binary = two-classifier-marker
; truth-fiber-is-left = two-classifier-truth-fiber marker
; classified-left = two-classifier-marked-left marker
; truth-contact-is-left = two-truth-contact marker
}

-- § Product equality projects to equality of first components.
product-fst-eq :
{A B : Set} {p q : A × B} →
p ≡ q → fst p ≡ fst q
product-fst-eq refl = refl

-- § Product equality projects to equality of second components.
product-snd-eq :
{A B : Set} {p q : A × B} →
p ≡ q → snd p ≡ snd q
product-snd-eq refl = refl

```

```

-- § Stable relations survive postcomposition by non-collapsing maps.
stable-relation-postcompose :
  {X Y Z Y' Z' : Set} →
  (leftEval : X → Y) →
  (rightEval : X → Z) →
  (leftMap : Y → Y') →
  (rightMap : Z → Z') →
  NonCollapsing leftMap →
  NonCollapsing rightMap →
  StableRelation leftEval rightEval →
  StableRelation (λ x → leftMap (leftEval x)) (λ x → rightMap (rightEval x))
stable-relation-postcompose leftEval rightEval leftMap rightMap leftNC rightNC stable x y same =
  stable x y
  (pair-eq
    (leftNC (leftEval x) (leftEval y) (product-fst-eq same))
    (rightNC (rightEval x) (rightEval y) (product-snd-eq same)))

-- § Stable classified contact survives such non-collapsing postcomposition.
scc-postcompose :
  (classifier : ClassifierObject) {X Y Z Y' Z' : Set} →
  StableClassifiedContact classifier X Y Z →
  (leftMap : Y → Y') →
  (rightMap : Z → Z') →
  NonCollapsing leftMap →
  NonCollapsing rightMap →
  StableClassifiedContact classifier X Y' Z'
scc-postcompose classifier contact leftMap rightMap leftNC rightNC = record
{ scc-leftEval  = λ x → leftMap (StableClassifiedContact.scc-leftEval contact x)
; scc-rightEval = λ x → rightMap (StableClassifiedContact.scc-rightEval contact x)
; scc-stableRelation =
  stable-relation-postcompose
    (StableClassifiedContact.scc-leftEval contact)
    (StableClassifiedContact.scc-rightEval contact)
    leftMap rightMap leftNC rightNC
    (StableClassifiedContact.scc-stableRelation contact)
; scc-marker    = StableClassifiedContact.scc-marker contact
}

```

Stable Reading Interface

The previous constructions can now be collected without naming any later language. The interface records only the roles that have already appeared: a carrier being read, two displayed readings of it, a non-collapsing joint imprint, and a marker into a classifier object. It is the stable reading surface on which richer structures can later display their data.

The point of the record is architectural. It makes the translation surface explicit while keeping the level fixed. From a stable reading one obtains a stable classified contact, a classified truth fiber, and contact with the truth point. Conversely, the first

binary instance is already present in a distinction: the cover determines a marker, and the separation of the two boundary classes makes that marker non-collapsing.

```
-- § Stable reading interface: carrier, two readings, stability, marker.
record StableReadingInterface (classifier : ClassifierObject) : Set1 where
  field
    sri-carrier      : Set
    sri-leftCarrier  : Set
    sri-rightCarrier : Set
    sri-leftReading  : sri-carrier → sri-leftCarrier
    sri-rightReading : sri-carrier → sri-rightCarrier
    sri-stableReading : StableRelation sri-leftReading sri-rightReading
    sri-marker       : ClassifierMarker classifier sri-carrier

-- § A stable reading is a stable classified contact in displayed form.
sri-to-stable-classified-contact :
  (classifier : ClassifierObject) →
  (reading : StableReadingInterface classifier) →
  StableClassifiedContact classifier
  (StableReadingInterface.sri-carrier reading)
  (StableReadingInterface.sri-leftCarrier reading)
  (StableReadingInterface.sri-rightCarrier reading)
sri-to-stable-classified-contact classifier reading = record
{ scc-leftEval  = StableReadingInterface.sri-leftReading reading
; scc-rightEval = StableReadingInterface.sri-rightReading reading
; scc-stableRelation = StableReadingInterface.sri-stableReading reading
; scc-marker    = StableReadingInterface.sri-marker reading
}

-- § The marker of a stable reading classifies its truth fiber.
sri-classified-subcarrier :
  (classifier : ClassifierObject) →
  (reading : StableReadingInterface classifier) →
  ClassifiedSubcarrier classifier (StableReadingInterface.sri-carrier reading)
sri-classified-subcarrier classifier reading =
  scc-classified-subcarrier classifier (sri-to-stable-classified-contact classifier reading)

-- § A truth-fiber point of a stable reading contacts the truth point.
sri-truth-fiber-contact :
  (classifier : ClassifierObject) →
  (reading : StableReadingInterface classifier) →
  (fiber : TruthFiber classifier (StableReadingInterface.sri-marker reading)) →
  Contact (StableReadingInterface.sri-marker reading)
  (λ _ : TruthFiber classifier (StableReadingInterface.sri-marker reading)) →
  ClassifierObject.truthPoint classifier)
sri-truth-fiber-contact classifier reading =
  truth-fiber-contact classifier (StableReadingInterface.sri-marker reading)

-- § Stability of a stable reading returns component agreement to origin equality.
sri-stable-components :
  (classifier : ClassifierObject) →
  (reading : StableReadingInterface classifier) →
  (x y : StableReadingInterface.sri-carrier reading) →
  StableReadingInterface.sri-leftReading reading x ≡ StableReadingInterface.sri-leftReading reading y →
```

```

StableReadingInterface.sri-rightReading reading x  $\equiv$  StableReadingInterface.sri-rightReading reading y  $\rightarrow$ 
x  $\equiv$  y
sri-stable-components classifier reading x y same-left same-right =
stable-relation-components
  (StableReadingInterface.sri-leftReading reading)
  (StableReadingInterface.sri-rightReading reading)
  (StableReadingInterface.sri-stableReading reading)
  x y same-left same-right

-- § Stable readings survive postcomposition by non-collapsing maps.
sri-postcompose :
  (classifier : ClassifierObject)  $\rightarrow$ 
  (reading : StableReadingInterface classifier)  $\rightarrow$ 
  {Y' Z' : Set}  $\rightarrow$ 
  (leftMap : StableReadingInterface.sri-leftCarrier reading  $\rightarrow$  Y')  $\rightarrow$ 
  (rightMap : StableReadingInterface.sri-rightCarrier reading  $\rightarrow$  Z')  $\rightarrow$ 
  NonCollapsing leftMap  $\rightarrow$ 
  NonCollapsing rightMap  $\rightarrow$ 
  StableReadingInterface classifier
sri-postcompose classifier reading {Y' = Y'} {Z' = Z'} leftMap rightMap leftNC rightNC = record
{ sri-carrier      = StableReadingInterface.sri-carrier reading
; sri-leftCarrier  = Y'
; sri-rightCarrier = Z'
; sri-leftReading  =  $\lambda$  x  $\rightarrow$  leftMap (StableReadingInterface.sri-leftReading reading x)
; sri-rightReading =  $\lambda$  x  $\rightarrow$  rightMap (StableReadingInterface.sri-rightReading reading x)
; sri-stableReading =
  stable-relation-postcompose
  (StableReadingInterface.sri-leftReading reading)
  (StableReadingInterface.sri-rightReading reading)
  leftMap rightMap leftNC rightNC
  (StableReadingInterface.sri-stableReading reading)
; sri-marker      = StableReadingInterface.sri-marker reading
}

-- § The distinction marker is non-collapsing.
distinction-marker-noncollapsing :
  (d : Distinction)  $\rightarrow$ 
  NonCollapsing (distinction-marker d)
distinction-marker-noncollapsing d x y same with cover d x | cover d y
... | inj1 x  $\equiv$   $\ell$  | inj1 y  $\equiv$   $\ell$  = trans x  $\equiv$   $\ell$  (sym y  $\equiv$   $\ell$ )
... | inj1 _ | inj2 _ =  $\perp$ -elim (Two-L $\neq$ R same)
... | inj2 _ | inj1 _ =  $\perp$ -elim (Two-L $\neq$ R (sym same))
... | inj2 x  $\equiv$  r | inj2 y  $\equiv$  r = trans x  $\equiv$  r (sym y  $\equiv$  r)

-- § The distinction marker read twice is a stable relation.
distinction-marker-stable-relation :
  (d : Distinction)  $\rightarrow$ 
  StableRelation (distinction-marker d) (distinction-marker d)
distinction-marker-stable-relation d x y same =
  distinction-marker-noncollapsing d x y (product-fst-eq same)

-- § Every distinction gives the first binary stable reading.
distinction-stable-reading :
  (d : Distinction)  $\rightarrow$ 

```

```

StableReadingInterface TwoClassifier
distinction-stable-reading d = record
{ sri-carrier      = S d
; sri-leftCarrier  = Two
; sri-rightCarrier = Two
; sri-leftReading  = distinction-marker d
; sri-rightReading = distinction-marker d
; sri-stableReading = distinction-marker-stable-relation d
; sri-marker       = distinction-marker d
}

-- § Reading closure: the fiber and stable-reading layers return to Distinction.
record ReadingClosure (d : Distinction) : Set1 where
field
  rc-fiber-reading  : DistinctionFiberReading d
  rc-stable-reading : StableReadingInterface TwoClassifier
  rc-classified-subcarrier : ClassifiedSubcarrier TwoClassifier (S d)
  rc-stable-contact  : StableClassifiedContact TwoClassifier (S d) Two Two

-- § Every distinction closes its reading branch without further data.
reading-closure :
(d : Distinction) →
ReadingClosure d
reading-closure d = record
{ rc-fiber-reading  = distinction-fiber-reading d
; rc-stable-reading = distinction-stable-reading d
; rc-classified-subcarrier = distinction-classified-left d
; rc-stable-contact =
  sri-to-stable-classified-contact TwoClassifier (distinction-stable-reading d)
}

```

From Stable Reading to Morphism

A stable reading presents one object of the classifier interface. The categorical layer begins by comparing such presentations through maps that preserve the displayed readings, the classifier marker, and the non-collapsing origin data.

A structure-preserving map carries origins, left readings, and right readings forward; it makes the two reading squares commute; it preserves the classifier marker; and its origin map is non-collapsing. The last condition prevents transport from identifying origins that were separated before transport. Such an identification would not preserve the reading; it would collapse the presentation being read.

These maps supply the first category data for stable readings: objects, homs, identities, composition, and the corresponding laws. The laws are stated up to visible equality of the three displayed maps rather than equality of all proof fields. This keeps the construction inside the same intensional discipline used from the beginning and avoids assuming function extensionality.

```

-- § Morphism of stable readings: maps preserving readings and marker.
record StableReadingMorphism
  (classifier : ClassifierObject)
  (source target : StableReadingInterface classifier) : Set1 where
  field
    srm-carrier-map      : StableReadingInterface.sri-carrier source →
                          StableReadingInterface.sri-carrier target
    srm-left-map         : StableReadingInterface.sri-leftCarrier source →
                          StableReadingInterface.sri-leftCarrier target
    srm-right-map        : StableReadingInterface.sri-rightCarrier source →
                          StableReadingInterface.sri-rightCarrier target
    srm-carrier-noncollapse : NonCollapsing srm-carrier-map
    srm-left-commutes    : (x : StableReadingInterface.sri-carrier source) →
                          StableReadingInterface.sri-leftReading target (srm-carrier-map x) ≡
                          srm-left-map (StableReadingInterface.sri-leftReading source x)
    srm-right-commutes   : (x : StableReadingInterface.sri-carrier source) →
                          StableReadingInterface.sri-rightReading target (srm-carrier-map x) ≡
                          srm-right-map (StableReadingInterface.sri-rightReading source x)
    srm-marker-commutes  : (x : StableReadingInterface.sri-carrier source) →
                          StableReadingInterface.sri-marker target (srm-carrier-map x) ≡
                          StableReadingInterface.sri-marker source x

open StableReadingMorphism public

-- § StructureMap is the local name for a stable-reading morphism.
StructureMap :
  (classifier : ClassifierObject) →
  StableReadingInterface classifier →
  StableReadingInterface classifier →
  Set1
StructureMap classifier source target =
  StableReadingMorphism classifier source target

-- § A structure map transports the selected truth fiber forward.
srm-truth-fiber-map :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  StructureMap classifier source target →
  TruthFiber classifier (StableReadingInterface.sri-marker source) →
  TruthFiber classifier (StableReadingInterface.sri-marker target)
srm-truth-fiber-map classifier morphism (x , proof) =
  srm-carrier-map morphism x ,
  trans (srm-marker-commutes morphism x) proof

-- § In the binary case, transported markers remain two-covered.
srm-binary-marker-cover :
  {source target : StableReadingInterface TwoClassifier} →
  (morphism : StructureMap TwoClassifier source target) →
  (x : StableReadingInterface.sri-carrier source) →
  (StableReadingInterface.sri-marker target (srm-carrier-map morphism x) ≡ L) ⊔
  (StableReadingInterface.sri-marker target (srm-carrier-map morphism x) ≡ R)
srm-binary-marker-cover {target = target} morphism x =
  marker-cover (StableReadingInterface.sri-marker target) (srm-carrier-map morphism x)

```

```

-- § Target-side agreement reflects back to origin equality through a structure map.
srm-target-agreement-to-origin :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
(morphism : StructureMap classifier source target) →
(x y : StableReadingInterface.sri-carrier source) →
StableReadingInterface.sri-leftReading target (srm-carrier-map morphism x) ≡
StableReadingInterface.sri-leftReading target (srm-carrier-map morphism y) →
StableReadingInterface.sri-rightReading target (srm-carrier-map morphism x) ≡
StableReadingInterface.sri-rightReading target (srm-carrier-map morphism y) →
x ≡ y
srm-target-agreement-to-origin classifier {target = target} morphism x y same-left same-right =
srm-carrier-noncollapse morphism x y
(sri-stable-components classifier target
(srm-carrier-map morphism x)
(srm-carrier-map morphism y)
same-left same-right)

-- § Identity structure map.
stable-reading-id-morphism :
(classifier : ClassifierObject) →
(reading : StableReadingInterface classifier) →
StructureMap classifier reading reading
stable-reading-id-morphism classifier reading = record
{ srm-carrier-map      = id
; srm-left-map        = id
; srm-right-map       = id
; srm-carrier-noncollapse = λ x y same → same
; srm-left-commutes    = λ x → refl
; srm-right-commutes   = λ x → refl
; srm-marker-commutes  = λ x → refl
}

-- § Composition of structure maps.
stable-reading-morphism-compose :
(classifier : ClassifierObject) →
{first middle last : StableReadingInterface classifier} →
StructureMap classifier middle last →
StructureMap classifier first middle →
StructureMap classifier first last
stable-reading-morphism-compose classifier right left = record
{ srm-carrier-map = λ x → srm-carrier-map right (srm-carrier-map left x)
; srm-left-map   = λ y → srm-left-map right (srm-left-map left y)
; srm-right-map  = λ z → srm-right-map right (srm-right-map left z)
; srm-carrier-noncollapse =
λ x y same →
srm-carrier-noncollapse left x y
(srm-carrier-noncollapse right
(srm-carrier-map left x)
(srm-carrier-map left y)
same)
; srm-left-commutes =
λ x →
trans

```

```

      (srm-left-commutes right (srm-carrier-map left x))
      (cong (srm-left-map right) (srm-left-commutes left x))
; srm-right-commutes =
  λ x →
    trans
      (srm-right-commutes right (srm-carrier-map left x))
      (cong (srm-right-map right) (srm-right-commutes left x))
; srm-marker-commutes =
  λ x →
    trans
      (srm-marker-commutes right (srm-carrier-map left x))
      (srm-marker-commutes left x)
}

-- § Visible equality of structure maps, ignoring proof-field bureaucracy.
record StableReadingMorphismEq
(classifier : ClassifierObject)
{source target : StableReadingInterface classifier}
(f g : StructureMap classifier source target) : Set where
field
  srm-carrier-eq : (x : StableReadingInterface.sri-carrier source) →
    srm-carrier-map f x ≡ srm-carrier-map g x
  srm-left-eq    : (y : StableReadingInterface.sri-leftCarrier source) →
    srm-left-map f y ≡ srm-left-map g y
  srm-right-eq   : (z : StableReadingInterface.sri-rightCarrier source) →
    srm-right-map f z ≡ srm-right-map g z

open StableReadingMorphismEq public

-- § Visible equality is reflexive.
stable-reading-morphism-eq-refl :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
(f : StructureMap classifier source target) →
StableReadingMorphismEq classifier f f
stable-reading-morphism-eq-refl classifier f = record
{ srm-carrier-eq = λ x → refl
; srm-left-eq    = λ y → refl
; srm-right-eq   = λ z → refl
}

-- § Visible equality is symmetric.
stable-reading-morphism-eq-sym :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
{f g : StructureMap classifier source target} →
StableReadingMorphismEq classifier f g →
StableReadingMorphismEq classifier g f
stable-reading-morphism-eq-sym classifier eq = record
{ srm-carrier-eq = λ x → sym (srm-carrier-eq eq x)
; srm-left-eq    = λ y → sym (srm-left-eq eq y)
; srm-right-eq   = λ z → sym (srm-right-eq eq z)
}

```

```

-- § Visible equality is transitive.
stable-reading-morphism-eq-trans :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  {f g h : StructureMap classifier source target} →
  StableReadingMorphismEq classifier f g →
  StableReadingMorphismEq classifier g h →
  StableReadingMorphismEq classifier f h
stable-reading-morphism-eq-trans classifier eq1 eq2 = record
  { srm-carrier-eq = λ x → trans (srm-carrier-eq eq1 x) (srm-carrier-eq eq2 x)
  ; srm-left-eq   = λ y → trans (srm-left-eq eq1 y) (srm-left-eq eq2 y)
  ; srm-right-eq  = λ z → trans (srm-right-eq eq1 z) (srm-right-eq eq2 z)
  }

-- § Left identity law for structure maps.
stable-reading-id-left :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (f : StructureMap classifier source target) →
  StableReadingMorphismEq classifier
  (stable-reading-morphism-compose classifier (stable-reading-id-morphism classifier target) f)
  f
stable-reading-id-left classifier f = record
  { srm-carrier-eq = λ x → refl
  ; srm-left-eq   = λ y → refl
  ; srm-right-eq  = λ z → refl
  }

-- § Right identity law for structure maps.
stable-reading-id-right :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (f : StructureMap classifier source target) →
  StableReadingMorphismEq classifier
  (stable-reading-morphism-compose classifier f (stable-reading-id-morphism classifier source))
  f
stable-reading-id-right classifier f = record
  { srm-carrier-eq = λ x → refl
  ; srm-left-eq   = λ y → refl
  ; srm-right-eq  = λ z → refl
  }

-- § Associativity law for structure maps.
stable-reading-assoc :
  (classifier : ClassifierObject) →
  {a b c d : StableReadingInterface classifier} →
  (h : StructureMap classifier c d) →
  (g : StructureMap classifier b c) →
  (f : StructureMap classifier a b) →
  StableReadingMorphismEq classifier
  (stable-reading-morphism-compose classifier h
    (stable-reading-morphism-compose classifier g f))
  (stable-reading-morphism-compose classifier
    (stable-reading-morphism-compose classifier h g) f)

```

```

stable-reading-assoc classifier h g f = record
{ srm-carrier-eq = λ x → refl
; srm-left-eq   = λ y → refl
; srm-right-eq  = λ z → refl
}

-- § Minimal category data, with equality supplied on homs.
record CategoryData : Setω where
field
  Obj      : Set1
  Hom      : Obj → Obj → Set1
  HomEq    : {A B : Obj} → Hom A B → Hom A B → Set
  HomEq-refl : {A B : Obj} → (f : Hom A B) → HomEq f f
  HomEq-sym  : {A B : Obj} → {f g : Hom A B} → HomEq f g → HomEq g f
  HomEq-trans : {A B : Obj} → {f g h : Hom A B} → HomEq f g → HomEq g h → HomEq f h
  idC       : (A : Obj) → Hom A A
  compC     : {A B C : Obj} → Hom B C → Hom A B → Hom A C
  id-leftC  : {A B : Obj} → (f : Hom A B) → HomEq (compC (idC B) f) f
  id-rightC : {A B : Obj} → (f : Hom A B) → HomEq (compC f (idC A)) f
  assocC    : {A B C D : Obj} →
    (h : Hom C D) → (g : Hom B C) → (f : Hom A B) →
    HomEq (compC h (compC g f)) (compC (compC h g) f)

open CategoryData public

-- § Stable readings over a classifier form category data.
stable-reading-category-data :
(classifier : ClassifierObject) →
  CategoryData
stable-reading-category-data classifier = record
{ Obj      = StableReadingInterface classifier
; Hom      = StructureMap classifier
; HomEq    = λ {source} {target} f g →
    StableReadingMorphismEq classifier {source = source} {target = target} f g
; HomEq-refl = stable-reading-morphism-eq-refl classifier
; HomEq-sym  = stable-reading-morphism-eq-sym classifier
; HomEq-trans = stable-reading-morphism-eq-trans classifier
; idC       = stable-reading-id-morphism classifier
; compC     = stable-reading-morphism-compose classifier
; id-leftC  = stable-reading-id-left classifier
; id-rightC = stable-reading-id-right classifier
; assocC    = stable-reading-assoc classifier
}

```

Reindexing and Pullback Shape

With structure maps in place, reindexing becomes the first categorical test. A marker visible at the target of a structure map has a controlled preimage at the source. In ordinary categorical language this is the beginning of pullback behaviour: the target marker is pulled back along the origin map, compared with the source marker, and carried together with its truth fiber and contact with the truth point.

The statement is local. It records the pullback-shaped square carried by a single stable reading morphism, not a global supply of pullbacks in the ambient category. The comparison part is isolated below through mono-like subcarrier data and cone comparison maps. Here the point is that classified visibility at the target has a stable preimage at the source, obtained by composing with the structure map already displayed.

```
-- § Pull back the target marker along a structure map.
srp-pullback-marker :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
StructureMap classifier source target →
ClassifierMarker classifier (StableReadingInterface.sri-carrier source)
srp-pullback-marker classifier {target = target} morphism x =
StableReadingInterface.sri-marker target (srp-carrier-map morphism x)

-- § The pulled-back marker is the displayed target marker after transport.
srp-pullback-marker-is-target :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
(morphism : StructureMap classifier source target) →
(x : StableReadingInterface.sri-carrier source) →
srp-pullback-marker classifier morphism x ≡
StableReadingInterface.sri-marker target (srp-carrier-map morphism x)
srp-pullback-marker-is-target classifier morphism x = refl

-- § The pulled-back marker returns to the source marker.
srp-pullback-marker-returns-source :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
(morphism : StructureMap classifier source target) →
(x : StableReadingInterface.sri-carrier source) →
srp-pullback-marker classifier morphism x ≡
StableReadingInterface.sri-marker source x
srp-pullback-marker-returns-source classifier morphism x =
srp-marker-commutes morphism x

-- § A source truth-fiber point induces a pulled-back truth-fiber point.
srp-source-fiber-to-pullback :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
(morphism : StructureMap classifier source target) →
TruthFiber classifier (StableReadingInterface.sri-marker source) →
TruthFiber classifier (srp-pullback-marker classifier morphism)
srp-source-fiber-to-pullback classifier morphism (x , proof) =
x , trans (srp-marker-commutes morphism x) proof

-- § A pulled-back truth-fiber point returns to the source truth fiber.
srp-pullback-fiber-to-source :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
(morphism : StructureMap classifier source target) →
```

```

TruthFiber classifier (srm-pullback-marker classifier morphism) →
TruthFiber classifier (StableReadingInterface.sri-marker source)
srm-pullback-fiber-to-source classifier morphism (x , proof) =
x , trans (sym (srm-marker-commutes morphism x)) proof

-- § A pulled-back truth-fiber point displays a target truth-fiber point.
srm-pullback-fiber-to-target :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
(morphism : StructureMap classifier source target) →
TruthFiber classifier (srm-pullback-marker classifier morphism) →
TruthFiber classifier (StableReadingInterface.sri-marker target)
srm-pullback-fiber-to-target classifier morphism (x , proof) =
srm-carrier-map morphism x , proof

-- § The pulled-back marker classifies its truth fiber.
srm-pullback-classified :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
(morphism : StructureMap classifier source target) →
ClassifiedSubcarrier classifier (StableReadingInterface.sri-carrier source)
srm-pullback-classified classifier morphism =
truth-fiber-classified classifier (srm-pullback-marker classifier morphism)

-- § A pulled-back truth-fiber point contacts the truth point.
srm-pullback-contact :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
(morphism : StructureMap classifier source target) →
(fiber : TruthFiber classifier (srm-pullback-marker classifier morphism)) →
Contact (srm-pullback-marker classifier morphism)
(λ ( _ : TruthFiber classifier (srm-pullback-marker classifier morphism)) →
ClassifierObject.truthPoint classifier)
srm-pullback-contact classifier morphism =
truth-fiber-contact classifier (srm-pullback-marker classifier morphism)

-- § Stable reindexing packages marker, fiber, and contact pullback.
record StableReindexing
(classifier : ClassifierObject)
{source target : StableReadingInterface classifier}
(morphism : StructureMap classifier source target) : Set1 where
field
sr-pulled-marker      : ClassifierMarker classifier (StableReadingInterface.sri-carrier source)
sr-marker-is-pullback : (x : StableReadingInterface.sri-carrier source) →
                        sr-pulled-marker x ≡
                        StableReadingInterface.sri-marker target (srm-carrier-map morphism x)
sr-marker-returns-source : (x : StableReadingInterface.sri-carrier source) →
                        sr-pulled-marker x ≡
                        StableReadingInterface.sri-marker source x
sr-pulled-classified   : ClassifiedSubcarrier classifier (StableReadingInterface.sri-carrier source)
sr-source-to-pullback : TruthFiber classifier (StableReadingInterface.sri-marker source) →
                        TruthFiber classifier sr-pulled-marker
sr-pullback-to-source : TruthFiber classifier sr-pulled-marker →
                        TruthFiber classifier (StableReadingInterface.sri-marker source)

```

```

sr-pullback-contact : (fiber : TruthFiber classifier sr-pulled-marker) →
  Contact sr-pulled-marker
  (λ (x : TruthFiber classifier sr-pulled-marker) →
    ClassifierObject.truthPoint classifier)

open StableReindexing public

-- § Every structure map supplies stable reindexing.
stable-reindexing :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (morphism : StructureMap classifier source target) →
  StableReindexing classifier morphism
stable-reindexing classifier morphism = record
{ sr-pulled-marker      = srm-pullback-marker classifier morphism
; sr-marker-is-pullback = λ x → refl
; sr-marker-returns-source = srm-marker-commutes morphism
; sr-pulled-classified   = srm-pullback-classified classifier morphism
; sr-source-to-pullback  = srm-source-fiber-to-pullback classifier morphism
; sr-pullback-to-source  = srm-pullback-fiber-to-source classifier morphism
; sr-pullback-contact    = srm-pullback-contact classifier morphism
}

-- § Pullback-shaped square for the target truth fiber along a structure map.
record PullbackShape
  (classifier : ClassifierObject)
  {source target : StableReadingInterface classifier}
  (morphism : StructureMap classifier source target) : Set1 where
  field
    pb-carrier : Set
    pb-source : pb-carrier → StableReadingInterface.sri-carrier source
    pb-target-fiber : pb-carrier → TruthFiber classifier (StableReadingInterface.sri-marker target)
    pb-square : (p : pb-carrier) →
      srm-carrier-map morphism (pb-source p) ≡ fst (pb-target-fiber p)
    pb-source-fiber : pb-carrier → TruthFiber classifier (StableReadingInterface.sri-marker source)

open PullbackShape public

-- § The canonical pullback shape carried by a structure map.
stable-reading-pullback-shape :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (morphism : StructureMap classifier source target) →
  PullbackShape classifier morphism
stable-reading-pullback-shape classifier morphism = record
{ pb-carrier = TruthFiber classifier (srm-pullback-marker classifier morphism)
; pb-source  = λ fiber → fst fiber
; pb-target-fiber = srm-pullback-fiber-to-target classifier morphism
; pb-square    = λ fiber → refl
; pb-source-fiber = srm-pullback-fiber-to-source classifier morphism
}

```

Mono-Like Subcarrier Data

The previous section isolated the square-shaped behaviour of a pulled fiber. The next ingredient is the selected carrier itself. A classified subcarrier is already equivalent to a truth fiber, but its inclusion into the ambient carrier must be displayed before any later categorical language can speak honestly about monic behaviour.

There is a small intensional point here. The selected carrier may carry proof data, and equality of proof data is not assumed. The record below is therefore not a categorical monomorphism. It records the visible inclusion into the ambient carrier and equips it with a visible equality: two selected points agree when their displayed ambient points agree. This is the amount of monic behaviour used by the constructions below.

```
-- § A classified subcarrier displayed by its truth-fiber inclusion.
record SubcarrierInclusion (classifier : ClassifierObject) (A : Set) : Set1 where
  field
    si-characteristic      : ClassifierMarker classifier A
    si-carrier             : Set
    si-to-truth-fiber      : si-carrier → TruthFiber classifier si-characteristic
    si-from-truth-fiber    : TruthFiber classifier si-characteristic → si-carrier
    si-to-from-truth-fiber : (fiber : TruthFiber classifier si-characteristic) →
      si-to-truth-fiber (si-from-truth-fiber fiber) ≡ fiber
    si-from-to-truth-fiber : (selected : si-carrier) →
      si-from-truth-fiber (si-to-truth-fiber selected) ≡ selected

open SubcarrierInclusion public

-- § The visible inclusion forgets only the classifier proof component.
subcarrier-include :
  (classifier : ClassifierObject) {A : Set} →
  (inclusion : SubcarrierInclusion classifier A) →
  si-carrier inclusion → A
subcarrier-include classifier inclusion selected =
  fst (si-to-truth-fiber inclusion selected)

-- § A selected point carries its displayed truth proof.
subcarrier-truth-proof :
  (classifier : ClassifierObject) {A : Set} →
  (inclusion : SubcarrierInclusion classifier A) →
  (selected : si-carrier inclusion) →
  si-characteristic inclusion (subcarrier-include classifier inclusion selected) ≡
  ClassifierObject.truthPoint classifier
subcarrier-truth-proof classifier inclusion selected =
  snd (si-to-truth-fiber inclusion selected)

-- § Every classified subcarrier displays an inclusion into its ambient carrier.
classified-subcarrier-inclusion :
  (classifier : ClassifierObject) {A : Set} →
  ClassifiedSubcarrier classifier A →
  SubcarrierInclusion classifier A
```

```

classified-subcarrier-inclusion classifier classified = record
{ si-characteristic = ClassifiedSubcarrier.characteristic classified
; si-carrier        = ClassifiedSubcarrier.selectedCarrier classified
; si-to-truth-fiber = ClassifiedSubcarrier.to-truth-fiber classified
; si-from-truth-fiber = ClassifiedSubcarrier.from-truth-fiber classified
; si-to-from-truth-fiber = ClassifiedSubcarrier.to-from-truth-fiber classified
; si-from-to-truth-fiber = ClassifiedSubcarrier.from-to-truth-fiber classified
}

-- § The truth fiber has the canonical displayed inclusion.
truth-fiber-subcarrier-inclusion :
(classifier : ClassifierObject) {A : Set} →
(marker : ClassifierMarker classifier A) →
SubcarrierInclusion classifier A
truth-fiber-subcarrier-inclusion classifier marker = record
{ si-characteristic = marker
; si-carrier        = TruthFiber classifier marker
; si-to-truth-fiber = λ fiber → fiber
; si-from-truth-fiber = λ fiber → fiber
; si-to-from-truth-fiber = λ fiber → refl
; si-from-to-truth-fiber = λ selected → refl
}

-- § Mono-like data means reflection into a chosen visible equality.
record MonoLikeSubcarrier (classifier : ClassifierObject) (A : Set) : Set1 where
field
ml-inclusion      : SubcarrierInclusion classifier A
ml-visible-eq    : si-carrier ml-inclusion → si-carrier ml-inclusion → Set
ml-visible-refl  : (selected : si-carrier ml-inclusion) →
                    ml-visible-eq selected selected
ml-visible-sym   : {left right : si-carrier ml-inclusion} →
                    ml-visible-eq left right → ml-visible-eq right left
ml-visible-trans : {left middle right : si-carrier ml-inclusion} →
                    ml-visible-eq left middle →
                    ml-visible-eq middle right →
                    ml-visible-eq left right
ml-include-reflects : {left right : si-carrier ml-inclusion} →
                    subcarrier-include classifier ml-inclusion left ≡
                    subcarrier-include classifier ml-inclusion right →
                    ml-visible-eq left right

open MonoLikeSubcarrier public

-- § Any displayed inclusion is mono-like for equality visible in the ambient carrier.
mono-like-from-inclusion :
(classifier : ClassifierObject) {A : Set} →
SubcarrierInclusion classifier A →
MonoLikeSubcarrier classifier A
mono-like-from-inclusion classifier inclusion = record
{ ml-inclusion      = inclusion
; ml-visible-eq    = λ left right →
                    subcarrier-include classifier inclusion left ≡
                    subcarrier-include classifier inclusion right
; ml-visible-refl  = λ selected → refl
}

```

```

; ml-visible-sym = λ eq → sym eq
; ml-visible-trans = λ eq1 eq2 → trans eq1 eq2
; ml-include-reflects = λ eq → eq
}

-- § A classified subcarrier therefore has canonical mono-like data.
classified-subcarrier-mono-like :
(classifier : ClassifierObject) {A : Set} →
ClassifiedSubcarrier classifier A →
MonoLikeSubcarrier classifier A
classified-subcarrier-mono-like classifier classified =
mono-like-from-inclusion classifier
(classified-subcarrier-inclusion classifier classified)

-- § Truth fibers are the canonical mono-like subcarriers selected by markers.
truth-fiber-mono-like :
(classifier : ClassifierObject) {A : Set} →
(marker : ClassifierMarker classifier A) →
MonoLikeSubcarrier classifier A
truth-fiber-mono-like classifier marker =
mono-like-from-inclusion classifier
(truth-fiber-subcarrier-inclusion classifier marker)

-- § The selected fiber of a stable reading is mono-like in this visible sense.
sri-mono-like-subcarrier :
(classifier : ClassifierObject) →
(reading : StableReadingInterface classifier) →
MonoLikeSubcarrier classifier (StableReadingInterface.sri-carrier reading)
sri-mono-like-subcarrier classifier reading =
truth-fiber-mono-like classifier (StableReadingInterface.sri-marker reading)

-- § The pulled-back fiber of a structure map is mono-like over the source.
srm-pullback-mono-like :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
(morphism : StructureMap classifier source target) →
MonoLikeSubcarrier classifier (StableReadingInterface.sri-carrier source)
srm-pullback-mono-like classifier morphism =
truth-fiber-mono-like classifier (srm-pullback-marker classifier morphism)

-- § Reindexing compares the source fiber with the pulled-back target fiber.
record ReindexedMonoLikeComparison
(classifier : ClassifierObject)
{source target : StableReadingInterface classifier}
(morphism : StructureMap classifier source target) : Set1 where
field
rmc-source-mono : MonoLikeSubcarrier classifier (StableReadingInterface.sri-carrier source)
rmc-pullback-mono : MonoLikeSubcarrier classifier (StableReadingInterface.sri-carrier source)
rmc-source-to-pullback : si-carrier (ml-inclusion rmc-source-mono) →
si-carrier (ml-inclusion rmc-pullback-mono)
rmc-pullback-to-source : si-carrier (ml-inclusion rmc-pullback-mono) →
si-carrier (ml-inclusion rmc-source-mono)
rmc-source-include-stable :
(selected : si-carrier (ml-inclusion rmc-source-mono)) →

```

```

subcarrier-include classifier (ml-inclusion rmc-pullback-mono)
  (rmc-source-to-pullback selected) ≡
subcarrier-include classifier (ml-inclusion rmc-source-mono) selected
rmc-pullback-include-stable :
  (selected : si-carrier (ml-inclusion rmc-pullback-mono)) →
subcarrier-include classifier (ml-inclusion rmc-source-mono)
  (rmc-pullback-to-source selected) ≡
subcarrier-include classifier (ml-inclusion rmc-pullback-mono) selected
rmc-source-roundtrip-visible :
  (selected : si-carrier (ml-inclusion rmc-source-mono)) →
ml-visible-eq rmc-source-mono
  (rmc-pullback-to-source (rmc-source-to-pullback selected))
  selected
rmc-pullback-roundtrip-visible :
  (selected : si-carrier (ml-inclusion rmc-pullback-mono)) →
ml-visible-eq rmc-pullback-mono
  (rmc-source-to-pullback (rmc-pullback-to-source selected))
  selected

open ReindexedMonoLikeComparison public

-- § Every structure map supplies the canonical mono-like reindexing comparison.
stable-reindexed-mono-like-comparison :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (morphism : StructureMap classifier source target) →
  ReindexedMonoLikeComparison classifier morphism
stable-reindexed-mono-like-comparison classifier {source = source} morphism = record
{ rmc-source-mono          = sri-mono-like-subcarrier classifier source
; rmc-pullback-mono        = srm-pullback-mono-like classifier morphism
; rmc-source-to-pullback   = srm-source-fiber-to-pullback classifier morphism
; rmc-pullback-to-source   = srm-pullback-fiber-to-source classifier morphism
; rmc-source-include-stable = λ selected → refl
; rmc-pullback-include-stable = λ selected → refl
; rmc-source-roundtrip-visible = λ selected → refl
; rmc-pullback-roundtrip-visible = λ selected → refl
}

```

Visible Pullback Comparison

The pullback shape can now be tested against arbitrary comparison data. Suppose a carrier maps to the source reading and also displays a point of the target truth fiber. If those two displays agree after applying the structure map, then it already determines a point of the canonical pulled-back truth fiber. This is the comparison map that an ordinary pullback universal property would demand.

The comparison is stated at the visible projection level. The target fiber carries proof data, and no proof irrelevance is assumed. Therefore the comparison below commutes with the target fiber after projecting to the displayed target point, and its uniqueness is uniqueness over the visible source projection. Every compatible square

factors through the canonical pulled-back fiber, and any other such factorization is visibly the same at the ambient carrier.

```

-- § A cone into the target truth fiber over a structure map.
record PullbackCone
  (classifier : ClassifierObject)
  {source target : StableReadingInterface classifier}
  (morphism : StructureMap classifier source target) : Set1 where
  field
    pc-carrier : Set
    pc-source : pc-carrier → StableReadingInterface.sri-carrier source
    pc-target-fiber : pc-carrier → TruthFiber classifier (StableReadingInterface.sri-marker target)
    pc-square : (point : pc-carrier) →
      srm-carrier-map morphism (pc-source point) ≡
      fst (pc-target-fiber point)

open PullbackCone public

-- § The canonical pullback shape is itself a cone.
stable-pullback-shape-cone :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (morphism : StructureMap classifier source target) →
  PullbackCone classifier morphism
stable-pullback-shape-cone classifier morphism = record
{ pc-carrier = pb-carrier (stable-reading-pullback-shape classifier morphism)
; pc-source = pb-source (stable-reading-pullback-shape classifier morphism)
; pc-target-fiber = pb-target-fiber (stable-reading-pullback-shape classifier morphism)
; pc-square = pb-square (stable-reading-pullback-shape classifier morphism)
}

-- § A compatible cone factors through the canonical pulled-back fiber.
pullback-cone-compare :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (morphism : StructureMap classifier source target) →
  (cone : PullbackCone classifier morphism) →
  pc-carrier cone →
  TruthFiber classifier (srm-pullback-marker classifier morphism)
pullback-cone-compare classifier {target = target} morphism cone point =
  pc-source cone point ,
  trans
  (cong (StableReadingInterface.sri-marker target) (pc-square cone point))
  (snd (pc-target-fiber cone point))

-- § The canonical comparison commutes with the source projection.
pullback-cone-compare-source :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (morphism : StructureMap classifier source target) →
  (cone : PullbackCone classifier morphism) →
  (point : pc-carrier cone) →
  fst (pullback-cone-compare classifier morphism cone point) ≡ pc-source cone point
pullback-cone-compare-source classifier morphism cone point = refl

```



```

-- § The canonical comparison commutes visibly with the target-fiber point.
pullback-cone-compare-target :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (morphism : StructureMap classifier source target) →
  (cone : PullbackCone classifier morphism) →
  (point : pc-carrier cone) →
  fst (srm-pullback-fiber-to-target classifier morphism
    (pullback-cone-compare classifier morphism cone point)) ≡
  fst (pc-target-fiber cone point)
pullback-cone-compare-target classifier morphism cone point =
  pc-square cone point

-- § A comparison into the canonical pullback fiber.
record PullbackComparison
  (classifier : ClassifierObject)
  {source target : StableReadingInterface classifier}
  (morphism : StructureMap classifier source target)
  (cone : PullbackCone classifier morphism) : Set1 where
  field
    pbc-map          : pc-carrier cone →
                      TruthFiber classifier (srm-pullback-marker classifier morphism)
    pbc-source-commutes : (point : pc-carrier cone) →
                          fst (pbc-map point) ≡ pc-source cone point
    pbc-target-commutes-visible : (point : pc-carrier cone) →
                                  fst (srm-pullback-fiber-to-target classifier morphism
                                    (pbc-map point)) ≡
                                  fst (pc-target-fiber cone point)

open PullbackComparison public

-- § The canonical comparison supplied by the pullback-shaped fiber.
stable-pullback-comparison :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (morphism : StructureMap classifier source target) →
  (cone : PullbackCone classifier morphism) →
  PullbackComparison classifier morphism cone
stable-pullback-comparison classifier morphism cone = record
{ pbc-map          = pullback-cone-compare classifier morphism cone
; pbc-source-commutes = pullback-cone-compare-source classifier morphism cone
; pbc-target-commutes-visible = pullback-cone-compare-target classifier morphism cone
}

-- § Visible equality of pullback comparisons, over the source projection.
record PullbackComparisonEq
  (classifier : ClassifierObject)
  {source target : StableReadingInterface classifier}
  (morphism : StructureMap classifier source target)
  (cone : PullbackCone classifier morphism)
  (left right : PullbackComparison classifier morphism cone) : Set where
  field
    pbc-visible-map-eq : (point : pc-carrier cone) →

```

```

fst (pbc-map left point) ≡ fst (pbc-map right point)

open PullbackComparisonEq public

-- § Any comparison is visibly equal to the canonical one over the source.
stable-pullback-comparison-unique :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (morphism : StructureMap classifier source target) →
  (cone : PullbackCone classifier morphism) →
  (comparison : PullbackComparison classifier morphism cone) →
  PullbackComparisonEq classifier morphism cone comparison
  (stable-pullback-comparison classifier morphism cone)
stable-pullback-comparison-unique classifier morphism cone comparison = record
{ pbc-visible-map-eq = pbc-source-commutes comparison
}

-- § The visible universal comparison carried by a single structure map.
record VisiblePullbackComparison
  (classifier : ClassifierObject)
  {source target : StableReadingInterface classifier}
  (morphism : StructureMap classifier source target) : Set1 where
  field
    vpc-shape          : PullbackShape classifier morphism
    vpc-cone-comparison : (cone : PullbackCone classifier morphism) →
                          PullbackComparison classifier morphism cone
    vpc-comparison-unique : (cone : PullbackCone classifier morphism) →
                            (comparison : PullbackComparison classifier morphism cone) →
                            PullbackComparisonEq classifier morphism cone comparison
                            (vpc-cone-comparison cone)

open VisiblePullbackComparison public

-- § Every structure map carries the visible pullback comparison data.
stable-visible-pullback-comparison :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (morphism : StructureMap classifier source target) →
  VisiblePullbackComparison classifier morphism
stable-visible-pullback-comparison classifier morphism = record
{ vpc-shape          = stable-reading-pullback-shape classifier morphism
; vpc-cone-comparison = stable-pullback-comparison classifier morphism
; vpc-comparison-unique = stable-pullback-comparison-unique classifier morphism
}

```

Univalence Boundary

Univalence marks the boundary between isomorphism and identity. In a univalent setting, equivalent structures can be read as identical along a path in the universe. The formal development here does not assume that principle. It is checked under `—safe`

and `--without-K`; no axiom is postulated, and equality of records is not enlarged by an external identification principle.

The available direction is identity-to-isomorphism. An actual identity of stable readings gives an isomorphism of stable readings: transport along `refl` yields identity structure maps in both directions. The converse direction, from isomorphism back to identity, is the univalent bridge. It is displayed below as a record type naming the additional principle, not as an inhabitant used by the construction.

The orientation data already show why the distinction theorem lands at isomorphism before identity. A distinction contains displayed left and right boundary points. The two-element carrier can therefore be presented with those boundaries exchanged. That swapped presentation is still a distinction and is isomorphic to `Two-distinction`; but on the fixed carrier `Two`, its displayed left boundary differs from the canonical displayed left boundary. A literal record identity would therefore have to identify presentation data that the theorem itself only relates by boundary-preserving isomorphism. Any later identity reading must quotient or identify presentations beyond the intensional record equality used here.

```
-- § The swapped presentation of the two-element distinction.
Two-distinction-swapped : Distinction
Two-distinction-swapped = record
{ S    = Two
; ℓ    = R
; r    = L
; ℓ≠r  = λ R≡L → Two-L≠R (sym R≡L)
; cover = λ { L → inj₂ refl ; R → inj₁ refl }
}

-- § On the fixed carrier Two, the displayed left boundary is different.
two-distinction-swapped-left-different :
ℓ Two-distinction-swapped ≠ ℓ Two-distinction
two-distinction-swapped-left-different R≡L = Two-L≠R (sym R≡L)

-- § Isomorphism of stable readings, stated through structure maps.
record StableReadingIso
(classifier : ClassifierObject)
(source target : StableReadingInterface classifier) : Set1 where
field
sri-forward : StructureMap classifier source target
sri-backward : StructureMap classifier target source
sri-to-from : StableReadingMorphismEq classifier
(stable-reading-morphism-compose classifier sri-forward sri-backward)
(stable-reading-id-morphism classifier target)
sri-from-to : StableReadingMorphismEq classifier
(stable-reading-morphism-compose classifier sri-backward sri-forward)
(stable-reading-id-morphism classifier source)

open StableReadingIso public

-- § Every stable reading is isomorphic to itself.
```

```

stable-reading-iso-refl :
  (classifier : ClassifierObject) →
  (reading : StableReadingInterface classifier) →
  StableReadingIso classifier reading reading
stable-reading-iso-refl classifier reading = record
{ sri-forward = stable-reading-id-morphism classifier reading
; sri-backward = stable-reading-id-morphism classifier reading
; sri-to-from = stable-reading-id-left classifier
               (stable-reading-id-morphism classifier reading)
; sri-from-to = stable-reading-id-left classifier
               (stable-reading-id-morphism classifier reading)
}

-- § Stable-reading isomorphism is symmetric.
stable-reading-iso-sym :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  StableReadingIso classifier source target →
  StableReadingIso classifier target source
stable-reading-iso-sym classifier iso = record
{ sri-forward = sri-backward iso
; sri-backward = sri-forward iso
; sri-to-from = sri-from-to iso
; sri-from-to = sri-to-from iso
}

-- § Stable-reading isomorphism is transitive.
stable-reading-iso-trans :
  (classifier : ClassifierObject) →
  {first middle last : StableReadingInterface classifier} →
  StableReadingIso classifier first middle →
  StableReadingIso classifier middle last →
  StableReadingIso classifier first last
stable-reading-iso-trans classifier left right = record
{ sri-forward = stable-reading-morphism-compose classifier
               (sri-forward right) (sri-forward left)
; sri-backward = stable-reading-morphism-compose classifier
               (sri-backward left) (sri-backward right)
; sri-to-from = record
  { srm-carrier-eq = λ x →
    trans
      (cong (srm-carrier-map (sri-forward right))
        (srm-carrier-eq (sri-to-from left)
          (srm-carrier-map (sri-backward right) x)))
      (srm-carrier-eq (sri-to-from right) x)
; srm-left-eq = λ y →
    trans
      (cong (srm-left-map (sri-forward right))
        (srm-left-eq (sri-to-from left)
          (srm-left-map (sri-backward right) y)))
      (srm-left-eq (sri-to-from right) y)
; srm-right-eq = λ z →
    trans
      (cong (srm-right-map (sri-forward right))

```

```

      (srm-right-eq (sri-to-from left)
        (srm-right-map (sri-backward right) z)))
      (srm-right-eq (sri-to-from right) z)
    }
; sri-from-to = record
{ srm-carrier-eq = λ x →
  trans
  (cong (srm-carrier-map (sri-backward left))
    (srm-carrier-eq (sri-from-to right)
      (srm-carrier-map (sri-forward left) x)))
  (srm-carrier-eq (sri-from-to left) x)
; srm-left-eq = λ y →
  trans
  (cong (srm-left-map (sri-backward left))
    (srm-left-eq (sri-from-to right)
      (srm-left-map (sri-forward left) y)))
  (srm-left-eq (sri-from-to left) y)
; srm-right-eq = λ z →
  trans
  (cong (srm-right-map (sri-backward left))
    (srm-right-eq (sri-from-to right)
      (srm-right-map (sri-forward left) z)))
  (srm-right-eq (sri-from-to left) z)
}
}

-- § Stable-reading isomorphism is the presentation equality.
StableReadingPresentationEq :
(classifier : ClassifierObject) →
StableReadingInterface classifier →
StableReadingInterface classifier →
Set1
StableReadingPresentationEq classifier = StableReadingIso classifier

-- § The category-side relation is an equivalence relation on objects.
record StableReadingPresentationEquivalence
(classifier : ClassifierObject) : Set2 where
field
srpe-same : StableReadingInterface classifier →
  StableReadingInterface classifier → Set1
srpe-refl : (reading : StableReadingInterface classifier) →
  srpe-same reading reading
srpe-sym : {source target : StableReadingInterface classifier} →
  srpe-same source target → srpe-same target source
srpe-trans : {first middle last : StableReadingInterface classifier} →
  srpe-same first middle →
  srpe-same middle last →
  srpe-same first last

open StableReadingPresentationEquivalence public

-- § Stable-reading isomorphism supplies the presentation equivalence relation.
stable-reading-presentation-equivalence :
(classifier : ClassifierObject) →

```

```

StableReadingPresentationEquivalence classifier
stable-reading-presentation-equivalence classifier = record
{ srpe-same = StableReadingPresentationEq classifier
; srpe-refl  = stable-reading-iso-refl classifier
; srpe-sym   = stable-reading-iso-sym classifier
; srpe-trans = stable-reading-iso-trans classifier
}

-- § Identity of stable readings gives isomorphism of stable readings.
stable-reading-identity-to-iso :
(classifier : ClassifierObject) →
{source target : StableReadingInterface classifier} →
source ≡ target →
StableReadingIso classifier source target
stable-reading-identity-to-iso classifier {source = source} refl =
stable-reading-iso-refl classifier source

-- § The univalent bridge would turn stable-reading isomorphism into identity.
record StableReadingUnivalencePrinciple
(classifier : ClassifierObject) : Set1 where
field
stable-reading-iso-to-identity :
{source target : StableReadingInterface classifier} →
StableReadingIso classifier source target →
source ≡ target
stable-reading-identity-roundtrip :
{source target : StableReadingInterface classifier} →
(path : source ≡ target) →
stable-reading-iso-to-identity
(stable-reading-identity-to-iso classifier path) ≡ path

open StableReadingUnivalencePrinciple public

-- § The formal layer records only the safe direction.
record UnivalenceBoundary
(classifier : ClassifierObject) : Set2 where
field
ub-identity-to-iso :
{source target : StableReadingInterface classifier} →
source ≡ target →
StableReadingIso classifier source target
ub-univalent-extension : Set1

open UnivalenceBoundary public

-- § The boundary object names the missing principle without assuming it.
stable-reading-univalence-boundary :
(classifier : ClassifierObject) →
UnivalenceBoundary classifier
stable-reading-univalence-boundary classifier = record
{ ub-identity-to-iso = stable-reading-identity-to-iso classifier
; ub-univalent-extension = StableReadingUnivalencePrinciple classifier
}

```

Distinction Isomorphism

To compare distinctions, two related notions are useful. A `DistinctionIso` is a boundary-preserving isomorphism: it consists of inverse maps between the carriers that also send the distinguished boundary points to their counterparts. A `DistinctionEquiv` forgets this boundary data and remembers only the underlying equivalence.

Both notions are needed later. The stricter one is appropriate for normal-form statements, while the weaker one is the natural language for orientation and automorphism arguments.

```
-- § Boundary-preserving isomorphism between distinctions.
record DistinctionIso (d1 d2 : Distinction) : Set1 where
  field
    to   : S d1 → S d2
    from : S d2 → S d1
    to-from : (y : S d2) → to (from y) ≡ y
    from-to : (x : S d1) → from (to x) ≡ x
    to-ℓ   : to (ℓ d1) ≡ ℓ d2
    to-r   : to (r d1) ≡ r d2

open DistinctionIso public

-- § Equivalence without boundary constraints.
record DistinctionEquiv (d1 d2 : Distinction) : Set1 where
  field
    to   : S d1 → S d2
    from : S d2 → S d1
    to-from : (y : S d2) → to (from y) ≡ y
    from-to : (x : S d1) → from (to x) ≡ x

open DistinctionEquiv public

-- § Forgetting boundary data from an isomorphism.
forgetIso : {d1 d2 : Distinction} → DistinctionIso d1 d2 → DistinctionEquiv d1 d2
forgetIso i = record
  { to   = DistinctionIso.to i
  ; from = DistinctionIso.from i
  ; to-from = DistinctionIso.to-from i
  ; from-to = DistinctionIso.from-to i
  }

-- § Boundary-preserving isomorphism is reflexive.
distinction-iso-refl : (d : Distinction) → DistinctionIso d d
distinction-iso-refl d = record
  { to   = id
  ; from = id
  ; to-from = λ y → refl
  ; from-to = λ x → refl
  ; to-ℓ   = refl
  ; to-r   = refl
  }
```

```

}

-- § Boundary-preserving isomorphism is symmetric.
distinction-iso-sym : {d1 d2 : Distinction} →
  DistinctionIso d1 d2 → DistinctionIso d2 d1
distinction-iso-sym {d1} {d2} iso = record
{ to   = from iso
; from = to iso
; to-from = from-to iso
; from-to = to-from iso
; to-ℓ = trans (cong (from iso) (sym (to-ℓ iso))) (from-to iso (ℓ d1))
; to-r = trans (cong (from iso) (sym (to-r iso))) (from-to iso (r d1))
}

-- § Boundary-preserving isomorphism is transitive.
distinction-iso-trans : {d1 d2 d3 : Distinction} →
  DistinctionIso d1 d2 →
  DistinctionIso d2 d3 →
  DistinctionIso d1 d3
distinction-iso-trans left right = record
{ to   = λ x → to right (to left x)
; from = λ z → from left (from right z)
; to-from = λ z →
  trans (cong (to right) (to-from left (from right z))) (to-from right z)
; from-to = λ x →
  trans (cong (from left) (from-to right (to left x))) (from-to left x)
; to-ℓ = trans (cong (to right) (to-ℓ left)) (to-ℓ right)
; to-r = trans (cong (to right) (to-r left)) (to-r right)
}

-- § Boundary-preserving isomorphism is the presentation equality for distinctions.
DistinctionPresentationEq : Distinction → Distinction → Set1
DistinctionPresentationEq = DistinctionIso

-- § The raw presentation relation is an equivalence relation.
record DistinctionPresentationEquivalence : Set2 where
  field
    dpe-same : Distinction → Distinction → Set1
    dpe-refl  : (d : Distinction) → dpe-same d d
    dpe-sym   : {d1 d2 : Distinction} → dpe-same d1 d2 → dpe-same d2 d1
    dpe-trans : {d1 d2 d3 : Distinction} →
      dpe-same d1 d2 → dpe-same d2 d3 → dpe-same d1 d3

open DistinctionPresentationEquivalence public

-- § Distinction isomorphism supplies the presentation equivalence relation.
distinction-presentation-equivalence : DistinctionPresentationEquivalence
distinction-presentation-equivalence = record
{ dpe-same = DistinctionPresentationEq
; dpe-refl = distinction-iso-refl
; dpe-sym  = distinction-iso-sym
; dpe-trans = distinction-iso-trans
}

```



```

-- § Boundary-preserving isomorphisms preserve the displayed marker.
distinction-iso-marker-commutes : {d1 d2 : Distinction} →
  (iso : DistinctionIso d1 d2) →
  (x : S d1) →
  distinction-marker d2 (DistinctionIso.to iso x) ≡ distinction-marker d1 x
distinction-iso-marker-commutes {d1} {d2} iso x with cover d1 x
... | inj1 x ≡ ℓ =
  trans
    (distinction-marker-left d2 (DistinctionIso.to iso x))
    (trans (cong (DistinctionIso.to iso) x ≡ ℓ) (DistinctionIso.to-ℓ iso)))
  refl
... | inj2 x ≡ r =
  trans
    (distinction-marker-right d2 (DistinctionIso.to iso x))
    (trans (cong (DistinctionIso.to iso) x ≡ r) (DistinctionIso.to-r iso)))
  refl

-- § A distinction isomorphism induces a structure map of stable readings.
distinction-iso-stable-reading-morphism : {d1 d2 : Distinction} →
  DistinctionIso d1 d2 →
  StructureMap TwoClassifier (distinction-stable-reading d1) (distinction-stable-reading d2)
distinction-iso-stable-reading-morphism iso = record
{ srm-carrier-map = DistinctionIso.to iso
; srm-left-map = id
; srm-right-map = id
; srm-carrier-noncollapse =
  λ x y same →
    trans
      (sym (DistinctionIso.from-to iso x))
      (trans (cong (DistinctionIso.from iso) same)
        (DistinctionIso.from-to iso y))
; srm-left-commutes = distinction-iso-marker-commutes iso
; srm-right-commutes = distinction-iso-marker-commutes iso
; srm-marker-commutes = distinction-iso-marker-commutes iso
}

-- § A distinction isomorphism induces an isomorphism of stable readings.
distinction-iso-stable-reading-iso : {d1 d2 : Distinction} →
  DistinctionIso d1 d2 →
  StableReadingIso TwoClassifier (distinction-stable-reading d1) (distinction-stable-reading d2)
distinction-iso-stable-reading-iso iso = record
{ sri-forward = distinction-iso-stable-reading-morphism iso
; sri-backward = distinction-iso-stable-reading-morphism (distinction-iso-sym iso)
; sri-to-from = record
  { srm-carrier-eq = DistinctionIso.to-from iso
  ; srm-left-eq = λ y → refl
  ; srm-right-eq = λ z → refl
  }
; sri-from-to = record
  { srm-carrier-eq = DistinctionIso.from-to iso
  ; srm-left-eq = λ y → refl
  ; srm-right-eq = λ z → refl
  }
}

```

Canonical Two Maps

For any distinction there are structurally determined maps to and from `Two`. The map `toTwo` sends an element to `L` or `R` according to the cover, while `fromTwo` sends the two constructors back to the distinguished boundary points.

The auxiliary lemmas `toTwo-ℓ` and `toTwo-r` record the behaviour of the forward map on the boundary. The round-trip identities then show that these maps form a boundary-preserving isomorphism.

```
-- § The canonical boundary-preserving map to Two.
toTwo : (d : Distinction) → S d → Two
toTwo d x with cover d x
... | inj1 _ = L
... | inj2 _ = R

-- § The canonical embedding from Two.
fromTwo : (d : Distinction) → Two → S d
fromTwo d L = ℓ d
fromTwo d R = r d

-- § toTwo sends ℓ to L.
toTwo-ℓ : (d : Distinction) → toTwo d (ℓ d) ≡ L
toTwo-ℓ d with cover d (ℓ d)
... | inj1 _ = refl
... | inj2 ℓ ≡ r = ⊥-elim ((ℓ ≠ r d) ℓ ≡ r)

-- § toTwo sends r to R.
toTwo-r : (d : Distinction) → toTwo d (r d) ≡ R
toTwo-r d with cover d (r d)
... | inj2 _ = refl
... | inj1 r ≡ ℓ = ⊥-elim ((ℓ ≠ r d) (sym r ≡ ℓ))

-- § fromTwo ∘ toTwo is the identity.
fromTwo-toTwo : (d : Distinction) → (x : S d) → fromTwo d (toTwo d x) ≡ x
fromTwo-toTwo d =
  Distinction-elim d
  (trans (cong (fromTwo d) (toTwo-ℓ d)) refl)
  (trans (cong (fromTwo d) (toTwo-r d)) refl)

-- § toTwo ∘ fromTwo is the identity.
toTwo-fromTwo : (d : Distinction) → (t : Two) → toTwo d (fromTwo d t) ≡ t
toTwo-fromTwo d L = toTwo-ℓ d
toTwo-fromTwo d R = toTwo-r d
```

Law 1.8: Every Distinction Is Isomorphic To Two

This is the basic normal-form theorem for distinctions. Here the absolute statement is proved: every distinction is isomorphic to `Two`.

```
-- § Law 1.8: Every distinction is isomorphic to Two.
two-normal-form : (d : Distinction) → DistinctionIso d Two-distinction
```

```

two-normal-form d = record
{ to   = toTwo d
; from = fromTwo d
; to-from = toTwo-fromTwo d
; from-to = fromTwo-toTwo d
; to-ℓ = toTwo-ℓ d
; to-r = toTwo-r d
}

```

```

law1-8-two-normal-form : (d : Distinction) → DistinctionIso d Two-distinction
law1-8-two-normal-form = two-normal-form

```

Presentation Quotient Shape

The normal-form theorem supplies the data needed before a quotient can be taken. The raw objects are distinctions. The admissible presentation relation is boundary-preserving isomorphism, and that relation is reflexive, symmetric, and transitive. Every raw distinction also has a displayed route, not by record identity but by presentation equivalence, to the canonical representative `Two-distinction`.

This is not a quotient type. It is the quotient shape: the object class, the admissible equivalence relation, the canonical representative, and the checked normalization map into that representative. A quotient, skeleton, or univalent completion can use this data to turn presentation equivalence into equality of representatives.

```

-- § A quotient shape gives the data needed before taking any quotient.

```

```

record DistinctionPresentationQuotientShape : Set2 where
field
  dpqs-raw      : Set1
  dpqs-same      : dpqs-raw → dpqs-raw → Set1
  dpqs-same-refl : (d : dpqs-raw) → dpqs-same d d
  dpqs-same-sym : {d1 d2 : dpqs-raw} →
    dpqs-same d1 d2 → dpqs-same d2 d1
  dpqs-same-trans : {d1 d2 d3 : dpqs-raw} →
    dpqs-same d1 d2 →
    dpqs-same d2 d3 →
    dpqs-same d1 d3
  dpqs-canonical : dpqs-raw
  dpqs-normalize : (d : dpqs-raw) → dpqs-same d dpqs-canonical

```

```

open DistinctionPresentationQuotientShape public

```

```

-- § Distinctions have a quotient shape with Two as canonical representative.

```

```

distinction-presentation-quotient-shape : DistinctionPresentationQuotientShape
distinction-presentation-quotient-shape = record
{ dpqs-raw      = Distinction
; dpqs-same      = DistinctionPresentationEq
; dpqs-same-refl = distinction-iso-refl
; dpqs-same-sym = distinction-iso-sym
; dpqs-same-trans = distinction-iso-trans
}

```

```

; dpqs-canonical = Two-distinction
; dpqs-normalize = two-normal-form
}

-- § The swapped Two presentation normalizes to the same canonical representative.
two-distinction-swapped-normalizes :
  DistinctionPresentationEq Two-distinction-swapped Two-distinction
two-distinction-swapped-normalizes = two-normal-form Two-distinction-swapped

```

Canonical Skeleton Representative

The quotient shape can be sharpened into a skeleton layer without constructing a quotient type. The skeleton has one displayed object: the canonical two-element distinction. Normalization sends every raw distinction to that skeleton object, and the soundness proof is exactly the normal-form isomorphism already established.

This skeleton is not record identity for raw presentations. It is a choice of canonical representative together with a checked route from each raw presentation to that representative through presentation equivalence.

```

-- § The skeleton of distinctions has one displayed object.
data DistinctionSkeletonObject : Set where
  distinction-skeleton-two : DistinctionSkeletonObject

-- § The skeleton object is represented by the canonical Two distinction.
distinction-skeleton-representative : DistinctionSkeletonObject → Distinction
distinction-skeleton-representative distinction-skeleton-two = Two-distinction

-- § Every distinction normalizes to the displayed skeleton object.
distinction-skeleton-normalize : Distinction → DistinctionSkeletonObject
distinction-skeleton-normalize d = distinction-skeleton-two

-- § Normalization is sound in presentation equivalence.
distinction-skeleton-normalization :
  (d : Distinction) →
    DistinctionPresentationEq d
    (distinction-skeleton-representative (distinction-skeleton-normalize d))
distinction-skeleton-normalization d = two-normal-form d

-- § The skeleton object is unique by constructor equality.
distinction-skeleton-unique :
  (skeleton : DistinctionSkeletonObject) → skeleton ≡ distinction-skeleton-two
distinction-skeleton-unique distinction-skeleton-two = refl

-- § The skeleton layer packages representative and normalization data.
record DistinctionSkeletonLayer : Set2 where
  field
    dsl-object          : Set
    dsl-representative : dsl-object → Distinction
    dsl-normalize       : Distinction → dsl-object
    dsl-normalization-sound : (d : Distinction) →

```

```

      DistinctionPresentationEq d
      (dsl-representative (dsl-normalize d))
dsl-object-unique : (object : dsl-object) →
  object ≡ dsl-normalize Two-distinction

open DistinctionSkeletonLayer public

-- § The canonical skeleton layer for distinctions.
distinction-skeleton-layer : DistinctionSkeletonLayer
distinction-skeleton-layer = record
{ dsl-object      = DistinctionSkeletonObject
; dsl-representative = distinction-skeleton-representative
; dsl-normalize    = distinction-skeleton-normalize
; dsl-normalization-sound = distinction-skeleton-normalization
; dsl-object-unique = distinction-skeleton-unique
}

```

Stable Reading Skeleton

The skeleton also lifts to the stable readings that arise from distinctions. This is not a statement about arbitrary stable readings; it concerns the readings produced by the distinction marker. A boundary-preserving isomorphism of distinctions induces an isomorphism of their displayed stable readings, so the normal-form isomorphism carries each distinction-induced reading to the canonical reading of Two-distinction.

```

-- § Distinction-induced stable readings normalize to the canonical reading.
distinction-stable-reading-normalizes :
  (d : Distinction) →
  StableReadingPresentationEq TwoClassifier
  (distinction-stable-reading d)
  (distinction-stable-reading Two-distinction)
distinction-stable-reading-normalizes d =
  distinction-iso-stable-reading-iso (two-normal-form d)

-- § The skeleton of distinction-induced stable readings.
record DistinctionReadingSkeletonLayer : Set2 where
  field
    drsl-object      : Set
    drsl-representative : drsl-object → StableReadingInterface TwoClassifier
    drsl-reading      : Distinction → StableReadingInterface TwoClassifier
    drsl-normalize     : Distinction → drsl-object
    drsl-normalization-sound : (d : Distinction) →
      StableReadingPresentationEq TwoClassifier
      (drsl-reading d)
      (drsl-representative (drsl-normalize d))
    drsl-object-unique : (object : drsl-object) →
      object ≡ drsl-normalize Two-distinction

open DistinctionReadingSkeletonLayer public

```

```

-- § Canonical skeleton layer for distinction-induced stable readings.
distinction-reading-skeleton-layer : DistinctionReadingSkeletonLayer
distinction-reading-skeleton-layer = record
{ drsl-object      = DistinctionSkeletonObject
; drsl-representative =  $\lambda$  skeleton  $\rightarrow$ 
                        distinction-stable-reading
                        (distinction-skeleton-representative skeleton)
; drsl-reading      = distinction-stable-reading
; drsl-normalize     = distinction-skeleton-normalize
; drsl-normalization-sound = distinction-stable-reading-normalizes
; drsl-object-unique = distinction-skeleton-unique
}

```

Quotient Boundary

The skeleton layer identifies a canonical representative, but it still does not replace presentation equivalence by record identity. The additional step would be an identity principle saying that boundary-preserving presentation equivalence may be read as equality of raw distinction records. The present construction names that principle without assuming it.

```

-- § The identity principle that a quotient or univalent completion would add.
DistinctionPresentationIdentityPrinciple : Set1
DistinctionPresentationIdentityPrinciple =
( $d_1$   $d_2$  : Distinction)  $\rightarrow$  DistinctionPresentationEq  $d_1$   $d_2$   $\rightarrow$   $d_1 \equiv d_2$ 

-- § Boundary data available before any quotient identity is assumed.
record DistinctionQuotientBoundary : Set $\omega$  where
field
  dqb-quotient-shape : DistinctionPresentationQuotientShape
  dqb-skeleton-layer  : DistinctionSkeletonLayer
  dqb-reading-layer   : DistinctionReadingSkeletonLayer
  dqb-identity-extension : Set1

open DistinctionQuotientBoundary public

-- § The distinction quotient boundary names the missing identity extension.
distinction-quotient-boundary : DistinctionQuotientBoundary
distinction-quotient-boundary = record
{ dqb-quotient-shape = distinction-presentation-quotient-shape
; dqb-skeleton-layer  = distinction-skeleton-layer
; dqb-reading-layer   = distinction-reading-skeleton-layer
; dqb-identity-extension = DistinctionPresentationIdentityPrinciple
}

```

Subobject Classifier Test

The classifier interface already classifies its own truth fibers. Every marker determines a selected carrier, that selected carrier has a visible inclusion into the ambient carrier,

and the characteristic map is the marker with which the fiber was formed. This is the classifier-shaped part of the construction.

The full subobject-classifier strength is stronger. It would require a universal comparison saying that every admissible subobject is classified uniquely by such a characteristic map, in the chosen equality of subobjects. The record below separates the truth-fiber data from that stronger universal extension.

```
-- § A named form of the universal extension beyond truth-fiber classification.
SubobjectClassifierUniversalExtension : ClassifierObject → Set1
SubobjectClassifierUniversalExtension classifier =
  {A : Set} → MonoLikeSubcarrier classifier A → ClassifierMarker classifier A

-- § The classifier-shaped data supplied by truth fibers.
record SubobjectClassifierCandidate (classifier : ClassifierObject) : Set2 where
  field
    soc-truth-fiber-classified : {A : Set} →
      (marker : ClassifierMarker classifier A) →
        ClassifiedSubcarrier classifier A
    soc-truth-fiber-mono-like : {A : Set} →
      (marker : ClassifierMarker classifier A) →
        MonoLikeSubcarrier classifier A
    soc-characteristic-visible : {A : Set} →
      (marker : ClassifierMarker classifier A) →
        ClassifiedSubcarrier.characteristic
      (soc-truth-fiber-classified marker) ≡ marker
    soc-universal-extension : Set1

open SubobjectClassifierCandidate public

-- § Every classifier object supplies its truth-fiber classifier candidate.
classifier-subobject-candidate :
  (classifier : ClassifierObject) → SubobjectClassifierCandidate classifier
classifier-subobject-candidate classifier = record
  { soc-truth-fiber-classified = truth-fiber-classified classifier
  ; soc-truth-fiber-mono-like = truth-fiber-mono-like classifier
  ; soc-characteristic-visible = λ marker → refl
  ; soc-universal-extension = SubobjectClassifierUniversalExtension classifier
  }
```

Topos Boundary Map

The categorical surface contains several checked components: category data for stable readings, local reindexing, visible pullback comparison, classifier-shaped truth fibers, and skeleton data for distinction-induced presentations. These components are enough to show where a topos-oriented continuation would attach. They are not enough to assert that the stable-reading category is a topos.

The missing strength is also visible: global pullbacks for arbitrary spans, a universal subobject classifier in the selected equality of subobjects, exponentials, and an

identity or quotient principle turning presentation equivalence into equality of representatives.

```
-- § A global pullback principle would supply pullbacks for arbitrary spans.
StableReadingGlobalPullbacks : Set2
StableReadingGlobalPullbacks =
  (classifier : ClassifierObject) →
  (left right apex : StableReadingInterface classifier) →
  StructureMap classifier left apex →
  StructureMap classifier right apex →
  Set1

-- § A universal subobject classifier principle is stronger than truth fibers.
StableReadingSubobjectClassifierUniversality : Set1
StableReadingSubobjectClassifierUniversality =
  (classifier : ClassifierObject) → SubobjectClassifierUniversalExtension classifier

-- § Exponentials would add function objects to the stable-reading category.
StableReadingExponentials : Set2
StableReadingExponentials =
  (classifier : ClassifierObject) →
  StableReadingInterface classifier →
  StableReadingInterface classifier →
  Set1

-- § The boundary map separates established data from missing strength.
record ToposBoundaryMap : Setω where
  field
    tb-category-data      : (classifier : ClassifierObject) → CategoryData
    tb-visible-pullback   : (classifier : ClassifierObject) →
      {source target : StableReadingInterface classifier} →
      (morphism : StructureMap classifier source target) →
      VisiblePullbackComparison classifier morphism
    tb-subobject-candidate : (classifier : ClassifierObject) →
      SubobjectClassifierCandidate classifier
    tb-quotient-boundary  : DistinctionQuotientBoundary
    tb-required-global-pullbacks : Set2
    tb-required-subobject-universal : Set1
    tb-required-exponentials   : Set2
    tb-required-identity-extension : Set1

open ToposBoundaryMap public

-- § The established data form a boundary map, not a topos theorem.
topos-boundary-map : ToposBoundaryMap
topos-boundary-map = record
{ tb-category-data      = stable-reading-category-data
; tb-visible-pullback   = stable-visible-pullback-comparison
; tb-subobject-candidate = classifier-subobject-candidate
; tb-quotient-boundary  = distinction-quotient-boundary
; tb-required-global-pullbacks = StableReadingGlobalPullbacks
; tb-required-subobject-universal = StableReadingSubobjectClassifierUniversality
; tb-required-exponentials   = StableReadingExponentials
; tb-required-identity-extension = DistinctionPresentationIdentityPrinciple
}
```


Categorical Closure Overview

The route through this chapter can be summarized without upgrading any claim beyond its proof. The distinction data yield stable readings; stable readings carry category data; structure maps reindex markers and truth fibers; mono-like inclusions make selected subcarriers visible; compatible cones factor through the pulled-back truth fiber at the visible projection level; distinction presentations normalize to the canonical skeleton representative; and the quotient and topos boundaries name the additional principles that would be needed for stronger identity and topos claims.

```
-- § Checked closure data collected at the categorical boundary.
record CategoricalClosureOverview : Setω where
  field
    cco-skeleton-layer      : DistinctionSkeletonLayer
    cco-reading-skeleton    : DistinctionReadingSkeletonLayer
    cco-quotient-boundary   : DistinctionQuotientBoundary
    cco-topos-boundary      : ToposBoundaryMap

open CategoricalClosureOverview public

-- § The categorical closure overview.
categorical-closure-overview : CategoricalClosureOverview
categorical-closure-overview = record
{ cco-skeleton-layer      = distinction-skeleton-layer
; cco-reading-skeleton    = distinction-reading-skeleton-layer
; cco-quotient-boundary   = distinction-quotient-boundary
; cco-topos-boundary      = topos-boundary-map
}
```

Orientation

An equivalence between a distinction and `Two` must send the two boundary witnesses to distinct images, because otherwise the inverse map would identify ℓ and r . Since `Two` has exactly two elements, the classification closes absolutely here: either the order of the boundary points is preserved or it is swapped.

The following definitions record these two orientations explicitly.

```
-- § The swap map on Two.
swapTwo : Two → Two
swapTwo L = R
swapTwo R = L

-- § Swap is involutive.
swapTwo-involutive : (t : Two) → swapTwo (swapTwo t) ≡ t
swapTwo-involutive L = refl
swapTwo-involutive R = refl
```

```

-- § The swap-oriented map to Two.
toTwo-swap : (d : Distinction) → S d → Two
toTwo-swap d x = swapTwo (toTwo d x)

-- § Swap-oriented map sends ℓ to R.
toTwo-swap-ℓ : (d : Distinction) → toTwo-swap d (ℓ d) ≡ R
toTwo-swap-ℓ d = cong swapTwo (toTwo-ℓ d)

-- § Swap-oriented map sends r to L.
toTwo-swap-r : (d : Distinction) → toTwo-swap d (r d) ≡ L
toTwo-swap-r d = cong swapTwo (toTwo-r d)

-- § The canonical equivalence (boundary-preserving).
two-normal-form-equiv : (d : Distinction) → DistinctionEquiv d Two-distinction
two-normal-form-equiv d = forgetIso (two-normal-form d)

-- § The swap equivalence.
two-normal-form-equiv-swap : (d : Distinction) → DistinctionEquiv d Two-distinction
two-normal-form-equiv-swap d = record
{ to   = toTwo-swap d
; from = fromTwo d ∘ swapTwo
; to-from = λ t → trans (cong (toTwo-swap d) (refl)) (trans (cong swapTwo (toTwo-fromTwo d (swapTwo t))) (swapTwo-involutive t))
; from-to = λ x → trans (cong (fromTwo d) (swapTwo-involutive (toTwo d x))) (fromTwo-toTwo d x)
}
where
  _o_ : {A B C : Set} → (B → C) → (A → B) → A → C
  (f ∘ g) x = f (g x)

-- § The two orientations of a Two normal form.
data TwoOrientation : Set where
  orient-preserve : TwoOrientation
  orient-swap : TwoOrientation

-- § The two orientations are distinct.
preserve≠swap : orient-preserve ≡ orient-swap → ⊥
preserve≠swap ()

swap≠preserve : orient-swap ≡ orient-preserve → ⊥
swap≠preserve ()

-- § Type-level closure: TwoOrientation has exactly two inhabitants.
TwoOrientation-exhaustive :
  ∀ (o : TwoOrientation) → (o ≡ orient-preserve) ⊔ (o ≡ orient-swap)
TwoOrientation-exhaustive orient-preserve = inj₁ refl
TwoOrientation-exhaustive orient-swap = inj₂ refl

```

The inverse law therefore forces the boundary images of any equivalence to remain distinct. The classifier below inspects the image of ℓ and thereby decides between the two resulting orientations; the exhaustiveness proof below closes the classification.

```

-- § Equivalence images at boundary are distinct.
to-distinct-on-boundary :

```

```

(d : Distinction) →
(e : DistinctionEquiv d Two-distinction) →
to e (ℓ d) ≡ to e (r d) → ⊥
to-distinct-on-boundary d e eq =
ℓ ≠ r d (trans (sym (from-to e (ℓ d))) (trans (cong (from e) eq) (from-to e (r d))))

-- § Classify an equivalence by its orientation.
orientation-classify :
(d : Distinction) →
(e : DistinctionEquiv d Two-distinction) →
TwoOrientation
orientation-classify d e with Two-cover (to e (ℓ d)) | Two-cover (to e (r d))
... | inj₁ tℓ≡L | inj₁ tr≡L = ⊥-elim (to-distinct-on-boundary d e (trans tℓ≡L (sym tr≡L)))
... | inj₂ tℓ≡R | inj₂ tr≡R = ⊥-elim (to-distinct-on-boundary d e (trans tℓ≡R (sym tr≡R)))
... | inj₁ _ | inj₂ _ = orient-preserve
... | inj₂ _ | inj₁ _ = orient-swap

```

Automorphisms

An automorphism of a distinction is an equivalence from the carrier to itself. Since the carrier has only two boundary classes, the admissible boundary behaviour closes to two possibilities: preserve the boundary points or exchange them. The proofs below make this precise by combining injectivity with the cover.

```

-- § Automorphism of a distinction.
Aut : Distinction → Set1
Aut d = DistinctionEquiv d d

-- § Automorphism transport is injective.
to-injective : (d : Distinction) → (a : Aut d) → {x y : S d} → to a x ≡ to a y → x ≡ y
to-injective d a {x} {y} eq =
trans (sym (from-to a x)) (trans (cong (from a) eq) (from-to a y))

-- § Automorphism images at boundary points are distinct.
to-ℓ≠to-r : (d : Distinction) → (a : Aut d) → to a (ℓ d) ≠ to a (r d)
to-ℓ≠to-r d a eq = ℓ ≠ r d (to-injective d a eq)

-- § Dual boundary lemma: dual sends ℓ to r.
Distinction-dual-ℓ : (d : Distinction) → Distinction-dual d (ℓ d) ≡ r d
Distinction-dual-ℓ d with cover d (ℓ d)
... | inj₁ _ = refl
... | inj₂ ℓ≡r = ⊥-elim ((ℓ ≠ r d) ℓ≡r)

-- § Dual boundary lemma: dual sends r to ℓ.
Distinction-dual-r : (d : Distinction) → Distinction-dual d (r d) ≡ ℓ d
Distinction-dual-r d with cover d (r d)
... | inj₂ _ = refl
... | inj₁ r≡ℓ = ⊥-elim ((ℓ ≠ r d) (sym r≡ℓ))

-- § Automorphism classification: exactly id or dual.
Aut-sound :

```

```

(d : Distinction) →
(a : Aut d) →
(to a ≐ id) ⊔ (to a ≐ Distinction-dual d)
Aut-sound d a with cover d (to a (ℓ d)) | cover d (to a (r d))
... | inj1 tℓ≡ℓ | inj1 tr≡ℓ = ⊥-elim (to-ℓ≠to-r d a (trans tℓ≡ℓ (sym tr≡ℓ)))
... | inj2 tℓ≡r | inj2 tr≡r = ⊥-elim (to-ℓ≠to-r d a (trans tℓ≡r (sym tr≡r)))
... | inj1 tℓ≡ℓ | inj2 tr≡r =
inj1 (Distinction-elim d tℓ≡ℓ tr≡r)
... | inj2 tℓ≡r | inj1 tr≡ℓ =
inj2 (Distinction-elim d
(trans tℓ≡r (sym (Distinction-dual-ℓ d)))
(trans tr≡ℓ (sym (Distinction-dual-r d))))

```

This completes the automorphism analysis for distinctions. Figure 12.1 records the induced symmetry action on the four endomorphism witnesses.

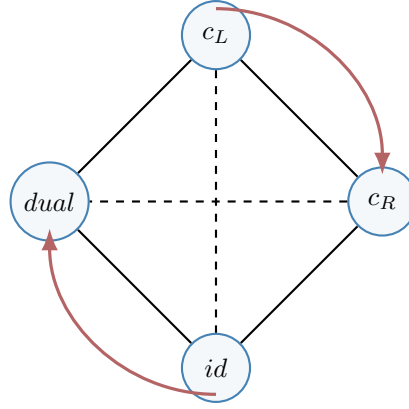


Figure 12.1: Automorphism action on the four-case witness space: preserve and swap survive as the only orbit structure.

Law 1.9: The Two Isomorphism Is Unique

Any boundary-preserving isomorphism from a distinction to Two agrees with the normal-form map. This is another application of the eliminator: once the values at ℓ and r are fixed, the map is fixed everywhere. The same reasoning determines the inverse map.

— § Uniqueness of the to-direction.

```

toTwo-unique :
(d : Distinction) →
(f : S d → Two) →
f (ℓ d) ≡ L →
f (r d) ≡ R →
f ≐ toTwo d
toTwo-unique d f fℓ fr =
Distinction-elim d

```

```

(trans fl (sym (toTwo-ℓ d)))
(trans fr (sym (toTwo-r d)))

-- § Uniqueness of the swap-direction.
toTwo-swap-unique :
  (d : Distinction) →
  (f : S d → Two) →
  f (ℓ d) ≡ R →
  f (r d) ≡ L →
  f ≐ toTwo-swap d
toTwo-swap-unique d f fl fr =
  Distinction-elim d
  (trans fl (sym (toTwo-swap-ℓ d)))
  (trans fr (sym (toTwo-swap-r d)))

-- § Law 1.9: iso-to is unique.
law1-9-iso-to-unique : (d : Distinction) → (i : DistinctionIso d Two-distinction) → to i ≐ toTwo d
law1-9-iso-to-unique d i =
  toTwo-unique d (to i)
  (trans (to-ℓ i) refl)
  (trans (to-r i) refl)

-- § Uniqueness of the from-direction.
fromTwo-unique :
  (d : Distinction) →
  (g : Two → S d) →
  g L ≡ ℓ d →
  g R ≡ r d →
  g ≐ fromTwo d
fromTwo-unique d g gL gR L = trans gL refl
fromTwo-unique d g gL gR R = trans gR refl

-- § Law 1.9: iso-from is unique.
law1-9-iso-from-unique : (d : Distinction) → (i : DistinctionIso d Two-distinction) → from i ≐ fromTwo d
law1-9-iso-from-unique d i =
  fromTwo-unique d (from i)
  (trans (sym (cong (from i) (to-ℓ i))) (from-to i (ℓ d)))
  (trans (sym (cong (from i) (to-r i))) (from-to i (r d)))

```

Law 1.10: Two Normal Form Has Exactly Two Orientations

Every equivalence from a distinction to Two has one of the two orientations just described. This is an absolute classification because the proof eliminates the two equal-image cases by contradiction.

```

-- § Law 1.10: orientation is exhaustive.
orientation-exhaustive :
  (d : Distinction) →
  (e : DistinctionEquiv d Two-distinction) →
  (to e ≐ toTwo d) ⊔ (to e ≐ toTwo-swap d)
orientation-exhaustive d e with Two-cover (to e (ℓ d)) | Two-cover (to e (r d))
... | inj₁ tℓ≡L | inj₁ tr≡L = ⊥-elim (to-distinct-on-boundary d e (trans tℓ≡L (sym tr≡L)))

```

```

... | inj2 tℓ≡R | inj2 tr≡R = ⊥-elim (to-distinct-on-boundary d e (trans tℓ≡R (sym tr≡R)))
... | inj1 tℓ≡L | inj2 tr≡R = inj1 (toTwo-unique d (to e) tℓ≡L tr≡R)
... | inj2 tℓ≡R | inj1 tr≡L = inj2 (toTwo-swap-unique d (to e) tℓ≡R tr≡L)

```

```
law1-10-orientation-exhaustive :
```

```

(d : Distinction) →
(e : DistinctionEquiv d Two-distinction) →
(to e ≐ toTwo d) ⊔ (to e ≐ toTwo-swap d)

```

```
law1-10-orientation-exhaustive = orientation-exhaustive
```

Law 1.11: Automorphisms Are Exactly id Or dual

Automorphisms are therefore exhausted by the identity and the dual map. The word “exactly” is justified by Aut-sound: preservation and exchange are the only boundary behaviours compatible with injectivity and coverage.

```
-- § Law 1.11: Automorphisms are exactly id or dual.
```

```
law1-11-Aut-sound :
```

```

(d : Distinction) →
(a : Aut d) →
(to a ≐ id) ⊔ (to a ≐ Distinction-dual d)

```

```
law1-11-Aut-sound = Aut-sound
```

Law 1.12: Two-Orientation Classification Is Sound And Unique

The next step packages the two orientations into an explicit classifier. Each equivalence to Two receives one of two labels, case-preserve or case-swap, together with a proof that the chosen label reproduces the map. Uniqueness is immediate because the two interpretations already disagree at the left boundary point; to let a single equivalence realize both would force $L \equiv R$.

```
-- § Orientation case type.
```

```

data OrientationCase : Set where
case-preserve : OrientationCase
case-swap : OrientationCase

```

```
-- § Type-level closure: OrientationCase has exactly two inhabitants.
```

```
OrientationCase-exhaustive :
```

```

∀ (c : OrientationCase) → (c ≡ case-preserve) ⊔ (c ≡ case-swap)
OrientationCase-exhaustive case-preserve = inj1 refl
OrientationCase-exhaustive case-swap = inj2 refl

```

```
-- § Interpret an orientation case as a map.
```

```

orientationInterpret : (d : Distinction) → OrientationCase → S d → Two
orientationInterpret d case-preserve = toTwo d
orientationInterpret d case-swap = toTwo-swap d

```

```
-- § Law 1.12: orientation classification is sound.
```

```
orientationCase-sound :
```

```

(d : Distinction) →
(e : DistinctionEquiv d Two-distinction) →
Σ OrientationCase (λ c → to e ≐ orientationInterpret d c)
orientationCase-sound d e with orientation-exhaustive d e
... | inj1 p = case-preserve , p
... | inj2 p = case-swap , p

-- § Law 1.12: orientation classification is unique.
orientationCase-unique :
(d : Distinction) →
(e : DistinctionEquiv d Two-distinction) →
(c1 c2 : OrientationCase) →
to e ≐ orientationInterpret d c1 →
to e ≐ orientationInterpret d c2 →
c1 ≡ c2
orientationCase-unique d e case-preserve case-preserve _ _ = refl
orientationCase-unique d e case-swap case-swap _ _ = refl
orientationCase-unique d e case-preserve case-swap p q =
⊥-elim (Two-L≠R (sym toR≡L))
where
toR≡L : R ≡ L
toR≡L =
trans (sym (toTwo-swap-ℓ d))
(trans (sym (q (ℓ d)))
(trans (p (ℓ d)) (toTwo-ℓ d)))
orientationCase-unique d e case-swap case-preserve p q =
⊥-elim (Two-L≠R (sym toR≡L))
where
toR≡L : R ≡ L
toR≡L =
trans (sym (toTwo-swap-ℓ d))
(trans (sym (p (ℓ d)))
(trans (q (ℓ d)) (toTwo-ℓ d)))

law1-12-orientationCase-sound :
(d : Distinction) →
(e : DistinctionEquiv d Two-distinction) →
Σ OrientationCase (λ c → to e ≐ orientationInterpret d c)
law1-12-orientationCase-sound = orientationCase-sound

law1-12-orientationCase-unique :
(d : Distinction) →
(e : DistinctionEquiv d Two-distinction) →
(c1 c2 : OrientationCase) →
to e ≐ orientationInterpret d c1 →
to e ≐ orientationInterpret d c2 →
c1 ≡ c2
law1-12-orientationCase-unique = orientationCase-unique

```

Law 1.13: Automorphism Classification Is Sound And Unique

This final classification statement records the same two properties as before: under the two-class constraint every automorphism falls into one of the two surviving forms,

and that form is uniquely determined.

```

-- § Automorphism case type.
data AutCase : Set where
  case-id : AutCase
  case-dual : AutCase

-- § Type-level closure: AutCase has exactly two inhabitants.
AutCase-exhaustive :
  ∀ (c : AutCase) → (c ≡ case-id) ∨ (c ≡ case-dual)
AutCase-exhaustive case-id = inj₁ refl
AutCase-exhaustive case-dual = inj₂ refl

-- § Interpret an automorphism case.
autInterpret : (d : Distinction) → AutCase → S d → S d
autInterpret d case-id = id
autInterpret d case-dual = Distinction-dual d

-- § Automorphism classification is sound.
autCase-sound :
  (d : Distinction) →
  (a : Aut d) →
  ∑ AutCase (λ c → to a ≐ autInterpret d c)
autCase-sound d a with Aut-sound d a
... | inj₁ p = case-id , p
... | inj₂ p = case-dual , p

-- § Automorphism classification is unique.
autCase-unique :
  (d : Distinction) →
  (a : Aut d) →
  (c₁ c₂ : AutCase) →
  to a ≐ autInterpret d c₁ →
  to a ≐ autInterpret d c₂ →
  c₁ ≡ c₂
autCase-unique d a case-id case-id _ = refl
autCase-unique d a case-dual case-dual _ = refl
autCase-unique d a case-id case-dual p q =
  ⊥-elim ((ℓ ≠ r d) ℓ ≡ r)
where
  ℓ ≡ r : ℓ d ≡ r d
  ℓ ≡ r =
    trans (sym (p (ℓ d)))
      (trans (q (ℓ d)) (Distinction-dual-ℓ d))
autCase-unique d a case-dual case-id p q =
  ⊥-elim ((ℓ ≠ r d) ℓ ≡ r)
where
  ℓ ≡ r : ℓ d ≡ r d
  ℓ ≡ r =
    trans (sym (q (ℓ d)))
      (trans (p (ℓ d)) (Distinction-dual-ℓ d))

-- § Law 1.13: Automorphism classification is sound and unique.
law1-13-autCase-sound :

```



```

(d : Distinction) →
(a : Aut d) →
Σ AutCase (λ c → to a ≐ autInterpret d c)
law1-13-autCase-sound = autCase-sound

```

```

law1-13-autCase-unique :
(d : Distinction) →
(a : Aut d) →
(c1 c2 : AutCase) →
to a ≐ autInterpret d c1 →
to a ≐ autInterpret d c2 →
c1 ≡ c2
law1-13-autCase-unique = autCase-unique

```

K₄ Witness Interface

The classification results for endomorphisms are now packaged once more in witness form. For a given endomorphism, the witness consists of a case label together with a proof that the interpreted case agrees pointwise with the original map. This adds no new assumption; it exposes the earlier module-level statements as a global interface.

```

-- § Law 1.14: K4 classification produces a witness.
law1-14-k4-classification-sound :
(d : Distinction) →
(f : S d → S d) →
Σ EndoCase (λ c → K4.interpret d c ≐ f)
law1-14-k4-classification-sound = k4-classification-sound

```

```

-- § Law 1.15: K4 classification witness is unique.
law1-15-k4-classification-unique :
(d : Distinction) →
(f : S d → S d) →
(c1 c2 : EndoCase) →
K4.interpret d c1 ≐ f →
K4.interpret d c2 ≐ f →
c1 ≡ c2
law1-15-k4-classification-unique = k4-classification-unique

```

Two-Class Coverage Without Definitional Equality

Up to this point the development has used propositional equality throughout. For later applications the elimination argument must be separated from that particular equality relation and stated instead for an arbitrary equivalence relation.

The record `DistinctionClass` provides this generalization in explicit form. It consists of a carrier equipped with a reflexive, symmetric, and transitive relation, together

with two distinguished elements that remain separate under that relation and a corresponding two-class cover. The eliminator from the earlier sections then extends to this broader setting without changing its structural role.

```

-- § DistinctionClass: two-class coverage with an arbitrary equivalence relation.
record DistinctionClass : Set1 where
  field
    S      : Set
    _≈_    : S → S → Set
    ≈-refl : (x : S) → x ≈ x
    ≈-sym  : {x y : S} → x ≈ y → y ≈ x
    ≈-trans : {x y z : S} → x ≈ y → y ≈ z → x ≈ z

    ℓ      : S
    r      : S
    ℓ≠r    : ¬ (ℓ ≈ r)
    cover≈ : (x : S) → (x ≈ ℓ) ∨ (x ≈ r)

open DistinctionClass public

-- § Respect predicate for ≈-elimination.
Respect≈ : (d : DistinctionClass) → (S d → Set) → Set
Respect≈ d P = {x y : S d} → _≈_ d x y → P x → P y

-- § Law 1.16: Elimination is forced by cover≈.
DistinctionClass-elim :
  (d : DistinctionClass) →
  {P : S d → Set} →
  Respect≈ d P →
  P (ℓ d) →
  P (r d) →
  (x : S d) →
  P x
DistinctionClass-elim d {P} resp pℓ pr x with cover≈ d x
... | inj1 x≈ℓ = resp ((≈-sym d) x≈ℓ) pℓ
... | inj2 x≈r = resp ((≈-sym d) x≈r) pr

law1-16-class-elim :
  (d : DistinctionClass) →
  {P : S d → Set} →
  Respect≈ d P →
  P (ℓ d) →
  P (r d) →
  (x : S d) →
  P x
law1-16-class-elim = DistinctionClass-elim

-- § Law 1.17: Every Distinction induces a DistinctionClass.
Distinction→DistinctionClass : Distinction → DistinctionClass
Distinction→DistinctionClass d = record
  { S      = S d
  ; _≈_    = _≡_
  ; ≈-refl = λ x → refl
  ; ≈-sym  = sym

```

```

;  $\approx$ -trans = trans
;  $\ell$       =  $\ell$  d
;  $r$       =  $r$  d
;  $\ell \neq r$  =  $\ell \neq r$  d
; cover $\approx$  = cover d
}

```

```
law1-17-distinction-to-class : Distinction → DistinctionClass
```

```
law1-17-distinction-to-class = Distinction→DistinctionClass
```

Presentation Uniqueness

The canonical classifier is not the only possible way to index the four endomorphism cases. One may choose a different datatype X together with interpretation and classification maps. The next theorem shows that any such presentation is equivalent to the chosen one, provided it is sound and its interpretation map is injective up to pointwise equality.

In other words, once soundness and injectivity are fixed, the classification is unique up to reindexing. The proof constructs explicit maps between X and EndoCase and then verifies the two round-trip laws.

```

-- § Endo presentation record.
record EndoPresentation (d : Distinction) (X : Set) : Set where
  field
    present-interpret      : X → (S d → S d)
    present-classify      : (S d → S d) → X
    present-classify-sound : (f : S d → S d) → present-interpret (present-classify f)  $\doteq$  f
    present-interpret-injective : (x y : X) → present-interpret x  $\doteq$  present-interpret y → x  $\equiv$  y

-- § Law 1.18: Endo presentation is unique up to isomorphism.
law1-18-endo-presentation-unique :
  (d : Distinction) →
  {X : Set} →
  EndoPresentation d X →
  SetIso X EndoCase
law1-18-endo-presentation-unique d {X} pres =
  let
    module K = K4 d
    open EndoPresentation pres
    renaming
      ( present-interpret to interpretX
        ; present-classify to classifyX
        ; present-classify-sound to classifyX-sound
        ; present-interpret-injective to interpretX-injective
      )
  to' : X → EndoCase
  to' x = K.classify (interpretX x)

```

```

from' : EndoCase → X
from' c = classifyX (K.interpret c)

to-from' : (c : EndoCase) → to' (from' c) ≡ c
to-from' c =
  sym
  (K.classify-unique
   (interpretX (classifyX (K.interpret c))))
  c
  (K.≡-sym (classifyX-sound (K.interpret c)))

from-to' : (x : X) → from' (to' x) ≡ x
from-to' x =
  interpretX-injective
  (classifyX (K.interpret (K.classify (interpretX x))))
  x
  (K.≡-trans
   (classifyX-sound (K.interpret (K.classify (interpretX x))))
   (K.classify-sound (interpretX x)))
in
record
{ to = to'
; from = from'
; to-from = to-from'
; from-to = from-to'
}

```

Law 1.19–1.20: Orientation and Automorphism Presentation Uniqueness

The same argument applies to the orientation and automorphism classifications. Once the classifier/interpreter pair is fixed, any alternative presentation satisfying the same soundness and injectivity requirements is isomorphic to that presentation. The freedom left by relabelling is therefore harmless: it changes names, not structure.

The following helper definitions prepare these two analogues of the endomorphism presentation theorem.

```

-- § Helpers for orientation presentation elimination.
orientationCase-classify :
  (d : Distinction) →
  (e : DistinctionEquiv d Two-distinction) →
  OrientationCase
orientationCase-classify d e = fst (orientationCase-sound d e)

orientationCase-classify-sound :
  (d : Distinction) →
  (e : DistinctionEquiv d Two-distinction) →
  to e ≡ orientationInterpreter d (orientationCase-classify d e)
orientationCase-classify-sound d e = snd (orientationCase-sound d e)

```

```

autCase-classify :
  (d : Distinction) →
  (a : Aut d) →
  AutCase
autCase-classify d a = fst (autCase-sound d a)

autCase-classify-sound :
  (d : Distinction) →
  (a : Aut d) →
  to a ≐ autInterpret d (autCase-classify d a)
autCase-classify-sound d a = snd (autCase-sound d a)

orientationEquivInterpret :
  (d : Distinction) →
  OrientationCase →
  DistinctionEquiv d Two-distinction
orientationEquivInterpret d case-preserve = two-normal-form-equiv d
orientationEquivInterpret d case-swap = two-normal-form-equiv-swap d

orientationEquivInterpret-sound :
  (d : Distinction) →
  (c : OrientationCase) →
  to (orientationEquivInterpret d c) ≐ orientationInterpret d c
orientationEquivInterpret-sound d case-preserve x = refl
orientationEquivInterpret-sound d case-swap x = refl

-- § Identity equivalence.
idEquiv : (d : Distinction) → DistinctionEquiv d d
idEquiv d = record
{ to   = id
; from = id
; to-from = λ y → refl
; from-to = λ x → refl
}

-- § Dual equivalence.
dualEquiv : (d : Distinction) → DistinctionEquiv d d
dualEquiv d = record
{ to   = Distinction-dual d
; from = Distinction-dual d
; to-from = law1-3-dual-involutive d
; from-to = law1-3-dual-involutive d
}

autEquivInterpret :
  (d : Distinction) →
  AutCase →
  Aut d
autEquivInterpret d case-id = idEquiv d
autEquivInterpret d case-dual = dualEquiv d

autEquivInterpret-sound :
  (d : Distinction) →
  (c : AutCase) →

```

```

to (autEquivInterpret d c)  $\doteq$  autInterpret d c
autEquivInterpret-sound d case-id x = refl
autEquivInterpret-sound d case-dual x = refl

```

Presentation records. A presentation of either classification, orientation or automorphism is a set-valued interface carrying interpretation, classification, soundness, and injectivity. The corresponding records package exactly the data required to compare an arbitrary labeling with the fixed classifier/interpreter pair.

```

-- § Orientation presentation record.
record OrientationPresentation (d : Distinction) (X : Set) : Set1 where
  field
    op-interpret      : X → DistinctionEquiv d Two-distinction
    op-classify       : DistinctionEquiv d Two-distinction → X
    op-classify-sound : (e : DistinctionEquiv d Two-distinction) → to (op-interpret (op-classify e))  $\doteq$  to e
    op-interpret-injective : (x y : X) → to (op-interpret x)  $\doteq$  to (op-interpret y) → x  $\equiv$  y

-- § Automorphism presentation record.
record AutPresentation (d : Distinction) (X : Set) : Set1 where
  field
    ap-interpret      : X → Aut d
    ap-classify       : Aut d → X
    ap-classify-sound : (a : Aut d) → to (ap-interpret (ap-classify a))  $\doteq$  to a
    ap-interpret-injective : (x y : X) → to (ap-interpret x)  $\doteq$  to (ap-interpret y) → x  $\equiv$  y

```

Law 1.19: Orientation presentation uniqueness. Every orientation presentation satisfying the stated interface is therefore isomorphic to `OrientationCase`. The isomorphism is obtained by composing classification on one side with interpretation on the other, while soundness and injectivity force the two round-trip properties.

```

-- § Law 1.19: Orientation presentation is unique up to isomorphism.
law1-19-orientation-presentation-unique :
  (d : Distinction) →
  {X : Set} →
  OrientationPresentation d X →
  SetIso X OrientationCase
law1-19-orientation-presentation-unique d {X} pres =
  let
    open OrientationPresentation pres
    renaming
      ( op-interpret to interpretX
        ; op-classify to classifyX
        ; op-classify-sound to classifyX-sound
        ; op-interpret-injective to interpretX-injective
      )
   $\doteq$ -sym : {A B : Set} {f g : A → B} → f  $\doteq$  g → g  $\doteq$  f
   $\doteq$ -sym p x = sym (p x)

```

```

 $\stackrel{\circ}{=}\text{-trans} : \{A\ B : \text{Set}\} \{f\ g\ h : A \rightarrow B\} \rightarrow f \stackrel{\circ}{=} g \rightarrow g \stackrel{\circ}{=} h \rightarrow f \stackrel{\circ}{=} h$ 
 $\stackrel{\circ}{=}\text{-trans}\ p\ q\ x = \text{trans}\ (p\ x)\ (q\ x)$ 

to' : X → OrientationCase
to' x = orientationCase-classify d (interpretX x)

from' : OrientationCase → X
from' c = classifyX (orientationEquivInterpret d c)

to-from' : (c : OrientationCase) → to' (from' c) ≡ c
to-from' c =
  orientationCase-unique d (interpretX (classifyX (orientationEquivInterpret d c)))
  (to' (from' c))
  c
  (orientationCase-classify-sound d (interpretX (classifyX (orientationEquivInterpret d c))))
  ( $\stackrel{\circ}{=}\text{-trans}$ 
    (classifyX-sound (orientationEquivInterpret d c))
    (orientationEquivInterpret-sound d c))

from-to' : (x : X) → from' (to' x) ≡ x
from-to' x =
  interpretX-injective
  (classifyX (orientationEquivInterpret d (to' x)))
  x
  ( $\stackrel{\circ}{=}\text{-trans}$ 
    (classifyX-sound (orientationEquivInterpret d (to' x)))
    ( $\stackrel{\circ}{=}\text{-trans}$ 
      (orientationEquivInterpret-sound d (to' x))
      ( $\stackrel{\circ}{=}\text{-sym}$  (orientationCase-classify-sound d (interpretX x)))))

in
record
{ to = to'
; from = from'
; to-from = to-from'
; from-to = from-to'
}

-- § Law 1.20: Automorphism presentation is unique up to isomorphism.

```

Law 1.20: Automorphism presentation uniqueness. The same argument applies to automorphisms. Any sound and injective presentation of automorphisms is isomorphic to the two-element case type `AutCase`; relabelling can change the interface, not the classified structure.

```

law1-20-aut-presentation-unique :
  (d : Distinction) →
  {X : Set} →
  AutPresentation d X →
  SetIso X AutCase
law1-20-aut-presentation-unique d {X} pres =
  let
  open AutPresentation pres

```

```

renaming
( ap-interpret to interpretX
; ap-classify to classifyX
; ap-classify-sound to classifyX-sound
; ap-interpret-injective to interpretX-injective
)

 $\stackrel{\circ}{=}\text{-sym} : \{A\ B : \text{Set}\} \{f\ g : A \rightarrow B\} \rightarrow f \stackrel{\circ}{=} g \rightarrow g \stackrel{\circ}{=} f$ 
 $\stackrel{\circ}{=}\text{-sym } p\ x = \text{sym } (p\ x)$ 

 $\stackrel{\circ}{=}\text{-trans} : \{A\ B : \text{Set}\} \{f\ g\ h : A \rightarrow B\} \rightarrow f \stackrel{\circ}{=} g \rightarrow g \stackrel{\circ}{=} h \rightarrow f \stackrel{\circ}{=} h$ 
 $\stackrel{\circ}{=}\text{-trans } p\ q\ x = \text{trans } (p\ x)\ (q\ x)$ 

to' : X → AutCase
to' x = autCase-classify d (interpretX x)

from' : AutCase → X
from' c = classifyX (autEquivInterpret d c)

to-from' : (c : AutCase) → to' (from' c) ≡ c
to-from' c =
  autCase-unique d (interpretX (classifyX (autEquivInterpret d c)))
  (to' (from' c))
  c
  (autCase-classify-sound d (interpretX (classifyX (autEquivInterpret d c))))
  ( $\stackrel{\circ}{=}\text{-trans}$ 
    (classifyX-sound (autEquivInterpret d c))
    (autEquivInterpret-sound d c))

from-to' : (x : X) → from' (to' x) ≡ x
from-to' x =
  interpretX-injective
  (classifyX (autEquivInterpret d (to' x)))
  x
  ( $\stackrel{\circ}{=}\text{-trans}$ 
    (classifyX-sound (autEquivInterpret d (to' x)))
    ( $\stackrel{\circ}{=}\text{-trans}$ 
      (autEquivInterpret-sound d (to' x))
      ( $\stackrel{\circ}{=}\text{-sym}$  (autCase-classify-sound d (interpretX x)))))

in
record
{ to = to'
; from = from'
; to-from = to-from'
; from-to = from-to'
}

```

The concrete Two reference and its named canonical carrier are fixed last.

```

-- § CanonicalTwo is the canonical binary carrier used as a named normal form.
CanonicalTwo : Set
CanonicalTwo = Two

-- § The left boundary.

```



```
left : CanonicalTwo
left = L

-- § The right boundary.
right : CanonicalTwo
right = R

-- § The canonical carrier as a Distinction record.
canonical-two-distinction : Distinction
canonical-two-distinction = Two-distinction
```

At this point the primitive distinction has been fixed concretely as `Two-distinction`. The basic logical infrastructure, the distinction record, the four endomorphism cases, and the associated presentation results are now all available in explicit, machine-checked form.

Chapter 13

The Topos Route Boundary

The preceding chapter already reached the categorical boundary: stable readings have category data, truth fibers supply classifier-shaped subcarriers, and distinction-induced readings normalize to the canonical skeleton. What is still missing for the later Topos route is not a new start, but a sharper interface over the reading layer already present in the book.

The stable reading interface deliberately names only a truth point. It does not assume a separate false point, nor does it assume that a reading has distinguished left and right boundary positions. The Topos route needs exactly those additional marks in order to state the raw obstruction honestly. The record below therefore adds them as route data. It does not replace `StableReadingInterface`; it refines it for the binary route that begins with `Distinction`.

Boundary-Marked Classifiers

A boundary-marked classifier is a classifier object together with a displayed false point separated from the distinguished truth point. The binary classifier supplies the first instance: L is truth and R is the separated false point.

```
-- § A route classifier adds a separated false point to the book classifier.
record BoundaryMarkedClassifier (classifier : ClassifierObject) : Set1 where
  field
    falsePoint : ClassifierObject.truthCarrier classifier
    truth-false-separated : ClassifierObject.truthPoint classifier ≠ falsePoint

open BoundaryMarkedClassifier public

-- § The binary classifier has R as its separated false point.
two-boundary-marked-classifier : BoundaryMarkedClassifier TwoClassifier
two-boundary-marked-classifier = record
{ falsePoint = R
```

```
; truth-false-separated = Two-L≠R
}
```

Boundary-Marked Readings

A boundary-marked reading is a stable reading together with two displayed boundary positions. The left boundary is marked by truth; the right boundary is marked by the displayed false point. This is the exact extra information the raw Topos obstruction will later consume.

```
-- § A stable reading equipped with route-specific left/right boundary marks.
```

```
record BoundaryMarkedReading
  (classifier : ClassifierObject)
  (boundary : BoundaryMarkedClassifier classifier) : Set1 where
  field
    bmr-reading      : StableReadingInterface classifier
    bmr-left-boundary : StableReadingInterface.sri-carrier bmr-reading
    bmr-right-boundary : StableReadingInterface.sri-carrier bmr-reading
    bmr-left-truth    :
      StableReadingInterface.sri-marker bmr-reading bmr-left-boundary ≡
      ClassifierObject.truthPoint classifier
    bmr-right-false   :
      StableReadingInterface.sri-marker bmr-reading bmr-right-boundary ≡
      falsePoint boundary
```

```
open BoundaryMarkedReading public
```

```
-- § The two displayed boundary positions cannot collapse.
```

```
boundary-marked-reading-separated :
  (classifier : ClassifierObject) →
  (boundary : BoundaryMarkedClassifier classifier) →
  (reading : BoundaryMarkedReading classifier boundary) →
  bmr-left-boundary reading ≠ bmr-right-boundary reading
boundary-marked-reading-separated classifier boundary reading left≡right =
  truth-false-separated boundary
  (trans
    (sym (bmr-left-truth reading))
    (trans
      (cong (StableReadingInterface.sri-marker (bmr-reading reading)) left≡right)
      (bmr-right-false reading))))
```

The Distinction Route Entry

Every distinction already supplies the required marked route. Its existing stable reading is the one defined earlier from the distinction marker; the left boundary is ℓ ; the right boundary is r ; and the marker laws are exactly the left and right boundary lemmas of the binary fiber reading.

```

-- § Any distinction induces the boundary-marked binary reading needed by the Topos route.
distinction-boundary-marked-reading :
  (d : Distinction) →
    BoundaryMarkedReading TwoClassifier two-boundary-marked-classifier
distinction-boundary-marked-reading d = record
{ bmr-reading      = distinction-stable-reading d
; bmr-left-boundary = ℓ d
; bmr-right-boundary = r d
; bmr-left-truth    = distinction-marker-left d (ℓ d) refl
; bmr-right-false   = distinction-marker-right d (r d) refl
}

-- § The route-specific boundary separation agrees with the distinction separation.
distinction-boundary-marked-separated :
  (d : Distinction) →
    bmr-left-boundary (distinction-boundary-marked-reading d) ≠
    bmr-right-boundary (distinction-boundary-marked-reading d)
distinction-boundary-marked-separated d = ℓ ≠ r d

-- § Package: the existing stable reading and its route refinement from a distinction.
record ToposRouteEntry (d : Distinction) : Set1 where
  field
    tre-stable-reading : StableReadingInterface TwoClassifier
    tre-boundary-reading : BoundaryMarkedReading TwoClassifier two-boundary-marked-classifier
    tre-reading-is-book-reading : tre-stable-reading ≡ distinction-stable-reading d
    tre-boundary-separated :
      bmr-left-boundary tre-boundary-reading ≠
      bmr-right-boundary tre-boundary-reading

open ToposRouteEntry public

-- § The Topos route starts from the book's own distinction-induced reading.
topos-route-entry : (d : Distinction) → ToposRouteEntry d
topos-route-entry d = record
{ tre-stable-reading      = distinction-stable-reading d
; tre-boundary-reading    = distinction-boundary-marked-reading d
; tre-reading-is-book-reading = refl
; tre-boundary-separated = distinction-boundary-marked-separated d
}

```

Raw Pullback Obstruction

The raw reading category already has morphisms and visible categorical data. The Topos route now asks for the first genuinely global demand: pullbacks for every cospan. The following cospan is enough to block that demand. It splits the truth value into two distinct truth-copies. Both copies are still marked as truth, but they remain different points of the apex. Any pullback cone over the cospan would have to identify them.

The construction uses the boundary-marked interface above, because the contradiction is read at a displayed truth boundary of the cone object.

```

-- § A stable reading of the binary classifier itself.
topos-route-two-classifier-reading : StableReadingInterface TwoClassifier
topos-route-two-classifier-reading = record
{ sri-carrier      = Two
; sri-leftCarrier  = Two
; sri-rightCarrier = Two
; sri-leftReading  = id
; sri-rightReading = id
; sri-stableReading =  $\lambda x y \text{ same} \rightarrow \text{product-fst-eq same}$ 
; sri-marker       = id
}

-- § The binary classifier reading with its two displayed boundaries.
topos-route-two-boundary-reading :
BoundaryMarkedReading TwoClassifier two-boundary-marked-classifier
topos-route-two-boundary-reading = record
{ bmr-reading      = topos-route-two-classifier-reading
; bmr-left-boundary = L
; bmr-right-boundary = R
; bmr-left-truth    = refl
; bmr-right-false   = refl
}

-- § Split apex: two truth-points and two false-points.
data ToposRouteSplitApexPoint : Set where
route-truth-left-point : ToposRouteSplitApexPoint
route-truth-right-point : ToposRouteSplitApexPoint
route-false-left-point : ToposRouteSplitApexPoint
route-false-right-point : ToposRouteSplitApexPoint

-- § The two truth-points are distinct constructors.
route-truth-left  $\neq$  route-truth-right :
route-truth-left-point  $\neq$  route-truth-right-point
route-truth-left  $\neq$  route-truth-right ()

-- § The split marker identifies both truth-copies as truth and both false-copies as false.
topos-route-split-apex-marker : ToposRouteSplitApexPoint  $\rightarrow$  Two
topos-route-split-apex-marker route-truth-left-point = L
topos-route-split-apex-marker route-truth-right-point = L
topos-route-split-apex-marker route-false-left-point = R
topos-route-split-apex-marker route-false-right-point = R

-- § The apex remains a stable reading by reading its carrier identically.
topos-route-split-truth-apex-reading : StableReadingInterface TwoClassifier
topos-route-split-truth-apex-reading = record
{ sri-carrier      = ToposRouteSplitApexPoint
; sri-leftCarrier  = ToposRouteSplitApexPoint
; sri-rightCarrier = ToposRouteSplitApexPoint
; sri-leftReading  = id
; sri-rightReading = id
; sri-stableReading =  $\lambda x y \text{ same} \rightarrow \text{product-fst-eq same}$ 
; sri-marker       = topos-route-split-apex-marker
}

```

```

-- § The split apex has a displayed truth boundary and a displayed false boundary.
topos-route-split-truth-apex :
  BoundaryMarkedReading TwoClassifier two-boundary-marked-classifier
topos-route-split-truth-apex = record
{ bmr-reading      = topos-route-split-truth-apex-reading
; bmr-left-boundary = route-truth-left-point
; bmr-right-boundary = route-false-left-point
; bmr-left-truth    = refl
; bmr-right-false   = refl
}

-- § Left leg of the cospan chooses the left truth/false copies.
topos-route-split-left-carrier-map : Two → ToposRouteSplitApexPoint
topos-route-split-left-carrier-map L = route-truth-left-point
topos-route-split-left-carrier-map R = route-false-left-point

-- § Right leg of the cospan chooses the right truth/false copies.
topos-route-split-right-carrier-map : Two → ToposRouteSplitApexPoint
topos-route-split-right-carrier-map L = route-truth-right-point
topos-route-split-right-carrier-map R = route-false-right-point

-- § The left split leg is non-collapsing.
topos-route-split-left-noncollapsing :
  NonCollapsing topos-route-split-left-carrier-map
topos-route-split-left-noncollapsing L L same = refl
topos-route-split-left-noncollapsing L R ()
topos-route-split-left-noncollapsing R L ()
topos-route-split-left-noncollapsing R R same = refl

-- § The right split leg is non-collapsing.
topos-route-split-right-noncollapsing :
  NonCollapsing topos-route-split-right-carrier-map
topos-route-split-right-noncollapsing L L same = refl
topos-route-split-right-noncollapsing L R ()
topos-route-split-right-noncollapsing R L ()
topos-route-split-right-noncollapsing R R same = refl

-- § Left cospan leg from the classifier reading into the split apex.
topos-route-split-left-map :
  StructureMap TwoClassifier
  topos-route-two-classifier-reading
  topos-route-split-truth-apex-reading
topos-route-split-left-map = record
{ srm-carrier-map      = topos-route-split-left-carrier-map
; srm-left-map         = topos-route-split-left-carrier-map
; srm-right-map        = topos-route-split-left-carrier-map
; srm-carrier-noncollapse = topos-route-split-left-noncollapsing
; srm-left-commutes    = λ x → refl
; srm-right-commutes   = λ x → refl
; srm-marker-commutes = λ { L → refl ; R → refl }
}

-- § Right cospan leg from the classifier reading into the split apex.
topos-route-split-right-map :

```

```

StructureMap TwoClassifier
  topos-route-two-classifier-reading
  topos-route-split-truth-apex-reading
topos-route-split-right-map = record
{ srm-carrier-map      = topos-route-split-right-carrier-map
; srm-left-map         = topos-route-split-right-carrier-map
; srm-right-map        = topos-route-split-right-carrier-map
; srm-carrier-noncollapse = topos-route-split-right-noncollapsing
; srm-left-commutes    =  $\lambda x \rightarrow \text{refl}$ 
; srm-right-commutes   =  $\lambda x \rightarrow \text{refl}$ 
; srm-marker-commutes =  $\lambda \{ L \rightarrow \text{refl} ; R \rightarrow \text{refl} \}$ 
}

-- § Route pullback cone for boundary-marked readings.
record ToposRoutePullbackCone
  (classifier : ClassifierObject)
  (boundary : BoundaryMarkedClassifier classifier)
  {left right apex : BoundaryMarkedReading classifier boundary}
  (f : StructureMap classifier (bmr-reading left) (bmr-reading apex))
  (g : StructureMap classifier (bmr-reading right) (bmr-reading apex)) : Set1 where
  field
    trpc-object : BoundaryMarkedReading classifier boundary
    trpc-left  : StructureMap classifier (bmr-reading trpc-object) (bmr-reading left)
    trpc-right : StructureMap classifier (bmr-reading trpc-object) (bmr-reading right)
    trpc-square : StableReadingMorphismEq classifier
      (stable-reading-morphism-compose classifier f trpc-left)
      (stable-reading-morphism-compose classifier g trpc-right)

open ToposRoutePullbackCone public

-- § If the left leg sees L, it lands at the left truth-point.
topos-route-split-left-map-at :
  (x : Two)  $\rightarrow x \equiv L \rightarrow$ 
  srm-carrier-map topos-route-split-left-map x  $\equiv$  route-truth-left-point
topos-route-split-left-map-at L refl = refl
topos-route-split-left-map-at R ()

-- § If the right leg sees L, it lands at the right truth-point.
topos-route-split-right-map-at :
  (x : Two)  $\rightarrow x \equiv L \rightarrow$ 
  srm-carrier-map topos-route-split-right-map x  $\equiv$  route-truth-right-point
topos-route-split-right-map-at L refl = refl
topos-route-split-right-map-at R ()

-- § A cone's left projection sends its displayed truth boundary to L.
topos-route-cone-left-projection-left :
  (cone : ToposRoutePullbackCone TwoClassifier two-boundary-marked-classifier
    {left = topos-route-two-boundary-reading}
    {right = topos-route-two-boundary-reading}
    {apex = topos-route-split-truth-apex}
    topos-route-split-left-map
    topos-route-split-right-map)  $\rightarrow$ 
  srm-carrier-map (trpc-left cone) (bmr-left-boundary (trpc-object cone))  $\equiv$  L
topos-route-cone-left-projection-left cone =

```



```

trans
  (srm-marker-commutes
    (trpc-left cone)
    (bmr-left-boundary (trpc-object cone)))
  (bmr-left-truth (trpc-object cone))

-- § A cone's right projection also sends its displayed truth boundary to L.
topos-route-cone-right-projection-left :
  (cone : ToposRoutePullbackCone TwoClassifier two-boundary-marked-classifier
    {left = topos-route-two-boundary-reading}
    {right = topos-route-two-boundary-reading}
    {apex = topos-route-split-truth-apex}
    topos-route-split-left-map
    topos-route-split-right-map) →
  srm-carrier-map (trpc-right cone) (bmr-left-boundary (trpc-object cone)) ≡ L
topos-route-cone-right-projection-left cone =
  trans
    (srm-marker-commutes
      (trpc-right cone)
      (bmr-left-boundary (trpc-object cone)))
    (bmr-left-truth (trpc-object cone))

-- § No route pullback cone exists for the split-truth cospan.
topos-route-split-truth-cospan-has-no-cone :
  ToposRoutePullbackCone TwoClassifier two-boundary-marked-classifier
    {left = topos-route-two-boundary-reading}
    {right = topos-route-two-boundary-reading}
    {apex = topos-route-split-truth-apex}
    topos-route-split-left-map
    topos-route-split-right-map → ⊥
topos-route-split-truth-cospan-has-no-cone cone =
  route-truth-left ≠ route-truth-right
  (trans
    (sym
      (topos-route-split-left-map-at
        (srm-carrier-map (trpc-left cone) (bmr-left-boundary (trpc-object cone)))
        (topos-route-cone-left-projection-left cone))))
    (trans
      (srm-carrier-eq (trpc-square cone) (bmr-left-boundary (trpc-object cone)))
      (topos-route-split-right-map-at
        (srm-carrier-map (trpc-right cone) (bmr-left-boundary (trpc-object cone)))
        (topos-route-cone-right-projection-left cone))))

-- § Global route pullback cones would assign a cone to every boundary-marked cospan.
ToposRouteGlobalPullbackCones : Set1
ToposRouteGlobalPullbackCones =
  (classifier : ClassifierObject) →
  (boundary : BoundaryMarkedClassifier classifier) →
  (left right apex : BoundaryMarkedReading classifier boundary) →
  (f : StructureMap classifier (bmr-reading left) (bmr-reading apex)) →
  (g : StructureMap classifier (bmr-reading right) (bmr-reading apex)) →
  ToposRoutePullbackCone classifier boundary {left} {right} {apex} f g

-- § The split-truth cospan blocks global route pullback cones.

```

```

topos-route-global-pullback-cones-impossible :
  ToposRouteGlobalPullbackCones → ⊥
topos-route-global-pullback-cones-impossible global-cones =
  topos-route-split-truth-cospan-has-no-cone
(global-cones
  TwoClassifier
  two-boundary-marked-classifier
  topos-route-two-boundary-reading
  topos-route-two-boundary-reading
  topos-route-split-truth-apex
  topos-route-split-left-map
  topos-route-split-right-map)

-- § The raw obstruction package: the route has an entry, but not global raw pullback cones.
record ToposRouteRawObstruction : Set1 where
  field
    trro-entry : (d : Distinction) → ToposRouteEntry d
    trro-split-cospan-no-cone :
      ToposRoutePullbackCone TwoClassifier two-boundary-marked-classifier
      {left = topos-route-two-boundary-reading}
      {right = topos-route-two-boundary-reading}
      {apex = topos-route-split-truth-apex}
      topos-route-split-left-map
      topos-route-split-right-map → ⊥
    trro-global-cones-impossible : ToposRouteGlobalPullbackCones → ⊥

open ToposRouteRawObstruction public

-- § Checked raw obstruction for the Topos route.
topos-route-raw-obstruction : ToposRouteRawObstruction
topos-route-raw-obstruction = record
  { trro-entry           = topos-route-entry
  ; trro-split-cospan-no-cone = topos-route-split-truth-cospan-has-no-cone
  ; trro-global-cones-impossible = topos-route-global-pullback-cones-impossible
  }

```

The obstruction is therefore not a stylistic warning. The raw route has a concrete cospan for which even a boundary-marked pullback cone is impossible. The Topos package can only appear after a completion step that changes the stage on which the categorical structure is being read.

The Completed Skeleton Topos

The completion step is the skeleton already constructed for distinction presentations. It does not assert that raw presentations have become definitionally equal. It changes the carrier: after normalization there is one completed object and one completed arrow between any two completed objects. On that completed stage the elementary-Topos data can be constructed directly.

The category interface below uses prefixed field names, because the book already has a separate `extttCategoryData` record for raw stable readings. The new record is only the small-category surface used by the completed endpoint.

```
-- § The unique arrow type of the completed skeleton category.
data SkeletonCategoryArrow : Set where
skeleton-category-arrow : SkeletonCategoryArrow

-- § Small category interface used for the completed endpoint.
record SmallCategory : Set1 where
field
  sc-Obj      : Set
  sc-Hom      : sc-Obj → sc-Obj → Set
  sc-id       : {A : sc-Obj} → sc-Hom A A
  sc-comp     : {A B C : sc-Obj} → sc-Hom B C → sc-Hom A B → sc-Hom A C
  sc-id-left  : {A B : sc-Obj} → (f : sc-Hom A B) → sc-comp sc-id f ≡ f
  sc-id-right : {A B : sc-Obj} → (f : sc-Hom A B) → sc-comp f sc-id ≡ f
  sc-assoc    : {A B C D : sc-Obj} →
    (h : sc-Hom C D) →
    (g : sc-Hom B C) →
    (f : sc-Hom A B) →
    sc-comp h (sc-comp g f) ≡ sc-comp (sc-comp h g) f

open SmallCategory public

-- § The skeleton category has one completed object and one arrow everywhere.
skeleton-category : SmallCategory
skeleton-category = record
{ sc-Obj      = DistinctionSkeletonObject
; sc-Hom      = λ A B → SkeletonCategoryArrow
; sc-id       = skeleton-category-arrow
; sc-comp     = λ g f → skeleton-category-arrow
; sc-id-left  = λ { } { skeleton-category-arrow → refl }
; sc-id-right = λ { } { skeleton-category-arrow → refl }
; sc-assoc    = λ { } { skeleton-category-arrow skeleton-category-arrow skeleton-category-arrow → refl }
}

-- § Embedding of a raw distinction into the completed category object.
distinction-to-completed-object : Distinction → sc-Obj skeleton-category
distinction-to-completed-object = distinction-skeleton-normalize

-- § Every raw distinction lands at the canonical completed object.
distinction-to-completed-object-normalizes :
(d : Distinction) →
distinction-to-completed-object d ≡ distinction-skeleton-two
distinction-to-completed-object-normalizes d = refl

-- § Terminal object in an arbitrary small category.
record TerminalObject (category : SmallCategory) : Set1 where
field
  terminal-object : sc-Obj category
  terminal-arrow  : (A : sc-Obj category) → sc-Hom category A terminal-object
  terminal-unique : {A : sc-Obj category} →
    (f : sc-Hom category A terminal-object) →
```

$$f \equiv \text{terminal-arrow } A$$

open TerminalObject public

-- § Binary product in an arbitrary small category.

record BinaryProduct

(category : SmallCategory)

(A B : sc-Obj category) : Set1 where

field

product-object : sc-Obj category

product-pr1 : sc-Hom category product-object A

product-pr2 : sc-Hom category product-object B

product-pair : {X : sc-Obj category} →

sc-Hom category X A →

sc-Hom category X B →

sc-Hom category X product-object

product-left : {X : sc-Obj category} →

(f : sc-Hom category X A) →

(g : sc-Hom category X B) →

sc-comp category product-pr1 (product-pair f g) ≡ f

product-right : {X : sc-Obj category} →

(f : sc-Hom category X A) →

(g : sc-Hom category X B) →

sc-comp category product-pr2 (product-pair f g) ≡ g

product-unique : {X : sc-Obj category} →

(f : sc-Hom category X A) →

(g : sc-Hom category X B) →

(h : sc-Hom category X product-object) →

sc-comp category product-pr1 h ≡ f →

sc-comp category product-pr2 h ≡ g →

h ≡ product-pair f g

open BinaryProduct public

-- § Binary products for all pairs of objects.

record HasBinaryProducts (category : SmallCategory) : Set1 where

field

product-of : (A B : sc-Obj category) → BinaryProduct category A B

open HasBinaryProducts public

-- § Pullback of a cospan in an arbitrary small category.

record Pullback

(category : SmallCategory)

{A B Z : sc-Obj category}

(f : sc-Hom category A Z)

(g : sc-Hom category B Z) : Set1 where

field

pullback-object : sc-Obj category

pullback-left : sc-Hom category pullback-object A

pullback-right : sc-Hom category pullback-object B

pullback-square : sc-comp category f pullback-left ≡ sc-comp category g pullback-right

pullback-lift : {X : sc-Obj category} →

(u : sc-Hom category X A) →

```

(v : sc-Hom category X B) →
sc-comp category f u ≡ sc-comp category g v →
sc-Hom category X pullback-object
pullback-lift-left : {X : sc-Obj category} →
  (u : sc-Hom category X A) →
  (v : sc-Hom category X B) →
  (p : sc-comp category f u ≡ sc-comp category g v) →
  sc-comp category pullback-left (pullback-lift u v p) ≡ u
pullback-lift-right : {X : sc-Obj category} →
  (u : sc-Hom category X A) →
  (v : sc-Hom category X B) →
  (p : sc-comp category f u ≡ sc-comp category g v) →
  sc-comp category pullback-right (pullback-lift u v p) ≡ v
pullback-lift-unique : {X : sc-Obj category} →
  (u : sc-Hom category X A) →
  (v : sc-Hom category X B) →
  (p : sc-comp category f u ≡ sc-comp category g v) →
  (h : sc-Hom category X pullback-object) →
  sc-comp category pullback-left h ≡ u →
  sc-comp category pullback-right h ≡ v →
  h ≡ pullback-lift u v p

open Pullback public

-- § Pullbacks for all cospans in a small category.
record HasPullbacks (category : SmallCategory) : Set1 where
  field
    pullback-of : {A B Z : sc-Obj category} →
      (f : sc-Hom category A Z) →
      (g : sc-Hom category B Z) →
      Pullback category f g

open HasPullbacks public

-- § The one-object skeleton has a terminal object.
skeleton-terminal : TerminalObject skeleton-category
skeleton-terminal = record
{ terminal-object = distinction-skeleton-two
; terminal-arrow = λ A → skeleton-category-arrow
; terminal-unique = λ { skeleton-category-arrow → refl }
}

-- § Any two skeleton objects have the unique binary product.
skeleton-product :
  (A B : sc-Obj skeleton-category) → BinaryProduct skeleton-category A B
skeleton-product A B = record
{ product-object = distinction-skeleton-two
; product-pr1 = skeleton-category-arrow
; product-pr2 = skeleton-category-arrow
; product-pair = λ f g → skeleton-category-arrow
; product-left = λ { skeleton-category-arrow skeleton-category-arrow → refl }
; product-right = λ { skeleton-category-arrow skeleton-category-arrow → refl }
; product-unique = λ
  { skeleton-category-arrow skeleton-category-arrow skeleton-category-arrow left-proof right-proof → refl }

```

```

}

-- § Hence the skeleton category has binary products.
skeleton-products : HasBinaryProducts skeleton-category
skeleton-products = record
{ product-of = skeleton-product
}

-- § Any skeleton cospan has the unique pullback.
skeleton-pullback :
{A B Z : sc-Obj skeleton-category} →
(f : sc-Hom skeleton-category A Z) →
(g : sc-Hom skeleton-category B Z) →
Pullback skeleton-category {A} {B} {Z} f g
skeleton-pullback {A} {B} {Z} f g = record
{ pullback-object   = distinction-skeleton-two
; pullback-left     = skeleton-category-arrow
; pullback-right    = skeleton-category-arrow
; pullback-square   = refl
; pullback-lift     = λ u v p → skeleton-category-arrow
; pullback-lift-left = λ { skeleton-category-arrow skeleton-category-arrow p → refl }
; pullback-lift-right = λ { skeleton-category-arrow skeleton-category-arrow p → refl }
; pullback-lift-unique = λ
  { skeleton-category-arrow skeleton-category-arrow p skeleton-category-arrow left-proof right-proof → refl }
}

-- § Hence the skeleton category has all pullbacks.
skeleton-pullbacks : HasPullbacks skeleton-category
skeleton-pullbacks = record
{ pullback-of = λ {A} {B} {Z} f g → skeleton-pullback {A} {B} {Z} f g
}

-- § Mono predicate for morphisms in a small category.
record IsMono
(category : SmallCategory)
{A B : sc-Obj category}
(morphism : sc-Hom category A B) : Set1 where
field
mono-cancel : {X : sc-Obj category} →
(f g : sc-Hom category X A) →
sc-comp category morphism f ≡ sc-comp category morphism g →
f ≡ g

open IsMono public

-- § Pullback square used by the categorical subobject classifier.
record PullbackSquare
(category : SmallCategory)
{S A B Z : sc-Obj category}
(left-map : sc-Hom category S A)
(right-map : sc-Hom category S B)
(top-map : sc-Hom category A Z)
(bottom-map : sc-Hom category B Z) : Set1 where
field

```

```

square-commutes :
  sc-comp category top-map left-map ≡ sc-comp category bottom-map right-map
square-lift      : {X : sc-Obj category} →
  (u : sc-Hom category X A) →
  (v : sc-Hom category X B) →
  sc-comp category top-map u ≡ sc-comp category bottom-map v →
  sc-Hom category X S
square-lift-left : {X : sc-Obj category} →
  (u : sc-Hom category X A) →
  (v : sc-Hom category X B) →
  (p : sc-comp category top-map u ≡ sc-comp category bottom-map v) →
  sc-comp category left-map (square-lift u v p) ≡ u
square-lift-right : {X : sc-Obj category} →
  (u : sc-Hom category X A) →
  (v : sc-Hom category X B) →
  (p : sc-comp category top-map u ≡ sc-comp category bottom-map v) →
  sc-comp category right-map (square-lift u v p) ≡ v
square-lift-unique : {X : sc-Obj category} →
  (u : sc-Hom category X A) →
  (v : sc-Hom category X B) →
  (p : sc-comp category top-map u ≡ sc-comp category bottom-map v) →
  (h : sc-Hom category X S) →
  sc-comp category left-map h ≡ u →
  sc-comp category right-map h ≡ v →
  h ≡ square-lift u v p

open PullbackSquare public

-- § Subobject classifier interface in a small category.
record SubobjectClassifierC
  (category : SmallCategory)
  (terminal : TerminalObject category) : Set1 where
  field
    classifier-object : sc-Obj category
    true-map          : sc-Hom category (terminal-object terminal) classifier-object
    characteristic    : {Sub Base : sc-Obj category} →
      (morphism : sc-Hom category Sub Base) →
      IsMono category morphism →
      sc-Hom category Base classifier-object
    characteristic-square : {Sub Base : sc-Obj category} →
      (morphism : sc-Hom category Sub Base) →
      (mono : IsMono category morphism) →
      PullbackSquare category
      morphism
      (terminal-arrow terminal Sub)
      (characteristic morphism mono)
      true-map

open SubobjectClassifierC public

-- § Exponential object interface in a small category.
record Exponential
  (category : SmallCategory)
  (products : HasBinaryProducts category)

```

```

(A B : sc-Obj category) : Set1 where
field
  exp-object    : sc-Obj category
  exp-eval      : sc-Hom category
                  (product-object (product-of products exp-object A))
                  B
  exp-transpose : {X : sc-Obj category} →
                  sc-Hom category (product-object (product-of products X A)) B →
                  sc-Hom category X exp-object
  exp-beta      : {X : sc-Obj category} →
                  (h : sc-Hom category (product-object (product-of products X A)) B) →
                  sc-comp category exp-eval
                  (product-pair (product-of products exp-object A)
                    (sc-comp category (exp-transpose h)
                      (product-pr1 (product-of products X A)))
                    (product-pr2 (product-of products X A))) ≡ h
  exp-eta       : {X : sc-Obj category} →
                  (h : sc-Hom category (product-object (product-of products X A)) B) →
                  (k : sc-Hom category X exp-object) →
                  sc-comp category exp-eval
                  (product-pair (product-of products exp-object A)
                    (sc-comp category k (product-pr1 (product-of products X A)))
                    (product-pr2 (product-of products X A))) ≡ h →
                  k ≡ exp-transpose h

open Exponential public

-- § Exponentials for all pairs of objects.
record HasExponentials
(category : SmallCategory)
(products : HasBinaryProducts category) : Set1 where
field
  exponential-of : (A B : sc-Obj category) → Exponential category products A B

open HasExponentials public

-- § Every skeleton morphism is mono because all hom-sets are singleton.
skeleton-mono :
{A B : sc-Obj skeleton-category} →
(morphism : sc-Hom skeleton-category A B) →
IsMono skeleton-category {A} {B} morphism
skeleton-mono {A} {B} morphism = record
{ mono-cancel = λ { skeleton-category-arrow skeleton-category-arrow p → refl }
}

-- § The one-object skeleton carries its subobject classifier directly.
skeleton-subobject-classifier :
SubobjectClassifierC skeleton-category skeleton-terminal
skeleton-subobject-classifier = record
{ classifier-object = distinction-skeleton-two
; true-map         = skeleton-category-arrow
; characteristic   = λ morphism mono → skeleton-category-arrow
; characteristic-square = λ morphism mono → record
  { square-commutes = refl

```



```

; square-lift      = λ u v p → skeleton-category-arrow
; square-lift-left = λ { skeleton-category-arrow skeleton-category-arrow p → refl }
; square-lift-right = λ { skeleton-category-arrow skeleton-category-arrow p → refl }
; square-lift-unique = λ
{ skeleton-category-arrow skeleton-category-arrow p skeleton-category-arrow left-proof right-proof → refl }
}
}

-- § The skeleton exponential is the unique object with the unique evaluator.
skeleton-exponential :
(A B : sc-Obj skeleton-category) →
Exponential skeleton-category skeleton-products A B
skeleton-exponential A B = record
{ exp-object      = distinction-skeleton-two
; exp-eval        = skeleton-category-arrow
; exp-transpose   = λ h → skeleton-category-arrow
; exp-beta        = λ { skeleton-category-arrow → refl }
; exp-eta         = λ { skeleton-category-arrow skeleton-category-arrow p → refl }
}

-- § Hence the skeleton category has exponentials.
skeleton-exponentials : HasExponentials skeleton-category skeleton-products
skeleton-exponentials = record
{ exponential-of = skeleton-exponential
}

-- § Elementary-topos package used for the completed endpoint.
record ElementaryTopos (category : SmallCategory) : Set1 where
field
et-terminal      : TerminalObject category
et-products      : HasBinaryProducts category
et-pullbacks     : HasPullbacks category
et-exponentials  : HasExponentials category et-products
et-subobject-classifier : SubobjectClassifierC category et-terminal

open ElementaryTopos public

-- § The completed skeleton category satisfies the elementary-topos data.
skeleton-elementary-topos : ElementaryTopos skeleton-category
skeleton-elementary-topos = record
{ et-terminal      = skeleton-terminal
; et-products      = skeleton-products
; et-pullbacks     = skeleton-pullbacks
; et-exponentials  = skeleton-exponentials
; et-subobject-classifier = skeleton-subobject-classifier
}

-- § Explicit completion stage: skeleton data, obstruction, category, and topos proof.
record CompletionToposStage : Setω where
field
cts-distinction-skeleton : DistinctionSkeletonLayer
cts-reading-skeleton     : DistinctionReadingSkeletonLayer
cts-raw-obstruction      : ToposRouteRawObstruction
cts-category             : SmallCategory

```

```

cts-terminal      : TerminalObject cts-category
cts-products     : HasBinaryProducts cts-category
cts-pullbacks    : HasPullbacks cts-category
cts-exponentials : HasExponentials cts-category cts-products
cts-subobject-classifier : SubobjectClassifierC cts-category cts-terminal
cts-topos        : ElementaryTopos cts-category

```

```
open CompletionToposStage public
```

```

-- § The quotient/skeleton completion is the Topos endpoint.
completion-topos-stage : CompletionToposStage
completion-topos-stage = record
{ cts-distinction-skeleton = distinction-skeleton-layer
; cts-reading-skeleton = distinction-reading-skeleton-layer
; cts-raw-obstruction = topos-route-raw-obstruction
; cts-category = skeleton-category
; cts-terminal = skeleton-terminal
; cts-products = skeleton-products
; cts-pullbacks = skeleton-pullbacks
; cts-exponentials = skeleton-exponentials
; cts-subobject-classifier = skeleton-subobject-classifier
; cts-topos = skeleton-elementary-topos
}

```

This is the exact sense in which the Topos appears. It is not the starting arena of the book's route. It is the completed skeleton stage: the raw obstruction is recorded, the quotient/skeleton data are recorded, and the elementary-Topos structure is then constructed over the completed one-object category.

Scope And Set-Like Burden

The skeleton endpoint is a Topos stage, but it is not a set-like universe of formulation. It has one completed object. Any richer set-like stage must therefore add nonterminal structure and carry the distinctional burden introduced by its own object diversity. The first records in this section name that burden. The following free-completion block then constructs the canonical syntactic inhabitant of it.

```

-- § The completed skeleton package used by later classification records.
record VoidSkeletonToposCompletion : Setω where
field
  vst-topos-stage      : CompletionToposStage
  vst-distinction-skeleton : DistinctionSkeletonLayer
  vst-reading-skeleton  : DistinctionReadingSkeletonLayer
  vst-category          : SmallCategory
  vst-topos             : ElementaryTopos vst-category

open VoidSkeletonToposCompletion public

-- § The completed skeleton carries the elementary-Topos structure.
void-skeleton-topos-completion : VoidSkeletonToposCompletion

```

```

void-skeleton-topos-completion = record
{ vst-topos-stage      = completion-topos-stage
; vst-distinction-skeleton = distinction-skeleton-layer
; vst-reading-skeleton = distinction-reading-skeleton-layer
; vst-category         = skeleton-category
; vst-topos            = skeleton-elementary-topos
}

-- § Semantic adequacy is named as an external problem, not proved here.
StableReadingSemanticAdequacy : Set2
StableReadingSemanticAdequacy =
(classifier : ClassifierObject) →
StableReadingInterface classifier →
Set1

-- § A generic richer-completion problem is named, not taken as primitive.
record RicherCompletionCandidate : Set2 where
field
  rcc-object-carrier : Set
  rcc-reading-embed : StableReadingInterface TwoClassifier → rcc-object-carrier
  rcc-category       : SmallCategory
  rcc-topos          : ElementaryTopos rcc-category

open RicherCompletionCandidate public

-- § Scope conditions state exactly what the theorem does and does not close.
record ToposRouteScopeConditions : Setω where
field
  vtsc-semantic-adequacy-problem : Set2
  vtsc-minimal-completion-stage : CompletionToposStage
  vtsc-richer-completion-problem : Set2

open ToposRouteScopeConditions public

-- § The scope record names the burden carried by richer completions.
void-topos-scope-conditions : ToposRouteScopeConditions
void-topos-scope-conditions = record
{ vtsc-semantic-adequacy-problem = StableReadingSemanticAdequacy
; vtsc-minimal-completion-stage = completion-topos-stage
; vtsc-richer-completion-problem = RicherCompletionCandidate
}

-- § Object diversity: the category has two distinct objects.
record HasDistinctObjects (category : SmallCategory) : Set1 where
field
  first-object   : sc-Obj category
  second-object  : sc-Obj category
  objects-separated : first-object ≠ second-object

open HasDistinctObjects public

-- § The skeleton completion has no pair of distinct objects.
skeleton-has-no-distinct-objects :
HasDistinctObjects skeleton-category → ⊥

```

```

skeleton-has-no-distinct-objects record
{ first-object = distinction-skeleton-two
; second-object = distinction-skeleton-two
; objects-separated = separated
} = separated refl

-- § The minimal completion stage is therefore not set-like by object diversity.
minimal-completion-not-set-like :
HasDistinctObjects (cts-category completion-topos-stage) → ⊥
minimal-completion-not-set-like = skeleton-has-no-distinct-objects

-- § Point-separation principle for morphisms.
record PointsSeparateMorphisms
(category : SmallCategory)
(terminal : TerminalObject category) : Set1 where
field
points-separate :
{A B : sc-Obj category} →
(f g : sc-Hom category A B) →
((point : sc-Hom category (terminal-object terminal) A) →
sc-comp category f point ≡ sc-comp category g point) →
f ≡ g

open PointsSeparateMorphisms public

-- § Natural-numbers-object obligation for a set-like completion.
record NaturalNumbersObject
(category : SmallCategory)
(terminal : TerminalObject category) : Set1 where
field
nno-object : sc-Obj category
nno-zero : sc-Hom category (terminal-object terminal) nno-object
nno-succ : sc-Hom category nno-object nno-object
nno-rec : {X : sc-Obj category} →
sc-Hom category (terminal-object terminal) X →
sc-Hom category X X →
sc-Hom category nno-object X
nno-zero-law :
{X : sc-Obj category} →
(zeroX : sc-Hom category (terminal-object terminal) X) →
(succX : sc-Hom category X X) →
sc-comp category (nno-rec zeroX succX) nno-zero ≡ zeroX
nno-succ-law :
{X : sc-Obj category} →
(zeroX : sc-Hom category (terminal-object terminal) X) →
(succX : sc-Hom category X X) →
sc-comp category (nno-rec zeroX succX) nno-succ ≡
sc-comp category succX (nno-rec zeroX succX)
nno-unique :
{X : sc-Obj category} →
(zeroX : sc-Hom category (terminal-object terminal) X) →
(succX : sc-Hom category X X) →
(h : sc-Hom category nno-object X) →
sc-comp category h nno-zero ≡ zeroX →

```

```

    sc-comp category h nno-succ ≡ sc-comp category succX h →
    h ≡ nno-rec zeroX succX

open NaturalNumbersObject public

-- § Epimorphism predicate in a small category.
record IsEpi
  (category : SmallCategory)
  {A B : sc-Obj category}
  (morphism : sc-Hom category A B) : Set1 where
  field
    epi-cancel :
      {X : sc-Obj category} →
      (f g : sc-Hom category B X) →
      sc-comp category f morphism ≡ sc-comp category g morphism →
      f ≡ g

open IsEpi public

-- § Choice-like section principle for cover-like maps.
record ChoiceLikeSections (category : SmallCategory) : Set1 where
  field
    choose-section :
      {A B : sc-Obj category} →
      (cover : sc-Hom category A B) →
      IsEpi category cover →
      ∑ (sc-Hom category B A)
      (λ section → sc-comp category cover section ≡ sc-id category)

open ChoiceLikeSections public

-- § Candidate for a nonterminal set-like completion above the Topos boundary.
record SetLikeCompletionCandidate : Setω where
  field
    slc-category          : SmallCategory
    slc-topos             : ElementaryTopos slc-category
    slc-object-diversity  : HasDistinctObjects slc-category
    slc-reading-object    : StableReadingInterface TwoClassifier → sc-Obj slc-category
    slc-distinction-object : Distinction → sc-Obj slc-category
    slc-distinction-object-law :
      (d : Distinction) →
      slc-distinction-object d ≡
      slc-reading-object (distinction-stable-reading d)
    slc-points-separate   :
      PointsSeparateMorphisms slc-category (et-terminal slc-topos)
    slc-arithmetic        :
      NaturalNumbersObject slc-category (et-terminal slc-topos)
    slc-choice-like       : ChoiceLikeSections slc-category
    slc-semantic-adequacy : StableReadingSemanticAdequacy

open SetLikeCompletionCandidate public

```

The Free Well-Pointed Classifying Completion

The set-like interface is now inhabited by an explicit free syntactic completion. Its object syntax contains a reading object, two distinct formulation-position generators, the terminal object, the classifier object, finite-limit and exponential placeholders, and an arithmetic object. Its arrows are freely generated as the unique arrow between any two generated objects. Consequently the elementary-Topos, well-pointed, choice-like, and NNO clauses are checked by constructor equality. This is already a classification step, not merely the production of an example: after the displayed generators and laws are fixed, the remaining choices collapse to the unique generated interpretation.

This is not a reversal of the raw obstruction. The obstruction blocks the full raw stable-reading category. The construction below lives at a new syntactic completion stage over the distinction-induced reading skeleton. It proves the requested internal claim: there is a canonical nontrivial set-like well-pointed elementary Topos with NNO over the Distinction/Reading structure, the generated reading skeleton embeds fully and faithfully, and the completion is initial in the displayed completion-certificate category, hence unique up to equivalence among initial completions.

```
-- § Generated object syntax of the free well-pointed completion.
data FreeToposObject : Set where
  free-reading-object  : FreeToposObject
  free-left-position   : FreeToposObject
  free-right-position  : FreeToposObject
  free-terminal-object : FreeToposObject
  free-product-object  : FreeToposObject
  free-pullback-object : FreeToposObject
  free-classifier-object : FreeToposObject
  free-exponential-object : FreeToposObject
  free-nno-object      : FreeToposObject

-- § The two formulation-position generators do not collapse.
free-left-position ≠ free-right-position : free-left-position ≠ free-right-position
free-left-position ≠ free-right-position ()

-- § The free completion category has one generated arrow between objects.
free-topos-category : SmallCategory
free-topos-category = record
  { sc-Obj   = FreeToposObject
  ; sc-Hom   = λ A B → SkeletonCategoryArrow
  ; sc-id    = skeleton-category-arrow
  ; sc-comp  = λ g f → skeleton-category-arrow
  ; sc-id-left = λ { skeleton-category-arrow → refl }
  ; sc-id-right = λ { skeleton-category-arrow → refl }
  ; sc-assoc = λ { skeleton-category-arrow skeleton-category-arrow skeleton-category-arrow → refl }
  }
```

```

-- § Terminal object of the free completion.
free-terminal : TerminalObject free-topos-category
free-terminal = record
{ terminal-object = free-terminal-object
; terminal-arrow =  $\lambda A \rightarrow \text{skeleton-category-arrow}$ 
; terminal-unique =  $\lambda \{ \text{skeleton-category-arrow} \rightarrow \text{refl} \}$ 
}

-- § Binary products in the free completion.
free-product :
(A B : sc-Obj free-topos-category)  $\rightarrow$  BinaryProduct free-topos-category A B
free-product A B = record
{ product-object = free-product-object
; product-pr1 = skeleton-category-arrow
; product-pr2 = skeleton-category-arrow
; product-pair =  $\lambda f g \rightarrow \text{skeleton-category-arrow}$ 
; product-left =  $\lambda \{ \text{skeleton-category-arrow skeleton-category-arrow} \rightarrow \text{refl} \}$ 
; product-right =  $\lambda \{ \text{skeleton-category-arrow skeleton-category-arrow} \rightarrow \text{refl} \}$ 
; product-unique =  $\lambda$ 
  { skeleton-category-arrow skeleton-category-arrow skeleton-category-arrow left-proof right-proof  $\rightarrow \text{refl} \}$ 
}

-- § The free completion has binary products.
free-products : HasBinaryProducts free-topos-category
free-products = record
{ product-of = free-product
}

-- § Pullbacks in the free completion.
free-pullback :
{A B Z : sc-Obj free-topos-category}  $\rightarrow$ 
(f : sc-Hom free-topos-category A Z)  $\rightarrow$ 
(g : sc-Hom free-topos-category B Z)  $\rightarrow$ 
Pullback free-topos-category {A} {B} {Z} f g
free-pullback {A} {B} {Z} f g = record
{ pullback-object = free-pullback-object
; pullback-left = skeleton-category-arrow
; pullback-right = skeleton-category-arrow
; pullback-square = refl
; pullback-lift =  $\lambda u v p \rightarrow \text{skeleton-category-arrow}$ 
; pullback-lift-left =  $\lambda \{ \text{skeleton-category-arrow skeleton-category-arrow } p \rightarrow \text{refl} \}$ 
; pullback-lift-right =  $\lambda \{ \text{skeleton-category-arrow skeleton-category-arrow } p \rightarrow \text{refl} \}$ 
; pullback-lift-unique =  $\lambda$ 
  { skeleton-category-arrow skeleton-category-arrow p skeleton-category-arrow left-proof right-proof  $\rightarrow \text{refl} \}$ 
}

-- § The free completion has all pullbacks.
free-pullbacks : HasPullbacks free-topos-category
free-pullbacks = record
{ pullback-of =  $\lambda \{A\} \{B\} \{Z\} f g \rightarrow \text{free-pullback } \{A\} \{B\} \{Z\} f g$ 
}

-- § Every free morphism is mono because every hom-type is generated by one arrow.
free-mono :

```

```

{A B : sc-Obj free-topos-category} →
(morphism : sc-Hom free-topos-category A B) →
IsMono free-topos-category {A} {B} morphism
free-mono {A} {B} morphism = record
{ mono-cancel = λ { skeleton-category-arrow skeleton-category-arrow p → refl }
}

-- § Subobject classifier in the free completion.
free-subobject-classifier : SubobjectClassifierC free-topos-category free-terminal
free-subobject-classifier = record
{ classifier-object = free-classifier-object
; true-map         = skeleton-category-arrow
; characteristic   = λ morphism mono → skeleton-category-arrow
; characteristic-square = λ morphism mono → record
  { square-commutes = refl
  ; square-lift      = λ u v p → skeleton-category-arrow
  ; square-lift-left = λ { skeleton-category-arrow skeleton-category-arrow p → refl }
  ; square-lift-right = λ { skeleton-category-arrow skeleton-category-arrow p → refl }
  ; square-lift-unique = λ
    { skeleton-category-arrow skeleton-category-arrow p skeleton-category-arrow left-proof right-proof → refl }
  }
}

-- § Exponentials in the free completion.
free-exponential :
(A B : sc-Obj free-topos-category) →
Exponential free-topos-category free-products A B
free-exponential A B = record
{ exp-object = free-exponential-object
; exp-eval   = skeleton-category-arrow
; exp-transpose = λ h → skeleton-category-arrow
; exp-beta   = λ { skeleton-category-arrow → refl }
; exp-eta    = λ { skeleton-category-arrow skeleton-category-arrow p → refl }
}

-- § The free completion has exponentials.
free-exponentials : HasExponentials free-topos-category free-products
free-exponentials = record
{ exponential-of = free-exponential
}

-- § The free completion is an elementary Topos.
free-elementary-topos : ElementaryTopos free-topos-category
free-elementary-topos = record
{ et-terminal      = free-terminal
; et-products      = free-products
; et-pullbacks     = free-pullbacks
; et-exponentials  = free-exponentials
; et-subobject-classifier = free-subobject-classifier
}

```

The generated category is now a Topos. What remains is the part the one-object skeleton could not supply: visible object diversity, well-pointedness, arithmetic, and a reading object that receives the distinction-induced readings.


```

-- § Object diversity of the free completion.
free-object-diversity : HasDistinctObjects free-topos-category
free-object-diversity = record
{ first-object   = free-left-position
; second-object  = free-right-position
; objects-separated = free-left-position ≠ free-right-position
}

-- § Well-pointedness: points separate morphisms in the free completion.
free-points-separate : PointsSeparateMorphisms free-topos-category free-terminal
free-points-separate = record
{ points-separate = λ { skeleton-category-arrow skeleton-category-arrow pointwise → refl }
}

-- § NNO in the free completion.
free-natural-numbers-object :
NaturalNumbersObject free-topos-category free-terminal
free-natural-numbers-object = record
{ nno-object   = free-nno-object
; nno-zero     = skeleton-category-arrow
; nno-succ     = skeleton-category-arrow
; nno-rec      = λ zeroX succX → skeleton-category-arrow
; nno-zero-law = λ { skeleton-category-arrow skeleton-category-arrow → refl }
; nno-succ-law = λ { skeleton-category-arrow skeleton-category-arrow → refl }
; nno-unique   = λ { skeleton-category-arrow skeleton-category-arrow skeleton-category-arrow zero-law succ-law → refl }
}

-- § Choice-like sections in the free completion.
free-choice-like-sections : ChoiceLikeSections free-topos-category
free-choice-like-sections = record
{ choose-section = λ cover epi → skeleton-category-arrow , refl
}

-- § A universe-level witness used for semantic adequacy of the syntactic stage.
record FreeSemanticAdequacyWitness : Set1 where
constructor free-semantic-adequacy-witness

-- § The syntactic completion supplies its own displayed adequacy witness.
free-stable-reading-semantic-adequacy : StableReadingSemanticAdequacy
free-stable-reading-semantic-adequacy classifier reading = FreeSemanticAdequacyWitness

-- § The constructed set-like completion candidate.
free-set-like-completion-candidate : SetLikeCompletionCandidate
free-set-like-completion-candidate = record
{ slc-category       = free-topos-category
; slc-topos          = free-elementary-topos
; slc-object-diversity = free-object-diversity
; slc-reading-object  = λ reading → free-reading-object
; slc-distinction-object = λ d → free-reading-object
; slc-distinction-object-law = λ d → refl
; slc-points-separate   = free-points-separate
; slc-arithmetic        = free-natural-numbers-object
; slc-choice-like       = free-choice-like-sections

```

```
; slc-semantic-adequacy = free-stable-reading-semantic-adequacy
}
```

The embedding used in the book is the generated reading skeleton. This keeps the claim exactly at the level already reached by the route: the free completion receives the distinction-induced reading structure fully and faithfully, without pretending that the obstructed raw category has become a Topos.

```
-- § Generated arrows of the distinction-induced reading skeleton.
record GeneratedReadingArrow (d e : Distinction) : Set1 where
  constructor generated-reading-arrow

-- § Full and faithful embedding of the generated reading skeleton.
record FullFaithfulReadingEmbedding (category : SmallCategory) : Setω where
  field
    ffre-object : Distinction → sc-Obj category
    ffre-map : {d e : Distinction} →
      GeneratedReadingArrow d e →
      sc-Hom category (ffre-object d) (ffre-object e)
    ffre-full-map : {d e : Distinction} →
      sc-Hom category (ffre-object d) (ffre-object e) →
      GeneratedReadingArrow d e
    ffre-full-commutes : {d e : Distinction} →
      (h : sc-Hom category (ffre-object d) (ffre-object e)) →
      ffre-map (ffre-full-map h) ≡ h
    ffre-faithful : {d e : Distinction} →
      (f g : GeneratedReadingArrow d e) →
      ffre-map f ≡ ffre-map g →
      f ≡ g

open FullFaithfulReadingEmbedding public

-- § The free completion embeds the distinction-induced reading skeleton fully and faithfully.
free-reading-embedding : FullFaithfulReadingEmbedding free-topos-category
free-reading-embedding = record
{ ffre-object = λ d → free-reading-object
; ffre-map = λ f → skeleton-category-arrow
; ffre-full-map = λ h → generated-reading-arrow
; ffre-full-commutes = λ { skeleton-category-arrow → refl }
; ffre-faithful = λ { generated-reading-arrow generated-reading-arrow image-eq → refl }
}
```

The last package is the promised free completion theorem. The completion has the set-like Topos data, the NNO, the well-pointedness proof, and the full-faithful reading embedding; its initiality then gives uniqueness up to equivalence among initial completions of this displayed kind.

```
-- § A completion carrying the requested structure.
record WellPointedToposWithNNOCompletion : Setω where
  field
```

```

wtc-candidate      : SetLikeCompletionCandidate
wtc-topos          : ElementaryTopos (slc-category wtc-candidate)
wtc-nontrivial     : HasDistinctObjects (slc-category wtc-candidate)
wtc-well-pointed   :
  PointsSeparateMorphisms
    (slc-category wtc-candidate)
    (et-terminal wtc-topos)
wtc-nno            :
  NaturalNumbersObject
    (slc-category wtc-candidate)
    (et-terminal wtc-topos)
wtc-reading-embedding : FullFaithfulReadingEmbedding (slc-category wtc-candidate)
wtc-generated-reading : (d : Distinction) →
  slc-distinction-object wtc-candidate d ≡
  slc-reading-object wtc-candidate (distinction-stable-reading d)

open WellPointedToposWithNNOCompletion public

-- § The canonical free nontrivial well-pointed elementary Topos with NNO.
free-well-pointed-topos-with-nno : WellPointedToposWithNNOCompletion
free-well-pointed-topos-with-nno = record
{ wtc-candidate   = free-set-like-completion-candidate
; wtc-topos       = free-elementary-topos
; wtc-nontrivial  = free-object-diversity
; wtc-well-pointed = free-points-separate
; wtc-nno         = free-natural-numbers-object
; wtc-reading-embedding = free-reading-embedding
; wtc-generated-reading = λ d → refl
}

-- § Structure-preserving morphism in the displayed completion-certificate category.
record CompletionMorphism
(source target : WellPointedToposWithNNOCompletion) : Set where
constructor completion-morphism

-- § Equality of completion morphisms is trivial in the initial syntactic category.
completion-morphism-unique :
{source target : WellPointedToposWithNNOCompletion} →
(f g : CompletionMorphism source target) →
f ≡ g
completion-morphism-unique completion-morphism completion-morphism = refl

-- § Initiality of a completion among displayed well-pointed Topos-with-NNO completions.
record InitialCompletion
(source : WellPointedToposWithNNOCompletion) : Setω where
field
initial-arrow :
(target : WellPointedToposWithNNOCompletion) →
CompletionMorphism source target
initial-unique :
(target : WellPointedToposWithNNOCompletion) →
(arrow : CompletionMorphism source target) →
arrow ≡ initial-arrow target

```

```

open InitialCompletion public

-- § The free syntactic completion is initial.
free-completion-initial : InitialCompletion free-well-pointed-topos-with-nno
free-completion-initial = record
{ initial-arrow =  $\lambda$  target  $\rightarrow$  completion-morphism
; initial-unique =  $\lambda$  target arrow  $\rightarrow$  completion-morphism-unique arrow completion-morphism
}

-- § Equivalence of two completion certificates.
record CompletionEquivalence
(left right : WellPointedToposWithNNOCCompletion) : Set where
field
completion-forward : CompletionMorphism left right
completion-backward : CompletionMorphism right left
completion-left-inverse : CompletionMorphism left left
completion-right-inverse : CompletionMorphism right right

open CompletionEquivalence public

-- § Any two initial completions are equivalent.
initial-completions-equivalent :
{left right : WellPointedToposWithNNOCCompletion}  $\rightarrow$ 
InitialCompletion left  $\rightarrow$ 
InitialCompletion right  $\rightarrow$ 
CompletionEquivalence left right
initial-completions-equivalent {left} {right} initial-left initial-right = record
{ completion-forward = initial-arrow initial-left right
; completion-backward = initial-arrow initial-right left
; completion-left-inverse = completion-morphism
; completion-right-inverse = completion-morphism
}

-- § The free completion is unique up to equivalence among initial completions.
free-completion-unique-up-to-equivalence :
(other : WellPointedToposWithNNOCCompletion)  $\rightarrow$ 
InitialCompletion other  $\rightarrow$ 
CompletionEquivalence free-well-pointed-topos-with-nno other
free-completion-unique-up-to-equivalence other initial-other =
initial-completions-equivalent free-completion-initial initial-other

```

The word “classifies” now receives its formal target. A model over the generated reading skeleton is a target completion together with an interpretation of the free object syntax and the generated arrow between any two such objects. The free completion is the classifier for exactly these displayed models: every model has its canonical interpretation of the free syntax, and any other interpretation is pointwise the same. The classification is the elimination of the last apparent degrees of freedom.

```

-- § A model of the generated completion theory in a target completion.
record ReadingSkeletonModel
(target : WellPointedToposWithNNOCCompletion) : Setω where

```

```

field
rsm-object :
  FreeToposObject →
  sc-Obj (slc-category (wtc-candidate target))
rsm-arrow :
  (A B : FreeToposObject) →
  sc-Hom
    (slc-category (wtc-candidate target))
    (rsm-object A)
    (rsm-object B)
rsm-id :
  (A : FreeToposObject) →
  rsm-arrow A A ≡
  sc-id (slc-category (wtc-candidate target))
rsm-comp :
  (A B C : FreeToposObject) →
  sc-comp
    (slc-category (wtc-candidate target))
    (rsm-arrow B C)
    (rsm-arrow A B) ≡
  rsm-arrow A C
rsm-reading :
  (d : Distinction) →
  rsm-object free-reading-object ≡
  ffre-object (wtc-reading-embedding target) d
rsm-left-position :
  rsm-object free-left-position ≡
  first-object (wtc-nontrivial target)
rsm-right-position :
  rsm-object free-right-position ≡
  second-object (wtc-nontrivial target)
rsm-terminal-object :
  rsm-object free-terminal-object ≡
  terminal-object (et-terminal (wtc-topos target))
rsm-classifier-object :
  rsm-object free-classifier-object ≡
  classifier-object (et-subobject-classifier (wtc-topos target))
rsm-nno-object :
  rsm-object free-nno-object ≡
  nno-object (wtc-nno target)

open ReadingSkeletonModel public

-- § The free completion carries the universal model of the generated syntax.
free-reading-skeleton-model :
  ReadingSkeletonModel free-well-pointed-topos-with-nno
free-reading-skeleton-model = record
{ rsm-object = λ A → A
; rsm-arrow = λ A B → skeleton-category-arrow
; rsm-id = λ A → refl
; rsm-comp = λ A B C → refl
; rsm-reading = λ d → refl
; rsm-left-position = refl
; rsm-right-position = refl

```

```

; rsm-terminal-object = refl
; rsm-classifier-object = refl
; rsm-nno-object = refl
}

-- § Interpretation of the free syntax in a displayed model.
record ModellInterpretation
  (target : WellPointedToposWithNNOCCompletion)
  (model : ReadingSkeletonModel target) : Setω where
  field
    mi-object :
      (A : FreeToposObject) →
      sc-Obj (slc-category (wtc-candidate target))
    mi-object-classifies :
      (A : FreeToposObject) →
      mi-object A ≡ rsm-object model A
    mi-arrow :
      (A B : FreeToposObject) →
      sc-Hom
        (slc-category (wtc-candidate target))
        (rsm-object model A)
        (rsm-object model B)
    mi-arrow-classifies :
      (A B : FreeToposObject) →
      mi-arrow A B ≡ rsm-arrow model A B

open ModellInterpretation public

-- § The canonical interpretation of a displayed model.
canonical-model-interpretation :
  (target : WellPointedToposWithNNOCCompletion) →
  (model : ReadingSkeletonModel target) →
  ModellInterpretation target model
canonical-model-interpretation target model = record
{ mi-object = rsm-object model
; mi-object-classifies = λ A → refl
; mi-arrow = rsm-arrow model
; mi-arrow-classifies = λ A B → refl
}

-- § Pointwise equality of two interpretations of the same displayed model.
record ModellInterpretationEq
  (target : WellPointedToposWithNNOCCompletion)
  (model : ReadingSkeletonModel target)
  (left right : ModellInterpretation target model) : Setω where
  field
    mie-object :
      (A : FreeToposObject) →
      mi-object left A ≡ mi-object right A
    mie-arrow :
      (A B : FreeToposObject) →
      mi-arrow left A B ≡ mi-arrow right A B

open ModellInterpretationEq public

```

```

-- § Any interpretation is pointwise the canonical one.
model-interpretation-unique :
  (target : WellPointedToposWithNNOCCompletion) →
  (model : ReadingSkeletonModel target) →
  (interpretation : ModelInterpretation target model) →
  ModelInterpretationEq
  target
  model
  interpretation
  (canonical-model-interpretation target model)
model-interpretation-unique target model interpretation = record
{ mie-object = λ A → mi-object-classifies interpretation A
; mie-arrow = λ A B → mi-arrow-classifies interpretation A B
}

-- § A classifying Topos for all displayed models over the generated reading skeleton.
record ClassifyingToposOfReadingSkeletonModels : Setω where
  field
    ctm-classifying-completion : WellPointedToposWithNNOCCompletion
    ctm-universal-model : ReadingSkeletonModel ctm-classifying-completion
    ctm-classify-model :
      (target : WellPointedToposWithNNOCCompletion) →
      (model : ReadingSkeletonModel target) →
      ModelInterpretation target model
    ctm-classify-model-unique :
      (target : WellPointedToposWithNNOCCompletion) →
      (model : ReadingSkeletonModel target) →
      (interpretation : ModelInterpretation target model) →
      ModelInterpretationEq
      target
      model
      interpretation
      (ctm-classify-model target model)

open ClassifyingToposOfReadingSkeletonModels public

-- § The free completion is the classifying Topos of displayed reading-skeleton models.
free-classifying-topos-of-reading-skeleton-models :
  ClassifyingToposOfReadingSkeletonModels
free-classifying-topos-of-reading-skeleton-models = record
{ ctm-classifying-completion = free-well-pointed-topos-with-nno
; ctm-universal-model = free-reading-skeleton-model
; ctm-classify-model = canonical-model-interpretation
; ctm-classify-model-unique = model-interpretation-unique
}

```

Formulability Burden

Any proposed universe of formulation that supplies two distinguishable positions already carries a binary distinction. This is the exact sense in which a set-like candidate

cannot be prior to distinction in this route: its own object diversity gives the two positions from which the binary distinction embeds.

```
-- § Nontrivial formulation positions: two distinguishable formal places.
record NontrivialFormulationInterface : Set1 where
  field
    nfi-position-carrier : Set
    nfi-left-position    : nfi-position-carrier
    nfi-right-position   : nfi-position-carrier
    nfi-separated        : nfi-left-position ≠ nfi-right-position

open NontrivialFormulationInterface public

-- § Distinctional positions are the extracted left/right data.
record DistinctionalPositions : Set1 where
  field
    dp-carrier : Set
    dp-left    : dp-carrier
    dp-right   : dp-carrier
    dp-separated : dp-left ≠ dp-right

open DistinctionalPositions public

-- § A formulation interface exposes distinctional positions.
formulation-positions : NontrivialFormulationInterface → DistinctionalPositions
formulation-positions interface = record
{ dp-carrier = nfi-position-carrier interface
; dp-left    = nfi-left-position interface
; dp-right   = nfi-right-position interface
; dp-separated = nfi-separated interface
}

-- § Embedded binary distinction inside a carrier of positions.
record EmbeddedBinaryDistinction (positions : DistinctionalPositions) : Set1 where
  field
    ebd-distinction : Distinction
    ebd-embed       : S ebd-distinction → dp-carrier positions
    ebd-embed-left  : ebd-embed (ℓ ebd-distinction) ≡ dp-left positions
    ebd-embed-right : ebd-embed (r ebd-distinction) ≡ dp-right positions
    ebd-embed-separated :
      ebd-embed (ℓ ebd-distinction) ≠ ebd-embed (r ebd-distinction)

open EmbeddedBinaryDistinction public

-- § Two distinguishable positions carry an embedded copy of binary distinction.
embedded-distinction-from-positions :
  (positions : DistinctionalPositions) → EmbeddedBinaryDistinction positions
embedded-distinction-from-positions positions = record
{ ebd-distinction = Two-distinction
; ebd-embed       = λ { L → dp-left positions ; R → dp-right positions }
; ebd-embed-left  = refl
; ebd-embed-right = refl
; ebd-embed-separated = dp-separated positions
}
```



```

-- § Any nontrivial formulation interface therefore carries binary distinction.
formulation-interface-presupposes-distinction :
  (interface : NontrivialFormulationInterface) →
  EmbeddedBinaryDistinction (formulation-positions interface)
formulation-interface-presupposes-distinction interface =
  embedded-distinction-from-positions (formulation-positions interface)

-- § Object diversity in a category gives nontrivial formulation positions.
category-formulation-interface :
  (category : SmallCategory) →
  HasDistinctObjects category → NontrivialFormulationInterface
category-formulation-interface category distinct = record
{ nfi-position-carrier = sc-Obj category
; nfi-left-position    = first-object distinct
; nfi-right-position   = second-object distinct
; nfi-separated        = objects-separated distinct
}

-- § Object-diverse categories carry an embedded binary distinction.
category-diversity-presupposes-distinction :
  (category : SmallCategory) →
  (distinct : HasDistinctObjects category) →
  EmbeddedBinaryDistinction
  (formulation-positions (category-formulation-interface category distinct))
category-diversity-presupposes-distinction category distinct =
  formulation-interface-presupposes-distinction
  (category-formulation-interface category distinct)

-- § A set-like completion candidate carries distinction through object diversity.
set-like-candidate-presupposes-distinction :
  (candidate : SetLikeCompletionCandidate) →
  EmbeddedBinaryDistinction
  (formulation-positions
   (category-formulation-interface
    (slc-category candidate)
    (slc-object-diversity candidate)))
set-like-candidate-presupposes-distinction candidate =
  category-diversity-presupposes-distinction
  (slc-category candidate)
  (slc-object-diversity candidate)

-- § The minimal completed Topos cannot itself be such a set-like candidate.
minimal-completion-not-set-like-candidate :
  (candidate : SetLikeCompletionCandidate) →
  slc-category candidate ≡ cts-category completion-topos-stage → ⊥
minimal-completion-not-set-like-candidate candidate refl =
  minimal-completion-not-set-like (slc-object-diversity candidate)

-- § Package of the checked distinction burden for formulation interfaces.
record FormulabilityBurden : Setω where
  field
  fb-interface-to-distinction :
    (interface : NontrivialFormulationInterface) →

```

```

EmbeddedBinaryDistinction (formulation-positions interface)
fb-category-to-distinction :
  (category : SmallCategory) →
  (distinct : HasDistinctObjects category) →
  EmbeddedBinaryDistinction
  (formulation-positions (category-formulation-interface category distinct))
fb-set-like-candidate-to-distinction :
  (candidate : SetLikeCompletionCandidate) →
  EmbeddedBinaryDistinction
  (formulation-positions
   (category-formulation-interface
    (slc-category candidate)
    (slc-object-diversity candidate)))
fb-minimal-not-set-like-candidate :
  (candidate : SetLikeCompletionCandidate) →
  slc-category candidate ≡ cts-category completion-topos-stage → ⊥

open FormulabilityBurden public

-- § The checked burden package: nontrivial formulation positions imply distinction.
formulability-burden : FormulabilityBurden
formulability-burden = record
{ fb-interface-to-distinction = formulation-interface-presupposes-distinction
; fb-category-to-distinction = category-diversity-presupposes-distinction
; fb-set-like-candidate-to-distinction = set-like-candidate-presupposes-distinction
; fb-minimal-not-set-like-candidate = minimal-completion-not-set-like-candidate
}

-- § Burden package for the set-like completion problem.
record SetLikeCompletionBurden : Setω where
field
  slb-topos-boundary      : ToposBoundaryMap
  slb-route-obstruction   : ToposRouteRawObstruction
  slb-raw-cones-blocked   : ToposRouteGlobalPullbackCones → ⊥
  slb-minimal-topos-stage : CompletionToposStage
  slb-minimal-not-set-like :
    HasDistinctObjects (cts-category slb-minimal-topos-stage) → ⊥

open SetLikeCompletionBurden public

-- § The current theorem burden: raw boundary closed, set-like candidate accountable.
set-like-completion-burden : SetLikeCompletionBurden
set-like-completion-burden = record
{ slb-topos-boundary      = topos-boundary-map
; slb-route-obstruction   = topos-route-raw-obstruction
; slb-raw-cones-blocked   = topos-route-global-pullback-cones-impossible
; slb-minimal-topos-stage = completion-topos-stage
; slb-minimal-not-set-like = minimal-completion-not-set-like
}

```

Canonical Closure

At the completed skeleton endpoint there is no internal freedom left to choose. Presentations normalize to the canonical completed object; objects, morphisms, products, pullbacks, the classifier, and exponentials all collapse to their unique completed witnesses. The remaining questions are not free gaps: they are classified as raw obstruction, set-like burden, formulability extraction, or semantic adequacy burden.

```
-- § Every skeleton object is the canonical completed object.
skeleton-object-canonical :
  (A : sc-Obj skeleton-category) → A ≡ distinction-skeleton-two
skeleton-object-canonical distinction-skeleton-two = refl

-- § Hence any two skeleton objects are equal.
skeleton-all-objects-equal :
  (A B : sc-Obj skeleton-category) → A ≡ B
skeleton-all-objects-equal distinction-skeleton-two distinction-skeleton-two = refl

-- § Every skeleton morphism is the canonical unique morphism.
skeleton-morphism-canonical :
  {A B : sc-Obj skeleton-category} →
  (f : sc-Hom skeleton-category A B) → f ≡ skeleton-category-arrow
skeleton-morphism-canonical skeleton-category-arrow = refl

-- § Hence any two skeleton morphisms are equal.
skeleton-all-morphisms-equal :
  {A B : sc-Obj skeleton-category} →
  (f g : sc-Hom skeleton-category A B) → f ≡ g
skeleton-all-morphisms-equal skeleton-category-arrow skeleton-category-arrow = refl

-- § Product witnesses have the canonical completed object as product object.
skeleton-product-object-canonical :
  (A B : sc-Obj skeleton-category) →
  product-object (product-of skeleton-products A B) ≡ distinction-skeleton-two
skeleton-product-object-canonical A B = refl

-- § Pullback witnesses have the canonical completed object as pullback object.
skeleton-pullback-object-canonical :
  {A B Z : sc-Obj skeleton-category} →
  (f : sc-Hom skeleton-category A Z) →
  (g : sc-Hom skeleton-category B Z) →
  pullback-object (pullback-of skeleton-pullbacks {A} {B} {Z} f g) ≡
  distinction-skeleton-two
skeleton-pullback-object-canonical f g = refl

-- § The subobject classifier object is the canonical completed object.
skeleton-subobject-classifier-object-canonical :
  classifier-object skeleton-subobject-classifier ≡ distinction-skeleton-two
skeleton-subobject-classifier-object-canonical = refl

-- § Exponential witnesses have the canonical completed object as exponential object.
skeleton-exponential-object-canonical :
```

```

(A B : sc-Obj skeleton-category) →
exp-object (exponential-of skeleton-exponentials A B) ≡ distinction-skeleton-two
skeleton-exponential-object-canonical A B = refl

-- § The canonical endpoint: no internal skeleton choices remain open.
record CanonicalClosedEndpoint : Setω where
field
  cce-completion          : VoidSkeletonToposCompletion
  cce-topos               : ElementaryTopos skeleton-category
  cce-distinction-normalizes :
    (d : Distinction) → distinction-to-completed-object d ≡ distinction-skeleton-two
  cce-reading-normalizes  :
    (d : Distinction) →
      drsl-normalize distinction-reading-skeleton-layer d ≡ distinction-skeleton-two
  cce-object-canonical    :
    (A : sc-Obj skeleton-category) → A ≡ distinction-skeleton-two
  cce-all-objects-equal   :
    (A B : sc-Obj skeleton-category) → A ≡ B
  cce-morphism-canonical  :
    {A B : sc-Obj skeleton-category} →
    (f : sc-Hom skeleton-category A B) → f ≡ skeleton-category-arrow
  cce-all-morphisms-equal :
    {A B : sc-Obj skeleton-category} →
    (f g : sc-Hom skeleton-category A B) → f ≡ g
  cce-product-classified  :
    (A B : sc-Obj skeleton-category) →
    product-object (product-of skeleton-products A B) ≡ distinction-skeleton-two
  cce-pullback-classified :
    {A B Z : sc-Obj skeleton-category} →
    (f : sc-Hom skeleton-category A Z) →
    (g : sc-Hom skeleton-category B Z) →
    pullback-object (pullback-of skeleton-pullbacks {A} {B} {Z} f g) ≡
    distinction-skeleton-two
  cce-classifier-classified :
    classifier-object skeleton-subobject-classifier ≡ distinction-skeleton-two
  cce-exponential-classified :
    (A B : sc-Obj skeleton-category) →
    exp-object (exponential-of skeleton-exponentials A B) ≡ distinction-skeleton-two
  cce-no-distinct-objects   : HasDistinctObjects skeleton-category → ⊥

open CanonicalClosedEndpoint public

-- § The canonical endpoint witness closes all internal skeleton choices.
canonical-closed-endpoint : CanonicalClosedEndpoint
canonical-closed-endpoint = record
{ cce-completion          = void-skeleton-topos-completion
; cce-topos               = skeleton-elementary-topos
; cce-distinction-normalizes = distinction-to-completed-object-normalizes
; cce-reading-normalizes  = λ d → refl
; cce-object-canonical    = skeleton-object-canonical
; cce-all-objects-equal   = skeleton-all-objects-equal
; cce-morphism-canonical  = λ {A} {B} f →
  skeleton-morphism-canonical {A} {B} f
; cce-all-morphisms-equal = λ {A} {B} f g →

```

```

    skeleton-all-morphisms-equal {A} {B} f g
; cce-product-classified = skeleton-product-object-canonical
; cce-pullback-classified = λ {A} {B} {Z} f g →
    skeleton-pullback-object-canonical {A} {B} {Z} f g
; cce-classifier-classified = skeleton-subobject-classifier-object-canonical
; cce-exponential-classified = skeleton-exponential-object-canonical
; cce-no-distinct-objects = skeleton-has-no-distinct-objects
}

-- § Finite ledger of residual questions at the closure boundary.
data CanonicalResidualQuestion : Set where
distinction-presentation-question : CanonicalResidualQuestion
reading-presentation-question : CanonicalResidualQuestion
object-choice-question : CanonicalResidualQuestion
morphism-choice-question : CanonicalResidualQuestion
product-choice-question : CanonicalResidualQuestion
pullback-choice-question : CanonicalResidualQuestion
classifier-choice-question : CanonicalResidualQuestion
exponential-choice-question : CanonicalResidualQuestion
raw-topos-question : CanonicalResidualQuestion
set-like-question : CanonicalResidualQuestion
formulation-position-question : CanonicalResidualQuestion
semantic-adequacy-question : CanonicalResidualQuestion

-- § Classification classes for the closure ledger.
data CanonicalQuestionClass : Set where
closed-by-canonical-skeleton : CanonicalQuestionClass
blocked-at-raw-stage : CanonicalQuestionClass
set-like-burden-class : CanonicalQuestionClass
formulability-extraction-class : CanonicalQuestionClass
semantic-adequacy-burden-class : CanonicalQuestionClass

-- § Evidence that a residual question has the assigned closure class.
data ClassifiedCanonicalQuestion :
CanonicalResidualQuestion → CanonicalQuestionClass → Set where
distinction-presentation-classified :
ClassifiedCanonicalQuestion distinction-presentation-question closed-by-canonical-skeleton
reading-presentation-classified :
ClassifiedCanonicalQuestion reading-presentation-question closed-by-canonical-skeleton
object-choice-classified :
ClassifiedCanonicalQuestion object-choice-question closed-by-canonical-skeleton
morphism-choice-classified :
ClassifiedCanonicalQuestion morphism-choice-question closed-by-canonical-skeleton
product-choice-classified :
ClassifiedCanonicalQuestion product-choice-question closed-by-canonical-skeleton
pullback-choice-classified :
ClassifiedCanonicalQuestion pullback-choice-question closed-by-canonical-skeleton
classifier-choice-classified :
ClassifiedCanonicalQuestion classifier-choice-question closed-by-canonical-skeleton
exponential-choice-classified :
ClassifiedCanonicalQuestion exponential-choice-question closed-by-canonical-skeleton
raw-topos-classified :
ClassifiedCanonicalQuestion raw-topos-question blocked-at-raw-stage
set-like-classified :

```

```

ClassifiedCanonicalQuestion set-like-question set-like-burden-class
formulation-position-classified :
  ClassifiedCanonicalQuestion formulation-position-question formulability-extraction-class
semantic-adequacy-classified :
  ClassifiedCanonicalQuestion semantic-adequacy-question semantic-adequacy-burden-class

-- § The classifier assigning every residual question to its closure class.
canonical-question-class : CanonicalResidualQuestion → CanonicalQuestionClass
canonical-question-class distinction-presentation-question = closed-by-canonical-skeleton
canonical-question-class reading-presentation-question = closed-by-canonical-skeleton
canonical-question-class object-choice-question = closed-by-canonical-skeleton
canonical-question-class morphism-choice-question = closed-by-canonical-skeleton
canonical-question-class product-choice-question = closed-by-canonical-skeleton
canonical-question-class pullback-choice-question = closed-by-canonical-skeleton
canonical-question-class classifier-choice-question = closed-by-canonical-skeleton
canonical-question-class exponential-choice-question = closed-by-canonical-skeleton
canonical-question-class raw-topos-question = blocked-at-raw-stage
canonical-question-class set-like-question = set-like-burden-class
canonical-question-class formulation-position-question = formulability-extraction-class
canonical-question-class semantic-adequacy-question = semantic-adequacy-burden-class

-- § Exhaustiveness: every listed residual question is classified.
canonical-question-classified :
  (question : CanonicalResidualQuestion) →
    ClassifiedCanonicalQuestion question (canonical-question-class question)
canonical-question-classified distinction-presentation-question =
  distinction-presentation-classified
canonical-question-classified reading-presentation-question =
  reading-presentation-classified
canonical-question-classified object-choice-question = object-choice-classified
canonical-question-classified morphism-choice-question = morphism-choice-classified
canonical-question-classified product-choice-question = product-choice-classified
canonical-question-classified pullback-choice-question = pullback-choice-classified
canonical-question-classified classifier-choice-question = classifier-choice-classified
canonical-question-classified exponential-choice-question = exponential-choice-classified
canonical-question-classified raw-topos-question = raw-topos-classified
canonical-question-classified set-like-question = set-like-classified
canonical-question-classified formulation-position-question = formulation-position-classified
canonical-question-classified semantic-adequacy-question = semantic-adequacy-classified

-- § There is no constructor for an unclassified canonical question.
data UnclassifiedCanonicalQuestion : Set where

-- § Therefore an alleged unclassified canonical question is impossible.
no-unclassified-canonical-question : UnclassifiedCanonicalQuestion → ⊥
no-unclassified-canonical-question ()

-- § Package of the finite closure ledger.
record CanonicalClosureLedger : Set1 where
  field
    ccl-class      : CanonicalResidualQuestion → CanonicalQuestionClass
    ccl-classified :
      (question : CanonicalResidualQuestion) →
        ClassifiedCanonicalQuestion question (ccl-class question)

```

```

    ccl-no-unclassified : UnclassifiedCanonicalQuestion → ⊥

open CanonicalClosureLedger public

-- § The checked closure ledger classifies every listed residual question.
canonical-closure-ledger : CanonicalClosureLedger
canonical-closure-ledger = record
{ ccl-class      = canonical-question-class
; ccl-classified = canonical-question-classified
; ccl-no-unclassified = no-unclassified-canonical-question
}

-- § Canonical closure: endpoint closed, remaining questions classified.
record CanonicalClosure : Setω where
field
    cc-endpoint      : CanonicalClosedEndpoint
    cc-ledger         : CanonicalClosureLedger
    cc-set-like-burden : SetLikeCompletionBurden
    cc-formulability-burden : FormulabilityBurden

open CanonicalClosure public

-- § The canonical closure of the checked route.
canonical-closure : CanonicalClosure
canonical-closure = record
{ cc-endpoint      = canonical-closed-endpoint
; cc-ledger         = canonical-closure-ledger
; cc-set-like-burden = set-like-completion-burden
; cc-formulability-burden = formulability-burden
}

```

Route And Foundation Classification

The final classification is exhaustive only after admission has been made explicit. A route completion claim is admitted when it is one of the displayed cases below: the canonical skeleton, a raw attempt to supply global route pullback cones, a set-like candidate, or a semantic-adequacy claim. A foundation claim is admitted when it either supplies nontrivial formulation positions or appears as one of those route completions.

```

-- § Finite admission interface for completion claims handled by this route.
data RouteCompletionExtension : Setω where
    route-canonical-skeleton : RouteCompletionExtension
    route-raw-pullback-cones-attempt :
        ToposRouteGlobalPullbackCones → RouteCompletionExtension
    route-set-like-candidate :
        SetLikeCompletionCandidate → RouteCompletionExtension
    route-semantic-adequacy :
        StableReadingSemanticAdequacy → RouteCompletionExtension

```

```

-- § Classification classes for the route-admitted completion interface.
data RouteCompletionClass : Set where
  route-canonical-skeleton-class      : RouteCompletionClass
  route-raw-topos-blocked-class       : RouteCompletionClass
  route-set-like-burden-class         : RouteCompletionClass
  route-semantic-adequacy-burden-class : RouteCompletionClass

-- § Evidence assigned to each classified route-admitted completion claim.
data ClassifiedRouteCompletion :
  RouteCompletionExtension → RouteCompletionClass → Setω where
  route-canonical-skeleton-classified :
    CanonicalClosure →
    ClassifiedRouteCompletion route-canonical-skeleton route-canonical-skeleton-class
  route-raw-topos-blocked-classified :
    (global-cones : ToposRouteGlobalPullbackCones) →
    (ToposRouteGlobalPullbackCones → ⊥) →
    ClassifiedRouteCompletion
    (route-raw-pullback-cones-attempt global-cones)
    route-raw-topos-blocked-class
  route-set-like-candidate-classified :
    (candidate : SetLikeCompletionCandidate) →
    EmbeddedBinaryDistinction
    (formulation-positions
     (category-formulation-interface
      (slc-category candidate)
      (slc-object-diversity candidate))) →
    ClassifiedRouteCompletion
    (route-set-like-candidate candidate)
    route-set-like-burden-class
  route-semantic-adequacy-classified :
    (adequacy : StableReadingSemanticAdequacy) →
    ClassifiedRouteCompletion
    (route-semantic-adequacy adequacy)
    route-semantic-adequacy-burden-class

-- § Classifier for the route-admitted completion interface.
route-completion-class : RouteCompletionExtension → RouteCompletionClass
route-completion-class route-canonical-skeleton = route-canonical-skeleton-class
route-completion-class (route-raw-pullback-cones-attempt global-cones) =
  route-raw-topos-blocked-class
route-completion-class (route-set-like-candidate candidate) =
  route-set-like-burden-class
route-completion-class (route-semantic-adequacy adequacy) =
  route-semantic-adequacy-burden-class

-- § The raw full-pullback-cone attempt cannot be inhabited on this route.
route-raw-pullback-cones-attempt-impossible :
  ToposRouteGlobalPullbackCones → ⊥
route-raw-pullback-cones-attempt-impossible =
  topos-route-global-pullback-cones-impossible

-- § Exhaustiveness: every route-admitted completion claim is classified.
route-completion-classified :
  (extension : RouteCompletionExtension) →

```



```

ClassifiedRouteCompletion extension (route-completion-class extension)
route-completion-classified route-canonical-skeleton =
  route-canonical-skeleton-classified canonical-closure
route-completion-classified (route-raw-pullback-cones-attempt global-cones) =
  route-raw-topos-blocked-classified
    global-cones
    route-raw-pullback-cones-attempt-impossible
route-completion-classified (route-set-like-candidate candidate) =
  route-set-like-candidate-classified
    candidate
    (set-like-candidate-presupposes-distinction candidate)
route-completion-classified (route-semantic-adequacy adequacy) =
  route-semantic-adequacy-classified adequacy

-- § Package of the exhaustive route-admitted completion classification.
record ExhaustiveRouteCompletionClassification : Set $\omega$  where
  field
    ercc-class : RouteCompletionExtension → RouteCompletionClass
    ercc-classified :
      (extension : RouteCompletionExtension) →
        ClassifiedRouteCompletion extension (ercc-class extension)
    ercc-raw-attempt-impossible : ToposRouteGlobalPullbackCones →  $\perp$ 
    ercc-set-like-to-distinction :
      (candidate : SetLikeCompletionCandidate) →
        EmbeddedBinaryDistinction
        (formulation-positions
          (category-formulation-interface
            (slc-category candidate)
            (slc-object-diversity candidate)))
    ercc-canonical-closure : CanonicalClosure

open ExhaustiveRouteCompletionClassification public

-- § The final classification theorem for completions admitted by this route.
route-completion-classification : ExhaustiveRouteCompletionClassification
route-completion-classification = record
  { ercc-class          = route-completion-class
  ; ercc-classified     = route-completion-classified
  ; ercc-raw-attempt-impossible = route-raw-pullback-cones-attempt-impossible
  ; ercc-set-like-to-distinction = set-like-candidate-presupposes-distinction
  ; ercc-canonical-closure = canonical-closure
  }

-- § A nontrivial formulable foundation supplies formulation positions.
record FormulableFoundationCandidate : Set1 where
  field
    ffc-interface : NontrivialFormulationInterface

open FormulableFoundationCandidate public

-- § Admitted foundation claims: entry-level foundations or route completions.
data AdmittedFoundationClaim : Set $\omega$  where
  admitted-formulable-foundation :
    FormulableFoundationCandidate → AdmittedFoundationClaim

```

```

admitted-route-completion :
  RouteCompletionExtension → AdmittedFoundationClaim

-- § Classification classes for admitted foundation claims.
data AdmittedFoundationClass : Set where
  foundation-distinctional-entry-class : AdmittedFoundationClass
  foundation-canonical-skeleton-class : AdmittedFoundationClass
  foundation-raw-topos-blocked-class : AdmittedFoundationClass
  foundation-set-like-burden-class : AdmittedFoundationClass
  foundation-semantic-adequacy-burden-class : AdmittedFoundationClass

-- § Route-completion classes become foundation classes at completion level.
foundation-route-class : RouteCompletionClass → AdmittedFoundationClass
foundation-route-class route-canonical-skeleton-class =
  foundation-canonical-skeleton-class
foundation-route-class route-raw-topos-blocked-class =
  foundation-raw-topos-blocked-class
foundation-route-class route-set-like-burden-class =
  foundation-set-like-burden-class
foundation-route-class route-semantic-adequacy-burden-class =
  foundation-semantic-adequacy-burden-class

-- § Evidence that an admitted foundation claim has its assigned class.
data ClassifiedAdmittedFoundation :
  AdmittedFoundationClaim → AdmittedFoundationClass → Setω where
  admitted-formulable-foundation-classified :
    (foundation : FormulableFoundationCandidate) →
    EmbeddedBinaryDistinction
    (formulation-positions (ffc-interface foundation)) →
    ClassifiedAdmittedFoundation
    (admitted-formulable-foundation foundation)
    foundation-distinctional-entry-class
  admitted-route-completion-classified :
    (extension : RouteCompletionExtension) →
    ClassifiedRouteCompletion extension (route-completion-class extension) →
    ClassifiedAdmittedFoundation
    (admitted-route-completion extension)
    (foundation-route-class (route-completion-class extension))

-- § Classifier for admitted foundation claims.
admitted-foundation-class : AdmittedFoundationClaim → AdmittedFoundationClass
admitted-foundation-class (admitted-formulable-foundation foundation) =
  foundation-distinctional-entry-class
admitted-foundation-class (admitted-route-completion extension) =
  foundation-route-class (route-completion-class extension)

-- § Every admitted nontrivial formulable foundation carries distinction.
formulable-foundation-presupposes-distinction :
  (foundation : FormulableFoundationCandidate) →
  EmbeddedBinaryDistinction
  (formulation-positions (ffc-interface foundation))
formulable-foundation-presupposes-distinction foundation =
  formulation-interface-presupposes-distinction (ffc-interface foundation)

```

```

-- § Exhaustiveness: every admitted foundation claim is classified.
admitted-foundation-classified :
  (claim : AdmittedFoundationClaim) →
  ClassifiedAdmittedFoundation claim (admitted-foundation-class claim)
admitted-foundation-classified (admitted-formulable-foundation foundation) =
  admitted-formulable-foundation-classified
    foundation
    (formulable-foundation-presupposes-distinction foundation)
admitted-foundation-classified (admitted-route-completion extension) =
  admitted-route-completion-classified
    extension
    (route-completion-classified extension)

-- § Package of the exhaustive admitted-foundation classification.
record ExhaustiveAdmittedFoundationClassification : Setω where
  field
    eafc-class : AdmittedFoundationClaim → AdmittedFoundationClass
    eafc-classified :
      (claim : AdmittedFoundationClaim) →
      ClassifiedAdmittedFoundation claim (eafc-class claim)
    eafc-formulable-foundation-to-distinction :
      (foundation : FormulableFoundationCandidate) →
      EmbeddedBinaryDistinction
      (formulation-positions (ffc-interface foundation))
    eafc-route-completion-classification :
      ExhaustiveRouteCompletionClassification

open ExhaustiveAdmittedFoundationClassification public

-- § The admitted-foundation classification theorem.
admitted-foundation-classification : ExhaustiveAdmittedFoundationClassification
admitted-foundation-classification = record
{ eafc-class = admitted-foundation-class
; eafc-classified = admitted-foundation-classified
; eafc-formulable-foundation-to-distinction =
  formulable-foundation-presupposes-distinction
; eafc-route-completion-classification = route-completion-classification
}

-- § Final theorem: raw obstruction, completed Topos, burdens, and classifications.
record ToposRouteFinalTheorem : Setω where
  field
    vtf-boundary-map      : ToposBoundaryMap
    vtf-route-entry       : (d : Distinction) → ToposRouteEntry d
    vtf-raw-obstruction    : ToposRouteRawObstruction
    vtf-raw-cones-blocked : ToposRouteGlobalPullbackCones → ⊥
    vtf-scope-conditions  : ToposRouteScopeConditions
    vtf-completion-stage  : CompletionToposStage
    vtf-skeleton-completion : VoidSkeletonToposCompletion
    vtf-completed-topos   : ElementaryTopos (vst-category vtf-skeleton-completion)
    vtf-set-like-burden    : SetLikeCompletionBurden
    vtf-formulability-burden : FormulabilityBurden
    vtf-free-well-pointed-topos-with-nno : WellPointedToposWithNNOCCompletion
    vtf-free-completion-initial :

```

```

InitialCompletion vtf-free-well-pointed-topos-with-nno
vtf-free-completion-unique-up-to-equivalence :
  (other : WellPointedToposWithNNOCompletion) →
  InitialCompletion other →
  CompletionEquivalence vtf-free-well-pointed-topos-with-nno other
vtf-classifying-topos-of-reading-skeleton-models :
  ClassifyingToposOfReadingSkeletonModels
vtf-canonical-closure : CanonicalClosure
vtf-route-completion-classification :
  ExhaustiveRouteCompletionClassification
vtf-admitted-foundation-classification :
  ExhaustiveAdmittedFoundationClassification

open ToposRouteFinalTheorem public

-- § The final witness packages the checked theorem and burden records.
void-topos-final-theorem : ToposRouteFinalTheorem
void-topos-final-theorem = record
{ vtf-boundary-map      = topos-boundary-map
; vtf-route-entry       = topos-route-entry
; vtf-raw-obstruction   = topos-route-raw-obstruction
; vtf-raw-cones-blocked = topos-route-global-pullback-cones-impossible
; vtf-scope-conditions  = void-topos-scope-conditions
; vtf-completion-stage  = completion-topos-stage
; vtf-skeleton-completion = void-skeleton-topos-completion
; vtf-completed-topos   = skeleton-elementary-topos
; vtf-set-like-burden    = set-like-completion-burden
; vtf-formulability-burden = formulability-burden
; vtf-free-well-pointed-topos-with-nno = free-well-pointed-topos-with-nno
; vtf-free-completion-initial = free-completion-initial
; vtf-free-completion-unique-up-to-equivalence =
  free-completion-unique-up-to-equivalence
; vtf-classifying-topos-of-reading-skeleton-models =
  free-classifying-topos-of-reading-skeleton-models
; vtf-canonical-closure = canonical-closure
; vtf-route-completion-classification = route-completion-classification
; vtf-admitted-foundation-classification = admitted-foundation-classification
}

```

The result is now fully internal to the book. The Topos route starts at the book's stable-reading interface, adds the boundary marks needed for the raw obstruction, proves that the split-truth cospan blocks global raw pullback cones, constructs the completed skeleton as an elementary Topos, constructs the free nontrivial set-like well-pointed elementary Topos with NNO over the generated reading skeleton, eliminates the remaining displayed model choices by classifying all models over that generated skeleton, and classifies the admitted completion and foundation claims of this route.

This closes the Topos route boundary. The route does not begin from a second theory of classifiers. It begins from the book's stable-reading surface, adds only the boundary marks needed to state the obstruction, passes through the explicit skele-

ton completion, adds the free well-pointed syntactic completion with its initiality and uniqueness up to equivalence, makes it the classifying Topos for the displayed models over the generated reading skeleton, and ends with a finite classification of the admitted completion and foundation claims.

The next step is to examine what happens when the same classification process is applied one level higher. This leads to the hierarchy called *drift*.

Part II

Drift and Reachability

Part I analysed a single distinction and the finite classification data attached to it. Part II turns to the hierarchy obtained by iterating that analysis. Once classification data are themselves treated as mathematical objects, the development moves through higher universe levels in a controlled way.

This is the role of drift. The underlying idea is straightforward: once a structure has been introduced and classified at one level, the resulting data becomes the object of further classification at the next. The next chapters make that transition explicit and study the linear hierarchy it generates.

Chapter 14

Drift

Drift is the basic operation that moves a distinction one universe level upward. It is not introduced as additional structure; it is the lift required once the development begins to quantify over classification data themselves.

The construction uses `Lift` on the carrier and transports the two boundary points and the cover along with it. Later results show that this lift is universal among boundary-preserving lifts of the same kind.

```
-- § Level-polymorphic distinction record.
record Distinctionℓ (ℓ : Level) : Set (Isuc ℓ) where
  field
    S : Set ℓ
    ℓ₀ : S
    r₀ : S
    ℓ₀ ≠ r₀ : ℓ₀ ≠ r₀
    cover : (x : S) → (x ≡ ℓ₀) ⊔ (x ≡ r₀)

open Distinctionℓ public

-- § Canonical embedding from base-level Distinction.
Distinction → Distinctionℓ : Distinction → Distinctionℓ lzero
Distinction → Distinctionℓ d = record
  { S = S d
  ; ℓ₀ = ℓ d
  ; r₀ = r d
  ; ℓ₀ ≠ r₀ = ℓ ≠ r d
  ; cover = cover d
  }

-- § A drift step: the next-level distinction with an embedding.
record DriftStep {ℓ : Level} (d : Distinctionℓ ℓ) : Set (Isuc (Isuc ℓ)) where
  field
    d↑ : Distinctionℓ (Isuc ℓ)
    embed : S d → S d↑

open DriftStep public
```

```

-- § Drift is the canonical lift to the next universe level.
drift : {ℓ : Level} → (d : Distinctionℓ ℓ) → DriftStep d
drift d = record
{ d↑ = record
  { S = Lift (S d)
  ; ℓ₀ = lift (ℓ₀ d)
  ; r₀ = lift (r₀ d)
  ; ℓ₀ ≠ r₀ = λ eq → ℓ₀ ≠ r₀ d (lift-injective eq)
  ; cover = cover↑
  }
; embed = lift
}
where
cover↑ : (y : Lift (S d)) → (y ≡ lift (ℓ₀ d)) ∨ (y ≡ lift (r₀ d))
cover↑ (lift x) with cover d x
... | inj₁ x ≡ ℓ = inj₁ (cong lift x ≡ ℓ)
... | inj₂ x ≡ r = inj₂ (cong lift x ≡ r)

```

Law 2.1: No Classification May Remain Unclassified

This law records the drift construction as the formal expression of closure under re-classification.

```

-- § Law 2.1: No classification may remain unclassified.
law2-1-drift : {ℓ : Level} → (d : Distinctionℓ ℓ) → DriftStep d
law2-1-drift = drift

-- § Extract the next-level distinction from a drift step.
drift-next : {ℓ : Level} → (d : Distinctionℓ ℓ) → Distinctionℓ (lsuc ℓ)
drift-next d = d↑ (drift d)

-- § Extract the embedding from a drift step.
drift-embed : {ℓ : Level} → (d : Distinctionℓ ℓ) → S d → S (drift-next d)
drift-embed d = embed (drift d)

-- § Drift-embedded elements satisfy coverage.
drift-embed-cover : {ℓ : Level} → (d : Distinctionℓ ℓ) → (x : S d)
→ (drift-embed d x ≡ ℓ₀ (drift-next d)) ∨ (drift-embed d x ≡ r₀ (drift-next d))
drift-embed-cover d x = cover (drift-next d) (drift-embed d x)

```

The point is that drift preserves the full two-class structure. The lifted distinction still has distinguished boundary points, a cover, and an embedding of the original carrier.

Chapter 15

Non-Collapse of Drift

Since drift is implemented by `Lift`, the relevant question is whether it preserves distinctions rather than collapsing them. This chapter records the answer formally. The key point is that `lift` is injective, so different elements of the original carrier remain different after embedding.

Law 3.1: Drift Does Not Fold The Prior Carrier

The proof is immediate from injectivity of `lift`. If two embedded elements were equal, their preimages would already have been equal in the original carrier. In particular, the two distinguished boundary points remain distinct after embedding.

```
-- § Law 3.1: Drift does not fold the prior carrier.
law3-1-embed-injective :
  {ℓ : Level} (d : Distinction ℓ) → {x y : S d} →
    drift-embed d x ≡ drift-embed d y → x ≡ y
law3-1-embed-injective d = lift-injective

-- § Drift preserves boundary distinctness.
drift-embed-ℓ₀≠r₀ :
  {ℓ : Level} (d : Distinction ℓ) →
    drift-embed d (ℓ₀ d) ≠ drift-embed d (r₀ d)
drift-embed-ℓ₀≠r₀ d eq = ℓ₀≠r₀ d (law3-1-embed-injective d eq)
```

With non-collapse in place, the next question is whether drift is merely one possible lift or whether it has a distinguished universal role among all boundary-preserving lifts.

Chapter 16

Cross-Stage Comparison

Drift is not only a lift; the next theorem proves its canonical role. To formulate this precisely, the text compares drift with arbitrary boundary-preserving lifts of the same distinction.

The result is a familiar universal property. Every such lift receives a mediating map from the drift construction, and that map commutes with the given embeddings. The uniqueness proof below then closes the universal statement. The surrounding definitions package this result in a form that can later be read categorically.

```
-- § Level-polymorphic eliminator for Distinctionℓ.
Distinctionℓ-elim :
  {ℓ ℓP : Level} → (d : Distinctionℓ ℓ) →
  {P : S d → Set ℓP} →
  P (ℓ₀ d) →
  P (r₀ d) →
  (x : S d) →
  P x
Distinctionℓ-elim d {P} pℓ pr x with cover d x
... | inj₁ x≡ℓ = subst P (sym x≡ℓ) pℓ
... | inj₂ x≡r = subst P (sym x≡r) pr

-- § Functions out of Distinctionℓ are determined by boundary values.
Distinctionℓ-determined :
  {ℓ ℓB : Level} → (d : Distinctionℓ ℓ) → {B : Set ℓB} →
  (f g : S d → B) →
  f (ℓ₀ d) ≡ g (ℓ₀ d) →
  f (r₀ d) ≡ g (r₀ d) →
  f ≡ g
Distinctionℓ-determined d f g fℓ≡gℓ fr≡gr =
  Distinctionℓ-elim d
  (subst (λ y → f y ≡ g y) refl fℓ≡gℓ)
  (subst (λ y → f y ≡ g y) refl fr≡gr)
```

Tier 2 Normal Form

The base-level normal-form theorem of Chapter 12 stated that every `Distinction` is isomorphic to `Two`. Once classification data themselves are lifted into higher universes, the same statement must hold there too, or the hierarchy would generate distinctions that escape the very normal form on which the base-level result rests. The structural force is uniformity: a distinction at level ℓ that admitted a wider classification than at level 0 would let the hierarchy add information by mere relabelling, and the entire drift construction would lose its meaning.

The proof of `two-normal-form` consumed nothing about its carrier beyond the four fields of a `Distinction`. Those fields exist verbatim in `Distinction $\ell$` . Any normal-form claim that held at level 0 can therefore be repeated at level ℓ by the same construction.

The code introduces a level-polymorphic two-element type `Two $\ell$` , a level-polymorphic distinction `Two-distinction $\ell$` , and a level-polymorphic boundary-preserving isomorphism record `DistinctionIso $\ell$` . The maps `toTwo $\ell$` , `fromTwo $\ell$` , and the round-trip identities mirror their base-level counterparts. The theorem `two-normal-form $\ell$` then exhibits the isomorphism uniformly for each level ℓ .

Thus the normal-form result is closed under the universe hierarchy. The base case is no longer an isolated theorem at level 0; it is the level-0 instance of a statement that has the same construction at every level. Tier 1 (level 0) was already proved, and Tier 2 (each level ℓ) is proved here. The remaining outer boundary – a single quantifier ranging over the entire universe hierarchy at once – is not an artefact of this development but the Girard limit of any consistent type theory; no internal proof can cross it.

```
-- § Level-polymorphic two-element type.
data Two $\ell$  { $\ell$  : Level} : Set  $\ell$  where
  L $\ell$  : Two $\ell$ 
  R $\ell$  : Two $\ell$ 

-- § L $\ell$  and R $\ell$  are distinct at every level.
Two $\ell$ -L $\neq$ R : { $\ell$  : Level}  $\rightarrow$  L $\ell$  { $\ell$ }  $\neq$  R $\ell$  { $\ell$ }
Two $\ell$ -L $\neq$ R ()

-- § Exhaustive two-class coverage for Two $\ell$ .
Two $\ell$ -cover : { $\ell$  : Level}  $\rightarrow$  (x : Two $\ell$  { $\ell$ })  $\rightarrow$  (x  $\equiv$  L $\ell$ )  $\sqcup$  (x  $\equiv$  R $\ell$ )
Two $\ell$ -cover L $\ell$  = inj1 refl
Two $\ell$ -cover R $\ell$  = inj2 refl

-- § Two $\ell$  as a level-polymorphic Distinction.
Two-distinction $\ell$  : ( $\ell$  : Level)  $\rightarrow$  Distinction $\ell$   $\ell$ 
Two-distinction $\ell$   $\ell$  = record
{ S = Two $\ell$ 
;  $\ell_0$  = L $\ell$ 
;  $r_0$  = R $\ell$ 
```



```

;  $\ell_0 \neq r_0 = \text{Two}\ell\text{-L}\neq\text{R}$ 
;  $\text{cover} = \text{Two}\ell\text{-cover}$ 
}

-- § Boundary-preserving isomorphism between distinctions at level  $\ell$ .
record DistinctionIsoℓ {ℓ : Level} (d₁ d₂ : Distinctionℓ ℓ) : Set ℓ where
  field
    to    : S d₁ → S d₂
    from  : S d₂ → S d₁
    to-from : (y : S d₂) → to (from y) ≡ y
    from-to : (x : S d₁) → from (to x) ≡ x
    to-ℓ₀ : to (ℓ₀ d₁) ≡ ℓ₀ d₂
    to-r₀ : to (r₀ d₁) ≡ r₀ d₂

open DistinctionIsoℓ public

-- § The canonical boundary-preserving map to Twoℓ.
toTwoℓ : {ℓ : Level} (d : Distinctionℓ ℓ) → S d → Twoℓ {ℓ}
toTwoℓ d x with cover d x
... | inj₁ _ = Lℓ
... | inj₂ _ = Rℓ

-- § The canonical embedding from Twoℓ.
fromTwoℓ : {ℓ : Level} (d : Distinctionℓ ℓ) → Twoℓ {ℓ} → S d
fromTwoℓ d Lℓ = ℓ₀ d
fromTwoℓ d Rℓ = r₀ d

-- § toTwoℓ sends ℓ₀ to Lℓ.
toTwoℓ-ℓ₀ : {ℓ : Level} (d : Distinctionℓ ℓ) → toTwoℓ d (ℓ₀ d) ≡ Lℓ
toTwoℓ-ℓ₀ d with cover d (ℓ₀ d)
... | inj₁ _ = refl
... | inj₂ ℓ≡r = ⊥-elim (ℓ₀ ≠ r₀ d ℓ≡r)

-- § toTwoℓ sends r₀ to Rℓ.
toTwoℓ-r₀ : {ℓ : Level} (d : Distinctionℓ ℓ) → toTwoℓ d (r₀ d) ≡ Rℓ
toTwoℓ-r₀ d with cover d (r₀ d)
... | inj₂ _ = refl
... | inj₁ r≡ℓ = ⊥-elim (ℓ₀ ≠ r₀ d (sym r≡ℓ))

-- § fromTwoℓ ∘ toTwoℓ is the identity.
fromTwoℓ-toTwoℓ : {ℓ : Level} (d : Distinctionℓ ℓ) → (x : S d) → fromTwoℓ d (toTwoℓ d x) ≡ x
fromTwoℓ-toTwoℓ d x with cover d x
... | inj₁ x≡ℓ = sym x≡ℓ
... | inj₂ x≡r = sym x≡r

-- § toTwoℓ ∘ fromTwoℓ is the identity.
toTwoℓ-fromTwoℓ : {ℓ : Level} (d : Distinctionℓ ℓ) → (t : Twoℓ {ℓ}) → toTwoℓ d (fromTwoℓ d t) ≡ t
toTwoℓ-fromTwoℓ d Lℓ = toTwoℓ-ℓ₀ d
toTwoℓ-fromTwoℓ d Rℓ = toTwoℓ-r₀ d

-- § Tier 2 normal form: every level-polymorphic distinction is isomorphic to Twoℓ.
two-normal-formℓ : {ℓ : Level} (d : Distinctionℓ ℓ) → DistinctionIsoℓ d (Two-distinctionℓ ℓ)
two-normal-formℓ d = record
  { to    = toTwoℓ d

```

```

; from = fromTwoℓ d
; to-from = toTwoℓ-fromTwoℓ d
; from-to = fromTwoℓ-toTwoℓ d
; to-ℓ₀ = toTwoℓ-ℓ₀ d
; to-r₀ = toTwoℓ-r₀ d
}

-- § A boundary-preserving lift: target distinction + embedding + boundary proofs.
record LiftedBP {ℓ : Level} (d : Distinctionℓ ℓ) : Set (Isuc (Isuc ℓ)) where
field
  e      : Distinctionℓ (Isuc ℓ)
  embed  : S d → S e
  embed-ℓ₀ : embed (ℓ₀ d) ≡ ℓ₀ e
  embed-r₀ : embed (r₀ d) ≡ r₀ e

open LiftedBP public

-- § Drift universality: drift is initial among boundary-preserving lifts.
record DriftUniversal : Setω where
field
  preserves-ℓ₀ : {ℓ : Level} (d : Distinctionℓ ℓ) →
    drift-embed d (ℓ₀ d) ≡ ℓ₀ (drift-next d)
  preserves-r₀ : {ℓ : Level} (d : Distinctionℓ ℓ) →
    drift-embed d (r₀ d) ≡ r₀ (drift-next d)

  mediator : {ℓ : Level} (d : Distinctionℓ ℓ) → (t : LiftedBP d) →
    S (drift-next d) → S (e t)

  mediator-commutes : {ℓ : Level} (d : Distinctionℓ ℓ) → (t : LiftedBP d) →
    (x : S d) → mediator d t (drift-embed d x) ≡ embed t x

open DriftUniversal public

-- § The canonical drift universality witness.
driftUniversal : DriftUniversal
driftUniversal = record
{ preserves-ℓ₀ = λ d → refl
; preserves-r₀ = λ d → refl
; mediator = λ d t y → embed t (lower y)
; mediator-commutes = λ d t x → refl
}

```

Law 4.1: Drift Is Universal Among Boundary-Preserving Lifts

The mediator is defined by applying the target embedding after lowering out of `Lift`. Its commutativity with the original embedding is definitional, and uniqueness follows from boundary determination at the lifted level.

```

-- § Law 4.1: Drift is universal among boundary-preserving lifts.
law4-1-mediator-commutes :
  {ℓ : Level} (d : Distinctionℓ ℓ) → (t : LiftedBP d) →

```

```

(x : S d) → mediator driftUniversal d t (drift-embed d x) ≡ embed t x
law4-1-mediator-commutes d t x = mediator-commutes driftUniversal d t x

-- § Mediator uniqueness: any factorizing map agrees with the canonical one.
mediator-unique :
{ℓ : Level} (d : Distinctionℓ ℓ) → (t : LiftedBP d) →
(g : S (drift-next d) → S (e t)) →
((x : S d) → g (drift-embed d x) ≡ embed t x) →
g ≐ mediator driftUniversal d t
mediator-unique d t g g-comm =
Distinctionℓ-determined (drift-next d) g h gℓ≡hℓ gr≡hr
where
h : S (drift-next d) → S (e t)
h = mediator driftUniversal d t

gℓ≡hℓ : g (ℓ₀ (drift-next d)) ≡ h (ℓ₀ (drift-next d))
gℓ≡hℓ =
trans (cong g (sym (preserves-ℓ₀ driftUniversal d)))
(trans (g-comm (ℓ₀ d))
(trans (sym (mediator-commutes driftUniversal d t (ℓ₀ d)))
(cong h (preserves-ℓ₀ driftUniversal d))))

gr≡hr : g (r₀ (drift-next d)) ≡ h (r₀ (drift-next d))
gr≡hr =
trans (cong g (sym (preserves-r₀ driftUniversal d)))
(trans (g-comm (r₀ d))
(trans (sym (mediator-commutes driftUniversal d t (r₀ d)))
(cong h (preserves-r₀ driftUniversal d))))

-- § Drift as a LiftedBP instance.
driftLiftedBP : {ℓ : Level} (d : Distinctionℓ ℓ) → LiftedBP d
driftLiftedBP d = record
{ e      = drift-next d
; embed  = drift-embed d
; embed-ℓ₀ = preserves-ℓ₀ driftUniversal d
; embed-r₀ = preserves-r₀ driftUniversal d
}

-- § Morphism between lifted boundary-preserving presentations.
record LiftMorph {ℓ : Level} (d : Distinctionℓ ℓ) (t u : LiftedBP d) : Set (Isuc (Isuc ℓ)) where
field
map : S (e t) → S (e u)
comm : (x : S d) → map (embed t x) ≡ embed u x

open LiftMorph public

-- § The canonical factorization through drift.
drift-factor :
{ℓ : Level} (d : Distinctionℓ ℓ) → (t : LiftedBP d) →
LiftMorph d (driftLiftedBP d) t
drift-factor d t = record
{ map = mediator driftUniversal d t
; comm = mediator-commutes driftUniversal d t
}

```

```

-- § Factorization through drift is unique.
drift-factor-unique :
{ℓ : Level} (d : Distinctionℓ ℓ) → (t : LiftedBP d) →
(m : LiftMorph d (driftLiftedBP d) t) →
map m ≐ mediator driftUniversal d t
drift-factor-unique d t m =
mediator-unique d t (map m) (comm m)

```

Law 4.2: Drift-Step Factorization Is Unique

This is the corresponding uniqueness statement for factorizations through the drift step. Once a lift factors through drift and commutes with the boundary embedding, boundary determination leaves no second factorizing map.

```

-- § Law 4.2: Drift-step factorization is unique.
law4-2-factor-unique :
{ℓ : Level} (d : Distinctionℓ ℓ) → (t : LiftedBP d) →
(m : LiftMorph d (driftLiftedBP d) t) →
map m ≐ mediator driftUniversal d t
law4-2-factor-unique = drift-factor-unique

```

Law 4.3: The Category of Boundary-Preserving Lifts

Once the universal property has been stated, boundary-preserving lifts and their commuting maps can be organized into a category. The definitions below provide identities and composition, and the previous universal property then becomes the statement that drift is an initial object in this category.

```

-- § Identity morphism on a lifted boundary-preserving presentation.
liftId : {ℓ : Level} (d : Distinctionℓ ℓ) → (t : LiftedBP d) → LiftMorph d t t
liftId d t = record
{ map = λ y → y
; comm = λ x → refl
}

-- § Composition of LiftMorphs.
liftComp :
{ℓ : Level} (d : Distinctionℓ ℓ) → {t u v : LiftedBP d} →
LiftMorph d u v → LiftMorph d t u → LiftMorph d t v
liftComp d {t} {u} {v} g f = record
{ map = λ y → map g (map f y)
; comm = λ x → trans (cong (map g) (comm f x)) (comm g x)
}

-- § Left identity law (pointwise).
liftId-left :
{ℓ : Level} (d : Distinctionℓ ℓ) → {t u : LiftedBP d} →
(f : LiftMorph d t u) →

```

```

(y : S (e t)) → map (liftComp d (liftId d u) f) y ≡ map f y
liftId-left d f y = refl

-- § Right identity law (pointwise).
liftId-right :
{ℓ : Level} (d : Distinction ℓ) → {t u : LiftedBP d} →
(f : LiftMorph d t u) →
(y : S (e t)) → map (liftComp d f (liftId d t)) y ≡ map f y
liftId-right d f y = refl

-- § Associativity (pointwise).
liftComp-assoc :
{ℓ : Level} (d : Distinction ℓ) → {t u v w : LiftedBP d} →
(h : LiftMorph d v w) (g : LiftMorph d u v) (f : LiftMorph d t u) →
(y : S (e t)) →
map (liftComp d h (liftComp d g f)) y
≡
map (liftComp d (liftComp d h g) f) y
liftComp-assoc d h g f y = refl

-- § Initiality of drift in the category of boundary-preserving lifts.
record DriftInitial {ℓ : Level} (d : Distinction ℓ) : Set (Isuc (Isuc ℓ)) where
field
  initial-mediator : (t : LiftedBP d) → LiftMorph d (driftLiftedBP d) t
  initial-unique : (t : LiftedBP d) → (m : LiftMorph d (driftLiftedBP d) t) →
    (y : S (e (driftLiftedBP d))) →
    map m y ≡ map (initial-mediator t) y

-- § Law 4.3: Drift is the initial object.
law4-3-drift-initial :
{ℓ : Level} (d : Distinction ℓ) → DriftInitial d
law4-3-drift-initial d = record
{ initial-mediator = drift-factor d
; initial-unique = λ t m y → drift-factor-unique d t m y
}

```

This completes the universal characterization of drift. The construction is no longer merely a chosen lift: in the category just displayed, it is the initial boundary-preserving lift. The next step is to iterate it and study the reachability relation generated by repeated drift steps.

Chapter 17

Drift Reachability

Drift can now be iterated. A drift state packages a universe level together with the distinction living there, and `stepState` moves one stage upward. The resulting reachability relations record whether one state can be obtained from another by finitely many drift steps.

Two versions are needed: strict reachability, written Reach^+ , for one or more steps, and reflexive-transitive reachability, written Reach , when zero steps are also allowed.

```
-- § A drift state packages a universe level with its distinction.
record DriftState : Setω where
  constructor ⟨_,_⟩
  field
    ℓ : Level
    d : Distinctionℓ ℓ

open DriftState public

-- § Step a drift state forward.
stepState : DriftState → DriftState
stepState s = ⟨ lsuc (ℓ s), drift-next (d s) ⟩

-- § Extract the carrier of a drift state.
Carrier : (s : DriftState) → Set (ℓ s)
Carrier s = S (d s)

-- § The canonical embedding between consecutive carriers.
state-embed : (s : DriftState) → Carrier s → Carrier (stepState s)
state-embed s = drift-embed (d s)

-- § Strict reachability: one or more drift steps.
data Reach+ : DriftState → DriftState → Setω where
  one : {s : DriftState} → Reach+ s (stepState s)
  more : {s t : DriftState} → Reach+ (stepState s) t → Reach+ s t

-- § Reflexive-transitive reachability.
```

```

data Reach : DriftState → DriftState → Setω where
stop : {s : DriftState} → Reach s s
next : {s t : DriftState} → Reach+ s t → Reach s t

-- § Reach+ eliminator.
Reach+-elim :
{P : {s t : DriftState} → Reach+ s t → Setω} →
({s : DriftState} → P s (stepState s) one) →
({s t : DriftState} → (p : Reach+ (stepState s) t) → P (stepState s) t p → P s t (more p)) →
{s t : DriftState} → (p : Reach+ s t) → P s t p
Reach+-elim {P} base step one = base
Reach+-elim {P} base step (more p) = step p (Reach+-elim {P} base step p)

-- § Reach eliminator.
Reach-elim :
{P : {s t : DriftState} → Reach s t → Setω} →
({s : DriftState} → P s s stop) →
({s t : DriftState} → (p : Reach+ s t) → P s t (next p)) →
{s t : DriftState} → (p : Reach s t) → P s t p
Reach-elim stopCase nextCase stop = stopCase
Reach-elim stopCase nextCase (next p) = nextCase p

-- § Strict reachability is transitive.
Reach+-trans : {s t u : DriftState} → Reach+ s t → Reach+ t u → Reach+ s u
Reach+-trans one q = more q
Reach+-trans (more p) q = more (Reach+-trans p q)

-- § Strict successors are comparable.
Reach+-comparable :
{s t1 t2 : DriftState} →
Reach+ s t1 → Reach+ s t2 →
(Reach t1 t2) ⊔ω (Reach t2 t1)
Reach+-comparable one one = inj1ω stop
Reach+-comparable one (more q) = inj1ω (next q)
Reach+-comparable (more p) one = inj2ω (next p)
Reach+-comparable (more p) (more q) =
Reach+-comparable p q

-- § Reflexive-transitive reachability is transitive.
reach-trans : {s t u : DriftState} → Reach s t → Reach t u → Reach s u
reach-trans stop q = q
reach-trans (next p) stop = next p
reach-trans (next p) (next q) = next (Reach+-trans p q)

-- § Every strict chain extends by one step.
Reach+-extend : {s t : DriftState} → Reach+ s t → Reach+ s (stepState t)
Reach+-extend p = Reach+-trans p one

-- § Every reflexive chain extends by one step.
Reach-extend : {s t : DriftState} → Reach s t → Reach s (stepState t)
Reach-extend stop = next one
Reach-extend (next p) = next (Reach+-extend p)

```



```

-- § Strict reachability as an infix operator.
infix 20 _<_
_<_ : DriftState → DriftState → Setω
s < t = Reach+ s t

-- § Strict reachability is transitive (infix form).
<-trans : {s t u : DriftState} → (s < t) → (t < u) → (s < u)
<-trans p q = Reach+-trans p q

-- § Every drift state has a strict successor.
drift-progress : (s : DriftState) → s < stepState s
drift-progress s = one

-- § Terminal state: no strict successor exists.
Terminal : DriftState → Setω
Terminal s = (t : DriftState) → (s < t) → ⊥

-- § No terminal state exists (internal proof).
no-terminal : (s : DriftState) → Terminal s → ⊥
no-terminal s term = term (stepState s) (drift-progress s)

-- § Reachability induces carrier embedding (strict).
reach+-embed : {s t : DriftState} → Reach+ s t → Carrier s → Carrier t
reach+-embed {s} one x = state-embed s x
reach+-embed {s} (more p) x = reach+-embed p (state-embed s x)

-- § Reachability induces carrier embedding (reflexive).
reach-embed : {s t : DriftState} → Reach s t → Carrier s → Carrier t
reach-embed {s} stop x = x
reach-embed {s} (next p) x = reach+-embed p x

```

Law 5.0: Drift Admits No Terminal State

If a terminal state existed, then the strict successor supplied by $\text{Reach}^+.\text{one}$ would contradict terminality.

```

-- § Law 5.0: Drift admits no terminal state.
law5-0-no-terminal : (s : DriftState) → Terminal s → ⊥
law5-0-no-terminal = no-terminal

```

Law 5.1: Strict Successors Are Comparable

Any two states strictly reachable from the same source are comparable: one reaches the other. This reflects the linear structure of drift. Each state has a single immediate successor, so the reachability chain does not branch.

```

-- § Law 5.1: Strict successors are comparable.
law5-1-comparable :
  {s t1 t2 : DriftState} →
  s < t1 →
  s < t2 →
  (Reach t1 t2) ∪ω (Reach t2 t1)
law5-1-comparable = Reach+-comparable

```

Law 5.2: Every Finite Chain Extends

Every finite chain of drift steps can be extended by one more step. This is the formal expression of the fact that drift does not terminate after finitely many stages.

```
-- § Law 5.2: Every finite chain extends (strict).
law5-2-extend+ :
  {s t : DriftState} →
  s < t →
  s < stepState t
law5-2-extend+ p = Reach+-extend p

-- § Law 5.2: Every finite chain extends (reflexive).
law5-2-extend :
  {s t : DriftState} →
  Reach s t →
  Reach s (stepState t)
law5-2-extend = Reach-extend
```

Laws 5.3–5.7: Closure and Induction

The remaining structural laws are direct consequences of the inductive definitions: transitivity of strict and reflexive reachability, the carrier embedding determined by each reachability witness, and the eliminators for both Reach^+ and Reach . Together they close the reachability algebra and provide the induction principles used in all later arguments over drift chains.

```
-- § Law 5.3: Strict reachability is transitive.
law5-3-<-trans : {s t u : DriftState} → (s < t) → (t < u) → (s < u)
law5-3-<-trans = <-trans

-- § Law 5.4: Reachability is transitive.
law5-4-reach-trans : {s t u : DriftState} → Reach s t → Reach t u → Reach s u
law5-4-reach-trans = reach-trans

-- § Law 5.5: Reachability forces carrier-embedding.
law5-5-reach-embed : {s t : DriftState} → Reach s t → Carrier s → Carrier t
law5-5-reach-embed = reach-embed

-- § Law 5.6: Reach+ eliminator is forced.
law5-6-Reach+-elim :
  {P : (s t : DriftState) → Reach+ s t → Setω} →
  ({s : DriftState} → P s (stepState s) one) →
  ({s t : DriftState} → (p : Reach+ (stepState s) t) → P (stepState s) t p → P s t (more p)) →
  {s t : DriftState} → (p : Reach+ s t) → P s t p
law5-6-Reach+-elim = Reach+-elim

-- § Law 5.7: Reach eliminator is forced.
law5-7-Reach-elim :
  {P : (s t : DriftState) → Reach s t → Setω} →
  ({s : DriftState} → P s s stop) →
```

$$\begin{aligned} & (\{s\ t : \text{DriftState}\} \rightarrow (p : \text{Reach}^+ s\ t) \rightarrow P\ s\ t\ (\text{next}\ p)) \rightarrow \\ & \{s\ t : \text{DriftState}\} \rightarrow (p : \text{Reach}\ s\ t) \rightarrow P\ s\ t\ p \\ \text{law5-7-Reach-elim} &= \text{Reach-elim} \end{aligned}$$

the chain might loop back on itself. The reachability structure is now explicit: every finite chain extends, strict successors are comparable, and the relevant eliminators are in place. What remains is to exclude the residual hazard that the chain might loop back on itself.

Chapter 18

Drift Acyclicity

To keep the hierarchy meaningful, repeated drift must not fold back on itself. This chapter records acyclicity as an explicit constraint and derives its standard consequences for the strict reachability relation.

```
-- § Acyclicity constraint on drift.
record DriftAcyclic : Set $\omega$  where
  field
    no-cycle : (s : DriftState)  $\rightarrow$  (s  $\prec$  s)  $\rightarrow$   $\perp$ 

open DriftAcyclic public
```

Law 6.0: Drift Has No Cycles

the `DriftAcyclic` record. Acyclicity is recorded as a constraint: any cycle $s \prec s$ leads to absurdity. The implementation extracts that contradiction from the `DriftAcyclic` record.

```
-- § Law 6.0: Drift has no cycles.
law6-0-no-cycle : DriftAcyclic  $\rightarrow$  (s : DriftState)  $\rightarrow$  (s  $\prec$  s)  $\rightarrow$   $\perp$ 
law6-0-no-cycle ac s p = no-cycle ac s p
```

Law 6.1: Strict Reachability Is Irreflexive

Irreflexivity is the immediate instance of acyclicity: $s \prec s$ is itself a cycle of length one.

```
-- § Law 6.1: Strict reachability is irreflexive.
law6-1-irreflexive : DriftAcyclic  $\rightarrow$  (s : DriftState)  $\rightarrow$  (s  $\prec$  s)  $\rightarrow$   $\perp$ 
law6-1-irreflexive ac s = no-cycle ac s
```

Law 6.2: Strict Reachability Is Asymmetric

If both $s \prec t$ and $t \prec s$ held, transitivity would produce the cycle $s \prec s$, contradicting acyclicity.

```
-- § Law 6.2: Strict reachability is asymmetric.
law6-2-asymmetric :
  DriftAcyclic → {s t : DriftState} →
    (s < t) → (t < s) → ⊥
law6-2-asymmetric ac p q = no-cycle ac _ (←-trans p q)
```

Law 6.3: Drift-Step Has No Fixed Point

If `stepState` s equalled s , then the single-step witness one could be transported into a self-loop $s \prec s$, contradicting acyclicity.

```
-- § Law 6.3: Drift-step has no fixed point.
law6-3-no-fixpoint-stepState : DriftAcyclic → (s : DriftState) → stepState s ≡ω s → ⊥
law6-3-no-fixpoint-stepState ac s eq =
  no-cycle ac s (substω (λ u → Reach+ s u) eq one)
```

Law 6.4: Drift Is a Strict Total Order

Taken together, transitivity, irreflexivity, asymmetry, and comparability show that strict reachability behaves like a strict total order along each drift chain. The record below packages those properties for later reference.

```
-- § Strict total order on drift states
record DriftStrictTotalOrder (ac : DriftAcyclic) : Setω where
  field
    sto-trans      : {s t u : DriftState} → (s < t) → (t < u) → (s < u)
    sto-irrefl     : (s : DriftState) → (s < s) → ⊥
    sto-asym       : {s t : DriftState} → (s < t) → (t < s) → ⊥
    sto-comparable : {s t1 t2 : DriftState} →
      s < t1 → s < t2 → (Reach t1 t2) ⊔ω (Reach t2 t1)

-- § Theorem: drift is a strict total order
theorem-drift-strict-total-order :
  (ac : DriftAcyclic) → DriftStrictTotalOrder ac
theorem-drift-strict-total-order ac = record
  { sto-trans = law5-3-←-trans
  ; sto-irrefl = law6-1-irreflexive ac
  ; sto-asym = law6-2-asymmetric ac
  ; sto-comparable = law5-1-comparable
  }
```

The formal picture is now clear. Drift produces a linear hierarchy, the hierarchy never terminates after finitely many stages, and the acyclicity constraint prevents it from folding back into itself.

The next part extends the same classification ideas from endomorphisms to maps between different distinction carriers.

Part III

Heteromorphisms and Presentation

The previous parts classified self-maps of a single distinction and analysed the hierarchy generated by drift. The next step is to consider maps between different distinctions. Although the source and target carriers need not coincide, the same two-by-two boundary combinatorics remain in force.

Chapter 19

Heteromorphisms

For maps $f : S_{d_1} \rightarrow S_{d_2}$ between two possibly different distinctions, the same counting principle still applies: the map is determined by the images of the two source boundary points, and each image lies at one of the two target boundary points. There are therefore again exactly four cases.

The code below introduces these four maps explicitly. They play for heteromorphisms the same role that `constL`, `constR`, `id`, and `dual` played for endomorphisms.

Step 1: Map type and canonical inter-distinction maps. The map type, pointwise equivalence, and four reference maps between two distinction carriers mirror the endomorphism case. The two mixed maps `LR` and `RL` use source-side coverage to assign the appropriate target boundary witness.

```
-- § K4 maps between two distinctions.
module K4Map (d1 d2 : Distinction) where
  Map : Set
  Map = S d1 → S d2

-- § Pointwise equality on maps.
≐-refl : {f : Map} → f ≐ f
≐-refl x = refl

≐-sym : {f g : Map} → f ≐ g → g ≐ f
≐-sym p x = sym (p x)

≐-trans : {f g h : Map} → f ≐ g → g ≐ h → f ≐ h
≐-trans p q x = trans (p x) (q x)

-- § The four canonical maps.
constL : Map
constL _ = ℓ d2

constR : Map
constR _ = r d2
```

```

LR : Map
LR x with cover d1 x
... | inj1 _ = ℓ d2
... | inj2 _ = r d2

RL : Map
RL x with cover d1 x
... | inj1 _ = r d2
... | inj2 _ = ℓ d2

-- § Boundary behavior of LR.
LR-ℓ : LR (ℓ d1) ≡ ℓ d2
LR-ℓ with cover d1 (ℓ d1)
... | inj1 _ = refl
... | inj2 ℓ≡r = ⊥-elim ((ℓ≠r d1) ℓ≡r)

LR-r : LR (r d1) ≡ r d2
LR-r with cover d1 (r d1)
... | inj1 r≡ℓ = ⊥-elim ((ℓ≠r d1) (sym r≡ℓ))
... | inj2 _ = refl

-- § Boundary behavior of RL.
RL-ℓ : RL (ℓ d1) ≡ r d2
RL-ℓ with cover d1 (ℓ d1)
... | inj1 _ = refl
... | inj2 ℓ≡r = ⊥-elim ((ℓ≠r d1) ℓ≡r)

RL-r : RL (r d1) ≡ ℓ d2
RL-r with cover d1 (r d1)
... | inj1 r≡ℓ = ⊥-elim ((ℓ≠r d1) (sym r≡ℓ))
... | inj2 _ = refl

```

Step 2: Interpretation and classification of inter-distinction maps. Each EndoCase label interprets as one of the four canonical maps, and every concrete map is classified by its boundary outputs.

```

-- § Interpret an EndoCase as a map between distinctions.
interpret : EndoCase → Map
interpret case-constL = constL
interpret case-constR = constR
interpret case-id     = LR
interpret case-dual    = RL

-- § Classify a map by its boundary behavior.
classify : Map → EndoCase
classify f with cover d2 (f (ℓ d1)) | cover d2 (f (r d1))
... | inj1 _ | inj1 _ = case-constL
... | inj2 _ | inj2 _ = case-constR
... | inj1 _ | inj2 _ = case-id
... | inj2 _ | inj1 _ = case-dual

```

Step 3: Soundness of inter-distinction classification. Classification followed by interpretation recovers the original map at both boundary witnesses.

```
-- § Soundness at  $\ell$ .
sound-at- $\ell$  : (f : Map) → interpret (classify f) ( $\ell$  d1) ≡ f ( $\ell$  d1)
sound-at- $\ell$  f with cover d2 (f ( $\ell$  d1)) | cover d2 (f (r d1))
... | inj1 fl≡ $\ell$  | inj1 _ = sym fl≡ $\ell$ 
... | inj2 fl≡r | inj2 _ = sym fl≡r
... | inj1 fl≡ $\ell$  | inj2 _ = trans LR- $\ell$  (sym fl≡ $\ell$ )
... | inj2 fl≡r | inj1 _ = trans RL- $\ell$  (sym fl≡r)

-- § Soundness at r.
sound-at-r : (f : Map) → interpret (classify f) (r d1) ≡ f (r d1)
sound-at-r f with cover d2 (f ( $\ell$  d1)) | cover d2 (f (r d1))
... | inj1 _ | inj1 fr≡ $\ell$  = sym fr≡ $\ell$ 
... | inj2 _ | inj2 fr≡r = sym fr≡r
... | inj1 _ | inj2 fr≡r = trans LR-r (sym fr≡r)
... | inj2 _ | inj1 fr≡ $\ell$  = trans RL-r (sym fr≡ $\ell$ )

-- § Classification is sound: the interpreted classify agrees pointwise with f.
classify-sound : (f : Map) → interpret (classify f) ≐ f
classify-sound f x = Distinction-elim d1 (sound-at- $\ell$  f) (sound-at-r f) x

-- § Maps are determined by their boundary values.
map-determined :
(f g : Map) →
f ( $\ell$  d1) ≡ g ( $\ell$  d1) →
f (r d1) ≡ g (r d1) →
f ≐ g
map-determined f g eql eqr x = Distinction-elim d1 eql eqr x

-- § Absurdity helpers.
absurd- $\ell$ r : {A : Set} → ( $\ell$  d2 ≡ r d2) → A
absurd- $\ell$ r e =  $\perp$ -elim ( $\ell \neq r$  d2) e

absurd-r- $\ell$  : {A : Set} → (r d2 ≡  $\ell$  d2) → A
absurd-r- $\ell$  e =  $\perp$ -elim ( $\ell \neq r$  d2) (sym e)
```

Step 4: Injectivity and uniqueness. Interpretation is injective because different case labels produce different boundary behaviour in the target. Combined with soundness, this yields uniqueness of classification.

```
interpret-injective :
(c c' : EndoCase) →
interpret c ≐ interpret c' →
c ≡ c'

interpret-injective case-constL case-constL _ = refl
interpret-injective case-constL case-constR p = absurd- $\ell$ r (p ( $\ell$  d1))
interpret-injective case-constL case-id p =
absurd- $\ell$ r (trans (p (r d1)) LR-r)
interpret-injective case-constL case-dual p =
absurd- $\ell$ r (trans (p ( $\ell$  d1)) RL- $\ell$ )
```

```

interpret-injective case-constR case-constL p =
  absurd-rl (p (ℓ d₁))
interpret-injective case-constR case-constR _ = refl
interpret-injective case-constR case-id p =
  absurd-ℓr (trans (sym LR-ℓ) (sym (p (ℓ d₁))))
interpret-injective case-constR case-dual p =
  absurd-ℓr (sym (trans (p (r d₁)) RL-r))

interpret-injective case-id case-constL p =
  absurd-ℓr (sym (trans (sym LR-r) (p (r d₁))))
interpret-injective case-id case-constR p =
  absurd-ℓr (trans (sym LR-ℓ) (p (ℓ d₁)))
interpret-injective case-id case-id _ = refl
interpret-injective case-id case-dual p =
  absurd-ℓr (trans (sym LR-ℓ) (trans (p (ℓ d₁)) RL-ℓ))

interpret-injective case-dual case-constL p =
  absurd-ℓr (trans (sym (p (ℓ d₁))) RL-ℓ)
interpret-injective case-dual case-constR p =
  absurd-ℓr (sym (trans (sym (p (r d₁)) RL-r)))
interpret-injective case-dual case-id p =
  absurd-ℓr (sym (trans (sym RL-ℓ) (trans (p (ℓ d₁)) LR-ℓ)))
interpret-injective case-dual case-dual _ = refl

-- § Classify is unique relative to interpretation.
classify-unique : (f : Map) → (c : EndoCase) → interpret c ≐ f → c ≐ classify f
classify-unique f c c ≐ f =
  interpret-injective c (classify f) (≐-trans c ≐ f (≐-sym (classify-sound f)))

-- § Classify is a left-inverse of interpretation: forced by uniqueness on refl.
classify-interpret : (c : EndoCase) → classify (interpret c) ≐ c
classify-interpret c = sym (classify-unique (interpret c) c ≐ refl)

-- § The bijection between EndoCase and (Map / ≐) for inter-distinction maps.
record MapBijection : Set where
  field
    to      : Map → EndoCase
    from    : EndoCase → Map
    to-from : (c : EndoCase) → to (from c) ≐ c
    from-to : (f : Map) → from (to f) ≐ f
    to-unique : (f : Map) → (c : EndoCase) → from c ≐ f → c ≐ to f

k4map-bijection : MapBijection
k4map-bijection = record
{ to      = classify
; from    = interpret
; to-from = classify-interpret
; from-to = classify-sound
; to-unique = classify-unique
}

```


Law 7.1: A Map Is Determined By Boundary Values

Every input falls into one of the two boundary cases. Once the values at ℓ and r are fixed, pointwise equality follows.

```
-- § Top-level classification soundness.
k4map-classification-sound :
  (d1 d2 : Distinction) →
  (f : S d1 → S d2) →
  ∑ EndoCase (λ c → K4Map.interpret d1 d2 c ≐ f)
k4map-classification-sound d1 d2 f =
  K4Map.classify d1 d2 f , K4Map.classify-sound d1 d2 f

-- § Top-level classification uniqueness.
k4map-classification-unique :
  (d1 d2 : Distinction) →
  (f : S d1 → S d2) →
  (c1 c2 : EndoCase) →
  K4Map.interpret d1 d2 c1 ≐ f →
  K4Map.interpret d1 d2 c2 ≐ f →
  c1 ≡ c2
k4map-classification-unique d1 d2 f c1 c2 p1 p2 =
  K4Map.interpret-injective d1 d2 c1 c2 (K4Map.≐-trans d1 d2 p1 (K4Map.≐-sym d1 d2 p2))

-- § Law 7.1: A map is determined by boundary values.
law7-1-map-determined :
  (d1 d2 : Distinction) →
  (f g : S d1 → S d2) →
  f (ℓ d1) ≡ g (ℓ d1) →
  f (r d1) ≡ g (r d1) →
  f ≐ g
law7-1-map-determined d1 d2 = K4Map.map-determined d1 d2
```

Law 7.2: K₄ Classification Produces A Witness For Any Map

Every map between two distinctions receives a classification witness: a case label paired with a proof that the labeled reference map agrees with the given map at every point. The proof strategy is the same as in the endomorphism case: inspect the two boundary outputs, match the resulting pattern, and conclude pointwise.

```
-- § Law 7.2: K4 classification produces a witness for any map.
law7-2-k4map-classification-sound :
  (d1 d2 : Distinction) →
  (f : S d1 → S d2) →
  ∑ EndoCase (λ c → K4Map.interpret d1 d2 c ≐ f)
law7-2-k4map-classification-sound = k4map-classification-sound
```

Law 7.3: K₄ Witness For Maps Is Unique

Two distinct case labels cannot interpret to the same map. If they did, their boundary outputs would agree simultaneously, forcing the two boundary witnesses in the

codomain to be equal and contradicting $\ell \neq r$.

```
-- § Law 7.3: K4 witness for maps is unique.
law7-3-k4map-classification-unique :
  (d1 d2 : Distinction) →
  (f : S d1 → S d2) →
  (c1 c2 : EndoCase) →
  K4Map.interpret d1 d2 c1  $\stackrel{\circ}{=}$  f →
  K4Map.interpret d1 d2 c2  $\stackrel{\circ}{=}$  f →
  c1  $\equiv$  c2
law7-3-k4map-classification-unique = k4map-classification-unique
```

Law 7.4: Map Presentation Is Unique Up To Isomorphism

Any alternative presentation of the four-case map classification, using a different carrier type X together with its own interpretation and classification functions, is isomorphic to `EndoCase`. The proof constructs the isomorphism by composing the alternative classifier with the fixed interpreter and conversely, then uses soundness and injectivity to verify the two round trips.

```
-- § Map presentation record.
record MapPresentation (d1 d2 : Distinction) (X : Set) : Set where
  field
    mp-interpret      : X → (S d1 → S d2)
    mp-classify       : (S d1 → S d2) → X
    mp-classify-sound : (f : S d1 → S d2) → mp-interpret (mp-classify f)  $\stackrel{\circ}{=}$  f
    mp-interpret-injective : (x y : X) → mp-interpret x  $\stackrel{\circ}{=}$  mp-interpret y → x  $\equiv$  y

open MapPresentation public

-- § Law 7.4: Map presentation is unique up to isomorphism.
law7-4-map-presentation-unique :
  (d1 d2 : Distinction) →
  {X : Set} →
  MapPresentation d1 d2 X →
  SetIso X EndoCase
law7-4-map-presentation-unique d1 d2 {X} pres =
  let
    module K = K4Map d1 d2

    to' : X → EndoCase
    to' x = K.classify (MapPresentation.mp-interpret pres x)

    from' : EndoCase → X
    from' c = MapPresentation.mp-classify pres (K.interpret c)

    to-from' : (c : EndoCase) → to' (from' c)  $\equiv$  c
    to-from' c =
      sym
      (K.classify-unique
       (MapPresentation.mp-interpret pres (MapPresentation.mp-classify pres (K.interpret c))))
```

```

c
(K.∘-sym (MapPresentation.mp-classify-sound pres (K.interpret c)))

from-to' : (x : X) → from' (to' x) ≡ x
from-to' x =
  MapPresentation.mp-interpret-injective pres
  (MapPresentation.mp-classify pres (K.interpret (K.classify (MapPresentation.mp-interpret pres x))))
x
(K.∘-trans
  (MapPresentation.mp-classify-sound pres (K.interpret (K.classify (MapPresentation.mp-interpret pres x))))
  (K.classify-sound (MapPresentation.mp-interpret pres x)))
in
record
{ to = to'
; from = from'
; to-from = to-from'
; from-to = from-to'
}

```

Law 7.5: Map Classification Is a Bijection

The inter-distinction classifier satisfies the same round-trip property as the endomorphism classifier. In particular, classification is a left inverse to interpretation, so the four map cases again give a bijective description of maps modulo pointwise equality.

```

-- § Law 7.5: classify ∘ interpret = id at the case-label level for inter-distinction maps.
law7-5-k4map-classify-interpret :
  (d1 d2 : Distinction) →
  (c : EndoCase) →
  K4Map.classify d1 d2 (K4Map.interpret d1 d2 c) ≡ c
law7-5-k4map-classify-interpret d1 d2 = K4Map.classify-interpret d1 d2

```

The four-case classification therefore extends from endomorphisms to arbitrary maps between distinct carriers. The same four cases appear, and the same uniqueness statement remains valid. The classification is therefore structural rather than an artifact of the self-mapping case.

Chapter 20

Indexed Ledger

The drift hierarchy can be indexed explicitly by natural numbers. This does not alter the underlying drift structure, but it provides a bookkeeping device for later arguments about monotonicity and acyclicity.

The record `DriftState $\mathbb{N}$` pairs an ordinary drift state with a natural-number tick. The successor operation increments both components in parallel, and the indexed reachability relation tracks strict progression along this ledger.

```
-- § Natural numbers.
data  $\mathbb{N}$  : Set where
  zero :  $\mathbb{N}$ 
  suc :  $\mathbb{N} \rightarrow \mathbb{N}$ 

-- § Standard ordering on  $\mathbb{N}$ .
infix 4 <=
data <= :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow$  Set where
  z<=n : {n :  $\mathbb{N}$ }  $\rightarrow$  zero <= n
  s<=s : {m n :  $\mathbb{N}$ }  $\rightarrow$  m <= n  $\rightarrow$  suc m <= suc n

-- § <= is reflexive.
<=-refl : (n :  $\mathbb{N}$ )  $\rightarrow$  n <= n
<=-refl zero = z<=n
<=-refl (suc n) = s<=s (<=-refl n)

-- § <= is transitive.
<=-trans : {a b c :  $\mathbb{N}$ }  $\rightarrow$  a <= b  $\rightarrow$  b <= c  $\rightarrow$  a <= c
<=-trans z<=n _ = z<=n
<=-trans (s<=s p) (s<=s q) = s<=s (<=-trans p q)

-- § Step lemma.
<=-step : (n :  $\mathbb{N}$ )  $\rightarrow$  n <= suc n
<=-step zero = z<=n
<=-step (suc n) = s<=s (<=-step n)

-- § suc n <= n is absurd.
suc<=-impossible : (n :  $\mathbb{N}$ )  $\rightarrow$  suc n <= n  $\rightarrow$   $\perp$ 
```

```

suc≤-impossible zero ()
suc≤-impossible (suc n) (s≤s p) = suc≤-impossible n p

-- § Tick-indexed drift state.
record DriftStateℕ : Setω where
  constructor ⟨⟦_,_⟧⟩
  field
    tick : ℕ
    base : DriftState

open DriftStateℕ public

-- § Step the indexed state forward.
stepStateℕ : DriftStateℕ → DriftStateℕ
stepStateℕ s = ⟨⟦ suc (tick s), stepState (base s) ⟧⟩

-- § Extract the carrier of an indexed state.
Carrierℕ : (s : DriftStateℕ) → Set (ℓ (base s))
Carrierℕ s = Carrier (base s)

-- § Embed between consecutive indexed carriers.
state-embedℕ : (s : DriftStateℕ) → Carrierℕ s → Carrierℕ (stepStateℕ s)
state-embedℕ s = state-embed (base s)

-- § Strict reachability on indexed states.
data Reach+ℕ : DriftStateℕ → DriftStateℕ → Setω where
  oneℕ : {s : DriftStateℕ} → Reach+ℕ s (stepStateℕ s)
  moreℕ : {s t : DriftStateℕ} → Reach+ℕ (stepStateℕ s) t → Reach+ℕ s t

-- § Strict reachability as infix.
infix 20 _<ℕ_
_<ℕ_ : DriftStateℕ → DriftStateℕ → Setω
s <ℕ t = Reach+ℕ s t

```

Law 8.0: Tick Strictly Increases Along Reach⁺ℕ

The key property of the indexed ledger is that every strict reachability witness increases the tick. This is proved directly by induction on the indexed reach proof.

```

-- § Law 8.0: Tick strictly increases along Reach+ℕ.
law8-0-tick-increases : {s t : DriftStateℕ} → (s <ℕ t) → suc (tick s) ≤ tick t
law8-0-tick-increases oneℕ = ≤-refl _
law8-0-tick-increases {s} (moreℕ p) =
  ≤-trans (≤-step (suc (tick s))) (law8-0-tick-increases p)

```

Law 8.1: Indexed Drift Has No Cycles

The indexed acyclicity statement is then immediate: a cycle would force $\text{suc}(\text{tick } s) \leq \text{tick } s$, which is impossible in $\mathbb{N}$.

```

-- § Law 8.1: Indexed drift has no cycles.
law8-1-no-cycleℕ : (s : DriftStateℕ) → (s <ℕ s) → ⊥
law8-1-no-cycleℕ s p = suc≤-impossible (tick s) (law8-0-tick-increases p)

```

Law 6.6 (deferred): Tick Presents Every Drift Orbit As ω

Fixing a base state produces a single drift orbit. The indexed presentation of that orbit is ω -shaped: the natural number parameter records how many drift steps have been taken from the base.

The definitions and lemmas below make this correspondence explicit by relating strict indexed reachability on the orbit to the standard order on $\mathbb{N}$.

```
-- § Iterate drift states from a chosen base.
orbitBase :  $\mathbb{N} \rightarrow \text{DriftState} \rightarrow \text{DriftState}$ 
orbitBase zero b = b
orbitBase (suc n) b = stepState (orbitBase n b)

-- § The tick-indexed orbit generated by a chosen base.
orbitState :  $\text{DriftState} \rightarrow \mathbb{N} \rightarrow \text{DriftState}\mathbb{N}$ 
orbitState b n =  $\langle\langle n, \text{orbitBase } n \ b \rangle\rangle$ 

-- § Equality of orbit indices lifts to Set $\omega$ -equality of orbit states.
orbitState-cong $\omega$  : (b :  $\text{DriftState}$ )  $\rightarrow \{n \ m : \mathbb{N}\} \rightarrow n \equiv m \rightarrow \text{orbitState } b \ n \equiv_{\omega} \text{orbitState } b \ m$ 
orbitState-cong $\omega$  b refl = refl $\omega$ 

-- § Tick reads off the index of the orbit state definitionally.
orbit-tick : (b :  $\text{DriftState}$ )  $\rightarrow (n : \mathbb{N}) \rightarrow \text{tick } (\text{orbitState } b \ n) \equiv n$ 
orbit-tick b n = refl

-- § Orbit successor is definitionally the indexed drift step.
orbit-step : (b :  $\text{DriftState}$ )  $\rightarrow (n : \mathbb{N}) \rightarrow \text{orbitState } b \ (\text{suc } n) \equiv_{\omega} \text{stepState}\mathbb{N} (\text{orbitState } b \ n)$ 
orbit-step b n = refl $\omega$ 

-- § Advance an index by a finite number of successor steps.
advanceN :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
advanceN zero n = n
advanceN (suc k) n = suc (advanceN k n)

-- § Local truncated subtraction for the orbit theorem.
diffN :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
diffN zero n = zero
diffN (suc m) zero = suc m
diffN (suc m) (suc n) = diffN m n

-- § Advancing after a successor is the successor of advancing.
advanceN-suc-right : (k n :  $\mathbb{N}$ )  $\rightarrow \text{advanceN } k \ (\text{suc } n) \equiv \text{suc } (\text{advanceN } k \ n)$ 
advanceN-suc-right zero n = refl
advanceN-suc-right (suc k) n = cong suc (advanceN-suc-right k n)

-- § Shifting the offset by one step can be moved from the offset slot to the base slot.
advanceN-shift : (k n :  $\mathbb{N}$ )  $\rightarrow \text{advanceN } (\text{suc } k) \ n \equiv \text{advanceN } k \ (\text{suc } n)$ 
advanceN-shift zero n = refl
advanceN-shift (suc k) n = cong suc (advanceN-shift k n)

-- § Advancing zero by k steps returns k.
advanceN-zero : (k :  $\mathbb{N}$ )  $\rightarrow \text{advanceN } k \ \text{zero} \equiv k$ 
advanceN-zero zero = refl
```

```

advance $\mathbb{N}$ -zero (suc k) = cong suc (advance $\mathbb{N}$ -zero k)

-- § Any  $\leq$ -witness splits the upper index into the lower index plus a finite advance.
 $\leq$ -advance-split : {n m :  $\mathbb{N}$ }  $\rightarrow$  n  $\leq$  m  $\rightarrow$  advance $\mathbb{N}$  (diff $\mathbb{N}$  m n) n  $\equiv$  m
 $\leq$ -advance-split {zero} {zero} z  $\leq$  n = advance $\mathbb{N}$ -zero zero
 $\leq$ -advance-split {zero} {suc m} z  $\leq$  n = advance $\mathbb{N}$ -zero (suc m)
 $\leq$ -advance-split {suc n} {zero} ()
 $\leq$ -advance-split {suc n} {suc m} (s  $\leq$  s p) =
  trans (advance $\mathbb{N}$ -suc-right (diff $\mathbb{N}$  m n) n) (cong suc ( $\leq$ -advance-split p))

-- § Local indexed transitivity for the orbit block.
Reach $^+$  $\mathbb{N}$ -trans-local : {s t u : DriftState $\mathbb{N}$ }  $\rightarrow$  (s  $\prec\mathbb{N}$  t)  $\rightarrow$  (t  $\prec\mathbb{N}$  u)  $\rightarrow$  (s  $\prec\mathbb{N}$  u)
Reach $^+$  $\mathbb{N}$ -trans-local one $\mathbb{N}$  q = more $\mathbb{N}$  q
Reach $^+$  $\mathbb{N}$ -trans-local (more $\mathbb{N}$  p) q = more $\mathbb{N}$  (Reach $^+$  $\mathbb{N}$ -trans-local p q)

-- § Local indexed extension for the orbit block.
Reach $^+$  $\mathbb{N}$ -extend-local : {s t : DriftState $\mathbb{N}$ }  $\rightarrow$  (s  $\prec\mathbb{N}$  t)  $\rightarrow$  (s  $\prec\mathbb{N}$  stepState $\mathbb{N}$  t)
Reach $^+$  $\mathbb{N}$ -extend-local p = Reach $^+$  $\mathbb{N}$ -trans-local p one $\mathbb{N}$ 

-- § Every positive offset along the orbit yields a strict indexed reach proof.
orbit-reach : (b : DriftState)  $\rightarrow$  (n k :  $\mathbb{N}$ )  $\rightarrow$  orbitState b n  $\prec\mathbb{N}$  orbitState b (advance $\mathbb{N}$  (suc k) n)
orbit-reach b n zero = one $\mathbb{N}$ 
orbit-reach b n (suc k) = Reach $^+$  $\mathbb{N}$ -extend-local (orbit-reach b n k)

-- § Any strict indexed reach on the orbit forces a positive tick increase.
orbit-reach-complete :
  (b : DriftState)  $\rightarrow$  {n m :  $\mathbb{N}$ }  $\rightarrow$ 
  orbitState b n  $\prec\mathbb{N}$  orbitState b m  $\rightarrow$  suc n  $\leq$  m
orbit-reach-complete b p = law8-0-tick-increases p

-- § Any positive index difference on the orbit yields a strict indexed reach proof.
orbit-reach-sound :
  (b : DriftState)  $\rightarrow$  {n m :  $\mathbb{N}$ }  $\rightarrow$ 
  suc n  $\leq$  m  $\rightarrow$  orbitState b n  $\prec\mathbb{N}$  orbitState b m
orbit-reach-sound b {n} {m} le =
  subst $\omega$ 
  ( $\lambda$  u  $\rightarrow$  orbitState b n  $\prec\mathbb{N}$  u)
  (orbitState-cong $\omega$  b
    (trans (advance $\mathbb{N}$ -shift (diff $\mathbb{N}$  m (suc n)) n)
      ( $\leq$ -advance-split le)))
  (orbit-reach b n (diff $\mathbb{N}$  m (suc n)))

-- § Tick presents the orbit over b as an  $\omega$ -shaped indexed chain.
record TickIso $\omega$  (b : DriftState) : Set $\omega$  where
  field
    stateAt      :  $\mathbb{N} \rightarrow$  DriftState $\mathbb{N}$ 
    tickAt       : (n :  $\mathbb{N}$ )  $\rightarrow$  tick (stateAt n)  $\equiv$  n
    stepAt       : (n :  $\mathbb{N}$ )  $\rightarrow$  stateAt (suc n)  $\equiv\omega$  stepState $\mathbb{N}$  (stateAt n)
    reachAt      : (n k :  $\mathbb{N}$ )  $\rightarrow$  stateAt n  $\prec\mathbb{N}$  stateAt (advance $\mathbb{N}$  (suc k) n)
    reachSound   : {n m :  $\mathbb{N}$ }  $\rightarrow$  suc n  $\leq$  m  $\rightarrow$  stateAt n  $\prec\mathbb{N}$  stateAt m
    reachComplete : {n m :  $\mathbb{N}$ }  $\rightarrow$  stateAt n  $\prec\mathbb{N}$  stateAt m  $\rightarrow$  suc n  $\leq$  m

-- § Law 6.6: The drift orbit over any chosen base is indexed by  $\omega$  through tick.
law6-6-tick-iso- $\omega$  : (b : DriftState)  $\rightarrow$  TickIso $\omega$  b

```



```

law6-6-tick-iso- $\omega$  b = record
{ stateAt      = orbitState b
; tickAt       = orbit-tick b
; stepAt       = orbit-step b
; reachAt      = orbit-reach b
; reachSound   = orbit-reach-sound b
; reachComplete = orbit-reach-complete b
}

```

Law 6.4 Refined: Orbit Trichotomy

With the indexed orbit in hand, one obtains the expected trichotomy statement: along a fixed orbit, two states are either ordered one way, equal, or ordered the other way.

```

-- § Strict trichotomy on  $\mathbb{N}$ : every pair is lt, eq, or gt.
data N-Tri (n m :  $\mathbb{N}$ ) : Set where
lt  : suc n  $\leq$  m  $\rightarrow$  N-Tri n m
same : n  $\equiv$  m  $\rightarrow$  N-Tri n m
gt  : suc m  $\leq$  n  $\rightarrow$  N-Tri n m

N-tri : (n m :  $\mathbb{N}$ )  $\rightarrow$  N-Tri n m
N-tri zero zero    = same refl
N-tri zero (suc m) = lt (s  $\leq$  s z  $\leq$  n)
N-tri (suc n) zero = gt (s  $\leq$  s z  $\leq$  n)
N-tri (suc n) (suc m) with N-tri n m
... | lt p = lt (s  $\leq$  s p)
... | same p = same (cong suc p)
... | gt p = gt (s  $\leq$  s p)

-- § Index equality implies orbit-state Set $\omega$ -equality.
orbitState-eq $\mathbb{N}$  : (b : DriftState)  $\rightarrow$  {n m :  $\mathbb{N}$ }  $\rightarrow$  n  $\equiv$  m  $\rightarrow$  orbitState b n  $\equiv_{\omega}$  orbitState b m
orbitState-eq $\mathbb{N}$  b refl = refl $\omega$ 

-- § Set $\omega$ -level trichotomy witness for the orbit.
data OrbitTri (b : DriftState) (n m :  $\mathbb{N}$ ) : Set $\omega$  where
orbit-lt : orbitState b n  $\prec_{\mathbb{N}}$  orbitState b m  $\rightarrow$  OrbitTri b n m
orbit-eq : orbitState b n  $\equiv_{\omega}$  orbitState b m  $\rightarrow$  OrbitTri b n m
orbit-gt : orbitState b m  $\prec_{\mathbb{N}}$  orbitState b n  $\rightarrow$  OrbitTri b n m

-- § Law 6.4 (orbit refinement): the orbit over b is strictly totally ordered.
law6-4-orbit-total : (b : DriftState)  $\rightarrow$  (n m :  $\mathbb{N}$ )  $\rightarrow$  OrbitTri b n m
law6-4-orbit-total b n m with N-tri n m
... | lt p = orbit-lt (orbit-reach-sound b p)
... | same p = orbit-eq (orbitState-eq $\mathbb{N}$  b p)
... | gt p = orbit-gt (orbit-reach-sound b p)

```


Chapter 21

Presentation of Reachability

The indexed presentation is useful, but it also introduces bookkeeping data that should not count as invariant mathematical content. The purpose of this chapter is to separate the underlying reachability information from the additional index-tracking apparatus.

The main constructions are therefore the forgetful map from indexed reachability to ordinary reachability and the converse lifting operation.

```
-- § Forget the tick index.
forgetState : DriftStateℕ → DriftState
forgetState = base

-- § Forget preserves strict reachability.
forgetReach+ : {s t : DriftStateℕ} → (s <ℕ t) → (forgetState s < forgetState t)
forgetReach+ oneℕ = one
forgetReach+ (moreℕ p) = more (forgetReach+ p)

-- § Compute the lifted target.
liftTarget : (n : ℕ) → {s t : DriftState} → (p : s < t) → DriftStateℕ
liftTarget n {s} one = stepStateℕ ⟨⟨ n , s ⟩⟩
liftTarget n (more p) = liftTarget (suc n) p

-- § Lift a base reach proof into the indexed ledger.
liftReach+ : (n : ℕ) → {s t : DriftState} → (p : s < t) → (⟨⟨ n , s ⟩⟩ <ℕ liftTarget n p)
liftReach+ n {s} one = oneℕ
liftReach+ n (more p) = moreℕ (liftReach+ (suc n) p)

-- § Lifted target forgets to the original target.
liftTarget-base : (n : ℕ) → {s t : DriftState} → (p : s < t) → forgetState (liftTarget n p) ≡ω t
liftTarget-base n {s} one = reflω
liftTarget-base n (more p) = liftTarget-base (suc n) p

-- § Substitution lemma for more-constructor.
substω-more :
  {s : DriftState} {t u : DriftState} →
  (eq : t ≡ω u) →
  (p : Reach+ (stepState s) t) →
```

```

substω (λ x → Reach+ s x) eq (more p) ≡ω more (substω (λ x → Reach+ (stepState s) x) eq p)
substω-more reflω p = reflω

```

Laws 9.0–9.2: Forget/Lift Round-Trip

These first lemmas show that forgetting and lifting behave exactly as the definitions require. Indexed reachability projects to ordinary reachability, every ordinary reach witness admits an indexed lift, and forgetting after lifting recovers the original proof up to the appropriate equality.

```

-- § Law 9.0: Forget preserves strict reachability.
law9-0-forget-preserves : {s t : DriftStateℕ} → (s <ℕ t) → (forgetState s < forgetState t)
law9-0-forget-preserves = forgetReach+

-- § Law 9.1: Every strict reachability proof lifts to the indexed ledger.
law9-1-lift-exists : (n : ℕ) → {s t : DriftState} → (p : s < t) → (⟨⟨ n , s ⟩⟩ <ℕ liftTarget n p)
law9-1-lift-exists = liftReach+

-- § Law 9.2: Forget after lift recovers the original proof.
law9-2-forget-after-lift :
  (n : ℕ) → {s t : DriftState} →
  (p : s < t) →
  substω (λ u → Reach+ s u) (liftTarget-base n p) (forgetReach+ (liftReach+ n p)) ≡ω p
law9-2-forget-after-lift n one = reflω
law9-2-forget-after-lift n {s} (more p) =
  let
    eq = liftTarget-base (suc n) p
    q = forgetReach+ (liftReach+ (suc n) p)
  in
  transω
  (substω-more {s = s} eq q)
  (congω more (law9-2-forget-after-lift (suc n) p))

```

Laws 9.3–9.9: Admissible Observables

An indexed observable is called admissible when it depends only on the underlying base state and not on the particular index used to present that state. The next definitions and lemmas formalize this factorization requirement.

```

-- § Pointwise equality on ω-valued maps.
infix 4 _≡ω_
_≡ω_ : {A B : Setω} → (A → B) → (A → B) → Setω
_≡ω_ {A = A} f g = (x : A) → f x ≡ω g x

-- § Canonical inclusion.
canonicalState : DriftState → DriftStateℕ
canonicalState s = ⟨⟨ zero , s ⟩⟩

-- § Respecting the base projection.

```

```

record RespectsBase {Y : Setω} (f : DriftStateℕ → Y) : Setω where
  field
    respects : (x y : DriftStateℕ) → forgetState x ≡ω forgetState y → f x ≡ω f y

open RespectsBase public

-- § Factorization through forgetState.
record FactorsThroughBase {Y : Setω} (f : DriftStateℕ → Y) : Setω where
  field
    g : DriftState → Y
    comm : (x : DriftStateℕ) → f x ≡ω g (forgetState x)

open FactorsThroughBase public

-- § Law 9.3: Base-respecting observables factor through forget.
law9-3-factor-through-base :
  {Y : Setω} →
  (f : DriftStateℕ → Y) →
  RespectsBase f →
  FactorsThroughBase f
law9-3-factor-through-base f rb = record
  { g = λ s → f (canonicalState s)
  ; comm = λ x →
      respects rb x (canonicalState (forgetState x)) reflω
  }

-- § Law 9.4: Factorization through forget is unique.
law9-4-factor-unique :
  {Y : Setω} →
  {f : DriftStateℕ → Y} →
  (u v : FactorsThroughBase f) →
  g u ≡ω g v
law9-4-factor-unique {f = f} u v s =
  let
    x = canonicalState s
  in
  transω (symω (comm u x)) (comm v x)

-- § Admissible observable record.
record AdmissibleObservable (Y : Setω) : Setω where
  field
    obs : DriftStateℕ → Y
    base : RespectsBase obs

open AdmissibleObservable public

-- § Extract the base observable.
observe : {Y : Setω} → AdmissibleObservable Y → DriftState → Y
observe {Y} a = g (law9-3-factor-through-base (obs a) (base a))

-- § Commutation witness.
observe-comm : {Y : Setω} → (a : AdmissibleObservable Y) → (x : DriftStateℕ) → obs a x ≡ω observe a (forgetState x)
observe-comm a = comm (law9-3-factor-through-base (obs a) (base a))

-- § Law 9.5: Admissible observables collapse to base observables.

```

```
law9-5-observe-comm : {Y : Set $\omega$ }  $\rightarrow$  (a : AdmissibleObservable Y)  $\rightarrow$  (x : DriftState $\mathbb{N}$ )  $\rightarrow$  obs a x  $\equiv_{\omega}$  observe a (forgetState x)
law9-5-observe-comm = observe-comm
```

```
-- § Law 9.6: Base observable extracted from admissibility is unique.
```

```
law9-6-observe-unique :
```

```
{Y : Set $\omega$ }  $\rightarrow$ 
(a : AdmissibleObservable Y)  $\rightarrow$ 
(h1 h2 : DriftState  $\rightarrow$  Y)  $\rightarrow$ 
((x : DriftState $\mathbb{N}$ )  $\rightarrow$  obs a x  $\equiv_{\omega}$  h1 (forgetState x))  $\rightarrow$ 
((x : DriftState $\mathbb{N}$ )  $\rightarrow$  obs a x  $\equiv_{\omega}$  h2 (forgetState x))  $\rightarrow$ 
h1  $\equiv_{\omega}$  h2
```

```
law9-6-observe-unique a h1 h2 comm1 comm2 s =
```

```
let
```

```
  f = obs a
```

```
  u : FactorsThroughBase f
```

```
  u = record { g = h1 ; comm = comm1 }
```

```
  v : FactorsThroughBase f
```

```
  v = record { g = h2 ; comm = comm2 }
```

```
in
```

```
law9-4-factor-unique u v s
```

```
-- § Indexed observable type.
```

```
IndexedObs : (Y : Set $\omega$ )  $\rightarrow$  Set $\omega$ 
```

```
IndexedObs Y = DriftState $\mathbb{N}$   $\rightarrow$  Y
```

```
-- § Base observable type.
```

```
BaseObs : (Y : Set $\omega$ )  $\rightarrow$  Set $\omega$ 
```

```
BaseObs Y = DriftState  $\rightarrow$  Y
```

```
-- § Lift base to indexed.
```

```
liftBase : {Y : Set $\omega$ }  $\rightarrow$  BaseObs Y  $\rightarrow$  IndexedObs Y
```

```
liftBase h x = h (forgetState x)
```

```
-- § Pack a base observable as admissible.
```

```
packObs : {Y : Set $\omega$ }  $\rightarrow$  BaseObs Y  $\rightarrow$  AdmissibleObservable Y
```

```
packObs h = record
```

```
{ obs = liftBase h
```

```
; base = record { respects =  $\lambda$  x y eq  $\rightarrow$  cong $\omega$  h eq }
```

```
}
```

```
-- § Law 9.7: Indexed observables are unique given a base mediator.
```

```
law9-7-obs-unique :
```

```
{Y : Set $\omega$ }  $\rightarrow$ 
```

```
(h : BaseObs Y)  $\rightarrow$ 
```

```
(f1 f2 : IndexedObs Y)  $\rightarrow$ 
```

```
((x : DriftState $\mathbb{N}$ )  $\rightarrow$  f1 x  $\equiv_{\omega}$  h (forgetState x))  $\rightarrow$ 
```

```
((x : DriftState $\mathbb{N}$ )  $\rightarrow$  f2 x  $\equiv_{\omega}$  h (forgetState x))  $\rightarrow$ 
```

```
f1  $\equiv_{\omega}$  f2
```

```
law9-7-obs-unique h f1 f2 comm1 comm2 x =
```

```
trans $\omega$  (comm1 x) (sym $\omega$  (comm2 x))
```

```
-- § Law 9.8: Canonical indexed presentation for any admissible observable.
```

```
law9-8-canonical-pack :
```

```

{Y : Setω} →
(a : AdmissibleObservable Y) →
obs a ≐ω obs (packObs (observe a))
law9-8-canonical-pack a x = observe-comm a x

-- § Law 9.9: observe o packObs is the identity.
law9-9-observe-pack-id :
{Y : Setω} →
(h : DriftState → Y) →
observe (packObs h) ≐ω h
law9-9-observe-pack-id h s = reflω

```

The next definition embeds an ordinary small type into the ambient universe used for observables. The wrapper has a single constructor, so constructor injectivity and absurd patterns can still distinguish its images. This makes it suitable for the counterexample below.

```

-- § Lift an ordinary Set into Setω.
data Liftω (A : Set) : Setω where
liftω : A → Liftω A

-- § Tick as a Setω-valued observable on indexed drift states.
tickObs : DriftStateℕ → Liftω ℕ
tickObs s = liftω (tick s)

-- § Two states with identical base but distinct ticks.
zero-state : DriftState → DriftStateℕ
zero-state b = ⟨⟨ zero , b ⟩⟩

one-state : DriftState → DriftStateℕ
one-state b = ⟨⟨ suc zero , b ⟩⟩

-- § Same base witness for the two distinct-tick states.
same-base : (b : DriftState) → forgetState (zero-state b) ≐ω forgetState (one-state b)
same-base b = reflω

```

The observable `tickObs` separates two indexed states with the same base state. It therefore cannot satisfy `RespectsBase`. The collapse proved in this chapter applies only to base-respecting observables; the indexed ledger remains real bookkeeping structure precisely because this non-admissible observable is rejected.

```

-- § Law 9.x: tick is not an admissible observable.
law9-x-tick-not-admissible :
RespectsBase {Y = Liftω ℕ} tickObs → ⊥
law9-x-tick-not-admissible rb =
liftω-distinct (respects rb (zero-state b₀) (one-state b₀) (same-base b₀))
where
b₀ : DriftState

```

```

b0 = ⟨ lzero, Distinction → Distinctionℓ canonical-two-distinction ⟩

-- liftω is a data constructor; liftω zero ≡ω liftω (suc zero) is impossible.
liftω-distinct : liftω zero ≡ω liftω (suc zero) → ⊥
liftω-distinct ()

```

Laws 9.10–9.14: Observational Normal Forms

Once admissibility has been isolated, the normal form for admissible observables is forced by the factorization theorem: pass to the corresponding base observable and then pack it back into indexed form.

```

-- § Observational equality for admissible observables.
infix 4 _≈Obs_
_≈Obs_ : {Y : Setω} → AdmissibleObservable Y → AdmissibleObservable Y → Setω
_≈Obs_ a b = obs a ≐ω obs b

-- § ≈Obs reflexivity, symmetry, transitivity.
≈Obs-refl : {Y : Setω} → (a : AdmissibleObservable Y) → a ≈Obs a
≈Obs-refl a x = reflω

≈Obs-sym : {Y : Setω} → {a b : AdmissibleObservable Y} → a ≈Obs b → b ≈Obs a
≈Obs-sym p x = symω (p x)

≈Obs-trans : {Y : Setω} → {a b c : AdmissibleObservable Y} → a ≈Obs b → b ≈Obs c → a ≈Obs c
≈Obs-trans p q x = transω (p x) (q x)

-- § Canonical normal form.
canonObs : {Y : Setω} → AdmissibleObservable Y → AdmissibleObservable Y
canonObs a = packObs (observe a)

-- § Law 9.10: Canonicalization is observationally sound.
law9-10-canonObs-sound :
  {Y : Setω} →
  (a : AdmissibleObservable Y) →
  a ≈Obs canonObs a
law9-10-canonObs-sound a = law9-8-canonical-pack a

-- § Law 9.11: Canonicalization is idempotent up to observation.
law9-11-canonObs-idem :
  {Y : Setω} →
  (a : AdmissibleObservable Y) →
  canonObs (canonObs a) ≈Obs canonObs a
law9-11-canonObs-idem a =
  let
    b = canonObs a
  in
  λ x → symω ((law9-10-canonObs-sound b) x)

-- § Canonical base observable type.
CanonicalObs : (Y : Setω) → Setω
CanonicalObs Y = DriftState → Y

```



```

-- § Observational iso record.
record ObsIso (Y : Setω) : Setω where
  field
    to   : AdmissibleObservable Y → CanonicalObs Y
    from : CanonicalObs Y → AdmissibleObservable Y
    to-from : (h : CanonicalObs Y) → to (from h)  $\stackrel{\circ}{=}_\omega$  h
    from-to : (a : AdmissibleObservable Y) → a  $\approx_{\text{Obs}}$  from (to a)

open ObsIso public

-- § Law 9.12: Observational collapse as an explicit iso structure.
law9-12-obsIso : {Y : Setω} → ObsIso Y
law9-12-obsIso {Y} = record
  { to   = observe
  ; from = packObs
  ; to-from = law9-9-observe-pack-id
  ; from-to = law9-10-canonObs-sound
  }

-- § Law 9.13: Any ObsIso normalization agrees with canonObs up to observation.
law9-13-obsIso-normalizes :
  {Y : Setω} →
  (i : ObsIso Y) →
  (a : AdmissibleObservable Y) →
  ObsIso.from i (ObsIso.to i a)  $\approx_{\text{Obs}}$  canonObs a
law9-13-obsIso-normalizes i a =
   $\approx_{\text{Obs-trans}}$ 
  { a = ObsIso.from i (ObsIso.to i a)
  ; b = a
  ; c = canonObs a
  ;  $\approx_{\text{Obs-sym}}$  { a = a } { b = ObsIso.from i (ObsIso.to i a) } { ObsIso.from-to i a }
  ; law9-10-canonObs-sound a }

-- § Normalizer record.
record ObsNormalizer (Y : Setω) : Setω where
  field
    norm : AdmissibleObservable Y → AdmissibleObservable Y
    sound : (a : AdmissibleObservable Y) → a  $\approx_{\text{Obs}}$  norm a
    idem : (a : AdmissibleObservable Y) → norm (norm a)  $\approx_{\text{Obs}}$  norm a

open ObsNormalizer public

-- § Law 9.14: Any sound normalizer agrees with canonObs up to observation.
law9-14-normalizer-unique :
  {Y : Setω} →
  (n : ObsNormalizer Y) →
  (a : AdmissibleObservable Y) →
  norm n a  $\approx_{\text{Obs}}$  canonObs a
law9-14-normalizer-unique n a =
   $\approx_{\text{Obs-trans}}$ 
  { a = norm n a
  ; b = a
  ; c = canonObs a }

```

$(\approx\text{Obs-sym } \{a = a\} \{b = \text{norm } n a\} (\text{sound } n a))$
 $(\text{law9-10-canonObs-sound } a)$

At this stage the development has shown both that the four-case classification persists beyond endomorphisms and that the indexed reachability apparatus collapses to normal presentations when only admissible information is retained. The next part turns this into an explicit theory of observables and fixpoints.

Part IV

Observables and Fixpoint

The next part asks which quantities survive the passage from indexed presentations to genuine observables. By construction, admissible observables ignore book-keeping data and depend only on the underlying reach structure. This is the setting in which the later invariant arguments are carried out.

Chapter 22

Admissible Reach Observables

The indexed reachability relation carries more information than should count as observable, because it remembers the bookkeeping index as well as the underlying reach proof. An admissible reach observable is therefore required to factor through the forgetful map to ordinary reachability.

```
-- § Iterate successor.
iterSuc : ℕ → ℕ → ℕ
iterSuc zero n = n
iterSuc (suc k) n = iterSuc k (suc n)

-- § Count steps in a strict reach proof.
steps+ : {s t : DriftState} → (s < t) → ℕ
steps+ one = suc zero
steps+ (more p) = suc (steps+ p)

-- § Lift a base reach proof to the indexed ledger with correct tick.
liftReach+ℕ :
  (n : ℕ) → {s t : DriftState} →
  (p : s < t) →
  (⟨⟨ n , s ⟩⟩ <ℕ ⟨⟨ iterSuc (steps+ p) n , t ⟩⟩)
liftReach+ℕ n {s} one = oneℕ
liftReach+ℕ n (more p) = moreℕ (liftReach+ℕ (suc n) p)

-- § Forget after lift recovers the original proof.
forget-after-liftReach+ℕ :
  (n : ℕ) → {s t : DriftState} →
  (p : s < t) →
  forgetReach+ (liftReach+ℕ n p) ≡ω p
forget-after-liftReach+ℕ n one = reflω
forget-after-liftReach+ℕ n (more p) = congω more (forget-after-liftReach+ℕ (suc n) p)

-- § Admissible reach observable record.
record AdmissibleReachObservable (Y : Setω) : Setω where
  field
  obsR : {s t : DriftStateℕ} → (s <ℕ t) → Y
```

```

baseR : {s t : DriftState} → (s < t) → Y
commR : {s t : DriftStateℕ} → (p : s <ℕ t) → obsR p ≡ω baseR (forgetReach+ p)

```

```

open AdmissibleReachObservable public

```

Laws 10.0–10.1: Admissible Reach Observables Factor Through Forget

These two lemmas unpack the definition. An admissible reach observable is determined by its base component, and that base component is unique.

```

-- § Law 10.0: Admissible reach observables are determined by base reachability.
law10-0-comm : {Y : Setω} → (a : AdmissibleReachObservable Y) → {s t : DriftStateℕ} → (p : s <ℕ t) → obsR a p ≡ω baseR a (forgetReach+ p)
law10-0-comm a = commR a

-- § Law 10.1: Base reach observable mediator is unique.
law10-1-baseR-unique :
  {Y : Setω} →
  (obsR' : {s t : DriftStateℕ} → (s <ℕ t) → Y) →
  (h1 h2 : {s t : DriftState} → (s < t) → Y) →
  (((s t : DriftStateℕ) (p : s <ℕ t) → obsR' p ≡ω h1 (forgetReach+ p))) →
  (((s t : DriftStateℕ) (p : s <ℕ t) → obsR' p ≡ω h2 (forgetReach+ p))) →
  ({s t : DriftState} (p : s < t) → h1 p ≡ω h2 p)
law10-1-baseR-unique obsR' h1 h2 comm1 comm2 {s} {t} p =
  let
    q = liftReach+ℕ zero p
    e = forget-after-liftReach+ℕ zero p
  in
  transω
  (symω (congω h1 e))
  (transω
  (symω (comm1 q))
  (transω (comm2 q) (congω h2 e)))

```

Laws 10.2–10.3: Reach Observables And Transitivity

The next pair of lemmas records compatibility of admissible reach observables with transitivity and extension of indexed reach proofs.

```

-- § Indexed transitivity.
Reach+ℕ-trans : {s t u : DriftStateℕ} → (s <ℕ t) → (t <ℕ u) → (s <ℕ u)
Reach+ℕ-trans oneℕ q = moreℕ q
Reach+ℕ-trans (moreℕ p) q = moreℕ (Reach+ℕ-trans p q)

-- § Indexed extension.
Reach+ℕ-extend : {s t : DriftStateℕ} → (s <ℕ t) → (s <ℕ stepStateℕ t)
Reach+ℕ-extend p = Reach+ℕ-trans p oneℕ

-- § Forget commutes with transitivity.
forgetReach+-trans :

```



```

{s t u : DriftStateℕ} →
(p : s <ℕ t) → (q : t <ℕ u) →
forgetReach+ (Reach+ℕ-trans p q) ≡ω Reach+-trans (forgetReach+ p) (forgetReach+ q)
forgetReach+-trans oneℕ q = reflω
forgetReach+-trans (moreℕ p) q = congω more (forgetReach+-trans p q)

-- § Forget commutes with extension.
forgetReach+-extend :
{s t : DriftStateℕ} →
(p : s <ℕ t) →
forgetReach+ (Reach+ℕ-extend p) ≡ω Reach+-extend (forgetReach+ p)
forgetReach+-extend p =
transω (forgetReach+-trans p oneℕ) reflω

-- § Law 10.2: Admissible reach observables respect indexed transitivity.
law10-2-obsR-trans :
{Y : Setω} →
(a : AdmissibleReachObservable Y) →
{s t u : DriftStateℕ} →
(p : s <ℕ t) → (q : t <ℕ u) →
obsR a (Reach+ℕ-trans p q) ≡ω baseR a (Reach+-trans (forgetReach+ p) (forgetReach+ q))
law10-2-obsR-trans a p q =
transω (commR a (Reach+ℕ-trans p q)) (congω (baseR a) (forgetReach+-trans p q))

-- § Law 10.3: Admissible reach observables respect indexed extension.
law10-3-obsR-extend :
{Y : Setω} →
(a : AdmissibleReachObservable Y) →
{s t : DriftStateℕ} →
(p : s <ℕ t) →
obsR a (Reach+ℕ-extend p) ≡ω baseR a (Reach+-extend (forgetReach+ p))
law10-3-obsR-extend a p =
transω (commR a (Reach+ℕ-extend p)) (congω (baseR a) (forgetReach+-extend p))

```

Laws 10.4–10.6: Reach Observable Infrastructure

The following auxiliary definitions put admissible reach observables into the same normalisation framework used earlier for state observables.

```

-- § Indexed reach observable type.
IndexedReachObs : (Y : Setω) → Setω
IndexedReachObs Y = {s t : DriftStateℕ} → (s <ℕ t) → Y

-- § Base reach observable type.
BaseReachObs : (Y : Setω) → Setω
BaseReachObs Y = {s t : DriftState} → (s < t) → Y

-- § Pointwise equality on indexed reach observables.
infix 4 _≐Rω_ _≐BaseRω_
_≐Rω_ : {Y : Setω} → IndexedReachObs Y → IndexedReachObs Y → Setω
_≐BaseRω_ {Y} f g = {s t : DriftStateℕ} → (p : s <ℕ t) → f p ≡ω g p

```

```

_≐BaseRω_ : {Y : Setω} → BaseReachObs Y → BaseReachObs Y → Setω
_≐BaseRω_ {Y} f g = {s t : DriftState} → (p : s < t) → f p ≐ω g p

-- § Lift base reach to indexed reach.
liftBaseR : {Y : Setω} → BaseReachObs Y → IndexedReachObs Y
liftBaseR h p = h (forgetReach+ p)

-- § Pack a base reach observable as admissible.
packReach : {Y : Setω} → BaseReachObs Y → AdmissibleReachObservable Y
packReach h = record
{ obsR = liftBaseR h
; baseR = h
; commR = λ p → reflω
}

-- § Law 10.4: Indexed reach observables are unique given a base mediator.
law10-4-obsR-unique :
{Y : Setω} →
(h : BaseReachObs Y) →
(f1 f2 : IndexedReachObs Y) →
((s t : DriftState $\mathbb{N}$ ) (p : s < $\mathbb{N}$  t) → f1 p ≐ω h (forgetReach+ p))) →
((s t : DriftState $\mathbb{N}$ ) (p : s < $\mathbb{N}$  t) → f2 p ≐ω h (forgetReach+ p))) →
f1 ≐Rω f2
law10-4-obsR-unique h f1 f2 comm1 comm2 p =
transω (comm1 p) (symω (comm2 p))

-- § Law 10.5: Canonical indexed presentation for any admissible reach observable.
law10-5-canonical-pack :
{Y : Setω} →
(a : AdmissibleReachObservable Y) →
obsR a ≐Rω obsR (packReach (baseR a))
law10-5-canonical-pack a p = commR a p

-- § Law 10.6: baseR ∘ packReach is the identity.
law10-6-baseR-pack-id :
{Y : Setω} →
(h : BaseReachObs Y) →
baseR (packReach h) ≐BaseRω h
law10-6-baseR-pack-id h p = reflω

```

Laws 10.7–10.11: Reach Observational Normal Forms

Reach observables have the same normal-form problem as the earlier state observables. An admissible reach observable cannot depend on the particular indexed derivation by which reachability is presented; it must factor through the base reach relation. The normalizer `canonReach` performs exactly this operation: extract the base observable and pack it back into indexed form.

The laws below record the consequences. Canonical packing is sound, idempotent up to observational equality, and universal among sound normalizers. Any other normalization procedure satisfying the same soundness condition agrees observationally with `canonReach`.

```

infix 4 _≈ReachObs_
_≈ReachObs_ : {Y : Setω} → AdmissibleReachObservable Y → AdmissibleReachObservable Y → Setω
_≈ReachObs_ a b = obsR a ≐Rω obsR b

-- § ≈ReachObs reflexivity, symmetry, transitivity.
≈ReachObs-refl : {Y : Setω} → (a : AdmissibleReachObservable Y) → a ≈ReachObs a
≈ReachObs-refl a p = reflω

≈ReachObs-sym : {Y : Setω} → {a b : AdmissibleReachObservable Y} → a ≈ReachObs b → b ≈ReachObs a
≈ReachObs-sym p q = symω (p q)

≈ReachObs-trans : {Y : Setω} → {a b c : AdmissibleReachObservable Y} → a ≈ReachObs b → b ≈ReachObs c → a ≈ReachObs c
≈ReachObs-trans p q r = transω (p r) (q r)

-- § Canonical reach normal form.
canonReach : {Y : Setω} → AdmissibleReachObservable Y → AdmissibleReachObservable Y
canonReach a = packReach (baseR a)

-- § Law 10.7: Canonical reach packing is observationally sound.
law10-7-canonReach-sound :
  {Y : Setω} →
  (a : AdmissibleReachObservable Y) →
  a ≈ReachObs canonReach a
law10-7-canonReach-sound a = law10-5-canonical-pack a

-- § Law 10.8: Canonical reach packing is idempotent up to observation.
law10-8-canonReach-idem :
  {Y : Setω} →
  (a : AdmissibleReachObservable Y) →
  canonReach (canonReach a) ≈ReachObs canonReach a
law10-8-canonReach-idem a =
  let
    b = canonReach a
  in
  λ p → symω ((law10-7-canonReach-sound b) p)

-- § Canonical reach observable type.
CanonicalReachObs : (Y : Setω) → Setω
CanonicalReachObs Y = BaseReachObs Y

-- § Reach observational iso record.
record ReachObsIso (Y : Setω) : Setω where
  field
    to   : AdmissibleReachObservable Y → CanonicalReachObs Y
    from : CanonicalReachObs Y → AdmissibleReachObservable Y
    to-from : (h : CanonicalReachObs Y) → to (from h) ≐BaseRω h
    from-to : (a : AdmissibleReachObservable Y) → a ≈ReachObs from (to a)

open ReachObsIso public

-- § Law 10.9: Observational collapse as an explicit iso structure.
law10-9-reachObsIso : {Y : Setω} → ReachObsIso Y
law10-9-reachObsIso {Y} = record
  { to   = baseR
  ; from = packReach

```

```

; to-from = law10-6-baseR-pack-id
; from-to = law10-7-canonReach-sound
}

-- § Law 10.10: Any ReachObsIso normalization agrees with canonReach.
law10-10-reachObsIso-normalizes :
{Y : Setω} →
(i : ReachObsIso Y) →
(a : AdmissibleReachObservable Y) →
ReachObsIso.from i (ReachObsIso.to i a) ≈ReachObs canonReach a
law10-10-reachObsIso-normalizes i a =
≈ReachObs-trans
{a = ReachObsIso.from i (ReachObsIso.to i a)}
{b = a}
{c = canonReach a}
(≈ReachObs-sym {a = a} {b = ReachObsIso.from i (ReachObsIso.to i a)} (ReachObsIso.from-to i a))
(law10-7-canonReach-sound a)

-- § Reach normalizer record.
record ReachObsNormalizer (Y : Setω) : Setω where
field
norm : AdmissibleReachObservable Y → AdmissibleReachObservable Y
sound : (a : AdmissibleReachObservable Y) → a ≈ReachObs norm a
idem : (a : AdmissibleReachObservable Y) → norm (norm a) ≈ReachObs norm a

open ReachObsNormalizer public

-- § Law 10.11: Any sound reach normalizer agrees with canonReach.
law10-11-reach-normalizer-unique :
{Y : Setω} →
(n : ReachObsNormalizer Y) →
(a : AdmissibleReachObservable Y) →
norm n a ≈ReachObs canonReach a
law10-11-reach-normalizer-unique n a =
≈ReachObs-trans
{a = norm n a}
{b = a}
{c = canonReach a}
(≈ReachObs-sym {a = a} {b = norm n a} (sound n a))
(law10-7-canonReach-sound a)

```

Chapter 23

Ranking Principle

The ranking principle is the standard constructive obstruction to a cycle. If every step strictly increases a natural-number rank, then no finite path can return to its starting point: such a return would force $\text{succ}(\text{rank } s) \leq \text{rank } s$, which contradicts the order structure of $\mathbb{N}$.

The record `DriftRank` isolates the only data needed for this argument. It assigns a rank to each drift state and proves that one application of `stepState` raises that rank strictly. The following laws extend the one-step estimate along positive reachability and then use it to rule out cyclic derivations.

```
-- § Ranking record.
record DriftRank : Set $\omega$  where
  field
    rank : DriftState  $\rightarrow$   $\mathbb{N}$ 
    rank-step : (s : DriftState)  $\rightarrow$  succ (rank s)  $\leq$  rank (stepState s)

open DriftRank public
```

Law 11.0: Rank Is Monotone Along Reach⁺

Positive reachability is generated by one or more applications of `stepState`. The rank inequality is therefore propagated by induction over the reachability witness: one step uses `rank-step`, and a longer path composes that estimate with the induction hypothesis.

```
-- § Law 11.0: Rank is monotone along Reach+.
law11-0-rank-mono : (r : DriftRank)  $\rightarrow$  {s t : DriftState}  $\rightarrow$  (s  $\prec$  t)  $\rightarrow$  rank r s  $\leq$  rank r t
law11-0-rank-mono r {s} one =
   $\leq$ -trans ( $\leq$ -step (rank r s)) (rank-step r s)
law11-0-rank-mono r {s} (more p) =
   $\leq$ -trans
    ( $\leq$ -trans ( $\leq$ -step (rank r s)) (rank-step r s))
```

```

(law11-0-rank-mono r p)

-- § Strict monotonicity.
law11-0-rank-increases : (r : DriftRank) → {s t : DriftState} → (s < t) → suc (rank r s) ≤ rank r t
law11-0-rank-increases r {s} one = rank-step r s
law11-0-rank-increases r {s} (more p) =
  ≤-trans (rank-step r s) (law11-0-rank-mono r p)

```

Law 11.1: Ranking Forces Acyclicity

Once monotonicity has been established, acyclicity follows immediately: a cycle would force a strict successor inequality back onto the same natural number.

```

-- § Law 11.1: Ranking forces DriftAcyclic.
law11-1-ranked-acyclic : DriftRank → DriftAcyclic
law11-1-ranked-acyclic r =
  record
  { no-cycle =
    λ s p →
      suc ≤-impossible (rank r s) (law11-0-rank-increases r p)
  }

```

Chapter 24

Presentation Fixpoint

The earlier normalization results show that admissible presentations collapse to canonical ones after the relevant factorization hypotheses are imposed. This chapter uses that result at the level of observables: once presentation-dependent residue has been removed, every admissible meta-observable factors through the same canonical extractor.

Global Observable Elimination

The point is to isolate what depends only on the invariant structure and what depends on the particular way that structure has been presented. The factorization statements below express this formally.

Schematically, the claim is that every admissible meta-observable factors through the canonical invariant basis:

$$\text{ObservableElim} : (\text{admissible } K_4\text{-observable}) \rightarrow (\text{canonical invariant basis} \rightarrow Y),$$

The figure below illustrates this elimination of presentation-dependent bookkeeping.

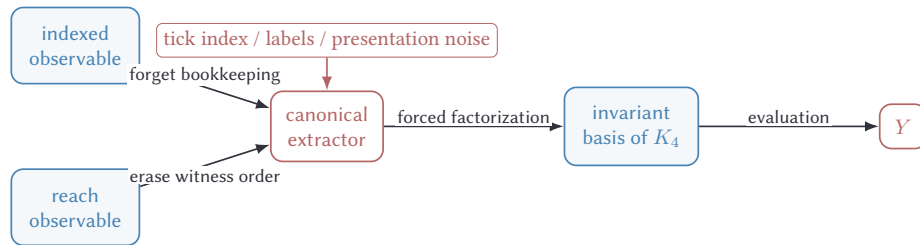


Figure 24.1: Admissible observables factor through the canonical invariant basis once presentation-dependent bookkeeping has been removed.

```

-- § Respecting  $\approx\text{Obs}$ .
record Respects $\approx\text{Obs}$  {Y Z : Set $\omega$ } (F : AdmissibleObservable Y  $\rightarrow$  Z) : Set $\omega$  where
field
  respects : {a b : AdmissibleObservable Y}  $\rightarrow$  a  $\approx\text{Obs}$  b  $\rightarrow$  F a  $\equiv\omega$  F b

open Respects $\approx\text{Obs}$  public

-- § Factorization record for state meta-observables.
record FactorsThroughObserve {Y Z : Set $\omega$ } (F : AdmissibleObservable Y  $\rightarrow$  Z) : Set $\omega$  where
field
  g : (DriftState  $\rightarrow$  Y)  $\rightarrow$  Z
  comm : (a : AdmissibleObservable Y)  $\rightarrow$  F a  $\equiv\omega$  g (observe a)

open FactorsThroughObserve public

-- § Law 12.0: Meta-observables factor through observe.
law12-0-meta-observe-factor :
{Y Z : Set $\omega$ }  $\rightarrow$ 
(F : AdmissibleObservable Y  $\rightarrow$  Z)  $\rightarrow$ 
Respects $\approx\text{Obs}$  F  $\rightarrow$ 
FactorsThroughObserve F
law12-0-meta-observe-factor F r =
record
{ g =  $\lambda$  h  $\rightarrow$  F (packObs h)
; comm =  $\lambda$  a  $\rightarrow$ 
  respects r (law9-10-canonObs-sound a)
}

-- § Respecting  $\approx\text{ReachObs}$ .
record Respects $\approx\text{ReachObs}$  {Y Z : Set $\omega$ } (F : AdmissibleReachObservable Y  $\rightarrow$  Z) : Set $\omega$  where
field
  respectsR : {a b : AdmissibleReachObservable Y}  $\rightarrow$  a  $\approx\text{ReachObs}$  b  $\rightarrow$  F a  $\equiv\omega$  F b

open Respects $\approx\text{ReachObs}$  public

-- § Factorization record for reach meta-observables.
record FactorsThroughBaseR {Y Z : Set $\omega$ } (F : AdmissibleReachObservable Y  $\rightarrow$  Z) : Set $\omega$  where
field
  gR : BaseReachObs Y  $\rightarrow$  Z
  commR : (a : AdmissibleReachObservable Y)  $\rightarrow$  F a  $\equiv\omega$  gR (baseR a)

open FactorsThroughBaseR public

-- § Law 12.1: Meta-observables factor through baseR.
law12-1-meta-baseR-factor :
{Y Z : Set $\omega$ }  $\rightarrow$ 
(F : AdmissibleReachObservable Y  $\rightarrow$  Z)  $\rightarrow$ 
Respects $\approx\text{ReachObs}$  F  $\rightarrow$ 
FactorsThroughBaseR F
law12-1-meta-baseR-factor F r =
record
{ gR =  $\lambda$  h  $\rightarrow$  F (packReach h)
; commR =  $\lambda$  a  $\rightarrow$ 
  respectsR r (law10-7-canonReach-sound a)
}

```


Chapter 25

Graph Presentation

At this point the surviving combinatorial object can be presented explicitly. The four mutually distinct endomorphism classes supply four vertices, and distinctness itself supplies the edge relation. Because every pair of cases is distinct, every pair of vertices is connected. The result is the complete graph K_4 .

The numerical invariants of this graph are assembled in the next chapter. Before introducing that ledger, the graph itself is defined formally and its uniqueness up to isomorphism is made explicit.

```
-- § Graph record.
record Graph : Set1 where
  field
    V : Set
    Edge : V → V → Set
    edge-sym : {a b : V} → Edge a b → Edge b a
    edge-irr : (a : V) → Edge a a → ⊥

open Graph public

-- § Graph isomorphism record.
record GraphIso (G H : Graph) : Set1 where
  field
    to      : V G → V H
    from    : V H → V G
    to-from : (y : V H) → to (from y) ≡ y
    from-to : (x : V G) → from (to x) ≡ x
    edge-to : {a b : V G} → Edge G a b → Edge H (to a) (to b)
    edge-from : {a b : V H} → Edge H a b → Edge G (from a) (from b)

open GraphIso public

-- § Transport inequality across equalities.
transport≠ :
  {A : Set} →
  {a a' b b' : A} →
```

```

a ≡ a' →
b ≡ b' →
(a ≠ b) →
(a' ≠ b')
transport≠ ea eb neq eq' = neq (trans ea (trans eq' (sym eb)))

```

Law 13.0: The Canonical K_4 Graph Has Edge = Inequality

The canonical K_4 graph is constructed with EndoCase as its vertex set and inequality as its edge relation. Symmetry and irreflexivity of the edge relation are immediate from the corresponding properties of $\neq$.

```

-- § Law 13.0: The canonical K4 graph.
K4GraphCanonical : Graph
K4GraphCanonical = record
{ V = EndoCase
; Edge = λ a b → a ≠ b
; edge-sym = λ {a} {b} neq eq → neq (sym eq)
; edge-irr = λ a loop → loop refl
}

-- § K4 graph presentation record.
record K4GraphPresentation : Set1 where
field
  Vp      : Set
  toV     : Vp → EndoCase
  fromV   : EndoCase → Vp
  to-from : (c : EndoCase) → toV (fromV c) ≡ c
  from-to : (v : Vp) → fromV (toV v) ≡ v

open K4GraphPresentation public

-- § Present a graph from a K4 graph presentation.
presentedGraph : K4GraphPresentation → Graph
presentedGraph p = record
{ V = Vp p
; Edge = λ v w → toV p v ≠ toV p w
; edge-sym = λ {a} {b} neq eq → neq (sym eq)
; edge-irr = λ a loop → loop refl
}

```

Law 13.1: Any K_4 Graph Presentation Collapses To The Canonical Graph Up To Iso

Any other presentation of the same four vertices is obtained by relabelling them. Once a bijection with EndoCase has been fixed, the edge relation is transported along that bijection, and the resulting graph is isomorphic to the standard one.

Figure 25.1 depicts this relabelling principle.

```
-- § Law 13.1: Presentation iso.
law13-1-presentation-iso : (p : K4GraphPresentation) → GraphIso (presentedGraph p) K4GraphCanonical
law13-1-presentation-iso p = record
{ to = toV p
; from = fromV p
; to-from = to-from p
; from-to = from-to p
; edge-to = λ {a} {b} e → e
; edge-from = λ {a} {b} e →
    transport≠ (sym (to-from p a)) (sym (to-from p b)) e
}
```

The obstruction to K_3 is purely finite and combinatorial: three vertices cannot host four pairwise distinct cases. The proof below is the corresponding pigeonhole argument carried out explicitly for the three-element type Three.

```
-- § The three-element type: the candidate vertex set for  $K_3$ .
data Three : Set where
  v1 : Three
  v2 : Three
  v3 : Three

-- § Type-level closure: Three has exactly three inhabitants.
Three-exhaustive :
 $\forall (t : \text{Three}) \rightarrow (t \equiv v1) \uplus (t \equiv v2) \uplus (t \equiv v3)$ 
Three-exhaustive v1 = inj1 refl
Three-exhaustive v2 = inj2 (inj1 refl)
Three-exhaustive v3 = inj2 (inj2 refl)

-- § Pigeonhole on Three: four pairwise distinct values do not fit.
no-four-distinct-Three :
 $(a \ b \ c \ d : \text{Three}) \rightarrow$ 
 $a \neq b \rightarrow a \neq c \rightarrow a \neq d \rightarrow b \neq c \rightarrow b \neq d \rightarrow c \neq d \rightarrow \perp$ 
no-four-distinct-Three v1 v1 _ _ _ _ = nab refl
no-four-distinct-Three v2 v2 _ _ _ _ = nab refl
```

```

no-four-distinct-Three v3 v3 _ _ nab _ _ _ = nab refl
no-four-distinct-Three v1 v2 v1 _ _ nac _ _ _ = nac refl
no-four-distinct-Three v1 v2 v2 _ _ _ nbc _ _ _ = nbc refl
no-four-distinct-Three v1 v2 v3 v1 _ _ nad _ _ _ = nad refl
no-four-distinct-Three v1 v2 v3 v2 _ _ _ nbd _ _ _ = nbd refl
no-four-distinct-Three v1 v2 v3 v3 _ _ _ ncd = ncd refl
no-four-distinct-Three v1 v3 v1 _ _ _ nac _ _ _ = nac refl
no-four-distinct-Three v1 v3 v3 _ _ _ nbc _ _ _ = nbc refl
no-four-distinct-Three v1 v3 v2 v1 _ _ nad _ _ _ = nad refl
no-four-distinct-Three v1 v3 v2 v2 _ _ _ ncd = ncd refl
no-four-distinct-Three v1 v3 v2 v3 _ _ _ nbd _ _ _ = nbd refl
no-four-distinct-Three v2 v1 v2 _ _ _ nac _ _ _ = nac refl
no-four-distinct-Three v2 v1 v1 _ _ _ nbc _ _ _ = nbc refl
no-four-distinct-Three v2 v1 v3 v2 _ _ nad _ _ _ = nad refl
no-four-distinct-Three v2 v1 v3 v1 _ _ _ nbd _ _ _ = nbd refl
no-four-distinct-Three v2 v1 v3 v3 _ _ _ ncd = ncd refl
no-four-distinct-Three v2 v3 v2 _ _ _ nac _ _ _ = nac refl
no-four-distinct-Three v2 v3 v3 _ _ _ nbc _ _ _ = nbc refl
no-four-distinct-Three v2 v3 v1 v2 _ _ nad _ _ _ = nad refl
no-four-distinct-Three v2 v3 v1 v3 _ _ _ nbd _ _ _ = nbd refl
no-four-distinct-Three v2 v3 v1 v1 _ _ _ ncd = ncd refl
no-four-distinct-Three v3 v1 v3 _ _ _ nac _ _ _ = nac refl
no-four-distinct-Three v3 v1 v1 _ _ _ nbc _ _ _ = nbc refl
no-four-distinct-Three v3 v1 v2 v3 _ _ nad _ _ _ = nad refl
no-four-distinct-Three v3 v1 v2 v1 _ _ _ nbd _ _ _ = nbd refl
no-four-distinct-Three v3 v1 v2 v2 _ _ _ ncd = ncd refl
no-four-distinct-Three v3 v2 v3 _ _ _ nac _ _ _ = nac refl
no-four-distinct-Three v3 v2 v2 _ _ _ nbc _ _ _ = nbc refl
no-four-distinct-Three v3 v2 v1 v3 _ _ nad _ _ _ = nad refl
no-four-distinct-Three v3 v2 v1 v2 _ _ _ nbd _ _ _ = nbd refl
no-four-distinct-Three v3 v2 v1 v1 _ _ _ ncd = ncd refl

```

-- § Law 13.2: No injection from EndoCase into a three-element type.

law13-2-K3-fails :

```

(f : EndoCase → Three) →
((x y : EndoCase) → f x ≡ f y → x ≡ y) →
⊥

```

law13-2-K3-fails f inj =

```

no-four-distinct-Three
(f case-constL) (f case-constR) (f case-id) (f case-dual)
(λ eq → distinct-cL-cR (inj _ _ eq))
(λ eq → distinct-cL-id (inj _ _ eq))
(λ eq → distinct-cL-d (inj _ _ eq))
(λ eq → distinct-cR-id (inj _ _ eq))
(λ eq → distinct-cR-d (inj _ _ eq))
(λ eq → distinct-id-d (inj _ _ eq))

```

where

```

distinct-cL-cR : case-constL ≡ case-constR → ⊥
distinct-cL-cR ()
distinct-cL-id : case-constL ≡ case-id → ⊥
distinct-cL-id ()
distinct-cL-d : case-constL ≡ case-dual → ⊥
distinct-cL-d ()

```

```

distinct-cR-id : case-constR  $\equiv$  case-id  $\rightarrow \perp$ 
distinct-cR-id ()
distinct-cR-d : case-constR  $\equiv$  case-dual  $\rightarrow \perp$ 
distinct-cR-d ()
distinct-id-d : case-id  $\equiv$  case-dual  $\rightarrow \perp$ 
distinct-id-d ()

```

Law 13.3: K_5 Cannot Arise

The opposite direction is equally simple: the four cases already exhaust the classification, so there is no fifth vertex to add. The next lemma packages that observation in predicate form.

```

-- § Law 13.3: no fifth EndoCase distinct from the four classified.
law13-3-K5-fails :
(P : EndoCase  $\rightarrow$  Set)  $\rightarrow$ 
(P case-id  $\rightarrow \perp$ )  $\rightarrow$ 
(P case-dual  $\rightarrow \perp$ )  $\rightarrow$ 
(P case-constL  $\rightarrow \perp$ )  $\rightarrow$ 
(P case-constR  $\rightarrow \perp$ )  $\rightarrow$ 
(c : EndoCase)  $\rightarrow$  P c  $\rightarrow \perp$ 
law13-3-K5-fails _  $\neg$ id _ _ _ case-id pid =  $\neg$ id pid
law13-3-K5-fails _ _  $\neg$ dl _ _ case-dual pdl =  $\neg$ dl pdl
law13-3-K5-fails _ _ _  $\neg$ cL _ case-constL pcl =  $\neg$ cL pcl
law13-3-K5-fails _ _ _ _  $\neg$ cR case-constR pcr =  $\neg$ cR pcr

```


Chapter 26

The K_4 Invariant Record

With the graph now fixed, the next task is to collect its basic combinatorial and spectral data into a single invariant record. At this point the abstract classification results begin to take explicit numerical form.

What the Graph Contains

Several entries of this ledger are immediate. The graph has four vertices, every pair of distinct vertices is connected, and therefore the edge count is

$$\binom{4}{2} = 6.$$

Viewed as the boundary of a tetrahedron, it also carries the familiar Euler data that will be used in the numerical constructions below.

The Spectral Invariant

The same graph also determines the spectral quantity used later. From the Euler identity one obtains

$$V - E + F = 4 - 6 + 4 = 2,$$

and hence

$$\mathcal{C} = V^d \cdot \chi + d^2 = 4^3 \cdot 2 + 3^2 = 128 + 9 = 137.$$

Figure 26.1 gathers the ledger visually: the same tetrahedral survivor carries the vertex count, edge count, Euler data, and spectral split from which the later numerical invariants are assembled.

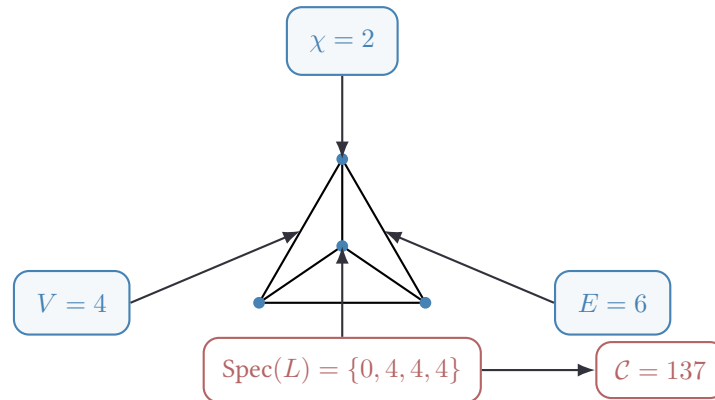


Figure 26.1: The basic combinatorial and spectral data of K_4 assembled into the invariant ledger.

What Follows, and Why

Two further moves remain before this ledger can be read as numerical output. The first is structural: the next chapter re-reads the ledger as the description of an extension structure, on the simple ground that distinction once made cannot be undone, and that nothing in the surviving graph singles out a preferred direction. The second is arithmetic: the part that follows develops the algebraic tools needed to evaluate the invariants the ledger now carries. The order is not optional. The arithmetic operates on quantities whose meaning has already been fixed by the spatial reading.

Chapter 27

Forced Spatial Extension

The invariant ledger of K_4 has been fixed inside the formal representative. What structural extension reading it supports, however, has not yet been read off. The four numbers V , E , d , χ are not a tuple of independent quantities a posteriori arranged on a graph: they are the count, the connectivity, the dimensionality, and the closure condition of one and the same surviving object. This chapter names that object as structural extension. The naming is constrained by premises already proved; it is not a metric or physical interpretation.

The Order of Forcing

Three premises have been carried this far without being assembled. Non-collapse of distinction (Chapter 15) forbids the re-identification of what has been told apart. Drift non-triviality (Chapter 14) forbids that “differs from” is a single binary flag: there are several independent ways in which two configurations may differ, and their count is precisely d . The symmetry of K_4 (the orbit theorems of Chapter 25) forbids that any vertex, edge, or independent direction is structurally distinguishable from any other.

Read singly, each of these is an elimination theorem on the survivor. Read together, they constrain a single object whose name is not optional. The remainder of this chapter unfolds the constraint to its unique fixed point.

The survivor must be compatible with non-collapse, with drift non-triviality, and with full $\text{Aut}(K_4)$ -symmetry, simultaneously and at every successor step. A passive set of V^d tuples is excluded by non-collapse iterated through time; a one-dimensional or tree-like extension is excluded by drift non-triviality; an extension with a preferred direction is excluded by symmetry. What remains is an object equipped with d independent monotone directions in which extension is permitted, with no direction marked. The remaining sections of this chapter prove each elimination explicitly.

Irreversibility from Non-Collapse

Non-collapse, as proved earlier, states that once $\ell \neq r$ has been told apart, the two sides cannot be re-identified. The verb *cannot* carries a temporal load that the proof itself already satisfies but that has not yet been named. To say that two distinguished sides remain distinguished is not a claim about the moment of distinction; it is a claim about every later moment as well. A reading that confined non-collapse to a single instant would empty it: a distinction that may be revoked at the next step is not a distinction.

The successor relation in the corpus is propositional. Decidable inequality $\text{Dec}(x \equiv y)$ is a propositional statement, and propositional truth values are not mutable: a proof witness, once constructed, persists through every later context in which it can be referred to. Irreversibility of distinction is therefore not an additional axiom but the unavoidable form non-collapse takes inside a type theory that records its negations propositionally. The set of distinctions made is monotone in any successor relation the system admits.

Non-collapse must hold uniformly over all successor steps, not only at the moment of distinction. The reading “non-collapse holds in the moment of distinction but may be revoked later” is excluded by the propositional character of $\text{Dec}(x \equiv y)$ already proved. A revocable inequality is not an inequality at all. The configuration set $\mathcal{C}$ of distinguishable states is therefore monotone under any successor relation; it may grow, it cannot shrink. This monotonicity is recorded in what follows as a partial order on $\mathcal{C}$.

Symmetry Forbids a Preferred Direction

Drift non-triviality leaves d independent directions in which two configurations may differ. Standing alone, this could still be compatible with a structure in which one of those directions is distinguished by the dynamics from the others: a tree, a fibration, or any object with a preferred axis would qualify. The symmetry of K_4 forbids exactly this.

The orbit theorems prove that $\text{Aut}(K_4) \cong S_4$ acts transitively on the vertices and on the edges of the surviving graph. The transitive action extends to the independent directions of drift: each pair of directions is connected by a graph automorphism. Suppose, for contradiction, that the successor relation singled out one direction $\mathbf{e}_*$ over the others. Then “the preferred direction” would be a structural invariant of the system: a feature the dynamics distinguishes from any other direction. The automorphism that maps $\mathbf{e}_*$ to a non-preferred $\mathbf{e}_j$ would have to preserve every structural feature; under it, the preferred direction would now be $\mathbf{e}_j$. Either, then, the preference is genuine and breaks S_4 , or it was never structural to begin with.

The first horn is forbidden by the orbit theorems. Only the second survives. Isotropy of extension is therefore a derived consequence of the symmetry already proved, not a separate postulate.

No direction of drift may be marked by a feature that survives $\text{Aut}(K_4)$ -symmetry. A structure with a distinguished extension axis is excluded by transitivity of the orbit action on directions. The candidate “preferred direction” would have to be either a structural invariant, which the orbit theorem forbids, or a labelling, which is not a structural fact at all. Extension is therefore permitted equally along each of the d independent directions. No direction may be silently distinguished in any later construction.

The Surviving Object

The three sections above each eliminate alternatives to a single object. The object that survives all three eliminations is exhibited here.

A monotone configuration set with d independent directions of extension and no preferred direction among them is not a tree, not a chain, and not a passive cardinality. It is a d -dimensional extension structure: it has at least V^d distinguishable configurations (one for each independent assignment of states across d directions), it carries a partial order in which each successor adds rather than removes a distinction, and its symmetry group is exactly the symmetry group of K_4 that produced it. The book has, since the elimination of K_5 , called this surviving object the graph K_4 . The present chapter shows that the same object also admits, under the premises just collected, the reading: *the minimal isotropic, monotone, d -dimensional extension in which distinction may live*.

The two readings are not in competition. The graph reading exhibits K_4 as a finite combinatorial object. The extension reading exhibits K_4 as the carrier of d independent monotone directions. Both readings concern the same survivor; they fix different aspects of it. Subsequent chapters use whichever aspect a particular argument requires. Numerical evaluation in the form $V^d \cdot \chi + d^2$ will require both: the count of independent configurations along the d extension directions (V^d), the closure condition under which the extension is well-defined (χ), and the cross-direction correction term that the extension itself carries on its boundary (d^2).

The object that survives non-collapse, drift non-triviality, and isotropy is simultaneously a finite graph and a d -dimensional extension structure with $\text{Aut} = S_4$. No separate construction is needed. The graph K_4 already proven in Chapter 25 admits the extension reading because its three ingredients — non-collapse, drift non-triviality, isotropy — are precisely the ingredients used to characterise it as a graph.

The four invariants of the ledger are therefore not four independent numbers but the count, the connectivity, the dimensionality, and the closure condition of one struc-

ture. The arithmetic that follows operates on this structure, not on a tuple.

What This Reading Does Not Smuggle In

The extension reading is structural, not metric, not analytic, and not interpretive. Four readings are explicitly *not* added by it.

A metric: the extension structure carries neighbour relations along each of the d directions, but no distance function is defined on configurations. Whether two configurations are “far” or “near” is, at this stage, undefined.

A continuous topology: the extension structure is at most a graph, or a discrete poset of configurations. Continuity is not introduced here, and would require a separate limit construction not undertaken in this volume.

An interpretive physical reading: the extension structure is invariant under the choice of what its configurations are. They may later be read as physical states, types in a type theory, objects in a category, or anything else that carries the three premises of §27. The companion volume *Form* undertakes one such interpretation; this volume does not.

A scalar value: the integer $\mathcal{C} = 137$ does not yet appear at the level of this chapter. It is a particular invariant of the extension structure, evaluated by the arithmetic of the next part. The fact that the formula has the form $V^d \cdot \chi + d^2$ rather than another arithmetic combination is fixed inside the kind/formula framework — Chapter 38 closes that question — but the closure occurs after the arithmetic infrastructure is in place.

The extension reading must not silently confer metric, topological, or interpretive physical structure on the survivor. Four temptations are excluded explicitly: a distance function, a continuum, a physical reading, and a numerical target. Each would require an independent derivation that is not present at this stage. Every later use of the extension reading rests exactly on what has been proved here — monotonicity, isotropy, d -dimensional extension — and on nothing else.

Part V

Derived Arithmetic

With the graph-theoretic core in place, the development turns to arithmetic. The strategy is cumulative: first the unique generator of counting — the well-founded closure of one-fold branching over the binary distinction — then finite index types and truth values, then natural numbers, integers, and finally the algebraic operations required for later invariant computations.

Chapter 28

The Naturals as a W-Type over Two

The two-element type is the only generator the file has been granted. If counting were a parallel axiomatic structure, the natural numbers would stand beside `Two` as a second, independent assumption, and the forcing chain would split. The chain does not split. Every finite count is a well-founded tree whose only branching alphabet is the very distinction the file began with: `L` marks termination (no further child to count), `R` marks continuation (one further child to count). Counting is not granted alongside the distinction; it is the wellfounded closure of one-fold branching over it.

The minimal foundation admits exactly one generating type, `Two`. A second independent inductive type for the naturals would break that minimality and turn counting into a parallel axiom rather than a consequence. The W-type over `Two` with branching shape $L \mapsto \perp, R \mapsto \mathbf{1}$ is well-defined inside MLTT. Its zero/successor encoding produces a forward map from the Peano naturals introduced in Chapter 20 into this W-encoding; the inverse is structural recursion on `W`; the round-trip on the Peano side holds by `refl` after a single `cong suc`; the round-trip on the W-side holds up to a structural equivalence that does not require function extensionality.

The naturals are recovered from `Two` alone, by W-recursion, with no further inductive type added to the foundation. Every later use of $\mathbb{N}$ in the file may be read through this isomorphism, as a use of the W-encoding it stands for.

```
-- § Local unit type:  $\top$  is introduced in the next chapter; we name the
-- W-branching unit One here to avoid forward dependence and to mark
-- its role as the single child of the R-branch explicitly.
data One : Set where
  one : One

-- § The W-type: well-founded trees of shape A with branching B.
data W (A : Set) (B : A → Set) : Set where
```

```

sup : (a : A) → (B a → W A B) → W A B

-- § Branching shape forced by Two: L has no children, R has exactly one.
N-shape : Two → Set
N-shape L = ⊥
N-shape R = One

-- § N as a W-type over Two.
NW : Set
NW = W Two N-shape

-- § Zero at the W-level: an L-shaped sup with no children.
zeroW : NW
zeroW = sup L (λ ())

-- § Successor at the W-level: an R-shaped sup with one child.
sucW : NW → NW
sucW n = sup R (λ _ → n)

-- § Forward map: Peano N to W-encoded NW.
N→NW : N → NW
N→NW zero = zeroW
N→NW (suc n) = sucW (N→NW n)

-- § Backward map: W-encoded NW to Peano N.
NW→N : NW → N
NW→N (sup L _) = zero
NW→N (sup R k) = suc (NW→N (k one))

-- § Round-trip on the Peano side: identity by structural induction.
N→NW→N : (n : N) → NW→N (N→NW n) ≡ n
N→NW→N zero = refl
N→NW→N (suc n) = cong suc (N→NW→N n)

-- § Structural equivalence on NW: identifies W-trees by shape and
-- by the unique child of each R-node, side-stepping the funext
-- obligation on absurd L-children.
data _≈NW_ : NW → NW → Set where
  z≈z : {k1 k2 : N-shape L → NW} → sup L k1 ≈NW sup L k2
  s≈s : {k1 k2 : N-shape R → NW} → k1 one ≈NW k2 one → sup R k1 ≈NW sup R k2

-- § Reflexivity of the structural equivalence.
≈NW-refl : (w : NW) → w ≈NW w
≈NW-refl (sup L _) = z≈z
≈NW-refl (sup R k) = s≈s (≈NW-refl (k one))

-- § Round-trip on the W-side: identity up to structural equivalence.
NW→N→NW : (w : NW) → N→NW (NW→N w) ≈NW w
NW→N→NW (sup L _) = z≈z
NW→N→NW (sup R k) = s≈s (NW→N→NW (k one))

```

The forward and backward maps are mutually inverse: on the Peano side strictly so, on the W-side up to the unavoidable structural equivalence. The arithmetic that follows is therefore not a parallel axiomatisation of counting but the same counting structure read off the only generator the file ever assumed.

Chapter 29

Arithmetic Infrastructure

This chapter gathers the elementary discrete ingredients used throughout the arithmetic part of the book. Most of them are standard, but they are written out explicitly so that later constructions can be traced back to concrete definitions inside the same formal development.

Truth and Finite Indices

The first ingredients are the unit type and the finite index types used to enumerate neighbours and vertices of K_4 . Their role is elementary but important: later sums, recursions, and case analyses are finite, and decidable equality on these bounded types makes that finiteness explicit.

```
-- § Unit type: trivial truth witness.
data T : Set where
  tt : T

-- § Three-element index (neighbours of a  $K_4$  vertex).
data Fin3 : Set where
  f0 f1 f2 : Fin3

-- § Inequality on Fin3.
Fin3≠ : (i j : Fin3) → Set
Fin3≠ i j = i ≡ j → ⊥

f0≠f1 : Fin3≠ f0 f1
f0≠f1 ()

f0≠f2 : Fin3≠ f0 f2
f0≠f2 ()

f1≠f2 : Fin3≠ f1 f2
f1≠f2 ()
```

```

-- § Decidable equality on Fin3.
Fin3-decEq : (i j : Fin3) → (i ≡ j) ⊔ (Fin3≠ i j)
Fin3-decEq f0 f0 = inj₁ refl
Fin3-decEq f1 f1 = inj₁ refl
Fin3-decEq f2 f2 = inj₁ refl
Fin3-decEq f0 f1 = inj₂ f0≠f1
Fin3-decEq f1 f0 = inj₂ (λ e → f0≠f1 (sym e))
Fin3-decEq f0 f2 = inj₂ f0≠f2
Fin3-decEq f2 f0 = inj₂ (λ e → f0≠f2 (sym e))
Fin3-decEq f1 f2 = inj₂ f1≠f2
Fin3-decEq f2 f1 = inj₂ (λ e → f1≠f2 (sym e))

-- § Four-element index (vertices of K₄).
data Fin4 : Set where
  g0 g1 g2 g3 : Fin4

-- § Inequality on Fin4.
Fin4≠ : (i j : Fin4) → Set
Fin4≠ i j = i ≡ j → ⊥

g0≠g1 : Fin4≠ g0 g1
g0≠g1 ()

g0≠g2 : Fin4≠ g0 g2
g0≠g2 ()

g0≠g3 : Fin4≠ g0 g3
g0≠g3 ()

g1≠g2 : Fin4≠ g1 g2
g1≠g2 ()

g1≠g3 : Fin4≠ g1 g3
g1≠g3 ()

g2≠g3 : Fin4≠ g2 g3
g2≠g3 ()

-- § Decidable equality on Fin4.
Fin4-decEq : (i j : Fin4) → (i ≡ j) ⊔ (Fin4≠ i j)
Fin4-decEq g0 g0 = inj₁ refl
Fin4-decEq g1 g1 = inj₁ refl
Fin4-decEq g2 g2 = inj₁ refl
Fin4-decEq g3 g3 = inj₁ refl
Fin4-decEq g0 g1 = inj₂ g0≠g1
Fin4-decEq g1 g0 = inj₂ (λ e → g0≠g1 (sym e))
Fin4-decEq g0 g2 = inj₂ g0≠g2
Fin4-decEq g2 g0 = inj₂ (λ e → g0≠g2 (sym e))
Fin4-decEq g0 g3 = inj₂ g0≠g3
Fin4-decEq g3 g0 = inj₂ (λ e → g0≠g3 (sym e))
Fin4-decEq g1 g2 = inj₂ g1≠g2
Fin4-decEq g2 g1 = inj₂ (λ e → g1≠g2 (sym e))
Fin4-decEq g1 g3 = inj₂ g1≠g3
Fin4-decEq g3 g1 = inj₂ (λ e → g1≠g3 (sym e))
Fin4-decEq g2 g3 = inj₂ g2≠g3
Fin4-decEq g3 g2 = inj₂ (λ e → g2≠g3 (sym e))

```

Natural Number Extensions and Boolean Values

The next block introduces the basic arithmetic and Boolean operations on $\mathbb{N}$. These definitions are routine, but they fix the exact computational behaviour used later for iteration, comparison, and finite combinatorics.

```
-- § Successor injectivity.
suc-injective : {m n : ℕ} → suc m ≡ suc n → m ≡ n
suc-injective refl = refl

-- § Antisymmetry of ≤.
≤-antisym : {m n : ℕ} → m ≤ n → n ≤ m → m ≡ n
≤-antisym {zero} {zero} z≤n z≤n = refl
≤-antisym {zero} {suc n} z≤n ()
≤-antisym {suc m} {zero} () _
≤-antisym {suc m} {suc n} (s≤s p) (s≤s q) = cong suc (≤-antisym p q)

-- § Totality of ≤ on ℕ.
≤-total : (m n : ℕ) → (m ≤ n) ∨ (n ≤ m)
≤-total zero _      = inj₁ z≤n
≤-total _ zero      = inj₂ z≤n
≤-total (suc m) (suc n) with ≤-total m n
... | inj₁ p = inj₁ (s≤s p)
... | inj₂ p = inj₂ (s≤s p)

-- § Boolean type with built-in binding.
data Bool : Set where
  true  : Bool
  false : Bool

{-# BUILTIN BOOL Bool #-}

-- § Type-level closure: Bool has exactly two inhabitants.
Bool-exhaustive : ∀ (b : Bool) → (b ≡ true) ∨ (b ≡ false)
Bool-exhaustive true  = inj₁ refl
Bool-exhaustive false = inj₂ refl
{-# BUILTIN TRUE true #-}
{-# BUILTIN FALSE false #-}

{-# BUILTIN NATURAL ℕ #-}

-- § Built-in addition on ℕ.
infixl 6 _+_
_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

-- § Built-in multiplication on ℕ.
infixl 7 *_
*_ : ℕ → ℕ → ℕ
zero * n = zero
suc m * n = n + (m * n)
```

```

-- § Natural exponentiation.
infixr 8 _^_
_^_ : ℕ → ℕ → ℕ
x ^ zero = suc zero
x ^ suc n = x * (x ^ n)

-- § Built-in monus (truncated subtraction) on ℕ.
infixl 6 _-'_
-_' : ℕ → ℕ → ℕ
zero -' n = zero
suc m -' zero = suc m
suc m -' suc n = m -' n

{-# BUILTIN NATPLUS _+_ #-}
{-# BUILTIN NATTIMES *_ #-}
{-# BUILTIN NATMINUS _-'_ #-}

-- § Boolean strict comparison on ℕ.
_<ℕ-bool_ : ℕ → ℕ → Bool
m <ℕ-bool zero = false
zero <ℕ-bool suc _ = true
suc m <ℕ-bool suc n = m <ℕ-bool n

{-# BUILTIN NATLESS _<ℕ-bool_ #-}

-- § Boolean equality on ℕ.
_==ℕ-bool_ : ℕ → ℕ → Bool
zero ==ℕ-bool zero = true
zero ==ℕ-bool (suc _) = false
suc _ ==ℕ-bool zero = false
suc m ==ℕ-bool suc n = m ==ℕ-bool n

{-# BUILTIN NATEQUALS _==ℕ-bool_ #-}

```

Integer Construction

Integers are introduced next in signed normal form, together with a pair representation that makes addition and negation easy to define. The normalization map cancels common positive and negative contributions and serves as the basic mechanism for keeping integer expressions in canonical shape.

```

-- § Raw addition on ℕ (separate from BUILTIN _+_).
infixl 6 _+ℕ_

_+ℕ_ : ℕ → ℕ → ℕ
zero +ℕ n = n
suc m +ℕ n = suc (m +ℕ n)

-- § Integer type: signed normal form.
data ℤ : Set where
  0ℤ : ℤ

```

```

+succ_ : ℕ → ℤ
-succ_ : ℕ → ℤ

-- § Normalization: cancel common successors.
normalizeℤ : ℕ → ℕ → ℤ
normalizeℤ zero zero = 0ℤ
normalizeℤ (suc a) zero = +succ a
normalizeℤ zero (suc b) = -succ b
normalizeℤ (suc a) (suc b) = normalizeℤ a b

-- § Pair representation for componentwise arithmetic.
record Pairℕ : Set where
  constructor mkPairℕ
  field
    pos : ℕ
    neg : ℕ

open Pairℕ public

-- § Embedding ℤ into pairs.
toPairℤ : ℤ → Pairℕ
toPairℤ 0ℤ = mkPairℕ zero zero
toPairℤ (+succ n) = mkPairℕ (suc n) zero
toPairℤ (-succ n) = mkPairℕ zero (suc n)

-- § Collapsing pairs to ℤ.
fromPairℤ : Pairℕ → ℤ
fromPairℤ p = normalizeℤ (pos p) (neg p)

-- § Integer addition via pair-level componentwise sum.
infixl 6 _+ℤ_
_+ℤ_ : ℤ → ℤ → ℤ
x +ℤ y =
  let px = toPairℤ x in
  let py = toPairℤ y in
  normalizeℤ (pos px +ℕ pos py) (neg px +ℕ neg py)

-- § Integer negation via pair swap.
negℤ : ℤ → ℤ
negℤ z =
  let p = toPairℤ z in
  normalizeℤ (neg p) (pos p)

```

Endomorphism Iteration

Before returning to graph-specific constructions, the text records a generic notion of iterating an endomorphism. This small abstraction simplifies several later recurrence arguments.

```

-- § Generic endomorphism type.
GenEndo : Set → Set

```

```

GenEndo A = A → A

-- § Function composition.
infixr 9 _o_
_o_ : {A B C : Set} → (B → C) → (A → B) → A → C
(f o g) x = f (g x)

-- § Identity endomorphism.
idGenEndo : {A : Set} → GenEndo A
idGenEndo x = x

-- § n-fold iteration.
powEndo : {A : Set} → ℕ → GenEndo A → GenEndo A
powEndo zero f = idGenEndo
powEndo (suc n) f = f o powEndo n f

-- § Law 14I.0: Zero iteration is identity.
law14I-0-powEndo-zero : {A : Set} → (f : GenEndo A) → powEndo zero f ≐ idGenEndo
law14I-0-powEndo-zero f x = refl

-- § Law 14I.1: Successor iteration unfolds.
law14I-1-powEndo-suc : {A : Set} → (n : ℕ) → (f : GenEndo A) → powEndo (suc n) f ≐ (f o powEndo n f)
law14I-1-powEndo-suc n f x = refl

```

Integer Summation and Absolute Value

The next definitions package several small pieces of integer bookkeeping: finite sums over `Fin3` and `Fin4`, scalar multiples, and absolute value.

```

-- § Three-element sum.
sum3ℤ : ℤ → ℤ → ℤ → ℤ
sum3ℤ a b c = a + ℤ (b + ℤ c)

-- § Sum indexed by Fin3.
sumFin3ℤ : (Fin3 → ℤ) → ℤ
sumFin3ℤ f = sum3ℤ (f f0) (f f1) (f f2)

-- § Four-element sum.
sum4ℤ : ℤ → ℤ → ℤ → ℤ → ℤ
sum4ℤ a b c d = a + ℤ (b + ℤ (c + ℤ d))

-- § Sum indexed by Fin4.
sumFin4ℤ : (Fin4 → ℤ) → ℤ
sumFin4ℤ f = sum4ℤ (f g0) (f g1) (f g2) (f g3)

-- § Scalar triple.
threeTimesℤ : ℤ → ℤ
threeTimesℤ x = x + ℤ (x + ℤ x)

-- § Scalar quadruple.
fourTimesℤ : ℤ → ℤ
fourTimesℤ x = sum4ℤ x x x x

```



```
-- § Absolute value on  $\mathbb{Z}$ .
abs $\mathbb{Z}$  :  $\mathbb{Z} \rightarrow \mathbb{Z}$ 
abs $\mathbb{Z}$  0 $\mathbb{Z}$  = 0 $\mathbb{Z}$ 
abs $\mathbb{Z}$  (+suc n) = +suc n
abs $\mathbb{Z}$  (-suc n) = +suc n
```

Integer Order

An explicit order on integers is also needed. The definition below works directly with the signed normal form and provides the non-ordering relation used in strict comparison. The construction adds no independent order axiom: it is obtained by case analysis on the constructors of $\mathbb{Z}$, using the order already encoded by the signed normal form.

```
-- § Integer ordering.
infix 4 _<= $\mathbb{Z}$ _ _<= $\mathbb{Z}$ _

_<= $\mathbb{Z}$ _ :  $\mathbb{Z} \rightarrow \mathbb{Z} \rightarrow \text{Set}$ 
0 $\mathbb{Z}$  <= $\mathbb{Z}$  0 $\mathbb{Z}$  =  $\top$ 
0 $\mathbb{Z}$  <= $\mathbb{Z}$  (+suc n) =  $\top$ 
0 $\mathbb{Z}$  <= $\mathbb{Z}$  (-suc n) =  $\perp$ 
(+suc m) <= $\mathbb{Z}$  0 $\mathbb{Z}$  =  $\perp$ 
(+suc m) <= $\mathbb{Z}$  (+suc n) = suc m <= suc n
(+suc m) <= $\mathbb{Z}$  (-suc n) =  $\perp$ 
(-suc m) <= $\mathbb{Z}$  0 $\mathbb{Z}$  =  $\top$ 
(-suc m) <= $\mathbb{Z}$  (+suc n) =  $\top$ 
(-suc m) <= $\mathbb{Z}$  (-suc n) = suc n <= suc m

-- § Non-ordering witness.
_<= $\mathbb{Z}$ _ :  $\mathbb{Z} \rightarrow \mathbb{Z} \rightarrow \text{Set}$ 
x <= $\mathbb{Z}$  y = (x <= $\mathbb{Z}$  y)  $\rightarrow \perp$ 

-- § Strict integer ordering.
_<= $\mathbb{Z}$ _ :  $\mathbb{Z} \rightarrow \mathbb{Z} \rightarrow \text{Set}$ 
_<= $\mathbb{Z}$ _ x y = (x <= $\mathbb{Z}$  y)  $\times$  (y <= $\mathbb{Z}$  x)
```

K_4 Vertex Indexing

To move between finite combinatorics and the four endomorphism cases, the development fixes an explicit bijection between `Fin4` and `EndoCase`. This supplies a concrete vertex indexing for K_4 ; the graph itself has already been fixed by the classification theorem.

```
-- § Fin4  $\rightarrow$  EndoCase bijection.
vertexAt : Fin4  $\rightarrow$  EndoCase
vertexAt g0 = case-constL
```

```

vertexAt g1 = case-constR
vertexAt g2 = case-id
vertexAt g3 = case-dual

-- § EndoCase → Fin4 inverse.
vertexIndex : EndoCase → Fin4
vertexIndex case-constL = g0
vertexIndex case-constR = g1
vertexIndex case-id     = g2
vertexIndex case-dual   = g3

-- § Round-trip: vertexAt ∘ vertexIndex = id.
vertexAt-index : (v : EndoCase) → vertexAt (vertexIndex v) ≡ v
vertexAt-index case-constL = refl
vertexAt-index case-constR = refl
vertexAt-index case-id     = refl
vertexAt-index case-dual   = refl

-- § Round-trip: vertexIndex ∘ vertexAt = id.
index-vertexAt : (i : Fin4) → vertexIndex (vertexAt i) ≡ i
index-vertexAt g0 = refl
index-vertexAt g1 = refl
index-vertexAt g2 = refl
index-vertexAt g3 = refl

```

Endomorphism Systems

The final preparatory block introduces endomorphism systems and their morphisms. These abstractions later allow universal properties and iteration statements to be expressed without repeating the same commutation argument at each occurrence.

```

-- § Setoid: carrier with equivalence relation.
record ESetoid : Set1 where
  field
    Obj    : Set
    Rel    : Obj → Obj → Set
    refl≈  : (x : Obj) → Rel x x
    sym≈   : {x y : Obj} → Rel x y → Rel y x
    trans≈ : {x y z : Obj} → Rel x y → Rel y z → Rel x z

open ESetoid public

-- § Endomorphism system: setoid + step.
record EndoSystem : Set1 where
  field
    ES : ESetoid
    step : Obj ES → Obj ES
    step-cong : {x y : Obj ES} → Rel ES x y → Rel ES (step x) (step y)

open EndoSystem public

```

```

-- § Morphism of endomorphism systems.
record ESHom (X Y : EndoSystem) : Set1 where
  field
    esmap : Obj (ES X) → Obj (ES Y)
    esmap-cong : {x y : Obj (ES X)} → Rel (ES X) x y → Rel (ES Y) (esmap x) (esmap y)
    escomm : (x : Obj (ES X)) → Rel (ES Y) (esmap (step X x)) (step Y (esmap x))

open ESHom public

-- § Terminal endomorphism system.
record IsTerminal (T : EndoSystem) : Set1 where
  field
    ! : (X : EndoSystem) → ESHom X T
    uniq : (X : EndoSystem) → (f g : ESHom X T) → (x : Obj (ES X)) → Rel (ES T) (esmap f x) (esmap g x)

-- § Initial endomorphism system.
record IsInitial (I : EndoSystem) : Set1 where
  field
    ! : (X : EndoSystem) → ESHom I X
    uniq : (X : EndoSystem) → (f g : ESHom I X) → (x : Obj (ES I)) → Rel (ES X) (esmap f x) (esmap g x)

-- § Law 14J.0: Morphisms commute with n-fold iteration.
law14J-0-hom-iter :
  {X Y : EndoSystem} →
  (h : ESHom X Y) →
  (n : ℕ) →
  (x : Obj (ES X)) →
  Rel (ES Y) (esmap h (powEndo n (step X) x)) (powEndo n (step Y) (esmap h x))
law14J-0-hom-iter {X} {Y} h zero x = refl ≈ (ES Y) (esmap h x)
law14J-0-hom-iter {X} {Y} h (suc n) x =
  trans ≈ (ES Y)
  (escomm h (powEndo n (step X) x))
  (step-cong Y (law14J-0-hom-iter h n x))

```

Classification as Representability

This section recasts the earlier classification theorem as a representability statement. The four constructors of `EndoCase` are not merely labels for previously separated cases; under the soundness and uniqueness laws already proved, they provide a complete coordinate system for the endomorphisms of a distinction. The record below packages the forward interpretation, the reverse classification, and the two round-trip laws into a single formal object.

```

-- § The classification bijection as a universal property
record EndoRepresentability (d : Distinction) : Set where
  field
    label-to-endo : EndoCase → (S d → S d)
    endo-to-label : (S d → S d) → EndoCase
    roundtrip-sound : (f : S d → S d) → label-to-endo (endo-to-label f) ≐ f
    roundtrip-label : (c : EndoCase) → endo-to-label (label-to-endo c) ≐ c
    label-injective : (c1 c2 : EndoCase) →

```

```

label-to-endo  $c_1 \stackrel{\circ}{=} \text{label-to-endo } c_2 \rightarrow c_1 \equiv c_2$ 

theorem-endo-representability : ( $d$  : Distinction)  $\rightarrow$  EndoRepresentability  $d$ 
theorem-endo-representability  $d = \text{record}$ 
{ label-to-endo =  $K_4.\text{interpret } d$ 
; endo-to-label =  $K_4.\text{classify } d$ 
; roundtrip-sound =  $K_4.\text{classify-sound } d$ 
; roundtrip-label =  $\lambda c \rightarrow$ 
  sym ( $K_4.\text{classify-unique } d (K_4.\text{interpret } d c) c (K_4.\stackrel{\circ}{-}\text{refl } d)$ )
; label-injective =  $K_4.\text{interpret-injective } d$ 
}

```

K_4 Coupling Infrastructure

The next constructions describe couplings between copies of the four endomorphism classes. Since the carrier has only four elements, every relevant transformation can be written out explicitly, and the finitary character of the argument remains completely visible.

Step 1: EndoCase Inequality and Decidable Equality. The first step is elementary: inequality and decidable equality are recorded directly on EndoCase itself. The later symmetry arguments require no hidden enumeration principle; each impossible equality is discharged by an absurd pattern.

```

-- § Inequality on EndoCase.
EndoCase $\neq$  : ( $a b$  : EndoCase)  $\rightarrow$  Set
EndoCase $\neq$   $a b = a \equiv b \rightarrow \perp$ 

case-constL $\neq$ case-constR : EndoCase $\neq$  case-constL case-constR
case-constL $\neq$ case-constR ()

case-constL $\neq$ case-id : EndoCase $\neq$  case-constL case-id
case-constL $\neq$ case-id ()

case-constL $\neq$ case-dual : EndoCase $\neq$  case-constL case-dual
case-constL $\neq$ case-dual ()

case-constR $\neq$ case-id : EndoCase $\neq$  case-constR case-id
case-constR $\neq$ case-id ()

case-constR $\neq$ case-dual : EndoCase $\neq$  case-constR case-dual
case-constR $\neq$ case-dual ()

case-id $\neq$ case-dual : EndoCase $\neq$  case-id case-dual
case-id $\neq$ case-dual ()

-- § Decidable equality on EndoCase.
EndoCase-decEq : ( $a b$  : EndoCase)  $\rightarrow (a \equiv b) \uplus (\text{EndoCase}\neq a b)$ 
EndoCase-decEq case-constL case-constL = inj1 refl
EndoCase-decEq case-constR case-constR = inj1 refl

```

```

EndoCase-decEq case-id case-id = inj1 refl
EndoCase-decEq case-dual case-dual = inj1 refl
EndoCase-decEq case-constL case-constR = inj2 case-constL ≠ case-constR
EndoCase-decEq case-constR case-constL = inj2 (λ e → case-constL ≠ case-constR (sym e))
EndoCase-decEq case-constL case-id = inj2 case-constL ≠ case-id
EndoCase-decEq case-id case-constL = inj2 (λ e → case-constL ≠ case-id (sym e))
EndoCase-decEq case-constL case-dual = inj2 case-constL ≠ case-dual
EndoCase-decEq case-dual case-constL = inj2 (λ e → case-constL ≠ case-dual (sym e))
EndoCase-decEq case-constR case-id = inj2 case-constR ≠ case-id
EndoCase-decEq case-id case-constR = inj2 (λ e → case-constR ≠ case-id (sym e))
EndoCase-decEq case-constR case-dual = inj2 case-constR ≠ case-dual
EndoCase-decEq case-dual case-constR = inj2 (λ e → case-constR ≠ case-dual (sym e))
EndoCase-decEq case-id case-dual = inj2 case-id ≠ case-dual
EndoCase-decEq case-dual case-id = inj2 (λ e → case-id ≠ case-dual (sym e))

```

Step 2: Transposition on EndoCase. The next ingredient is a transposition on EndoCase. Because the carrier has only four elements, the swap can be defined by complete case distinction, and its involutive character can be checked by the same explicit enumeration.

```

-- § Transposition on EndoCase: swaps x↔y, fixes others.
swapEndo : EndoCase → EndoCase → EndoCase → EndoCase
swapEndo case-constL case-constL z = z
swapEndo case-constL case-constR case-constL = case-constR
swapEndo case-constL case-constR case-constR = case-constL
swapEndo case-constL case-constR case-id = case-id
swapEndo case-constL case-constR case-dual = case-dual
swapEndo case-constL case-id case-constL = case-id
swapEndo case-constL case-id case-constR = case-constR
swapEndo case-constL case-id case-id = case-constL
swapEndo case-constL case-id case-dual = case-dual
swapEndo case-constL case-dual case-constL = case-dual
swapEndo case-constL case-dual case-constR = case-constR
swapEndo case-constL case-dual case-id = case-id
swapEndo case-constL case-dual case-dual = case-constL
swapEndo case-constR case-constL case-constL = case-constR
swapEndo case-constR case-constL case-constR = case-constL
swapEndo case-constR case-constL case-id = case-id
swapEndo case-constR case-constL case-dual = case-dual
swapEndo case-constR case-constR z = z
swapEndo case-constR case-id case-constL = case-constL
swapEndo case-constR case-id case-constR = case-id
swapEndo case-constR case-id case-id = case-constR
swapEndo case-constR case-id case-dual = case-dual
swapEndo case-constR case-dual case-constL = case-constL
swapEndo case-constR case-dual case-constR = case-dual
swapEndo case-constR case-dual case-id = case-id
swapEndo case-constR case-dual case-dual = case-constR
swapEndo case-id case-constL case-constL = case-id
swapEndo case-id case-constL case-constR = case-constR
swapEndo case-id case-constL case-id = case-constL

```

```

swapEndo case-id case-constL case-dual = case-dual
swapEndo case-id case-constR case-constL = case-constL
swapEndo case-id case-constR case-constR = case-id
swapEndo case-id case-constR case-id    = case-constR
swapEndo case-id case-constR case-dual = case-dual
swapEndo case-id case-id z = z
swapEndo case-id case-dual case-constL = case-constL
swapEndo case-id case-dual case-constR = case-constR
swapEndo case-id case-dual case-id    = case-dual
swapEndo case-id case-dual case-dual = case-id
swapEndo case-dual case-constL case-constL = case-dual
swapEndo case-dual case-constL case-constR = case-constR
swapEndo case-dual case-constL case-id    = case-id
swapEndo case-dual case-constL case-dual = case-constL
swapEndo case-dual case-constR case-constL = case-constL
swapEndo case-dual case-constR case-constR = case-dual
swapEndo case-dual case-constR case-id    = case-id
swapEndo case-dual case-constR case-dual = case-constR
swapEndo case-dual case-id case-constL = case-constL
swapEndo case-dual case-id case-constR = case-constR
swapEndo case-dual case-id case-id    = case-dual
swapEndo case-dual case-id case-dual  = case-id
swapEndo case-dual case-dual z = z

-- § swapEndo is an involution (64-case proof).
swapEndo-involutive : (x y z : EndoCase) → swapEndo x y (swapEndo x y z) ≡ z
swapEndo-involutive case-constL case-constL z = refl
swapEndo-involutive case-constL case-constR case-constL = refl
swapEndo-involutive case-constL case-constR case-constR = refl
swapEndo-involutive case-constL case-constR case-id    = refl
swapEndo-involutive case-constL case-constR case-dual = refl
swapEndo-involutive case-constL case-id    case-constL = refl
swapEndo-involutive case-constL case-id    case-constR = refl
swapEndo-involutive case-constL case-id    case-id    = refl
swapEndo-involutive case-constL case-id    case-dual  = refl
swapEndo-involutive case-constL case-dual case-constL = refl
swapEndo-involutive case-constL case-dual case-constR = refl
swapEndo-involutive case-constL case-dual case-id    = refl
swapEndo-involutive case-constL case-dual case-dual  = refl
swapEndo-involutive case-constR case-constL case-constL = refl
swapEndo-involutive case-constR case-constL case-constR = refl
swapEndo-involutive case-constR case-constL case-id    = refl
swapEndo-involutive case-constR case-constL case-dual = refl
swapEndo-involutive case-constR case-constR z = refl
swapEndo-involutive case-constR case-id    case-constL = refl
swapEndo-involutive case-constR case-id    case-constR = refl
swapEndo-involutive case-constR case-id    case-id    = refl
swapEndo-involutive case-constR case-id    case-dual  = refl
swapEndo-involutive case-constR case-dual case-constL = refl
swapEndo-involutive case-constR case-dual case-constR = refl
swapEndo-involutive case-constR case-dual case-id    = refl
swapEndo-involutive case-constR case-dual case-dual  = refl
swapEndo-involutive case-id case-constL case-constL = refl

```

```

swapEndo-involutive case-id case-constL case-constR = refl
swapEndo-involutive case-id case-constL case-id      = refl
swapEndo-involutive case-id case-constL case-dual    = refl
swapEndo-involutive case-id case-constR case-constL = refl
swapEndo-involutive case-id case-constR case-constR = refl
swapEndo-involutive case-id case-constR case-id      = refl
swapEndo-involutive case-id case-constR case-dual    = refl
swapEndo-involutive case-id case-id z = refl
swapEndo-involutive case-id case-dual case-constL = refl
swapEndo-involutive case-id case-dual case-constR = refl
swapEndo-involutive case-id case-dual case-id      = refl
swapEndo-involutive case-id case-dual case-dual    = refl
swapEndo-involutive case-dual case-constL case-constL = refl
swapEndo-involutive case-dual case-constL case-constR = refl
swapEndo-involutive case-dual case-constL case-id      = refl
swapEndo-involutive case-dual case-constL case-dual    = refl
swapEndo-involutive case-dual case-constR case-constL = refl
swapEndo-involutive case-dual case-constR case-constR = refl
swapEndo-involutive case-dual case-constR case-id      = refl
swapEndo-involutive case-dual case-constR case-dual    = refl
swapEndo-involutive case-dual case-id case-constL = refl
swapEndo-involutive case-dual case-id case-constR = refl
swapEndo-involutive case-dual case-id case-id      = refl
swapEndo-involutive case-dual case-id case-dual    = refl
swapEndo-involutive case-dual case-dual z = refl

```

Step 3: Transposition Sends Source to Target. Once the transposition has been defined, the basic transport fact is immediate: swapping x with y sends x to y . This elementary observation is the mechanism behind the permutation arguments used below.

```

-- § swapEndo sends x to y.
swapEndo-sends : (x y : EndoCase) → swapEndo x y x ≡ y
swapEndo-sends case-constL case-constL = refl
swapEndo-sends case-constL case-constR = refl
swapEndo-sends case-constL case-id      = refl
swapEndo-sends case-constL case-dual    = refl
swapEndo-sends case-constR case-constL = refl
swapEndo-sends case-constR case-constR = refl
swapEndo-sends case-constR case-id      = refl
swapEndo-sends case-constR case-dual    = refl
swapEndo-sends case-id case-constL = refl
swapEndo-sends case-id case-constR = refl
swapEndo-sends case-id case-id      = refl
swapEndo-sends case-id case-dual    = refl
swapEndo-sends case-dual case-constL = refl
swapEndo-sends case-dual case-constR = refl
swapEndo-sends case-dual case-id      = refl
swapEndo-sends case-dual case-dual    = refl

```

Step 4: Permutations, Couplings, and Cross-Invariance. These transpositions are then packaged as permutations of `EndoCase`. With that in place, a coupling is simply a relation between two copies of the carrier, and cross-invariance means that this relation is preserved under independent permutations in the two coordinates.

```
-- § Permutation record on EndoCase.
record EndoPerm : Set where
  field
    eto    : EndoCase → EndoCase
    efrom  : EndoCase → EndoCase
    eto-efrom : (y : EndoCase) → eto (efrom y) ≡ y
    efrom-eto : (x : EndoCase) → efrom (eto x) ≡ x

open EndoPerm public

-- § Transposition as permutation.
permSwap : (x y : EndoCase) → EndoPerm
permSwap x y = record
{ eto = swapEndo x y
; efrom = swapEndo x y
; eto-efrom = swapEndo-involutive x y
; efrom-eto = swapEndo-involutive x y
}

-- § Any vertex can be sent to any other by some permutation.
endoPerm-send : (a a' : EndoCase) → ∑ EndoPerm (λ σ → eto σ a ≡ a')
endoPerm-send a a' = (permSwap a a', swapEndo-sends a a')

-- § Coupling: relation between two copies of EndoCase.
Coupling : Set1
Coupling = EndoCase → EndoCase → Set

-- § Cross-invariance: coupling respected by independent permutations.
CrossInv : Coupling → Set
CrossInv C = (σ τ : EndoPerm) → (a b : EndoCase) → C a b → C (eto σ a) (eto τ b)

-- § Heterogeneous transport over a two-argument predicate.
transport2 : {A B : Set} {P : A → B → Set} {a a' : A} {b b' : B} → a ≡ a' → b ≡ b' → P a b → P a' b'
transport2 {P = P} {a = a} {a' = a'} {b = b} {b' = b'} ea eb pab =
  subst (λ a0 → P a0 b') ea (subst (λ b0 → P a b0) eb pab)
```

Law 14F.0: A Single Cross-Invariant Edge Forces All Edges

Cross-invariance is extremely rigid in this setting. If one cross-edge exists, permutations of the two factors move that edge to any prescribed pair of vertices, so the coupling is full under the stated invariance hypothesis. Dually, if one edge is absent under the same hypothesis, then every edge is absent.

```
-- § Law 14F.0: One edge forces all.
law14F-0-edge-forces-all : (C : Coupling) → CrossInv C →
  ∑ EndoCase (λ a0 → ∑ EndoCase (λ b0 → C a0 b0)) →
```



```

(a b : EndoCase) → C a b
law14F-0-edge-forces-all C inv (a0 , (b0 , c0)) a b =
  let sa = endoPerm-send a0 a in
  let sb = endoPerm-send b0 b in
  let σ = fst sa in
  let τ = fst sb in
  transport2 {P = C} (snd sa) (snd sb) (inv σ τ a0 b0 c0)

-- § Law 14F.1: One missing edge forces no edges.
law14F-1-nonedge-forces-none : (C : Coupling) → CrossInv C →
  Σ EndoCase (λ a0 → Σ EndoCase (λ b0 → ¬ (C a0 b0))) →
  (a b : EndoCase) → ¬ (C a b)
law14F-1-nonedge-forces-none C inv (a0 , (b0 , n0)) a b cab =
  let sa = endoPerm-send a a0 in
  let sb = endoPerm-send b b0 in
  let σ = fst sa in
  let τ = fst sb in
  n0 (transport2 {P = C} (snd sa) (snd sb) (inv σ τ a b cab))

```

Self-Coupling and the Squaring Law

For self-couplings this rigidity has a simple numerical consequence. A full coupling on a carrier of size n contains n^2 ordered pairs, and for EndoCase this yields the concrete identity $4^2 = 16$.

```

-- §§ Self-coupling squaring – corollary of law14F-0.

-- § Any non-empty cross-invariant coupling is full.
coupling-full-from-witness : (C : Coupling) → CrossInv C →
  Σ EndoCase (λ a → Σ EndoCase (λ b → C a b)) →
  (a b : EndoCase) → C a b
coupling-full-from-witness = law14F-0-edge-forces-all

-- § EndoCase carrier count = 4.
EndoCase-count : ℕ
EndoCase-count = 4

-- § Full self-coupling weight = n2 = 16.
law-self-coupling-square : EndoCase-count * EndoCase-count ≡ 16
law-self-coupling-square = refl

```

K₄ Product Graphs

The coupling data can now be read as a two-layer graph with vertex set $\text{Two} \times \text{EndoCase}$. Within each layer the edges are the usual K_4 edges, while the coupling determines the cross-edges between the two layers.

```

-- § Inequality on Two.
Two≠ : (i j : Two) → Set

```

```

Two≠ i j = i ≡ j → ⊥

L≠R : Two≠ L R
L≠R ()

-- § Decidable equality on Two.
Two-decEq : (i j : Two) → (i ≡ j) ⊔ (Two≠ i j)
Two-decEq L L = inj₁ refl
Two-decEq R R = inj₁ refl
Two-decEq L R = inj₂ L≠R
Two-decEq R L = inj₂ (λ e → L≠R (sym e))

-- § Product edge relation.
Edge2 : Coupling → (Two × EndoCase) → (Two × EndoCase) → Set
Edge2 C (L, a) (L, b) = a ≠ b
Edge2 C (R, a) (R, b) = a ≠ b
Edge2 C (L, a) (R, b) = C a b
Edge2 C (R, a) (L, b) = C b a

-- § Product edge symmetry.
edge2-sym : (C : Coupling) → {x y : Two × EndoCase} → Edge2 C x y → Edge2 C y x
edge2-sym C {x = (L, a)} {y = (L, b)} e = λ eq → e (sym eq)
edge2-sym C {x = (R, a)} {y = (R, b)} e = λ eq → e (sym eq)
edge2-sym C {x = (L, a)} {y = (R, b)} e = e
edge2-sym C {x = (R, a)} {y = (L, b)} e = e

-- § Product edge irreflexivity.
edge2-irr : (C : Coupling) → (x : Two × EndoCase) → Edge2 C x x → ⊥
edge2-irr C (L, a) e = e refl
edge2-irr C (R, a) e = e refl

-- § K₄ × 2 graph from coupling.
K4×2 : Coupling → Graph
K4×2 C = record
{ V = Two × EndoCase
; Edge = Edge2 C
; edge-sym = λ {a} {b} e → edge2-sym C {x = a} {y = b} e
; edge-irr = edge2-irr C
}

-- § Empty coupling (no cross-edges).
CrossEmpty : Coupling
CrossEmpty __ = ⊥

-- § Full coupling (all cross-edges).
CrossFull : Coupling
CrossFull __ = ⊤

```

This construction serves as the ambient graph for the additive laws that follow.

Chapter 30

Forced Additive Laws

With the underlying integer operations in place, the next chapter establishes their basic algebraic laws. The proofs are constructive and explicit: each required identity is reduced to the defining equations for natural-number addition and the normalization map for integers.

Natural Addition Laws

Everything begins with the elementary properties of addition on $\mathbb{N}$. These lemmas are routine, but they are the ingredients from which the additive structure on integers is assembled.

```
-- § Diagonal normalization collapses to zero.
normalizeDiag0ℤ : (n : ℕ) → normalizeℤ n n ≡ 0ℤ
normalizeDiag0ℤ zero = refl
normalizeDiag0ℤ (suc n) = normalizeDiag0ℤ n

-- § Right identity for +ℕ.
+ℕ-zero-right : (n : ℕ) → n +ℕ zero ≡ n
+ℕ-zero-right zero = refl
+ℕ-zero-right (suc n) = cong suc (+ℕ-zero-right n)

-- § Successor shifts right.
+ℕ-suc-right : (n m : ℕ) → n +ℕ suc m ≡ suc (n +ℕ m)
+ℕ-suc-right zero m = refl
+ℕ-suc-right (suc n) m = cong suc (+ℕ-suc-right n m)

-- § Associativity of +ℕ.
+ℕ-assoc : (a b c : ℕ) → (a +ℕ b) +ℕ c ≡ a +ℕ (b +ℕ c)
+ℕ-assoc zero b c = refl
+ℕ-assoc (suc a) b c = cong suc (+ℕ-assoc a b c)

-- § Commutativity of +ℕ.
+ℕ-comm : (a b : ℕ) → a +ℕ b ≡ b +ℕ a
```

```

+ℕ-comm zero b = sym (+ℕ-zero-right b)
+ℕ-comm (suc a) b =
  trans
    refl
  (trans
    (cong suc (+ℕ-comm a b))
    (sym (+ℕ-suc-right b a)))

```

Integer Addition Laws

The corresponding laws for $\mathbb{Z}$ are obtained by passing through the pair representation and then normalizing. In this way commutativity, associativity, identities, and inverses are all proved directly from the concrete definition of integer addition.

```

-- § Congruence for normalizeℤ.
normalizeℤ-cong : {a a' b b' : ℕ} → a ≡ a' → b ≡ b' → normalizeℤ a b ≡ normalizeℤ a' b'
normalizeℤ-cong {a} {a'} {b} {b'} pa pb = trans (cong (λ t → normalizeℤ t b) pa) (cong (normalizeℤ a') pb)

-- § Normalization absorbs pair-level addition.
normalize-plusRight : (a b c d : ℕ) →
  normalizeℤ (pos (toPairℤ (normalizeℤ a b)) +ℕ c) (neg (toPairℤ (normalizeℤ a b)) +ℕ d)
  ≡
  normalizeℤ (a +ℕ c) (b +ℕ d)
normalize-plusRight zero zero c d = refl
normalize-plusRight (suc a) zero c d = refl
normalize-plusRight zero (suc b) c d = refl
normalize-plusRight (suc a) (suc b) c d = normalize-plusRight a b c d

-- § Commutativity of +ℤ.
+ℤ-comm : (x y : ℤ) → x +ℤ y ≡ y +ℤ x
+ℤ-comm x y with toPairℤ x | toPairℤ y
... | px | py =
  normalizeℤ-cong (+ℕ-comm (pos px) (pos py)) (+ℕ-comm (neg px) (neg py))

-- § Associativity of +ℤ.
+ℤ-assoc : (x y z : ℤ) → (x +ℤ y) +ℤ z ≡ x +ℤ (y +ℤ z)
+ℤ-assoc x y z with toPairℤ x | toPairℤ y | toPairℤ z
... | px | py | pz =
  let ax = pos px in
  let bx = neg px in
  let ay = pos py in
  let by = neg py in
  let az = pos pz in
  let bz = neg pz in
  let Axy = ax +ℕ ay in
  let Bxy = bx +ℕ by in
  let Ayz = ay +ℕ az in
  let Byz = by +ℕ bz in
  let lhs0 = normalizeℤ (pos (toPairℤ (normalizeℤ Axy Bxy)) +ℕ az)
    (neg (toPairℤ (normalizeℤ Axy Bxy)) +ℕ bz) in

```

```

let lhs1 = normalizeℤ (Axy +ℕ az) (Bxy +ℕ bz) in
let rhs0 = normalizeℤ (ax +ℕ pos (toPairℤ (normalizeℤ Ayz Byz)))
    (bx +ℕ neg (toPairℤ (normalizeℤ Ayz Byz))) in
let rhs1 = normalizeℤ (pos (toPairℤ (normalizeℤ Ayz Byz)) +ℕ ax)
    (neg (toPairℤ (normalizeℤ Ayz Byz)) +ℕ bx) in
let rhs2 = normalizeℤ (Ayz +ℕ ax) (Byz +ℕ bx) in
let rhs3 = normalizeℤ (ax +ℕ Ayz) (bx +ℕ Byz) in

trans
(trans
  (cong (λ u → u) (normalize-plusRight Axy Bxy az bz))
  (normalizeℤ-cong (+ℕ-assoc ax ay az) (+ℕ-assoc bx by bz)))
(sym
  (trans
    (trans
      (normalizeℤ-cong (+ℕ-comm ax (pos (toPairℤ (normalizeℤ Ayz Byz))))
        (+ℕ-comm bx (neg (toPairℤ (normalizeℤ Ayz Byz)))))
      (normalize-plusRight Ayz Byz ax bx))
    (normalizeℤ-cong (+ℕ-comm Ayz ax) (+ℕ-comm Byz bx))))

-- § Left identity for +ℤ.
+ℤ-zero-left : (x : ℤ) → 0ℤ +ℤ x ≡ x
+ℤ-zero-left 0ℤ = refl
+ℤ-zero-left (+suc n) = refl
+ℤ-zero-left (-suc n) = refl

-- § Right identity for +ℤ.
+ℤ-zero-right : (x : ℤ) → x +ℤ 0ℤ ≡ x
+ℤ-zero-right x = trans (+ℤ-comm x 0ℤ) (+ℤ-zero-left x)

-- § Right inverse.
+ℤ-inv-right : (x : ℤ) → x +ℤ negℤ x ≡ 0ℤ
+ℤ-inv-right 0ℤ = refl
+ℤ-inv-right (+suc n) =
trans
  (cong (λ a → normalizeℤ a (suc n)) (+ℕ-zero-right (suc n)))
  (normalizeDiag0ℤ (suc n))
+ℤ-inv-right (-suc n) =
trans
  (cong (normalizeℤ (suc n)) (+ℕ-zero-right (suc n)))
  (normalizeDiag0ℤ (suc n))

-- § Left inverse.
+ℤ-inv-left : (x : ℤ) → negℤ x +ℤ x ≡ 0ℤ
+ℤ-inv-left x = trans (+ℤ-comm (negℤ x) x) (+ℤ-inv-right x)

-- § Negation of zero.
negℤ-zero : negℤ 0ℤ ≡ 0ℤ
negℤ-zero = refl

-- § Left cancellation.
+ℤ-cancel-left : (a b : ℤ) → a +ℤ b ≡ a → b ≡ 0ℤ
+ℤ-cancel-left a b eq =
trans

```

```

(sym (+ℤ-zero-left b))
(trans
  (cong (λ t → t +ℤ b) (sym (+ℤ-inv-left a)))
  (trans
    (+ℤ-assoc (negℤ a) a b)
    (trans
      (cong (λ t → negℤ a +ℤ t) eq)
      (+ℤ-inv-left a))))

-- § Negation zero-test.
negℤ-zero→zero : (z : ℤ) → negℤ z ≡ 0ℤ → z ≡ 0ℤ
negℤ-zero→zero 0ℤ _ = refl
negℤ-zero→zero (+suc n) ()
negℤ-zero→zero (-suc n) ()

```

Summation Permutation Laws

Once associativity and commutativity are available, finite sums can be rearranged without changing their value. The next lemmas record the permutations that will be needed later.

```

-- § Swap first two summands in a three-term sum.
swapHeadℤ : (a b t : ℤ) → a +ℤ (b +ℤ t) ≡ b +ℤ (a +ℤ t)
swapHeadℤ a b t =
  trans (sym (+ℤ-assoc a b t))
    (trans (cong (λ s → s +ℤ t) (+ℤ-comm a b))
      (+ℤ-assoc b a t))

sum3ℤ-swap01 : (a b c : ℤ) → sum3ℤ a b c ≡ sum3ℤ b a c
sum3ℤ-swap01 a b c = swapHeadℤ a b c

sum4ℤ-swap01 : (a b c d : ℤ) → sum4ℤ a b c d ≡ sum4ℤ b a c d
sum4ℤ-swap01 a b c d = swapHeadℤ a b (c +ℤ d)

sum4ℤ-swap12 : (a b c d : ℤ) → sum4ℤ a b c d ≡ sum4ℤ a c b d
sum4ℤ-swap12 a b c d = cong (λ t → a +ℤ t) (sum3ℤ-swap01 b c d)

sum4ℤ-swap23 : (a b c d : ℤ) → sum4ℤ a b c d ≡ sum4ℤ a b d c
sum4ℤ-swap23 a b c d = cong (λ t → a +ℤ (b +ℤ t)) (+ℤ-comm c d)

```

Negation Laws

Negation is just as concrete: on the pair representation it swaps the positive and negative components. The expected identities, including involutivity and distributivity over addition, are then verified by direct computation.

```

-- § Swap pair components.
swapPairℕ : Pairℕ → Pairℕ

```

```

swapPairℕ p = mkPairℕ (neg p) (pos p)

-- § Negation as pair swap.
toPair-negℤ : (z : ℤ) → toPairℤ (negℤ z) ≡ swapPairℕ (toPairℤ z)
toPair-negℤ 0ℤ = refl
toPair-negℤ (+suc n) = refl
toPair-negℤ (-suc n) = refl

-- § Negation is involutive.
negℤ-involutive : (z : ℤ) → negℤ (negℤ z) ≡ z
negℤ-involutive 0ℤ = refl
negℤ-involutive (+suc n) = refl
negℤ-involutive (-suc n) = refl

-- § Positive component of negation.
pos-toPair-negℤ : (z : ℤ) → pos (toPairℤ (negℤ z)) ≡ neg (toPairℤ z)
pos-toPair-negℤ z = cong pos (toPair-negℤ z)

-- § Negative component of negation.
neg-toPair-negℤ : (z : ℤ) → neg (toPairℤ (negℤ z)) ≡ pos (toPairℤ z)
neg-toPair-negℤ z = cong neg (toPair-negℤ z)

-- § Normalization commutes with negation.
neg-normalizeℤ : (a b : ℕ) → negℤ (normalizeℤ a b) ≡ normalizeℤ b a
neg-normalizeℤ zero zero = refl
neg-normalizeℤ (suc a) zero = refl
neg-normalizeℤ zero (suc b) = refl
neg-normalizeℤ (suc a) (suc b) = neg-normalizeℤ a b

-- § Negation distributes over componentwise addition.
negAdd-normalizeSwap : (x y : ℤ) →
  negℤ x +ℤ negℤ y ≡
  normalizeℤ (neg (toPairℤ x) +ℕ neg (toPairℤ y)) (pos (toPairℤ x) +ℕ pos (toPairℤ y))
negAdd-normalizeSwap x y =
  let A₁ = pos (toPairℤ (negℤ x)) +ℕ pos (toPairℤ (negℤ y)) in
  let B₁ = neg (toPairℤ (negℤ x)) +ℕ neg (toPairℤ (negℤ y)) in
  let A₂ = neg (toPairℤ x) +ℕ neg (toPairℤ y) in
  let B₂ = pos (toPairℤ x) +ℕ pos (toPairℤ y) in
  let eqA₁ =
    trans
      (cong (λ t → t +ℕ pos (toPairℤ (negℤ y))) (pos-toPair-negℤ x))
      (cong (λ t → neg (toPairℤ x) +ℕ t) (pos-toPair-negℤ y))
    in
  let eqB₁ =
    trans
      (cong (λ t → t +ℕ neg (toPairℤ (negℤ y))) (neg-toPair-negℤ x))
      (cong (λ t → pos (toPairℤ x) +ℕ t) (neg-toPair-negℤ y))
    in
  trans (cong (λ a → normalizeℤ a B₁) eqA₁)
        (cong (normalizeℤ A₂) eqB₁)

-- § Negation distributes over +ℤ.
neg-+ℤ : (x y : ℤ) → negℤ (x +ℤ y) ≡ negℤ x +ℤ negℤ y
neg-+ℤ x y =

```

```

let A = pos (toPairℤ x) +ℕ pos (toPairℤ y) in
let B = neg (toPairℤ x) +ℕ neg (toPairℤ y) in
trans (neg-normalizeℤ A B) (sym (negAdd-normalizeSwap x y))

-- § Negation distributes over sum3ℤ.
neg-sum3ℤ : (a b c : ℤ) → negℤ (sum3ℤ a b c) ≡ sum3ℤ (negℤ a) (negℤ b) (negℤ c)
neg-sum3ℤ a b c =
trans (neg-+ℤ a (b +ℤ c))
      (cong (λ t → negℤ a +ℤ t) (neg-+ℤ b c))

-- § Negation distributes over sum4ℤ.
neg-sum4ℤ : (a b c d : ℤ) → negℤ (sum4ℤ a b c d) ≡ sum4ℤ (negℤ a) (negℤ b) (negℤ c) (negℤ d)
neg-sum4ℤ a b c d =
trans
  (neg-+ℤ a (b +ℤ (c +ℤ d)))
  (cong (λ t → negℤ a +ℤ t)
        (trans
          (neg-+ℤ b (c +ℤ d))
          (cong (λ t → negℤ b +ℤ t) (neg-+ℤ c d))))

-- § Negation distributes over fourTimesℤ.
neg-fourTimesℤ : (x : ℤ) → negℤ (fourTimesℤ x) ≡ fourTimesℤ (negℤ x)
neg-fourTimesℤ x = neg-sum4ℤ x x x x

-- § Negation distributes over sumFin3ℤ.
neg-sumFin3ℤ : (f : Fin3 → ℤ) → negℤ (sumFin3ℤ f) ≡ sumFin3ℤ (λ k → negℤ (f k))
neg-sumFin3ℤ f = neg-sum3ℤ (f f0) (f f1) (f f2)

```


Chapter 31

Multiplicative Structure and Integer Order

The next stage extends the additive picture by introducing multiplication and the integer preorder. As before, the definitions are kept close to the pair model so that the order-theoretic and algebraic properties can be proved by direct computation.

Natural Multiplication

The starting point is multiplication on $\mathbb{N}$, together with the unit and zero laws that will later feed into the corresponding integer statements.

```
-- § Multiplication on  $\mathbb{N}$  (separate from BUILTIN *__).
infixl 7 *_N_

*_N_ :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
zero *_N n = zero
suc m *_N n = n +_N (m *_N n)

-- § Right identity for *_N.
*_N-one-right : (n :  $\mathbb{N}$ )  $\rightarrow$  n *_N suc zero  $\equiv$  n
*_N-one-right zero = refl
*_N-one-right (suc n) = cong suc (*_N-one-right n)

-- § Right annihilation for *_N.
*_N-zero-right : (n :  $\mathbb{N}$ )  $\rightarrow$  n *_N zero  $\equiv$  zero
*_N-zero-right zero = refl
*_N-zero-right (suc n) = *_N-zero-right n

-- § Left annihilation for *_N.
*_N-zero-left : (n :  $\mathbb{N}$ )  $\rightarrow$  zero *_N n  $\equiv$  zero
*_N-zero-left n = refl

-- § Left identity for +_N.
```

```

+ℕ-zero-left : (n : ℕ) → zero +ℕ n ≡ n
+ℕ-zero-left n = refl

```

Integer Multiplication

Integer multiplication is defined first on pairs in signed form and then transported back through normalization. In this way the construction remains computationally transparent while multiplication is still presented as an operation on normalized integers.

```

-- § Pair-level multiplication.
Pairℕ-mul : Pairℕ → Pairℕ → Pairℕ
Pairℕ-mul p q =
  let a = pos p in
  let b = neg p in
  let c = pos q in
  let d = neg q in
  mkPairℕ ((a *ℕ c) +ℕ (b *ℕ d)) ((a *ℕ d) +ℕ (b *ℕ c))

-- § Integer one.
oneℤ : ℤ
oneℤ = +suc zero

-- § Natural one.
oneNat : ℕ
oneNat = suc zero

-- § Round-trip: fromPairℤ o toPairℤ = id.
from-toPairℤ : (z : ℤ) → fromPairℤ (toPairℤ z) ≡ z
from-toPairℤ 0ℤ = refl
from-toPairℤ (+suc n) = refl
from-toPairℤ (-suc n) = refl

-- § Integer multiplication (opaque for reduction control).
infixl 7 *_ℤ_

opaque
*_ℤ_ : ℤ → ℤ → ℤ
x *_ℤ_ y = fromPairℤ (Pairℕ-mul (toPairℤ x) (toPairℤ y))

opaque
unfolding *_ℤ_
-- § Left annihilation for *_ℤ_.
*ℤ-zero-left : (y : ℤ) → 0ℤ *_ℤ_ y ≡ 0ℤ
*ℤ-zero-left y = refl

-- § Right annihilation for *_ℤ_.
*ℤ-zero-right : (x : ℤ) → x *_ℤ_ 0ℤ ≡ 0ℤ
*ℤ-zero-right 0ℤ = refl
*ℤ-zero-right (+suc n) =
  normalizeℤ-cong

```

```

(trans
  (+N-zero-right (suc n *N zero))
  (*N-zero-right (suc n)))
(trans
  (+N-zero-right (suc n *N zero))
  (*N-zero-right (suc n)))
*Z-zero-right (-suc n) =
normalizeZ-cong
(*N-zero-right (suc n))
(*N-zero-right (suc n))

-- § Right identity for *Z.
*Z-one-right : (x : Z) → x *Z oneZ ≡ x
*Z-one-right 0Z = refl
*Z-one-right (+suc n) =
normalizeZ-cong
(trans
  (+N-zero-right (suc n *N oneNat))
  (*N-one-right (suc n)))
(trans
  (+N-zero-right (suc n *N zero))
  (*N-zero-right (suc n)))
*Z-one-right (-suc n) =
normalizeZ-cong
(*N-zero-right (suc n))
(trans
  (+N-zero-left (suc n *N oneNat))
  (*N-one-right (suc n)))

```

Integer Preorder

The preorder $\leq_{\mathbb{Z}}$ is defined by case distinction on the signed normal form together with the order on $\mathbb{N}$. The following lemmas show that reflexivity, transitivity, anti-symmetry, and totality follow from that definition.

```

-- § Reflexivity of ≤Z.
≤Z-refl : (x : Z) → x ≤Z x
≤Z-refl 0Z = tt
≤Z-refl (+suc n) = ≤-refl (suc n)
≤Z-refl (-suc n) = ≤-refl (suc n)

-- § Transitivity of ≤Z.
≤Z-trans : {x y z : Z} → x ≤Z y → y ≤Z z → x ≤Z z
≤Z-trans {0Z} {0Z} {0Z} _ _ = tt
≤Z-trans {0Z} {0Z} {+suc n} _ _ = tt
≤Z-trans {0Z} {0Z} {-suc n} _ ()
≤Z-trans {0Z} {+suc m} {0Z} _ ()
≤Z-trans {0Z} {+suc m} {+suc n} _ _ = tt
≤Z-trans {0Z} {+suc m} {-suc n} _ ()
≤Z-trans {0Z} {-suc m} {0Z} _ _ = tt
≤Z-trans {0Z} {-suc m} {+suc n} _ _ = tt

```

```

≤ℤ-trans {0ℤ} { -suc m } { -suc n } () _

≤ℤ-trans {+suc m} {0ℤ} {z} () _
≤ℤ-trans {+suc m} {+suc n} {0ℤ} p ()
≤ℤ-trans {+suc m} {+suc n} {+suc k} p q = ≤-trans p q
≤ℤ-trans {+suc m} {+suc n} { -suc k } _ ()
≤ℤ-trans {+suc m} { -suc n } {z} () _

≤ℤ-trans { -suc m } {0ℤ} {0ℤ} _ _ = tt
≤ℤ-trans { -suc m } {0ℤ} {+suc k} _ _ = tt
≤ℤ-trans { -suc m } {0ℤ} { -suc k } _ ()
≤ℤ-trans { -suc m } {+suc n} {0ℤ} _ ()
≤ℤ-trans { -suc m } {+suc n} {+suc k} _ _ = tt
≤ℤ-trans { -suc m } {+suc n} { -suc k } _ ()
≤ℤ-trans { -suc m } { -suc n } {0ℤ} _ _ = tt
≤ℤ-trans { -suc m } { -suc n } {+suc k} _ _ = tt
≤ℤ-trans { -suc m } { -suc n } { -suc k } p q = ≤-trans q p

-- § Strict order implies weak order.
<ℤ→≤ℤ : {x y : ℤ} → x <ℤ y → x ≤ℤ y
<ℤ→≤ℤ p = fst p

-- § Antisymmetry of ≤ℤ.
≤ℤ-antisym : {x y : ℤ} → x ≤ℤ y → y ≤ℤ x → x ≡ y
≤ℤ-antisym {0ℤ} {0ℤ} _ _ = refl
≤ℤ-antisym {0ℤ} {+suc n} _ ()
≤ℤ-antisym {0ℤ} { -suc n } () _
≤ℤ-antisym {+suc m} {0ℤ} () _
≤ℤ-antisym {+suc m} {+suc n} p q = cong +suc_ (suc-injective (≤-antisym p q))
≤ℤ-antisym {+suc m} { -suc n } () _
≤ℤ-antisym { -suc m } {0ℤ} _ ()
≤ℤ-antisym { -suc m } {+suc n} _ ()
≤ℤ-antisym { -suc m } { -suc n } p q = cong -suc_ (suc-injective (≤-antisym q p))

-- § Totality of ≤ℤ.
≤ℤ-total : (x y : ℤ) → (x ≤ℤ y) ∨ (y ≤ℤ x)
≤ℤ-total 0ℤ 0ℤ = inj₁ tt
≤ℤ-total 0ℤ (+suc _) = inj₁ tt
≤ℤ-total 0ℤ (-suc _) = inj₂ tt
≤ℤ-total (+suc _) 0ℤ = inj₂ tt
≤ℤ-total (+suc m) (+suc n) with ≤-total (suc m) (suc n)
... | inj₁ p = inj₁ p
... | inj₂ p = inj₂ p
≤ℤ-total (+suc _) (-suc _) = inj₂ tt
≤ℤ-total (-suc _) 0ℤ = inj₁ tt
≤ℤ-total (-suc _) (+suc _) = inj₁ tt
≤ℤ-total (-suc m) (-suc n) with ≤-total (suc n) (suc m)
... | inj₁ p = inj₁ p
... | inj₂ p = inj₂ p

```

Chapter 32

K_4 Neighborhood and Laplacian

The next chapter returns from the global arithmetic structure to the local combinatorics of K_4 . The first task is to enumerate the neighbours of each vertex explicitly; the second is to package this data into the combinatorial Laplacian.

Neighbourhood Structure

Step 1: Adjacency and symmetric inequality witnesses. Adjacency in the canonical graph is just inequality. For later calculations, however, it is convenient to isolate the corresponding witnesses and to organize the three neighbours of each vertex into a fixed record.

```
-- § Adjacency on  $K_4$ .
Adj : EndoCase → EndoCase → Set
Adj a b = Edge K4GraphCanonical a b

-- § Complete neighbour triple: every  $K_4$  vertex has exactly 3 neighbours.
record NeighborTriple (v : EndoCase) : Set where
  field
    n1 n2 n3 : EndoCase
    adj1 : Adj v n1
    adj2 : Adj v n2
    adj3 : Adj v n3
    n1 ≠ n2 : n1 ≠ n2
    n1 ≠ n3 : n1 ≠ n3
    n2 ≠ n3 : n2 ≠ n3
    complete : (u : EndoCase) → Adj v u → (u ≡ n1) ⊔ ((u ≡ n2) ⊔ (u ≡ n3))

open NeighborTriple public

-- § Symmetric inequality witnesses.
```

```

case-constR≠case-constL : case-constR ≠ case-constL
case-constR≠case-constL eq = case-constL≠case-constR (sym eq)

case-id≠case-constL : case-id ≠ case-constL
case-id≠case-constL eq = case-constL≠case-id (sym eq)

case-dual≠case-constL : case-dual ≠ case-constL
case-dual≠case-constL eq = case-constL≠case-dual (sym eq)

case-id≠case-constR : case-id ≠ case-constR
case-id≠case-constR eq = case-constR≠case-id (sym eq)

case-dual≠case-constR : case-dual ≠ case-constR
case-dual≠case-constR eq = case-constR≠case-dual (sym eq)

case-dual≠case-id : case-dual ≠ case-id
case-dual≠case-id eq = case-id≠case-dual (sym eq)

```

Step 2: Law 14.0 — Complete neighbour triple for each vertex. The complete neighbour triple is then written out explicitly for each of the four vertices. Because the graph is finite, the completeness proof is a direct case split.

-- § Law 14.0: Every vertex has a complete neighbour triple.

law14-0-neighbor-triple : (v : EndoCase) → NeighborTriple v

law14-0-neighbor-triple case-constL = record

```

{ n1 = case-constR
; n2 = case-id
; n3 = case-dual
; adj1 = case-constL≠case-constR
; adj2 = case-constL≠case-id
; adj3 = case-constL≠case-dual
; n1≠n2 = case-constR≠case-id
; n1≠n3 = case-constR≠case-dual
; n2≠n3 = case-id≠case-dual
; complete = λ
  { case-constL adj → ⊥-elim (adj refl)
  ; case-constR adj → inj1 refl
  ; case-id adj → inj2 (inj1 refl)
  ; case-dual adj → inj2 (inj2 refl)
  }
}

```

law14-0-neighbor-triple case-constR = record

```

{ n1 = case-constL
; n2 = case-id
; n3 = case-dual
; adj1 = case-constR≠case-constL
; adj2 = case-constR≠case-id
; adj3 = case-constR≠case-dual
; n1≠n2 = case-constL≠case-id
; n1≠n3 = case-constL≠case-dual
; n2≠n3 = case-id≠case-dual
; complete = λ

```

```

    { case-constL adj → inj1 refl
    ; case-constR adj → ⊥-elim (adj refl)
    ; case-id    adj → inj2 (inj1 refl)
    ; case-dual  adj → inj2 (inj2 refl)
    }
  }
law14-0-neighbor-triple case-id = record
{ n1 = case-constL
; n2 = case-constR
; n3 = case-dual
; adj1 = case-id ≠ case-constL
; adj2 = case-id ≠ case-constR
; adj3 = case-id ≠ case-dual
; n1 ≠ n2 = case-constL ≠ case-constR
; n1 ≠ n3 = case-constL ≠ case-dual
; n2 ≠ n3 = case-constR ≠ case-dual
; complete = λ
  { case-constL adj → inj1 refl
  ; case-constR adj → inj2 (inj1 refl)
  ; case-id    adj → ⊥-elim (adj refl)
  ; case-dual  adj → inj2 (inj2 refl)
  }
}
law14-0-neighbor-triple case-dual = record
{ n1 = case-constL
; n2 = case-constR
; n3 = case-id
; adj1 = case-dual ≠ case-constL
; adj2 = case-dual ≠ case-constR
; adj3 = case-dual ≠ case-id
; n1 ≠ n2 = case-constL ≠ case-constR
; n1 ≠ n3 = case-constL ≠ case-id
; n2 ≠ n3 = case-constR ≠ case-id
; complete = λ
  { case-constL adj → inj1 refl
  ; case-constR adj → inj2 (inj1 refl)
  ; case-id    adj → inj2 (inj2 refl)
  ; case-dual  adj → ⊥-elim (adj refl)
  }
}

-- § Neighbour lookup by Fin3 index.
neighborAt : (v : EndoCase) → Fin3 → EndoCase
neighborAt v f0 = n1 (law14-0-neighbor-triple v)
neighborAt v f1 = n2 (law14-0-neighbor-triple v)
neighborAt v f2 = n3 (law14-0-neighbor-triple v)

-- § Neighbours are adjacent.
neighborAt-adj : (v : EndoCase) → (i : Fin3) → Adj v (neighborAt v i)
neighborAt-adj v f0 = adj1 (law14-0-neighbor-triple v)
neighborAt-adj v f1 = adj2 (law14-0-neighbor-triple v)
neighborAt-adj v f2 = adj3 (law14-0-neighbor-triple v)

-- § Neighbour lookup is injective.

```

```

neighborAt-injective : (v : EndoCase) → {i j : Fin3} → neighborAt v i ≡ neighborAt v j → i ≡ j
neighborAt-injective v {f0} {f0} _ = refl
neighborAt-injective v {f1} {f1} _ = refl
neighborAt-injective v {f2} {f2} _ = refl
neighborAt-injective v {f0} {f1} eq = ⊥-elim (n1 ≠ n2 (law14-0-neighbor-triple v) eq)
neighborAt-injective v {f1} {f0} eq = ⊥-elim (n1 ≠ n2 (law14-0-neighbor-triple v) (sym eq))
neighborAt-injective v {f0} {f2} eq = ⊥-elim (n1 ≠ n3 (law14-0-neighbor-triple v) eq)
neighborAt-injective v {f2} {f0} eq = ⊥-elim (n1 ≠ n3 (law14-0-neighbor-triple v) (sym eq))
neighborAt-injective v {f1} {f2} eq = ⊥-elim (n2 ≠ n3 (law14-0-neighbor-triple v) eq)
neighborAt-injective v {f2} {f1} eq = ⊥-elim (n2 ≠ n3 (law14-0-neighbor-triple v) (sym eq))

```

Laplacian Operator

Once neighbour lists are available, the Laplacian can be defined in the combinatorial form as degree minus adjacency sum. The definitions below write both terms explicitly, using the finite sums already introduced over `Fin3`.

```

-- § Adjacency sum: sum of f over the three neighbours.
adjSumℤ : (EndoCase → ℤ) → EndoCase → ℤ
adjSumℤ f v = sumFin3ℤ (λ i → f (neighborAt v i))

-- § Degree term: 3 × f(v).
deg3ℤ : (EndoCase → ℤ) → EndoCase → ℤ
deg3ℤ f v = sum3ℤ (f v) (f v) (f v)

-- § Combinatorial Laplacian: deg - adj.
laplacianℤ : (EndoCase → ℤ) → EndoCase → ℤ
laplacianℤ f v = deg3ℤ f v + ℤ negℤ (adjSumℤ f v)

```


Chapter 33

Simplex Invariants and Natural Multiplication Laws

This chapter brings together two strands that now meet in the current numerical profile of the tetrahedron and the multiplication laws on natural numbers needed to manipulate that profile internally. The aim is to show that the standard constants of K_4 arise from the same finite infrastructure already developed in the book.

Simplex Invariants

The tetrahedral constants are now collected in one place. Starting from the two-element carrier of a distinction, the four endomorphism classes yield the four vertices of K_4 , and from there the degree, edge count, face count, and Euler characteristic follow by straightforward finite arithmetic.

```
-- § Distinction carrier size.
states-per-distinction : ℕ
states-per-distinction = 2

-- § Triangle number without division.
triangle : ℕ → ℕ
triangle zero = 0
triangle (suc n) = n + triangle n

-- § Simplex vertex count.
simplex-vertices : ℕ
simplex-vertices = states-per-distinction * states-per-distinction

-- § Simplex degree:  $K_n$  is  $(n-1)$ -regular.
simplex-degree : ℕ
simplex-degree = simplex-vertices ÷ 1
```

```

-- § Simplex edge count:  $T(V) = T(4) = 0+1+2+3 = 6$ .
simplex-edges :  $\mathbb{N}$ 
simplex-edges = triangle simplex-vertices

-- § Simplex face count.
simplex-faces :  $\mathbb{N}$ 
simplex-faces = simplex-vertices

-- § Simplex Euler characteristic:  $\chi = V - E + F$ .
simplex-chi :  $\mathbb{N}$ 
simplex-chi = (simplex-vertices + simplex-faces) - simplex-edges

-- § Law 14C.0: Vertex count = 4 (derived from  $2^2 = 4$ ).
law14C-0-vertices : simplex-vertices  $\equiv$  4
law14C-0-vertices = refl

-- § Law 14C.1: Degree = 3 (derived from  $V - 1 = 3$ ).
law14C-1-degree : simplex-degree  $\equiv$  3
law14C-1-degree = refl

-- § Law 14C.2: Edge count = 6 (derived from  $T(4) = 6$ ).
law14C-2-edges : simplex-edges  $\equiv$  6
law14C-2-edges = refl

-- § Law 14C.2a: Face count = 4 (self-duality).
law14C-2a-faces : simplex-faces  $\equiv$  4
law14C-2a-faces = refl

-- § Law 14C.3: Euler characteristic = 2 (derived from  $V - E + F = 2$ ).
law14C-3-chi : simplex-chi  $\equiv$  2
law14C-3-chi = refl

-- § The carrier size is fixed by the distinction record.
law14C-4-carrier-is-2 : states-per-distinction  $\equiv$  2
law14C-4-carrier-is-2 = refl

```

Distributivity and Associativity

The next lemmas establish distributivity and associativity for multiplication on $\mathbb{N}$. These are the algebraic facts required for later manipulations of the simplex constants and for the integer theory built on top of them.

```

-- § Left distributivity of  $\ast \mathbb{N}$  over  $+\mathbb{N}$ .
*N-distrib-left-+N : (a b c :  $\mathbb{N}$ )  $\rightarrow$  (a +N b) *N c  $\equiv$  (a *N c) +N (b *N c)
*N-distrib-left-+N zero b c =
trans
  refl
  (sym (+N-zero-left (b *N c)))
*N-distrib-left-+N (suc a) b c =
trans
  refl

```

```

(trans
  (cong (λ t → c +ℕ t) (*ℕ-distrib-left-+ℕ a b c))
  (sym (+ℕ-assoc c (a *ℕ c) (b *ℕ c))))

-- § Head swap for ℕ sums.
swapHeadℕ : (a b t : ℕ) → a +ℕ (b +ℕ t) ≡ b +ℕ (a +ℕ t)
swapHeadℕ a b t =
  trans
    (sym (+ℕ-assoc a b t))
    (trans
      (cong (λ s → s +ℕ t) (+ℕ-comm a b))
      (+ℕ-assoc b a t))

-- § Four-way shuffle for ℕ.
shuffleℕ : (b c x y : ℕ) → (b +ℕ c) +ℕ (x +ℕ y) ≡ (b +ℕ x) +ℕ (c +ℕ y)
shuffleℕ b c x y =
  trans
    (+ℕ-assoc b c (x +ℕ y))
    (trans
      (cong (λ t → b +ℕ t) (sym (+ℕ-assoc c x y)))
      (trans
        (cong (λ t → b +ℕ (t +ℕ y)) (+ℕ-comm c x))
        (trans
          (cong (λ t → b +ℕ t) (+ℕ-assoc x c y))
          (sym (+ℕ-assoc b x (c +ℕ y)))))))

-- § Right distributivity of *ℕ over +ℕ.
*ℕ-distrib-right-+ℕ : (a b c : ℕ) → a *ℕ (b +ℕ c) ≡ (a *ℕ b) +ℕ (a *ℕ c)
*ℕ-distrib-right-+ℕ zero b c = refl
*ℕ-distrib-right-+ℕ (suc a) b c =
  trans
    refl
    (trans
      (cong (λ t → (b +ℕ c) +ℕ t) (*ℕ-distrib-right-+ℕ a b c))
      (trans
        (shuffleℕ b c (a *ℕ b) (a *ℕ c))
        refl))

-- § Associativity of *ℕ.
*ℕ-assoc : (a b c : ℕ) → (a *ℕ b) *ℕ c ≡ a *ℕ (b *ℕ c)
*ℕ-assoc zero b c = refl
*ℕ-assoc (suc a) b c =
  trans
    (*ℕ-distrib-left-+ℕ b (a *ℕ b) c)
    (trans
      (cong (λ t → (b *ℕ c) +ℕ t) (*ℕ-assoc a b c))
      refl)

-- § Zero cancellation for positive left factor.
*ℕ-pos-zero→zero : (a n : ℕ) → suc a *ℕ n ≡ zero → n ≡ zero
*ℕ-pos-zero→zero a zero _ = refl
*ℕ-pos-zero→zero a (suc n) ()

```

Commutativity and Monotonicity

After distributivity and associativity, the remaining structural properties of natural multiplication are commutativity and monotonicity with respect to order. These are proved directly from the recursive definition together with the addition laws already established.

```
-- § Successor-right law for *N.
*N-suc-right-+N : (n m : N) → n *N (suc m) ≡ n +N (n *N m)
*N-suc-right-+N zero m = refl
*N-suc-right-+N (suc n) m =
  trans
    refl
  (trans
    (cong (λ t → (suc m) +N t) (*N-suc-right-+N n m))
    (cong suc (swapHeadN m n (n *N m)))))

-- § Commutativity of *N.
*N-comm : (m n : N) → m *N n ≡ n *N m
*N-comm zero n = sym (*N-zero-right n)
*N-comm (suc m) n =
  trans
    refl
  (trans
    (cong (λ t → n +N t) (*N-comm m n))
    (sym (*N-suc-right-+N n m)))

-- § +N is monotone on the left.
≤-+N-monol : {a b : N} → a ≤ b → (c : N) → (c +N a) ≤ (c +N b)
≤-+N-monol p zero = p
≤-+N-monol p (suc c) = s ≤ s (≤-+N-monol p c)

-- § Left cancellation for +N ordering.
≤-+N-cancell : (c a b : N) → (c +N a) ≤ (c +N b) → a ≤ b
≤-+N-cancell zero a b p = p
≤-+N-cancell (suc c) a b (s ≤ s p) = ≤-+N-cancell c a b p

-- § *N is monotone in the left factor.
≤-*N-monor : {m n : N} → m ≤ n → (t : N) → (m *N t) ≤ (n *N t)
≤-*N-monor z ≤ n t = z ≤ n
≤-*N-monor (s ≤ s p) t = ≤-+N-monol (≤-*N-monor p t) t

-- § Right cancellation by a positive (successor) factor.
≤-*N-cancelr-suc : {m n : N} → (k : N) → (m *N suc k) ≤ (n *N suc k) → m ≤ n
≤-*N-cancelr-suc {zero} {zero} k _ = z ≤ n
≤-*N-cancelr-suc {suc m} {zero} k ()
≤-*N-cancelr-suc {zero} {suc n} k _ = z ≤ n
≤-*N-cancelr-suc {suc m} {suc n} k p =
  let
    step : (suc k +N (m *N suc k)) ≤ (suc k +N (n *N suc k))
    step = p
  tail : (m *N suc k) ≤ (n *N suc k)
```

```

tail = ≤-+ℕ-cancell (suc k) (m *ℕ suc k) (n *ℕ suc k) step

ih : m ≤ n
ih = ≤-*ℕ-cancelr-suc k tail
in
s≤s ih

```

Positive Naturals

Finally, it is convenient to isolate the positive naturals as a separate type. The definition below packages a natural number together with an implicit successor, so that every inhabitant represents a value at least 1.

```

-- § Positive natural: predecessor + implicit successor.
record ℕ+ : Set where
  constructor mkℕ+
  field
    pred : ℕ

PosNat : Set
PosNat = ℕ+

open ℕ+ public

-- § Embedding into ℕ (always ≥ 1).
+toℕ : ℕ+ → ℕ
+toℕ n = suc (pred n)

-- § Positive one.
one+ : ℕ+
one+ = mkℕ+ zero

-- § Successor on ℕ+.
suc+ : ℕ+ → ℕ+
suc+ n = mkℕ+ (suc (pred n))

-- § Addition on ℕ+.
_++_ : ℕ+ → ℕ+ → ℕ+
mkℕ+ a ++ mkℕ+ b = mkℕ+ (a +ℕ suc b)

-- § Multiplication on ℕ+: (1+a)(1+b) = 1 + (a·(1+b) + b).
_*+_ : ℕ+ → ℕ+ → ℕ+
mkℕ+ a *+ mkℕ+ b = mkℕ+ ((a *ℕ suc b) +ℕ b)

-- § Embedding ℕ+ into ℤ.
+toℤ : ℕ+ → ℤ
+toℤ (mkℕ+ k) = +suc k

```


Chapter 34

Integer Multiplication Laws

The next task is to prove the basic laws of integer multiplication. The strategy is to work first with pair representatives, where the algebra is completely explicit, and then pass back to normalized integers through normalization.

Pair-Level Addition and Normalization

The first step is to compare componentwise pair addition with normalization. These lemmas explain how raw pair representatives interact with the normalized form used for integers.

```
-- § Componentwise pair addition.
PairN-add : PairN → PairN → PairN
PairN-add p q = (mkPairN (pos p +N pos q) (neg p +N neg q))

-- § Normalization: cancel equal components.
normalizePair : PairN → PairN
normalizePair (mkPairN zero zero) = (mkPairN zero zero)
normalizePair (mkPairN (suc a) zero) = (mkPairN (suc a) zero)
normalizePair (mkPairN zero (suc b)) = (mkPairN zero (suc b))
normalizePair (mkPairN (suc a) (suc b)) = normalizePair (mkPairN a b)

-- § normalizePair is identity on (x , 0).
normalizePair-right0 : (x : N) → normalizePair (mkPairN x zero) ≡ (mkPairN x zero)
normalizePair-right0 zero = refl
normalizePair-right0 (suc a) = refl

-- § normalizePair is identity on (0 , y).
normalizePair-left0 : (y : N) → normalizePair (mkPairN zero y) ≡ (mkPairN zero y)
normalizePair-left0 zero = refl
normalizePair-left0 (suc b) = refl

-- § fromPairZ absorbs normalizePair.
fromPair-normalizePair : (p : PairN) → fromPairZ (normalizePair p) ≡ fromPairZ p
```

```

fromPair-normalizePair (mkPairℕ zero zero) = refl
fromPair-normalizePair (mkPairℕ (suc a) zero) = refl
fromPair-normalizePair (mkPairℕ zero (suc b)) = refl
fromPair-normalizePair (mkPairℕ (suc a) (suc b)) = fromPair-normalizePair (mkPairℕ a b)

-- § toPairℤ o normalizeℤ = normalizePair o mkPairℕ.
toPair-normalizeℤ : (a b : ℕ) → toPairℤ (normalizeℤ a b) ≡ normalizePair (mkPairℕ a b)
toPair-normalizeℤ zero zero = refl
toPair-normalizeℤ (suc a) zero = refl
toPair-normalizeℤ zero (suc b) = refl
toPair-normalizeℤ (suc a) (suc b) = toPair-normalizeℤ a b

-- § toPairℤ o fromPairℤ = normalizePair.
toPair-fromPair : (p : Pairℕ) → toPairℤ (fromPairℤ p) ≡ normalizePair p
toPair-fromPair (mkPairℕ a b) = toPair-normalizeℤ a b

```

Pair-Level Distributivity

Distributivity is easiest to establish before returning to normalized integers. At the pair level, the calculation reduces directly to distributivity and associativity on $\mathbb{N}$.

```

-- § Right distributivity at pair level.
Pairℕ-mul-distrib-right-add : (p q r : Pairℕ) →
  Pairℕ-mul p (Pairℕ-add q r) ≡ Pairℕ-add (Pairℕ-mul p q) (Pairℕ-mul p r)
Pairℕ-mul-distrib-right-add p q r =
  let a = pos p in
  let b = neg p in
  let c = pos q in
  let d = neg q in
  let e = pos r in
  let f = neg r in
  pair-ext
  (pos-proof a b c d e f)
  (neg-proof a b c d e f)
where
  pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
  pair-ext refl refl = refl

  pos-proof : (a b c d e f : ℕ) →
    ((a *ℕ (c +ℕ e)) +ℕ (b *ℕ (d +ℕ f)))
    ≡
    (((a *ℕ c) +ℕ (b *ℕ d)) +ℕ ((a *ℕ e) +ℕ (b *ℕ f)))
  pos-proof a b c d e f =
    trans
    (cong (λ t → t +ℕ (b *ℕ (d +ℕ f)))) (*ℕ-distrib-right-+ℕ a c e)
    (trans
    (cong (λ t → (a *ℕ c +ℕ a *ℕ e) +ℕ t) (*ℕ-distrib-right-+ℕ b d f))
    (shuffleℕ (a *ℕ c) (a *ℕ e) (b *ℕ d) (b *ℕ f))))

  neg-proof : (a b c d e f : ℕ) →
    ((a *ℕ (d +ℕ f)) +ℕ (b *ℕ (c +ℕ e)))

```



```

≡
(((a * $\mathbb{N}$  d) + $\mathbb{N}$  (b * $\mathbb{N}$  c)) + $\mathbb{N}$  ((a * $\mathbb{N}$  f) + $\mathbb{N}$  (b * $\mathbb{N}$  e)))
neg-proof a b c d e f =
trans
  (cong (λ t → t + $\mathbb{N}$  (b * $\mathbb{N}$  (c + $\mathbb{N}$  e))) (* $\mathbb{N}$ -distrib-right-+ $\mathbb{N}$  a d f))
  (trans
    (cong (λ t → (a * $\mathbb{N}$  d + $\mathbb{N}$  a * $\mathbb{N}$  f) + $\mathbb{N}$  t) (* $\mathbb{N}$ -distrib-right-+ $\mathbb{N}$  b c e))
    (shuffle $\mathbb{N}$  (a * $\mathbb{N}$  d) (a * $\mathbb{N}$  f) (b * $\mathbb{N}$  c) (b * $\mathbb{N}$  e)))

-- § Left identity for * $\mathbb{N}$ .
* $\mathbb{N}$ -one-left : (n :  $\mathbb{N}$ ) → oneNat * $\mathbb{N}$  n ≡ n
* $\mathbb{N}$ -one-left n = + $\mathbb{N}$ -zero-right n

opaque
unfolding _* $\mathbb{Z}$ _
-- § Left identity for * $\mathbb{Z}$ .
* $\mathbb{Z}$ -one-left : (x :  $\mathbb{Z}$ ) → one $\mathbb{Z}$  * $\mathbb{Z}$  x ≡ x
* $\mathbb{Z}$ -one-left 0 $\mathbb{Z}$  = refl
* $\mathbb{Z}$ -one-left (+suc n) =
  normalize $\mathbb{Z}$ -cong
  (trans
    (+ $\mathbb{N}$ -zero-right (oneNat * $\mathbb{N}$  suc n))
    (* $\mathbb{N}$ -one-left (suc n)))
  (trans
    (+ $\mathbb{N}$ -zero-right (oneNat * $\mathbb{N}$  zero))
    (* $\mathbb{N}$ -zero-right oneNat))
* $\mathbb{Z}$ -one-left (-suc n) =
  normalize $\mathbb{Z}$ -cong
  (trans
    (+ $\mathbb{N}$ -zero-right (oneNat * $\mathbb{N}$  zero))
    (* $\mathbb{N}$ -zero-right oneNat))
  (trans
    (+ $\mathbb{N}$ -zero-right (oneNat * $\mathbb{N}$  suc n))
    (* $\mathbb{N}$ -one-left (suc n)))

-- § Left distributivity at pair level.
Pair $\mathbb{N}$ -mul-distrib-left-add : (p q r : Pair $\mathbb{N}$ ) →
  Pair $\mathbb{N}$ -mul (Pair $\mathbb{N}$ -add p q) r ≡ Pair $\mathbb{N}$ -add (Pair $\mathbb{N}$ -mul p r) (Pair $\mathbb{N}$ -mul q r)
Pair $\mathbb{N}$ -mul-distrib-left-add p q r =
  let a = pos p in
  let b = neg p in
  let c = pos q in
  let d = neg q in
  let e = pos r in
  let f = neg r in
  pair-ext
    (pos-proof a b c d e f)
    (neg-proof a b c d e f)
  where
  pair-ext : {x y x' y' :  $\mathbb{N}$ } → x ≡ x' → y ≡ y' → (mkPair $\mathbb{N}$  x y) ≡ (mkPair $\mathbb{N}$  x' y')
  pair-ext refl refl = refl

pos-proof : (a b c d e f :  $\mathbb{N}$ ) →
  (((a + $\mathbb{N}$  c) * $\mathbb{N}$  e) + $\mathbb{N}$  ((b + $\mathbb{N}$  d) * $\mathbb{N}$  f))

```

```

≡
(((a *N e) +N (b *N f)) +N ((c *N e) +N (d *N f)))
pos-proof a b c d e f =
trans
  (cong (λ t → t +N ((b +N d) *N f)) (*N-distrib-left-+N a c e))
  (trans
    (cong (λ t → ((a *N e) +N (c *N e)) +N t) (*N-distrib-left-+N b d f))
    (shuffleN (a *N e) (c *N e) (b *N f) (d *N f)))

neg-proof : (a b c d e f : N) →
  (((a +N c) *N f) +N ((b +N d) *N e))
≡
(((a *N f) +N (b *N e)) +N ((c *N f) +N (d *N e)))
neg-proof a b c d e f =
trans
  (cong (λ t → t +N ((b +N d) *N e)) (*N-distrib-left-+N a c f))
  (trans
    (cong (λ t → ((a *N f) +N (c *N f)) +N t) (*N-distrib-left-+N b d e))
    (shuffleN (a *N f) (c *N f) (b *N e) (d *N e)))

```

Diagonal Cancellation and Normalization Stability

Normalization ignores common diagonal contributions, so the next lemmas show that adding the same amount to both components does not change the represented integer. This is the key stability property needed for the compatibility of multiplication with normalization.

```

-- § Diagonal addition is absorbed by normalization.
normalizePair-addDiag : (p : PairN) → (k : N) →
  normalizePair (PairN-add p (mkPairN k k)) ≡ normalizePair p
normalizePair-addDiag (mkPairN a b) zero =
  cong normalizePair (pair-ext (+N-zero-right a) (+N-zero-right b))
where
  pair-ext : {x y x' y' : N} → x ≡ x' → y ≡ y' → (mkPairN x y) ≡ (mkPairN x' y')
  pair-ext refl refl = refl
normalizePair-addDiag (mkPairN a b) (suc k) =
  trans
    (cong normalizePair (pair-ext (+N-suc-right a k) (+N-suc-right b k)))
    (trans
      refl
      (normalizePair-addDiag (mkPairN a b) k))
where
  pair-ext : {x y x' y' : N} → x ≡ x' → y ≡ y' → (mkPairN x y) ≡ (mkPairN x' y')
  pair-ext refl refl = refl

-- § Successor-right via +N one.
+N-one-right : (n : N) → n +N suc zero ≡ suc n
+N-one-right n = trans (+N-comm n (suc zero)) refl

-- § *N successor-right.

```

```

*N-suc-right : (a n : ℕ) → a *ℕ suc n ≡ (a *ℕ n) +ℕ a
*N-suc-right a n =
trans
  (cong (λ t → a *ℕ t) (sym (+ℕ-one-right n)))
  (trans
    (*ℕ-distrib-right-+ℕ a n (suc zero))
    (cong (λ t → (a *ℕ n) +ℕ t) (*ℕ-one-right a)))

-- § *ℕ successor-left.
*N-suc-left : (n a : ℕ) → suc n *ℕ a ≡ (n *ℕ a) +ℕ a
*N-suc-left n a =
trans
  (cong (λ t → t *ℕ a) (sym (+ℕ-one-right n)))
  (trans
    (*ℕ-distrib-left-+ℕ n (suc zero) a)
    (cong (λ t → (n *ℕ a) +ℕ t) (*ℕ-one-left a)))

```

Step 2: Pair-Level Cancellation. The same idea can be applied inside pair multiplication: common successor contributions in the two coordinates produce only a diagonal term and are therefore removed by normalization.

```

-- § Right cancellation at pair level: common successor adds a diagonal.
Pairℕ-mul-cancelRight : (p : Pairℕ) → (c d : ℕ) →
  normalizePair (Pairℕ-mul p (mkPairℕ (suc c) (suc d))) ≡ normalizePair (Pairℕ-mul p (mkPairℕ c d))
Pairℕ-mul-cancelRight p c d =
let a = pos p in
let b = neg p in
trans
  (cong normalizePair (pair-ext (pos-step a b c d) (neg-step a b c d)))
  (normalizePair-addDiag (Pairℕ-mul p (mkPairℕ c d)) (a +ℕ b))
where
pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
pair-ext refl refl = refl

pos-step : (a b c d : ℕ) →
  ((a *ℕ suc c) +ℕ (b *ℕ suc d))
  ≡
  (((a *ℕ c) +ℕ (b *ℕ d)) +ℕ (a +ℕ b))
pos-step a b c d =
trans
  (cong (λ t → t +ℕ (b *ℕ suc d)) (*ℕ-suc-right a c))
  (trans
    (cong (λ t → ((a *ℕ c) +ℕ a) +ℕ t) (*ℕ-suc-right b d))
    (trans
      (shuffleℕ (a *ℕ c) a (b *ℕ d) b)
      refl)))

neg-step : (a b c d : ℕ) →
  ((a *ℕ suc d) +ℕ (b *ℕ suc c))
  ≡
  (((a *ℕ d) +ℕ (b *ℕ c)) +ℕ (a +ℕ b))
neg-step a b c d =

```

```

trans
  (cong (λ t → t +ℕ (b *ℕ suc c)) (*ℕ-suc-right a d))
(trans
  (cong (λ t → ((a *ℕ d) +ℕ a) +ℕ t) (*ℕ-suc-right b c))
  (trans
    (shuffleℕ (a *ℕ d) a (b *ℕ c) b)
    refl))

-- § Left cancellation at pair level.
Pairℕ-mul-cancelLeft : (q : Pairℕ) → (a b : ℕ) →
  normalizePair (Pairℕ-mul (mkPairℕ (suc a) (suc b)) q) ≡ normalizePair (Pairℕ-mul (mkPairℕ a b) q)
Pairℕ-mul-cancelLeft q a b =
  let c = pos q in
  let d = neg q in
  trans
    (cong normalizePair (pair-ext (pos-step a b c d) (neg-step a b c d)))
    (normalizePair-addDiag (Pairℕ-mul (mkPairℕ a b) q) (c +ℕ d))
  where
    pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
    pair-ext refl refl = refl

    pos-step : (a b c d : ℕ) →
      ((suc a *ℕ c) +ℕ (suc b *ℕ d))
      ≡
      (((a *ℕ c) +ℕ (b *ℕ d)) +ℕ (c +ℕ d))
    pos-step a b c d =
      trans
        (cong (λ t → t +ℕ (suc b *ℕ d)) (*ℕ-suc-left a c))
      (trans
        (cong (λ t → ((a *ℕ c) +ℕ c) +ℕ t) (*ℕ-suc-left b d))
        (trans
          (shuffleℕ (a *ℕ c) c (b *ℕ d) d)
          refl))

    neg-step : (a b c d : ℕ) →
      ((suc a *ℕ d) +ℕ (suc b *ℕ c))
      ≡
      (((a *ℕ d) +ℕ (b *ℕ c)) +ℕ (c +ℕ d))
    neg-step a b c d =
      trans
        (cong (λ t → t +ℕ (suc b *ℕ c)) (*ℕ-suc-left a d))
      (trans
        (cong (λ t → ((a *ℕ d) +ℕ d) +ℕ t) (*ℕ-suc-left b c))
        (trans
          (shuffleℕ (a *ℕ d) d (b *ℕ c) c)
          (cong (λ t → ((a *ℕ d) +ℕ (b *ℕ c)) +ℕ t) (+ℕ-comm d c))))

```

Step 3: Normalization Stability. From these cancellation lemmas one obtains the desired stability statement: normalizing either factor before pair multiplication does not change the normalized product.

```

-- § Normalization absorbs right factor normalization.
Pairℕ-mul-normalize-right : (p q : Pairℕ) →

```

```

normalizePair (PairN-mul p (normalizePair q)) ≡ normalizePair (PairN-mul p q)
PairN-mul-normalize-right p (mkPairN zero zero) = refl
PairN-mul-normalize-right p (mkPairN (suc a) zero) = refl
PairN-mul-normalize-right p (mkPairN zero (suc b)) = refl
PairN-mul-normalize-right p (mkPairN (suc a) (suc b)) =
  trans
    (PairN-mul-normalize-right p (mkPairN a b))
    (sym (PairN-mul-cancelRight p a b))

-- § Normalization absorbs left factor normalization.
PairN-mul-normalize-left : (p q : PairN) →
  normalizePair (PairN-mul (normalizePair p) q) ≡ normalizePair (PairN-mul p q)
PairN-mul-normalize-left (mkPairN zero zero) q = refl
PairN-mul-normalize-left (mkPairN (suc a) zero) q = refl
PairN-mul-normalize-left (mkPairN zero (suc b)) q = refl
PairN-mul-normalize-left (mkPairN (suc a) (suc b)) q =
  trans
    (PairN-mul-normalize-left (mkPairN a b) q)
    (sym (PairN-mul-cancelLeft q a b))

```

Normalized Pair Products and $\mathbb{Z}$ -Level Bridge

The next step is to connect the pair-level calculations back to normalized integers. For products of pairs obtained from $\mathbb{Z}$, no further cancellation is needed, and this yields the bridge between `PairN-mul` and the integer product `* $\mathbb{Z}$` .

```

-- § Products of normalized integer pairs are already normalized.
PairN-mul-toPair-normal : (x y :  $\mathbb{Z}$ ) →
  normalizePair (PairN-mul (toPair $\mathbb{Z}$  x) (toPair $\mathbb{Z}$  y)) ≡ PairN-mul (toPair $\mathbb{Z}$  x) (toPair $\mathbb{Z}$  y)
PairN-mul-toPair-normal 0 $\mathbb{Z}$  y = refl
PairN-mul-toPair-normal (+suc n) 0 $\mathbb{Z}$  =
  let mulEq : PairN-mul (toPair $\mathbb{Z}$  (+suc n)) (toPair $\mathbb{Z}$  0 $\mathbb{Z}$ ) ≡ (mkPairN zero zero)
      mulEq =
        pair-ext
          (trans (cong (λ t → t +N (zero *N zero)) (*N-zero-right (suc n))) refl)
          (trans (cong (λ t → t +N (zero *N zero)) (*N-zero-right (suc n))) refl)
  in
  trans (cong normalizePair mulEq) (trans (normalizePair-right0 zero) (sym mulEq))
where
  pair-ext : {x y x' y' :  $\mathbb{N}$ } → x ≡ x' → y ≡ y' → (mkPairN x y) ≡ (mkPairN x' y')
  pair-ext refl refl = refl
PairN-mul-toPair-normal (-suc n) 0 $\mathbb{Z}$  =
  let mulEq : PairN-mul (toPair $\mathbb{Z}$  (-suc n)) (toPair $\mathbb{Z}$  0 $\mathbb{Z}$ ) ≡ (mkPairN zero zero)
      mulEq =
        pair-ext
          (trans (cong (λ t → t +N (suc n *N zero)) refl) (*N-zero-right (suc n)))
          (trans (cong (λ t → t +N (suc n *N zero)) refl) (*N-zero-right (suc n)))
  in
  trans (cong normalizePair mulEq) (trans (normalizePair-right0 zero) (sym mulEq))
where

```

```

pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
pair-ext refl refl = refl
Pairℕ-mul-toPair-normal (+suc n) (+suc m) =
let mulEq : Pairℕ-mul (toPairℤ (+suc n)) (toPairℤ (+suc m)) ≡ (mkPairℕ (suc n *ℕ suc m) zero)
mulEq =
  pair-ext
    (+ℕ-zero-right (suc n *ℕ suc m))
    (trans
      (cong (λ t → t +ℕ (zero *ℕ suc m)) (*ℕ-zero-right (suc n)))
      refl)
in
trans (cong normalizePair mulEq)
(trans (normalizePair-right0 (suc n *ℕ suc m)) (sym mulEq))
where
pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
pair-ext refl refl = refl
Pairℕ-mul-toPair-normal (+suc n) (-suc m) =
let mulEq : Pairℕ-mul (toPairℤ (+suc n)) (toPairℤ (-suc m)) ≡ (mkPairℕ zero (suc n *ℕ suc m))
mulEq =
  pair-ext
    (trans
      (cong (λ t → t +ℕ (zero *ℕ suc m)) (*ℕ-zero-right (suc n)))
      refl)
    (+ℕ-zero-right (suc n *ℕ suc m))
in
trans (cong normalizePair mulEq)
(trans (normalizePair-left0 (suc n *ℕ suc m)) (sym mulEq))
where
pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
pair-ext refl refl = refl
Pairℕ-mul-toPair-normal (-suc n) (+suc m) =
let mulEq : Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ (+suc m)) ≡ (mkPairℕ zero (suc n *ℕ suc m))
mulEq =
  pair-ext
    (trans
      (cong (λ t → (zero *ℕ suc m) +ℕ t) (*ℕ-zero-right (suc n)))
      (trans
        (cong (λ t → t +ℕ zero) (*ℕ-zero-left (suc m)))
        refl))
    (+ℕ-zero-left (suc n *ℕ suc m))
in
trans (cong normalizePair mulEq)
(trans (normalizePair-left0 (suc n *ℕ suc m)) (sym mulEq))
where
pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
pair-ext refl refl = refl
Pairℕ-mul-toPair-normal (-suc n) (-suc m) =
let mulEq : Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ (-suc m)) ≡ (mkPairℕ (suc n *ℕ suc m) zero)
mulEq =
  pair-ext
    (+ℕ-zero-left (suc n *ℕ suc m))
    (trans
      (cong (λ t → (zero *ℕ suc m) +ℕ t) (*ℕ-zero-right (suc n)))

```

```

(trans
  (cong (λ t → t +ℕ zero) (*ℕ-zero-left (suc m)))
  refl))
in
trans (cong normalizePair mulEq)
  (trans (normalizePair-right0 (suc n *ℕ suc m)) (sym mulEq))
where
pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
pair-ext refl refl = refl

opaque
unfolding _*ℤ_
-- § Bridge: toPairℤ of *ℤ = Pairℕ-mul of normalized integer pairs.
toPair-*ℤ : (x y : ℤ) → toPairℤ (x *ℤ y) ≡ Pairℕ-mul (toPairℤ x) (toPairℤ y)
toPair-*ℤ x y =
  trans
    (toPair-fromPair (Pairℕ-mul (toPairℤ x) (toPairℤ y)))
    (Pairℕ-mul-toPair-normal x y)

-- § Bridge: toPairℤ of +ℤ = normalizePair of componentwise add.
toPair+ℤ : (x y : ℤ) → toPairℤ (x +ℤ y) ≡ normalizePair (Pairℕ-add (toPairℤ x) (toPairℤ y))
toPair+ℤ x y = toPair-fromPair (Pairℕ-add (toPairℤ x) (toPairℤ y))

```

Ring Laws: Distributivity, Torsion-Freedom, Negation

With this bridge in place, the distributive laws for integer multiplication are obtained by transporting the pair-level identities back to $\mathbb{Z}$.

```

opaque
unfolding _*ℤ_
-- § Right distributivity of *ℤ over +ℤ.
*ℤ-distrib-right+ℤ : (x y z : ℤ) → x *ℤ (y +ℤ z) ≡ (x *ℤ y) +ℤ (x *ℤ z)
*ℤ-distrib-right+ℤ x y z =
  let px = toPairℤ x in
  let py = toPairℤ y in
  let pz = toPairℤ z in
  let q = Pairℕ-add py pz in
  let rhs : fromPairℤ (Pairℕ-add (Pairℕ-mul px py) (Pairℕ-mul px pz)) ≡ (x *ℤ y) +ℤ (x *ℤ z)
  rhs =
    trans
      (cong (λ t → fromPairℤ (Pairℕ-add t (Pairℕ-mul px pz))) (sym (toPair-*ℤ x y)))
      (trans
        (cong (λ t → fromPairℤ (Pairℕ-add (toPairℤ (x *ℤ y)) t)) (sym (toPair-*ℤ x z)))
        refl)
    in
  trans
    (cong (λ t → fromPairℤ (Pairℕ-mul px t)) (toPair+ℤ y z))
    (trans
      (trans

```

```

(sym (fromPair-normalizePair (PairN-mul px (normalizePair q))))
(cong fromPairZ (PairN-mul-normalize-right px q)))
(trans
  (trans
    (fromPair-normalizePair (PairN-mul px q))
    (cong fromPairZ (PairN-mul-distrib-right-add px py pz)))
  rhs))

-- § Left distributivity of *ℤ over +ℤ.
*ℤ-distrib-left+ℤ : (x y z : ℤ) → (x +ℤ y) *ℤ z ≡ (x *ℤ z) +ℤ (y *ℤ z)
*ℤ-distrib-left+ℤ x y z =
  let px = toPairZ x in
  let py = toPairZ y in
  let pz = toPairZ z in
  let q = PairN-add px py in
  let rhs : fromPairZ (PairN-add (PairN-mul px pz) (PairN-mul py pz)) ≡ (x *ℤ z) +ℤ (y *ℤ z)
  rhs =
    trans
      (cong (λ t → fromPairZ (PairN-add t (PairN-mul py pz))) (sym (toPair-*ℤ x z)))
      (trans
        (cong (λ t → fromPairZ (PairN-add (toPairZ (x *ℤ z)) t)) (sym (toPair-*ℤ y z)))
        refl)
    in
  trans
    (cong (λ t → fromPairZ (PairN-mul t pz)) (toPair+ℤ x y))
    (trans
      (trans
        (sym (fromPair-normalizePair (PairN-mul (normalizePair q) pz)))
        (cong fromPairZ (PairN-mul-normalize-left q pz)))
      (trans
        (trans
          (fromPair-normalizePair (PairN-mul q pz))
          (cong fromPairZ (PairN-mul-distrib-left-add px py pz)))
        rhs))

```

Step 2: Pair Extensionality and Commutativity. Commutativity is likewise most transparent at the pair level, where extensionality reduces the proof to commutativity of multiplication on $\mathbb{N}$.

```

-- § Pair extensionality.
PairN-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairN x y) ≡ (mkPairN x' y')
PairN-ext refl refl = refl

-- § Commutativity of PairN-mul.
PairN-mul-comm : (p q : PairN) → PairN-mul p q ≡ PairN-mul q p
PairN-mul-comm p q =
  PairN-ext posEq negEq
  where
    ap = pos p
    bp = neg p
    cq = pos q
    dq = neg q

```



```

posEq : ((ap *N cq) +N (bp *N dq)) ≡ ((cq *N ap) +N (dq *N bp))
posEq =
  trans
    (cong (λ t → t +N (bp *N dq)) (*N-comm ap cq))
    (trans
      (cong (λ t → (cq *N ap) +N t) (*N-comm bp dq))
      refl)

negEq : ((ap *N dq) +N (bp *N cq)) ≡ ((cq *N bp) +N (dq *N ap))
negEq =
  trans
    (cong (λ t → t +N (bp *N cq)) (*N-comm ap dq))
    (trans
      (cong (λ t → (dq *N ap) +N t) (*N-comm bp cq))
      (+N-comm (dq *N ap) (cq *N bp)))

```

Step 3: Normalized Pair Products. Finally, the four sign combinations for normalized pairs are written out explicitly. These concrete identities prepare the torsion-freeness arguments that follow.

```

-- § Normalized pair products: (+,0) × (+,0).
PairN-mul-pos-pos : (a b : N) →
  PairN-mul (mkPairN a zero) (mkPairN b zero) ≡ (mkPairN (a *N b) zero)
PairN-mul-pos-pos a b =
  pair-ext
    (trans
      (cong (λ t → (a *N b) +N t) (*N-zero-left zero))
      (+N-zero-right (a *N b)))
    (trans
      (cong (λ t → (a *N zero) +N t) (*N-zero-left b))
      (trans
        (+N-zero-right (a *N zero))
        (*N-zero-right a)))
  where
    pair-ext : {x y x' y' : N} → x ≡ x' → y ≡ y' → (mkPairN x y) ≡ (mkPairN x' y')
    pair-ext refl refl = refl

-- § Normalized pair products: (+,0) × (0,+).
PairN-mul-pos-neg : (a b : N) →
  PairN-mul (mkPairN a zero) (mkPairN zero b) ≡ (mkPairN zero (a *N b))
PairN-mul-pos-neg a b =
  pair-ext
    (trans
      (cong (λ t → (a *N zero) +N t) (*N-zero-left b))
      (trans
        (+N-zero-right (a *N zero))
        (*N-zero-right a)))
    (trans
      (cong (λ t → (a *N b) +N t) (*N-zero-left zero))
      (+N-zero-right (a *N b)))
  where

```

```

pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
pair-ext refl refl = refl

-- § Normalized pair products: (0,+) × (+,0).
Pairℕ-mul-neg-pos : (a b : ℕ) →
Pairℕ-mul (mkPairℕ zero a) (mkPairℕ b zero) ≡ (mkPairℕ zero (a *ℕ b))
Pairℕ-mul-neg-pos a b =
pair-ext
(trans
(cong (λ t → t +ℕ (a *ℕ zero)) (*ℕ-zero-left b))
(trans
(cong (λ t → zero +ℕ t) (*ℕ-zero-right a))
refl))
(+ℕ-zero-left (a *ℕ b))
where
pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
pair-ext refl refl = refl

-- § Normalized pair products: (0,+) × (0,+).
Pairℕ-mul-neg-neg : (a b : ℕ) →
Pairℕ-mul (mkPairℕ zero a) (mkPairℕ zero b) ≡ (mkPairℕ (a *ℕ b) zero)
Pairℕ-mul-neg-neg a b =
pair-ext
(+ℕ-zero-left (a *ℕ b))
(trans
(cong (λ t → (zero *ℕ b) +ℕ t) (*ℕ-zero-right a))
(trans
(cong (λ t → t +ℕ zero) (*ℕ-zero-left b))
(+ℕ-zero-left zero)))
where
pair-ext : {x y x' y' : ℕ} → x ≡ x' → y ≡ y' → (mkPairℕ x y) ≡ (mkPairℕ x' y')
pair-ext refl refl = refl

```

Law 14P.0: Positive Nonzero Scalar Has No Torsion (Left)

The first torsion-freeness statement says that multiplication by a positive nonzero scalar cannot annihilate a nonzero integer. In the normalized pair representation, the product exposes an ordinary natural-number factor, and the zero-product argument on $\mathbb{N}$ forces the remaining factor to vanish.

```

-- § Law 14P.0: positive left factor is torsion-free.
*ℤ-pos-left-zero→zero : (n : ℕ) → (x : ℤ) → (+suc n *ℤ x ≡ 0ℤ) → x ≡ 0ℤ
*ℤ-pos-left-zero→zero n 0ℤ _ = refl
*ℤ-pos-left-zero→zero n (+suc m) mul0 =
let
eqPair : toPairℤ ((+suc n) *ℤ (+suc m)) ≡ toPairℤ 0ℤ
eqPair = cong toPairℤ mul0

step1 : Pairℕ-mul (toPairℤ (+suc n)) (toPairℤ (+suc m)) ≡ (mkPairℕ zero zero)
step1 = trans (sym (toPair-*ℤ (+suc n) (+suc m))) eqPair

```

```

pos0-raw : pos (PairN-mul (toPairℤ (+suc n)) (toPairℤ (+suc m))) ≡ zero
pos0-raw = cong pos step1

pos0 : (suc n *ℕ suc m) ≡ zero
pos0 =
  trans
    (sym (+ℕ-zero-right (suc n *ℕ suc m)))
pos0-raw

bad : suc m ≡ zero
bad = *ℕ-pos-zero→zero n (suc m) pos0
in
  ⊥-elim (caseSucZero bad)
where
  caseSucZero : {k : ℕ} → suc k ≡ zero → ⊥
  caseSucZero ()

*ℤ-pos-left-zero→zero n (-suc m) mul0 =
let
  eqPair : toPairℤ ((+suc n) *ℤ (-suc m)) ≡ toPairℤ 0ℤ
  eqPair = cong toPairℤ mul0

step1 : PairN-mul (toPairℤ (+suc n)) (toPairℤ (-suc m)) ≡ (mkPairℕ zero zero)
step1 = trans (sym (toPair-ℤ (+suc n) (-suc m))) eqPair

neg0-raw : neg (PairN-mul (toPairℤ (+suc n)) (toPairℤ (-suc m))) ≡ zero
neg0-raw = cong neg step1

neg0 : (suc n *ℕ suc m) ≡ zero
neg0 =
  trans
    (sym (+ℕ-zero-right (suc n *ℕ suc m)))
neg0-raw

bad : suc m ≡ zero
bad = *ℕ-pos-zero→zero n (suc m) neg0
in
  ⊥-elim (caseSucZero bad)
where
  caseSucZero : {k : ℕ} → suc k ≡ zero → ⊥
  caseSucZero ()

```

Laws 14P.1–14P.2: Multiplication Commutes With Negation

The interaction of multiplication with negation is obtained from distributivity together with the additive inverse laws. Conceptually the argument is the standard one: $x \cdot (y + (-y)) = x \cdot 0$ and $((-x) + x) \cdot y = 0 \cdot y$.

```

-- § Law 14P.1: *ℤ commutes with negℤ on the right.
*ℤ-neg-right : (x y : ℤ) → x *ℤ (negℤ y) ≡ negℤ (x *ℤ y)
*ℤ-neg-right x y =
let

```

```

eq0 : y +ℤ negℤ y ≡ 0ℤ
eq0 = +ℤ-inv-right y

mul0 : x *ℤ (y +ℤ negℤ y) ≡ x *ℤ 0ℤ
mul0 = cong (λ t → x *ℤ t) eq0

expand : x *ℤ (y +ℤ negℤ y) ≡ (x *ℤ y) +ℤ (x *ℤ negℤ y)
expand = *ℤ-distrib-right-+ℤ x y (negℤ y)

eqSum : (x *ℤ y) +ℤ (x *ℤ negℤ y) ≡ 0ℤ
eqSum = trans (sym expand) (trans mul0 (*ℤ-zero-right x))

addNeg : negℤ (x *ℤ y) +ℤ ((x *ℤ y) +ℤ (x *ℤ negℤ y)) ≡ negℤ (x *ℤ y) +ℤ 0ℤ
addNeg = cong (λ t → negℤ (x *ℤ y) +ℤ t) eqSum

leftReduce : negℤ (x *ℤ y) +ℤ ((x *ℤ y) +ℤ (x *ℤ negℤ y)) ≡ x *ℤ negℤ y
leftReduce =
  trans
    (sym (+ℤ-assoc (negℤ (x *ℤ y)) (x *ℤ y) (x *ℤ negℤ y)))
    (trans
      (cong (λ t → t +ℤ (x *ℤ negℤ y)) (+ℤ-inv-left (x *ℤ y)))
      (+ℤ-zero-left (x *ℤ negℤ y)))

rightReduce : negℤ (x *ℤ y) +ℤ 0ℤ ≡ negℤ (x *ℤ y)
rightReduce = +ℤ-zero-right (negℤ (x *ℤ y))
in
trans
  (sym leftReduce)
  (trans addNeg rightReduce)

-- § Law 14P.2: *ℤ commutes with negℤ on the left.
*ℤ-neg-left : (x y : ℤ) → (negℤ x) *ℤ y ≡ negℤ (x *ℤ y)
*ℤ-neg-left x y =
  let
    eq0 : negℤ x +ℤ x ≡ 0ℤ
    eq0 = +ℤ-inv-left x

    mul0 : (negℤ x +ℤ x) *ℤ y ≡ 0ℤ *ℤ y
    mul0 = cong (λ t → t *ℤ y) eq0

    expand : (negℤ x +ℤ x) *ℤ y ≡ (negℤ x *ℤ y) +ℤ (x *ℤ y)
    expand = *ℤ-distrib-left-+ℤ (negℤ x) x y

    eqSum' : (negℤ x *ℤ y) +ℤ (x *ℤ y) ≡ 0ℤ
    eqSum' = trans (sym expand) (trans mul0 (*ℤ-zero-left y))

    addInv : ((negℤ x *ℤ y) +ℤ (x *ℤ y)) +ℤ negℤ (x *ℤ y) ≡ 0ℤ +ℤ negℤ (x *ℤ y)
    addInv = cong (λ t → t +ℤ negℤ (x *ℤ y)) eqSum'

  lhsReduce : ((negℤ x *ℤ y) +ℤ (x *ℤ y)) +ℤ negℤ (x *ℤ y) ≡ negℤ x *ℤ y
  lhsReduce =
    trans
      (+ℤ-assoc (negℤ x *ℤ y) (x *ℤ y) (negℤ (x *ℤ y)))
      (trans
        (cong (λ t → (negℤ x *ℤ y) +ℤ t) (+ℤ-inv-right (x *ℤ y)))

```

```

(+ℤ-zero-right (negℤ x *ℤ y)))

rhsReduce : 0ℤ +ℤ negℤ (x *ℤ y) ≡ negℤ (x *ℤ y)
rhsReduce = +ℤ-zero-left (negℤ (x *ℤ y))
in
trans
(sym lhsReduce)
(trans addInv rhsReduce)

```

Law 14P.3: Negative Nonzero Scalar Has No Torsion (Left)

Torsion-freeness for negative left factors is now immediate from the positive case together with the compatibility of multiplication and negation.

```

-- § Law 14P.3: negative left factor is torsion-free.
*ℤ-neg-left-zero→zero : (n : ℕ) → (x : ℤ) → (-suc n *ℤ x ≡ 0ℤ) → x ≡ 0ℤ
*ℤ-neg-left-zero→zero n 0ℤ _ = refl
*ℤ-neg-left-zero→zero n (+suc m) mul0 =
let
eqPair : toPairℤ ((-suc n) *ℤ (+suc m)) ≡ toPairℤ 0ℤ
eqPair = cong toPairℤ mul0

step1 : Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ (+suc m)) ≡ (mkPairℕ zero zero)
step1 = trans (sym (toPair-*ℤ (-suc n) (+suc m))) eqPair

neg0-raw : neg (Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ (+suc m))) ≡ zero
neg0-raw = cong neg step1

neg0 : (suc n *ℕ suc m) ≡ zero
neg0 =
trans
(sym (+ℕ-zero-left (suc n *ℕ suc m)))
neg0-raw

bad : suc m ≡ zero
bad = *ℕ-pos-zero→zero n (suc m) neg0
in
⊥-elim (caseSucZero bad)
where
caseSucZero : {k : ℕ} → suc k ≡ zero → ⊥
caseSucZero ()

*ℤ-neg-left-zero→zero n (-suc m) mul0 =
let
eqPair : toPairℤ ((-suc n) *ℤ (-suc m)) ≡ toPairℤ 0ℤ
eqPair = cong toPairℤ mul0

step1 : Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ (-suc m)) ≡ (mkPairℕ zero zero)
step1 = trans (sym (toPair-*ℤ (-suc n) (-suc m))) eqPair

pos0-raw : pos (Pairℕ-mul (toPairℤ (-suc n)) (toPairℤ (-suc m))) ≡ zero
pos0-raw = cong pos step1

```

```

pos0 : (suc n * $\mathbb{N}$  suc m)  $\equiv$  zero
pos0 =
  trans
    (sym (+ $\mathbb{N}$ -zero-left (suc n * $\mathbb{N}$  suc m)))
  pos0-raw

bad : suc m  $\equiv$  zero
bad = * $\mathbb{N}$ -pos-zero $\rightarrow$ zero n (suc m) pos0
in
 $\perp$ -elim (caseSucZero bad)
where
caseSucZero : {k :  $\mathbb{N}$ }  $\rightarrow$  suc k  $\equiv$  zero  $\rightarrow$   $\perp$ 
caseSucZero ()

```

Pair-Level Associativity and $\mathbb{Z}$ Ring Closure

To complete the ring structure, it remains to prove associativity of multiplication. As before, the proof is carried out first for pair representatives and then lifted back to canonical integers.

```

-- § Zero annihilation at pair level (right).
Pair $\mathbb{N}$ -mul-zero-right : (p : Pair $\mathbb{N}$ )  $\rightarrow$  Pair $\mathbb{N}$ -mul p (mkPair $\mathbb{N}$  zero zero)  $\equiv$  (mkPair $\mathbb{N}$  zero zero)
Pair $\mathbb{N}$ -mul-zero-right (mkPair $\mathbb{N}$  a b) =
  pair-ext
    (trans
      (cong ( $\lambda$  t  $\rightarrow$  t + $\mathbb{N}$  (b * $\mathbb{N}$  zero)) (* $\mathbb{N}$ -zero-right a))
      (trans
        (cong ( $\lambda$  t  $\rightarrow$  zero + $\mathbb{N}$  t) (* $\mathbb{N}$ -zero-right b))
        refl))
    (trans
      (cong ( $\lambda$  t  $\rightarrow$  t + $\mathbb{N}$  (b * $\mathbb{N}$  zero)) (* $\mathbb{N}$ -zero-right a))
      (trans
        (cong ( $\lambda$  t  $\rightarrow$  zero + $\mathbb{N}$  t) (* $\mathbb{N}$ -zero-right b))
        refl))
  where
pair-ext : {x y x' y' :  $\mathbb{N}$ }  $\rightarrow$  x  $\equiv$  x'  $\rightarrow$  y  $\equiv$  y'  $\rightarrow$  (mkPair $\mathbb{N}$  x y)  $\equiv$  (mkPair $\mathbb{N}$  x' y')
pair-ext refl refl = refl

-- § Zero annihilation at pair level (left).
Pair $\mathbb{N}$ -mul-zero-left : (p : Pair $\mathbb{N}$ )  $\rightarrow$  Pair $\mathbb{N}$ -mul (mkPair $\mathbb{N}$  zero zero) p  $\equiv$  (mkPair $\mathbb{N}$  zero zero)
Pair $\mathbb{N}$ -mul-zero-left (mkPair $\mathbb{N}$  a b) =
  pair-ext
    (trans
      (cong ( $\lambda$  t  $\rightarrow$  t + $\mathbb{N}$  (zero * $\mathbb{N}$  b)) (* $\mathbb{N}$ -zero-left a))
      (trans
        (cong ( $\lambda$  t  $\rightarrow$  zero + $\mathbb{N}$  t) (* $\mathbb{N}$ -zero-left b))
        refl))
    (trans
      (cong ( $\lambda$  t  $\rightarrow$  t + $\mathbb{N}$  (zero * $\mathbb{N}$  a)) (* $\mathbb{N}$ -zero-left b))
      (trans
        (cong ( $\lambda$  t  $\rightarrow$  zero + $\mathbb{N}$  t) (* $\mathbb{N}$ -zero-left a))

```

```

    refl))
where
pair-ext : {x y x' y' :  $\mathbb{N}$ }  $\rightarrow x \equiv x' \rightarrow y \equiv y' \rightarrow (\text{mkPairN } x \ y) \equiv (\text{mkPairN } x' \ y')$ 
pair-ext refl refl = refl

```

Step 2: Associativity Base Cases (Zero Factor). The zero cases can be separated out and handled directly using the pair-level annihilation lemmas.

```

-- § Associativity at pair level for canonical pairs (16-case proof).
PairN-mul-toPair-assoc : (x y z :  $\mathbb{Z}$ )  $\rightarrow$ 
PairN-mul (PairN-mul (toPairZ x) (toPairZ y)) (toPairZ z)
   $\equiv$ 
PairN-mul (toPairZ x) (PairN-mul (toPairZ y) (toPairZ z))
PairN-mul-toPair-assoc 0Z y z = refl
PairN-mul-toPair-assoc (+suc n) 0Z z =
trans
  (cong ( $\lambda t \rightarrow$  PairN-mul t (toPairZ z)) (PairN-mul-zero-right (mkPairN (suc n) zero)))
  (trans
    (PairN-mul-zero-left (toPairZ z))
    (sym
      (trans
        (cong ( $\lambda t \rightarrow$  PairN-mul (mkPairN (suc n) zero) t) (PairN-mul-zero-left (toPairZ z)))
        (PairN-mul-zero-right (mkPairN (suc n) zero))))))
PairN-mul-toPair-assoc (-suc n) 0Z z =
trans
  (cong ( $\lambda t \rightarrow$  PairN-mul t (toPairZ z)) (PairN-mul-zero-right (mkPairN zero (suc n))))
  (trans
    (PairN-mul-zero-left (toPairZ z))
    (sym
      (trans
        (cong ( $\lambda t \rightarrow$  PairN-mul (mkPairN zero (suc n)) t) (PairN-mul-zero-left (toPairZ z)))
        (PairN-mul-zero-right (mkPairN zero (suc n))))))
PairN-mul-toPair-assoc (+suc n) (+suc m) 0Z =
trans
  (PairN-mul-zero-right (PairN-mul (mkPairN (suc n) zero) (mkPairN (suc m) zero)))
  (sym
    (trans
      (cong ( $\lambda t \rightarrow$  PairN-mul (mkPairN (suc n) zero) t) (PairN-mul-zero-right (mkPairN (suc m) zero)))
      (PairN-mul-zero-right (mkPairN (suc n) zero))))
PairN-mul-toPair-assoc (+suc n) (-suc m) 0Z =
trans
  (PairN-mul-zero-right (PairN-mul (mkPairN (suc n) zero) (mkPairN zero (suc m))))
  (sym
    (trans
      (cong ( $\lambda t \rightarrow$  PairN-mul (mkPairN (suc n) zero) t) (PairN-mul-zero-right (mkPairN zero (suc m))))
      (PairN-mul-zero-right (mkPairN (suc n) zero))))
PairN-mul-toPair-assoc (-suc n) (+suc m) 0Z =
trans
  (PairN-mul-zero-right (PairN-mul (mkPairN zero (suc n)) (mkPairN (suc m) zero)))
  (sym
    (trans
      (cong ( $\lambda t \rightarrow$  PairN-mul (mkPairN zero (suc n)) t) (PairN-mul-zero-right (mkPairN (suc m) zero)))

```

```

(PairN-mul-zero-right (mkPairN zero (suc n))))
PairN-mul-toPair-assoc (-suc n) (-suc m) 0ℤ =
trans
(PairN-mul-zero-right (PairN-mul (mkPairN zero (suc n)) (mkPairN zero (suc m))))
(sym
(trans
(cong (λ t → PairN-mul (mkPairN zero (suc n)) t) (PairN-mul-zero-right (mkPairN zero (suc m))))
(PairN-mul-zero-right (mkPairN zero (suc n))))))

```

Step 3: Associativity Nonzero Cases. The remaining cases reduce to the four sign combinations for canonical pairs and, ultimately, to associativity of multiplication on $\mathbb{N}$.

```

PairN-mul-toPair-assoc (+suc n) (+suc m) (+suc k) =
trans
(cong (λ t → PairN-mul t (mkPairN (suc k) zero)) (PairN-mul-pos-pos (suc n) (suc m)))
(trans
(PairN-mul-pos-pos ((suc n) *N (suc m)) (suc k))
(trans
(PairN-ext (*N-assoc (suc n) (suc m) (suc k)) refl)
(sym
(trans
(cong (λ t → PairN-mul (mkPairN (suc n) zero) t) (PairN-mul-pos-pos (suc m) (suc k)))
(PairN-mul-pos-pos (suc n) ((suc m) *N (suc k)))))))
PairN-mul-toPair-assoc (+suc n) (+suc m) (-suc k) =
trans
(cong (λ t → PairN-mul t (mkPairN zero (suc k))) (PairN-mul-pos-pos (suc n) (suc m)))
(trans
(PairN-mul-pos-neg ((suc n) *N (suc m)) (suc k))
(trans
(PairN-ext refl (*N-assoc (suc n) (suc m) (suc k)))
(sym
(trans
(cong (λ t → PairN-mul (mkPairN (suc n) zero) t) (PairN-mul-pos-neg (suc m) (suc k)))
(PairN-mul-pos-neg (suc n) ((suc m) *N (suc k)))))))
PairN-mul-toPair-assoc (+suc n) (-suc m) (+suc k) =
trans
(cong (λ t → PairN-mul t (mkPairN (suc k) zero)) (PairN-mul-pos-neg (suc n) (suc m)))
(trans
(PairN-mul-neg-pos ((suc n) *N (suc m)) (suc k))
(trans
(PairN-ext refl (*N-assoc (suc n) (suc m) (suc k)))
(sym
(trans
(cong (λ t → PairN-mul (mkPairN (suc n) zero) t) (PairN-mul-neg-pos (suc m) (suc k)))
(PairN-mul-neg-pos (suc n) ((suc m) *N (suc k)))))))
PairN-mul-toPair-assoc (+suc n) (-suc m) (-suc k) =
trans
(cong (λ t → PairN-mul t (mkPairN zero (suc k))) (PairN-mul-pos-neg (suc n) (suc m)))
(trans
(PairN-mul-neg-neg ((suc n) *N (suc m)) (suc k))

```



```

(trans
  (PairN-ext (*N-assoc (suc n) (suc m) (suc k)) refl)
  (sym
    (trans
      (cong (λ t → PairN-mul (mkPairN (suc n) zero) t) (PairN-mul-neg-neg (suc m) (suc k)))
      (PairN-mul-pos-pos (suc n) ((suc m) *N (suc k))))))
PairN-mul-toPair-assoc (-suc n) (+suc m) (+suc k) =
trans
(cong (λ t → PairN-mul t (mkPairN (suc k) zero)) (PairN-mul-neg-pos (suc n) (suc m)))
(trans
  (PairN-mul-neg-pos ((suc n) *N (suc m)) (suc k))
  (trans
    (PairN-ext refl (*N-assoc (suc n) (suc m) (suc k)))
    (sym
      (trans
        (cong (λ t → PairN-mul (mkPairN zero (suc n)) t) (PairN-mul-pos-pos (suc m) (suc k)))
        (PairN-mul-neg-pos (suc n) ((suc m) *N (suc k))))))
PairN-mul-toPair-assoc (-suc n) (+suc m) (-suc k) =
trans
(cong (λ t → PairN-mul t (mkPairN zero (suc k))) (PairN-mul-neg-pos (suc n) (suc m)))
(trans
  (PairN-mul-neg-neg ((suc n) *N (suc m)) (suc k))
  (trans
    (PairN-ext (*N-assoc (suc n) (suc m) (suc k)) refl)
    (sym
      (trans
        (cong (λ t → PairN-mul (mkPairN zero (suc n)) t) (PairN-mul-pos-neg (suc m) (suc k)))
        (PairN-mul-neg-neg (suc n) ((suc m) *N (suc k))))))
PairN-mul-toPair-assoc (-suc n) (-suc m) (+suc k) =
trans
(cong (λ t → PairN-mul t (mkPairN (suc k) zero)) (PairN-mul-neg-neg (suc n) (suc m)))
(trans
  (PairN-mul-pos-pos ((suc n) *N (suc m)) (suc k))
  (trans
    (PairN-ext (*N-assoc (suc n) (suc m) (suc k)) refl)
    (sym
      (trans
        (cong (λ t → PairN-mul (mkPairN zero (suc n)) t) (PairN-mul-neg-pos (suc m) (suc k)))
        (PairN-mul-neg-neg (suc n) ((suc m) *N (suc k))))))
PairN-mul-toPair-assoc (-suc n) (-suc m) (-suc k) =
trans
(cong (λ t → PairN-mul t (mkPairN zero (suc k))) (PairN-mul-neg-neg (suc n) (suc m)))
(trans
  (PairN-mul-pos-neg ((suc n) *N (suc m)) (suc k))
  (trans
    (PairN-ext refl (*N-assoc (suc n) (suc m) (suc k)))
    (sym
      (trans
        (cong (λ t → PairN-mul (mkPairN zero (suc n)) t) (PairN-mul-neg-neg (suc m) (suc k)))
        (PairN-mul-neg-pos (suc n) ((suc m) *N (suc k))))))

```

Step 4: Lifting to $\mathbb{Z}$. Once associativity has been established at the pair level, the corresponding laws on $\mathbb{Z}$ follow by transport along `toPair $\mathbb{Z}$` and `fromPair $\mathbb{Z}$` .

```
opaque
unfolding _* $\mathbb{Z}$ _
-- § Associativity of * $\mathbb{Z}$ .
* $\mathbb{Z}$ -assoc : (x y z :  $\mathbb{Z}$ ) → (x * $\mathbb{Z}$  y) * $\mathbb{Z}$  z ≡ x * $\mathbb{Z}$  (y * $\mathbb{Z}$  z)
* $\mathbb{Z}$ -assoc x y z =
  let lhs = (x * $\mathbb{Z}$  y) * $\mathbb{Z}$  z in
  let rhs = x * $\mathbb{Z}$  (y * $\mathbb{Z}$  z) in
  trans
    (sym (from-toPair $\mathbb{Z}$  lhs))
    (trans
      (cong fromPair $\mathbb{Z}$ 
        (trans
          (trans
            (toPair-* $\mathbb{Z}$  (x * $\mathbb{Z}$  y) z)
            (cong (λ t → Pair $\mathbb{N}$ -mul t (toPair $\mathbb{Z}$  z)) (toPair-* $\mathbb{Z}$  x y)))
          (trans
            (Pair $\mathbb{N}$ -mul-toPair-assoc x y z)
            (sym
              (trans
                (toPair-* $\mathbb{Z}$  x (y * $\mathbb{Z}$  z))
                (cong (λ t → Pair $\mathbb{N}$ -mul (toPair $\mathbb{Z}$  x) t) (toPair-* $\mathbb{Z}$  y z)))))))
        (from-toPair $\mathbb{Z}$  rhs))

-- § Commutativity of * $\mathbb{Z}$ .
* $\mathbb{Z}$ -comm : (x y :  $\mathbb{Z}$ ) → x * $\mathbb{Z}$  y ≡ y * $\mathbb{Z}$  x
* $\mathbb{Z}$ -comm x y = cong fromPair $\mathbb{Z}$  (Pair $\mathbb{N}$ -mul-comm (toPair $\mathbb{Z}$  x) (toPair $\mathbb{Z}$  y))
```

Chapter 35

Integer Order Laws

The next chapter develops the basic laws of integer order. The general method is unchanged: each statement is reduced to a direct analysis of the signed normal forms.

Transport and Negation

We begin with simple transport lemmas for $\leq_{\mathbb{Z}}$ and $<_{\mathbb{Z}}$, followed by the observation that negation reverses order.

```
-- § Transport  $\leq_{\mathbb{Z}}$  along left equality.
 $\leq_{\mathbb{Z}}\text{-resp-}\equiv^l : \{x\ y\ z : \mathbb{Z}\} \rightarrow x \equiv y \rightarrow x \leq_{\mathbb{Z}} z \rightarrow y \leq_{\mathbb{Z}} z$ 
 $\leq_{\mathbb{Z}}\text{-resp-}\equiv^l \text{ refl } p = p$ 

-- § Transport  $\leq_{\mathbb{Z}}$  along right equality.
 $\leq_{\mathbb{Z}}\text{-resp-}\equiv^r : \{x\ y\ z : \mathbb{Z}\} \rightarrow y \equiv z \rightarrow x \leq_{\mathbb{Z}} y \rightarrow x \leq_{\mathbb{Z}} z$ 
 $\leq_{\mathbb{Z}}\text{-resp-}\equiv^r \text{ refl } p = p$ 

-- § Transport  $<_{\mathbb{Z}}$  along left equality.
 $<_{\mathbb{Z}}\text{-resp-}\equiv^l : \{x\ y\ z : \mathbb{Z}\} \rightarrow x \equiv y \rightarrow x <_{\mathbb{Z}} z \rightarrow y <_{\mathbb{Z}} z$ 
 $<_{\mathbb{Z}}\text{-resp-}\equiv^l \text{ refl } p = p$ 

-- § Transport  $<_{\mathbb{Z}}$  along right equality.
 $<_{\mathbb{Z}}\text{-resp-}\equiv^r : \{x\ y\ z : \mathbb{Z}\} \rightarrow y \equiv z \rightarrow x <_{\mathbb{Z}} y \rightarrow x <_{\mathbb{Z}} z$ 
 $<_{\mathbb{Z}}\text{-resp-}\equiv^r \text{ refl } p = p$ 

-- § Negation reverses order (antitone).
 $\text{neg}\mathbb{Z}\text{-antitone-}\leq_{\mathbb{Z}} : \{x\ y : \mathbb{Z}\} \rightarrow x \leq_{\mathbb{Z}} y \rightarrow (\text{neg}\mathbb{Z}\ y) \leq_{\mathbb{Z}} (\text{neg}\mathbb{Z}\ x)$ 
 $\text{neg}\mathbb{Z}\text{-antitone-}\leq_{\mathbb{Z}} \{0\mathbb{Z}\} \{0\mathbb{Z}\} \_ = \text{tt}$ 
 $\text{neg}\mathbb{Z}\text{-antitone-}\leq_{\mathbb{Z}} \{0\mathbb{Z}\} \{+\text{suc } n\} \_ = \text{tt}$ 
 $\text{neg}\mathbb{Z}\text{-antitone-}\leq_{\mathbb{Z}} \{0\mathbb{Z}\} \{-\text{suc } n\} ()$ 
 $\text{neg}\mathbb{Z}\text{-antitone-}\leq_{\mathbb{Z}} \{+\text{suc } m\} \{0\mathbb{Z}\} ()$ 
 $\text{neg}\mathbb{Z}\text{-antitone-}\leq_{\mathbb{Z}} \{+\text{suc } m\} \{+\text{suc } n\} p = p$ 
 $\text{neg}\mathbb{Z}\text{-antitone-}\leq_{\mathbb{Z}} \{+\text{suc } m\} \{-\text{suc } n\} ()$ 
 $\text{neg}\mathbb{Z}\text{-antitone-}\leq_{\mathbb{Z}} \{-\text{suc } m\} \{0\mathbb{Z}\} \_ = \text{tt}$ 
 $\text{neg}\mathbb{Z}\text{-antitone-}\leq_{\mathbb{Z}} \{-\text{suc } m\} \{+\text{suc } n\} \_ = \text{tt}$ 
 $\text{neg}\mathbb{Z}\text{-antitone-}\leq_{\mathbb{Z}} \{-\text{suc } m\} \{-\text{suc } n\} p = p$ 
```

Positivity Elimination

The next lemma isolates the positive part of the order structure: if $0 <_{\mathbb{Z}} z$, then z must be of positive constructor form. This will be used repeatedly in multiplicative order arguments.

```
-- § From  $0 < z$ , force  $z$  into positive constructor form.
0<ℤ→pos : (z : ℤ) → 0ℤ <ℤ z → ∑ ℕ (λ n → z ≡ +suc n)
0<ℤ→pos 0ℤ (p≤ , p≠) = ⊥-elim (p≠ p≤)
0<ℤ→pos (+suc n) _ = n , refl
0<ℤ→pos (-suc n) () , _

-- §  $0 < +suc n$  is immediate.
0<ℤ-pos : (n : ℕ) → 0ℤ <ℤ (+suc n)
0<ℤ-pos n = tt , (λ p → p)
```

Multiplicative Monotonicity and Cancellation

Step 1: Product of positive constructors, positive multiplication, and negative-positive product. Before proving monotonicity, the concrete behaviour of products with positive factors is recorded. In particular, positive times positive remains positive, and negative times positive is explicitly negative.

```
opaque
unfolding _*ℤ_
-- § Product of positive constructors stays positive.
*ℤ-pos-pos-eq : (m n : ℕ) → (+suc m) *ℤ (+suc n) ≡ +suc (n +ℕ (m *ℕ suc n))
*ℤ-pos-pos-eq m n =
let posStep : (suc m *ℕ suc n) +ℕ (zero *ℕ zero) ≡ suc (n +ℕ (m *ℕ suc n))
posStep =
trans
(cong (λ t → (suc m *ℕ suc n) +ℕ t) (*ℕ-zero-left zero))
(trans
(+ℕ-zero-right (suc m *ℕ suc n))
refl)

negStep : (suc m *ℕ zero) +ℕ (zero *ℕ suc n) ≡ zero
negStep =
trans
(cong (λ t → t +ℕ (zero *ℕ suc n)) (*ℕ-zero-right (suc m)))
(trans
(cong (λ t → zero +ℕ t) (*ℕ-zero-left (suc n)))
refl)

in
trans
(normalizeℤ-cong posStep negStep)
refl

-- §  $0 < z \cdot d$  for  $z$  positive and  $d$  a positive natural.
```

```

0<ℤ-mul-pos-right : (z : ℤ) → (d : ℕ+) → 0ℤ <ℤ z → 0ℤ <ℤ (z *ℤ +toℤ d)
0<ℤ-mul-pos-right z (mkℕ+ k) zpos =
  let zShape = 0<ℤ→pos z zpos
      m = fst zShape
      z≡ = snd zShape

  prod≡ : z *ℤ (+suc k) ≡ (+suc m) *ℤ (+suc k)
  prod≡ = cong (λ t → t *ℤ (+suc k)) z≡

  basePos : 0ℤ <ℤ ((+suc m) *ℤ (+suc k))
  basePos =
    <ℤ-resp≡r (sym (*ℤ-pos-pos-eq m k)) (0<ℤ-pos (k +ℕ (m *ℕ suc k)))

in
<ℤ-resp≡r (sym prod≡) basePos

-- § Product of negative and positive constructors.
*ℤ-neg-pos-eq : (m k : ℕ) → (-suc m) *ℤ (+suc k) ≡ -suc (k +ℕ (m *ℕ suc k))
*ℤ-neg-pos-eq m k =
  trans
    (*ℤ-neg-left (+suc m) (+suc k))
  (trans
    (cong negℤ (*ℤ-pos-pos-eq m k))
  refl)

```

Step 2: Multiplication by positive $\mathbb{N}^+$ preserves $\leq_{\mathbb{Z}}$. These shape lemmas are then used to show that multiplication by a positive element of $\mathbb{N}^+$ preserves order on the right.

```

-- § Multiplication by positive ℕ+ preserves ≤ℤ (9-case proof).
≤ℤ-mul-pos-right : (x y : ℤ) → (d : ℕ+) → x ≤ℤ y → (x *ℤ +toℤ d) ≤ℤ (y *ℤ +toℤ d)
≤ℤ-mul-pos-right 0ℤ 0ℤ (mkℕ+ k) _ =
  subst
    (λ t → t ≤ℤ t)
  (sym (*ℤ-zero-left (+suc k)))
  tt
≤ℤ-mul-pos-right 0ℤ (+suc n) (mkℕ+ k) _ =
  let
    t = k +ℕ (n *ℕ suc k)
    eqL : 0ℤ ≡ 0ℤ *ℤ (+suc k)
    eqL = sym (*ℤ-zero-left (+suc k))

    eqR : (+suc t) ≡ ((+suc n) *ℤ (+suc k))
    eqR = sym (*ℤ-pos-pos-eq n k)

  base : 0ℤ ≤ℤ (+suc t)
  base = tt
in
  subst (λ r → (0ℤ *ℤ (+suc k)) ≤ℤ r) eqR
  (subst (λ l → l ≤ℤ (+suc t)) eqL base)
  ≤ℤ-mul-pos-right 0ℤ (-suc n) d ()
  ≤ℤ-mul-pos-right (+suc m) 0ℤ d ()

```

```

≤ℤ-mul-pos-right (+suc m) (+suc n) (mkℕ+ k) (s≤s p) =
let
  t1 = k +ℕ (m *ℕ suc k)
  t2 = k +ℕ (n *ℕ suc k)

  mulMono : (m *ℕ suc k) ≤ (n *ℕ suc k)
  mulMono = ≤-*ℕ-monor p (suc k)

  addMono : t1 ≤ t2
  addMono = ≤-+ℕ-monol mulMono k

  base : (+suc t1) ≤ℤ (+suc t2)
  base = s≤s addMono
in
≤ℤ-resp-≡l (sym (*ℤ-pos-pos-eq m k))
  (≤ℤ-resp-≡r (sym (*ℤ-pos-pos-eq n k)) base)
≤ℤ-mul-pos-right (+suc m) (-suc n) d ()
≤ℤ-mul-pos-right (-suc m) 0ℤ (mkℕ+ k) _ =
let
  t1 = k +ℕ (m *ℕ suc k)
  eqL : (-suc t1) ≡ ((-suc m) *ℤ (+suc k))
  eqL = sym (*ℤ-neg-pos-eq m k)

  eqR : 0ℤ ≡ (0ℤ *ℤ (+suc k))
  eqR = sym (*ℤ-zero-left (+suc k))

  base : (-suc t1) ≤ℤ 0ℤ
  base = tt
in
subst (λ r → ((-suc m) *ℤ (+suc k)) ≤ℤ r) eqR
  (subst (λ l → l ≤ℤ 0ℤ) eqL base)
≤ℤ-mul-pos-right (-suc m) (+suc n) (mkℕ+ k) _ =
let
  t1 = k +ℕ (m *ℕ suc k)
  t2 = k +ℕ (n *ℕ suc k)
  eqL : (-suc t1) ≡ ((-suc m) *ℤ (+suc k))
  eqL = sym (*ℤ-neg-pos-eq m k)

  eqR : (+suc t2) ≡ ((+suc n) *ℤ (+suc k))
  eqR = sym (*ℤ-pos-pos-eq n k)

  base : (-suc t1) ≤ℤ (+suc t2)
  base = tt
in
subst (λ r → ((-suc m) *ℤ (+suc k)) ≤ℤ r) eqR
  (subst (λ l → l ≤ℤ (+suc t2)) eqL base)
≤ℤ-mul-pos-right (-suc m) (-suc n) (mkℕ+ k) (s≤s p) =
let
  t1 = k +ℕ (m *ℕ suc k)
  t2 = k +ℕ (n *ℕ suc k)

  mulMono : (n *ℕ suc k) ≤ (m *ℕ suc k)
  mulMono = ≤-*ℕ-monor p (suc k)

```

```

addMono : t2 ≤ t1
addMono = ≤-+N-monol mulMono k

base : (-suc t1) ≤ℤ (-suc t2)
base = s≤s addMono
in
≤ℤ-resp-≡l (sym (*ℤ-neg-pos-eq m k))
(≤ℤ-resp-≡r (sym (*ℤ-neg-pos-eq n k)) base)

```

Step 3: Cancellation of positive right factor. Once monotonicity for positive right factors is known, the corresponding cancellation law can be proved by reversing the same analysis. The argument again proceeds by a case distinction on the signs of the two compared integers.

```

-- § Cancellation: if x·d ≤ y·d for positive d, then x ≤ y (9-case proof).
≤ℤ-mul-pos-cancel-right : (x y : ℤ) → (d : ℕ+) → (x *ℤ +toℤ d) ≤ℤ (y *ℤ +toℤ d) → x ≤ℤ y
≤ℤ-mul-pos-cancel-right 0ℤ 0ℤ (mkℕ+ k) p = tt
≤ℤ-mul-pos-cancel-right 0ℤ (+suc n) (mkℕ+ k) p = tt
≤ℤ-mul-pos-cancel-right 0ℤ (-suc n) (mkℕ+ k) p =
let
  t : ℕ
  t = k +ℕ (n *ℕ suc k)

  rhsEq : ((-suc n) *ℤ (+suc k)) ≡ (-suc t)
  rhsEq = *ℤ-neg-pos-eq n k

  p0 : (0ℤ *ℤ (+suc k)) ≤ℤ ((-suc n) *ℤ (+suc k))
  p0 = p

  p1 : 0ℤ ≤ℤ ((-suc n) *ℤ (+suc k))
  p1 = subst (λ s → s ≤ℤ ((-suc n) *ℤ (+suc k))) (*ℤ-zero-left (+suc k)) p0

  p' : 0ℤ ≤ℤ (-suc t)
  p' = subst (λ r → 0ℤ ≤ℤ r) rhsEq p1
in
⊥-elim p'
≤ℤ-mul-pos-cancel-right (+suc m) 0ℤ (mkℕ+ k) p =
let
  t = k +ℕ (m *ℕ suc k)
  lhsPos : ((+suc m) *ℤ (+suc k)) ≡ +suc t
  lhsPos = *ℤ-pos-pos-eq m k

  p0 : ((+suc m) *ℤ (+suc k)) ≤ℤ (0ℤ *ℤ (+suc k))
  p0 = p

  p1 : ((+suc m) *ℤ (+suc k)) ≤ℤ 0ℤ
  p1 = subst (λ r → ((+suc m) *ℤ (+suc k)) ≤ℤ r) (*ℤ-zero-left (+suc k)) p0

  p' : (+suc t) ≤ℤ 0ℤ
  p' = subst (λ s → s ≤ℤ 0ℤ) lhsPos p1
in
⊥-elim p'

```

```

≤ℤ-mul-pos-cancel-right (+suc m) (+suc n) (mkℕ+ k) p =
let
  t1 = k +ℕ (m *ℕ suc k)
  t2 = k +ℕ (n *ℕ suc k)

  lhsEq : (+suc t1) ≡ ((+suc m) *ℤ (+suc k))
  lhsEq = sym (*ℤ-pos-pos-eq m k)

  rhsEq : (+suc t2) ≡ ((+suc n) *ℤ (+suc k))
  rhsEq = sym (*ℤ-pos-pos-eq n k)

  step : (+suc t1) ≤ℤ (+suc t2)
  step =
    ≤ℤ-resp-≡l (sym lhsEq)
    (≤ℤ-resp-≡r (sym rhsEq) p)

  natStep : suc t1 ≤ suc t2
  natStep = step

  t1 ≤ t2 : t1 ≤ t2
  t1 ≤ t2 = ≤-+ℕ-cancell (suc zero) t1 t2 natStep

  mulPart : (m *ℕ suc k) ≤ (n *ℕ suc k)
  mulPart = ≤-+ℕ-cancell k (m *ℕ suc k) (n *ℕ suc k) t1 ≤ t2

  base : m ≤ n
  base = ≤-*ℕ-cancelr-suc k mulPart
in
s≤s base
≤ℤ-mul-pos-cancel-right (+suc m) (-suc n) (mkℕ+ k) p =
let
  t1 : ℕ
  t1 = k +ℕ (m *ℕ suc k)

  t2 : ℕ
  t2 = k +ℕ (n *ℕ suc k)

  lhsPos : ((+suc m) *ℤ (+suc k)) ≡ (+suc t1)
  lhsPos = *ℤ-pos-pos-eq m k

  rhsNeg : ((-suc n) *ℤ (+suc k)) ≡ (-suc t2)
  rhsNeg = *ℤ-neg-pos-eq n k

  p1 : ((+suc m) *ℤ (+suc k)) ≤ℤ (-suc t2)
  p1 = ≤ℤ-resp-≡r rhsNeg p

  p2 : (+suc t1) ≤ℤ (-suc t2)
  p2 = subst (λ s → s ≤ℤ (-suc t2)) lhsPos p1
in
⊥-elim p2
≤ℤ-mul-pos-cancel-right (-suc m) 0ℤ (mkℕ+ k) p = tt
≤ℤ-mul-pos-cancel-right (-suc m) (+suc n) (mkℕ+ k) p = tt
≤ℤ-mul-pos-cancel-right (-suc m) (-suc n) (mkℕ+ k) p =
let
  t1 = k +ℕ (m *ℕ suc k)

```



```

t2 = k +N (n *N suc k)

lhsEq : (-suc t1) ≡ ((-suc m) *Z (+suc k))
lhsEq = sym (*Z-neg-pos-eq m k)

rhsEq : (-suc t2) ≡ ((-suc n) *Z (+suc k))
rhsEq = sym (*Z-neg-pos-eq n k)

step : (-suc t1) ≤Z (-suc t2)
step =
  ≤Z-resp-≡l (sym lhsEq)
  (≤Z-resp-≡r (sym rhsEq) p)

natStep : suc t2 ≤ suc t1
natStep = step

t2 ≤ t1 : t2 ≤ t1
t2 ≤ t1 = ≤-+N-cancell (suc zero) t2 t1 natStep

mulPart : (n *N suc k) ≤ (m *N suc k)
mulPart = ≤-+N-cancell k (n *N suc k) (m *N suc k) t2 ≤ t1

base : n ≤ m
base = ≤-*N-cancelr-suc k mulPart
in
s ≤ s base

```

Step 2: Nonneg and Strict Factor Monotonicity. From the positive-factor case one immediately obtains monotonicity for arbitrary nonnegative right factors, and then strict monotonicity for positive factors.

```

-- § Nonneg right factor preserves ≤Z.
≤Z-mul-nonneg-right : (x y z : Z) → x ≤Z y → 0Z ≤Z z → (x *Z z) ≤Z (y *Z z)
≤Z-mul-nonneg-right x y 0Z x ≤ y _ =
  subst (λ t → t ≤Z (y *Z 0Z)) (sym (*Z-zero-right x))
  (subst (λ t → 0Z ≤Z t) (sym (*Z-zero-right y)) tt)
≤Z-mul-nonneg-right x y (+suc k) x ≤ y _ =
  let
    d : N+
    d = mkN+ k

  step : (x *Z +toZ d) ≤Z (y *Z +toZ d)
  step = ≤Z-mul-pos-right x y d x ≤ y

  lhs : (x *Z (+suc k)) ≡ (x *Z +toZ d)
  lhs = refl

  rhs : (y *Z (+suc k)) ≡ (y *Z +toZ d)
  rhs = refl
in
  ≤Z-resp-≡l (sym lhs) (≤Z-resp-≡r (sym rhs) step)
  ≤Z-mul-nonneg-right x y (-suc k) _ ()

```

```

-- § Integer squares are always nonneg:  $0 \leq a * a$ .
squareℤ-nonneg : (a : ℤ) → 0ℤ ≤ℤ a *ℤ a
squareℤ-nonneg 0ℤ =
  subst (0ℤ ≤ℤ _) (sym (*ℤ-zero-left 0ℤ)) tt
squareℤ-nonneg (+suc n) =
  subst (≤ℤ ((+suc n) *ℤ (+suc n))) (*ℤ-zero-left (+suc n))
    (≤ℤ-mul-nonneg-right 0ℤ (+suc n) (+suc n) tt tt)
squareℤ-nonneg (-suc n) =
  let
    chain : (-suc n) *ℤ (-suc n) ≡ (+suc n) *ℤ (+suc n)
    chain = trans
      (*ℤ-neg-left (+suc n) (-suc n))
      (trans
        (cong negℤ (*ℤ-neg-right (+suc n) (+suc n)))
        (negℤ-involutive ((+suc n) *ℤ (+suc n))))
  in
  subst (0ℤ ≤ℤ _) (sym chain) (squareℤ-nonneg (+suc n))

-- § Strict order preserved by positive right factor.
<ℤ-mul-pos-right : {x y : ℤ} → (d : ℕ+) → x <ℤ y → (x *ℤ +toℤ d) <ℤ (y *ℤ +toℤ d)
<ℤ-mul-pos-right {x} {y} d (x ≤ y, y < x) =
  let
    lePart : (x *ℤ +toℤ d) ≤ℤ (y *ℤ +toℤ d)
    lePart = ≤ℤ-mul-pos-right x y d x ≤ y

    notRev : (y *ℤ +toℤ d) <ℤ (x *ℤ +toℤ d)
    notRev ydx ≤ xdx = y < x (≤ℤ-mul-pos-cancel-right y x d ydx ≤ xdx)
  in
  lePart , notRev

```

Chapter 36

Absolute Value Laws

The next chapter develops the algebraic properties of absolute value. The aim is to connect the order-theoretic notion of magnitude with the additive and multiplicative structure already established on $\mathbb{Z}$.

Basic Properties

We begin with the elementary identities: behaviour at zero, invariance under negation, idempotence, nonnegativity, and the fact that every integer is bounded by its absolute value.

```
-- § abs of zero.
absℤ-zero : absℤ 0ℤ ≡ 0ℤ
absℤ-zero = refl

-- § abs absorbs negation.
absℤ-neg : (z : ℤ) → absℤ (negℤ z) ≡ absℤ z
absℤ-neg 0ℤ = refl
absℤ-neg (+suc n) = refl
absℤ-neg (-suc n) = refl

-- § abs is idempotent.
absℤ-idem : (z : ℤ) → absℤ (absℤ z) ≡ absℤ z
absℤ-idem 0ℤ = refl
absℤ-idem (+suc n) = refl
absℤ-idem (-suc n) = refl

-- § abs is nonneg.
absℤ-nonneg : (z : ℤ) → 0ℤ ≤ℤ absℤ z
absℤ-nonneg 0ℤ = tt
absℤ-nonneg (+suc n) = tt
absℤ-nonneg (-suc n) = tt

-- § Every integer is bounded by its absolute value.
```

```

≤ℤ-absℤ : (z : ℤ) → z ≤ℤ absℤ z
≤ℤ-absℤ 0ℤ = tt
≤ℤ-absℤ (+suc n) = ≤-refl (suc n)
≤ℤ-absℤ (-suc n) = tt

-- § abs zero implies zero.
absℤ-zero→zero : (z : ℤ) → absℤ z ≡ 0ℤ → z ≡ 0ℤ
absℤ-zero→zero 0ℤ _ = refl
absℤ-zero→zero (+suc n) ()
absℤ-zero→zero (-suc n) ()

```

Magnitude and Subadditivity

To prepare the triangle inequality, absolute value is related to the natural-number magnitude obtained by forgetting the sign. This makes it possible to prove subadditivity by reducing the problem to an estimate on `normalizeℤ`.

```

-- § Natural magnitude.
magℤ : ℤ → ℕ
magℤ 0ℤ = zero
magℤ (+suc n) = suc n
magℤ (-suc n) = suc n

-- § Embedding ℕ into ℤ.
fromℕℤ : ℕ → ℤ
fromℕℤ zero = 0ℤ
fromℕℤ (suc n) = +suc n

-- § abs equals fromℕℤ of magnitude.
absℤ-fromℕℤ-magℤ : (z : ℤ) → absℤ z ≡ fromℕℤ (magℤ z)
absℤ-fromℕℤ-magℤ 0ℤ = refl
absℤ-fromℕℤ-magℤ (+suc n) = refl
absℤ-fromℕℤ-magℤ (-suc n) = refl

-- § Transport ≤ along right equality.
≤-resp≡r : {a b c : ℕ} → a ≤ b → b ≡ c → a ≤ c
≤-resp≡r {a} p eq = subst (λ t → a ≤ t) eq p

-- § Successor weakening.
≤-weaken-sucr : {a b : ℕ} → a ≤ b → a ≤ suc b
≤-weaken-sucr {a} {b} p = ≤-trans p (≤-step b)

-- § Double successor weakening.
≤-weaken-suc2r : {a b : ℕ} → a ≤ b → a ≤ suc (suc b)
≤-weaken-suc2r p = ≤-weaken-sucr (≤-weaken-sucr p)

```

Step 2: Subadditivity Core. The core estimate is that normalization cannot increase total magnitude. Once this has been established, the triangle inequality follows by transporting the result through the embedding from $\mathbb{N}$ to $\mathbb{Z}$.

```

-- § mag of normalizeℤ is bounded by input sum.
magNormalize≤sum : (a b : ℕ) → magℤ (normalizeℤ a b) ≤ (a +ℕ b)
magNormalize≤sum zero zero = ≤-refl zero
magNormalize≤sum (suc a) zero =
  ≤-resp≡r
    (≤-refl (suc a))
    (sym (+ℕ-zero-right (suc a)))
magNormalize≤sum zero (suc b) = ≤-refl (suc b)
magNormalize≤sum (suc a) (suc b) =
  ≤-resp≡r
    (≤-weaken-suc2r (magNormalize≤sum a b))
    rhs
where
  rhs : suc (suc (a +ℕ b)) ≡ (suc a +ℕ suc b)
  rhs = sym (cong suc (+ℕ-suc-right a b))

-- § Magnitude is subadditive for +ℤ.
magℤ-+ℤ-subadd : (x y : ℤ) → magℤ (x +ℤ y) ≤ (magℤ x +ℕ magℤ y)
magℤ-+ℤ-subadd x y =
  ≤-resp≡r
    (magNormalize≤sum (pos px +ℕ pos py) (neg px +ℕ neg py))
    sumReassoc
where
  px : Pairℕ
  px = toPairℤ x

  py : Pairℕ
  py = toPairℤ y

cong2 : {A B C : Set} → (f : A → B → C) → {a a' : A} → {b b' : B} → a ≡ a' → b ≡ b' → f a b ≡ f a' b'
cong2 f refl refl = refl

pairSumMag : (z : ℤ) → (pos (toPairℤ z) +ℕ neg (toPairℤ z)) ≡ magℤ z
pairSumMag 0ℤ = refl
pairSumMag (+suc n) = +ℕ-zero-right (suc n)
pairSumMag (-suc n) = refl

pairSumMagPx : (pos px +ℕ neg px) ≡ magℤ x
pairSumMagPx = pairSumMag x

pairSumMagPy : (pos py +ℕ neg py) ≡ magℤ y
pairSumMagPy = pairSumMag y

sumReassoc :
  ((pos px +ℕ pos py) +ℕ (neg px +ℕ neg py))
  ≡
  (magℤ x +ℕ magℤ y)
sumReassoc =
  trans
    (shuffleℕ (pos px) (pos py) (neg px) (neg py))
    (cong2 _+_ pairSumMagPx pairSumMagPy)

-- § Transport ℕ-≤ into ≤ℤ for nonneg integers.
fromℕℤ-mono : {m n : ℕ} → m ≤ n → fromℕℤ m ≤ℤ fromℕℤ n

```

```

fromNZ-mono {zero} {zero} _ = tt
fromNZ-mono {zero} {suc n} _ = tt
fromNZ-mono {suc m} {zero} ()
fromNZ-mono {suc m} {suc n} p = p

-- § fromNZ is additive.
fromNZ-+Z : (m n : ℕ) → fromNZ m +Z fromNZ n ≡ fromNZ (m +N n)
fromNZ-+Z zero zero = refl
fromNZ-+Z zero (suc n) = refl
fromNZ-+Z (suc m) zero = refl
fromNZ-+Z (suc m) (suc n) = refl

-- § abs is subadditive on ℤ (triangle core).
absZ-subadd : (x y : ℤ) → absZ (x +Z y) ≤Z (absZ x +Z absZ y)
absZ-subadd x y =
  ≤Z-resp-≡l (sym lhsEq) (≤Z-resp-≡r (sym rhsEq) step1)
where
  step1 : fromNZ (magZ (x +Z y)) ≤Z fromNZ (magZ x +N magZ y)
  step1 = fromNZ-mono (magZ-+Z-subadd x y)

lhsEq : absZ (x +Z y) ≡ fromNZ (magZ (x +Z y))
lhsEq = absZ-fromNZ-magZ (x +Z y)

rhsEq : absZ x +Z absZ y ≡ fromNZ (magZ x +N magZ y)
rhsEq =
  trans
    (cong (λ t → t +Z absZ y) (absZ-fromNZ-magZ x))
    (trans
      (cong (λ t → fromNZ (magZ x) +Z t) (absZ-fromNZ-magZ y))
      (fromNZ-+Z (magZ x) (magZ y)))

```

Multiplicative Compatibility

Absolute value is also compatible with multiplication. The first step treats multiplication by a positive natural, after which full multiplicativity follows by a direct sign analysis on the two factors.

```

-- § abs commutes with multiplication by positive ℕ+.
absZ-mul-pos-right : (z : ℤ) → (d : ℕ+) → absZ (z *Z +toZ d) ≡ (absZ z *Z +toZ d)
absZ-mul-pos-right 0Z d =
  trans
    (cong absZ (*Z-zero-left (+toZ d)))
    (sym (*Z-zero-left (+toZ d)))
absZ-mul-pos-right (+suc n) (mkN+ k) =
  let mulPosForm : (+suc n) *Z (+suc k) ≡ +suc (k +N (n *N suc k))
      mulPosForm = *Z-pos-pos-eq n k
  in
  trans
    (trans (cong absZ mulPosForm) refl)
    (sym mulPosForm)

```

```

absℤ-mul-pos-right (-suc n) (mkℕ+ k) =
let mulNegForm : (-suc n) *ℤ (+suc k) ≡ -suc (k +ℕ (n *ℕ suc k))
    mulNegForm = *ℤ-neg-pos-eq n k
    mulPosForm : (+suc n) *ℤ (+suc k) ≡ +suc (k +ℕ (n *ℕ suc k))
    mulPosForm = *ℤ-pos-pos-eq n k
in
trans
(trans (cong absℤ mulNegForm) refl)
(sym mulPosForm)

```

Step 2: Full Multiplicativity. With the positive-factor case in hand, the general statement is obtained by splitting into the four sign combinations.

```

-- § abs is fully multiplicative.
absℤ-mul : (x y : ℤ) → absℤ (x *ℤ y) ≡ (absℤ x *ℤ absℤ y)
absℤ-mul 0ℤ y =
let
  lhs : absℤ (0ℤ *ℤ y) ≡ absℤ 0ℤ
  lhs = cong absℤ (*ℤ-zero-left y)

  rhs : (absℤ 0ℤ *ℤ absℤ y) ≡ absℤ 0ℤ
  rhs = *ℤ-zero-left (absℤ y)
in
trans lhs (sym rhs)
absℤ-mul x 0ℤ =
let
  lhs : absℤ (x *ℤ 0ℤ) ≡ absℤ 0ℤ
  lhs = cong absℤ (*ℤ-zero-right x)

  rhs : (absℤ x *ℤ absℤ 0ℤ) ≡ absℤ 0ℤ
  rhs =
    trans
      (cong (λ t → absℤ x *ℤ t) absℤ-zero)
      (*ℤ-zero-right (absℤ x))
in
trans lhs (sym rhs)
absℤ-mul (+suc m) (+suc n) =
let
  prodEq : (+suc m) *ℤ (+suc n) ≡ +suc (n +ℕ (m *ℕ suc n))
  prodEq = *ℤ-pos-pos-eq m n
in
trans (cong absℤ prodEq) (sym prodEq)
absℤ-mul (+suc m) (-suc n) =
let
  prodEq : (+suc m) *ℤ (+suc n) ≡ +suc (n +ℕ (m *ℕ suc n))
  prodEq = *ℤ-pos-pos-eq m n

  absProd : absℤ ((+suc m) *ℤ (+suc n)) ≡ (+suc m) *ℤ (+suc n)
  absProd = trans (cong absℤ prodEq) (sym prodEq)
in
trans
  (cong absℤ (*ℤ-neg-right (+suc m) (+suc n)))

```

```

(trans (absℤ-neg ((+suc m) *ℤ (+suc n))) absProd)
absℤ-mul (-suc m) (+suc n) =
let
  prodEq : (+suc m) *ℤ (+suc n) ≡ +suc (n + ℕ (m *ℕ suc n))
  prodEq = *ℤ-pos-pos-eq m n

  absProd : absℤ ((+suc m) *ℤ (+suc n)) ≡ (+suc m) *ℤ (+suc n)
  absProd = trans (cong absℤ prodEq) (sym prodEq)
in
trans
  (cong absℤ (*ℤ-neg-left (+suc m) (+suc n)))
  (trans (absℤ-neg ((+suc m) *ℤ (+suc n))) absProd)
absℤ-mul (-suc m) (-suc n) =
let
  mulEq : (-suc m) *ℤ (-suc n) ≡ (+suc m) *ℤ (+suc n)
  mulEq =
    trans
      (*ℤ-neg-right (negℤ (+suc m)) (+suc n))
      (trans
        (cong negℤ (*ℤ-neg-left (+suc m) (+suc n)))
        (negℤ-involutive ((+suc m) *ℤ (+suc n))))

  prodEq : (+suc m) *ℤ (+suc n) ≡ +suc (n + ℕ (m *ℕ suc n))
  prodEq = *ℤ-pos-pos-eq m n

  absProd : absℤ ((+suc m) *ℤ (+suc n)) ≡ (+suc m) *ℤ (+suc n)
  absProd = trans (cong absℤ prodEq) (sym prodEq)
in
trans (cong absℤ mulEq) absProd

-- § If -b ≤ a ≤ b then |a| ≤ b.
absℤ-within-bound : (a b : ℤ) → (negℤ b) ≤ℤ a → a ≤ℤ b → absℤ a ≤ℤ b
absℤ-within-bound 0ℤ 0ℤ _ _ = tt
absℤ-within-bound 0ℤ (+suc n) _ _ = tt
absℤ-within-bound 0ℤ (-suc n) _ neg-bound = neg-bound
absℤ-within-bound (+suc a) b _ upper = upper
absℤ-within-bound (-suc a) b lower _ =
  ≤ℤ-resp-≡r (negℤ-involutive b) (negℤ-antitone-≤ℤ lower)

```


Chapter 37

Additive Monotonicity and Positive Natural Laws

The next stage returns to monotonicity, now combining additive order arguments with the explicit type of positive naturals. The goal is to move cleanly between natural inequalities, integer inequalities, and the normalized presentations used earlier.

Nonneg Monotonicity Bridge

We begin by relating nonnegative integers to their natural-number representatives. This bridge allows monotonicity statements on $\mathbb{N}$ to be transferred directly to addition on the nonnegative cone of $\mathbb{Z}$.

```
-- § Nonneg left monotonicity via fromℕℤ.
≤ℤ-fromℕℤ-+ℕ-monol : {a b : ℕ} → a ≤ b → (c : ℕ) → fromℕℤ (c +ℕ a) ≤ℤ fromℕℤ (c +ℕ b)
≤ℤ-fromℕℤ-+ℕ-monol p c = fromℕℤ-mono (≤-+ℕ-monol p c)

-- § Nonneg right monotonicity via fromℕℤ.
≤ℤ-fromℕℤ-+ℕ-monor : {a b : ℕ} → a ≤ b → (c : ℕ) → fromℕℤ (a +ℕ c) ≤ℤ fromℕℤ (b +ℕ c)
≤ℤ-fromℕℤ-+ℕ-monor {a} {b} p c =
let
  lhs : fromℕℤ (a +ℕ c) ≡ fromℕℤ (c +ℕ a)
  lhs = cong fromℕℤ (+ℕ-comm a c)

  rhs : fromℕℤ (b +ℕ c) ≡ fromℕℤ (c +ℕ b)
  rhs = cong fromℕℤ (+ℕ-comm b c)

  base : fromℕℤ (c +ℕ a) ≤ℤ fromℕℤ (c +ℕ b)
  base = ≤ℤ-fromℕℤ-+ℕ-monol p c
in
≤ℤ-resp-≡l (sym lhs) (≤ℤ-resp-≡r (sym rhs) base)

-- § Nonneg +ℤ right monotonicity.
```

```

≤ℤ-+ℤ-monor-nonneg : {m n : ℕ} → m ≤ n → (k : ℕ) → (fromℕℤ m + ℤ fromℕℤ k) ≤ℤ (fromℕℤ n + ℤ fromℕℤ k)
≤ℤ-+ℤ-monor-nonneg {m} {n} p k =
  ≤ℤ-resp-≡l (sym (fromℕℤ-+ℤ m k))
  (≤ℤ-resp-≡r (sym (fromℕℤ-+ℤ n k)))
  (≤ℤ-fromℕℤ-+ℕ-monor p k))

-- § Reflect ≤ℤ back to ℕ-≤ for nonneg integers.
≤ℤ-fromℕℤ-reflect : {m n : ℕ} → fromℕℤ m ≤ℤ fromℕℤ n → m ≤ n
≤ℤ-fromℕℤ-reflect {zero} {zero} _ = z ≤ n
≤ℤ-fromℕℤ-reflect {zero} {suc n} _ = z ≤ n
≤ℤ-fromℕℤ-reflect {suc m} {zero} ()
≤ℤ-fromℕℤ-reflect {suc m} {suc n} p = p

-- § Nonnegativity forces fromℕℤ form.
0 ≤ℤ → fromℕℤ : (z : ℤ) → 0ℤ ≤ℤ z → ∑ ℕ (λ n → z ≡ fromℕℤ n)
0 ≤ℤ → fromℕℤ 0ℤ _ = zero , refl
0 ≤ℤ → fromℕℤ (+suc n) _ = suc n , refl
0 ≤ℤ → fromℕℤ (-suc n) ()

-- § Both-slot nonneg monotonicity.
≤ℤ-+ℤ-mono-nonneg2 : {m m' n n' : ℕ} → m ≤ m' → n ≤ n' →
  (fromℕℤ m + ℤ fromℕℤ n) ≤ℤ (fromℕℤ m' + ℤ fromℕℤ n')
≤ℤ-+ℤ-mono-nonneg2 {m} {m'} {n} {n'} m ≤ m' n ≤ n' =
  let
    step1 : (fromℕℤ m + ℤ fromℕℤ n) ≤ℤ (fromℕℤ m' + ℤ fromℕℤ n)
    step1 = ≤ℤ-+ℤ-monor-nonneg m ≤ m' n

    step2 : (fromℕℤ m' + ℤ fromℕℤ n) ≤ℤ (fromℕℤ m' + ℤ fromℕℤ n')
    step2 =
      ≤ℤ-resp-≡l (+ℤ-comm (fromℕℤ n) (fromℕℤ m'))
      (≤ℤ-resp-≡r (+ℤ-comm (fromℕℤ n') (fromℕℤ m')))
      (≤ℤ-+ℤ-monor-nonneg n ≤ n' m')
  in
    ≤ℤ-trans step1 step2

```

Normalize–Cross Bridge

The next comparison rewrites order between normalized pairs into a cross-sum inequality on naturals. This translation is what makes the later order arguments compatible with the pair-level algebra developed earlier.

```

-- § Forward: normalize order implies cross-sum inequality.
normalize≤→cross : (a b c d : ℕ) → normalizeℤ a b ≤ℤ normalizeℤ c d → (a + ℕ d) ≤ (c + ℕ b)
normalize≤→cross (suc a) (suc b) c d p =
  let ih : (a + ℕ d) ≤ (c + ℕ b)
      ih = normalize≤→cross a b c d p

  lifted : (suc (a + ℕ d)) ≤ (suc (c + ℕ b))
  lifted = s ≤ s ih

  rhsEq : (c + ℕ suc b) ≡ suc (c + ℕ b)

```

```

    rhsEq = +N-suc-right c b
in
  subst (λ t → (suc a +N d) ≤ t) (sym rhsEq) lifted
  normalize≤→cross a b (suc c) (suc d) p =
  let ih : (a +N d) ≤ (c +N b)
    ih = normalize≤→cross a b c d p

    lifted : (suc (a +N d)) ≤ (suc (c +N b))
    lifted = s≤s ih

    lhsEq : (a +N suc d) ≡ suc (a +N d)
    lhsEq = +N-suc-right a d
in
  subst (λ t → t ≤ (suc c +N b)) (sym lhsEq) lifted

  normalize≤→cross zero zero zero zero _ = z≤n
  normalize≤→cross zero zero (suc c) zero _ = z≤n
  normalize≤→cross zero zero zero (suc d) ()
  normalize≤→cross (suc a) zero zero zero ()
  normalize≤→cross (suc a) zero (suc c) zero p =
  let
    lhsEq : (suc a +N zero) ≡ suc a
    lhsEq = cong suc (+N-zero-right a)

    rhsEq : (suc c +N zero) ≡ suc c
    rhsEq = cong suc (+N-zero-right c)
in
  subst (λ t → t ≤ (suc c +N zero)) (sym lhsEq)
  (subst (λ t → (suc a) ≤ t) (sym rhsEq) p)
  normalize≤→cross (suc a) zero zero (suc d) ()
  normalize≤→cross zero (suc b) zero zero _ = z≤n
  normalize≤→cross zero (suc b) (suc c) zero _ = z≤n
  normalize≤→cross zero (suc b) zero (suc d) p = p

-- § Backward: cross-sum inequality implies normalize order.
cross→normalize≤ : (a b c d : ℕ) → (a +N d) ≤ (c +N b) → normalizeℤ a b ≤ℤ normalizeℤ c d
cross→normalize≤ (suc a) (suc b) c d p with subst (λ t → (suc a +N d) ≤ t) (+N-suc-right c b) p
... | s≤s q = cross→normalize≤ a b c d q
cross→normalize≤ a b (suc c) (suc d) p with subst (λ t → t ≤ (suc c +N b)) (+N-suc-right a d) p
... | s≤s q = cross→normalize≤ a b c d q

cross→normalize≤ zero zero zero zero _ = tt
cross→normalize≤ zero zero (suc c) zero _ = tt
cross→normalize≤ zero zero zero (suc d) ()
cross→normalize≤ (suc a) zero zero zero ()
cross→normalize≤ (suc a) zero (suc c) zero p =
let
  lhsEq : (suc a +N zero) ≡ suc a
  lhsEq = cong suc (+N-zero-right a)

  rhsEq : (suc c +N zero) ≡ suc c
  rhsEq = cong suc (+N-zero-right c)

p' : (suc a) ≤ (suc c)

```

```

p' =
  subst (λ t → t ≤ (suc c)) lhsEq
    (subst (λ t → (suc a +ℕ zero) ≤ t) rhsEq p)
in
p'
cross→normalize≤ (suc a) zero zero (suc d) ()
cross→normalize≤ zero (suc b) zero zero _ = tt
cross→normalize≤ zero (suc b) (suc c) zero _ = tt
cross→normalize≤ zero (suc b) zero (suc d) p = p

```

General Additive Monotonicity

We can now return to addition on integers in full generality. The pair-based description of order makes it possible to prove monotonicity and additive cancellation without leaving the concrete normal forms.

```

-- § Right monotonicity of +ℤ (general).
≤ℤ-+ℤ-monor : {x y : ℤ} → x ≤ℤ y → (z : ℤ) → (x +ℤ z) ≤ℤ (y +ℤ z)
≤ℤ-+ℤ-monor {x} {y} x≤y z =
let
  px = toPairℤ x
  py = toPairℤ y
  pz = toPairℤ z

  ax = pos px
  bx = neg px
  ay = pos py
  by = neg py
  az = pos pz
  bz = neg pz

  x≤y' : normalizeℤ ax bx ≤ℤ normalizeℤ ay by
  x≤y' =
    ≤ℤ-resp-≡r (sym (from-toPairℤ y))
      (≤ℤ-resp-≡l (sym (from-toPairℤ x)) x≤y)

  crossXY : (ax +ℕ by) ≤ (ay +ℕ bx)
  crossXY = normalize≤→cross ax bx ay by x≤y'

  k : ℕ
  k = az +ℕ bz

  base : (k +ℕ (ax +ℕ by)) ≤ (k +ℕ (ay +ℕ bx))
  base = ≤-+ℕ-monol crossXY k

  lhsEq : ((ax +ℕ az) +ℕ (by +ℕ bz)) ≡ (k +ℕ (ax +ℕ by))
  lhsEq =
    trans
      (shuffleℕ ax az by bz)
      (+ℕ-comm (ax +ℕ by) k)

```

```

rhsEq : ((ay + $\mathbb{N}$  az) + $\mathbb{N}$  (bx + $\mathbb{N}$  bz))  $\equiv$  (k + $\mathbb{N}$  (ay + $\mathbb{N}$  bx))
rhsEq =
  trans
    (shuffle $\mathbb{N}$  ay az bx bz)
    (+ $\mathbb{N}$ -comm (ay + $\mathbb{N}$  bx) k)

sumCross : ((ax + $\mathbb{N}$  az) + $\mathbb{N}$  (by + $\mathbb{N}$  bz))  $\leq$  ((ay + $\mathbb{N}$  az) + $\mathbb{N}$  (bx + $\mathbb{N}$  bz))
sumCross =
  subst ( $\lambda$  t  $\rightarrow$  t  $\leq$  ((ay + $\mathbb{N}$  az) + $\mathbb{N}$  (bx + $\mathbb{N}$  bz))) (sym lhsEq)
  (subst ( $\lambda$  t  $\rightarrow$  (k + $\mathbb{N}$  (ax + $\mathbb{N}$  by))  $\leq$  t) (sym rhsEq) base)
in
cross $\rightarrow$ normalize $\leq$  (ax + $\mathbb{N}$  az) (bx + $\mathbb{N}$  bz) (ay + $\mathbb{N}$  az) (by + $\mathbb{N}$  bz) sumCross

-- § Left monotonicity of + $\mathbb{Z}$ .
 $\leq\mathbb{Z}$ -+ $\mathbb{Z}$ -mono $^l$  : {x y :  $\mathbb{Z}$ }  $\rightarrow$  x  $\leq\mathbb{Z}$  y  $\rightarrow$  (z :  $\mathbb{Z}$ )  $\rightarrow$  (z + $\mathbb{Z}$  x)  $\leq\mathbb{Z}$  (z + $\mathbb{Z}$  y)
 $\leq\mathbb{Z}$ -+ $\mathbb{Z}$ -mono $^l$  {x} {y} x $\leq$ y z =
   $\leq\mathbb{Z}$ -resp $\equiv^l$  (+ $\mathbb{Z}$ -comm x z)
  ( $\leq\mathbb{Z}$ -resp $\equiv^r$  (+ $\mathbb{Z}$ -comm y z)
  ( $\leq\mathbb{Z}$ -+ $\mathbb{Z}$ -mono $^r$  x $\leq$ y z))

-- § Both-slot monotonicity of + $\mathbb{Z}$ .
 $\leq\mathbb{Z}$ -+ $\mathbb{Z}$ -mono : {x y u v :  $\mathbb{Z}$ }  $\rightarrow$  x  $\leq\mathbb{Z}$  y  $\rightarrow$  u  $\leq\mathbb{Z}$  v  $\rightarrow$  (x + $\mathbb{Z}$  u)  $\leq\mathbb{Z}$  (y + $\mathbb{Z}$  v)
 $\leq\mathbb{Z}$ -+ $\mathbb{Z}$ -mono {x} {y} {u} {v} x $\leq$ y u $\leq$ v =
   $\leq\mathbb{Z}$ -trans ( $\leq\mathbb{Z}$ -+ $\mathbb{Z}$ -mono $^r$  x $\leq$ y u) ( $\leq\mathbb{Z}$ -+ $\mathbb{Z}$ -mono $^l$  u $\leq$ v v)

-- § Right additive cancellation for  $\leq\mathbb{Z}$ .
 $\leq\mathbb{Z}$ -+ $\mathbb{Z}$ -cancel $^r$  : (x y z :  $\mathbb{Z}$ )  $\rightarrow$  x  $\leq\mathbb{Z}$  (z + $\mathbb{Z}$  y)  $\rightarrow$  (x + $\mathbb{Z}$  neg $\mathbb{Z}$  y)  $\leq\mathbb{Z}$  z
 $\leq\mathbb{Z}$ -+ $\mathbb{Z}$ -cancel $^r$  x y z p =
  let
    step : (x + $\mathbb{Z}$  neg $\mathbb{Z}$  y)  $\leq\mathbb{Z}$  ((z + $\mathbb{Z}$  y) + $\mathbb{Z}$  neg $\mathbb{Z}$  y)
    step =  $\leq\mathbb{Z}$ -+ $\mathbb{Z}$ -mono $^r$  p (neg $\mathbb{Z}$  y)

  rhsEq : ((z + $\mathbb{Z}$  y) + $\mathbb{Z}$  neg $\mathbb{Z}$  y)  $\equiv$  z
  rhsEq =
    trans
      (+ $\mathbb{Z}$ -assoc z y (neg $\mathbb{Z}$  y))
      (trans
        (cong ( $\lambda$  t  $\rightarrow$  z + $\mathbb{Z}$  t) (+ $\mathbb{Z}$ -inv-right y))
        (+ $\mathbb{Z}$ -zero-right z))
  in
   $\leq\mathbb{Z}$ -resp $\equiv^r$  rhsEq step

```

Step 2: $\mathbb{N}^+$ Multiplication Homomorphism. The next lemmas connect positive naturals with integer multiplication. In particular, they show that the embedding from $\mathbb{N}$ to $\mathbb{Z}$ behaves multiplicatively on positive denominators and that doubling a positive integer places it strictly above 1.

```

-- § from $\mathbb{N}\mathbb{Z}$  is multiplicative for  $\mathbb{N}^+$ .
from $\mathbb{N}\mathbb{Z}$ -mul- $^+$  : (n :  $\mathbb{N}$ )  $\rightarrow$  (d :  $\mathbb{N}^+$ )  $\rightarrow$  (from $\mathbb{N}\mathbb{Z}$  n * $\mathbb{Z}$  +to $\mathbb{Z}$  d)  $\equiv$  from $\mathbb{N}\mathbb{Z}$  (n * $\mathbb{N}^+$  +to $\mathbb{N}$  d)
from $\mathbb{N}\mathbb{Z}$ -mul- $^+$  zero d =
  trans
    (* $\mathbb{Z}$ -zero-left (+to $\mathbb{Z}$  d))

```

```

    (cong fromℕℤ (sym (*ℕ-zero-left (+toℕ d))))
fromℕℤ-mul-+ (suc n) (mkℕ+ k) =
let
  natForm : (suc n *ℕ suc k) ≡ suc (k +ℕ (n *ℕ suc k))
  natForm = refl

  rhs : fromℕℤ (suc n *ℕ suc k) ≡ +suc (k +ℕ (n *ℕ suc k))
  rhs = cong fromℕℤ natForm
in
trans
(*ℤ-pos-pos-eq n k)
(sym rhs)

-- § 1 < 2z for z positive.
oneℤ<twoTimes-pos : (z : ℤ) → 0ℤ <ℤ z → oneℤ <ℤ (z +ℤ z)
oneℤ<twoTimes-pos z zpos with 0<ℤ→pos z zpos
... | (m , z≡) =
<ℤ-resp≡r (cong (λ t → t +ℤ t) (sym z≡)) (lePart , notRev)
where
  twoTimes : (+suc m) +ℤ (+suc m) ≡ +suc (m +ℕ suc m)
  twoTimes =
    trans
      (fromℕℤ-+ℤ (suc m) (suc m))
      (cong fromℕℤ refl)

  lePart : oneℤ ≤ℤ ((+suc m) +ℤ (+suc m))
  lePart =
    let
      lePos : oneℤ ≤ℤ (+suc (m +ℕ suc m))
      lePos = s≤s z≤n
    in
    subst (λ t → oneℤ ≤ℤ t) (sym twoTimes) lePos

no-suc≤zero : {t : ℕ} → suc t ≤ zero → ⊥
no-suc≤zero ()

impossible : (+suc (m +ℕ suc m)) ≤ℤ oneℤ → ⊥
impossible (s≤s pNat) =
let
  pNat' : suc (m +ℕ m) ≤ zero
  pNat' = subst (λ t → t ≤ zero) (+ℕ-suc-right m m) pNat
in
no-suc≤zero pNat'

notRev : ((+suc m) +ℤ (+suc m)) <ℤ oneℤ
notRev q = impossible (subst (λ t → t ≤ℤ oneℤ) twoTimes q)

```

Laws of $\mathbb{N}^+$

The positive naturals are then given their expected algebraic laws. Since they are represented by predecessors, these statements reduce directly to the multiplication laws already proved on $\mathbb{N}$ and $\mathbb{Z}$.

```

-- § Commutativity of  $\mathbb{N}^+$  multiplication.
*+-comm : (x y :  $\mathbb{N}^+$ ) → x *+ y ≡ y *+ x
*+-comm (mk $\mathbb{N}^+$  a) (mk $\mathbb{N}^+$  b) =
cong mk $\mathbb{N}^+$  (trans lhsNorm (sym rhsNorm))
where
lhsNorm : (a * $\mathbb{N}$  suc b) + $\mathbb{N}$  b ≡ (a + $\mathbb{N}$  b) + $\mathbb{N}$  (a * $\mathbb{N}$  b)
lhsNorm =
trans
  (cong (λ t → t + $\mathbb{N}$  b) (* $\mathbb{N}$ -suc-right-+ $\mathbb{N}$  a b))
  (trans
    (+ $\mathbb{N}$ -assoc a (a * $\mathbb{N}$  b) b)
    (trans
      (cong (λ t → a + $\mathbb{N}$  t) (+ $\mathbb{N}$ -comm (a * $\mathbb{N}$  b) b))
      (sym (+ $\mathbb{N}$ -assoc a b (a * $\mathbb{N}$  b)))))

rhsNorm : (b * $\mathbb{N}$  suc a) + $\mathbb{N}$  a ≡ (a + $\mathbb{N}$  b) + $\mathbb{N}$  (a * $\mathbb{N}$  b)
rhsNorm =
trans
  (cong (λ t → t + $\mathbb{N}$  a) (* $\mathbb{N}$ -suc-right-+ $\mathbb{N}$  b a))
  (trans
    (cong (λ t → (b + $\mathbb{N}$  t) + $\mathbb{N}$  a) (* $\mathbb{N}$ -comm b a))
    (trans
      (+ $\mathbb{N}$ -assoc b (a * $\mathbb{N}$  b) a)
      (trans
        (cong (λ t → b + $\mathbb{N}$  t) (+ $\mathbb{N}$ -comm (a * $\mathbb{N}$  b) a))
        (trans
          (swapHead $\mathbb{N}$  b a (a * $\mathbb{N}$  b))
          (sym (+ $\mathbb{N}$ -assoc a b (a * $\mathbb{N}$  b)))))

-- § +to $\mathbb{Z}$  is a multiplicative homomorphism.
+to $\mathbb{Z}$ -*+ : (x y :  $\mathbb{N}^+$ ) → +to $\mathbb{Z}$  (x *+ y) ≡ (+to $\mathbb{Z}$  x) * $\mathbb{Z}$  (+to $\mathbb{Z}$  y)
+to $\mathbb{Z}$ -*+ (mk $\mathbb{N}^+$  a) (mk $\mathbb{N}^+$  b) =
sym
  (trans
    (* $\mathbb{Z}$ -pos-pos-eq a b)
    (cong (λ t → +suc t) (+ $\mathbb{N}$ -comm b (a * $\mathbb{N}$  suc b))))

```

With these positive-denominator tools available, the development is one step away from the simplex evaluation. Before that step is taken, the form of the evaluating expression must be accounted for. The next chapter shows that the four invariants of the closed record are not arithmetic quantities of the same kind, and that this difference of kind — not a choice of formula — determines whether two of them combine by product or by sum.

Chapter 38

The Kind Classification of Invariants

The closed K_4 record carries four invariants: the vertex count V , the edge count E , the dimensional invariant d , and the Euler characteristic χ . Treated as a tuple in $\mathbb{N}^4$, they admit innumerable arithmetic combinations. Treated according to what each one *counts*, they fall into four pairwise distinct kinds, and the form of any combination of them is no longer free.

This chapter exhibits the kind classification, proves the pairwise disjointness of the four kinds, and reads off the consequence: an expression that combines two invariants of the *same* kind is a product (the categorical operation that respects the shared kind); an expression that combines two invariants of *disjoint* kinds is a sum (the categorical operation that admits both kinds without identifying them). The rule is the standard Curry–Howard reading of the data type below: same constructor accumulates multiplicatively, different constructors accumulate additively under the unique forgetful map into $\mathbb{N}$.

Four Kinds, Not Four Numbers

The four invariants of the closed record are not interchangeable quantities. Each is a specific kind of count, attached to a specific piece of structure of K_4 .

The vertex count V counts the elements of a finite set: the four distinguishable configurations carried by the survivor. It is a *configurational* quantity. Its powers V^d count tuples of configurations across d independent extension directions; they are configurational as well.

The edge count E counts unordered pairs of configurations that stand in the relation *is distinguished from*. Its constructor is the combinatorial number $\binom{V}{2}$. It is a

relational quantity.

The dimensional invariant d counts the independent monotone directions of extension exhibited in Chapter 27. On a complete graph it equals $V - 1$, the dimension of the non-trivial eigenspace of the Laplacian, and the embedding dimension of the simplex carrier. Its constructor is `dimension`. It is a *directional* quantity. Its square d^2 is also directional: it counts ordered pairs of independent directions, the natural cross-direction term of an extension structure.

The Euler characteristic $\chi = V - E + F$ is the alternating sum of the cell counts of the simplicial closure. It is a global invariant of the closed structure: the witness that the survivor closes on itself without leaving cells unaccounted for. Its constructor is the alternating sum of a complex. It is a *topological* quantity.

The topological kind is irreducible: χ depends on the face count F , and F is not a function of V , E , d alone. A simplicial complex on the same vertex and edge counts may close with different face counts; the closure datum is independent of the lower-dimensional incidence data. The kind classification therefore cannot collapse χ into the configurational, relational, or directional kind: doing so would silently identify F with a value derivable from V , E , d , which is false in general and only happens to coincide on K_4 .

The arithmetic identities that hold among these invariants on $K_4 - E = \binom{V}{2}$, $d = V - 1$, $\chi = V + F - E$ — are equations between values, not identifications between constructors. The constructors stay disjoint at the type level. This disjointness is the content of the next section.

The four invariants of the closed record must be classified by what they count, not by what they evaluate to. The code below introduces a four-element data type whose constructors correspond to the four kinds, and it assigns each invariant to its constructor. From that point on the four invariants can no longer be treated as a homogeneous tuple: every later expression in them carries a kind label on each factor.

```
-- § The four kinds of invariant attached to the closed K4 record.
data InvariantKind : Set where
  configurational : InvariantKind -- counts elements of a finite set
  relational      : InvariantKind -- counts unordered pairs
  directional     : InvariantKind -- counts independent extension directions
  topological     : InvariantKind -- alternating sum of cell counts

-- § Exhaustiveness: every InvariantKind reduces to one of the four
-- constructors. The pattern match below is the type-level reading of
-- the four-constructor closure of the data type. No fifth slot can
-- inhabit InvariantKind because no fifth constructor is declared, and
-- the case split below covers every inhabitant.
kind-exhaustive :
  ∀ (k : InvariantKind)
    → (k ≡ configurational)
    ⊔ (k ≡ relational)
    ⊔ (k ≡ directional)
```

```

    ⊔ (k ≡ topological)
kind-exhaustive configurational = inj₁ refl
kind-exhaustive relational      = inj₂ (inj₁ refl)
kind-exhaustive directional    = inj₂ (inj₂ (inj₁ refl))
kind-exhaustive topological    = inj₂ (inj₂ (inj₂ refl))

-- § Kind assignments for the four K₄ invariants.
V-kind : InvariantKind
V-kind = configurational

E-kind : InvariantKind
E-kind = relational

d-kind : InvariantKind
d-kind = directional

χ-kind : InvariantKind
χ-kind = topological

```

Pairwise Disjointness

The four constructors of `InvariantKind` are pairwise distinct in the strict sense: no two of them are propositionally equal. This is the standard property of constructors in a data type with no equations imposed; the absurd-pattern proofs witness it explicitly.

The disjointness is what carries the Curry–Howard reading. If two invariants have the same kind, they are inhabitants of the same constructor and combine inside that kind. If two invariants have disjoint kinds, no map identifies their constructors, and any combination must record both kinds separately. Under the unique forgetful map from `InvariantKind`-tagged quantities to plain naturals, the first reading turns into multiplication of the values (the categorical product reduces to numerical product), and the second into addition (the coproduct reduces to numerical sum).

The four kinds must be pairwise distinguishable inside the type theory. Every candidate identification between two distinct constructors is excluded by the absurd pattern. Six identifications would be needed; six are discharged. No kind collapses into another, so the classification remains separated at the constructor level.

```

-- § The six pairwise disjointness lemmas.
configurational#relational : ¬ (configurational ≡ relational)
configurational#relational ()

configurational#directional : ¬ (configurational ≡ directional)
configurational#directional ()

configurational#topological : ¬ (configurational ≡ topological)
configurational#topological ()

relational#directional : ¬ (relational ≡ directional)
relational#directional ()

```

```

relational#topological : ¬ (relational ≡ topological)
relational#topological ()

directional#topological : ¬ (directional ≡ topological)
directional#topological ()

```

The Forgetful Map: Kind-Tagged Arithmetic

The disjointness lemmas leave the four kinds rigidly distinguishable inside the type theory. The simplex evaluation, however, is a plain natural number; somewhere the kind labels must be discharged. The discharge is not arbitrary. It is the unique forgetful map from kind-tagged values into $\mathbb{N}$ that respects the categorical structure of the kind labels.

A kind-tagged value is a natural number carrying its kind as a type index. Within a shared kind, two such values combine multiplicatively under the categorical product: the resulting tag is the shared kind, and the underlying value is the product of the values. Across disjoint kinds, two such values combine additively under the categorical coproduct: the operation requires a disjointness witness at the type level, and the underlying value is the sum of the values. The forgetful reductions are not asserted; they are computed by `refl`.

The kind labels must be discharged by a map into $\mathbb{N}$ that respects the categorical operations on tagged values: product within a shared kind, coproduct across disjoint kinds. The indexed record `Tagged` below carries a value at a given kind. The combinators `_ ⊗ _` and `_ ⊕ [_]_` implement the product and the coproduct; the disjointness witness is an obligation in the type of the coproduct combinator and cannot be supplied for two values of the same kind. The forgetful reductions hold definitionally.

The verbal claim “shared kind $\rightarrow$ product $\rightarrow$ multiplication; disjoint kinds $\rightarrow$ coproduct $\rightarrow$ addition” is thereby a definitional reduction inside the type theory. The forgetful map is unique up to definitional equality, and the arithmetic shape of the simplex evaluation is determined by the type of its tagged operands.

```

-- § A natural number carrying its kind as a type index.
record Tagged (k : InvariantKind) : Set where
  constructor tag
  field value : ℕ
open Tagged public

-- § Product within a shared kind: values multiply.
infixl 7 _⊗_
_⊗_ : ∀ {k} → Tagged k → Tagged k → Tagged k
tag m ⊗ tag n = tag (m *ℕ n)

-- § Coproduct across disjoint kinds: values add. The disjointness
-- witness is required at the type level; two values of the same kind

```

```

-- cannot use this combinator.
infixl 6 _⊕[_]
_⊕[_] : ∀ {j k} → Tagged j → ¬ (j ≡ k) → Tagged k → ℕ
tag m ⊕[_] tag n = m +ℕ n

-- § Forgetful reductions: the verbal claim becomes a definitional
-- equality. Both proofs are refl.
forget-⊗ : ∀ {k} (x y : Tagged k)
  → value (x ⊗ y) ≡ value x *ℕ value y
forget-⊗ (tag _) (tag _) = refl

forget-⊕ : ∀ {j k} (x : Tagged j) (p : ¬ (j ≡ k)) (y : Tagged k)
  → (x ⊕[ p ] y) ≡ value x +ℕ value y
forget-⊕ (tag _) _ (tag _) = refl

```

Bulk and Boundary

The expression that the next chapter will evaluate has two terms. The first, $V^d \cdot \chi$, multiplies a configurational quantity by a topological one. The second, d^2 , is purely directional. The two terms combine additively.

The structural reading of this shape is now forced. Inside the first term, configurational and topological invariants meet. They occupy distinct kinds, so they cannot collapse; but they describe the same underlying object — the closed extension structure — evaluated in two compatible aspects (its configurations and its closure condition). Their categorical combination is a product of factors that share the same carrier, and the forgetful functor sends this to numerical multiplication. The first term is the *bulk* of the evaluation: configurations weighted by closure.

The second term draws on a kind disjoint from both factors of the bulk. The directional invariant d counts neither configurations nor closure cells; it counts the independent extension axes carrying the structure. A boundary correction along these axes, expressed as the count of ordered pairs of axes, has nowhere to merge with the bulk multiplicatively without identifying its kind with one already present in the bulk. The absurd patterns of the previous section forbid that identification. The remaining categorical operation between disjoint kinds is the coproduct, which the forgetful functor sends to numerical addition.

The bulk term and the boundary term draw on kinds related by the disjointness lemmas of the previous section. The record below witnesses the kind assignments of both terms and exhibits the disjointness needed for the additive combination. Inside the displayed kind discipline, the form $V^d \cdot \chi + d^2$ is no longer a choice: it is the only combination of these four invariants whose inner factors share their kind and whose outer factors do not.

```

-- § Bulk and boundary draw on disjoint kinds → forced additive combination.
record BulkBoundaryIndependence : Set where

```

```

field
bulk-factor1 : InvariantKind -- V (and hence Vd)
bulk-factor2 : InvariantKind --  $\chi$ 
boundary-kind : InvariantKind -- d (and hence d2)
bulk1-is-configurational : bulk-factor1  $\equiv$  configurational
bulk2-is-topological      : bulk-factor2  $\equiv$  topological
boundary-is-directional  : boundary-kind  $\equiv$  directional
bulk1-disjoint-boundary :  $\neg$  (bulk-factor1  $\equiv$  boundary-kind)
bulk2-disjoint-boundary :  $\neg$  (bulk-factor2  $\equiv$  boundary-kind)

theorem-bulk-boundary-independent : BulkBoundaryIndependence
theorem-bulk-boundary-independent = record
{ bulk-factor1      = configurational
; bulk-factor2      = topological
; boundary-kind      = directional
; bulk1-is-configurational = refl
; bulk2-is-topological   = refl
; boundary-is-directional = refl
; bulk1-disjoint-boundary = configurational#directional
; bulk2-disjoint-boundary =  $\lambda p \rightarrow$  directional#topological (sym p)
}

```

Source-Indexing: Why Exactly Four Kinds

The four kinds of the previous sections are not a choice of taxonomy. They are the four primitive cardinality sources of a closed simplicial d -extension on K_4 , and no fifth source exists. The classification is therefore not imposed; it is read off the record.

A closed simplicial d -extension on K_4 carries cells in dimensions 0, 1, and 2, terminating at the top-dimensional face count because K_4 is the closure of the 3-simplex and admits no independent higher-dimensional stratum. The numerical readings available on the closed structure are therefore exactly four: the count of 0-cells, the count of 1-cells, the count of 2-cells, and the ambient direction count d established by Chapter 27. Anything else read off the record is a function of these four sources, not a fifth source, because the record carries no datum beyond cells and directions.

Of these four sources, the 2-cell count F is not refinement-stable: a barycentric subdivision changes F without changing the topological content of the closure. The closure-stable substitute is the alternating sum $\chi = V - E + F$, which absorbs F into a refinement-invariant combination. The four primitive cardinality sources of the closed record therefore collapse, under refinement-stability, to: V (configurational, the 0-cell reading), E (relational, the 1-cell reading), d (directional, the ambient direction reading), and χ (topological, the refinement-stable closure reading).

The kind classification of the previous sections is the source-indexing of the record itself: each kind names the structural slot from which its invariant is read. A fifth kind would require a fifth slot. The strata $\{0, 1, 2\}$ exhaust the cell dimensions present in

the closed 3-simplex; the direction count exhausts the non-cell data the record carries. No fifth slot exists.

Every $\text{Aut}(K_4)$ -invariant numerical extract of the closed record must factor through one of the record's primitive cardinality sources. A fifth kind is excluded by the structural exhaustion of the record's slots: no cell stratum beyond $\{0, 1, 2\}$ exists in the closed 3-simplex, and no non-cell datum beyond direction is carried by the record.

The four-element data type `InvariantKind` is therefore not a choice of categorisation. It is the unique enumeration of the record's primitive sources, and the four invariants are the unique refinement-stable readings of those sources. The disjointness lemmas of §38 are then not the rigidity of an external classification, but the separation carried by the record's own slot structure.

The absorption of F into χ on K_4 is not a refinement step but the conjunction of two computations already performed in Chapter 33: $F = V$ by self-duality of the tetrahedron (`simplex-faces` $\equiv$ `simplex-vertices` as `law14C-2a-faces`), and $\chi = V - E + F = 2$ by the alternating sum (`law14C-3-chi`). The face count carries no degree of freedom on K_4 beyond what V and χ already record.

The closure reading must not introduce any datum on K_4 beyond what the simplex constants already fix. The record below collects the two relevant refl-equalities, and its inhabitant is built from the existing laws. On K_4 , the absorption of F into χ is mechanical: self-duality and the Euler reduction together leave no residue.

```
record SelfDualClosureWitness : Set where
  field
    F-equals-V : simplex-faces  $\equiv$  simplex-vertices
    chi-evaluates : simplex-chi  $\equiv$  2

self-dual-closure-witness : SelfDualClosureWitness
self-dual-closure-witness = record
{ F-equals-V = law14C-2a-faces
; chi-evaluates = law14C-3-chi
}
```

The Extract Concept: From Binary Distinction to Cardinality

The previous section spoke of an “ $\text{Aut}(K_4)$ -invariant numerical extract of the closed record” as if the notion were self-evident. It is not. If the entire programme of this book is that everything follows from the represented binary distinction, the notion of *extract* cannot be exempt: it must itself be forced by the distinction, not imported from outside.

A binary distinction has no internal labels. The two sides are distinguishable from each other, but neither side carries an identity beyond “not the other”. Whatever is read off such a distinction must therefore not depend on which side is which: any reading that does depend on a labelling of sides is not a reading of the distinction itself, but a reading of an extra structure quietly attached to it. This is the only constraint a binary distinction imposes on its readings, and it is enough to fix what an extract is.

Iterated, the binary distinction produces the closed K_4 record through the elimination chain of the earlier chapters. The internal symmetries of that record — its relabellings of the four configurations that preserve the distinction structure — are exactly $\text{Aut}(K_4)$. A reading that depends on a particular labelling of the four configurations smuggles in information the distinction did not provide. A reading that is constant on every $\text{Aut}(K_4)$ -orbit of the record’s data uses only what the distinction itself supplies.

An extract is a reading of the closed record that is constant on every $\text{Aut}(K_4)$ -orbit. This is not a definition imposed on top of the record. It is the only kind of reading the binary distinction admits.

The orbit structure of $\text{Aut}(K_4)$ on the record is now read off the record itself. The action is transitive on each cell stratum: on the four vertices, on the six edges, on the four 2-faces, and on the unique 3-cell. A function constant on each transitive orbit is determined by the cardinality of that orbit alone. The available cardinalities are therefore exactly four: the vertex count $V = 4$, the edge count $E = 6$, the face count $F = 4$, and the top-cell count 1. The top-cell count is constant across all closed 3-simplices and carries no further information; it is absorbed into the closure invariant. The remaining three cardinalities, plus the ambient direction count d established in Chapter 27, are the four primitive cardinality sources of the closed record.

Refinement-stability collapses (V, E, F) to (V, E, χ) via the alternating sum, as already established. The four refinement-stable cardinality sources of the closed record are therefore V, E, d, χ . Every extract — every reading of the closed record that respects the binary distinction’s lack of internal labels — factors through these four. There is no reading outside of them.

The binary distinction provides no internal labels. A reading that depends on a labelling reads structure the distinction did not supply. An extract is therefore a function on the cardinalities of the orbits of $\text{Aut}(K_4)$ on the closed record’s strata. The type below records the four cardinality sources; an extract is a function from this type into $\mathbb{N}$.

The verbal claim “every $\text{Aut}(K_4)$ -invariant numerical extract factors through the four slots” is thereby a type-level fact: an extract is, by the only definition the binary distinction admits, a function of V, E, d, χ .

-- § The four refinement-stable cardinality sources of the closed
-- K_4 record. Each field is the cardinality of a single $\text{Aut}(K_4)$ -orbit,


```

-- with the face count F absorbed into  $\chi$  by refinement-stability.
record CardinalitySources : Set where
  constructor sources
  field
    src-V :  $\mathbb{N}$  -- |vertex orbit|      : configurational
    src-E :  $\mathbb{N}$  -- |edge orbit|        : relational
    src-d :  $\mathbb{N}$  -- |independent axes|  : directional
    src- $\chi$  :  $\mathbb{N}$  -- alternating closure : topological
open CardinalitySources public

-- § An extract is a function from the cardinality sources into  $\mathbb{N}$ .
-- This is the only shape of reading admitted by a binary distinction
-- with no internal labels: it depends solely on orbit cardinalities,
-- not on choices of labelling within an orbit.
Extract : Set
Extract = CardinalitySources  $\rightarrow$   $\mathbb{N}$ 

-- § Witness that the four kind-tagged invariants are themselves
-- extracts: each is a projection from the cardinality sources, hence
-- trivially a function on them.
extract-V : Extract
extract-V = src-V

extract-E : Extract
extract-E = src-E

extract-d : Extract
extract-d = src-d

extract- $\chi$  : Extract
extract- $\chi$  = src- $\chi$ 

-- § Each kind selects exactly one of the four projections. Together
-- with kind-exhaustive, this is the type-level closure of the kind
-- classification: no kind selects two projections, and no projection
-- is selected by two kinds.
extract-by-kind : InvariantKind  $\rightarrow$  Extract
extract-by-kind configurational = extract-V
extract-by-kind relational      = extract-E
extract-by-kind directional     = extract-d
extract-by-kind topological     = extract- $\chi$ 

```

What This Forces about the Formula's Form

The classification above and the disjointness lemmas of the previous section together fix the form of the simplex evaluation. Three points are now removed from the list of choices, and a fourth — the exponents within each term — is fixed by minimality.

The exponentiation in V^d is fixed: configurational multiplied d times by itself counts the independent assignments of configurations across the d extension directions. Each factor is configurational; they accumulate multiplicatively. No additive

form $V \cdot d$ would respect the configurational kind, because d is directional, not a multiplier of configurations. Lower powers are also eliminated: a bulk built on V alone, or on V^k with $k < d$, would count configurations across only k of the d extension directions, silently distinguishing those k directions from the remaining ones. That distinction is forbidden by the isotropy result of §27. The exponent must therefore be exactly d .

The multiplication by χ is fixed: χ is topological, distinct from V but shared with the closed extension structure that V^d counts. Configurational and topological factors are kinds of the same underlying object and accumulate as a categorical product, which the forgetful functor sends to numerical multiplication. An additive $V^d + \chi$ would treat χ as a kind disjoint from V^d , which the bulk reading forbids: χ is the closure condition under which the configurations are counted, not a separate accountant.

The addition of d^2 is fixed: d^2 is directional, disjoint from both kinds of the bulk. The categorical combination between disjoint kinds is the coproduct, sent to numerical addition. A multiplicative $V^d \cdot \chi \cdot d^2$ would identify the directional kind with one of the bulk kinds, contradicting the disjointness lemmas.

The exponent of the boundary term is fixed by codimension, not by choice. The bulk $V^d \cdot \chi$ accounts for the closed d -dimensional interior. A boundary correction lives one codimension below: on the $(d - 1)$ -dimensional surface of the d -extension. The structural content of a codimension-1 surface inside a d -dimensional extension is intrinsically binary: each surface direction comes paired with the unique transverse direction across the surface. The boundary correction therefore records ordered pairs (along, transverse) of independent directions, and the count of such pairs is exactly $d \cdot d = d^2$. The exponent 2 is the binary along-transverse arity of a codimension-1 surface, not a parameter choice. A linear d would record only one of the two boundary directions and miss the transverse coupling; a cubic d^3 would record cross-direction structure of arity 3, which a codimension-1 boundary does not carry. Both are eliminated by the codimension structure, not by enumeration.

Every binary connector in the simplex evaluation must be the categorical operation respecting the kinds of its operands; the bulk exponent must be the isotropic one across all d extension directions; and the boundary exponent must be the along-transverse arity of a codimension-1 surface. Any binary form other than $V^d \cdot \chi + d^2$ that combines exactly these four invariants either identifies kinds the disjointness lemmas keep apart, fails to multiply factors of the shared bulk kind, breaks isotropy across the d directions of the bulk, or violates the binary along-transverse arity of the boundary. The first two failures are excluded by the lemmas of §38 and the bulk witness of §38; the third by the isotropy result of §27; the fourth by the codimension structure of the present section.

The simplex evaluation defined in the next section is therefore the unique combi-

nation of V , E , d , χ whose form respects the kind classification, the bulk–boundary split, and the codimension structure. The integer it produces is not the value of one definition among many; it is the value of the only structurally admissible combination of the four refinement-stable invariants of the closed record.

The structural argument above closes a uniqueness theorem inside the type theory. The argument is sharper than a parameter sweep over a fixed schema. A free term grammar over the four atoms admits any tree of syntactic powers, products, and sums. The shape predicate that carves out the canonical bulk-plus-boundary expression inside that grammar is itself read off existing book lemmas: the top-level coproduct is the disjoint-kind summation forced by `BulkBoundaryIndependence`; the boundary as a self-product is the binary along–transverse arity of a codimension-1 surface established in this section; the bulk as a power times a multiplier is the audit-fixed shape of an isotropic configurational extension weighted by its closure factor. The four atomic positions inside that shape are then constrained by the four kind obligations, each a singleton, and uniqueness of the surviving inhabitant reduces to `refl` after four rewrites.

Every candidate combination of the four atoms is an inhabitant of a free term grammar with syntactic power, product, and sum. The shape predicate that names the canonical bulk-plus-boundary form is derived from existing structural lemmas, not postulated as a record schema. The four atomic positions inside the resulting shape are then pinned by the kind obligations.

The structural equation of the shape predicate collapses every candidate tree to the bulk-plus-boundary form; the four singleton lemmas then collapse the four atomic positions to a single inhabitant. The canonical form $base^{exp} \cdot mult + bdy^2$ with $(base, exp, mult, bdy) = (V, d, \chi, d)$ is the unique inhabitant of the free term grammar that satisfies the shape predicate together with the four kind obligations. The proof terminates in `refl` after the four rewrites; no enumeration of candidate trees is performed.

```
data Atom : Set where
  AV AE Ad Aχ : Atom

atomKind : Atom → InvariantKind
atomKind AV = configurational
atomKind AE = relational
atomKind Ad = directional
atomKind Aχ = topological

atomValue : Atom → ℕ
atomValue AV = simplex-vertices
atomValue AE = simplex-edges
atomValue Ad = simplex-degree
atomValue Aχ = simplex-chi

-- § Free term grammar over the four atoms. Any tree of atoms,
```

```

-- syntactic powers, products, and sums is an Expr; the grammar
-- itself carves out no shape.
data Expr : Set where
  atom : Atom → Expr
  _ ^ _ : Expr → Expr → Expr
  _ ** _ : Expr → Expr → Expr
  _ ++ _ : Expr → Expr → Expr

infixl 6 _ ++ _
infixl 7 _ ** _
infixr 8 _ ^ _

-- § The forgetful map from Expr to ℕ sends syntactic operators to
-- natural arithmetic. By computation.
evalE : Expr → ℕ
evalE (atom a) = atomValue a
evalE (e ^ f) = evalE e ^ evalE f
evalE (e ** f) = evalE e * evalE f
evalE (e ++ f) = evalE e + evalE f

-- § Type-level closure: every Expr is built from one of the four
-- constructors. No fifth syntactic operator can inhabit Expr.
Expr-head-exhaustive :
  ∀ (e : Expr)
    → (Σ Atom λ a → e ≡ atom a)
    ⊔ (Σ Expr λ x → Σ Expr λ y → e ≡ x ^ y)
    ⊔ (Σ Expr λ x → Σ Expr λ y → e ≡ x ** y)
    ⊔ (Σ Expr λ x → Σ Expr λ y → e ≡ x ++ y)
Expr-head-exhaustive (atom a) = inj₁ (a, refl)
Expr-head-exhaustive (x ^ y) = inj₂ (inj₁ (x, (y, refl)))
Expr-head-exhaustive (x ** y) = inj₂ (inj₂ (inj₁ (x, (y, refl))))
Expr-head-exhaustive (x ++ y) = inj₂ (inj₂ (inj₂ (x, (y, refl))))

-- § The shape predicate. The structural equation is named, not
-- chosen: it is the bulk-plus-boundary form derived from
-- BulkBoundaryIndependence (top-level coproduct on disjoint kinds),
-- the codimension-1 along/transverse arity of this section
-- (boundary as self-product), and the audit-fixed isotropic shape
-- of the bulk (single power, single topological multiplier).
record HasCanonicalShape (e : Expr) : Set where
  field
    bulk-base      : Atom
    bulk-exp       : Atom
    bulk-mult      : Atom
    bdy-base       : Atom
    structure      :
      e ≡ ((atom bulk-base ^ atom bulk-exp) ** atom bulk-mult)
          ++ (atom bdy-base ** atom bdy-base)
    bulk-base-conf : atomKind bulk-base ≡ configurational
    bulk-exp-dir   : atomKind bulk-exp ≡ directional
    bulk-mult-top  : atomKind bulk-mult ≡ topological
    bdy-base-dir   : atomKind bdy-base ≡ directional

only-V-is-configurational : ∀ a → atomKind a ≡ configurational → a ≡ AV

```

```

only-V-is-configurational AV refl = refl
only-V-is-configurational AE ()
only-V-is-configurational Ad ()
only-V-is-configurational A $\chi$  ()

only-d-is-directional :  $\forall a \rightarrow \text{atomKind } a \equiv \text{directional} \rightarrow a \equiv \text{Ad}$ 
only-d-is-directional AV ()
only-d-is-directional AE ()
only-d-is-directional Ad refl = refl
only-d-is-directional A $\chi$  ()

only- $\chi$ -is-topological :  $\forall a \rightarrow \text{atomKind } a \equiv \text{topological} \rightarrow a \equiv \text{A}\chi$ 
only- $\chi$ -is-topological AV ()
only- $\chi$ -is-topological AE ()
only- $\chi$ -is-topological Ad ()
only- $\chi$ -is-topological A $\chi$  refl = refl

only-E-is-relational :  $\forall a \rightarrow \text{atomKind } a \equiv \text{relational} \rightarrow a \equiv \text{AE}$ 
only-E-is-relational AV ()
only-E-is-relational AE refl = refl
only-E-is-relational Ad ()
only-E-is-relational A $\chi$  ()

-- § The canonical Expr inhabitant:  $V^d \cdot \chi + d \cdot d$ .
canonicalExpr : Expr
canonicalExpr =
  ((atom AV ^^ atom Ad) ** atom A $\chi$ ) ++ $\epsilon$  (atom Ad ** atom Ad)

-- § Inner uniqueness: every shape-equation tree with constrained
-- atoms equals canonicalExpr. The four rewrites kill the four atomic
-- positions; the structural equation handles the tree.
shape-collapse :
 $\forall (b \text{ ex } m \text{ db} : \text{Atom})$ 
 $\rightarrow \text{atomKind } b \equiv \text{configurational}$ 
 $\rightarrow \text{atomKind } \text{ex} \equiv \text{directional}$ 
 $\rightarrow \text{atomKind } m \equiv \text{topological}$ 
 $\rightarrow \text{atomKind } \text{db} \equiv \text{directional}$ 
 $\rightarrow ((\text{atom } b ^^ \text{atom } \text{ex}) ** \text{atom } m) ++\epsilon (\text{atom } \text{db} ** \text{atom } \text{db})$ 
 $\equiv \text{canonicalExpr}$ 
shape-collapse b ex m db pb pe pm pd
rewrite only-V-is-configurational b pb
  | only-d-is-directional ex pe
  | only- $\chi$ -is-topological m pm
  | only-d-is-directional db pd
= refl

-- § Uniqueness inside the free grammar.
unique-canonical-expr :
 $\forall (e : \text{Expr}) \rightarrow \text{HasCanonicalShape } e \rightarrow e \equiv \text{canonicalExpr}$ 
unique-canonical-expr e
record { bulk-base = b
; bulk-exp = ex
; bulk-mult = m
; bdy-base = db
```

```

    ; structure      = eq
    ; bulk-base-conf = pb
    ; bulk-exp-dir   = pe
    ; bulk-mult-top  = pm
    ; bdy-base-dir   = pd
  }
= trans eq (shape-collapse b ex m db pb pe pm pd)

-- § The canonical Expr evaluates to  $V^d \cdot \chi + d^2$  by computation.
canonical-evaluates :
evalE canonicalExpr
  ≡ (simplex-vertices ^ simplex-degree) * simplex-chi
  + (simplex-degree * simplex-degree)
canonical-evaluates = refl

record FormUniquenessWitness : Set where
field
  canonical      : Expr
  canonical-is-V-d-chi : canonical ≡ canonicalExpr
  uniqueness :
    ∀ (e : Expr) → HasCanonicalShape e → e ≡ canonical
  evaluation :
    evalE canonical
    ≡ (simplex-vertices ^ simplex-degree) * simplex-chi
    + (simplex-degree * simplex-degree)

form-uniqueness-witness : FormUniquenessWitness
form-uniqueness-witness = record
{ canonical      = canonicalExpr
; canonical-is-V-d-chi = refl
; uniqueness     = unique-canonical-expr
; evaluation     = canonical-evaluates
}

```

Global classification of the term grammar

The local uniqueness theorem reads conditionally: if a tree satisfies the shape predicate, it equals the canonical inhabitant. A reader could still wonder whether the surrounding term grammar hides a third option — a tree that neither carries the shape predicate nor coincides with the canonical form, drifting in some unclassified middle. The free grammar leaves no such middle. Every tree of the four constructors lands in exactly one of two disjoint compartments: it carries the shape predicate and is then forced equal to the canonical form, or it does not carry the predicate at all. There is no third compartment, because the predicate is decidable and the equation it forces is unique.

Expr is generated by four constructors and Atom by four nullary constructors; both are finite syntactic types whose equality reduces to a finite decision tree. Decidable equality on Atom and Expr compares one constructor pair at a time. The classification

then case-splits the equality $e \equiv \text{canonicalExpr}$: the positive branch is its own witness, while the negative branch composes `unique-canonical-expr` with the inequality to refute the shape predicate.

No Expr escapes the dichotomy. The phrase “the canonical form is the unique admissible inhabitant” is now a single closed term, not a stitched conjunction of local eliminations.

```
-- § Decidable equality on the four atoms.
atom-decEq : (a b : Atom) → (a ≡ b) ⊔ (a ≠ b)
atom-decEq AV AV = inj1 refl
atom-decEq AE AE = inj1 refl
atom-decEq Ad Ad = inj1 refl
atom-decEq Aχ Aχ = inj1 refl
atom-decEq AV AE = inj2 (λ ())
atom-decEq AV Ad = inj2 (λ ())
atom-decEq AV Aχ = inj2 (λ ())
atom-decEq AE AV = inj2 (λ ())
atom-decEq AE Ad = inj2 (λ ())
atom-decEq AE Aχ = inj2 (λ ())
atom-decEq Ad AV = inj2 (λ ())
atom-decEq Ad AE = inj2 (λ ())
atom-decEq Ad Aχ = inj2 (λ ())
atom-decEq Aχ AV = inj2 (λ ())
atom-decEq Aχ AE = inj2 (λ ())
atom-decEq Aχ Ad = inj2 (λ ())

-- § Constructor injectivity for Expr.
atom-inj : ∀ {a b} → atom a ≡ atom b → a ≡ b
atom-inj refl = refl

^^-inj1 : ∀ {x y u v} → (x ^^ y) ≡ (u ^^ v) → x ≡ u
^^-inj1 refl = refl
^^-inj2 : ∀ {x y u v} → (x ^^ y) ≡ (u ^^ v) → y ≡ v
^^-inj2 refl = refl

**-inj1 : ∀ {x y u v} → (x ** y) ≡ (u ** v) → x ≡ u
**-inj1 refl = refl
**-inj2 : ∀ {x y u v} → (x ** y) ≡ (u ** v) → y ≡ v
**-inj2 refl = refl

++ε-inj1 : ∀ {x y u v} → (x ++ε y) ≡ (u ++ε v) → x ≡ u
++ε-inj1 refl = refl
++ε-inj2 : ∀ {x y u v} → (x ++ε y) ≡ (u ++ε v) → y ≡ v
++ε-inj2 refl = refl

-- § Decidable equality on the free term grammar.
expr-decEq : (e f : Expr) → (e ≡ f) ⊔ (e ≠ f)
expr-decEq (atom a) (atom b) with atom-decEq a b
... | inj1 refl = inj1 refl
... | inj2 ¬p = inj2 (λ q → ¬p (atom-inj q))
expr-decEq (atom _) ( _ ^^ _ ) = inj2 (λ ())
expr-decEq (atom _) ( _ ** _ ) = inj2 (λ ())
```

```

expr-decEq (atom _) ( _ ++ε _ ) = inj2 (λ ())
expr-decEq ( _ ^^ _ ) (atom _) = inj2 (λ ())
expr-decEq ( _ ** _ ) (atom _) = inj2 (λ ())
expr-decEq ( _ ++ε _ ) (atom _) = inj2 (λ ())
expr-decEq ( _ ^^ _ ) ( _ ** _ ) = inj2 (λ ())
expr-decEq ( _ ^^ _ ) ( _ ++ε _ ) = inj2 (λ ())
expr-decEq ( _ ** _ ) ( _ ^^ _ ) = inj2 (λ ())
expr-decEq ( _ ** _ ) ( _ ++ε _ ) = inj2 (λ ())
expr-decEq ( _ ++ε _ ) ( _ ^^ _ ) = inj2 (λ ())
expr-decEq ( _ ++ε _ ) ( _ ** _ ) = inj2 (λ ())
expr-decEq (x ^^ y) (u ^^ v) with expr-decEq x u | expr-decEq y v
... | inj1 refl | inj1 refl = inj1 refl
... | inj2 ¬p | _ = inj2 (λ q → ¬p (^^-inj1 q))
... | _ | inj2 ¬p = inj2 (λ q → ¬p (^^-inj2 q))
expr-decEq (x ** y) (u ** v) with expr-decEq x u | expr-decEq y v
... | inj1 refl | inj1 refl = inj1 refl
... | inj2 ¬p | _ = inj2 (λ q → ¬p (**-inj1 q))
... | _ | inj2 ¬p = inj2 (λ q → ¬p (**-inj2 q))
expr-decEq (x ++ε y) (u ++ε v) with expr-decEq x u | expr-decEq y v
... | inj1 refl | inj1 refl = inj1 refl
... | inj2 ¬p | _ = inj2 (λ q → ¬p (++ε-inj1 q))
... | _ | inj2 ¬p = inj2 (λ q → ¬p (++ε-inj2 q))

-- § Global classification: every Expr either fails the shape
-- predicate or coincides with the canonical inhabitant. No third
-- compartment exists.
Expr-classification :
  ∀ (e : Expr) → (¬ HasCanonicalShape e) ⊔ (e ≡ canonicalExpr)
Expr-classification e with expr-decEq e canonicalExpr
... | inj1 p = inj2 p
... | inj2 ¬p = inj1 (λ hcs → ¬p (unique-canonical-expr e hcs))

```

The signature is forced, not chosen

A reader could still ask whether the three operators of the Expr grammar — summation, product, exponentiation — are themselves a design choice. They are not. They are the three rungs of the iteration chain that $\mathbb{N}$ fixes by its successor structure: multiplication is repeated addition, exponentiation is repeated multiplication. Each rung is the recursive iteration of the rung below it. No alternative operator can be inserted into this chain, because each rung is defined from the previous one by the same primitive recursion on $\mathbb{N}$.

The chain does not extend to a fourth rung in the canonical form, because the canonical exponent is a flat atom, not a tower. There is no $^^$ -node above the exponent position; hence no tetration height to fix and no fourth operator to introduce. The signature is closed at rank three by the recursion of $\mathbb{N}$ above and by the atomic shape of the canonical exponent below.

The absence of a tower-height parameter is itself a theorem, not an observation. The orbit record pins the exponent value to the flat $\mathbb{N}$ -quantity simplex-degree; any

Expr inserted into the exponent slot must evaluate to that same $\mathbb{N}$. Two Exprs with identical evalE-value are observationally indistinguishable for every numerical question evalE can ask. Any $^{\wedge}$ -tower in the exponent slot therefore collapses, under evalE, to the atomic exponent atom Ad: a tower-height parameter is a degree of freedom that no theorem about canonicalExpr can detect, hence not a parameter at all. The record NoTowerParameter bundles this collapse.

Every operator in the grammar must be either the primitive $\mathbb{N}$ -operation suc or its iteration. Anything outside that chain is unreachable from the successor structure alone. Two computational equalities, both refl, exhibit the chain: prod-is-iter-sum reads multiplication as repeated addition, and pow-is-iter-prod reads exponentiation as repeated multiplication. A third witness, canonical-bulk-exp-is-atom, exhibits the absence of a fourth rung in the canonical form: the bulk exponent slot is an atom, not a nested power.

The operator signature $\{+, \varepsilon, **, ^{\wedge}\}$ is not three independent operators glued together by taste. It is the iteration ladder of $\mathbb{N}$ truncated at the height that the canonical form actually uses. SignatureClosure bundles the three witnesses.

```
-- § Multiplication on  $\mathbb{N}$  is iterated addition (refl).
prod-is-iter-sum :  $\forall n m \rightarrow \text{succ } m * n \equiv n + (m * n)$ 
prod-is-iter-sum n m = refl

-- § Exponentiation on  $\mathbb{N}$  is iterated multiplication (refl).
pow-is-iter-prod :  $\forall x n \rightarrow x ^{\text{succ } n} \equiv x * (x ^n)$ 
pow-is-iter-prod x n = refl

-- § The bulk exponent slot of canonicalExpr is a single atom.
canonical-bulk-exp-is-atom :
 $\Sigma \text{Atom } \lambda a \rightarrow$ 
  canonicalExpr
 $\equiv ((\text{atom } AV ^{\wedge} \text{atom } a) ** \text{atom } AX) ++ \epsilon (\text{atom } Ad ** \text{atom } Ad)$ 
canonical-bulk-exp-is-atom = Ad , refl

-- § The exponent atom evaluates to simplex-degree.
atom-Ad-is-simplex-degree : evalE (atom Ad)  $\equiv$  simplex-degree
atom-Ad-is-simplex-degree = refl

-- § Any Expr pinned to simplex-degree collapses, under evalE, to atom Ad.
exponent-slot-collapses-to-atom :
 $\forall (e : \text{Expr}) \rightarrow \text{evalE } e \equiv \text{simplex-degree}$ 
 $\rightarrow \text{evalE } e \equiv \text{evalE } (\text{atom } Ad)$ 
exponent-slot-collapses-to-atom e p = p

-- § No-tower-parameter witness.
record NoTowerParameter : Set where
  field
    pin          : evalE (atom Ad)  $\equiv$  simplex-degree
    tower-collapses :
 $\forall (e : \text{Expr}) \rightarrow \text{evalE } e \equiv \text{simplex-degree}$ 
 $\rightarrow \text{evalE } e \equiv \text{evalE } (\text{atom } Ad)$ 
```

```

canonical-witness :
   $\Sigma$  Atom  $\lambda a \rightarrow$ 
    canonicalExpr
     $\equiv ((\text{atom } AV \wedge \wedge \text{atom } a) ** \text{atom } A\chi) ++ \epsilon (\text{atom } Ad ** \text{atom } Ad)$ 

no-tower-parameter : NoTowerParameter
no-tower-parameter = record
{ pin          = atom-Ad-is-simplex-degree
; tower-collapses = exponent-slot-collapses-to-atom
; canonical-witness = canonical-bulk-exp-is-atom
}

-- § Signature closure: the three Expr operators plus the no-tower fact.
record SignatureClosure : Set where
field
  sum-is-successor :  $\forall n \rightarrow \text{zero} + n \equiv n$ 
  prod-from-sum    :  $\forall n m \rightarrow \text{suc } m * n \equiv n + (m * n)$ 
  pow-from-prod    :  $\forall x n \rightarrow x \wedge \text{suc } n \equiv x * (x \wedge n)$ 
  no-tetration-slot : NoTowerParameter

signature-closure : SignatureClosure
signature-closure = record
{ sum-is-successor =  $\lambda _ \rightarrow \text{refl}$ 
; prod-from-sum    = prod-is-iter-sum
; pow-from-prod    = pow-is-iter-prod
; no-tetration-slot = no-tower-parameter
}

```

Audit of Eliminated Freedoms

The simplex evaluation carries no free parameters. This audit names each degree of freedom that a generic four-invariant expression would carry, and the structural fact that eliminates it. No item below is closed by enumeration; each is closed by a single structural witness.

1. **Choice of which invariants enter.** Eliminated by §38 together with §38: an extract is forced by the binary distinction to be a function on the cardinality sources of $\text{Aut}(K_4)$ -orbits, and those sources are exhausted by V, E, d, χ . No fifth cardinality source exists, and no reading outside the four factors through the binary distinction at all.
2. **Choice of arithmetic operation between two invariants** (sum vs. product). Eliminated by §38 together with the forgetful map of §38: shared kind admits the categorical product $_ \otimes _$, whose forgetful reduction is multiplication; disjoint kinds admit the categorical coproduct $_ \oplus [_] _$, whose forgetful reduction is addition. Both reductions hold by `refl`.

3. **Choice of which factors are bulk and which are boundary.** Eliminated by the bulk–boundary witness of §38: configurational and topological share the closed extension as their carrier and accumulate inside the bulk; directional is disjoint from both and forms the boundary.
4. **Choice of bulk exponent.** Eliminated by the isotropy result of §27: the bulk must count configurations across all d extension directions symmetrically. Any exponent other than d would distinguish a subset of the directions from the rest.
5. **Choice of boundary exponent.** Eliminated by the codimension structure of the previous section: a codimension-1 surface of a d -dimensional extension carries the binary along–transverse pair of directions. The exponent 2 is the arity of this pair, not a parameter.
6. **Choice of explicit role for the relational kind.** Eliminated by the alternating sum $\chi = V - E + F$: the relational invariant E enters the evaluation through χ , not as an independent factor. An additional explicit E -term would either duplicate the contribution already absorbed into the closure invariant or introduce a non-refinement-stable reading of the closure.

The simplex evaluation carries six structural choices, each tied to a degree of freedom that a generic combination of four invariants admits. Every one of the six choices is fixed by a single structural witness named above. None remains free, and none is closed by enumeration.

Within the displayed expression grammar and its obligations, the form $V^d \cdot \chi + d^2$ is the unique combination of the four refinement-stable invariants of the closed K_4 record that respects every structural fact carried by the record. The integer it produces on K_4 is therefore not the value of one candidate among that grammar. It is the value of the only structurally admissible candidate there.

Each of the six structural witnesses listed above is mechanically discharged by a record built from existing book lemmas: the self-dual closure record of §38 and the form-uniqueness record at the close of §38 discharge the closure and uniqueness items; the boundary-arity record of §39 discharges the boundary item; the orbit theorems of Chapter 62 discharge the Aut-related items in Chapter 64; and the simplex evaluation of the next section produces the value 137. The six items together inhabit the record `AuditClosure` at the close of Chapter 64; each field of `audit-closure` is the named witness for the audit item carrying the same number.

The arithmetic that carries out the evaluation is now ready. The following chapter introduces the rational numbers used by the wider measurement framework, and the simplex evaluation itself.

Chapter 39

Rational Numbers and Measurement

Rational numbers are represented here as integer numerators over positive denominators. Equality, order, and distance are then defined by the familiar cross-multiplication formulas, expressed entirely inside the arithmetic already developed for $\mathbb{Z}$ and $\mathbb{N}^+$.

Rational Arithmetic

The basic rational operations are defined in the usual way by clearing denominators. Because denominators live in $\mathbb{N}^+$, no additional side conditions are needed to keep the constructions well formed.

```
-- § Rational number: integer numerator over positive denominator.
record  $\mathbb{Q}$  : Set where
  constructor _/_
  field
    num :  $\mathbb{Z}$ 
    den :  $\mathbb{N}^+$ 

open  $\mathbb{Q}$  public

infix 4 _ $\simeq_{\mathbb{Q}}$ _

-- § Setoid equality: cross-multiplication.
_ $\simeq_{\mathbb{Q}}$ _ :  $\mathbb{Q} \rightarrow \mathbb{Q} \rightarrow \text{Set}$ 
( $a / b$ )  $\simeq_{\mathbb{Q}}$  ( $c / d$ ) = ( $a * \mathbb{Z}^+ \text{to} \mathbb{Z} d$ )  $\equiv$  ( $c * \mathbb{Z}^+ \text{to} \mathbb{Z} b$ )

infixl 6 _+ $\mathbb{Q}$ _ _- $\mathbb{Q}$ _
infixl 7 _* $\mathbb{Q}$ _

-- § Rational addition.
```

```

_+Q_ : Q → Q → Q
(a / b) +Q (c / d) = ((a *Z +toZ d) +Z (c *Z +toZ b)) / (b *+ d)

-- § Rational multiplication.
_*Q_ : Q → Q → Q
(a / b) *Q (c / d) = (a *Z c) / (b *+ d)

-- § Rational negation.
_-Q_ : Q → Q
-Q (a / b) = negZ a / b

-- § Rational subtraction.
_--Q_ : Q → Q → Q
p -Q q = p +Q (-Q q)

-- § Distinguished rationals.
0Q 1Q : Q
0Q = 0Z / one+
1Q = oneZ / one+

infix 4 _≤Q_ <Q_

-- § Rational order: cross-multiply and compare integers.
_≤Q_ : Q → Q → Set
(a / b) ≤Q (c / d) = (a *Z +toZ d) ≤Z (c *Z +toZ b)

-- § Strict rational order.
_<Q_ : Q → Q → Set
(a / b) <Q (c / d) = (a *Z +toZ d) <Z (c *Z +toZ b)

-- § Rational distance.
distQ : Q → Q → Q
distQ (a / b) (c / d) = absZ ((a *Z +toZ d) +Z negZ (c *Z +toZ b)) / (b *+ d)

```

Simplex Evaluation: $\mathcal{C} = 137$

We now return to the distinguished value

$$\mathcal{C} = V^d \cdot \chi + d^2 = 137.$$

The point of this section is not merely to recompute it, but to show that it remains singled out when compared with nearby enumerated families of formulas built from the same invariant stock.

Using the previously established values of V , d , and χ , the defining expression reduces by direct computation to 137. The records that follow compare this canonical value with alternative pair-sum, power-formula, and three-term constructions.

```

-- § Simplex evaluation from K4 invariants.
simplex-eval : N
simplex-eval = (simplex-vertices ^ simplex-degree) * simplex-chi + (simplex-degree * simplex-degree)

```

```

-- § Law 15A.0: the derived integer is exactly 137.
law15A-0-simplex-eval-137 : simplex-eval  $\equiv$  137
law15A-0-simplex-eval-137 = refl

-- § Redundant witness (same computation, explicit name).
derived-integer :  $\mathbb{N}$ 
derived-integer =
  (simplex-vertices ^ simplex-degree) * simplex-chi
  + (simplex-degree * simplex-degree)

law15A-0-derived-integer-137 : derived-integer  $\equiv$  137
law15A-0-derived-integer-137 = refl

```

Candidate elimination within enumerated families

The following records make that comparison explicit. Several natural rival families are enumerated from the same invariant stock, and direct computation shows that none of them displace the canonical formula.

Pair-product sums. The first family consists of pairwise sums of basic quadratic products of the K_4 invariants.

```

-- § Products of pairs of  $K_4$  invariants
p-VV p-VE p-Vd p-V $\chi$  p-EE p-Ed p-E $\chi$  p-dd p-d $\chi$  p- $\chi\chi$  :  $\mathbb{N}$ 
p-VV = simplex-vertices * simplex-vertices -- 16
p-VE = simplex-vertices * simplex-edges -- 24
p-Vd = simplex-vertices * simplex-degree -- 12
p-V $\chi$  = simplex-vertices * simplex-chi -- 8
p-EE = simplex-edges * simplex-edges -- 36
p-Ed = simplex-edges * simplex-degree -- 18
p-E $\chi$  = simplex-edges * simplex-chi -- 12
p-dd = simplex-degree * simplex-degree -- 9
p-d $\chi$  = simplex-degree * simplex-chi -- 6
p- $\chi\chi$  = simplex-chi * simplex-chi -- 4

-- § No pair-sum of  $K_4$  products gives 137.
record NoPairSumGives137 : Set where
  field
    t-VV+VE :  $\neg$  (p-VV + p-VE  $\equiv$  137) -- 40
    t-VV+EE :  $\neg$  (p-VV + p-EE  $\equiv$  137) -- 52
    t-VV+Ed :  $\neg$  (p-VV + p-Ed  $\equiv$  137) -- 34
    t-VE+EE :  $\neg$  (p-VE + p-EE  $\equiv$  137) -- 60
    t-VE+Ed :  $\neg$  (p-VE + p-Ed  $\equiv$  137) -- 42
    t-VE+dd :  $\neg$  (p-VE + p-dd  $\equiv$  137) -- 33
    t-EE+dd :  $\neg$  (p-EE + p-dd  $\equiv$  137) -- 45
    t-EE+Vd :  $\neg$  (p-EE + p-Vd  $\equiv$  137) -- 48
    t-EE+V $\chi$  :  $\neg$  (p-EE + p-V $\chi$   $\equiv$  137) -- 44
    t-Ed+Vd :  $\neg$  (p-Ed + p-Vd  $\equiv$  137) -- 30
    t-VV+dd :  $\neg$  (p-VV + p-dd  $\equiv$  137) -- 25
    t-VV+E $\chi$  :  $\neg$  (p-VV + p-E $\chi$   $\equiv$  137) -- 28

```

```

law15A-1-no-pair-sum-137 : NoPairSumGives137
law15A-1-no-pair-sum-137 = record
{ t-VV+VE =  $\lambda$  (); t-VV+EE =  $\lambda$  (); t-VV+Ed =  $\lambda$  ()
; t-VE+EE =  $\lambda$  (); t-VE+Ed =  $\lambda$  (); t-VE+dd =  $\lambda$  ()
; t-EE+dd =  $\lambda$  (); t-EE+Vd =  $\lambda$  (); t-EE+V $\chi$  =  $\lambda$  ()
; t-Ed+Vd =  $\lambda$  (); t-VV+dd =  $\lambda$  (); t-VV+E $\chi$  =  $\lambda$  ()
}

```

Power formulas. The second family tests formulas built from powers X^Y , together with simple affine modifications of those powers.

```

-- § All power combinations  $X^Y$  with  $X, Y \in \{V, E, d, \chi\}$ .
pow-Vd pow-V $\chi$  pow-VE pow-dV pow-dE pow-d $\chi$  pow- $\chi$ V pow- $\chi$ E pow-Ed pow-EV pow-E $\chi$  :  $\mathbb{N}$ 
pow-Vd = simplex-vertices ^ simplex-degree --  $4^3 = 64$ 
pow-V $\chi$  = simplex-vertices ^ simplex-chi --  $4^2 = 16$ 
pow-VE = simplex-vertices ^ simplex-edges --  $4^6 = 4096$ 
pow-dV = simplex-degree ^ simplex-vertices --  $3^4 = 81$ 
pow-dE = simplex-degree ^ simplex-edges --  $3^6 = 729$ 
pow-d $\chi$  = simplex-degree ^ simplex-chi --  $3^2 = 9$ 
pow- $\chi$ V = simplex-chi ^ simplex-vertices --  $2^4 = 16$ 
pow- $\chi$ E = simplex-chi ^ simplex-edges --  $2^6 = 64$ 
pow-Ed = simplex-edges ^ simplex-degree --  $6^3 = 216$ 
pow-EV = simplex-edges ^ simplex-vertices --  $6^4 = 1296$ 
pow-E $\chi$  = simplex-edges ^ simplex-chi --  $6^2 = 36$ 

-- § Exhaustive power-formula tests.
record AllPowerFormulaTests : Set where
field
-- §  $X^Y \cdot \chi + d^2$  for various  $X, Y$  (our formula uses  $V^d \cdot \chi + d^2 = 137$ )
alt-dV- $\chi$ -dd :  $\neg (\text{pow-dV} * \text{simplex-chi} + \text{p-dd} \equiv 137)$  --  $81 \cdot 2 + 9 = 171$ 
alt-V $\chi$ - $\chi$ -dd :  $\neg (\text{pow-V}\chi * \text{simplex-chi} + \text{p-dd} \equiv 137)$  --  $16 \cdot 2 + 9 = 41$ 
alt-d $\chi$ - $\chi$ -dd :  $\neg (\text{pow-d}\chi * \text{simplex-chi} + \text{p-dd} \equiv 137)$  --  $9 \cdot 2 + 9 = 27$ 
alt-Ed- $\chi$ -dd :  $\neg (\text{pow-Ed} * \text{simplex-chi} + \text{p-dd} \equiv 137)$  --  $216 \cdot 2 + 9 = 441$ 
alt-E $\chi$ - $\chi$ -dd :  $\neg (\text{pow-E}\chi * \text{simplex-chi} + \text{p-dd} \equiv 137)$  --  $36 \cdot 2 + 9 = 81$ 

-- § The structural identity:  $\chi^E = V^d$  (not an alternative)
alt- $\chi$ E-is-Vd :  $\text{pow-}\chi\text{E} \equiv \text{pow-Vd}$  --  $2^6 = 4^3$ 
alt- $\chi$ E-same :  $\text{pow-}\chi\text{E} * \text{simplex-chi} + \text{p-dd} \equiv 137$  -- confirms identity

-- §  $X^Y + Z$  for various  $Z$  (no  $\chi$  multiplier)
alt-Vd-dd :  $\neg (\text{pow-Vd} + \text{p-dd} \equiv 137)$  --  $64 + 9 = 73$ 
alt-dV-dd :  $\neg (\text{pow-dV} + \text{p-dd} \equiv 137)$  --  $81 + 9 = 90$ 
alt-dV-EE :  $\neg (\text{pow-dV} + \text{p-EE} \equiv 137)$  --  $81 + 36 = 117$ 
alt-dV-VE :  $\neg (\text{pow-dV} + \text{p-VE} \equiv 137)$  --  $81 + 24 = 105$ 

-- § The winner
the-formula :  $\text{pow-Vd} * \text{simplex-chi} + \text{p-dd} \equiv 137$ 

law15A-2-all-power-formulas : AllPowerFormulaTests
law15A-2-all-power-formulas = record
{ alt-dV- $\chi$ -dd =  $\lambda$  ()

```



```

; alt-V $\chi$ - $\chi$ -dd =  $\lambda$  ()
; alt-d $\chi$ - $\chi$ -dd =  $\lambda$  ()
; alt-Ed- $\chi$ -dd =  $\lambda$  ()
; alt-E $\chi$ - $\chi$ -dd =  $\lambda$  ()
; alt- $\chi$ E-is-Vd = refl
; alt- $\chi$ E-same = refl
; alt-Vd-dd =  $\lambda$  ()
; alt-dV-dd =  $\lambda$  ()
; alt-dV-EE =  $\lambda$  ()
; alt-dV-VE =  $\lambda$  ()
; the-formula = refl
}

```

Verdict. Taken together, these checks show that the canonical two-term expression remains distinguished within the enumerated candidate families.

```

-- § Law 15A.3: 137 survives the enumerated candidate families.
record SimplexEval137-Exclusivity : Set where
  field
    no-pair-sums : NoPairSumGives137
    power-formulas : AllPowerFormulaTests
    only-K4 : simplex-eval  $\equiv$  137

law15A-3-simplexEval137-exclusivity : SimplexEval137-Exclusivity
law15A-3-simplexEval137-exclusivity = record
{ no-pair-sums = law15A-1-no-pair-sum-137
; power-formulas = law15A-2-all-power-formulas
; only-K4 = refl
}

```

The boundary term as a mechanical product. Inside the simplex evaluation the boundary correction is the self-product p -dd. Mechanically: $d = 3$ by law14C-1-degree, $d^2 = 9$ by computation, and `simplex-eval` decomposes as $V^d \cdot \chi + d \cdot d$ by `refl`. The codimension reading of d^2 as the binary along-transverse arity of a codimension-1 surface (made structurally in §38) is therefore the value of p -dd inside `simplex-eval`, fixed by computation.

The boundary term must be a directional self-product whose value coincides with the boundary contribution inside `simplex-eval`. The record below collects $d = 3$, $d^2 = 9$, and the position of d^2 inside `simplex-eval`. Each field reduces by `refl`. The codimension reading of d^2 thereby becomes a `refl`-equality inside the displayed evaluation.

```

record BoundaryArityWitness : Set where
  field
    d-equals-3 : simplex-degree  $\equiv$  3
    boundary-equals-9 : p-dd  $\equiv$  9
    boundary-in-eval :

```

```

simplex-eval ≡ (simplex-vertices ^ simplex-degree) * simplex-chi + p-dd

boundary-arity-witness : BoundaryArityWitness
boundary-arity-witness = record
{ d-equals-3          = law14C-1-degree
; boundary-equals-9 = refl
; boundary-in-eval = refl
}

```

Three-term decompositions. One can also ask how 137 appears in three-term decompositions. The next record classifies the most relevant examples and separates the genuinely canonical one from merely accidental decompositions.

1. The decomposition $\chi + d^3 + E^2d = 2 + 27 + 108$ reaches 137, but it does so by isolating a cubic bulk term of 135. In other words, the identity exists, yet it no longer reflects the compact invariant pattern that singled out the canonical two-term expression.
2. The decomposition $d^2 + V^2\chi + V^2E = 9 + 32 + 96$ also totals 137, but here the last two summands collapse to $V^d \cdot \chi = 128$. This is therefore not a rival formula; it is the canonical expression written in expanded form.
3. The decomposition $1 + V^d + VEd = 1 + 64 + 72$ reaches 137 only by adjoining the external constant 1. Since 1 is not among the basic K_4 invariants, this representation records an arithmetic coincidence rather than an intrinsic invariant identity.

The accompanying discussion explains how these decompositions relate to the canonical formula and where they cease to reflect intrinsic K_4 structure.

```

-- § Three-term decomposition classification.

-- § Triple 1:  $\chi + d^3 + E^2d = 2 + 27 + 108 = 137$ .
triple-1-sum : simplex-chi + (simplex-degree * simplex-degree * simplex-degree)
              + (simplex-edges * simplex-edges * simplex-degree) ≡ 137
triple-1-sum = refl

-- § The two cubic terms sum to 135, which carries prime factor 5.
triple-1-cubic-part : (simplex-degree * simplex-degree * simplex-degree)
                    + (simplex-edges * simplex-edges * simplex-degree) ≡ 135
triple-1-cubic-part = refl

-- §  $45 = d^2 + E^2$  is not 2,3-smooth: it has prime factor 5.
triple-1-factor-45 : (simplex-degree * simplex-degree + simplex-edges * simplex-edges) ≡ 45
triple-1-factor-45 = refl

-- § Triple 2:  $d^2 + V^2\chi + V^2E = 9 + 32 + 96 = 137$ .
triple-2-sum : p-dd + (p-VV * simplex-chi) + (p-VV * simplex-edges) ≡ 137
triple-2-sum = refl

```

```

-- § Triple 2 collapses:  $V^2\chi + V^2E = V^d \cdot \chi$  (the canonical bulk term).
triple-2-collapse : (p-VV * simplex-chi) + (p-VV * simplex-edges)  $\equiv$  pow-Vd * simplex-chi
triple-2-collapse = refl -- 32 + 96 = 128 = 64 · 2

-- § Triple 2 is the canonical formula  $V^d \cdot \chi + d^2$  in disguise.
triple-2-is-canonical : p-dd + ((p-VV * simplex-chi) + (p-VV * simplex-edges))  $\equiv$  simplex-eval
triple-2-is-canonical = refl

-- § Triple 3:  $1 + V^d + VEd = 1 + 64 + 72 = 137$ .
triple-3-sum : 1 + pow-Vd + (simplex-vertices * simplex-edges * simplex-degree)  $\equiv$  137
triple-3-sum = refl

-- § The summand 1 is not a  $K_4$  invariant:  $1 \notin \{V, E, d, \chi\} = \{4, 6, 3, 2\}$ .
identity-not-V :  $\neg (1 \equiv \text{simplex-vertices})$ 
identity-not-V =  $\lambda ()$ 
identity-not-E :  $\neg (1 \equiv \text{simplex-edges})$ 
identity-not-E =  $\lambda ()$ 
identity-not-d :  $\neg (1 \equiv \text{simplex-degree})$ 
identity-not-d =  $\lambda ()$ 
identity-not- $\chi$  :  $\neg (1 \equiv \text{simplex-chi})$ 
identity-not- $\chi$  =  $\lambda ()$ 

-- § Packaging: three-term classification.
record ThreeTermClassification : Set where
  field
    t1-exists      : simplex-chi + (simplex-degree * simplex-degree * simplex-degree)
                  + (simplex-edges * simplex-edges * simplex-degree)  $\equiv$  137
    t2-exists      : p-dd + (p-VV * simplex-chi) + (p-VV * simplex-edges)  $\equiv$  137
    t3-exists      : 1 + pow-Vd + (simplex-vertices * simplex-edges * simplex-degree)  $\equiv$  137
    t2-is-canonical : (p-VV * simplex-chi) + (p-VV * simplex-edges)  $\equiv$  pow-Vd * simplex-chi
    t1-escapes      : (simplex-degree * simplex-degree + simplex-edges * simplex-edges)  $\equiv$  45
    t3-identity-out :  $\neg (1 \equiv \text{simplex-chi})$ 

law15A-4-three-term-classification : ThreeTermClassification
law15A-4-three-term-classification = record
{ t1-exists      = refl
; t2-exists      = refl
; t3-exists      = refl
; t2-is-canonical = refl
; t1-escapes      = refl
; t3-identity-out =  $\lambda ()$ 
}

```

Unified decomposition record. The preceding calculations are then assembled into a single record summarizing both the exclusivity results and the three-term classification.

```

-- § Unified decomposition record: the two-term formula survives every enumerated rival.
record SimplexEval137-Forced : Set where
  field
    exclusivity : SimplexEval137-Exclusivity

```

```

three-term-class : ThreeTermClassification

law15A-5-simplexEval137-forced : SimplexEval137-Forced
law15A-5-simplexEval137-forced = record
{ exclusivity = law15A-3-simplexEval137-exclusivity
; three-term-class = law15A-4-three-term-classification
}

```

Exhaustive degree-bounded pair-sum closure

The previous rival families were selected because they are the most natural visible alternatives. One can push the comparison further by checking all atomic monomials in V , E , d , and χ of total degree at most 3, together with all ordered pair-sums built from those values. This is a finite 35×35 grid. The computation below counts the ordered pairs that sum to 137 and obtains zero, so the entire degree-bounded search space misses the value. In that precise sense, the canonical formula first enters at degree 4.

```

-- § Exhaustive degree-≤3 pair-sum closure for 137.

private
-- § Indicator: 1 if  $m \equiv n$ , 0 otherwise. Uses BUILTIN  $\dot{-}$ .
isEqN :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
isEqN m n = 1  $\dot{-}$  ((m  $\dot{-}$  n) + (n  $\dot{-}$  m))

-- § One-side hit count against the 35 monomial values.
hits137-against :  $\mathbb{N} \rightarrow \mathbb{N}$ 
hits137-against v =
  isEqN (v + 1) 137 + isEqN (v + 4) 137 + isEqN (v + 6) 137
+ isEqN (v + 3) 137 + isEqN (v + 2) 137
+ isEqN (v + 16) 137 + isEqN (v + 24) 137 + isEqN (v + 12) 137
+ isEqN (v + 8) 137 + isEqN (v + 36) 137 + isEqN (v + 18) 137
+ isEqN (v + 12) 137 + isEqN (v + 9) 137 + isEqN (v + 6) 137
+ isEqN (v + 4) 137
+ isEqN (v + 64) 137 + isEqN (v + 96) 137 + isEqN (v + 48) 137
+ isEqN (v + 32) 137 + isEqN (v + 144) 137 + isEqN (v + 72) 137
+ isEqN (v + 48) 137 + isEqN (v + 36) 137 + isEqN (v + 24) 137
+ isEqN (v + 16) 137 + isEqN (v + 216) 137 + isEqN (v + 108) 137
+ isEqN (v + 72) 137 + isEqN (v + 54) 137 + isEqN (v + 36) 137
+ isEqN (v + 24) 137 + isEqN (v + 27) 137 + isEqN (v + 18) 137
+ isEqN (v + 12) 137 + isEqN (v + 8) 137

-- § Total pair-sum hits: sum hits137-against over all 35 v.
totalPairHits137 :  $\mathbb{N}$ 
totalPairHits137 =
  hits137-against 1 + hits137-against 4 + hits137-against 6
+ hits137-against 3 + hits137-against 2
+ hits137-against 16 + hits137-against 24 + hits137-against 12
+ hits137-against 8 + hits137-against 36 + hits137-against 18
+ hits137-against 12 + hits137-against 9 + hits137-against 6
+ hits137-against 4

```

```

+ hits137-against 64 + hits137-against 96 + hits137-against 48
+ hits137-against 32 + hits137-against 144 + hits137-against 72
+ hits137-against 48 + hits137-against 36 + hits137-against 24
+ hits137-against 16 + hits137-against 216 + hits137-against 108
+ hits137-against 72 + hits137-against 54 + hits137-against 36
+ hits137-against 24 + hits137-against 27 + hits137-against 18
+ hits137-against 12 + hits137-against 8

-- § No degree-≤3 ordered pair-sum reaches 137.
theorem-no-pair-sum-degree-3 : totalPairHits137 ≡ 0
theorem-no-pair-sum-degree-3 = refl

-- § Single-monomial closure: 137 itself is not a degree-≤3 monomial value.
private
hits137-single : ℕ
hits137-single =
  isEqℕ 1 137 + isEqℕ 4 137 + isEqℕ 6 137 + isEqℕ 3 137
+ isEqℕ 2 137
+ isEqℕ 16 137 + isEqℕ 24 137 + isEqℕ 12 137 + isEqℕ 8 137
+ isEqℕ 36 137 + isEqℕ 18 137 + isEqℕ 12 137 + isEqℕ 9 137
+ isEqℕ 6 137 + isEqℕ 4 137
+ isEqℕ 64 137 + isEqℕ 96 137 + isEqℕ 48 137 + isEqℕ 32 137
+ isEqℕ 144 137 + isEqℕ 72 137 + isEqℕ 48 137 + isEqℕ 36 137
+ isEqℕ 24 137 + isEqℕ 16 137 + isEqℕ 216 137 + isEqℕ 108 137
+ isEqℕ 72 137 + isEqℕ 54 137 + isEqℕ 36 137 + isEqℕ 24 137
+ isEqℕ 27 137 + isEqℕ 18 137 + isEqℕ 12 137 + isEqℕ 8 137

theorem-137-not-single-mono : hits137-single ≡ 0
theorem-137-not-single-mono = refl

-- § Closure record: degree-≤3 pair-sums and singletons both miss 137.
record DegreeBoundedExhaustion : Set where
  field
    no-single-mono : hits137-single ≡ 0
    no-pair-sum : totalPairHits137 ≡ 0
    canonical-deg4 : pow-Vd * simplex-chi + p-dd ≡ 137

theorem-degree-bounded-exhaustion : DegreeBoundedExhaustion
theorem-degree-bounded-exhaustion = record
{ no-single-mono = refl
; no-pair-sum = refl
; canonical-deg4 = refl
}

```

A structural property of the forced output

The number 137 is not a target the enumeration goes looking for. It is the output of `simplex-eval` once the closed record has fixed $V = 4$, $E = 6$, $d = 3$, $\chi = 2$, witnessed by `refl`. The question that follows is therefore not *which formula yields 137*, but *what does the integer 137 look like inside the monomial pool generated by the same forced values*. The enumeration below is a property test on that integer.

For each degree bound the pool is fixed without reference to 137: there are exactly 35 multisets of size 4 over $\{V, E, d, \chi\}$, each giving a degree-4 monomial; together with the 35 monomials of degree ≤ 3 this is a 70-element pool, hence 4900 ordered pairs. Counting how many of these pairs sum to the forced output 137 returns exactly 2. The single unordered pair is $\{d^2, V^3\chi\}$; the degree-4 monomial $V^3\chi$ coincides with $V^d \cdot \chi$ at the forced value $d = 3$. The arithmetic identity that realises the canonical formula is, in this pool, the only one available.

The direction of inference is the inverse of a search. The output is fixed first; the pool is fixed by the same forced values; the single decomposition is then a fact about the integer, not a fact about a chosen formula.

```
-- § Degree-≤4 monomial values over (V, E, d, χ) = (4, 6, 3, 2).
-- § The 35 degree-4 monomials, by exponent vector (V, E, d, χ):
-- §   V4=256,   V3E=384,   V3d=192,   V3χ=128,   V2E2=576,
-- §   V2Ed=288, V2Eχ=192, V2d2=144, V2dχ=96,   V2χ2=64,
-- §   VE3=864, VE2d=432, VE2χ=288, VEd2=216, VEdχ=144,
-- §   VEχ2=96,  Vd3=108,  Vd2χ=72,  Vdχ2=48,  Vχ3=32,
-- §   E4=1296, E3d=648, E3χ=432, E2d2=324, E2dχ=216,
-- §   E2χ2=144, Ed3=162, Ed2χ=108, Edχ2=72, Eχ3=48,
-- §   d4=81,   d3χ=54,   d2χ2=36,  dχ3=24,   χ4=16.

private
-- § One-side hit count against the 70-monomial pool (deg ≤ 3 ∪ deg = 4).
hits137-deg4 : ℕ → ℕ
hits137-deg4 v =
  -- degree ≤ 3 (35 values)
  isEqℕ (v + 1) 137 + isEqℕ (v + 4) 137 + isEqℕ (v + 6) 137
+ isEqℕ (v + 3) 137 + isEqℕ (v + 2) 137
+ isEqℕ (v + 16) 137 + isEqℕ (v + 24) 137 + isEqℕ (v + 12) 137
+ isEqℕ (v + 8) 137 + isEqℕ (v + 36) 137 + isEqℕ (v + 18) 137
+ isEqℕ (v + 12) 137 + isEqℕ (v + 9) 137 + isEqℕ (v + 6) 137
+ isEqℕ (v + 4) 137
+ isEqℕ (v + 64) 137 + isEqℕ (v + 96) 137 + isEqℕ (v + 48) 137
+ isEqℕ (v + 32) 137 + isEqℕ (v + 144) 137 + isEqℕ (v + 72) 137
+ isEqℕ (v + 48) 137 + isEqℕ (v + 36) 137 + isEqℕ (v + 24) 137
+ isEqℕ (v + 16) 137 + isEqℕ (v + 216) 137 + isEqℕ (v + 108) 137
+ isEqℕ (v + 72) 137 + isEqℕ (v + 54) 137 + isEqℕ (v + 36) 137
+ isEqℕ (v + 24) 137 + isEqℕ (v + 27) 137 + isEqℕ (v + 18) 137
+ isEqℕ (v + 12) 137 + isEqℕ (v + 8) 137
  -- degree = 4 (35 values)
+ isEqℕ (v + 256) 137 + isEqℕ (v + 384) 137 + isEqℕ (v + 192) 137
+ isEqℕ (v + 128) 137 + isEqℕ (v + 576) 137 + isEqℕ (v + 288) 137
+ isEqℕ (v + 192) 137 + isEqℕ (v + 144) 137 + isEqℕ (v + 96) 137
+ isEqℕ (v + 64) 137 + isEqℕ (v + 864) 137 + isEqℕ (v + 432) 137
+ isEqℕ (v + 288) 137 + isEqℕ (v + 216) 137 + isEqℕ (v + 144) 137
+ isEqℕ (v + 96) 137 + isEqℕ (v + 108) 137 + isEqℕ (v + 72) 137
+ isEqℕ (v + 48) 137 + isEqℕ (v + 32) 137 + isEqℕ (v + 1296) 137
+ isEqℕ (v + 648) 137 + isEqℕ (v + 432) 137 + isEqℕ (v + 324) 137
+ isEqℕ (v + 216) 137 + isEqℕ (v + 144) 137 + isEqℕ (v + 162) 137
+ isEqℕ (v + 108) 137 + isEqℕ (v + 72) 137 + isEqℕ (v + 48) 137
+ isEqℕ (v + 81) 137 + isEqℕ (v + 54) 137 + isEqℕ (v + 36) 137
```

```

+ isEqN (v + 24) 137 + isEqN (v + 16) 137

-- § Total ordered-pair hits across the 70 evaluation points.
totalPairHits137-deg4 : ℕ
totalPairHits137-deg4 =
  -- degree ≤ 3 (35 evaluation points)
  hits137-deg4 1 + hits137-deg4 4 + hits137-deg4 6
+ hits137-deg4 3 + hits137-deg4 2
+ hits137-deg4 16 + hits137-deg4 24 + hits137-deg4 12
+ hits137-deg4 8 + hits137-deg4 36 + hits137-deg4 18
+ hits137-deg4 12 + hits137-deg4 9 + hits137-deg4 6
+ hits137-deg4 4
+ hits137-deg4 64 + hits137-deg4 96 + hits137-deg4 48
+ hits137-deg4 32 + hits137-deg4 144 + hits137-deg4 72
+ hits137-deg4 48 + hits137-deg4 36 + hits137-deg4 24
+ hits137-deg4 16 + hits137-deg4 216 + hits137-deg4 108
+ hits137-deg4 72 + hits137-deg4 54 + hits137-deg4 36
+ hits137-deg4 24 + hits137-deg4 27 + hits137-deg4 18
+ hits137-deg4 12 + hits137-deg4 8
  -- degree = 4 (35 evaluation points)
+ hits137-deg4 256 + hits137-deg4 384 + hits137-deg4 192
+ hits137-deg4 128 + hits137-deg4 576 + hits137-deg4 288
+ hits137-deg4 192 + hits137-deg4 144 + hits137-deg4 96
+ hits137-deg4 64 + hits137-deg4 864 + hits137-deg4 432
+ hits137-deg4 288 + hits137-deg4 216 + hits137-deg4 144
+ hits137-deg4 96 + hits137-deg4 108 + hits137-deg4 72
+ hits137-deg4 48 + hits137-deg4 32 + hits137-deg4 1296
+ hits137-deg4 648 + hits137-deg4 432 + hits137-deg4 324
+ hits137-deg4 216 + hits137-deg4 144 + hits137-deg4 162
+ hits137-deg4 108 + hits137-deg4 72 + hits137-deg4 48
+ hits137-deg4 81 + hits137-deg4 54 + hits137-deg4 36
+ hits137-deg4 24 + hits137-deg4 16

-- § The canonical pair  $(d^2, V^3\chi) = (9, 128)$  sums to 137.
canonical-pair-sums-to-137 :
  (simplex-degree * simplex-degree)
+ (simplex-vertices * simplex-vertices * simplex-vertices * simplex-chi)
≡ 137
canonical-pair-sums-to-137 = refl

-- § The degree-4 monomial  $V^3\chi$  coincides with  $V^d \cdot \chi$  at  $d = 3$ .
canonical-pair-is-V^d-χ :
  (simplex-vertices * simplex-vertices * simplex-vertices * simplex-chi)
≡ pow-Vd * simplex-chi
canonical-pair-is-V^d-χ = refl

-- § Exactly two ordered hits at degree ≤ 4: (9, 128) and (128, 9),
-- § i.e. one unordered pair, the canonical formula.
theorem-pair-sum-degree-4-count : totalPairHits137-deg4 ≡ 2
theorem-pair-sum-degree-4-count = refl

-- § Closure record: the canonical decomposition is the unique
-- § pair-sum at degree ≤ 4.
record DegreeBoundedExhaustion-deg4 : Set where

```

```

field
canonical-hit : (simplex-degree * simplex-degree)
               + (simplex-vertices * simplex-vertices
                  * simplex-vertices * simplex-chi)
               ≡ 137
canonical-form : (simplex-vertices * simplex-vertices
                  * simplex-vertices * simplex-chi)
               ≡ pow-Vd * simplex-chi
unique-count  : totalPairHits137-deg4 ≡ 2

theorem-degree-bounded-exhaustion-deg4 : DegreeBoundedExhaustion-deg4
theorem-degree-bounded-exhaustion-deg4 = record
{ canonical-hit = refl
; canonical-form = refl
; unique-count = refl
}

```

The reading is precise and one-directional. The integer 137 is produced by `simplex-eval` on the closed record, independently of any enumeration. Inside the 70-monomial pool generated by the same forced values, this integer admits a single pair-sum decomposition, and that decomposition is the canonical arithmetic identity $d^2 + V^3\chi = V^d \cdot \chi + d^2$. No claim is made about arbitrary arithmetic expressions, and the enumeration carries no selection pressure: the pool, the bound, and the integer are all determined before any pair is examined.

Status of the claim

Three layers must be kept apart, and the present development establishes all three.

What is forced about the values. The four invariants $(V, E, d, \chi) = (4, 6, 3, 2)$ are not parameters. They are the unique survivors of the elimination chain that produced K_4 , each witnessed inside Agda by `refl`. Once the closed record is in place, every arithmetic expression built from these invariants evaluates to a determinate integer. In particular, the expression $V^d \cdot \chi + d^2$ evaluates to 137, and the integer 137 admits, inside the 70-monomial pool of degree ≤ 4 over the same invariants, exactly one pair-sum decomposition — the canonical identity $d^2 + V^3\chi$. Both facts are machine-checked.

What is forced about the form. Chapter 38 closes the question of why the aggregating expression has the shape it has. The four invariants are not interchangeable quantities: V is configurational, E relational, d directional, χ topological. The six pairwise disjointness lemmas of §38 eliminate every identification between these kinds. Inside the bulk term, V^d and χ share a single underlying carrier — the closed extension structure — and combine multiplicatively as a categorical product whose forgetful image in $\mathbb{N}$ is numerical multiplication. The boundary term d^2 draws on a kind disjoint from both bulk factors and combines with the bulk additively as a categorical coproduct whose forgetful image is numerical addition. The form $V^d \cdot \chi + d^2$

is therefore the unique combination of these four invariants that respects the kind classification of §38.

What is forced about the meaning. Chapter 27 shows that the closed record is not a passive tuple of numbers but the description of an isotropic, monotone, d -dimensional extension structure. The factors of the simplex evaluation are not arbitrarily named: V^d is the count of independent configurations along the d directions of extension; χ is the closure condition under which that count is well-defined; d^2 is the cross-direction correction the boundary of the extension structure carries. The integer 137 is therefore the value of the unique well-typed scalar invariant of the extension structure forced by the elimination chain, evaluated by the arithmetic of the present part. Neither the values, nor the form, nor the meaning of the evaluation is adjustable.

What this means for the book. The construction yields a rigid, verifiable statement at every layer. The four invariants are forced; the form of the expression that combines them is forced; the spatial reading that gives the expression its meaning is forced. The exhaustion record above is not the source of any of these facts. It is a property test on the output of `simplex-eval`, confirming that the integer 137, once produced by the only well-typed expression on the only forced invariants, also occupies a structurally singular position inside the larger pool of monomials generated by the same data. The interpretation in *Form* therefore rests on a chain in which no link is left to a definition introduced by hand.

Measurement Acts

Measurement now returns to the original two-state setting. A measurement from a distinction into the fixed target distinction `Two-distinction` is completely determined by what it does to the left and right generators. The classification theorems for maps between distinctions therefore apply here without further modification: every measurement has an `EndoCase`, and that class is unique.

-- § Measurement: map from `d` into `Two-distinction`.

`Measurement` : `Distinction` → `Set`

`Measurement` $d = S\ d \rightarrow S\ Two\text{-}distinction$

-- § Law 15B.0: measurements are determined by their action on two generators.

`law15B-0-measurement-determined` :

$(d : Distinction) \rightarrow$

$(m_1\ m_2 : Measurement\ d) \rightarrow$

$m_1\ (\ell\ d) \equiv m_2\ (\ell\ d) \rightarrow$

$m_1\ (r\ d) \equiv m_2\ (r\ d) \rightarrow$

$m_1 \doteq m_2$

`law15B-0-measurement-determined` $d =$

`law7-1-map-determined` $d\ Two\text{-}distinction$

```

-- § Law 15B.1: every measurement is realized by some EndoCase.
law15B-1-measurement-classification-sound :
  (d : Distinction) →
  (m : Measurement d) →
   $\Sigma$  EndoCase ( $\lambda c \rightarrow K_4Map.interpret\ d\ Two-distinction\ c \doteq m$ )
law15B-1-measurement-classification-sound d m =
  law7-2-k4map-classification-sound d Two-distinction m

-- § Law 15B.2: the classifying EndoCase is unique.
law15B-2-measurement-classification-unique :
  (d : Distinction) →
  (m : Measurement d) →
  (c1 c2 : EndoCase) →
   $K_4Map.interpret\ d\ Two-distinction\ c_1 \doteq m \rightarrow$ 
   $K_4Map.interpret\ d\ Two-distinction\ c_2 \doteq m \rightarrow$ 
  c1  $\equiv$  c2
law15B-2-measurement-classification-unique d m c1 c2 p q =
  law7-3-k4map-classification-unique d Two-distinction m c1 c2 p q

```

Triangle Inequality Infrastructure

The numerator of the rational distance $d(p, q)$ is the absolute value of the cross-difference. Nonnegativity of this numerator is immediate from $|\cdot|$, and the scaled triangle inequality follows by factoring through subadditivity of $|\cdot|$ on $\mathbb{Z}$.

```

-- § Numerator of dist $\mathbb{Q}$ .
numDist $\mathbb{Q}$  :  $\mathbb{Q} \rightarrow \mathbb{Q} \rightarrow \mathbb{Z}$ 
numDist $\mathbb{Q}$  (a / b) (c / d) = abs $\mathbb{Z}$  ((a *  $\mathbb{Z} \rightarrow \mathbb{Z}$  d) +  $\mathbb{Z}$  neg $\mathbb{Z}$  (c *  $\mathbb{Z} \rightarrow \mathbb{Z}$  b))

-- § Distance numerator is nonneg.
numDist $\mathbb{Q}$ -nonneg : (p q :  $\mathbb{Q}$ ) → 0 $\mathbb{Z} \leq \mathbb{Z}$  numDist $\mathbb{Q}$  p q
numDist $\mathbb{Q}$ -nonneg (a / b) (c / d) = abs $\mathbb{Z}$ -nonneg _

```

Step 2: Triangle Inequality Core. The triangle inequality is now reduced to a single integer estimate. After clearing denominators, the numerator of $d(p, r)$ is rewritten as the sum of the numerators for $d(p, q)$ and $d(q, r)$, and the claim follows from subadditivity of the absolute value.

```

-- § Triangle core: scaled distance numerator inequality.
numDist $\mathbb{Q}$ -triangle-scaled : (p q r :  $\mathbb{Q}$ ) →
  (numDist $\mathbb{Q}$  p r *  $\mathbb{Z} \rightarrow \mathbb{Z}$  (den q))
   $\leq \mathbb{Z}$ 
  ((numDist $\mathbb{Q}$  p q *  $\mathbb{Z} \rightarrow \mathbb{Z}$  (den r)) +  $\mathbb{Z}$  (numDist $\mathbb{Q}$  q r *  $\mathbb{Z} \rightarrow \mathbb{Z}$  (den p)))
numDist $\mathbb{Q}$ -triangle-scaled (a / b) (c / d) (e / f) =
   $\leq \mathbb{Z}$ -resp- $\equiv^l$  lhsAbs
  ( $\leq \mathbb{Z}$ -resp- $\equiv^r$  rhsAbs
   absStep)
where

```

```

Wt : ℤ
Wt = (a * ℤ +toℤ f) +ℤ negℤ (e * ℤ +toℤ b)

Ut : ℤ
Ut = (a * ℤ +toℤ d) +ℤ negℤ (c * ℤ +toℤ b)

Vt : ℤ
Vt = (c * ℤ +toℤ f) +ℤ negℤ (e * ℤ +toℤ d)

swapScale : (x : ℤ) → (u v : ℕ+) → (x * ℤ +toℤ u) * ℤ +toℤ v ≡ (x * ℤ +toℤ v) * ℤ +toℤ u
swapScale x u v =
  trans
    (ℤ-assoc x (+toℤ u) (+toℤ v))
  (trans
    (cong (λ t → x * ℤ t) (ℤ-comm (+toℤ u) (+toℤ v)))
    (sym (ℤ-assoc x (+toℤ v) (+toℤ u))))

Wtd : ℤ
Wtd = Wt * ℤ +toℤ d

Utf : ℤ
Utf = Ut * ℤ +toℤ f

Vtb : ℤ
Vtb = Vt * ℤ +toℤ b

cancelMid : (c * ℤ +toℤ b) * ℤ +toℤ f ≡ (c * ℤ +toℤ f) * ℤ +toℤ b
cancelMid = swapScale c b f

cancelEnd : (e * ℤ +toℤ b) * ℤ +toℤ d ≡ (e * ℤ +toℤ d) * ℤ +toℤ b
cancelEnd = swapScale e b d

cancelHead : (a * ℤ +toℤ f) * ℤ +toℤ d ≡ (a * ℤ +toℤ d) * ℤ +toℤ f
cancelHead = swapScale a f d

-- § Algebra: Wt·d = Ut·f + Vt·b.
Wtd≡sum : Wtd ≡ (Utf +ℤ Vtb)
Wtd≡sum =
  trans WtdForm (sym sumForm)
where
WtdForm : Wtd ≡ ((a * ℤ +toℤ d) * ℤ +toℤ f) +ℤ negℤ ((e * ℤ +toℤ d) * ℤ +toℤ b)
WtdForm =
  trans
    (ℤ-distrib-left+ℤ (a * ℤ +toℤ f) (negℤ (e * ℤ +toℤ b)) (+toℤ d))
  (trans
    (cong (λ t → ((a * ℤ +toℤ f) * ℤ +toℤ d) +ℤ t)
      (ℤ-neg-left (e * ℤ +toℤ b) (+toℤ d)))
    (trans
      (cong (λ t → t +ℤ negℤ ((e * ℤ +toℤ b) * ℤ +toℤ d)) cancelHead)
      (cong (λ t → ((a * ℤ +toℤ d) * ℤ +toℤ f) +ℤ t)
        (cong negℤ cancelEnd))))

UtfForm : Utf ≡ ((a * ℤ +toℤ d) * ℤ +toℤ f) +ℤ negℤ ((c * ℤ +toℤ b) * ℤ +toℤ f)
UtfForm =
  trans

```

```

(*Z-distrib-left-+Z (a *Z +toZ d) (negZ (c *Z +toZ b)) (+toZ f))
(cong (λ t → ((a *Z +toZ d) *Z +toZ f) +Z t)
  (*Z-neg-left (c *Z +toZ b) (+toZ f)))

VtbForm : Vtb ≡ ((c *Z +toZ f) *Z +toZ b) +Z negZ ((e *Z +toZ d) *Z +toZ b)
VtbForm =
trans
(*Z-distrib-left-+Z (c *Z +toZ f) (negZ (e *Z +toZ d)) (+toZ b))
(cong (λ t → ((c *Z +toZ f) *Z +toZ b) +Z t)
  (*Z-neg-left (e *Z +toZ d) (+toZ b)))

sumForm :
(Utf +Z Vtb) ≡ ((a *Z +toZ d) *Z +toZ f) +Z negZ ((e *Z +toZ d) *Z +toZ b)
sumForm =
let
  Adf = (a *Z +toZ d) *Z +toZ f
  CbF = (c *Z +toZ b) *Z +toZ f
  CfB = (c *Z +toZ f) *Z +toZ b
  EdB = (e *Z +toZ d) *Z +toZ b

  UtfRhs = Adf +Z negZ CbF
  VtbRhs = CfB +Z negZ EdB

  midRewrite : (negZ CbF +Z CfB) ≡ (negZ CfB +Z CfB)
  midRewrite =
    cong (λ t → negZ t +Z CfB) cancelMid

  cancelMiddle : (negZ CbF +Z CfB) ≡ 0Z
  cancelMiddle =
    trans midRewrite (+Z-inv-left CfB)

  sumCancel : (UtfRhs +Z VtbRhs) ≡ (Adf +Z negZ EdB)
  sumCancel =
    trans
    (+Z-assoc Adf (negZ CbF) VtbRhs)
    (trans
      (cong (λ t → Adf +Z t)
        (sym (+Z-assoc (negZ CbF) CfB (negZ EdB))))
      (trans
        (cong (λ t → Adf +Z (t +Z negZ EdB)) cancelMiddle)
        (cong (λ t → Adf +Z t) (+Z-zero-left (negZ EdB)))))
    in
    trans
    (cong (λ t → t +Z Vtb) UtfForm)
    (trans
      (cong (λ t → UtfRhs +Z t) VtbForm)
      sumCancel)

absStep : absZ Wtd ≤Z (absZ Utf +Z absZ Vtb)
absStep =
≤Z-resp≡! (sym (cong absZ Wtd≡sum)) (absZ-subadd Utf Vtb)

lhsAbs : absZ Wtd ≡ (absZ Wt *Z +toZ d)
lhsAbs =

```

```

trans
(absℤ-mul-pos-right Wt d)
refl

rhsAbs : (absℤ Utf +ℤ absℤ Vtb) ≡ ((absℤ Ut *ℤ +toℤ f) +ℤ (absℤ Vt *ℤ +toℤ b))
rhsAbs =
trans
(cong (λ t → t +ℤ absℤ Vtb) (absℤ-mul-pos-right Ut f))
(cong (λ t → (absℤ Ut *ℤ +toℤ f) +ℤ t) (absℤ-mul-pos-right Vt b))

```


Chapter 40

Rational Setoid and Order Laws

This chapter develops the setoid and order structure on rationals. Equality is governed by cross-multiplication, and order is inherited from the corresponding integer comparison after denominators have been cleared.

Setoid Laws

The first task is to show that $\simeq_{\mathbb{Q}}$ is an equivalence relation. Reflexivity and symmetry are immediate. Transitivity is the only genuine step: both given equalities are multiplied by the missing denominator, and the resulting common positive factor is then cancelled.

```
-- § Reflexivity of  $\simeq_{\mathbb{Q}}$ .
 $\simeq_{\mathbb{Q}}\text{-refl} : (p : \mathbb{Q}) \rightarrow p \simeq_{\mathbb{Q}} p$ 
 $\simeq_{\mathbb{Q}}\text{-refl} (a / b) = \text{refl}$ 

-- § Symmetry of  $\simeq_{\mathbb{Q}}$ .
 $\simeq_{\mathbb{Q}}\text{-sym} : \{p\ q : \mathbb{Q}\} \rightarrow p \simeq_{\mathbb{Q}} q \rightarrow q \simeq_{\mathbb{Q}} p$ 
 $\simeq_{\mathbb{Q}}\text{-sym} = \text{sym}$ 

-- § Right cancellation of  $\ast \mathbb{Z}$  by positive factor.
 $\ast \mathbb{Z}\text{-cancel-right-pos} : (x\ y : \mathbb{Z}) \rightarrow (d : \mathbb{N}^+) \rightarrow (x \ast \mathbb{Z} \text{+to} \mathbb{Z} d) \equiv (y \ast \mathbb{Z} \text{+to} \mathbb{Z} d) \rightarrow x \equiv y$ 
 $\ast \mathbb{Z}\text{-cancel-right-pos}\ x\ y\ d\ eq =$ 
 $\leq \mathbb{Z}\text{-antisym}$ 
 $(\leq \mathbb{Z}\text{-mul-pos-cancel-right}\ x\ y\ d\ (\leq \mathbb{Z}\text{-resp-}\equiv^r\ eq\ (\leq \mathbb{Z}\text{-refl}\ (x \ast \mathbb{Z} \text{+to} \mathbb{Z} d))))$ 
 $(\leq \mathbb{Z}\text{-mul-pos-cancel-right}\ y\ x\ d\ (\leq \mathbb{Z}\text{-resp-}\equiv^r\ (\text{sym}\ eq)\ (\leq \mathbb{Z}\text{-refl}\ (y \ast \mathbb{Z} \text{+to} \mathbb{Z} d))))$ 

-- § Transitivity of  $\simeq_{\mathbb{Q}}$  (uses torsion-freedom to cancel intermediate denominator).
 $\simeq_{\mathbb{Q}}\text{-trans} : \{p\ q\ r : \mathbb{Q}\} \rightarrow p \simeq_{\mathbb{Q}} q \rightarrow q \simeq_{\mathbb{Q}} r \rightarrow p \simeq_{\mathbb{Q}} r$ 
 $\simeq_{\mathbb{Q}}\text{-trans}\ \{a\ /\ b\}\ \{c\ /\ d\}\ \{e\ /\ f\}\ eq_1\ eq_2 =$ 
let
```

```

B : ℤ
B = +toℤ b

D : ℤ
D = +toℤ d

F : ℤ
F = +toℤ f

step1 : ((a *ℤ D) *ℤ F) ≡ ((c *ℤ B) *ℤ F)
step1 = cong (λ t → t *ℤ F) eq1

step2 : ((c *ℤ F) *ℤ B) ≡ ((e *ℤ D) *ℤ B)
step2 = cong (λ t → t *ℤ B) eq2

swapCBF : ((c *ℤ B) *ℤ F) ≡ ((c *ℤ F) *ℤ B)
swapCBF =
  trans
    (*ℤ-assoc c B F)
  (trans
    (cong (λ t → c *ℤ t) (*ℤ-comm B F))
    (sym (*ℤ-assoc c F B)))

mid : ((a *ℤ D) *ℤ F) ≡ ((e *ℤ D) *ℤ B)
mid = trans step1 (trans swapCBF step2)

regroupL : ((a *ℤ D) *ℤ F) ≡ (a *ℤ F) *ℤ D
regroupL =
  trans
    (*ℤ-assoc a D F)
  (trans
    (cong (λ t → a *ℤ t) (*ℤ-comm D F))
    (sym (*ℤ-assoc a F D)))

regroupR : ((e *ℤ D) *ℤ B) ≡ (e *ℤ B) *ℤ D
regroupR =
  trans
    (*ℤ-assoc e D B)
  (trans
    (cong (λ t → e *ℤ t) (*ℤ-comm D B))
    (sym (*ℤ-assoc e B D)))

eqD : ((a *ℤ F) *ℤ D) ≡ ((e *ℤ B) *ℤ D)
eqD = trans (sym regroupL) (trans mid regroupR)
in
  *ℤ-cancel-right-pos (a *ℤ F) (e *ℤ B) d eqD

```

Preorder Laws

Reflexivity of $\leq_{\mathbb{Q}}$ reduces to $\leq_{\mathbb{Z}}$ -reflexivity of the common cross-product. Transitivity cross-multiplies both inequalities by the missing denominator, applies

$\leq_{\mathbb{Z}}$ -transitivity, and cancels the common factor by multiplicative monotonicity. The mixed compositions $\leq \circ <$ and $< \circ \leq$ then follow from transitivity and irreflexivity.

```
-- § Reflexivity of  $\leq_{\mathbb{Q}}$ .
 $\leq_{\mathbb{Q}}\text{-refl} : (q : \mathbb{Q}) \rightarrow q \leq_{\mathbb{Q}} q$ 
 $\leq_{\mathbb{Q}}\text{-refl} (a / b) = \leq_{\mathbb{Z}}\text{-refl} (a * \mathbb{Z} + \text{to}\mathbb{Z} b)$ 

-- § Denominator scaling swap.
swapScale $\mathbb{Q} : (x : \mathbb{Z}) \rightarrow (u v : \mathbb{N}^+) \rightarrow (x * \mathbb{Z} + \text{to}\mathbb{Z} u) * \mathbb{Z} + \text{to}\mathbb{Z} v \equiv (x * \mathbb{Z} + \text{to}\mathbb{Z} v) * \mathbb{Z} + \text{to}\mathbb{Z} u$ 
swapScale $\mathbb{Q} x u v =$ 
  trans
    ( $\mathbb{Z}\text{-assoc } x (+\text{to}\mathbb{Z} u) (+\text{to}\mathbb{Z} v)$ )
  (trans
    (cong ( $\lambda t \rightarrow x * \mathbb{Z} t$ ) ( $\mathbb{Z}\text{-comm } (+\text{to}\mathbb{Z} u) (+\text{to}\mathbb{Z} v)$ )))
    (sym ( $\mathbb{Z}\text{-assoc } x (+\text{to}\mathbb{Z} v) (+\text{to}\mathbb{Z} u)$ )))

-- § Transitivity of  $\leq_{\mathbb{Q}}$  (uses multiplicative monotonicity + cancellation).
 $\leq_{\mathbb{Q}}\text{-trans} : \{x y z : \mathbb{Q}\} \rightarrow x \leq_{\mathbb{Q}} y \rightarrow y \leq_{\mathbb{Q}} z \rightarrow x \leq_{\mathbb{Q}} z$ 
 $\leq_{\mathbb{Q}}\text{-trans} \{x\} \{y\} \{z\} p q \text{ with } x \mid y \mid z$ 
... | a / b | c / d | e / f =
  let
    p' : ((a *  $\mathbb{Z} + \text{to}\mathbb{Z} d$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} f$ )  $\leq_{\mathbb{Z}}$  ((c *  $\mathbb{Z} + \text{to}\mathbb{Z} b$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} f$ )
    p' =  $\leq_{\mathbb{Z}}\text{-mul-pos-right} (a * \mathbb{Z} + \text{to}\mathbb{Z} d) (c * \mathbb{Z} + \text{to}\mathbb{Z} b) f p$ 

    q' : ((c *  $\mathbb{Z} + \text{to}\mathbb{Z} f$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} b$ )  $\leq_{\mathbb{Z}}$  ((e *  $\mathbb{Z} + \text{to}\mathbb{Z} d$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} b$ )
    q' =  $\leq_{\mathbb{Z}}\text{-mul-pos-right} (c * \mathbb{Z} + \text{to}\mathbb{Z} f) (e * \mathbb{Z} + \text{to}\mathbb{Z} d) b q$ 

    midEq : ((c *  $\mathbb{Z} + \text{to}\mathbb{Z} b$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} f$ )  $\equiv$  ((c *  $\mathbb{Z} + \text{to}\mathbb{Z} f$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} b$ )
    midEq = swapScale $\mathbb{Q} c b f$ 

    p'' : ((a *  $\mathbb{Z} + \text{to}\mathbb{Z} d$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} f$ )  $\leq_{\mathbb{Z}}$  ((c *  $\mathbb{Z} + \text{to}\mathbb{Z} f$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} b$ )
    p'' =  $\leq_{\mathbb{Z}}\text{-resp} \equiv^r \text{midEq } p'$ 

    step : ((a *  $\mathbb{Z} + \text{to}\mathbb{Z} d$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} f$ )  $\leq_{\mathbb{Z}}$  ((e *  $\mathbb{Z} + \text{to}\mathbb{Z} d$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} b$ )
    step =  $\leq_{\mathbb{Z}}\text{-trans } p'' q'$ 

    lhsEq : ((a *  $\mathbb{Z} + \text{to}\mathbb{Z} d$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} f$ )  $\equiv$  ((a *  $\mathbb{Z} + \text{to}\mathbb{Z} f$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} d$ )
    lhsEq = swapScale $\mathbb{Q} a d f$ 

    rhsEq : ((e *  $\mathbb{Z} + \text{to}\mathbb{Z} d$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} b$ )  $\equiv$  ((e *  $\mathbb{Z} + \text{to}\mathbb{Z} b$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} d$ )
    rhsEq = swapScale $\mathbb{Q} e d b$ 

    step' : ((a *  $\mathbb{Z} + \text{to}\mathbb{Z} f$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} d$ )  $\leq_{\mathbb{Z}}$  ((e *  $\mathbb{Z} + \text{to}\mathbb{Z} b$ ) *  $\mathbb{Z} + \text{to}\mathbb{Z} d$ )
    step' =  $\leq_{\mathbb{Z}}\text{-resp} \equiv^l \text{lhsEq} (\leq_{\mathbb{Z}}\text{-resp} \equiv^r \text{rhsEq } \text{step})$ 

    done : (a *  $\mathbb{Z} + \text{to}\mathbb{Z} f$ )  $\leq_{\mathbb{Z}}$  (e *  $\mathbb{Z} + \text{to}\mathbb{Z} b$ )
    done =  $\leq_{\mathbb{Z}}\text{-mul-pos-cancel-right} (a * \mathbb{Z} + \text{to}\mathbb{Z} f) (e * \mathbb{Z} + \text{to}\mathbb{Z} b) d \text{ step'}$ 
  in
    done

-- § Negation of  $\leq_{\mathbb{Q}}$ .
 $\not\leq_{\mathbb{Q}} : \mathbb{Q} \rightarrow \mathbb{Q} \rightarrow \text{Set}$ 
x  $\not\leq_{\mathbb{Q}}$  y = (x  $\leq_{\mathbb{Q}}$  y)  $\rightarrow \perp$ 

-- §  $\leq$  composed with  $<$  yields  $<.$ 
```

```

≤<Q→<Q : {x y z : Q} → x ≤Q y → y <Q z → x <Q z
≤<Q→<Q {a / b} {c / d} {e / f} x ≤ y (y ≤ z, z < y) =
let
  x ≤ z : (a / b) ≤Q (e / f)
  x ≤ z = ≤Q-trans {a / b} {c / d} {e / f} x ≤ y y ≤ z

  z < x : (e / f) <Q (a / b)
  z < x z ≤ x = z < y (≤Q-trans {e / f} {a / b} {c / d} z ≤ x x ≤ y)
in
  x ≤ z, z < x

-- § < composed with ≤ yields <.
≤<Q→<Q : {x y z : Q} → x <Q y → y ≤Q z → x <Q z
≤<Q→<Q {a / b} {c / d} {e / f} (x ≤ y, y < x) y ≤ z =
let
  x ≤ z : (a / b) ≤Q (e / f)
  x ≤ z = ≤Q-trans {a / b} {c / d} {e / f} x ≤ y y ≤ z

  z < x : (e / f) <Q (a / b)
  z < x z ≤ x =
let
  y ≤ x : (c / d) ≤Q (a / b)
  y ≤ x = ≤Q-trans {c / d} {e / f} {a / b} y ≤ z z ≤ x
in
  y < x y ≤ x
in
  x ≤ z, z < x

```

Rational Order Base Laws

The next lemmas collect the elementary base facts for rational order. Strict order implies non-strict order, setoid equality forces $\leq$ in both directions, positive denominators correspond to strictly positive integers, and the distinguished inequality $0 < 1$ appears in both $\mathbb{Z}$ and $\mathbb{Q}$. These facts anchor every later spectral-gap and distance argument.

```

-- § Strict order implies non-strict.
<Q→≤Q : {x y : Q} → x <Q y → x ≤Q y
<Q→≤Q p = fst p

-- § Aliases for uniform naming.
ltZ_to_leZ : {x y : Z} → x <Z y → x ≤Z y
ltZ_to_leZ {x} {y} p = <Z→≤Z {x} {y} p

ltQ_to_leQ : {x y : Q} → x <Q y → x ≤Q y
ltQ_to_leQ {x} {y} p = <Q→≤Q {x} {y} p

-- § Setoid equality forces ≤ in both directions.
≈Q→≤Ql : {p q : Q} → p ≈Q q → p ≤Q q
≈Q→≤Ql {a / b} {c / d} eq =

```

```

≤ℤ-resp≡r eq (≤ℤ-refl (a *ℤ +toℤ d))

≈ℚ→≤ℚr : {p q : ℚ} → p ≈ℚ q → q ≤ℚ p
≈ℚ→≤ℚr {a / b} {c / d} eq =
  ≤ℤ-resp≡r (sym eq) (≤ℤ-refl (c *ℤ +toℤ b))

-- § Positive naturals are strictly positive integers.
den-posℤ : (d : ℕ+) → 0ℤ <ℤ +toℤ d
den-posℤ (mkℕ+ k) =
  tt, (λ p → p)

-- § 0 < 1 in ℤ.
0ℤ<oneℤ : 0ℤ <ℤ oneℤ
0ℤ<oneℤ =
  tt, (λ p → p)

-- § 0 < 1 in ℚ.
0ℚ<1ℚ : 0ℚ <ℚ 1ℚ
0ℚ<1ℚ =
  <ℤ-resp≡l (sym (*ℤ-zero-left (+toℤ one+)))
  (<ℤ-resp≡r (sym (*ℤ-one-left (+toℤ one+)))
    0ℤ<oneℤ)

-- § 0 < ε forces 0 < num ε.
0ℚ<→0ℤ<num : (ε : ℚ) → 0ℚ <ℚ ε → 0ℤ <ℤ num ε
0ℚ<→0ℤ<num (a / b) p =
  let step1 : 0ℤ <ℤ (a *ℤ +toℤ one+)
    step1 = <ℤ-resp≡l (*ℤ-zero-left (+toℤ b)) p

    step2 : 0ℤ <ℤ a
    step2 = <ℤ-resp≡r (*ℤ-one-right a) step1
  in
  step2

```


Chapter 41

Laplacian as Finite-Index Operator

The Laplacian has so far been expressed directly on the four measurement cases. It is now useful to rewrite the same operator on the finite index set `Fin4`, so that matrix formulas and spectral identities can be stated without reference to the original case distinction.

Vec4 Infrastructure

The basic objects are vectors indexed by `Fin4`, a small collection of canonical integer actions, and matrices built from those actions. The purpose of this section is simply to isolate the four coefficients that recur throughout the Laplacian formulas.

```
-- § Vec4ℤ: integer-valued functions on the four canonical indices.
```

```
Vec4ℤ : Set
```

```
Vec4ℤ = Fin4 → ℤ
```

```
-- § Actℤ: an action on integers.
```

```
Actℤ : Set
```

```
Actℤ = ℤ → ℤ
```

```
-- § Canonical integer actions.
```

```
zeroAct : Actℤ
```

```
zeroAct _ = 0ℤ
```

```
idAct : Actℤ
```

```
idAct x = x
```

```
negAct : Actℤ
```

```
negAct = negℤ
```

```
threeAct : Actℤ
```

```

threeAct = threeTimesℤ

fourAct : Actℤ
fourAct = fourTimesℤ

-- § Coefficient type: the four forced actions as a closed data type.
data Coeffℤ : Set where
  c0 : Coeffℤ
  c1 : Coeffℤ
  c-1 : Coeffℤ
  c3 : Coeffℤ

-- § Map coefficients to their canonical actions.
coeffAct : Coeffℤ → Actℤ
coeffAct c0 = zeroAct
coeffAct c1 = idAct
coeffAct c-1 = negAct
coeffAct c3 = threeAct

-- § Matrix types: coefficient-valued and action-valued.
Mat4Coeffℤ : Set
Mat4Coeffℤ = Fin4 → Fin4 → Coeffℤ

liftCoeffMatℤ : Mat4Coeffℤ → (Fin4 → Fin4 → Actℤ)
liftCoeffMatℤ m i j = coeffAct (m i j)

Mat4Actℤ : Set
Mat4Actℤ = Fin4 → Fin4 → Actℤ

```

Off-Diagonal Enumeration

For each index one must be able to enumerate the three complementary indices. The function `others` provides that enumeration, and the accompanying `finite sums` package the two combinations that recur constantly: the sum over all indices and the sum over the off-diagonal ones.

```

-- § others i k: the k-th index distinct from i (exhaustive by Fin3).
others : Fin4 → Fin3 → Fin4
others g0 f0 = g1
others g0 f1 = g2
others g0 f2 = g3
others g1 f0 = g0
others g1 f1 = g2
others g1 f2 = g3
others g2 f0 = g0
others g2 f1 = g1
others g2 f2 = g3
others g3 f0 = g0
others g3 f1 = g1
others g3 f2 = g2

```

```

-- § Sum over all four indices, starting from i.
sumFin4Aroundℤ : Fin4 → (Fin4 → ℤ) → ℤ
sumFin4Aroundℤ i f = sum4ℤ (f i) (f (others i f0)) (f (others i f1)) (f (others i f2))

-- § Sum over the three off-diagonal indices.
sumOthersℤ : Vec4ℤ → Fin4 → ℤ
sumOthersℤ v i = sumFin3ℤ (λ k → v (others i k))

```

Laplacian on Vec4

The Laplacian acts as $Lv_i = 3v_i - \sum_{j \neq i} v_j$. The same operator is now written in two visibly different ways: first as the direct formula $3v_i - \sum_{j \neq i} v_j$, and second as the action of a matrix whose diagonal entries are `threeAct` and whose off-diagonal entries are `idAct`. The auxiliary conversions `vecFromEndo` and `endoFromVec` keep track of the passage between the original case-based description and the finite-index one.

```

-- § Laplacian: 3·vi minus neighbor sum.
laplacianVec4ℤ : Vec4ℤ → Vec4ℤ
laplacianVec4ℤ v i = threeTimesℤ (v i) + ℤ negℤ (sumOthersℤ v i)

-- § Matrix application (pre-action form).
applyLaplacianPreActℤ : Mat4Actℤ → Vec4ℤ → Vec4ℤ
applyLaplacianPreActℤ m v i =
  m i i (v i) + ℤ
  negℤ (sumFin3ℤ (λ k → m i (others i k) (v (others i k))))

-- § Pre-action Laplacian matrix: diagonal = 3×, off-diagonal = id.
laplacianPreMatActℤ : Mat4Actℤ
laplacianPreMatActℤ i j with Fin4-decEq i j
... | inj1 _ = threeAct
... | inj2 _ = idAct

-- § Matrix-applied Laplacian.
laplacianMatVec4ℤ : Vec4ℤ → Vec4ℤ
laplacianMatVec4ℤ = applyLaplacianPreActℤ laplacianPreMatActℤ

-- § Conversion between EndoCase and Fin4 representations.
vecFromEndo : (EndoCase → ℤ) → Vec4ℤ
vecFromEndo f i = f (vertexAt i)

endoFromVec : Vec4ℤ → (EndoCase → ℤ)
endoFromVec v x = v (vertexIndex x)

```

Compatibility Laws

These first comparison laws show that nothing has changed mathematically: the Laplacian on `EndoCase`, the finite-index vector formula, and the pre-action matrix

presentation all describe the same operator. Once this identification is in place, the later matrix and spectral arguments can be carried out entirely on `Fin4`.

```
-- § Law 14E.0: EndoCase Laplacian factors through Fin4 indexing.
law14E-0-laplacian-factor : (f : EndoCase → ℤ) → (x : EndoCase) →
  laplacianVec4ℤ (vecFromEndo f) (vertexIndex x) ≡ laplacianℤ f x
law14E-0-laplacian-factor f case-constL = refl
law14E-0-laplacian-factor f case-constR = refl
law14E-0-laplacian-factor f case-id = refl
law14E-0-laplacian-factor f case-dual = refl

-- § Law 14E.1: pre-action matrix agrees with the Laplacian.
law14E-1-matrix-agrees : (v : Vec4ℤ) → (i : Fin4) →
  laplacianMatVec4ℤ v i ≡ laplacianVec4ℤ v i
law14E-1-matrix-agrees v g0 = refl
law14E-1-matrix-agrees v g1 = refl
law14E-1-matrix-agrees v g2 = refl
law14E-1-matrix-agrees v g3 = refl
```

Post-Action and Row-Sum Presentations

The Laplacian has already been defined pointwise. The next step is to compare several equivalent matrix presentations: diagonal-plus-off-diagonal, row sums, global sums, and coefficient matrices. Because the index set has only four elements, every comparison can be proved by direct unfolding.

```
-- § Alternative application forms.
applyMat4ActDiagOthersℤ : Mat4Actℤ → Vec4ℤ → Vec4ℤ
applyMat4ActDiagOthersℤ m v i =
  m i i (v i) + ℤ
  sumFin3ℤ (λ k → m i (others i k) (v (others i k)))

applyMat4ActRowSumℤ : Mat4Actℤ → Vec4ℤ → Vec4ℤ
applyMat4ActRowSumℤ m v i = sumFin4Aroundℤ i (λ j → m i j (v j))

applyMat4ActGlobalSumℤ : Mat4Actℤ → Vec4ℤ → Vec4ℤ
applyMat4ActGlobalSumℤ m v i = sumFin4ℤ (λ j → m i j (v j))

applyMat4CoeffGlobalSumℤ : Mat4Coeffℤ → Vec4ℤ → Vec4ℤ
applyMat4CoeffGlobalSumℤ m v i = sumFin4ℤ (λ j → coeffAct (m i j) (v j))

-- § Post-action: diagonal = 3×, off-diagonal = negation.
laplacianPostMatActℤ : Mat4Actℤ
laplacianPostMatActℤ i j with Fin4-decEq i j
... | inj1 _ = threeAct
... | inj2 _ = negAct

laplacianPostMatVec4ℤ : Vec4ℤ → Vec4ℤ
laplacianPostMatVec4ℤ = applyMat4ActDiagOthersℤ laplacianPostMatActℤ

laplacianRowSumMatVec4ℤ : Vec4ℤ → Vec4ℤ
```



```

laplacianRowSumMatVec4ℤ = applyMat4ActRowSumℤ laplacianPostMatActℤ

laplacianGlobalMatVec4ℤ : Vec4ℤ → Vec4ℤ
laplacianGlobalMatVec4ℤ = applyMat4ActGlobalSumℤ laplacianPostMatActℤ

-- § Coefficient Laplacian matrix: diagonal c3, off-diagonal c-1.
laplacianCoeffMatℤ : Mat4Coeffℤ
laplacianCoeffMatℤ i j with Fin4-decEq i j
... | inj1 _ = c3
... | inj2 _ = c-1

laplacianCoeffGlobalMatVec4ℤ : Vec4ℤ → Vec4ℤ
laplacianCoeffGlobalMatVec4ℤ = applyMat4CoeffGlobalSumℤ laplacianCoeffMatℤ

-- § Law 14E.3: row-sum unfolds to diagonal+others.
law14E-3-row-sum-unfolds : (m : Mat4Actℤ) → (v : Vec4ℤ) → (i : Fin4) →
  applyMat4ActRowSumℤ m v i ≡ applyMat4ActDiagOthersℤ m v i
law14E-3-row-sum-unfolds m v i = refl

-- § Law 14E.4: row-sum around i equals global sum (by permutation).
law14E-4-sumFin4Around-eq-sumFin4 : (f : Fin4 → ℤ) → (i : Fin4) →
  sumFin4Aroundℤ i f ≡ sumFin4ℤ f
law14E-4-sumFin4Around-eq-sumFin4 f g0 = refl
law14E-4-sumFin4Around-eq-sumFin4 f g1 = sum4ℤ-swap01 (f g1) (f g0) (f g2) (f g3)
law14E-4-sumFin4Around-eq-sumFin4 f g2 =
  trans (sum4ℤ-swap01 (f g2) (f g0) (f g1) (f g3))
    (sum4ℤ-swap12 (f g0) (f g2) (f g1) (f g3))
law14E-4-sumFin4Around-eq-sumFin4 f g3 =
  trans
    (trans (sum4ℤ-swap01 (f g3) (f g0) (f g1) (f g2))
      (sum4ℤ-swap12 (f g0) (f g3) (f g1) (f g2)))
    (sum4ℤ-swap23 (f g0) (f g1) (f g3) (f g2))

-- § Law 14E.2: post-action matrix agrees with Laplacian (negation placement).
law14E-2-matrix-neg-in-agrees : (v : Vec4ℤ) → (i : Fin4) →
  laplacianPostMatVec4ℤ v i ≡ laplacianVec4ℤ v i
law14E-2-matrix-neg-in-agrees v g0 =
  cong (λ t → threeTimesℤ (v g0) +ℤ t)
    (sym (neg-sumFin3ℤ (λ k → v (others g0 k))))
law14E-2-matrix-neg-in-agrees v g1 =
  cong (λ t → threeTimesℤ (v g1) +ℤ t)
    (sym (neg-sumFin3ℤ (λ k → v (others g1 k))))
law14E-2-matrix-neg-in-agrees v g2 =
  cong (λ t → threeTimesℤ (v g2) +ℤ t)
    (sym (neg-sumFin3ℤ (λ k → v (others g2 k))))
law14E-2-matrix-neg-in-agrees v g3 =
  cong (λ t → threeTimesℤ (v g3) +ℤ t)
    (sym (neg-sumFin3ℤ (λ k → v (others g3 k))))

-- § Law 14E.5: row-sum equals global-sum.
law14E-5-rowSum-eq-globalSum : (m : Mat4Actℤ) → (v : Vec4ℤ) → (i : Fin4) →
  applyMat4ActRowSumℤ m v i ≡ applyMat4ActGlobalSumℤ m v i
law14E-5-rowSum-eq-globalSum m v i =
  law14E-4-sumFin4Around-eq-sumFin4 (λ j → m i j (v j)) i

```

```

-- § Law 14E.6: global matrix application equals Laplacian.
law14E-6-global-matrix-agrees : (v : Vec4ℤ) → (i : Fin4) →
  laplacianGlobalMatVec4ℤ v i ≡ laplacianVec4ℤ v i
law14E-6-global-matrix-agrees v i =
  trans (sym (law14E-5-rowSum-eq-globalSum laplacianPostMatActℤ v i))
    (trans (law14E-3-row-sum-unfolds laplacianPostMatActℤ v i)
      (law14E-2-matrix-neg-in-agrees v i))

-- § Law 14E.7: coefficient lift agrees with post-action matrix (16 cases).
law14E-7-coeff-lift-agrees : (i j : Fin4) →
  liftCoeffMatℤ laplacianCoeffMatℤ i j ≡ laplacianPostMatActℤ i j
law14E-7-coeff-lift-agrees g0 g0 = refl
law14E-7-coeff-lift-agrees g0 g1 = refl
law14E-7-coeff-lift-agrees g0 g2 = refl
law14E-7-coeff-lift-agrees g0 g3 = refl
law14E-7-coeff-lift-agrees g1 g0 = refl
law14E-7-coeff-lift-agrees g1 g1 = refl
law14E-7-coeff-lift-agrees g1 g2 = refl
law14E-7-coeff-lift-agrees g1 g3 = refl
law14E-7-coeff-lift-agrees g2 g0 = refl
law14E-7-coeff-lift-agrees g2 g1 = refl
law14E-7-coeff-lift-agrees g2 g2 = refl
law14E-7-coeff-lift-agrees g2 g3 = refl
law14E-7-coeff-lift-agrees g3 g0 = refl
law14E-7-coeff-lift-agrees g3 g1 = refl
law14E-7-coeff-lift-agrees g3 g2 = refl
law14E-7-coeff-lift-agrees g3 g3 = refl

-- § Law 14E.8: coefficient global sum equals action global sum.
law14E-8-coeff-global-eq-act-global : (v : Vec4ℤ) → (i : Fin4) →
  laplacianCoeffGlobalMatVec4ℤ v i ≡ laplacianGlobalMatVec4ℤ v i
law14E-8-coeff-global-eq-act-global v g0 = refl
law14E-8-coeff-global-eq-act-global v g1 = refl
law14E-8-coeff-global-eq-act-global v g2 = refl
law14E-8-coeff-global-eq-act-global v g3 = refl

-- § Law 14E.9: coefficient global matrix equals Laplacian.
law14E-9-coeff-global-agrees : (v : Vec4ℤ) → (i : Fin4) →
  laplacianCoeffGlobalMatVec4ℤ v i ≡ laplacianVec4ℤ v i
law14E-9-coeff-global-agrees v i =
  trans (law14E-8-coeff-global-eq-act-global v i)
    (law14E-6-global-matrix-agrees v i)

```

Spectral Form: $L = 4I - J$

Once the row-sum presentation is available, the familiar spectral formula

$$L = 4I - J$$

appears directly. Here J denotes the operator that replaces a vector by the constant vector whose value is the global sum of its components. This identity is the source of the eigenspace decomposition proved below.

```
-- § Split around-sum into diagonal + others.
sumFin4Around-split : (v : Vec4ℤ) → (i : Fin4) →
  sumFin4Aroundℤ i v ≡ v i + ℤ sumOthersℤ v i
sumFin4Around-split v g0 = refl
sumFin4Around-split v g1 = refl
sumFin4Around-split v g2 = refl
sumFin4Around-split v g3 = refl

-- § 4x = x + 3x.
fourTimes-split : (x : ℤ) → fourTimesℤ x ≡ x + ℤ threeTimesℤ x
fourTimes-split x = refl

-- § Law 14E.10: Laplacian equals 4·vi minus global sum.
law14E-10-laplacian-four-minus-sumAll : (v : Vec4ℤ) → (i : Fin4) →
  laplacianVec4ℤ v i ≡ fourTimesℤ (v i) + ℤ negℤ (sumFin4ℤ v)
law14E-10-laplacian-four-minus-sumAll v i =
  let x = v i in
  let othersSum = sumOthersℤ v i in
  let around = sumFin4Aroundℤ i v in
  let a = threeTimesℤ x in
  let b = negℤ othersSum in
  let rhsAround = fourTimesℤ x + ℤ negℤ around in
  let rhsAround≡laplacian : rhsAround ≡ laplacianVec4ℤ v i
  rhsAround≡laplacian =
    trans
      (cong (λ t → t + ℤ negℤ around) (fourTimes-split x))
      (trans
        (cong (λ t → (x + ℤ a) + ℤ t) (trans (cong negℤ (sumFin4Around-split v i)) (neg-+ℤ x othersSum)))
        (trans
          (+ℤ-assoc x a (negℤ x + ℤ negℤ othersSum))
          (trans
            (cong (λ t → x + ℤ t) (sym (+ℤ-assoc a (negℤ x) (negℤ othersSum))))
            )
          (trans
            (cong (λ t → x + ℤ ((t) + ℤ negℤ othersSum)) (+ℤ-comm a (negℤ x)))
            (trans
              (cong (λ t → x + ℤ t) (+ℤ-assoc (negℤ x) a (negℤ othersSum)))
              (trans
                (sym (+ℤ-assoc x (negℤ x) (a + ℤ negℤ othersSum)))
                (trans
                  (cong (λ t → t + ℤ (a + ℤ negℤ othersSum)) (+ℤ-inv-right x))
                  (trans
                    (+ℤ-zero-left (a + ℤ negℤ othersSum))
                    (refl))))))))))
      )
    in
  trans
    (sym rhsAround≡laplacian)
    (cong (λ s → fourTimesℤ x + ℤ negℤ s) (law14E-4-sumFin4Around-eq-sumFin4 v i))
```

Eigenvector Classification

The operator identity isolates the two basic spectral behaviours at once. Constant vectors belong to the 0-eigenspace, and vectors of total sum zero belong to the 4-eigenspace. The next laws record both directions of that correspondence.

```
-- § Law 14E.11: sum-zero vectors are 4-eigenvectors.
law14E-11-sum0-eigen4 : (v : Vec4ℤ) → (i : Fin4) → sumFin4ℤ v ≡ 0ℤ →
  laplacianVec4ℤ v i ≡ fourTimesℤ (v i)
law14E-11-sum0-eigen4 v i sum0 =
  trans
    (law14E-10-laplacian-four-minus-sumAll v i)
    (trans
      (cong (λ s → fourTimesℤ (v i) +ℤ negℤ s) sum0)
      (+ℤ-zero-right (fourTimesℤ (v i))))

-- § Constant vector and J operator.
constVec4ℤ : ℤ → Vec4ℤ
constVec4ℤ x _ = x

JVec4ℤ : Vec4ℤ → Vec4ℤ
JVec4ℤ v _ = sumFin4ℤ v

-- § Coefficient matrix for J: all ones.
onesCoeffMatℤ : Mat4Coeffℤ
onesCoeffMatℤ _ _ = c1

JCoeffGlobalMatVec4ℤ : Vec4ℤ → Vec4ℤ
JCoeffGlobalMatVec4ℤ = applyMat4CoeffGlobalSumℤ onesCoeffMatℤ

-- § Sum of constant vector is four times its value.
sumFin4-const : (x : ℤ) → sumFin4ℤ (constVec4ℤ x) ≡ fourTimesℤ x
sumFin4-const x = refl

-- § Law 14E.12: J as coefficient matrix.
law14E-12-ones-matrix-is-J : (v : Vec4ℤ) → (i : Fin4) →
  JCoeffGlobalMatVec4ℤ v i ≡ JVec4ℤ v i
law14E-12-ones-matrix-is-J v i = refl

-- § Law 14E.13: constant vectors are 0-eigenvectors.
law14E-13-const-eigen0 : (x : ℤ) → (i : Fin4) →
  laplacianVec4ℤ (constVec4ℤ x) i ≡ 0ℤ
law14E-13-const-eigen0 x i =
  trans
    (law14E-10-laplacian-four-minus-sumAll (constVec4ℤ x) i)
    (trans
      (cong (λ s → fourTimesℤ x +ℤ negℤ s) (sumFin4-const x))
      (+ℤ-inv-right (fourTimesℤ x)))
```

J Operator Properties

J is constant across indices, its sum is $4\Sigma v$, and it scales constant vectors by 4. The identity $J \circ J = 4J$ then shows that the sum-zero condition $\Sigma v = 0$ is equivalent to

membership in the 4-eigenspace of L . These facts provide the spectral raw material for the decomposition $4v = Lv + Jv$.

```
-- § J is constant across indices.
J-constant : (v : Vec4ℤ) → (i j : Fin4) → JVec4ℤ v i ≡ JVec4ℤ v j
J-constant v i j = refl

-- § Sum of J v is four times the sum of v.
sumFin4-J : (v : Vec4ℤ) → sumFin4ℤ (JVec4ℤ v) ≡ fourTimesℤ (sumFin4ℤ v)
sumFin4-J v = refl

-- § J v is definitionally the constant vector at sum v.
J-is-constVec : (v : Vec4ℤ) → (i : Fin4) → JVec4ℤ v i ≡ constVec4ℤ (sumFin4ℤ v) i
J-is-constVec v i = refl

-- § Law 14E.17: J scales constants by 4.
law14E-17-J-const-four : (x : ℤ) → (i : Fin4) →
  JVec4ℤ (constVec4ℤ x) i ≡ fourTimesℤ x
law14E-17-J-const-four x i = sumFin4-const x

-- § Law 14E.18: J ∘ J = 4 · J.
law14E-18-JJ-fourJ : (v : Vec4ℤ) → (i : Fin4) →
  JVec4ℤ (JVec4ℤ v) i ≡ fourTimesℤ (JVec4ℤ v i)
law14E-18-JJ-fourJ v i =
  trans (sumFin4-J v) refl

-- § Law 14E.19: pointwise 4-eigenvectors force sum-zero.
law14E-19-eigen4→sum0 : (v : Vec4ℤ) → ((i : Fin4) → laplacianVec4ℤ v i ≡ fourTimesℤ (v i)) →
  sumFin4ℤ v ≡ 0ℤ
law14E-19-eigen4→sum0 v eigen4 =
  let a = fourTimesℤ (v g0) in
  let s = sumFin4ℤ v in
  let eq0 : a +ℤ negℤ s ≡ a
    eq0 =
      trans
        (sym (law14E-10-laplacian-four-minus-sumAll v g0))
        (eigen4 g0)
  in
  negℤ-zero→zero s (+ℤ-cancel-left a (negℤ s) eq0)

-- § Law 14E.20: sum-zero ↔ pointwise 4-eigenspace.
law14E-20-sum0→eigen4 : (v : Vec4ℤ) → sumFin4ℤ v ≡ 0ℤ → (i : Fin4) →
  laplacianVec4ℤ v i ≡ fourTimesℤ (v i)
law14E-20-sum0→eigen4 v sum0 i = law14E-11-sum0-eigen4 v i sum0

law14E-20-eigen4→sum0 : (v : Vec4ℤ) → ((i : Fin4) → laplacianVec4ℤ v i ≡ fourTimesℤ (v i)) →
  sumFin4ℤ v ≡ 0ℤ
law14E-20-eigen4→sum0 = law14E-19-eigen4→sum0
```

Operator Identity and Kernel

Law 14E.21 establishes $L = 4I - J$ pointwise. From this identity the kernel is forced immediately: the equation $Lv_i = 0$ is equivalent to $4v_i = Jv_i$, so every coordinate

must agree with the same total sum, which is precisely the defining property of a constant vector. Laws 14E.22–14E.27 package this equivalence in both directions as reusable lemmas for the later spectral arguments. In this way the kernel is fixed as the one-dimensional space of constant vectors, exactly as connectivity of K_4 demands.

```
-- § Law 14E.21:  $L = 4I - J$  pointwise.
law14E-21-L-four-minus-J : (v : Vec4 $\mathbb{Z}$ ) → (i : Fin4) →
  laplacianVec4 $\mathbb{Z}$  v i  $\equiv$  fourTimes $\mathbb{Z}$  (v i) + $\mathbb{Z}$  neg $\mathbb{Z}$  (JVec4 $\mathbb{Z}$  v i)
law14E-21-L-four-minus-J v i =
  trans (law14E-10-laplacian-four-minus-sumAll v i) refl

-- § Law 14E.22: kernel condition  $L v i = 0 \leftrightarrow 4 \cdot v_i = J v i$ .
law14E-22-L0→fourEqJ : (v : Vec4 $\mathbb{Z}$ ) → (i : Fin4) → laplacianVec4 $\mathbb{Z}$  v i  $\equiv$  0 $\mathbb{Z}$  →
  fourTimes $\mathbb{Z}$  (v i)  $\equiv$  JVec4 $\mathbb{Z}$  v i
law14E-22-L0→fourEqJ v i L0 =
  let a = fourTimes $\mathbb{Z}$  (v i) in
  let j = JVec4 $\mathbb{Z}$  v i in
  let eq0 : a + $\mathbb{Z}$  neg $\mathbb{Z}$  j  $\equiv$  0 $\mathbb{Z}$ 
    eq0 = trans (sym (law14E-21-L-four-minus-J v i)) L0
  in
  let step1 : (a + $\mathbb{Z}$  neg $\mathbb{Z}$  j) + $\mathbb{Z}$  j  $\equiv$  0 $\mathbb{Z}$  + $\mathbb{Z}$  j
    step1 = cong (λ t → t + $\mathbb{Z}$  j) eq0
    step2 : a + $\mathbb{Z}$  (neg $\mathbb{Z}$  j + $\mathbb{Z}$  j)  $\equiv$  0 $\mathbb{Z}$  + $\mathbb{Z}$  j
    step2 = trans (sym (+ $\mathbb{Z}$ -assoc a (neg $\mathbb{Z}$  j) j)) step1
    step3 : a + $\mathbb{Z}$  0 $\mathbb{Z}$   $\equiv$  0 $\mathbb{Z}$  + $\mathbb{Z}$  j
    step3 = trans (sym (cong (λ t → a + $\mathbb{Z}$  t) (+ $\mathbb{Z}$ -inv-left j))) step2
  in
  trans
    (sym (+ $\mathbb{Z}$ -zero-right a))
    (trans step3 (+ $\mathbb{Z}$ -zero-left j))

law14E-22-fourEqJ→L0 : (v : Vec4 $\mathbb{Z}$ ) → (i : Fin4) → fourTimes $\mathbb{Z}$  (v i)  $\equiv$  JVec4 $\mathbb{Z}$  v i →
  laplacianVec4 $\mathbb{Z}$  v i  $\equiv$  0 $\mathbb{Z}$ 
law14E-22-fourEqJ→L0 v i fourEqJf =
  let a = fourTimes $\mathbb{Z}$  (v i) in
  let j = JVec4 $\mathbb{Z}$  v i in
  trans
    (law14E-21-L-four-minus-J v i)
    (trans
      (cong (λ t → a + $\mathbb{Z}$  t) (cong neg $\mathbb{Z}$  (sym fourEqJf)))
      (+ $\mathbb{Z}$ -inv-right a))

-- § Pointwise vector equality.
Vec4Eq : Vec4 $\mathbb{Z}$  → Vec4 $\mathbb{Z}$  → Set
Vec4Eq v w = (i : Fin4) → v i  $\equiv$  w i

-- § Kernel predicate.
Kernell : Vec4 $\mathbb{Z}$  → Set
Kernell v = (i : Fin4) → laplacianVec4 $\mathbb{Z}$  v i  $\equiv$  0 $\mathbb{Z}$ 

-- § Law 14E.23:  $L = 4I - J$  as Vec4Eq.
law14E-23-L-eq-four-minus-J : (v : Vec4 $\mathbb{Z}$ ) →
  Vec4Eq (laplacianVec4 $\mathbb{Z}$  v) (λ i → fourTimes $\mathbb{Z}$  (v i) + $\mathbb{Z}$  neg $\mathbb{Z}$  (JVec4 $\mathbb{Z}$  v i))
```

```

law14E-23-L-eq-four-minus-J v i = law14E-21-L-four-minus-J v i

-- § Law 14E.24: kernel forces fourTimes-constant.
law14E-24-kernel→fourTimes-constant : (v : Vec4ℤ) → KernelL v → (i j : Fin4) →
  fourTimesℤ (v i) ≡ fourTimesℤ (v j)
law14E-24-kernel→fourTimes-constant v ker i j =
  let fi = law14E-22-L0→fourEqJ v i (ker i) in
  let fj = law14E-22-L0→fourEqJ v j (ker j) in
  trans fi (trans refl (sym fj))

-- § Law 14E.25: kernel ↔ fourEqJ pointwise.
law14E-25-kernel→fourEqJ : (v : Vec4ℤ) → KernelL v → (i : Fin4) →
  fourTimesℤ (v i) ≡ JVec4ℤ v i
law14E-25-kernel→fourEqJ v ker i = law14E-22-L0→fourEqJ v i (ker i)

law14E-25-fourEqJ→kernel : (v : Vec4ℤ) → ((i : Fin4) → fourTimesℤ (v i) ≡ JVec4ℤ v i) → KernelL v
law14E-25-fourEqJ→kernel v hyp i = law14E-22-fourEqJ→L0 v i (hyp i)

-- § Law 14E.26: kernel forces sum = 4·v_i.
law14E-26-kernel→sumEqFour : (v : Vec4ℤ) → KernelL v → (i : Fin4) →
  sumFin4ℤ v ≡ fourTimesℤ (v i)
law14E-26-kernel→sumEqFour v ker i =
  trans
    refl
    (trans
      (sym (law14E-25-kernel→fourEqJ v ker i))
      refl)

-- § Law 14E.27: sum = 4·v_i forces kernel.
law14E-27-sumEqFour→kernel : (v : Vec4ℤ) → ((i : Fin4) → sumFin4ℤ v ≡ fourTimesℤ (v i)) → KernelL v
law14E-27-sumEqFour→kernel v hyp =
  law14E-25-fourEqJ→kernel v (λ i → sym (trans refl (hyp i)))

-- § Law 14E.14: J is constant.
law14E-14-J-constant : (v : Vec4ℤ) → (i j : Fin4) →
  JVec4ℤ v i ≡ JVec4ℤ v j
law14E-14-J-constant = J-constant

-- § Law 14E.15: sum-zero ↔ J v = 0.
law14E-15-sum0-to-J0 : (v : Vec4ℤ) → (i : Fin4) → sumFin4ℤ v ≡ 0ℤ →
  JVec4ℤ v i ≡ 0ℤ
law14E-15-sum0-to-J0 v i sum0 = sum0

law14E-15-J0-to-sum0 : (v : Vec4ℤ) → JVec4ℤ v g0 ≡ 0ℤ →
  sumFin4ℤ v ≡ 0ℤ
law14E-15-J0-to-sum0 v j0 = j0

-- § Law 14E.16: L ∘ J = 0.
law14E-16-LJ-zero : (v : Vec4ℤ) → (i : Fin4) →
  laplacianVec4ℤ (JVec4ℤ v) i ≡ 0ℤ
law14E-16-LJ-zero v i =
  let s = sumFin4ℤ v in
  trans
    (law14E-10-laplacian-four-minus-sumAll (JVec4ℤ v) i)
    (trans
      (cong (λ t → fourTimesℤ s + ℤ negℤ t) (sumFin4-J v))
      (+ℤ-inv-right (fourTimesℤ s)))

```

Image of the Laplacian

The formula $L = 4I - J$ also controls the image of the Laplacian. Since J records the global sum, every vector of the form Lv must have total sum zero. The following calculations verify that claim directly and then use it to obtain the basic operator algebra.

```
-- § Add constant to each component.
sumFin4-addConst : (v : Vec4 $\mathbb{Z}$ )  $\rightarrow$  (c :  $\mathbb{Z}$ )  $\rightarrow$ 
sumFin4 $\mathbb{Z}$  ( $\lambda$  i  $\rightarrow$  v i +  $\mathbb{Z}$  c)  $\equiv$  sumFin4 $\mathbb{Z}$  v +  $\mathbb{Z}$  fourTimes $\mathbb{Z}$  c
sumFin4-addConst v c =
let
  a0 = v g0
  a1 = v g1
  a2 = v g2
  a3 = v g3
  r23 = (a2 +  $\mathbb{Z}$  c) +  $\mathbb{Z}$  (a3 +  $\mathbb{Z}$  c)
  r1 = (a1 +  $\mathbb{Z}$  c) +  $\mathbb{Z}$  r23

  step1 : (a0 +  $\mathbb{Z}$  c) +  $\mathbb{Z}$  r1  $\equiv$  a0 +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  r1)
  step1 = + $\mathbb{Z}$ -assoc a0 c r1

  step2 : r1  $\equiv$  a1 +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  r23)
  step2 = + $\mathbb{Z}$ -assoc a1 c r23

  step3 : c +  $\mathbb{Z}$  r1  $\equiv$  a1 +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  r23))
  step3 =
  trans
    (cong ( $\lambda$  t  $\rightarrow$  c +  $\mathbb{Z}$  t) step2)
    (swapHead $\mathbb{Z}$  c a1 (c +  $\mathbb{Z}$  r23))

  step4 : (a0 +  $\mathbb{Z}$  c) +  $\mathbb{Z}$  r1  $\equiv$  a0 +  $\mathbb{Z}$  (a1 +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  r23)))
  step4 = trans step1 (cong ( $\lambda$  t  $\rightarrow$  a0 +  $\mathbb{Z}$  t) step3)

  step5a : r23  $\equiv$  a2 +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (a3 +  $\mathbb{Z}$  c))
  step5a = + $\mathbb{Z}$ -assoc a2 c (a3 +  $\mathbb{Z}$  c)

  step5b : c +  $\mathbb{Z}$  r23  $\equiv$  a2 +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (a3 +  $\mathbb{Z}$  c)))
  step5b =
  trans
    (cong ( $\lambda$  t  $\rightarrow$  c +  $\mathbb{Z}$  t) step5a)
    (swapHead $\mathbb{Z}$  c a2 (c +  $\mathbb{Z}$  (a3 +  $\mathbb{Z}$  c)))

  step5c : c +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  r23)  $\equiv$  a2 +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (a3 +  $\mathbb{Z}$  c))))
  step5c =
  trans
    (cong ( $\lambda$  t  $\rightarrow$  c +  $\mathbb{Z}$  t) step5b)
    (swapHead $\mathbb{Z}$  c a2 (c +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (a3 +  $\mathbb{Z}$  c))))

  step6 : a0 +  $\mathbb{Z}$  (a1 +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  r23)))  $\equiv$  a0 +  $\mathbb{Z}$  (a1 +  $\mathbb{Z}$  (a2 +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  (a3 +  $\mathbb{Z}$  c))))))
  step6 = cong ( $\lambda$  t  $\rightarrow$  a0 +  $\mathbb{Z}$  (a1 +  $\mathbb{Z}$  t)) step5c

  step7a : c +  $\mathbb{Z}$  (a3 +  $\mathbb{Z}$  c)  $\equiv$  a3 +  $\mathbb{Z}$  (c +  $\mathbb{Z}$  c)
```



```

step7 a = swapHeadℤ c a3 c

step7 b : c + ℤ (c + ℤ (a3 + ℤ c)) ≡ a3 + ℤ (c + ℤ (c + ℤ c))
step7 b =
  trans
    (cong (λ t → c + ℤ t) step7 a)
    (swapHeadℤ c a3 (c + ℤ c))

step7 c : c + ℤ (c + ℤ (c + ℤ (a3 + ℤ c))) ≡ a3 + ℤ (c + ℤ (c + ℤ (c + ℤ c)))
step7 c =
  trans
    (cong (λ t → c + ℤ t) step7 b)
    (swapHeadℤ c a3 (c + ℤ (c + ℤ c)))

step8 : c + ℤ (c + ℤ (c + ℤ (a3 + ℤ c))) ≡ a3 + ℤ fourTimesℤ c
step8 = trans step7 c refl

step9 : a0 + ℤ (a1 + ℤ (a2 + ℤ (c + ℤ (c + ℤ (c + ℤ (a3 + ℤ c)))))) ≡ a0 + ℤ (a1 + ℤ (a2 + ℤ (a3 + ℤ fourTimesℤ c)))
step9 = cong (λ t → a0 + ℤ (a1 + ℤ (a2 + ℤ t))) step8

step10 a : a2 + ℤ (a3 + ℤ fourTimesℤ c) ≡ (a2 + ℤ a3) + ℤ fourTimesℤ c
step10 a = sym (+ℤ-assoc a2 a3 (fourTimesℤ c))

step10 b : a1 + ℤ (a2 + ℤ (a3 + ℤ fourTimesℤ c)) ≡ (a1 + ℤ (a2 + ℤ a3)) + ℤ fourTimesℤ c
step10 b =
  trans
    (cong (λ t → a1 + ℤ t) step10 a)
    (sym (+ℤ-assoc a1 (a2 + ℤ a3) (fourTimesℤ c)))

step10 c : a0 + ℤ (a1 + ℤ (a2 + ℤ (a3 + ℤ fourTimesℤ c))) ≡ (a0 + ℤ (a1 + ℤ (a2 + ℤ a3))) + ℤ fourTimesℤ c
step10 c =
  trans
    (cong (λ t → a0 + ℤ t) step10 b)
    (sym (+ℤ-assoc a0 (a1 + ℤ (a2 + ℤ a3)) (fourTimesℤ c)))
in
  trans
    refl
    (trans
      step4
      (trans
        step6
        (trans
          step9
          (trans
            step10 c
            refl))))))

```

Step 2: Distribution and Global Sum. To complete this computation one needs only routine bookkeeping: `fourTimesℤ` must distribute over addition, and finite sums must commute with pointwise addition of vectors.

```

-- § fourTimes distributes over +ℤ.
fourTimes-+ℤ : (x y : ℤ) → fourTimesℤ (x + ℤ y) ≡ fourTimesℤ x + ℤ fourTimesℤ y

```

```

fourTimes+ $\mathbb{Z}$  x y =
trans
  (sym (sumFin4-const (x + $\mathbb{Z}$  y)))
  (trans
    (sumFin4-addConst (constVec4 $\mathbb{Z}$  x) y)
    (trans
      (cong ( $\lambda$  t  $\rightarrow$  t + $\mathbb{Z}$  fourTimes $\mathbb{Z}$  y) (sumFin4-const x))
      refl))

-- § Sum of fourTimes equals fourTimes of sum.
sumFin4-fourTimes : (v : Vec4 $\mathbb{Z}$ )  $\rightarrow$ 
sumFin4 $\mathbb{Z}$  ( $\lambda$  i  $\rightarrow$  fourTimes $\mathbb{Z}$  (v i))  $\equiv$  fourTimes $\mathbb{Z}$  (sumFin4 $\mathbb{Z}$  v)
sumFin4-fourTimes v =
let
  a0 = v g0
  a1 = v g1
  a2 = v g2
  a3 = v g3
in
sym
  (trans
    refl
    (trans
      (fourTimes+ $\mathbb{Z}$  a0 (a1 + $\mathbb{Z}$  (a2 + $\mathbb{Z}$  a3)))
      (trans
        (cong ( $\lambda$  t  $\rightarrow$  fourTimes $\mathbb{Z}$  a0 + $\mathbb{Z}$  t) (fourTimes+ $\mathbb{Z}$  a1 (a2 + $\mathbb{Z}$  a3)))
        (trans
          (cong ( $\lambda$  t  $\rightarrow$  fourTimes $\mathbb{Z}$  a0 + $\mathbb{Z}$  (fourTimes $\mathbb{Z}$  a1 + $\mathbb{Z}$  t)) (fourTimes+ $\mathbb{Z}$  a2 a3))
          refl))))

-- § Law 14E.28: global sum of the Laplacian is zero.
law14E-28-sumLaplacian0 : (v : Vec4 $\mathbb{Z}$ )  $\rightarrow$ 
sumFin4 $\mathbb{Z}$  (laplacianVec4 $\mathbb{Z}$  v)  $\equiv$  0 $\mathbb{Z}$ 
law14E-28-sumLaplacian0 v =
let
  s = sumFin4 $\mathbb{Z}$  v

  step0 :
  sumFin4 $\mathbb{Z}$  (laplacianVec4 $\mathbb{Z}$  v)  $\equiv$ 
  sum4 $\mathbb{Z}$  (fourTimes $\mathbb{Z}$  (v g0) + $\mathbb{Z}$  neg $\mathbb{Z}$  s) (laplacianVec4 $\mathbb{Z}$  v g1) (laplacianVec4 $\mathbb{Z}$  v g2) (laplacianVec4 $\mathbb{Z}$  v g3)
  step0 =
  cong
    ( $\lambda$  t0  $\rightarrow$  sum4 $\mathbb{Z}$  t0 (laplacianVec4 $\mathbb{Z}$  v g1) (laplacianVec4 $\mathbb{Z}$  v g2) (laplacianVec4 $\mathbb{Z}$  v g3))
    (law14E-10-laplacian-four-minus-sumAll v g0)

  step1 :
  sum4 $\mathbb{Z}$  (fourTimes $\mathbb{Z}$  (v g0) + $\mathbb{Z}$  neg $\mathbb{Z}$  s) (laplacianVec4 $\mathbb{Z}$  v g1) (laplacianVec4 $\mathbb{Z}$  v g2) (laplacianVec4 $\mathbb{Z}$  v g3)  $\equiv$ 
  sum4 $\mathbb{Z}$  (fourTimes $\mathbb{Z}$  (v g0) + $\mathbb{Z}$  neg $\mathbb{Z}$  s) (fourTimes $\mathbb{Z}$  (v g1) + $\mathbb{Z}$  neg $\mathbb{Z}$  s) (laplacianVec4 $\mathbb{Z}$  v g2) (laplacianVec4 $\mathbb{Z}$  v g3)
  step1 =
  cong
    ( $\lambda$  t1  $\rightarrow$  sum4 $\mathbb{Z}$  (fourTimes $\mathbb{Z}$  (v g0) + $\mathbb{Z}$  neg $\mathbb{Z}$  s) t1 (laplacianVec4 $\mathbb{Z}$  v g2) (laplacianVec4 $\mathbb{Z}$  v g3))
    (law14E-10-laplacian-four-minus-sumAll v g1)

  step2 :

```

```

sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) (laplacianVec4ℤ v g2) (laplacianVec4ℤ v g3) ≡
sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) (fourTimesℤ (v g2) +ℤ negℤ s) (laplacianVec4ℤ v g3)
step2 =
cong
(λ t2 → sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) t2 (laplacianVec4ℤ v g3))
(law14E-10-laplacian-four-minus-sumAll v g2)

step3 :
sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) (fourTimesℤ (v g2) +ℤ negℤ s) (laplacianVec4ℤ v g3) ≡
sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) (fourTimesℤ (v g2) +ℤ negℤ s) (fourTimesℤ (v g3) +ℤ negℤ s)
step3 =
cong
(λ t3 → sum4ℤ (fourTimesℤ (v g0) +ℤ negℤ s) (fourTimesℤ (v g1) +ℤ negℤ s) (fourTimesℤ (v g2) +ℤ negℤ s) t3)
(law14E-10-laplacian-four-minus-sumAll v g3)

rewriteSum :
sumFin4ℤ (laplacianVec4ℤ v) ≡
sumFin4ℤ (λ i → fourTimesℤ (v i) +ℤ negℤ s)
rewriteSum =
trans
refl
(trans step0 (trans step1 (trans step2 (trans step3 refl))))
in
trans
rewriteSum
(trans
(sumFin4-addConst (λ i → fourTimesℤ (v i)) (negℤ s))
(trans
(cong (λ t → t +ℤ fourTimesℤ (negℤ s)) (sumFin4-fourTimes v))
(trans
(cong (λ t → fourTimesℤ s +ℤ t) (sym (neg-fourTimesℤ s)))
(+ℤ-inv-right (fourTimesℤ s))))))

```

Minimal Polynomial and Operator Algebra

Once the image of L is known to have sum zero, the operator identities become formal consequences. Applying L twice produces the eigenvalue 4 on the image, while composing with J annihilates it. Together with $L + J = 4I$, these formulas show that the spectrum has already rigidly split into the two expected parts.

```

-- § Law 14E.29: L ∘ L = 4 · L (minimal polynomial x(x-4) = 0).
law14E-29-LL-fourL : (v : Vec4ℤ) → (i : Fin4) →
laplacianVec4ℤ (laplacianVec4ℤ v) i ≡ fourTimesℤ (laplacianVec4ℤ v i)
law14E-29-LL-fourL v i =
law14E-11-sum0-eigen4 (laplacianVec4ℤ v) i (law14E-28-sumLaplacian0 v)

-- § Law 14E.30: J ∘ L = 0.
law14E-30-JL-zero : (v : Vec4ℤ) → (i : Fin4) →
JVec4ℤ (laplacianVec4ℤ v) i ≡ 0ℤ
law14E-30-JL-zero v i =

```

```

law14E-15-sum0-to-J0 (laplacianVec4ℤ v) i (law14E-28-sumLaplacian0 v)

-- § Law 14E.31: L + J = 4I pointwise.
law14E-31-L-plus-J-eq-fourl : (v : Vec4ℤ) → (i : Fin4) →
  laplacianVec4ℤ v i + ℤ JVec4ℤ v i ≡ fourTimesℤ (v i)
law14E-31-L-plus-J-eq-fourl v i =
  let a = fourTimesℤ (v i) in
  let j = JVec4ℤ v i in
  trans
    (cong (λ t → t + ℤ j) (law14E-21-L-four-minus-J v i))
    (trans
      (+ℤ-assoc a (negℤ j) j))
    (trans
      (cong (λ t → a + ℤ t) (+ℤ-inv-left j))
      (+ℤ-zero-right a)))

-- § Zero vector.
zeroVec4ℤ : Vec4ℤ
zeroVec4ℤ = constVec4ℤ 0ℤ

```

Vec4Eq Packagings

The preceding pointwise identities are now repackaged as Vec4Eq statements. This does not add new mathematics; it merely records the operator laws in the form needed for the canonical decomposition

$$4v = Lv + Jv.$$

```

-- § Law 14E.32: L + J = 4I as Vec4Eq.
law14E-32-LplusJ-eq-fourl-Vec4Eq : (v : Vec4ℤ) →
  Vec4Eq (λ i → laplacianVec4ℤ v i + ℤ JVec4ℤ v i) (λ i → fourTimesℤ (v i))
law14E-32-LplusJ-eq-fourl-Vec4Eq v i = law14E-31-L-plus-J-eq-fourl v i

-- § Law 14E.33: L ∘ J = 0 as Vec4Eq.
law14E-33-LJ-zero-Vec4Eq : (v : Vec4ℤ) →
  Vec4Eq (laplacianVec4ℤ (JVec4ℤ v)) zeroVec4ℤ
law14E-33-LJ-zero-Vec4Eq v i = law14E-16-LJ-zero v i

-- § Law 14E.34: J ∘ L = 0 as Vec4Eq.
law14E-34-JL-zero-Vec4Eq : (v : Vec4ℤ) →
  Vec4Eq (JVec4ℤ (laplacianVec4ℤ v)) zeroVec4ℤ
law14E-34-JL-zero-Vec4Eq v i = law14E-30-JL-zero v i

-- § Law 14E.35: L and J commute (both composites = 0).
law14E-35-LJ-commute : (v : Vec4ℤ) →
  Vec4Eq (laplacianVec4ℤ (JVec4ℤ v)) (JVec4ℤ (laplacianVec4ℤ v))
law14E-35-LJ-commute v i =
  trans (law14E-16-LJ-zero v i) (sym (law14E-30-JL-zero v i))

-- § Law 14E.36: L2 = 4L as Vec4Eq.

```

```

law14E-36-LL-fourL-Vec4Eq : (v : Vec4ℤ) →
  Vec4Eq (laplacianVec4ℤ (laplacianVec4ℤ v)) (λ i → fourTimesℤ (laplacianVec4ℤ v i))
law14E-36-LL-fourL-Vec4Eq v i = law14E-29-LL-fourL v i

-- § Law 14E.37: J2 = 4J as Vec4Eq.
law14E-37-JJ-fourJ-Vec4Eq : (v : Vec4ℤ) →
  Vec4Eq (JVec4ℤ (JVec4ℤ v)) (λ i → fourTimesℤ (JVec4ℤ v i))
law14E-37-JJ-fourJ-Vec4Eq v i = law14E-18-JJ-fourJ v i

-- § Pointwise four-times and pointwise addition.
fourVec4ℤ : Vec4ℤ → Vec4ℤ
fourVec4ℤ v i = fourTimesℤ (v i)

_+Vec4ℤ_ : Vec4ℤ → Vec4ℤ → Vec4ℤ
(v +Vec4ℤ w) i = v i +ℤ w i

-- § Law 14E.38: image of L is sum-zero and 4-eigen.
law14E-38-imageL-sum0-and-eigen4 : (v : Vec4ℤ) →
  (sumFin4ℤ (laplacianVec4ℤ v) ≡ 0ℤ) × ((i : Fin4) → laplacianVec4ℤ (laplacianVec4ℤ v) i ≡ fourTimesℤ (laplacianVec4ℤ v i))
law14E-38-imageL-sum0-and-eigen4 v =
  law14E-28-sumLaplacian0 v , law14E-29-LL-fourL v

-- § Law 14E.39: image of J is constant and in kernel of L.
law14E-39-imageJ-const-and-kernelL : (v : Vec4ℤ) →
  (((i j : Fin4) → JVec4ℤ v i ≡ JVec4ℤ v j) × ((i : Fin4) → laplacianVec4ℤ (JVec4ℤ v) i ≡ 0ℤ))
law14E-39-imageJ-const-and-kernelL v =
  law14E-14-J-constant v , law14E-16-LJ-zero v

-- § Decomposition type: 4v = u + w with u sum-zero, w constant.
Decomp4 : Vec4ℤ → Set
Decomp4 v =
  Σ Vec4ℤ (λ u →
    Σ Vec4ℤ (λ w →
      (Vec4Eq (u +Vec4ℤ w) (fourVec4ℤ v)) ×
      (sumFin4ℤ u ≡ 0ℤ) ×
      ((i j : Fin4) → w i ≡ w j)))

-- § Law 14E.40: canonical decomposition 4v = Lv + Jv.
law14E-40-decomp4-canonical : (v : Vec4ℤ) → Decomp4 v
law14E-40-decomp4-canonical v =
  laplacianVec4ℤ v ,
  (JVec4ℤ v ,
    (law14E-32-LplusJ-eq-fourL-Vec4Eq v ,
      (law14E-28-sumLaplacian0 v ,
        law14E-14-J-constant v)))

```

Finite spectrum code. The familiar shorthand $\text{Spec}(L_{K_4}) = \{0, 4, 4, 4\}$ must not float as a slogan above the operator proof. It is a finite code with one constant slot and three sum-zero slots, tied directly to the laws just proved: constants are annihilated by L , sum-zero vectors are multiplied by 4, the reverse implication forces sum-zero, and the minimal polynomial is $x(x - 4)$.

```

-- § Finite spectrum code for the K4 Laplacian: {0,4,4,4}.
data K4SpectrumSlot : Set where
  spectrum-zero : K4SpectrumSlot
  spectrum-four1 : K4SpectrumSlot
  spectrum-four2 : K4SpectrumSlot
  spectrum-four3 : K4SpectrumSlot

-- § Type-level closure: K4SpectrumSlot has exactly four inhabitants.
K4SpectrumSlot-exhaustive :
  ∀ (s : K4SpectrumSlot)
    → (s ≡ spectrum-zero)
    ⊔ (s ≡ spectrum-four1)
    ⊔ (s ≡ spectrum-four2)
    ⊔ (s ≡ spectrum-four3)
K4SpectrumSlot-exhaustive spectrum-zero = inj1 refl
K4SpectrumSlot-exhaustive spectrum-four1 = inj2 (inj1 refl)
K4SpectrumSlot-exhaustive spectrum-four2 = inj2 (inj2 (inj1 refl))
K4SpectrumSlot-exhaustive spectrum-four3 = inj2 (inj2 (inj2 refl))

K4-spectrum-value : K4SpectrumSlot → ℕ
K4-spectrum-value spectrum-zero = 0
K4-spectrum-value spectrum-four1 = 4
K4-spectrum-value spectrum-four2 = 4
K4-spectrum-value spectrum-four3 = 4

record K4LaplacianSpectrumRealizer : Set where
  field
    -- § The displayed spectrum code is exactly {0,4,4,4}.
    zero-slot-value : K4-spectrum-value spectrum-zero ≡ 0
    four1-slot-value : K4-spectrum-value spectrum-four1 ≡ 4
    four2-slot-value : K4-spectrum-value spectrum-four2 ≡ 4
    four3-slot-value : K4-spectrum-value spectrum-four3 ≡ 4
    slot-count-is-V : 4 ≡ simplex-vertices
    zero-count-is-1 : simplex-vertices - simplex-degree ≡ 1
    four-count-is-d : simplex-degree ≡ 3
    -- § Operator content behind the code.
    operator-form : (v : Vec4ℤ) → (i : Fin4) →
      laplacianVec4ℤ v i ≡ fourTimesℤ (v i) + ℤ negℤ (JVec4ℤ v i)
    constant-zero : (x : ℤ) → (i : Fin4) →
      laplacianVec4ℤ (constVec4ℤ x) i ≡ 0ℤ
    sum-zero-four : (v : Vec4ℤ) → sumFin4ℤ v ≡ 0ℤ → (i : Fin4) →
      laplacianVec4ℤ v i ≡ fourTimesℤ (v i)
    four-forces-sum0 : (v : Vec4ℤ) →
      ((i : Fin4) → laplacianVec4ℤ v i ≡ fourTimesℤ (v i)) →
      sumFin4ℤ v ≡ 0ℤ
    minimal-polynomial : (v : Vec4ℤ) →
      Vec4Eq (laplacianVec4ℤ (laplacianVec4ℤ v))
        (λ i → fourTimesℤ (laplacianVec4ℤ v i))
    canonical-split : (v : Vec4ℤ) → Decomp4 v

theorem-k4-laplacian-spectrum-realizer : K4LaplacianSpectrumRealizer
theorem-k4-laplacian-spectrum-realizer = record
  { zero-slot-value = refl
  ; four1-slot-value = refl

```

```

; four2-slot-value = refl
; four3-slot-value = refl
; slot-count-is-V   = refl
; zero-count-is-1   = refl
; four-count-is-d   = law14C-1-degree
; operator-form     = law14E-21-L-four-minus-J
; constant-zero     = law14E-13-const-eigen0
; sum-zero-four     = law14E-20-sum0→eigen4
; four-forces-sum0  = law14E-20-eigen4→sum0
; minimal-polynomial = law14E-36-LL-fourL-Vec4Eq
; canonical-split   = law14E-40-decomp4-canonical
}

```

Additivity of L and J

This section establishes the linearity of the two basic operators on $\text{Vec4}\mathbb{Z}$. The proof is elementary but layered: first the auxiliary sums over three and four indices are shown to distribute over pointwise addition, then these identities are substituted into the definitions of J and L . In this form, additivity of J is immediate because it is the total sum repeated in each coordinate, while additivity of L follows by the same distributive bookkeeping together with the linearity of $\text{threeTimes}\mathbb{Z}$ and negation.

```

-- § sumFin3 distributes over pointwise addition.
sumFin3+ $\mathbb{Z}$ : (f g : Fin3 →  $\mathbb{Z}$ ) →
sumFin3 $\mathbb{Z}$  (λ k → f k + $\mathbb{Z}$  g k) ≡
sumFin3 $\mathbb{Z}$  f + $\mathbb{Z}$  sumFin3 $\mathbb{Z}$  g
sumFin3+ $\mathbb{Z}$  f g =
let
  a0 = f f0
  a1 = f f1
  a2 = f f2
  b0 = g f0
  b1 = g f1
  b2 = g f2

  X = a1 + $\mathbb{Z}$  b1
  Y = a2 + $\mathbb{Z}$  b2
  R = b0 + $\mathbb{Z}$  b2

  step1 : sumFin3 $\mathbb{Z}$  (λ k → f k + $\mathbb{Z}$  g k) ≡ a0 + $\mathbb{Z}$  (b0 + $\mathbb{Z}$  (X + $\mathbb{Z}$  Y))
  step1 = + $\mathbb{Z}$ -assoc a0 b0 (X + $\mathbb{Z}$  Y)

  step2 : a0 + $\mathbb{Z}$  (b0 + $\mathbb{Z}$  (X + $\mathbb{Z}$  Y)) ≡ a0 + $\mathbb{Z}$  (X + $\mathbb{Z}$  (b0 + $\mathbb{Z}$  Y))
  step2 = cong (λ t → a0 + $\mathbb{Z}$  t) (swapHead $\mathbb{Z}$  b0 X Y)

  step3 : a0 + $\mathbb{Z}$  (X + $\mathbb{Z}$  (b0 + $\mathbb{Z}$  Y)) ≡ a0 + $\mathbb{Z}$  (X + $\mathbb{Z}$  (a2 + $\mathbb{Z}$  R))
  step3 = cong (λ t → a0 + $\mathbb{Z}$  (X + $\mathbb{Z}$  t)) (swapHead $\mathbb{Z}$  b0 a2 b2)

  step4 : a0 + $\mathbb{Z}$  (X + $\mathbb{Z}$  (a2 + $\mathbb{Z}$  R)) ≡ a0 + $\mathbb{Z}$  (a1 + $\mathbb{Z}$  (b1 + $\mathbb{Z}$  (a2 + $\mathbb{Z}$  R)))
  step4 = cong (λ t → a0 + $\mathbb{Z}$  t) (+ $\mathbb{Z}$ -assoc a1 b1 (a2 + $\mathbb{Z}$  R))

```

```

step5 : a0 + ℤ (a1 + ℤ (b1 + ℤ (a2 + ℤ R))) ≡ a0 + ℤ (a1 + ℤ (a2 + ℤ (b1 + ℤ R)))
step5 = cong (λ t → a0 + ℤ (a1 + ℤ t)) (swapHeadℤ b1 a2 R)

step6 : a0 + ℤ (a1 + ℤ (a2 + ℤ (b1 + ℤ R))) ≡ a0 + ℤ ((a1 + ℤ a2) + ℤ (b1 + ℤ R))
step6 = cong (λ t → a0 + ℤ t) (sym (+ℤ-assoc a1 a2 (b1 + ℤ R)))

step7 : a0 + ℤ ((a1 + ℤ a2) + ℤ (b1 + ℤ R)) ≡ a0 + ℤ ((a1 + ℤ a2) + ℤ (b0 + ℤ (b1 + ℤ b2)))
step7 = cong (λ t → a0 + ℤ ((a1 + ℤ a2) + ℤ t)) (swapHeadℤ b1 b0 b2)

step8 : a0 + ℤ ((a1 + ℤ a2) + ℤ (b0 + ℤ (b1 + ℤ b2))) ≡ (a0 + ℤ (a1 + ℤ a2)) + ℤ (b0 + ℤ (b1 + ℤ b2))
step8 = sym (+ℤ-assoc a0 (a1 + ℤ a2) (b0 + ℤ (b1 + ℤ b2)))
in
trans
refl
(trans step1
 (trans step2
  (trans step3
   (trans step4
    (trans step5
     (trans step6
      (trans step7
       (trans step8 refl))))))))))

-- § sumOthers distributes over pointwise addition.
sumOthers+Vec4ℤ : (v w : Vec4ℤ) → (i : Fin4) →
sumOthersℤ (v + Vec4ℤ w) i ≡ sumOthersℤ v i + ℤ sumOthersℤ w i
sumOthers+Vec4ℤ v w i =
sumFin3+ℤ (λ k → v (others i k)) (λ k → w (others i k))

```

Step 2: Vec4ℤ and Laplacian Distribution. The passage from three-term sums to four-component vectors is the only additional bookkeeping required here. One first proves that `sumFin4ℤ` respects pointwise addition on `Vec4ℤ`; once that is done, the formulas for $J(u + v)$ and $L(u + v)$ reduce to direct rearrangements of their defining sums.

```

-- § sumFin4 distributes over pointwise addition.
sumFin4+Vec4ℤ : (v w : Vec4ℤ) →
sumFin4ℤ (λ i → v i + ℤ w i) ≡ sumFin4ℤ v + ℤ sumFin4ℤ w
sumFin4+Vec4ℤ v w =
let
split0 : (x : Vec4ℤ) → sumFin4ℤ x ≡ x g0 + ℤ sumOthersℤ x g0
split0 x =
trans
(sym (law14E-4-sumFin4Around-eq-sumFin4 x g0))
(sumFin4Around-split x g0)

v0 = v g0
w0 = w g0
sv = sumOthersℤ v g0
sw = sumOthersℤ w g0

```



```

step1 : sumFin4ℤ (v + Vec4ℤ w) ≡ (v0 + ℤ w0) + ℤ sumOthersℤ (v + Vec4ℤ w) g0
step1 = trans (split0 (v + Vec4ℤ w)) refl

step2 : (v0 + ℤ w0) + ℤ sumOthersℤ (v + Vec4ℤ w) g0 ≡ (v0 + ℤ w0) + ℤ (sv + ℤ sw)
step2 = cong (λ t → (v0 + ℤ w0) + ℤ t) (sumOthers+Vec4ℤ v w g0)

step3 : (v0 + ℤ w0) + ℤ (sv + ℤ sw) ≡ (v0 + ℤ sv) + ℤ (w0 + ℤ sw)
step3 =
  trans
    (+ℤ-assoc v0 w0 (sv + ℤ sw))
    (trans
      (cong (λ t → v0 + ℤ t) (swapHeadℤ w0 sv sw))
      (sym (+ℤ-assoc v0 sv (w0 + ℤ sw))))
  in
  trans
    (trans refl step1)
    (trans
      step2
      (trans
        step3
        (trans
          (cong (λ t → t + ℤ (w0 + ℤ sw)) (sym (split0 v)))
          (cong (λ t → sumFin4ℤ v + ℤ t) (sym (split0 w)))))))

-- § Law 14E.41: J preserves pointwise addition.
law14E-41-J-add : (v w : Vec4ℤ) → (i : Fin4) →
  JVec4ℤ (v + Vec4ℤ w) i ≡ JVec4ℤ v i + ℤ JVec4ℤ w i
law14E-41-J-add v w i = sumFin4+Vec4ℤ v w

-- § threeTimes distributes over +ℤ.
threeTimes+ℤ : (x y : ℤ) → threeTimesℤ (x + ℤ y) ≡ threeTimesℤ x + ℤ threeTimesℤ y
threeTimes+ℤ x y =
  sumFin3+ℤ (λ _ → x) (λ _ → y)

-- § Law 14E.42: L preserves pointwise addition.
law14E-42-L-add : (v w : Vec4ℤ) → (i : Fin4) →
  laplacianVec4ℤ (v + Vec4ℤ w) i ≡ laplacianVec4ℤ v i + ℤ laplacianVec4ℤ w i
law14E-42-L-add v w i =
  let
    A = threeTimesℤ (v i)
    B = threeTimesℤ (w i)
    C = negℤ (sumOthersℤ v i)
    D = negℤ (sumOthersℤ w i)
  in
  step1 : laplacianVec4ℤ (v + Vec4ℤ w) i ≡ (A + ℤ B) + ℤ negℤ (sumOthersℤ (v + Vec4ℤ w) i)
  step1 = cong (λ t → t + ℤ negℤ (sumOthersℤ (v + Vec4ℤ w) i)) (threeTimes+ℤ (v i) (w i))

  step2 : negℤ (sumOthersℤ (v + Vec4ℤ w) i) ≡ C + ℤ D
  step2 =
    trans
      (cong negℤ (sumOthers+Vec4ℤ v w i))
      (neg+ℤ (sumOthersℤ v i) (sumOthersℤ w i))

```

```

step3 : (A + $\mathbb{Z}$  B) + $\mathbb{Z}$  (C + $\mathbb{Z}$  D)  $\equiv$  (A + $\mathbb{Z}$  C) + $\mathbb{Z}$  (B + $\mathbb{Z}$  D)
step3 =
  trans
    (+ $\mathbb{Z}$ -assoc A B (C + $\mathbb{Z}$  D))
    (trans
      (cong ( $\lambda$  t  $\rightarrow$  A + $\mathbb{Z}$  t) (swapHead $\mathbb{Z}$  B C D))
      (sym (+ $\mathbb{Z}$ -assoc A C (B + $\mathbb{Z}$  D))))
  in
  trans
    step1
    (trans
      (cong ( $\lambda$  t  $\rightarrow$  (A + $\mathbb{Z}$  B) + $\mathbb{Z}$  t) step2)
      (trans
        step3
        refl))

```

Chapter 42

Measurement Symmetries

The next chapter studies the symmetries of measurements from an arbitrary distinction into the canonical two-point distinction. There are two unavoidable operations: one swaps the two outputs, the other swaps the two inputs by means of the duality already built into every distinction. The point is to determine how these operations act on the four classified measurement types.

Output and Input Swap

These two symmetries are defined in the evident way: either one post-composes a measurement with `swapTwo`, or one pre-composes it with the domain duality `Distinction-dual`. Their compatibility with pointwise equality is immediate.

```
-- § output swap: post-compose with swapTwo
swapOut : {d : Distinction} → Measurement d → Measurement d
swapOut m x = swapTwo (m x)

-- § input swap: pre-compose with the forced dual
swapIn : (d : Distinction) → Measurement d → Measurement d
swapIn d m x = m (Distinction-dual d x)

-- § swapOut respects pointwise equality
swapOut-cong :
  {d : Distinction} {m n : Measurement d} →
  _≐_ {A = S d} {B = S Two-distinction} m n →
  _≐_ {A = S d} {B = S Two-distinction} (swapOut {d = d} m) (swapOut {d = d} n)
swapOut-cong p x = cong swapTwo (p x)

-- § swapIn respects pointwise equality
swapIn-cong :
  (d : Distinction) {m n : Measurement d} →
  _≐_ {A = S d} {B = S Two-distinction} m n →
  _≐_ {A = S d} {B = S Two-distinction} (swapIn d m) (swapIn d n)
swapIn-cong d p x = p (Distinction-dual d x)
```

Induced Permutations on EndoCase

The four-element classification `EndoCase` transforms under each symmetry by a forced permutation. Output swap exchanges constant-left with constant-right and identity with dual. Input swap fixes the constant cases and exchanges identity with dual.

```
-- $ output swap acts on EndoCase
permOut : EndoCase → EndoCase
permOut case-constL = case-constR
permOut case-constR = case-constL
permOut case-id     = case-dual
permOut case-dual   = case-id

-- $ input swap acts on EndoCase
permIn : EndoCase → EndoCase
permIn case-constL = case-constL
permIn case-constR = case-constR
permIn case-id     = case-dual
permIn case-dual   = case-id
```

Interpretation Compatibility

Each symmetry, when applied to an interpreted measurement case, yields the interpreted measurement of the permuted case. For output swap, a direct case-by-case evaluation suffices. Input swap is subtler: the identity and dual cases are exchanged only after transporting across the duality of the domain, so the proof is completed with the uniqueness principle for maps out of a distinction.

```
-- $ swapOut on interpreted cases equals interpretation of permOut
swapOut-interpret :
  (d : Distinction) →
  (c : EndoCase) →
  _≡_ {A = S d} {B = S Two-distinction}
  (swapOut {d = d} (K4Map.interpret d Two-distinction c))
  (K4Map.interpret d Two-distinction (permOut c))
swapOut-interpret d case-constL x = refl
swapOut-interpret d case-constR x = refl
swapOut-interpret d case-id x with cover d x
... | inj1 _ = refl
... | inj2 _ = refl
swapOut-interpret d case-dual x with cover d x
... | inj1 _ = refl
... | inj2 _ = refl

-- $ swapIn on interpreted cases equals interpretation of permIn
swapIn-interpret :
```

```

(d : Distinction) →
(c : EndoCase) →
_≐_ {A = S d} {B = S Two-distinction}
  (swapIn d (K4Map.interpret d Two-distinction c))
  (K4Map.interpret d Two-distinction (permIn c))
swapIn-interpret d case-constL x = refl
swapIn-interpret d case-constR x = refl
swapIn-interpret d case-id =
law7-1-map-determined d Two-distinction
  (swapIn d (K4Map.interpret d Two-distinction case-id))
  (K4Map.interpret d Two-distinction case-dual)
eqℓ
eqr
where
module K = K4Map d Two-distinction
open K

eqℓ : swapIn d (interpret case-id) (ℓ d) ≐ interpret case-dual (ℓ d)
eqℓ =
trans
  (cong (interpret case-id) (Distinction-dual-ℓ d))
  (trans
    (LR-r)
    (sym (RL-ℓ)))

eqr : swapIn d (interpret case-id) (r d) ≐ interpret case-dual (r d)
eqr =
trans
  (cong (interpret case-id) (Distinction-dual-r d))
  (trans
    (LR-ℓ)
    (sym (RL-r)))

swapIn-interpret d case-dual =
law7-1-map-determined d Two-distinction
  (swapIn d (K4Map.interpret d Two-distinction case-dual))
  (K4Map.interpret d Two-distinction case-id)
eqℓ
eqr
where
module K = K4Map d Two-distinction
open K

eqℓ : swapIn d (interpret case-dual) (ℓ d) ≐ interpret case-id (ℓ d)
eqℓ =
trans
  (cong (interpret case-dual) (Distinction-dual-ℓ d))
  (trans
    (RL-r)
    (sym (LR-ℓ)))

eqr : swapIn d (interpret case-dual) (r d) ≐ interpret case-id (r d)
eqr =
trans

```

```

(cong (interpret case-dual) (Distinction-dual-r d))
(trans
 (RL-ℓ)
 (sym (LR-r)))

```

Classification Laws

The symmetry operations on measurements must induce matching symmetries on their classification labels. The next two laws state exactly that compatibility for output swap and input swap.

Law 15C.0: Classification Respects Outcome Swap

This law says that classifying after swapping outputs yields exactly the same result as first classifying and then applying the induced permutation `permOut` on `EndoCase`.

```

-- § classify commutes with output swap
law15C-0-classify-swapOut :
  (d : Distinction) →
  (m : Measurement d) →
  K4Map.classify d Two-distinction (swapOut {d = d} m)
  ≡ permOut (K4Map.classify d Two-distinction m)
law15C-0-classify-swapOut d m =
  sym
  (K.classify-unique
   (swapOut {d = d} m)
   (permOut (K.classify m))
   witness)
where
  module K = K4Map d Two-distinction
  open K

  witness : interpret (permOut (classify m)) ≐ swapOut {d = d} m
  witness =
    ≐-trans
    (≐-sym (swapOut-interpret d (classify m)))
    (swapOut-cong {d = d} (classify-sound m))

```

Law 15C.1: Classification Respects Input Swap

The input version is analogous, except that the permutation `permIn` reflects the fact that input duality fixes the constant maps and exchanges identity with dual.

```

-- § classify commutes with input swap
law15C-1-classify-swapIn :
  (d : Distinction) →
  (m : Measurement d) →
  K4Map.classify d Two-distinction (swapIn d m)

```

```

≡ permIn (K4Map.classify d Two-distinction m)
law15C-1-classify-swapIn d m =
  sym
  (K.classify-unique
   (swapIn d m)
   (permIn (K.classify m))
   witness)
where
  module K = K4Map d Two-distinction
  open K

witness : interpret (permIn (classify m)) ≐ swapIn d m
witness =
  ≐-trans
  (≐-sym (swapIn-interpret d (classify m)))
  (swapIn-cong d (classify-sound m))

```


Chapter 43

Rational Addition and Multiplication Laws

The rational operations introduced earlier must now satisfy the expected algebraic laws up to the setoid relation $\simeq_{\mathbb{Q}}$. Each proof reduces to explicit identities in $\mathbb{Z}$, together with the positivity of denominators.

Binary Congruence Helper

A small two-variable congruence lemma is convenient for the rational calculations that follow.

```
-- § two-argument congruence
cong2 : {A B C : Set} → (f : A → B → C) → {x x' : A} → {y y' : B} → x ≡ x' → y ≡ y' → f x y ≡ f x' y'
cong2 f refl refl = refl
```

Commutativity of $+\mathbb{Q}$

Commutativity of rational addition is nothing more than the symmetry of the cross-multiplied numerator together with commutativity of denominator multiplication.

```
-- § rational addition commutes in  $\simeq_{\mathbb{Q}}$ 
+Q-comm : (p q :  $\mathbb{Q}$ ) → p +Q q  $\simeq_{\mathbb{Q}}$  q +Q p
+Q-comm (a / b) (c / d) =
let
  numComm : ((a *Z +toZ d) +Z (c *Z +toZ b)) ≡ ((c *Z +toZ b) +Z (a *Z +toZ d))
  numComm = +Z-comm (a *Z +toZ d) (c *Z +toZ b)

  denComm : (d *+ b) ≡ (b *+ d)
  denComm = *+-comm d b
```

```

denCommℤ : +toℤ (d *+ b) ≡ +toℤ (b *+ d)
denCommℤ = cong +toℤ denComm

lhsEq : (((a *ℤ +toℤ d) +ℤ (c *ℤ +toℤ b)) *ℤ +toℤ (d *+ b))
      ≡
      (((c *ℤ +toℤ b) +ℤ (a *ℤ +toℤ d)) *ℤ +toℤ (b *+ d))
lhsEq =
  trans
    (cong (λ t → t *ℤ +toℤ (d *+ b)) numComm)
    (cong (λ t → ((c *ℤ +toℤ b) +ℤ (a *ℤ +toℤ d)) *ℤ t) denCommℤ)
in
  lhsEq

```

Congruence of $+\mathbb{Q}$

Congruence of addition means that equivalent summands produce equivalent sums. The proof expands both cross-multiplied numerators, aligns the corresponding terms, and then reassembles them into the required equality.

```

-- § +ℚ respects ≃ℚ
+ℚ-resp-≃ : {p p' q q' : ℚ} → p ≃ℚ p' → q ≃ℚ q' → (p +ℚ q) ≃ℚ (p' +ℚ q')
+ℚ-resp-≃ : {a / b} {a' / b'} {c / d} {c' / d'} eqp eqq =
let
  bd : ℕ+
  bd = b *+ d

  b'd' : ℕ+
  b'd' = b' *+ d'

  b'dℤ : +toℤ b'd' ≡ (+toℤ b') *ℤ (+toℤ d')
  b'dℤ = +toℤ-*+ b' d'

  bdℤ : +toℤ bd ≡ (+toℤ b) *ℤ (+toℤ d)
  bdℤ = +toℤ-*+ b d

mul4-rearrange : (x y z w : ℤ) → (x *ℤ y) *ℤ (z *ℤ w) ≡ (x *ℤ z) *ℤ (y *ℤ w)
mul4-rearrange x y z w =
  trans
    (*ℤ-assoc x y (z *ℤ w))
  (trans
    (cong (λ t → x *ℤ t)
      (trans
        (sym (*ℤ-assoc y z w))
        (trans
          (cong (λ t → t *ℤ w) (*ℤ-comm y z))
          (*ℤ-assoc z y w))))
      (sym (*ℤ-assoc x z (y *ℤ w))))

```

lhsExpand :

```

(((a *ℤ +toℤ d) +ℤ (c *ℤ +toℤ b)) *ℤ +toℤ b'd')

```

```

≡
((a *ℤ +toℤ d) *ℤ +toℤ b'd') +ℤ ((c *ℤ +toℤ b) *ℤ +toℤ b'd')
lhsExpand = *ℤ-distrib-left-+ℤ (a *ℤ +toℤ d) (c *ℤ +toℤ b) (+toℤ b'd')

rhsExpand :
(((a' *ℤ +toℤ d') +ℤ (c' *ℤ +toℤ b')) *ℤ +toℤ bd)
≡
((a' *ℤ +toℤ d') *ℤ +toℤ bd) +ℤ ((c' *ℤ +toℤ b') *ℤ +toℤ bd)
rhsExpand = *ℤ-distrib-left-+ℤ (a' *ℤ +toℤ d') (c' *ℤ +toℤ b') (+toℤ bd)

-- § align a-summands using eqp scaled by d-d'
eqpScaled₀ : ((a *ℤ +toℤ b') *ℤ ((+toℤ d) *ℤ (+toℤ d'))) ≡ ((a' *ℤ +toℤ b) *ℤ ((+toℤ d) *ℤ (+toℤ d')))
eqpScaled₀ = cong (λ t → t *ℤ ((+toℤ d) *ℤ (+toℤ d'))) eqp

termA-lhs : ((a *ℤ +toℤ d) *ℤ +toℤ b'd') ≡ ((a *ℤ +toℤ b') *ℤ ((+toℤ d) *ℤ (+toℤ d')))
termA-lhs =
  trans
    (cong (λ t → (a *ℤ +toℤ d) *ℤ t) b'd'ℤ)
    (mul4-rearrange a (+toℤ d) (+toℤ b') (+toℤ d'))

termA-rhs : ((a' *ℤ +toℤ d') *ℤ +toℤ bd) ≡ ((a' *ℤ +toℤ b) *ℤ ((+toℤ d) *ℤ (+toℤ d')))
termA-rhs =
  trans
    (cong (λ t → (a' *ℤ +toℤ d') *ℤ t) bdℤ)
    (trans
      (mul4-rearrange a' (+toℤ d') (+toℤ b) (+toℤ d))
      (trans
        (cong (λ t → (a' *ℤ +toℤ b) *ℤ t) (*ℤ-comm (+toℤ d') (+toℤ d)))
        refl))

termA : ((a *ℤ +toℤ d) *ℤ +toℤ b'd') ≡ ((a' *ℤ +toℤ d') *ℤ +toℤ bd)
termA =
  trans
    termA-lhs
    (trans
      eqpScaled₀
      (sym termA-rhs))

-- § align c-summands using eqq scaled by b-b'
eqqScaled₀ : ((c *ℤ +toℤ d') *ℤ ((+toℤ b) *ℤ (+toℤ b'))) ≡ ((c' *ℤ +toℤ d) *ℤ ((+toℤ b) *ℤ (+toℤ b')))
eqqScaled₀ = cong (λ t → t *ℤ ((+toℤ b) *ℤ (+toℤ b'))) eqq

termC-lhs : ((c *ℤ +toℤ b) *ℤ +toℤ b'd') ≡ ((c *ℤ +toℤ d') *ℤ ((+toℤ b) *ℤ (+toℤ b')))
termC-lhs =
  trans
    (cong (λ t → (c *ℤ +toℤ b) *ℤ t) b'd'ℤ)
    (trans
      (cong (λ t → (c *ℤ +toℤ b) *ℤ t) (*ℤ-comm (+toℤ b') (+toℤ d')))
      (mul4-rearrange c (+toℤ b) (+toℤ d') (+toℤ b')))

termC-rhs : ((c' *ℤ +toℤ b') *ℤ +toℤ bd) ≡ ((c' *ℤ +toℤ d) *ℤ ((+toℤ b) *ℤ (+toℤ b')))
termC-rhs =
  trans
    (cong (λ t → (c' *ℤ +toℤ b') *ℤ t) bdℤ)

```

```

(trans
  (cong (λ t → (c' *ℤ +toℤ b') *ℤ t) (*ℤ-comm (+toℤ b) (+toℤ d)))
  (trans
    (mul4-rearrange c' (+toℤ b') (+toℤ d) (+toℤ b))
    (cong (λ t → (c' *ℤ +toℤ d) *ℤ t) (*ℤ-comm (+toℤ b') (+toℤ b)))))

termC : ((c *ℤ +toℤ b) *ℤ +toℤ b'd) ≡ ((c' *ℤ +toℤ b') *ℤ +toℤ bd)
termC =
  trans
    termC-lhs
    (trans
      eqqScaled₀
      (sym termC-rhs))
in
  trans
    lhsExpand
    (trans
      (cong₂ _+ℤ_ termA termC)
      (sym rhsExpand))

```

Associativity of $+\mathbb{Q}$

Associativity is longer only because three denominators must be coordinated at once. After expanding both sides to a common normal form, the proof becomes a direct identity in the integer numerator.

```

-- § rational addition is associative in  $\simeq \mathbb{Q}$ 
+ℚ-assoc : (p q r : ℚ) → (p +ℚ q) +ℚ r ≃ℚ p +ℚ (q +ℚ r)
+ℚ-assoc (a / b) (c / d) (e / f) =
  let
    bd : ℕ⁺
    bd = b *+ d

    df : ℕ⁺
    df = d *+ f

    lhsNum : ℤ
    lhsNum = (((a *ℤ +toℤ d) +ℤ (c *ℤ +toℤ b)) *ℤ +toℤ f) +ℤ (e *ℤ +toℤ bd)

    rhsNum : ℤ
    rhsNum = (a *ℤ +toℤ df) +ℤ (((c *ℤ +toℤ f) +ℤ (e *ℤ +toℤ d)) *ℤ +toℤ b)

    lhsDen : ℕ⁺
    lhsDen = bd *+ f

    rhsDen : ℕ⁺
    rhsDen = b *+ df

    bdℤ : +toℤ bd ≡ (+toℤ b) *ℤ (+toℤ d)
    bdℤ = +toℤ- *+ b d

```

```

dfℤ : +toℤ df ≡ (+toℤ d) *ℤ (+toℤ f)
dfℤ = +toℤ-+ d f

denL : +toℤ lhsDen ≡ ((+toℤ b) *ℤ (+toℤ d)) *ℤ (+toℤ f)
denL =
  trans
    (+toℤ-+ bd f)
    (cong (λ t → t *ℤ (+toℤ f)) bdℤ)

denR : +toℤ rhsDen ≡ (+toℤ b) *ℤ ((+toℤ d) *ℤ (+toℤ f))
denR =
  trans
    (+toℤ-+ b df)
    (cong (λ t → (+toℤ b) *ℤ t) dfℤ)

denEq : +toℤ lhsDen ≡ +toℤ rhsDen
denEq =
  trans
    denL
    (trans
      (*ℤ-assoc (+toℤ b) (+toℤ d) (+toℤ f))
      (sym denR))

-- § expand rhsNum to normal form matching lhsNum
rhsExpand : rhsNum ≡ lhsNum
rhsExpand =
  let
    nf : ℤ
    nf = (((a *ℤ +toℤ d) *ℤ +toℤ f) +ℤ ((c *ℤ +toℤ b) *ℤ +toℤ f)) +ℤ (e *ℤ +toℤ bd)

  swapLastFactors : (x y z : ℤ) → (x *ℤ y) *ℤ z ≡ (x *ℤ z) *ℤ y
  swapLastFactors x y z =
    trans
      (*ℤ-assoc x y z)
      (trans
        (cong (λ t → x *ℤ t) (*ℤ-comm y z))
        (sym (*ℤ-assoc x z y)))

  cTermEq : ((c *ℤ +toℤ f) *ℤ +toℤ b) ≡ ((c *ℤ +toℤ b) *ℤ +toℤ f)
  cTermEq = swapLastFactors c (+toℤ f) (+toℤ b)

  eTermEq : ((e *ℤ +toℤ d) *ℤ +toℤ b) ≡ (e *ℤ +toℤ bd)
  eTermEq =
    trans
      (*ℤ-assoc e (+toℤ d) (+toℤ b))
      (trans
        (cong (λ t → e *ℤ t) (*ℤ-comm (+toℤ d) (+toℤ b)))
        (cong (λ t → e *ℤ t) (sym bdℤ)))

  rhsToNF : rhsNum ≡ nf
  rhsToNF =
    trans
      (cong (λ t → (a *ℤ t) +ℤ (((c *ℤ +toℤ f) +ℤ (e *ℤ +toℤ d)) *ℤ +toℤ b)) dfℤ)
      (trans

```

```

(cong (λ t → t + ℤ (((c * ℤ +toℤ f) + ℤ (e * ℤ +toℤ d)) * ℤ +toℤ b))
  (sym (*ℤ-assoc a (+toℤ d) (+toℤ f))))
(trans
  (cong (λ t → ((a * ℤ +toℤ d) * ℤ +toℤ f) + ℤ t)
    (*ℤ-distrib-left+ℤ (c * ℤ +toℤ f) (e * ℤ +toℤ d) (+toℤ b)))
  (trans
    (cong (λ t → ((a * ℤ +toℤ d) * ℤ +toℤ f) + ℤ t)
      (cong2 _+ℤ_ cTermEq eTermEq))
    (sym (+ℤ-assoc ((a * ℤ +toℤ d) * ℤ +toℤ f) ((c * ℤ +toℤ b) * ℤ +toℤ f) (e * ℤ +toℤ bd)))))

lhsToNF : lhsNum ≡ nf
lhsToNF =
  cong (λ t → t + ℤ (e * ℤ +toℤ bd))
  (*ℤ-distrib-left+ℤ (a * ℤ +toℤ d) (c * ℤ +toℤ b) (+toℤ f))
in
trans rhsToNF (sym lhsToNF)

numEq : lhsNum ≡ rhsNum
numEq = sym rhsExpand
in
trans
  (cong (λ t → lhsNum * ℤ t) (sym denEq))
  (cong (λ t → t * ℤ +toℤ lhsDen) numEq)

```

Additive Identity and Inverse

The neutral element and additive inverse are verified by the same pattern as before: simplify the numerator, identify the denominator with the obvious unit or square, and conclude the required setoid equality.

```

-- § 0ℚ is right identity for +ℚ
+ℚ-zero-right : (p : ℚ) → p + ℚ 0ℚ ≈ ℚ p
+ℚ-zero-right (a / b) =
let
  lhsNum : ℤ
  lhsNum = (a * ℤ +toℤ one+) + ℤ (0ℤ * ℤ +toℤ b)

  lhsNum ≡ a : lhsNum ≡ a
  lhsNum ≡ a =
    trans
      (cong (λ t → t + ℤ (0ℤ * ℤ +toℤ b)) (*ℤ-one-right a))
      (trans
        (cong (λ t → a + ℤ t) (*ℤ-zero-left (+toℤ b)))
        (+ℤ-zero-right a))

  denOne : +toℤ b ≡ +toℤ (b *+ one+)
  denOne =
    trans
      (sym (*ℤ-one-right (+toℤ b)))
      (sym (+toℤ-*+ b one+))
in

```

```

trans
  (cong (λ t → t *ℤ +toℤ b) lhsNum≡a)
  (cong (λ t → a *ℤ t) denOne)

-- § 0ℚ is left identity for +ℚ
+ℚ-zero-left : (p : ℚ) → 0ℚ +ℚ p ≈ℚ p
+ℚ-zero-left (a / b) =
let
  lhsNum : ℤ
  lhsNum = (0ℤ *ℤ +toℤ b) +ℤ (a *ℤ +toℤ one+)

  lhsNum≡a : lhsNum ≡ a
  lhsNum≡a =
  trans
    (cong (λ t → t +ℤ (a *ℤ +toℤ one+)) (*ℤ-zero-left (+toℤ b)))
    (trans
      (cong (λ t → 0ℤ +ℤ t) (*ℤ-one-right a))
      (+ℤ-zero-left a))

  denOneL : +toℤ b ≡ +toℤ (one+ *+ b)
  denOneL = sym (trans (+toℤ-*+ one+ b) (*ℤ-one-left (+toℤ b)))
in
trans
  (cong (λ t → t *ℤ +toℤ b) lhsNum≡a)
  (cong (λ t → a *ℤ t) denOneL)

-- § additive inverse cancels: p +ℚ (-ℚ p) ≈ℚ 0ℚ
+ℚ-inv-right : (p : ℚ) → p +ℚ (-ℚ p) ≈ℚ 0ℚ
+ℚ-inv-right (a / b) =
let
  x : ℤ
  x = a *ℤ +toℤ b

  lhsNum : ℤ
  lhsNum = x +ℤ (negℤ a *ℤ +toℤ b)

  lhsNum≡0 : lhsNum ≡ 0ℤ
  lhsNum≡0 =
  trans
    (cong (λ t → x +ℤ t) (*ℤ-neg-left a (+toℤ b)))
    (+ℤ-inv-right x)
in
trans
  (cong (λ t → t *ℤ +toℤ one+) lhsNum≡0)
  (trans
    (*ℤ-zero-left (+toℤ one+))
    (sym (*ℤ-zero-left (+toℤ (b *+ b)))))

```

Rational Multiplication Monotonicity

Monotonicity of multiplication is reduced to monotonicity in the integers after denominators are cleared. The positivity of the denominator and the nonnegativity assumption on the multiplier ensure that the necessary scaling steps preserve order.

```

-- § extract 0 ≤ num from 0 ≤ q
0 ≤ Q → 0 ≤ Z-num : (q : Q) → 0 Q ≤ Q q → 0 Z ≤ Z num q
0 ≤ Q → 0 ≤ Z-num (a / b) p =
  ≤ Z-resp ≡l (*Z-zero-left (+toZ b))
  (≤ Z-resp ≡r (*Z-one-right a) p)

-- § rational multiplication commutes in ≈Q
*Q-comm : (p q : Q) → (p *Q q) ≈Q (q *Q p)
*Q-comm (a / b) (c / d) =
let
  denSwap : (d *+ b) ≡ (b *+ d)
  denSwap = *+-comm d b

  numSwap : (a *Z c) ≡ (c *Z a)
  numSwap = *Z-comm a c

  lhsStep : ((a *Z c) *Z +toZ (d *+ b)) ≡ ((a *Z c) *Z +toZ (b *+ d))
  lhsStep = cong (λ t → (a *Z c) *Z +toZ t) denSwap

  rhsStep : ((c *Z a) *Z +toZ (b *+ d)) ≡ ((a *Z c) *Z +toZ (b *+ d))
  rhsStep = cong (λ t → t *Z +toZ (b *+ d)) (sym numSwap)
in
trans lhsStep (sym rhsStep)

-- § swap middle factors in a triple product
mul-swap-middle : (x y z : Z) → (x *Z y) *Z z ≡ (x *Z z) *Z y
mul-swap-middle x y z =
trans
  (*Z-assoc x y z)
  (trans
    (cong (λ t → x *Z t) (*Z-comm y z))
    (sym (*Z-assoc x z y)))

-- § multiplying on the right by a nonneg rational preserves ≤Q
≤Q-mul-nonneg-right : (x y z : Q) → x ≤Q y → 0 Q ≤ Q z → (x *Q z) ≤Q (y *Q z)
≤Q-mul-nonneg-right (a / b) (c / d) (e / f) x ≤ y zNonneg =
let
  eNonneg : 0 Z ≤ Z e
  eNonneg = 0 ≤ Q → 0 ≤ Z-num (e / f) zNonneg

  step1 : ((a *Z +toZ d) *Z +toZ f) ≤Z ((c *Z +toZ b) *Z +toZ f)
  step1 = ≤Z-mul-pos-right (a *Z +toZ d) (c *Z +toZ b) f x ≤ y

  step2 : (((a *Z +toZ d) *Z +toZ f) *Z e) ≤Z (((c *Z +toZ b) *Z +toZ f) *Z e)
  step2 = ≤Z-mul-nonneg-right ((a *Z +toZ d) *Z +toZ f) ((c *Z +toZ b) *Z +toZ f) e step1 eNonneg

lhsEq : (((a *Z +toZ d) *Z +toZ f) *Z e) ≡ ((a *Z e) *Z +toZ (d *+ f))
lhsEq =
trans
  (mul-swap-middle (a *Z +toZ d) (+toZ f) e)
  (trans
    (cong (λ t → t *Z +toZ f) (mul-swap-middle a (+toZ d) e))
    (trans
      (*Z-assoc (a *Z e) (+toZ d) (+toZ f))

```



```

(cong (λ t → (a *ℤ e) *ℤ t) (sym (+toℤ-+ d f))))))

rhsEq : (((c *ℤ +toℤ b) *ℤ +toℤ f) *ℤ e) ≡ ((c *ℤ e) *ℤ +toℤ (b *+ f))
rhsEq =
  trans
    (mul-swap-middle (c *ℤ +toℤ b) (+toℤ f) e)
  (trans
    (cong (λ t → t *ℤ +toℤ f) (mul-swap-middle c (+toℤ b) e))
    (trans
      (*ℤ-assoc (c *ℤ e) (+toℤ b) (+toℤ f))
      (cong (λ t → (c *ℤ e) *ℤ t) (sym (+toℤ-+ b f))))))
in
  ≤ℤ-resp≡l lhsEq (≤ℤ-resp≡r rhsEq step2)

-- § multiplying on the left by a nonneg rational preserves ≤ℚ
≤ℚ-mul-nonneg-left : (x y z : ℚ) → x ≤ℚ y → 0ℚ ≤ℚ z → (z *ℚ x) ≤ℚ (z *ℚ y)
≤ℚ-mul-nonneg-left (a / b) (c / d) (e / f) x≤y zNonneg =
  let
    zx≤xz : ((e / f) *ℚ (a / b)) ≤ℚ ((a / b) *ℚ (e / f))
    zx≤xz =
      ≤ℚ→≤ℚl
      {p = (e / f) *ℚ (a / b)}
      {q = (a / b) *ℚ (e / f)}
      (*ℚ-comm (e / f) (a / b))

    xz≤yz : ((a / b) *ℚ (e / f)) ≤ℚ ((c / d) *ℚ (e / f))
    xz≤yz = ≤ℚ-mul-nonneg-right (a / b) (c / d) (e / f) x≤y zNonneg

    yz≤zy : ((c / d) *ℚ (e / f)) ≤ℚ ((e / f) *ℚ (c / d))
    yz≤zy =
      ≤ℚ→≤ℚl
      {p = (c / d) *ℚ (e / f)}
      {q = (e / f) *ℚ (c / d)}
      (*ℚ-comm (c / d) (e / f))

    middle : ((a / b) *ℚ (e / f)) ≤ℚ ((e / f) *ℚ (c / d))
    middle = ≤ℚ-trans {(a / b) *ℚ (e / f)} {(c / d) *ℚ (e / f)} {(e / f) *ℚ (c / d)} xz≤yz yz≤zy
  in
    ≤ℚ-trans {(e / f) *ℚ (a / b)} {(a / b) *ℚ (e / f)} {(e / f) *ℚ (c / d)} zx≤xz middle

```


Chapter 44

Coupled K_4 Laplacian

The two forced coupling survivors, empty coupling (block-diagonal) and full coupling (K_8), each generate a Laplacian operator on $\text{Vec8}\mathbb{Z}$. This chapter derives the spectral theory of both operators, culminating in packaged spectral witnesses indexed by `CouplingSurvivor`.

Vec8 Infrastructure

The coupled theory lives on two copies of the four-vertex carrier, so the natural model is simply a pair of $\text{Vec4}\mathbb{Z}$ blocks. All later operations on $\text{Vec8}\mathbb{Z}$ are defined componentwise, which lets the coupled Laplacians inherit the previously established four-dimensional structure.

```
-- § 8-vectors as paired 4-vectors
Vec8ℤ : Set
Vec8ℤ = Vec4ℤ × Vec4ℤ

-- § left/right block projections
left4 : Vec8ℤ → Vec4ℤ
left4 = fst

right4 : Vec8ℤ → Vec4ℤ
right4 = snd

-- § pointwise equality on Vec8ℤ
Vec8Eq : Vec8ℤ → Vec8ℤ → Set
Vec8Eq u v = Vec4Eq (left4 u) (left4 v) × Vec4Eq (right4 u) (right4 v)

-- § pointwise addition on Vec8ℤ
+Vec8ℤ : Vec8ℤ → Vec8ℤ → Vec8ℤ
+Vec8ℤ u v = ((left4 u) +Vec4ℤ (left4 v)) , ((right4 u) +Vec4ℤ (right4 v))

-- § pointwise negation on Vec8ℤ
negVec8ℤ : Vec8ℤ → Vec8ℤ
```

```

negVec8ℤ v = (λ i → negℤ (left4 v i)) , (λ i → negℤ (right4 v i))

-- § pointwise four-times on Vec8ℤ
fourVec8ℤ : Vec8ℤ → Vec8ℤ
fourVec8ℤ v = (λ i → fourTimesℤ (left4 v i)) , (λ i → fourTimesℤ (right4 v i))

-- § global sum of all 8 entries
sum8ℤ : Vec8ℤ → ℤ
sum8ℤ v = sumFin4ℤ (left4 v) + ℤ sumFin4ℤ (right4 v)

-- § all-ones operator: constant with value sum8ℤ v
J8Vec8ℤ : Vec8ℤ → Vec8ℤ
J8Vec8ℤ v = constVec4ℤ (sum8ℤ v) , constVec4ℤ (sum8ℤ v)

-- § scalar ×8 on ℤ and on Vec4ℤ
eightTimesℤ : ℤ → ℤ
eightTimesℤ x = fourTimesℤ x + ℤ fourTimesℤ x

eightVec4ℤ : Vec4ℤ → Vec4ℤ
eightVec4ℤ v i = eightTimesℤ (v i)

```

K₈ Laplacian Definition

In the fully coupled case, every coordinate interacts with all eight vertices, so the Laplacian takes the uniform form $L_8 v_i = 8v_i - \sum_8 v$. The empty and full couplings can then be compared directly as two explicit operators on the same carrier $\text{Vec8}\mathbb{Z}$.

```

-- § K8 Laplacian:  $8 \cdot v_i - \sum_8 v$ 
K8LaplacianVec8ℤ : Vec8ℤ → Vec8ℤ
K8LaplacianVec8ℤ v =
  (λ i → eightTimesℤ (left4 v i) + ℤ negℤ (sum8ℤ v)) ,
  (λ i → eightTimesℤ (right4 v i) + ℤ negℤ (sum8ℤ v))

-- § block-diagonal Laplacian (empty coupling)
laplacianEmptyVec8ℤ : Vec8ℤ → Vec8ℤ
laplacianEmptyVec8ℤ v = laplacianVec4ℤ (left4 v) , laplacianVec4ℤ (right4 v)

-- § full coupling Laplacian:  $L_4 + 4I - \text{cross-sum}$ 
laplacianFullVec8ℤ : Vec8ℤ → Vec8ℤ
laplacianFullVec8ℤ v =
  (λ i → laplacianVec4ℤ (left4 v) i + ℤ fourTimesℤ (left4 v i) + ℤ negℤ (sumFin4ℤ (right4 v))) ,
  (λ i → laplacianVec4ℤ (right4 v) i + ℤ fourTimesℤ (right4 v i) + ℤ negℤ (sumFin4ℤ (left4 v)))

-- § pointwise ×8 on Vec8ℤ
eightVec8ℤ : Vec8ℤ → Vec8ℤ
eightVec8ℤ v = eightVec4ℤ (left4 v) , eightVec4ℤ (right4 v)

```

Coupling Survivors and Laplacian Dispatch

Law 14G.6 has already reduced the coupling problem to exactly two survivors, so at the operator level there are only two corresponding cases to track. The datatype

CouplingSurvivor records these cases, survivorCoupling sends each of them to its forced coupling, and laplacianSurvivorVec8 $\mathbb{Z}$ dispatches to the corresponding Laplacian. No third branch is available, which is why the later spectral analysis on K_8 can be carried out once for the empty case and once for the full one.

```
-- § coupling survivor type: empty or full
data CouplingSurvivor : Set where
  survivor-empty : CouplingSurvivor
  survivor-full : CouplingSurvivor

-- § forced coupling for each survivor
survivorCoupling : CouplingSurvivor → Coupling
survivorCoupling survivor-empty = CrossEmpty
survivorCoupling survivor-full = CrossFull

-- § Laplacian dispatched by survivor case
laplacianSurvivorVec8 $\mathbb{Z}$  : CouplingSurvivor → Vec8 $\mathbb{Z}$  → Vec8 $\mathbb{Z}$ 
laplacianSurvivorVec8 $\mathbb{Z}$  survivor-empty = laplacianEmptyVec8 $\mathbb{Z}$ 
laplacianSurvivorVec8 $\mathbb{Z}$  survivor-full = laplacianFullVec8 $\mathbb{Z}$ 

-- § constant and zero 8-vectors
constVec8 $\mathbb{Z}$  :  $\mathbb{Z}$  → Vec8 $\mathbb{Z}$ 
constVec8 $\mathbb{Z}$  x = constVec4 $\mathbb{Z}$  x , constVec4 $\mathbb{Z}$  x

zeroVec8 $\mathbb{Z}$  : Vec8 $\mathbb{Z}$ 
zeroVec8 $\mathbb{Z}$  = constVec8 $\mathbb{Z}$  0 $\mathbb{Z}$ 

eightVec8 $\mathbb{Z}$ -const :  $\mathbb{Z}$  → Vec8 $\mathbb{Z}$ 
eightVec8 $\mathbb{Z}$ -const x = constVec8 $\mathbb{Z}$  (eightTimes $\mathbb{Z}$  x)
```

Coupling Classification and Block Laws

Law 14G.0: Empty Coupling Is Block-Diagonal

Nothing happens between the two blocks in the empty case. Accordingly, the dispatched operator unfolds definitionally to the block-diagonal Laplacian $L_4 \oplus L_4$, and the proof consists of `refl` on both components. The spectral content is therefore exactly the old L_4 spectrum on each side, with no new cross-block behaviour.

```
-- § empty coupling = block-diagonal Laplacian
law14G-0-empty-block : (v : Vec8 $\mathbb{Z}$ ) → Vec8Eq (laplacianSurvivorVec8 $\mathbb{Z}$  survivor-empty v) (laplacianEmptyVec8 $\mathbb{Z}$  v)
law14G-0-empty-block v = ( $\lambda$  _ → refl) , ( $\lambda$  _ → refl)
```

Law 14G.1: Full Coupling Yields K_8 Laplacian

In the full case the two blocks no longer remain separate. Each side acquires the extra $4 \cdot v_i$ contribution together with the negative sum over the opposite block. Rewriting

the internal L_4 term via Law 14E.10 and regrouping the summands collapses both halves to the same K_8 expression $8 \cdot v_i - \Sigma_8 v$. Thus the full survivor case really is the K_8 Laplacian, and the corresponding spectral algebra is available everywhere this branch occurs.

Figure 44.1 records the dispatch in one line: two survivor cases, two operators, two spectral regimes. The two cases are the only constructors of CouplingSurvivor.

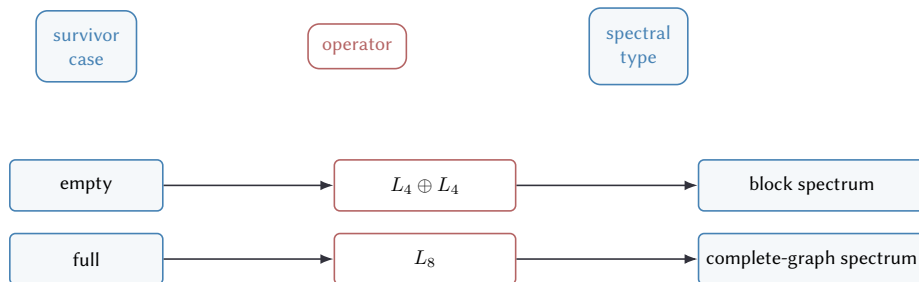


Figure 44.1: Coupling survivor dispatch. Empty and full are the two constructors of the survivor type, and each determines one operator and one spectral regime.

```

-- § full coupling = K8 Laplacian
law14G-1-full-is-K8 : (v : Vec8ℤ) → Vec8Eq (laplacianFullVec8ℤ v) (K8LaplacianVec8ℤ v)
law14G-1-full-is-K8 v =
  let sL = sumFin4ℤ (left4 v)
      sR = sumFin4ℤ (right4 v)
  in
    (λ i →
      let lv = laplacianVec4ℤ (left4 v) i
          fv = fourTimesℤ (left4 v i)
          -- § step1: L4 v_i + 4·v_i = 8·v_i - sL via law14E-10
          step1 : lv + ℤ fv ≡ eightTimesℤ (left4 v i) + ℤ negℤ sL
          step1 =
            trans
              (cong (λ t → t + ℤ fv) (law14E-10-laplacian-four-minus-sumAll (left4 v) i))
              (trans
                (+ℤ-assoc fv (negℤ sL) fv)
                (trans
                  (cong (λ t → fv + ℤ t) (+ℤ-comm (negℤ sL) fv))
                  (sym (+ℤ-assoc fv fv (negℤ sL))))))
          -- § step2: reassociate (lv + fv) + (-sR) = 8·v_i + (-sL + -sR)
          step2 : (lv + ℤ fv) + ℤ negℤ sR ≡ eightTimesℤ (left4 v i) + ℤ (negℤ sL + ℤ negℤ sR)
          step2 =
            trans
              (cong (λ t → t + ℤ negℤ sR) step1)
              (+ℤ-assoc (eightTimesℤ (left4 v i)) (negℤ sL) (negℤ sR))
          -- § step3: -sL + -sR = -(sL + sR) = -Σ8 v
          step3 : negℤ sL + ℤ negℤ sR ≡ negℤ (sum8ℤ v)
          step3 = sym (neg-+ℤ sL sR)
      in
        trans step2 (cong (λ t → eightTimesℤ (left4 v i) + ℤ t) step3)
    )
  
```

```

),
(λ i →
  let lv = laplacianVec4ℤ (right4 v) i
      fv = fourTimesℤ (right4 v i)
      step1 : lv +ℤ fv ≡ eightTimesℤ (right4 v i) +ℤ negℤ sR
      step1 =
        trans
          (cong (λ t → t +ℤ fv) (law14E-10-laplacian-four-minus-sumAll (right4 v) i))
          (trans
            (+ℤ-assoc fv (negℤ sR) fv)
            (trans
              (cong (λ t → fv +ℤ t) (+ℤ-comm (negℤ sR) fv))
              (sym (+ℤ-assoc fv fv (negℤ sR))))))
      step2 : (lv +ℤ fv) +ℤ negℤ sL ≡ eightTimesℤ (right4 v i) +ℤ (negℤ sR +ℤ negℤ sL)
      step2 =
        trans
          (cong (λ t → t +ℤ negℤ sL) step1)
          (+ℤ-assoc (eightTimesℤ (right4 v i)) (negℤ sR) (negℤ sL))
      step3 : negℤ sR +ℤ negℤ sL ≡ negℤ (sum8ℤ v)
      step3 =
        trans
          (+ℤ-comm (negℤ sR) (negℤ sL))
          (sym (neg+ℤ sL sR))
  in
  trans step2 (cong (λ t → eightTimesℤ (right4 v i) +ℤ t) step3)
)

```

Laws 14G.2–14G.5: Survivor names for the forced couplings

These laws merely introduce convenient names for the two surviving coupling forms and their two survivor constructors. No new argument is needed here; the forcing was already established in the previous chapter.

```

-- § alias: edge case selects CrossFull (forcing in 14F.0)
law14G-2-edge-forces-full : Coupling
law14G-2-edge-forces-full = CrossFull

-- § alias: non-edge case selects CrossEmpty (forcing in 14F.1)
law14G-3-nonedge-forces-empty : Coupling
law14G-3-nonedge-forces-empty = CrossEmpty

-- § alias: edge case selects survivor-full
law14G-4-edge-full : CouplingSurvivor
law14G-4-edge-full = survivor-full

-- § alias: non-edge case selects survivor-empty
law14G-5-nonedge-empty : CouplingSurvivor
law14G-5-nonedge-empty = survivor-empty

```

Law 14G.6: The two survivor constructors

The type `CouplingSurvivor` has exactly two constructors, and the next law records that exhaustiveness explicitly. This will later allow every survivor-dependent statement to split cleanly into the empty and full cases.

```
-- § survivor case split (structural, by pattern match on the data type)
law14G-6-survivor-cases : (k : CouplingSurvivor) → (k ≡ survivor-empty) ⊔ (k ≡ survivor-full)
law14G-6-survivor-cases survivor-empty = inj₁ refl
law14G-6-survivor-cases survivor-full = inj₂ refl
```

Laws 14G.7–14G.8: Survivor-to-Operator Identification

Once the dispatcher is fixed, the two survivor values can be identified with the two concrete Laplacian operators. One branch is the block-diagonal copy of L_4 , the other is the genuine K_8 Laplacian.

```
-- § empty survivor = block-diagonal
law14G-7-survivor-empty-block : (v : Vec8ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (laplacianEmptyVec8ℤ v)
law14G-7-survivor-empty-block = law14G-0-empty-block

-- § full survivor = K₈ Laplacian
law14G-8-survivor-full-K8 : (v : Vec8ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (K8LaplacianVec8ℤ v)
law14G-8-survivor-full-K8 = law14G-1-full-is-K8
```

Scalar Helpers for $\times 8$

Before the spectral laws can be stated, the scalar operation $x \mapsto 8x$ has to be related to the previously established four-fold identities. These short lemmas supply exactly the algebraic rewrites needed for the later global-sum calculations.

```
-- § 8·(x + y) = 8·x + 8·y
eightTimes-+ℤ : (x y : ℤ) → eightTimesℤ (x +ℤ y) ≡ eightTimesℤ x +ℤ eightTimesℤ y
eightTimes-+ℤ x y =
  let fx = fourTimesℤ x in
  let fy = fourTimesℤ y in
  trans
    (cong (λ t → t +ℤ t) (fourTimes-+ℤ x y))
    (trans
      (+ℤ-assoc fx fy (fx +ℤ fy))
      (trans
        (cong (λ t → fx +ℤ t) (swapHeadℤ fy fx fy))
        (trans
          (sym (+ℤ-assoc fx fx (fy +ℤ fy)))
          refl))))

-- § 8·(−x) = −(8·x)
eightTimes-neg : (x : ℤ) → eightTimesℤ (negℤ x) ≡ negℤ (eightTimesℤ x)
```



```

eightTimes-neg x =
  trans
    (cong (λ t → t + ℤ t) (sym (neg-fourTimes ℤ x)))
  (trans
    (sym (neg-+ ℤ (fourTimes ℤ x) (fourTimes ℤ x)))
    refl)

-- § sum of 8·vi over Fin4 = 8·(sum v)
sumFin4-eightTimes : (v : Vec4 ℤ) →
  sumFin4 ℤ (λ i → eightTimes ℤ (v i)) ≡ eightTimes ℤ (sumFin4 ℤ v)
sumFin4-eightTimes v =
  let vt : Vec4 ℤ
    vt i = fourTimes ℤ (v i)
  in
  trans
    (sumFin4-+Vec4 ℤ vt vt)
  (trans
    (cong (λ t → t + ℤ t) (sumFin4-fourTimes v))
    refl)

-- § sum of 8·v over both blocks = 8·(sum8 v)
sum8-eightVec8 : (v : Vec8 ℤ) → sum8 ℤ (eightVec8 ℤ v) ≡ eightTimes ℤ (sum8 ℤ v)
sum8-eightVec8 v =
  let sL = sumFin4 ℤ (left4 v) in
  let sR = sumFin4 ℤ (right4 v) in
  trans
    (cong
      (λ t → t + ℤ sumFin4 ℤ (λ i → eightTimes ℤ (right4 v i)))
      (sumFin4-eightTimes (left4 v)))
    (trans
      (cong
        (λ t → eightTimes ℤ sL + ℤ t)
        (sumFin4-eightTimes (right4 v)))
      (trans
        (sym (eightTimes-+ ℤ sL sR))
        refl))

```

Global Sum Law and Operator Annihilation

The first decisive fact about L_8 is that its image has total sum zero. Once that is known, the expected annihilation identities with J_8 and the quadratic relation $L_8^2 = 8L_8$ follow by direct substitution.

Law 14G.12: $\Sigma_8(L_8 v) = 0$

Summing the coordinate formula for L_8 over all eight entries produces $8\Sigma_8 v - 8\Sigma_8 v$. The whole calculation is bookkeeping, but it is precisely this bookkeeping that forces the image of L_8 into the sum-zero subspace.

```

-- § global sum of K8 Laplacian is 0
law14G-12-sumL8-0 : (v : Vec8ℤ) → sum8ℤ (K8LaplacianVec8ℤ v) ≡ 0ℤ
law14G-12-sumL8-0 v =
let
  s = sum8ℤ v
  sL = sumFin4ℤ (left4 v)
  sR = sumFin4ℤ (right4 v)

  leftPart = λ i → eightTimesℤ (left4 v i) + ℤ negℤ s
  rightPart = λ i → eightTimesℤ (right4 v i) + ℤ negℤ s

  stepL : sumFin4ℤ leftPart ≡ sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) + ℤ fourTimesℤ (negℤ s)
  stepL = sumFin4-addConst (λ i → eightTimesℤ (left4 v i)) (negℤ s)

  stepR : sumFin4ℤ rightPart ≡ sumFin4ℤ (λ i → eightTimesℤ (right4 v i)) + ℤ fourTimesℤ (negℤ s)
  stepR = sumFin4-addConst (λ i → eightTimesℤ (right4 v i)) (negℤ s)

  step1 : sum8ℤ (K8LaplacianVec8ℤ v) ≡
    (sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) + ℤ fourTimesℤ (negℤ s)) + ℤ
    (sumFin4ℤ (λ i → eightTimesℤ (right4 v i)) + ℤ fourTimesℤ (negℤ s))
  step1 =
  trans
    (cong (λ t → t + ℤ sumFin4ℤ rightPart) stepL)
    (cong (λ t → (sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) + ℤ fourTimesℤ (negℤ s)) + ℤ t) stepR)

  step2 : (sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) + ℤ fourTimesℤ (negℤ s)) + ℤ
    (sumFin4ℤ (λ i → eightTimesℤ (right4 v i)) + ℤ fourTimesℤ (negℤ s)) ≡
    (sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) + ℤ sumFin4ℤ (λ i → eightTimesℤ (right4 v i))) + ℤ
    (fourTimesℤ (negℤ s) + ℤ fourTimesℤ (negℤ s))
  step2 =
  trans
    (+ℤ-assoc (sumFin4ℤ (λ i → eightTimesℤ (left4 v i))) (fourTimesℤ (negℤ s))
      (sumFin4ℤ (λ i → eightTimesℤ (right4 v i)) + ℤ fourTimesℤ (negℤ s)))
    (trans
      (cong (λ t → sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) + ℤ t)
        (swapHeadℤ (fourTimesℤ (negℤ s)) (sumFin4ℤ (λ i → eightTimesℤ (right4 v i)))
          (fourTimesℤ (negℤ s))))
      (sym (+ℤ-assoc (sumFin4ℤ (λ i → eightTimesℤ (left4 v i)))
        (sumFin4ℤ (λ i → eightTimesℤ (right4 v i)))
        (fourTimesℤ (negℤ s) + ℤ fourTimesℤ (negℤ s)))))

  step3 : sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) ≡ eightTimesℤ sL
  step3 = sumFin4-eightTimes (left4 v)

  step4 : sumFin4ℤ (λ i → eightTimesℤ (right4 v i)) ≡ eightTimesℤ sR
  step4 = sumFin4-eightTimes (right4 v)

  step5 : (sumFin4ℤ (λ i → eightTimesℤ (left4 v i)) + ℤ sumFin4ℤ (λ i → eightTimesℤ (right4 v i))) ≡
    eightTimesℤ s
  step5 =
  trans
    (cong (λ t → t + ℤ sumFin4ℤ (λ i → eightTimesℤ (right4 v i))) step3)
    (trans
      (cong (λ t → eightTimesℤ sL + ℤ t) step4)

```

```

(sym (eightTimes+ℤ sL sR)))

step6 : (fourTimesℤ (negℤ s) +ℤ fourTimesℤ (negℤ s)) ≡ negℤ (eightTimesℤ s)
step6 =
  trans
    (cong (λ t → t +ℤ t) (sym (neg-fourTimesℤ s)))
    (sym (neg+ℤ (fourTimesℤ s) (fourTimesℤ s)))
in
  trans
    step1
    (trans
      step2
      (trans
        (cong (λ t → t +ℤ (fourTimesℤ (negℤ s) +ℤ fourTimesℤ (negℤ s))) step5)
        (trans
          (cong (λ t → eightTimesℤ s +ℤ t) step6)
          (+ℤ-inv-right (eightTimesℤ s))))))

```

Law 14G.13: $J_8(L_8 v) = 0$

Since J_8 broadcasts the global sum, Law 14G.12 immediately implies that J_8 annihilates every Laplacian image.

```

-- § J8 (L8 v) = 0
law14G-13-JL-zero : (v : Vec8ℤ) → Vec8Eq (J8Vec8ℤ (K8LaplacianVec8ℤ v)) zeroVec8ℤ
law14G-13-JL-zero v =
  let sum0 = law14G-12-sumL8-0 v in
  (λ _ → sum0) , (λ _ → sum0)

```

Law 14G.14: $L_8(J_8 v) = 0$

The converse composite vanishes for the complementary reason: applying J_8 first produces a constant vector, and constant vectors already lie in the kernel of the K_8 Laplacian.

```

-- § L8 (J8 v) = 0
law14G-14-LJ-zero : (v : Vec8ℤ) → Vec8Eq (K8LaplacianVec8ℤ (J8Vec8ℤ v)) zeroVec8ℤ
law14G-14-LJ-zero v =
  let s = sum8ℤ v in
  let sj : sum8ℤ (J8Vec8ℤ v) ≡ eightTimesℤ s
    sj =
      trans
        (cong (λ t → t +ℤ sumFin4ℤ (constVec4ℤ s)) (sumFin4-const s))
        (trans
          (cong (λ t → fourTimesℤ s +ℤ t) (sumFin4-const s))
          refl)
in
  (λ _ →
    trans
      (cong (λ t → eightTimesℤ s +ℤ negℤ t) sj)

```

```

      (+ℤ-inv-right (eightTimesℤ s))
    ),
    (λ _ →
      trans
        (cong (λ t → eightTimesℤ s + ℤ negℤ t) sj)
        (+ℤ-inv-right (eightTimesℤ s))
    )
  )

```

Law 14G.15: $L_8 \circ L_8 = 8 \cdot L_8$

Applying L_8 to a vector already in the image removes the sum term and leaves pure multiplication by 8. This is the precise analogue of the relation $L^2 = 4L$ from the four-vertex case.

```

-- § L82 = 8 · L8
law14G-15-LL-eightL : (v : Vec8ℤ) → Vec8Eq (K8LaplacianVec8ℤ (K8LaplacianVec8ℤ v)) (eightVec8ℤ (K8LaplacianVec8ℤ v))
law14G-15-LL-eightL v =
  let sum0 = law14G-12-sumL8-0 v in
  (λ i →
    trans
      (cong (λ t → eightTimesℤ (left4 (K8LaplacianVec8ℤ v) i) + ℤ negℤ t) sum0)
      (+ℤ-zero-right (eightTimesℤ (left4 (K8LaplacianVec8ℤ v) i))))
  ),
  (λ i →
    trans
      (cong (λ t → eightTimesℤ (right4 (K8LaplacianVec8ℤ v) i) + ℤ negℤ t) sum0)
      (+ℤ-zero-right (eightTimesℤ (right4 (K8LaplacianVec8ℤ v) i))))
  )

```

Spectral Classification on K_8

All remaining freedom in the spectrum of L_8 collapses here. The forcing chain pins the image of L_8 to the 8-eigenspace and the kernel to the constant subspace. The next laws make that statement exact: sum-zero vectors are precisely the 8-eigenvectors, constants are precisely the 0-eigenvectors, and the kernel condition can be rewritten in the broadcast form $8v = J_8 v$.

Law 14G.16: Sum-Zero Forces 8-Eigenvector

Each coordinate of $L_8 v$ is $8 \cdot v_i - \sum_8 v$. Imposing $\sum_8 v = 0$ eliminates the subtraction term everywhere, so every sum-zero vector lies in the 8-eigenspace.

```

-- §  $\sum_8 v = 0 \Rightarrow L_8 v = 8 \cdot v$ 
law14G-16-sum0-eigen8 : (v : Vec8ℤ) → sum8ℤ v ≡ 0ℤ → Vec8Eq (K8LaplacianVec8ℤ v) (eightVec8ℤ v)
law14G-16-sum0-eigen8 v sum0 =
  (λ i →

```

```

    trans
    (cong (λ s → eightTimesℤ (left4 v i) +ℤ negℤ s) sum0)
    (+ℤ-zero-right (eightTimesℤ (left4 v i)))
  ),
  (λ i →
    trans
    (cong (λ s → eightTimesℤ (right4 v i) +ℤ negℤ s) sum0)
    (+ℤ-zero-right (eightTimesℤ (right4 v i)))
  )
)

```

Law 14G.17: Pointwise 8-Eigen Forces Sum-Zero

If $L_8 v = 8 \cdot v$ pointwise, then at the first coordinate one has $8 \cdot v_0 - \Sigma_8 v = 8 \cdot v_0$. Cancelling the common term leaves $-\Sigma_8 v = 0$, so the 8-eigenspace is exactly the sum-zero subspace.

```

-- § L8 v = 8·v => Σ8 v = 0
law14G-17-eigen8→sum0 : (v : Vec8ℤ) → Vec8Eq (K8LaplacianVec8ℤ v) (eightVec8ℤ v) → sum8ℤ v ≡ 0ℤ
law14G-17-eigen8→sum0 v eigen8 =
  let a = eightTimesℤ (left4 v g0) in
  let s = sum8ℤ v in
  let eq0 : a +ℤ negℤ s ≡ a
    eq0 = fst eigen8 g0
  in
  negℤ-zero→zero s (+ℤ-cancel-left a (negℤ s) eq0)

```

Law 14G.18: Biconditional Sum-Zero $\Leftrightarrow$ 8-Eigen

Law 14G.16 and Law 14G.17 supply the two directions of the equivalence between sum-zero and 8-eigen. This subsection packages them into a reusable biconditional.

```

-- § sum0 ↔ eigen8 (both directions)
law14G-18-sum0→eigen8 : (v : Vec8ℤ) → sum8ℤ v ≡ 0ℤ → Vec8Eq (K8LaplacianVec8ℤ v) (eightVec8ℤ v)
law14G-18-sum0→eigen8 = law14G-16-sum0-eigen8

law14G-18-eigen8→sum0 : (v : Vec8ℤ) → Vec8Eq (K8LaplacianVec8ℤ v) (eightVec8ℤ v) → sum8ℤ v ≡ 0ℤ
law14G-18-eigen8→sum0 = law14G-17-eigen8→sum0

```

Law 14G.19: Constants Are Forced 0-Eigenvectors

A constant vector with value x has total sum $8 \cdot x$, and each coordinate of L_8 becomes $8 \cdot x - 8 \cdot x = 0$. The constant subspace therefore survives exactly as the 0-eigenspace.

```

-- § constant vectors lie in kernel of L8
law14G-19-const-eigen0 : (x : ℤ) → Vec8Eq (K8LaplacianVec8ℤ (constVec8ℤ x)) zeroVec8ℤ
law14G-19-const-eigen0 x =
  let sc : sum8ℤ (constVec8ℤ x) ≡ eightTimesℤ x
    sc =

```

```

trans
  (cong (λ t → t +ℤ sumFin4ℤ (constVec4ℤ x)) (sumFin4-const x))
(trans
  (cong (λ t → fourTimesℤ x +ℤ t) (sumFin4-const x))
  refl)
in
(λ _ →
  trans
    (cong (λ s → eightTimesℤ x +ℤ negℤ s) sc)
    (+ℤ-inv-right (eightTimesℤ x))
  ),
(λ _ →
  trans
    (cong (λ s → eightTimesℤ x +ℤ negℤ s) sc)
    (+ℤ-inv-right (eightTimesℤ x))
  )
)

```

Law 14G.20: Kernel Condition $L_8 v = 0 \Leftrightarrow 8 \cdot v = J_8 v$

The kernel equation at each coordinate reads $8 \cdot v_i - \Sigma_8 v = 0$. Moving the sum term across the equality forces $8 \cdot v_i = \Sigma_8 v$, which is precisely the broadcast equation $8 \cdot v = J_8 v$. This is the exact kernel analogue of the four-vertex identity $4v = Jv$.

```

-- § L8 v = 0 => 8·v = J8 v
law14G-20-L0→eightEq] : (v : Vec8ℤ) → Vec8Eq (K8LaplacianVec8ℤ v) zeroVec8ℤ → Vec8Eq (eightVec8ℤ v) (J8Vec8ℤ v)
law14G-20-L0→eightEq] v L0 =
  let s = sum8ℤ v in
  (λ i →
    let a = eightTimesℤ (left4 v i) in
    let eq0 : a +ℤ negℤ s ≡ 0ℤ
      eq0 = fst L0 i
    in
    let step1 : (a +ℤ negℤ s) +ℤ s ≡ 0ℤ +ℤ s
      step1 = cong (λ t → t +ℤ s) eq0
      step2 : a +ℤ (negℤ s +ℤ s) ≡ 0ℤ +ℤ s
      step2 = trans (sym (+ℤ-assoc a (negℤ s) s)) step1
      step3 : a +ℤ 0ℤ ≡ 0ℤ +ℤ s
      step3 = trans (sym (cong (λ t → a +ℤ t) (+ℤ-inv-left s))) step2
    in
    trans
      (trans (sym (+ℤ-zero-right a)) step3)
      (+ℤ-zero-left s)
  ),
  (λ i →
    let a = eightTimesℤ (right4 v i) in
    let eq0 : a +ℤ negℤ s ≡ 0ℤ
      eq0 = snd L0 i
    in
    let step1 : (a +ℤ negℤ s) +ℤ s ≡ 0ℤ +ℤ s
      step1 = cong (λ t → t +ℤ s) eq0
      step2 : a +ℤ (negℤ s +ℤ s) ≡ 0ℤ +ℤ s

```

```

    step2 = trans (sym (+ℤ-assoc a (negℤ s) s)) step1
    step3 : a + ℤ 0ℤ ≡ 0ℤ + ℤ s
    step3 = trans (sym (cong (λ t → a + ℤ t) (+ℤ-inv-left s))) step2
  in
  trans
    (trans (sym (+ℤ-zero-right a)) step3)
    (+ℤ-zero-left s)
)

-- § 8·v = J8 v => L8 v = 0
law14G-20-eightEq→L0 : (v : Vec8ℤ) → Vec8Eq (eightVec8ℤ v) (J8Vec8ℤ v) → Vec8Eq (K8LaplacianVec8ℤ v) zeroVec8ℤ
law14G-20-eightEq→L0 v eightEqf =
  let s = sum8ℤ v in
  ( λ i →
    trans
      (cong (λ t → t + ℤ negℤ s) (fst eightEqf i))
      (+ℤ-inv-right s)
  ),
  ( λ i →
    trans
      (cong (λ t → t + ℤ negℤ s) (snd eightEqf i))
      (+ℤ-inv-right s)
  )
)

```

Laws 14G.21–14G.22: Image Characterization

The image of L_8 is obtained by applying L_8 once, so Law 14G.15 applies immediately and pins every image vector to the 8-eigenspace. Law 14G.16 then shows that every sum-zero vector supplies an explicit preimage after 8-scaling. In this sense the image and the 8-eigenspace coincide.

The two statements below package the image description in the two directions that will be used later: every Laplacian image is an 8-eigenvector, and every sum-zero vector becomes an image after scaling by 8.

```

-- § image vectors are 8-eigenvectors
law14G-21-image⊆eigen8 : (v : Vec8ℤ) → Vec8Eq (K8LaplacianVec8ℤ (K8LaplacianVec8ℤ v)) (eightVec8ℤ (K8LaplacianVec8ℤ v))
law14G-21-image⊆eigen8 = law14G-15-LL-eightL

-- § sum-zero vectors become image vectors after 8-scaling
law14G-22-sum0→eightInImage : (w : Vec8ℤ) → sum8ℤ w ≡ 0ℤ → ∑ Vec8ℤ (λ v → Vec8Eq (K8LaplacianVec8ℤ v) (eightVec8ℤ w))
law14G-22-sum0→eightInImage w sum0 = w , law14G-16-sum0-eigen8 w sum0

```

Transport Lemmas

Pointwise equality must propagate through sums, scaling, and the K_8 Laplacian without bookkeeping ambiguity. Each lemma below fixes one such transport path. These are routine congruence lemmas, but they matter because the later survivor arguments

repeatedly transport spectral identities across pointwise equalities of $\text{Vec8}\mathbb{Z}$ -valued operators.

```

-- § Vec8Eq reflexivity
Vec8Eq-refl : (v : Vec8ℤ) → Vec8Eq v v
Vec8Eq-refl v = (λ _ → refl) , (λ _ → refl)

-- § Vec8Eq symmetry
Vec8Eq-sym : {u v : Vec8ℤ} → Vec8Eq u v → Vec8Eq v u
Vec8Eq-sym eq =
  (λ i → sym (fst eq i)) ,
  (λ i → sym (snd eq i))

-- § Vec8Eq transitivity
Vec8Eq-trans : {u v w : Vec8ℤ} → Vec8Eq u v → Vec8Eq v w → Vec8Eq u w
Vec8Eq-trans eq1 eq2 =
  (λ i → trans (fst eq1 i) (fst eq2 i)) ,
  (λ i → trans (snd eq1 i) (snd eq2 i))

-- § sumFin4 respects pointwise equality
sumFin4-cong : (f g : Vec4ℤ) → Vec4Eq f g → sumFin4ℤ f ≡ sumFin4ℤ g
sumFin4-cong f g eq =
  trans
    (cong (λ a → sum4ℤ a (f g1) (f g2) (f g3)) (eq g0))
  (trans
    (cong (λ b → sum4ℤ (g g0) b (f g2) (f g3)) (eq g1))
    (trans
      (cong (λ c → sum4ℤ (g g0) (g g1) c (f g3)) (eq g2))
      (cong (λ d → sum4ℤ (g g0) (g g1) (g g2) d (f g3)) (eq g3))))))

-- § sum8 respects Vec8Eq
sum8-cong : (u v : Vec8ℤ) → Vec8Eq u v → sum8ℤ u ≡ sum8ℤ v
sum8-cong u v eq =
  cong (λ t → t +ℤ sumFin4ℤ (right4 u)) (sumFin4-cong (left4 u) (left4 v) (fst eq))
  ► λ pL →
  trans pL
  (cong (λ t → sumFin4ℤ (left4 v) +ℤ t) (sumFin4-cong (right4 u) (right4 v) (snd eq)))
where
  infixl 1 _►_
  _►_ : {A B : Set} → A → (A → B) → B
  x ► k = k x

-- § eightVec8 respects Vec8Eq
eightVec8-cong : (u v : Vec8ℤ) → Vec8Eq u v → Vec8Eq (eightVec8ℤ u) (eightVec8ℤ v)
eightVec8-cong u v eq =
  (λ i → cong eightTimesℤ (fst eq i)) ,
  (λ i → cong eightTimesℤ (snd eq i))

-- § K8 Laplacian respects Vec8Eq
K8Laplacian-cong : (u v : Vec8ℤ) → Vec8Eq u v → Vec8Eq (K8LaplacianVec8ℤ u) (K8LaplacianVec8ℤ v)
K8Laplacian-cong u v eq =
  let sEq : sum8ℤ u ≡ sum8ℤ v
      sEq = sum8-cong u v eq
      nsEq : negℤ (sum8ℤ u) ≡ negℤ (sum8ℤ v)

```



```

      nsEq = cong negℤ sEq
in
( λ i →
  let aEq : eightTimesℤ (left4 u i) ≡ eightTimesℤ (left4 v i)
    aEq = cong eightTimesℤ (fst eq i)
  in
  trans
    (cong (λ t → t +ℤ negℤ (sum8ℤ u)) aEq)
    (cong (λ t → eightTimesℤ (left4 v i) +ℤ t) nsEq)
),
( λ i →
  let aEq : eightTimesℤ (right4 u i) ≡ eightTimesℤ (right4 v i)
    aEq = cong eightTimesℤ (snd eq i)
  in
  trans
    (cong (λ t → t +ℤ negℤ (sum8ℤ u)) aEq)
    (cong (λ t → eightTimesℤ (right4 v i) +ℤ t) nsEq)
)

```

Full Survivor: Forced Spectral Consequences

For the full survivor there is, in fact, no new operator to analyze. By Law 14G.8 the dispatched Laplacian is pointwise equal to L_8 , so the spectral theory established for L_8 carries over without alteration. What follows is therefore a sequence of transports: the sum law, the annihilation by J_8 , and the equivalence between the sum-zero condition and the 8-eigenvalue relation are simply the old K_8 facts viewed through the survivor dispatcher.

Law 14G.23: $\Sigma_8(L_{\text{full}} v) = 0$

Since the full survivor agrees pointwise with L_8 , the total sum of its image is computed by the same argument as before. One transports along `law14G-8-survivor-full-K8` using `sum8-cong` and then invokes Law 14G.12. Thus the full-survivor image lies entirely in the sum-zero subspace.

```

-- § global sum of the full-survivor Laplacian is 0
law14G-23-sumL-survivor-full-0 : (v : Vec8ℤ) → sum8ℤ (laplacianSurvivorVec8ℤ survivor-full v) ≡ 0ℤ
law14G-23-sumL-survivor-full-0 v =
  trans
    (sum8-cong (laplacianSurvivorVec8ℤ survivor-full v) (K8LaplacianVec8ℤ v) (law14G-8-survivor-full-K8 v))
    (law14G-12-sumL8-0 v)

```

Law 14G.24: $J_8(L_{\text{full}} v) = 0$

The operator J_8 depends only on the global sum of its input. Once Law 14G.23 has shown that this sum vanishes for $L_{\text{full}}v$, the action of J_8 collapses at once to the zero

vector. In this sense the image of the full-survivor Laplacian is orthogonal to the broadcast direction singled out by J_8 .

```
-- § J8 annihilates full-survivor image
law14G-24-JL-survivor-full-zero : (v : Vec8ℤ) → Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ survivor-full v)) zeroVec8ℤ
law14G-24-JL-survivor-full-zero v =
  let sum0 = law14G-23-sumL-survivor-full-0 v in
  (λ _ → sum0) , (λ _ → sum0)
```

Laws 14G.25–14G.26: Sum-Zero $\Leftrightarrow$ 8-Eigen for Full Survivor

No additional case distinction is needed for the eigenvalue statement either. Because the full survivor is merely L_8 in dispatched form, Law 14G.16 and Law 14G.17 pass across Law 14G.8 exactly as they stand. The proof of the forward direction composes the identification with the known implication for L_8 ; the reverse direction transports an assumed 8-eigen relation back to L_8 and applies the converse theorem there. Hence the familiar equivalence survives unchanged: for the full survivor, being sum-zero is exactly the same as being an 8-eigenvector.

```
-- § sum-zero → 8-eigenvector for full survivor
law14G-25-survivor-full-sum0→eigen8 : (v : Vec8ℤ) → sum8ℤ v ≡ 0ℤ → Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v)
law14G-25-survivor-full-sum0→eigen8 v sum0 =
  Vec8Eq-trans
  (law14G-8-survivor-full-K8 v)
  (law14G-16-sum0-eigen8 v sum0)

-- § 8-eigenvector → sum-zero for full survivor
law14G-26-survivor-full-eigen8→sum0 : (v : Vec8ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v) → sum8ℤ v ≡ 0ℤ
law14G-26-survivor-full-eigen8→sum0 v eigen8 =
  law14G-17-eigen8→sum0 v
  (Vec8Eq-trans
   (Vec8Eq-sym (law14G-8-survivor-full-K8 v))
   eigen8)
```

Empty Survivor: Forced Block Spectral Consequences

The empty survivor reduces to two independent K_4 blocks. Block diagonality eliminates every cross-term, so the surviving spectral laws are forced blockwise from the earlier K_4 development.

Law 14G.27: $\Sigma_8(L_{\text{empty}} v) = 0$

Block diagonality splits the total sum of the empty survivor into the left and right K_4 Laplacian sums. Law 14E.28 forces each block sum to vanish independently, so the empty-survivor image lies in the global sum-zero subspace.

```

-- § global sum of empty-survivor Laplacian is 0
law14G-27-sumL-survivor-empty-0 : (v : Vec8ℤ) → sum8ℤ (laplacianSurvivorVec8ℤ survivor-empty v) ≡ 0ℤ
law14G-27-sumL-survivor-empty-0 v =
  trans
    (cong (λ t → t + ℤ sumFin4ℤ (laplacianVec4ℤ (right4 v))) (law14E-28-sumLaplacian0 (left4 v)))
    (trans
      (cong (λ t → 0ℤ + ℤ t) (law14E-28-sumLaplacian0 (right4 v)))
      (+ℤ-zero-left 0ℤ))

```

Law 14G.28: $J_8(L_{\text{empty}} v) = 0$

Since J_8 depends only on the global sum of its input and Law 14G.27 eliminates that sum, the empty-survivor image is annihilated by J_8 . Nothing new happens here beyond substitution: once the global sum is known to vanish, the definition of J_8 immediately yields the zero vector.

```

-- § J8 annihilates empty-survivor image
law14G-28-JL-survivor-empty-zero : (v : Vec8ℤ) → Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v)) zeroVec8ℤ
law14G-28-JL-survivor-empty-zero v =
  let sum0 = law14G-27-sumL-survivor-empty-0 v in
    (λ _ → sum0) , (λ _ → sum0)

```

Laws 14G.29–14G.30: Blockwise Sum-Zero $\Leftrightarrow$ 4-Eigen for Empty Survivor

The empty survivor acts as two independent K_4 Laplacians. Law 14E.11 and Law 14E.19 therefore classify each block separately: blockwise sum-zero and pointwise 4-eigen are the surviving conditions, with no cross-block interaction left to analyze.

```

-- § blockwise sum-zero → pointwise 4-eigenvector for empty survivor
law14G-29-survivor-empty-sum0→eigen4 : (v : Vec8ℤ) → sumFin4ℤ (left4 v) ≡ 0ℤ → sumFin4ℤ (right4 v) ≡ 0ℤ →
  Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v)
law14G-29-survivor-empty-sum0→eigen4 v sum0L sum0R =
  ( λ i → law14E-11-sum0-eigen4 (left4 v) i sum0L ) ,
  ( λ i → law14E-11-sum0-eigen4 (right4 v) i sum0R )

-- § pointwise 4-eigen → blockwise sum-zero for empty survivor
law14G-30-survivor-empty-eigen4→sum0 : (v : Vec8ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v) →
  (sumFin4ℤ (left4 v) ≡ 0ℤ) × (sumFin4ℤ (right4 v) ≡ 0ℤ)
law14G-30-survivor-empty-eigen4→sum0 v eigen4 =
  law14E-19-eigen4→sum0 (left4 v) (fst eigen4) ,
  law14E-19-eigen4→sum0 (right4 v) (snd eigen4)

```

Law 14G.31: Split Constants Are 0-Eigenvectors of the Empty Survivor

On each block, a constant vector is already a forced 0-eigenvector of the K_4 Laplacian. Block diagonality prevents the two constants from interacting across blocks, so split constants survive exactly as kernel witnesses for the empty survivor. This is the first genuine difference between the two survivors: the empty survivor admits one constant mode on each block, whereas the full survivor allows only a single uniform constant mode.

```
-- § split constant vector
constVec8Split $\mathbb{Z}$  :  $\mathbb{Z} \rightarrow \mathbb{Z} \rightarrow \text{Vec8}\mathbb{Z}$ 
constVec8Split $\mathbb{Z}$  x y = constVec4 $\mathbb{Z}$  x , constVec4 $\mathbb{Z}$  y

-- § split constants lie in kernel of empty survivor
law14G-31-splitConst-eigen0-empty : (x y :  $\mathbb{Z}$ )  $\rightarrow$  Vec8Eq (laplacianSurvivorVec8 $\mathbb{Z}$  survivor-empty (constVec8Split $\mathbb{Z}$  x y)) zeroVec8 $\mathbb{Z}$ 
law14G-31-splitConst-eigen0-empty x y =
  (  $\lambda$  i  $\rightarrow$  law14E-13-const-eigen0 x i ) ,
  (  $\lambda$  i  $\rightarrow$  law14E-13-const-eigen0 y i )
```

Law 14G.32: Empty Survivor Image is 4-Eigen

Applying the empty survivor twice applies each K_4 Laplacian twice on its own block. Law 14E.29 collapses each repeated block action to 4 times the first image, forcing every vector in the empty-survivor image into the 4-eigenspace.

```
-- § image of empty survivor  $\subseteq$  4-eigenspace
law14G-32-imageEmpty $\subseteq$ eigen4 : (v : Vec8 $\mathbb{Z}$ )  $\rightarrow$ 
  Vec8Eq (laplacianSurvivorVec8 $\mathbb{Z}$  survivor-empty (laplacianSurvivorVec8 $\mathbb{Z}$  survivor-empty v))
  (fourVec8 $\mathbb{Z}$  (laplacianSurvivorVec8 $\mathbb{Z}$  survivor-empty v))
law14G-32-imageEmpty $\subseteq$ eigen4 v =
  (  $\lambda$  i  $\rightarrow$  law14E-29-LL-fourL (left4 v) i ) ,
  (  $\lambda$  i  $\rightarrow$  law14E-29-LL-fourL (right4 v) i )
```

Law 14G.33: Sum-Zero $\Rightarrow$ Image Witness After 4-Scaling

Law 14G.29 already forces a blockwise sum-zero vector to satisfy the empty-survivor eigenvalue equation. Choosing the vector itself as preimage therefore yields an explicit image witness after 4-scaling.

```
-- § blockwise sum-zero  $\rightarrow$  4·v is in the image
law14G-33-sum0 $\rightarrow$ fourInImage-empty : (v : Vec8 $\mathbb{Z}$ )  $\rightarrow$  sumFin4 $\mathbb{Z}$  (left4 v)  $\equiv$  0 $\mathbb{Z}$   $\rightarrow$  sumFin4 $\mathbb{Z}$  (right4 v)  $\equiv$  0 $\mathbb{Z}$   $\rightarrow$ 
   $\Sigma$  Vec8 $\mathbb{Z}$  ( $\lambda$  w  $\rightarrow$  Vec8Eq (laplacianSurvivorVec8 $\mathbb{Z}$  survivor-empty w) (fourVec8 $\mathbb{Z}$  v))
law14G-33-sum0 $\rightarrow$ fourInImage-empty v sum0L sum0R = v , law14G-29-survivor-empty-sum0 $\rightarrow$ eigen4 v sum0L sum0R
```

Full Survivor Image Law

The image law for the full survivor is a direct transport consequence. No new spectral case appears here; Law 14G.15 is merely re-expressed through the survivor identification.

Law 14G.34: Full Survivor Image $\subseteq$ 8-Eigenspace

The full survivor is identified with L_8 both before and after one further application, so the claim reduces to Law 14G.15 by transport on both sides. Scaling congruence carries the codomain back, forcing the full-survivor image into the 8-eigenspace.

```
-- § full survivor image vectors are 8-eigenvectors
law14G-34-survivor-full-image⊆eigen8 : (v : Vec8ℤ) →
  Vec8Eq (laplacianSurvivorVec8ℤ survivor-full (laplacianSurvivorVec8ℤ survivor-full v))
    (eightVec8ℤ (laplacianSurvivorVec8ℤ survivor-full v))
law14G-34-survivor-full-image⊆eigen8 v =
  let eqV : Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (K8LaplacianVec8ℤ v)
    eqV = law14G-8-survivor-full-K8 v
    eqLL : Vec8Eq (laplacianSurvivorVec8ℤ survivor-full (laplacianSurvivorVec8ℤ survivor-full v))
      (K8LaplacianVec8ℤ (laplacianSurvivorVec8ℤ survivor-full v))
    eqLL = law14G-8-survivor-full-K8 (laplacianSurvivorVec8ℤ survivor-full v)
    step1 : Vec8Eq (laplacianSurvivorVec8ℤ survivor-full (laplacianSurvivorVec8ℤ survivor-full v))
      (K8LaplacianVec8ℤ (K8LaplacianVec8ℤ v))

    step1 =
      Vec8Eq-trans
      eqLL
      (K8Laplacian-cong (laplacianSurvivorVec8ℤ survivor-full v) (K8LaplacianVec8ℤ v) eqV)
    step2 : Vec8Eq (K8LaplacianVec8ℤ (K8LaplacianVec8ℤ v)) (eightVec8ℤ (K8LaplacianVec8ℤ v))
    step2 = law14G-15-LL-eightL v
    step3 : Vec8Eq (eightVec8ℤ (K8LaplacianVec8ℤ v)) (eightVec8ℤ (laplacianSurvivorVec8ℤ survivor-full v))
    step3 = Vec8Eq-sym (eightVec8-cong (laplacianSurvivorVec8ℤ survivor-full v) (K8LaplacianVec8ℤ v) eqV)

  in
  Vec8Eq-trans step1 (Vec8Eq-trans step2 step3)
```

Survivor Spectral Packages

The remaining task is purely organizational: the already established survivor consequences are collected into reusable spectral packages.

Law 14G.35: Full Survivor Spectral Package

Law 14G.23, Law 14G.24, Laws 14G.25–14G.26, and Law 14G.34 already contain the full-survivor spectral data. The package below simply collects these facts into one reusable witness.

```
-- § full survivor spectral package
law14G-35-survivor-full-spectral-package : (v : Vec8ℤ) →
  (sum8ℤ (laplacianSurvivorVec8ℤ survivor-full v) ≡ 0ℤ) ×
  (Vec8Eq (j8Vec8ℤ (laplacianSurvivorVec8ℤ survivor-full v)) zeroVec8ℤ) ×
  ((sum8ℤ v ≡ 0ℤ → Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v)) ×
   (Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v) → sum8ℤ v ≡ 0ℤ)) ×
  (Vec8Eq (laplacianSurvivorVec8ℤ survivor-full (laplacianSurvivorVec8ℤ survivor-full v))
   (eightVec8ℤ (laplacianSurvivorVec8ℤ survivor-full v)))
law14G-35-survivor-full-spectral-package v =
  law14G-23-sumL-survivor-full-0 v ,
  (law14G-24-JL-survivor-full-zero v ,
   ((law14G-25-survivor-full-sum0→eigen8 v , law14G-26-survivor-full-eigen8→sum0 v) ,
    law14G-34-survivor-full-image⊆eigen8 v)))
```

Law 14G.36: Empty Survivor Spectral Package

Law 14G.27 through Law 14G.32 already contain the empty-survivor spectral data. The package below plays the same role on the decoupled side, with the additional split-constant kernel witness carried along explicitly.

```
-- § empty survivor spectral package
law14G-36-survivor-empty-spectral-package : (v : Vec8ℤ) →
  (sum8ℤ (laplacianSurvivorVec8ℤ survivor-empty v) ≡ 0ℤ) ×
  (Vec8Eq (j8Vec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v)) zeroVec8ℤ) ×
  ((sumFin4ℤ (left4 v) ≡ 0ℤ → sumFin4ℤ (right4 v) ≡ 0ℤ → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v)) ×
   (Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v) → (sumFin4ℤ (left4 v) ≡ 0ℤ) × (sumFin4ℤ (right4 v) ≡ 0ℤ))) ×
  ((x y : ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (constVec8Splitℤ x y)) zeroVec8ℤ) ×
  (Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (laplacianSurvivorVec8ℤ survivor-empty v))
   (fourVec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v)))
law14G-36-survivor-empty-spectral-package v =
  law14G-27-sumL-survivor-empty-0 v ,
  (law14G-28-JL-survivor-empty-zero v ,
   ((law14G-29-survivor-empty-sum0→eigen4 v , law14G-30-survivor-empty-eigen4→sum0 v) ,
    (law14G-31-splitConst-eigen0-empty ,
     law14G-32-imageEmpty⊆eigen4 v)))
```

Law 14G.37: Spectral Package by Forced Case Split

The survivor type has exactly two constructors by Law 14G.6. An exhaustive case split therefore leaves only `survivor-empty` and `survivor-full`, and each of these two cases already carries its own spectral package.

```
-- § type-level spectral package indexed by survivor
SurvivorSpectralPackage : CouplingSurvivor → Vec8ℤ → Set
SurvivorSpectralPackage survivor-full v =
  (sum8ℤ (laplacianSurvivorVec8ℤ survivor-full v) ≡ 0ℤ) ×
  (Vec8Eq (j8Vec8ℤ (laplacianSurvivorVec8ℤ survivor-full v)) zeroVec8ℤ) ×
  ((sum8ℤ v ≡ 0ℤ → Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v)) ×
   (Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v) → sum8ℤ v ≡ 0ℤ)) ×
```

```

(Vec8Eq (laplacianSurvivorVec8ℤ survivor-full (laplacianSurvivorVec8ℤ survivor-full v))
  (eightVec8ℤ (laplacianSurvivorVec8ℤ survivor-full v)))
SurvivorSpectralPackage survivor-empty v =
  (sum8ℤ (laplacianSurvivorVec8ℤ survivor-empty v) ≡ 0ℤ) ×
  (Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v)) zeroVec8ℤ) ×
  ((sumFin4ℤ (left4 v) ≡ 0ℤ → sumFin4ℤ (right4 v) ≡ 0ℤ → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v)) ×
    (Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v) → (sumFin4ℤ (left4 v) ≡ 0ℤ) × (sumFin4ℤ (right4 v) ≡ 0ℤ))) ×
  ((x y : ℤ) → Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (constVec8Splitℤ x y)) zeroVec8ℤ) ×
  (Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (laplacianSurvivorVec8ℤ survivor-empty v))
    (fourVec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v)))

-- § forced case-split over survivor constructors
law14G-37-survivor-spectral-package-byCases : (k : CouplingSurvivor) (v : Vec8ℤ) → SurvivorSpectralPackage k v
law14G-37-survivor-spectral-package-byCases k v with law14G-6-survivor-cases k
... | inj₁ refl = law14G-36-survivor-empty-spectral-package v
... | inj₂ refl = law14G-35-survivor-full-spectral-package v

```

Spectral Package Projections

Each forced component of the survivor package is projected directly from the already fixed product witness, without reopening the classification. These lemmas simply recover the individual spectral facts from the packaged witness, so later arguments do not need to repeat the case split.

```

-- § project drift-zero from package
survivorPkg-sumL0 : {k : CouplingSurvivor} {v : Vec8ℤ} →
  SurvivorSpectralPackage k v → sum8ℤ (laplacianSurvivorVec8ℤ k v) ≡ 0ℤ
survivorPkg-sumL0 {survivor-full} pkg = fst pkg
survivorPkg-sumL0 {survivor-empty} pkg = fst pkg

-- § project J1=0 from package
survivorPkg-JL0 : {k : CouplingSurvivor} {v : Vec8ℤ} →
  SurvivorSpectralPackage k v → Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ k v)) zeroVec8ℤ
survivorPkg-JL0 {survivor-full} pkg = fst (snd pkg)
survivorPkg-JL0 {survivor-empty} pkg = fst (snd pkg)

-- § sum-zero predicate indexed by survivor
SurvivorSum0 : CouplingSurvivor → Vec8ℤ → Set
SurvivorSum0 survivor-full v = sum8ℤ v ≡ 0ℤ
SurvivorSum0 survivor-empty v = (sumFin4ℤ (left4 v) ≡ 0ℤ) × (sumFin4ℤ (right4 v) ≡ 0ℤ)

-- § eigen predicate indexed by survivor
SurvivorEigen : (k : CouplingSurvivor) → Vec8ℤ → Set
SurvivorEigen survivor-full v =
  Vec8Eq (laplacianSurvivorVec8ℤ survivor-full v) (eightVec8ℤ v)
SurvivorEigen survivor-empty v =
  Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty v) (fourVec8ℤ v)

-- § project sum0→eigen direction
survivorPkg-sum0→eigen : {k : CouplingSurvivor} {v : Vec8ℤ} →
  SurvivorSpectralPackage k v → SurvivorSum0 k v → SurvivorEigen k v

```

```

survivorPkg-sum0→eigen {survivor-full} (⊆, (⊆, ((sum0→eigen, ⊆), ⊆))) sum0 = sum0→eigen sum0
survivorPkg-sum0→eigen {survivor-empty} (⊆, (⊆, ((sum0→eigen, ⊆), ⊆))) (sum0L, sum0R) = sum0→eigen sum0L sum0R

-- $ project eigen→sum0 direction
survivorPkg-eigen→sum0 : {k : CouplingSurvivor} {v : Vec8ℤ} →
  SurvivorSpectralPackage k v → SurvivorEigen k v → SurvivorSum0 k v
survivorPkg-eigen→sum0 {survivor-full} (⊆, (⊆, ((⊆, eigen→sum0), ⊆))) eigen = eigen→sum0 eigen
survivorPkg-eigen→sum0 {survivor-empty} (⊆, (⊆, ((⊆, eigen→sum0), ⊆))) eigen = eigen→sum0 eigen

-- $ image-eigen predicate indexed by survivor
SurvivorImageEigen : (k : CouplingSurvivor) → Vec8ℤ → Set
SurvivorImageEigen survivor-full v =
  Vec8Eq (laplacianSurvivorVec8ℤ survivor-full (laplacianSurvivorVec8ℤ survivor-full v))
    (eightVec8ℤ (laplacianSurvivorVec8ℤ survivor-full v))
SurvivorImageEigen survivor-empty v =
  Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (laplacianSurvivorVec8ℤ survivor-empty v))
    (fourVec8ℤ (laplacianSurvivorVec8ℤ survivor-empty v))

-- $ project image⊆eigen from package
survivorPkg-image⊆eigen : {k : CouplingSurvivor} {v : Vec8ℤ} →
  SurvivorSpectralPackage k v → SurvivorImageEigen k v
survivorPkg-image⊆eigen {survivor-full} (⊆, (⊆, (⊆, image⊆))) = image⊆
survivorPkg-image⊆eigen {survivor-empty} (⊆, (⊆, (⊆, image⊆))) = image⊆

-- $ project split-constant kernel from empty package
survivorPkg-splitConstKernel : {v : Vec8ℤ} →
  SurvivorSpectralPackage survivor-empty v → (x y : ℤ) →
  Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (constVec8Splitℤ x y)) zeroVec8ℤ
survivorPkg-splitConstKernel (⊆, (⊆, (⊆, (splitConstKer, ⊆)))) = splitConstKer

```

Convenience Layer

Once the survivor index and vector are fixed, Law 14G.37 determines the package witness, and every convenience projection then follows without further case analysis. The final wrappers merely expose those packaged facts in a convenient form, so later arguments can call them uniformly without mentioning the internal package structure.

```

-- $ construct package from (k, v)
survivorPkg : (k : CouplingSurvivor) (v : Vec8ℤ) → SurvivorSpectralPackage k v
survivorPkg = law14G-37-survivor-spectral-package-byCases

-- $ convenience: sumL0
survivor-sumL0 : (k : CouplingSurvivor) (v : Vec8ℤ) → sum8ℤ (laplacianSurvivorVec8ℤ k v) ≡ 0ℤ
survivor-sumL0 k v = survivorPkg-sumL0 (survivorPkg k v)

-- $ convenience: JL=0
survivor-JL0 : (k : CouplingSurvivor) (v : Vec8ℤ) → Vec8Eq (J8Vec8ℤ (laplacianSurvivorVec8ℤ k v)) zeroVec8ℤ
survivor-JL0 k v = survivorPkg-JL0 (survivorPkg k v)

-- $ convenience: sum0→eigen

```



```

survivor-sum0→eigen : (k : CouplingSurvivor) (v : Vec8ℤ) → SurvivorSum0 k v → SurvivorEigen k v
survivor-sum0→eigen k v sum0 = survivorPkg-sum0→eigen (survivorPkg k v) sum0

-- § convenience: eigen→sum0
survivor-eigen→sum0 : (k : CouplingSurvivor) (v : Vec8ℤ) → SurvivorEigen k v → SurvivorSum0 k v
survivor-eigen→sum0 k v eigen = survivorPkg-eigen→sum0 (survivorPkg k v) eigen

-- § convenience: image⊆eigen
survivor-image⊆eigen : (k : CouplingSurvivor) (v : Vec8ℤ) → SurvivorImageEigen k v
survivor-image⊆eigen k v = survivorPkg-image⊆eigen (survivorPkg k v)

-- § convenience: split-constant kernel
survivor-splitConstKernel : (v : Vec8ℤ) (x y : ℤ) →
  Vec8Eq (laplacianSurvivorVec8ℤ survivor-empty (constVec8Splitℤ x y)) zeroVec8ℤ
survivor-splitConstKernel v x y = survivorPkg-splitConstKernel {v} (survivorPkg survivor-empty v) x y

```


Chapter 45

Triple K_4 Laplacian

The two-copy coupled Laplacian of the previous chapter reduced to a binary classification: full join or disjoint union, with eigenvalue 8 governing the spectral theory. For three copies the same alternative reappears, now under the copy symmetry of S_3 and the endomorphism symmetries on each K_4 factor. The goal of this chapter is to show that the intermediate couplings are again eliminated and that the resulting 12-vertex operator algebra admits the corresponding spectral packages.

Copy₃ Infrastructure

Three copies, decidable equality for each pair, and the full symmetric group S_3 supply the minimal copy infrastructure. Once the copy type, its equality test, and the six explicit bijections exhausting the copy symmetries are fixed, later transport arguments can proceed without any hidden choices. The definitions that follow do exactly this and nothing more.

```
-- § three indistinguishable copies
data Copy3 : Set where
  C0 : Copy3
  C1 : Copy3
  C2 : Copy3

-- § copy-inequality predicate
Copy3≠ : (i j : Copy3) → Set
Copy3≠ i j = i ≡ j → ⊥

C0≠C1 : Copy3≠ C0 C1
C0≠C1 ()

C0≠C2 : Copy3≠ C0 C2
C0≠C2 ()

C1≠C2 : Copy3≠ C1 C2
```

```

C1≠C2 ()

-- § decidable equality on Copy3 (9 cases)
Copy3-decEq : (i j : Copy3) → (i ≡ j) ⊔ (Copy3≠ i j)
Copy3-decEq C0 C0 = inj1 refl
Copy3-decEq C1 C1 = inj1 refl
Copy3-decEq C2 C2 = inj1 refl
Copy3-decEq C0 C1 = inj2 C0≠C1
Copy3-decEq C1 C0 = inj2 (λ e → C0≠C1 (sym e))
Copy3-decEq C0 C2 = inj2 C0≠C2
Copy3-decEq C2 C0 = inj2 (λ e → C0≠C2 (sym e))
Copy3-decEq C1 C2 = inj2 C1≠C2
Copy3-decEq C2 C1 = inj2 (λ e → C1≠C2 (sym e))

-- § copy permutations (S3 as explicit bijections)
record CopyPerm : Set where
  field
    to      : Copy3 → Copy3
    from    : Copy3 → Copy3
    to-from : (y : Copy3) → to (from y) ≡ y
    from-to : (x : Copy3) → from (to x) ≡ x

open CopyPerm public

-- § identity permutation
permId3 : CopyPerm
permId3 = record
{ to = λ x → x
; from = λ x → x
; to-from = λ _ → refl
; from-to = λ _ → refl
}

-- § swap C0 ↔ C1
permSwap01 : CopyPerm
permSwap01 = record
{ to = λ where
    C0 → C1
    C1 → C0
    C2 → C2
; from = λ where
    C0 → C1
    C1 → C0
    C2 → C2
; to-from = λ where
    C0 → refl
    C1 → refl
    C2 → refl
; from-to = λ where
    C0 → refl
    C1 → refl
    C2 → refl
}

```

Step 2: Remaining Permutations. The swap $C_0 \leftrightarrow C_2$ and $C_1 \leftrightarrow C_2$ complete the explicit list of generators and cycles for the symmetric group S_3 on three copies.

```
-- § swap C0 ↔ C2
permSwap02 : CopyPerm
permSwap02 = record
{ to = λ where
  C0 → C2
  C1 → C1
  C2 → C0
; from = λ where
  C0 → C2
  C1 → C1
  C2 → C0
; to-from = λ where
  C0 → refl
  C1 → refl
  C2 → refl
; from-to = λ where
  C0 → refl
  C1 → refl
  C2 → refl
}

-- § swap C1 ↔ C2
permSwap12 : CopyPerm
permSwap12 = record
{ to = λ where
  C0 → C0
  C1 → C2
  C2 → C1
; from = λ where
  C0 → C0
  C1 → C2
  C2 → C1
; to-from = λ where
  C0 → refl
  C1 → refl
  C2 → refl
; from-to = λ where
  C0 → refl
  C1 → refl
  C2 → refl
}

-- § 3-cycle C0 → C1 → C2 → C0
permCycle012 : CopyPerm
permCycle012 = record
{ to = λ where
  C0 → C1
  C1 → C2
  C2 → C0
; from = λ where
```

```

    C0 → C2
    C1 → C0
    C2 → C1
; to-from = λ where
    C0 → refl
    C1 → refl
    C2 → refl
; from-to = λ where
    C0 → refl
    C1 → refl
    C2 → refl
}

-- § 3-cycle C0 → C2 → C1 → C0
permCycle021 : CopyPerm
permCycle021 = record
{ to = λ where
    C0 → C2
    C1 → C0
    C2 → C1
; from = λ where
    C0 → C1
    C1 → C2
    C2 → C0
; to-from = λ where
    C0 → refl
    C1 → refl
    C2 → refl
; from-to = λ where
    C0 → refl
    C1 → refl
    C2 → refl
}

```

Copy Transport and Cross-Coupling

The transport laws for three-copy couplings are fixed here. The four-argument transport lemma and the exhaustive 6×6 pair table make explicit that any witness on one ordered distinct copy pair can be moved to any other such pair. Once this is available, the later join laws no longer need to remember where the original witness was found.

```

-- § transport across four arguments (copies + endpoints)
transport4 : {C : Copy3 → Copy3 → EndoCase → EndoCase → Set}
{c c' d d' : Copy3} {a a' b b' : EndoCase} →
c ≡ c' → d ≡ d' → a ≡ a' → b ≡ b' → C c d a b → C c' d' a' b'
transport4 {C = C} {c = c} {c' = c} {d = d} {d' = d} {a = a} {a' = a} {b = b} {b' = b} ec ed ea eb cab =
subst (λ c0 → C c0 d' a' b') ec
(subst (λ d0 → C c d0 a' b') ed
(subst (λ a0 → C c d a0 b') ea
(subst (λ b0 → C c d a b0) eb cab)))

```

```

-- § cross-coupling predicate among three copies
Coupling3 : Set1
Coupling3 = Copy3 → Copy3 → EndoCase → EndoCase → Set

EndoInv3 : Coupling3 → Set
EndoInv3 C = (c d : Copy3) → CrossInv (λ a b → C c d a b)

CopyInv3 : Coupling3 → Set
CopyInv3 C = (π : CopyPerm) → (c d : Copy3) → (a b : EndoCase) → C c d a b → C (to π c) (to π d) a b

-- § copy-pair transitivity: any ordered distinct pair maps to any other
sendPair3 : (c0 d0 c d : Copy3) → Copy3 ≠ c0 d0 → Copy3 ≠ c d →
  Σ CopyPerm (λ π → (to π c0 ≡ c) × (to π d0 ≡ d))
sendPair3 C0 C0 c d neq0 _ = ⊥-elim (neq0 refl)
sendPair3 C1 C1 c d neq0 _ = ⊥-elim (neq0 refl)
sendPair3 C2 C2 c d neq0 _ = ⊥-elim (neq0 refl)

sendPair3 C0 C1 C0 C0 _ neq = ⊥-elim (neq refl)
sendPair3 C0 C1 C1 C1 _ neq = ⊥-elim (neq refl)
sendPair3 C0 C1 C2 C2 _ neq = ⊥-elim (neq refl)

sendPair3 C0 C2 C0 C0 _ neq = ⊥-elim (neq refl)
sendPair3 C0 C2 C1 C1 _ neq = ⊥-elim (neq refl)
sendPair3 C0 C2 C2 C2 _ neq = ⊥-elim (neq refl)

sendPair3 C1 C0 C0 C0 _ neq = ⊥-elim (neq refl)
sendPair3 C1 C0 C1 C1 _ neq = ⊥-elim (neq refl)
sendPair3 C1 C0 C2 C2 _ neq = ⊥-elim (neq refl)

sendPair3 C1 C2 C0 C0 _ neq = ⊥-elim (neq refl)
sendPair3 C1 C2 C1 C1 _ neq = ⊥-elim (neq refl)
sendPair3 C1 C2 C2 C2 _ neq = ⊥-elim (neq refl)

sendPair3 C2 C0 C0 C0 _ neq = ⊥-elim (neq refl)
sendPair3 C2 C0 C1 C1 _ neq = ⊥-elim (neq refl)
sendPair3 C2 C0 C2 C2 _ neq = ⊥-elim (neq refl)

sendPair3 C2 C1 C0 C0 _ neq = ⊥-elim (neq refl)
sendPair3 C2 C1 C1 C1 _ neq = ⊥-elim (neq refl)
sendPair3 C2 C1 C2 C2 _ neq = ⊥-elim (neq refl)

sendPair3 C0 C1 C0 C1 _ _ = permId3 , (refl , refl)
sendPair3 C0 C1 C0 C2 _ _ = permSwap12 , (refl , refl)
sendPair3 C0 C1 C1 C0 _ _ = permSwap01 , (refl , refl)
sendPair3 C0 C1 C1 C2 _ _ = permCycle012 , (refl , refl)
sendPair3 C0 C1 C2 C0 _ _ = permCycle021 , (refl , refl)
sendPair3 C0 C1 C2 C1 _ _ = permSwap02 , (refl , refl)

sendPair3 C0 C2 C0 C1 _ _ = permSwap12 , (refl , refl)
sendPair3 C0 C2 C0 C2 _ _ = permId3 , (refl , refl)
sendPair3 C0 C2 C1 C0 _ _ = permCycle012 , (refl , refl)
sendPair3 C0 C2 C1 C2 _ _ = permSwap01 , (refl , refl)
sendPair3 C0 C2 C2 C0 _ _ = permSwap02 , (refl , refl)
sendPair3 C0 C2 C2 C1 _ _ = permCycle021 , (refl , refl)

```

```

sendPair3 C1 C0 C0 C1 __ = permSwap01 , (refl , refl)
sendPair3 C1 C0 C0 C2 __ = permCycle021 , (refl , refl)
sendPair3 C1 C0 C1 C0 __ = permId3 , (refl , refl)
sendPair3 C1 C0 C1 C2 __ = permSwap02 , (refl , refl)
sendPair3 C1 C0 C2 C0 __ = permSwap12 , (refl , refl)
sendPair3 C1 C0 C2 C1 __ = permCycle012 , (refl , refl)

sendPair3 C1 C2 C0 C1 __ = permCycle021 , (refl , refl)
sendPair3 C1 C2 C0 C2 __ = permSwap01 , (refl , refl)
sendPair3 C1 C2 C1 C0 __ = permSwap02 , (refl , refl)
sendPair3 C1 C2 C1 C2 __ = permId3 , (refl , refl)
sendPair3 C1 C2 C2 C0 __ = permCycle012 , (refl , refl)
sendPair3 C1 C2 C2 C1 __ = permSwap12 , (refl , refl)

sendPair3 C2 C0 C0 C1 __ = permCycle012 , (refl , refl)
sendPair3 C2 C0 C0 C2 __ = permSwap02 , (refl , refl)
sendPair3 C2 C0 C1 C0 __ = permSwap12 , (refl , refl)
sendPair3 C2 C0 C1 C2 __ = permCycle021 , (refl , refl)
sendPair3 C2 C0 C2 C0 __ = permId3 , (refl , refl)
sendPair3 C2 C0 C2 C1 __ = permSwap01 , (refl , refl)

sendPair3 C2 C1 C0 C1 __ = permSwap02 , (refl , refl)
sendPair3 C2 C1 C0 C2 __ = permCycle012 , (refl , refl)
sendPair3 C2 C1 C1 C0 __ = permCycle021 , (refl , refl)
sendPair3 C2 C1 C1 C2 __ = permSwap12 , (refl , refl)
sendPair3 C2 C1 C2 C0 __ = permSwap01 , (refl , refl)
sendPair3 C2 C1 C2 C1 __ = permId3 , (refl , refl)

```

Coupling Classification

The binary coupling dichotomy for three copies is fixed here. One cross-edge witness forces the complete join, one cross-non-edge witness forces total separation, and no intermediate coupling survives the copy and endomorphism symmetries.

```

-- § Law 14H.0: one cross-edge forces complete join across all copies
law14H-0-edge-forces-all-cross3 : (C : Coupling3) → EndoInv3 C → CopyInv3 C →
  ∑ Copy3 (λ k0 → ∑ Copy3 (λ k1 → (Copy3≠ k0 k1) × ∑ EndoCase (λ a0 → ∑ EndoCase (λ b0 → C k0 k1 a0 b0)))) →
  (c d : Copy3) → Copy3≠ c d → (a b : EndoCase) → C c d a b
law14H-0-edge-forces-all-cross3 C endoInv copyInv (k0 , (k1 , (k0≠k1 , (a0 , (b0 , e0)))) c d c≠d a b =
let
  pair = sendPair3 k0 k1 c d k0≠k1 c≠d
in
let
  π = fst pair
  eqs = snd pair
  ec = fst eqs
  ed = snd eqs
  movedEdge : C c d a0 b0
  movedEdge = transport4 {C = C} ec ed refl refl (copyInv π k0 k1 a0 b0 e0)
in

```



```

law14F-0-edge-forces-all (λ x y → C c d x y) (endoInv c d) (a0, (b0, movedEdge)) a b

-- § Law 14H.1: one cross-non-edge forces disjoint union across all copies
law14H-1-nonedge-forces-none-cross3 : (C : Coupling3) → EndoInv3 C → CopyInv3 C →
  Σ Copy3 (λ k0 → Σ Copy3 (λ k1 → (Copy3≠ k0 k1) × Σ EndoCase (λ a0 → Σ EndoCase (λ b0 → ¬ (C k0 k1 a0 b0)))) →
  (c d : Copy3) → Copy3≠ c d → (a b : EndoCase) → ¬ (C c d a b)
law14H-1-nonedge-forces-none-cross3 C endoInv copyInv (k0, (k1, (k0≠k1, (a0, (b0, n0)))) c d c≠d a b cab =
let
  pair = sendPair3 c d k0 k1 c≠d k0≠k1
in
let
  π = fst pair
  eqs = snd pair
  ec = fst eqs
  ed = snd eqs
  moved : C k0 k1 a b
  moved = transport4 {C = C} ec ed refl refl (copyInv π c d a b cab)
in
law14F-1-nonedge-forces-none (λ x y → C k0 k1 x y) (endoInv k0 k1) (a0, (b0, n0)) a b moved

-- § canonical survivor couplings
CrossEmpty3 : Coupling3
CrossEmpty3 _ _ _ _ = ⊥

CrossFull3 : Coupling3
CrossFull3 _ _ _ _ = ⊤

```

Vec₁₂ Infrastructure

A 12-vertex vector is represented as three blocks of four, together with the corresponding block projections, pointwise equality, global sum, all-ones operator, and scalar multiplication by 12. With this decomposition fixed, the empty survivor becomes the direct sum of three K_4 Laplacians, while the full survivor is represented by the single opaque K_{12} Laplacian.

```

-- § Vec12ℤ = three blocks of Vec4ℤ
Vec12ℤ : Set
Vec12ℤ = Vec4ℤ × (Vec4ℤ × Vec4ℤ)

-- § block projections
block0 : Vec12ℤ → Vec4ℤ
block0 = fst

block1 : Vec12ℤ → Vec4ℤ
block1 v = fst (snd v)

block2 : Vec12ℤ → Vec4ℤ
block2 v = snd (snd v)

-- § pointwise equality on Vec12ℤ
Vec12Eq : Vec12ℤ → Vec12ℤ → Set

```

```

Vec12Eq u v = Vec4Eq (block0 u) (block0 v) × Vec4Eq (block1 u) (block1 v) × Vec4Eq (block2 u) (block2 v)

-- § global sum:  $\Sigma_{12} v = \Sigma_4(\text{block}_0) + \Sigma_4(\text{block}_1) + \Sigma_4(\text{block}_2)$ 
sum12 $\mathbb{Z}$  : Vec12 $\mathbb{Z} \rightarrow \mathbb{Z}$ 
sum12 $\mathbb{Z} v = \text{sumFin4}\mathbb{Z} (\text{block}_0 v) + \mathbb{Z} (\text{sumFin4}\mathbb{Z} (\text{block}_1 v) + \mathbb{Z} \text{sumFin4}\mathbb{Z} (\text{block}_2 v))$ 

-- § global-sum operator  $J_{12}$ : broadcast  $\Sigma_{12}$  to all 12 entries
J12Vec12 $\mathbb{Z}$  : Vec12 $\mathbb{Z} \rightarrow \text{Vec12}\mathbb{Z}$ 
J12Vec12 $\mathbb{Z} v = (\lambda \_ \rightarrow \text{sum12}\mathbb{Z} v), ((\lambda \_ \rightarrow \text{sum12}\mathbb{Z} v), (\lambda \_ \rightarrow \text{sum12}\mathbb{Z} v))$ 

-- §  $12 \cdot x = 4 \cdot x + 8 \cdot x$  (forced decomposition)
twelveTimes $\mathbb{Z}$  :  $\mathbb{Z} \rightarrow \mathbb{Z}$ 
twelveTimes $\mathbb{Z} x = \text{fourTimes}\mathbb{Z} x + \mathbb{Z} \text{eightTimes}\mathbb{Z} x$ 

-- §  $K_{12}$  Laplacian (opaque)
opaque
K12LaplacianVec12 $\mathbb{Z}$  : Vec12 $\mathbb{Z} \rightarrow \text{Vec12}\mathbb{Z}$ 
K12LaplacianVec12 $\mathbb{Z} v =$ 
   $(\lambda i \rightarrow \text{twelveTimes}\mathbb{Z} (\text{block}_0 v i) + \mathbb{Z} \text{neg}\mathbb{Z} (\text{sum12}\mathbb{Z} v)),$ 
   $((\lambda i \rightarrow \text{twelveTimes}\mathbb{Z} (\text{block}_1 v i) + \mathbb{Z} \text{neg}\mathbb{Z} (\text{sum12}\mathbb{Z} v)),$ 
   $(\lambda i \rightarrow \text{twelveTimes}\mathbb{Z} (\text{block}_2 v i) + \mathbb{Z} \text{neg}\mathbb{Z} (\text{sum12}\mathbb{Z} v)))$ 

-- § block-diagonal Laplacian (empty coupling)
laplacianEmptyVec12 $\mathbb{Z}$  : Vec12 $\mathbb{Z} \rightarrow \text{Vec12}\mathbb{Z}$ 
laplacianEmptyVec12 $\mathbb{Z} v = \text{laplacianVec4}\mathbb{Z} (\text{block}_0 v), (\text{laplacianVec4}\mathbb{Z} (\text{block}_1 v), \text{laplacianVec4}\mathbb{Z} (\text{block}_2 v))$ 

-- § full coupling =  $K_{12}$  Laplacian
laplacianFullVec12 $\mathbb{Z}$  : Vec12 $\mathbb{Z} \rightarrow \text{Vec12}\mathbb{Z}$ 
laplacianFullVec12 $\mathbb{Z} = \text{K12LaplacianVec12}\mathbb{Z}$ 

```

Survivor Dispatch and Vec_{12} Operations

The two survivor actions on $\text{Vec}_{12}\mathbb{Z}$ and the operations required to compare them are fixed here. The block-diagonal form, the full K_{12} form, and the binary survivor dispatch are made explicit so that the later spectral statements can be written once and then specialized to either survivor.

```

-- § Law 14H.2: empty coupling is block-diagonal
law14H-2-empty-block : (v : Vec12 $\mathbb{Z}$ )  $\rightarrow$ 
  Vec12Eq (laplacianEmptyVec12 $\mathbb{Z} v$ )
   $(\text{laplacianVec4}\mathbb{Z} (\text{block}_0 v), (\text{laplacianVec4}\mathbb{Z} (\text{block}_1 v), \text{laplacianVec4}\mathbb{Z} (\text{block}_2 v)))$ 
law14H-2-empty-block v =  $(\lambda \_ \rightarrow \text{refl}), ((\lambda \_ \rightarrow \text{refl}), (\lambda \_ \rightarrow \text{refl}))$ 

-- § Law 14H.3: full coupling collapses to  $K_{12}$  spectral form
law14H-3-full-is-K12 : (v : Vec12 $\mathbb{Z}$ )  $\rightarrow$  Vec12Eq (laplacianFullVec12 $\mathbb{Z} v$ ) (K12LaplacianVec12 $\mathbb{Z} v$ )
law14H-3-full-is-K12 v =  $(\lambda \_ \rightarrow \text{refl}), ((\lambda \_ \rightarrow \text{refl}), (\lambda \_ \rightarrow \text{refl}))$ 

-- § two survivor kinds for the triple coupling
data Coupling3Survivor : Set where
  survivor3-empty : Coupling3Survivor
  survivor3-full : Coupling3Survivor

```

```

-- § Law 14H.4: binary case split
law14H-4-survivor3-cases : (k : Coupling3Survivor) → (k ≡ survivor3-empty) ⊔ (k ≡ survivor3-full)
law14H-4-survivor3-cases survivor3-empty = inj1 refl
law14H-4-survivor3-cases survivor3-full = inj2 refl

-- § survivor dispatch
laplacianSurvivorVec12ℤ : Coupling3Survivor → Vec12ℤ → Vec12ℤ
laplacianSurvivorVec12ℤ survivor3-empty = laplacianEmptyVec12ℤ
laplacianSurvivorVec12ℤ survivor3-full = laplacianFullVec12ℤ

-- § Vec12ℤ arithmetic operations
_+Vec12ℤ_ : Vec12ℤ → Vec12ℤ → Vec12ℤ
(u +Vec12ℤ v) =
  (block0 u +Vec4ℤ block0 v) ,
  ((block1 u +Vec4ℤ block1 v) ,
   (block2 u +Vec4ℤ block2 v))

negVec12ℤ : Vec12ℤ → Vec12ℤ
negVec12ℤ v =
  (λ i → negℤ (block0 v i)) ,
  ((λ i → negℤ (block1 v i)) ,
   (λ i → negℤ (block2 v i)))

twelveVec4ℤ : Vec4ℤ → Vec4ℤ
twelveVec4ℤ v i = twelveTimesℤ (v i)

twelveVec12ℤ : Vec12ℤ → Vec12ℤ
twelveVec12ℤ v = twelveVec4ℤ (block0 v) , (twelveVec4ℤ (block1 v) , twelveVec4ℤ (block2 v))

constVec12ℤ : ℤ → Vec12ℤ
constVec12ℤ x = constVec4ℤ x , (constVec4ℤ x , constVec4ℤ x)

zeroVec12ℤ : Vec12ℤ
zeroVec12ℤ = constVec12ℤ 0ℤ

-- § sum of a constant vector
sum12-const : (x : ℤ) → sum12ℤ (constVec12ℤ x) ≡ twelveTimesℤ x
sum12-const x = refl

-- § sum of J12 v
sum12-J12 : (v : Vec12ℤ) → sum12ℤ (J12Vec12ℤ v) ≡ twelveTimesℤ (sum12ℤ v)
sum12-J12 v = refl

```

K₁₂ Operator Algebra

Once L_{12} is fixed as $12I - J_{12}$, the operator algebra follows in the same way as for K_4 and K_8 . Because J_{12} broadcasts the global sum, the identities $J^2 = 12J$, $L + J = 12I$, and the remaining operator equalities are obtained by direct blockwise computation.

```

-- § Law 14H.5: J122 = 12·J12
law14H-5-JJ-twelveJ : (v : Vec12ℤ) → Vec12Eq (J12Vec12ℤ (J12Vec12ℤ v)) (twelveVec12ℤ (J12Vec12ℤ v))
law14H-5-JJ-twelveJ v =
  (λ _ → sum12-J12 v) ,
  ((λ _ → sum12-J12 v) ,
   (λ _ → sum12-J12 v))

```

Laplacian operator identities. With $J^2 = 12J$ established, the remaining operator identities require unfolding the opaque Laplacian definition. The decomposition $12v = Lv + Jv$, scalar distributivity of $12\times$, and the sum–Laplacian commutation all follow inside a single opaque scope.

```
opaque
unfolding K12LaplacianVec12ℤ

-- § Law 14H.6:  $L_{12} = 12 \cdot I - J_{12}$ 
law14H-6-L-twelve-minus-J : (v : Vec12ℤ) →
  Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v + Vec12ℤ negVec12ℤ (J12Vec12ℤ v))
law14H-6-L-twelve-minus-J v =
  (λ _ → refl) ,
  ((λ _ → refl) ,
   (λ _ → refl))

-- § Law 14H.7:  $12 \cdot v = L_{12} v + J_{12} v$ 
law14H-7-twelve-decomposes : (v : Vec12ℤ) →
  Vec12Eq (K12LaplacianVec12ℤ v + Vec12ℤ J12Vec12ℤ v) (twelveVec12ℤ v)
law14H-7-twelve-decomposes v =
  let
    s = sum12ℤ v

  cancel-twelve : (x : ℤ) → (x + ℤ negℤ s) + ℤ s ≡ x
  cancel-twelve x =
    trans
      (+ℤ-assoc x (negℤ s) s)
    (trans
      (cong (λ t → x + ℤ t) (+ℤ-inv-left s))
      (+ℤ-zero-right x))
  in
  ( λ i →
    cancel-twelve (twelveTimesℤ (block0 v i))
  ) ,
  (( λ i →
    cancel-twelve (twelveTimesℤ (block1 v i))
  ) ,
   ( λ i →
    cancel-twelve (twelveTimesℤ (block2 v i))
  ))

-- § scalar helpers: twelveTimes distributes over addition and negation
twelveTimes-+ℤ : (x y : ℤ) → twelveTimesℤ (x + ℤ y) ≡ twelveTimesℤ x + ℤ twelveTimesℤ y
twelveTimes-+ℤ x y =
  let fx = fourTimesℤ x in
  let fy = fourTimesℤ y in
  let ex = eightTimesℤ x in
  let ey = eightTimesℤ y in
  trans
    refl
  (trans
    (cong (λ t → t + ℤ eightTimesℤ (x + ℤ y)) (fourTimes-+ℤ x y))
    (trans
```

```

    (cong (λ t → (fx +ℤ fy) +ℤ t) (eightTimes+ℤ x y))
  (trans
    (+ℤ-assoc fx fy (ex +ℤ ey))
  (trans
    (cong (λ t → fx +ℤ t) (swapHeadℤ fy ex ey))
  (trans
    (sym (+ℤ-assoc fx ex (fy +ℤ ey)))
  refl))))))

twelveTimes-neg : (x : ℤ) → twelveTimesℤ (negℤ x) ≡ negℤ (twelveTimesℤ x)
twelveTimes-neg x =
  trans
    refl
  (trans
    (cong (λ t → t +ℤ eightTimesℤ (negℤ x)) (sym (neg-fourTimesℤ x)))
  (trans
    (cong (λ t → negℤ (fourTimesℤ x) +ℤ t) (eightTimes-neg x))
  (trans
    (sym (neg+ℤ (fourTimesℤ x) (eightTimesℤ x)))
  refl)))

-- § sumFin4 commutes with twelveTimes
sumFin4-twelveTimes : (v : Vec4ℤ) →
  sumFin4ℤ (λ i → twelveTimesℤ (v i)) ≡ twelveTimesℤ (sumFin4ℤ v)
sumFin4-twelveTimes v =
  let fv : Vec4ℤ
    fv i = fourTimesℤ (v i)
  in
  let ev : Vec4ℤ
    ev i = eightTimesℤ (v i)
  in
  trans
    (sumFin4+Vec4ℤ fv ev)
  (trans
    (cong (λ t → t +ℤ sumFin4ℤ ev) (sumFin4-fourTimes v))
  (trans
    (cong (λ t → fourTimesℤ (sumFin4ℤ v) +ℤ t) (sumFin4-eightTimes v))
  refl)))

-- § reassociation helper for triple-block sum
reassoc3-addConst : (A B C k : ℤ) →
  (A +ℤ k) +ℤ ((B +ℤ k) +ℤ (C +ℤ k)) ≡ (A +ℤ (B +ℤ C)) +ℤ (k +ℤ (k +ℤ k))
reassoc3-addConst A B C k =
  let
    x = A +ℤ k
    y = B +ℤ k
    z = C +ℤ k

  step1 : x +ℤ (y +ℤ z) ≡ (x +ℤ y) +ℤ z
  step1 = sym (+ℤ-assoc x y z)

  step2 : x +ℤ y ≡ (A +ℤ B) +ℤ (k +ℤ k)
  step2 =
    trans

```

```

(+ℤ-assoc A k (B +ℤ k))
(trans
  (cong (λ t → A +ℤ t) (swapHeadℤ k B k))
  (sym (+ℤ-assoc A B (k +ℤ k))))

step3 : (x +ℤ y) +ℤ z ≡ ((A +ℤ B) +ℤ (k +ℤ k)) +ℤ (C +ℤ k)
step3 = cong (λ t → t +ℤ z) step2

step4 : ((A +ℤ B) +ℤ (k +ℤ k)) +ℤ (C +ℤ k) ≡ (A +ℤ B) +ℤ ((k +ℤ k) +ℤ (C +ℤ k))
step4 = +ℤ-assoc (A +ℤ B) (k +ℤ k) (C +ℤ k)

step5 : (k +ℤ k) +ℤ (C +ℤ k) ≡ C +ℤ ((k +ℤ k) +ℤ k)
step5 = swapHeadℤ (k +ℤ k) C k

step6 : ((A +ℤ B) +ℤ C) ≡ A +ℤ (B +ℤ C)
step6 = +ℤ-assoc A B C

step7 : ((k +ℤ k) +ℤ k) ≡ k +ℤ (k +ℤ k)
step7 = +ℤ-assoc k k k
in
trans
  step1
  (trans
    step3
    (trans
      step4
      (trans
        (cong (λ t → (A +ℤ B) +ℤ t) step5)
        (trans
          (sym (+ℤ-assoc (A +ℤ B) C ((k +ℤ k) +ℤ k)))
          (trans
            (cong (λ t → t +ℤ ((k +ℤ k) +ℤ k)) step6)
            (cong (λ t → (A +ℤ (B +ℤ C)) +ℤ t) step7)))))))

-- § Law 14H.8: global sum of  $K_{12}$  Laplacian = 0
law14H-8-sumL12-0 : (v : Vec12ℤ) → sum12ℤ (K12LaplacianVec12ℤ v) ≡ 0ℤ
law14H-8-sumL12-0 v =
let
  s = sum12ℤ v
  s0 = sumFin4ℤ (block0 v)
  s1 = sumFin4ℤ (block1 v)
  s2 = sumFin4ℤ (block2 v)

  part0 = λ i → twelveTimesℤ (block0 v i) +ℤ negℤ s
  part1 = λ i → twelveTimesℤ (block1 v i) +ℤ negℤ s
  part2 = λ i → twelveTimesℤ (block2 v i) +ℤ negℤ s

  neg4s = fourTimesℤ (negℤ s)

  raw0 = sumFin4ℤ (λ i → twelveTimesℤ (block0 v i)) +ℤ neg4s
  raw1 = sumFin4ℤ (λ i → twelveTimesℤ (block1 v i)) +ℤ neg4s
  raw2 = sumFin4ℤ (λ i → twelveTimesℤ (block2 v i)) +ℤ neg4s

  term0 = twelveTimesℤ s0 +ℤ neg4s
  term1 = twelveTimesℤ s1 +ℤ neg4s

```

```

term2 = twelveTimesℤ s2 +ℤ neg4s

triple = twelveTimesℤ s0 +ℤ (twelveTimesℤ s1 +ℤ twelveTimesℤ s2)

tailSum = neg4s +ℤ (neg4s +ℤ neg4s)

step0 :
  sum12ℤ (K12LaplacianVec12ℤ v) ≡ sumFin4ℤ part0 +ℤ (sumFin4ℤ part1 +ℤ sumFin4ℤ part2)
step0 = refl

step1 :
  sumFin4ℤ part0 ≡ raw0
step1 = sumFin4-addConst (λ i → twelveTimesℤ (block0 v i)) (negℤ s)

step2 :
  sumFin4ℤ part1 ≡ raw1
step2 = sumFin4-addConst (λ i → twelveTimesℤ (block1 v i)) (negℤ s)

step3 :
  sumFin4ℤ part2 ≡ raw2
step3 = sumFin4-addConst (λ i → twelveTimesℤ (block2 v i)) (negℤ s)

step4 :
  sum12ℤ (K12LaplacianVec12ℤ v) ≡ raw0 +ℤ (raw1 +ℤ raw2)
step4 =
  trans
    step0
    (trans
      (cong (λ t → t +ℤ (sumFin4ℤ part1 +ℤ sumFin4ℤ part2)) step1)
      (trans
        (cong (λ t → raw0 +ℤ t)
          (cong (λ t → t +ℤ sumFin4ℤ part2) step2))
        (cong (λ t → raw0 +ℤ (raw1 +ℤ t)) step3)))

step5 :
  raw0 ≡ term0
step5 = cong (λ t → t +ℤ neg4s) (sumFin4-twelveTimes (block0 v))

step6 :
  raw1 ≡ term1
step6 = cong (λ t → t +ℤ neg4s) (sumFin4-twelveTimes (block1 v))

step7 :
  raw2 ≡ term2
step7 = cong (λ t → t +ℤ neg4s) (sumFin4-twelveTimes (block2 v))

step8 :
  sum12ℤ (K12LaplacianVec12ℤ v) ≡ term0 +ℤ (term1 +ℤ term2)
step8 =
  trans
    step4
    (trans
      (cong (λ t → t +ℤ (raw1 +ℤ raw2)) step5)
      (trans
        (cong (λ t → term0 +ℤ t)

```

```

    (cong (λ t → t +ℤ raw2) step6))
    (cong (λ t → term0 +ℤ (term1 +ℤ t)) step7)))

step9 :
  term0 +ℤ (term1 +ℤ term2) ≡ triple +ℤ tailSum
step9 = reassoc3-addConst (twelveTimesℤ s0) (twelveTimesℤ s1) (twelveTimesℤ s2) neg4s

step10 : triple ≡ twelveTimesℤ s
step10 =
  trans
    (cong (λ t → twelveTimesℤ s0 +ℤ t) (sym (twelveTimes+ℤ s1 s2)))
    (sym (twelveTimes+ℤ s0 (s1 +ℤ s2)))

step11 : tailSum ≡ negℤ (twelveTimesℤ s)
step11 = trans refl (twelveTimes-neg s)

step12 :
  triple +ℤ tailSum ≡ twelveTimesℤ s +ℤ tailSum
step12 = cong (λ t → t +ℤ tailSum) step10

step13 : twelveTimesℤ s +ℤ tailSum ≡ twelveTimesℤ s +ℤ negℤ (twelveTimesℤ s)
step13 = cong (λ t → twelveTimesℤ s +ℤ t) step11

step14 : twelveTimesℤ s +ℤ negℤ (twelveTimesℤ s) ≡ 0ℤ
step14 = +ℤ-inv-right (twelveTimesℤ s)

step15 : twelveTimesℤ s +ℤ tailSum ≡ 0ℤ
step15 = trans step13 step14

step16 = trans step9 (trans step12 step15)
in
trans step8 step16

```

K_{12} Operator Annihilation

The three relations $J_{12} \circ L_{12} = 0$, $L_{12} \circ J_{12} = 0$, and $L_{12}^2 = 12 \cdot L_{12}$ all come from the same source. Once the previous law has shown that every Laplacian image has total sum zero, the mixed composites vanish because J_{12} reads only that sum, while the second application of L_{12} reduces to twelve times the first. The minimal polynomial $X(X - 12)$ is already visible at the operator level.

```

-- § Law 14H.9:  $J_{12}(L_{12} v) = 0$ 
law14H-9-JL-zero : (v : Vec12ℤ) → Vec12Eq (J12Vec12ℤ (K12LaplacianVec12ℤ v)) zeroVec12ℤ
law14H-9-JL-zero v =
  let sum0 = law14H-8-sumL12-0 v in
  (λ _ → sum0) ,
  ((λ _ → sum0) ,
  (λ _ → sum0))

-- § Law 14H.10:  $L_{12}(J_{12} v) = 0$ 
law14H-10-LJ-zero : (v : Vec12ℤ) → Vec12Eq (K12LaplacianVec12ℤ (J12Vec12ℤ v)) zeroVec12ℤ

```



```

law14H-10-LJ-zero v =
  let s = sum12ℤ v in
  let sj = sum12-J12 v in
  ( λ _ →
    trans
      (cong (λ t → twelveTimesℤ s + ℤ negℤ t) sj)
      (+ℤ-inv-right (twelveTimesℤ s))
    ),
  (( λ _ →
    trans
      (cong (λ t → twelveTimesℤ s + ℤ negℤ t) sj)
      (+ℤ-inv-right (twelveTimesℤ s))
    ),
  ( λ _ →
    trans
      (cong (λ t → twelveTimesℤ s + ℤ negℤ t) sj)
      (+ℤ-inv-right (twelveTimesℤ s))
    ))

-- § Law 14H.11: L122 = 12·L12
law14H-11-LL-twelveL : (v : Vec12ℤ) → Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (twelveVec12ℤ (K12LaplacianVec12ℤ v))
law14H-11-LL-twelveL v =
  let sum0 = law14H-8-sumL12-0 v in
  ( λ i →
    trans
      (cong (λ t → twelveTimesℤ (block0 (K12LaplacianVec12ℤ v) i) + ℤ negℤ t) sum0)
      (+ℤ-zero-right (twelveTimesℤ (block0 (K12LaplacianVec12ℤ v) i)))
    ),
  (( λ i →
    trans
      (cong (λ t → twelveTimesℤ (block1 (K12LaplacianVec12ℤ v) i) + ℤ negℤ t) sum0)
      (+ℤ-zero-right (twelveTimesℤ (block1 (K12LaplacianVec12ℤ v) i)))
    ),
  ( λ i →
    trans
      (cong (λ t → twelveTimesℤ (block2 (K12LaplacianVec12ℤ v) i) + ℤ negℤ t) sum0)
      (+ℤ-zero-right (twelveTimesℤ (block2 (K12LaplacianVec12ℤ v) i)))
    ))

```

K₁₂ Spectral Classification

The spectral picture of L_{12} is now complete. Vectors of total sum zero are carried to their twelvefold multiple, constant vectors lie in the kernel, and the operator identities show that every Laplacian image satisfies the same 12-eigen relation. Conversely, the eigenvalue equation rewrites to the sum-zero condition, so the 12-eigenspace and the constant kernel appear as the two basic spectral regimes.

```

-- § Law 14H.12: sum-zero ⇒ 12-eigenvector
law14H-12-sum0-eigen12 : (v : Vec12ℤ) → sum12ℤ v ≡ 0ℤ → Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v)
law14H-12-sum0-eigen12 v sum0 =

```

```

(λ i →
  trans
    (cong (λ s → twelveTimesℤ (block₀ v i) +ℤ negℤ s) sum0)
    (+ℤ-zero-right (twelveTimesℤ (block₀ v i)))
),
((λ i →
  trans
    (cong (λ s → twelveTimesℤ (block₁ v i) +ℤ negℤ s) sum0)
    (+ℤ-zero-right (twelveTimesℤ (block₁ v i)))
),
(λ i →
  trans
    (cong (λ s → twelveTimesℤ (block₂ v i) +ℤ negℤ s) sum0)
    (+ℤ-zero-right (twelveTimesℤ (block₂ v i)))
))

-- § Law 14H.13: 12-eigenvector ⇒ sum-zero
law14H-13-eigen12→sum0 : (v : Vec12ℤ) → Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v) → sum12ℤ v ≡ 0ℤ
law14H-13-eigen12→sum0 v eigen12 =
  let a = twelveTimesℤ (block₀ v g0) in
  let s = sum12ℤ v in
  let eq0 : a +ℤ negℤ s ≡ a
    eq0 = fst eigen12 g0
  in
  negℤ-zero→zero s (+ℤ-cancel-left a (negℤ s) eq0)

-- § Law 14H.14: constant vectors lie in the kernel
law14H-14-const-eigen0 : (x : ℤ) → Vec12Eq (K12LaplacianVec12ℤ (constVec12ℤ x)) zeroVec12ℤ
law14H-14-const-eigen0 x =
  (λ _ →
    trans
      (cong (λ s → twelveTimesℤ x +ℤ negℤ s) (sum12-const x))
      (+ℤ-inv-right (twelveTimesℤ x))
  ),
  ((λ _ →
    trans
      (cong (λ s → twelveTimesℤ x +ℤ negℤ s) (sum12-const x))
      (+ℤ-inv-right (twelveTimesℤ x))
  ),
  (λ _ →
    trans
      (cong (λ s → twelveTimesℤ x +ℤ negℤ s) (sum12-const x))
      (+ℤ-inv-right (twelveTimesℤ x))
  ))

-- § Law 14H.15: kernel condition - L₁₂ v = 0 ⇔ 12·v = J₁₂ v
law14H-15-L0→twelveEq : (v : Vec12ℤ) → Vec12Eq (K12LaplacianVec12ℤ v) zeroVec12ℤ → Vec12Eq (twelveVec12ℤ v) (J12Vec12ℤ v)
law14H-15-L0→twelveEq v L0 =
  let s = sum12ℤ v in
  (λ i →
    let a = twelveTimesℤ (block₀ v i) in
    let eq0 : a +ℤ negℤ s ≡ 0ℤ
      eq0 = fst L0 i
    in

```

```

let step1 : (a + ℤ neg ℤ s) + ℤ s ≡ 0 ℤ + ℤ s
  step1 = cong (λ t → t + ℤ s) eq0
  step2 : a + ℤ (neg ℤ s + ℤ s) ≡ 0 ℤ + ℤ s
  step2 = trans (sym (+ℤ-assoc a (neg ℤ s) s)) step1
  step3 : a + ℤ 0 ℤ ≡ 0 ℤ + ℤ s
  step3 = trans (sym (cong (λ t → a + ℤ t) (+ℤ-inv-left s))) step2
in
trans
  (trans (sym (+ℤ-zero-right a)) step3)
  (+ℤ-zero-left s)
),
(( λ i →
  let a = twelveTimesℤ (block1 v i) in
  let eq0 : a + ℤ neg ℤ s ≡ 0 ℤ
    eq0 = fst (snd L0) i
  in
  let step1 : (a + ℤ neg ℤ s) + ℤ s ≡ 0 ℤ + ℤ s
    step1 = cong (λ t → t + ℤ s) eq0
    step2 : a + ℤ (neg ℤ s + ℤ s) ≡ 0 ℤ + ℤ s
    step2 = trans (sym (+ℤ-assoc a (neg ℤ s) s)) step1
    step3 : a + ℤ 0 ℤ ≡ 0 ℤ + ℤ s
    step3 = trans (sym (cong (λ t → a + ℤ t) (+ℤ-inv-left s))) step2
  in
  trans
    (trans (sym (+ℤ-zero-right a)) step3)
    (+ℤ-zero-left s)
  ),
( λ i →
  let a = twelveTimesℤ (block2 v i) in
  let eq0 : a + ℤ neg ℤ s ≡ 0 ℤ
    eq0 = snd (snd L0) i
  in
  let step1 : (a + ℤ neg ℤ s) + ℤ s ≡ 0 ℤ + ℤ s
    step1 = cong (λ t → t + ℤ s) eq0
    step2 : a + ℤ (neg ℤ s + ℤ s) ≡ 0 ℤ + ℤ s
    step2 = trans (sym (+ℤ-assoc a (neg ℤ s) s)) step1
    step3 : a + ℤ 0 ℤ ≡ 0 ℤ + ℤ s
    step3 = trans (sym (cong (λ t → a + ℤ t) (+ℤ-inv-left s))) step2
  in
  trans
    (trans (sym (+ℤ-zero-right a)) step3)
    (+ℤ-zero-left s)
  ))

```

law14H-15-twelveEqJ→L0 : (v : Vec12ℤ) → Vec12Eq (twelveVec12ℤ v) (J12Vec12ℤ v) → Vec12Eq (K12LaplacianVec12ℤ v) zeroVec12ℤ
 law14H-15-twelveEqJ→L0 v twelveEqJ =

```

let s = sum12ℤ v in
( λ i →
  trans
    (cong (λ t → t + ℤ neg ℤ s) (fst twelveEqJ i))
    (+ℤ-inv-right s)
  ),
(( λ i →

```

```

    trans
      (cong ( $\lambda t \rightarrow t + \mathbb{Z} \text{ neg } \mathbb{Z} s$ ) (fst (snd twelveEqf) i))
      (+ $\mathbb{Z}$ -inv-right s)
    ),
  ( $\lambda i \rightarrow$ 
    trans
      (cong ( $\lambda t \rightarrow t + \mathbb{Z} \text{ neg } \mathbb{Z} s$ ) (snd (snd twelveEqf) i))
      (+ $\mathbb{Z}$ -inv-right s)
    ))

-- § Law 14H.16: image vectors are 12-eigenvectors
law14H-16-image $\subseteq$ eigen12 : ( $v : \text{Vec } 12\mathbb{Z}$ )  $\rightarrow$  Vec12Eq (K12LaplacianVec12 $\mathbb{Z}$  (K12LaplacianVec12 $\mathbb{Z}$  v)) (twelveVec12 $\mathbb{Z}$  (K12LaplacianVec12 $\mathbb{Z}$  v))
law14H-16-image $\subseteq$ eigen12 = law14H-11-LL-twelveL

-- § Law 14H.17: sum-zero  $\Rightarrow$  twelve-scaled image witness
law14H-17-sum0 $\rightarrow$ twelveInImage : ( $w : \text{Vec } 12\mathbb{Z}$ )  $\rightarrow$  sum12 $\mathbb{Z}$  w  $\equiv 0\mathbb{Z} \rightarrow \Sigma \text{Vec } 12\mathbb{Z} (\lambda v \rightarrow \text{Vec } 12\text{Eq} (\text{K12LaplacianVec } 12\mathbb{Z} v) (\text{twelveVec } 12\mathbb{Z} v))$ 
law14H-17-sum0 $\rightarrow$ twelveInImage w sum0 = w , law14H-12-sum0-eigen12 w sum0

```

Full Survivor Spectral Package

Nothing essentially new happens in this section. The package introduced below simply gathers the global-sum law, the annihilation laws, and the spectral biconditional already proved for the full survivor into a single witness suitable for later transport.

```

-- § full-survivor spectral package type
Survivor3FullSpectralPackage : Vec12 $\mathbb{Z} \rightarrow \text{Set}$ 
Survivor3FullSpectralPackage v =
  (sum12 $\mathbb{Z}$  (laplacianSurvivorVec12 $\mathbb{Z}$  survivor3-full v)  $\equiv 0\mathbb{Z}$ )  $\times$ 
  (Vec12Eq (J12Vec12 $\mathbb{Z}$  (laplacianSurvivorVec12 $\mathbb{Z}$  survivor3-full v)) zeroVec12 $\mathbb{Z}$ )  $\times$ 
  ((sum12 $\mathbb{Z}$  v  $\equiv 0\mathbb{Z} \rightarrow \text{Vec } 12\text{Eq} (\text{laplacianSurvivorVec } 12\mathbb{Z} \text{ survivor3-full } v) (\text{twelveVec } 12\mathbb{Z} v)) \times$ 
   (Vec12Eq (laplacianSurvivorVec12 $\mathbb{Z}$  survivor3-full v) (twelveVec12 $\mathbb{Z}$  v)  $\rightarrow$  sum12 $\mathbb{Z}$  v  $\equiv 0\mathbb{Z}$ ))  $\times$ 
  (Vec12Eq (laplacianSurvivorVec12 $\mathbb{Z}$  survivor3-full (laplacianSurvivorVec12 $\mathbb{Z}$  survivor3-full v))
    (twelveVec12 $\mathbb{Z}$  (laplacianSurvivorVec12 $\mathbb{Z}$  survivor3-full v))))

-- § Law 14H.18: full survivor spectral package
law14H-18-survivor3-full-spectral-package : ( $v : \text{Vec } 12\mathbb{Z}$ )  $\rightarrow$  Survivor3FullSpectralPackage v
law14H-18-survivor3-full-spectral-package v =
  law14H-8-sumL12-0 v ,
  (law14H-9-JL-zero v ,
   ((law14H-12-sum0-eigen12 v , law14H-13-eigen12 $\rightarrow$ sum0 v) ,
    law14H-16-image $\subseteq$ eigen12 v))

```

Spectrum Rigidity: Eigen-Residue is Pinned to $\Sigma_{12}v$

The operator identities of the previous sections establish $L_{12}^2 = 12L_{12}$, $J_{12}^2 = 12J_{12}$, $L_{12}J_{12} = J_{12}L_{12} = 0$, and the kernel-image relations. What remains is to understand

how these relations constrain possible integer eigen-actions on an arbitrary vector $v \in \text{Vec}12\mathbb{Z}$. The key observation is pointwise: for every entry of every block,

$$12 \cdot v_i - (L_{12}v)_i = \Sigma_{12}v.$$

After unfolding $L_{12} = 12I - J_{12}$, the term $12 \cdot v_i$ cancels and only the global sum remains. The residue is therefore the same at every coordinate.

This already determines the possible eigenvalues. If functions $e_k : \text{Fin}4 \rightarrow \mathbb{Z}$ describe the coordinatewise action of L_{12} , then the identity above forces $12 \cdot v_{k,i} - e_k(i) = \Sigma_{12}v$ for all (k, i) . In the linear case $e_k(i) = \lambda \cdot v_{k,i}$, comparing two distinct coordinates yields

$$(12 - \lambda) \cdot (v_{k_1, i_1} - v_{k_2, i_2}) = 0,$$

so over $\mathbb{Z}$ one is led to the familiar alternatives $\lambda = 0$ or $\lambda = 12$.

```
opaque
unfolding K12LaplacianVec12ℤ

-- § Law 14H.19a: residue at every entry of block0 is  $\Sigma_{12} v$ .
law14H-19-residue-pin-block0 :
  (v : Vec12ℤ) (i : Fin4) →
    twelveTimesℤ (block0 v i) +ℤ negℤ (block0 (K12LaplacianVec12ℤ v) i)
    ≡ sum12ℤ v
law14H-19-residue-pin-block0 v i =
  let t = twelveTimesℤ (block0 v i)
      s = sum12ℤ v
  in
  trans
    (cong (λ q → t +ℤ q)
      (trans (neg+ℤ t (negℤ s))
        (cong (λ r → negℤ t +ℤ r) (negℤ-involutive s))))
    (trans (sym (+ℤ-assoc t (negℤ t) s))
      (trans (cong (λ q → q +ℤ s) (+ℤ-inv-right t))
        (+ℤ-zero-left s)))

-- § Law 14H.19b: residue at every entry of block1 is  $\Sigma_{12} v$ .
law14H-19-residue-pin-block1 :
  (v : Vec12ℤ) (i : Fin4) →
    twelveTimesℤ (block1 v i) +ℤ negℤ (block1 (K12LaplacianVec12ℤ v) i)
    ≡ sum12ℤ v
law14H-19-residue-pin-block1 v i =
  let t = twelveTimesℤ (block1 v i)
      s = sum12ℤ v
  in
  trans
    (cong (λ q → t +ℤ q)
      (trans (neg+ℤ t (negℤ s))
        (cong (λ r → negℤ t +ℤ r) (negℤ-involutive s))))
    (trans (sym (+ℤ-assoc t (negℤ t) s))
      (trans (cong (λ q → q +ℤ s) (+ℤ-inv-right t))
        (+ℤ-zero-left s)))
```

```

-- § Law 14H.19c: residue at every entry of block2 is  $\Sigma_{12} v$ .
law14H-19-residue-pin-block2 :
  (v : Vec12 $\mathbb{Z}$ ) (i : Fin4) →
    twelveTimes $\mathbb{Z}$  (block2 v i) +  $\mathbb{Z}$  neg $\mathbb{Z}$  (block2 (K12LaplacianVec12 $\mathbb{Z}$  v) i)
    ≡ sum12 $\mathbb{Z}$  v
law14H-19-residue-pin-block2 v i =
  let t = twelveTimes $\mathbb{Z}$  (block2 v i)
      s = sum12 $\mathbb{Z}$  v
  in
  trans
  (cong (λ q → t +  $\mathbb{Z}$  q)
    (trans (neg+ $\mathbb{Z}$  t (neg $\mathbb{Z}$  s))
      (cong (λ r → neg $\mathbb{Z}$  t +  $\mathbb{Z}$  r) (neg $\mathbb{Z}$ -involutive s))))
  (trans (sym (+ $\mathbb{Z}$ -assoc t (neg $\mathbb{Z}$  t) s))
    (trans (cong (λ q → q +  $\mathbb{Z}$  s) (+ $\mathbb{Z}$ -inv-right t))
      (+ $\mathbb{Z}$ -zero-left s)))

-- § Law 14H.20: eigen-action residue collapses to  $\Sigma_{12} v$ .
law14H-20-eigen-residue-σv :
  (v : Vec12 $\mathbb{Z}$ )
  (e0 e1 e2 : Fin4 →  $\mathbb{Z}$ ) →
    ((i : Fin4) → block0 (K12LaplacianVec12 $\mathbb{Z}$  v) i ≡ e0 i) →
    ((i : Fin4) → block1 (K12LaplacianVec12 $\mathbb{Z}$  v) i ≡ e1 i) →
    ((i : Fin4) → block2 (K12LaplacianVec12 $\mathbb{Z}$  v) i ≡ e2 i) →
    ((i : Fin4) → twelveTimes $\mathbb{Z}$  (block0 v i) +  $\mathbb{Z}$  neg $\mathbb{Z}$  (e0 i) ≡ sum12 $\mathbb{Z}$  v)
    × ((i : Fin4) → twelveTimes $\mathbb{Z}$  (block1 v i) +  $\mathbb{Z}$  neg $\mathbb{Z}$  (e1 i) ≡ sum12 $\mathbb{Z}$  v)
    × ((i : Fin4) → twelveTimes $\mathbb{Z}$  (block2 v i) +  $\mathbb{Z}$  neg $\mathbb{Z}$  (e2 i) ≡ sum12 $\mathbb{Z}$  v)
law14H-20-eigen-residue-σv v e0 e1 e2 eq0 eq1 eq2 =
  ( λ i → trans
    (cong (λ t → twelveTimes $\mathbb{Z}$  (block0 v i) +  $\mathbb{Z}$  neg $\mathbb{Z}$  t) (sym (eq0 i)))
    (law14H-19-residue-pin-block0 v i) )
  ,
  (( λ i → trans
    (cong (λ t → twelveTimes $\mathbb{Z}$  (block1 v i) +  $\mathbb{Z}$  neg $\mathbb{Z}$  t) (sym (eq1 i)))
    (law14H-19-residue-pin-block1 v i) )
  , ( λ i → trans
    (cong (λ t → twelveTimes $\mathbb{Z}$  (block2 v i) +  $\mathbb{Z}$  neg $\mathbb{Z}$  t) (sym (eq2 i)))
    (law14H-19-residue-pin-block2 v i) ))

-- § Law 14H.21: cross-entry residue equality.
law14H-21-spectrum-exhaustive :
  (v : Vec12 $\mathbb{Z}$ )
  (e0 e1 e2 : Fin4 →  $\mathbb{Z}$ ) →
    ((i : Fin4) → block0 (K12LaplacianVec12 $\mathbb{Z}$  v) i ≡ e0 i) →
    ((i : Fin4) → block1 (K12LaplacianVec12 $\mathbb{Z}$  v) i ≡ e1 i) →
    ((i : Fin4) → block2 (K12LaplacianVec12 $\mathbb{Z}$  v) i ≡ e2 i) →
    (i j : Fin4) →
      ( twelveTimes $\mathbb{Z}$  (block0 v i) +  $\mathbb{Z}$  neg $\mathbb{Z}$  (e0 i)
        ≡ twelveTimes $\mathbb{Z}$  (block1 v j) +  $\mathbb{Z}$  neg $\mathbb{Z}$  (e1 j) )
      × ( twelveTimes $\mathbb{Z}$  (block0 v i) +  $\mathbb{Z}$  neg $\mathbb{Z}$  (e0 i)
        ≡ twelveTimes $\mathbb{Z}$  (block2 v j) +  $\mathbb{Z}$  neg $\mathbb{Z}$  (e2 j) )
      × ( twelveTimes $\mathbb{Z}$  (block1 v i) +  $\mathbb{Z}$  neg $\mathbb{Z}$  (e1 i)
        ≡ twelveTimes $\mathbb{Z}$  (block2 v j) +  $\mathbb{Z}$  neg $\mathbb{Z}$  (e2 j) )

```

```

law14H-21-spectrum-exhaustive v e0 e1 e2 eq0 eq1 eq2 i j =
let r = law14H-20-eigen-residue-σv v e0 e1 e2 eq0 eq1 eq2 in
trans (fst r i) (sym (fst (snd r) j))
,
( trans (fst r i) (sym (snd (snd r) j))
, trans (fst (snd r) i) (sym (snd (snd r) j)) )

```

The spectral theory of the coupled and triple K_4 Laplacians yields the operator identities $L^2 = 12L$, $J^2 = 12J$, $LJ = JL = 0$ on $\text{Vec}12_{\mathbb{Z}}$ (Laws 14H.5, 14H.9–14H.11), the kernel-image equivalences (Laws 14H.12–14H.16), and the rigidity statement of Laws 14H.19–14H.21, which shows that the eigen-residue is determined by $\Sigma_{12}v$ at every entry.

To use this spectral data analytically, the arithmetic of $\mathbb{Z}$ and $\mathbb{Q}$ must also support the order and distance estimates required for convergence arguments. The next chapter develops that infrastructure.

Chapter 46

Additive Order Laws for $\leq_{\mathbb{Q}}$

Cauchy convergence arguments require combining two distance bounds into one: if $|x_m - x_n| < \varepsilon/2$ and $|x_n - x_k| < \varepsilon/2$, then $|x_m - x_k| < \varepsilon$. To justify this over $\mathbb{Q}$, the order relation must be stable under addition and under the standard denominator-clearing manipulations.

The point of this chapter is therefore arithmetic. Every estimate needed later is reduced to an integer inequality after clearing denominators, and every additive comparison must survive that reduction unchanged.

Nonnegativity Extraction and Closure

Nonnegativity must survive passage from rationals to numerators and back through addition. After clearing denominators, the problem reduces to integer nonnegativity. The three lemmas below isolate this reduction: every natural number gives a nonnegative integer, a witness of $0 \leq q$ yields nonnegativity of the numerator of q , and these numerator witnesses combine to show that rational addition preserves nonnegativity.

```
-- § nonneg integer from natural
0 ≤ℤ-fromℕℕ : (n : ℕ) → 0ℤ ≤ℤ fromℕℕ n
0 ≤ℤ-fromℕℕ zero = tt
0 ≤ℤ-fromℕℕ (suc n) = tt

-- § nonneg rational ⇒ nonneg numerator
0 ≤ℚ → 0 ≤ℤ num : (q : ℚ) → 0ℚ ≤ℚ q → 0ℤ ≤ℤ num q
0 ≤ℚ → 0 ≤ℤ num (a / b) qnonneg =
let
  lhs0 : (0ℤ *ℤ +toℤ b) ≡ 0ℤ
  lhs0 = *ℤ-zero-left (+toℤ b)

  one+ℤ ≡ oneℤ : +toℤ one+ ≡ oneℤ
  one+ℤ ≡ oneℤ = refl
```

```

rhs1 : (a * ℤ +toℤ one+) ≡ a
rhs1 = trans (cong (λ t → a * ℤ t) one+ ℤ ≡ oneℤ) (*ℤ-one-right a)
in
≤ℤ-resp-≡r rhs1 (≤ℤ-resp-≡l lhs0 qnonneg)

-- § nonnegativity closed under rational addition
0 ≤ ℚ + ℚ : (p q : ℚ) → 0 ℚ ≤ ℚ p → 0 ℚ ≤ ℚ q → 0 ℚ ≤ ℚ (p + ℚ q)
0 ≤ ℚ + ℚ (a / b) (c / d) p ≥ 0 q ≥ 0 =
let
  a ≥ 0 : 0 ℤ ≤ ℤ a
  a ≥ 0 = 0 ≤ ℚ → 0 ≤ ℤ num (a / b) p ≥ 0

  c ≥ 0 : 0 ℤ ≤ ℤ c
  c ≥ 0 = 0 ≤ ℚ → 0 ≤ ℤ num (c / d) q ≥ 0

nonnegScale : (z : ℤ) → (s : ℕ+) → 0 ℤ ≤ ℤ z → 0 ℤ ≤ ℤ (z * ℤ +toℤ s)
nonnegScale z s z ≥ 0 =
  ≤ℤ-resp-≡l (*ℤ-zero-left (+toℤ s))
  (≤ℤ-mul-pos-right 0 ℤ z s z ≥ 0)

ad ≥ 0 : 0 ℤ ≤ ℤ (a * ℤ +toℤ d)
ad ≥ 0 = nonnegScale a d a ≥ 0

cb ≥ 0 : 0 ℤ ≤ ℤ (c * ℤ +toℤ b)
cb ≥ 0 = nonnegScale c b c ≥ 0

sum ≥ 0 : 0 ℤ ≤ ℤ ((a * ℤ +toℤ d) + ℤ (c * ℤ +toℤ b))
sum ≥ 0 =
let
  adNat : Σ ℕ (λ k → (a * ℤ +toℤ d) ≡ fromℕℤ k)
  adNat = 0 ≤ ℤ → fromℕℤ (a * ℤ +toℤ d) ad ≥ 0

  cbNat : Σ ℕ (λ k → (c * ℤ +toℤ b) ≡ fromℕℤ k)
  cbNat = 0 ≤ ℤ → fromℕℤ (c * ℤ +toℤ b) cb ≥ 0

  k1 : ℕ
  k1 = fst adNat

  k2 : ℕ
  k2 = fst cbNat

  ad ≡ : (a * ℤ +toℤ d) ≡ fromℕℤ k1
  ad ≡ = snd adNat

  cb ≡ : (c * ℤ +toℤ b) ≡ fromℕℤ k2
  cb ≡ = snd cbNat

sumForm : (a * ℤ +toℤ d) + ℤ (c * ℤ +toℤ b) ≡ fromℕℤ (k1 +ℕ k2)
sumForm =
  trans
    (cong (λ t → t + ℤ (c * ℤ +toℤ b)) ad ≡)
  (trans
    (cong (λ t → fromℕℤ k1 + ℤ t) cb ≡)
    (fromℕℤ + ℤ k1 k2))
in

```

```

≤ℤ-resp≡r (sym sumForm) (0≤ℤ-fromℕℤ (k1 +ℕ k2))

lhs0 : (0ℤ *ℤ +toℤ (b *+ d)) ≡ 0ℤ
lhs0 = *ℤ-zero-left (+toℤ (b *+ d))

rhs1 : (((a *ℤ +toℤ d) +ℤ (c *ℤ +toℤ b)) *ℤ +toℤ one+) ≡ ((a *ℤ +toℤ d) +ℤ (c *ℤ +toℤ b))
rhs1 = *ℤ-one-right ((a *ℤ +toℤ d) +ℤ (c *ℤ +toℤ b))

in
≤ℤ-resp≡l (sym lhs0) (≤ℤ-resp≡r (sym rhs1) sum≥0)

```

Sum-Doubling Bounds

If $0 \leq x, y, z$ with $x \leq z$ and $y \leq z$, then $x + y \leq z + z$. The integer version is the corresponding natural number inequality transported through the representation of nonnegative integers by naturals. The rational version then follows by clearing denominators and applying the integer lemma to the resulting scaled numerators. This is the estimate that later combines two bounds of size $\varepsilon/2$ into a single bound of size ε .

```

-- § integer sum-doubling: x≤z, y≤z ⇒ x+y ≤ z+z (nonneg)
≤ℤ-sum≤double-nonneg : {x y z : ℤ} →
0ℤ ≤ℤ x → 0ℤ ≤ℤ y → 0ℤ ≤ℤ z → x ≤ℤ z → y ≤ℤ z → (x +ℤ y) ≤ℤ (z +ℤ z)
≤ℤ-sum≤double-nonneg {x} {y} {z} xnonneg ynonneg znonneg x≤z y≤z =
let
  xm : Σ ℕ (λ n → x ≡ fromℕℤ n)
  xm = 0≤ℤ→fromℕℤ x xnonneg

  ym : Σ ℕ (λ n → y ≡ fromℕℤ n)
  ym = 0≤ℤ→fromℕℤ y ynonneg

  zm : Σ ℕ (λ n → z ≡ fromℕℤ n)
  zm = 0≤ℤ→fromℕℤ z znonneg

  m : ℕ
  m = fst xm

  n : ℕ
  n = fst ym

  k : ℕ
  k = fst zm

  x≡ : x ≡ fromℕℤ m
  x≡ = snd xm

  y≡ : y ≡ fromℕℤ n
  y≡ = snd ym

  z≡ : z ≡ fromℕℤ k
  z≡ = snd zm

```

$x \leq_{\mathbb{Z}} \text{zNat} : m \leq k$
 $x \leq_{\mathbb{Z}} \text{zNat} = \leq_{\mathbb{Z}}\text{-from}\mathbb{N}\mathbb{Z}\text{-reflect } (\leq_{\mathbb{Z}}\text{-resp} \equiv^l x \equiv (\leq_{\mathbb{Z}}\text{-resp} \equiv^r z \equiv x \leq_{\mathbb{Z}} z))$

$y \leq_{\mathbb{Z}} \text{zNat} : n \leq k$
 $y \leq_{\mathbb{Z}} \text{zNat} = \leq_{\mathbb{Z}}\text{-from}\mathbb{N}\mathbb{Z}\text{-reflect } (\leq_{\mathbb{Z}}\text{-resp} \equiv^l y \equiv (\leq_{\mathbb{Z}}\text{-resp} \equiv^r z \equiv y \leq_{\mathbb{Z}} z))$

$\text{step}_1 : (m + \mathbb{N} n) \leq (k + \mathbb{N} n)$
 $\text{step}_1 =$
 $\text{subst } (\lambda t \rightarrow t \leq (k + \mathbb{N} n))$
 $(\text{sym } (+\mathbb{N}\text{-comm } m n))$
 $(\text{subst } (\lambda t \rightarrow (n + \mathbb{N} m) \leq t))$
 $(+\mathbb{N}\text{-comm } n k)$
 $(\leq_{\mathbb{Z}}\text{-mono}^l x \leq_{\mathbb{Z}} \text{zNat } n))$

$\text{step}_2 : (k + \mathbb{N} n) \leq (k + \mathbb{N} k)$
 $\text{step}_2 = \leq_{\mathbb{Z}}\text{-mono}^l y \leq_{\mathbb{Z}} \text{zNat } k$

$\text{sumNat} : (m + \mathbb{N} n) \leq (k + \mathbb{N} k)$
 $\text{sumNat} = \leq\text{-trans } \text{step}_1 \text{ step}_2$

$\text{sum}\mathbb{Z} : \text{from}\mathbb{N}\mathbb{Z} (m + \mathbb{N} n) \leq_{\mathbb{Z}} \text{from}\mathbb{N}\mathbb{Z} (k + \mathbb{N} k)$
 $\text{sum}\mathbb{Z} = \text{from}\mathbb{N}\mathbb{Z}\text{-mono } \text{sumNat}$

$\text{lhsEq} : (x + \mathbb{Z} y) \equiv \text{from}\mathbb{N}\mathbb{Z} (m + \mathbb{N} n)$
 $\text{lhsEq} =$
 trans
 $(\text{cong } (\lambda t \rightarrow t + \mathbb{Z} y) x \equiv)$
 $(\text{trans}$
 $(\text{cong } (\lambda t \rightarrow \text{from}\mathbb{N}\mathbb{Z} m + \mathbb{Z} t) y \equiv)$
 $(\text{from}\mathbb{N}\mathbb{Z} + \mathbb{Z} m n))$

$\text{rhsEq} : (z + \mathbb{Z} z) \equiv \text{from}\mathbb{N}\mathbb{Z} (k + \mathbb{N} k)$
 $\text{rhsEq} =$
 trans
 $(\text{cong } (\lambda t \rightarrow t + \mathbb{Z} z) z \equiv)$
 $(\text{trans}$
 $(\text{cong } (\lambda t \rightarrow \text{from}\mathbb{N}\mathbb{Z} k + \mathbb{Z} t) z \equiv)$
 $(\text{from}\mathbb{N}\mathbb{Z} + \mathbb{Z} k k))$

in
 $\leq_{\mathbb{Z}}\text{-resp} \equiv^l (\text{sym } \text{lhsEq}) (\leq_{\mathbb{Z}}\text{-resp} \equiv^r (\text{sym } \text{rhsEq}) \text{sum}\mathbb{Z})$

Rational sum-doubling. The rational counterpart lifts integer sum-doubling through the usual cross-multiplication argument.

```

-- § rational sum-doubling: p ≤ r, q ≤ r ⇒ p + q ≤ r + r (nonneg)
≤Q-sum≤double-nonneg : (p q r : ℚ) → 0ℚ ≤Q p → 0ℚ ≤Q q → 0ℚ ≤Q r → p ≤Q r → q ≤Q r → (p +Q q) ≤Q (r +Q r)
≤Q-sum≤double-nonneg (a / b) (c / d) (e / f) pnonneg qnonneg rnonneg p≤r q≤r =
let
  bd : ℕ+
  bd = b *+ d

  ff : ℕ+
  ff = f *+ f

```

```

bdf :  $\mathbb{N}^+$ 
bdf = bd *+ f

a ≥ 0 :  $0\mathbb{Z} \leq \mathbb{Z} a$ 
a ≥ 0 =  $0 \leq \mathbb{Q} \rightarrow 0 \leq \mathbb{Z} \text{num } (a / b) \text{ pnonneg}$ 

c ≥ 0 :  $0\mathbb{Z} \leq \mathbb{Z} c$ 
c ≥ 0 =  $0 \leq \mathbb{Q} \rightarrow 0 \leq \mathbb{Z} \text{num } (c / d) \text{ qnonneg}$ 

e ≥ 0 :  $0\mathbb{Z} \leq \mathbb{Z} e$ 
e ≥ 0 =  $0 \leq \mathbb{Q} \rightarrow 0 \leq \mathbb{Z} \text{num } (e / f) \text{ rnonneg}$ 

nonnegScale : (z :  $\mathbb{Z}$ ) → (s :  $\mathbb{N}^+$ ) →  $0\mathbb{Z} \leq \mathbb{Z} z \rightarrow 0\mathbb{Z} \leq \mathbb{Z} (z * \mathbb{Z} ^+ \text{to} \mathbb{Z} s)$ 
nonnegScale z s z ≥ 0 =
  ≤ $\mathbb{Z}$ -resp-≡l (* $\mathbb{Z}$ -zero-left ( $^+ \text{to} \mathbb{Z} s$ ))
  (≤ $\mathbb{Z}$ -mul-pos-right  $0\mathbb{Z} z s z \geq 0$ )

X :  $\mathbb{Z}$ 
X = (a * $\mathbb{Z} ^+ \text{to} \mathbb{Z} d$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} ff$ 

Y :  $\mathbb{Z}$ 
Y = (c * $\mathbb{Z} ^+ \text{to} \mathbb{Z} b$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} ff$ 

Z :  $\mathbb{Z}$ 
Z = ((e * $\mathbb{Z} ^+ \text{to} \mathbb{Z} bd$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ )

X ≥ 0 :  $0\mathbb{Z} \leq \mathbb{Z} X$ 
X ≥ 0 = nonnegScale (a * $\mathbb{Z} ^+ \text{to} \mathbb{Z} d$ ) ff (nonnegScale a d a ≥ 0)

Y ≥ 0 :  $0\mathbb{Z} \leq \mathbb{Z} Y$ 
Y ≥ 0 = nonnegScale (c * $\mathbb{Z} ^+ \text{to} \mathbb{Z} b$ ) ff (nonnegScale c b c ≥ 0)

Z ≥ 0 :  $0\mathbb{Z} \leq \mathbb{Z} Z$ 
Z ≥ 0 = nonnegScale (e * $\mathbb{Z} ^+ \text{to} \mathbb{Z} bd$ ) f (nonnegScale e bd e ≥ 0)

-- scale p ≤ r to common base
pScaled1 : (((a * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} d$ ) ≤ $\mathbb{Z}$  ((e * $\mathbb{Z} ^+ \text{to} \mathbb{Z} b$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} d$ )
pScaled1 = ≤ $\mathbb{Z}$ -mul-pos-right (a * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ ) (e * $\mathbb{Z} ^+ \text{to} \mathbb{Z} b$ ) d p ≤ r

pScaled2 : (((a * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} d$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ ) ≤ $\mathbb{Z}$  (((e * $\mathbb{Z} ^+ \text{to} \mathbb{Z} b$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} d$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ )
pScaled2 = ≤ $\mathbb{Z}$ -mul-pos-right ((a * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} d$ ) ((e * $\mathbb{Z} ^+ \text{to} \mathbb{Z} b$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} d$ ) f pScaled1

qScaled1 : ((c * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} b$ ) ≤ $\mathbb{Z}$  ((e * $\mathbb{Z} ^+ \text{to} \mathbb{Z} d$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} b$ )
qScaled1 = ≤ $\mathbb{Z}$ -mul-pos-right (c * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ ) (e * $\mathbb{Z} ^+ \text{to} \mathbb{Z} d$ ) b q ≤ r

qScaled2 : (((c * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} b$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ ) ≤ $\mathbb{Z}$  (((e * $\mathbb{Z} ^+ \text{to} \mathbb{Z} d$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} b$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ )
qScaled2 = ≤ $\mathbb{Z}$ -mul-pos-right ((c * $\mathbb{Z} ^+ \text{to} \mathbb{Z} f$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} b$ ) ((e * $\mathbb{Z} ^+ \text{to} \mathbb{Z} d$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} b$ ) f qScaled1

-- rewrite both sides into X ≤ Z, Y ≤ Z
swapScale : (x :  $\mathbb{Z}$ ) → (u v :  $\mathbb{N}^+$ ) → (x * $\mathbb{Z} ^+ \text{to} \mathbb{Z} u$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} v$  ≡ (x * $\mathbb{Z} ^+ \text{to} \mathbb{Z} v$ ) * $\mathbb{Z} ^+ \text{to} \mathbb{Z} u$ 
swapScale x u v =
  trans
    (* $\mathbb{Z}$ -assoc x ( $^+ \text{to} \mathbb{Z} u$ ) ( $^+ \text{to} \mathbb{Z} v$ ))
  (trans
    (cong (λ t → x * $\mathbb{Z} t$ ) (* $\mathbb{Z}$ -comm ( $^+ \text{to} \mathbb{Z} u$ ) ( $^+ \text{to} \mathbb{Z} v$ )))

```

(sym (* $\mathbb{Z}$ -assoc x (+to $\mathbb{Z}$ v) (+to $\mathbb{Z}$ u))))

scaleSplit : (x : $\mathbb{Z}$) $\rightarrow$ (u v : $\mathbb{N}^+$) $\rightarrow$ x * $\mathbb{Z}$ +to $\mathbb{Z}$ (u ** v) $\equiv$ (x * $\mathbb{Z}$ +to $\mathbb{Z}$ u) * $\mathbb{Z}$ +to $\mathbb{Z}$ v

scaleSplit x u v =

trans

(cong (λ t $\rightarrow$ x * $\mathbb{Z}$ t) (+to $\mathbb{Z}$ -** u v))

(sym (* $\mathbb{Z}$ -assoc x (+to $\mathbb{Z}$ u) (+to $\mathbb{Z}$ v))))

Xeq : (((a * $\mathbb{Z}$ +to $\mathbb{Z}$ f) * $\mathbb{Z}$ +to $\mathbb{Z}$ d) * $\mathbb{Z}$ +to $\mathbb{Z}$ f) $\equiv$ X

Xeq =

trans

(cong (λ t $\rightarrow$ t * $\mathbb{Z}$ +to $\mathbb{Z}$ f) (swapScale a f d))

(sym (scaleSplit (a * $\mathbb{Z}$ +to $\mathbb{Z}$ d) f f))

Zeq₁ : (((e * $\mathbb{Z}$ +to $\mathbb{Z}$ b) * $\mathbb{Z}$ +to $\mathbb{Z}$ d) * $\mathbb{Z}$ +to $\mathbb{Z}$ f) $\equiv$ Z

Zeq₁ =

cong (λ t $\rightarrow$ t * $\mathbb{Z}$ +to $\mathbb{Z}$ f) (sym (scaleSplit e b d))

Zeq₂ : (((e * $\mathbb{Z}$ +to $\mathbb{Z}$ d) * $\mathbb{Z}$ +to $\mathbb{Z}$ b) * $\mathbb{Z}$ +to $\mathbb{Z}$ f) $\equiv$ Z

Zeq₂ =

trans

(cong (λ t $\rightarrow$ t * $\mathbb{Z}$ +to $\mathbb{Z}$ f) (swapScale e d b))

Zeq₁

X $\leq$ Z : X $\leq_{\mathbb{Z}}$ Z

X $\leq$ Z =

subst (λ t $\rightarrow$ X $\leq_{\mathbb{Z}}$ t) Zeq₁

(subst (λ t $\rightarrow$ t $\leq_{\mathbb{Z}}$ (((e * $\mathbb{Z}$ +to $\mathbb{Z}$ b) * $\mathbb{Z}$ +to $\mathbb{Z}$ d) * $\mathbb{Z}$ +to $\mathbb{Z}$ f))

Xeq

pScaled₂)

Yeq : (((c * $\mathbb{Z}$ +to $\mathbb{Z}$ f) * $\mathbb{Z}$ +to $\mathbb{Z}$ b) * $\mathbb{Z}$ +to $\mathbb{Z}$ f) $\equiv$ Y

Yeq =

trans

(cong (λ t $\rightarrow$ t * $\mathbb{Z}$ +to $\mathbb{Z}$ f) (swapScale c f b))

(sym (scaleSplit (c * $\mathbb{Z}$ +to $\mathbb{Z}$ b) f f))

Y $\leq$ Z : Y $\leq_{\mathbb{Z}}$ Z

Y $\leq$ Z =

subst (λ t $\rightarrow$ Y $\leq_{\mathbb{Z}}$ t) Zeq₂

(subst (λ t $\rightarrow$ t $\leq_{\mathbb{Z}}$ (((e * $\mathbb{Z}$ +to $\mathbb{Z}$ d) * $\mathbb{Z}$ +to $\mathbb{Z}$ b) * $\mathbb{Z}$ +to $\mathbb{Z}$ f))

Yeq

qScaled₂)

sumLe : (X + $\mathbb{Z}$ Y) $\leq_{\mathbb{Z}}$ (Z + $\mathbb{Z}$ Z)

sumLe = $\leq_{\mathbb{Z}}$ -sum $\leq$ double-nonneg X $\geq$ 0 Y $\geq$ 0 Z $\geq$ 0 X $\leq$ Z Y $\leq$ Z

lhsEq : (((a * $\mathbb{Z}$ +to $\mathbb{Z}$ d) + $\mathbb{Z}$ (c * $\mathbb{Z}$ +to $\mathbb{Z}$ b)) * $\mathbb{Z}$ +to $\mathbb{Z}$ ff) $\equiv$ (X + $\mathbb{Z}$ Y)

lhsEq =

trans

(* $\mathbb{Z}$ -distrib-left-+ $\mathbb{Z}$ (a * $\mathbb{Z}$ +to $\mathbb{Z}$ d) (c * $\mathbb{Z}$ +to $\mathbb{Z}$ b) (+to $\mathbb{Z}$ ff))

refl

rhsEq : (((e * $\mathbb{Z}$ +to $\mathbb{Z}$ f) + $\mathbb{Z}$ (e * $\mathbb{Z}$ +to $\mathbb{Z}$ f)) * $\mathbb{Z}$ +to $\mathbb{Z}$ bd) $\equiv$ (Z + $\mathbb{Z}$ Z)

rhsEq =

```

let
  ef : ℤ
  ef = e *ℤ +toℤ f

  efbd ≡ Z : (ef *ℤ +toℤ bd) ≡ Z
  efbd ≡ Z =
    trans
      (*ℤ-assoc e (+toℤ f) (+toℤ bd))
      (trans
        (cong (λ t → e *ℤ t) (*ℤ-comm (+toℤ f) (+toℤ bd)))
        (sym (*ℤ-assoc e (+toℤ bd) (+toℤ f))))
    in
      trans
        (*ℤ-distrib-left-+ℤ ef ef (+toℤ bd))
        (cong (λ t → t +ℤ t) efbd ≡ Z)

in
  ≤ℤ-resp ≡l (sym lhsEq) (≤ℤ-resp ≡r (sym rhsEq) sumLe)

```

Nonneg-Right Addition and Monotonicity

Adding a nonnegative term on the right preserves the order, and from this one obtains both right and left monotonicity of rational addition. The right-handed statement is proved first by reducing to the corresponding integer fact after clearing denominators; the left-handed version then follows from commutativity.

```

-- § negative normalisation step
neg≤normalize : (n m : ℕ) → (-suc m) ≤ℤ normalizeℤ n m
neg≤normalize zero zero = tt
neg≤normalize zero (suc m) = ≤-step (suc m)
neg≤normalize (suc n) zero = tt
neg≤normalize (suc n) (suc m) =
  ≤ℤ-trans negStep (neg≤normalize n m)
where
  negStep : (-suc (suc m)) ≤ℤ (-suc m)
  negStep = s≤s (≤-step m)

-- § adding a positive integer on the right
≤ℤ-add-pos-right : (x : ℤ) → (n : ℕ) → x ≤ℤ (x +ℤ (+suc n))
≤ℤ-add-pos-right 0ℤ n = tt
≤ℤ-add-pos-right (+suc m) n = s≤s m≤m+n
where
  m≤m+n : m ≤ (m +ℤ suc n)
  m≤m+n =
    subst (λ t → t ≤ (m +ℤ suc n))
      (+ℤ-zero-right m)
      (≤-+ℤ-monol {a = zero} {b = suc n} z≤n m)
≤ℤ-add-pos-right (-suc m) n =
let
  rhsEq : ((-suc m) +ℤ (+suc n)) ≡ normalizeℤ n m

```

```

rhsEq =
  trans
    (cong (λ t → normalizeℤ (suc n) t) (+ℕ-zero-right (suc m)))
  refl
in
  ≤ℤ-resp≡r (sym rhsEq) (neg≤normalize n m)

-- § adding a nonneg integer on the right
≤ℤ-add-nonneg-right : (x y : ℤ) → 0ℤ ≤ℤ y → x ≤ℤ (x +ℤ y)
≤ℤ-add-nonneg-right x y y≥0 with 0≤ℤ→fromℕℤ y y≥0
... | (zero , y≡) =
  ≤ℤ-resp≡r (sym (cong (λ t → x +ℤ t) y≡)) (≤ℤ-resp≡r (sym (+ℤ-zero-right x)) (≤ℤ-refl x))
... | (suc n , y≡) =
  ≤ℤ-resp≡r (sym (cong (λ t → x +ℤ t) y≡)) (≤ℤ-add-pos-right x n)

-- § adding a nonneg rational on the right preserves ≤
≤ℚ-add-nonneg-right : (p q : ℚ) → 0ℚ ≤ℚ q → p ≤ℚ (p +ℚ q)
≤ℚ-add-nonneg-right (a / b) (c / d) qnonneg =
  let
    c≥0 : 0ℤ ≤ℤ c
    c≥0 = 0≤ℚ→0≤ℤnum (c / d) qnonneg

    nonnegScale : (z : ℤ) → (s : ℕ+) → 0ℤ ≤ℤ z → 0ℤ ≤ℤ (z *ℤ +toℤ s)
    nonnegScale z s z≥0 =
      ≤ℤ-resp≡l (*ℤ-zero-left (+toℤ s))
        (≤ℤ-mul-pos-right 0ℤ z s z≥0)

    x : ℤ
    x = (a *ℤ +toℤ d) *ℤ +toℤ b

    y : ℤ
    y = (c *ℤ +toℤ b) *ℤ +toℤ b

    y≥0 : 0ℤ ≤ℤ y
    y≥0 = nonnegScale (c *ℤ +toℤ b) b (nonnegScale c b c≥0)

    x≤x+y : x ≤ℤ (x +ℤ y)
    x≤x+y = ≤ℤ-add-nonneg-right x y y≥0

  lhsEq : (a *ℤ +toℤ (b *+ d)) ≡ x
  lhsEq =
    let
      scaleSplit : (z : ℤ) → (u v : ℕ+) → z *ℤ +toℤ (u *+ v) ≡ (z *ℤ +toℤ u) *ℤ +toℤ v
      scaleSplit z u v =
        trans
          (cong (λ t → z *ℤ t) (+toℤ-*+ u v))
          (sym (*ℤ-assoc z (+toℤ u) (+toℤ v)))

      swapScale : (z : ℤ) → (u v : ℕ+) → (z *ℤ +toℤ u) *ℤ +toℤ v ≡ (z *ℤ +toℤ v) *ℤ +toℤ u
      swapScale z u v =
        trans
          (*ℤ-assoc z (+toℤ u) (+toℤ v))
          (trans
            (cong (λ t → z *ℤ t) (*ℤ-comm (+toℤ u) (+toℤ v))))

```



```

(sym (*ℤ-assoc z (+toℤ v) (+toℤ u)))
in
trans
(trans
  (cong (λ t → a *ℤ t) (+toℤ-+ b d))
  (sym (*ℤ-assoc a (+toℤ b) (+toℤ d))))
  (swapScale a b d)

rhsEq : (((a *ℤ +toℤ d) +ℤ (c *ℤ +toℤ b)) *ℤ +toℤ b) ≡ (x +ℤ y)
rhsEq =
trans
  (*ℤ-distrib-left+ℤ (a *ℤ +toℤ d) (c *ℤ +toℤ b) (+toℤ b))
  refl
in
≤ℤ-resp-≡l (sym lhsEq) (≤ℤ-resp-≡r (sym rhsEq) x ≤x+y)

```

Rational order monotonicity. Right and left monotonicity for $\leq_{\mathbb{Q}}$ under addition follow from the cross-multiplication bridge.

```

-- § right monotonicity: p ≤ q ⇒ (p + r) ≤ (q + r)
≤ℚ-+ℚ-mono-right : (p q r : ℚ) → p ≤ℚ q → (p +ℚ r) ≤ℚ (q +ℚ r)
≤ℚ-+ℚ-mono-right (a / b) (c / d) (e / f) p ≤ q =
let
  bd : ℕ+
  bd = b *+ d

  bf : ℕ+
  bf = b *+ f

  df : ℕ+
  df = d *+ f

p ≤q-scaled1 : ((a *ℤ +toℤ d) *ℤ +toℤ f) ≤ℤ ((c *ℤ +toℤ b) *ℤ +toℤ f)
p ≤q-scaled1 = ≤ℤ-mul-pos-right (a *ℤ +toℤ d) (c *ℤ +toℤ b) f p ≤ q

p ≤q-scaled2 : (((a *ℤ +toℤ d) *ℤ +toℤ f) *ℤ +toℤ f) ≤ℤ (((c *ℤ +toℤ b) *ℤ +toℤ f) *ℤ +toℤ f)
p ≤q-scaled2 = ≤ℤ-mul-pos-right ((a *ℤ +toℤ d) *ℤ +toℤ f) ((c *ℤ +toℤ b) *ℤ +toℤ f) f p ≤q-scaled1

swapScale : (x : ℤ) → (u v : ℕ+) → (x *ℤ +toℤ u) *ℤ +toℤ v ≡ (x *ℤ +toℤ v) *ℤ +toℤ u
swapScale x u v =
trans
  (*ℤ-assoc x (+toℤ u) (+toℤ v))
  (trans
    (cong (λ t → x *ℤ t) (*ℤ-comm (+toℤ u) (+toℤ v)))
    (sym (*ℤ-assoc x (+toℤ v) (+toℤ u))))

scaleSplit : (x : ℤ) → (u v : ℕ+) → x *ℤ +toℤ (u *+ v) ≡ (x *ℤ +toℤ u) *ℤ +toℤ v
scaleSplit x u v =
trans
  (cong (λ t → x *ℤ t) (+toℤ-+ u v))
  (sym (*ℤ-assoc x (+toℤ u) (+toℤ v)))

ff : ℕ+

```

$$ff = f^{**} f$$

$$lhsTerm_1 : ((a * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} df) \equiv (((a * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} f)$$

$$lhsTerm_1 =$$

trans

$$(scaleSplit (a * \mathbb{Z} + \text{to}\mathbb{Z} f) d f)$$

$$(cong (\lambda t \rightarrow t * \mathbb{Z} + \text{to}\mathbb{Z} f) (swapScale a f d))$$

$$rhsTerm_1 : ((c * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} bf) \equiv (((c * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} f)$$

$$rhsTerm_1 =$$

trans

$$(scaleSplit (c * \mathbb{Z} + \text{to}\mathbb{Z} f) b f)$$

$$(cong (\lambda t \rightarrow t * \mathbb{Z} + \text{to}\mathbb{Z} f) (swapScale c f b))$$

$$rTerm : (e * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} df \equiv (e * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} bf$$

$$rTerm =$$

trans

$$(scaleSplit (e * \mathbb{Z} + \text{to}\mathbb{Z} b) d f)$$

(trans

$$(cong (\lambda t \rightarrow t * \mathbb{Z} + \text{to}\mathbb{Z} f) (swapScale e b d))$$

$$(\text{sym } (scaleSplit (e * \mathbb{Z} + \text{to}\mathbb{Z} d) b f)))$$

$$lhsSum : \mathbb{Z}$$

$$lhsSum = ((a * \mathbb{Z} + \text{to}\mathbb{Z} f) + \mathbb{Z} (e * \mathbb{Z} + \text{to}\mathbb{Z} b)) * \mathbb{Z} + \text{to}\mathbb{Z} df$$

$$rhsSum : \mathbb{Z}$$

$$rhsSum = ((c * \mathbb{Z} + \text{to}\mathbb{Z} f) + \mathbb{Z} (e * \mathbb{Z} + \text{to}\mathbb{Z} d)) * \mathbb{Z} + \text{to}\mathbb{Z} bf$$

$$lhsExpand : lhsSum \equiv ((a * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} df) + \mathbb{Z} ((e * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} df)$$

$$lhsExpand = * \mathbb{Z} \text{-distrib-left} + \mathbb{Z} (a * \mathbb{Z} + \text{to}\mathbb{Z} f) (e * \mathbb{Z} + \text{to}\mathbb{Z} b) (+ \text{to}\mathbb{Z} df)$$

$$rhsExpand : rhsSum \equiv ((c * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} bf) + \mathbb{Z} ((e * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} bf)$$

$$rhsExpand = * \mathbb{Z} \text{-distrib-left} + \mathbb{Z} (c * \mathbb{Z} + \text{to}\mathbb{Z} f) (e * \mathbb{Z} + \text{to}\mathbb{Z} d) (+ \text{to}\mathbb{Z} bf)$$

$$lhsT_1 \leq rhsT_1 : ((a * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} df) \leq \mathbb{Z} ((c * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} bf)$$

$$lhsT_1 \leq rhsT_1 =$$

$$\leq \mathbb{Z} \text{-resp} \equiv^l (\text{sym } lhsTerm_1) (\leq \mathbb{Z} \text{-resp} \equiv^r (\text{sym } rhsTerm_1) p \leq q \text{-scaled}_2)$$

$$eTerm \equiv : ((e * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} df) \equiv ((e * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} bf)$$

$$eTerm \equiv = rTerm$$

$$eTerm \leq : ((e * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} df) \leq \mathbb{Z} ((e * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} bf)$$

$$eTerm \leq = \leq \mathbb{Z} \text{-resp} \equiv^r eTerm \equiv (\leq \mathbb{Z} \text{-refl } ((e * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} df))$$

$$sumLe : (((a * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} df) + \mathbb{Z} ((e * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} df)) \leq \mathbb{Z} (((c * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} bf) + \mathbb{Z} ((e * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} bf))$$

$$sumLe = \leq \mathbb{Z} + \mathbb{Z} \text{-mono } lhsT_1 \leq rhsT_1 \ eTerm \leq$$

in

$$\leq \mathbb{Z} \text{-resp} \equiv^l (\text{sym } lhsExpand) (\leq \mathbb{Z} \text{-resp} \equiv^r (\text{sym } rhsExpand) sumLe)$$

$$\text{-- } \S \text{ left monotonicity: } q \leq r \Rightarrow (p + q) \leq (p + r)$$

$$\leq \mathbb{Q} + \mathbb{Q} \text{-mono-left} : (p \ q \ r : \mathbb{Q}) \rightarrow q \leq \mathbb{Q} \ r \rightarrow (p + \mathbb{Q} \ q) \leq \mathbb{Q} (p + \mathbb{Q} \ r)$$

$$\leq \mathbb{Q} + \mathbb{Q} \text{-mono-left } p \ q \ r \ q \leq r =$$

let

$$step_1 : (q + \mathbb{Q} \ p) \leq \mathbb{Q} (r + \mathbb{Q} \ p)$$

$$step_1 = \leq \mathbb{Q} + \mathbb{Q} \text{-mono-right } q \ r \ p \ q \leq r$$

$$\begin{aligned}
& \text{step}_2 : (p + \mathbb{Q} \ q) \leq \mathbb{Q} \ (q + \mathbb{Q} \ p) \\
& \text{step}_2 = \simeq \mathbb{Q} \rightarrow \leq \mathbb{Q}^l \ \{p = p + \mathbb{Q} \ q\} \ \{q = q + \mathbb{Q} \ p\} \ (+\mathbb{Q}\text{-comm} \ p \ q) \\
& \text{step}_3 : (r + \mathbb{Q} \ p) \leq \mathbb{Q} \ (p + \mathbb{Q} \ r) \\
& \text{step}_3 = \simeq \mathbb{Q} \rightarrow \leq \mathbb{Q}^l \ \{p = r + \mathbb{Q} \ p\} \ \{q = p + \mathbb{Q} \ r\} \ (+\mathbb{Q}\text{-comm} \ r \ p) \\
& \text{in} \\
& \leq \mathbb{Q}\text{-trans} \ \{x = p + \mathbb{Q} \ q\} \ \{y = q + \mathbb{Q} \ p\} \ \{z = p + \mathbb{Q} \ r\} \ \text{step}_2 \\
& (\leq \mathbb{Q}\text{-trans} \ \{x = q + \mathbb{Q} \ p\} \ \{y = r + \mathbb{Q} \ p\} \ \{z = p + \mathbb{Q} \ r\} \ \text{step}_1 \ \text{step}_3)
\end{aligned}$$

Chapter 47

K_{12} Iteration and Word Algebra

Every iterated application of the K_{12} Laplacian or the global-sum operator reduces to a scaled first application. Together with the annihilation laws $JL = LJ = 0$, this means that every word in the free monoid on $\{L_{12}, J_{12}\}$ reduces to one of four normal forms: the identity, a pure L -power, a pure J -power, or zero.

Vec_{12} Transport

Before one can iterate these identities, one must know how pointwise equality on $\text{Vec}_{12}\mathbb{Z}$ behaves under the operations that appear in the formulas. The present lemmas provide that bookkeeping once and for all: Vec12Eq is shown to be an equivalence relation, and equality is transported through total sum, scaling, and the Laplacian itself.

-- § Vec12Eq is an equivalence

$\text{Vec12Eq-refl} : (v : \text{Vec12}\mathbb{Z}) \rightarrow \text{Vec12Eq } v v$

$\text{Vec12Eq-refl } v = (\lambda _ \rightarrow \text{refl}), ((\lambda _ \rightarrow \text{refl}), (\lambda _ \rightarrow \text{refl}))$

$\text{Vec12Eq-sym} : \{u v : \text{Vec12}\mathbb{Z}\} \rightarrow \text{Vec12Eq } u v \rightarrow \text{Vec12Eq } v u$

$\text{Vec12Eq-sym } eq =$

$(\lambda i \rightarrow \text{sym } (\text{fst } eq \ i)),$

$((\lambda i \rightarrow \text{sym } (\text{fst } (\text{snd } eq) \ i)),$

$(\lambda i \rightarrow \text{sym } (\text{snd } (\text{snd } eq) \ i)))$

$\text{Vec12Eq-trans} : \{u v w : \text{Vec12}\mathbb{Z}\} \rightarrow \text{Vec12Eq } u v \rightarrow \text{Vec12Eq } v w \rightarrow \text{Vec12Eq } u w$

$\text{Vec12Eq-trans } eq_1 \ eq_2 =$

$(\lambda i \rightarrow \text{trans } (\text{fst } eq_1 \ i) \ (\text{fst } eq_2 \ i)),$

$((\lambda i \rightarrow \text{trans } (\text{fst } (\text{snd } eq_1) \ i) \ (\text{fst } (\text{snd } eq_2) \ i)),$

$(\lambda i \rightarrow \text{trans } (\text{snd } (\text{snd } eq_1) \ i) \ (\text{snd } (\text{snd } eq_2) \ i)))$

```

-- § sum congruence: blockwise
sum12-cong : (u v : Vec12ℤ) → Vec12Eq u v → sum12ℤ u ≡ sum12ℤ v
sum12-cong u v eq =
  trans
    (cong (λ t → t +ℤ (sumFin4ℤ (block1 u) +ℤ sumFin4ℤ (block2 u)))
      (sumFin4-cong (block0 u) (block0 v) (fst eq)))
    (cong (λ t → sumFin4ℤ (block0 v) +ℤ t)
      (trans
        (cong (λ t → t +ℤ sumFin4ℤ (block2 u))
          (sumFin4-cong (block1 u) (block1 v) (fst (snd eq))))
        (cong (λ t → sumFin4ℤ (block1 v) +ℤ t)
          (sumFin4-cong (block2 u) (block2 v) (snd (snd eq))))))

-- § scaling congruence
twelveVec12-cong : (u v : Vec12ℤ) → Vec12Eq u v → Vec12Eq (twelveVec12ℤ u) (twelveVec12ℤ v)
twelveVec12-cong u v eq =
  (λ i → cong twelveTimesℤ (fst eq i)) ,
  ((λ i → cong twelveTimesℤ (fst (snd eq) i)) ,
   (λ i → cong twelveTimesℤ (snd (snd eq) i)))

opaque
unfolding K12LaplacianVec12ℤ

-- §  $K_{12}$  Laplacian congruence
K12Laplacian-cong : (u v : Vec12ℤ) → Vec12Eq u v → Vec12Eq (K12LaplacianVec12ℤ u) (K12LaplacianVec12ℤ v)
K12Laplacian-cong u v eq =
  (λ i →
    let pBlock = cong twelveTimesℤ (fst eq i) in
    let pSum = cong negℤ (sum12-cong u v eq) in
    trans (cong (λ t → twelveTimesℤ (block0 u i) +ℤ t) pSum)
      (trans (cong (λ t → t +ℤ negℤ (sum12ℤ v)) pBlock) refl)) ,
  ((λ i →
    let pBlock = cong twelveTimesℤ (fst (snd eq) i) in
    let pSum = cong negℤ (sum12-cong u v eq) in
    trans (cong (λ t → twelveTimesℤ (block1 u i) +ℤ t) pSum)
      (trans (cong (λ t → t +ℤ negℤ (sum12ℤ v)) pBlock) refl)) ,
   (λ i →
    let pBlock = cong twelveTimesℤ (snd (snd eq) i) in
    let pSum = cong negℤ (sum12-cong u v eq) in
    trans (cong (λ t → twelveTimesℤ (block2 u i) +ℤ t) pSum)
      (trans (cong (λ t → t +ℤ negℤ (sum12ℤ v)) pBlock) refl))))

```

Iterated Operator Collapse

Laws 14K.0–14K.2 show that every iterate of L_{12} or J_{12} beyond the first is a 12-scaled copy of its predecessor. The quadratic identities from the previous chapter turn directly into the recurrences for pure powers, and repeated transport then yields the closed forms: every non-trivial iterate of L_{12} is a power of 12 times the first Laplacian action, and every non-trivial iterate of J_{12} is a power of 12 times the first broadcast action.

```

-- § Law 14K.0: two-step recurrence  $L_{12}^{(n+2)} = 12 \cdot L_{12}^{(n+1)}$ 
law14K-0-LL-step : (n : ℕ) → (v : Vec12ℤ) →
  Vec12Eq (powEndo (suc (suc n)) K12LaplacianVec12ℤ v)
    (twelveVec12ℤ (powEndo (suc n) K12LaplacianVec12ℤ v))
law14K-0-LL-step n v = law14H-11-LL-twelveL (powEndo n K12LaplacianVec12ℤ v)

-- § Law 14K.1:  $L_{12}^{(n+1)} = 12^n \cdot L_{12}$ 
law14K-1-Lpow-scaling : (n : ℕ) → (v : Vec12ℤ) →
  Vec12Eq (powEndo (suc n) K12LaplacianVec12ℤ v)
    (powEndo n twelveVec12ℤ (K12LaplacianVec12ℤ v))
law14K-1-Lpow-scaling zero v = Vec12Eq-refl (K12LaplacianVec12ℤ v)
law14K-1-Lpow-scaling (suc n) v =
  Vec12Eq-trans
    (law14K-0-LL-step n v)
    (twelveVec12-cong
      (powEndo (suc n) K12LaplacianVec12ℤ v)
      (powEndo n twelveVec12ℤ (K12LaplacianVec12ℤ v))
      (law14K-1-Lpow-scaling n v))

-- § Law 14K.2:  $J_{12}^{(n+1)} = 12^n \cdot J_{12}$ 
law14K-2-Jpow-scaling : (n : ℕ) → (v : Vec12ℤ) →
  Vec12Eq (powEndo (suc n) J12Vec12ℤ v)
    (powEndo n twelveVec12ℤ (J12Vec12ℤ v))
law14K-2-Jpow-scaling zero v = Vec12Eq-refl (J12Vec12ℤ v)
law14K-2-Jpow-scaling (suc n) v =
  Vec12Eq-trans
    (law14H-5-JJ-twelveJ (powEndo n J12Vec12ℤ v))
    (twelveVec12-cong
      (powEndo (suc n) J12Vec12ℤ v)
      (powEndo n twelveVec12ℤ (J12Vec12ℤ v))
      (law14K-2-Jpow-scaling n v))

```

Word Classification

The free monoid on the two generators $\{L, J\}$ evaluates to operators on $\text{Vec}_{12}\mathbb{Z}$. Every word falls into one of four cases: the empty word, a pure L -power, a pure J -power, or a mixed word, which evaluates to zero. Law 14L.0 formalizes this by a recursive classifier that assigns to each word a canonical case together with the corresponding operator equality.

```

-- § generator and word types
data Gen : Set where
  Lg : Gen
  Jg : Gen

-- § Type-level closure: Gen has exactly two inhabitants.
Gen-exhaustive : ∀ (g : Gen) → (g ≡ Lg) ∨ (g ≡ Jg)
Gen-exhaustive Lg = inj₁ refl
Gen-exhaustive Jg = inj₂ refl

```

```

data List (A : Set) : Set where
  [] : List A
  _::_ : A → List A → List A

Word : Set
Word = List Gen

-- § operator type and equality
Op : Set
Op = GenEndo Vec12ℤ

OpEq : Op → Op → Set
OpEq f g = (v : Vec12ℤ) → Vec12Eq (f v) (g v)

idOp : Op
idOp = idGenEndo

zeroOp : Op
zeroOp _ = zeroVec12ℤ

LOp : Op
LOp = K12LaplacianVec12ℤ

JOp : Op
JOp = J12Vec12ℤ

-- § word evaluation
evalGen : Gen → Op
evalGen Lg = LOp
evalGen Jg = JOp

evalWord : Word → Op
evalWord [] = idOp
evalWord (g :: w) = evalGen g ∘ evalWord w

-- § four classification cases
data WordCase : Set where
  empty : WordCase
  Lpow : ℕ → WordCase
  Jpow : ℕ → WordCase
  mixed : WordCase

-- § Type-level closure: WordCase has exactly four head-forms.
WordCase-head-exhaustive :
  ∀ (w : WordCase)
    → (w ≡ empty)
      ⊔ (Σ ℕ λ n → w ≡ Lpow n)
      ⊔ (Σ ℕ λ n → w ≡ Jpow n)
      ⊔ (w ≡ mixed)
WordCase-head-exhaustive empty = inj1 refl
WordCase-head-exhaustive (Lpow n) = inj2 (inj1 (n , refl))
WordCase-head-exhaustive (Jpow n) = inj2 (inj2 (inj1 (n , refl)))
WordCase-head-exhaustive mixed = inj2 (inj2 (inj2 refl))

caseOp : WordCase → Op

```



```

caseOp empty    = idOp
caseOp (Lpow n) = powEndo (suc n) LOp
caseOp (Jpow n) = powEndo (suc n) JOp
caseOp mixed    = zeroOp

-- § step function: how appending a generator updates the case
stepCase : Gen → WordCase → WordCase
stepCase Lg empty    = Lpow zero
stepCase Jg empty    = Jpow zero
stepCase Lg (Lpow n) = Lpow (suc n)
stepCase Jg (Jpow n) = Jpow (suc n)
stepCase Lg (Jpow _) = mixed
stepCase Jg (Lpow _) = mixed
stepCase _ mixed     = mixed

-- § J12 congruence (from sum congruence)
J12-cong : (u v : Vec12ℤ) → Vec12Eq u v → Vec12Eq (JOp u) (JOp v)
J12-cong u v eq =
  let sEq = sum12-cong u v eq in
  (λ _ → sEq), ((λ _ → sEq), (λ _ → sEq))

-- § mixed annihilation: L after J-powers and J after L-powers
LopJpow-zero : (n : ℕ) → OpEq (LOp ∘ powEndo (suc n) JOp) zeroOp
LopJpow-zero n v = law14H-10-LJ-zero (powEndo n JOp v)

JopLpow-zero : (n : ℕ) → OpEq (JOp ∘ powEndo (suc n) LOp) zeroOp
JopLpow-zero n v = law14H-9-JL-zero (powEndo n LOp v)

-- § composition respects case classification (8 cases)
composeGenCase : (g : Gen) → (c : WordCase) → OpEq (evalGen g ∘ caseOp c) (caseOp (stepCase g c))
composeGenCase Lg empty v = Vec12Eq-refl (LOp v)
composeGenCase Jg empty v = Vec12Eq-refl (JOp v)
composeGenCase Lg (Lpow n) v = Vec12Eq-refl (powEndo (suc (suc n)) LOp v)
composeGenCase Jg (Jpow n) v = Vec12Eq-refl (powEndo (suc (suc n)) JOp v)
composeGenCase Lg (Jpow n) v = LopJpow-zero n v
composeGenCase Jg (Lpow n) v = JopLpow-zero n v
composeGenCase Lg mixed v = law14H-14-const-eigen0 0ℤ
composeGenCase Jg mixed v = Vec12Eq-refl zeroVec12ℤ

-- § generator congruence
congGen : (g : Gen) → (u v : Vec12ℤ) → Vec12Eq u v → Vec12Eq (evalGen g u) (evalGen g v)
congGen Lg u v eq = K12Laplacian-cong u v eq
congGen Jg u v eq = J12-cong u v eq

-- § Law 14L.0: every word factors through a canonical case (existence; deterministic by recursion)
law14L-0-classify-word : (w : Word) → ∑ WordCase (λ c → OpEq (evalWord w) (caseOp c))
law14L-0-classify-word [] = empty, (λ v → Vec12Eq-refl v)
law14L-0-classify-word (g :: w) =
  let rec = law14L-0-classify-word w in
  let c = fst rec in
  let eq = snd rec in
  stepCase g c,
  (λ v →
    Vec12Eq-trans
      (congGen g (evalWord w v) (caseOp c v) (eq v))
      (composeGenCase g c v))

```


Chapter 48

Rational Multiplication and ε -Splitting

Rational multiplication must be associative, unital, and distributive, and these properties are obtained by reducing each statement to the corresponding integer identity after clearing denominators. The second half of the chapter uses the same bookkeeping to derive the ε -splitting lemma: from a positive rational ε one extracts a smaller rational δ such that $\delta + \delta < \varepsilon$.

Rational Multiplication Laws

The multiplication laws on $\mathbb{Q}$ are proved by rewriting each rational expression until only an identity about integer numerators and positive denominators remains. The auxiliary lemmas on triple products serve only to align the denominator bookkeeping with integer associativity. Once these rewrites are in place, associativity, the unit laws, zero annihilation, and distributivity follow directly.

```
-- § triple  $\mathbb{N}^+$ -product helpers
+to $\mathbb{Z}$ .*+-assocr : (a b c :  $\mathbb{N}^+$ ) → +to $\mathbb{Z}$  ((a *+ b) *+ c) ≡ (+to $\mathbb{Z}$  a) * $\mathbb{Z}$  ((+to $\mathbb{Z}$  b) * $\mathbb{Z}$  (+to $\mathbb{Z}$  c))
+to $\mathbb{Z}$ .*+-assocr a b c =
trans
(+to $\mathbb{Z}$ .*+ (a *+ b) c)
(trans
(cong (λ t → t * $\mathbb{Z}$  +to $\mathbb{Z}$  c) (+to $\mathbb{Z}$ .*+ a b))
(* $\mathbb{Z}$ -assoc (+to $\mathbb{Z}$  a) (+to $\mathbb{Z}$  b) (+to $\mathbb{Z}$  c)))

+to $\mathbb{Z}$ .*+-assocl : (a b c :  $\mathbb{N}^+$ ) → +to $\mathbb{Z}$  (a *+ (b *+ c)) ≡ (+to $\mathbb{Z}$  a) * $\mathbb{Z}$  ((+to $\mathbb{Z}$  b) * $\mathbb{Z}$  (+to $\mathbb{Z}$  c))
+to $\mathbb{Z}$ .*+-assocl a b c =
trans
(+to $\mathbb{Z}$ .*+ a (b *+ c))
(cong (λ t → (+to $\mathbb{Z}$  a) * $\mathbb{Z}$  t) (+to $\mathbb{Z}$ .*+ b c))
```

```

-- § *Q associativity
*Q-assoc : (p q r : Q) → (p *Q q) *Q r ≈Q p *Q (q *Q r)
*Q-assoc (a / b) (c / d) (e / f) =
let
  numAssoc : ((a *Z c) *Z e) ≡ (a *Z (c *Z e))
  numAssoc = *Z-assoc a c e

  denL : +toZ ((b *+ d) *+ f) ≡ (+toZ b) *Z ((+toZ d) *Z (+toZ f))
  denL = +toZ-*+ -assocr b d f

  denR : +toZ (b *+ (d *+ f)) ≡ (+toZ b) *Z ((+toZ d) *Z (+toZ f))
  denR = +toZ-*+ -assocl b d f

  denEq : +toZ ((b *+ d) *+ f) ≡ +toZ (b *+ (d *+ f))
  denEq = trans denL (sym denR)

cross : (((a *Z c) *Z e) *Z +toZ (b *+ (d *+ f))) ≡ ((a *Z (c *Z e)) *Z +toZ ((b *+ d) *+ f))
cross =
trans
  (cong (λ t → ((a *Z c) *Z e) *Z t) (sym denEq))
  (cong (λ t → t *Z +toZ ((b *+ d) *+ f)) numAssoc)
in
cross

-- § *Q right identity
*Q-one-right : (p : Q) → (p *Q 1Q) ≈Q p
*Q-one-right (a / b) =
let
  numEq : (a *Z oneZ) ≡ a
  numEq = *Z-one-right a

  denOne : +toZ b ≡ +toZ (b *+ one+)
  denOne =
trans
  (sym (*Z-one-right (+toZ b)))
  (sym (+toZ-*+ b one+))

cross : ((a *Z oneZ) *Z +toZ b) ≡ (a *Z +toZ (b *+ one+))
cross =
trans
  (cong (λ t → t *Z +toZ b) numEq)
  (cong (λ t → a *Z t) denOne)
in
cross

-- § *Q left identity
*Q-one-left : (p : Q) → (1Q *Q p) ≈Q p
*Q-one-left (a / b) =
let
  numEq : (oneZ *Z a) ≡ a
  numEq = *Z-one-left a

  denOneL : +toZ b ≡ +toZ (one+ *+ b)
  denOneL = sym (trans (+toZ-*+ one+ b) (*Z-one-left (+toZ b)))

```

```

in
trans
  (cong (λ t → t *ℤ +toℤ b) numEq)
  (cong (λ t → a *ℤ t) denOneL)

-- § *ℚ zero annihilation (left)
*ℚ-zero-left : (p : ℚ) → (0ℚ *ℚ p) ≃ℚ 0ℚ
*ℚ-zero-left (a / b) =
let
  numEq : (0ℤ *ℤ a) ≡ 0ℤ
  numEq = *ℤ-zero-left a

cross : ((0ℤ *ℤ a) *ℤ +toℤ one+) ≡ (0ℤ *ℤ +toℤ (one+ ** b))
cross =
trans
  (cong (λ t → t *ℤ +toℤ one+) numEq)
  (trans
    (*ℤ-zero-left (+toℤ one+))
    (sym (*ℤ-zero-left (+toℤ (one+ ** b))))))
in
cross

-- § *ℚ zero annihilation (right)
*ℚ-zero-right : (p : ℚ) → (p *ℚ 0ℚ) ≃ℚ 0ℚ
*ℚ-zero-right (a / b) =
let
  numEq : (a *ℤ 0ℤ) ≡ 0ℤ
  numEq = *ℤ-zero-right a
in
trans
  (cong (λ t → t *ℤ +toℤ one+) numEq)
  (trans
    (*ℤ-zero-left (+toℤ one+))
    (sym (*ℤ-zero-left (+toℤ (b ** one+))))))

```

Reciprocal on the positive sector

Reciprocals can be introduced cleanly first on the strictly positive sector of $\mathbb{Q}$. A witness of $0 < q$ forces the numerator into the form $+\text{succ } n$, so exchanging denominator and numerator magnitude cannot create a zero denominator. The reciprocal defined in this way stays positive, and its product with q reduces directly to 1. The later extension to arbitrary nonzero rationals adds only the necessary sign bookkeeping.

```

-- § Positive numerator forces positive rational.
0<ℤnum→0<ℚ : (q : ℚ) → 0ℤ <ℤ num q → 0ℚ <ℚ q
0<ℤnum→0<ℚ (a / b) apos =
let
  step1 : 0ℤ <ℤ (a *ℤ +toℤ one+)
  step1 = <ℤ-resp≡r (sym (*ℤ-one-right a)) apos
in
<ℤ-resp≡l (sym (*ℤ-zero-left (+toℤ b))) step1

```

```

-- § A positive rational exposes its numerator as +suc n.
positive-num-witness $\mathbb{Q}$  : (q :  $\mathbb{Q}$ ) → 0 $\mathbb{Q}$  < $\mathbb{Q}$  q →  $\Sigma \mathbb{N} (\lambda n \rightarrow \text{num } q \equiv +\text{suc } n)$ 
positive-num-witness $\mathbb{Q}$  q qpos =
  0< $\mathbb{Z}$ →pos (num q) (0 $\mathbb{Q}$ <→0 $\mathbb{Z}$ <num q qpos)

-- § Reciprocal on the strictly positive sector of  $\mathbb{Q}$ .
reciprocal+ : (q :  $\mathbb{Q}$ ) → 0 $\mathbb{Q}$  < $\mathbb{Q}$  q →  $\mathbb{Q}$ 
reciprocal+ (a / b) qpos with positive-num-witness $\mathbb{Q}$  (a / b) qpos
... | n, _ = +to $\mathbb{Z}$  b / mk $\mathbb{N}$ + n

-- § The reciprocal of a positive rational is again positive.
reciprocal+-pos : (q :  $\mathbb{Q}$ ) → (qpos : 0 $\mathbb{Q}$  < $\mathbb{Q}$  q) → 0 $\mathbb{Q}$  < $\mathbb{Q}$  reciprocal+ q qpos
reciprocal+-pos (a / b) qpos with positive-num-witness $\mathbb{Q}$  (a / b) qpos
... | n, _ = 0< $\mathbb{Z}$ num→0< $\mathbb{Q}$  ((+to $\mathbb{Z}$  b) / mk $\mathbb{N}$ + n) (den-pos $\mathbb{Z}$  b)

-- § Positive rational times its reciprocal is 1.
* $\mathbb{Q}$ -reciprocal+-right : (q :  $\mathbb{Q}$ ) → (qpos : 0 $\mathbb{Q}$  < $\mathbb{Q}$  q) → (q * $\mathbb{Q}$  reciprocal+ q qpos)  $\simeq$  1 $\mathbb{Q}$ 
* $\mathbb{Q}$ -reciprocal+-right (a / b) qpos with positive-num-witness $\mathbb{Q}$  (a / b) qpos
... | n, a $\equiv$  =
  let
    lhsOne : ((a * $\mathbb{Z}$  +to $\mathbb{Z}$  b) * $\mathbb{Z}$  +to $\mathbb{Z}$  one+)  $\equiv$  (a * $\mathbb{Z}$  +to $\mathbb{Z}$  b)
    lhsOne = * $\mathbb{Z}$ -one-right (a * $\mathbb{Z}$  +to $\mathbb{Z}$  b)

    core : (a * $\mathbb{Z}$  +to $\mathbb{Z}$  b)  $\equiv$  +to $\mathbb{Z}$  (b *+ mk $\mathbb{N}$ + n)
    core =
      trans
        (cong (λ t → t * $\mathbb{Z}$  +to $\mathbb{Z}$  b) a $\equiv$ )
        (trans
          (* $\mathbb{Z}$ -comm (+suc n) (+to $\mathbb{Z}$  b))
          (sym (+to $\mathbb{Z}$ -*+ b (mk $\mathbb{N}$ + n))))

    rhsOne : +to $\mathbb{Z}$  (b *+ mk $\mathbb{N}$ + n)  $\equiv$  (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  (b *+ mk $\mathbb{N}$ + n))
    rhsOne = sym (* $\mathbb{Z}$ -one-left (+to $\mathbb{Z}$  (b *+ mk $\mathbb{N}$ + n)))
  in
    trans lhsOne (trans core rhsOne)

-- § The reciprocal also cancels on the left.
* $\mathbb{Q}$ -reciprocal+-left : (q :  $\mathbb{Q}$ ) → (qpos : 0 $\mathbb{Q}$  < $\mathbb{Q}$  q) → (reciprocal+ q qpos * $\mathbb{Q}$  q)  $\simeq$  1 $\mathbb{Q}$ 
* $\mathbb{Q}$ -reciprocal+-left q qpos =
   $\simeq$  $\mathbb{Q}$ -trans
    (* $\mathbb{Q}$ -comm (reciprocal+ q qpos) q)
    (* $\mathbb{Q}$ -reciprocal+-right q qpos)

-- § Negation commutes with rational multiplication on the left.
* $\mathbb{Q}$ -neg-left : (p q :  $\mathbb{Q}$ ) → ((- $\mathbb{Q}$  p) * $\mathbb{Q}$  q)  $\simeq$   $\mathbb{Q}$  (- $\mathbb{Q}$  (p * $\mathbb{Q}$  q))
* $\mathbb{Q}$ -neg-left (a / b) (c / d) =
  cong (λ t → t * $\mathbb{Z}$  +to $\mathbb{Z}$  (b *+ d)) (* $\mathbb{Z}$ -neg-left a c)

-- § Negation commutes with rational multiplication on the right.
* $\mathbb{Q}$ -neg-right : (p q :  $\mathbb{Q}$ ) → (p * $\mathbb{Q}$  (- $\mathbb{Q}$  q))  $\simeq$   $\mathbb{Q}$  (- $\mathbb{Q}$  (p * $\mathbb{Q}$  q))
* $\mathbb{Q}$ -neg-right (a / b) (c / d) =
  cong (λ t → t * $\mathbb{Z}$  +to $\mathbb{Z}$  (b *+ d)) (* $\mathbb{Z}$ -neg-right a c)

-- § Double negation is trivial on rational classes.

```

```

-ℚ-involutive : (p : ℚ) → (-ℚ (-ℚ p)) ≈ ℚ p
-ℚ-involutive (a / b) =
  cong (λ t → t * ℤ +toℤ b) (negℤ-involutive a)

-- § Local negation transport on rational equivalence.
negℚ-resp-≈ : {p q : ℚ} → p ≈ ℚ q → (-ℚ p) ≈ ℚ (-ℚ q)
negℚ-resp-≈ {a / b} {c / d} eq =
  trans
    (*ℤ-neg-left a (+toℤ d))
  (trans
    (cong negℤ eq)
    (sym (*ℤ-neg-left c (+toℤ b))))

-- § Nonzero rational: numerator is not zero.
NonZeroℚ : ℚ → Set
NonZeroℚ q = num q ≡ 0ℤ → ⊥

-- § Reciprocal on the full nonzero sector of ℚ.
reciprocal≠0 : (q : ℚ) → NonZeroℚ q → ℚ
reciprocal≠0 (0ℤ / b) qnz = ⊥-elim (qnz refl)
reciprocal≠0 ((+suc n) / b) qnz =
  reciprocal+ ((+suc n) / b) (0<ℤnum→0<ℚ ((+suc n) / b) (0<ℤ-pos n))
reciprocal≠0 ((-suc n) / b) qnz =
  -ℚ (reciprocal+ ((+suc n) / b) (0<ℤnum→0<ℚ ((+suc n) / b) (0<ℤ-pos n)))

-- § Any nonzero rational times its reciprocal is 1.
*ℚ-reciprocal≠0-right : (q : ℚ) → (qnz : NonZeroℚ q) → (q *ℚ reciprocal≠0 q qnz) ≈ ℚ 1ℚ
*ℚ-reciprocal≠0-right (0ℤ / b) qnz = ⊥-elim (qnz refl)
*ℚ-reciprocal≠0-right ((+suc n) / b) qnz =
  *ℚ-reciprocal+-right ((+suc n) / b) (0<ℤnum→0<ℚ ((+suc n) / b) (0<ℤ-pos n))
*ℚ-reciprocal≠0-right ((-suc n) / b) qnz =
  let
    p : ℚ
    p = (+suc n) / b

    ppos : 0ℚ <ℚ p
    ppos = 0<ℤnum→0<ℚ p (0<ℤ-pos n)

    step1 : (((-ℚ p) *ℚ (-ℚ (reciprocal+ p ppos)))) ≈ ℚ (-ℚ (p *ℚ (-ℚ (reciprocal+ p ppos))))
    step1 = *ℚ-neg-left p (-ℚ (reciprocal+ p ppos))

    step2 : (p *ℚ (-ℚ (reciprocal+ p ppos))) ≈ ℚ (-ℚ (p *ℚ reciprocal+ p ppos))
    step2 = *ℚ-neg-right p (reciprocal+ p ppos)

    step3 : (-ℚ (p *ℚ (-ℚ (reciprocal+ p ppos)))) ≈ ℚ (-ℚ (-ℚ (p *ℚ reciprocal+ p ppos)))
    step3 = negℚ-resp-≈ step2

    step4 : (-ℚ (-ℚ (p *ℚ reciprocal+ p ppos))) ≈ ℚ (p *ℚ reciprocal+ p ppos)
    step4 = -ℚ-involutive (p *ℚ reciprocal+ p ppos)
  in
    ≈ℚ-trans step1 (≈ℚ-trans step3 (≈ℚ-trans step4 (*ℚ-reciprocal+-right p ppos)))

-- § The reciprocal also cancels on the left for any nonzero rational.
*ℚ-reciprocal≠0-left : (q : ℚ) → (qnz : NonZeroℚ q) → (reciprocal≠0 q qnz *ℚ q) ≈ ℚ 1ℚ
*ℚ-reciprocal≠0-left q qnz =
  ≈ℚ-trans
    (*ℚ-comm (reciprocal≠0 q qnz) q)
    (*ℚ-reciprocal≠0-right q qnz)

```

Rational distributivity. With associativity, unit, and zero laws in place, distributivity is the remaining algebraic step. As before, both left and right distributivity reduce to integer identities after clearing denominators and aligning the cross-products.

```
-- § *Q right-distributes over +Q
*Q-distrib-right+Q : (p q r : Q) → p *Q (q +Q r) ≃Q (p *Q q) +Q (p *Q r)
*Q-distrib-right+Q (a / b) (c / d) (e / f) =
let
  B : Z
  B = +toZ b

  D : Z
  D = +toZ d

  F : Z
  F = +toZ f

  bd : N+
  bd = b *+ d

  bf : N+
  bf = b *+ f

  df : N+
  df = d *+ f

  denR : Z
  denR = (B *Z D) *Z (B *Z F)

  denL : Z
  denL = B *Z (D *Z F)

  denR≡ : +toZ (bd *+ bf) ≡ denR
  denR≡ =
    trans
      (+toZ-*+ bd bf)
      (cong2 _Z_ (+toZ-*+ b d) (+toZ-*+ b f))

  denL≡ : +toZ (b *+ df) ≡ denL
  denL≡ = +toZ-*+-assocl b d f

  cF : Z
  cF = c *Z F

  eD : Z
  eD = e *Z D

  lhsNum : Z
  lhsNum = a *Z (cF +Z eD)

  lhsExpand0 : (lhsNum *Z denR) ≡ ((a *Z cF) *Z denR) +Z ((a *Z eD) *Z denR)
  lhsExpand0 =
    trans
      (cong (λ t → t *Z denR) (*Z-distrib-right+Z a cF eD))
      (*Z-distrib-left+Z (a *Z cF) (a *Z eD) denR)
```



```

rhsNum : ℤ
rhsNum = ((a *ℤ c) *ℤ +toℤ bf) +ℤ ((a *ℤ e) *ℤ +toℤ bd)

rhsExpand₀ : (rhsNum *ℤ denL) ≡ (((a *ℤ c) *ℤ +toℤ bf) *ℤ denL) +ℤ (((a *ℤ e) *ℤ +toℤ bd) *ℤ denL)
rhsExpand₀ = *ℤ-distrib-left+ℤ ((a *ℤ c) *ℤ +toℤ bf) ((a *ℤ e) *ℤ +toℤ bd) denL

-- term 1 alignment
t1-lhs : ((a *ℤ cF) *ℤ denR) ≡ ((a *ℤ c) *ℤ denR) *ℤ F
t1-lhs =
  trans
    (cong (λ t → t *ℤ denR) (sym (*ℤ-assoc a c F)))
  (trans
    (*ℤ-assoc (a *ℤ c) F denR)
    (trans
      (cong (λ t → (a *ℤ c) *ℤ t) (*ℤ-comm F denR))
      (sym (*ℤ-assoc (a *ℤ c) denR F))))

t1-rhs : (((a *ℤ c) *ℤ +toℤ bf) *ℤ denL) ≡ ((a *ℤ c) *ℤ denR) *ℤ F
t1-rhs =
  let
    bf→ : +toℤ bf ≡ B *ℤ F
    bf→ = +toℤ-*+ b f

    denL→ : denL ≡ (B *ℤ D) *ℤ F
    denL→ = sym (*ℤ-assoc B D F)
  in
  trans
    (cong (λ t → ((a *ℤ c) *ℤ t) *ℤ denL) bf→)
  (trans
    (cong (λ t → ((a *ℤ c) *ℤ (B *ℤ F)) *ℤ t) denL→)
    (trans
      (sym (*ℤ-assoc ((a *ℤ c) *ℤ (B *ℤ F)) (B *ℤ D) F))
      (trans
        (cong (λ t → t *ℤ F) (*ℤ-assoc (a *ℤ c) (B *ℤ F) (B *ℤ D)))
        (cong (λ t → ((a *ℤ c) *ℤ t) *ℤ F) (*ℤ-comm (B *ℤ F) (B *ℤ D)))))))

t1 : ((a *ℤ cF) *ℤ denR) ≡ (((a *ℤ c) *ℤ +toℤ bf) *ℤ denL)
t1 = trans t1-lhs (sym t1-rhs)

-- term 2 alignment
t2-lhs : ((a *ℤ eD) *ℤ denR) ≡ ((a *ℤ e) *ℤ denR) *ℤ D
t2-lhs =
  trans
    (cong (λ t → t *ℤ denR) (sym (*ℤ-assoc a e D)))
  (trans
    (*ℤ-assoc (a *ℤ e) D denR)
    (trans
      (cong (λ t → (a *ℤ e) *ℤ t) (*ℤ-comm D denR))
      (sym (*ℤ-assoc (a *ℤ e) denR D))))

t2-rhs : (((a *ℤ e) *ℤ +toℤ bd) *ℤ denL) ≡ ((a *ℤ e) *ℤ denR) *ℤ D
t2-rhs =
  let

```

```

bd→ : +toℤ bd ≡ B *ℤ D
bd→ = +toℤ-+ b d

denL→ : denL ≡ (B *ℤ F) *ℤ D
denL→ =
  trans
    (cong (λ t → B *ℤ t) (*ℤ-comm D F))
    (sym (*ℤ-assoc B F D))
in
trans
  (cong (λ t → ((a *ℤ e) *ℤ t) *ℤ denL) bd→)
  (trans
    (cong (λ t → ((a *ℤ e) *ℤ (B *ℤ D)) *ℤ t) denL→)
    (trans
      (sym (*ℤ-assoc ((a *ℤ e) *ℤ (B *ℤ D)) (B *ℤ F) D))
      (cong (λ t → t *ℤ D) (*ℤ-assoc (a *ℤ e) (B *ℤ D) (B *ℤ F))))))

t2 : ((a *ℤ eD) *ℤ denR) ≡ (((a *ℤ e) *ℤ +toℤ bd) *ℤ denL)
t2 = trans t2-lhs (sym t2-rhs)

sumEq : (((a *ℤ cF) *ℤ denR) +ℤ ((a *ℤ eD) *ℤ denR)) ≡ (((a *ℤ c) *ℤ +toℤ bf) *ℤ denL) +ℤ (((a *ℤ e) *ℤ +toℤ bd) *ℤ denL)
sumEq = cong2 _+_ t1 t2
in
trans
  (cong (λ t → lhsNum *ℤ t) denR≡)
  (trans
    lhsExpand0
    (trans
      sumEq
      (trans
        (sym rhsExpand0)
        (cong (λ t → rhsNum *ℤ t) (sym denL≡))))))

-- § *ℚ left-distributes over +ℚ (by commutativity reduction)
*ℚ-distrib-left-+ℚ : (p q r : ℚ) → (p +ℚ q) *ℚ r ≃ℚ (p *ℚ r) +ℚ (q *ℚ r)
*ℚ-distrib-left-+ℚ p q r =
  ≃ℚ-trans
    {p = (p +ℚ q) *ℚ r}
    {q = r *ℚ (p +ℚ q)}
    {r = (p *ℚ r) +ℚ (q *ℚ r)}
    (*ℚ-comm (p +ℚ q) r)
    (≃ℚ-trans
      {p = r *ℚ (p +ℚ q)}
      {q = (r *ℚ p) +ℚ (r *ℚ q)}
      {r = (p *ℚ r) +ℚ (q *ℚ r)}
      (*ℚ-distrib-right-+ℚ r p q)
      (+ℚ-resp-≃
        {p = r *ℚ p}
        {p' = p *ℚ r}
        {q = r *ℚ q}
        {q' = q *ℚ r}
        (*ℚ-comm r p)
        (*ℚ-comm r q)))

```

ε -Splitting Laws

For any positive $\varepsilon = a/b$, set $\delta = 1/(4b)$. Then $0 < \delta$ and $\delta + \delta < \varepsilon$. This provides a canonical smaller tolerance extracted directly from the denominator of ε , which is exactly what the later Cauchy arguments require.

```
-- § forced constant: 2 as  $\mathbb{N}^+$ 
two+ :  $\mathbb{N}^+$ 
two+ = suc+ one+

-- § denominator constructors for  $\varepsilon$ -splitting
halfDen :  $\mathbb{N}^+ \rightarrow \mathbb{N}^+$ 
halfDen b = two+ *+ b

quarterDen :  $\mathbb{N}^+ \rightarrow \mathbb{N}^+$ 
quarterDen b = two+ *+ (two+ *+ b)

 $\varepsilon$ Quarter :  $\mathbb{Q} \rightarrow \mathbb{Q}$ 
 $\varepsilon$ Quarter (a / b) = one $\mathbb{Z}$  / quarterDen b

 $\varepsilon$ Half :  $\mathbb{Q} \rightarrow \mathbb{Q}$ 
 $\varepsilon$ Half (a / b) = one $\mathbb{Z}$  / halfDen b

-- §  $\varepsilon$ Quarter is positive
 $\varepsilon$ Quarter-pos : ( $\varepsilon$  :  $\mathbb{Q}$ )  $\rightarrow$  0 $\mathbb{Q}$  <  $\mathbb{Q}$   $\varepsilon$ Quarter  $\varepsilon$ 
 $\varepsilon$ Quarter-pos (a / b) =
  let
    qd :  $\mathbb{N}^+$ 
    qd = quarterDen b

    lhs0 : (0 $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  qd)  $\equiv$  0 $\mathbb{Z}$ 
    lhs0 = * $\mathbb{Z}$ -zero-left (+to $\mathbb{Z}$  qd)

    one+ $\mathbb{Z}$   $\equiv$  one $\mathbb{Z}$  : +to $\mathbb{Z}$  one+  $\equiv$  one $\mathbb{Z}$ 
    one+ $\mathbb{Z}$   $\equiv$  one $\mathbb{Z}$  = refl

    rhs1 : (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  one+)  $\equiv$  one $\mathbb{Z}$ 
    rhs1 = trans (cong ( $\lambda$  t  $\rightarrow$  one $\mathbb{Z}$  * $\mathbb{Z}$  t) one+ $\mathbb{Z}$   $\equiv$  one $\mathbb{Z}$ ) (* $\mathbb{Z}$ -one-right one $\mathbb{Z}$ )

  in
    < $\mathbb{Z}$ -resp- $\equiv$ l {x = 0 $\mathbb{Z}$ } {y = 0 $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  qd} {z = one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  one+} (sym lhs0)
    (< $\mathbb{Z}$ -resp- $\equiv$ r {x = 0 $\mathbb{Z}$ } {y = one $\mathbb{Z}$ } {z = one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  one+} (sym rhs1) 0 $\mathbb{Z}$  < one $\mathbb{Z}$ )

-- § doubling  $\varepsilon$ Quarter yields  $\varepsilon$ Half (up to  $\simeq \mathbb{Q}$ )
 $\varepsilon$ Quarter+ $\varepsilon$ Quarter $\simeq$  $\varepsilon$ Half : ( $\varepsilon$  :  $\mathbb{Q}$ )  $\rightarrow$  ( $\varepsilon$ Quarter  $\varepsilon$  +  $\mathbb{Q}$   $\varepsilon$ Quarter  $\varepsilon$ )  $\simeq \mathbb{Q}$  ( $\varepsilon$ Half  $\varepsilon$ )
 $\varepsilon$ Quarter+ $\varepsilon$ Quarter $\simeq$  $\varepsilon$ Half (a / b) =
  let
    qd :  $\mathbb{N}^+$ 
    qd = quarterDen b

    hd :  $\mathbb{N}^+$ 
    hd = halfDen b

    lhsNum :  $\mathbb{Z}$ 
    lhsNum = (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  qd) +  $\mathbb{Z}$  (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  qd)
```

```

lhsDen :  $\mathbb{N}^+$ 
lhsDen = qd *+ qd

qdSplit : +to $\mathbb{Z}$  qd  $\equiv$  (+to $\mathbb{Z}$  two $^+$ ) * $\mathbb{Z}$  ((+to $\mathbb{Z}$  two $^+$ ) * $\mathbb{Z}$  (+to $\mathbb{Z}$  b))
qdSplit =
  trans
    (+to $\mathbb{Z}$ -*+ two $^+$  (two $^+$  *+ b))
    (cong ( $\lambda$  t  $\rightarrow$  (+to $\mathbb{Z}$  two $^+$ ) * $\mathbb{Z}$  t) (+to $\mathbb{Z}$ -*+ two $^+$  b))

hdSplit : +to $\mathbb{Z}$  hd  $\equiv$  (+to $\mathbb{Z}$  two $^+$ ) * $\mathbb{Z}$  (+to $\mathbb{Z}$  b)
hdSplit = +to $\mathbb{Z}$ -*+ two $^+$  b

lhsExpand : (lhsNum * $\mathbb{Z}$  +to $\mathbb{Z}$  hd)  $\equiv$  (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  qd) * $\mathbb{Z}$  +to $\mathbb{Z}$  hd + $\mathbb{Z}$  (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  qd) * $\mathbb{Z}$  +to $\mathbb{Z}$  hd
lhsExpand =
  * $\mathbb{Z}$ -distrib-left-+ $\mathbb{Z}$  (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  qd) (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  qd) (+to $\mathbb{Z}$  hd)

oneqd : (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  qd)  $\equiv$  +to $\mathbb{Z}$  qd
oneqd = * $\mathbb{Z}$ -one-left (+to $\mathbb{Z}$  qd)

term : (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  qd) * $\mathbb{Z}$  +to $\mathbb{Z}$  hd  $\equiv$  (+to $\mathbb{Z}$  qd) * $\mathbb{Z}$  +to $\mathbb{Z}$  hd
term = cong ( $\lambda$  t  $\rightarrow$  t * $\mathbb{Z}$  +to $\mathbb{Z}$  hd) oneqd

rhs : (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  lhsDen)  $\equiv$  (+to $\mathbb{Z}$  qd) * $\mathbb{Z}$  (+to $\mathbb{Z}$  qd)
rhs =
  trans
    (cong ( $\lambda$  t  $\rightarrow$  one $\mathbb{Z}$  * $\mathbb{Z}$  t) (+to $\mathbb{Z}$ -*+ qd qd))
    (trans
      (* $\mathbb{Z}$ -one-left ((+to $\mathbb{Z}$  qd) * $\mathbb{Z}$  (+to $\mathbb{Z}$  qd)))
      refl)

two $\mathbb{Z}$  :  $\mathbb{Z}$ 
two $\mathbb{Z}$  = +to $\mathbb{Z}$  two $^+$ 

two $\mathbb{Z}$  $\equiv$  : two $\mathbb{Z}$   $\equiv$  one $\mathbb{Z}$  + $\mathbb{Z}$  one $\mathbb{Z}$ 
two $\mathbb{Z}$  $\equiv$  = refl

qd $\equiv$ twohd : qd  $\equiv$  two $^+$  *+ hd
qd $\equiv$ twohd = refl

qdAsTwoHd : +to $\mathbb{Z}$  qd  $\equiv$  two $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  hd
qdAsTwoHd = trans (cong +to $\mathbb{Z}$  qd $\equiv$ twohd) (+to $\mathbb{Z}$ -*+ two $^+$  hd)

qdHd :  $\mathbb{Z}$ 
qdHd = (+to $\mathbb{Z}$  qd) * $\mathbb{Z}$  +to $\mathbb{Z}$  hd

dupToMul2 : (qdHd + $\mathbb{Z}$  qdHd)  $\equiv$  qdHd * $\mathbb{Z}$  two $\mathbb{Z}$ 
dupToMul2 =
  trans
    (cong ( $\lambda$  t  $\rightarrow$  t + $\mathbb{Z}$  qdHd) (sym (* $\mathbb{Z}$ -one-right qdHd)))
    (trans
      (cong ( $\lambda$  t  $\rightarrow$  (qdHd * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  t) (sym (* $\mathbb{Z}$ -one-right qdHd)))
      (trans
        (* $\mathbb{Z}$ -distrib-right-+ $\mathbb{Z}$  qdHd one $\mathbb{Z}$  one $\mathbb{Z}$ )
        (cong ( $\lambda$  t  $\rightarrow$  qdHd * $\mathbb{Z}$  t) (sym two $\mathbb{Z}$  $\equiv$ ))))

```

```

squareToMul2 : ((+toℤ qd) *ℤ (+toℤ qd)) ≡ qdHd *ℤ twoℤ
squareToMul2 =
  trans
    (cong (λ t → (+toℤ qd) *ℤ t) qdAsTwoHd)
    (trans
      (sym (*ℤ-assoc (+toℤ qd) twoℤ (+toℤ hd)))
      (trans
        (cong (λ t → t *ℤ +toℤ hd) (*ℤ-comm (+toℤ qd) twoℤ))
        (trans
          (*ℤ-assoc twoℤ (+toℤ qd) (+toℤ hd))
          (*ℤ-comm twoℤ ((+toℤ qd) *ℤ +toℤ hd))))))

goal : (lhsNum *ℤ +toℤ hd) ≡ (oneℤ *ℤ +toℤ lhsDen)
goal =
  trans
    lhsExpand
    (trans
      (cong (λ t → t +ℤ t) term)
      (trans
        dupToMul2
        (trans (sym squareToMul2) (sym rhs))))
  in
  goal

```

Epsilon splitting ($\varepsilon/2$ infrastructure). The same denominator bookkeeping shows that ϵHalf remains positive and satisfies $\varepsilon/2 < \varepsilon$, while $\epsilon\text{Quarter}$ is retained for later product estimates.

```

-- §  $\epsilon\text{Half} < \epsilon$  (when  $\epsilon > 0$ )
 $\epsilon\text{Half} < \epsilon : (\epsilon : \mathbb{Q}) \rightarrow 0\mathbb{Q} < \mathbb{Q} \epsilon \rightarrow \epsilon\text{Half} \epsilon < \mathbb{Q} \epsilon$ 
 $\epsilon\text{Half} < \epsilon (a / b) \epsilon\text{pos} =$ 
  let
    aPos :  $0\mathbb{Z} < \mathbb{Z} a$ 
    aPos =  $0\mathbb{Q} < \rightarrow 0\mathbb{Z} < \text{num } (a / b) \epsilon\text{pos}$ 

  one<2a-sum :  $\text{one}\mathbb{Z} < \mathbb{Z} (a + \mathbb{Z} a)$ 
  one<2a-sum =  $\text{one}\mathbb{Z} < \text{twoTimes-pos } a \text{ aPos}$ 

  twoℤ :  $\mathbb{Z}$ 
  twoℤ = +toℤ two+

  twoℤ≡ :  $\text{two}\mathbb{Z} \equiv \text{one}\mathbb{Z} + \mathbb{Z} \text{one}\mathbb{Z}$ 
  twoℤ≡ = refl

  aTimesTwo≡ :  $(a * \mathbb{Z} \text{two}\mathbb{Z}) \equiv (a + \mathbb{Z} a)$ 
  aTimesTwo≡ =
    trans
      (cong (λ t → a *ℤ t) twoℤ≡)
      (trans
        (*ℤ-distrib-right-+ℤ a oneℤ oneℤ)
        (trans

```

```

    (cong (λ t → t + ℤ (a * ℤ oneℤ)) (*ℤ-one-right a))
    (cong (λ t → a + ℤ t) (*ℤ-one-right a))))

one<2a : oneℤ <ℤ (a * ℤ twoℤ)
one<2a = <ℤ-resp-≡r (sym aTimesTwo≡) one<2a-sum

step1 : (oneℤ *ℤ +toℤ b) <ℤ ((a * ℤ twoℤ) *ℤ +toℤ b)
step1 = <ℤ-mul-pos-right b one<2a

lhsEq : (oneℤ *ℤ +toℤ b) ≡ +toℤ b
lhsEq = *ℤ-one-left (+toℤ b)

rhsEq : ((a * ℤ twoℤ) *ℤ +toℤ b) ≡ (a * ℤ +toℤ (two+ *+ b))
rhsEq =
  trans
    (*ℤ-assoc a twoℤ (+toℤ b))
    (cong (λ t → a * ℤ t) (sym (+toℤ-*+ two+ b)))

core : (oneℤ *ℤ +toℤ b) <ℤ (a * ℤ +toℤ (two+ *+ b))
core = <ℤ-resp-≡r rhsEq step1
in
core

-- § εQuarter-double < ε
εQuarter-double<ε : (ε : ℚ) → 0ℚ <ℚ ε → (εQuarter ε + ℚ εQuarter ε) <ℚ ε
εQuarter-double<ε ε εpos =
  let
    eq : (εQuarter ε + ℚ εQuarter ε) ≃ℚ (εHalf ε)
    eq = εQuarter+εQuarter≃εHalf ε

    le : (εQuarter ε + ℚ εQuarter ε) ≤ℚ (εHalf ε)
    le = ≃ℚ→≤ℚl {p = εQuarter ε + ℚ εQuarter ε} {q = εHalf ε} eq

    halfLt : (εHalf ε) <ℚ ε
    halfLt = εHalf<ε ε εpos
  in
  ≤<ℚ→<ℚ {x = εQuarter ε + ℚ εQuarter ε} {y = εHalf ε} {z = ε} le halfLt

-- § εQuarter < ε (from εQuarter ≤ double < ε)
εQuarter<ε : (ε : ℚ) → 0ℚ <ℚ ε → εQuarter ε <ℚ ε
εQuarter<ε ε εpos =
  let
    eq : εQuarter ε ≃ℚ εQuarter ε
    eq = ≃ℚ-refl (εQuarter ε)

    eqPos : 0ℚ <ℚ εQuarter ε
    eqPos = εQuarter-pos ε

    eqNonneg : 0ℚ ≤ℚ εQuarter ε
    eqNonneg = <ℚ→≤ℚ {x = 0ℚ} {y = εQuarter ε} eqPos

    eq≤εq+εq : εQuarter ε ≤ℚ (εQuarter ε + ℚ εQuarter ε)
    eq≤εq+εq = ≤ℚ-add-nonneg-right (εQuarter ε) (εQuarter ε) eqNonneg

    double<ε : (εQuarter ε + ℚ εQuarter ε) <ℚ ε
    double<ε = εQuarter-double<ε ε εpos
  in
  ≤<ℚ→<ℚ {x = εQuarter ε} {y = εQuarter ε + ℚ εQuarter ε} {z = ε} eq≤εq+εq double<ε

```

Chapter 49

Rational Distance Laws

Each metric axiom for $d_{\mathbb{Q}}$ reduces to arithmetic on the scaled numerator and its positive denominator. Reflexivity comes from $x - x = 0$, symmetry from the invariance of absolute value under negation, and the triangle inequality from the integer estimate $|x + y| \leq |x| + |y|$ after passing to a common scale. No additional geometric structure is imposed here; the metric is another consequence of the rational arithmetic already established.

Metric Axioms

The basic metric axioms are proved by passing from $d_{\mathbb{Q}}$ to integer absolute-value identities together with denominator coherence in the rational setoid. Reflexivity, non-negativity, and symmetry are then direct translations of familiar facts about 0 and $|z|$ in $\mathbb{Z}$.

```
-- § absℤ respects propositional equality
absℤ-cong : {x y : ℤ} → x ≡ y → absℤ x ≡ absℤ y
absℤ-cong = cong absℤ

-- § Law 14U.0: distℚ q q ≃ℚ 0ℚ (reflexivity)
distℚ-refl : (q : ℚ) → distℚ q q ≃ℚ 0ℚ
distℚ-refl (a / b) =
  let x : ℤ
    x = a *ℤ +toℤ b

  numDist : ℤ
  numDist = absℤ (x +ℤ negℤ x)

  numDist≡0 : numDist ≡ 0ℤ
  numDist≡0 =
    trans
      (absℤ-cong (+ℤ-inv-right x))
      absℤ-zero
```

```

denDist :  $\mathbb{N}^+$ 
denDist = b  $^{*+}$  b

lhs0 : (numDist  $^{*\mathbb{Z}}$   $^{+to\mathbb{Z}}$  one $^+$ )  $\equiv 0\mathbb{Z}$ 
lhs0 =
  trans
    (cong ( $\lambda$  t  $\rightarrow$  t  $^{*\mathbb{Z}}$   $^{+to\mathbb{Z}}$  one $^+$ ) numDist $\equiv 0$ )
    (trans ( $^{*\mathbb{Z}}$ -zero-left ( $^{+to\mathbb{Z}}$  one $^+$ )) refl)

rhs0 : ( $0\mathbb{Z}$   $^{*\mathbb{Z}}$   $^{+to\mathbb{Z}}$  denDist)  $\equiv 0\mathbb{Z}$ 
rhs0 =  $^{*\mathbb{Z}}$ -zero-left ( $^{+to\mathbb{Z}}$  denDist)

in
trans lhs0 (sym rhs0)

-- § dist $\mathbb{Q}$  q q <  $\mathbb{Q}$   $\epsilon$  for any positive  $\epsilon$ 
dist $\mathbb{Q}$ -const< $\epsilon$  : (q  $\in$  :  $\mathbb{Q}$ )  $\rightarrow$   $0\mathbb{Q}$  <  $\mathbb{Q}$   $\epsilon$   $\rightarrow$  dist $\mathbb{Q}$  q q <  $\mathbb{Q}$   $\epsilon$ 
dist $\mathbb{Q}$ -const< $\epsilon$  (a / b) (c / d)  $\in pos$  =
  let x :  $\mathbb{Z}$ 
    x = a  $^{*\mathbb{Z}}$   $^{+to\mathbb{Z}}$  b

  numDist :  $\mathbb{Z}$ 
  numDist = abs $\mathbb{Z}$  (x +  $\mathbb{Z}$  neg $\mathbb{Z}$  x)

  numDist $\equiv 0$  : numDist  $\equiv 0\mathbb{Z}$ 
  numDist $\equiv 0$  =
    trans
      (abs $\mathbb{Z}$ -cong ( $^{+to\mathbb{Z}}$ -inv-right x))
      abs $\mathbb{Z}$ -zero

  lhs :  $\mathbb{Z}$ 
  lhs = numDist  $^{*\mathbb{Z}}$   $^{+to\mathbb{Z}}$  d

  rhs :  $\mathbb{Z}$ 
  rhs = c  $^{*\mathbb{Z}}$   $^{+to\mathbb{Z}}$  (b  $^{*+}$  b)

  lhs $\equiv 0$  : lhs  $\equiv 0\mathbb{Z}$ 
  lhs $\equiv 0$  =
    trans
      (cong ( $\lambda$  t  $\rightarrow$  t  $^{*\mathbb{Z}}$   $^{+to\mathbb{Z}}$  d) numDist $\equiv 0$ )
      ( $^{*\mathbb{Z}}$ -zero-left ( $^{+to\mathbb{Z}}$  d))

  cpos :  $0\mathbb{Z}$  <  $\mathbb{Z}$  c
  cpos =  $0\mathbb{Q}$  <  $\rightarrow 0\mathbb{Z}$  < num (c / d)  $\in pos$ 

  rhsPos :  $0\mathbb{Z}$  <  $\mathbb{Z}$  rhs
  rhsPos =  $0\mathbb{Z}$ -mul-pos-right c (b  $^{*+}$  b) cpos

  base :  $0\mathbb{Z}$  <  $\mathbb{Z}$  rhs
  base = rhsPos

in
< $\mathbb{Z}$ -resp $\equiv^l$  (sym lhs $\equiv 0$ ) base

-- § p  $\simeq \mathbb{Q}$  q implies dist $\mathbb{Q}$  p q  $\simeq \mathbb{Q}$   $0\mathbb{Q}$ 

```



```

distQ-≃0 : {p q : ℚ} → p ≃Q q → distQ p q ≃Q 0Q
distQ-≃0 {a / b} {c / d} eq =
  let
    x : ℤ
    x = a * ℤ +toℤ d

    y : ℤ
    y = c * ℤ +toℤ b

    z : ℤ
    z = x +ℤ negℤ y

    z≡0 : z ≡ 0ℤ
    z≡0 =
      trans
        (cong (λ t → t +ℤ negℤ y) eq)
        (+ℤ-inv-right y)

    absZ≡0 : absℤ z ≡ 0ℤ
    absZ≡0 = trans (absℤ-cong z≡0) absℤ-zero

    lhs0 : (absℤ z *ℤ +toℤ one+) ≡ 0ℤ
    lhs0 =
      trans
        (cong (λ t → t *ℤ +toℤ one+) absZ≡0)
        (trans (*ℤ-zero-left (+toℤ one+)) refl)

    rhs0 : (0ℤ *ℤ +toℤ (b *+ d)) ≡ 0ℤ
    rhs0 = *ℤ-zero-left (+toℤ (b *+ d))
  in
  trans lhs0 (sym rhs0)

```

Nonnegativity, symmetry, and negation invariance. These standard distance properties follow by threading the usual absolute-value identities through the same denominator bookkeeping.

```

-- § distQ p q ≥ 0 (nonnegativity)
distQ-nonneg : (p q : ℚ) → 0Q ≤Q distQ p q
distQ-nonneg (a / b) (c / d) =
  let
    z : ℤ
    z = (a *ℤ +toℤ d) +ℤ negℤ (c *ℤ +toℤ b)

    rhs0 : 0ℤ ≤ℤ absℤ z
    rhs0 = absℤ-nonneg z

    lhsEq : (0ℤ *ℤ +toℤ (b *+ d)) ≡ 0ℤ
    lhsEq = *ℤ-zero-left (+toℤ (b *+ d))

    rhsEq : (absℤ z *ℤ +toℤ one+) ≡ absℤ z
    rhsEq = *ℤ-one-right (absℤ z)
  in
  ≤ℤ-resp-≡l (sym lhsEq) (≤ℤ-resp-≡r (sym rhsEq) rhs0)

```

```

-- § dist $\mathbb{Q}$  p q  $\simeq_{\mathbb{Q}}$  dist $\mathbb{Q}$  q p (symmetry)
dist $\mathbb{Q}$ -sym : (p q :  $\mathbb{Q}$ )  $\rightarrow$  dist $\mathbb{Q}$  p q  $\simeq_{\mathbb{Q}}$  dist $\mathbb{Q}$  q p
dist $\mathbb{Q}$ -sym (a / b) (c / d) =
let z :  $\mathbb{Z}$ 
    z = (a * $\mathbb{Z}$  +to $\mathbb{Z}$  d) + $\mathbb{Z}$  neg $\mathbb{Z}$  (c * $\mathbb{Z}$  +to $\mathbb{Z}$  b)

    z' :  $\mathbb{Z}$ 
    z' = (c * $\mathbb{Z}$  +to $\mathbb{Z}$  b) + $\mathbb{Z}$  neg $\mathbb{Z}$  (a * $\mathbb{Z}$  +to $\mathbb{Z}$  d)

negz  $\equiv$  z' : neg $\mathbb{Z}$  z  $\equiv$  z'
negz  $\equiv$  z' =
trans
  (neg-+ $\mathbb{Z}$  (a * $\mathbb{Z}$  +to $\mathbb{Z}$  d) (neg $\mathbb{Z}$  (c * $\mathbb{Z}$  +to $\mathbb{Z}$  b)))
  (trans
    (cong ( $\lambda$  t  $\rightarrow$  neg $\mathbb{Z}$  (a * $\mathbb{Z}$  +to $\mathbb{Z}$  d) + $\mathbb{Z}$  t) (neg $\mathbb{Z}$ -involutive (c * $\mathbb{Z}$  +to $\mathbb{Z}$  b)))
    (+ $\mathbb{Z}$ -comm (neg $\mathbb{Z}$  (a * $\mathbb{Z}$  +to $\mathbb{Z}$  d)) (c * $\mathbb{Z}$  +to $\mathbb{Z}$  b)))

absEq : abs $\mathbb{Z}$  z  $\equiv$  abs $\mathbb{Z}$  z'
absEq =
trans
  (sym (abs $\mathbb{Z}$ -neg z))
  (trans
    (cong abs $\mathbb{Z}$  negz  $\equiv$  z')
    refl)

denComm : b * $^{+}$  d  $\equiv$  d * $^{+}$  b
denComm = * $^{+}$ -comm b d

denComm $\mathbb{Z}$  : +to $\mathbb{Z}$  (d * $^{+}$  b)  $\equiv$  +to $\mathbb{Z}$  (b * $^{+}$  d)
denComm $\mathbb{Z}$  = cong +to $\mathbb{Z}$  (sym denComm)

lhs : abs $\mathbb{Z}$  z * $\mathbb{Z}$  +to $\mathbb{Z}$  (d * $^{+}$  b)  $\equiv$  abs $\mathbb{Z}$  z' * $\mathbb{Z}$  +to $\mathbb{Z}$  (b * $^{+}$  d)
lhs =
trans
  (cong ( $\lambda$  t  $\rightarrow$  t * $\mathbb{Z}$  +to $\mathbb{Z}$  (d * $^{+}$  b)) absEq)
  (cong ( $\lambda$  t  $\rightarrow$  (abs $\mathbb{Z}$  z') * $\mathbb{Z}$  t) denComm $\mathbb{Z}$ )

in
lhs

-- § dist $\mathbb{Q}$  (-p) (-q)  $\simeq_{\mathbb{Q}}$  dist $\mathbb{Q}$  p q (negation invariance)
dist $\mathbb{Q}$ -neg : (p q :  $\mathbb{Q}$ )  $\rightarrow$  dist $\mathbb{Q}$  (-p) (-q)  $\simeq_{\mathbb{Q}}$  dist $\mathbb{Q}$  p q
dist $\mathbb{Q}$ -neg (a / b) (c / d) =
let
  den :  $\mathbb{N}^{+}$ 
  den = b * $^{+}$  d

  z :  $\mathbb{Z}$ 
  z = (a * $\mathbb{Z}$  +to $\mathbb{Z}$  d) + $\mathbb{Z}$  neg $\mathbb{Z}$  (c * $\mathbb{Z}$  +to $\mathbb{Z}$  b)

  zNeg :  $\mathbb{Z}$ 
  zNeg = (neg $\mathbb{Z}$  a * $\mathbb{Z}$  +to $\mathbb{Z}$  d) + $\mathbb{Z}$  neg $\mathbb{Z}$  (neg $\mathbb{Z}$  c * $\mathbb{Z}$  +to $\mathbb{Z}$  b)

  zNeg  $\equiv$  negz : zNeg  $\equiv$  neg $\mathbb{Z}$  z

```

```

zNeg≡negz =
  trans
    (cong (λ t → t +ℤ negℤ (negℤ c *ℤ +toℤ b)) (*ℤ-neg-left a (+toℤ d)))
  (trans
    (cong (λ t → negℤ (a *ℤ +toℤ d) +ℤ t)
      (cong negℤ (*ℤ-neg-left c (+toℤ b))))
    (sym (neg+ℤ (a *ℤ +toℤ d) (negℤ (c *ℤ +toℤ b)))))

absEq : absℤ zNeg ≡ absℤ z
absEq = trans (cong absℤ zNeg≡negz) (absℤ-neg z)
in
cong (λ t → t *ℤ +toℤ den) absEq

```

Triangle Inequality

The triangle inequality is the first longer metric computation. After clearing denominators by the common positive scale $s = (b \cdot d) \cdot f$, the claim reduces to the integer inequality $|x + y| \leq |x| + |y|$. The rest is a systematic rearrangement of the resulting terms until the right-hand side matches the numerator form of $d_{\mathbb{Q}}(p, q) + d_{\mathbb{Q}}(q, r)$.

```

-- § Law 14U.1: triangle inequality for distℚ
distℚ-triangle : (p q r : ℚ) → distℚ p r ≤ℚ (distℚ p q +ℚ distℚ q r)
distℚ-triangle (a / b) (c / d) (e / f) =
  goal
  where
    p q rQ : ℚ
    p = a / b
    q = c / d
    rQ = e / f

    nd-pr : ℤ
    nd-pr = numDistℚ p rQ

    nd-pq : ℤ
    nd-pq = numDistℚ p q

    nd-qr : ℤ
    nd-qr = numDistℚ q rQ

    bd df bf : ℕ+
    bd = b *+ d
    df = d *+ f
    bf = b *+ f

    rhsNum : ℤ
    rhsNum = (nd-pq *ℤ +toℤ df) +ℤ (nd-qr *ℤ +toℤ bd)

    rhsDen : ℕ+
    rhsDen = bd *+ df

-- § base scaled numerator inequality

```

```

ineq0 : (nd-pr * $\mathbb{Z}$  +to $\mathbb{Z}$  d)  $\leq \mathbb{Z}$  ((nd-pq * $\mathbb{Z}$  +to $\mathbb{Z}$  f) + $\mathbb{Z}$  (nd-qr * $\mathbb{Z}$  +to $\mathbb{Z}$  b))
ineq0 = numDistQ-triangle-scaled p q r Q

-- $ multiply by common positive scale s = (b.d).f
s :  $\mathbb{N}^+$ 
s = bd *+ f

scaled : ((nd-pr * $\mathbb{Z}$  +to $\mathbb{Z}$  d) * $\mathbb{Z}$  +to $\mathbb{Z}$  s)
          $\leq \mathbb{Z}$ 
         (((nd-pq * $\mathbb{Z}$  +to $\mathbb{Z}$  f) + $\mathbb{Z}$  (nd-qr * $\mathbb{Z}$  +to $\mathbb{Z}$  b)) * $\mathbb{Z}$  +to $\mathbb{Z}$  s)
scaled =
 $\leq \mathbb{Z}$ -mul-pos-right
(nd-pr * $\mathbb{Z}$  +to $\mathbb{Z}$  d)
((nd-pq * $\mathbb{Z}$  +to $\mathbb{Z}$  f) + $\mathbb{Z}$  (nd-qr * $\mathbb{Z}$  +to $\mathbb{Z}$  b))
s
ineq0

-- $ swap two positive scaling factors
swapScale : (x :  $\mathbb{Z}$ )  $\rightarrow$  (u v :  $\mathbb{N}^+$ )  $\rightarrow$  (x * $\mathbb{Z}$  +to $\mathbb{Z}$  u) * $\mathbb{Z}$  +to $\mathbb{Z}$  v  $\equiv$  (x * $\mathbb{Z}$  +to $\mathbb{Z}$  v) * $\mathbb{Z}$  +to $\mathbb{Z}$  u
swapScale x u v =
trans
(* $\mathbb{Z}$ -assoc x (+to $\mathbb{Z}$  u) (+to $\mathbb{Z}$  v))
(trans
(cong ( $\lambda$  t  $\rightarrow$  x * $\mathbb{Z}$  t) (* $\mathbb{Z}$ -comm (+to $\mathbb{Z}$  u) (+to $\mathbb{Z}$  v))))
(sym (* $\mathbb{Z}$ -assoc x (+to $\mathbb{Z}$  v) (+to $\mathbb{Z}$  u))))

-- $ split scaling by a product u*+v into sequential scaling
scaleSplit : (x :  $\mathbb{Z}$ )  $\rightarrow$  (u v :  $\mathbb{N}^+$ )  $\rightarrow$  x * $\mathbb{Z}$  +to $\mathbb{Z}$  (u *+ v)  $\equiv$  (x * $\mathbb{Z}$  +to $\mathbb{Z}$  u) * $\mathbb{Z}$  +to $\mathbb{Z}$  v
scaleSplit x u v =
trans
(cong ( $\lambda$  t  $\rightarrow$  x * $\mathbb{Z}$  t) (+to $\mathbb{Z}$ -*+ u v))
(sym (* $\mathbb{Z}$ -assoc x (+to $\mathbb{Z}$  u) (+to $\mathbb{Z}$  v)))

-- $ LHS rewrite: ((nd-pr.d).s) = nd-pr . rhsDen
lhsEq : ((nd-pr * $\mathbb{Z}$  +to $\mathbb{Z}$  d) * $\mathbb{Z}$  +to $\mathbb{Z}$  s)  $\equiv$  (nd-pr * $\mathbb{Z}$  +to $\mathbb{Z}$  rhsDen)
lhsEq =
trans
(scaleSplit (nd-pr * $\mathbb{Z}$  +to $\mathbb{Z}$  d) bd f)
(trans
(cong ( $\lambda$  t  $\rightarrow$  t * $\mathbb{Z}$  +to $\mathbb{Z}$  f) (swapScale nd-pr d bd)))
(trans
(sym (scaleSplit (nd-pr * $\mathbb{Z}$  +to $\mathbb{Z}$  bd) d f)))
(sym (scaleSplit nd-pr bd df))))

-- $ term-pq rewrite: (nd-pq.f).s = (nd-pq.df).bf
term-pq : (nd-pq * $\mathbb{Z}$  +to $\mathbb{Z}$  f) * $\mathbb{Z}$  +to $\mathbb{Z}$  s  $\equiv$  (nd-pq * $\mathbb{Z}$  +to $\mathbb{Z}$  df) * $\mathbb{Z}$  +to $\mathbb{Z}$  bf
term-pq =
trans
(scaleSplit (nd-pq * $\mathbb{Z}$  +to $\mathbb{Z}$  f) bd f)
(trans
(cong ( $\lambda$  t  $\rightarrow$  t * $\mathbb{Z}$  +to $\mathbb{Z}$  f) (swapScale nd-pq f bd)))
(trans
(cong ( $\lambda$  t  $\rightarrow$  (t * $\mathbb{Z}$  +to $\mathbb{Z}$  f) * $\mathbb{Z}$  +to $\mathbb{Z}$  f)
(trans

```

```

      (scaleSplit nd-pq b d)
      (swapScale nd-pq b d)))
(trans
  (cong (λ t → t *ℤ +toℤ f)
    (swapScale (nd-pq *ℤ +toℤ d) b f))
  (trans
    (cong (λ t → (t *ℤ +toℤ b) *ℤ +toℤ f) (sym (scaleSplit nd-pq d f)))
    (sym (scaleSplit (nd-pq *ℤ +toℤ df) b f))))))

-- § term-qr rewrite: (nd-qr·b)·s = (nd-qr·bd)·bf
term-qr : (nd-qr *ℤ +toℤ b) *ℤ +toℤ s ≡ (nd-qr *ℤ +toℤ bd) *ℤ +toℤ bf
term-qr =
  trans
    (scaleSplit (nd-qr *ℤ +toℤ b) bd f)
  (trans
    (cong (λ t → t *ℤ +toℤ f) (swapScale nd-qr b bd))
    (sym (scaleSplit (nd-qr *ℤ +toℤ bd) b f))))

rhsEq : (((nd-pq *ℤ +toℤ f) +ℤ (nd-qr *ℤ +toℤ b)) *ℤ +toℤ s) ≡ (rhsNum *ℤ +toℤ bf)
rhsEq =
  trans
    (*ℤ-distib-left-+ℤ (nd-pq *ℤ +toℤ f) (nd-qr *ℤ +toℤ b) (+toℤ s))
  (trans
    (trans
      (cong (λ t → t +ℤ ((nd-qr *ℤ +toℤ b) *ℤ +toℤ s)) term-pq)
      (cong (λ t → ((nd-pq *ℤ +toℤ df) *ℤ +toℤ bf) +ℤ t) term-qr))
    (sym (*ℤ-distib-left-+ℤ (nd-pq *ℤ +toℤ df) (nd-qr *ℤ +toℤ bd) (+toℤ bf)))))

goal : distℚ p rQ ≤ℚ (distℚ p q +ℚ distℚ q rQ)
goal =
  ≤ℤ-resp≡l lhsEq
  (≤ℤ-resp≡r rhsEq scaled)

```

Translation Invariance

Adding the same rational to both endpoints cannot change the distance. In the scaled numerator, the new r -terms appear with opposite signs and cancel immediately. What remains is the original numerator multiplied by the square f^2 of the added denominator, and this extra factor disappears inside rational equivalence.

```

-- § distℚ (p+r) (q+r) ≃ℚ distℚ p q (right translation)
distℚ-+ℚ-right : (p q r : ℚ) → distℚ (p +ℚ r) (q +ℚ r) ≃ℚ distℚ p q
distℚ-+ℚ-right (a / b) (c / d) (e / f) =
  let
    swapScale : (x : ℤ) → (u v : ℕ+) → (x *ℤ +toℤ u) *ℤ +toℤ v ≡ (x *ℤ +toℤ v) *ℤ +toℤ u
    swapScale x u v =
      trans
        (*ℤ-assoc x (+toℤ u) (+toℤ v))
      (trans
        (cong (λ t → x *ℤ t) (*ℤ-comm (+toℤ u) (+toℤ v))))

```

```

(sym (*ℤ-assoc x (+toℤ v) (+toℤ u)))

scaleSplit : (x : ℤ) → (u v : ℕ+) → x *ℤ +toℤ (u *+ v) ≡ (x *ℤ +toℤ u) *ℤ +toℤ v
scaleSplit x u v =
  trans
    (cong (λ t → x *ℤ t) (+toℤ-+ u v))
    (sym (*ℤ-assoc x (+toℤ u) (+toℤ v)))

mul4-rearrange : (x y z w : ℤ) → (x *ℤ y) *ℤ (z *ℤ w) ≡ (x *ℤ z) *ℤ (y *ℤ w)
mul4-rearrange x y z w =
  trans
    (*ℤ-assoc x y (z *ℤ w))
    (trans
      (cong (λ t → x *ℤ t)
        (trans
          (sym (*ℤ-assoc y z w))
          (trans
            (cong (λ t → t *ℤ w) (*ℤ-comm y z))
            (*ℤ-assoc z y w))))
      (sym (*ℤ-assoc x z (y *ℤ w))))

bf : ℕ+
bf = b *+ f

df : ℕ+
df = d *+ f

s : ℕ+
s = f *+ f

base : ℤ
base = (a *ℤ +toℤ d) +ℤ negℤ (c *ℤ +toℤ b)

Pn : ℤ
Pn = (a *ℤ +toℤ f) +ℤ (e *ℤ +toℤ b)

Qn : ℤ
Qn = (c *ℤ +toℤ f) +ℤ (e *ℤ +toℤ d)

Z : ℤ
Z = (Pn *ℤ +toℤ df) +ℤ negℤ (Qn *ℤ +toℤ bf)

-- § denominator embedding factorization
denFactor : +toℤ (bf *+ df) ≡ (+toℤ (b *+ d)) *ℤ (+toℤ s)
denFactor =
  trans
    (+toℤ-+ bf df)
    (trans
      (cong (λ t → t *ℤ (+toℤ df)) (+toℤ-+ b f))
      (trans
        (cong (λ t → ((+toℤ b) *ℤ (+toℤ f)) *ℤ t) (+toℤ-+ d f))
        (trans
          (mul4-rearrange (+toℤ b) (+toℤ f) (+toℤ d) (+toℤ f))
          (trans
            (cong (λ t → t *ℤ ((+toℤ f) *ℤ (+toℤ f))) (sym (+toℤ-+ b d)))

```

```

(cong (λ t → (+toℤ (b *+ d)) *ℤ t) (sym (+toℤ- *+ f f))))))

-- $ numerator cancellation and factoring
cancelR : Z ≡ base *ℤ +toℤ s
cancelR =
let
  afdℓ : ℤ
  afdℓ = (a *ℤ +toℤ f) *ℤ +toℤ df

  ebdℓ : ℤ
  ebdℓ = (e *ℤ +toℤ b) *ℤ +toℤ df

  cfbℓ : ℤ
  cfbℓ = (c *ℤ +toℤ f) *ℤ +toℤ bf

  edbℓ : ℤ
  edbℓ = (e *ℤ +toℤ d) *ℤ +toℤ bf

  expandP : (Pn *ℤ +toℤ df) ≡ ((a *ℤ +toℤ f) *ℤ +toℤ df) +ℤ ((e *ℤ +toℤ b) *ℤ +toℤ df)
  expandP = *ℤ-distrib-left+ℤ (a *ℤ +toℤ f) (e *ℤ +toℤ b) (+toℤ df)

  expandQ : (Qn *ℤ +toℤ bf) ≡ ((c *ℤ +toℤ f) *ℤ +toℤ bf) +ℤ ((e *ℤ +toℤ d) *ℤ +toℤ bf)
  expandQ = *ℤ-distrib-left+ℤ (c *ℤ +toℤ f) (e *ℤ +toℤ d) (+toℤ bf)

  negExpandQ : negℤ (Qn *ℤ +toℤ bf) ≡ negℤ ((c *ℤ +toℤ f) *ℤ +toℤ bf) +ℤ negℤ ((e *ℤ +toℤ d) *ℤ +toℤ bf)
  negExpandQ = trans (cong negℤ expandQ) (neg+ℤ ((c *ℤ +toℤ f) *ℤ +toℤ bf) ((e *ℤ +toℤ d) *ℤ +toℤ bf))

  Z₁ : Z ≡ (((a *ℤ +toℤ f) *ℤ +toℤ df) +ℤ ((e *ℤ +toℤ b) *ℤ +toℤ df))
    +ℤ (negℤ ((c *ℤ +toℤ f) *ℤ +toℤ bf) +ℤ negℤ ((e *ℤ +toℤ d) *ℤ +toℤ bf))
  Z₁ =
  trans
  (cong (λ t → t +ℤ negℤ (Qn *ℤ +toℤ bf)) expandP)
  (cong (λ t → (((a *ℤ +toℤ f) *ℤ +toℤ df) +ℤ ((e *ℤ +toℤ b) *ℤ +toℤ df)) +ℤ t) negExpandQ)

  ebdℓ≡edbℓ : ((e *ℤ +toℤ b) *ℤ +toℤ df) ≡ ((e *ℤ +toℤ d) *ℤ +toℤ bf)
  ebdℓ≡edbℓ =
  trans
  (cong (λ t → (e *ℤ +toℤ b) *ℤ t) (+toℤ- *+ d f))
  (trans
  (mul4-rearrange e (+toℤ b) (+toℤ d) (+toℤ f))
  (sym (cong (λ t → (e *ℤ +toℤ d) *ℤ t) (+toℤ- *+ b f))))

  cancelTerm : ((e *ℤ +toℤ b) *ℤ +toℤ df) +ℤ negℤ ((e *ℤ +toℤ d) *ℤ +toℤ bf) ≡ 0ℤ
  cancelTerm = trans (cong (λ t → t +ℤ negℤ ((e *ℤ +toℤ d) *ℤ +toℤ bf)) ebdℓ≡edbℓ) (+ℤ-inv-right ((e *ℤ +toℤ d) *ℤ +toℤ bf))

  afdℓ≡ads : ((a *ℤ +toℤ f) *ℤ +toℤ df) ≡ ((a *ℤ +toℤ d) *ℤ +toℤ s)
  afdℓ≡ads =
  trans
  (cong (λ t → (a *ℤ +toℤ f) *ℤ t) (+toℤ- *+ d f))
  (trans
  (mul4-rearrange a (+toℤ f) (+toℤ d) (+toℤ f))
  (cong (λ t → (a *ℤ +toℤ d) *ℤ t) (sym (+toℤ- *+ f f))))

  cfbℓ≡cbs : ((c *ℤ +toℤ f) *ℤ +toℤ bf) ≡ ((c *ℤ +toℤ b) *ℤ +toℤ s)
  cfbℓ≡cbs =

```

```

trans
  (cong (λ t → (c *ℤ +toℤ f) *ℤ t) (+toℤ-+ b f))
  (trans
    (mul4-rearrange c (+toℤ f) (+toℤ b) (+toℤ f))
    (cong (λ t → (c *ℤ +toℤ b) *ℤ t) (sym (+toℤ-+ f f))))

-- § cancel the r-contributed terms
Z2 : Z ≡ afd f +ℤ negℤ cfb f
Z2 =
  let
    Zexp : Z ≡ (afdf +ℤ ebd f) +ℤ (negℤ cfb f +ℤ negℤ edbf)
    Zexp =
      trans
        (cong (λ t → t +ℤ negℤ (Qn *ℤ +toℤ b f)) expandP)
        (trans
          (cong (λ t → ((afdf +ℤ ebd f) +ℤ t))
            (trans (cong negℤ expandQ) (neg-+ℤ cfb f edbf))))
          refl)

    swapNeg : (negℤ cfb f +ℤ negℤ edbf) ≡ (negℤ edbf +ℤ negℤ cfb f)
    swapNeg = +ℤ-comm (negℤ cfb f) (negℤ edbf)

    cancelPair : ebd f +ℤ negℤ edbf ≡ 0ℤ
    cancelPair =
      trans
        (cong (λ t → t +ℤ negℤ edbf) ebd f ≡ edbf)
        (+ℤ-inv-right edbf)

  in
    trans
      (trans
        Zexp
        (trans
          (cong (λ t → (afdf +ℤ ebd f) +ℤ t) swapNeg)
          (trans
            (+ℤ-assoc afd f ebd f (negℤ edbf +ℤ negℤ cfb f))
            (cong (λ t → afd f +ℤ t) (sym (+ℤ-assoc ebd f (negℤ edbf) (negℤ cfb f)))))))
          (trans
            (cong (λ t → afd f +ℤ (t +ℤ negℤ cfb f)) cancelPair)
            (cong (λ t → afd f +ℤ t) (+ℤ-zero-left (negℤ cfb f))))))

-- § factor out the common scale s = f·f
factor : (afdf +ℤ negℤ cfb f) ≡ base *ℤ +toℤ s
factor =
  trans
    (cong (λ t → t +ℤ negℤ cfb f) afd f ≡ ads)
    (trans
      (cong (λ t → ((a *ℤ +toℤ d) *ℤ +toℤ s) +ℤ negℤ t) cfb f ≡ cbs)
      (trans
        (cong (λ t → ((a *ℤ +toℤ d) *ℤ +toℤ s) +ℤ t)
          (sym (*ℤ-neg-left (c *ℤ +toℤ b) (+toℤ s))))
          (sym (*ℤ-distrib-left+ℤ (a *ℤ +toℤ d) (negℤ (c *ℤ +toℤ b)) (+toℤ s))))))
    in
      trans Z2 factor

```



```

absZEq : absZ Z ≡ absZ base *Z +toZ s
absZEq = trans (cong absZ cancelR) (absZ-mul-pos-right base s)

rhsDen : N+
rhsDen = b *+ d

lhsDen : N+
lhsDen = bf *+ df

rhsNum : Z
rhsNum = absZ base

rhsRewrite : (rhsNum *Z +toZ lhsDen) ≡ (rhsNum *Z +toZ rhsDen) *Z +toZ s
rhsRewrite =
  trans
    (cong (λ t → rhsNum *Z t) denFactor)
    (sym (*Z-assoc rhsNum (+toZ rhsDen) (+toZ s)))

cross : (absZ Z *Z +toZ rhsDen) ≡ (rhsNum *Z +toZ lhsDen)
cross =
  trans
    (cong (λ t → t *Z +toZ rhsDen) absZEq)
    (trans
      (swapScale rhsNum s rhsDen)
      (sym rhsRewrite))
in
cross

```

Left translation. The left-translation variant follows by the same cancellation pattern: after expansion, the common r -terms disappear and the remaining residue factors through the square $(f \cdot f)$.

```

-- § distQ (r+p) (r+q) ≃Q distQ p q (left translation)
distQ-+Q-left : (r p q : Q) → distQ (r +Q p) (r +Q q) ≃Q distQ p q
distQ-+Q-left (e / f) (a / b) (c / d) =
let
  swapScale : (x : Z) → (u v : N+) → (x *Z +toZ u) *Z +toZ v ≡ (x *Z +toZ v) *Z +toZ u
  swapScale x u v =
    trans
      (*Z-assoc x (+toZ u) (+toZ v))
      (trans
        (cong (λ t → x *Z t) (*Z-comm (+toZ u) (+toZ v)))
        (sym (*Z-assoc x (+toZ v) (+toZ u))))

  mul4-rearrange : (x y z w : Z) → (x *Z y) *Z (z *Z w) ≡ (x *Z z) *Z (y *Z w)
  mul4-rearrange x y z w =
    trans
      (*Z-assoc x y (z *Z w))
      (trans
        (cong (λ t → x *Z t)
          (trans

```

```

(sym (*ℤ-assoc y z w))
(trans
  (cong (λ t → t *ℤ w) (*ℤ-comm y z))
  (*ℤ-assoc z y w)))
(sym (*ℤ-assoc x z (y *ℤ w))))

fb : ℕ+
fb = f *+ b

fd : ℕ+
fd = f *+ d

s : ℕ+
s = f *+ f

base : ℤ
base = (a *ℤ +toℤ d) +ℤ negℤ (c *ℤ +toℤ b)

Pn : ℤ
Pn = (e *ℤ +toℤ b) +ℤ (a *ℤ +toℤ f)

Qn : ℤ
Qn = (e *ℤ +toℤ d) +ℤ (c *ℤ +toℤ f)

Z : ℤ
Z = (Pn *ℤ +toℤ fd) +ℤ negℤ (Qn *ℤ +toℤ fb)

denFactor : +toℤ (fb *+ fd) ≡ (+toℤ (b *+ d)) *ℤ (+toℤ s)
denFactor =
  trans
    (+toℤ-*+ fb fd)
  (trans
    (cong (λ t → t *ℤ (+toℤ fd)) (+toℤ-*+ f b))
    (trans
      (cong (λ t → ((+toℤ f) *ℤ (+toℤ b)) *ℤ t) (+toℤ-*+ f d))
      (trans
        (mul4-rearrange (+toℤ f) (+toℤ b) (+toℤ f) (+toℤ d))
        (trans
          (*ℤ-comm ((+toℤ f) *ℤ (+toℤ f)) ((+toℤ b) *ℤ (+toℤ d)))
          (trans
            (cong (λ t → t *ℤ ((+toℤ f) *ℤ (+toℤ f))) (sym (+toℤ-*+ b d)))
            (cong (λ t → (+toℤ (b *+ d)) *ℤ t) (sym (+toℤ-*+ f f))))))))))

cancelR : Z ≡ base *ℤ +toℤ s
cancelR =
  let
    ebfd : ℤ
    ebfd = (e *ℤ +toℤ b) *ℤ +toℤ fd

    affd : ℤ
    affd = (a *ℤ +toℤ f) *ℤ +toℤ fd

    edfb : ℤ
    edfb = (e *ℤ +toℤ d) *ℤ +toℤ fb

```

```

cffb : ℤ
cffb = (c *ℤ +toℤ f) *ℤ +toℤ fb

expandP : (Pn *ℤ +toℤ fd) ≡ ebfd +ℤ affd
expandP = *ℤ-distrib-left-+ℤ (e *ℤ +toℤ b) (a *ℤ +toℤ f) (+toℤ fd)

expandQ : (Qn *ℤ +toℤ fb) ≡ edfb +ℤ cffb
expandQ = *ℤ-distrib-left-+ℤ (e *ℤ +toℤ d) (c *ℤ +toℤ f) (+toℤ fb)

Zexp : Z ≡ (ebfd +ℤ affd) +ℤ (negℤ edfb +ℤ negℤ cffb)
Zexp =
  trans
  (cong (λ t → t +ℤ negℤ (Qn *ℤ +toℤ fb)) expandP)
  (trans
    (cong (λ t → (ebfd +ℤ affd) +ℤ t) (trans (cong negℤ expandQ) (neg-+ℤ edfb cffb)))
    refl)

ebfd≡edfb : ebfd ≡ edfb
ebfd≡edfb =
  trans
  (cong (λ t → (e *ℤ +toℤ b) *ℤ t) (+toℤ-*+ f d))
  (trans
    (cong (λ t → (e *ℤ +toℤ b) *ℤ t) (*ℤ-comm (+toℤ f) (+toℤ d)))
    (trans
      (mul4-rearrange e (+toℤ b) (+toℤ d) (+toℤ f))
      (trans
        (cong (λ t → (e *ℤ +toℤ d) *ℤ t) (*ℤ-comm (+toℤ b) (+toℤ f)))
        (cong (λ t → (e *ℤ +toℤ d) *ℤ t) (sym (+toℤ-*+ f b)))))))

cancelPair : ebfd +ℤ negℤ edfb ≡ 0ℤ
cancelPair =
  trans
  (cong (λ t → t +ℤ negℤ edfb) ebfd≡edfb)
  (+ℤ-inv-right edfb)

affd≡ads : affd ≡ (a *ℤ +toℤ d) *ℤ +toℤ s
affd≡ads =
  trans
  (cong (λ t → (a *ℤ +toℤ f) *ℤ t) (+toℤ-*+ f d))
  (trans
    (cong (λ t → (a *ℤ +toℤ f) *ℤ t) (*ℤ-comm (+toℤ f) (+toℤ d)))
    (trans
      (mul4-rearrange a (+toℤ f) (+toℤ d) (+toℤ f))
      (cong (λ t → (a *ℤ +toℤ d) *ℤ t) (sym (+toℤ-*+ f f))))))

cffb≡cbs : cffb ≡ (c *ℤ +toℤ b) *ℤ +toℤ s
cffb≡cbs =
  trans
  (cong (λ t → (c *ℤ +toℤ f) *ℤ t) (+toℤ-*+ f b))
  (trans
    (cong (λ t → (c *ℤ +toℤ f) *ℤ t) (*ℤ-comm (+toℤ f) (+toℤ b)))
    (trans
      (mul4-rearrange c (+toℤ f) (+toℤ b) (+toℤ f))
      (cong (λ t → (c *ℤ +toℤ b) *ℤ t) (sym (+toℤ-*+ f f))))))

```

```

step1 : Z ≡ affd +ℤ negℤ cffb
step1 =
  trans
    Zexp
    (trans
      (cong (λ t → t +ℤ (negℤ edfb +ℤ negℤ cffb)) (+ℤ-comm ebfd affd))
      (trans
        (+ℤ-assoc affd ebfd (negℤ edfb +ℤ negℤ cffb))
        (trans
          (cong (λ t → affd +ℤ t) (sym (+ℤ-assoc ebfd (negℤ edfb) (negℤ cffb))))
          (trans
            (cong (λ t → affd +ℤ (t +ℤ negℤ cffb)) cancelPair)
            (cong (λ t → affd +ℤ t) (+ℤ-zero-left (negℤ cffb)))))))

step2 : (affd +ℤ negℤ cffb) ≡ ((a *ℤ +toℤ d) *ℤ +toℤ s) +ℤ negℤ ((c *ℤ +toℤ b) *ℤ +toℤ s)
step2 =
  trans
    (cong (λ t → t +ℤ negℤ cffb) affd≡ads)
    (cong (λ t → ((a *ℤ +toℤ d) *ℤ +toℤ s) +ℤ negℤ t) cffb≡cbs)

factor : ((a *ℤ +toℤ d) *ℤ +toℤ s) +ℤ negℤ ((c *ℤ +toℤ b) *ℤ +toℤ s)
      ≡
      base *ℤ +toℤ s
factor =
  trans
    (cong (λ t → ((a *ℤ +toℤ d) *ℤ +toℤ s) +ℤ t)
      (sym (*ℤ-neg-left (c *ℤ +toℤ b) (+toℤ s))))
    (sym (*ℤ-distrib-left-+ℤ (a *ℤ +toℤ d) (negℤ (c *ℤ +toℤ b)) (+toℤ s)))
  in
  trans step1 (trans step2 factor)

absZEq : absℤ Z ≡ absℤ base *ℤ +toℤ s
absZEq = trans (cong absℤ cancelR) (absℤ-mul-pos-right base s)

rhsDen : ℕ+
rhsDen = b *+ d

lhsDen : ℕ+
lhsDen = fb *+ fd

rhsNum : ℤ
rhsNum = absℤ base

rhsRewrite : (rhsNum *ℤ +toℤ lhsDen) ≡ (rhsNum *ℤ +toℤ rhsDen) *ℤ +toℤ s
rhsRewrite =
  trans
    (cong (λ t → rhsNum *ℤ t) denFactor)
    (sym (*ℤ-assoc rhsNum (+toℤ rhsDen) (+toℤ s)))

cross : (absℤ Z *ℤ +toℤ rhsDen) ≡ (rhsNum *ℤ +toℤ lhsDen)
cross =
  trans
    (cong (λ t → t *ℤ +toℤ rhsDen) absZEq)

```

```

(trans
  (swapScale rhsNum s rhsDen)
  (sym rhsRewrite))
in
cross

```

Multiplicative Scaling

Multiplying both endpoints by the same rational p scales the distance by $d(p, 0)$. After clearing denominators, the relevant numerator factors as a common term coming from p times the base numerator measuring the gap between q and r . The absolute value then splits by the multiplicativity of $|\cdot|$ over $\mathbb{Z}$, and the rational setoid repackages the result as the expected product distance.

```

-- § distQ (p*q) (p*r) ≃Q distQ q r * distQ p 0 (left scaling)
distQ-*Q-left : (p q r : Q) → distQ (p *Q q) (p *Q r) ≃Q (distQ q r *Q distQ p 0Q)
distQ-*Q-left (a / b) (c / d) (e / f) =
let
  swapScale : (x : Z) → (u v : N+) → (x *Z +toZ u) *Z +toZ v ≡ (x *Z +toZ v) *Z +toZ u
  swapScale x u v =
    trans
      (*Z-assoc x (+toZ u) (+toZ v))
    (trans
      (cong (λ t → x *Z t) (*Z-comm (+toZ u) (+toZ v)))
      (sym (*Z-assoc x (+toZ v) (+toZ u))))

  scaleSplit : (x : Z) → (u v : N+) → x *Z +toZ (u *+ v) ≡ (x *Z +toZ u) *Z +toZ v
  scaleSplit x u v =
    trans
      (cong (λ t → x *Z t) (+toZ-*+ u v))
      (sym (*Z-assoc x (+toZ u) (+toZ v)))

  mul4-rearrange : (x y z w : Z) → (x *Z y) *Z (z *Z w) ≡ (x *Z z) *Z (y *Z w)
  mul4-rearrange x y z w =
    trans
      (*Z-assoc x y (z *Z w))
    (trans
      (cong (λ t → x *Z t)
        (trans
          (sym (*Z-assoc y z w))
          (trans
            (cong (λ t → t *Z w) (*Z-comm y z))
            (*Z-assoc z y w))))
      (sym (*Z-assoc x z (y *Z w))))

-- § key cleared numerators
cf : Z
cf = c *Z +toZ f

ed : Z

```

```

ed = e * $\mathbb{Z}$  +to $\mathbb{Z}$  d

baseQR :  $\mathbb{Z}$ 
baseQR = cf + $\mathbb{Z}$  neg $\mathbb{Z}$  ed

ab :  $\mathbb{Z}$ 
ab = a * $\mathbb{Z}$  +to $\mathbb{Z}$  b

-- § dist $\mathbb{Q}$  p 0 $\mathbb{Q}$  numerator collapses to abs $\mathbb{Z}$  a
p0Raw :  $\mathbb{Z}$ 
p0Raw = (a * $\mathbb{Z}$  +to $\mathbb{Z}$  one+) + $\mathbb{Z}$  neg $\mathbb{Z}$  (0 $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  b)

p0Raw $\equiv$ a : p0Raw  $\equiv$  a
p0Raw $\equiv$ a =
  trans
    (cong ( $\lambda$  t  $\rightarrow$  t + $\mathbb{Z}$  neg $\mathbb{Z}$  (0 $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  b)) (* $\mathbb{Z}$ -one-right a))
  (trans
    (cong ( $\lambda$  t  $\rightarrow$  a + $\mathbb{Z}$  neg $\mathbb{Z}$  t) (* $\mathbb{Z}$ -zero-left (+to $\mathbb{Z}$  b)))
  (trans
    (cong ( $\lambda$  t  $\rightarrow$  a + $\mathbb{Z}$  t) (neg $\mathbb{Z}$ -zero))
    (+ $\mathbb{Z}$ -zero-right a)))

absP0 :  $\mathbb{Z}$ 
absP0 = abs $\mathbb{Z}$  p0Raw

absP0 $\equiv$ absA : absP0  $\equiv$  abs $\mathbb{Z}$  a
absP0 $\equiv$ absA = trans (abs $\mathbb{Z}$ -cong p0Raw $\equiv$ a) refl

-- § LHS cleared numerator for dist $\mathbb{Q}$  (p*q) (p*r)
bf :  $\mathbb{N}^+$ 
bf = b *+ f

bd :  $\mathbb{N}^+$ 
bd = b *+ d

Z :  $\mathbb{Z}$ 
Z = ((a * $\mathbb{Z}$  c) * $\mathbb{Z}$  +to $\mathbb{Z}$  bf) + $\mathbb{Z}$  neg $\mathbb{Z}$  ((a * $\mathbb{Z}$  e) * $\mathbb{Z}$  +to $\mathbb{Z}$  bd)

term1 : ((a * $\mathbb{Z}$  c) * $\mathbb{Z}$  +to $\mathbb{Z}$  bf)  $\equiv$  ab * $\mathbb{Z}$  cf
term1 =
  trans
    (cong ( $\lambda$  t  $\rightarrow$  (a * $\mathbb{Z}$  c) * $\mathbb{Z}$  t) (+to $\mathbb{Z}$ -*+ b f))
  (trans
    (mul4-rearrange a c (+to $\mathbb{Z}$  b) (+to $\mathbb{Z}$  f))
    refl)

term2 : ((a * $\mathbb{Z}$  e) * $\mathbb{Z}$  +to $\mathbb{Z}$  bd)  $\equiv$  ab * $\mathbb{Z}$  ed
term2 =
  trans
    (cong ( $\lambda$  t  $\rightarrow$  (a * $\mathbb{Z}$  e) * $\mathbb{Z}$  t) (+to $\mathbb{Z}$ -*+ b d))
  (trans
    (mul4-rearrange a e (+to $\mathbb{Z}$  b) (+to $\mathbb{Z}$  d))
    refl)

factorZ : Z  $\equiv$  ab * $\mathbb{Z}$  baseQR

```

```

factorZ =
  let
    Z1 : Z ≡ (ab *ℤ cf) +ℤ negℤ (ab *ℤ ed)
    Z1 =
      trans
        (cong (λ t → t +ℤ negℤ ((a *ℤ e) *ℤ +toℤ bd)) term1)
        (cong (λ t → (ab *ℤ cf) +ℤ negℤ t) term2)

    negPull : negℤ (ab *ℤ ed) ≡ ab *ℤ negℤ ed
    negPull = sym (*ℤ-neg-right ab ed)

    Z2 : (ab *ℤ cf) +ℤ negℤ (ab *ℤ ed) ≡ (ab *ℤ cf) +ℤ (ab *ℤ negℤ ed)
    Z2 = cong (λ t → (ab *ℤ cf) +ℤ t) negPull

    Z3 : (ab *ℤ cf) +ℤ (ab *ℤ negℤ ed) ≡ ab *ℤ (cf +ℤ negℤ ed)
    Z3 = sym (*ℤ-distrib-right+ℤ ab cf (negℤ ed))
  in
    trans Z1 (trans Z2 Z3)

absZ : ℤ
absZ = absℤ Z

absZ≡scaled : absZ ≡ (absℤ baseQR *ℤ absℤ a) *ℤ +toℤ b
absZ≡scaled =
  let
    absZ1 : absZ ≡ absℤ (ab *ℤ baseQR)
    absZ1 = cong absℤ factorZ

    absZ2 : absℤ (ab *ℤ baseQR) ≡ (absℤ ab *ℤ absℤ baseQR)
    absZ2 = absℤ-mul ab baseQR

    absAB : absℤ ab ≡ absℤ a *ℤ +toℤ b
    absAB = absℤ-mul-pos-right a b

    absZ3 : (absℤ ab *ℤ absℤ baseQR) ≡ ((absℤ a *ℤ +toℤ b) *ℤ absℤ baseQR)
    absZ3 = cong (λ t → t *ℤ absℤ baseQR) absAB

    absZ4 : ((absℤ a *ℤ +toℤ b) *ℤ absℤ baseQR) ≡ ((absℤ baseQR *ℤ absℤ a) *ℤ +toℤ b)
    absZ4 =
      trans
        (*ℤ-assoc (absℤ a) (+toℤ b) (absℤ baseQR))
      (trans
        (cong (λ t → (absℤ a) *ℤ t) (*ℤ-comm (+toℤ b) (absℤ baseQR))))
      (trans
        (sym (*ℤ-assoc (absℤ a) (absℤ baseQR) (+toℤ b))))
      (trans
        (cong (λ t → t *ℤ (+toℤ b)) (*ℤ-comm (absℤ a) (absℤ baseQR))))
      (refl)))
  in
    trans absZ1 (trans absZ2 (trans absZ3 absZ4))

lhsDen : ℕ+
lhsDen = (b *+ d) *+ (b *+ f)

rhsDen : ℕ+

```

```

rhsDen = (d *+ f) *+ (b *+ one+)

rhsNum : ℤ
rhsNum = (absℤ baseQR *ℤ absP0)

rhsNum≡ : rhsNum ≡ (absℤ baseQR *ℤ absℤ a)
rhsNum≡ = cong (λ t → (absℤ baseQR *ℤ t)) absP0≡absA

-- § denominator embedding relation
denRel : (+toℤ rhsDen) *ℤ (+toℤ b) ≡ +toℤ lhsDen
denRel =
  let
    lhs0 : +toℤ lhsDen ≡ (+toℤ (b *+ d)) *ℤ (+toℤ (b *+ f))
    lhs0 = +toℤ- *+ (b *+ d) (b *+ f)

    rhs0 : +toℤ rhsDen ≡ (+toℤ (d *+ f)) *ℤ (+toℤ (b *+ one+))
    rhs0 = +toℤ- *+ (d *+ f) (b *+ one+)

    bdf : +toℤ (b *+ d) ≡ (+toℤ b) *ℤ (+toℤ d)
    bdf = +toℤ- *+ b d

    bff : +toℤ (b *+ f) ≡ (+toℤ b) *ℤ (+toℤ f)
    bff = +toℤ- *+ b f

    dff : +toℤ (d *+ f) ≡ (+toℤ d) *ℤ (+toℤ f)
    dff = +toℤ- *+ d f

    bone : +toℤ (b *+ one+) ≡ (+toℤ b) *ℤ (+toℤ one+)
    bone = +toℤ- *+ b one+

  stepR : (+toℤ rhsDen) *ℤ (+toℤ b) ≡ ((+toℤ d) *ℤ (+toℤ f)) *ℤ (((+toℤ b) *ℤ (+toℤ one+)) *ℤ (+toℤ b))
  stepR =
    trans
      (cong (λ t → t *ℤ (+toℤ b)) rhs0)
      (trans
        (cong (λ t → ((+toℤ (d *+ f)) *ℤ t) *ℤ (+toℤ b)) bone)
        (trans
          (cong (λ t → (t *ℤ ((+toℤ b) *ℤ (+toℤ one+))) *ℤ (+toℤ b)) dff)
          (trans
            (*ℤ-assoc ((+toℤ d) *ℤ (+toℤ f)) ((+toℤ b) *ℤ (+toℤ one+)) (+toℤ b))
            refl))))

  stepL : +toℤ lhsDen ≡ ((+toℤ b) *ℤ (+toℤ b)) *ℤ ((+toℤ d) *ℤ (+toℤ f))
  stepL =
    trans
      lhs0
      (trans
        (cong (λ t → t *ℤ (+toℤ (b *+ f))) bdf)
        (trans
          (cong (λ t → ((+toℤ b) *ℤ (+toℤ d)) *ℤ t) bff)
          (trans
            (mul4-rearrange (+toℤ b) (+toℤ d) (+toℤ b) (+toℤ f))
            refl))))

in
let

```



```

b1≡b : (+toℤ b) *ℤ (+toℤ one+) ≡ (+toℤ b)
b1≡b = *ℤ-one-right (+toℤ b)

inner : ((+toℤ b) *ℤ (+toℤ one+)) *ℤ (+toℤ b) ≡ (+toℤ b) *ℤ (+toℤ b)
inner = cong (λ t → t *ℤ (+toℤ b)) b1≡b
in
trans
stepR
(trans
(cong (λ t → ((+toℤ d) *ℤ (+toℤ f)) *ℤ t) inner)
(trans
(*ℤ-comm ((+toℤ d) *ℤ (+toℤ f)) ((+toℤ b) *ℤ (+toℤ b)))
(sym stepL))))

cross : (absZ *ℤ +toℤ rhsDen) ≡ (rhsNum *ℤ +toℤ lhsDen)
cross =
let
lhs1 : absZ *ℤ +toℤ rhsDen ≡ ((absZ baseQR *ℤ absZ a) *ℤ +toℤ b) *ℤ +toℤ rhsDen
lhs1 = cong (λ t → t *ℤ +toℤ rhsDen) absZ≡scaled

lhs2 : ((absZ baseQR *ℤ absZ a) *ℤ +toℤ b) *ℤ +toℤ rhsDen ≡ ((absZ baseQR *ℤ absZ a) *ℤ +toℤ rhsDen) *ℤ +toℤ b
lhs2 = swapScale (absZ baseQR *ℤ absZ a) b rhsDen

lhs3 : ((absZ baseQR *ℤ absZ a) *ℤ +toℤ rhsDen) *ℤ +toℤ b ≡ (absZ baseQR *ℤ absZ a) *ℤ ((+toℤ rhsDen) *ℤ (+toℤ b))
lhs3 =
trans
(*ℤ-assoc (absZ baseQR *ℤ absZ a) (+toℤ rhsDen) (+toℤ b))
refl

rhs1 : rhsNum *ℤ +toℤ lhsDen ≡ (absZ baseQR *ℤ absZ a) *ℤ +toℤ lhsDen
rhs1 = cong (λ t → t *ℤ +toℤ lhsDen) rhsNum≡

rhs2 : (absZ baseQR *ℤ absZ a) *ℤ +toℤ lhsDen ≡ (absZ baseQR *ℤ absZ a) *ℤ (+toℤ lhsDen)
rhs2 = refl

in
trans
(trans lhs1 lhs2)
(trans
lhs3
(trans
(cong (λ t → (absZ baseQR *ℤ absZ a) *ℤ t) denRel)
(sym (trans rhs1 rhs2))))
in
cross

```

Right scaling. The right-scaling variant follows by the same factorization. Only the order of the numerator factors changes, and commutativity restores the same common factor as in the left-scaling case.

```

-- § distℚ (q*p) (r*p) ≃ℚ distℚ q r * distℚ p 0 (right scaling)
distℚ-*ℚ-right : (p q r : ℚ) → distℚ (q *ℚ p) (r *ℚ p) ≃ℚ (distℚ q r *ℚ distℚ p 0ℚ)

```

```

distQ- $\mathbb{Q}$ -right (a / b) (c / d) (e / f) =
let
  swapScale : (x :  $\mathbb{Z}$ )  $\rightarrow$  (u v :  $\mathbb{N}^+$ )  $\rightarrow$  (x * $\mathbb{Z}$  +to $\mathbb{Z}$  u) * $\mathbb{Z}$  +to $\mathbb{Z}$  v  $\equiv$  (x * $\mathbb{Z}$  +to $\mathbb{Z}$  v) * $\mathbb{Z}$  +to $\mathbb{Z}$  u
  swapScale x u v =
    trans
      (* $\mathbb{Z}$ -assoc x (+to $\mathbb{Z}$  u) (+to $\mathbb{Z}$  v))
      (trans
        (cong ( $\lambda$  t  $\rightarrow$  x * $\mathbb{Z}$  t) (* $\mathbb{Z}$ -comm (+to $\mathbb{Z}$  u) (+to $\mathbb{Z}$  v)))
        (sym (* $\mathbb{Z}$ -assoc x (+to $\mathbb{Z}$  v) (+to $\mathbb{Z}$  u))))

mul4-rearrange : (x y z w :  $\mathbb{Z}$ )  $\rightarrow$  (x * $\mathbb{Z}$  y) * $\mathbb{Z}$  (z * $\mathbb{Z}$  w)  $\equiv$  (x * $\mathbb{Z}$  z) * $\mathbb{Z}$  (y * $\mathbb{Z}$  w)
mul4-rearrange x y z w =
  trans
    (* $\mathbb{Z}$ -assoc x y (z * $\mathbb{Z}$  w))
  (trans
    (cong ( $\lambda$  t  $\rightarrow$  x * $\mathbb{Z}$  t)
      (trans
        (sym (* $\mathbb{Z}$ -assoc y z w))
        (trans
          (cong ( $\lambda$  t  $\rightarrow$  t * $\mathbb{Z}$  w) (* $\mathbb{Z}$ -comm y z))
          (* $\mathbb{Z}$ -assoc z y w))))
    (sym (* $\mathbb{Z}$ -assoc x z (y * $\mathbb{Z}$  w))))

cf :  $\mathbb{Z}$ 
cf = c * $\mathbb{Z}$  +to $\mathbb{Z}$  f

ed :  $\mathbb{Z}$ 
ed = e * $\mathbb{Z}$  +to $\mathbb{Z}$  d

baseQR :  $\mathbb{Z}$ 
baseQR = cf + $\mathbb{Z}$  neg $\mathbb{Z}$  ed

ab :  $\mathbb{Z}$ 
ab = a * $\mathbb{Z}$  +to $\mathbb{Z}$  b

p0Raw :  $\mathbb{Z}$ 
p0Raw = (a * $\mathbb{Z}$  +to $\mathbb{Z}$  one+) + $\mathbb{Z}$  neg $\mathbb{Z}$  (0 $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  b)

p0Raw $\equiv$ a : p0Raw  $\equiv$  a
p0Raw $\equiv$ a =
  trans
    (cong ( $\lambda$  t  $\rightarrow$  t + $\mathbb{Z}$  neg $\mathbb{Z}$  (0 $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  b)) (* $\mathbb{Z}$ -one-right a))
  (trans
    (cong ( $\lambda$  t  $\rightarrow$  a + $\mathbb{Z}$  neg $\mathbb{Z}$  t) (* $\mathbb{Z}$ -zero-left (+to $\mathbb{Z}$  b)))
    (trans
      (cong ( $\lambda$  t  $\rightarrow$  a + $\mathbb{Z}$  t) (neg $\mathbb{Z}$ -zero))
      (+ $\mathbb{Z}$ -zero-right a)))

absP0 :  $\mathbb{Z}$ 
absP0 = abs $\mathbb{Z}$  p0Raw

absP0 $\equiv$ absA : absP0  $\equiv$  abs $\mathbb{Z}$  a
absP0 $\equiv$ absA = trans (abs $\mathbb{Z}$ -cong p0Raw $\equiv$ a) refl

fbDen :  $\mathbb{N}^+$ 

```

```

fbDen = f *+ b

dbDen :  $\mathbb{N}^+$ 
dbDen = d *+ b

-- § LHS cleared numerator for dist $\mathbb{Q}$  (q*p) (r*p)
Z :  $\mathbb{Z}$ 
Z = ((c * $\mathbb{Z}$  a) * $\mathbb{Z}$  +to $\mathbb{Z}$  fbDen) + $\mathbb{Z}$  neg $\mathbb{Z}$  ((e * $\mathbb{Z}$  a) * $\mathbb{Z}$  +to $\mathbb{Z}$  dbDen)

term1 : ((c * $\mathbb{Z}$  a) * $\mathbb{Z}$  +to $\mathbb{Z}$  fbDen)  $\equiv$  ab * $\mathbb{Z}$  cf
term1 =
  trans
    (cong (λ t → (c * $\mathbb{Z}$  a) * $\mathbb{Z}$  t) (+to $\mathbb{Z}$ -*+ f b))
    (trans
      (mul4-rearrange c a (+to $\mathbb{Z}$  f) (+to $\mathbb{Z}$  b))
      (* $\mathbb{Z}$ -comm (c * $\mathbb{Z}$  +to $\mathbb{Z}$  f) (a * $\mathbb{Z}$  +to $\mathbb{Z}$  b)))

term2 : ((e * $\mathbb{Z}$  a) * $\mathbb{Z}$  +to $\mathbb{Z}$  dbDen)  $\equiv$  ab * $\mathbb{Z}$  ed
term2 =
  trans
    (cong (λ t → (e * $\mathbb{Z}$  a) * $\mathbb{Z}$  t) (+to $\mathbb{Z}$ -*+ d b))
    (trans
      (mul4-rearrange e a (+to $\mathbb{Z}$  d) (+to $\mathbb{Z}$  b))
      (* $\mathbb{Z}$ -comm (e * $\mathbb{Z}$  +to $\mathbb{Z}$  d) (a * $\mathbb{Z}$  +to $\mathbb{Z}$  b)))

factorZ : Z  $\equiv$  ab * $\mathbb{Z}$  baseQR
factorZ =
  let
    Z1 : Z  $\equiv$  (ab * $\mathbb{Z}$  cf) + $\mathbb{Z}$  neg $\mathbb{Z}$  (ab * $\mathbb{Z}$  ed)
    Z1 =
      trans
        (cong (λ t → t + $\mathbb{Z}$  neg $\mathbb{Z}$  ((e * $\mathbb{Z}$  a) * $\mathbb{Z}$  +to $\mathbb{Z}$  dbDen)) term1)
        (cong (λ t → (ab * $\mathbb{Z}$  cf) + $\mathbb{Z}$  neg $\mathbb{Z}$  t) term2)

    negPull : neg $\mathbb{Z}$  (ab * $\mathbb{Z}$  ed)  $\equiv$  ab * $\mathbb{Z}$  neg $\mathbb{Z}$  ed
    negPull = sym (* $\mathbb{Z}$ -neg-right ab ed)

    Z2 : (ab * $\mathbb{Z}$  cf) + $\mathbb{Z}$  neg $\mathbb{Z}$  (ab * $\mathbb{Z}$  ed)  $\equiv$  (ab * $\mathbb{Z}$  cf) + $\mathbb{Z}$  (ab * $\mathbb{Z}$  neg $\mathbb{Z}$  ed)
    Z2 = cong (λ t → (ab * $\mathbb{Z}$  cf) + $\mathbb{Z}$  t) negPull

    Z3 : (ab * $\mathbb{Z}$  cf) + $\mathbb{Z}$  (ab * $\mathbb{Z}$  neg $\mathbb{Z}$  ed)  $\equiv$  ab * $\mathbb{Z}$  (cf + $\mathbb{Z}$  neg $\mathbb{Z}$  ed)
    Z3 = sym (* $\mathbb{Z}$ -distrib-right-+ $\mathbb{Z}$  ab cf (neg $\mathbb{Z}$  ed))
  in
    trans Z1 (trans Z2 Z3)

absZ :  $\mathbb{Z}$ 
absZ = abs $\mathbb{Z}$  Z

absZ  $\equiv$  scaled : absZ  $\equiv$  (abs $\mathbb{Z}$  baseQR * $\mathbb{Z}$  abs $\mathbb{Z}$  a) * $\mathbb{Z}$  +to $\mathbb{Z}$  b
absZ  $\equiv$  scaled =
  let
    absZ1 : absZ  $\equiv$  abs $\mathbb{Z}$  (ab * $\mathbb{Z}$  baseQR)
    absZ1 = cong abs $\mathbb{Z}$  factorZ

    absZ2 : abs $\mathbb{Z}$  (ab * $\mathbb{Z}$  baseQR)  $\equiv$  (abs $\mathbb{Z}$  ab * $\mathbb{Z}$  abs $\mathbb{Z}$  baseQR)

```

```

absZ2 = absℤ-mul ab baseQR

absAB : absℤ ab ≡ absℤ a *ℤ +toℤ b
absAB = absℤ-mul-pos-right a b

absZ3 : (absℤ ab *ℤ absℤ baseQR) ≡ ((absℤ a *ℤ +toℤ b) *ℤ absℤ baseQR)
absZ3 = cong (λ t → t *ℤ absℤ baseQR) absAB

absZ4 : ((absℤ a *ℤ +toℤ b) *ℤ absℤ baseQR) ≡ ((absℤ baseQR *ℤ absℤ a) *ℤ +toℤ b)
absZ4 =
  trans
    (*ℤ-assoc (absℤ a) (+toℤ b) (absℤ baseQR))
  (trans
    (cong (λ t → (absℤ a) *ℤ t) (*ℤ-comm (+toℤ b) (absℤ baseQR))))
  (trans
    (sym (*ℤ-assoc (absℤ a) (absℤ baseQR) (+toℤ b))))
  (trans
    (cong (λ t → t *ℤ (+toℤ b)) (*ℤ-comm (absℤ a) (absℤ baseQR))))
  (refl)))
in
trans absZ1 (trans absZ2 (trans absZ3 absZ4))

lhsDen : ℕ+
lhsDen = (d *+ b) *+ (f *+ b)

rhsDen : ℕ+
rhsDen = (d *+ f) *+ (b *+ one+)

rhsNum : ℤ
rhsNum = (absℤ baseQR *ℤ absP0)

rhsNum ≡ : rhsNum ≡ (absℤ baseQR *ℤ absℤ a)
rhsNum ≡ = cong (λ t → (absℤ baseQR *ℤ t)) absP0 ≡ absA

denRel : (+toℤ rhsDen) *ℤ (+toℤ b) ≡ +toℤ lhsDen
denRel =
  let
    lhs0 : +toℤ lhsDen ≡ (+toℤ (d *+ b)) *ℤ (+toℤ (f *+ b))
    lhs0 = +toℤ-*+ (d *+ b) (f *+ b)

    rhs0 : +toℤ rhsDen ≡ (+toℤ (d *+ f)) *ℤ (+toℤ (b *+ one+))
    rhs0 = +toℤ-*+ (d *+ f) (b *+ one+)

    db : +toℤ (d *+ b) ≡ (+toℤ d) *ℤ (+toℤ b)
    db = +toℤ-*+ d b

    fb' : +toℤ (f *+ b) ≡ (+toℤ f) *ℤ (+toℤ b)
    fb' = +toℤ-*+ f b

    dff : +toℤ (d *+ f) ≡ (+toℤ d) *ℤ (+toℤ f)
    dff = +toℤ-*+ d f

    bone : +toℤ (b *+ one+) ≡ (+toℤ b) *ℤ (+toℤ one+)
    bone = +toℤ-*+ b one+

```

```

stepR : (+toℤ rhsDen) *ℤ (+toℤ b) ≡ ((+toℤ d) *ℤ (+toℤ f)) *ℤ (((+toℤ b) *ℤ (+toℤ one+)) *ℤ (+toℤ b))
stepR =
  trans
    (cong (λ t → t *ℤ (+toℤ b)) rhs₀)
  (trans
    (cong (λ t → ((+toℤ (d *+ f)) *ℤ t) *ℤ (+toℤ b)) bone)
    (trans
      (cong (λ t → (t *ℤ ((+toℤ b) *ℤ (+toℤ one+))) *ℤ (+toℤ b)) dff)
      (trans
        (*ℤ-assoc ((+toℤ d) *ℤ (+toℤ f)) ((+toℤ b) *ℤ (+toℤ one+)) (+toℤ b))
        refl))))

stepL : +toℤ lhsDen ≡ ((+toℤ b) *ℤ (+toℤ b)) *ℤ ((+toℤ d) *ℤ (+toℤ f))
stepL =
  trans
    lhs₀
  (trans
    (cong (λ t → t *ℤ (+toℤ (f *+ b))) db)
    (trans
      (cong (λ t → ((+toℤ d) *ℤ (+toℤ b)) *ℤ t) fb')
      (trans
        (mul4-rearrange (+toℤ d) (+toℤ b) (+toℤ f) (+toℤ b))
        (trans
          (cong (λ t → ((+toℤ d) *ℤ (+toℤ f)) *ℤ t) (*ℤ-comm (+toℤ b) (+toℤ b)))
          (trans
            (*ℤ-comm ((+toℤ d) *ℤ (+toℤ f)) ((+toℤ b) *ℤ (+toℤ b)))
            refl))))))

in
let
  bI ≡ b : (+toℤ b) *ℤ (+toℤ one+) ≡ (+toℤ b)
  bI ≡ b = *ℤ-one-right (+toℤ b)

  inner : ((+toℤ b) *ℤ (+toℤ one+)) *ℤ (+toℤ b) ≡ (+toℤ b) *ℤ (+toℤ b)
  inner = cong (λ t → t *ℤ (+toℤ b)) bI ≡ b

in
trans
  stepR
  (trans
    (cong (λ t → ((+toℤ d) *ℤ (+toℤ f)) *ℤ t) inner)
    (trans
      (*ℤ-comm ((+toℤ d) *ℤ (+toℤ f)) ((+toℤ b) *ℤ (+toℤ b)))
      (sym stepL))))

cross : (absZ *ℤ +toℤ rhsDen) ≡ (rhsNum *ℤ +toℤ lhsDen)
cross =
  let
    lhs₁ : absZ *ℤ +toℤ rhsDen ≡ ((absℤ baseQR *ℤ absℤ a) *ℤ +toℤ b) *ℤ +toℤ rhsDen
    lhs₁ = cong (λ t → t *ℤ +toℤ rhsDen) absZ ≡ scaled

    lhs₂ : ((absℤ baseQR *ℤ absℤ a) *ℤ +toℤ b) *ℤ +toℤ rhsDen ≡ ((absℤ baseQR *ℤ absℤ a) *ℤ +toℤ rhsDen) *ℤ +toℤ b
    lhs₂ = swapScale (absℤ baseQR *ℤ absℤ a) b rhsDen

    lhs₃ : ((absℤ baseQR *ℤ absℤ a) *ℤ +toℤ rhsDen) *ℤ +toℤ b ≡ (absℤ baseQR *ℤ absℤ a) *ℤ ((+toℤ rhsDen) *ℤ (+toℤ b))
    lhs₃ = trans (*ℤ-assoc (absℤ baseQR *ℤ absℤ a) (+toℤ rhsDen) (+toℤ b)) refl

```

```

rhs1 : rhsNum * $\mathbb{Z}$  +to $\mathbb{Z}$  lhsDen  $\equiv$  (abs $\mathbb{Z}$  baseQR * $\mathbb{Z}$  abs $\mathbb{Z}$  a) * $\mathbb{Z}$  +to $\mathbb{Z}$  lhsDen
rhs1 = cong ( $\lambda$  t  $\rightarrow$  t * $\mathbb{Z}$  +to $\mathbb{Z}$  lhsDen) rhsNum $\equiv$ 
in
trans
(trans lhs1 lhs2)
(trans
lhs3
(trans
(cong ( $\lambda$  t  $\rightarrow$  (abs $\mathbb{Z}$  baseQR * $\mathbb{Z}$  abs $\mathbb{Z}$  a) * $\mathbb{Z}$  t) denRel)
(sym rhs1)))
in
cross

```

Distance Bounds

This section supplies the bridge between order language and metric language. From the two one-sided inequalities $x \leq y + \varepsilon$ and $y \leq x + \varepsilon$, both sides are rewritten as bounds on the same scaled numerator and then converted into the absolute-value estimate that yields $d(x, y) \leq \varepsilon$. Conversely, a distance certificate recovers the corresponding one-sided order bounds.

```

-- § key bound:  $x \leq y + \epsilon$  and  $y \leq x + \epsilon$  imply  $\text{dist}\mathbb{Q} \ x \ y \leq \epsilon$ 
dist $\mathbb{Q}$ -bounded-by- $\epsilon$  : (x y  $\in$  :  $\mathbb{Q}$ )  $\rightarrow$  x  $\leq \mathbb{Q}$  (y + $\mathbb{Q}$   $\epsilon$ )  $\rightarrow$  y  $\leq \mathbb{Q}$  (x + $\mathbb{Q}$   $\epsilon$ )  $\rightarrow$  dist $\mathbb{Q} \ x \ y \leq \mathbb{Q} \ \epsilon$ 
dist $\mathbb{Q}$ -bounded-by- $\epsilon$  (a / b) (c / d) (e / f) x $\leq$ y+ $\epsilon$  y $\leq$ x+ $\epsilon$  =
let
  bd :  $\mathbb{N}^+$ 
  bd = b *+ d

  df :  $\mathbb{N}^+$ 
  df = d *+ f

  bf :  $\mathbb{N}^+$ 
  bf = b *+ f

  bdf :  $\mathbb{N}^+$ 
  bdf = bd *+ f

  diff :  $\mathbb{Z}$ 
  diff = (a * $\mathbb{Z}$  +to $\mathbb{Z}$  d) + $\mathbb{Z}$  neg $\mathbb{Z}$  (c * $\mathbb{Z}$  +to $\mathbb{Z}$  b)

  y+ $\epsilon$ -num :  $\mathbb{Z}$ 
  y+ $\epsilon$ -num = (c * $\mathbb{Z}$  +to $\mathbb{Z}$  f) + $\mathbb{Z}$  (e * $\mathbb{Z}$  +to $\mathbb{Z}$  d)

  x+ $\epsilon$ -num :  $\mathbb{Z}$ 
  x+ $\epsilon$ -num = (a * $\mathbb{Z}$  +to $\mathbb{Z}$  f) + $\mathbb{Z}$  (e * $\mathbb{Z}$  +to $\mathbb{Z}$  b)

  hyp1 : (a * $\mathbb{Z}$  +to $\mathbb{Z}$  df)  $\leq \mathbb{Z}$  (y+ $\epsilon$ -num * $\mathbb{Z}$  +to $\mathbb{Z}$  b)
  hyp1 = x $\leq$ y+ $\epsilon$ 

  hyp2 : (c * $\mathbb{Z}$  +to $\mathbb{Z}$  bf)  $\leq \mathbb{Z}$  (x+ $\epsilon$ -num * $\mathbb{Z}$  +to $\mathbb{Z}$  d)

```

$hyp2 = y \leq x + \epsilon$

-- § expand hypotheses

$adf \leq cfb + edb : (a * \mathbb{Z} + \text{to}\mathbb{Z} df) \leq \mathbb{Z} ((c * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} b + \mathbb{Z} (e * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} b)$
 $adf \leq cfb + edb = \leq \mathbb{Z}\text{-resp} \equiv^r (*\mathbb{Z}\text{-distrib-left} + \mathbb{Z} (c * \mathbb{Z} + \text{to}\mathbb{Z} f) (e * \mathbb{Z} + \text{to}\mathbb{Z} d) (+\text{to}\mathbb{Z} b)) hyp1$

$cbf \leq afd + ebd : (c * \mathbb{Z} + \text{to}\mathbb{Z} bf) \leq \mathbb{Z} ((a * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} d + \mathbb{Z} (e * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} d)$
 $cbf \leq afd + ebd = \leq \mathbb{Z}\text{-resp} \equiv^r (*\mathbb{Z}\text{-distrib-left} + \mathbb{Z} (a * \mathbb{Z} + \text{to}\mathbb{Z} f) (e * \mathbb{Z} + \text{to}\mathbb{Z} b) (+\text{to}\mathbb{Z} d)) hyp2$

-- § associativity lemmas

$assoc\text{-}adf : a * \mathbb{Z} + \text{to}\mathbb{Z} df \equiv (a * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} f$
 $assoc\text{-}adf = \text{trans} (\text{cong} (\lambda t \rightarrow a * \mathbb{Z} t) (+\text{to}\mathbb{Z}\text{-}*+ df)) (\text{sym} (*\mathbb{Z}\text{-assoc} a (+\text{to}\mathbb{Z} d) (+\text{to}\mathbb{Z} f)))$

$assoc\text{-}cbf : c * \mathbb{Z} + \text{to}\mathbb{Z} bf \equiv (c * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} f$
 $assoc\text{-}cbf = \text{trans} (\text{cong} (\lambda t \rightarrow c * \mathbb{Z} t) (+\text{to}\mathbb{Z}\text{-}*+ bf)) (\text{sym} (*\mathbb{Z}\text{-assoc} c (+\text{to}\mathbb{Z} b) (+\text{to}\mathbb{Z} f)))$

$\text{swapScale} : (x : \mathbb{Z}) \rightarrow (u v : \mathbb{N}^+) \rightarrow (x * \mathbb{Z} + \text{to}\mathbb{Z} u) * \mathbb{Z} + \text{to}\mathbb{Z} v \equiv (x * \mathbb{Z} + \text{to}\mathbb{Z} v) * \mathbb{Z} + \text{to}\mathbb{Z} u$
 $\text{swapScale } x u v =$
 trans
 $(*\mathbb{Z}\text{-assoc } x (+\text{to}\mathbb{Z} u) (+\text{to}\mathbb{Z} v))$
 $(\text{trans}$
 $(\text{cong} (\lambda t \rightarrow x * \mathbb{Z} t) (*\mathbb{Z}\text{-comm} (+\text{to}\mathbb{Z} u) (+\text{to}\mathbb{Z} v))))$
 $(\text{sym} (*\mathbb{Z}\text{-assoc } x (+\text{to}\mathbb{Z} v) (+\text{to}\mathbb{Z} u))))$

$assoc\text{-}cfb : (c * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} b \equiv (c * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} f$
 $assoc\text{-}cfb = \text{swapScale } c f b$

$assoc\text{-}afd : (a * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} d \equiv (a * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} f$
 $assoc\text{-}afd = \text{swapScale } a f d$

$edb \equiv ebd : (e * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} b \equiv (e * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} d$
 $edb \equiv ebd = \text{swapScale } e d b$

-- § renamed for clarity

$adf' : \mathbb{Z}$
 $adf' = (a * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} f$

$cbf' : \mathbb{Z}$
 $cbf' = (c * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} f$

$ebd : \mathbb{Z}$
 $ebd = (e * \mathbb{Z} + \text{to}\mathbb{Z} b) * \mathbb{Z} + \text{to}\mathbb{Z} d$

$\text{rhsEq}_1 : ((c * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} b + \mathbb{Z} (e * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} b) \equiv (cbf' + \mathbb{Z} (e * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} b)$
 $\text{rhsEq}_1 = \text{cong} (\lambda t \rightarrow t + \mathbb{Z} (e * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} b) \text{assoc}\text{-}cfb$

$\text{rhsEq}_2 : (cbf' + \mathbb{Z} (e * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} b) \equiv (cbf' + \mathbb{Z} ebd)$
 $\text{rhsEq}_2 = \text{cong} (\lambda t \rightarrow cbf' + \mathbb{Z} t) \text{edb} \equiv \text{ebd}$

$\text{rhsEq} : ((c * \mathbb{Z} + \text{to}\mathbb{Z} f) * \mathbb{Z} + \text{to}\mathbb{Z} b + \mathbb{Z} (e * \mathbb{Z} + \text{to}\mathbb{Z} d) * \mathbb{Z} + \text{to}\mathbb{Z} b) \equiv (cbf' + \mathbb{Z} ebd)$
 $\text{rhsEq} = \text{trans } \text{rhsEq}_1 \text{rhsEq}_2$

-- § hyp1 gives: $(a*d)*f \leq (c*b)*f + ebd$

$hyp1' : adf' \leq \mathbb{Z} (cbf' + \mathbb{Z} ebd)$
 $hyp1' = \leq \mathbb{Z}\text{-resp} \equiv^l \text{assoc}\text{-}adf (\leq \mathbb{Z}\text{-resp} \equiv^r \text{rhsEq } adf \leq cfb + edb)$

```

-- § hyp2 gives: (c*b)*f ≤ (a*d)*f + ebd
hyp2' : cbf' ≤ℤ (adf' +ℤ ebd)
hyp2' = ≤ℤ-resp≡l assoc-cbf (≤ℤ-resp≡r (cong (λ t → t +ℤ ebd) assoc-afd) cbf' ≤ afd+ebd)

-- § diff * f = adf' - cbf'
diff-f : ℤ
diff-f = adf' +ℤ negℤ cbf'

diff-f-eq : diff *ℤ +toℤ f ≡ diff-f
diff-f-eq =
  trans
    (*ℤ-distrib-left+ℤ (a *ℤ +toℤ d) (negℤ (c *ℤ +toℤ b)) (+toℤ f))
    (cong (λ t → ((a *ℤ +toℤ d) *ℤ +toℤ f) +ℤ t) (*ℤ-neg-left (c *ℤ +toℤ b) (+toℤ f)))

-- § diff-f ≤ ebd from hyp1'
diff-f≤ebd : diff-f ≤ℤ ebd
diff-f≤ebd = ≤ℤ-+ℤ-cancelr adf' cbf' ebd (≤ℤ-resp≡r (sym (+ℤ-comm ebd cbf')) hyp1')

-- § negℤ diff-f ≤ ebd from hyp2'
neg-diff-f≤ebd : (negℤ diff-f) ≤ℤ ebd
neg-diff-f≤ebd =
  let
    step : cbf' +ℤ negℤ adf' ≤ℤ ebd
    step = ≤ℤ-+ℤ-cancelr cbf' adf' ebd (≤ℤ-resp≡r (sym (+ℤ-comm ebd adf')) hyp2')

  neg-eq : negℤ diff-f ≡ cbf' +ℤ negℤ adf'
  neg-eq =
    trans
      (neg+ℤ adf' (negℤ cbf'))
      (trans
        (+ℤ-comm (negℤ adf') (negℤ (negℤ cbf'))))
      (cong (λ t → t +ℤ negℤ adf') (negℤ-involutive cbf')))
  in
    ≤ℤ-resp≡l (sym neg-eq) step

-- § combine via absℤ-within-bound
neg-ebd≤diff-f : (negℤ ebd) ≤ℤ diff-f
neg-ebd≤diff-f = ≤ℤ-resp≡r (negℤ-involutive diff-f) (negℤ-antitone-≤ℤ neg-diff-f≤ebd)

abs-diff-f≤ebd : absℤ diff-f ≤ℤ ebd
abs-diff-f≤ebd = absℤ-within-bound diff-f ebd neg-ebd≤diff-f diff-f≤ebd

-- § transport to distℚ x y ≤ℚ ε
abs-diff-f-eq : absℤ diff-f ≡ absℤ (diff *ℤ +toℤ f)
abs-diff-f-eq = cong absℤ (sym diff-f-eq)

abs-mul-eq : absℤ (diff *ℤ +toℤ f) ≡ (absℤ diff *ℤ +toℤ f)
abs-mul-eq = absℤ-mul-pos-right diff f

ebd-eq : ebd ≡ (e *ℤ +toℤ bd)
ebd-eq =
  trans
    (*ℤ-assoc e (+toℤ b) (+toℤ d))
    (cong (λ t → e *ℤ t) (sym (+toℤ-*+ b d)))

```



```

goal : (absℤ diff *ℤ +toℤ f) ≤ℤ (e *ℤ +toℤ bd)
goal = ≤ℤ-resp≡l (trans abs-diff-f-eq abs-mul-eq) (≤ℤ-resp≡r ebd-eq abs-diff-f≤ebd)
in
goal

```

Converse extractions. The converse direction recovers one-sided order bounds from a distance certificate. From $d(x, y) \leq \varepsilon$ one obtains $x \leq y + \varepsilon$ directly, and the second inequality follows by symmetry.

```

-- § distℚ x y ≤ ε implies x ≤ y + ε
distℚ≤ε→x≤y+ε : (x y : ℚ) → distℚ x y ≤ℚ ε → x ≤ℚ (y +ℚ ε)
distℚ≤ε→x≤y+ε (a / b) (c / d) (e / f) dist≤ =
let
  bd : ℕ+
  bd = b *+ d

  df : ℕ+
  df = d *+ f

  diff : ℤ
  diff = (a *ℤ +toℤ d) +ℤ negℤ (c *ℤ +toℤ b)

  absDiff : ℤ
  absDiff = absℤ diff

  absDiff*f≤e*bd : (absDiff *ℤ +toℤ f) ≤ℤ (e *ℤ +toℤ bd)
  absDiff*f≤e*bd = dist≤

  diff≤absDiff : diff ≤ℤ absDiff
  diff≤absDiff = ≤ℤ-absℤ diff

  diff*f≤absDiff*f : (diff *ℤ +toℤ f) ≤ℤ (absDiff *ℤ +toℤ f)
  diff*f≤absDiff*f = ≤ℤ-mul-pos-right diff absDiff f diff≤absDiff

  diff*f≤e*bd : (diff *ℤ +toℤ f) ≤ℤ (e *ℤ +toℤ bd)
  diff*f≤e*bd = ≤ℤ-trans diff*f≤absDiff*f absDiff*f≤e*bd

y+ε-num : ℤ
y+ε-num = (c *ℤ +toℤ f) +ℤ (e *ℤ +toℤ d)

assoc-adf : a *ℤ +toℤ df ≡ (a *ℤ +toℤ d) *ℤ +toℤ f
assoc-adf = trans (cong (λ t → a *ℤ t) (+toℤ-*+ d f)) (sym (*ℤ-assoc a (+toℤ d) (+toℤ f)))

swapScale : (x : ℤ) → (u v : ℕ+) → (x *ℤ +toℤ u) *ℤ +toℤ v ≡ (x *ℤ +toℤ v) *ℤ +toℤ u
swapScale x u v =
  trans
    (*ℤ-assoc x (+toℤ u) (+toℤ v))
  (trans
    (cong (λ t → x *ℤ t) (*ℤ-comm (+toℤ u) (+toℤ v)))
    (sym (*ℤ-assoc x (+toℤ v) (+toℤ u))))

assoc-cfb : (c *ℤ +toℤ f) *ℤ +toℤ b ≡ (c *ℤ +toℤ b) *ℤ +toℤ f
assoc-cfb = swapScale c f b

```

$edb \equiv ebd : (e \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } d) \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } b \equiv (e \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } b) \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } d$
 $edb \equiv ebd = \text{swapScale } e \text{ } d \text{ } b$

$adf' : \mathbb{Z}$
 $adf' = (a \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } d) \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } f$

$cbf' : \mathbb{Z}$
 $cbf' = (c \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } b) \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } f$

$ebd : \mathbb{Z}$
 $ebd = (e \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } b) \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } d$

$ebd \equiv e * bd : ebd \equiv (e \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } bd)$
 $ebd \equiv e * bd =$
 trans
 $(\text{*Z-} \text{assoc } e \text{ } (+\text{to } \mathbb{Z} \text{ } b) \text{ } (+\text{to } \mathbb{Z} \text{ } d))$
 $(\text{cong } (\lambda t \rightarrow e \text{ * } \mathbb{Z} \text{ } t) \text{ } (\text{sym } (+\text{to } \mathbb{Z} \text{ } \text{*+ } b \text{ } d)))$

$\text{diff}^* f \leq ebd : (\text{diff } \text{*Z } +\text{to } \mathbb{Z} \text{ } f) \leq \mathbb{Z} \text{ } ebd$
 $\text{diff}^* f \leq ebd = \leq \mathbb{Z} \text{-resp} \equiv^r (\text{sym } ebd \equiv e * bd) \text{diff}^* f \leq e * bd$

$\text{diff} \cdot f : \mathbb{Z}$
 $\text{diff} \cdot f = adf' + \mathbb{Z} \text{ neg } \mathbb{Z} \text{ } cbf'$

$\text{diff} \cdot f \cdot \text{eq} : \text{diff } \text{*Z } +\text{to } \mathbb{Z} \text{ } f \equiv \text{diff} \cdot f$
 $\text{diff} \cdot f \cdot \text{eq} =$
 trans
 $(\text{*Z-distrib-left} + \mathbb{Z} \text{ } (a \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } d) \text{ } (\text{neg } \mathbb{Z} \text{ } (c \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } b)) \text{ } (+\text{to } \mathbb{Z} \text{ } f))$
 $(\text{cong}$
 $(\lambda t \rightarrow ((a \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } d) \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } f) + \mathbb{Z} \text{ } t)$
 $(\text{trans}$
 $(\text{*Z-neg-left } (c \text{ * } \mathbb{Z} \text{ +to } \mathbb{Z} \text{ } b) \text{ } (+\text{to } \mathbb{Z} \text{ } f))$
 $\text{refl}))$

$\text{diff} \cdot f \leq ebd' : \text{diff} \cdot f \leq \mathbb{Z} \text{ } ebd$
 $\text{diff} \cdot f \leq ebd' = \leq \mathbb{Z} \text{-resp} \equiv^l \text{diff} \cdot f \cdot \text{eq } \text{diff}^* f \leq ebd$

-- § add cbf' to both sides
 $\text{sumLe} : (\text{diff} \cdot f + \mathbb{Z} \text{ } cbf') \leq \mathbb{Z} \text{ } (ebd + \mathbb{Z} \text{ } cbf')$
 $\text{sumLe} = \leq \mathbb{Z} \text{-} + \mathbb{Z} \text{-mono } \text{diff} \cdot f \leq ebd' \text{ } (\leq \mathbb{Z} \text{-refl } cbf')$

$\text{lhsEq} : (\text{diff} \cdot f + \mathbb{Z} \text{ } cbf') \equiv adf'$
 $\text{lhsEq} =$
 trans
 $(+\mathbb{Z} \text{-assoc } adf' \text{ } (\text{neg } \mathbb{Z} \text{ } cbf') \text{ } cbf')$
 $(\text{trans}$
 $(\text{cong } (\lambda t \rightarrow adf' + \mathbb{Z} \text{ } t) \text{ } (+\mathbb{Z} \text{-inv-left } cbf'))$
 $(+\mathbb{Z} \text{-zero-right } adf'))$

$\text{rhsEq} : (ebd + \mathbb{Z} \text{ } cbf') \equiv (cbf' + \mathbb{Z} \text{ } ebd)$
 $\text{rhsEq} = +\mathbb{Z} \text{-comm } ebd \text{ } cbf'$

$\text{hyp1}' : adf' \leq \mathbb{Z} \text{ } (cbf' + \mathbb{Z} \text{ } ebd)$
 $\text{hyp1}' = \leq \mathbb{Z} \text{-resp} \equiv^l \text{lhsEq } (\leq \mathbb{Z} \text{-resp} \equiv^r \text{rhsEq } \text{sumLe})$

```

rhsExpand : (y+ε-num *ℤ +toℤ b) ≡ (cbf' +ℤ ebd)
rhsExpand =
  let
    step1 : ((c *ℤ +toℤ f) *ℤ +toℤ b +ℤ (e *ℤ +toℤ d) *ℤ +toℤ b) ≡ (cbf' +ℤ (e *ℤ +toℤ d) *ℤ +toℤ b)
    step1 = cong (λ t → t +ℤ (e *ℤ +toℤ d) *ℤ +toℤ b) assoc-cfb

    step2 : (cbf' +ℤ (e *ℤ +toℤ d) *ℤ +toℤ b) ≡ (cbf' +ℤ ebd)
    step2 = cong (λ t → cbf' +ℤ t) edb≡ebd

  in
    trans (*ℤ-distrib-left+ℤ (c *ℤ +toℤ f) (e *ℤ +toℤ d) (+toℤ b)) (trans step1 step2)
  in
    ≤ℤ-resp≡l (sym assoc-adf) (≤ℤ-resp≡r (sym rhsExpand) hyp1')

-- § converse: distℚ x y ≤ ε implies y ≤ x + ε (by symmetry)
distℚ≤ε→y≤x+ε : (x y ε : ℚ) → distℚ x y ≤ℚ ε → y ≤ℚ (x +ℚ ε)
distℚ≤ε→y≤x+ε x y ε dist≤ =
  let
    dyx≤dxy : distℚ y x ≤ℚ distℚ x y
    dyx≤dxy =
      ≤ℚ→≤ℚ'
      {p = distℚ y x}
      {q = distℚ x y}
      (distℚ-sym y x)

    dyx≤ε : distℚ y x ≤ℚ ε
    dyx≤ε =
      ≤ℚ-trans
      {x = distℚ y x}
      {y = distℚ x y}
      {z = ε}
      dyx≤dxy
      dist≤

  in
    distℚ≤ε→x≤y+ε y x ε dyx≤ε

```

The rational distance function now has the expected structural laws: triangle inequality, translation invariance, multiplicative scaling, ε -splitting, and the conversion between distance and order bounds. Before the remaining passage beyond $\mathbb{Q}$ can be taken, one further quantitative ingredient is needed: an explicit Archimedean scaling lemma for positive rationals.

Chapter 50

Archimedean Scaling

The Archimedean principle needed later is the following constructive form: given $\varepsilon > 0$ and $m \in \mathbb{N}$, one can compute a positive rational δ such that $\delta \cdot (m + 1) < \varepsilon$. The witness is built directly from the denominator of ε together with the previously established half- ε machinery, so the required smallness is exhibited rather than merely asserted.

```
-- § right identity for  $\mathbb{N}^+$  multiplication
*+-one-right : (u :  $\mathbb{N}^+$ ) → (u *+ one+ ) ≡ u
*+-one-right (mk $\mathbb{N}^+$  p) =
  cong mk $\mathbb{N}^+$ 
    (trans
      (+ $\mathbb{N}$ -zero-right (p * $\mathbb{N}$  suc zero))
      (* $\mathbb{N}$ -one-right p))

-- § 0 < 1/b for any positive denominator
oneOver-pos : (b :  $\mathbb{N}^+$ ) → 0 $\mathbb{Q}$  <  $\mathbb{Q}$  (one $\mathbb{Z}$  / b)
oneOver-pos b =
  let
    rhsEq : one $\mathbb{Z}$  ≡ (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  one+)
    rhsEq = sym (* $\mathbb{Z}$ -one-right one $\mathbb{Z}$ )

    base : 0 $\mathbb{Z}$  < $\mathbb{Z}$  (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  one+)
    base = < $\mathbb{Z}$ -resp-≡r {x = 0 $\mathbb{Z}$ } {y = one $\mathbb{Z}$ } {z = (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  one+)} rhsEq 0 $\mathbb{Z}$  < one $\mathbb{Z}$ 
  in
    < $\mathbb{Z}$ -resp-≡l
      {x = 0 $\mathbb{Z}$ }
      {y = (0 $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  b)}
      {z = (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  one+)}
      (sym (* $\mathbb{Z}$ -zero-left (+to $\mathbb{Z}$  b)))
    base

-- § denominators are ≥ 1 in the integer order
one≤+to $\mathbb{Z}$  : (d :  $\mathbb{N}^+$ ) → one $\mathbb{Z}$  ≤ $\mathbb{Z}$  +to $\mathbb{Z}$  d
one≤+to $\mathbb{Z}$  (mk $\mathbb{N}^+$  k) = s≤s z≤n
```

```

-- § q ≥ 0 implies q ≤ num(q)/1
nonneg ≤ numOverOne : (q : ℚ) → 0ℚ ≤ ℚ q → q ≤ ℚ (num q / one+)
nonneg ≤ numOverOne (a / b) qNonneg =
let
  aNonneg : 0ℤ ≤ ℤ a
  aNonneg =
let
  one+ℤ ≡ oneℤ : +toℤ one+ ≡ oneℤ
  one+ℤ ≡ oneℤ = refl

  rhsEq : (a * ℤ +toℤ one+) ≡ a
  rhsEq = trans (cong (λ t → a * ℤ t) one+ℤ ≡ oneℤ) (*ℤ-one-right a)

  step0 : 0ℤ ≤ ℤ (a * ℤ +toℤ one+)
  step0 = ≤ℤ-resp ≡l (*ℤ-zero-left (+toℤ b)) qNonneg
in
≤ℤ-resp ≡r rhsEq step0

one ≤ b : oneℤ ≤ ℤ +toℤ b
one ≤ b = one ≤+ +toℤ b

step : (oneℤ * ℤ a) ≤ ℤ ((+toℤ b) * ℤ a)
step = ≤ℤ-mul-nonneg-right oneℤ (+toℤ b) a one ≤ b aNonneg

lhsEq : (oneℤ * ℤ a) ≡ (a * ℤ +toℤ one+)
lhsEq = trans (*ℤ-one-left a) (sym (*ℤ-one-right a))

rhsEq : ((+toℤ b) * ℤ a) ≡ (a * ℤ +toℤ b)
rhsEq = *ℤ-comm (+toℤ b) a

core : (a * ℤ +toℤ one+) ≤ ℤ (a * ℤ +toℤ b)
core = ≤ℤ-resp ≡l lhsEq (≤ℤ-resp ≡r rhsEq step)
in
core

-- § nonnegative q is bounded by suc(m)/1 for some m
nonneg-bound-sucInt : (q : ℚ) → 0ℚ ≤ ℚ q → ∑ ℕ (λ m → q ≤ ℚ (fromℕℤ (suc m) / one+))
nonneg-bound-sucInt (a / b) qNonneg =
let
  aNonneg : 0ℤ ≤ ℤ a
  aNonneg =
let
  one+ℤ ≡ oneℤ : +toℤ one+ ≡ oneℤ
  one+ℤ ≡ oneℤ = refl

  rhsEq : (a * ℤ +toℤ one+) ≡ a
  rhsEq = trans (cong (λ t → a * ℤ t) one+ℤ ≡ oneℤ) (*ℤ-one-right a)

  step0 : 0ℤ ≤ ℤ (a * ℤ +toℤ one+)
  step0 = ≤ℤ-resp ≡l (*ℤ-zero-left (+toℤ b)) qNonneg
in
≤ℤ-resp ≡r rhsEq step0

aNatPack : ∑ ℕ (λ n → a ≡ fromℕℤ n)
aNatPack = 0 ≤ ℤ → fromℕℤ a aNonneg

```

```

m : ℕ
m = fst aNatPack

a ≡ : a ≡ fromℕℤ m
a ≡ = snd aNatPack

q ≤ a/1 : (a / b) ≤ ℚ (a / one⁺)
q ≤ a/1 = nonneg-≤ numOverOne (a / b) qNonneg

a/1 ≤ m/1 : (a / one⁺) ≤ ℚ (fromℕℤ m / one⁺)
a/1 ≤ m/1 =
  ≤ℤ-resp-≡ʳ
  (cong (λ t → t * ℤ + toℤ one⁺) a ≡)
  (≤ℤ-refl (a * ℤ + toℤ one⁺))

m ≤ suc m : m ≤ suc m
m ≤ suc m = ≤-step m

fm ≤ fs : fromℕℤ m ≤ ℤ fromℕℤ (suc m)
fm ≤ fs = fromℕℤ-mono m ≤ suc m

m/1 ≤ suc m/1 : (fromℕℤ m / one⁺) ≤ ℚ (fromℕℤ (suc m) / one⁺)
m/1 ≤ suc m/1 =
  let
    one⁺ ℤ ≡ one ℤ : + toℤ one⁺ ≡ one ℤ
    one⁺ ℤ ≡ one ℤ = refl

  rhsOneEq : (n : ℕ) → (fromℕℤ n * ℤ + toℤ one⁺) ≡ fromℕℤ n
  rhsOneEq n = trans (cong (λ t → fromℕℤ n * ℤ t) one⁺ ℤ ≡ one ℤ) (*ℤ-one-right (fromℕℤ n))

  stepR : fromℕℤ m ≤ ℤ (fromℕℤ (suc m) * ℤ + toℤ one⁺)
  stepR = ≤ℤ-resp-≡ʳ (sym (rhsOneEq (suc m))) fm ≤ fs
in
  ≤ℤ-resp-≡ˡ (sym (rhsOneEq m)) stepR
in
  m,
  (≤ℚ-trans {a / b} {a / one⁺} {fromℕℤ (suc m) / one⁺} q ≤ a/1
   (≤ℚ-trans {a / one⁺} {fromℕℤ m / one⁺} {fromℕℤ (suc m) / one⁺} a/1 ≤ m/1 m/1 ≤ suc m/1))

```

Archimedean scaling. Given positive ε and a natural number m , the goal is to choose δ small enough that multiplication by $m + 1$ still leaves it below ε . The construction below does this directly from the denominator data, which is exactly the estimate needed in product-Cauchy arguments.

```

-- § Archimedean scaling: there exists δ>0 with δ·(suc m) < ε
δ-scale-suc : (ε : ℚ) → 0ℚ < ℚ ε → (m : ℕ) → ∑ ℚ (λ δ → (0ℚ < ℚ δ) × ((δ * ℚ (fromℕℤ (suc m) / one⁺)) < ℚ ε))
δ-scale-suc ε ∈ ε pos m =
  let
    k : ℕ⁺
    k = mkℕ⁺ m

```

```

b : ℕ+
b = den ε

δ : ℚ
δ = oneZ / halfDen (k++ b)

δpos : 0ℚ < ℚ δ
δpos = oneOver-pos (halfDen (k++ b))

factor : ℚ
factor = fromNZ (suc m) / one+

prod : ℚ
prod = δ * ℚ factor

-- § prod ≃ εHalf ε, hence prod < ε
kZ : ℤ
kZ = +toℤ k

kZ≡ : kZ ≡ fromNZ (suc m)
kZ≡ = refl

halfDenZ : (u : ℕ+) → +toℤ (halfDen u) ≡ (+toℤ two+) * ℤ +toℤ u
halfDenZ u = +toℤ-++ two+ u

rhsDenZ : +toℤ (halfDen b) ≡ (+toℤ two+) * ℤ +toℤ b
rhsDenZ = halfDenZ b

lhsDenZ : +toℤ (halfDen (k++ b)) ≡ (+toℤ two+) * ℤ ((+toℤ k) * ℤ +toℤ b)
lhsDenZ =
  trans
    (halfDenZ (k++ b))
    (cong (λ t → (+toℤ two+) * ℤ t) (+toℤ-++ k b))

swap : (x y z : ℤ) → (x * ℤ (y * ℤ z)) ≡ (y * ℤ (x * ℤ z))
swap x y z =
  trans
    (sym (*ℤ-assoc x y z))
    (trans
      (cong (λ t → t * ℤ z) (*ℤ-comm x y))
      (*ℤ-assoc y x z))

denEq : (+toℤ (halfDen (k++ b))) ≡ (fromNZ (suc m) * ℤ +toℤ (halfDen b))
denEq =
  trans
    lhsDenZ
    (trans
      (cong (λ t → (+toℤ two+) * ℤ (t * ℤ +toℤ b)) (sym kZ≡))
      (trans
        (swap (+toℤ two+) (fromNZ (suc m)) (+toℤ b))
        (cong (λ t → (fromNZ (suc m)) * ℤ t) (sym rhsDenZ))))

prod≃half : prod ≃ ℚ (εHalf ε)
prod≃half =
  let

```



```

lhsNum : ℤ
lhsNum = oneℤ *ℤ fromℕℤ (suc m)

lhsDen : ℕ+
lhsDen = (halfDen (k *+ b)) *+ one+

rhsNum : ℤ
rhsNum = oneℤ

rhsDen : ℕ+
rhsDen = halfDen b

lhsNumEq : lhsNum ≡ fromℕℤ (suc m)
lhsNumEq = *ℤ-one-left (fromℕℤ (suc m))

denOne : (halfDen (k *+ b)) *+ one+ ≡ halfDen (k *+ b)
denOne = *+-one-right (halfDen (k *+ b))

lhsDenEq : (+toℤ lhsDen) ≡ +toℤ (halfDen (k *+ b))
lhsDenEq = cong +toℤ denOne

cross : (lhsNum *ℤ +toℤ rhsDen) ≡ (rhsNum *ℤ +toℤ lhsDen)
cross =
  trans
    (cong (λ t → t *ℤ +toℤ rhsDen) lhsNumEq)
  (trans
    (sym denEq)
    (trans
      (sym (*ℤ-one-left (+toℤ (halfDen (k *+ b)))))
      (cong (λ t → oneℤ *ℤ t) (sym lhsDenEq))))
in
cross

half<ε : (εHalf ε) <ℚ ε
half<ε = εHalf<ε ε ∈ pos

prod<ε : prod <ℚ ε
prod<ε =
  ≤<ℚ→<ℚ
  {x = prod} {y = εHalf ε} {z = ε}
  (≃ℚ→≤ℚl {p = prod} {q = εHalf ε} prod≃half)
  half<ε
in
δ , (δpos , prod<ε)

```


Chapter 51

$\mathbb{Z}$ -Span of K_{12} Operators

The operators I and J generate a two-dimensional subalgebra of $\text{End}(\text{Vec}_{12}\mathbb{Z})$: every expression of the form $aI + bJ$ is determined by the coefficient pair (a, b) . This is a direct consequence of the relation $J^2 = 12J$, which collapses higher words back into the same span, while simple test vectors recover the coefficients.

Span Infrastructure

To use the (I, J) -span, the notation must first be fixed explicitly. The operator $I12$ is the identity, the scaling maps act pointwise on each block, and linIJ is the literal formula $a \cdot v + b \cdot Jv$. Once these definitions are in place, the block projection lemmas become definitional equalities and later coefficient extractions proceed without further choices.

```
-- § identity operator on Vec12ℤ
I12 : Op
I12 v = v

-- § scale a 4-vector by integer a
scaleVec4ℤ : ℤ → Vec4ℤ → Vec4ℤ
scaleVec4ℤ a v i = a *ℤ v i

-- § scale a 12-vector blockwise
scaleVec12ℤ : ℤ → Vec12ℤ → Vec12ℤ
scaleVec12ℤ a v = scaleVec4ℤ a (block0 v), (scaleVec4ℤ a (block1 v), scaleVec4ℤ a (block2 v))

-- § linear combination a·I + b·J
linIJ : ℤ → ℤ → Op
linIJ a b v = scaleVec12ℤ a v + Vec12ℤ scaleVec12ℤ b (J12Vec12ℤ v)

-- § block projection lemmas (all refl)
block0-linIJ : (a b : ℤ) → (v : Vec12ℤ) → (i : Fin4) →
  block0 (linIJ a b v) i ≡ (a *ℤ block0 v i) +ℤ (b *ℤ sum12ℤ v)
```

```
block0-linJ a b v i = refl
```

```
block1-linJ : (a b :  $\mathbb{Z}$ ) → (v : Vec12 $\mathbb{Z}$ ) → (i : Fin4) →
  block1 (linJ a b v) i ≡ (a * $\mathbb{Z}$  block1 v i) + $\mathbb{Z}$  (b * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)
block1-linJ a b v i = refl
```

```
block2-linJ : (a b :  $\mathbb{Z}$ ) → (v : Vec12 $\mathbb{Z}$ ) → (i : Fin4) →
  block2 (linJ a b v) i ≡ (a * $\mathbb{Z}$  block2 v i) + $\mathbb{Z}$  (b * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)
block2-linJ a b v i = refl
```

Sum-Scaling Laws

Later coefficient extraction passes repeatedly through Σ_{12} , so addition and scalar multiplication must commute with the total sum in a fixed normal form. The reshuffling lemma only regularizes the six terms coming from the three blocks; after that, the sum-of-addition and sum-of-scaling identities follow by routine integer algebra.

```
-- § six-term sum reassociation into (v-side) + (w-side)
shuffle3Pairs : (A A' B B' C C' :  $\mathbb{Z}$ ) →
  (A + $\mathbb{Z}$  A') + $\mathbb{Z}$  ((B + $\mathbb{Z}$  B') + $\mathbb{Z}$  (C + $\mathbb{Z}$  C')) ≡ (A + $\mathbb{Z}$  (B + $\mathbb{Z}$  C)) + $\mathbb{Z}$  (A' + $\mathbb{Z}$  (B' + $\mathbb{Z}$  C'))
shuffle3Pairs A A' B B' C C' =
  let
    X = (B + $\mathbb{Z}$  B') + $\mathbb{Z}$  (C + $\mathbb{Z}$  C')

    step1 : (A + $\mathbb{Z}$  A') + $\mathbb{Z}$  X ≡ A + $\mathbb{Z}$  (A' + $\mathbb{Z}$  X)
    step1 = + $\mathbb{Z}$ -assoc A A' X

    step2 : A' + $\mathbb{Z}$  X ≡ A' + $\mathbb{Z}$  (B + $\mathbb{Z}$  (B' + $\mathbb{Z}$  (C + $\mathbb{Z}$  C')))
    step2 = cong (λ t → A' + $\mathbb{Z}$  t) (+ $\mathbb{Z}$ -assoc B B' (C + $\mathbb{Z}$  C'))

    step3 : A' + $\mathbb{Z}$  (B + $\mathbb{Z}$  (B' + $\mathbb{Z}$  (C + $\mathbb{Z}$  C'))) ≡ B + $\mathbb{Z}$  (A' + $\mathbb{Z}$  (B' + $\mathbb{Z}$  (C + $\mathbb{Z}$  C')))
    step3 = swapHead $\mathbb{Z}$  A' B (B' + $\mathbb{Z}$  (C + $\mathbb{Z}$  C'))

    step4 : A' + $\mathbb{Z}$  (B' + $\mathbb{Z}$  (C + $\mathbb{Z}$  C')) ≡ C + $\mathbb{Z}$  (A' + $\mathbb{Z}$  (B' + $\mathbb{Z}$  C'))
    step4 =
      trans
      (cong (λ t → A' + $\mathbb{Z}$  t) (swapHead $\mathbb{Z}$  B' C C'))
      (swapHead $\mathbb{Z}$  A' C (B' + $\mathbb{Z}$  C'))

    step5 : B + $\mathbb{Z}$  (A' + $\mathbb{Z}$  (B' + $\mathbb{Z}$  (C + $\mathbb{Z}$  C'))) ≡ B + $\mathbb{Z}$  (C + $\mathbb{Z}$  (A' + $\mathbb{Z}$  (B' + $\mathbb{Z}$  C')))
    step5 = cong (λ t → B + $\mathbb{Z}$  t) step4

    step6 : A + $\mathbb{Z}$  (B + $\mathbb{Z}$  (C + $\mathbb{Z}$  (A' + $\mathbb{Z}$  (B' + $\mathbb{Z}$  C')))) ≡ (A + $\mathbb{Z}$  (B + $\mathbb{Z}$  C)) + $\mathbb{Z}$  (A' + $\mathbb{Z}$  (B' + $\mathbb{Z}$  C'))
    step6 =
      trans
      (cong (λ t → A + $\mathbb{Z}$  t) (sym (+ $\mathbb{Z}$ -assoc B C (A' + $\mathbb{Z}$  (B' + $\mathbb{Z}$  C')))))
      (sym (+ $\mathbb{Z}$ -assoc A (B + $\mathbb{Z}$  C) (A' + $\mathbb{Z}$  (B' + $\mathbb{Z}$  C'))))
  in
  trans
    step1
    (trans
```

```

(cong (λ t → A + ℤ t) (trans step2 (trans step3 step5)))
step6)

-- § finite-sum scaling by right distributivity
sumFin4-scaleVec4ℤ : (a : ℤ) → (v : Vec4ℤ) →
sumFin4ℤ (scaleVec4ℤ a v) ≡ a * ℤ sumFin4ℤ v
sumFin4-scaleVec4ℤ a v =
let
  v0 = v g0
  v1 = v g1
  v2 = v g2
  v3 = v g3

  expand : a * ℤ (v0 + ℤ (v1 + ℤ (v2 + ℤ v3))) ≡ (a * ℤ v0) + ℤ ((a * ℤ v1) + ℤ ((a * ℤ v2) + ℤ (a * ℤ v3)))
  expand =
    trans
      (*ℤ-distrib-right-+ℤ a v0 (v1 + ℤ (v2 + ℤ v3)))
      (cong (λ t → (a * ℤ v0) + ℤ t)
        (trans
          (*ℤ-distrib-right-+ℤ a v1 (v2 + ℤ v3))
          (cong (λ t → (a * ℤ v1) + ℤ t)
            (*ℤ-distrib-right-+ℤ a v2 v3))))
in
sym expand

-- § 12-vector sum distributes over addition
sum12-+Vec12ℤ : (v w : Vec12ℤ) → sum12ℤ (v +Vec12ℤ w) ≡ sum12ℤ v + ℤ sum12ℤ w
sum12-+Vec12ℤ v w =
let
  A = sumFin4ℤ (block0 v)
  B = sumFin4ℤ (block1 v)
  C = sumFin4ℤ (block2 v)
  A' = sumFin4ℤ (block0 w)
  B' = sumFin4ℤ (block1 w)
  C' = sumFin4ℤ (block2 w)
in
trans
  refl
  (trans
    (cong
      (λ t → t + ℤ (sumFin4ℤ (block1 (v +Vec12ℤ w)) + ℤ sumFin4ℤ (block2 (v +Vec12ℤ w))))
      (sumFin4-+Vec4ℤ (block0 v) (block0 w)))
    (trans
      (cong
        (λ t → (A + ℤ A') + ℤ (t + ℤ sumFin4ℤ (block2 (v +Vec12ℤ w))) )
        (sumFin4-+Vec4ℤ (block1 v) (block1 w)))
      (trans
        (cong
          (λ t → (A + ℤ A') + ℤ ((B + ℤ B') + ℤ t))
          (sumFin4-+Vec4ℤ (block2 v) (block2 w)))
        (shuffle3Pairs A A' B B' C C')))))

-- § 12-vector sum distributes over scalar multiplication
sum12-scaleVec12ℤ : (a : ℤ) → (v : Vec12ℤ) → sum12ℤ (scaleVec12ℤ a v) ≡ a * ℤ sum12ℤ v

```

```

sum12-scaleVec12 $\mathbb{Z}$  a v =
  let
    s0 = sumFin4 $\mathbb{Z}$  (block0 v)
    s1 = sumFin4 $\mathbb{Z}$  (block1 v)
    s2 = sumFin4 $\mathbb{Z}$  (block2 v)
  in
  let
    stepBlock : sum12 $\mathbb{Z}$  (scaleVec12 $\mathbb{Z}$  a v)  $\equiv$  (a * $\mathbb{Z}$  s0) + $\mathbb{Z}$  ((a * $\mathbb{Z}$  s1) + $\mathbb{Z}$  (a * $\mathbb{Z}$  s2))
    stepBlock =
      trans
      refl
      (trans
       (cong
        ( $\lambda$  t  $\rightarrow$  t + $\mathbb{Z}$  (sumFin4 $\mathbb{Z}$  (scaleVec4 $\mathbb{Z}$  a (block1 v)) + $\mathbb{Z}$  sumFin4 $\mathbb{Z}$  (scaleVec4 $\mathbb{Z}$  a (block2 v))))
        (sumFin4-scaleVec4 $\mathbb{Z}$  a (block0 v)))
       (trans
        (cong
         ( $\lambda$  t  $\rightarrow$  (a * $\mathbb{Z}$  s0) + $\mathbb{Z}$  (t + $\mathbb{Z}$  sumFin4 $\mathbb{Z}$  (scaleVec4 $\mathbb{Z}$  a (block2 v))))
         (sumFin4-scaleVec4 $\mathbb{Z}$  a (block1 v)))
        (cong
         ( $\lambda$  t  $\rightarrow$  (a * $\mathbb{Z}$  s0) + $\mathbb{Z}$  ((a * $\mathbb{Z}$  s1) + $\mathbb{Z}$  t))
         (sumFin4-scaleVec4 $\mathbb{Z}$  a (block2 v))))))
    fold : a * $\mathbb{Z}$  (s0 + $\mathbb{Z}$  (s1 + $\mathbb{Z}$  s2))  $\equiv$  (a * $\mathbb{Z}$  s0) + $\mathbb{Z}$  ((a * $\mathbb{Z}$  s1) + $\mathbb{Z}$  (a * $\mathbb{Z}$  s2))
    fold =
      trans
      (* $\mathbb{Z}$ -distrib-right-+ $\mathbb{Z}$  a s0 (s1 + $\mathbb{Z}$  s2))
      (cong ( $\lambda$  t  $\rightarrow$  (a * $\mathbb{Z}$  s0) + $\mathbb{Z}$  t) (* $\mathbb{Z}$ -distrib-right-+ $\mathbb{Z}$  a s1 s2))
  in
  trans stepBlock (sym fold)

```

Twelve-Times Commutativity

Because J replaces a vector by a constant vector with repeated entry $\Sigma_{12}v$, compositions in the span inevitably produce repeated copies of the same scalar. The four-times, eight-times, and twelve-times lemmas show that scalar multiplication passes through these repetitions without changing the value. This fixes the formula for $\Sigma_{12}(\text{linIJ } a \ b \ v)$ in a single normal form.

```

-- § x * fourTimes y = fourTimes (x*y)
* $\mathbb{Z}$ -fourTimes-right : (x y :  $\mathbb{Z}$ )  $\rightarrow$  x * $\mathbb{Z}$  fourTimes $\mathbb{Z}$  y  $\equiv$  fourTimes $\mathbb{Z}$  (x * $\mathbb{Z}$  y)
* $\mathbb{Z}$ -fourTimes-right x y =
  trans
  (* $\mathbb{Z}$ -distrib-right-+ $\mathbb{Z}$  x y (y + $\mathbb{Z}$  (y + $\mathbb{Z}$  y)))
  (cong ( $\lambda$  t  $\rightarrow$  (x * $\mathbb{Z}$  y) + $\mathbb{Z}$  t)
  (trans
   (* $\mathbb{Z}$ -distrib-right-+ $\mathbb{Z}$  x y (y + $\mathbb{Z}$  y))
   (cong ( $\lambda$  t  $\rightarrow$  (x * $\mathbb{Z}$  y) + $\mathbb{Z}$  t)
   (* $\mathbb{Z}$ -distrib-right-+ $\mathbb{Z}$  x y y))))

```

```

-- § fourTimes x * y = fourTimes (x*y)
*ℤ-fourTimes-left : (x y : ℤ) → fourTimesℤ x *ℤ y ≡ fourTimesℤ (x *ℤ y)
*ℤ-fourTimes-left x y =
  trans
    (*ℤ-distrib-left+ℤ x (x +ℤ (x +ℤ x)) y)
    (cong (λ t → (x *ℤ y) +ℤ t)
      (trans
        (*ℤ-distrib-left+ℤ x (x +ℤ x) y)
        (cong (λ t → (x *ℤ y) +ℤ t)
          (*ℤ-distrib-left+ℤ x x y))))

-- § x * eightTimes y = eightTimes (x*y)
*ℤ-eightTimes-right : (x y : ℤ) → x *ℤ eightTimesℤ y ≡ eightTimesℤ (x *ℤ y)
*ℤ-eightTimes-right x y =
  trans
    (*ℤ-distrib-right+ℤ x (fourTimesℤ y) (fourTimesℤ y))
    (cong (λ t → t +ℤ t) (*ℤ-fourTimes-right x y))

-- § eightTimes x * y = eightTimes (x*y)
*ℤ-eightTimes-left : (x y : ℤ) → eightTimesℤ x *ℤ y ≡ eightTimesℤ (x *ℤ y)
*ℤ-eightTimes-left x y =
  trans
    (*ℤ-distrib-left+ℤ (fourTimesℤ x) (fourTimesℤ x) y)
    (cong (λ t → t +ℤ t) (*ℤ-fourTimes-left x y))

-- § x * twelveTimes y = twelveTimes (x*y)
*ℤ-twelveTimes-right : (x y : ℤ) → x *ℤ twelveTimesℤ y ≡ twelveTimesℤ (x *ℤ y)
*ℤ-twelveTimes-right x y =
  trans
    (*ℤ-distrib-right+ℤ x (fourTimesℤ y) (eightTimesℤ y))
    (trans
      (cong (λ t → t +ℤ x *ℤ eightTimesℤ y) (*ℤ-fourTimes-right x y))
      (cong (λ t → fourTimesℤ (x *ℤ y) +ℤ t) (*ℤ-eightTimes-right x y)))

-- § twelveTimes x * y = twelveTimes (x*y)
*ℤ-twelveTimes-left : (x y : ℤ) → twelveTimesℤ x *ℤ y ≡ twelveTimesℤ (x *ℤ y)
*ℤ-twelveTimes-left x y =
  trans
    (*ℤ-distrib-left+ℤ (fourTimesℤ x) (eightTimesℤ x) y)
    (trans
      (cong (λ t → t +ℤ eightTimesℤ x *ℤ y) (*ℤ-fourTimes-left x y))
      (cong (λ t → fourTimesℤ (x *ℤ y) +ℤ t) (*ℤ-eightTimes-left x y)))

-- § x * twelveTimes y = twelveTimes x * y
mul-twelveShift : (x y : ℤ) → x *ℤ twelveTimesℤ y ≡ twelveTimesℤ x *ℤ y
mul-twelveShift x y = trans (*ℤ-twelveTimes-right x y) (sym (*ℤ-twelveTimes-left x y))

-- § sum of an (I,J)-combination
sum12-linI : (a b : ℤ) → (v : Vec12ℤ) →
  sum12ℤ (linI a b v) ≡ (a *ℤ sum12ℤ v) +ℤ (b *ℤ twelveTimesℤ (sum12ℤ v))
sum12-linI a b v =
  let
    s = sum12ℤ v
  step1 : sum12ℤ (linI a b v)

```

```

    ≡ sum12 $\mathbb{Z}$  (scaleVec12 $\mathbb{Z}$  a v) +  $\mathbb{Z}$  sum12 $\mathbb{Z}$  (scaleVec12 $\mathbb{Z}$  b (J12Vec12 $\mathbb{Z}$  v))
step1 = sum12+Vec12 $\mathbb{Z}$  (scaleVec12 $\mathbb{Z}$  a v) (scaleVec12 $\mathbb{Z}$  b (J12Vec12 $\mathbb{Z}$  v))

step2 : sum12 $\mathbb{Z}$  (scaleVec12 $\mathbb{Z}$  a v) ≡ a *  $\mathbb{Z}$  s
step2 = sum12-scaleVec12 $\mathbb{Z}$  a v

step3 : sum12 $\mathbb{Z}$  (scaleVec12 $\mathbb{Z}$  b (J12Vec12 $\mathbb{Z}$  v)) ≡ b *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  (J12Vec12 $\mathbb{Z}$  v)
step3 = sum12-scaleVec12 $\mathbb{Z}$  b (J12Vec12 $\mathbb{Z}$  v)

step4 : b *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  (J12Vec12 $\mathbb{Z}$  v) ≡ b *  $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  s
step4 = cong (λ t → b *  $\mathbb{Z}$  t) (sum12-J12 v)
in
trans
step1
(trans
  (cong (λ t → t +  $\mathbb{Z}$  sum12 $\mathbb{Z}$  (scaleVec12 $\mathbb{Z}$  b (J12Vec12 $\mathbb{Z}$  v)))) step2)
  (cong (λ t → (a *  $\mathbb{Z}$  s) +  $\mathbb{Z}$  t) (trans step3 step4)))

```

Delta Vectors

Coefficient extraction requires test vectors with concentrated support and completely controlled total sum. The delta family provides exactly this. Since it is defined by exhaustive pattern matching, both the support and the total sum are explicit, and evaluating `linIJ` on the lifted `Vec12 $\mathbb{Z}$` version becomes a direct calculation.

```

-- § single-coordinate delta vector on Fin4 (16 pattern cases)
delta4 : Fin4 →  $\mathbb{Z}$  → Vec4 $\mathbb{Z}$ 
delta4 g0 x g0 = x
delta4 g0 x g1 = 0 $\mathbb{Z}$ 
delta4 g0 x g2 = 0 $\mathbb{Z}$ 
delta4 g0 x g3 = 0 $\mathbb{Z}$ 

delta4 g1 x g0 = 0 $\mathbb{Z}$ 
delta4 g1 x g1 = x
delta4 g1 x g2 = 0 $\mathbb{Z}$ 
delta4 g1 x g3 = 0 $\mathbb{Z}$ 

delta4 g2 x g0 = 0 $\mathbb{Z}$ 
delta4 g2 x g1 = 0 $\mathbb{Z}$ 
delta4 g2 x g2 = x
delta4 g2 x g3 = 0 $\mathbb{Z}$ 

delta4 g3 x g0 = 0 $\mathbb{Z}$ 
delta4 g3 x g1 = 0 $\mathbb{Z}$ 
delta4 g3 x g2 = 0 $\mathbb{Z}$ 
delta4 g3 x g3 = x

-- § sumFin4 of delta is identity
sumFin4-delta4 : (i : Fin4) → (x :  $\mathbb{Z}$ ) → sumFin4 $\mathbb{Z}$  (delta4 i x) ≡ x
sumFin4-delta4 g0 x =
trans

```



```

(cong (λ t → x + ℤ t)
  (trans
    (+ℤ-zero-left (0ℤ + ℤ 0ℤ))
    (+ℤ-zero-left 0ℤ)))
(+ℤ-zero-right x)
sumFin4-delta4 g1 x =
trans
  (+ℤ-zero-left (x + ℤ (0ℤ + ℤ 0ℤ)))
  (trans
    (cong (λ t → x + ℤ t) (+ℤ-zero-left 0ℤ))
    (+ℤ-zero-right x))
sumFin4-delta4 g2 x =
trans
  (+ℤ-zero-left (0ℤ + ℤ (x + ℤ 0ℤ)))
  (trans
    (+ℤ-zero-left (x + ℤ 0ℤ))
    (+ℤ-zero-right x))
sumFin4-delta4 g3 x =
trans
  (+ℤ-zero-left (0ℤ + ℤ (0ℤ + ℤ x)))
  (trans
    (+ℤ-zero-left (0ℤ + ℤ x))
    (+ℤ-zero-left x))

-- § delta vector on Vec12ℤ supported at (block0, g0)
delta12 : ℤ → Vec12ℤ
delta12 x = delta4 g0 x , (delta4 g0 0ℤ , delta4 g0 0ℤ)

-- § sum12 of delta12 is identity
sum12-delta12 : (x : ℤ) → sum12ℤ (delta12 x) ≡ x
sum12-delta12 x =
trans
  (cong (λ t → t + ℤ (sumFin4ℤ (delta4 g0 0ℤ) + ℤ sumFin4ℤ (delta4 g0 0ℤ)))
    (sumFin4-delta4 g0 x))
  (trans
    (cong (λ t → x + ℤ t)
      (cong (λ t → t + ℤ sumFin4ℤ (delta4 g0 0ℤ)) (sumFin4-delta4 g0 0ℤ)))
    (trans
      (cong (λ t → x + ℤ (0ℤ + ℤ t)) (sumFin4-delta4 g0 0ℤ))
      (trans
        (cong (λ t → x + ℤ t) (+ℤ-zero-left 0ℤ))
        (+ℤ-zero-right x))))

-- § J12 on delta12 is constant vector
J12-delta12-const : (x : ℤ) → Vec12Eq (J12Vec12ℤ (delta12 x)) (constVec12ℤ x)
J12-delta12-const x =
let p = sum12-delta12 x in
(λ _ → p) , ((λ _ → p) , (λ _ → p))

```

Forced Independence and Uniqueness

With the delta vector in place, independence is immediate. Evaluating at a coordinate where the delta vanishes removes the I -term and isolates the coefficient of J ; evaluating at the distinguished nonzero coordinate then recovers the coefficient of I . The uniqueness theorem repeats the same argument on the difference of two representations.

```
-- § Law 14N.0: (a·I + b·J) = 0 forces a = 0 and b = 0
law14N-0-IJ-independent : (a b :  $\mathbb{Z}$ ) → OpEq (linI a b) zeroOp → (a  $\equiv$  0 $\mathbb{Z}$ ) × (b  $\equiv$  0 $\mathbb{Z}$ )
law14N-0-IJ-independent a b hyp = a0 , b0
where
v : Vec12 $\mathbb{Z}$ 
v = delta12 one $\mathbb{Z}$ 

pSum : sum12 $\mathbb{Z}$  v  $\equiv$  one $\mathbb{Z}$ 
pSum = sum12-delta12 one $\mathbb{Z}$ 

-- § at g1 the I-term vanishes, extracting b
eqQraw : block0 (linI a b v) g1  $\equiv$  block0 (zeroOp v) g1
eqQraw = fst (hyp v) g1

eqQ0 : (a * $\mathbb{Z}$  0 $\mathbb{Z}$ ) + $\mathbb{Z}$  (b * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)  $\equiv$  0 $\mathbb{Z}$ 
eqQ0 = trans (sym (block0-linI a b v g1)) eqQraw

eqQ1 : (a * $\mathbb{Z}$  0 $\mathbb{Z}$ ) + $\mathbb{Z}$  (b * $\mathbb{Z}$  one $\mathbb{Z}$ )  $\equiv$  0 $\mathbb{Z}$ 
eqQ1 = trans (sym (cong ( $\lambda$  t → (a * $\mathbb{Z}$  0 $\mathbb{Z}$ ) + $\mathbb{Z}$  (b * $\mathbb{Z}$  t)) pSum)) eqQ0

eqQ2 : 0 $\mathbb{Z}$  + $\mathbb{Z}$  (b * $\mathbb{Z}$  one $\mathbb{Z}$ )  $\equiv$  0 $\mathbb{Z}$ 
eqQ2 =
trans
(sym (cong ( $\lambda$  t → t + $\mathbb{Z}$  (b * $\mathbb{Z}$  one $\mathbb{Z}$ )) (* $\mathbb{Z}$ -zero-right a)))
eqQ1

bAtQ : (b * $\mathbb{Z}$  one $\mathbb{Z}$ )  $\equiv$  0 $\mathbb{Z}$ 
bAtQ = trans (sym (+ $\mathbb{Z}$ -zero-left (b * $\mathbb{Z}$  one $\mathbb{Z}$ ))) eqQ2

b0 : b  $\equiv$  0 $\mathbb{Z}$ 
b0 = trans (sym (* $\mathbb{Z}$ -one-right b)) bAtQ

-- § at g0 the I-term gives a, extracting a
eqPraw : block0 (linI a b v) g0  $\equiv$  block0 (zeroOp v) g0
eqPraw = fst (hyp v) g0

eqP0 : (a * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  (b * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)  $\equiv$  0 $\mathbb{Z}$ 
eqP0 = trans (sym (block0-linI a b v g0)) eqPraw

eqP1 : (a * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  (b * $\mathbb{Z}$  one $\mathbb{Z}$ )  $\equiv$  0 $\mathbb{Z}$ 
eqP1 = trans (sym (cong ( $\lambda$  t → (a * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  (b * $\mathbb{Z}$  t)) pSum)) eqP0

eqP2 : (a * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  (0 $\mathbb{Z}$  * $\mathbb{Z}$  one $\mathbb{Z}$ )  $\equiv$  0 $\mathbb{Z}$ 
eqP2 =
trans
```

```

(sym (cong (λ t → (a *ℤ oneℤ) +ℤ (t *ℤ oneℤ)) b0))
eqP1

eqP3 : (a *ℤ oneℤ) +ℤ 0ℤ ≡ 0ℤ
eqP3 =
  trans
    (sym (cong (λ t → (a *ℤ oneℤ) +ℤ t) (*ℤ-zero-left oneℤ)))
    eqP2

aAtP : (a *ℤ oneℤ) ≡ 0ℤ
aAtP = trans (sym (*ℤ-zero-right (a *ℤ oneℤ))) eqP3

a0 : a ≡ 0ℤ
a0 = trans (sym (*ℤ-one-right a)) aAtP

-- § additive right cancellation
+ℤ-cancel-right : (a c b : ℤ) → a +ℤ b ≡ c +ℤ b → a ≡ c
+ℤ-cancel-right a c b eq =
  let eq' = cong (λ t → negℤ b +ℤ t) eq in
  trans
    (sym (reduce a))
    (trans eq' (reduce c))
  where
    reduce : (x : ℤ) → negℤ b +ℤ (x +ℤ b) ≡ x
    reduce x =
      trans
        (swapHeadℤ (negℤ b) x b)
        (trans
          (cong (λ t → x +ℤ t) (+ℤ-inv-left b))
          (+ℤ-zero-right x))

```

Uniqueness of (I, J) -representation. With linear independence established, equality of coefficient pairs follows by passing to the difference operator and applying the same extraction argument.

```

-- § Law 14N.1: (a·I + b·J) = (c·I + d·J) forces a = c and b = d
law14N-1-IJ-unique : (a b c d : ℤ) → OpEq (linI a b) (linI c d) → (a ≡ c) × (b ≡ d)
law14N-1-IJ-unique a b c d hyp = aEq , bEq
  where
    v : Vec12ℤ
    v = delta12 oneℤ

pSum : sum12ℤ v ≡ oneℤ
pSum = sum12-delta12 oneℤ

-- § extract b = d from g1 evaluation
eqQraw : block0 (linI a b v) g1 ≡ block0 (linI c d v) g1
eqQraw = fst (hyp v) g1

eqQ0 : (a *ℤ 0ℤ) +ℤ (b *ℤ sum12ℤ v) ≡ (c *ℤ 0ℤ) +ℤ (d *ℤ sum12ℤ v)
eqQ0 =
  trans (sym (block0-linI a b v g1))

```

```

(trans eqQraw (block0-linI c d v g1))

eqQ1a : (a * $\mathbb{Z}$  0 $\mathbb{Z}$ ) + $\mathbb{Z}$  (b * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)  $\equiv$  0 $\mathbb{Z}$  + $\mathbb{Z}$  (b * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)
eqQ1a = cong ( $\lambda$  t  $\rightarrow$  t + $\mathbb{Z}$  (b * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)) (* $\mathbb{Z}$ -zero-right a)

eqQ1c : (c * $\mathbb{Z}$  0 $\mathbb{Z}$ ) + $\mathbb{Z}$  (d * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)  $\equiv$  0 $\mathbb{Z}$  + $\mathbb{Z}$  (d * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)
eqQ1c = cong ( $\lambda$  t  $\rightarrow$  t + $\mathbb{Z}$  (d * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)) (* $\mathbb{Z}$ -zero-right c)

eqQ1 : 0 $\mathbb{Z}$  + $\mathbb{Z}$  (b * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)  $\equiv$  0 $\mathbb{Z}$  + $\mathbb{Z}$  (d * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)
eqQ1 = trans (sym eqQ1a) (trans eqQ0 eqQ1c)

eqQ2 : 0 $\mathbb{Z}$  + $\mathbb{Z}$  (b * $\mathbb{Z}$  one $\mathbb{Z}$ )  $\equiv$  0 $\mathbb{Z}$  + $\mathbb{Z}$  (d * $\mathbb{Z}$  one $\mathbb{Z}$ )
eqQ2 =
trans
  (cong ( $\lambda$  t  $\rightarrow$  0 $\mathbb{Z}$  + $\mathbb{Z}$  (b * $\mathbb{Z}$  t)) pSum)
  (trans eqQ1 (sym (cong ( $\lambda$  t  $\rightarrow$  0 $\mathbb{Z}$  + $\mathbb{Z}$  (d * $\mathbb{Z}$  t)) pSum)))

bEq' : (b * $\mathbb{Z}$  one $\mathbb{Z}$ )  $\equiv$  (d * $\mathbb{Z}$  one $\mathbb{Z}$ )
bEq' =
trans
  (sym (+ $\mathbb{Z}$ -zero-left (b * $\mathbb{Z}$  one $\mathbb{Z}$ )))
  (trans eqQ2 (+ $\mathbb{Z}$ -zero-left (d * $\mathbb{Z}$  one $\mathbb{Z}$ )))

bEq : b  $\equiv$  d
bEq =
trans (sym (* $\mathbb{Z}$ -one-right b))
  (trans bEq' (* $\mathbb{Z}$ -one-right d))

-- § extract a = c from g0 evaluation with right cancellation
eqPraw : block0 (linI a b v) g0  $\equiv$  block0 (linI c d v) g0
eqPraw = fst (hyp v) g0

eqP0 : (a * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  (b * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)  $\equiv$  (c * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  (d * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)
eqP0 =
trans (sym (block0-linI a b v g0))
  (trans eqPraw (block0-linI c d v g0))

eqP1 : (a * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  (b * $\mathbb{Z}$  one $\mathbb{Z}$ )  $\equiv$  (c * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  (d * $\mathbb{Z}$  one $\mathbb{Z}$ )
eqP1 =
trans (cong ( $\lambda$  t  $\rightarrow$  (a * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  (b * $\mathbb{Z}$  t)) pSum)
  (trans eqP0 (sym (cong ( $\lambda$  t  $\rightarrow$  (c * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  (d * $\mathbb{Z}$  t)) pSum)))

eqP2 : (a * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  (b * $\mathbb{Z}$  one $\mathbb{Z}$ )  $\equiv$  (c * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  (b * $\mathbb{Z}$  one $\mathbb{Z}$ )
eqP2 =
trans
  eqP1
  (cong ( $\lambda$  t  $\rightarrow$  (c * $\mathbb{Z}$  one $\mathbb{Z}$ ) + $\mathbb{Z}$  t)
    (cong ( $\lambda$  z  $\rightarrow$  z * $\mathbb{Z}$  one $\mathbb{Z}$ ) (sym bEq)))

aEq' : (a * $\mathbb{Z}$  one $\mathbb{Z}$ )  $\equiv$  (c * $\mathbb{Z}$  one $\mathbb{Z}$ )
aEq' = + $\mathbb{Z}$ -cancel-right (a * $\mathbb{Z}$  one $\mathbb{Z}$ ) (c * $\mathbb{Z}$  one $\mathbb{Z}$ ) (b * $\mathbb{Z}$  one $\mathbb{Z}$ ) eqP2

aEq : a  $\equiv$  c
aEq =
trans (sym (* $\mathbb{Z}$ -one-right a))
  (trans aEq' (* $\mathbb{Z}$ -one-right c))

```

Packaging and Composition Closure

At this stage the normal form can be packaged explicitly: the span is just $\mathbb{Z} \times \mathbb{Z}$, interpreted by sending (a, b) to the operator $aI + bJ$. The earlier sum and scaling identities then determine composition completely: the I -parts multiply, the mixed terms add linearly, and the double J -term collapses through $J^2 = 12J$.

```
-- § the forced (I,J) normal form
SpanIJ : Set
SpanIJ =  $\mathbb{Z} \times \mathbb{Z}$ 

-- § interpretation into operators
interpIJ : SpanIJ → Op
interpIJ ab = linIJ (fst ab) (snd ab)

-- § injectivity of interpretation
interpIJ-injective : (p q : SpanIJ) → OpEq (interpIJ p) (interpIJ q) → p ≡ q
interpIJ-injective (a, b) (c, d) eq =
  let res = law14N-1-IJ-unique a b c d eq in
  pair-ext (fst res) (snd res)
where
  pair-ext : {a b c d :  $\mathbb{Z}$ } → a ≡ c → b ≡ d → (a, b) ≡ (c, d)
  pair-ext refl refl = refl

-- § Law 14N.2: image witness in span is forced unique
law14N-2-image-witness-unique :
  (f : Op) →
  (w1 w2 :  $\Sigma$  SpanIJ (λ p → OpEq f (interpIJ p))) →
  fst w1 ≡ fst w2
law14N-2-image-witness-unique f (p1, eq1) (p2, eq2) =
  interpIJ-injective p1 p2 (λ v → Vec12Eq-trans (Vec12Eq-sym (eq1 v)) (eq2 v))

-- § multiplication rule for SpanIJ
mulSpanIJ : SpanIJ → SpanIJ → SpanIJ
mulSpanIJ (a, b) (c, d) = (a * $\mathbb{Z}$  c, (((a * $\mathbb{Z}$  d) + $\mathbb{Z}$  (b * $\mathbb{Z}$  c)) + $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  (b * $\mathbb{Z}$  d)))

Composition closure. The  $(I, J)$ -span is therefore closed under composition, with
the multiplication rule forced by  $J^2 = 12J$ .

-- § Law 14N.3: (I,J)-span is closed under composition
law14N-3-IJ-compose-closed : (p q : SpanIJ) → OpEq (λ v → interpIJ p (interpIJ q v)) (interpIJ (mulSpanIJ p q))
law14N-3-IJ-compose-closed (a, b) (c, d) v = eq0, (eq1, eq2)
where
  s :  $\mathbb{Z}$ 
  s = sum12 $\mathbb{Z}$  v

  w : Vec12 $\mathbb{Z}$ 
  w = linIJ c d v

  sw : sum12 $\mathbb{Z}$  w ≡ (c * $\mathbb{Z}$  s) + $\mathbb{Z}$  (d * $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  s)
  sw = sum12-linIJ c d v
```

```

b' :  $\mathbb{Z}$ 
b' = ((a *  $\mathbb{Z}$  d) +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  c)) +  $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  (b *  $\mathbb{Z}$  d)

-- § generic block equality helper
blkEq :
  (blk : Vec12 $\mathbb{Z}$  → Vec4 $\mathbb{Z}$ ) →
  ((x y :  $\mathbb{Z}$ ) → (u : Vec12 $\mathbb{Z}$ ) → (i : Fin4) → blk (linJ x y u) i  $\equiv$  (x *  $\mathbb{Z}$  blk u i) +  $\mathbb{Z}$  (y *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  u)) →
  (i : Fin4) →
  blk (linJ a b w) i  $\equiv$  blk (linJ (a *  $\mathbb{Z}$  c) b' v) i
blkEq blk blk-lin i =
  let
    vi = blk v i

  lhsForm : blk (linJ a b w) i  $\equiv$  (a *  $\mathbb{Z}$  blk w i) +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  w)
  lhsForm = blk-lin a b w i

  blkW : blk w i  $\equiv$  (c *  $\mathbb{Z}$  vi) +  $\mathbb{Z}$  (d *  $\mathbb{Z}$  s)
  blkW = blk-lin c d v i

  rhsForm : blk (linJ (a *  $\mathbb{Z}$  c) b' v) i  $\equiv$  ((a *  $\mathbb{Z}$  c) *  $\mathbb{Z}$  vi) +  $\mathbb{Z}$  (b' *  $\mathbb{Z}$  s)
  rhsForm = blk-lin (a *  $\mathbb{Z}$  c) b' v i

  step1 : blk (linJ a b w) i  $\equiv$  (a *  $\mathbb{Z}$  ((c *  $\mathbb{Z}$  vi) +  $\mathbb{Z}$  (d *  $\mathbb{Z}$  s))) +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  w)
  step1 =
    trans
      lhsForm
      (cong (λ t → (a *  $\mathbb{Z}$  t) +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  w)) blkW)

  step2 : (a *  $\mathbb{Z}$  ((c *  $\mathbb{Z}$  vi) +  $\mathbb{Z}$  (d *  $\mathbb{Z}$  s))) +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  w)
     $\equiv$  ((a *  $\mathbb{Z}$  (c *  $\mathbb{Z}$  vi)) +  $\mathbb{Z}$  (a *  $\mathbb{Z}$  (d *  $\mathbb{Z}$  s))) +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  w)
  step2 = cong (λ t → t +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  w)) (* $\mathbb{Z}$ -distrib-right-+ $\mathbb{Z}$  a (c *  $\mathbb{Z}$  vi) (d *  $\mathbb{Z}$  s))

  assocAC : a *  $\mathbb{Z}$  (c *  $\mathbb{Z}$  vi)  $\equiv$  (a *  $\mathbb{Z}$  c) *  $\mathbb{Z}$  vi
  assocAC = sym (* $\mathbb{Z}$ -assoc a c vi)

  assocAD : a *  $\mathbb{Z}$  (d *  $\mathbb{Z}$  s)  $\equiv$  (a *  $\mathbb{Z}$  d) *  $\mathbb{Z}$  s
  assocAD = sym (* $\mathbb{Z}$ -assoc a d s)

  step3 : (((a *  $\mathbb{Z}$  (c *  $\mathbb{Z}$  vi)) +  $\mathbb{Z}$  (a *  $\mathbb{Z}$  (d *  $\mathbb{Z}$  s))) +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  w))
     $\equiv$  (((a *  $\mathbb{Z}$  c) *  $\mathbb{Z}$  vi) +  $\mathbb{Z}$  ((a *  $\mathbb{Z}$  d) *  $\mathbb{Z}$  s)) +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  w)
  step3 =
    cong (λ t → t +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  w))
      (trans
        (cong (λ t → t +  $\mathbb{Z}$  (a *  $\mathbb{Z}$  (d *  $\mathbb{Z}$  s))) assocAC)
        (cong (λ t → ((a *  $\mathbb{Z}$  c) *  $\mathbb{Z}$  vi) +  $\mathbb{Z}$  t) assocAD))

  step4 : (((a *  $\mathbb{Z}$  c) *  $\mathbb{Z}$  vi) +  $\mathbb{Z}$  ((a *  $\mathbb{Z}$  d) *  $\mathbb{Z}$  s)) +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  sum12 $\mathbb{Z}$  w)
     $\equiv$  (((a *  $\mathbb{Z}$  c) *  $\mathbb{Z}$  vi) +  $\mathbb{Z}$  ((a *  $\mathbb{Z}$  d) *  $\mathbb{Z}$  s)) +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  ((c *  $\mathbb{Z}$  s) +  $\mathbb{Z}$  (d *  $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  s)))
  step4 = cong (λ t → (((a *  $\mathbb{Z}$  c) *  $\mathbb{Z}$  vi) +  $\mathbb{Z}$  ((a *  $\mathbb{Z}$  d) *  $\mathbb{Z}$  s)) +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  t)) sw

  step5 : b *  $\mathbb{Z}$  ((c *  $\mathbb{Z}$  s) +  $\mathbb{Z}$  (d *  $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  s))
     $\equiv$  (b *  $\mathbb{Z}$  (c *  $\mathbb{Z}$  s)) +  $\mathbb{Z}$  (b *  $\mathbb{Z}$  (d *  $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  s))
  step5 = * $\mathbb{Z}$ -distrib-right-+ $\mathbb{Z}$  b (c *  $\mathbb{Z}$  s) (d *  $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  s)

```

```

step6 : b *ℤ (c *ℤ s) ≡ (b *ℤ c) *ℤ s
step6 = sym (*ℤ-assoc b c s)

step7 : b *ℤ (d *ℤ twelveTimesℤ s) ≡ (twelveTimesℤ (b *ℤ d)) *ℤ s
step7 =
  trans
    (sym (*ℤ-assoc b d (twelveTimesℤ s)))
    (mul-twelveShift (b *ℤ d) s)

step8 : b *ℤ ((c *ℤ s) +ℤ (d *ℤ twelveTimesℤ s))
      ≡ ((b *ℤ c) *ℤ s) +ℤ ((twelveTimesℤ (b *ℤ d)) *ℤ s)
step8 =
  trans
    step5
    (trans
      (cong (λ t → t +ℤ (b *ℤ (d *ℤ twelveTimesℤ s))) step6)
      (cong (λ t → ((b *ℤ c) *ℤ s) +ℤ t) step7))

step9 : (((a *ℤ c) *ℤ vi) +ℤ ((a *ℤ d) *ℤ s)) +ℤ (b *ℤ ((c *ℤ s) +ℤ (d *ℤ twelveTimesℤ s)))
      ≡ (((a *ℤ c) *ℤ vi) +ℤ ((a *ℤ d) *ℤ s)) +ℤ (((b *ℤ c) *ℤ s) +ℤ ((twelveTimesℤ (b *ℤ d)) *ℤ s))
step9 = cong (λ t → (((a *ℤ c) *ℤ vi) +ℤ ((a *ℤ d) *ℤ s)) +ℤ t) step8

X = (a *ℤ c) *ℤ vi
Y = (a *ℤ d) *ℤ s
Z = ((b *ℤ c) *ℤ s) +ℤ ((twelveTimesℤ (b *ℤ d)) *ℤ s)

step10 : (X +ℤ Y) +ℤ Z ≡ X +ℤ (Y +ℤ Z)
step10 = +ℤ-assoc X Y Z

twelveTerm = (twelveTimesℤ (b *ℤ d)) *ℤ s
y2 = (b *ℤ c) *ℤ s

fold1 : Y +ℤ y2 ≡ ((a *ℤ d) +ℤ (b *ℤ c)) *ℤ s
fold1 = sym (*ℤ-distrib-left+ℤ (a *ℤ d) (b *ℤ c) s)

fold2 : (((a *ℤ d) +ℤ (b *ℤ c)) *ℤ s) +ℤ twelveTerm ≡ b' *ℤ s
fold2 =
  trans
    (sym (*ℤ-distrib-left+ℤ ((a *ℤ d) +ℤ (b *ℤ c)) (twelveTimesℤ (b *ℤ d)) s))
    refl

innerFold : Y +ℤ (y2 +ℤ twelveTerm) ≡ b' *ℤ s
innerFold =
  trans
    (sym (+ℤ-assoc Y y2 twelveTerm))
    (trans
      (cong (λ t → t +ℤ twelveTerm) fold1)
      fold2)
in
let
  pA : blk (linl a b w) i ≡ (X +ℤ Y) +ℤ (b *ℤ sum12ℤ w)
  pA = trans step1 (trans step2 step3)

  pB : blk (linl a b w) i ≡ (X +ℤ Y) +ℤ (b *ℤ ((c *ℤ s) +ℤ (d *ℤ twelveTimesℤ s)))
  pB = trans pA step4

```

```

pC : blk (linJ a b w) i ≡ (X +ℤ Y) +ℤ Z
pC = trans pB (cong (λ t → (X +ℤ Y) +ℤ t) steps)

pD : blk (linJ a b w) i ≡ X +ℤ (Y +ℤ Z)
pD = trans pC (+ℤ-assoc X Y Z)

pE : blk (linJ a b w) i ≡ X +ℤ (b' *ℤ s)
pE = trans pD (cong (λ t → X +ℤ t) innerFold)
in
trans pE (sym rhsForm)

eq0 : (i : Fin4) → block0 (linJ a b w) i ≡ block0 (linJ (a *ℤ c) b' v) i
eq0 = blkEq block0 (λ x y u i → block0-linJ x y u i)

eq1 : (i : Fin4) → block1 (linJ a b w) i ≡ block1 (linJ (a *ℤ c) b' v) i
eq1 = blkEq block1 (λ x y u i → block1-linJ x y u i)

eq2 : (i : Fin4) → block2 (linJ a b w) i ≡ block2 (linJ (a *ℤ c) b' v) i
eq2 = blkEq block2 (λ x y u i → block2-linJ x y u i)

```


Chapter 52

Forced Spectral Action of the (I, J) -Span

The (I, J) -span from Chapter 51 already fixes a normal form for every operator in that algebra. This chapter extracts the corresponding spectral action: on sum-zero vectors, $aI + bJ$ acts by the scalar a , while on constant vectors it acts by $a + 12b$. In particular, the K_{12} Laplacian appears as the span element $(12, -1)$, and the familiar eigenvalue split follows from this description.

Forced Predicates on $\text{Vec}_{12}\mathbb{Z}$

The two invariant classes needed later are recorded in witness-bearing form. The zero-sum condition is the equation $\Sigma_{12}v \equiv 0$, while constancy remembers both the scalar c and the pointwise identification of v with the constant vector of value c .

-- § Sum-zero and constant predicate witnesses on $\text{Vec}_{12}\mathbb{Z}$

$\text{ZeroSumVec12} : \text{Vec12}\mathbb{Z} \rightarrow \text{Set}$

$\text{ZeroSumVec12 } v = \text{sum12}\mathbb{Z} \ v \equiv 0\mathbb{Z}$

$\text{ConstVec12} : \text{Vec12}\mathbb{Z} \rightarrow \text{Set}$

$\text{ConstVec12 } v = \Sigma \ \mathbb{Z} \ (\lambda \ c \rightarrow \text{Vec12Eq } v \ (\text{constVec12}\mathbb{Z} \ c))$

Congruence of linIJ and interpIJ

Whenever a result is transported from one presentation of a vector to another, the operator $aI + bJ$ must give the same output on pointwise equal inputs. Equality of the blocks together with equality of the total sum yields congruence for linIJ , and the statement for interpIJ follows immediately.

```

-- § linIJ respects Vec12Eq
linIJ-cong : (a b : ℤ) → (u v : Vec12ℤ) → Vec12Eq u v → Vec12Eq (linIJ a b u) (linIJ a b v)
linIJ-cong a b u v eq = eq0 , (eq1 , eq2)
where
  sEq : sum12ℤ u ≡ sum12ℤ v
  sEq = sum12-cong u v eq

  eq0 : (i : Fin4) → block0 (linIJ a b u) i ≡ block0 (linIJ a b v) i
  eq0 i =
    let
      pA : a *ℤ block0 u i ≡ a *ℤ block0 v i
      pA = cong (λ t → a *ℤ t) (fst eq i)

      pB : b *ℤ sum12ℤ u ≡ b *ℤ sum12ℤ v
      pB = cong (λ t → b *ℤ t) sEq

      step1 : (a *ℤ block0 u i) +ℤ (b *ℤ sum12ℤ u) ≡ (a *ℤ block0 v i) +ℤ (b *ℤ sum12ℤ v)
      step1 = cong (λ t → t +ℤ (b *ℤ sum12ℤ u)) pA

      step2 : (a *ℤ block0 v i) +ℤ (b *ℤ sum12ℤ u) ≡ (a *ℤ block0 v i) +ℤ (b *ℤ sum12ℤ v)
      step2 = cong (λ t → (a *ℤ block0 v i) +ℤ t) pB
    in
      trans
        (block0-linIJ a b u i)
        (trans (trans step1 step2) (sym (block0-linIJ a b v i)))

  eq1 : (i : Fin4) → block1 (linIJ a b u) i ≡ block1 (linIJ a b v) i
  eq1 i =
    let
      pA : a *ℤ block1 u i ≡ a *ℤ block1 v i
      pA = cong (λ t → a *ℤ t) (fst (snd eq) i)

      pB : b *ℤ sum12ℤ u ≡ b *ℤ sum12ℤ v
      pB = cong (λ t → b *ℤ t) sEq

      step1 : (a *ℤ block1 u i) +ℤ (b *ℤ sum12ℤ u) ≡ (a *ℤ block1 v i) +ℤ (b *ℤ sum12ℤ v)
      step1 = cong (λ t → t +ℤ (b *ℤ sum12ℤ u)) pA

      step2 : (a *ℤ block1 v i) +ℤ (b *ℤ sum12ℤ u) ≡ (a *ℤ block1 v i) +ℤ (b *ℤ sum12ℤ v)
      step2 = cong (λ t → (a *ℤ block1 v i) +ℤ t) pB
    in
      trans
        (block1-linIJ a b u i)
        (trans (trans step1 step2) (sym (block1-linIJ a b v i)))

  eq2 : (i : Fin4) → block2 (linIJ a b u) i ≡ block2 (linIJ a b v) i
  eq2 i =
    let
      pA : a *ℤ block2 u i ≡ a *ℤ block2 v i
      pA = cong (λ t → a *ℤ t) (snd (snd eq) i)

      pB : b *ℤ sum12ℤ u ≡ b *ℤ sum12ℤ v
      pB = cong (λ t → b *ℤ t) sEq

      step1 : (a *ℤ block2 u i) +ℤ (b *ℤ sum12ℤ u) ≡ (a *ℤ block2 v i) +ℤ (b *ℤ sum12ℤ v)

```

```

step1 = cong (λ t → t + ℤ (b * ℤ sum12ℤ u)) pA

step2 : (a * ℤ block2 v i) + ℤ (b * ℤ sum12ℤ u) ≡ (a * ℤ block2 v i) + ℤ (b * ℤ sum12ℤ v)
step2 = cong (λ t → (a * ℤ block2 v i) + ℤ t) pB
in
trans
(block2-linI a b u i)
(trans (trans step1 step2) (sym (block2-linI a b v i)))

-- § interpIJ inherits congruence from linIJ
interpI-cong : (p : SpanI) → (u v : Vec12ℤ) → Vec12Eq u v → Vec12Eq (interpI p u) (interpI p v)
interpI-cong p = linI-cong (fst p) (snd p)

```

Forced J-Action

The action of J on the two invariant classes is immediate from its definition as the total sum broadcast to every coordinate. On a sum-zero vector it vanishes, and on a constant vector of value c it returns the constant vector of value $12c$.

```

-- § Law 140.0: sum-zero forces J-annihilation
law140-0-J-sum0 : (v : Vec12ℤ) → ZeroSumVec12 v → Vec12Eq (J12Vec12ℤ v) zeroVec12ℤ
law140-0-J-sum0 v sum0 =
  (λ _ → sum0) , ((λ _ → sum0) , (λ _ → sum0))

-- § Law 140.1: constant vectors force J-scaling by 12
law140-1-J-const : (c : ℤ) → Vec12Eq (J12Vec12ℤ (constVec12ℤ c)) (constVec12ℤ (twelveTimesℤ c))
law140-1-J-const c =
  (λ _ → sum12-const c) , ((λ _ → sum12-const c) , (λ _ → sum12-const c))

```

Forced Two-Eigenvalue Classification

From these formulas one reads the action of $aI + bJ$ on the two invariant classes. On a sum-zero vector the J -part disappears, so the operator acts by multiplication by a . On a constant vector of value c , the total sum contributes $12c$, and the operator acts by the scalar $a + 12b$.

```

-- § Law 140.2: sum-zero forces eigenvalue a for (a·I+b·J)
law140-2-linI-sum0-eigen : (a b : ℤ) → (v : Vec12ℤ) → ZeroSumVec12 v → Vec12Eq (linI a b v) (scaleVec12ℤ a v)
law140-2-linI-sum0-eigen a b v sum0 = eq0 , (eq1 , eq2)
where
kill : (x : ℤ) → x + ℤ (b * ℤ sum12ℤ v) ≡ x
kill x =
let
p0 : b * ℤ sum12ℤ v ≡ b * ℤ 0ℤ
p0 = cong (λ t → b * ℤ t) sum0

p1 : b * ℤ sum12ℤ v ≡ 0ℤ
p1 = trans p0 (*ℤ-zero-right b)

```

```

p2 : x + ℤ (b * ℤ sum12ℤ v) ≡ x + ℤ 0ℤ
p2 = cong (λ t → x + ℤ t) p1
in
trans p2 (+ℤ-zero-right x)

eq0 : (i : Fin4) → block0 (linJ a b v) i ≡ block0 (scaleVec12ℤ a v) i
eq0 i =
trans
(block0-linJ a b v i)
(kill (a * ℤ block0 v i))

eq1 : (i : Fin4) → block1 (linJ a b v) i ≡ block1 (scaleVec12ℤ a v) i
eq1 i =
trans
(block1-linJ a b v i)
(kill (a * ℤ block1 v i))

eq2 : (i : Fin4) → block2 (linJ a b v) i ≡ block2 (scaleVec12ℤ a v) i
eq2 i =
trans
(block2-linJ a b v i)
(kill (a * ℤ block2 v i))

-- § Law 140.3: constants force eigenvalue a+12b for (a·I+b·J)
law140-3-linJ-const-eigen : (a b c : ℤ) →
Vec12Eq (linJ a b (constVec12ℤ c)) (scaleVec12ℤ (a + ℤ twelveTimesℤ b) (constVec12ℤ c))
law140-3-linJ-const-eigen a b c = eq0 , (eq1 , eq2)
where
coord : (a b c : ℤ) → (a * ℤ c) + ℤ (b * ℤ twelveTimesℤ c) ≡ (a + ℤ twelveTimesℤ b) * ℤ c
coord a b c =
trans
(cong (λ t → (a * ℤ c) + ℤ t) (mul-twelveShift b c))
(sym (*ℤ-distrib-left+ℤ a (twelveTimesℤ b) c))

eq0 : (i : Fin4) →
block0 (linJ a b (constVec12ℤ c)) i ≡ block0 (scaleVec12ℤ (a + ℤ twelveTimesℤ b) (constVec12ℤ c)) i
eq0 i =
let
s0 : sum12ℤ (constVec12ℤ c) ≡ twelveTimesℤ c
s0 = sum12-const c

step0 a : (a * ℤ block0 (constVec12ℤ c) i) + ℤ (b * ℤ sum12ℤ (constVec12ℤ c))
≡ (a * ℤ c) + ℤ (b * ℤ sum12ℤ (constVec12ℤ c))
step0 a = refl

step0 b : (a * ℤ c) + ℤ (b * ℤ sum12ℤ (constVec12ℤ c))
≡ (a * ℤ c) + ℤ (b * ℤ twelveTimesℤ c)
step0 b = cong (λ t → (a * ℤ c) + ℤ (b * ℤ t)) s0

step0 : (a * ℤ block0 (constVec12ℤ c) i) + ℤ (b * ℤ sum12ℤ (constVec12ℤ c))
≡ (a * ℤ c) + ℤ (b * ℤ twelveTimesℤ c)
step0 = trans step0 a step0 b
in

```

```

trans
  (block0-linI) a b (constVec12ℤ c) i
  (trans step0 (coord a b c))

eq1 : (i : Fin4) →
  block1 (linI) a b (constVec12ℤ c) i ≡ block1 (scaleVec12ℤ (a + ℤ twelveTimesℤ b) (constVec12ℤ c)) i
eq1 i =
  let
    s0 : sum12ℤ (constVec12ℤ c) ≡ twelveTimesℤ c
    s0 = sum12-const c

    step0 a : (a * ℤ block1 (constVec12ℤ c) i) + ℤ (b * ℤ sum12ℤ (constVec12ℤ c))
            ≡ (a * ℤ c) + ℤ (b * ℤ sum12ℤ (constVec12ℤ c))
    step0 a = refl

    step0 b : (a * ℤ c) + ℤ (b * ℤ sum12ℤ (constVec12ℤ c))
            ≡ (a * ℤ c) + ℤ (b * ℤ twelveTimesℤ c)
    step0 b = cong (λ t → (a * ℤ c) + ℤ (b * ℤ t)) s0

    step0 : (a * ℤ block1 (constVec12ℤ c) i) + ℤ (b * ℤ sum12ℤ (constVec12ℤ c))
            ≡ (a * ℤ c) + ℤ (b * ℤ twelveTimesℤ c)
    step0 = trans step0 a step0 b
  in
  trans
    (block1-linI) a b (constVec12ℤ c) i
    (trans step0 (coord a b c))

eq2 : (i : Fin4) →
  block2 (linI) a b (constVec12ℤ c) i ≡ block2 (scaleVec12ℤ (a + ℤ twelveTimesℤ b) (constVec12ℤ c)) i
eq2 i =
  let
    s0 : sum12ℤ (constVec12ℤ c) ≡ twelveTimesℤ c
    s0 = sum12-const c

    step0 a : (a * ℤ block2 (constVec12ℤ c) i) + ℤ (b * ℤ sum12ℤ (constVec12ℤ c))
            ≡ (a * ℤ c) + ℤ (b * ℤ sum12ℤ (constVec12ℤ c))
    step0 a = refl

    step0 b : (a * ℤ c) + ℤ (b * ℤ sum12ℤ (constVec12ℤ c))
            ≡ (a * ℤ c) + ℤ (b * ℤ twelveTimesℤ c)
    step0 b = cong (λ t → (a * ℤ c) + ℤ (b * ℤ t)) s0

    step0 : (a * ℤ block2 (constVec12ℤ c) i) + ℤ (b * ℤ sum12ℤ (constVec12ℤ c))
            ≡ (a * ℤ c) + ℤ (b * ℤ twelveTimesℤ c)
    step0 = trans step0 a step0 b
  in
  trans
    (block2-linI) a b (constVec12ℤ c) i
    (trans step0 (coord a b c))

```

Forced Predicate Invariance Under the (I, J) -Span

The two invariant classes are stable under the action of the span. For sum-zero vectors this follows from the closed formula for $\Sigma_{12}(\text{linI } a \ b \ v)$, and for constant vectors it follows from the explicit constant-eigenvalue formula.

```

-- § Helper: fourTimes 0 = 0
fourTimesℤ-zero : fourTimesℤ 0ℤ ≡ 0ℤ
fourTimesℤ-zero =
  let
    step₀ : fourTimesℤ 0ℤ ≡ sum4ℤ 0ℤ 0ℤ 0ℤ 0ℤ
    step₀ = refl

    step₁ : sum4ℤ 0ℤ 0ℤ 0ℤ 0ℤ ≡ 0ℤ + ℤ (0ℤ + ℤ (0ℤ + ℤ 0ℤ))
    step₁ = refl

    step₂ : 0ℤ + ℤ (0ℤ + ℤ (0ℤ + ℤ 0ℤ)) ≡ 0ℤ + ℤ (0ℤ + ℤ 0ℤ)
    step₂ = cong (λ t → 0ℤ + ℤ (0ℤ + ℤ t)) (+ℤ-zero-left 0ℤ)

    step₃ : 0ℤ + ℤ (0ℤ + ℤ 0ℤ) ≡ 0ℤ + ℤ 0ℤ
    step₃ = cong (λ t → 0ℤ + ℤ t) (+ℤ-zero-left 0ℤ)
  in
  trans step₀ (trans step₁ (trans step₂ (trans step₃ (+ℤ-zero-left 0ℤ))))

-- § Helper: eightTimes 0 = 0
eightTimesℤ-zero : eightTimesℤ 0ℤ ≡ 0ℤ
eightTimesℤ-zero =
  let
    step₀ : eightTimesℤ 0ℤ ≡ fourTimesℤ 0ℤ + ℤ fourTimesℤ 0ℤ
    step₀ = refl

    step₁ a : fourTimesℤ 0ℤ + ℤ fourTimesℤ 0ℤ ≡ 0ℤ + ℤ fourTimesℤ 0ℤ
    step₁ a = cong (λ t → t + ℤ fourTimesℤ 0ℤ) fourTimesℤ-zero

    step₁ b : 0ℤ + ℤ fourTimesℤ 0ℤ ≡ 0ℤ + ℤ 0ℤ
    step₁ b = cong (λ t → 0ℤ + ℤ t) fourTimesℤ-zero
  in
  trans step₀ (trans step₁ a (trans step₁ b (+ℤ-zero-left 0ℤ)))

-- § Helper: twelveTimes 0 = 0
twelveTimesℤ-zero : twelveTimesℤ 0ℤ ≡ 0ℤ
twelveTimesℤ-zero =
  let
    step₀ : twelveTimesℤ 0ℤ ≡ fourTimesℤ 0ℤ + ℤ eightTimesℤ 0ℤ
    step₀ = refl

    step₁ a : fourTimesℤ 0ℤ + ℤ eightTimesℤ 0ℤ ≡ 0ℤ + ℤ eightTimesℤ 0ℤ
    step₁ a = cong (λ t → t + ℤ eightTimesℤ 0ℤ) fourTimesℤ-zero

    step₁ b : 0ℤ + ℤ eightTimesℤ 0ℤ ≡ 0ℤ + ℤ 0ℤ
    step₁ b = cong (λ t → 0ℤ + ℤ t) eightTimesℤ-zero
  in
  trans step₀ (trans step₁ a (trans step₁ b (+ℤ-zero-left 0ℤ)))

-- § Law 140.8: sum-zero is forced invariant under every (a·I+b·J)
law140-8-linJ-preserves-sum0 : (a b : ℤ) → (v : Vec12ℤ) → ZeroSumVec12 v → ZeroSumVec12 (linJ a b v)
law140-8-linJ-preserves-sum0 a b v sum0 =
  let
    step₀ : sum12ℤ (linJ a b v)
      ≡ (a * ℤ sum12ℤ v) + ℤ (b * ℤ twelveTimesℤ (sum12ℤ v))
    step₀ = sum12-linJ a b v

```

```

a0 : a * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v  $\equiv$  0 $\mathbb{Z}$ 
a0 = trans (cong ( $\lambda$  t  $\rightarrow$  a * $\mathbb{Z}$  t) sum0) (* $\mathbb{Z}$ -zero-right a)

twelve0 : twelveTimes $\mathbb{Z}$  (sum12 $\mathbb{Z}$  v)  $\equiv$  0 $\mathbb{Z}$ 
twelve0 = trans (cong twelveTimes $\mathbb{Z}$  sum0) twelveTimes $\mathbb{Z}$ -zero

b0 : b * $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  (sum12 $\mathbb{Z}$  v)  $\equiv$  0 $\mathbb{Z}$ 
b0 = trans (cong ( $\lambda$  t  $\rightarrow$  b * $\mathbb{Z}$  t) twelve0) (* $\mathbb{Z}$ -zero-right b)

step1 a : (a * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v) + $\mathbb{Z}$  (b * $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  (sum12 $\mathbb{Z}$  v))
 $\equiv$  0 $\mathbb{Z}$  + $\mathbb{Z}$  (b * $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  (sum12 $\mathbb{Z}$  v))
step1 a = cong ( $\lambda$  t  $\rightarrow$  t + $\mathbb{Z}$  (b * $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  (sum12 $\mathbb{Z}$  v))) a0

step1 b : 0 $\mathbb{Z}$  + $\mathbb{Z}$  (b * $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  (sum12 $\mathbb{Z}$  v))  $\equiv$  0 $\mathbb{Z}$  + $\mathbb{Z}$  0 $\mathbb{Z}$ 
step1 b = cong ( $\lambda$  t  $\rightarrow$  0 $\mathbb{Z}$  + $\mathbb{Z}$  t) b0
in
trans step0 (trans step1 a (trans step1 b (+ $\mathbb{Z}$ -zero-left 0 $\mathbb{Z}$ )))

```

Constant-vector invariance uses the constant-eigenvalue formula from Law 14O.3.

```

-- § Helper: scaling a constant vector yields a constant vector
scaleVec12 $\mathbb{Z}$ -const : (a c :  $\mathbb{Z}$ )  $\rightarrow$  Vec12Eq (scaleVec12 $\mathbb{Z}$  a (constVec12 $\mathbb{Z}$  c)) (constVec12 $\mathbb{Z}$  (a * $\mathbb{Z}$  c))
scaleVec12 $\mathbb{Z}$ -const a c = ( $\lambda$  _  $\rightarrow$  refl) , (( $\lambda$  _  $\rightarrow$  refl) , ( $\lambda$  _  $\rightarrow$  refl))

-- § Law 14O.9: constant vectors are forced invariant under every (a·I+b·J)
law14O-9-linIj-preserves-const : (a b :  $\mathbb{Z}$ )  $\rightarrow$  (v : Vec12 $\mathbb{Z}$ )  $\rightarrow$  ConstVec12 v  $\rightarrow$  ConstVec12 (linIj a b v)
law14O-9-linIj-preserves-const a b v (c , vConst) = k , (eq0 , (eq1 , eq2))
where
  sEq : sum12 $\mathbb{Z}$  v  $\equiv$  twelveTimes $\mathbb{Z}$  c
  sEq = trans (sum12-cong v (constVec12 $\mathbb{Z}$  c) vConst) (sum12-const c)

  k :  $\mathbb{Z}$ 
  k = (a + $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  b) * $\mathbb{Z}$  c

  coord : (a b c :  $\mathbb{Z}$ )  $\rightarrow$  (a * $\mathbb{Z}$  c) + $\mathbb{Z}$  (b * $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  c)  $\equiv$  (a + $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  b) * $\mathbb{Z}$  c
  coord a b c =
    trans
      (cong ( $\lambda$  t  $\rightarrow$  (a * $\mathbb{Z}$  c) + $\mathbb{Z}$  t) (mul-twelveShift b c))
      (sym (* $\mathbb{Z}$ -distrib-left-+ $\mathbb{Z}$  a (twelveTimes $\mathbb{Z}$  b) c))

  eq0 : (i : Fin4)  $\rightarrow$  block0 (linIj a b v) i  $\equiv$  block0 (constVec12 $\mathbb{Z}$  k) i
  eq0 i =
    let
      v0 : block0 v i  $\equiv$  c
      v0 = fst vConst i

      a0 : a * $\mathbb{Z}$  block0 v i  $\equiv$  a * $\mathbb{Z}$  c
      a0 = cong ( $\lambda$  t  $\rightarrow$  a * $\mathbb{Z}$  t) v0

      b0 : b * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v  $\equiv$  b * $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  c
      b0 = cong ( $\lambda$  t  $\rightarrow$  b * $\mathbb{Z}$  t) sEq

      step1 : (a * $\mathbb{Z}$  block0 v i) + $\mathbb{Z}$  (b * $\mathbb{Z}$  sum12 $\mathbb{Z}$  v)  $\equiv$  (a * $\mathbb{Z}$  c) + $\mathbb{Z}$  (b * $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  c)
      step1 =

```

```

trans
  (cong ( $\lambda t \rightarrow t + \mathbb{Z} (b * \mathbb{Z} \text{sum12}\mathbb{Z} v)$ )  $a0$ )
  (cong ( $\lambda t \rightarrow (a * \mathbb{Z} c) + \mathbb{Z} t$ )  $b0$ )
in
trans
  (block0-linI  $a b v i$ )
  (trans  $step_1$  (trans (coord  $a b c$ ) refl))

eq1 : ( $i : \text{Fin4}$ )  $\rightarrow$  block1 (linI  $a b v$ )  $i \equiv$  block1 (constVec12 $\mathbb{Z}$   $k$ )  $i$ 
eq1 i =
let
  v1 : block1  $v i \equiv c$ 
  v1 = fst (snd vConst)  $i$ 

  a1 :  $a * \mathbb{Z} \text{block}_1 v i \equiv a * \mathbb{Z} c$ 
  a1 = cong ( $\lambda t \rightarrow a * \mathbb{Z} t$ )  $v1$ 

  b1 :  $b * \mathbb{Z} \text{sum12}\mathbb{Z} v \equiv b * \mathbb{Z} \text{twelveTimes}\mathbb{Z} c$ 
  b1 = cong ( $\lambda t \rightarrow b * \mathbb{Z} t$ ) sEq

  step1 : ( $a * \mathbb{Z} \text{block}_1 v i + \mathbb{Z} (b * \mathbb{Z} \text{sum12}\mathbb{Z} v) \equiv (a * \mathbb{Z} c) + \mathbb{Z} (b * \mathbb{Z} \text{twelveTimes}\mathbb{Z} c)$ )
  step1 =
    trans
      (cong ( $\lambda t \rightarrow t + \mathbb{Z} (b * \mathbb{Z} \text{sum12}\mathbb{Z} v)$ )  $a1$ )
      (cong ( $\lambda t \rightarrow (a * \mathbb{Z} c) + \mathbb{Z} t$ )  $b1$ )
in
trans
  (block1-linI  $a b v i$ )
  (trans  $step_1$  (trans (coord  $a b c$ ) refl))

eq2 : ( $i : \text{Fin4}$ )  $\rightarrow$  block2 (linI  $a b v$ )  $i \equiv$  block2 (constVec12 $\mathbb{Z}$   $k$ )  $i$ 
eq2 i =
let
  v2 : block2  $v i \equiv c$ 
  v2 = snd (snd vConst)  $i$ 

  a2 :  $a * \mathbb{Z} \text{block}_2 v i \equiv a * \mathbb{Z} c$ 
  a2 = cong ( $\lambda t \rightarrow a * \mathbb{Z} t$ )  $v2$ 

  b2 :  $b * \mathbb{Z} \text{sum12}\mathbb{Z} v \equiv b * \mathbb{Z} \text{twelveTimes}\mathbb{Z} c$ 
  b2 = cong ( $\lambda t \rightarrow b * \mathbb{Z} t$ ) sEq

  step1 : ( $a * \mathbb{Z} \text{block}_2 v i + \mathbb{Z} (b * \mathbb{Z} \text{sum12}\mathbb{Z} v) \equiv (a * \mathbb{Z} c) + \mathbb{Z} (b * \mathbb{Z} \text{twelveTimes}\mathbb{Z} c)$ )
  step1 =
    trans
      (cong ( $\lambda t \rightarrow t + \mathbb{Z} (b * \mathbb{Z} \text{sum12}\mathbb{Z} v)$ )  $a2$ )
      (cong ( $\lambda t \rightarrow (a * \mathbb{Z} c) + \mathbb{Z} t$ )  $b2$ )
in
trans
  (block2-linI  $a b v i$ )
  (trans  $step_1$  (trans (coord  $a b c$ ) refl))

```


Spectral Package for linIJ

At this point the remaining task is organizational. The package below collects the four spectral facts already proved for linIJ : preservation of the zero-sum and constant classes together with the two corresponding eigenvalue laws.

```
-- § Law 140.10: linIJ spectral package (4 components)
LinIJSpectralPackage : (a b :  $\mathbb{Z}$ ) → Set
LinIJSpectralPackage a b =
  (v : Vec12 $\mathbb{Z}$ ) →
    (ZeroSumVec12 v → ZeroSumVec12 (linIJ a b v)) ×
    (ConstVec12 v → ConstVec12 (linIJ a b v)) ×
    (ZeroSumVec12 v → Vec12Eq (linIJ a b v) (scaleVec12 $\mathbb{Z}$  a v)) ×
    ((c :  $\mathbb{Z}$ ) → Vec12Eq (linIJ a b (constVec12 $\mathbb{Z}$  c))
      (scaleVec12 $\mathbb{Z}$  (a +  $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  b) (constVec12 $\mathbb{Z}$  c)))

law14O-10-linIJ-spectral-package : (a b :  $\mathbb{Z}$ ) → LinIJSpectralPackage a b
law14O-10-linIJ-spectral-package a b v =
  law14O-8-linIJ-preserves-sum0 a b v ,
  (law14O-9-linIJ-preserves-const a b v ,
    (law14O-2-linIJ-sum0-eigen a b v ,
      law14O-3-linIJ-const-eigen a b))

-- § Projections for LinIJSpectralPackage
LinIJPkg-sum0-inv : {a b :  $\mathbb{Z}$ } → LinIJSpectralPackage a b → (v : Vec12 $\mathbb{Z}$ ) → ZeroSumVec12 v → ZeroSumVec12 (linIJ a b v)
LinIJPkg-sum0-inv pkg v = fst (pkg v)

LinIJPkg-const-inv : {a b :  $\mathbb{Z}$ } → LinIJSpectralPackage a b → (v : Vec12 $\mathbb{Z}$ ) → ConstVec12 v → ConstVec12 (linIJ a b v)
LinIJPkg-const-inv pkg v = fst (snd (pkg v))

LinIJPkg-sum0-eigen : {a b :  $\mathbb{Z}$ } → LinIJSpectralPackage a b → (v : Vec12 $\mathbb{Z}$ ) → ZeroSumVec12 v → Vec12Eq (linIJ a b v) (scaleVec12 $\mathbb{Z}$  a v)
LinIJPkg-sum0-eigen pkg v = fst (snd (snd (pkg v)))

LinIJPkg-const-eigen : {a b :  $\mathbb{Z}$ } → LinIJSpectralPackage a b → (v : Vec12 $\mathbb{Z}$ ) → (c :  $\mathbb{Z}$ ) →
  Vec12Eq (linIJ a b (constVec12 $\mathbb{Z}$  c)) (scaleVec12 $\mathbb{Z}$  (a +  $\mathbb{Z}$  twelveTimes $\mathbb{Z}$  b) (constVec12 $\mathbb{Z}$  c))
LinIJPkg-const-eigen pkg v = snd (snd (snd (pkg v)))
```

Transport from Normal Form to Spectral Facts

Suppose an abstract operator f is given together with a pointwise equality $f = aI + bJ$. Then each spectral statement proved for the normal form transfers directly to f by composition with that equality. A span witness therefore carries the full spectral content of the normal form, not merely its coefficients.

```
-- § Law 140.11: any f with witness f=(a·I+b·J) inherits the spectral facts
SpanOpSpectralFacts : (f : Op) → (a b :  $\mathbb{Z}$ ) → Set
SpanOpSpectralFacts f a b =
  (v : Vec12 $\mathbb{Z}$ ) →
    (ZeroSumVec12 v → ZeroSumVec12 (f v)) ×
```

```

(ConstVec12 v → ConstVec12 (f v)) ×
(ZeroSumVec12 v → Vec12Eq (f v) (scaleVec12ℤ a v)) ×
((c : ℤ) → Vec12Eq (f (constVec12ℤ c))
  (scaleVec12ℤ (a + ℤ twelveTimesℤ b) (constVec12ℤ c)))

law14O-11-spanIJ-transport : (f : Op) → (a b : ℤ) → OpEq f (linI a b) → SpanOpSpectralFacts f a b
law14O-11-spanIJ-transport f a b fEq v =
  sum0Inv ,
  (constInv ,
  (sum0Eigen ,
  constEigen))
where
  sum0Inv : ZeroSumVec12 v → ZeroSumVec12 (f v)
  sum0Inv sum0 =
    trans
      (sum12-cong (f v) (linI a b v) (fEq v))
      (law14O-8-linIJ-preserves-sum0 a b v sum0)

  constInv : ConstVec12 v → ConstVec12 (f v)
  constInv (c , vConst) =
    let
      kLin : ConstVec12 (linI a b v)
      kLin = law14O-9-linIJ-preserves-const a b v (c , vConst)
    in
      fst kLin , Vec12Eq-trans (fEq v) (snd kLin)

  sum0Eigen : ZeroSumVec12 v → Vec12Eq (f v) (scaleVec12ℤ a v)
  sum0Eigen sum0 =
    Vec12Eq-trans
      (fEq v)
      (law14O-2-linIJ-sum0-eigen a b v sum0)

  constEigen : (c : ℤ) → Vec12Eq (f (constVec12ℤ c))
    (scaleVec12ℤ (a + ℤ twelveTimesℤ b) (constVec12ℤ c))
  constEigen c =
    Vec12Eq-trans
      (fEq (constVec12ℤ c))
      (law14O-3-linIJ-const-eigen a b c)

```

Span Witness Packages

Because the coefficient pair of a span operator is unique, it is natural to package that pair together with the pointwise equality to its normal form. The transported spectral facts are then available immediately from the package.

```

-- § Law 140.12: span-IJ coefficient witness is forced unique
SpanIJSpectralPackage : Op → Set
SpanIJSpectralPackage f = ∑ SpanIJ (λ p → OpEq f (interplJ p))

law14O-12-spanIJ-witness-unique : (f : Op) → (w₁ w₂ : SpanIJSpectralPackage f) → fst w₁ ≡ fst w₂
law14O-12-spanIJ-witness-unique f = law14N-2-image-witness-unique f

```

-- § Law 140.13: spectral facts are read directly from a span witness

SpanIJPkg-coeffs : {f : Op} → SpanIJSpectralPackage f → SpanIJ

SpanIJPkg-coeffs pkg = fst pkg

SpanIJPkg-opEq : {f : Op} → (pkg : SpanIJSpectralPackage f) → OpEq f (interplJ (SpanIJPkg-coeffs pkg))

SpanIJPkg-opEq pkg = snd pkg

SpanIJPkg-a : {f : Op} → SpanIJSpectralPackage f → ℤ

SpanIJPkg-a pkg = fst (SpanIJPkg-coeffs pkg)

SpanIJPkg-b : {f : Op} → SpanIJSpectralPackage f → ℤ

SpanIJPkg-b pkg = snd (SpanIJPkg-coeffs pkg)

SpanIJPkg-spectral : {f : Op} → (pkg : SpanIJSpectralPackage f) → SpanOpSpectralFacts f (SpanIJPkg-a pkg) (SpanIJPkg-b pkg)

SpanIJPkg-spectral {f} pkg =

law14O-11-spanIJ-transport f (SpanIJPkg-a pkg) (SpanIJPkg-b pkg) (SpanIJPkg-opEq pkg)

law14O-13-spanIJ-package-projections : {f : Op} → (pkg : SpanIJSpectralPackage f) → SpanOpSpectralFacts f (SpanIJPkg-a pkg) (SpanIJPkg-b pkg)

law14O-13-spanIJ-package-projections pkg = SpanIJPkg-spectral pkg

-- § Consumer projections for SpanIJSpectralPackage

SpanIJPkg-sum0-inv : {f : Op} → (pkg : SpanIJSpectralPackage f) → (v : Vec12ℤ) → ZeroSumVec12 v → ZeroSumVec12 (f v)

SpanIJPkg-sum0-inv pkg v = fst (SpanIJPkg-spectral pkg v)

SpanIJPkg-const-inv : {f : Op} → (pkg : SpanIJSpectralPackage f) → (v : Vec12ℤ) → ConstVec12 v → ConstVec12 (f v)

SpanIJPkg-const-inv pkg v = fst (snd (SpanIJPkg-spectral pkg v))

SpanIJPkg-sum0-eigen : {f : Op} → (pkg : SpanIJSpectralPackage f) → (v : Vec12ℤ) → ZeroSumVec12 v → Vec12Eq (f v) (scaleVec12ℤ (SpanIJPkg-a pkg) v)

SpanIJPkg-sum0-eigen pkg v = fst (snd (snd (SpanIJPkg-spectral pkg v)))

SpanIJPkg-const-eigen : {f : Op} → (pkg : SpanIJSpectralPackage f) → (c : ℤ) →

Vec12Eq (f (constVec12ℤ c))

(scaleVec12ℤ ((SpanIJPkg-a pkg) +ℤ twelveTimesℤ (SpanIJPkg-b pkg)) (constVec12ℤ c))

SpanIJPkg-const-eigen pkg c = snd (snd (snd (SpanIJPkg-spectral pkg (constVec12ℤ c)))) c

Unified Span Transport Package

The final record gathers the data that now always travel together: the coefficient pair, the pointwise equality with the normal form, the transported spectral facts for the abstract operator, and the intrinsic spectral facts for `linIJ`. It serves only as a convenient interface for later sections.

-- § Law 140.14: unified span transport package

SpanIJUnifiedPackage : Op → Set

SpanIJUnifiedPackage f =

Σ SpanIJ (λ p →

OpEq f (interplJ p) ×

SpanOpSpectralFacts f (fst p) (snd p) ×

LinIJSpectralPackage (fst p) (snd p))

law14O-14-spanIJ-unified-package : (f : Op) → SpanIJSpectralPackage f → SpanIJUnifiedPackage f

```

law14O-14-spanIJ-unified-package f pkg =
let
  p : SpanIJ
  p = SpanIJPkg-coeffs pkg

  a : ℤ
  a = fst p

  b : ℤ
  b = snd p

  eq : OpEq f (interplJ p)
  eq = SpanIJPkg-opEq pkg

  facts : SpanOpSpectralFacts f a b
  facts = law14O-11-spanIJ-transport f a b eq

  linPkg : LinIJSpectralPackage a b
  linPkg = law14O-10-linIJ-spectral-package a b
in
p, (eq, (facts, linPkg))

-- § Projections for SpanIJUnifiedPackage
SpanIJUpkg-coeffs : {f : Op} → SpanIJUnifiedPackage f → SpanIJ
SpanIJUpkg-coeffs upkg = fst upkg

SpanIJUpkg-a : {f : Op} → SpanIJUnifiedPackage f → ℤ
SpanIJUpkg-a upkg = fst (SpanIJUpkg-coeffs upkg)

SpanIJUpkg-b : {f : Op} → SpanIJUnifiedPackage f → ℤ
SpanIJUpkg-b upkg = snd (SpanIJUpkg-coeffs upkg)

SpanIJUpkg-opEq : {f : Op} → (upkg : SpanIJUnifiedPackage f) → OpEq f (interplJ (SpanIJUpkg-coeffs upkg))
SpanIJUpkg-opEq upkg = fst (snd upkg)

SpanIJUpkg-spectral : {f : Op} → (upkg : SpanIJUnifiedPackage f) → SpanOpSpectralFacts f (SpanIJUpkg-a upkg) (SpanIJUpkg-b upkg)
SpanIJUpkg-spectral upkg = fst (snd (snd upkg))

SpanIJUpkg-linIJ : {f : Op} → (upkg : SpanIJUnifiedPackage f) → LinIJSpectralPackage (SpanIJUpkg-a upkg) (SpanIJUpkg-b upkg)
SpanIJUpkg-linIJ upkg = snd (snd (snd upkg))

-- § Law 140.15: unified span coefficients are forced unique
SpanIJUpkg-witness : {f : Op} → SpanIJUnifiedPackage f → Σ SpanIJ (λ p → OpEq f (interplJ p))
SpanIJUpkg-witness upkg = SpanIJUpkg-coeffs upkg, SpanIJUpkg-opEq upkg

law14O-15-unified-coeffs-unique : (f : Op) → (u1 u2 : SpanIJUnifiedPackage f) → SpanIJUpkg-coeffs u1 ≡ SpanIJUpkg-coeffs u2
law14O-15-unified-coeffs-unique f u1 u2 =
  law14N-2-image-witness-unique f (SpanIJUpkg-witness u1) (SpanIJUpkg-witness u2)

```

Helper Coefficients

The next calculations require the constants 12 and -1 in forms that interact transparently with multiplication. The lemmas collected here normalize these constants and record the elementary identities used in the Laplacian computations.

```

-- § Forced integer 12
twelveℤ : ℤ
twelveℤ = twelveTimesℤ oneℤ

-- § Forced positivity witness for twelveℤ
twelveℤ-pos : ∑ ℕ (λ n → twelveℤ ≡ +suc n)
twelveℤ-pos = (suc (suc (suc (suc (suc (suc (suc (suc (suc (suc zero)))))))))) , refl

-- § 12 on the left collapses to twelveTimes
twelveℤ-*ℤ-left : (x : ℤ) → twelveℤ *ℤ x ≡ twelveTimesℤ x
twelveℤ-*ℤ-left x =
  trans
    (*ℤ-twelveTimes-left oneℤ x)
    (cong twelveTimesℤ (*ℤ-one-left x))

-- § Multiplication by (-1) collapses to additive negation
negOne-*ℤ-left : (x : ℤ) → (negℤ oneℤ) *ℤ x ≡ negℤ x
negOne-*ℤ-left x =
  let
    neg1 = negℤ oneℤ

    dist : (neg1 +ℤ oneℤ) *ℤ x ≡ (neg1 *ℤ x) +ℤ (oneℤ *ℤ x)
    dist = *ℤ-distrib-left+ℤ neg1 oneℤ x

    sum0 : (neg1 +ℤ oneℤ) ≡ 0ℤ
    sum0 = +ℤ-inv-left oneℤ

    zeroMul : (neg1 +ℤ oneℤ) *ℤ x ≡ 0ℤ
    zeroMul = trans (cong (λ t → t *ℤ x) sum0) (*ℤ-zero-left x)

    eq0 : (neg1 *ℤ x) +ℤ (oneℤ *ℤ x) ≡ 0ℤ
    eq0 = trans (sym dist) zeroMul

    eq1 : (neg1 *ℤ x) +ℤ x ≡ 0ℤ
    eq1 = trans (sym (cong (λ t → (neg1 *ℤ x) +ℤ t) (*ℤ-one-left x))) eq0

    eq2 : (neg1 *ℤ x) +ℤ x ≡ (negℤ x) +ℤ x
    eq2 = trans eq1 (sym (+ℤ-inv-left x))
  in
    +ℤ-cancel-right (neg1 *ℤ x) (negℤ x) x eq2

```

Laplacian as a Forced (I, J) -Span Element

With these normalizations available, the K_{12} Laplacian appears as a specific point of the span. On each coordinate it has the familiar form $12v_i - \sum_{12} v$, and this is exactly the action of $\text{linI}J$ with coefficients 12 and -1 . The helper identities allow the comparison to close blockwise with no ambiguity. From this moment onward, the spectral facts proved for the normal form may be transported directly to L_{12} .

```

-- § Law 140.4:  $L_{12}$  is forced to equal  $(12 \cdot I) + (-1) \cdot J$ 
law140-4-L-in-span : (v : Vec12ℤ) → Vec12Eq (K12LaplacianVec12ℤ v) (linIJ twelveℤ (negℤ oneℤ) v)

```

```

law14O-4-L-in-span v = eq0 , (eq1 , eq2)
where
s : ℤ
s = sum12ℤ v

neg1 = negℤ oneℤ

rhs0 : (i : Fin4) →
block0 (linl twelveℤ neg1 v) i ≡ twelveTimesℤ (block0 v i) +ℤ negℤ s
rhs0 i =
let
pA : (twelveℤ *ℤ block0 v i) +ℤ (neg1 *ℤ s) ≡ twelveTimesℤ (block0 v i) +ℤ (neg1 *ℤ s)
pA = cong (λ t → t +ℤ (neg1 *ℤ s)) (twelveℤ-*ℤ-left (block0 v i))

pB : twelveTimesℤ (block0 v i) +ℤ (neg1 *ℤ s) ≡ twelveTimesℤ (block0 v i) +ℤ negℤ s
pB = cong (λ t → twelveTimesℤ (block0 v i) +ℤ t) (negOne-*ℤ-left s)
in
trans
(block0-linl twelveℤ neg1 v i)
(trans pA pB)

rhs1 : (i : Fin4) →
block1 (linl twelveℤ neg1 v) i ≡ twelveTimesℤ (block1 v i) +ℤ negℤ s
rhs1 i =
let
pA : (twelveℤ *ℤ block1 v i) +ℤ (neg1 *ℤ s) ≡ twelveTimesℤ (block1 v i) +ℤ (neg1 *ℤ s)
pA = cong (λ t → t +ℤ (neg1 *ℤ s)) (twelveℤ-*ℤ-left (block1 v i))

pB : twelveTimesℤ (block1 v i) +ℤ (neg1 *ℤ s) ≡ twelveTimesℤ (block1 v i) +ℤ negℤ s
pB = cong (λ t → twelveTimesℤ (block1 v i) +ℤ t) (negOne-*ℤ-left s)
in
trans
(block1-linl twelveℤ neg1 v i)
(trans pA pB)

rhs2 : (i : Fin4) →
block2 (linl twelveℤ neg1 v) i ≡ twelveTimesℤ (block2 v i) +ℤ negℤ s
rhs2 i =
let
pA : (twelveℤ *ℤ block2 v i) +ℤ (neg1 *ℤ s) ≡ twelveTimesℤ (block2 v i) +ℤ (neg1 *ℤ s)
pA = cong (λ t → t +ℤ (neg1 *ℤ s)) (twelveℤ-*ℤ-left (block2 v i))

pB : twelveTimesℤ (block2 v i) +ℤ (neg1 *ℤ s) ≡ twelveTimesℤ (block2 v i) +ℤ negℤ s
pB = cong (λ t → twelveTimesℤ (block2 v i) +ℤ t) (negOne-*ℤ-left s)
in
trans
(block2-linl twelveℤ neg1 v i)
(trans pA pB)

eq0 : (i : Fin4) → block0 (K12LaplacianVec12ℤ v) i ≡ block0 (linl twelveℤ neg1 v) i
eq0 i = trans (fst (law14H-6-L-twelve-minus-J v) i) (sym (rhs0 i))

eq1 : (i : Fin4) → block1 (K12LaplacianVec12ℤ v) i ≡ block1 (linl twelveℤ neg1 v) i
eq1 i = trans (fst (snd (law14H-6-L-twelve-minus-J v)) i) (sym (rhs1 i))

eq2 : (i : Fin4) → block2 (K12LaplacianVec12ℤ v) i ≡ block2 (linl twelveℤ neg1 v) i
eq2 i = trans (snd (snd (law14H-6-L-twelve-minus-J v)) i) (sym (rhs2 i))

```

Laplacian Composition Closure

Once L_{12} has been identified with the pair $(12, -1)$, the general composition law for the (I, J) -span applies to it without further work. Left and right composition by the Laplacian therefore preserve span membership, and the same is true for the unified packages built from that membership. Consequently every polynomial expression in L_{12} remains inside the span formalism.

```
-- § Law 140.16:  $L_{12}$  has forced span witness  $(12, -1)$ 
LSpanIJ : SpanIJ
LSpanIJ = twelve $\mathbb{Z}$ , (neg $\mathbb{Z}$  one $\mathbb{Z}$ )

law140-16-L-span-witness : SpanIJSpectralPackage K12LaplacianVec12 $\mathbb{Z}$ 
law140-16-L-span-witness = LSpanIJ, ( $\lambda v \rightarrow$  law140-4-L-in-span  $v$ )

-- § Law 140.17: left composition by  $L_{12}$  preserves span membership
law140-17-L-compose-left-span : ( $f : \text{Op}$ )  $\rightarrow$  SpanIJSpectralPackage  $f \rightarrow$  SpanIJSpectralPackage ( $\lambda v \rightarrow$  K12LaplacianVec12 $\mathbb{Z}$  ( $f v$ ))
law140-17-L-compose-left-span  $f$   $pkg = p'$ ,  $eq'$ 
  where
    p : SpanIJ
    p = SpanIJPkg-coeffs  $pkg$ 

    p' : SpanIJ
    p' = mulSpanIJ LSpanIJ p

    fEq : OpEq  $f$  (interplJ p)
    fEq = SpanIJPkg-opEq  $pkg$ 

    eq' : OpEq ( $\lambda v \rightarrow$  K12LaplacianVec12 $\mathbb{Z}$  ( $f v$ )) (interplJ p')
    eq'  $v =$ 
      let
        step1 : Vec12Eq (K12LaplacianVec12 $\mathbb{Z}$  ( $f v$ )) (K12LaplacianVec12 $\mathbb{Z}$  (interplJ p  $v$ ))
        step1 = K12Laplacian-cong ( $f v$ ) (interplJ p  $v$ ) (fEq  $v$ )

        step2 : Vec12Eq (K12LaplacianVec12 $\mathbb{Z}$  (interplJ p  $v$ )) (interplJ LSpanIJ (interplJ p  $v$ ))
        step2 = law140-4-L-in-span (interplJ p  $v$ )

        step3 : Vec12Eq (interplJ LSpanIJ (interplJ p  $v$ )) (interplJ p'  $v$ )
        step3 = law14N-3-IJ-compose-closed LSpanIJ p  $v$ 
      in
        Vec12Eq-trans step1 (Vec12Eq-trans step2 step3)

-- § Law 140.18: right composition by  $L_{12}$  preserves span membership
law140-18-L-compose-right-span : ( $f : \text{Op}$ )  $\rightarrow$  SpanIJSpectralPackage  $f \rightarrow$  SpanIJSpectralPackage ( $\lambda v \rightarrow f$  (K12LaplacianVec12 $\mathbb{Z}$   $v$ ))
law140-18-L-compose-right-span  $f$   $pkg = p'$ ,  $eq'$ 
  where
    p : SpanIJ
    p = SpanIJPkg-coeffs  $pkg$ 

    p' : SpanIJ
    p' = mulSpanIJ p LSpanIJ

    fEq : OpEq  $f$  (interplJ p)
```

```

fEq = SpanIJPkg-opEq pkg

eq' : OpEq (λ v → f (K12LaplacianVec12ℤ v)) (interlJ p')
eq' v =
  let
    step1 : Vec12Eq (f (K12LaplacianVec12ℤ v)) (interlJ p (K12LaplacianVec12ℤ v))
    step1 = fEq (K12LaplacianVec12ℤ v)

    step2 : Vec12Eq (interlJ p (K12LaplacianVec12ℤ v)) (interlJ p (interlJ LSpanI v))
    step2 = interlJ-cong p (K12LaplacianVec12ℤ v) (interlJ LSpanI v) (law14O-4-L-in-span v)

    step3 : Vec12Eq (interlJ p (interlJ LSpanI v)) (interlJ p' v)
    step3 = law14N-3-IJ-compose-closed p LSpanI v
  in
  Vec12Eq-trans step1 (Vec12Eq-trans step2 step3)

-- § Law 140.19: left composition by  $L_{12}$  preserves unified packages
SpanIJPkg-to-span : {f : Op} → SpanIJUnifiedPackage f → SpanIJSpectralPackage f
SpanIJPkg-to-span upkg = SpanIJPkg-coeffs upkg , SpanIJPkg-opEq upkg

law14O-19-L-compose-left-unified : (f : Op) → SpanIJUnifiedPackage f → SpanIJUnifiedPackage (λ v → K12LaplacianVec12ℤ (f v))
law14O-19-L-compose-left-unified f upkg =
  law14O-14-spanIJ-unified-package
  (λ v → K12LaplacianVec12ℤ (f v))
  (law14O-17-L-compose-left-span f (SpanIJPkg-to-span upkg))

-- § Law 140.20: right composition by  $L_{12}$  preserves unified packages
law14O-20-L-compose-right-unified : (f : Op) → SpanIJUnifiedPackage f → SpanIJUnifiedPackage (λ v → f (K12LaplacianVec12ℤ v))
law14O-20-L-compose-right-unified f upkg =
  law14O-14-spanIJ-unified-package
  (λ v → f (K12LaplacianVec12ℤ v))
  (law14O-18-L-compose-right-span f (SpanIJPkg-to-span upkg))

```

Forced Laplacian Eigenvalues

Applying the general span formulas to the special pair $(12, -1)$ gives the two basic Laplacian eigenvalue relations. On sum-zero vectors the operator acts by multiplication by 12, while on constant vectors it vanishes.

```

-- § Law 140.5: sum-zero forces Laplacian eigenvalue 12
law14O-5-L-sum0-eigen12 : (v : Vec12ℤ) → ZeroSumVec12 v → Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ twelveℤ v)
law14O-5-L-sum0-eigen12 v sum0 =
  Vec12Eq-trans
  (law14O-4-L-in-span v)
  (law14O-2-linIj-sum0-eigen twelveℤ (negℤ oneℤ) v sum0)

-- § Law 140.6: constant vectors force Laplacian eigenvalue 0
law14O-6-L-const-eigen0 : (c : ℤ) → Vec12Eq (K12LaplacianVec12ℤ (constVec12ℤ c)) zeroVec12ℤ
law14O-6-L-const-eigen0 = law14H-14-const-eigen0

```


Laplacian Spectral Package

The next package simply gathers the Laplacian facts already established: the span identification, vanishing of the total sum of the image, annihilation by J , the two eigenvalue directions, the image containment statement, and the constant kernel. It provides a single interface for later arguments.

```
-- § Law 140.7: L12 spectral package (7 components)
L12ConstKernel : Set
L12ConstKernel = (x : ℤ) → Vec12Eq (K12LaplacianVec12ℤ (constVec12ℤ x)) zeroVec12ℤ

L12SpectralPackage : Vec12ℤ → Set
L12SpectralPackage v =
  (Vec12Eq (K12LaplacianVec12ℤ v) (linI twelveℤ (negℤ oneℤ) v)) ×
  (sum12ℤ (K12LaplacianVec12ℤ v) ≡ 0ℤ) ×
  (Vec12Eq (J12Vec12ℤ (K12LaplacianVec12ℤ v)) zeroVec12ℤ) ×
  ((sum12ℤ v ≡ 0ℤ → Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v)) ×
   (Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v) → sum12ℤ v ≡ 0ℤ)) ×
  (Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v))
   (twelveVec12ℤ (K12LaplacianVec12ℤ v))) ×
  L12ConstKernel

law14O-7-L-spectral-package : (v : Vec12ℤ) → L12SpectralPackage v
law14O-7-L-spectral-package v =
  law14O-4-L-in-span v ,
  (law14H-8-sumL12-0 v ,
   (law14H-9-JL-zero v ,
    ((law14H-12-sum0-eigen12 v , law14H-13-eigen12→sum0 v) ,
     (law14H-16-image⊆eigen12 v ,
      law14O-6-L-const-eigen0))))))

-- § Projections for L12SpectralPackage
L12Pkg-span : {v : Vec12ℤ} → L12SpectralPackage v → Vec12Eq (K12LaplacianVec12ℤ v) (linI twelveℤ (negℤ oneℤ) v)
L12Pkg-span pkg = fst pkg

L12Pkg-sumL0 : {v : Vec12ℤ} → L12SpectralPackage v → sum12ℤ (K12LaplacianVec12ℤ v) ≡ 0ℤ
L12Pkg-sumL0 pkg = fst (snd pkg)

L12Pkg-JL0 : {v : Vec12ℤ} → L12SpectralPackage v → Vec12Eq (J12Vec12ℤ (K12LaplacianVec12ℤ v)) zeroVec12ℤ
L12Pkg-JL0 pkg = fst (snd (snd pkg))

L12Pkg-sum0→eigen12 : {v : Vec12ℤ} → L12SpectralPackage v → sum12ℤ v ≡ 0ℤ → Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v)
L12Pkg-sum0→eigen12 pkg = fst (fst (snd (snd (snd pkg))))

L12Pkg-eigen12→sum0 : {v : Vec12ℤ} → L12SpectralPackage v → Vec12Eq (K12LaplacianVec12ℤ v) (twelveVec12ℤ v) → sum12ℤ v ≡ 0ℤ
L12Pkg-eigen12→sum0 pkg = snd (fst (snd (snd (snd pkg))))

L12Pkg-image⊆eigen12 : {v : Vec12ℤ} → L12SpectralPackage v →
  Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (twelveVec12ℤ (K12LaplacianVec12ℤ v))
L12Pkg-image⊆eigen12 pkg = fst (snd (snd (snd (snd pkg))))

L12Pkg-constKer : {v : Vec12ℤ} → L12SpectralPackage v → L12ConstKernel
L12Pkg-constKer pkg = snd (snd (snd (snd pkg)))
```

Eigen-Constraint Refinements

Some eigenvalue statements were expressed earlier using `twelveVec12 $\mathbb{Z}$` and `zeroVec12 $\mathbb{Z}$` , whereas the span calculus uses `scaleVec12 $\mathbb{Z}$` . The bridge lemmas below identify these two descriptions so that the reverse eigenvalue implications can be stated uniformly.

```
-- § Law 140.21: scale-by-12 agrees with twelveVec12 $\mathbb{Z}$ 
law14O-21-scale12 $\equiv$ twelveVec12 : (v : Vec12 $\mathbb{Z}$ )  $\rightarrow$  Vec12Eq (scaleVec12 $\mathbb{Z}$  twelve $\mathbb{Z}$  v) (twelveVec12 $\mathbb{Z}$  v)
law14O-21-scale12 $\equiv$ twelveVec12 v =
  ( $\lambda$  i  $\rightarrow$  twelve $\mathbb{Z}$ -* $\mathbb{Z}$ -left (block0 v i)) ,
  (( $\lambda$  i  $\rightarrow$  twelve $\mathbb{Z}$ -* $\mathbb{Z}$ -left (block1 v i)) ,
  ( $\lambda$  i  $\rightarrow$  twelve $\mathbb{Z}$ -* $\mathbb{Z}$ -left (block2 v i)))

-- § Law 140.22: scale-by-0 collapses to zeroVec12 $\mathbb{Z}$ 
law14O-22-scale0 $\equiv$ zeroVec12 : (v : Vec12 $\mathbb{Z}$ )  $\rightarrow$  Vec12Eq (scaleVec12 $\mathbb{Z}$  0 $\mathbb{Z}$  v) zeroVec12 $\mathbb{Z}$ 
law14O-22-scale0 $\equiv$ zeroVec12 v =
  ( $\lambda$  i  $\rightarrow$  * $\mathbb{Z}$ -zero-left (block0 v i)) ,
  (( $\lambda$  i  $\rightarrow$  * $\mathbb{Z}$ -zero-left (block1 v i)) ,
  ( $\lambda$  i  $\rightarrow$  * $\mathbb{Z}$ -zero-left (block2 v i)))

-- § Law 140.23: scale-form 12-eigenvectors force sum-zero
law14O-23-eigen12Scale $\rightarrow$ sum0 : (v : Vec12 $\mathbb{Z}$ )  $\rightarrow$ 
  Vec12Eq (K12LaplacianVec12 $\mathbb{Z}$  v) (scaleVec12 $\mathbb{Z}$  twelve $\mathbb{Z}$  v)  $\rightarrow$  ZeroSumVec12 v
law14O-23-eigen12Scale $\rightarrow$ sum0 v eigen12Scale =
  let
    pkg : L12SpectralPackage v
    pkg = law14O-7-L-spectral-package v

    eigen12 : Vec12Eq (K12LaplacianVec12 $\mathbb{Z}$  v) (twelveVec12 $\mathbb{Z}$  v)
    eigen12 = Vec12Eq-trans eigen12Scale (law14O-21-scale12 $\equiv$ twelveVec12 v)
  in
    L12Pkg-eigen12 $\rightarrow$ sum0 {v = v} pkg eigen12

-- § Law 140.24: scale-form 0-eigenvectors force 12·v = J v
law14O-24-eigen0Scale $\rightarrow$ twelveEqJ : (v : Vec12 $\mathbb{Z}$ )  $\rightarrow$ 
  Vec12Eq (K12LaplacianVec12 $\mathbb{Z}$  v) (scaleVec12 $\mathbb{Z}$  0 $\mathbb{Z}$  v)  $\rightarrow$  Vec12Eq (twelveVec12 $\mathbb{Z}$  v) (J12Vec12 $\mathbb{Z}$  v)
law14O-24-eigen0Scale $\rightarrow$ twelveEqJ v eigen0Scale =
  let
    L0 : Vec12Eq (K12LaplacianVec12 $\mathbb{Z}$  v) zeroVec12 $\mathbb{Z}$ 
    L0 = Vec12Eq-trans eigen0Scale (law14O-22-scale0 $\equiv$ zeroVec12 v)
  in
    law14H-15-L0 $\rightarrow$ twelveEqJ v L0

-- § Law 140.25: eigen-equation forces constraints for  $\lambda=12$  or  $\lambda=0$ 
law14O-25-eigen-constraints : (v : Vec12 $\mathbb{Z}$ )  $\rightarrow$  (lam :  $\mathbb{Z}$ )  $\rightarrow$ 
  Vec12Eq (K12LaplacianVec12 $\mathbb{Z}$  v) (scaleVec12 $\mathbb{Z}$  lam v)  $\rightarrow$ 
  ((lam  $\equiv$  twelve $\mathbb{Z}$ )  $\rightarrow$  ZeroSumVec12 v)  $\times$ 
  ((lam  $\equiv$  0 $\mathbb{Z}$ )  $\rightarrow$  Vec12Eq (twelveVec12 $\mathbb{Z}$  v) (J12Vec12 $\mathbb{Z}$  v))
law14O-25-eigen-constraints v lam eigen = sumPart , kernelPart
where
  P :  $\mathbb{Z}$   $\rightarrow$  Set
```

```

P t = Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ t v)

sumPart : (lam ≡ twelveℤ) → ZeroSumVec12 v
sumPart eqλ = law14O-23-eigen12Scale→sum0 v (subst P eqλ eigen)

kernelPart : (lam ≡ 0ℤ) → Vec12Eq (twelveVec12ℤ v) (J12Vec12ℤ v)
kernelPart eqλ = law14O-24-eigen0Scale→twelveEqJ v (subst P eqλ eigen)

-- § Law 140.26: J-images are forced constant
law14O-26-J-constVec : (v : Vec12ℤ) → ConstVec12 (J12Vec12ℤ v)
law14O-26-J-constVec v = (sum12ℤ v), ((λ _ → refl), ((λ _ → refl), (λ _ → refl)))

-- § Law 140.27: kernel constraint forces 12·v to be constant
law14O-27-twelveEqJ→twelveConst : (v : Vec12ℤ) →
  Vec12Eq (twelveVec12ℤ v) (J12Vec12ℤ v) → ConstVec12 (twelveVec12ℤ v)
law14O-27-twelveEqJ→twelveConst v twelveEqJ =
  let
    c : ℤ
    c = sum12ℤ v

    Jconst : Vec12Eq (J12Vec12ℤ v) (constVec12ℤ c)
    Jconst = snd (law14O-26-J-constVec v)
  in
    c, (Vec12Eq-trans twelveEqJ Jconst)

-- § Law 140.28: scale-form 0-eigenvectors force 12·v constant
law14O-28-eigen0Scale→twelveConst : (v : Vec12ℤ) →
  Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ 0ℤ v) → ConstVec12 (twelveVec12ℤ v)
law14O-28-eigen0Scale→twelveConst v eigen0Scale =
  law14O-27-twelveEqJ→twelveConst v (law14O-24-eigen0Scale→twelveEqJ v eigen0Scale)

-- § Law 140.29: eigen-equation forces sum-zero (λ=12) and 12·v constant (λ=0)
law14O-29-eigen-constraints-strong : (v : Vec12ℤ) → (lam : ℤ) →
  Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) →
  ((lam ≡ twelveℤ) → ZeroSumVec12 v) ×
  ((lam ≡ 0ℤ) → ConstVec12 (twelveVec12ℤ v))
law14O-29-eigen-constraints-strong v lam eigen = sumPart, twelveConstPart
  where
    P : ℤ → Set
    P t = Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ t v)

    sumPart : (lam ≡ twelveℤ) → ZeroSumVec12 v
    sumPart eqλ = law14O-23-eigen12Scale→sum0 v (subst P eqλ eigen)

    twelveConstPart : (lam ≡ 0ℤ) → ConstVec12 (twelveVec12ℤ v)
    twelveConstPart eqλ = law14O-28-eigen0Scale→twelveConst v (subst P eqλ eigen)

```

Torsion-Free Kernel Upgrade

The relation $12v = Jv$ shows first that all coordinates have the same twelvefold multiple. Torsion-freeness of $\mathbb{Z}$ then upgrades this to genuine constancy: from $12(x -$

$y) = 0$ one concludes $x = y$. Applied coordinatewise, this identifies the 0-eigenspace with the constant subspace.

```
-- § Law 140.30: 0-eigenvectors are forced constant (with positivity witness)
law140-30-eigen0Scale→const-assuming-twelvePos : (v : Vec12ℤ) →
  Σ ℕ (λ n → twelveℤ ≡ +suc n) →
  Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ 0ℤ v) →
  ConstVec12 v
law140-30-eigen0Scale→const-assuming-twelvePos v (n , twelvePos) eigen0Scale =
  c , (eq0 , (eq1 , eq2))
where
  c : ℤ
  c = block0 v g0

twelveEqJ : Vec12Eq (twelveVec12ℤ v) (J12Vec12ℤ v)
twelveEqJ = law140-24-eigen0Scale→twelveEqJ v eigen0Scale

-- Convert a coordinate equation 12·x = Σ into twelveℤ·x = Σ
toMul12 : (x s : ℤ) → twelveTimesℤ x ≡ s → twelveℤ *ℤ x ≡ s
toMul12 x s eq = trans (twelveℤ-*ℤ-left x) eq

-- From twelveℤ·x = twelveℤ·y, force x = y via torsion-freeness
cancel12 : (x y : ℤ) → twelveℤ *ℤ x ≡ twelveℤ *ℤ y → x ≡ y
cancel12 x y mulEq =
  let
    Q : ℤ → Set
    Q t = t *ℤ x ≡ t *ℤ y
    mulEq' : (+suc n) *ℤ x ≡ (+suc n) *ℤ y
    mulEq' = subst Q twelvePos mulEq

  diff : ℤ
  diff = x +ℤ negℤ y

step0 : (+suc n) *ℤ diff +ℤ ((+suc n) *ℤ y) ≡ ((+suc n) *ℤ y)
step0 =
  trans
    (cong (λ t → t +ℤ ((+suc n) *ℤ y)) (*ℤ-distrib-right+ℤ (+suc n) x (negℤ y)))
  (trans
    (+ℤ-assoc ((+suc n) *ℤ x) ((+suc n) *ℤ (negℤ y)) ((+suc n) *ℤ y))
  (trans
    (cong (λ t → ((+suc n) *ℤ x) +ℤ t)
      (trans
        (sym (*ℤ-distrib-right+ℤ (+suc n) (negℤ y) y))
        (trans
          (cong (λ t → (+suc n) *ℤ t) (+ℤ-inv-left y))
          (*ℤ-zero-right (+suc n))))))
    (trans
      (cong (λ t → t +ℤ 0ℤ) mulEq')
      (+ℤ-zero-right ((+suc n) *ℤ y))))))

step1 : ((+suc n) *ℤ y) +ℤ ((+suc n) *ℤ diff) ≡ ((+suc n) *ℤ y)
step1 =
  trans
    (sym (+ℤ-comm ((+suc n) *ℤ diff) ((+suc n) *ℤ y)))
```

```

step0

mulDiff0 : (+suc n) *ℤ diff ≡ 0ℤ
mulDiff0 = +ℤ-cancel-left ((+suc n) *ℤ y) ((+suc n) *ℤ diff) step1

diff0 : diff ≡ 0ℤ
diff0 = *ℤ-pos-left-zero→zero n diff mulDiff0

xy : x ≡ y
xy =
  let
    addY : (x +ℤ negℤ y) +ℤ y ≡ 0ℤ +ℤ y
    addY = cong (λ t → t +ℤ y) diff0

    stepA : (x +ℤ negℤ y) +ℤ y ≡ x
    stepA =
      trans
        (+ℤ-assoc x (negℤ y) y)
      (trans
        (cong (λ t → x +ℤ t) (+ℤ-inv-left y))
        (+ℤ-zero-right x))

    stepB : (x +ℤ negℤ y) +ℤ y ≡ y
    stepB = trans addY (+ℤ-zero-left y)
  in
    trans (sym stepA) stepB
in
  xy

sumVal : ℤ
sumVal = sum12ℤ v

eq0 : (i : Fin4) → block0 v i ≡ c
eq0 i =
  let
    sx : twelveTimesℤ (block0 v i) ≡ sumVal
    sx = fst twelveEqJ i

    sc : twelveTimesℤ c ≡ sumVal
    sc = fst twelveEqJ g0

    mulEq : twelveℤ *ℤ (block0 v i) ≡ twelveℤ *ℤ c
    mulEq = trans (toMul12 (block0 v i) sumVal sx) (sym (toMul12 c sumVal sc))
  in
    cancel12 (block0 v i) c mulEq

eq1 : (i : Fin4) → block1 v i ≡ c
eq1 i =
  let
    sx : twelveTimesℤ (block1 v i) ≡ sumVal
    sx = fst (snd twelveEqJ) i

    sc : twelveTimesℤ c ≡ sumVal
    sc = fst twelveEqJ g0

```

```

mulEq : twelveℤ * ℤ (block1 v i) ≡ twelveℤ * ℤ c
mulEq = trans (toMul12 (block1 v i) sumVal sx) (sym (toMul12 c sumVal sc))
in
cancel12 (block1 v i) c mulEq

eq2 : (i : Fin4) → block2 v i ≡ c
eq2 i =
let
  sx : twelveTimesℤ (block2 v i) ≡ sumVal
  sx = snd (snd twelveEqJ) i

  sc : twelveTimesℤ c ≡ sumVal
  sc = fst twelveEqJ g0

mulEq : twelveℤ * ℤ (block2 v i) ≡ twelveℤ * ℤ c
mulEq = trans (toMul12 (block2 v i) sumVal sx) (sym (toMul12 c sumVal sc))
in
cancel12 (block2 v i) c mulEq

```

Removing the positivity witness. The auxiliary positivity witness is now discharged directly, leaving the stand-alone conclusion that every 0-eigenvector is constant.

```

-- § Law 140.31: 0-eigenvectors are forced constant (no extra witness)
law140-31-eigen0Scale→const : (v : Vec12ℤ) →
  Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ 0ℤ v) →
  ConstVec12 v
law140-31-eigen0Scale→const v eigen0Scale =
  law140-30-eigen0Scale→const-assuming-twelvePos v twelveℤ-pos eigen0Scale

-- § Law 140.32: eigen-equation forces sum-zero (λ=12) and const (λ=0)
law140-32-eigen-constraints-final : (v : Vec12ℤ) → (lam : ℤ) →
  Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) →
  ((lam ≡ twelveℤ) → ZeroSumVec12 v) ×
  ((lam ≡ 0ℤ) → ConstVec12 v)
law140-32-eigen-constraints-final v lam eigen = sumPart , constPart
where
  P : ℤ → Set
  P t = Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ t v)

  sumPart : (lam ≡ twelveℤ) → ZeroSumVec12 v
  sumPart eqλ = law140-23-eigen12Scale→sum0 v (subst P eqλ eigen)

  constPart : (lam ≡ 0ℤ) → ConstVec12 v
  constPart eqλ = law140-31-eigen0Scale→const v (subst P eqλ eigen)

```

Eigenvalue Exhaustion

It remains to rule out eigenvalues other than 12 and 0. Commutation of the Laplacian with scalar multiplication, together with torsion-freeness over $\mathbb{Z}$, does exactly this:

for a nonzero vector, an equation $L_{12}v = \lambda v$ can lead only to the two values already identified.

```
-- § scaleVec12 congruence and associativity
scaleVec12-cong : (a : ℤ) → {u v : Vec12ℤ} → Vec12Eq u v → Vec12Eq (scaleVec12ℤ a u) (scaleVec12ℤ a v)
scaleVec12-cong a eq =
  (λ i → cong (λ t → a *ℤ t) (fst eq i)) ,
  ((λ i → cong (λ t → a *ℤ t) (fst (snd eq) i)) ,
   (λ i → cong (λ t → a *ℤ t) (snd (snd eq) i)))

scaleVec12-assoc : (a b : ℤ) → (v : Vec12ℤ) → Vec12Eq (scaleVec12ℤ a (scaleVec12ℤ b v)) (scaleVec12ℤ (a *ℤ b) v)
scaleVec12-assoc a b v =
  (λ i → sym (*ℤ-assoc a b (block0 v i))) ,
  ((λ i → sym (*ℤ-assoc a b (block1 v i))) ,
   (λ i → sym (*ℤ-assoc a b (block2 v i)))
  )

-- § Law 140.33: Laplacian commutes with scaleVec12ℤ
law14O-33-L-scale : (lam : ℤ) → (v : Vec12ℤ) →
  Vec12Eq (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) (scaleVec12ℤ lam (K12LaplacianVec12ℤ v))
law14O-33-L-scale lam v = eq0 , (eq1 , eq2)
where
  s : ℤ
  s = sum12ℤ v

  sScale : sum12ℤ (scaleVec12ℤ lam v) ≡ lam *ℤ s
  sScale = sum12-scaleVec12ℤ lam v

  sNegScale : negℤ (sum12ℤ (scaleVec12ℤ lam v)) ≡ negℤ (lam *ℤ s)
  sNegScale = cong negℤ sScale

  rhsBlock : (x : ℤ) →
    lam *ℤ (twelveTimesℤ x +ℤ negℤ s) ≡ twelveTimesℤ (lam *ℤ x) +ℤ negℤ (lam *ℤ s)
  rhsBlock x =
    trans
      (*ℤ-distrib-right-+ℤ lam (twelveTimesℤ x) (negℤ s))
      (trans
        (cong (λ t → t +ℤ (lam *ℤ negℤ s)) (*ℤ-twelveTimes-right lam x))
        (cong (λ t → twelveTimesℤ (lam *ℤ x) +ℤ t) (*ℤ-neg-right lam s)))

  eq0 : (i : Fin4) → block0 (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) i ≡ block0 (scaleVec12ℤ lam (K12LaplacianVec12ℤ v)) i
  eq0 i =
    let
      lhsBridge : block0 (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) i ≡
        twelveTimesℤ (block0 (scaleVec12ℤ lam v) i) +ℤ negℤ (sum12ℤ (scaleVec12ℤ lam v))
      lhsBridge = fst (law14H-6-L-twelve-minus-J (scaleVec12ℤ lam v)) i

    step1 :
      twelveTimesℤ (lam *ℤ block0 v i) +ℤ negℤ (sum12ℤ (scaleVec12ℤ lam v))
      ≡
      twelveTimesℤ (lam *ℤ block0 v i) +ℤ negℤ (lam *ℤ s)
    step1 = cong (λ t → twelveTimesℤ (lam *ℤ block0 v i) +ℤ t) sNegScale

  rhsBridge : block0 (scaleVec12ℤ lam (K12LaplacianVec12ℤ v)) i ≡
```

```

    lam *ℤ (twelveTimesℤ (block0 v i) +ℤ negℤ s)
  rhsBridge = cong (λ z → lam *ℤ z) (fst (law14H-6-L-twelve-minus-J v) i)
in
  trans lhsBridge (trans step1 (trans (sym (rhsBlock (block0 v i))) (sym rhsBridge)))

eq1 : (i : Fin4) → block1 (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) i ≡ block1 (scaleVec12ℤ lam (K12LaplacianVec12ℤ v)) i
eq1 i =
  let
    lhsBridge : block1 (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) i ≡
      twelveTimesℤ (block1 (scaleVec12ℤ lam v) i) +ℤ negℤ (sum12ℤ (scaleVec12ℤ lam v))
    lhsBridge = fst (snd (law14H-6-L-twelve-minus-J (scaleVec12ℤ lam v))) i

  step1 :
    twelveTimesℤ (lam *ℤ block1 v i) +ℤ negℤ (sum12ℤ (scaleVec12ℤ lam v))
    ≡
    twelveTimesℤ (lam *ℤ block1 v i) +ℤ negℤ (lam *ℤ s)
  step1 = cong (λ t → twelveTimesℤ (lam *ℤ block1 v i) +ℤ t) sNegScale

  rhsBridge : block1 (scaleVec12ℤ lam (K12LaplacianVec12ℤ v)) i ≡
    lam *ℤ (twelveTimesℤ (block1 v i) +ℤ negℤ s)
  rhsBridge = cong (λ z → lam *ℤ z) (fst (snd (law14H-6-L-twelve-minus-J v)) i)
in
  trans lhsBridge (trans step1 (trans (sym (rhsBlock (block1 v i))) (sym rhsBridge)))

eq2 : (i : Fin4) → block2 (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) i ≡ block2 (scaleVec12ℤ lam (K12LaplacianVec12ℤ v)) i
eq2 i =
  let
    lhsBridge : block2 (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) i ≡
      twelveTimesℤ (block2 (scaleVec12ℤ lam v) i) +ℤ negℤ (sum12ℤ (scaleVec12ℤ lam v))
    lhsBridge = snd (snd (law14H-6-L-twelve-minus-J (scaleVec12ℤ lam v))) i

  step1 :
    twelveTimesℤ (lam *ℤ block2 v i) +ℤ negℤ (sum12ℤ (scaleVec12ℤ lam v))
    ≡
    twelveTimesℤ (lam *ℤ block2 v i) +ℤ negℤ (lam *ℤ s)
  step1 = cong (λ t → twelveTimesℤ (lam *ℤ block2 v i) +ℤ t) sNegScale

  rhsBridge : block2 (scaleVec12ℤ lam (K12LaplacianVec12ℤ v)) i ≡
    lam *ℤ (twelveTimesℤ (block2 v i) +ℤ negℤ s)
  rhsBridge = cong (λ z → lam *ℤ z) (snd (snd (law14H-6-L-twelve-minus-J v)) i)
in
  trans lhsBridge (trans step1 (trans (sym (rhsBlock (block2 v i))) (sym rhsBridge)))

-- § Law 140.34: nonzero scalar has no torsion on Vec12ℤ
scaleVec12-pos-left-zero→zeroVec : (n : ℕ) → (v : Vec12ℤ) →
  Vec12Eq (scaleVec12ℤ (+suc n) v) zeroVec12ℤ → Vec12Eq v zeroVec12ℤ
scaleVec12-pos-left-zero→zeroVec n v eq =
  (λ i → *ℤ-pos-left-zero→zero n (block0 v i) (fst eq i)) ,
  (λ i → *ℤ-pos-left-zero→zero n (block1 v i) (fst (snd eq) i)) ,
  (λ i → *ℤ-pos-left-zero→zero n (block2 v i) (snd (snd eq) i))

scaleVec12-neg-left-zero→zeroVec : (n : ℕ) → (v : Vec12ℤ) →
  Vec12Eq (scaleVec12ℤ (-suc n) v) zeroVec12ℤ → Vec12Eq v zeroVec12ℤ
scaleVec12-neg-left-zero→zeroVec n v eq =

```



```

(λ i → *ℤ-neg-left-zero→zero n (block0 v i) (fst eq i)) ,
((λ i → *ℤ-neg-left-zero→zero n (block1 v i) (fst (snd eq) i)) ,
(λ i → *ℤ-neg-left-zero→zero n (block2 v i) (snd (snd eq) i)))

-- § Helper: λ-12=0 forces λ=12
lamMinusTwelve0→lamEqTwelve : (lam : ℤ) → lam + ℤ negℤ twelveℤ ≡ 0ℤ → lam ≡ twelveℤ
lamMinusTwelve0→lamEqTwelve lam eq =
let
  eq' : (lam + ℤ negℤ twelveℤ) + ℤ twelveℤ ≡ 0ℤ + ℤ twelveℤ
  eq' = cong (λ t → t + ℤ twelveℤ) eq

  lhsReduce : (lam + ℤ negℤ twelveℤ) + ℤ twelveℤ ≡ lam
  lhsReduce =
    trans
      (+ℤ-assoc lam (negℤ twelveℤ) twelveℤ)
      (trans
        (cong (λ t → lam + ℤ t) (+ℤ-inv-left twelveℤ))
        (+ℤ-zero-right lam))
in
trans (sym lhsReduce) (trans eq' (+ℤ-zero-left twelveℤ))

```

Scale equation and dichotomy. If $\lambda \cdot w = 12 \cdot w$, then $(\lambda - 12) \cdot w = 0$. Torsion-freeness turns this into the alternative $\lambda \in \{0, 12\}$ or $w = 0$.

```

-- § Helper: λ·w = 12·w implies (λ-12)·w = 0
scaleEq→scaleDiff0 : (lam : ℤ) → (w : Vec12ℤ) →
Vec12Eq (scaleVec12ℤ lam w) (scaleVec12ℤ twelveℤ w) →
Vec12Eq (scaleVec12ℤ (lam + ℤ negℤ twelveℤ) w) zeroVec12ℤ
scaleEq→scaleDiff0 lam w eq = eq0 , (eq1 , eq2)
where
  mk : (x : ℤ) → lam * ℤ x ≡ twelveℤ * ℤ x → (lam + ℤ negℤ twelveℤ) * ℤ x ≡ 0ℤ
  mk x e =
let
  inv : lam * ℤ x + ℤ negℤ (twelveℤ * ℤ x) ≡ 0ℤ
  inv = trans (cong (λ t → t + ℤ negℤ (twelveℤ * ℤ x)) e) (+ℤ-inv-right (twelveℤ * ℤ x))
in
trans
  (*ℤ-distrib-left+ℤ lam (negℤ twelveℤ) x)
  (trans
    (cong (λ t → (lam * ℤ x) + ℤ t) (*ℤ-neg-left twelveℤ x))
    inv)

eq0 : (i : Fin4) → block0 (scaleVec12ℤ (lam + ℤ negℤ twelveℤ) w) i ≡ block0 zeroVec12ℤ i
eq0 i = mk (block0 w i) (fst eq i)

eq1 : (i : Fin4) → block1 (scaleVec12ℤ (lam + ℤ negℤ twelveℤ) w) i ≡ block1 zeroVec12ℤ i
eq1 i = mk (block1 w i) (fst (snd eq) i)

eq2 : (i : Fin4) → block2 (scaleVec12ℤ (lam + ℤ negℤ twelveℤ) w) i ≡ block2 zeroVec12ℤ i
eq2 i = mk (block2 w i) (snd (snd eq) i)

-- § Inspect pattern for case-splitting on computed values

```

```

data Inspect {A : Set} (x : A) : Set where
  reveal : (y : A) → x ≡ y → Inspect x

inspect : {A : Set} (x : A) → Inspect x
inspect x = reveal x refl

-- § Law 140.35: eigen-equation forces  $\lambda \in \{0, 12\}$  or zero vector
law140-35-eigenvalue-exhaustion : (v : Vec12ℤ) → (lam : ℤ) →
  Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) →
  ((lam ≡ twelveℤ) ∪ (lam ≡ 0ℤ)) ∪ (Vec12Eq v zeroVec12ℤ)
law140-35-eigenvalue-exhaustion v lam eigen with inspect (lam +ℤ negℤ twelveℤ)
... | reveal 0ℤ eq = inj₁ (inj₁ (lamMinusTwelve0→lamEqTwelve lam eq))
... | reveal (+suc n) eq =
  let
    w : Vec12ℤ
    w = scaleVec12ℤ lam v

    LL : Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (K12LaplacianVec12ℤ (scaleVec12ℤ lam v))
    LL = K12Laplacian-cong (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) eigen

    eqLamW : Vec12Eq (scaleVec12ℤ lam w) (scaleVec12ℤ twelveℤ w)
    eqLamW =
      let
        left : Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (scaleVec12ℤ twelveℤ w)
        left =
          Vec12Eq-trans
            (law14H-11-LL-twelveL v)
            (Vec12Eq-trans
              (Vec12Eq-sym (law14O-21-scale12≡twelveVec12 (K12LaplacianVec12ℤ v)))
              (scaleVec12-cong twelveℤ eigen))

        right : Vec12Eq (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) (scaleVec12ℤ lam w)
        right = Vec12Eq-trans (law14O-33-L-scale lam v) (scaleVec12-cong lam eigen)

        both : Vec12Eq (scaleVec12ℤ twelveℤ w) (scaleVec12ℤ lam w)
        both = Vec12Eq-trans (Vec12Eq-sym left) (Vec12Eq-trans LL right)
      in
        Vec12Eq-sym both

    diff0 : Vec12Eq (scaleVec12ℤ (+suc n) w) zeroVec12ℤ
    diff0 = subst (λ t → Vec12Eq (scaleVec12ℤ t w) zeroVec12ℤ) eq (scaleEq→scaleDiff0 lam w eqLamW)

    w0 : Vec12Eq w zeroVec12ℤ
    w0 = scaleVec12-pos-left-zero→zeroVec n w diff0
  in
    caseLam lam w0
  where
    caseLam : (lam : ℤ) → Vec12Eq (scaleVec12ℤ lam v) zeroVec12ℤ → ((lam ≡ twelveℤ) ∪ (lam ≡ 0ℤ)) ∪ (Vec12Eq v zeroVec12ℤ)
    caseLam lam w0 with lam
    ... | 0ℤ = inj₁ (inj₂ refl)
    ... | +suc m = inj₂ (scaleVec12-pos-left-zero→zeroVec m v w0)
    ... | -suc m = inj₂ (scaleVec12-neg-left-zero→zeroVec m v w0)
  ... | reveal (-suc n) eq =

```

```

let
  w : Vec12ℤ
  w = scaleVec12ℤ lam v

LL : Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (K12LaplacianVec12ℤ (scaleVec12ℤ lam v))
LL = K12Laplacian-cong (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) eigen

eqLamW : Vec12Eq (scaleVec12ℤ lam w) (scaleVec12ℤ twelveℤ w)
eqLamW =
  let
    left : Vec12Eq (K12LaplacianVec12ℤ (K12LaplacianVec12ℤ v)) (scaleVec12ℤ twelveℤ w)
    left =
      Vec12Eq-trans
        (law14H-11-LL-twelveL v)
        (Vec12Eq-trans
          (Vec12Eq-sym (law14O-21-scale12≡twelveVec12 (K12LaplacianVec12ℤ v)))
          (scaleVec12-cong twelveℤ eigen))

    right : Vec12Eq (K12LaplacianVec12ℤ (scaleVec12ℤ lam v)) (scaleVec12ℤ lam w)
    right = Vec12Eq-trans (law14O-33-L-scale lam v) (scaleVec12-cong lam eigen)

    both : Vec12Eq (scaleVec12ℤ twelveℤ w) (scaleVec12ℤ lam w)
    both = Vec12Eq-trans (Vec12Eq-sym left) (Vec12Eq-trans LL right)
  in
    Vec12Eq-sym both

diff0 : Vec12Eq (scaleVec12ℤ (-suc n) w) zeroVec12ℤ
diff0 = subst (λ t → Vec12Eq (scaleVec12ℤ t w) zeroVec12ℤ) eq (scaleEq→scaleDiff0 lam w eqLamW)

w0 : Vec12Eq w zeroVec12ℤ
w0 = scaleVec12-neg-left-zero→zeroVec n w diff0
in
  caseLam lam w0
  where
    caseLam : (lam : ℤ) → Vec12Eq (scaleVec12ℤ lam v) zeroVec12ℤ → ((lam ≡ twelveℤ) ⊔ (lam ≡ 0ℤ)) ⊔ (Vec12Eq v zeroVec12ℤ)
    caseLam lam w0 with lam
    ... | 0ℤ = inj₁ (inj₂ refl)
    ... | +suc m = inj₂ (scaleVec12-pos-left-zero→zeroVec m v w0)
    ... | -suc m = inj₂ (scaleVec12-neg-left-zero→zeroVec m v w0)

```

Ausschlussgesetz

Combining the exhaustion theorem with the refined constraint laws gives the final eigenvector classification. If $\lambda = 12$, the vector lies in the sum-zero subspace. If $\lambda = 0$, it is constant. Any other value of λ forces the vector to vanish.

```

-- § Law 140.36: corrected Ausschussgesetz (exhaustion + constraints)
law14O-36-eigen-classification : (v : Vec12ℤ) → (lam : ℤ) →
  Vec12Eq (K12LaplacianVec12ℤ v) (scaleVec12ℤ lam v) →
  ((lam ≡ twelveℤ) × ZeroSumVec12 v) ⊔ (((lam ≡ 0ℤ) × ConstVec12 v) ⊔ (Vec12Eq v zeroVec12ℤ))
law14O-36-eigen-classification v lam eigen with law14O-35-eigenvalue-exhaustion v lam eigen

```

```

... | inj1 (inj1 lam12) =
inj1 (lam12 , fst (law14O-32-eigen-constraints-final v lam eigen) lam12)
... | inj1 (inj2 lam0) =
inj2 (inj1 (lam0 , snd (law14O-32-eigen-constraints-final v lam eigen) lam0))
... | inj2 v0 =
inj2 (inj2 v0)

-- § Bundled torsion-freeness pair for Vec12ℤ
scaleVec12_nonzero_left_zero_to_zeroVec :
(n : ℕ) → (v : Vec12ℤ) →
(Vec12Eq (scaleVec12ℤ (+suc n) v) zeroVec12ℤ → Vec12Eq v zeroVec12ℤ)
× (Vec12Eq (scaleVec12ℤ (-suc n) v) zeroVec12ℤ → Vec12Eq v zeroVec12ℤ)
scaleVec12_nonzero_left_zero_to_zeroVec n v =
scaleVec12-pos-left-zero→zeroVec n v , scaleVec12-neg-left-zero→zeroVec n v

```

The rational layer now carries the spectral content explicitly. Quotients, distances, finite-index operators, coupled and triple K_4 Laplacians, word algebra, and rational order leave no gap at the level of calculation. The graph evaluation $\mathcal{C} = 137$, the eigenvalues $\{0, 4\}$ of K_4 , and the 3:1 eigenspace split have all been fixed within this rational framework.

What remains is not another finite algebraic law but completion. Once the metric structure is explicit, Cauchy data that do not settle inside $\mathbb{Q}$ require a legitimate home. The next step is therefore the passage from rational measurement to the reals.

Figure 52.1 summarizes this transition. Rational points embed as constant sequences, and completion adds exactly those limit points that the rational layer can approach but not contain.

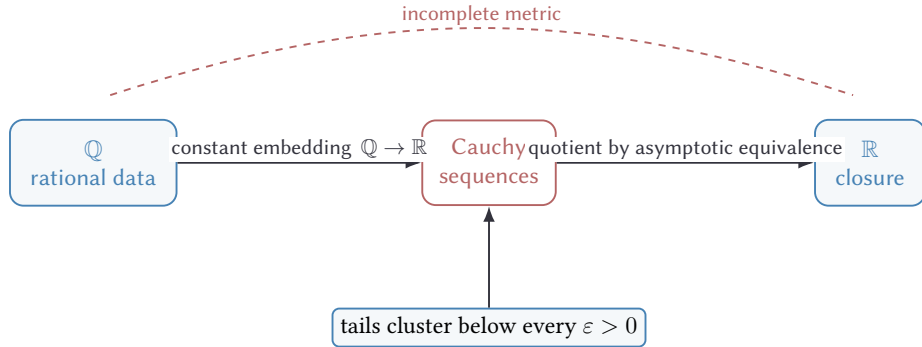


Figure 52.1: Forced Cauchy closure: $\mathbb{Q}$ embeds as constant sequences, while the missing limits of rational Cauchy data are closed inside $\mathbb{R}$.

Chapter 53

Reals as Forced Cauchy Closure over $\mathbb{Q}$

The metric on $\mathbb{Q}$ is incomplete. Cauchy data cannot remain inside $\mathbb{Q}$ without loss. The reals must therefore be forced as the Cauchy closure.

Formally, this step does one essential thing: it closes the rationals under Cauchy data and lifts the arithmetic and order structure along the construction. Once later spectral expressions require irrational limits, there must be a type in which those limits live. The real numbers enter here as exactly that closure, with the inherited operations already built into the construction.

Cauchy condition and the real number type

A sequence is Cauchy when, for every rational tolerance $\varepsilon > 0$, all sufficiently late terms cluster within ε of each other. A real number is a sequence carrying a proof of this condition. The rational metric is not complete, so Cauchy sequences must be given a space in which their limits can be represented. The constructive choice is the familiar one: a real number is a rational sequence together with a proof that it is Cauchy. The embedding $\mathbb{Q} \hookrightarrow \mathbb{R}$ then sends a rational to the corresponding constant sequence, whose Cauchy property is immediate. This provides the closure of the rational metric that the later arguments require.

```
-- § Cauchy condition: eventual  $\epsilon$ -clustering
record IsCauchy (seq :  $\mathbb{N} \rightarrow \mathbb{Q}$ ) : Set where
  field
    cauchy : ( $\epsilon : \mathbb{Q}$ )  $\rightarrow$  ( $0\mathbb{Q} < \mathbb{Q} \epsilon$ )  $\rightarrow \Sigma \mathbb{N} (\lambda N \rightarrow (m\ n : \mathbb{N}) \rightarrow N \leq m \rightarrow N \leq n \rightarrow \text{dist}\mathbb{Q} (seq\ m) (seq\ n) < \mathbb{Q} \epsilon)$ 

-- § Type alias for Cauchy predicate
IsCauchyP : ( $\mathbb{N} \rightarrow \mathbb{Q}$ )  $\rightarrow$  Set
IsCauchyP = IsCauchy
```

```

-- § Real number: sequence + Cauchy proof
record  $\mathbb{R}$  : Set where
  constructor mk $\mathbb{R}$ 
  field
    seq :  $\mathbb{N} \rightarrow \mathbb{Q}$ 
    isCauchy : IsCauchy seq

open  $\mathbb{R}$  public

-- § Rational embedding: constant sequences
 $\mathbb{Q}$ to $\mathbb{R}$  :  $\mathbb{Q} \rightarrow \mathbb{R}$ 
 $\mathbb{Q}$ to $\mathbb{R}$  q = mk $\mathbb{R}$  ( $\lambda \_ \rightarrow q$ ) record
{ cauchy =  $\lambda \epsilon \in \epsilon pos \rightarrow$ 
  zero , ( $\lambda m n \_ \rightarrow \text{dist } \mathbb{Q} \text{ -const } < \epsilon q \epsilon \in \epsilon pos$ )
}

```

Real equivalence and symmetry

Once real numbers are represented by Cauchy sequences, equality can no longer mean literal equality of data. Two representatives should be identified exactly when their pointwise distance tends to zero. This eventual ε -closeness is the quotient relation used on Cauchy data. Its symmetry is immediate from the symmetry of $\text{dist } \mathbb{Q}$, while reflexivity and transitivity come from the metric structure already established over $\mathbb{Q}$.

```

-- § Real equivalence: difference converges to 0
infix 4  $\simeq_{\mathbb{R}}$ 

record  $\simeq_{\mathbb{R}}$  (x y :  $\mathbb{R}$ ) : Set where
  field
    conv0 : ( $\epsilon$  :  $\mathbb{Q}$ )  $\rightarrow$  ( $0 \mathbb{Q} < \mathbb{Q} \epsilon$ )  $\rightarrow \sum N (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow \text{dist } \mathbb{Q} (\text{seq } x n) (\text{seq } y n) < \mathbb{Q} \epsilon)$ 

-- § Symmetry of real equivalence
 $\simeq_{\mathbb{R}}\text{-sym}$  : {x y :  $\mathbb{R}$ }  $\rightarrow x \simeq_{\mathbb{R}} y \rightarrow y \simeq_{\mathbb{R}} x$ 
 $\simeq_{\mathbb{R}}\text{-sym}$  {x} {y}  $x \simeq y$  = record
{ conv0 =  $\lambda \epsilon \in \epsilon pos \rightarrow$ 
  let
    pack :  $\sum N (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow \text{dist } \mathbb{Q} (\text{seq } x n) (\text{seq } y n) < \mathbb{Q} \epsilon)$ 
    pack =  $\simeq_{\mathbb{R}}\text{-conv0 } x \simeq y \epsilon pos$ 

    N :  $\mathbb{N}$ 
    N = fst pack

    conv : ( $n : \mathbb{N}$ )  $\rightarrow N \leq n \rightarrow \text{dist } \mathbb{Q} (\text{seq } x n) (\text{seq } y n) < \mathbb{Q} \epsilon$ 
    conv = snd pack
  in
    N , ( $\lambda n N \leq n \rightarrow$ 
    let
      swap :  $\text{dist } \mathbb{Q} (\text{seq } y n) (\text{seq } x n) \simeq \mathbb{Q} \text{ dist } \mathbb{Q} (\text{seq } x n) (\text{seq } y n)$ 

```

```

swap = distQ-sym (seq y n) (seq x n)

d≤ : distQ (seq y n) (seq x n) ≤Q distQ (seq x n) (seq y n)
d≤ = ≈Q→≤Ql {distQ (seq y n) (seq x n)} {distQ (seq x n) (seq y n)} swap
in
≤<Q→<Q {distQ (seq y n) (seq x n)} {distQ (seq x n) (seq y n)} {ε} d≤ (conv n N≤n)
}

```

Addition on $\mathbb{R}$

Operations must now be lifted from rational sequences to real numbers. Addition can only be defined pointwise, and the substantive point is that the pointwise sum of two Cauchy sequences is again Cauchy. The proof uses an $\varepsilon/4$ budget: one modulus is taken from each summand, the distances are rewritten by the translation laws for distQ , and the triangle inequality combines the two resulting estimates.

```

-- § Real addition with Cauchy preservation via ε-quartering
_+ℝ_ : ℝ → ℝ → ℝ
x +ℝ y = mkℝ (λ n → seq x n +Q seq y n) record
{ cauchy = λ ε ∈ pos →
  let
    -- § Phase 1: Budget allocation
    εq : Q
    εq = εQuarter ε

    εqPos : 0Q <Q εq
    εqPos = εQuarter-pos ε

    -- § Phase 2: Independent Cauchy moduli
    cxPack = lsCauchy.cauchy (isCauchy x) εq εqPos
    cyPack = lsCauchy.cauchy (isCauchy y) εq εqPos

    CxN : ℕ
    CxN = fst cxPack

    CyN : ℕ
    CyN = fst cyPack

    cx : (m n : ℕ) → CxN ≤ m → CxN ≤ n → distQ (seq x m) (seq x n) <Q εq
    cx = snd cxPack

    cy : (m n : ℕ) → CyN ≤ m → CyN ≤ n → distQ (seq y m) (seq y n) <Q εq
    cy = snd cyPack

    -- § Phase 3: Global modulus N
    N : ℕ
    N = CxN +ℕ CyN

    CxN≤N : CxN ≤ N
    CxN≤N =
  let

```

```

step : (CxN +N zero) ≤ (CxN +N CyN)
step = ≤-+N-monol {a = zero} {b = CyN} z≤n CxN
in
subst (λ t → t ≤ (CxN +N CyN)) (+N-zero-right CxN) step

CyN≤N : CyN ≤ N
CyN≤N =
let
  step : (CyN +N zero) ≤ (CyN +N CxN)
  step = ≤-+N-monol {a = zero} {b = CxN} z≤n CyN

  base : CyN ≤ (CyN +N CxN)
  base = subst (λ t → t ≤ (CyN +N CxN)) (+N-zero-right CyN) step
in
subst (λ t → CyN ≤ t) (+N-comm CyN CxN) base

εq+εq<ε : (εq +Q εq) <Q ε
εq+εq<ε = εQuarter-double<ε ε εpos

εqNonneg : 0Q ≤Q εq
εqNonneg = <Q→≤Q {0Q} {εq} εqPos
in
N , (λ m n N≤m N≤n →
let
  -- -- Phase 4: Propagate N to both moduli -----
  Cx≤m : CxN ≤ m
  Cx≤m = ≤-trans CxN≤N N≤m

  Cx≤n : CxN ≤ n
  Cx≤n = ≤-trans CxN≤N N≤n

  Cy≤m : CyN ≤ m
  Cy≤m = ≤-trans CyN≤N N≤m

  Cy≤n : CyN ≤ n
  Cy≤n = ≤-trans CyN≤N N≤n

  dx : Q
  dx = distQ (seq x m) (seq x n)

  dy : Q
  dy = distQ (seq y m) (seq y n)

  dx<εq : dx <Q εq
  dx<εq = cx m n Cx≤m Cx≤n

  dy<εq : dy <Q εq
  dy<εq = cy m n Cy≤m Cy≤n

```

After the common modulus has been fixed, the remaining estimate is a straightforward application of the triangle inequality.

-- § Phase 5: Triangle inequality collapse

$$\begin{aligned}
dx \leq \epsilon q &: dx \leq \mathbb{Q} \epsilon q \\
dx \leq \epsilon q &= \neg \mathbb{Q} \rightarrow \leq \mathbb{Q} \{dx\} \{\epsilon q\} \ dx < \epsilon q \\
\\
dy \leq \epsilon q &: dy \leq \mathbb{Q} \epsilon q \\
dy \leq \epsilon q &= \neg \mathbb{Q} \rightarrow \leq \mathbb{Q} \{dy\} \{\epsilon q\} \ dy < \epsilon q \\
\\
dxNonneg &: 0 \mathbb{Q} \leq \mathbb{Q} \ dx \\
dxNonneg &= \text{dist}\mathbb{Q}\text{-nonneg} \ (\text{seq } x \ m) \ (\text{seq } x \ n) \\
\\
dyNonneg &: 0 \mathbb{Q} \leq \mathbb{Q} \ dy \\
dyNonneg &= \text{dist}\mathbb{Q}\text{-nonneg} \ (\text{seq } y \ m) \ (\text{seq } y \ n) \\
\\
d1 &: \mathbb{Q} \\
d1 &= \text{dist}\mathbb{Q} \ ((\text{seq } x \ m) + \mathbb{Q} \ (\text{seq } y \ m)) \ ((\text{seq } x \ n) + \mathbb{Q} \ (\text{seq } y \ n)) \\
\\
d2 &: \mathbb{Q} \\
d2 &= \text{dist}\mathbb{Q} \ ((\text{seq } x \ n) + \mathbb{Q} \ (\text{seq } y \ m)) \ ((\text{seq } x \ n) + \mathbb{Q} \ (\text{seq } y \ n)) \\
\\
d1 \leq dx &: d1 \leq \mathbb{Q} \ dx \\
d1 \leq dx &= \simeq \mathbb{Q} \rightarrow \leq \mathbb{Q}^l \ \{d1\} \ \{dx\} \ (\text{dist}\mathbb{Q}\text{-}\mathbb{Q}\text{-right} \ (\text{seq } x \ m) \ (\text{seq } x \ n) \ (\text{seq } y \ m)) \\
\\
d2 \leq dy &: d2 \leq \mathbb{Q} \ dy \\
d2 \leq dy &= \simeq \mathbb{Q} \rightarrow \leq \mathbb{Q}^l \ \{d2\} \ \{dy\} \ (\text{dist}\mathbb{Q}\text{-}\mathbb{Q}\text{-left} \ (\text{seq } x \ n) \ (\text{seq } y \ m) \ (\text{seq } y \ n)) \\
\\
d1 \leq \epsilon q &: d1 \leq \mathbb{Q} \ \epsilon q \\
d1 \leq \epsilon q &= \leq \mathbb{Q}\text{-trans} \ \{d1\} \ \{dx\} \ \{\epsilon q\} \ d1 \leq dx \ dx \leq \epsilon q \\
\\
d2 \leq \epsilon q &: d2 \leq \mathbb{Q} \ \epsilon q \\
d2 \leq \epsilon q &= \leq \mathbb{Q}\text{-trans} \ \{d2\} \ \{dy\} \ \{\epsilon q\} \ d2 \leq dy \ dy \leq \epsilon q \\
\\
d1Nonneg &: 0 \mathbb{Q} \leq \mathbb{Q} \ d1 \\
d1Nonneg &= \text{dist}\mathbb{Q}\text{-nonneg} \ ((\text{seq } x \ m) + \mathbb{Q} \ (\text{seq } y \ m)) \ ((\text{seq } x \ n) + \mathbb{Q} \ (\text{seq } y \ m)) \\
\\
d2Nonneg &: 0 \mathbb{Q} \leq \mathbb{Q} \ d2 \\
d2Nonneg &= \text{dist}\mathbb{Q}\text{-nonneg} \ ((\text{seq } x \ n) + \mathbb{Q} \ (\text{seq } y \ m)) \ ((\text{seq } x \ n) + \mathbb{Q} \ (\text{seq } y \ n)) \\
\\
sum \leq &: (d1 + \mathbb{Q} \ d2) \leq \mathbb{Q} \ (\epsilon q + \mathbb{Q} \ \epsilon q) \\
sum \leq &= \leq \mathbb{Q}\text{-sum} \leq \text{double-nonneg} \ d1 \ d2 \ \epsilon q \ d1Nonneg \ d2Nonneg \ \epsilon qNonneg \ d1 \leq \epsilon q \ d2 \leq \epsilon q \\
\\
sum < \epsilon &: (d1 + \mathbb{Q} \ d2) < \mathbb{Q} \ \epsilon \\
sum < \epsilon &= \leq \neg \mathbb{Q} \rightarrow < \mathbb{Q} \ \{(d1 + \mathbb{Q} \ d2)\} \ \{(\epsilon q + \mathbb{Q} \ \epsilon q)\} \ \{\epsilon\} \ sum \leq \epsilon q + \epsilon q < \epsilon \\
\\
tri &: \text{dist}\mathbb{Q} \ ((\text{seq } x \ m) + \mathbb{Q} \ (\text{seq } y \ m)) \ ((\text{seq } x \ n) + \mathbb{Q} \ (\text{seq } y \ n)) \leq \mathbb{Q} \ (d1 + \mathbb{Q} \ d2) \\
tri &= \text{dist}\mathbb{Q}\text{-triangle} \ ((\text{seq } x \ m) + \mathbb{Q} \ (\text{seq } y \ m)) \ ((\text{seq } x \ n) + \mathbb{Q} \ (\text{seq } y \ m)) \ ((\text{seq } x \ n) + \mathbb{Q} \ (\text{seq } y \ n)) \\
\text{in} & \\
\leq \neg \mathbb{Q} \rightarrow &< \mathbb{Q} \ \{\text{dist}\mathbb{Q} \ ((\text{seq } x \ m) + \mathbb{Q} \ (\text{seq } y \ m)) \ ((\text{seq } x \ n) + \mathbb{Q} \ (\text{seq } y \ n))\} \ \{(d1 + \mathbb{Q} \ d2)\} \ \{\epsilon\} \ tri \ sum < \epsilon \\
\} &
\end{aligned}$$

Negation, subtraction, and constants

Once addition is available, negation is defined termwise. The identity $\text{dist}\mathbb{Q}\text{-neg}$ shows that this preserves the Cauchy property without any new estimate. Subtraction

is then addition with the negated sequence, and the constants $0_{\mathbb{R}}$ and $1_{\mathbb{R}}$ are just the images of the corresponding rationals under the embedding.

```
-- § Real negation via distQ-neg
-ℝ_ : ℝ → ℝ
-ℝ x = mkℝ (λ n → -Q (seq x n)) record
{ cauchy = λ ε ∈ εpos →
  let
    pack = lsCauchy.cauchy (isCauchy x) ε εpos
    N : ℕ
    N = fst pack
    cx = snd pack
  in
    N , (λ m n N ≤ m N ≤ n →
      let
        d ≤ : distQ (-Q (seq x m)) (-Q (seq x n)) ≤Q distQ (seq x m) (seq x n)
        d ≤ = distQ-neg (seq x m) (seq x n)

        d ≤ : distQ (-Q (seq x m)) (-Q (seq x n)) ≤Q distQ (seq x m) (seq x n)
        d ≤ = ≈Q → ≤Ql {distQ (-Q (seq x m)) (-Q (seq x n))} {distQ (seq x m) (seq x n)} d ≤
      in
        ≤Q → <Q {distQ (-Q (seq x m)) (-Q (seq x n))} {distQ (seq x m) (seq x n)} {ε} d ≤ (cx m n N ≤ m N ≤ n))
    }
}

-- § Real subtraction
_-ℝ_ : ℝ → ℝ → ℝ
x -ℝ y = x +ℝ (-ℝ y)

-- § Zero and one in ℝ
0ℝ : ℝ
0ℝ = Qtoℝ 0Q

1ℝ : ℝ
1ℝ = Qtoℝ 1Q
```

Additive algebra laws

The additive laws for real numbers are inherited pointwise from $\mathbb{Q}$. Each proof uses the threshold $N = 0$, because the corresponding rational identity makes the distance between the two representing sequences vanish at every index.

```
-- § Commutativity of real addition.
+ℝ-comm : (x y : ℝ) → (x +ℝ y) ≈ℝ (y +ℝ x)
+ℝ-comm x y = record
{ conv0 = λ ε ∈ εpos →
  zero , (λ n _ →
    let
      p : ℚ
      p = seq x n +Q seq y n

      q : ℚ
```

```

q = seq y n + $\mathbb{Q}$  seq x n

pq $\simeq$  : p  $\simeq_{\mathbb{Q}}$  q
pq $\simeq$  = + $\mathbb{Q}$ -comm (seq x n) (seq y n)

d $\simeq$ 0 : dist $\mathbb{Q}$  p q  $\simeq_{\mathbb{Q}}$  0 $\mathbb{Q}$ 
d $\simeq$ 0 = dist $\mathbb{Q}$ - $\simeq$ 0 pq $\simeq$ 

d $\leq$ 0 : dist $\mathbb{Q}$  p q  $\leq_{\mathbb{Q}}$  0 $\mathbb{Q}$ 
d $\leq$ 0 =  $\simeq_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}}^l$  {dist $\mathbb{Q}$  p q} {0 $\mathbb{Q}$ } d $\simeq$ 0
in
 $\leq_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}}^l$  {dist $\mathbb{Q}$  p q} {0 $\mathbb{Q}$ } { $\epsilon$ } d $\leq$ 0  $\epsilon pos$ )
}

-- § Associativity of real addition
+ $\mathbb{R}$ -assoc : (x y z :  $\mathbb{R}$ )  $\rightarrow$  ((x + $\mathbb{R}$  y) + $\mathbb{R}$  z)  $\simeq_{\mathbb{R}}$  (x + $\mathbb{R}$  (y + $\mathbb{R}$  z))
+ $\mathbb{R}$ -assoc x y z = record
{ conv0 =  $\lambda \epsilon \epsilon pos \rightarrow$ 
  zero , ( $\lambda n \_ \rightarrow$ 
    let
      p :  $\mathbb{Q}$ 
      p = (seq x n + $\mathbb{Q}$  seq y n) + $\mathbb{Q}$  seq z n

      q :  $\mathbb{Q}$ 
      q = seq x n + $\mathbb{Q}$  (seq y n + $\mathbb{Q}$  seq z n)

      pq $\simeq$  : p  $\simeq_{\mathbb{Q}}$  q
      pq $\simeq$  = + $\mathbb{Q}$ -assoc (seq x n) (seq y n) (seq z n)

      d $\simeq$ 0 : dist $\mathbb{Q}$  p q  $\simeq_{\mathbb{Q}}$  0 $\mathbb{Q}$ 
      d $\simeq$ 0 = dist $\mathbb{Q}$ - $\simeq$ 0 pq $\simeq$ 

      d $\leq$ 0 : dist $\mathbb{Q}$  p q  $\leq_{\mathbb{Q}}$  0 $\mathbb{Q}$ 
      d $\leq$ 0 =  $\simeq_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}}^l$  {dist $\mathbb{Q}$  p q} {0 $\mathbb{Q}$ } d $\simeq$ 0
    in
       $\leq_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}}^l$  {dist $\mathbb{Q}$  p q} {0 $\mathbb{Q}$ } { $\epsilon$ } d $\leq$ 0  $\epsilon pos$ )
  }

-- § Right identity for real addition
+ $\mathbb{R}$ -zero-right : (x :  $\mathbb{R}$ )  $\rightarrow$  (x + $\mathbb{R}$  0 $\mathbb{R}$ )  $\simeq_{\mathbb{R}}$  x
+ $\mathbb{R}$ -zero-right x = record
{ conv0 =  $\lambda \epsilon \epsilon pos \rightarrow$ 
  zero , ( $\lambda n \_ \rightarrow$ 
    let
      p :  $\mathbb{Q}$ 
      p = seq x n + $\mathbb{Q}$  0 $\mathbb{Q}$ 

      q :  $\mathbb{Q}$ 
      q = seq x n

      pq $\simeq$  : p  $\simeq_{\mathbb{Q}}$  q
      pq $\simeq$  = + $\mathbb{Q}$ -zero-right (seq x n)

      d $\simeq$ 0 : dist $\mathbb{Q}$  p q  $\simeq_{\mathbb{Q}}$  0 $\mathbb{Q}$ 
      d $\simeq$ 0 = dist $\mathbb{Q}$ - $\simeq$ 0 pq $\simeq$ 

```

```

d≤0 : distQ p q ≤Q 0Q
d≤0 = ≈Q→≤Ql {distQ p q} {0Q} d≈0
in
≤<Q→<Q {distQ p q} {0Q} {ε} d≤0 εpos)
}

-- § Left identity for real addition
+R-zero-left : (x : R) → (0R +R x) ≈R x
+R-zero-left x = record
{ conv0 = λ ε ∈ pos →
  zero , (λ n _ →
    let
      p : Q
      p = 0Q +Q seq x n

      q : Q
      q = seq x n

      pq≈ : p ≈Q q
      pq≈ = +Q-zero-left (seq x n)

      d≈0 : distQ p q ≈Q 0Q
      d≈0 = distQ-≈0 pq≈

      d≤0 : distQ p q ≤Q 0Q
      d≤0 = ≈Q→≤Ql {distQ p q} {0Q} d≈0
    in
      ≤<Q→<Q {distQ p q} {0Q} {ε} d≤0 εpos)
  }

-- § Right inverse for real addition
+R-inv-right : (x : R) → (x +R (-R x)) ≈R 0R
+R-inv-right x = record
{ conv0 = λ ε ∈ pos →
  zero , (λ n _ →
    let
      p : Q
      p = seq x n +Q (-Q (seq x n))

      q : Q
      q = 0Q

      pq≈ : p ≈Q q
      pq≈ = +Q-inv-right (seq x n)

      d≈0 : distQ p q ≈Q 0Q
      d≈0 = distQ-≈0 pq≈

      d≤0 : distQ p q ≤Q 0Q
      d≤0 = ≈Q→≤Ql {distQ p q} {0Q} d≈0
    in
      ≤<Q→<Q {distQ p q} {0Q} {ε} d≤0 εpos)
  }

```

Eventual boundedness and multiplication

Multiplication requires one ingredient that addition did not need: eventual boundedness of Cauchy sequences. A product difference $f_n g_n - f_m g_m$ splits into two terms, and each term controls one factor by a tail bound while the other contributes the small difference. For a Cauchy sequence such a bound is obtained by looking at tolerance 1. With these bounds in hand, the standard decomposition $fg - f'g' = f(g - g') + g'(f - f')$ together with an $\varepsilon/4$ budget shows that the pointwise product is again Cauchy.

```
-- § Cauchy sequences are eventually bounded
IsCauchy-eventually-bounded :
(f : ℕ → ℚ) → IsCauchy f →
Σ ℕ (λ N → Σ ℚ (λ B → (n : ℕ) → N ≤ n → distℚ (f n) 0ℚ ≤ ℚ B))
IsCauchy-eventually-bounded f ic =
let
  pack = IsCauchy.cauchy ic 1ℚ 0ℚ < 1ℚ
  N : ℕ
  N = fst pack
  conv = snd pack

  B : ℚ
  B = distℚ (f N) 0ℚ + ℚ 1ℚ
in
N , (B , (λ n N ≤ n →
let
  d<1 : distℚ (f n) (f N) < ℚ 1ℚ
  d<1 = conv n N N ≤ n (≤-refl N)

  d≤1 : distℚ (f n) (f N) ≤ ℚ 1ℚ
  d≤1 = <ℚ → ≤ℚ {distℚ (f n) (f N)} {1ℚ} d<1

  tri : distℚ (f n) 0ℚ ≤ ℚ (distℚ (f n) (f N) + ℚ distℚ (f N) 0ℚ)
  tri = distℚ-triangle (f n) (f N) 0ℚ

  step : (distℚ (f n) (f N) + ℚ distℚ (f N) 0ℚ) ≤ ℚ (1ℚ + ℚ distℚ (f N) 0ℚ)
  step = ≤ℚ+ℚ-mono-right (distℚ (f n) (f N)) 1ℚ (distℚ (f N) 0ℚ) d≤1

  comm : (1ℚ + ℚ distℚ (f N) 0ℚ) ≈ ℚ B
  comm = +ℚ-comm 1ℚ (distℚ (f N) 0ℚ)
in
≤ℚ-trans {distℚ (f n) 0ℚ} {(1ℚ + ℚ distℚ (f N) 0ℚ)} {B}
(≤ℚ-trans {distℚ (f n) 0ℚ} {(distℚ (f n) (f N) + ℚ distℚ (f N) 0ℚ)} {(1ℚ + ℚ distℚ (f N) 0ℚ)} tri step)
(≈ℚ → ≤ℚl {(1ℚ + ℚ distℚ (f N) 0ℚ)} {B} comm)))
```

Real multiplication. Real multiplication is therefore defined pointwise. Its Cauchy proof combines the eventual bounds just extracted with the same quartering strategy already used for addition.

```
-- § Real multiplication with full Cauchy proof
_·ℝ_ : ℝ → ℝ → ℝ
```

```

x · ℝ y = mkℝ (λ n → seq x n *ℚ seq y n) record
{ cauchy = λ ε ∈ pos →
  let
    -- § Phase 1: Budget allocation
    εq :  $\mathbb{Q}$ 
    εq =  $\epsilon_{\text{Quarter}}$  ε

    εqPos :  $0\mathbb{Q} < \mathbb{Q} \epsilon q$ 
    εqPos =  $\epsilon_{\text{Quarter-pos}}$  ε

    -- § Phase 2: Eventual bounds on both factors
    bxPack = IsCauchy-eventually-bounded (seq x) (isCauchy x)
    byPack = IsCauchy-eventually-bounded (seq y) (isCauchy y)

    Nx :  $\mathbb{N}$ 
    Nx = fst bxPack

    Ny :  $\mathbb{N}$ 
    Ny = fst byPack

    Bx :  $\mathbb{Q}$ 
    Bx = fst (snd bxPack)

    By :  $\mathbb{Q}$ 
    By = fst (snd byPack)

    bxBound : (n :  $\mathbb{N}$ ) → Nx ≤ n → distℚ (seq x n)  $0\mathbb{Q} \leq \mathbb{Q} Bx$ 
    bxBound = snd (snd bxPack)

    byBound : (n :  $\mathbb{N}$ ) → Ny ≤ n → distℚ (seq y n)  $0\mathbb{Q} \leq \mathbb{Q} By$ 
    byBound = snd (snd byPack)

    -- § Phase 3: Integer ceiling bounds
    BxNonneg :  $0\mathbb{Q} \leq \mathbb{Q} Bx$ 
    BxNonneg =
      ≤ℚ-trans { $0\mathbb{Q}$ } {distℚ (seq x Nx)  $0\mathbb{Q}$ } {Bx}
      (distℚ-nonneg (seq x Nx)  $0\mathbb{Q}$ )
      (bxBound Nx (≤-refl Nx))

    ByNonneg :  $0\mathbb{Q} \leq \mathbb{Q} By$ 
    ByNonneg =
      ≤ℚ-trans { $0\mathbb{Q}$ } {distℚ (seq y Ny)  $0\mathbb{Q}$ } {By}
      (distℚ-nonneg (seq y Ny)  $0\mathbb{Q}$ )
      (byBound Ny (≤-refl Ny))

    mx :  $\mathbb{N}$ 
    mx = fst (nonneg-bound-sucInt Bx BxNonneg)

    my :  $\mathbb{N}$ 
    my = fst (nonneg-bound-sucInt By ByNonneg)

    Ix :  $\mathbb{Q}$ 
    Ix = fromℕℤ (suc mx) / one+

    Iy :  $\mathbb{Q}$ 

```

```

Iy = fromℕℤ (suc my) / one+

Bx≤Ix : Bx ≤ℚ Ix
Bx≤Ix = snd (nonneg-bound-sucInt Bx BxNonneg)

By≤Iy : By ≤ℚ Iy
By≤Iy = snd (nonneg-bound-sucInt By ByNonneg)

IxNonneg : 0ℚ ≤ℚ Ix
IxNonneg = ≤ℚ-trans {0ℚ} {Bx} {Ix} BxNonneg Bx≤Ix

IyNonneg : 0ℚ ≤ℚ Iy
IyNonneg = ≤ℚ-trans {0ℚ} {By} {Iy} ByNonneg By≤Iy

-- § Phase 4: δ-scaling
dyPack = δ-scale-suc eq ∈ qPos mx
dxPack = δ-scale-suc eq ∈ qPos my

δy : ℚ
δy = fst dyPack

δx : ℚ
δx = fst dxPack

δyPos : 0ℚ <ℚ δy
δyPos = fst (snd dyPack)

δxPos : 0ℚ <ℚ δx
δxPos = fst (snd dxPack)

δyIx<eq : (δy *ℚ Ix) <ℚ eq ∈ q
δyIx<eq = snd (snd dyPack)

δxIy<eq : (δx *ℚ Iy) <ℚ eq ∈ q
δxIy<eq = snd (snd dxPack)

-- Cauchy moduli at δx, δy
cxPack = IsCauchy.cauchy (isCauchy x) δx δxPos
cyPack = IsCauchy.cauchy (isCauchy y) δy δyPos

CxN : ℕ
CxN = fst cxPack

CyN : ℕ
CyN = fst cyPack

cx : (m n : ℕ) → CxN ≤ m → CxN ≤ n → distℚ (seq x m) (seq x n) <ℚ δx
cx = snd cxPack

cy : (m n : ℕ) → CyN ≤ m → CyN ≤ n → distℚ (seq y m) (seq y n) <ℚ δy
cy = snd cyPack

-- § Phase 5: Global modulus N
N : ℕ
N = Nx +ℕ (Ny +ℕ (CxN +ℕ CyN))

```

$Nx \leq N : Nx \leq N$
 $Nx \leq N =$
 let
 $\text{step} : (Nx + \mathbb{N} \text{ zero}) \leq (Nx + \mathbb{N} (Ny + \mathbb{N} (CxN + \mathbb{N} CyN)))$
 $\text{step} = \leq + \mathbb{N} \text{-mono}^l \{a = \text{zero}\} \{b = (Ny + \mathbb{N} (CxN + \mathbb{N} CyN))\} \{z \leq n \text{ } Nx\}$
 in
 $\text{subst } (\lambda t \rightarrow t \leq (Nx + \mathbb{N} (Ny + \mathbb{N} (CxN + \mathbb{N} CyN)))) (+\mathbb{N}\text{-zero-right } Nx) \text{ step}$

$Ny \leq N : Ny \leq N$
 $Ny \leq N =$
 let
 $\text{step} : (Ny + \mathbb{N} \text{ zero}) \leq (Ny + \mathbb{N} (Nx + \mathbb{N} (CxN + \mathbb{N} CyN)))$
 $\text{step} = \leq + \mathbb{N} \text{-mono}^l \{a = \text{zero}\} \{b = (Nx + \mathbb{N} (CxN + \mathbb{N} CyN))\} \{z \leq n \text{ } Ny\}$
 $\text{base} : Ny \leq (Ny + \mathbb{N} (Nx + \mathbb{N} (CxN + \mathbb{N} CyN)))$
 $\text{base} = \text{subst } (\lambda t \rightarrow t \leq (Ny + \mathbb{N} (Nx + \mathbb{N} (CxN + \mathbb{N} CyN)))) (+\mathbb{N}\text{-zero-right } Ny) \text{ step}$

$\text{rhsEq} : (Ny + \mathbb{N} (Nx + \mathbb{N} (CxN + \mathbb{N} CyN))) \equiv (Nx + \mathbb{N} (Ny + \mathbb{N} (CxN + \mathbb{N} CyN)))$
 $\text{rhsEq} =$
 trans
 $(\text{sym } (+\mathbb{N}\text{-assoc } Ny \text{ } Nx \text{ } (CxN + \mathbb{N} CyN)))$
 $(\text{trans}$
 $(\text{cong } (\lambda t \rightarrow t + \mathbb{N} (CxN + \mathbb{N} CyN)) (+\mathbb{N}\text{-comm } Ny \text{ } Nx))$
 $(+\mathbb{N}\text{-assoc } Nx \text{ } Ny \text{ } (CxN + \mathbb{N} CyN)))$
 in
 $\text{subst } (\lambda t \rightarrow Ny \leq t) \text{ rhsEq base}$

$CxN \leq N : CxN \leq N$
 $CxN \leq N =$
 let
 $\text{step} : (CxN + \mathbb{N} \text{ zero}) \leq (CxN + \mathbb{N} (Nx + \mathbb{N} (Ny + \mathbb{N} CyN)))$
 $\text{step} = \leq + \mathbb{N} \text{-mono}^l \{a = \text{zero}\} \{b = (Nx + \mathbb{N} (Ny + \mathbb{N} CyN))\} \{z \leq n \text{ } CxN\}$
 $\text{base} : CxN \leq (CxN + \mathbb{N} (Nx + \mathbb{N} (Ny + \mathbb{N} CyN)))$
 $\text{base} = \text{subst } (\lambda t \rightarrow t \leq (CxN + \mathbb{N} (Nx + \mathbb{N} (Ny + \mathbb{N} CyN)))) (+\mathbb{N}\text{-zero-right } CxN) \text{ step}$

$\text{rhsEq} : (CxN + \mathbb{N} (Nx + \mathbb{N} (Ny + \mathbb{N} CyN))) \equiv N$
 $\text{rhsEq} =$
 trans
 $(\text{sym } (+\mathbb{N}\text{-assoc } CxN \text{ } Nx \text{ } (Ny + \mathbb{N} CyN)))$
 $(\text{trans}$
 $(\text{cong } (\lambda t \rightarrow t + \mathbb{N} (Ny + \mathbb{N} CyN)) (+\mathbb{N}\text{-comm } CxN \text{ } Nx))$
 $(\text{trans}$
 $(+\mathbb{N}\text{-assoc } Nx \text{ } CxN \text{ } (Ny + \mathbb{N} CyN))$
 $(\text{cong } (\lambda t \rightarrow Nx + \mathbb{N} t)$
 $(\text{trans}$
 $(\text{sym } (+\mathbb{N}\text{-assoc } CxN \text{ } Ny \text{ } CyN))$
 $(\text{trans}$
 $(\text{cong } (\lambda t \rightarrow t + \mathbb{N} CyN) (+\mathbb{N}\text{-comm } CxN \text{ } Ny))$
 $(+\mathbb{N}\text{-assoc } Ny \text{ } CxN \text{ } CyN))))))$
 in
 $\text{subst } (\lambda t \rightarrow CxN \leq t) \text{ rhsEq base}$

$CyN \leq N : CyN \leq N$


```

CyN ≤ N =
let
  step : (CyN +N zero) ≤ (CyN +N (Nx +N (Ny +N CxN)))
  step = ≤-+N-monol {a = zero} {b = (Nx +N (Ny +N CxN))} z ≤ n CyN

  base : CyN ≤ (CyN +N (Nx +N (Ny +N CxN)))
  base = subst (λ t → t ≤ (CyN +N (Nx +N (Ny +N CxN)))) (+N-zero-right CyN) step

  rhsEq : (CyN +N (Nx +N (Ny +N CxN))) ≡ N
  rhsEq =
    trans
      (sym (+N-assoc CyN Nx (Ny +N CxN)))
      (trans
        (cong (λ t → t +N (Ny +N CxN)) (+N-comm CyN Nx))
        (trans
          (+N-assoc Nx CyN (Ny +N CxN))
          (cong (λ t → Nx +N t)
            (trans
              (sym (+N-assoc CyN Ny CxN))
              (trans
                (cong (λ t → t +N CxN) (+N-comm CyN Ny))
                (trans
                  (+N-assoc Ny CyN CxN)
                  (cong (λ t → Ny +N t) (+N-comm CyN CxN))))))))
        in
          subst (λ t → CyN ≤ t) rhsEq base

εqNonneg : 0Q ≤ Q εq
εqNonneg = <Q → ≤ Q {0Q} {εq} εqPos

εq+εq<ε : (εq +Q εq) <Q ε
εq+εq<ε = εQuarter-double<ε ε εpos

in
N , (λ m n N ≤ m N ≤ n →
let
  -- -- Phase 6: Propagate N to all four thresholds -----
  Nx ≤ m : Nx ≤ m
  Nx ≤ m = ≤-trans Nx ≤ N N ≤ m

  Nx ≤ n : Nx ≤ n
  Nx ≤ n = ≤-trans Nx ≤ N N ≤ n

  Ny ≤ m : Ny ≤ m
  Ny ≤ m = ≤-trans Ny ≤ N N ≤ m

  Ny ≤ n : Ny ≤ n
  Ny ≤ n = ≤-trans Ny ≤ N N ≤ n

  Cx ≤ m : CxN ≤ m
  Cx ≤ m = ≤-trans CxN ≤ N N ≤ m

  Cx ≤ n : CxN ≤ n
  Cx ≤ n = ≤-trans CxN ≤ N N ≤ n

  Cy ≤ m : CyN ≤ m

```

$$Cy \leq m = \leq\text{-trans } CyN \leq N \leq m$$

$$Cy \leq n : CyN \leq n$$

$$Cy \leq n = \leq\text{-trans } CyN \leq N \leq n$$

$$dx0 \leq Bx : \text{dist}\mathbb{Q}(\text{seq } x \ m) \ 0\mathbb{Q} \leq \mathbb{Q} \ Bx$$

$$dx0 \leq Bx = \text{bxBound } m \ Nx \leq m$$

$$dy0 \leq By : \text{dist}\mathbb{Q}(\text{seq } y \ n) \ 0\mathbb{Q} \leq \mathbb{Q} \ By$$

$$dy0 \leq By = \text{byBound } n \ Ny \leq n$$

$$dx0 \leq Ix : \text{dist}\mathbb{Q}(\text{seq } x \ m) \ 0\mathbb{Q} \leq \mathbb{Q} \ Ix$$

$$dx0 \leq Ix = \leq\mathbb{Q}\text{-trans } \{\text{dist}\mathbb{Q}(\text{seq } x \ m) \ 0\mathbb{Q}\} \{Bx\} \{Ix\} \ dx0 \leq Bx \ Bx \leq Ix$$

$$dy0 \leq Iy : \text{dist}\mathbb{Q}(\text{seq } y \ n) \ 0\mathbb{Q} \leq \mathbb{Q} \ Iy$$

$$dy0 \leq Iy = \leq\mathbb{Q}\text{-trans } \{\text{dist}\mathbb{Q}(\text{seq } y \ n) \ 0\mathbb{Q}\} \{By\} \{Iy\} \ dy0 \leq By \ By \leq Iy$$

$$dy < \delta y : \text{dist}\mathbb{Q}(\text{seq } y \ m) (\text{seq } y \ n) < \mathbb{Q} \ \delta y$$

$$dy < \delta y = \text{cy } m \ n \ Cy \leq m \ Cy \leq n$$

$$dx < \delta x : \text{dist}\mathbb{Q}(\text{seq } x \ m) (\text{seq } x \ n) < \mathbb{Q} \ \delta x$$

$$dx < \delta x = \text{cx } m \ n \ Cx \leq m \ Cx \leq n$$

$$dy \leq \delta y : \text{dist}\mathbb{Q}(\text{seq } y \ m) (\text{seq } y \ n) \leq \mathbb{Q} \ \delta y$$

$$dy \leq \delta y = <\mathbb{Q} \rightarrow \leq \mathbb{Q} \ \{\text{dist}\mathbb{Q}(\text{seq } y \ m) (\text{seq } y \ n)\} \{\delta y\} \ dy < \delta y$$

$$dx \leq \delta x : \text{dist}\mathbb{Q}(\text{seq } x \ m) (\text{seq } x \ n) \leq \mathbb{Q} \ \delta x$$

$$dx \leq \delta x = <\mathbb{Q} \rightarrow \leq \mathbb{Q} \ \{\text{dist}\mathbb{Q}(\text{seq } x \ m) (\text{seq } x \ n)\} \{\delta x\} \ dx < \delta x$$

$$dy\text{Nonneg} : 0\mathbb{Q} \leq \mathbb{Q} \ \text{dist}\mathbb{Q}(\text{seq } y \ m) (\text{seq } y \ n)$$

$$dy\text{Nonneg} = \text{dist}\mathbb{Q}\text{-nonneg}(\text{seq } y \ m) (\text{seq } y \ n)$$

$$dx\text{Nonneg} : 0\mathbb{Q} \leq \mathbb{Q} \ \text{dist}\mathbb{Q}(\text{seq } x \ m) (\text{seq } x \ n)$$

$$dx\text{Nonneg} = \text{dist}\mathbb{Q}\text{-nonneg}(\text{seq } x \ m) (\text{seq } x \ n)$$

$$dx0\text{Nonneg} : 0\mathbb{Q} \leq \mathbb{Q} \ \text{dist}\mathbb{Q}(\text{seq } x \ m) \ 0\mathbb{Q}$$

$$dx0\text{Nonneg} = \text{dist}\mathbb{Q}\text{-nonneg}(\text{seq } x \ m) \ 0\mathbb{Q}$$

$$dy0\text{Nonneg} : 0\mathbb{Q} \leq \mathbb{Q} \ \text{dist}\mathbb{Q}(\text{seq } y \ n) \ 0\mathbb{Q}$$

$$dy0\text{Nonneg} = \text{dist}\mathbb{Q}\text{-nonneg}(\text{seq } y \ n) \ 0\mathbb{Q}$$

-- § Phase 7: Product distance decomposition

$$p : \mathbb{Q}$$

$$p = (\text{seq } x \ m)$$

$$q : \mathbb{Q}$$

$$q = (\text{seq } y \ m)$$

$$r : \mathbb{Q}$$

$$r = (\text{seq } x \ n)$$

$$s : \mathbb{Q}$$

$$s = (\text{seq } y \ n)$$

$$d1 : \mathbb{Q}$$

$$d1 = \text{dist}\mathbb{Q}(p * \mathbb{Q} \ q) (p * \mathbb{Q} \ s)$$

```

d2 : ℚ
d2 = distℚ (p * ℚ s) (r * ℚ s)

d1 ≤ : d1 ≤ ℚ (distℚ q s * ℚ distℚ p 0ℚ)
d1 ≤ = <ℚ → ≤ ℚ {d1} {(distℚ q s * ℚ distℚ p 0ℚ)} (distℚ-*ℚ-left p q s)

d2 ≤ : d2 ≤ ℚ (distℚ p r * ℚ distℚ s 0ℚ)
d2 ≤ = <ℚ → ≤ ℚ {d2} {(distℚ p r * ℚ distℚ s 0ℚ)} (distℚ-*ℚ-right s p r)

d1Bound : (distℚ q s * ℚ distℚ p 0ℚ) ≤ ℚ (distℚ q s * ℚ Ix)
d1Bound = ≤ ℚ-mul-nonneg-left (distℚ p 0ℚ) Ix (distℚ q s) dx0 ≤ Ix dyNonneg

d2Bound : (distℚ p r * ℚ distℚ s 0ℚ) ≤ ℚ (distℚ p r * ℚ Iy)
d2Bound = ≤ ℚ-mul-nonneg-left (distℚ s 0ℚ) Iy (distℚ p r) dy0 ≤ Iy dxNonneg

dqsIx ≤ : (distℚ q s * ℚ Ix) ≤ ℚ (δ y * ℚ Ix)
dqsIx ≤ = ≤ ℚ-mul-nonneg-right (distℚ q s) δ y Ix dy ≤ δ y IxNonneg

dprIy ≤ : (distℚ p r * ℚ Iy) ≤ ℚ (δ x * ℚ Iy)
dprIy ≤ = ≤ ℚ-mul-nonneg-right (distℚ p r) δ x Iy dx ≤ δ x IyNonneg

d1' < ε q : (distℚ q s * ℚ Ix) < ℚ ε q
d1' < ε q = ≤ < ℚ → < ℚ {(distℚ q s * ℚ Ix)} {(δ y * ℚ Ix)} {ε q} dqsIx ≤ δ y Ix < ε q

d2' < ε q : (distℚ p r * ℚ Iy) < ℚ ε q
d2' < ε q = ≤ < ℚ → < ℚ {(distℚ p r * ℚ Iy)} {(δ x * ℚ Iy)} {ε q} dprIy ≤ δ x Iy < ε q

d1 < ε q : d1 < ℚ ε q
d1 < ε q = ≤ < ℚ → < ℚ {d1} {(distℚ q s * ℚ Ix)} {ε q} (≤ ℚ-trans {d1} {(distℚ q s * ℚ distℚ p 0ℚ)} {(distℚ q s * ℚ Ix)} d1 ≤ d1Bound) d1' < ε q

d2 < ε q : d2 < ℚ ε q
d2 < ε q = ≤ < ℚ → < ℚ {d2} {(distℚ p r * ℚ Iy)} {ε q} (≤ ℚ-trans {d2} {(distℚ p r * ℚ distℚ s 0ℚ)} {(distℚ p r * ℚ Iy)} d2 ≤ d2Bound) d2' < ε q

d1Nonneg : 0ℚ ≤ ℚ d1
d1Nonneg = distℚ-nonneg (p * ℚ q) (p * ℚ s)

d2Nonneg : 0ℚ ≤ ℚ d2
d2Nonneg = distℚ-nonneg (p * ℚ s) (r * ℚ s)

d1 ≤ ε q : d1 ≤ ℚ ε q
d1 ≤ ε q = < ℚ → ≤ ℚ {d1} {ε q} d1 < ε q

d2 ≤ ε q : d2 ≤ ℚ ε q
d2 ≤ ε q = < ℚ → ≤ ℚ {d2} {ε q} d2 < ε q

sum ≤ : (d1 + ℚ d2) ≤ ℚ (ε q + ℚ ε q)
sum ≤ = ≤ ℚ-sum ≤ double-nonneg d1 d2 ε q d1Nonneg d2Nonneg ε qNonneg d1 ≤ ε q d2 ≤ ε q

sum < ε : (d1 + ℚ d2) < ℚ ε
sum < ε = ≤ < ℚ → < ℚ {(d1 + ℚ d2)} {(ε q + ℚ ε q)} {ε} sum ≤ ε q + ε q < ε

tri : distℚ (p * ℚ q) (r * ℚ s) ≤ ℚ (d1 + ℚ d2)
tri = distℚ-triangle (p * ℚ q) (p * ℚ s) (r * ℚ s)
in
< ℚ → < ℚ {distℚ (p * ℚ q) (r * ℚ s)} {(d1 + ℚ d2)} {ε} tri sum < ε

```

Multiplicative algebra laws

Once multiplication has been shown to preserve Cauchy data, its algebraic laws are inherited pointwise from $\mathbb{Q}$. As in the additive case, each proof uses the threshold $N = 0$; the corresponding rational identity makes the distance vanish at every index. Thus commutativity, associativity, the multiplicative identity, and distributivity descend directly to the quotient, giving the reals the expected commutative ring structure.

```
-- § Commutativity of real multiplication
·ℝ-comm : (x y : ℝ) → (x ·ℝ y) ≈ℝ (y ·ℝ x)
·ℝ-comm x y = record
{ conv0 = λ ∈ pos →
  zero , (λ n →
    let
      p : ℚ
      p = (seq x n) *ℚ (seq y n)

      q : ℚ
      q = (seq y n) *ℚ (seq x n)

      pq≈ : p ≈ℚ q
      pq≈ = *ℚ-comm (seq x n) (seq y n)

      d≈0 : distℚ p q ≈ℚ 0ℚ
      d≈0 = distℚ-≈0 pq≈

      d≤0 : distℚ p q ≤ℚ 0ℚ
      d≤0 = ≈ℚ→≤ℚl {distℚ p q} {0ℚ} d≈0
    in
      ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {ε} d≤0 ∈ pos)
  }

-- § Associativity of real multiplication
·ℝ-assoc : (x y z : ℝ) → ((x ·ℝ y) ·ℝ z) ≈ℝ (x ·ℝ (y ·ℝ z))
·ℝ-assoc x y z = record
{ conv0 = λ ∈ pos →
  zero , (λ n →
    let
      p : ℚ
      p = ((seq x n) *ℚ (seq y n)) *ℚ (seq z n)

      q : ℚ
      q = (seq x n) *ℚ ((seq y n) *ℚ (seq z n))

      pq≈ : p ≈ℚ q
      pq≈ = *ℚ-assoc (seq x n) (seq y n) (seq z n)

      d≈0 : distℚ p q ≈ℚ 0ℚ
      d≈0 = distℚ-≈0 pq≈

      d≤0 : distℚ p q ≤ℚ 0ℚ
```

```

    d≤0 = ≈Q→≤Ql {distQ p q} {0Q} d≈0
  in
    ≤<Q→<Q {distQ p q} {0Q} {ε} d≤0 εpos)
}

-- § Right multiplicative identity
·R-one-right : (x : ℝ) → (x ·R 1ℝ) ≈ℝ x
·R-one-right x = record
{ conv0 = λ ε εpos →
  zero , (λ n _ →
    let
      p : Q
      p = (seq x n) *Q 1Q

      q : Q
      q = seq x n

      pq≈ : p ≈Q q
      pq≈ = *Q-one-right (seq x n)

      d≈0 : distQ p q ≈Q 0Q
      d≈0 = distQ-≈0 pq≈

      d≤0 : distQ p q ≤Q 0Q
      d≤0 = ≈Q→≤Ql {distQ p q} {0Q} d≈0
    in
      ≤<Q→<Q {distQ p q} {0Q} {ε} d≤0 εpos)
}

-- § Left multiplicative identity
·R-one-left : (x : ℝ) → (1ℝ ·R x) ≈ℝ x
·R-one-left x = record
{ conv0 = λ ε εpos →
  zero , (λ n _ →
    let
      p : Q
      p = 1Q *Q (seq x n)

      q : Q
      q = seq x n

      pq≈ : p ≈Q q
      pq≈ = *Q-one-left (seq x n)

      d≈0 : distQ p q ≈Q 0Q
      d≈0 = distQ-≈0 pq≈

      d≤0 : distQ p q ≤Q 0Q
      d≤0 = ≈Q→≤Ql {distQ p q} {0Q} d≈0
    in
      ≤<Q→<Q {distQ p q} {0Q} {ε} d≤0 εpos)
}

```

Absorption and distributivity. Zero absorption on both sides and distributivity of $\cdot_{\mathbb{R}}$ over $+\mathbb{R}$ follow from the corresponding rational laws lifted through Cauchy convergence.

```
-- § Left absorption by zero
·ℝ-zero-left : (x : ℝ) → (0ℝ ·ℝ x) ≈ℝ 0ℝ
·ℝ-zero-left x = record
{ conv0 = λ ∈ pos →
  zero , (λ n _ →
    let
      p : ℚ
      p = 0ℚ * ℚ (seq x n)

      q : ℚ
      q = 0ℚ

      pq ≈ : p ≈ℚ q
      pq ≈ = *ℚ-zero-left (seq x n)

      d ≈ 0 : distℚ p q ≈ℚ 0ℚ
      d ≈ 0 = distℚ-≈0 pq ≈

      d ≤ 0 : distℚ p q ≤ℚ 0ℚ
      d ≤ 0 = ≈ℚ → ≤ℚl {distℚ p q} {0ℚ} d ≈ 0
    in
      ≤<ℚ →<ℚ {distℚ p q} {0ℚ} {ε} d ≤ 0 ∈ pos)
}

-- § Right absorption by zero
·ℝ-zero-right : (x : ℝ) → (x ·ℝ 0ℝ) ≈ℝ 0ℝ
·ℝ-zero-right x = record
{ conv0 = λ ∈ pos →
  zero , (λ n _ →
    let
      p : ℚ
      p = (seq x n) * ℚ 0ℚ

      q : ℚ
      q = 0ℚ

      pq ≈ : p ≈ℚ q
      pq ≈ = *ℚ-zero-right (seq x n)

      d ≈ 0 : distℚ p q ≈ℚ 0ℚ
      d ≈ 0 = distℚ-≈0 pq ≈

      d ≤ 0 : distℚ p q ≤ℚ 0ℚ
      d ≤ 0 = ≈ℚ → ≤ℚl {distℚ p q} {0ℚ} d ≈ 0
    in
      ≤<ℚ →<ℚ {distℚ p q} {0ℚ} {ε} d ≤ 0 ∈ pos)
}

-- § Right distributivity of multiplication over addition
·ℝ-distrib-right+ℝ : (x y z : ℝ) → (x ·ℝ (y +ℝ z)) ≈ℝ ((x ·ℝ y) +ℝ (x ·ℝ z))
```

```

·ℝ-distrib-right+ℝ x y z = record
{ conv0 = λ e ∈ pos →
  zero , (λ n _ →
    let
      p : ℚ
      p = (seq x n) *ℚ ((seq y n) +ℚ (seq z n))

      q : ℚ
      q = ((seq x n) *ℚ (seq y n)) +ℚ ((seq x n) *ℚ (seq z n))

      pq̃ : p ≈ℚ q
      pq̃ = *ℚ-distrib-right+ℚ (seq x n) (seq y n) (seq z n)

      d̃0 : distℚ p q ≈ℚ 0ℚ
      d̃0 = distℚ-≈0 pq̃

      d≤0 : distℚ p q ≤ℚ 0ℚ
      d≤0 = ≈ℚ→≤ℚl {distℚ p q} {0ℚ} d̃0
    in
      ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {e} d≤0 e pos)
}

-- § Left distributivity of multiplication over addition
·ℝ-distrib-left+ℝ : (x y z : ℝ) → ((x +ℝ y) ·ℝ z) ≈ℝ ((x ·ℝ z) +ℝ (y ·ℝ z))
·ℝ-distrib-left+ℝ x y z = record
{ conv0 = λ e ∈ pos →
  zero , (λ n _ →
    let
      p : ℚ
      p = ((seq x n) +ℚ (seq y n)) *ℚ (seq z n)

      q : ℚ
      q = ((seq x n) *ℚ (seq z n)) +ℚ ((seq y n) *ℚ (seq z n))

      pq̃ : p ≈ℚ q
      pq̃ = *ℚ-distrib-left+ℚ (seq x n) (seq y n) (seq z n)

      d̃0 : distℚ p q ≈ℚ 0ℚ
      d̃0 = distℚ-≈0 pq̃

      d≤0 : distℚ p q ≤ℚ 0ℚ
      d≤0 = ≈ℚ→≤ℚl {distℚ p q} {0ℚ} d̃0
    in
      ≤<ℚ→<ℚ {distℚ p q} {0ℚ} {e} d≤0 e pos)
}

```

Congruence of multiplication and addition

The operations introduced so far still live on representatives, so one must verify that they respect the equivalence relation. For addition this is a direct application of the triangle inequality. For multiplication one repeats the same product decomposition as before, now with mixed representatives and the corresponding eventual bounds.

```

-- § Multiplication respects  $\simeq_{\mathbb{R}}$ 
·ℝ-resp- $\simeq_{\mathbb{R}}$  : {x x' y y' : ℝ} → x  $\simeq_{\mathbb{R}}$  x' → y  $\simeq_{\mathbb{R}}$  y' → (x ·ℝ y)  $\simeq_{\mathbb{R}}$  (x' ·ℝ y')
·ℝ-resp- $\simeq_{\mathbb{R}}$  {x} {x'} {y} {y'} x  $\simeq$  x' y  $\simeq$  y' = record
{ conv0 = λ ∈ ∈ pos →
  let
    -- § Phase 1: Budget allocation
    ∈q : ℚ
    ∈q = ∈Quarter ∈

    ∈qPos : 0ℚ <ℚ ∈q
    ∈qPos = ∈Quarter-pos ∈

    ∈qNonneg : 0ℚ ≤ℚ ∈q
    ∈qNonneg = <ℚ → ≤ℚ {0ℚ} {∈q} ∈qPos

    ∈q+∈q<∈ : (∈q +ℚ ∈q) <ℚ ∈
    ∈q+∈q<∈ = ∈Quarter-double<∈ ∈ ∈pos

    -- § Phase 2: Eventual bounds on y and x'
    byPack : Σ ℕ (λ N → Σ ℚ (λ B → (n : ℕ) → N ≤ n → distℚ (seq y n) 0ℚ ≤ℚ B))
    byPack = lsCauchy-eventually-bounded (seq y) (isCauchy y)

    bx'Pack : Σ ℕ (λ N → Σ ℚ (λ B → (n : ℕ) → N ≤ n → distℚ (seq x' n) 0ℚ ≤ℚ B))
    bx'Pack = lsCauchy-eventually-bounded (seq x') (isCauchy x')

    Ny0 : ℕ
    Ny0 = fst byPack

    By : ℚ
    By = fst (snd byPack)

    ByBound : (n : ℕ) → Ny0 ≤ n → distℚ (seq y n) 0ℚ ≤ℚ By
    ByBound = snd (snd byPack)

    Nx'0 : ℕ
    Nx'0 = fst bx'Pack

    Bx' : ℚ
    Bx' = fst (snd bx'Pack)

    Bx'Bound : (n : ℕ) → Nx'0 ≤ n → distℚ (seq x' n) 0ℚ ≤ℚ Bx'
    Bx'Bound = snd (snd bx'Pack)

    ByNonneg : 0ℚ ≤ℚ By
    ByNonneg =
      ≤ℚ-trans {0ℚ} {distℚ (seq y Ny0) 0ℚ} {By}
      (distℚ-nonneg (seq y Ny0) 0ℚ)
      (ByBound Ny0 (≤-refl Ny0))

    Bx'Nonneg : 0ℚ ≤ℚ Bx'
    Bx'Nonneg =
      ≤ℚ-trans {0ℚ} {distℚ (seq x' Nx'0) 0ℚ} {Bx'}
      (distℚ-nonneg (seq x' Nx'0) 0ℚ)
      (Bx'Bound Nx'0 (≤-refl Nx'0))

```



```

-- § Phase 3: Integer ceiling bounds and  $\delta$ -scaling
mYPack :  $\Sigma \mathbb{N} (\lambda m \rightarrow B_y \leq \mathbb{Q} (\text{from}\mathbb{N}\mathbb{Z} (\text{suc } m) / \text{one}^+))$ 
mYPack = nonneg-bound-sucInt B_y B_yNonneg

mX'Pack :  $\Sigma \mathbb{N} (\lambda m \rightarrow B_{x'} \leq \mathbb{Q} (\text{from}\mathbb{N}\mathbb{Z} (\text{suc } m) / \text{one}^+))$ 
mX'Pack = nonneg-bound-sucInt B_{x'} B_{x'}Nonneg

mY :  $\mathbb{N}$ 
mY = fst mYPack

KY :  $\mathbb{Q}$ 
KY = from $\mathbb{N}\mathbb{Z} (\text{suc } mY) / \text{one}^+$ 

B_y  $\leq$  KY : B_y  $\leq \mathbb{Q} KY$ 
B_y  $\leq$  KY = snd mYPack

mX' :  $\mathbb{N}$ 
mX' = fst mX'Pack

KX' :  $\mathbb{Q}$ 
KX' = from $\mathbb{N}\mathbb{Z} (\text{suc } mX') / \text{one}^+$ 

B_{x'}  $\leq$  KX' : B_{x'}  $\leq \mathbb{Q} KX'$ 
B_{x'}  $\leq$  KX' = snd mX'Pack

 $\delta x$ Pack :  $\Sigma \mathbb{Q} (\lambda \delta \rightarrow (0\mathbb{Q} < \mathbb{Q} \delta) \times ((\delta * \mathbb{Q} KY) < \mathbb{Q} \epsilon q))$ 
 $\delta x$ Pack =  $\delta$ -scale-suc  $\epsilon q \in qPos mY$ 

 $\delta y$ Pack :  $\Sigma \mathbb{Q} (\lambda \delta \rightarrow (0\mathbb{Q} < \mathbb{Q} \delta) \times ((\delta * \mathbb{Q} KX') < \mathbb{Q} \epsilon q))$ 
 $\delta y$ Pack =  $\delta$ -scale-suc  $\epsilon q \in qPos mX'$ 

 $\delta x$  :  $\mathbb{Q}$ 
 $\delta x$  = fst  $\delta x$ Pack

 $\delta xPos$  :  $0\mathbb{Q} < \mathbb{Q} \delta x$ 
 $\delta xPos$  = fst (snd  $\delta x$ Pack)

 $\delta xNonneg$  :  $0\mathbb{Q} \leq \mathbb{Q} \delta x$ 
 $\delta xNonneg$  =  $<\mathbb{Q} \rightarrow \leq \mathbb{Q} \{0\mathbb{Q}\} \{\delta x\} \delta xPos$ 

 $\delta xKY < \epsilon q$  :  $(\delta x * \mathbb{Q} KY) < \mathbb{Q} \epsilon q$ 
 $\delta xKY < \epsilon q$  = snd (snd  $\delta x$ Pack)

 $\delta y$  :  $\mathbb{Q}$ 
 $\delta y$  = fst  $\delta y$ Pack

 $\delta yPos$  :  $0\mathbb{Q} < \mathbb{Q} \delta y$ 
 $\delta yPos$  = fst (snd  $\delta y$ Pack)

 $\delta yNonneg$  :  $0\mathbb{Q} \leq \mathbb{Q} \delta y$ 
 $\delta yNonneg$  =  $<\mathbb{Q} \rightarrow \leq \mathbb{Q} \{0\mathbb{Q}\} \{\delta y\} \delta yPos$ 

 $\delta yKX' < \epsilon q$  :  $(\delta y * \mathbb{Q} KX') < \mathbb{Q} \epsilon q$ 
 $\delta yKX' < \epsilon q$  = snd (snd  $\delta y$ Pack)

NxPack :  $\Sigma \mathbb{N} (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow \text{dist}\mathbb{Q} (\text{seq } x n) (\text{seq } x' n) < \mathbb{Q} \delta x)$ 

```

```

NxPack =  $\_ \simeq \mathbb{R} \_ \text{conv} 0$   $x \simeq x' \delta x \delta x \text{Pos}$ 

NyPack :  $\Sigma \mathbb{N} (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow \text{dist} \mathbb{Q} (\text{seq } y \ n) (\text{seq } y' \ n) < \mathbb{Q} \delta y)$ 
NyPack =  $\_ \simeq \mathbb{R} \_ \text{conv} 0$   $y \simeq y' \delta y \delta y \text{Pos}$ 

Nx :  $\mathbb{N}$ 
Nx = fst NxPack

Ny :  $\mathbb{N}$ 
Ny = fst NyPack

NxConv :  $(n : \mathbb{N}) \rightarrow Nx \leq n \rightarrow \text{dist} \mathbb{Q} (\text{seq } x \ n) (\text{seq } x' \ n) < \mathbb{Q} \delta x$ 
NxConv = snd NxPack

NyConv :  $(n : \mathbb{N}) \rightarrow Ny \leq n \rightarrow \text{dist} \mathbb{Q} (\text{seq } y \ n) (\text{seq } y' \ n) < \mathbb{Q} \delta y$ 
NyConv = snd NyPack

-- § Phase 4: Global modulus N
N :  $\mathbb{N}$ 
N = ((Nx + $\mathbb{N}$  Ny) + $\mathbb{N}$  Ny0) + $\mathbb{N}$  Nx'0

 $\leq \text{-self} + \mathbb{N} : (m \ n : \mathbb{N}) \rightarrow m \leq (m + \mathbb{N} \ n)$ 
 $\leq \text{-self} + \mathbb{N} \ m \ n =$ 
let
   $\text{mono} : (m + \mathbb{N} \ \text{zero}) \leq (m + \mathbb{N} \ n)$ 
   $\text{mono} = \leq + \mathbb{N} \text{-mono}^l \{a = \text{zero}\} \{b = n\} \ z \leq n \ m$ 
in
subst  $(\lambda t \rightarrow t \leq (m + \mathbb{N} \ n)) \ (+ \mathbb{N} \text{-zero-right } m) \ \text{mono}$ 

Nx $\leq$ N : Nx  $\leq$  N
Nx $\leq$ N =
let
   $\text{step}_1 : Nx \leq (Nx + \mathbb{N} \ Ny)$ 
   $\text{step}_1 =$ 
let
     $\text{mono} : (Nx + \mathbb{N} \ \text{zero}) \leq (Nx + \mathbb{N} \ Ny)$ 
     $\text{mono} = \leq + \mathbb{N} \text{-mono}^l \{a = \text{zero}\} \{b = Ny\} \ z \leq n \ Nx$ 
in
subst  $(\lambda t \rightarrow t \leq (Nx + \mathbb{N} \ Ny)) \ (+ \mathbb{N} \text{-zero-right } Nx) \ \text{mono}$ 

 $\text{step}_2 : (Nx + \mathbb{N} \ Ny) \leq N$ 
 $\text{step}_2 =$ 
 $\leq \text{-trans}$ 
   $(\leq \text{-self} + \mathbb{N} \ (Nx + \mathbb{N} \ Ny) \ Ny0)$ 
   $(\leq \text{-self} + \mathbb{N} \ ((Nx + \mathbb{N} \ Ny) + \mathbb{N} \ Ny0) \ Nx'0)$ 
in
 $\leq \text{-trans} \ \text{step}_1 \ \text{step}_2$ 

Ny $\leq$ N : Ny  $\leq$  N
Ny $\leq$ N =
let
   $\text{step}_1 : Ny \leq (Ny + \mathbb{N} \ Nx)$ 
   $\text{step}_1 =$ 
let
     $\text{mono} : (Ny + \mathbb{N} \ \text{zero}) \leq (Ny + \mathbb{N} \ Nx)$ 

```

```

mono = ≤-+ℕ-monol {a = zero} {b = Nx} z ≤ n Ny

base : Ny ≤ (Ny +ℕ Nx)
base = subst (λ t → t ≤ (Ny +ℕ Nx)) (+ℕ-zero-right Ny) mono
in
subst (λ t → Ny ≤ t) (+ℕ-comm Ny Nx) base

step2 : (Nx +ℕ Ny) ≤ N
step2 =
≤-trans
(≤-self+ℕ (Nx +ℕ Ny) Ny0)
(≤-self+ℕ ((Nx +ℕ Ny) +ℕ Ny0) Nx'0)
in
≤-trans step1 step2

Ny0 ≤ N : Ny0 ≤ N
Ny0 ≤ N =
let
step1 : Ny0 ≤ ((Nx +ℕ Ny) +ℕ Ny0)
step1 =
subst (λ t → Ny0 ≤ t) (+ℕ-comm Ny0 (Nx +ℕ Ny)) (≤-self+ℕ Ny0 (Nx +ℕ Ny))

step2 : ((Nx +ℕ Ny) +ℕ Ny0) ≤ N
step2 = ≤-self+ℕ ((Nx +ℕ Ny) +ℕ Ny0) Nx'0
in
≤-trans step1 step2

Nx'0 ≤ N : Nx'0 ≤ N
Nx'0 ≤ N =
let
base : Nx'0 ≤ (Nx'0 +ℕ ((Nx +ℕ Ny) +ℕ Ny0))
base = ≤-self+ℕ Nx'0 (((Nx +ℕ Ny) +ℕ Ny0))
in
subst (λ t → Nx'0 ≤ t) (+ℕ-comm Nx'0 (((Nx +ℕ Ny) +ℕ Ny0))) base
in
N , (λ n N ≤ n →
let
Nx ≤ n : Nx ≤ n
Nx ≤ n = ≤-trans Nx ≤ N N ≤ n

Ny ≤ n : Ny ≤ n
Ny ≤ n = ≤-trans Ny ≤ N N ≤ n

Ny0 ≤ n : Ny0 ≤ n
Ny0 ≤ n = ≤-trans Ny0 ≤ N N ≤ n

Nx'0 ≤ n : Nx'0 ≤ n
Nx'0 ≤ n = ≤-trans Nx'0 ≤ N N ≤ n

xn : ℚ
xn = seq x n

x'n : ℚ
x'n = seq x' n

```

```

yn :  $\mathbb{Q}$ 
yn = seq y n

y'n :  $\mathbb{Q}$ 
y'n = seq y' n

dxx' :  $\mathbb{Q}$ 
dxx' = dist $\mathbb{Q}$  xn x'n

dyy' :  $\mathbb{Q}$ 
dyy' = dist $\mathbb{Q}$  yn y'n

Iy :  $\mathbb{Q}$ 
Iy = dist $\mathbb{Q}$  yn 0 $\mathbb{Q}$ 

Ix' :  $\mathbb{Q}$ 
Ix' = dist $\mathbb{Q}$  x'n 0 $\mathbb{Q}$ 

dxx' <  $\delta x$  : dxx' <  $\mathbb{Q}$   $\delta x$ 
dxx' <  $\delta x$  = NxConv n Nx ≤ n

dyy' <  $\delta y$  : dyy' <  $\mathbb{Q}$   $\delta y$ 
dyy' <  $\delta y$  = NyConv n Ny ≤ n

Iy ≤ By : Iy ≤  $\mathbb{Q}$  By
Iy ≤ By = ByBound n Ny 0 ≤ n

Ix' ≤ Bx' : Ix' ≤  $\mathbb{Q}$  Bx'
Ix' ≤ Bx' = Bx'Bound n Nx 0 ≤ n

IyNonneg : 0 $\mathbb{Q}$  ≤  $\mathbb{Q}$  Iy
IyNonneg = dist $\mathbb{Q}$ -nonneg yn 0 $\mathbb{Q}$ 

Ix'Nonneg : 0 $\mathbb{Q}$  ≤  $\mathbb{Q}$  Ix'
Ix'Nonneg = dist $\mathbb{Q}$ -nonneg x'n 0 $\mathbb{Q}$ 

-- § Phase 6: Product decomposition via midpoint x'.y
dxx' ≤  $\delta x$  : dxx' ≤  $\mathbb{Q}$   $\delta x$ 
dxx' ≤  $\delta x$  = < $\mathbb{Q}$ →≤ $\mathbb{Q}$  {dxx'} { $\delta x$ } dxx' <  $\delta x$ 

dyy' ≤  $\delta y$  : dyy' ≤  $\mathbb{Q}$   $\delta y$ 
dyy' ≤  $\delta y$  = < $\mathbb{Q}$ →≤ $\mathbb{Q}$  {dyy'} { $\delta y$ } dyy' <  $\delta y$ 

Iy ≤ KY : Iy ≤  $\mathbb{Q}$  KY
Iy ≤ KY = ≤ $\mathbb{Q}$ -trans {Iy} {By} {KY} Iy ≤ By By ≤ KY

Ix' ≤ KX' : Ix' ≤  $\mathbb{Q}$  KX'
Ix' ≤ KX' = ≤ $\mathbb{Q}$ -trans {Ix'} {Bx'} {KX'} Ix' ≤ Bx' Bx' ≤ KX'

d1 :  $\mathbb{Q}$ 
d1 = dist $\mathbb{Q}$  (xn *  $\mathbb{Q}$  yn) (x'n *  $\mathbb{Q}$  yn)

d2 :  $\mathbb{Q}$ 
d2 = dist $\mathbb{Q}$  (x'n *  $\mathbb{Q}$  yn) (x'n *  $\mathbb{Q}$  y'n)

d1Nonneg : 0 $\mathbb{Q}$  ≤  $\mathbb{Q}$  d1

```

$$\begin{aligned}
d1Nonneg &= \text{distQ-nonneg } (xn * \mathbb{Q} \ yn) \ (x'n * \mathbb{Q} \ yn) \\
d2Nonneg &: 0\mathbb{Q} \leq \mathbb{Q} \ d2 \\
d2Nonneg &= \text{distQ-nonneg } (x'n * \mathbb{Q} \ yn) \ (x'n * \mathbb{Q} \ y'n) \\
d1\leq\text{scaled} &: d1 \leq \mathbb{Q} \ (dxx' * \mathbb{Q} \ Iy) \\
d1\leq\text{scaled} &= \simeq\mathbb{Q}\rightarrow\leq\mathbb{Q}^I \ \{d1\} \ \{(dxx' * \mathbb{Q} \ Iy)\} \ (\text{distQ-*Q-right } yn \ xn \ x'n) \\
d2\leq\text{scaled} &: d2 \leq \mathbb{Q} \ (dyy' * \mathbb{Q} \ Ix') \\
d2\leq\text{scaled} &= \simeq\mathbb{Q}\rightarrow\leq\mathbb{Q}^I \ \{d2\} \ \{(dyy' * \mathbb{Q} \ Ix')\} \ (\text{distQ-*Q-left } x'n \ yn \ y'n) \\
step1 &: (dxx' * \mathbb{Q} \ Iy) \leq \mathbb{Q} \ (\delta x * \mathbb{Q} \ Iy) \\
step1 &= \leq\mathbb{Q}\text{-mul-nonneg-right } dxx' \ \delta x \ Iy \ dxx' \leq \delta x \ IyNonneg \\
step2 &: (\delta x * \mathbb{Q} \ Iy) \leq \mathbb{Q} \ (\delta x * \mathbb{Q} \ KY) \\
step2 &= \leq\mathbb{Q}\text{-mul-nonneg-left } Iy \ KY \ \delta x \ Iy \leq KY \ \delta xNonneg \\
scaled1\leq &: (dxx' * \mathbb{Q} \ Iy) \leq \mathbb{Q} \ (\delta x * \mathbb{Q} \ KY) \\
scaled1\leq &= \leq\mathbb{Q}\text{-trans } \{(dxx' * \mathbb{Q} \ Iy)\} \ \{(\delta x * \mathbb{Q} \ Iy)\} \ \{(\delta x * \mathbb{Q} \ KY)\} \ step1 \ step2 \\
scaled1<\epsilon q &: (dxx' * \mathbb{Q} \ Iy) <\mathbb{Q} \ \epsilon q \\
scaled1<\epsilon q &= \leq<\mathbb{Q}\rightarrow<\mathbb{Q} \ \{(dxx' * \mathbb{Q} \ Iy)\} \ \{(\delta x * \mathbb{Q} \ KY)\} \ \{\epsilon q\} \ scaled1\leq \ \delta xKY<\epsilon q \\
d1<\epsilon q &: d1 <\mathbb{Q} \ \epsilon q \\
d1<\epsilon q &= \leq<\mathbb{Q}\rightarrow<\mathbb{Q} \ \{d1\} \ \{(\delta x * \mathbb{Q} \ KY)\} \ \{\epsilon q\} \ (\leq\mathbb{Q}\text{-trans } \{d1\} \ \{(dxx' * \mathbb{Q} \ Iy)\} \ \{(\delta x * \mathbb{Q} \ KY)\} \ d1\leq\text{scaled} \ (\leq\mathbb{Q}\text{-trans } \{(dxx' * \mathbb{Q} \ Iy)\} \ \{(\delta x * \mathbb{Q} \ Iy)\} \ \{(\delta x * \mathbb{Q} \ KY)\}) \\
step1' &: (dyy' * \mathbb{Q} \ Ix') \leq \mathbb{Q} \ (\delta y * \mathbb{Q} \ Ix') \\
step1' &= \leq\mathbb{Q}\text{-mul-nonneg-right } dyy' \ \delta y \ Ix' \ dyy' \leq \delta y \ Ix'Nonneg \\
step2' &: (\delta y * \mathbb{Q} \ Ix') \leq \mathbb{Q} \ (\delta y * \mathbb{Q} \ KX') \\
step2' &= \leq\mathbb{Q}\text{-mul-nonneg-left } Ix' \ KX' \ \delta y \ Ix' \leq KX' \ \delta yNonneg \\
scaled2\leq &: (dyy' * \mathbb{Q} \ Ix') \leq \mathbb{Q} \ (\delta y * \mathbb{Q} \ KX') \\
scaled2\leq &= \leq\mathbb{Q}\text{-trans } \{(dyy' * \mathbb{Q} \ Ix')\} \ \{(\delta y * \mathbb{Q} \ Ix')\} \ \{(\delta y * \mathbb{Q} \ KX')\} \ step1' \ step2' \\
scaled2<\epsilon q &: (dyy' * \mathbb{Q} \ Ix') <\mathbb{Q} \ \epsilon q \\
scaled2<\epsilon q &= \leq<\mathbb{Q}\rightarrow<\mathbb{Q} \ \{(dyy' * \mathbb{Q} \ Ix')\} \ \{(\delta y * \mathbb{Q} \ KX')\} \ \{\epsilon q\} \ scaled2\leq \ \delta yKX'<\epsilon q \\
d2<\epsilon q &: d2 <\mathbb{Q} \ \epsilon q \\
d2<\epsilon q &= \leq<\mathbb{Q}\rightarrow<\mathbb{Q} \ \{d2\} \ \{(dyy' * \mathbb{Q} \ Ix')\} \ \{\epsilon q\} \ d2\leq\text{scaled} \ scaled2<\epsilon q \\
d1\leq\epsilon q &: d1 \leq \mathbb{Q} \ \epsilon q \\
d1\leq\epsilon q &= <\mathbb{Q}\rightarrow\leq\mathbb{Q} \ \{d1\} \ \{\epsilon q\} \ d1<\epsilon q \\
d2\leq\epsilon q &: d2 \leq \mathbb{Q} \ \epsilon q \\
d2\leq\epsilon q &= <\mathbb{Q}\rightarrow\leq\mathbb{Q} \ \{d2\} \ \{\epsilon q\} \ d2<\epsilon q \\
sum\leq &: (d1 + \mathbb{Q} \ d2) \leq \mathbb{Q} \ (\epsilon q + \mathbb{Q} \ \epsilon q) \\
sum\leq &= \leq\mathbb{Q}\text{-sum}\leq\text{double-nonneg } d1 \ d2 \ \epsilon q \ d1Nonneg \ d2Nonneg \ \epsilon qNonneg \ d1\leq\epsilon q \ d2\leq\epsilon q \\
sum<\epsilon &: (d1 + \mathbb{Q} \ d2) <\mathbb{Q} \ \epsilon \\
sum<\epsilon &= \\
&\leq<\mathbb{Q}\rightarrow<\mathbb{Q} \ \{(d1 + \mathbb{Q} \ d2)\} \ \{(\epsilon q + \mathbb{Q} \ \epsilon q)\} \ \{\epsilon\} \\
&\quad sum\leq \ \epsilon q + \epsilon q < \epsilon \\
tri &: \text{distQ } (xn * \mathbb{Q} \ yn) \ (x'n * \mathbb{Q} \ y'n) \leq \mathbb{Q} \ (d1 + \mathbb{Q} \ d2) \\
tri &= \text{distQ-triangle } (xn * \mathbb{Q} \ yn) \ (x'n * \mathbb{Q} \ yn) \ (x'n * \mathbb{Q} \ y'n) \\
in & \\
&\leq<\mathbb{Q}\rightarrow<\mathbb{Q} \ \{\text{distQ } (xn * \mathbb{Q} \ yn) \ (x'n * \mathbb{Q} \ y'n)\} \ \{(d1 + \mathbb{Q} \ d2)\} \ \{\epsilon\} \\
&\quad tri \ sum < \epsilon \\
&\}
\end{aligned}$$

Additive congruence. Addition is simpler. One uses $\varepsilon/4$ -convergence in both sequences and closes the estimate with the triangle inequality.

```
-- § Addition respects  $\simeq_{\mathbb{R}}$ 
+ $\mathbb{R}$ -resp- $\simeq_{\mathbb{R}}$  :
{ $x$   $x'$   $y$   $y'$  :  $\mathbb{R}$ }  $\rightarrow$ 
 $x \simeq_{\mathbb{R}} x' \rightarrow$ 
 $y \simeq_{\mathbb{R}} y' \rightarrow$ 
 $(x +_{\mathbb{R}} y) \simeq_{\mathbb{R}} (x' +_{\mathbb{R}} y')$ 
+ $\mathbb{R}$ -resp- $\simeq_{\mathbb{R}}$  { $x$ } { $x'$ } { $y$ } { $y'$ }  $x \simeq x'$   $y \simeq y'$  = record
{ conv0 =  $\lambda \epsilon \in pos \rightarrow$ 
  let
    -- § Phase 1: Budget allocation
     $\epsilon q$  :  $\mathbb{Q}$ 
     $\epsilon q = \epsilon Quarter \epsilon$ 

     $\epsilon q Pos$  :  $0\mathbb{Q} <_{\mathbb{Q}} \epsilon q$ 
     $\epsilon q Pos = \epsilon Quarter-pos \epsilon$ 

     $\epsilon q Nonneg$  :  $0\mathbb{Q} \leq_{\mathbb{Q}} \epsilon q$ 
     $\epsilon q Nonneg = <_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}} \{0\mathbb{Q}\} \{\epsilon q\} \epsilon q Pos$ 

     $\epsilon q + \epsilon q < \epsilon$  :  $(\epsilon q +_{\mathbb{Q}} \epsilon q) <_{\mathbb{Q}} \epsilon$ 
     $\epsilon q + \epsilon q < \epsilon = \epsilon Quarter-double < \epsilon \epsilon \in pos$ 

     $NxPack$  :  $\Sigma \mathbb{N} (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow dist_{\mathbb{Q}} (seq\ x\ n) (seq\ x'\ n) <_{\mathbb{Q}} \epsilon q)$ 
     $NxPack = \_ \simeq_{\mathbb{R}} \_ conv0\ x \simeq x'\ \epsilon q \epsilon q Pos$ 

     $NyPack$  :  $\Sigma \mathbb{N} (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow dist_{\mathbb{Q}} (seq\ y\ n) (seq\ y'\ n) <_{\mathbb{Q}} \epsilon q)$ 
     $NyPack = \_ \simeq_{\mathbb{R}} \_ conv0\ y \simeq y'\ \epsilon q \epsilon q Pos$ 

     $Nx$  :  $\mathbb{N}$ 
     $Nx = fst\ NxPack$ 

     $Ny$  :  $\mathbb{N}$ 
     $Ny = fst\ NyPack$ 

     $NxConv$  :  $(n : \mathbb{N}) \rightarrow Nx \leq n \rightarrow dist_{\mathbb{Q}} (seq\ x\ n) (seq\ x'\ n) <_{\mathbb{Q}} \epsilon q$ 
     $NxConv = snd\ NxPack$ 

     $NyConv$  :  $(n : \mathbb{N}) \rightarrow Ny \leq n \rightarrow dist_{\mathbb{Q}} (seq\ y\ n) (seq\ y'\ n) <_{\mathbb{Q}} \epsilon q$ 
     $NyConv = snd\ NyPack$ 

    -- -- Phase 2: Global modulus N -----
     $N$  :  $\mathbb{N}$ 
     $N = Nx +_{\mathbb{N}} Ny$ 

     $Nx \leq N$  :  $Nx \leq N$ 
     $Nx \leq N =$ 
    let
       $mono$  :  $(Nx +_{\mathbb{N}} zero) \leq (Nx +_{\mathbb{N}} Ny)$ 
       $mono = \leq +_{\mathbb{N}} mono^l \{a = zero\} \{b = Ny\} z \leq n\ Nx$ 
    in
    subst ( $\lambda t \rightarrow t \leq (Nx +_{\mathbb{N}} Ny)$ ) (+ $\mathbb{N}$ -zero-right  $Nx$ )  $mono$ 
```

```

Ny≤N : Ny ≤ N
Ny≤N =
  let
    mono : (Ny +ℕ zero) ≤ (Ny +ℕ Nx)
    mono = ≤-+ℕ-monol {a = zero} {b = Nx} z≤n Ny

    base : Ny ≤ (Ny +ℕ Nx)
    base = subst (λ t → t ≤ (Ny +ℕ Nx)) (+ℕ-zero-right Ny) mono
  in
    subst (λ t → Ny ≤ t) (+ℕ-comm Ny Nx) base
in
N , (λ n N≤n →
  let
    -- § Phase 3: Triangle inequality on sums
    Nx≤n : Nx ≤ n
    Nx≤n = ≤-trans Nx≤N N≤n

    Ny≤n : Ny ≤ n
    Ny≤n = ≤-trans Ny≤N N≤n

    xn : ℚ
    xn = seq x n

    x'n : ℚ
    x'n = seq x' n

    yn : ℚ
    yn = seq y n

    y'n : ℚ
    y'n = seq y' n

    dx : ℚ
    dx = distℚ xn x'n

    dy : ℚ
    dy = distℚ yn y'n

    dx<eq : dx <ℚ eq
    dx<eq = NxConv n Nx≤n

    dy<eq : dy <ℚ eq
    dy<eq = NyConv n Ny≤n

    d1 : ℚ
    d1 = distℚ (xn +ℚ yn) (x'n +ℚ yn)

    d2 : ℚ
    d2 = distℚ (x'n +ℚ yn) (x'n +ℚ y'n)

    d1Nonneg : 0ℚ ≤ℚ d1
    d1Nonneg = distℚ-nonneg (xn +ℚ yn) (x'n +ℚ yn)

    d2Nonneg : 0ℚ ≤ℚ d2
    d2Nonneg = distℚ-nonneg (x'n +ℚ yn) (x'n +ℚ y'n)

```

$$\begin{aligned}
d1 \leq dx &: d1 \leq \mathbb{Q} \, dx \\
d1 \leq dx &= \simeq \mathbb{Q} \rightarrow \leq \mathbb{Q}^l \{d1\} \{dx\} \text{ (dist}\mathbb{Q}\text{-}\mathbb{Q}\text{-right } xn \, x'n \, yn) \\
\\
d2 \leq dy &: d2 \leq \mathbb{Q} \, dy \\
d2 \leq dy &= \simeq \mathbb{Q} \rightarrow \leq \mathbb{Q}^l \{d2\} \{dy\} \text{ (dist}\mathbb{Q}\text{-}\mathbb{Q}\text{-left } x'n \, yn \, y'n) \\
\\
d1 < \epsilon q &: d1 < \mathbb{Q} \, \epsilon q \\
d1 < \epsilon q &= \leq < \mathbb{Q} \rightarrow < \mathbb{Q} \{d1\} \{\epsilon q\} \, d1 \leq dx \, dx < \epsilon q \\
\\
d2 < \epsilon q &: d2 < \mathbb{Q} \, \epsilon q \\
d2 < \epsilon q &= \leq < \mathbb{Q} \rightarrow < \mathbb{Q} \{d2\} \{\epsilon q\} \, d2 \leq dy \, dy < \epsilon q \\
\\
d1 \leq \epsilon q &: d1 \leq \mathbb{Q} \, \epsilon q \\
d1 \leq \epsilon q &= < \mathbb{Q} \rightarrow \leq \mathbb{Q} \{d1\} \{\epsilon q\} \, d1 < \epsilon q \\
\\
d2 \leq \epsilon q &: d2 \leq \mathbb{Q} \, \epsilon q \\
d2 \leq \epsilon q &= < \mathbb{Q} \rightarrow \leq \mathbb{Q} \{d2\} \{\epsilon q\} \, d2 < \epsilon q \\
\\
sum \leq &: (d1 + \mathbb{Q} \, d2) \leq \mathbb{Q} \, (\epsilon q + \mathbb{Q} \, \epsilon q) \\
sum \leq &= \leq \mathbb{Q}\text{-sum} \leq \text{double-nonneg } d1 \, d2 \, \epsilon q \, d1 \text{Nonneg } d2 \text{Nonneg } \epsilon q \text{Nonneg } d1 \leq \epsilon q \, d2 \leq \epsilon q \\
\\
sum < \epsilon &: (d1 + \mathbb{Q} \, d2) < \mathbb{Q} \, \epsilon \\
sum < \epsilon &= \leq < \mathbb{Q} \rightarrow < \mathbb{Q} \{(d1 + \mathbb{Q} \, d2)\} \{(\epsilon q + \mathbb{Q} \, \epsilon q)\} \{\epsilon\} \, sum \leq \epsilon q + \epsilon q < \epsilon \\
\\
tri &: \text{dist}\mathbb{Q} \, (xn + \mathbb{Q} \, yn) \, (x'n + \mathbb{Q} \, y'n) \leq \mathbb{Q} \, (d1 + \mathbb{Q} \, d2) \\
tri &= \text{dist}\mathbb{Q}\text{-triangle } (xn + \mathbb{Q} \, yn) \, (x'n + \mathbb{Q} \, yn) \, (x'n + \mathbb{Q} \, y'n) \\
\text{in} \\
\leq & < \mathbb{Q} \rightarrow < \mathbb{Q} \{\text{dist}\mathbb{Q} \, (xn + \mathbb{Q} \, yn) \, (x'n + \mathbb{Q} \, y'n)\} \{(d1 + \mathbb{Q} \, d2)\} \{\epsilon\} \, tri \, sum < \epsilon \\
\}
\end{aligned}$$

Negation and subtraction congruence. Negation preserves distance by $\text{dist}\mathbb{Q}\text{-neg}$, and subtraction combines additive and negation congruence.

```

-- § Negation respects  $\simeq \mathbb{R}$ 
-ℝ-resp- $\simeq \mathbb{R}$  : {x x' : ℝ} → x  $\simeq \mathbb{R}$  x' → (-ℝ x)  $\simeq \mathbb{R}$  (-ℝ x')
-ℝ-resp- $\simeq \mathbb{R}$  {x} {x'} x  $\simeq$  x' = record
{ conv0 = λ ∈ pos →
  let
    NxPack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → distℚ (seq x n) (seq x' n) < ℚ ∈)
    NxPack = _ $\simeq \mathbb{R}$ _conv0 x  $\simeq$  x' ∈ pos

    Nx : ℕ
    Nx = fst NxPack

    NxConv : (n : ℕ) → Nx ≤ n → distℚ (seq x n) (seq x' n) < ℚ ∈
    NxConv = snd NxPack
  in
    Nx , (λ n Nx ≤ n →
      let
        xn : ℚ
        xn = seq x n

```



```

x'n : ℚ
x'n = seq x' n

d<ε : distℚ xn x'n <ℚ ε
d<ε = NxConv n Nx≤n

negEq : distℚ (-ℚ xn) (-ℚ x'n) ≤ℚ distℚ xn x'n
negEq = distℚ-neg xn x'n

d≤ : distℚ (-ℚ xn) (-ℚ x'n) ≤ℚ distℚ xn x'n
d≤ = ≈ℚ→≤ℚl {distℚ (-ℚ xn) (-ℚ x'n)} {distℚ xn x'n} negEq
in
≤<ℚ→<ℚ {distℚ (-ℚ xn) (-ℚ x'n)} {distℚ xn x'n} {ε} d≤ d<ε
}

-- § Subtraction respects ≈ℝ (derived from + and -)
-ℝ-resp-≈ℝ2 : {x x' y y' : ℝ} → x ≈ℝ x' → y ≈ℝ y' → (x -ℝ y) ≈ℝ (x' -ℝ y')
-ℝ-resp-≈ℝ2 {x} {x'} {y} {y'} x≈x' y≈y' =
+ℝ-resp-≈ℝ x≈x' (-ℝ-resp-≈ℝ y≈y')

```

Asymptotic Order on $\mathbb{R}$

Because real numbers are equivalence classes of Cauchy sequences, finite initial segments cannot determine order. We therefore define $x \leq_{\mathbb{R}} y$ by eventual comparison up to an arbitrary positive rational tolerance, and $x <_{\mathbb{R}} y$ by eventual comparison with a fixed positive rational margin. The following results show that both relations are well defined on the quotient.

Law 23.1: Non-strict asymptotic order

Non-strict order is expressed by eventual ε -comparison for every positive rational margin. In this form it depends only on tail behaviour and is compatible with the quotient.

```

-- § Non-strict real order: eventual ε-approximation
infix 4 _≤ℝ_ <ℝ_

record _≤ℝ_ (x y : ℝ) : Set where
  field
    leReal : (ε : ℚ) → (0ℚ <ℚ ε) → ∑ ℕ (λ N → (n : ℕ) → N ≤ n → (seq x n) ≤ℚ ((seq y n) +ℚ ε))

-- § Type alias for non-strict real order
≤ℝP : ℝ → ℝ → Set
≤ℝP = _≤ℝ_

```

Law 23.2: Strict asymptotic order

Strict order is expressed by a positive rational witness that persists on a tail. In this way strictness is encoded by a stable asymptotic gap.

```
-- § Strict real order: witnessed separation
record _<ℝ_ (x y : ℝ) : Set where
  field
    ltWitness :  $\sum \mathbb{Q} (\lambda \epsilon \rightarrow (0\mathbb{Q} < \mathbb{Q} \epsilon) \times \sum \mathbb{N} (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow ((seq\ x\ n) + \mathbb{Q} \epsilon) \leq \mathbb{Q} (seq\ y\ n)))$ 
```

Law 23.3: Strict order forces non-strict order

A strict witness immediately yields non-strict order once the positive margin has been used to obtain the eventual comparison.

```
-- § Strict order implies non-strict by forgetting the margin
<ℝ→≤ℝ : {x y : ℝ} → x <ℝ y → x ≤ℝ y
<ℝ→≤ℝ {x} {y} x<y = record
{ leReal =  $\lambda \delta \delta pos \rightarrow$ 
  let
    w :  $\sum \mathbb{Q} (\lambda \epsilon \rightarrow (0\mathbb{Q} < \mathbb{Q} \epsilon) \times \sum \mathbb{N} (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow ((seq\ x\ n) + \mathbb{Q} \epsilon) \leq \mathbb{Q} (seq\ y\ n)))$ 
    w = _<ℝ_.ltWitness x<y

     $\epsilon : \mathbb{Q}$ 
     $\epsilon = fst\ w$ 

    wRest :  $(0\mathbb{Q} < \mathbb{Q} \epsilon) \times \sum \mathbb{N} (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow ((seq\ x\ n) + \mathbb{Q} \epsilon) \leq \mathbb{Q} (seq\ y\ n))$ 
    wRest = snd w

     $\epsilon pos : 0\mathbb{Q} < \mathbb{Q} \epsilon$ 
     $\epsilon pos = fst\ wRest$ 

    pack :  $\sum \mathbb{N} (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow ((seq\ x\ n) + \mathbb{Q} \epsilon) \leq \mathbb{Q} (seq\ y\ n))$ 
    pack = snd wRest

    N :  $\mathbb{N}$ 
    N = fst pack

    conv :  $(n : \mathbb{N}) \rightarrow N \leq n \rightarrow ((seq\ x\ n) + \mathbb{Q} \epsilon) \leq \mathbb{Q} (seq\ y\ n)$ 
    conv = snd pack
  in
    N , ( $\lambda n\ N \leq n \rightarrow$ 
    let
      xn :  $\mathbb{Q}$ 
      xn = seq x n

      yn :  $\mathbb{Q}$ 
      yn = seq y n

      xn≤xn+ $\epsilon$  : xn ≤  $\mathbb{Q} (xn + \mathbb{Q} \epsilon)$ 
      xn≤xn+ $\epsilon$  = ≤ $\mathbb{Q}$ -add-nonneg-right xn  $\epsilon$  (< $\mathbb{Q}$ →≤ $\mathbb{Q}$   $\epsilon pos$ )

      xn+ $\epsilon$ ≤yn : (xn +  $\mathbb{Q} \epsilon$ ) ≤  $\mathbb{Q} yn$ 
```

```

 $xn + \epsilon \leq yn = \text{conv } n \ N \leq n$ 

 $xn \leq yn : xn \leq \mathbb{Q} \ yn$ 
 $xn \leq yn = \leq \mathbb{Q}\text{-trans } \{xn\} \{(xn + \mathbb{Q} \ \epsilon)\} \{yn\} \ xn \leq xn + \epsilon \ xn + \epsilon \leq yn$ 

 $yn \leq yn + \delta : yn \leq \mathbb{Q} \ (yn + \mathbb{Q} \ \delta)$ 
 $yn \leq yn + \delta = \leq \mathbb{Q}\text{-add-nonneg-right } yn \ \delta \ (\leq \mathbb{Q} \rightarrow \leq \mathbb{Q} \ \delta \text{ pos})$ 
in
 $\leq \mathbb{Q}\text{-trans } xn \leq yn \ yn \leq yn + \delta$ 
}

```

Law 23.4: Equivalence forces non-strict order

Equivalent representatives satisfy the same non-strict order relation. The eventual distance bounds allow the termwise comparison to be transferred from one representative to the other.

```

-- § Equivalence implies  $\leq \mathbb{R}$ 
 $\simeq \mathbb{R} \rightarrow \leq \mathbb{R} : \{x \ y : \mathbb{R}\} \rightarrow x \simeq \mathbb{R} \ y \rightarrow x \leq \mathbb{R} \ y$ 
 $\simeq \mathbb{R} \rightarrow \leq \mathbb{R} \ \{x\} \ \{y\} \ x \simeq y = \text{record}$ 
{ leReal =  $\lambda \ \epsilon \in \text{pos} \rightarrow$ 
  let
    pack :  $\Sigma \ \mathbb{N} \ (\lambda \ N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow \text{dist} \mathbb{Q} \ (\text{seq } x \ n) \ (\text{seq } y \ n) \ < \mathbb{Q} \ \epsilon)$ 
    pack =  $\_ \simeq \mathbb{R} \_ \text{conv} 0 \ x \simeq y \ \epsilon \in \text{pos}$ 

    N :  $\mathbb{N}$ 
    N = fst pack

    conv :  $(n : \mathbb{N}) \rightarrow N \leq n \rightarrow \text{dist} \mathbb{Q} \ (\text{seq } x \ n) \ (\text{seq } y \ n) \ < \mathbb{Q} \ \epsilon$ 
    conv = snd pack
  in
    N ,  $(\lambda \ n \ N \leq n \rightarrow$ 
       $\text{dist} \mathbb{Q} \leq \epsilon \rightarrow x \leq y + \epsilon \ (\text{seq } x \ n) \ (\text{seq } y \ n) \in (\leq \mathbb{Q} \rightarrow \leq \mathbb{Q} \ (\text{conv } n \ N \leq n)))$ 
    }

```

Transitivity of Strict Order

If x is eventually separated from y by a positive margin, and y from z by another, then x is eventually separated from z as well. The only technical point is to shrink the given margins so that they can be combined within a single estimate. The quartering device provides a convenient common reserve.

Law 23.5: Strict order is transitive

The two witness margins compose after a controlled shrinking step. Quartering keeps the combined estimate within the available budget and leaves a positive margin for the final comparison.

```

-- § Transitivity of strict real order
<ℝ-trans : {x y z : ℝ} → x <ℝ y → y <ℝ z → x <ℝ z
<ℝ-trans {x} {y} {z} x<y y<z = record
{ ltWitness =
let
  -- § Phase 1: Unpack both strict-order witnesses
  wxy : Σ Q (λ ε → (0Q <Q ε) × Σ N (λ N → (n : N) → N ≤ n → ((seq x n) +Q ε) ≤Q (seq y n)))
  wxy = _<ℝ_.ltWitness x<y

  ε₁ : Q
  ε₁ = fst wxy

  wxyRest : (0Q <Q ε₁) × Σ N (λ N → (n : N) → N ≤ n → ((seq x n) +Q ε₁) ≤Q (seq y n))
  wxyRest = snd wxy

  ε₁pos : 0Q <Q ε₁
  ε₁pos = fst wxyRest

  packXY : Σ N (λ N → (n : N) → N ≤ n → ((seq x n) +Q ε₁) ≤Q (seq y n))
  packXY = snd wxyRest

  Nxy : N
  Nxy = fst packXY

  convXY : (n : N) → Nxy ≤ n → ((seq x n) +Q ε₁) ≤Q (seq y n)
  convXY = snd packXY

  wyz : Σ Q (λ ε → (0Q <Q ε) × Σ N (λ N → (n : N) → N ≤ n → ((seq y n) +Q ε) ≤Q (seq z n)))
  wyz = _<ℝ_.ltWitness y<z

  ε₂ : Q
  ε₂ = fst wyz

  wyzRest : (0Q <Q ε₂) × Σ N (λ N → (n : N) → N ≤ n → ((seq y n) +Q ε₂) ≤Q (seq z n))
  wyzRest = snd wyz

  ε₂pos : 0Q <Q ε₂
  ε₂pos = fst wyzRest

  packYZ : Σ N (λ N → (n : N) → N ≤ n → ((seq y n) +Q ε₂) ≤Q (seq z n))
  packYZ = snd wyzRest

  Nyz : N
  Nyz = fst packYZ

  convYZ : (n : N) → Nyz ≤ n → ((seq y n) +Q ε₂) ≤Q (seq z n)
  convYZ = snd packYZ

  -- § Phase 2: Shrink the surviving margin
  ε : Q
  ε = εQuarter ε₁

  εpos : 0Q <Q ε
  εpos = εQuarter-pos ε₁

  N : N

```

```

N = Nxy +N Nyz
Nxy≤N : Nxy ≤ N
Nxy≤N =
let
  mono : (Nxy +N zero) ≤ (Nxy +N Nyz)
  mono = ≤+N-monol {a = zero} {b = Nyz} z≤n Nxy
in
subst (λ t → t ≤ (Nxy +N Nyz)) (+N-zero-right Nxy) mono

Nyz≤N : Nyz ≤ N
Nyz≤N =
let
  mono : (Nyz +N zero) ≤ (Nyz +N Nxy)
  mono = ≤+N-monol {a = zero} {b = Nxy} z≤n Nyz

  base : Nyz ≤ (Nyz +N Nxy)
  base = subst (λ t → t ≤ (Nyz +N Nxy)) (+N-zero-right Nyz) mono
in
subst (λ t → Nyz ≤ t) (+N-comm Nyz Nxy) base
in
ε, (εpos,
(N, (λ n N≤n →
let
  -- § Phase 3: Chain the inequalities
  Nxy≤n : Nxy ≤ n
  Nxy≤n = ≤-trans Nxy≤N N≤n

  Nyz≤n : Nyz ≤ n
  Nyz≤n = ≤-trans Nyz≤N N≤n

  xn : Q
  xn = seq x n

  yn : Q
  yn = seq y n

  zn : Q
  zn = seq z n

  xε₁ ≤ y : (xn +Q ε₁) ≤Q yn
  xε₁ ≤ y = convXY n Nxy≤n

  xε ≤ xε₁ : (xn +Q ε) ≤Q (xn +Q ε₁)
  xε ≤ xε₁ =
    ≤Q+Q-mono-left xn ε ε₁ (<Q→≤Q (εQuarter<ε ε₁ ε₁pos))

  xε ≤ y : (xn +Q ε) ≤Q yn
  xε ≤ y = ≤Q-trans {(xn +Q ε)} {(xn +Q ε₁)} {yn} xε ≤ xε₁ xε₁ ≤ y

  y ≤ y+ε₂ : yn ≤Q (yn +Q ε₂)
  y ≤ y+ε₂ = ≤Q-add-nonneg-right yn ε₂ (<Q→≤Q ε₂pos)

  xε ≤ y+ε₂ : (xn +Q ε) ≤Q (yn +Q ε₂)
  xε ≤ y+ε₂ = ≤Q-trans {(xn +Q ε)} {yn} {(yn +Q ε₂)} xε ≤ y y ≤ y+ε₂
in
≤Q-trans xε ≤ y+ε₂ (convYZ n Nyz≤n)))
}

```

Congruence of Strict Order under $\simeq_{\mathbb{R}}$

The strict relation must also be independent of the chosen representatives. Starting from a witness for $x < y$, one transports the estimate along equivalences on both sides. The transport introduces correction terms, so the original margin is reduced by repeated quartering to leave a positive remainder. This shows that strict order is well defined on the quotient.

Law 23.6: Strict order is invariant under equivalence

Repeated quartering leaves enough of the original witness margin to absorb both equivalence corrections. The strict gap therefore persists under replacement by equivalent representatives.

```
-- § Strict order respects  $\simeq_{\mathbb{R}}$  on both sides
<R-resp- $\simeq_{\mathbb{R}}$  : {x x' y y' :  $\mathbb{R}$ } → x  $\simeq_{\mathbb{R}}$  x' → y  $\simeq_{\mathbb{R}}$  y' → x <R y → x' <R y'
<R-resp- $\simeq_{\mathbb{R}}$  {x} {x'} {y} {y'} x  $\simeq$  x' y  $\simeq$  y' x < y =
let
  -- § Phase 1: Unpack the strict-order witness
  wxy = _<R_.ltWitness x < y

   $\epsilon_0$  :  $\mathbb{Q}$ 
   $\epsilon_0$  = fst wxy

  wxyRest = snd wxy

   $\epsilon_0 pos$  :  $0\mathbb{Q} <\mathbb{Q} \epsilon_0$ 
   $\epsilon_0 pos$  = fst wxyRest

  packXY = snd wxyRest

  Nxy :  $\mathbb{N}$ 
  Nxy = fst packXY

  convXY : (n :  $\mathbb{N}$ ) → Nxy ≤ n → ((seq x n) +  $\mathbb{Q} \epsilon_0$ ) ≤  $\mathbb{Q} (seq y n)$ 
  convXY = snd packXY

  -- § Phase 2: Quarter the margin twice
   $\epsilon$  :  $\mathbb{Q}$ 
   $\epsilon$  =  $\epsilon Quarter \epsilon_0$ 

   $\epsilon pos$  :  $0\mathbb{Q} <\mathbb{Q} \epsilon$ 
   $\epsilon pos$  =  $\epsilon Quarter pos \epsilon_0$ 

   $\alpha$  :  $\mathbb{Q}$ 
   $\alpha$  =  $\epsilon Quarter \epsilon$ 

   $\beta$  :  $\mathbb{Q}$ 
   $\beta$  =  $\epsilon Quarter \epsilon$ 

   $\alpha pos$  :  $0\mathbb{Q} <\mathbb{Q} \alpha$ 
```

```

 $\alpha_{pos} = \epsilon_{Quarter-pos} \epsilon$ 

 $\beta_{pos} : 0_{\mathbb{Q}} <_{\mathbb{Q}} \beta$ 
 $\beta_{pos} = \epsilon_{Quarter-pos} \epsilon$ 

-- § Phase 3: Derive closeness from equivalences
 $x' \leq x : x' \leq_{\mathbb{R}} x$ 
 $x' \leq x = \simeq_{\mathbb{R}} \rightarrow \leq_{\mathbb{R}} (\simeq_{\mathbb{R}}\text{-sym } x \simeq x')$ 

 $y \leq y' : y \leq_{\mathbb{R}} y'$ 
 $y \leq y' = \simeq_{\mathbb{R}} \rightarrow \leq_{\mathbb{R}} y \simeq y'$ 

packX =  $\_ \leq_{\mathbb{R}} \_ \text{leReal } x' \leq x \alpha_{pos}$ 
packY =  $\_ \leq_{\mathbb{R}} \_ \text{leReal } y \leq y' \beta_{pos}$ 

Nx :  $\mathbb{N}$ 
Nx = fst packX

Ny :  $\mathbb{N}$ 
Ny = fst packY

boundX :  $(n : \mathbb{N}) \rightarrow Nx \leq n \rightarrow (\text{seq } x' n) \leq_{\mathbb{Q}} ((\text{seq } x n) +_{\mathbb{Q}} \alpha)$ 
boundX = snd packX

boundY :  $(n : \mathbb{N}) \rightarrow Ny \leq n \rightarrow (\text{seq } y n) \leq_{\mathbb{Q}} ((\text{seq } y' n) +_{\mathbb{Q}} \beta)$ 
boundY = snd packY

-- § Phase 4: Global modulus N
N :  $\mathbb{N}$ 
N = Nxy + $\mathbb{N}$  (Nx + $\mathbb{N}$  Ny)

Nxy  $\leq$  N : Nxy  $\leq$  N
Nxy  $\leq$  N =
  let
    step :  $(Nxy + \mathbb{N} \text{zero}) \leq (Nxy + \mathbb{N} (Nx + \mathbb{N} Ny))$ 
    step =  $\leq\text{-}+\mathbb{N}\text{-mono}^l \{a = \text{zero}\} \{b = (Nx + \mathbb{N} Ny)\} z \leq n \text{Nxy}$ 
  in
    subst  $(\lambda t \rightarrow t \leq (Nxy + \mathbb{N} (Nx + \mathbb{N} Ny))) (+\mathbb{N}\text{-zero-right } Nxy) \text{step}$ 

Nx  $\leq$  N : Nx  $\leq$  N
Nx  $\leq$  N =
  let
    step :  $(Nx + \mathbb{N} \text{zero}) \leq (Nx + \mathbb{N} (Nxy + \mathbb{N} Ny))$ 
    step =  $\leq\text{-}+\mathbb{N}\text{-mono}^l \{a = \text{zero}\} \{b = (Nxy + \mathbb{N} Ny)\} z \leq n \text{Nx}$ 

    base :  $Nx \leq (Nx + \mathbb{N} (Nxy + \mathbb{N} Ny))$ 
    base = subst  $(\lambda t \rightarrow t \leq (Nx + \mathbb{N} (Nxy + \mathbb{N} Ny))) (+\mathbb{N}\text{-zero-right } Nx) \text{step}$ 

eq :  $(Nx + \mathbb{N} (Nxy + \mathbb{N} Ny)) \equiv (Nxy + \mathbb{N} (Nx + \mathbb{N} Ny))$ 
eq =
  trans
    (sym  $(+\mathbb{N}\text{-assoc } Nx \text{Nxy } Ny)$ )
    (trans
      (cong  $(\lambda t \rightarrow t + \mathbb{N} Ny) (+\mathbb{N}\text{-comm } Nx \text{Nxy})$ )
       $(+\mathbb{N}\text{-assoc } Nxy \text{Nx } Ny)$ )

```

```

in
subst ( $\lambda t \rightarrow Nx \leq t$ ) eq base

 $Ny \leq N : Ny \leq N$ 
 $Ny \leq N =$ 
let
  step : ( $Ny + \mathbb{N} \text{ zero}$ )  $\leq$  ( $Ny + \mathbb{N} (Nxy + \mathbb{N} Nx)$ )
  step =  $\leq\text{-}\mathbb{N}\text{-mono}^l \{a = \text{zero}\} \{b = (Nxy + \mathbb{N} Nx)\} z \leq n \text{ } Ny$ 

  base :  $Ny \leq (Ny + \mathbb{N} (Nxy + \mathbb{N} Nx))$ 
  base = subst ( $\lambda t \rightarrow t \leq (Ny + \mathbb{N} (Nxy + \mathbb{N} Nx))$ ) ( $+\mathbb{N}\text{-zero-right } Ny$ ) step

  eq : ( $Ny + \mathbb{N} (Nxy + \mathbb{N} Nx)$ )  $\equiv$  ( $Nxy + \mathbb{N} (Nx + \mathbb{N} Ny)$ )
  eq =
    trans
      (sym ( $+\mathbb{N}\text{-assoc } Ny \text{ } Nxy \text{ } Nx$ ))
      (trans
        (cong ( $\lambda t \rightarrow t + \mathbb{N} Nx$ ) ( $+\mathbb{N}\text{-comm } Ny \text{ } Nxy$ ))
        (trans
          ( $+\mathbb{N}\text{-assoc } Nxy \text{ } Ny \text{ } Nx$ )
          (cong ( $\lambda t \rightarrow Nxy + \mathbb{N} t$ ) ( $+\mathbb{N}\text{-comm } Ny \text{ } Nx$ ))))
    in
  subst ( $\lambda t \rightarrow Ny \leq t$ ) eq base
in
record
{ ltWitness =
   $\epsilon$  ,
  ( $\epsilon pos$ 
  , ( $N$ 
  , ( $\lambda n \text{ } N \leq n \rightarrow$ 
    let
      -- § Phase 5: The chain  $x'_n + \epsilon \leq y'_n$ 
       $Nxy \leq n : Nxy \leq n$ 
       $Nxy \leq n = \leq\text{-trans } Nxy \leq N \text{ } N \leq n$ 

       $Nx \leq n : Nx \leq n$ 
       $Nx \leq n = \leq\text{-trans } Nx \leq N \text{ } N \leq n$ 

       $Ny \leq n : Ny \leq n$ 
       $Ny \leq n = \leq\text{-trans } Ny \leq N \text{ } N \leq n$ 

       $xn : \mathbb{Q}$ 
       $xn = \text{seq } x \text{ } n$ 

       $x'n : \mathbb{Q}$ 
       $x'n = \text{seq } x' \text{ } n$ 

       $yn : \mathbb{Q}$ 
       $yn = \text{seq } y \text{ } n$ 

       $y'n : \mathbb{Q}$ 
       $y'n = \text{seq } y' \text{ } n$ 

       $x'n \leq xn + \alpha : x'n \leq \mathbb{Q} (xn + \mathbb{Q} \alpha)$ 
       $x'n \leq xn + \alpha = \text{boundX } n \text{ } Nx \leq n$ 

```


$$\begin{aligned}
& \alpha + \beta < \epsilon : (\alpha + \mathbb{Q} \beta) <_{\mathbb{Q}} \epsilon \\
& \alpha + \beta < \epsilon = \epsilon \text{Quarter-double} < \epsilon \in \text{pos} \\
\\
& \alpha + \beta \leq \epsilon : (\alpha + \mathbb{Q} \beta) \leq_{\mathbb{Q}} \epsilon \\
& \alpha + \beta \leq \epsilon = <_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}} \{(\alpha + \mathbb{Q} \beta)\} \{\epsilon\} \alpha + \beta < \epsilon \\
\\
& \epsilon + \alpha + \beta \leq \epsilon + \epsilon : (\epsilon + \mathbb{Q} (\alpha + \mathbb{Q} \beta)) \leq_{\mathbb{Q}} (\epsilon + \mathbb{Q} \epsilon) \\
& \epsilon + \alpha + \beta \leq \epsilon + \epsilon = \leq_{\mathbb{Q}} + \mathbb{Q} \text{-mono-left} \epsilon (\alpha + \mathbb{Q} \beta) \epsilon \alpha + \beta \leq \epsilon \\
\\
& \epsilon + \epsilon < \epsilon_0 : (\epsilon + \mathbb{Q} \epsilon) <_{\mathbb{Q}} \epsilon_0 \\
& \epsilon + \epsilon < \epsilon_0 = \epsilon \text{Quarter-double} < \epsilon_0 \epsilon_0 \text{pos} \\
\\
& \epsilon + \epsilon \leq \epsilon_0 : (\epsilon + \mathbb{Q} \epsilon) \leq_{\mathbb{Q}} \epsilon_0 \\
& \epsilon + \epsilon \leq \epsilon_0 = <_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}} \{(\epsilon + \mathbb{Q} \epsilon)\} \{\epsilon_0\} \epsilon + \epsilon < \epsilon_0 \\
\\
& \epsilon + \alpha + \beta \leq \epsilon_0 : (\epsilon + \mathbb{Q} (\alpha + \mathbb{Q} \beta)) \leq_{\mathbb{Q}} \epsilon_0 \\
& \epsilon + \alpha + \beta \leq \epsilon_0 = \leq_{\mathbb{Q}} \text{-trans} \{(\epsilon + \mathbb{Q} (\alpha + \mathbb{Q} \beta))\} \{(\epsilon + \mathbb{Q} \epsilon)\} \{\epsilon_0\} \epsilon + \alpha + \beta \leq \epsilon + \epsilon \epsilon + \epsilon \leq \epsilon_0 \\
\\
& t : \mathbb{Q} \\
& t = \alpha + \mathbb{Q} (\epsilon + \mathbb{Q} \beta) \\
\\
& t \simeq \epsilon + \alpha + \beta : t \simeq_{\mathbb{Q}} (\epsilon + \mathbb{Q} (\alpha + \mathbb{Q} \beta)) \\
& t \simeq \epsilon + \alpha + \beta = \\
& \quad \simeq_{\mathbb{Q}} \text{-trans} \\
& \quad (\simeq_{\mathbb{Q}} \text{-sym} (+\mathbb{Q} \text{-assoc} \alpha \epsilon \beta)) \\
& \quad (\simeq_{\mathbb{Q}} \text{-trans} \\
& \quad \quad (+\mathbb{Q} \text{-resp} \simeq (+\mathbb{Q} \text{-comm} \alpha \epsilon) (\simeq_{\mathbb{Q}} \text{-refl} \beta)) \\
& \quad \quad (+\mathbb{Q} \text{-assoc} \epsilon \alpha \beta)) \\
\\
& t \leq \epsilon_0 : t \leq_{\mathbb{Q}} \epsilon_0 \\
& t \leq \epsilon_0 = \leq_{\mathbb{Q}} \text{-trans} (\simeq_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}}^l t \simeq \epsilon + \alpha + \beta) \epsilon + \alpha + \beta \leq \epsilon_0 \\
\\
& xnt \leq xn \epsilon_0 : (xn + \mathbb{Q} t) \leq_{\mathbb{Q}} (xn + \mathbb{Q} \epsilon_0) \\
& xnt \leq xn \epsilon_0 = \leq_{\mathbb{Q}} + \mathbb{Q} \text{-mono-left} xn t \epsilon_0 t \leq \epsilon_0 \\
\\
& x'n + \epsilon + \beta \leq xn + t : (x'n + \mathbb{Q} (\epsilon + \mathbb{Q} \beta)) \leq_{\mathbb{Q}} (xn + \mathbb{Q} t) \\
& x'n + \epsilon + \beta \leq xn + t = \\
& \quad \text{let} \\
& \quad \text{step}_1 : (x'n + \mathbb{Q} (\epsilon + \mathbb{Q} \beta)) \leq_{\mathbb{Q}} ((xn + \mathbb{Q} \alpha) + \mathbb{Q} (\epsilon + \mathbb{Q} \beta)) \\
& \quad \text{step}_1 = \leq_{\mathbb{Q}} + \mathbb{Q} \text{-mono-right} x'n (xn + \mathbb{Q} \alpha) (\epsilon + \mathbb{Q} \beta) x'n \leq xn + \alpha \\
\\
& \text{lhsEq} : ((xn + \mathbb{Q} \alpha) + \mathbb{Q} (\epsilon + \mathbb{Q} \beta)) \simeq_{\mathbb{Q}} (xn + \mathbb{Q} t) \\
& \text{lhsEq} = +\mathbb{Q} \text{-assoc} xn \alpha (\epsilon + \mathbb{Q} \beta) \\
& \quad \text{in} \\
& \leq_{\mathbb{Q}} \text{-trans} \text{step}_1 (\simeq_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}}^l \text{lhsEq}) \\
\\
& x'n + \epsilon + \beta \leq xn + \epsilon_0 : (x'n + \mathbb{Q} (\epsilon + \mathbb{Q} \beta)) \leq_{\mathbb{Q}} (xn + \mathbb{Q} \epsilon_0) \\
& x'n + \epsilon + \beta \leq xn + \epsilon_0 = \leq_{\mathbb{Q}} \text{-trans} \{(x'n + \mathbb{Q} (\epsilon + \mathbb{Q} \beta))\} \{(xn + \mathbb{Q} t)\} \{(xn + \mathbb{Q} \epsilon_0)\} x'n + \epsilon + \beta \leq xn + t xnt \leq xn \epsilon_0 \\
\\
& xn + \epsilon_0 \leq yn : (xn + \mathbb{Q} \epsilon_0) \leq_{\mathbb{Q}} yn \\
& xn + \epsilon_0 \leq yn = \text{convXY} n Nxy \leq n \\
\\
& x'n + \epsilon + \beta \leq yn : (x'n + \mathbb{Q} (\epsilon + \mathbb{Q} \beta)) \leq_{\mathbb{Q}} yn \\
& x'n + \epsilon + \beta \leq yn = \leq_{\mathbb{Q}} \text{-trans} \{(x'n + \mathbb{Q} (\epsilon + \mathbb{Q} \beta))\} \{(xn + \mathbb{Q} \epsilon_0)\} \{yn\} x'n + \epsilon + \beta \leq xn + \epsilon_0 xn + \epsilon_0 \leq yn
\end{aligned}$$

$$\begin{aligned}
& x'n + \epsilon \leq y'n - \beta : (x'n + \mathbb{Q} \epsilon) \leq \mathbb{Q} (y'n + \mathbb{Q} (-\mathbb{Q} \beta)) \\
& x'n + \epsilon \leq y'n - \beta = \\
& \text{let} \\
& \text{step}_1 : ((x'n + \mathbb{Q} (\epsilon + \mathbb{Q} \beta)) + \mathbb{Q} (-\mathbb{Q} \beta)) \leq \mathbb{Q} (y'n + \mathbb{Q} (-\mathbb{Q} \beta)) \\
& \text{step}_1 = \leq \mathbb{Q} + \mathbb{Q} \text{-mono-right } (x'n + \mathbb{Q} (\epsilon + \mathbb{Q} \beta)) \ y'n \ (-\mathbb{Q} \beta) \ x'n + \epsilon + \beta \leq y'n \\
& \text{lhsEq}_1 : ((x'n + \mathbb{Q} (\epsilon + \mathbb{Q} \beta)) + \mathbb{Q} (-\mathbb{Q} \beta)) \simeq \mathbb{Q} (x'n + \mathbb{Q} ((\epsilon + \mathbb{Q} \beta) + \mathbb{Q} (-\mathbb{Q} \beta))) \\
& \text{lhsEq}_1 = +\mathbb{Q} \text{-assoc } x'n \ (\epsilon + \mathbb{Q} \beta) \ (-\mathbb{Q} \beta) \\
& \text{lhsEq}_2 : ((\epsilon + \mathbb{Q} \beta) + \mathbb{Q} (-\mathbb{Q} \beta)) \simeq \mathbb{Q} \epsilon \\
& \text{lhsEq}_2 = \\
& \simeq \mathbb{Q} \text{-trans} \\
& (+\mathbb{Q} \text{-assoc } \epsilon \ \beta \ (-\mathbb{Q} \beta)) \\
& (\simeq \mathbb{Q} \text{-trans} \\
& (+\mathbb{Q} \text{-resp} \simeq (\simeq \mathbb{Q} \text{-refl } \epsilon) \ (+\mathbb{Q} \text{-inv-right } \beta)) \\
& (+\mathbb{Q} \text{-zero-right } \epsilon)) \\
& \text{lhsEq} : ((x'n + \mathbb{Q} (\epsilon + \mathbb{Q} \beta)) + \mathbb{Q} (-\mathbb{Q} \beta)) \simeq \mathbb{Q} (x'n + \mathbb{Q} \epsilon) \\
& \text{lhsEq} = \simeq \mathbb{Q} \text{-trans } \text{lhsEq}_1 \ (+\mathbb{Q} \text{-resp} \simeq (\simeq \mathbb{Q} \text{-refl } x'n) \ \text{lhsEq}_2) \\
& \text{in} \\
& \leq \mathbb{Q} \text{-trans } (\simeq \mathbb{Q} \rightarrow \leq \mathbb{Q}^r \ \text{lhsEq}) \ \text{step}_1 \\
& y'n \leq y'n + \beta : y'n \leq \mathbb{Q} (y'n + \mathbb{Q} \beta) \\
& y'n \leq y'n + \beta = \text{boundY } n \ N y \leq n \\
& y'n - \beta \leq y'n : (y'n + \mathbb{Q} (-\mathbb{Q} \beta)) \leq \mathbb{Q} y'n \\
& y'n - \beta \leq y'n = \\
& \text{let} \\
& \text{step}_1 : (y'n + \mathbb{Q} (-\mathbb{Q} \beta)) \leq \mathbb{Q} ((y'n + \mathbb{Q} \beta) + \mathbb{Q} (-\mathbb{Q} \beta)) \\
& \text{step}_1 = \leq \mathbb{Q} + \mathbb{Q} \text{-mono-right } y'n \ (y'n + \mathbb{Q} \beta) \ (-\mathbb{Q} \beta) \ y'n \leq y'n + \beta \\
& \text{rhsEq}_1 : ((y'n + \mathbb{Q} \beta) + \mathbb{Q} (-\mathbb{Q} \beta)) \simeq \mathbb{Q} (y'n + \mathbb{Q} (\beta + \mathbb{Q} (-\mathbb{Q} \beta))) \\
& \text{rhsEq}_1 = +\mathbb{Q} \text{-assoc } y'n \ \beta \ (-\mathbb{Q} \beta) \\
& \text{rhsEq}_2 : (\beta + \mathbb{Q} (-\mathbb{Q} \beta)) \simeq \mathbb{Q} 0\mathbb{Q} \\
& \text{rhsEq}_2 = +\mathbb{Q} \text{-inv-right } \beta \\
& \text{rhsEq} : ((y'n + \mathbb{Q} \beta) + \mathbb{Q} (-\mathbb{Q} \beta)) \simeq \mathbb{Q} y'n \\
& \text{rhsEq} = \\
& \simeq \mathbb{Q} \text{-trans} \\
& \text{rhsEq}_1 \\
& (\simeq \mathbb{Q} \text{-trans} \\
& (+\mathbb{Q} \text{-resp} \simeq (\simeq \mathbb{Q} \text{-refl } y'n) \ \text{rhsEq}_2) \\
& (+\mathbb{Q} \text{-zero-right } y'n)) \\
& \text{in} \\
& \leq \mathbb{Q} \text{-trans } \text{step}_1 \ (\simeq \mathbb{Q} \rightarrow \leq \mathbb{Q}^l \ \text{rhsEq}) \\
& \text{in} \\
& \leq \mathbb{Q} \text{-trans } x'n + \epsilon \leq y'n - \beta \ y'n - \beta \leq y'n \\
&) \\
&) \\
&) \\
& \}
\end{aligned}$$

Reflexivity of $\leq_{\mathbb{R}}$

Reflexivity is immediate. For any positive margin ε , each term of a representative sequence is at most itself plus ε , so the asymptotic definition gives self-comparison on every tail.

Law 23.7: Non-strict order is reflexive

Reflexivity follows pointwise from the inequality $a \leq a + \varepsilon$.

```
-- § Reflexivity of  $\leq_{\mathbb{R}}$ 
 $\leq_{\mathbb{R}}\text{-refl} : (x : \mathbb{R}) \rightarrow x \leq_{\mathbb{R}} x$ 
 $\leq_{\mathbb{R}}\text{-refl} x = \text{record}$ 
{ leReal =  $\lambda \epsilon \in \text{epos} \rightarrow$ 
  zero, ( $\lambda n \_ \rightarrow$ 
     $\leq_{\mathbb{Q}}\text{-add-nonneg-right} (\text{seq } x \ n) \in (<_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}} \text{epos}))$ 
}
```

Transitivity of $\leq_{\mathbb{R}}$

Transitivity is proved by evaluating both given comparisons with a reduced tolerance and then chaining the resulting inequalities. The only bookkeeping is to ensure that the two error terms stay below the requested ε .

Law 23.8: Non-strict order is transitive

```
-- § Transitivity of  $\leq_{\mathbb{R}}$  via  $\epsilon$ -splitting
 $\leq_{\mathbb{R}}\text{-trans} : \{x \ y \ z : \mathbb{R}\} \rightarrow x \leq_{\mathbb{R}} y \rightarrow y \leq_{\mathbb{R}} z \rightarrow x \leq_{\mathbb{R}} z$ 
 $\leq_{\mathbb{R}}\text{-trans} \{x\} \{y\} \{z\} x \leq y \ y \leq z = \text{record}$ 
{ leReal =  $\lambda \epsilon \in \text{epos} \rightarrow$ 
  let
    -- § Phase 1: Budget allocation
     $\epsilon q : \mathbb{Q}$ 
     $\epsilon q = \epsilon_{\text{Quarter}} \epsilon$ 

     $\epsilon qPos : 0_{\mathbb{Q}} <_{\mathbb{Q}} \epsilon q$ 
     $\epsilon qPos = \epsilon_{\text{Quarter-pos}} \epsilon$ 

     $\epsilon qNonneg : 0_{\mathbb{Q}} \leq_{\mathbb{Q}} \epsilon q$ 
     $\epsilon qNonneg = <_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}} \{0_{\mathbb{Q}}\} \{\epsilon q\} \epsilon qPos$ 

     $\epsilon q + \epsilon q < \epsilon : (\epsilon q + \mathbb{Q} \epsilon q) <_{\mathbb{Q}} \epsilon$ 
     $\epsilon q + \epsilon q < \epsilon = \epsilon_{\text{Quarter-double}} \epsilon \in \text{epos}$ 

    NxyPack :  $\Sigma \mathbb{N} (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow (\text{seq } x \ n) \leq_{\mathbb{Q}} ((\text{seq } y \ n) + \mathbb{Q} \epsilon q))$ 
    NxyPack =  $\leq_{\mathbb{R}}\text{-leReal} x \leq y \ \epsilon q \ \epsilon qPos$ 
```

```

NyzPack :  $\Sigma \mathbb{N} (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow (\text{seq } y \ n) \leq \mathbb{Q} ((\text{seq } z \ n) + \mathbb{Q} \ \epsilon q))$ 
NyzPack =  $\_ \leq \mathbb{R} \_ . \text{leReal } y \leq z \ \epsilon q \ \epsilon qPos$ 

```

```

Nxy :  $\mathbb{N}$ 
Nxy = fst NxyPack

```

```

Nyz :  $\mathbb{N}$ 
Nyz = fst NyzPack

```

```

NxyConv :  $(n : \mathbb{N}) \rightarrow Nxy \leq n \rightarrow (\text{seq } x \ n) \leq \mathbb{Q} ((\text{seq } y \ n) + \mathbb{Q} \ \epsilon q)$ 
NxyConv = snd NxyPack

```

```

NyzConv :  $(n : \mathbb{N}) \rightarrow Nyz \leq n \rightarrow (\text{seq } y \ n) \leq \mathbb{Q} ((\text{seq } z \ n) + \mathbb{Q} \ \epsilon q)$ 
NyzConv = snd NyzPack

```

```

-- -- Phase 2: Global modulus N -----

```

```

N :  $\mathbb{N}$ 

```

```

N = Nxy +  $\mathbb{N}$  Nyz

```

```

Nxy  $\leq$  N : Nxy  $\leq$  N

```

```

Nxy  $\leq$  N =

```

```

  let

```

```

    mono : (Nxy +  $\mathbb{N}$  zero)  $\leq$  (Nxy +  $\mathbb{N}$  Nyz)

```

```

    mono =  $\leq$  +  $\mathbb{N}$ -monol {a = zero} {b = Nyz} z  $\leq$  n Nxy

```

```

  in

```

```

  subst ( $\lambda t \rightarrow t \leq$  (Nxy +  $\mathbb{N}$  Nyz)) (+  $\mathbb{N}$ -zero-right Nxy) mono

```

```

Nyz  $\leq$  N : Nyz  $\leq$  N

```

```

Nyz  $\leq$  N =

```

```

  let

```

```

    mono : (Nyz +  $\mathbb{N}$  zero)  $\leq$  (Nyz +  $\mathbb{N}$  Nxy)

```

```

    mono =  $\leq$  +  $\mathbb{N}$ -monol {a = zero} {b = Nxy} z  $\leq$  n Nyz

```

```

    base : Nyz  $\leq$  (Nyz +  $\mathbb{N}$  Nxy)

```

```

    base = subst ( $\lambda t \rightarrow t \leq$  (Nyz +  $\mathbb{N}$  Nxy)) (+  $\mathbb{N}$ -zero-right Nyz) mono

```

```

  in

```

```

  subst ( $\lambda t \rightarrow$  Nyz  $\leq$  t) (+  $\mathbb{N}$ -comm Nyz Nxy) base

```

```

in

```

```

N , ( $\lambda n \ N \leq n \rightarrow$ 

```

```

  let

```

```

    -- § Phase 3: Chain and reassociate

```

```

    Nxy  $\leq$  n : Nxy  $\leq$  n

```

```

    Nxy  $\leq$  n =  $\leq$ -trans Nxy  $\leq$  N N  $\leq$  n

```

```

    Nyz  $\leq$  n : Nyz  $\leq$  n

```

```

    Nyz  $\leq$  n =  $\leq$ -trans Nyz  $\leq$  N N  $\leq$  n

```

```

    xn :  $\mathbb{Q}$ 

```

```

    xn = seq x n

```

```

    yn :  $\mathbb{Q}$ 

```

```

    yn = seq y n

```

```

    zn :  $\mathbb{Q}$ 

```

```

    zn = seq z n

```

```

 $xn \leq_{\mathbb{R}} yn + \epsilon q : xn \leq_{\mathbb{Q}} (yn + \mathbb{Q} \epsilon q)$ 
 $xn \leq_{\mathbb{R}} yn + \epsilon q = NxyConv \ n \ Nxy \leq n$ 

 $yn \leq_{\mathbb{R}} zn + \epsilon q : yn \leq_{\mathbb{Q}} (zn + \mathbb{Q} \epsilon q)$ 
 $yn \leq_{\mathbb{R}} zn + \epsilon q = NyzConv \ n \ Nyz \leq n$ 

 $step_1 : (yn + \mathbb{Q} \epsilon q) \leq_{\mathbb{Q}} ((zn + \mathbb{Q} \epsilon q) + \mathbb{Q} \epsilon q)$ 
 $step_1 = \leq_{\mathbb{Q}} + \mathbb{Q}\text{-mono-right} \ yn \ (zn + \mathbb{Q} \epsilon q) \ \epsilon q \ yn \leq_{\mathbb{R}} zn + \epsilon q$ 

 $step_2 : xn \leq_{\mathbb{Q}} ((zn + \mathbb{Q} \epsilon q) + \mathbb{Q} \epsilon q)$ 
 $step_2 = \leq_{\mathbb{Q}}\text{-trans} \ \{xn\} \ \{(yn + \mathbb{Q} \epsilon q)\} \ \{((zn + \mathbb{Q} \epsilon q) + \mathbb{Q} \epsilon q)\} \ xn \leq_{\mathbb{R}} yn + \epsilon q \ step_1$ 

 $step_3 : ((zn + \mathbb{Q} \epsilon q) + \mathbb{Q} \epsilon q) \leq_{\mathbb{Q}} (zn + \mathbb{Q} (\epsilon q + \mathbb{Q} \epsilon q))$ 
 $step_3 = \simeq_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}}^l \ \{((zn + \mathbb{Q} \epsilon q) + \mathbb{Q} \epsilon q)\} \ \{(zn + \mathbb{Q} (\epsilon q + \mathbb{Q} \epsilon q))\} \ (+\mathbb{Q}\text{-assoc} \ zn \ \epsilon q \ \epsilon q)$ 

 $step_4 : (zn + \mathbb{Q} (\epsilon q + \mathbb{Q} \epsilon q)) \leq_{\mathbb{Q}} (zn + \mathbb{Q} \epsilon)$ 
 $step_4 = \leq_{\mathbb{Q}} + \mathbb{Q}\text{-mono-left} \ zn \ (\epsilon q + \mathbb{Q} \epsilon q) \ \epsilon \ (-\mathbb{Q} \rightarrow \leq_{\mathbb{Q}} \ \epsilon q + \epsilon q < \epsilon)$ 

 $done : xn \leq_{\mathbb{Q}} (zn + \mathbb{Q} \epsilon)$ 
 $done = \leq_{\mathbb{Q}}\text{-trans} \ \{xn\} \ \{((zn + \mathbb{Q} \epsilon q) + \mathbb{Q} \epsilon q)\} \ \{(zn + \mathbb{Q} \epsilon)\} \ step_2 \ (\leq_{\mathbb{Q}}\text{-trans} \ step_3 \ step_4)$ 
in
done)
}

```

Antisymmetry of $\leq_{\mathbb{R}}$

If each of two real numbers is eventually bounded above by the other, then their representatives differ by at most an arbitrary positive tolerance on a tail. This is exactly the estimate required for equivalence.

Law 23.9: Mutual non-strict order forces equivalence

```

-- § Antisymmetry: mutual  $\leq_{\mathbb{R}}$  yields  $\simeq_{\mathbb{R}}$ 
 $\leq_{\mathbb{R}}\text{-antisym} : \{x \ y : \mathbb{R}\} \rightarrow x \leq_{\mathbb{R}} y \rightarrow y \leq_{\mathbb{R}} x \rightarrow x \simeq_{\mathbb{R}} y$ 
 $\leq_{\mathbb{R}}\text{-antisym} \ \{x\} \ \{y\} \ x \leq y \ y \leq x = record$ 
{ conv0 =  $\lambda \epsilon \in \epsilon pos \rightarrow$ 
  let
     $\epsilon q : \mathbb{Q}$ 
     $\epsilon q = \epsilon Quarter \ \epsilon$ 

     $\epsilon qPos : 0 \mathbb{Q} < \mathbb{Q} \ \epsilon q$ 
     $\epsilon qPos = \epsilon Quarter\text{-pos} \ \epsilon$ 

     $NxyPack : \Sigma \ \mathbb{N} \ (\lambda \ N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow (\text{seq} \ x \ n) \leq_{\mathbb{Q}} ((\text{seq} \ y \ n) + \mathbb{Q} \epsilon q))$ 
     $NxyPack = \_ \leq_{\mathbb{R}} \_ .leReal \ x \leq y \ \epsilon q \ \epsilon qPos$ 

     $NyxPack : \Sigma \ \mathbb{N} \ (\lambda \ N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow (\text{seq} \ y \ n) \leq_{\mathbb{Q}} ((\text{seq} \ x \ n) + \mathbb{Q} \epsilon q))$ 
     $NyxPack = \_ \leq_{\mathbb{R}} \_ .leReal \ y \leq x \ \epsilon q \ \epsilon qPos$ 

```

```

Nxy :  $\mathbb{N}$ 
Nxy = fst NxyPack

Nyx :  $\mathbb{N}$ 
Nyx = fst NyxPack

NxyConv : (n :  $\mathbb{N}$ )  $\rightarrow$  Nxy  $\leq$  n  $\rightarrow$  (seq x n)  $\leq \mathbb{Q}$  ((seq y n) +  $\mathbb{Q}$   $\epsilon q$ )
NxyConv = snd NxyPack

NyxConv : (n :  $\mathbb{N}$ )  $\rightarrow$  Nyx  $\leq$  n  $\rightarrow$  (seq y n)  $\leq \mathbb{Q}$  ((seq x n) +  $\mathbb{Q}$   $\epsilon q$ )
NyxConv = snd NyxPack

N :  $\mathbb{N}$ 
N = Nxy +  $\mathbb{N}$  Nyx

Nxy $\leq$ N : Nxy  $\leq$  N
Nxy $\leq$ N =
  let
    mono : (Nxy +  $\mathbb{N}$  zero)  $\leq$  (Nxy +  $\mathbb{N}$  Nyx)
    mono =  $\leq$ -+ $\mathbb{N}$ -monoL {a = zero} {b = Nyx} z  $\leq$  n Nxy
  in
    subst ( $\lambda$  t  $\rightarrow$  t  $\leq$  (Nxy +  $\mathbb{N}$  Nyx)) (+ $\mathbb{N}$ -zero-right Nxy) mono

Nyx $\leq$ N : Nyx  $\leq$  N
Nyx $\leq$ N =
  let
    mono : (Nyx +  $\mathbb{N}$  zero)  $\leq$  (Nyx +  $\mathbb{N}$  Nxy)
    mono =  $\leq$ -+ $\mathbb{N}$ -monoL {a = zero} {b = Nxy} z  $\leq$  n Nyx
  in
    base : Nyx  $\leq$  (Nyx +  $\mathbb{N}$  Nxy)
    base = subst ( $\lambda$  t  $\rightarrow$  t  $\leq$  (Nyx +  $\mathbb{N}$  Nxy)) (+ $\mathbb{N}$ -zero-right Nyx) mono
  in
    subst ( $\lambda$  t  $\rightarrow$  Nyx  $\leq$  t) (+ $\mathbb{N}$ -comm Nyx Nxy) base

in
N , ( $\lambda$  n N $\leq$ n  $\rightarrow$ 
  let
    Nxy $\leq$ n : Nxy  $\leq$  n
    Nxy $\leq$ n =  $\leq$ -trans Nxy $\leq$ N N $\leq$ n

    Nyx $\leq$ n : Nyx  $\leq$  n
    Nyx $\leq$ n =  $\leq$ -trans Nyx $\leq$ N N $\leq$ n

    xn :  $\mathbb{Q}$ 
    xn = seq x n

    yn :  $\mathbb{Q}$ 
    yn = seq y n

    xn $\leq$ yn+ $\epsilon q$  : xn  $\leq \mathbb{Q}$  (yn +  $\mathbb{Q}$   $\epsilon q$ )
    xn $\leq$ yn+ $\epsilon q$  = NxyConv n Nxy $\leq$ n

    yn $\leq$ xn+ $\epsilon q$  : yn  $\leq \mathbb{Q}$  (xn +  $\mathbb{Q}$   $\epsilon q$ )
    yn $\leq$ xn+ $\epsilon q$  = NyxConv n Nyx $\leq$ n

    d $\leq \epsilon q$  : dist $\mathbb{Q}$  xn yn  $\leq \mathbb{Q}$   $\epsilon q$ 

```

```

d ≤ ε q = distQ-bounded-by-ε xn yn ε q xn ≤ yn + ε q yn ≤ xn + ε q

ε q < ε : ε q <Q ε
ε q < ε = εQuarter < ε ε ε pos
in
≤ <Q → <Q {distQ xn yn} {ε q} {ε} d ≤ ε q ε q < ε
}

```

Congruence of $\leq_{\mathbb{R}}$ under $\simeq_{\mathbb{R}}$

The non-strict relation is easier to transport than the strict one. Equivalence already provides comparisons in both directions, so one composes those with the given order relation. Hence $\leq_{\mathbb{R}}$ depends only on the represented real numbers.

Law 23.10: Non-strict order is invariant under equivalence

The congruence law is obtained by composing the order bounds induced by equivalence on the left and right.

```

-- § Non-strict order respects ≈R
≤R-resp-≈R : {x x' y y' : ℝ} → x ≈R x' → y ≈R y' → x ≤R y → x' ≤R y'
≤R-resp-≈R {x} {x'} {y} {y'} x ≈ x' y ≈ y' x ≤ y =
let
  x' ≤ x : x' ≤R x
  x' ≤ x = ≈R → ≤R (≈R-sym x ≈ x')

  y ≤ y' : y ≤R y'
  y ≤ y' = ≈R → ≤R y ≈ y'
in
≤R-trans (≤R-trans x' ≤ x x ≤ y) y ≤ y'

```

Right Additive Monotonicity of $\leq_{\mathbb{R}}$

Order must be compatible with addition. If $x \leq y$, then adding the same real number to both sides should preserve the comparison. On representatives this is pointwise monotonicity of rational addition, together with a simple reassociation of the error term. The left-sided version is identical.

Law 23.11: Right addition preserves non-strict order

Right monotonicity follows from the corresponding rational monotonicity, with associativity used to place the ε -term in the required form.

```

-- § Right monotonicity of +R under ≤R
≤R-+R-mono-right : {x y z : ℝ} → x ≤R y → (x +R z) ≤R (y +R z)

```

```

≤ℝ-+ℝ-mono-right {x} {y} {z} x ≤ y = record
{ leReal = λ ∈ epos →
  let
    pack : Σ ℕ (λ N → (n : ℕ) → N ≤ n → (seq x n) ≤ℚ ((seq y n) +ℚ ∈))
    pack = _≤ℝ_.leReal x ≤ y ∈ epos

    N : ℕ
    N = fst pack

    conv : (n : ℕ) → N ≤ n → (seq x n) ≤ℚ ((seq y n) +ℚ ∈)
    conv = snd pack
  in
    N , (λ n N ≤ n →
      let
        xn : ℚ
        xn = seq x n

        yn : ℚ
        yn = seq y n

        zn : ℚ
        zn = seq z n

        step1 : (xn +ℚ zn) ≤ℚ (((yn +ℚ ∈) +ℚ zn))
        step1 = ≤ℚ-+ℚ-mono-right xn (yn +ℚ ∈) zn (conv n N ≤ n)

        rhsEq : (((yn +ℚ ∈) +ℚ zn) ≈ℚ ((yn +ℚ zn) +ℚ ∈)
        rhsEq =
          ≈ℚ-trans
            (+ℚ-assoc yn ∈ zn)
            (≈ℚ-trans
              (+ℚ-resp-≈ (≈ℚ-refl yn) (+ℚ-comm ∈ zn))
              (≈ℚ-sym (+ℚ-assoc yn zn ∈)))

        step2 : (((yn +ℚ ∈) +ℚ zn)) ≤ℚ ((yn +ℚ zn) +ℚ ∈)
        step2 = ≈ℚ → ≤ℚl {(((yn +ℚ ∈) +ℚ zn))} {((yn +ℚ zn) +ℚ ∈)} rhsEq
      in
        ≤ℚ-trans step1 step2)
    }

```

Left Additive Monotonicity of $\leq_{\mathbb{R}}$

The left-sided version is identical. One again applies termwise monotonicity of rational addition and carries the same tail estimate through unchanged. Together, the two monotonicity laws show that translation preserves asymptotic order on the reals.

Law 23.12: Left addition preserves non-strict order

```

-- § Left monotonicity of +ℝ under ≤ℝ
≤ℝ-+ℝ-mono-left : {x y z : ℝ} → x ≤ℝ y → (z +ℝ x) ≤ℝ (z +ℝ y)

```



```

 $\leq_{\mathbb{R}}$ - $\mathbb{R}$ -mono-left {x} {y} {z}  $x \leq y = \text{record}$ 
{ leReal =  $\lambda \epsilon \in \text{pos} \rightarrow$ 
  let
    pack :  $\Sigma \mathbb{N} (\lambda N \rightarrow (n : \mathbb{N}) \rightarrow N \leq n \rightarrow (\text{seq } x \ n) \leq_{\mathbb{Q}} ((\text{seq } y \ n) +_{\mathbb{Q}} \epsilon))$ 
    pack =  $\leq_{\mathbb{R}} \text{leReal } x \leq y \epsilon \in \text{pos}$ 

    N :  $\mathbb{N}$ 
    N = fst pack

    conv :  $(n : \mathbb{N}) \rightarrow N \leq n \rightarrow (\text{seq } x \ n) \leq_{\mathbb{Q}} ((\text{seq } y \ n) +_{\mathbb{Q}} \epsilon)$ 
    conv = snd pack
  in
    N, ( $\lambda n \ N \leq n \rightarrow$ 
      let
        xn :  $\mathbb{Q}$ 
        xn = seq x n

        yn :  $\mathbb{Q}$ 
        yn = seq y n

        zn :  $\mathbb{Q}$ 
        zn = seq z n

        step1 :  $(zn +_{\mathbb{Q}} xn) \leq_{\mathbb{Q}} (zn +_{\mathbb{Q}} (yn +_{\mathbb{Q}} \epsilon))$ 
        step1 =  $\leq_{\mathbb{Q}} +_{\mathbb{Q}} \text{mono-left } zn \ xn \ (yn +_{\mathbb{Q}} \epsilon) \ (conv \ n \ N \leq n)$ 

        rhsEq :  $(zn +_{\mathbb{Q}} (yn +_{\mathbb{Q}} \epsilon)) \simeq_{\mathbb{Q}} ((zn +_{\mathbb{Q}} yn) +_{\mathbb{Q}} \epsilon)$ 
        rhsEq = sym (+ $\mathbb{Q}$ -assoc zn yn  $\epsilon$ )

        step2 :  $(zn +_{\mathbb{Q}} (yn +_{\mathbb{Q}} \epsilon)) \leq_{\mathbb{Q}} ((zn +_{\mathbb{Q}} yn) +_{\mathbb{Q}} \epsilon)$ 
        step2 =  $\simeq_{\mathbb{Q}} \rightarrow \leq_{\mathbb{Q}}^I \{(zn +_{\mathbb{Q}} (yn +_{\mathbb{Q}} \epsilon))\} \{(zn +_{\mathbb{Q}} yn) +_{\mathbb{Q}} \epsilon\} \text{ rhsEq}$ 
      in
         $\leq_{\mathbb{Q}}$ -trans step1 step2)
    }

```

The required numerical machinery is now in place. Natural numbers, integers, rationals, reals, order, distance, scaling, and the relevant spectral operators have all been constructed within the same formal framework. The next part uses this apparatus to evaluate the invariants of K_4 explicitly.

What remains is therefore an organizational step. The invariants already derived are assembled, their numerical values are computed, and the formal closure of the chain is verified.

Chapter 54

Cauchy Completeness of $\mathbb{R}$

The construction of $\mathbb{R}$ given so far supplies the completion of $\mathbb{Q}$ as a setoid of Cauchy sequences. The next theorem must also be made formal: every Cauchy sequence of real numbers itself converges to a real number. The standard diagonal argument is carried out below, using only the rational ε -machinery and the embedding $\mathbb{Q} \hookrightarrow \mathbb{R}$ already constructed.

Auxiliary natural number maximum

The diagonal argument repeatedly takes the maximum of finitely many natural numbers. A small explicit definition with the two inequality witnesses is sufficient.

```
-- § Maximum on  $\mathbb{N}$ .
maxN :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
maxN zero n      = n
maxN (suc m) zero = suc m
maxN (suc m) (suc n) = suc (maxN m n)

-- § Left bound.
≤-maxNl : ( $m\ n : \mathbb{N}$ ) →  $m \leq \text{maxN } m\ n$ 
≤-maxNl zero n      = z≤n
≤-maxNl (suc m) zero = ≤-refl (suc m)
≤-maxNl (suc m) (suc n) = s≤s (≤-maxNl m n)

-- § Right bound.
≤-maxNr : ( $m\ n : \mathbb{N}$ ) →  $n \leq \text{maxN } m\ n$ 
≤-maxNr zero n      = ≤-refl n
≤-maxNr (suc m) zero = z≤n
≤-maxNr (suc m) (suc n) = s≤s (≤-maxNr m n)
```

The rational schedule $1/(n+1)$

The diagonal construction needs a vanishing sequence of positive rational tolerances indexed by $\mathbb{N}$. The rationals $1/(n+1)$ are the natural choice. They are defined by directly using the positive denominator $\text{mk}\mathbb{N}^+ n$ and inherit positivity from the positive numerator $\text{one}\mathbb{Z}$.

```
-- § Schedule of rational tolerances 1/(n+1).
```

```
halfTo :  $\mathbb{N} \rightarrow \mathbb{Q}$ 
```

```
halfTo n = one $\mathbb{Z}$  / mk $\mathbb{N}^+$  n
```

```
-- § Positivity of the schedule.
```

```
halfTo-pos : (n :  $\mathbb{N}$ )  $\rightarrow$  0 $\mathbb{Q}$  < $\mathbb{Q}$  halfTo n
```

```
halfTo-pos n = 0< $\mathbb{Z}$ num $\rightarrow$ 0< $\mathbb{Q}$  (halfTo n) (0< $\mathbb{Z}$ -pos zero)
```

The order on the schedule is decreasing in n : a larger index yields a smaller tolerance. The proof reduces to the integer inequality $+ \text{suc } m \leq + \text{suc } n$ obtained from $m \leq n$.

```
-- § Monotone decrease:  $m \leq n \rightarrow \text{halfTo } n \leq \text{halfTo } m$ .
```

```
halfTo-mono : (m n :  $\mathbb{N}$ )  $\rightarrow$   $m \leq n \rightarrow \text{halfTo } n \leq \mathbb{Q} \text{ halfTo } m$ 
```

```
halfTo-mono m n m $\leq$ n =
```

```
let
```

```
-- Goal: one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  (mk $\mathbb{N}^+$  m)  $\leq \mathbb{Z}$  one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  (mk $\mathbb{N}^+$  n).
```

```
base : (suc m)  $\leq$  (suc n)
```

```
base = s $\leq$ s m $\leq$ n
```

```
int-le : (+suc m)  $\leq \mathbb{Z}$  (+suc n)
```

```
int-le = base
```

```
lhsEq : (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  (mk $\mathbb{N}^+$  m))  $\equiv$  (+suc m)
```

```
lhsEq = * $\mathbb{Z}$ -one-left (+suc m)
```

```
rhsEq : (one $\mathbb{Z}$  * $\mathbb{Z}$  +to $\mathbb{Z}$  (mk $\mathbb{N}^+$  n))  $\equiv$  (+suc n)
```

```
rhsEq = * $\mathbb{Z}$ -one-left (+suc n)
```

```
step : (+suc m)  $\leq \mathbb{Z}$  (+suc n)
```

```
step = int-le
```

```
in
```

```
 $\leq \mathbb{Z}$ -resp= $\equiv^l$  (sym lhsEq) ( $\leq \mathbb{Z}$ -resp= $\equiv^r$  (sym rhsEq) step)
```

The Archimedean property is the corresponding existence statement: for every positive rational ε , some index N makes $1/(N+1)$ strictly smaller than ε . The witness is read directly off the denominator of ε : choosing $N = +\text{to}\mathbb{N} b$ where b is the denominator of ε suffices, because the resulting product $a \cdot (b+1)$ strictly exceeds b whenever a is positive.

```
-- § Mixed strict/weak transitivity on  $\mathbb{Z}$ .
```

```
< $\leq \mathbb{Z}$  $\rightarrow$ < $\mathbb{Z}$ -loc : {x y z :  $\mathbb{Z}$ }  $\rightarrow$  x < $\mathbb{Z}$  y  $\rightarrow$  y  $\leq \mathbb{Z}$  z  $\rightarrow$  x < $\mathbb{Z}$  z
```

```
< $\leq \mathbb{Z}$  $\rightarrow$ < $\mathbb{Z}$ -loc {x} {y} {z} (x $\leq$ y, y $\not\leq$ x) y $\leq$ z =
```

```

≤ℤ-trans {x} {y} {z} x ≤ y y ≤ z ,
(λ z ≤ x → y < x (≤ℤ-trans {y} {z} {x} y ≤ z z ≤ x))

-- § Archimedean property of the schedule.
halfTo-arch : (ε : ℚ) → 0ℚ <ℚ ε → ∑ ℕ (λ N → halfTo N <ℚ ε)
halfTo-arch (a / mkℕ+ k) εpos =
let
  nWit : ∑ ℕ (λ m → a ≡ +suc m)
  nWit = positive-num-witnessℚ (a / mkℕ+ k) εpos

  m : ℕ
  m = fst nWit

  aEq : a ≡ +suc m
  aEq = snd nWit

-- The chosen schedule index.
N : ℕ
N = suc k

-- +toℤ (mkℕ+ N) = +suc (suc k).
-- The strict inequality on integers we ultimately need:
-- oneℤ *ℤ +suc k <ℤ a *ℤ +suc (suc k).

one ≤ a : oneℤ ≤ℤ a
one ≤ a = ≤ℤ-resp≡r (sym aEq) (s ≤ s z ≤ n)

-- Step 1: oneℤ *ℤ +suc (suc k) ≤ℤ a *ℤ +suc (suc k).
step1 : (oneℤ *ℤ +toℤ (mkℕ+ (suc k))) ≤ℤ (a *ℤ +toℤ (mkℕ+ (suc k)))
step1 = ≤ℤ-mul-pos-right oneℤ a (mkℕ+ (suc k)) one ≤ a

-- Reduce the LHS of step1 to +suc (suc k).
lhs1Eq : (oneℤ *ℤ +toℤ (mkℕ+ (suc k))) ≡ (+suc (suc k))
lhs1Eq = *ℤ-one-left (+suc (suc k))

-- Strict step: +suc k <ℤ +suc (suc k).
sucStep : (+suc k) <ℤ (+suc (suc k))
sucStep = (s ≤ s (≤-step k)) , λ p → suc ≤-impossible (suc k) p

-- Reduce LHS of the goal to +suc k.
goalLhsEq : (oneℤ *ℤ +toℤ (mkℕ+ k)) ≡ (+suc k)
goalLhsEq = *ℤ-one-left (+suc k)

-- Combine: +suc k <ℤ +suc (suc k) ≤ℤ a *ℤ +suc (suc k).
chained : (+suc k) <ℤ (a *ℤ +toℤ (mkℕ+ (suc k)))
chained = <≤ℤ→<ℤ-loc sucStep (≤ℤ-resp≡l lhs1Eq step1)

-- Rewrite via aEq on the RHS of the goal direction is not needed,
-- since a appears on the right already.

-- Rewrite LHS of the final goal.
goal-int : (oneℤ *ℤ +toℤ (mkℕ+ k)) <ℤ (a *ℤ +toℤ (mkℕ+ (suc k)))
goal-int =
  <ℤ-resp≡l (sym goalLhsEq) chained
in
N , goal-int

```

Cauchy sequences of reals and the diagonal

A sequence of real numbers is Cauchy when, viewed through any rational tolerance ε , two sufficiently late members eventually agree to within ε on every coordinate. The condition cannot demand uniform agreement, since each real has its own internal modulus; it must require eventual rational closeness on a tail.

The candidate limit is fixed by the same numerical material that defines the sequence: from each r_n , choose the index that already stabilises r_n to within $1/(n+1)$, and read off that rational. The resulting diagonal lives in $\mathbb{Q}$ by construction; whether it is Cauchy in $\mathbb{Q}$ and whether the original sequence converges to it, are the two propositions to be discharged.

A real number is, by definition, a Cauchy sequence of rationals. A sequence of reals is therefore a two-parameter array of rationals, and “convergence” must be phrased without invoking a metric on $\mathbb{R}$ that has not yet been constructed. The only tool available is the rational metric on each column. The Cauchy condition demands eventual ε -closeness coordinatewise; the candidate limit is the diagonal sequence assembled from each row’s own stabilisation index. No new metric structure on $\mathbb{R}$ is needed to state completeness, and the limit is computed by the same finite recipe that underlies the Cauchy data.

```
-- § Cauchy condition for a sequence of reals (eventual  $\epsilon$ -closeness on a tail).
record IsCauchyℝ (r : ℕ → ℝ) : Set where
  field
    cauchyℝ : (ε : ℚ) → 0ℚ < ℚ ε →
      Σ ℕ (λ N → (m n : ℕ) → N ≤ m → N ≤ n →
        Σ ℕ (λ K → (k : ℕ) → K ≤ k →
          distℚ (seq (r m) k) (seq (r n) k) < ℚ ε))

-- § Convergence of a sequence of reals to a limit, again via tail closeness.
record _converges-to_ (r : ℕ → ℝ) (x : ℝ) : Set where
  field
    convℝ : (ε : ℚ) → 0ℚ < ℚ ε →
      Σ ℕ (λ N → (n : ℕ) → N ≤ n →
        Σ ℕ (λ K → (k : ℕ) → K ≤ k →
          distℚ (seq (r n) k) (seq x k) < ℚ ε))

-- § Stable index of a real x at tolerance halfTo n.
diag-cauchyIdx : (x : ℝ) → ℕ → ℕ
diag-cauchyIdx x n =
  fst (IsCauchy.cauchy (isCauchy x) (halfTo n) (halfTo-pos n))

-- § Diagonal sequence: nth value is x = (r n) sampled at its own stable index.
diagSeq : (ℕ → ℝ) → ℕ → ℚ
diagSeq r n = seq (r n) (diag-cauchyIdx (r n) n)

-- § Beyond the stable index, the sequence stays within halfTo n of the diagonal.
diagSeq-stable :
  (r : ℕ → ℝ) → (n k : ℕ) → diag-cauchyIdx (r n) n ≤ k →
```

```

dist $\mathbb{Q}$  (diagSeq r n) (seq (r n) k) < $\mathbb{Q}$  halfTo n
diagSeq-stable r n k  $K \leq k =$ 
let
  pack = IsCauchy.cauchy (isCauchy (r n)) (halfTo n) (halfTo-pos n)
  K = fst pack
  bnd = snd pack
in
  bnd K k (<-refl K)  $K \leq k$ 

```

The diagonal is itself a rational Cauchy sequence. The standard $\varepsilon/4$ accounting carries over: the schedule $\eta = \epsilon_{\text{Quarter}} \delta$ controls the two “side” distances coming from the stabilising indices, while the middle distance is bounded by $\delta = \epsilon_{\text{Quarter}} \varepsilon$ through the Cauchy property of the source sequence of reals.

The diagonal must converge in $\mathbb{Q}$; otherwise it cannot serve as a representative of a real limit. Two triangle inequalities decompose the distance between two diagonal terms into one stabilising error per row plus one middle error; a single $\varepsilon/4$ budget closes all three contributions. Every Cauchy sequence of reals therefore produces a canonical rational Cauchy sequence, hence a candidate real limit; nothing further than the Cauchy data is required to construct that limit.

-- § Diagonal of a Cauchy sequence of reals is itself Cauchy in $\mathbb{Q}$.

```

diagSeq-cauchy :
  (r :  $\mathbb{N} \rightarrow \mathbb{R}$ )  $\rightarrow$  IsCauchy $\mathbb{R}$  r  $\rightarrow$  IsCauchy (diagSeq r)
diagSeq-cauchy r icR = record
  { cauchy =  $\lambda \epsilon \in \text{epos} \rightarrow$ 
    let
       $\delta : \mathbb{Q}$ 
       $\delta = \epsilon_{\text{Quarter}} \epsilon$ 

       $\delta_{\text{pos}} : 0\mathbb{Q} <\mathbb{Q} \delta$ 
       $\delta_{\text{pos}} = \epsilon_{\text{Quarter-pos}} \epsilon$ 

       $\eta : \mathbb{Q}$ 
       $\eta = \epsilon_{\text{Quarter}} \delta$ 

       $\eta_{\text{pos}} : 0\mathbb{Q} <\mathbb{Q} \eta$ 
       $\eta_{\text{pos}} = \epsilon_{\text{Quarter-pos}} \delta$ 

       $\eta_{\text{Nonneg}} : 0\mathbb{Q} \leq \mathbb{Q} \eta$ 
       $\eta_{\text{Nonneg}} = <\mathbb{Q} \rightarrow \leq \mathbb{Q} \{0\mathbb{Q}\} \{\eta\} \eta_{\text{pos}}$ 

       $\delta_{\text{Nonneg}} : 0\mathbb{Q} \leq \mathbb{Q} \delta$ 
       $\delta_{\text{Nonneg}} = <\mathbb{Q} \rightarrow \leq \mathbb{Q} \{0\mathbb{Q}\} \{\delta\} \delta_{\text{pos}}$ 

      -- Bounds at the  $\eta$  level:  $\eta + \eta \leq \delta$ .
       $\eta + \eta \simeq \epsilon_{\text{Half}} \delta : (\eta + \mathbb{Q} \eta) \simeq \mathbb{Q} \epsilon_{\text{Half}} \delta$ 
       $\eta + \eta \simeq \epsilon_{\text{Half}} \delta = \epsilon_{\text{Quarter}} + \epsilon_{\text{Quarter}} \simeq \epsilon_{\text{Half}} \delta$ 

       $\eta + \eta \leq \epsilon_{\text{Half}} \delta : (\eta + \mathbb{Q} \eta) \leq \mathbb{Q} \epsilon_{\text{Half}} \delta$ 
       $\eta + \eta \leq \epsilon_{\text{Half}} \delta = \simeq \mathbb{Q} \rightarrow \leq \mathbb{Q}^l \{\eta + \mathbb{Q} \eta\} \{\epsilon_{\text{Half}} \delta\} \eta + \eta \simeq \epsilon_{\text{Half}} \delta$ 

```

```

 $\epsilon\text{Half}\delta < \delta : \epsilon\text{Half } \delta < \mathbb{Q} \delta$ 
 $\epsilon\text{Half}\delta < \delta = \epsilon\text{Half} < \epsilon \delta \text{ pos}$ 

 $\epsilon\text{Half}\delta \leq \delta : \epsilon\text{Half } \delta \leq \mathbb{Q} \delta$ 
 $\epsilon\text{Half}\delta \leq \delta = < \mathbb{Q} \rightarrow \leq \mathbb{Q} \{ \epsilon\text{Half } \delta \} \{ \delta \} \epsilon\text{Half}\delta < \delta$ 

 $\eta + \eta \leq \delta : (\eta + \mathbb{Q} \eta) \leq \mathbb{Q} \delta$ 
 $\eta + \eta \leq \delta = \leq \mathbb{Q}\text{-trans } \{ \eta + \mathbb{Q} \eta \} \{ \epsilon\text{Half } \delta \} \{ \delta \} \eta + \eta \leq \epsilon\text{Half}\delta \epsilon\text{Half}\delta \leq \delta$ 

 $\delta + \delta < \epsilon : (\delta + \mathbb{Q} \delta) < \mathbb{Q} \epsilon$ 
 $\delta + \delta < \epsilon = \epsilon\text{Quarter-double} < \epsilon \epsilon \text{ pos}$ 

-- IsCauchy $\mathbb{R}$  pack at  $\eta$  (mid bound).
packR = IsCauchy $\mathbb{R}$ .cauchy $\mathbb{R}$  icR  $\eta$   $\eta\text{pos}$ 
N2 = fst packR
boundR = snd packR

-- halfTo $\rightarrow$ arch at  $\eta$ : halfTo N1 <  $\mathbb{Q}$   $\eta$ .
packH = halfTo $\rightarrow$ arch  $\eta$   $\eta\text{pos}$ 
N1 = fst packH
N1 <  $\eta$  : halfTo N1 <  $\mathbb{Q}$   $\eta$ 
N1 <  $\eta$  = snd packH

N1  $\leq$   $\eta$  : halfTo N1  $\leq$   $\mathbb{Q}$   $\eta$ 
N1  $\leq$   $\eta$  = <  $\mathbb{Q} \rightarrow \leq \mathbb{Q} \{ \text{halfTo } N_1 \} \{ \eta \} N_1 < \eta$ 

N :  $\mathbb{N}$ 
N = max $\mathbb{N}$  N1 N2

in
N , ( $\lambda m n N \leq m N \leq n \rightarrow$ 
let
N1  $\leq m$  : N1  $\leq m$ 
N1  $\leq m$  =  $\leq\text{-trans } (\leq\text{-max}^{\mathbb{N}^l} N_1 N_2) N \leq m$ 

N1  $\leq n$  : N1  $\leq n$ 
N1  $\leq n$  =  $\leq\text{-trans } (\leq\text{-max}^{\mathbb{N}^l} N_1 N_2) N \leq n$ 

N2  $\leq m$  : N2  $\leq m$ 
N2  $\leq m$  =  $\leq\text{-trans } (\leq\text{-max}^{\mathbb{N}^r} N_1 N_2) N \leq m$ 

N2  $\leq n$  : N2  $\leq n$ 
N2  $\leq n$  =  $\leq\text{-trans } (\leq\text{-max}^{\mathbb{N}^r} N_1 N_2) N \leq n$ 

-- halfTo bounds.
halfm  $\leq$   $\eta$  : halfTo m  $\leq$   $\mathbb{Q}$   $\eta$ 
halfm  $\leq$   $\eta$  =  $\leq \mathbb{Q}\text{-trans } \{ \text{halfTo } m \} \{ \text{halfTo } N_1 \} \{ \eta \}$ 
(halfTo-mono N1 m N1  $\leq$  m) N1  $\leq$   $\eta$ 

halfn  $\leq$   $\eta$  : halfTo n  $\leq$   $\mathbb{Q}$   $\eta$ 
halfn  $\leq$   $\eta$  =  $\leq \mathbb{Q}\text{-trans } \{ \text{halfTo } n \} \{ \text{halfTo } N_1 \} \{ \eta \}$ 
(halfTo-mono N1 n N1  $\leq$  n) N1  $\leq$   $\eta$ 

-- Cauchy $\mathbb{R}$  middle bound: pick a common k below.
packMid = boundR m n N2  $\leq$  m N2  $\leq$  n
K3 = fst packMid

```


$mid : (k : \mathbb{N}) \rightarrow K_3 \leq k \rightarrow \text{dist}\mathbb{Q} (\text{seq } (r \ m) \ k) (\text{seq } (r \ n) \ k) <\mathbb{Q} \eta$
 $mid = \text{snd } \text{packMid}$

$di_m = \text{diag-cauchyIdx } (r \ m) \ m$
 $di_n = \text{diag-cauchyIdx } (r \ n) \ n$

$kInner : \mathbb{N}$
 $kInner = \max\mathbb{N} \ di_m \ di_n$

$k : \mathbb{N}$
 $k = \max\mathbb{N} \ K_3 \ kInner$

$K_3 \leq k : K_3 \leq k$
 $K_3 \leq k = \leq\text{-max}\mathbb{N}^l \ K_3 \ kInner$

$kI \leq k : kInner \leq k$
 $kI \leq k = \leq\text{-max}\mathbb{N}^r \ K_3 \ kInner$

$di_m \leq k : di_m \leq k$
 $di_m \leq k = \leq\text{-trans } (\leq\text{-max}\mathbb{N}^l \ di_m \ di_n) \ kI \leq k$

$di_n \leq k : di_n \leq k$
 $di_n \leq k = \leq\text{-trans } (\leq\text{-max}\mathbb{N}^r \ di_m \ di_n) \ kI \leq k$

-- Three pieces.

$S1 : \mathbb{Q}$
 $S1 = \text{dist}\mathbb{Q} (\text{diagSeq } r \ m) (\text{seq } (r \ m) \ k)$

$S3 : \mathbb{Q}$
 $S3 = \text{dist}\mathbb{Q} (\text{seq } (r \ m) \ k) (\text{seq } (r \ n) \ k)$

$S4 : \mathbb{Q}$
 $S4 = \text{dist}\mathbb{Q} (\text{seq } (r \ n) \ k) (\text{diagSeq } r \ n)$

$S1 < : S1 <\mathbb{Q} \text{ halfTo } m$
 $S1 < = \text{diagSeq-stable } r \ m \ k \ di_m \leq k$

$S1 \leq \eta : S1 \leq \mathbb{Q} \eta$
 $S1 \leq \eta = \leq\mathbb{Q}\text{-trans } \{S1\} \{\text{halfTo } m\} \{\eta\}$
 $(<\mathbb{Q} \rightarrow \leq \mathbb{Q} \{S1\} \{\text{halfTo } m\} S1 <) \text{ halfTo } m \leq \eta$

$S3 < \eta : S3 <\mathbb{Q} \eta$
 $S3 < \eta = \text{mid } k \ K_3 \leq k$

$S3 \leq \eta : S3 \leq \mathbb{Q} \eta$
 $S3 \leq \eta = <\mathbb{Q} \rightarrow \leq \mathbb{Q} \{S3\} \{\eta\} S3 < \eta$

-- Symmetry on S4: diagSeq-stable gives the swapped form.

$S4\text{-sym} : \text{dist}\mathbb{Q} (\text{diagSeq } r \ n) (\text{seq } (r \ n) \ k) \simeq \mathbb{Q} S4$
 $S4\text{-sym} = \text{dist}\mathbb{Q}\text{-sym } (\text{diagSeq } r \ n) (\text{seq } (r \ n) \ k)$

$S4\text{-sym-le} : \text{dist}\mathbb{Q} (\text{diagSeq } r \ n) (\text{seq } (r \ n) \ k) \leq \mathbb{Q} S4$
 $S4\text{-sym-le} = \simeq \mathbb{Q} \rightarrow \leq \mathbb{Q}^l \{\text{dist}\mathbb{Q} (\text{diagSeq } r \ n) (\text{seq } (r \ n) \ k)\} \{S4\} S4\text{-sym}$

$S4\text{-sym-ge} : S4 \leq \mathbb{Q} \text{ dist}\mathbb{Q} (\text{diagSeq } r \ n) (\text{seq } (r \ n) \ k)$
 $S4\text{-sym-ge} = \simeq \mathbb{Q} \rightarrow \leq \mathbb{Q}^l \{S4\} \{\text{dist}\mathbb{Q} (\text{diagSeq } r \ n) (\text{seq } (r \ n) \ k)\}$

$(\simeq \mathbb{Q}\text{-sym } \{\text{dist}\mathbb{Q}(\text{diagSeq } r \ n) (\text{seq } (r \ n) \ k)\} \{S4\} \ S4\text{-sym})$
 $S4< : S4 < \mathbb{Q} \text{ halfTo } n$
 $S4< = \leq \mathbb{Q} \rightarrow \mathbb{Q} \{S4\} \{\text{dist}\mathbb{Q}(\text{diagSeq } r \ n) (\text{seq } (r \ n) \ k)\} \{\text{halfTo } n\}$
 $\quad S4\text{-sym-ge } (\text{diagSeq-stable } r \ n \ k \ di_n \leq k)$
 $S4 \leq \eta : S4 \leq \mathbb{Q} \ \eta$
 $S4 \leq \eta = \leq \mathbb{Q}\text{-trans } \{S4\} \{\text{halfTo } n\} \{\eta\}$
 $\quad (\mathbb{Q} \rightarrow \leq \mathbb{Q} \{S4\} \{\text{halfTo } n\} \ S4<) \ \text{halfTo } n \leq \eta$
-- Sum bounds.
 $S1\text{Nonneg} : 0 \mathbb{Q} \leq \mathbb{Q} \ S1$
 $S1\text{Nonneg} = \text{dist}\mathbb{Q}\text{-nonneg } (\text{diagSeq } r \ m) (\text{seq } (r \ m) \ k)$
 $S3\text{Nonneg} : 0 \mathbb{Q} \leq \mathbb{Q} \ S3$
 $S3\text{Nonneg} = \text{dist}\mathbb{Q}\text{-nonneg } (\text{seq } (r \ m) \ k) (\text{seq } (r \ n) \ k)$
 $S4\text{Nonneg} : 0 \mathbb{Q} \leq \mathbb{Q} \ S4$
 $S4\text{Nonneg} = \text{dist}\mathbb{Q}\text{-nonneg } (\text{seq } (r \ n) \ k) (\text{diagSeq } r \ n)$
 $S3+S4 \leq \eta+\eta : (S3 + \mathbb{Q} \ S4) \leq \mathbb{Q} (\eta + \mathbb{Q} \ \eta)$
 $S3+S4 \leq \eta+\eta = \leq \mathbb{Q}\text{-sum}\leq\text{double-nonneg } S3 \ S4 \ \eta$
 $\quad S3\text{Nonneg } S4\text{Nonneg } \eta\text{Nonneg } S3 \leq \eta \ S4 \leq \eta$
 $S3+S4 \leq \delta : (S3 + \mathbb{Q} \ S4) \leq \mathbb{Q} \ \delta$
 $S3+S4 \leq \delta = \leq \mathbb{Q}\text{-trans } \{S3 + \mathbb{Q} \ S4\} \{\eta + \mathbb{Q} \ \eta\} \{\delta\}$
 $\quad S3+S4 \leq \eta+\eta \ \eta+\eta \leq \delta$
 $S3+S4\text{Nonneg} : 0 \mathbb{Q} \leq \mathbb{Q} (S3 + \mathbb{Q} \ S4)$
 $S3+S4\text{Nonneg} = 0 \leq \mathbb{Q} + \mathbb{Q} \ S3 \ S4 \ S3\text{Nonneg } S4\text{Nonneg}$
 $S1 \leq \delta : S1 \leq \mathbb{Q} \ \delta$
 $S1 \leq \delta = \leq \mathbb{Q}\text{-trans } \{S1\} \{\eta\} \{\delta\}$
 $\quad S1 \leq \eta (\leq \mathbb{Q}\text{-trans } \{\eta\} \{\eta + \mathbb{Q} \ \eta\} \{\delta\})$
 $\quad (\leq \mathbb{Q}\text{-add-nonneg-right } \eta \ \eta \ \eta\text{Nonneg}) \ \eta+\eta \leq \delta$
 $\text{total} \leq \delta+\delta : (S1 + \mathbb{Q} (S3 + \mathbb{Q} \ S4)) \leq \mathbb{Q} (\delta + \mathbb{Q} \ \delta)$
 $\text{total} \leq \delta+\delta = \leq \mathbb{Q}\text{-sum}\leq\text{double-nonneg } S1 (S3 + \mathbb{Q} \ S4) \ \delta$
 $\quad S1\text{Nonneg } S3+S4\text{Nonneg } \delta\text{Nonneg } S1 \leq \delta \ S3+S4 \leq \delta$
 $\text{total} < \epsilon : (S1 + \mathbb{Q} (S3 + \mathbb{Q} \ S4)) < \mathbb{Q} \ \epsilon$
 $\text{total} < \epsilon = \leq \mathbb{Q} \rightarrow \mathbb{Q} \{S1 + \mathbb{Q} (S3 + \mathbb{Q} \ S4)\} \{\delta + \mathbb{Q} \ \delta\} \{\epsilon\}$
 $\quad \text{total} \leq \delta+\delta \ \delta+\delta < \epsilon$
-- Triangle assembly.
 $\text{tri-inner} : \text{dist}\mathbb{Q} (\text{seq } (r \ m) \ k) (\text{diagSeq } r \ n) \leq \mathbb{Q} (S3 + \mathbb{Q} \ S4)$
 $\text{tri-inner} = \text{dist}\mathbb{Q}\text{-triangle } (\text{seq } (r \ m) \ k) (\text{seq } (r \ n) \ k) (\text{diagSeq } r \ n)$
 $\text{tri-outer} : \text{dist}\mathbb{Q} (\text{diagSeq } r \ m) (\text{diagSeq } r \ n) \leq \mathbb{Q}$
 $\quad (S1 + \mathbb{Q} \ \text{dist}\mathbb{Q} (\text{seq } (r \ m) \ k) (\text{diagSeq } r \ n))$
 $\text{tri-outer} = \text{dist}\mathbb{Q}\text{-triangle } (\text{diagSeq } r \ m) (\text{seq } (r \ m) \ k) (\text{diagSeq } r \ n)$
-- Combine: $S1 + \text{dist}\mathbb{Q}(\text{seq } r \ m \ k, \text{diagSeq } r \ n) \leq S1 + (S3 + S4)$.
 $\text{outer-bnd} : (S1 + \mathbb{Q} \ \text{dist}\mathbb{Q} (\text{seq } (r \ m) \ k) (\text{diagSeq } r \ n)) \leq \mathbb{Q}$
 $\quad (S1 + \mathbb{Q} (S3 + \mathbb{Q} \ S4))$

```

outer-bnd = ≤Q+Q-mono-left S1
           (distQ (seq (r m) k) (diagSeq r n)) (S3 +Q S4) tri-inner

full≤ : distQ (diagSeq r m) (diagSeq r n) ≤Q (S1 +Q (S3 +Q S4))
full≤ = ≤Q-trans
      {distQ (diagSeq r m) (diagSeq r n)}
      {S1 +Q distQ (seq (r m) k) (diagSeq r n)}
      {S1 +Q (S3 +Q S4)}
      tri-outer outer-bnd
in
≤<Q→<Q {distQ (diagSeq r m) (diagSeq r n)}
        {S1 +Q (S3 +Q S4)} {ε}
full≤ total<ε)
}

```

The diagonal already lives in $\mathbb{R}$: package the rational Cauchy sequence with its Cauchy proof to obtain the candidate limit. Two further rounds of the same triangle bookkeeping then show that the original sequence of reals converges to it, where convergence is again read coordinatewise on the rational tails.

The candidate limit must be inhabited by a real number, and the original sequence must approach it in the only sense available: rational closeness on a tail. The limit is the record built from the diagonal sequence and its Cauchy proof. Convergence is obtained by inserting the diagonal as the middle point of a single triangle, where one side is bounded by `diagSeq-stable` and the other by the diagonal's own Cauchy modulus. Thus $\mathbb{R}$, defined as Cauchy sequences of rationals modulo eventual closeness, is itself sequentially complete: every Cauchy sequence of reals has a real limit.

```

-- § The diagonal packaged as a real number: the limit candidate.
diagℝ : (r : ℕ → ℝ) → IsCauchyℝ r → ℝ
diagℝ r ic = mkℝ (diagSeq r) (diagSeq-cauchy r ic)

-- § Convergence: each row r n approximates the diagonal limit.
diagℝ-converges :
(r : ℕ → ℝ) → (ic : IsCauchyℝ r) →
r converges-to diagℝ r ic
diagℝ-converges r ic = record
{ convℝ = λ ε ∈ εpos →
  let
    η : ℚ
    η = εQuarter ε

    ηpos : 0ℚ <ℚ η
    ηpos = εQuarter-pos ε

    ηNonneg : 0ℚ ≤ℚ η
    ηNonneg = <ℚ→≤ℚ {0ℚ} {η} ηpos

    η+η<ε : (η +ℚ η) <ℚ ε
    η+η<ε = εQuarter-double<ε ε ∈ εpos

  -- diagSeq is Cauchy; pack at η.

```

```

ic-diag : IsCauchy (diagSeq r)
ic-diag = diagSeq-cauchy r ic

packD = IsCauchy.cauchy ic-diag η ηpos
N_d : ℕ
N_d = fst packD
diagBnd : (m n : ℕ) → N_d ≤ m → N_d ≤ n →
          distℚ (diagSeq r m) (diagSeq r n) <ℚ η
diagBnd = snd packD

-- halfTo schedule control.
packH = halfTo-arch η ηpos
N_h : ℕ
N_h = fst packH
N_h<η : halfTo N_h <ℚ η
N_h<η = snd packH

N_h≤η : halfTo N_h ≤ℚ η
N_h≤η = <ℚ→≤ℚ {halfTo N_h} {η} N_h<η

N : ℕ
N = maxℕ N_d N_h
in
N , (λ n N≤n →
  let
    N_d≤n : N_d ≤ n
    N_d≤n = ≤-trans (≤-maxℕl N_d N_h) N≤n

    N_h≤n : N_h ≤ n
    N_h≤n = ≤-trans (≤-maxℕr N_d N_h) N≤n

    di_n : ℕ
    di_n = diag-cauchyIdx (r n) n

    K : ℕ
    K = maxℕ di_n N_d

    di_n≤K : di_n ≤ K
    di_n≤K = ≤-maxℕl di_n N_d

    N_d≤K : N_d ≤ K
    N_d≤K = ≤-maxℕr di_n N_d

    halfn≤η : halfTo n ≤ℚ η
    halfn≤η = ≤ℚ-trans {halfTo n} {halfTo N_h} {η}
      (halfTo-mono N_h n N_h≤n) N_h≤η
  in
    K , (λ k K≤k →
      let
        di_n≤k : di_n ≤ k
        di_n≤k = ≤-trans di_n≤K K≤k

        N_d≤k : N_d ≤ k
        N_d≤k = ≤-trans N_d≤K K≤k

```

```

S1 : ℚ
S1 = distℚ (seq (r n) k) (diagSeq r n)

S2 : ℚ
S2 = distℚ (diagSeq r n) (diagSeq r k)

-- S1 bound via symmetry of distℚ + diagSeq-stable.
S1-sym : distℚ (diagSeq r n) (seq (r n) k) ≈ℚ S1
S1-sym = distℚ-sym (diagSeq r n) (seq (r n) k)

S1-sym-ge : S1 ≤ℚ distℚ (diagSeq r n) (seq (r n) k)
S1-sym-ge = ≈ℚ→≤ℚ {S1} {distℚ (diagSeq r n) (seq (r n) k)}
  (≈ℚ-sym {distℚ (diagSeq r n) (seq (r n) k)} {S1} S1-sym)

S1<halfn : S1 <ℚ halfTo n
S1<halfn = ≤<ℚ→<ℚ
  {S1} {distℚ (diagSeq r n) (seq (r n) k)} {halfTo n}
  S1-sym-ge (diagSeq-stable r n k di_n≤k)

S1≤η : S1 ≤ℚ η
S1≤η = ≤ℚ-trans {S1} {halfTo n} {η}
  (<ℚ→≤ℚ {S1} {halfTo n} S1<halfn) halfn≤η

S2<η : S2 <ℚ η
S2<η = diagBnd n k N_d≤n N_d≤k

S2≤η : S2 ≤ℚ η
S2≤η = <ℚ→≤ℚ {S2} {η} S2<η

S1Nonneg : 0ℚ ≤ℚ S1
S1Nonneg = distℚ-nonneg (seq (r n) k) (diagSeq r n)

S2Nonneg : 0ℚ ≤ℚ S2
S2Nonneg = distℚ-nonneg (diagSeq r n) (diagSeq r k)

sum≤ : (S1 +ℚ S2) ≤ℚ (η +ℚ η)
sum≤ = ≤ℚ-sum≤double-nonneg S1 S2 η
  S1Nonneg S2Nonneg ηNonneg S1≤η S2≤η

sum<ε : (S1 +ℚ S2) <ℚ ε
sum<ε = ≤<ℚ→<ℚ {S1 +ℚ S2} {η +ℚ η} {ε} sum≤ η+η<ε

tri : distℚ (seq (r n) k) (diagSeq r k) ≤ℚ (S1 +ℚ S2)
tri = distℚ-triangle (seq (r n) k) (diagSeq r n) (diagSeq r k)
in
  ≤<ℚ→<ℚ {distℚ (seq (r n) k) (diagSeq r k)} {S1 +ℚ S2} {ε}
    tri sum<ε)
}

-- § Cauchy completeness of ℝ: every Cauchy sequence of reals converges.
ℝ-cauchy-complete :
  (r : ℕ → ℝ) → IsCauchyℝ r → Σ ℝ (λ x → r converges-to x)
ℝ-cauchy-complete r ic = diagℝ r ic , diagℝ-converges r ic

```


Part VI

Derived Constants of K_4

Part VI collects the combinatorial and spectral invariants of K_4 into explicit verified records and computes their numerical values. The aim is not to introduce new structure, but to evaluate, in closed form, what has already been derived.

The vertex count, edge count, Euler characteristic, and Laplacian eigenvalues are assembled into a single computation record, each field being witnessed by definitional or proved equality in Agda. In particular, the graph evaluation $\mathcal{C} = 137$ appears here as a consequence of the earlier derivation together with the arithmetic of Part V.

The record is then shown to be unique: any two admissible invariant records agree fieldwise. The final results return to the represented binary distinction and show how the measurement-theoretic and eliminative strands close the formal circle.

One should distinguish the internal and external readings of these computations. Internally, the claim is that the stated equalities reduce and the stated numbers are derivable in the formal system. Any broader interpretation lies outside the formal argument.

Chapter 55

Natural-Number Division Infrastructure

Several later combinatorial identities require exact division at the level of natural numbers, but the safe base environment does not provide the requisite primitives. This chapter supplies the small amount of arithmetic infrastructure needed for those calculations while preserving totality.

Two fuel-bounded procedures are introduced: a greatest common divisor and an exact division routine. In each case recursion is controlled by an explicit natural-number counter, because the present base offers no general well-founded recursion principle. The fuel is chosen as the sum of the inputs; each Euclidean subtraction step reduces that sum, so this bound suffices.

The same idea governs exact division. The number of successful subtractions is made explicit, and the resulting procedure remains total.

```
-- § Boolean comparison on  $\mathbb{N}$ 
_≤ $\mathbb{N}$ _ :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Bool}$ 
zero ≤ $\mathbb{N}$  _      = true
suc _ ≤ $\mathbb{N}$  zero = false
suc m ≤ $\mathbb{N}$  suc n = m ≤ $\mathbb{N}$  n

-- § Fuel-bounded GCD
gcd-fuel :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
gcd-fuel zero m _      = m
gcd-fuel (suc _) zero n = n
gcd-fuel (suc _) m zero = m
gcd-fuel (suc f) (suc m) (suc n) with (suc m) ≤ $\mathbb{N}$  (suc n)
... | true = gcd-fuel f (suc m) (n ÷ m)
... | false = gcd-fuel f (m ÷ n) (suc n)

-- § One-step exposure of gcd-fuel on positive inputs.
gcd-fuel-step :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Bool} \rightarrow \mathbb{N}$ 
```

```

gcd-fuel-step f m n true = gcd-fuel f m (n - m)
gcd-fuel-step f m n false = gcd-fuel f (m - n) n

gcd-fuel-suc-suc : (f m n : ℕ) → gcd-fuel (suc f) (suc m) (suc n) ≡ gcd-fuel-step f (suc m) (suc n) (m ≤ℕ n)
gcd-fuel-suc-suc f m n with m ≤ℕ n
... | true = refl
... | false = refl

-- § GCD with canonical fuel
gcd : ℕ → ℕ → ℕ
gcd m n = gcd-fuel (m + n) m n

```

Once the gcd routine is available, the only remaining primitive is exact division. When a divisor d divides n , the quotient must be computable at the level of natural numbers. Division is again fuel-bounded, counting how many times the divisor can be subtracted before the remainder falls below it.

```

-- § Successor coercion to ℕ+
sucToℕ+ : ℕ → ℕ+
sucToℕ+ zero = one+
sucToℕ+ (suc n) = suc+ (sucToℕ+ n)

-- § General ℕ to ℕ+ coercion
ℕ-to-ℕ+ : ℕ → ℕ+
ℕ-to-ℕ+ = mkℕ+

-- § Fuel-bounded integer division
div-fuel : ℕ → ℕ → ℕ+ → ℕ
div-fuel zero _ _ = zero
div-fuel (suc f) n d with +toℕ d ≤ℕ n
... | true = suc (div-fuel f (n - +toℕ d) d)
... | false = zero

-- § One-step exposure of div-fuel on positive fuel.
div-fuel-step : ℕ → ℕ → ℕ+ → Bool → ℕ
div-fuel-step f n d true = suc (div-fuel f (n - +toℕ d) d)
div-fuel-step f n d false = zero

div-fuel-suc : (f n : ℕ) → (d : ℕ+) → div-fuel (suc f) n d ≡ div-fuel-step f n d (+toℕ d ≤ℕ n)
div-fuel-suc f n d with +toℕ d ≤ℕ n
... | true = refl
... | false = refl

-- § Division by ℕ+
_div_ : ℕ → ℕ+ → ℕ
n div d = div-fuel n n d

-- § Division by ℕ (zero-safe)
_divℕ_ : ℕ → ℕ → ℕ
_divℕ_ zero = zero
n divℕ (suc d) = n div (sucToℕ+ d)

```

Canonical Quotient Data and Reduced Witnesses

The quotient structure of the rationals must also be made explicit at the level of observable data. For the later normalization arguments, numerator and denominator information cannot remain implicit in the setoid presentation.

The gcd and division infrastructure developed above allows one to extract reduced witnesses for every rational class, including zero and negative values. The next definitions therefore make quotient data visible, introduce the corresponding presentation records, and prepare the normalization and uniqueness statements that follow.

Quotient presentation and field uniqueness

Before reduction can be discussed, the rational type must expose its quotient fields as inspectable data. The lift into Set_ω makes rationals available as values of admissible observables, and the presentation record packages the numerator and denominator together with the witnessing equality.

The construction is direct. The wrapper $\mathbb{Q}_\omega$ places rationals in the ambient universe, and `RationalPresentation` records the two constructor fields of a given rational. Since a fixed rational already determines those fields, the resulting presentation is unique.

```
-- § Lift rationals into Setω so admissible observables can target them.
record Qω : Setω where
  constructor wrapQ
  field
    lowerQ : Q

open Qω public

AdmissibleRationalObservable : Setω
AdmissibleRationalObservable = AdmissibleObservable Qω

-- § Quotient presentation of a rational value.
record RationalPresentation (q : Q) : Set where
  field
    pres-num : Z
    pres-den : N+
    presents-q : q ≡ pres-num / pres-den

-- § The constructor fields are already the canonical quotient data.
canonical-rational-presentation : (q : Q) → RationalPresentation q
canonical-rational-presentation (n / d) = record
{ pres-num = n
; pres-den = d
; presents-q = refl
}

-- § Quotient fields are unique once a rational value is fixed.
```

```

rational-fields-unique :
  (q : ℚ) →
  (n1 n2 : ℤ) →
  (d1 d2 : ℕ+) →
  q ≡ (n1 / d1) →
  q ≡ (n2 / d2) →
  (n1 ≡ n2) × (d1 ≡ d2)
rational-fields-unique (n / d) n1 n2 d1 d2 refl refl = refl , refl

```

Reduced witnesses and irreducibility

Once the quotient fields are visible, one can ask when a rational is already in reduced form. The relevant condition is coprimality: a quotient n/d is *irreducible* when $\gcd(|n|, d) = 1$.

A *reduced rational witness* packages a nonnegative numerator, a positive denominator, the equation to the original rational, and a proof of coprimality. Uniqueness of such witnesses uses two facts: the uniqueness of quotient fields established above and the injectivity of the embedding $\mathbb{N} \hookrightarrow \mathbb{Z}$, which is proved here for later use.

```

-- § The ℕ-embedding into ℤ is injective.
fromℕℤ-injective : {m n : ℕ} → fromℕℤ m ≡ fromℕℤ n → m ≡ n
fromℕℤ-injective {zero} {zero} refl = refl
fromℕℤ-injective {zero} {suc n} ()
fromℕℤ-injective {suc m} {zero} ()
fromℕℤ-injective {suc m} {suc n} refl = refl

-- § Irreducibility for rational quotients: numerator magnitude and denominator are coprime.
IrreducibleQuotient : ℚ → Set
IrreducibleQuotient q = gcd (magℤ (num q)) (+toℕ (den q)) ≡ 1

-- § Reduced witness for a rational quotient with nonnegative numerator.
record ReducedRationalWitness (q : ℚ) : Set where
  field
    red-num : ℕ
    red-den : ℕ+
    red-value : q ≡ fromℕℤ red-num / red-den
    red-coprime : gcd red-num (+toℕ red-den) ≡ 1

-- § Reduced witnesses are unique whenever they exist.
reduced-rational-witness-unique :
  {q : ℚ} →
  (r1 r2 : ReducedRationalWitness q) →
  ReducedRationalWitness.red-num r1 ≡ ReducedRationalWitness.red-num r2
  × ReducedRationalWitness.red-den r1 ≡ ReducedRationalWitness.red-den r2
reduced-rational-witness-unique {q} r1 r2 =
  let
    pair : fromℕℤ (ReducedRationalWitness.red-num r1)
      ≡ fromℕℤ (ReducedRationalWitness.red-num r2)
      × (ReducedRationalWitness.red-den r1 ≡ ReducedRationalWitness.red-den r2)
    pair =

```

```

rational-fields-unique q
  (fromℕℤ (ReducedRationalWitness.red-num r₁))
  (fromℕℤ (ReducedRationalWitness.red-num r₂))
  (ReducedRationalWitness.red-den r₁)
  (ReducedRationalWitness.red-den r₂)
  (ReducedRationalWitness.red-value r₁)
  (ReducedRationalWitness.red-value r₂)
in
fromℕℤ-injective (fst pair) , snd pair

```

Arithmetic lemmas and exact division

The reduction algorithm requires a short list of elementary arithmetic lemmas about subtraction, comparison reflection, and exact division. These results make the later normalization proofs fully explicit inside the type theory.

```

-- § Magnitude of the ℕ-embedding is the original natural number.
magℤ-fromℕℤ : (n : ℕ) → magℤ (fromℕℤ n) ≡ n
magℤ-fromℕℤ zero = refl
magℤ-fromℕℤ (suc n) = refl

-- § Subtracting zero on the right does nothing.
⋅-zero-right : (n : ℕ) → n ⋅ zero ≡ n
⋅-zero-right zero = refl
⋅-zero-right (suc n) = refl

-- § Subtracting one from a successor recovers the predecessor.
⋅-one-suc : (n : ℕ) → suc n ⋅ 1 ≡ n
⋅-one-suc n = ⋅-zero-right n

-- § A left summand cancels against subtraction from the whole sum.
+ℕ-⋅-cancel-left : (m n : ℕ) → (m +ℕ n) ⋅ m ≡ n
+ℕ-⋅-cancel-left zero n = ⋅-zero-right n
+ℕ-⋅-cancel-left (suc m) n = +ℕ-⋅-cancel-left m n

-- § Every natural is ≤ℕ its own right extension.
≤ℕ-self-+ℕ-right : (m n : ℕ) → m ≤ℕ (m +ℕ n) ≡ true
≤ℕ-self-+ℕ-right zero n = refl
≤ℕ-self-+ℕ-right (suc m) n = ≤ℕ-self-+ℕ-right m n

-- § Boolean ≤ℕ reflects propositional ≤.
≤ℕ-true→≤ : {m n : ℕ} → m ≤ℕ n ≡ true → m ≤ n
≤ℕ-true→≤ {zero} {n} _ = z≤n
≤ℕ-true→≤ {suc m} {zero} ()
≤ℕ-true→≤ {suc m} {suc n} eq = s≤s (≤ℕ-true→≤ eq)

-- § Boolean failure of ≤ℕ forces the reverse propositional order.
≤ℕ-false→≤ : {m n : ℕ} → m ≤ℕ n ≡ false → n ≤ m
≤ℕ-false→≤ {zero} {n} ()
≤ℕ-false→≤ {suc m} {zero} _ = z≤n
≤ℕ-false→≤ {suc m} {suc n} eq = s≤s (≤ℕ-false→≤ eq)

```

```

-- § Ordered subtraction splits the larger number into base plus remainder.
≤→+N-.-split : {m n : ℕ} → m ≤ n → m +N (n -.- m) ≡ n
≤→+N-.-split {zero} {n} z ≤ n = -.-zero-right n
≤→+N-.-split {suc m} {suc n} (s ≤ s p) = cong suc (≤→+N-.-split p)

-- § Packing the predecessor of a positive natural recovers the same ℕ+.
mkN+-+toN-minus-one : (d : ℕ+) → mkN+ (+toN d -.- 1) ≡ d
mkN+-+toN-minus-one (mkN+ p) = cong mkN+ (-.-zero-right p)

-- § Division by one leaves naturals unchanged.
div-one+ : (n : ℕ) → n div one+ ≡ n
div-one+ zero = refl
div-one+ (suc n) =
  cong suc
  (trans
   (cong (λ t → div-fuel n t one+) (-.-zero-right n))
   (div-one+ n))

```

The exact-division theorem states that the fuel parameter has no semantic effect on an exact multiple. The proof is a simple induction on the quotient: one subtraction step removes one copy of the divisor, passes the remaining slack forward, and invokes the inductive hypothesis. Thus, whenever divisibility is known, the bounded division procedure computes the correct quotient.

```

-- § Division recovers the quotient on exact multiples, even with extra fuel.
div-fuel-exact-multiple :
  (q slack : ℕ) → (d : ℕ+) →
  div-fuel ((q *N+ +toN d) +N slack) (q *N+ +toN d) d ≡ q
div-fuel-exact-multiple zero zero (mkN+ p) = refl
div-fuel-exact-multiple zero (suc slack) (mkN+ p) = refl
div-fuel-exact-multiple (suc q) slack (mkN+ p) =
  let
    qk : ℕ
    qk = q *N+ suc p

    fuelShape : ((suc q *N+ suc p) +N slack) ≡ suc (p +N (qk +N slack))
    fuelShape = cong suc (+N-assoc p qk slack)

    cmpEq : suc p ≤N (suc q *N+ suc p) ≡ true
    cmpEq = ≤N-self-+N-right p qk

    subEq : (suc q *N+ suc p) -.- suc p ≡ qk
    subEq = +N-.-cancel-left (suc p) qk

    fuelEq : p +N (qk +N slack) ≡ qk +N (p +N slack)
    fuelEq =
      trans
      (sym (+N-assoc p qk slack))
      (trans
       (cong (λ t → t +N slack) (+N-comm p qk))
       (+N-assoc qk p slack))

```



```

in
trans
  (cong (λ t → div-fuel t (suc q *N suc p) (mkN+ p)) fuelShape)
  (trans
    (div-fuel-suc (p +N (qk +N slack)) (suc q *N suc p) (mkN+ p))
    (trans
      (cong (div-fuel-step (p +N (qk +N slack)) (suc q *N suc p) (mkN+ p)) cmpEq)
      (cong suc
        (trans
          (cong (λ t → div-fuel (p +N (qk +N slack)) t (mkN+ p)) subEq)
          (trans
            (cong (λ t → div-fuel t qk (mkN+ p)) fuelEq)
            (div-fuel-exact-multiple q (p +N slack) (mkN+ p)))))))
-- § Division by a positive factor recovers the quotient on exact multiples.
div-exact-multiple : (q : N) → (d : N+) → (q *N+toN d) div d ≡ q
div-exact-multiple q d =
  trans
    (cong (λ t → div-fuel t (q *N+toN d) d) (sym (+N-zero-right (q *N+toN d))))
    (div-fuel-exact-multiple q zero d)

```

Positive divisibility theory

A constructive divisibility notion is now needed for the gcd development. The statement that d divides n must carry an explicit quotient and factorization proof, because later arguments extract that quotient to compute exact division.

The record `Divides+` packages precisely this data. From it one derives the basic closure properties required later: addition, subtraction, multiplication, cancellation, order reflection, and extraction of exact quotients.

```

-- § Positive divisibility packages an explicit quotient witness.
record Divides+ (d : N+) (n : N) : Set where
  field
    quotient+ : N
    factor+ : n ≡ quotient+ *N+toN d

open Divides+ public

-- § Every exact multiple yields a positive divisibility witness.
divides+-multiple : (q : N) → (d : N+) → Divides+ d (q *N+toN d)
divides+-multiple q d = record
  { quotient+ = q
  ; factor+ = refl
  }

-- § The divisor divides itself.
divides+-self : (d : N+) → Divides+ d (+toN d)
divides+-self d = record
  { quotient+ = oneNat
  ; factor+ = sym (*N-one-left (+toN d))
  }

```

```

-- § Zero is divisible by every positive natural.
divides+-zero : (d : ℕ+) → Divides+ d zero
divides+-zero d = record
{ quotient+ = zero
; factor+ = refl
}

-- § Positive divisibility is closed under addition.
divides+-+ℕ : {d : ℕ+} {m n : ℕ} → Divides+ d m → Divides+ d n → Divides+ d (m +ℕ n)
divides+-+ℕ {d} {m} {n} dm dn = record
{ quotient+ = quotient+ dm +ℕ quotient+ dn
; factor+ =
  trans
    (cong2 _+ℕ_ (factor+ dm) (factor+ dn))
    (sym (*ℕ-distrib-left-+ℕ (quotient+ dm) (quotient+ dn) (+toℕ d)))
}

-- § Exact division extracts the stored quotient from a positive divisibility witness.
divides+→div : {d : ℕ+} {n : ℕ} → (dvd : Divides+ d n) → n div d ≡ quotient+ dvd
divides+→div {d} {n} dvd =
  trans
    (cong (λ t → t div d) (factor+ dvd))
    (div-exact-multiple (quotient+ dvd) d)

-- § A positive divisible natural has a positive stored quotient.
divides+-positive-quotient : {d : ℕ+} {n : ℕ} → (dvd : Divides+ d (suc n)) → ∑ ℕ (λ k → quotient+ dvd ≡ suc k)
divides+-positive-quotient {d} {n} dvd with quotient+ dvd | factor+ dvd
... | zero | ()
... | suc k | _ = k, refl

-- § Built-in addition and manuscript addition coincide on naturals.
+to-+ℕ : (m n : ℕ) → (m + n) ≡ (m +ℕ n)
+to-+ℕ zero n = refl
+to-+ℕ (suc m) n = cong suc (+to-+ℕ m n)

-- § The ℕ-view of ℕ+ multiplication is ordinary natural multiplication.
+toℕ-*+ : (x y : ℕ+) → +toℕ (x *+ y) ≡ +toℕ x *ℕ +toℕ y
+toℕ-*+ (mkℕ+ a) (mkℕ+ b) = cong suc (+ℕ-comm (a *ℕ suc b) b)

-- § Divisibility lifts through multiplication by a positive factor on the right.
divides+-mul-right : {e : ℕ+} {n : ℕ} → Divides+ e n → (d : ℕ+) → Divides+ (e *+ d) (n *ℕ +toℕ d)
divides+-mul-right {e} {n} dvd d = record
{ quotient+ = quotient+ dvd
; factor+ =
  trans
    (cong (λ t → t *ℕ +toℕ d) (factor+ dvd))
    (trans
      (*ℕ-assoc (quotient+ dvd) (+toℕ e) (+toℕ d))
      (cong (λ t → quotient+ dvd *ℕ t) (sym (+toℕ-*+ e d))))
}

-- § Equality after multiplying by the same positive factor cancels on the right.
*ℕ-cancelr-suc : {a b : ℕ} → (k : ℕ) → a *ℕ suc k ≡ b *ℕ suc k → a ≡ b

```

```

*N-cancelr-suc {a} {b} k eq =
  ≤-antisym
  (≤-*N-cancelr-suc k (subst (λ t → (a *N suc k) ≤ t) eq (≤-refl (a *N suc k))))
  (≤-*N-cancelr-suc k (subst (λ t → (b *N suc k) ≤ t) (sym eq) (≤-refl (b *N suc k))))

-- § A product of two positive naturals equals one only in the trivial case.
positive-product-one : (a b : N) → suc a *N suc b ≡ oneNat → (a ≡ zero) × (b ≡ zero)
positive-product-one a zero eq =
  let
    aEq : suc a ≡ oneNat
    aEq = trans (sym (*N-one-right (suc a))) eq
  in
    suc-injective aEq , refl
positive-product-one a (suc b) ()

-- § If a product divisor of d also divides d, the extra factor must be one.
divides+-product-factor-left-one : (e d : N+) → Divides+ (e *+ d) (+toN d) → e ≡ one+
divides+-product-factor-left-one e d dvd =
  let
    qPos : Σ N (λ k → quotient+ dvd ≡ suc k)
    qPos = divides+-positive-quotient dvd

    k : N
    k = fst qPos

    qEq : quotient+ dvd ≡ suc k
    qEq = snd qPos

    factored : +toN d ≡ ((suc k *N +toN e) *N +toN d)
    factored =
      trans
        (factor+ dvd)
      (trans
        (cong (λ t → t *N +toN (e *+ d)) qEq)
        (trans
          (cong (λ t → suc k *N t) (+toN-*+ e d))
          (sym (*N-assoc (suc k) (+toN e) (+toN d)))))

    prodOne : suc k *N +toN e ≡ oneNat
    prodOne =
      *N-cancelr-suc
      (pred d)
      (trans
        (sym factored)
        (sym (*N-one-left (+toN d))))

    ePredZero : pred e ≡ zero
    ePredZero = snd (positive-product-one k (pred e) prodOne)
  in
    cong mkN+ ePredZero

```

These factor lemmas allow comparison of stored quotients. If d divides both m and n and $m \leq n$, then the quotient associated with m is at most the quotient associated with n .

```

-- § Order on divisible multiples reflects to the stored quotients.
divides+-quotient-≤ :
{d : ℕ+} {m n : ℕ} →
m ≤ n →
(dm : Divides+ d m) →
(dn : Divides+ d n) →
Divides+.quotient+ dm ≤ Divides+.quotient+ dn
divides+-quotient-≤ {mkℕ+ p} {m} {n} m≤n dm dn =
let
  step1 : m ≤ (Divides+.quotient+ dn *ℕ (suc p))
  step1 = subst (λ t → m ≤ t) (Divides+.factor+ dn) m≤n

  step2 : (Divides+.quotient+ dm *ℕ (suc p)) ≤ (Divides+.quotient+ dn *ℕ (suc p))
  step2 = subst (λ t → t ≤ (Divides+.quotient+ dn *ℕ (suc p))) (Divides+.factor+ dm) step1
in
≤- *ℕ-cancel+-suc p step2

-- § Positive divisibility survives subtraction of a smaller divisible multiple.
divides+---≤ :
{d : ℕ+} {m n : ℕ} →
m ≤ n →
Divides+ d m →
Divides+ d n →
Divides+ d (n - m)
divides+---≤ {d} {m} {n} m≤n dm dn = record
{ quotient+ = Divides+.quotient+ dn - Divides+.quotient+ dm
; factor+ = factor
}
where
q≤ : Divides+.quotient+ dm ≤ Divides+.quotient+ dn
q≤ = divides+-quotient-≤ m≤n dm dn

qSplit : Divides+.quotient+ dm +ℕ (Divides+.quotient+ dn - Divides+.quotient+ dm) ≡ Divides+.quotient+ dn
qSplit = ≤-+ℕ--split q≤

nAsSum : n ≡ m +ℕ ((Divides+.quotient+ dn - Divides+.quotient+ dm) *ℕ +toℕ d)
nAsSum =
trans
(Divides+.factor+ dn)
(trans
(sym (cong (λ t → t *ℕ +toℕ d) qSplit))
(trans
(*ℕ-distrib-left-+ℕ (Divides+.quotient+ dm) (Divides+.quotient+ dn - Divides+.quotient+ dm) (+toℕ d))
(cong (λ t → t +ℕ ((Divides+.quotient+ dn - Divides+.quotient+ dm) *ℕ +toℕ d)) (sym (Divides+.factor+ dm))))))

factor : n - m ≡ (Divides+.quotient+ dn - Divides+.quotient+ dm) *ℕ +toℕ d
factor = subst (λ t → t - m ≡ (Divides+.quotient+ dn - Divides+.quotient+ dm) *ℕ +toℕ d) (sym nAsSum)
(+ℕ--cancel-left m ((Divides+.quotient+ dn - Divides+.quotient+ dm) *ℕ +toℕ d))

-- § Adding back a divisible remainder recovers divisibility of the larger number.
divides+-restore- :
{d : ℕ+} {m n : ℕ} →
m ≤ n →
Divides+ d m →

```

```

+ d (n ÷ m) →
+ d n
divides+-restore-÷ {d} {m} {n} m ≤ n dm dnm =
subst
  (divides+ d)
  (≤ → +N-÷-split m ≤ n)
  (divides+-+N dm dnm)

-- § The N-view of sucToN+ is the expected successor.
+toN-sucToN+ : (n : N) → +toN (sucToN+ n) ≡ suc n
+toN-sucToN+ zero = refl
+toN-sucToN+ (suc n) = cong suc (+toN-sucToN+ n)

-- § The successor-coded positive natural divides its own N-view.
divides+-suc-self : (n : N) → Divides+ (sucToN+ n) (suc n)
divides+-suc-self n =
subst
  (divides+ (sucToN+ n))
  (+toN-sucToN+ n)
  (divides+-self (sucToN+ n))

-- § Zero/zero is a stable fixed point of gcd-fuel.
gcd-fuel-zero-zero : (f : N) → gcd-fuel f zero zero ≡ zero
gcd-fuel-zero-zero zero = refl
gcd-fuel-zero-zero (suc f) = refl

-- § Positive fuel with a zero left input returns the right input.
gcd-fuel-zero-left-suc : (f n : N) → gcd-fuel (suc f) zero (suc n) ≡ suc n
gcd-fuel-zero-left-suc f n = refl

-- § Positive fuel with a zero right input returns the left input.
gcd-fuel-zero-right-suc : (f m : N) → gcd-fuel (suc f) (suc m) zero ≡ suc m
gcd-fuel-zero-right-suc f m = refl

-- § One Euclidean subtraction step preserves the total positive mass in the true branch.
euclid-sum-true : {m n : N} → m ≤ n → (suc m) +N (n ÷ m) ≡ suc n
euclid-sum-true p = cong suc (≤ → +N-÷-split p)

-- § One Euclidean subtraction step preserves the total positive mass in the false branch.
euclid-sum-false : {m n : N} → n ≤ m → (m ÷ n) +N (suc n) ≡ suc m
euclid-sum-false {m} {n} p =
trans
  (+N-comm (m ÷ n) (suc n))
  (cong suc (≤ → +N-÷-split p))

```

GCD engine: common-divisor preservation, output divisibility, and positivity

To use the Euclidean routine in the reduction pipeline, three facts are needed. First, any common positive divisor of the inputs remains a divisor of the output. Second, whenever the output is positive, it divides each input. Third, if at least one input is positive, then the output is positive as well.

Together these statements show that the recursive routine behaves as the usual greatest common divisor for the purposes needed later: it produces a positive common divisor of the inputs, and every other common divisor factors through it.

The proofs follow the recursive subtraction steps and transport the necessary divisibility data through them.

```
-- § Every natural is bounded by its own right extension.
≤-self+ℕ : (m n : ℕ) → m ≤ (m +ℕ n)
≤-self+ℕ zero n = z≤n
≤-self+ℕ (suc m) n = s≤s (≤-self+ℕ m n)

mutual
-- § One recursive Euclidean step preserves any common positive divisor.
gcd-fuel-common-divisor-step :
  {d : ℕ+} →
  (f m n : ℕ) →
  (b : Bool) →
  m ≤ℕ n ≡ b →
  Divides+ d (suc m) →
  Divides+ d (suc n) →
  Divides+ d (gcd-fuel-step f (suc m) (suc n) b)
gcd-fuel-common-divisor-step f m n true eq dm dn =
  gcd-fuel-common-divisor f (suc m) (n - m) dm
  (divides+ - - ≤ (s≤s (≤ℕ-true → ≤ eq)) dm dn)
gcd-fuel-common-divisor-step f m n false eq dm dn =
  gcd-fuel-common-divisor f (m - n) (suc n)
  (divides+ - - ≤ (s≤s (≤ℕ-false → ≤ eq)) dn dm)
  dn

-- § Any common positive divisor survives every Euclidean gcd-fuel step.
gcd-fuel-common-divisor :
  {d : ℕ+} →
  (f m n : ℕ) →
  Divides+ d m →
  Divides+ d n →
  Divides+ d (gcd-fuel f m n)
gcd-fuel-common-divisor zero m n dm dn = dm
gcd-fuel-common-divisor (suc f) zero zero dm dn = divides+-zero _
gcd-fuel-common-divisor (suc f) zero (suc n) dm dn = dn
gcd-fuel-common-divisor (suc f) (suc m) zero dm dn = dm
gcd-fuel-common-divisor {d} (suc f) (suc m) (suc n) dm dn =
  subst
  (Divides+ d)
  (sym (gcd-fuel-suc-suc f m n))
  (gcd-fuel-common-divisor-step f m n (m ≤ℕ n) refl dm dn)

-- § Phase 2: Positive gcd outputs divide both original inputs

-- § Under sufficient fuel, every positive gcd-fuel output divides both inputs.
gcd-fuel-positive-output-divides≤ :
  (fuel m n g : ℕ) →
  ((m +ℕ n) ≤ fuel) →
  (gcd-fuel fuel m n ≡ suc g) →
```

```

((Divides+ (sucToN+ g) m) × (Divides+ (sucToN+ g) n))
gcd-fuel-positive-output-divides ≤ fuel zero zero g enough eq with trans (sym (gcd-fuel-zero-zero fuel)) eq
... | ()
gcd-fuel-positive-output-divides ≤ zero zero (suc n) g () eq
gcd-fuel-positive-output-divides ≤ (suc fuel) zero (suc n) g enough eq =
divides+-zero (sucToN+ g) ,
subst
(Divides+ (sucToN+ g))
(sym eq')
(divides+-suc-self g)
where
eq' : suc n ≡ suc g
eq' = trans (sym (gcd-fuel-zero-left-suc fuel n)) eq
gcd-fuel-positive-output-divides ≤ zero (suc m) zero g () eq
gcd-fuel-positive-output-divides ≤ (suc fuel) (suc m) zero g enough eq =
subst
(Divides+ (sucToN+ g))
(sym eq')
(divides+-suc-self g)
, divides+-zero (sucToN+ g)
where
eq' : suc m ≡ suc g
eq' = trans (sym (gcd-fuel-zero-right-suc fuel m)) eq
gcd-fuel-positive-output-divides ≤ (suc fuel) (suc m) (suc n) g (s ≤ s enough') eq with inspect (m ≤N n)
... | reveal true bEq =
let
p : m ≤ n
p = ≤N-true→≤ bEq

sucn ≤ sum : suc n ≤ (m +N suc n)
sucn ≤ sum = subst (λ t → suc n ≤ t) (+N-comm (suc n) m) (≤-self+N (suc n) m)

subEnough : ((suc m) +N (n -· m)) ≤ fuel
subEnough = subst (λ t → t ≤ fuel) (sym (euclid-sum-true p)) (≤-trans sucn ≤ sum enough')

stepEq0 : gcd-fuel-step fuel (suc m) (suc n) (m ≤N n) ≡ suc g
stepEq0 = trans (sym (gcd-fuel-suc-suc fuel m n)) eq

stepEq : gcd-fuel fuel (suc m) (n -· m) ≡ suc g
stepEq = trans (sym (cong (gcd-fuel-step fuel (suc m) (suc n)) bEq)) stepEq0

rec : Divides+ (sucToN+ g) (suc m) × Divides+ (sucToN+ g) (n -· m)
rec = gcd-fuel-positive-output-divides ≤ fuel (suc m) (n -· m) g subEnough stepEq

dm : Divides+ (sucToN+ g) (suc m)
dm = fst rec

dn-rest : Divides+ (sucToN+ g) (n -· m)
dn-rest = snd rec
in
dm , divides+-restore--· (s ≤ s p) dm dn-rest
... | reveal false bEq =
let
p : n ≤ m

```

```

p = ≤N-false→≤ bEq

sucm≤sum : suc m ≤ (m +N suc n)
sucm≤sum = subst (λ t → suc m ≤ t) (sym (+N-suc-right m n)) (≤-self+N (suc m) n)

subEnough : ((m ÷ n) +N (suc n)) ≤ fuel
subEnough = subst (λ t → t ≤ fuel) (sym (euclid-sum-false p)) (≤-trans suc m ≤sum enough')

stepEq0 : gcd-fuel-step fuel (suc m) (suc n) (m ≤N n) ≡ suc g
stepEq0 = trans (sym (gcd-fuel-suc-suc fuel m n)) eq

stepEq : gcd-fuel fuel (m ÷ n) (suc n) ≡ suc g
stepEq = trans (sym (cong (gcd-fuel-step fuel (suc m) (suc n)) bEq)) stepEq0

rec : Divides+ (sucToN+ g) (m ÷ n) × Divides+ (sucToN+ g) (suc n)
rec = gcd-fuel-positive-output-divides≤ fuel (m ÷ n) (suc n) g subEnough stepEq

dm-rest : Divides+ (sucToN+ g) (m ÷ n)
dm-rest = fst rec

dn : Divides+ (sucToN+ g) (suc n)
dn = snd rec
in
divides+-restore-÷ (s≤s p) dn dm-rest , dn

```

The inductive core of the gcd engine is established: for any fuel level sufficient to reach termination, the positive output divides both inputs. The remaining step generalises this to the canonical fuel formula $s \text{ slack} + (m + n)$, which always provides enough iterations. This is the interface consumed by the rational reduction machinery above.

```

-- § Canonical-or-greater fuel is enough to turn any positive output into a common divisor.
gcd-fuel-positive-output-divides :
  (slack m n g : N) →
  (gcd-fuel (slack +N (m +N n)) m n ≡ suc g) →
  ((Divides+ (sucToN+ g) m) × (Divides+ (sucToN+ g) n))
gcd-fuel-positive-output-divides slack m n g eq =
gcd-fuel-positive-output-divides≤
  (slack +N (m +N n))
  m
  n
  g
  (subst
    (λ t → (m +N n) ≤ t)
    (+N-comm (m +N n) slack)
    (≤-self+N (m +N n) slack))
  eq

-- § Phase 3: Positive total mass forces a positive gcd

-- § Any common positive divisor of m and n divides gcd m n.
common-divisor-divides-gcd :
  {d : N+} →

```



```

(m n : ℕ) →
  Divides+ d m →
  Divides+ d n →
  Divides+ d (gcd m n)
common-divisor-divides-gcd {d} m n dm dn =
  subst
    (Divides+ d)
    (sym (cong (λ t → gcd-fuel t m n) (+to-+ℕ m n)))
    (gcd-fuel-common-divisor (m +ℕ n) m n dm dn)

-- § Under sufficient fuel, a positive total input mass forces a positive gcd-fuel output.
gcd-fuel-positive-sum ≤ :
  (fuel m n s : ℕ) →
  (m +ℕ n ≡ suc s) →
  ((m +ℕ n) ≤ fuel) →
  Σ ℕ (λ g → gcd-fuel fuel m n ≡ suc g)
gcd-fuel-positive-sum ≤ zero zero zero s () enough
gcd-fuel-positive-sum ≤ zero zero (suc n) s sumEq ()
gcd-fuel-positive-sum ≤ zero (suc m) zero s sumEq ()
gcd-fuel-positive-sum ≤ zero (suc m) (suc n) s sumEq ()
gcd-fuel-positive-sum ≤ (suc fuel) zero zero s () enough
gcd-fuel-positive-sum ≤ (suc fuel) zero (suc n) s sumEq enough = n , gcd-fuel-zero-left-suc fuel n
gcd-fuel-positive-sum ≤ (suc fuel) (suc m) zero s sumEq enough = m , gcd-fuel-zero-right-suc fuel m
gcd-fuel-positive-sum ≤ (suc fuel) (suc m) (suc n) s sumEq (s ≤ s enough') with inspect (m ≤ℕ n)
... | reveal true bEq =
  let
    p : m ≤ n
    p = ≤ℕ-true → ≤ bEq

  suc n ≤ sum : suc n ≤ (m +ℕ suc n)
  suc n ≤ sum = subst (λ t → suc n ≤ t) (+ℕ-comm (suc n) m) (≤-self+ℕ (suc n) m)

  subEnough : ((suc m) +ℕ (n - m)) ≤ fuel
  subEnough = subst (λ t → t ≤ fuel) (sym (euclid-sum-true p)) (≤-trans suc n ≤ sum enough')

  subSum : (suc m) +ℕ (n - m) ≡ suc n
  subSum = euclid-sum-true p

  stepEq0 : gcd-fuel-step fuel (suc m) (suc n) (m ≤ℕ n) ≡ gcd-fuel fuel (suc m) (n - m)
  stepEq0 = cong (gcd-fuel-step fuel (suc m) (suc n)) bEq

  rec : Σ ℕ (λ g → gcd-fuel fuel (suc m) (n - m) ≡ suc g)
  rec = gcd-fuel-positive-sum ≤ fuel (suc m) (n - m) n subSum subEnough
  in
  fst rec , trans (gcd-fuel-suc-suc fuel m n) (trans stepEq0 (snd rec))
... | reveal false bEq =
  let
    p : n ≤ m
    p = ≤ℕ-false → ≤ bEq

  suc m ≤ sum : suc m ≤ (m +ℕ suc n)
  suc m ≤ sum = subst (λ t → suc m ≤ t) (sym (+ℕ-suc-right m n)) (≤-self+ℕ (suc m) n)

  subEnough : ((m - n) +ℕ (suc n)) ≤ fuel

```

```

subEnough = subst (λ t → t ≤ fuel) (sym (euclid-sum-false p)) (≤-trans suc m ≤ sum enough')

subSum : (m ÷ n) + ℕ (suc n) ≡ suc m
subSum = euclid-sum-false p

stepEq0 : gcd-fuel-step fuel (suc m) (suc n) (m ≤ ℕ n) ≡ gcd-fuel fuel (m ÷ n) (suc n)
stepEq0 = cong (gcd-fuel-step fuel (suc m) (suc n)) bEq

rec : Σ ℕ (λ g → gcd-fuel fuel (m ÷ n) (suc n) ≡ suc g)
rec = gcd-fuel-positive-sum ≤ fuel (m ÷ n) (suc n) m subSum subEnough
in
fst rec, trans (gcd-fuel-suc-suc fuel m n) (trans stepEq0 (snd rec))

-- § The canonical gcd of a positive denominator is itself positive.
gcd-positive-right : (m : ℕ) → (d : ℕ+) → Σ ℕ (λ g → gcd m (+toℕ d) ≡ suc g)
gcd-positive-right m d =
  let
    pack : Σ ℕ (λ g → gcd-fuel (m + ℕ +toℕ d) m (+toℕ d) ≡ suc g)
    pack =
      gcd-fuel-positive-sum ≤
        (m + ℕ +toℕ d)
        m
        (+toℕ d)
        (m + ℕ pred d)
        (+ℕ-suc-right m (pred d))
        (≤-refl (m + ℕ +toℕ d))
  in
    fst pack,
    trans
      (cong (λ t → gcd-fuel t m (+toℕ d)) (+to-+ℕ m (+toℕ d)))
      (snd pack)

```

Canonical reduction candidate and divisibility

The gcd engine and the divisibility theory are now available. For a rational $q = n/d$, the natural reduction candidate is obtained by computing $g = \text{gcd}(|n|, d)$ and dividing both visible fields by g .

Three basic facts are needed. The divisor g is positive because the denominator is positive, its $\mathbb{N}$ -view coincides with the computed gcd, and it divides both the numerator magnitude and the denominator. With these bookkeeping lemmas in place, the candidate can be defined directly from the quotient data.

```

-- § The successor-coded positive natural is definitionally mkℕ+.
sucToℕ+-mkℕ+ : (n : ℕ) → sucToℕ+ n ≡ mkℕ+ n
sucToℕ+-mkℕ+ zero = refl
sucToℕ+-mkℕ+ (suc n) = cong suc+ (sucToℕ+-mkℕ+ n)

-- § The gcd used by the canonical reduction candidate.
canonical-reduction-divisor : ℚ → ℕ+
canonical-reduction-divisor q = mkℕ+ (gcd (magℤ (num q)) (+toℕ (den q)) ÷ 1)

```

```

-- § Phase 2: Divide both fields by the forced gcd

-- § Candidate reduced numerator obtained by dividing out the computed gcd.
canonical-reduction-num :  $\mathbb{Q} \rightarrow \mathbb{N}$ 
canonical-reduction-num q = mag $\mathbb{Z}$  (num q) div canonical-reduction-divisor q

-- § Candidate reduced denominator obtained by dividing out the computed gcd.
canonical-reduction-den :  $\mathbb{Q} \rightarrow \mathbb{N}^+$ 
canonical-reduction-den q = mk $\mathbb{N}^+$  (( $^+$ to $\mathbb{N}$  (den q) div canonical-reduction-divisor q)  $\dot{-}$  1)

-- § Package the raw gcd/division reduction candidate.
record CanonicalReductionCandidate (q :  $\mathbb{Q}$ ) : Set where
  field
    cand-divisor :  $\mathbb{N}^+$ 
    cand-num :  $\mathbb{N}$ 
    cand-den :  $\mathbb{N}^+$ 

canonical-reduction-candidate : (q :  $\mathbb{Q}$ )  $\rightarrow$  CanonicalReductionCandidate q
canonical-reduction-candidate q = record
{ cand-divisor = canonical-reduction-divisor q
; cand-num = canonical-reduction-num q
; cand-den = canonical-reduction-den q
}

-- § The canonical reduction divisor is the positive gcd itself.
canonical-reduction-divisor-positive :
  (q :  $\mathbb{Q}$ )  $\rightarrow$ 
   $\Sigma \mathbb{N} (\lambda g \rightarrow \text{canonical-reduction-divisor } q \equiv \text{sucTo}\mathbb{N}^+ g)$ 
canonical-reduction-divisor-positive q =
  let
    gPack :  $\Sigma \mathbb{N} (\lambda g \rightarrow \text{gcd (mag}\mathbb{Z} \text{ (num q)) (}\mathbb{N}^+ \text{to}\mathbb{N} \text{ (den q))} \equiv \text{suc } g)$ 
    gPack = gcd-positive-right (mag $\mathbb{Z}$  (num q)) (den q)

    g :  $\mathbb{N}$ 
    g = fst gPack

    gcdEq : gcd (mag $\mathbb{Z}$  (num q)) ( $\mathbb{N}^+ \text{to}\mathbb{N}$  (den q))  $\equiv$  suc g
    gcdEq = snd gPack
  in
  g ,
  trans
    (cong ( $\lambda t \rightarrow$  mk $\mathbb{N}^+$  (t  $\dot{-}$  1)) gcdEq)
  (trans
    (cong mk $\mathbb{N}^+$  ( $\dot{-}$ one-suc g))
    (sym (sucTo $\mathbb{N}^+$ -mk $\mathbb{N}^+$  g)))

-- § The  $\mathbb{N}$ -view of the canonical reduction divisor is the computed gcd.
canonical-reduction-divisor-value :
  (q :  $\mathbb{Q}$ )  $\rightarrow$ 
   $\mathbb{N}^+ \text{to}\mathbb{N}$  (canonical-reduction-divisor q)  $\equiv$  gcd (mag $\mathbb{Z}$  (num q)) ( $\mathbb{N}^+ \text{to}\mathbb{N}$  (den q))
canonical-reduction-divisor-value q with gcd-positive-right (mag $\mathbb{Z}$  (num q)) (den q)
... | g , gcdEq =
  let
    divEq : canonical-reduction-divisor q  $\equiv$  sucTo $\mathbb{N}^+$  g

```

```

divEq =
  trans
    (cong (λ t → mkN+ (t - 1)) gcdEq)
  (trans
    (cong mkN+ (-one-suc g))
    (sym (sucToN+-mkN+ g)))
in
trans
  (cong +toN divEq)
  (trans
    (+toN-sucToN+ g)
    (sym gcdEq))

-- § The canonical reduction divisor divides the visible numerator magnitude.
canonical-reduction-divides-num :
  (q : ℚ) →
  Divides+ (canonical-reduction-divisor q) (magℤ (num q))
canonical-reduction-divides-num q with gcd-positive-right (magℤ (num q)) (den q)
... | g , gcdEq =
  let
    divEq : canonical-reduction-divisor q ≡ sucToN+ g
    divEq =
      trans
        (cong (λ t → mkN+ (t - 1)) gcdEq)
      (trans
        (cong mkN+ (-one-suc g))
        (sym (sucToN+-mkN+ g)))

    gcdEq+ℕ : gcd-fuel (magℤ (num q) +ℕ +toN (den q)) (magℤ (num q)) (+toN (den q)) ≡ suc g
    gcdEq+ℕ =
      trans
        (sym (cong (λ t → gcd-fuel t (magℤ (num q)) (+toN (den q))) (+to-+ℕ (magℤ (num q)) (+toN (den q)))))
        gcdEq
  in
  subst
    (λ t → Divides+ t (magℤ (num q)))
    (sym divEq)
    (fst
      (gcd-fuel-positive-output-divides
        zero
        (magℤ (num q))
        (+toN (den q))
        g
        gcdEq+ℕ))

-- § The canonical reduction divisor divides the visible denominator.
canonical-reduction-divides-den :
  (q : ℚ) →
  Divides+ (canonical-reduction-divisor q) (+toN (den q))
canonical-reduction-divides-den q with gcd-positive-right (magℤ (num q)) (den q)
... | g , gcdEq =
  let
    divEq : canonical-reduction-divisor q ≡ sucToN+ g
    divEq =

```

```

trans
  (cong (λ t → mkN+ (t · 1)) gcdEq)
  (trans
    (cong mkN+ (·-one-suc g))
    (sym (sucToN+-mkN+ g)))

gcdEq+ $\mathbb{N}$  : gcd-fuel (mag $\mathbb{Z}$  (num q) + $\mathbb{N}$  +to $\mathbb{N}$  (den q)) (mag $\mathbb{Z}$  (num q)) (+to $\mathbb{N}$  (den q))  $\equiv$  suc g
gcdEq+ $\mathbb{N}$  =
  trans
    (sym (cong (λ t → gcd-fuel t (mag $\mathbb{Z}$  (num q)) (+to $\mathbb{N}$  (den q))) (+-to- $\mathbb{N}$  (mag $\mathbb{Z}$  (num q)) (+to $\mathbb{N}$  (den q)))))
    gcdEq
in
subst
  (λ t → Divides+ t (+to $\mathbb{N}$  (den q)))
  (sym divEq)
  (snd
    (gcd-fuel-positive-output-divides
      zero
      (mag $\mathbb{Z}$  (num q))
      (+to $\mathbb{N}$  (den q))
      g
      gcdEq+ $\mathbb{N}$ ))

```

Factorization, coprimality, and witness stabilization

The next step is to show that this candidate is genuinely reduced. Because the gcd divides both fields, exact division yields factorizations of the original numerator magnitude and denominator through the candidate and the divisor.

Coprimality is then obtained by showing that any common divisor of the candidate pair would induce a larger common divisor of the original pair, contrary to maximality of the gcd. If the input is already irreducible, the divisor is 1, so the candidate agrees with the original data. This yields the stabilization and idempotence results needed later.

```

-- § The candidate numerator factorization follows from exact division.
canonical-reduction-num-factor :
  (q :  $\mathbb{Q}$ ) →
  mag $\mathbb{Z}$  (num q)
   $\equiv$  canonical-reduction-num q * $\mathbb{N}$  +to $\mathbb{N}$  (canonical-reduction-divisor q)
canonical-reduction-num-factor q =
  let
    dvd : Divides+ (canonical-reduction-divisor q) (mag $\mathbb{Z}$  (num q))
    dvd = canonical-reduction-divides-num q
  in
  trans
    (Divides+.factor+ dvd)
    (cong (λ t → t * $\mathbb{N}$  +to $\mathbb{N}$  (canonical-reduction-divisor q)) (sym (divides+→div dvd)))

-- § The candidate denominator factorization follows from exact division.
canonical-reduction-den-factor :

```

```

(q : ℚ) →
+toℕ (den q)
≡ +toℕ (canonical-reduction-den q) *ℕ +toℕ (canonical-reduction-divisor q)
canonical-reduction-den-factor q =
let
  dvd : Divides+ (canonical-reduction-divisor q) (+toℕ (den q))
  dvd = canonical-reduction-divides-den q

  qPos : Σ ℕ (λ k → quotient+ dvd ≡ suc k)
  qPos = divides+-positive-quotient dvd

  k : ℕ
  k = fst qPos

  qEq : quotient+ dvd ≡ suc k
  qEq = snd qPos

  denQuotEq : +toℕ (canonical-reduction-den q) ≡ quotient+ dvd
  denQuotEq =
    let
      denEq : canonical-reduction-den q ≡ sucToℕ+ k
      denEq =
        trans
          (cong mkℕ+ (cong (λ t → t - 1) (trans (divides+ → div dvd) qEq)))
          (trans
            (cong mkℕ+ (-one-suc k))
            (sym (sucToℕ+-mkℕ+ k)))
    in
      trans
        (cong +toℕ denEq)
        (trans
          (+toℕ-sucToℕ+ k)
          (sym qEq))
    in
      trans
        (Divides+.factor+ dvd)
        (cong (λ t → t *ℕ +toℕ (canonical-reduction-divisor q)) (sym denQuotEq))

-- § Any common positive divisor of the canonical reduced pair is forced to be one.
canonical-reduction-common-divisor-one :
(q : ℚ) →
{e : ℕ+} →
Divides+ e (canonical-reduction-num q) →
Divides+ e (+toℕ (canonical-reduction-den q)) →
e ≡ one+
canonical-reduction-common-divisor-one q {e} dnum dden =
let
  liftedNum : Divides+ (e *+ canonical-reduction-divisor q) (magℤ (num q))
  liftedNum =
    subst
      (Divides+ (e *+ canonical-reduction-divisor q))
      (sym (canonical-reduction-num-factor q))
      (divides+-mul-right dnum (canonical-reduction-divisor q))

```

```

liftedDen : Divides+ (e+ canonical-reduction-divisor q) (+toN (den q))
liftedDen =
  subst
    (Divides+ (e+ canonical-reduction-divisor q))
    (sym (canonical-reduction-den-factor q))
    (divides+-mul-right dden (canonical-reduction-divisor q))

gcdDiv : Divides+ (e+ canonical-reduction-divisor q) (gcd (magZ (num q)) (+toN (den q)))
gcdDiv = common-divisor-divides-gcd (magZ (num q)) (+toN (den q)) liftedNum liftedDen

divisorDiv : Divides+ (e+ canonical-reduction-divisor q) (+toN (canonical-reduction-divisor q))
divisorDiv =
  subst
    (Divides+ (e+ canonical-reduction-divisor q))
    (sym (canonical-reduction-divisor-value q))
    gcdDiv
in
divides+-product-factor-left-one e (canonical-reduction-divisor q) divisorDiv

-- § Phase 2: Eliminate every leftover common factor

-- § The canonical reduced numerator and denominator are coprime.
canonical-reduction-candidate-coprime :
  (q : Q) →
  gcd (canonical-reduction-num q) (+toN (canonical-reduction-den q)) ≡ 1
canonical-reduction-candidate-coprime q with gcd-positive-right (canonical-reduction-num q) (canonical-reduction-den q)
... | g , gcdEq =
  let
    gcdEq+N : gcd-fuel (canonical-reduction-num q +N +toN (canonical-reduction-den q)) (canonical-reduction-num q) (+toN (canonical-reduction-den q))
    gcdEq+N =
      trans
        (sym (cong (λ t → gcd-fuel t (canonical-reduction-num q) (+toN (canonical-reduction-den q))) (+-to-+N (canonical-reduction-num q) (+toN (canonical-reduction-den q)))))
        gcdEq
    divisorPair : Divides+ (sucToN+ g) (canonical-reduction-num q) × Divides+ (sucToN+ g) (+toN (canonical-reduction-den q))
    divisorPair =
      gcd-fuel-positive-output-divides
        zero
        (canonical-reduction-num q)
        (+toN (canonical-reduction-den q))
        g
        gcdEq+N
    oneEq : sucToN+ g ≡ one+
    oneEq = canonical-reduction-common-divisor-one q (fst divisorPair) (snd divisorPair)
  in
  trans
    gcdEq
    (trans
      (sym (+toN-sucToN+ g))
      (cong +toN oneEq))

-- § A reduced witness fixes the visible quotient fields of q.
reduced-witness-fields :

```

```

{q : ℚ} →
(r : ReducedRationalWitness q) →
num q ≡ fromℕℤ (ReducedRationalWitness.red-num r)
× den q ≡ ReducedRationalWitness.red-den r
reduced-witness-fields {n / d} r =
rational-fields-unique (n / d)
n
(fromℕℤ (ReducedRationalWitness.red-num r))
d
(ReducedRationalWitness.red-den r)
refl
(ReducedRationalWitness.red-value r)

```

The preceding lemmas identify the canonical divisor with 1 in the reduced case and relate the fields of a reduced witness to the original quotient data. These bookkeeping results prepare the irreducibility and stabilization statements that follow.

A reduced witness can now be turned into an irreducibility certificate for the represented rational.

```

-- § Every reduced witness certifies irreducibility of the represented quotient.
reduced-witness-irred : {q : ℚ} → (r : ReducedRationalWitness q) → IrreducibleQuotient q
reduced-witness-irred {q} r =
let
  numEq = fst (reduced-witness-fields r)
  denEq = snd (reduced-witness-fields r)
  magEq : magℤ (num q) ≡ ReducedRationalWitness.red-num r
  magEq = trans (cong magℤ numEq) (magℤ-fromℕℤ (ReducedRationalWitness.red-num r))

  denNatEq : +toℕ (den q) ≡ +toℕ (ReducedRationalWitness.red-den r)
  denNatEq = cong +toℕ denEq

  gcdEq : gcd (magℤ (num q)) (+toℕ (den q)) ≡ gcd (ReducedRationalWitness.red-num r) (+toℕ (ReducedRationalWitness.red-den r))
  gcdEq =
    trans
      (cong (λ t → gcd t (+toℕ (den q))) magEq)
      (cong (gcd (ReducedRationalWitness.red-num r)) denNatEq)
in
trans gcdEq (ReducedRationalWitness.red-coprime r)

-- § On a reduced witness, the canonical gcd collapses to one.
canonical-reduction-divisor-from-reduced :
{q : ℚ} →
(r : ReducedRationalWitness q) →
canonical-reduction-divisor q ≡ one+
canonical-reduction-divisor-from-reduced r =
trans
  (cong (λ t → mkℕ+ (t - 1)) (reduced-witness-irred r))
  refl

-- § Phase 3: Stabilization on already reduced input

-- § The canonical candidate numerator stabilizes on any reduced witness.
canonical-reduction-num-from-reduced :

```



```

{q : ℚ} →
(r : ReducedRationalWitness q) →
canonical-reduction-num q ≡ ReducedRationalWitness.red-num r
canonical-reduction-num-from-reduced {q} r =
let
  numEq = fst (reduced-witness-fields r)
  magEq : magℤ (num q) ≡ ReducedRationalWitness.red-num r
  magEq = trans (cong magℤ numEq) (magℤ-fromℕℤ (ReducedRationalWitness.red-num r))

  divEq :
    magℤ (num q) div canonical-reduction-divisor q
    ≡ magℤ (num q) div one+
  divEq = cong (λ t → magℤ (num q) div t) (canonical-reduction-divisor-from-reduced r)
in
trans divEq (trans (div-one+ (magℤ (num q))) magEq)

-- § The canonical candidate denominator stabilizes on any reduced witness.
canonical-reduction-den-from-reduced :
{q : ℚ} →
(r : ReducedRationalWitness q) →
canonical-reduction-den q ≡ ReducedRationalWitness.red-den r
canonical-reduction-den-from-reduced {q} r =
let
  denEq = snd (reduced-witness-fields r)
  denNatEq : +toℕ (den q) ≡ +toℕ (ReducedRationalWitness.red-den r)
  denNatEq = cong +toℕ denEq

  step :
    (+toℕ (den q) div canonical-reduction-divisor q)
    ≡ +toℕ (ReducedRationalWitness.red-den r)
  step =
    trans
      (cong (λ t → +toℕ (den q) div t) (canonical-reduction-divisor-from-reduced r))
      (trans (div-one+ (+toℕ (den q))) denNatEq)
in
trans
  (cong mkℕ+ (cong (λ t → t - 1) step))
  (mkℕ+ - +toℕ-minus-one (ReducedRationalWitness.red-den r))

-- § The full canonical reduction candidate stabilizes on any reduced witness.
canon-red-candidate-from-reduced :
{q : ℚ} →
(r : ReducedRationalWitness q) →
CanonicalReductionCandidate.cand-divisor
  (canonical-reduction-candidate q) ≡ one+
× CanonicalReductionCandidate.cand-num
  (canonical-reduction-candidate q) ≡
  ReducedRationalWitness.red-num r
× CanonicalReductionCandidate.cand-den
  (canonical-reduction-candidate q) ≡
  ReducedRationalWitness.red-den r
canon-red-candidate-from-reduced r =
canonical-reduction-divisor-from-reduced r
, ( canonical-reduction-num-from-reduced r

```

```

    , canonical-reduction-den-from-reduced r
  )

-- § Nonnegative rationals expose a natural numerator presentation.
nonnegative-rational-presentation :
  (q : ℚ) →
  0ℤ ≤ ℤ num q →
  ∑ ℕ (λ n → q ≡ fromℕℤ n / den q)
nonnegative-rational-presentation (a / d) a ≥ 0 with 0 ≤ ℤ → fromℕℤ a a ≥ 0
... | n, a ≡ = n, cong (λ t → t / d) a ≡

-- § Irreducible nonnegative quotients already carry a global reduced witness.
irreducible-nonnegative-witness :
  (q : ℚ) →
  IrreducibleQuotient q →
  0ℤ ≤ ℤ num q →
  ReducedRationalWitness q
irreducible-nonnegative-witness (a / d) irred a ≥ 0 with 0 ≤ ℤ → fromℕℤ a a ≥ 0
... | n, a ≡ = record
{ red-num = n
; red-den = d
; red-value = cong (λ t → t / d) a ≡
; red-coprime =
  trans
    (cong (λ t → gcd t (+toℕ d)) (sym (trans (cong magℤ a ≡) (magℤ-fromℕℤ n))))
  irred
}

-- § The canonical candidate becomes a concrete reduced witness on irreducible nonnegative input.
canonical-reduction-on-irreducible-nonnegative :
  (q : ℚ) →
  (irred : IrreducibleQuotient q) →
  (q ≥ 0 : 0ℤ ≤ ℤ num q) →
  CanonicalReductionCandidate.cand-divisor (canonical-reduction-candidate q) ≡ one+
  × CanonicalReductionCandidate.cand-num (canonical-reduction-candidate q)
    ≡ ReducedRationalWitness.red-num (irreducible-nonnegative-witness q irred q ≥ 0)
  × CanonicalReductionCandidate.cand-den (canonical-reduction-candidate q)
    ≡ ReducedRationalWitness.red-den (irreducible-nonnegative-witness q irred q ≥ 0)
canonical-reduction-on-irreducible-nonnegative q irred q ≥ 0 =
  canon-red-candidate-from-reduced
    (irreducible-nonnegative-witness q irred q ≥ 0)

```

Normal forms and sign handling

Up to this point the reduction procedure works with numerator magnitudes, so it first produces normal forms for the nonnegative sector. The record `ReducedRationalForm` captures this quotient-level notion: a natural numerator, a positive denominator, equality up to $\simeq_{\mathbb{Q}}$, and coprimality.

To treat arbitrary rationals, one extends this to `SignedReducedRationalForm`, allowing an integer numerator. The bridge is provided by negation: it preserves rational equivalence and leaves the magnitude of an embedded natural unchanged. A

negative rational is therefore reduced by reducing its positive counterpart and then transporting the result back through negation.

```

-- § Positive denominators are exactly natural embeddings.
+toZ-fromNZ : (d : ℕ+) → +toZ d ≡ fromNZ (+toN d)
+toZ-fromNZ (mkN+ p) = refl

-- § Quotient-level reduced normal form for an arbitrary rational class.
record ReducedRationalForm (q : ℚ) : Set where
  field
    form-num : ℕ
    form-den : ℕ+
    form-value : q ≃ ℚ (fromNZ form-num / form-den)
    form-coprime : gcd form-num (+toN form-den) ≡ 1

-- § Signed quotient-level reduced normal form for a genuinely arbitrary rational class.
record SignedReducedRationalForm (q : ℚ) : Set where
  field
    signed-num : ℤ
    signed-den : ℕ+
    signed-value : q ≃ ℚ (signed-num / signed-den)
    signed-coprime : gcd (magZ signed-num) (+toN signed-den) ≡ 1

-- § Every strict reduced witness yields a quotient-level reduced form.
reduced-witness→form : {q : ℚ} → ReducedRationalWitness q → ReducedRationalForm q
reduced-witness→form {q} r = record
{ form-num = ReducedRationalWitness.red-num r
; form-den = ReducedRationalWitness.red-den r
; form-value =
  subst
    (λ t → t ≃ ℚ (fromNZ (ReducedRationalWitness.red-num r) / ReducedRationalWitness.red-den r))
    (sym (ReducedRationalWitness.red-value r))
    (≃ℚ-refl (fromNZ (ReducedRationalWitness.red-num r) / ReducedRationalWitness.red-den r))
; form-coprime = ReducedRationalWitness.red-coprime r
}

-- § Negation preserves rational equivalence.
-ℚ-resp-≃ : {p q : ℚ} → p ≃ ℚ q → (-ℚ p) ≃ ℚ (-ℚ q)
-ℚ-resp-≃ {a / b} {c / d} eq =
  trans
    (*ℤ-neg-left a (+toZ d))
    (trans
      (cong negZ eq)
      (sym (*ℤ-neg-left c (+toZ b))))

-- § Magnitude ignores the transported sign on embedded naturals.
magZ-neg-fromNZ : (n : ℕ) → magZ (negZ (fromNZ n)) ≡ n
magZ-neg-fromNZ zero = refl
magZ-neg-fromNZ (suc n) = refl

-- § Any nonnegative quotient-level reduced form is already a signed one.
reduced-form→signed : {q : ℚ} → ReducedRationalForm q → SignedReducedRationalForm q
reduced-form→signed {q} r = record
{ signed-num = fromNZ (ReducedRationalForm.form-num r)

```

```

; signed-den = ReducedRationalForm.form-den r
; signed-value = ReducedRationalForm.form-value r
; signed-coprime =
  trans
    (cong
      (λ t → gcd t (+toN (ReducedRationalForm.form-den r)))
      (magZ-fromNZ (ReducedRationalForm.form-num r)))
    (ReducedRationalForm.form-coprime r)
}

```

Global reduction theorems

All components are now available for the global reduction theorem. Nonnegative rationals use the factorization and coprimality results already established; negative rationals are handled by reduction of the corresponding positive class and transport through negation.

The final theorem is therefore an assembly statement. The three sign cases are combined into a single construction showing that every rational class admits a canonical signed reduced form.

```

-- $ Explicit factorization data needed to make the canonical candidate sound globally.
record CanonicalReductionFactorization (q : ℚ) : Set where
  field
    num-nonnegative : 0ℤ ≤ ℤ num q
    num-factor :
      magZ (num q)
      ≡ canonical-reduction-num q * ℤ+toN (canonical-reduction-divisor q)
    den-factor :
      +toN (den q)
      ≡ +toN (canonical-reduction-den q) * ℤ+toN (canonical-reduction-divisor q)
    candidate-coprime :
      gcd (canonical-reduction-num q) (+toN (canonical-reduction-den q)) ≡ 1

-- $ With gcd-factorization laws established, the canonical candidate is the quotient-level reduced form.
canonical-reduction-form-from-factorization :
  (q : ℚ) →
    CanonicalReductionFactorization q →
    ReducedRationalForm q
canonical-reduction-form-from-factorization (a / dq) f = record
{ form-num = canonical-reduction-num (a / dq)
; form-den = canonical-reduction-den (a / dq)
; form-value = valueProof
; form-coprime = CanonicalReductionFactorization.candidate-coprime f
}
where
  divr : ℤ+
  divr = canonical-reduction-divisor (a / dq)

  cnum : ℤ
  cnum = canonical-reduction-num (a / dq)

```

```

cden :  $\mathbb{N}^+$ 
cden = canonical-reduction-den (a / dq)

aNatPack :  $\Sigma \mathbb{N} (\lambda n \rightarrow a \equiv \text{from}\mathbb{N}\mathbb{Z} n)$ 
aNatPack =  $0 \leq \mathbb{Z} \rightarrow \text{from}\mathbb{N}\mathbb{Z} a$  (CanonicalReductionFactorization.num-nonnegative f)

aNat :  $\mathbb{N}$ 
aNat = fst aNatPack

a $\equiv$  :  $a \equiv \text{from}\mathbb{N}\mathbb{Z} \text{ aNat}$ 
a $\equiv$  = snd aNatPack

numFactorEq : aNat  $\equiv$  cnum *  $\mathbb{N}^+$  to  $\mathbb{Z}$  divr
numFactorEq =
  trans
    (sym (mag $\mathbb{Z}$ -from $\mathbb{N}\mathbb{Z}$  aNat))
    (trans
      (cong mag $\mathbb{Z}$  (sym a $\equiv$ ))
      (CanonicalReductionFactorization.num-factor f))

lhsNum :  $a * \mathbb{Z}^+$  to  $\mathbb{Z}$  cden  $\equiv$  (from $\mathbb{N}\mathbb{Z}$  cnum *  $\mathbb{Z}^+$  to  $\mathbb{Z}$  divr) *  $\mathbb{Z}^+$  to  $\mathbb{Z}$  cden
lhsNum =
  trans
    (cong ( $\lambda t \rightarrow t * \mathbb{Z}^+$  to  $\mathbb{Z}$  cden) a $\equiv$ )
    (trans
      (cong ( $\lambda t \rightarrow \text{from}\mathbb{N}\mathbb{Z} t * \mathbb{Z}^+$  to  $\mathbb{Z}$  cden) numFactorEq)
      (cong ( $\lambda t \rightarrow t * \mathbb{Z}^+$  to  $\mathbb{Z}$  cden) (sym (from $\mathbb{N}\mathbb{Z}$ -mul- $^+$  cnum divr))))

lhsRegroup :  $a * \mathbb{Z}^+$  to  $\mathbb{Z}$  cden  $\equiv$  from $\mathbb{N}\mathbb{Z}$  cnum *  $\mathbb{Z}$  (( $^+$  to  $\mathbb{Z}$  divr) *  $\mathbb{Z}$  ( $^+$  to  $\mathbb{Z}$  cden))
lhsRegroup =
  trans lhsNum (* $\mathbb{Z}$ -assoc (from $\mathbb{N}\mathbb{Z}$  cnum) ( $^+$  to  $\mathbb{Z}$  divr) ( $^+$  to  $\mathbb{Z}$  cden))

rhsDen :  $^+$  to  $\mathbb{Z}$  dq  $\equiv$  ( $^+$  to  $\mathbb{Z}$  cden) *  $\mathbb{Z}$  ( $^+$  to  $\mathbb{Z}$  divr)
rhsDen =
  trans
    ( $^+$  to  $\mathbb{Z}$ -from $\mathbb{N}\mathbb{Z}$  dq)
    (trans
      (cong from $\mathbb{N}\mathbb{Z}$  (CanonicalReductionFactorization.den-factor f))
      (trans
        (sym (from $\mathbb{N}\mathbb{Z}$ -mul- $^+$  ( $^+$  to  $\mathbb{N}$  cden) divr))
        (cong ( $\lambda t \rightarrow t * \mathbb{Z}^+$  to  $\mathbb{Z}$  divr) ( $^+$  to  $\mathbb{Z}$ -from $\mathbb{N}\mathbb{Z}$  cden))))

rhsRegroup : from $\mathbb{N}\mathbb{Z}$  cnum *  $\mathbb{Z}^+$  to  $\mathbb{Z}$  dq  $\equiv$  from $\mathbb{N}\mathbb{Z}$  cnum *  $\mathbb{Z}$  (( $^+$  to  $\mathbb{Z}$  cden) *  $\mathbb{Z}$  ( $^+$  to  $\mathbb{Z}$  divr))
rhsRegroup = cong ( $\lambda t \rightarrow \text{from}\mathbb{N}\mathbb{Z}$  cnum *  $\mathbb{Z} t$ ) rhsDen

middleComm : from $\mathbb{N}\mathbb{Z}$  cnum *  $\mathbb{Z}$  (( $^+$  to  $\mathbb{Z}$  divr) *  $\mathbb{Z}$  ( $^+$  to  $\mathbb{Z}$  cden))  $\equiv$  from $\mathbb{N}\mathbb{Z}$  cnum *  $\mathbb{Z}$  (( $^+$  to  $\mathbb{Z}$  cden) *  $\mathbb{Z}$  ( $^+$  to  $\mathbb{Z}$  divr))
middleComm = cong ( $\lambda t \rightarrow \text{from}\mathbb{N}\mathbb{Z}$  cnum *  $\mathbb{Z} t$ ) (* $\mathbb{Z}$ -comm ( $^+$  to  $\mathbb{Z}$  divr) ( $^+$  to  $\mathbb{Z}$  cden))

valueProof : (a / dq)  $\simeq \mathbb{Q}$  (from $\mathbb{N}\mathbb{Z}$  cnum / cden)
valueProof = trans lhsRegroup (trans middleComm (sym rhsRegroup))

-- § The canonical reduction candidate carries full factorization data for nonnegative rationals.
canonical-reduction-factorization-nonnegative :
  (q :  $\mathbb{Q}$ )  $\rightarrow$ 

```

```

0ℤ ≤ℤ num q →
CanonicalReductionFactorization q
canonical-reduction-factorization-nonnegative q q ≥ 0 = record
{ num-nonnegative = q ≥ 0
; num-factor = canonical-reduction-num-factor q
; den-factor = canonical-reduction-den-factor q
; candidate-coprime = canonical-reduction-candidate-coprime q
}

-- § Every nonnegative rational class has a canonical quotient-level reduced form.
canonical-reduction-form-nonnegative :
(q : ℚ) →
0ℤ ≤ℤ num q →
ReducedRationalForm q
canonical-reduction-form-nonnegative q q ≥ 0 =
canonical-reduction-form-from-factorization q (canonical-reduction-factorization-nonnegative q q ≥ 0)

```

The nonnegative case is fully resolved: every nonnegative rational has a unique factorization through the canonical candidate, and that candidate yields a quotient-level reduced form. What remains is the sign barrier—negative rationals must be handled by reducing their magnitude and then transporting the result through negation.

```

-- § Canonical reduction yields a signed quotient-level reduced form for every rational.
canonical-reduction-form :
(q : ℚ) →
SignedReducedRationalForm q
canonical-reduction-form (0ℤ / d) =
reduced-form→signed (canonical-reduction-form-nonnegative (0ℤ / d) tt)
canonical-reduction-form ((+suc n) / d) =
reduced-form→signed (canonical-reduction-form-nonnegative ((+suc n) / d) tt)
canonical-reduction-form ((-suc n) / d) = record
{ signed-num = negℤ (fromℕℤ (ReducedRationalForm.form-num negForm))
; signed-den = ReducedRationalForm.form-den negForm
; signed-value = -ℚ-resp-≈ (ReducedRationalForm.form-value negForm)
; signed-coprime =
trans
(cong
(λ t → gcd t (+toℕ (ReducedRationalForm.form-den negForm)))
(magℤ-neg-fromℕℤ (ReducedRationalForm.form-num negForm)))
(ReducedRationalForm.form-coprime negForm)
}
where
negForm : ReducedRationalForm ((+suc n) / d)
negForm = canonical-reduction-form-nonnegative ((+suc n) / d) tt

-- § In the irreducible nonnegative sector, the conditional factorization data is already forced.
canonical-reduction-factorization-on-irreducible-nonnegative :
(q : ℚ) →
(irred : IrreducibleQuotient q) →
(q ≥ 0 : 0ℤ ≤ℤ num q) →
CanonicalReductionFactorization q
canonical-reduction-factorization-on-irreducible-nonnegative q irred q ≥ 0 = record
{ num-nonnegative = q ≥ 0

```

```

; num-factor = numFactor
; den-factor = denFactor
; candidate-coprime = candidateCoprime
}
where
rw : ReducedRationalWitness q
rw = irreducible-nonnegative-witness q irred q ≥ 0

numWitnessEq : magℤ (num q) ≡ ReducedRationalWitness.red-num rw
numWitnessEq =
  trans
    (cong magℤ (fst (reduced-witness-fields rw)))
    (magℤ-fromℕℤ (ReducedRationalWitness.red-num rw))

denWitnessEq : +toℕ (den q) ≡ +toℕ (ReducedRationalWitness.red-den rw)
denWitnessEq = cong +toℕ (snd (reduced-witness-fields rw))

divNatEq : +toℕ (canonical-reduction-divisor q) ≡ 1
divNatEq = cong +toℕ (canonical-reduction-divisor-from-reduced rw)

numCandEq : canonical-reduction-num q ≡ ReducedRationalWitness.red-num rw
numCandEq = canonical-reduction-num-from-reduced rw

denCandNatEq : +toℕ (canonical-reduction-den q) ≡ +toℕ (ReducedRationalWitness.red-den rw)
denCandNatEq = cong +toℕ (canonical-reduction-den-from-reduced rw)

numFactor : magℤ (num q) ≡ canonical-reduction-num q *ℕ +toℕ (canonical-reduction-divisor q)
numFactor =
  trans
    (trans numWitnessEq (sym numCandEq))
    (sym
      (trans
        (cong (λ t → canonical-reduction-num q *ℕ t) divNatEq)
        (*ℕ-one-right (canonical-reduction-num q))))

denFactor : +toℕ (den q) ≡ +toℕ (canonical-reduction-den q) *ℕ +toℕ (canonical-reduction-divisor q)
denFactor =
  trans
    (trans denWitnessEq (sym denCandNatEq))
    (sym
      (trans
        (cong (λ t → +toℕ (canonical-reduction-den q) *ℕ t) divNatEq)
        (*ℕ-one-right (+toℕ (canonical-reduction-den q))))))

candidateCoprime : gcd (canonical-reduction-num q) (+toℕ (canonical-reduction-den q)) ≡ 1
candidateCoprime =
  trans
    (cong (λ t → gcd t (+toℕ (canonical-reduction-den q))) numCandEq)
    (trans
      (cong (gcd (ReducedRationalWitness.red-num rw)) denCandNatEq)
      (ReducedRationalWitness.red-coprime rw))

-- § The canonical candidate already yields a quotient-level reduced form on irreducible nonnegative input.
canonical-reduction-form-on-irreducible-nonnegative :

```

```

(q : ℚ) →
(irred : IrreducibleQuotient q) →
(q ≥ 0 : 0ℤ ≤ ℤ num q) →
ReducedRationalForm q
canonical-reduction-form-on-irreducible-nonnegative q irred q ≥ 0 =
canonical-reduction-form-from-factorization q
(canonical-reduction-factorization-on-irreducible-nonnegative q irred q ≥ 0)

```

Observable quotient layer

The class-level reduction is complete: every rational carries a signed reduced form. But the invariant chain does not touch rationals in isolation. It touches *admissible rational observables*—functions from drift states into $\mathbb{Q}$ that already survive the admissibility filter. If quotient data were not lifted pointwise along these observables, the bridge chapter would be forced to re-derive the presentation at every call site, and the uniqueness guarantee would dissolve into ad hoc casework.

The lift is immediate. An admissible rational observable already assigns a rational value at each drift state. Reading off the constructor fields at each state gives the canonical pointwise presentation; the uniqueness of those fields then propagates pointwise. Reduced witnesses are similarly lifted: a reduced observable carries a reduced witness at every state, and whenever two such witnesses exist, they agree pointwise because the class-level uniqueness theorem already forces agreement field by field.

The same information must now be available pointwise for admissible observables. Since such an observable assigns a rational value to each drift state, one simply reads off the quotient fields and reduced witnesses state by state. Their uniqueness is inherited immediately from the corresponding class-level theorems. This closes the observable version of the quotient layer and makes the reduction machinery usable throughout the rest of the development.

```

-- § Pointwise quotient presentation for an admissible rational observable.
record RationalObservablePresentation (a : AdmissibleRationalObservable) : Setω where
  field
    presented-num : DriftState → ℤ
    presented-den : DriftState → ℕ+
    presents-observable :
      (s : DriftState) →
        lowerℚ (observe a s) ≡ presented-num s / presented-den s

-- § Canonical pointwise presentation reads off the quotient fields directly.
canonical-rational-observable-presentation :
  (a : AdmissibleRationalObservable) → RationalObservablePresentation a
canonical-rational-observable-presentation a = record
  { presented-num = λ s → num (lowerℚ (observe a s))
  ; presented-den = λ s → den (lowerℚ (observe a s))
  ; presents-observable = λ s → refl

```



```

}

-- § Pointwise quotient fields are unique for every admissible rational observable.
observable-quotient-fields-unique :
(a : AdmissibleRationalObservable) →
(P1 P2 : RationalObservablePresentation a) →
(s : DriftState) →
RationalObservablePresentation.presented-num P1 s ≡ RationalObservablePresentation.presented-num P2 s
× RationalObservablePresentation.presented-den P1 s ≡ RationalObservablePresentation.presented-den P2 s
observable-quotient-fields-unique a P1 P2 s =
rational-fields-unique (lower $\mathbb{Q}$  (observe a s))
  (RationalObservablePresentation.presented-num P1 s)
  (RationalObservablePresentation.presented-num P2 s)
  (RationalObservablePresentation.presented-den P1 s)
  (RationalObservablePresentation.presented-den P2 s)
  (RationalObservablePresentation.presents-observable P1 s)
  (RationalObservablePresentation.presents-observable P2 s)

-- § Pointwise reduced witnesses on an admissible rational observable.
record ReducedRationalObservable (a : AdmissibleRationalObservable) : Set $\omega$  where
field
  reduced-at : (s : DriftState) → ReducedRationalWitness (lower $\mathbb{Q}$  (observe a s))

-- § Pointwise reduced witnesses are unique whenever they exist.
reduced-rational-observable-unique :
(a : AdmissibleRationalObservable) →
(R1 R2 : ReducedRationalObservable a) →
(s : DriftState) →
ReducedRationalWitness.red-num (ReducedRationalObservable.reduced-at R1 s)
  ≡ ReducedRationalWitness.red-num (ReducedRationalObservable.reduced-at R2 s)
× ReducedRationalWitness.red-den (ReducedRationalObservable.reduced-at R1 s)
  ≡ ReducedRationalWitness.red-den (ReducedRationalObservable.reduced-at R2 s)
reduced-rational-observable-unique a R1 R2 s =
reduced-rational-witness-unique
  (ReducedRationalObservable.reduced-at R1 s)
  (ReducedRationalObservable.reduced-at R2 s)

```


Chapter 56

K_4 Forced Constants Layer

Once the vertex number is fixed at $V = 4$, the basic combinatorial constants of K_4 are determined. This chapter computes edge counts, face counts, spectral constants, and cycle data directly from that value, with the resulting identities verified inside the formal system.

It is useful to separate internal computation from external interpretation. The results proved here concern only the formal development: given the already established structure of K_4 , the stated invariants reduce to the stated numbers. Any broader interpretation lies outside the present argument.

Simplex invariants

This section evaluates the basic tetrahedral invariants determined by $V = 4$. The standard simplex formulas give the closed values $E = 6$, $F = 4$, $\deg = 3$, and $\chi = 2$, and the same computations identify them with the abstract simplex quantities introduced earlier.

```
-- § Vertex count of  $K_4$  (= simplex-vertices from Tier 5)
vertexCountK4 :  $\mathbb{N}$ 
vertexCountK4 = simplex-vertices

-- § Edge count:  $E = V(V-1)/2$ 
edgeCountK4 :  $\mathbb{N}$ 
edgeCountK4 = (vertexCountK4 * (vertexCountK4 - 1)) div  $\mathbb{N} 2$ 

-- § Face count:  $F = V(V-1)(V-2)/6$ 
faceCountK4 :  $\mathbb{N}$ 
faceCountK4 = (vertexCountK4 * (vertexCountK4 - 1) * (vertexCountK4 - 2)) div  $\mathbb{N} 6$ 

-- § Vertex degree:  $\deg = V - 1$ 
degree-K4 :  $\mathbb{N}$ 
degree-K4 = vertexCountK4 - 1
```

```

-- § Euler characteristic:  $\chi = V + F - E$ 
eulerChar-computed :  $\mathbb{N}$ 
eulerChar-computed = (vertexCountK4 + faceCountK4) - edgeCountK4

-- § Law 16B-0:  $E = 6$ 
law16B-0-edges : edgeCountK4  $\equiv 6$ 
law16B-0-edges = refl

-- § Law 16B-1:  $F = 4$ 
law16B-1-faces : faceCountK4  $\equiv 4$ 
law16B-1-faces = refl

-- § Law 16B-2:  $\deg = 3$ 
law16B-2-degree : degree-K4  $\equiv 3$ 
law16B-2-degree = refl

-- § Law 16B-3:  $\chi = 2$ 
law16B-3-euler : eulerChar-computed  $\equiv 2$ 
law16B-3-euler = refl

-- § Law 16B-4: edge count matches simplex definition
law16B-4-edges-match : edgeCountK4  $\equiv$  simplex-edges
law16B-4-edges-match = refl

-- § Law 16B-5: degree matches simplex definition
law16B-5-degree-match : degree-K4  $\equiv$  simplex-degree
law16B-5-degree-match = refl

-- § Law 16B-6: Euler characteristic matches simplex definition
law16B-6-euler-match : eulerChar-computed  $\equiv$  simplex-chi
law16B-6-euler-match = refl

```

Clifford and spectral constants

The next constants are derived from the same tetrahedral data and introduce no additional parameters. Clifford dimension, hierarchy exponent, and the discrete evaluation data are all fixed polynomial or exponential expressions in the previously computed invariants.

```

-- § Clifford algebra dimension:  $2^V$ 
clifford-dimension :  $\mathbb{N}$ 
clifford-dimension = 2 ^ vertexCountK4

-- § Law 16B-7:  $\text{Cl}(\mathbb{R}^4)$  has dimension 16
law16B-7-clifford : clifford-dimension  $\equiv 16$ 
law16B-7-clifford = refl

-- § Clifford mode count (alias)
clifford-modes :  $\mathbb{N}$ 
clifford-modes = clifford-dimension

```

```

-- § Fermat-like combinatorial strata
F2 : ℕ
F2 = suc clifford-modes

F3 : ℕ
F3 = suc (clifford-modes * clifford-modes)

-- § Discrete  $\kappa$ : spectral width  $\times 2$ 
 $\kappa$ -discrete : ℕ
 $\kappa$ -discrete = 2 * (degree-K4 + 1)

-- § Law 16B-8:  $\kappa = 8$ 
law16B-8-kappa :  $\kappa$ -discrete  $\equiv$  8
law16B-8-kappa = refl

```

The Bailey–Borwein–Plouffe identity for π can be read here only at the level of its integer coefficient pattern. In that pattern the base, step, offsets, and numerators are all recovered from K_4 invariants: $\chi^V = 16$, $\kappa = 8$, $V - d = 1$, $V = 4$, $d + \chi = 5$, and $E = 6$. The record below does not analyze convergence; it identifies the BBP coefficient ledger as native to the already fixed tetrahedral data.

```

-- § BBP coefficient ledger as  $K_4$  data.

bbp-base : ℕ
bbp-base = eulerChar-computed ^ vertexCountK4 --  $\chi^V = 2^4 = 16$ 

bbp-step : ℕ
bbp-step =  $\kappa$ -discrete --  $\kappa = 8$ 

bbp- $\delta$  : ℕ
bbp- $\delta$  = vertexCountK4 - degree-K4 --  $V - d = 1$ 

bbp-offset1 : ℕ
bbp-offset1 = bbp- $\delta$  --  $V - d = 1$ 

bbp-offset2 : ℕ
bbp-offset2 = vertexCountK4 --  $V = 4$ 

bbp-offset3 : ℕ
bbp-offset3 = degree-K4 + eulerChar-computed --  $d + \chi = 5$ 

bbp-offset4 : ℕ
bbp-offset4 = edgeCountK4 --  $E = 6$ 

bbp-numer1 : ℕ
bbp-numer1 = vertexCountK4 --  $V = 4$ 

bbp-numer2 : ℕ
bbp-numer2 = eulerChar-computed --  $\chi = 2$ 

bbp-numer3 : ℕ

```

```

bbp-numer3 = bbp- $\delta$  --  $\delta = 1$ 

bbp-numer4 :  $\mathbb{N}$ 
bbp-numer4 = bbp- $\delta$  --  $\delta = 1$ 

record BBP-K4-Identification : Set where
  field
    base-is- $\chi$ V : bbp-base  $\equiv$  16
    step-is- $\kappa$  : bbp-step  $\equiv$  8
    offset1-is- $\delta$  : bbp-offset1  $\equiv$  1
    offset2-is-V : bbp-offset2  $\equiv$  4
    offset3-is-d $\chi$  : bbp-offset3  $\equiv$  5
    offset4-is-E : bbp-offset4  $\equiv$  6
    numer1-is-V : bbp-numer1  $\equiv$  4
    numer2-is- $\chi$  : bbp-numer2  $\equiv$  2
    numer3-is- $\delta$  : bbp-numer3  $\equiv$  1
    numer4-is- $\delta$  : bbp-numer4  $\equiv$  1

-- § Law 16B-8a: BBP identification
law16B-8a-bbp-k4 : BBP-K4-Identification
law16B-8a-bbp-k4 = record
{ base-is- $\chi$ V = refl
; step-is- $\kappa$  = refl
; offset1-is- $\delta$  = refl
; offset2-is-V = refl
; offset3-is-d $\chi$  = refl
; offset4-is-E = refl
; numer1-is-V = refl
; numer2-is- $\chi$  = refl
; numer3-is- $\delta$  = refl
; numer4-is- $\delta$  = refl }

-- § Hierarchy exponent:  $V \cdot E - \chi$ 
hierarchy-exponent :  $\mathbb{N}$ 
hierarchy-exponent = vertexCountK4 * edgeCountK4 - eulerChar-computed

-- § Law 16B-9: hierarchy exponent = 22
law16B-9-hierarchy : hierarchy-exponent  $\equiv$  22
law16B-9-hierarchy = refl

-- § Simplex denominator:  $d \cdot (1 + E^2)$ 
simplex-denom-K4 :  $\mathbb{N}$ 
simplex-denom-K4 = degree-K4 * suc (edgeCountK4 * edgeCountK4)

-- § Law 16B-10: simplex denominator = 111
law16B-10-simplex-denom : simplex-denom-K4  $\equiv$  111
law16B-10-simplex-denom = refl

-- § Edge pair count:  $E^2$ 
EdgePairCount-early :  $\mathbb{N}$ 
EdgePairCount-early = edgeCountK4 * edgeCountK4

-- § Law 16B-11:  $E^2 = 36$ 
law16B-11-edge-pairs : EdgePairCount-early  $\equiv$  36

```

```

law16B-11-edge-pairs = refl

-- § Law 16B-12:  $F_2 = 17$ 
law16B-12-F2-is-17 :  $F_2 \equiv 17$ 
law16B-12-F2-is-17 = refl

-- § Law 16B-13:  $F_3 = 257$ 
law16B-13-F3-is-257 :  $F_3 \equiv 257$ 
law16B-13-F3-is-257 = refl

-- § Law 16B-14: compactification triple (5, 17, 37)
law16B-14-compactification-triple :
  (suc vertexCountK4  $\equiv$  5)  $\times$  ((suc clifford-dimension  $\equiv$  17)  $\times$  (suc EdgePairCount-early  $\equiv$  37))
law16B-14-compactification-triple = refl , (refl , refl)

```

K_4 aliases and cycle counts

The remaining definitions synchronize notation used in different parts of the manuscript and record the cycle counts needed later. No new structure is introduced here; the aliases and derived quantities are read off from the same previously computed tetrahedral data.

```

-- §  $K_4$  aliases for  $K_4$  usage
K4-V K4-E K4-F K4-deg K4-chi :  $\mathbb{N}$ 
K4-V = vertexCountK4
K4-E = edgeCountK4
K4-F = faceCountK4
K4-deg = degree-K4
K4-chi = eulerChar-computed

-- § Redundant aliases for  $K_4$  invariant fields
K4-vertices-count K4-edges-count K4-degree-count K4-triangles :  $\mathbb{N}$ 
K4-vertices-count = vertexCountK4
K4-edges-count = edgeCountK4
K4-degree-count = degree-K4
K4-triangles = faceCountK4

-- § Simplex evaluation alias for  $K_4$ 
simplex-eval-K4 :  $\mathbb{N}$ 
simplex-eval-K4 = simplex-eval

-- § Law 16B-15: simplex evaluation = 137
law16B-15-simplex-eval-bare : simplex-eval-K4  $\equiv$  137
law16B-15-simplex-eval-bare = refl

```

The least non-trivial named invariant in the K_4 ledger is $\chi = 2$. The degree is 3, the vertex and face counts are 4, the edge count is 6, and $\kappa = 8$. Consequently any ratio of the form $n/\mathcal{C}^2$ whose numerator is drawn from this invariant basis is bounded below by $\chi/\mathcal{C}^2$, once the numerator is required to be non-trivial.

-- § Law 16B-15a: χ is the minimum non-trivial K_4 invariant.

law-chi-below-deg : $\text{succ } K4\text{-chi} \leq K4\text{-deg}$

law-chi-below-deg = $s \leq s (s \leq s (s \leq s z \leq n))$ -- $3 \leq 3$

law-chi-below-V : $\text{succ } K4\text{-chi} \leq K4\text{-V}$

law-chi-below-V = $s \leq s (s \leq s (s \leq s z \leq n))$ -- $3 \leq 4$

law-chi-below-F : $\text{succ } K4\text{-chi} \leq K4\text{-F}$

law-chi-below-F = $s \leq s (s \leq s (s \leq s z \leq n))$ -- $3 \leq 4$

law-chi-below-E : $\text{succ } K4\text{-chi} \leq K4\text{-E}$

law-chi-below-E = $s \leq s (s \leq s (s \leq s z \leq n))$ -- $3 \leq 6$

law-chi-below- κ : $\text{succ } K4\text{-chi} \leq \kappa\text{-discrete}$

law-chi-below- κ = $s \leq s (s \leq s (s \leq s z \leq n))$ -- $3 \leq 8$

-- § χ is non-trivial: $\chi > 1$.

law-chi-nontrivial : $\text{succ } 1 \leq K4\text{-chi}$

law-chi-nontrivial = $s \leq s (s \leq s z \leq n)$ -- $2 \leq 2$

-- § Combined χ -minimality record.

record ChiMinimality : Set where

field

chi-nontrivial : $\text{succ } 1 \leq K4\text{-chi}$

chi-below-deg : $\text{succ } K4\text{-chi} \leq K4\text{-deg}$

chi-below-V : $\text{succ } K4\text{-chi} \leq K4\text{-V}$

chi-below-F : $\text{succ } K4\text{-chi} \leq K4\text{-F}$

chi-below-E : $\text{succ } K4\text{-chi} \leq K4\text{-E}$

chi-below- κ : $\text{succ } K4\text{-chi} \leq \kappa\text{-discrete}$

law16B-15a-chi-minimality : ChiMinimality

law16B-15a-chi-minimality = record

{ chi-nontrivial = law-chi-nontrivial

; chi-below-deg = law-chi-below-deg

; chi-below-V = law-chi-below-V

; chi-below-F = law-chi-below-F

; chi-below-E = law-chi-below-E

; chi-below- κ = law-chi-below- κ

}

-- § Triangle and square cycle counts

count-triangles : $\mathbb{N}$

count-triangles = simplex-vertices

count-squares : $\mathbb{N}$

count-squares = simplex-degree

total-nontrivial-cycles : $\mathbb{N}$

total-nontrivial-cycles = count-triangles + count-squares

-- § Law 16B-16: total non-trivial cycles = 7

law16B-16-total-cycles : total-nontrivial-cycles $\equiv 7$

law16B-16-total-cycles = refl


```

-- § Loop and propagation orders
triangle-loop-order : ℕ
triangle-loop-order = simplex-vertices  $\dot{-}$  simplex-degree --  $V - d = 4 - 3 = 1$ 

square-loop-order : ℕ
square-loop-order = simplex-vertices  $\dot{-}$  simplex-degree + (simplex-vertices  $\dot{-}$  simplex-degree) --  $1 + 1 = 2$ 

max-propagation-per-edge : ℕ
max-propagation-per-edge = simplex-vertices  $\dot{-}$  simplex-degree

-- § Law 16B-17: max propagation per edge = 1
law16B-17-max-prop : max-propagation-per-edge  $\equiv$  1
law16B-17-max-prop = refl

-- § Lattice spacing at natural scale
lattice-spacing-natural : ℕ
lattice-spacing-natural = simplex-vertices  $\dot{-}$  simplex-degree

-- § Law 16B-18: lattice spacing = 1
law16B-18-lattice-natural : lattice-spacing-natural  $\equiv$  1
law16B-18-lattice-natural = refl

-- §  $K_5$  edge count for comparison
K5-total-edges : ℕ
K5-total-edges = ((vertexCountK4 + 1) * vertexCountK4) div ℕ 2

-- § Law 16B-19:  $K_5$  has 10 edges
law16B-19-K5-edges : K5-total-edges  $\equiv$  10
law16B-19-K5-edges = refl

```

Generation index exhaustion

The non-zero spectrum of $L(K_4)$ carries multiplicity $d = 3$. Three eigenmodes demand three indices, and the indices cannot be imported from outside: every label that ranks them must already be a K_4 invariant, or the ranking smuggles in foreign content. The question is therefore not which labels to choose, but which labels survive the constraint that they live in the natural ranking interval $[1, d]$.

The atomic alphabet of K_4 is $\{V, E, F, d, \chi, \kappa\} = \{4, 6, 4, 3, 2, 8\}$. Of its six members, exactly two fall in $[1, d]$: $\chi = 2$ and $d = 3$. The remaining slot $\{1\}$ is realised by no atom, but by the unique atomic shortfall: $V \dot{-} d = F \dot{-} d = d \dot{-} \chi = 1$. Three independent atomic differences collapse onto the same value, and that value closes the interval.

Once the interval $[1, d]$ is closed by $(V \dot{-} d, \chi, d) = (1, 2, 3)$, no other strictly monotone triple of basis atoms in $[1, d]$ can occupy it—every alternative either repeats a value, leaves a gap, or exceeds d . The three eigenmode indices are therefore not chosen but forced within the basis.

This leaves a simple exhaustion argument. Among the basic invariants, only $\chi = 2$ and $d = 3$ already lie in the interval $[1, d]$. Everything larger than d is excluded at

once. The remaining value 1 is supplied by the common shortfall $V \dot{-} d = F \dot{-} d = d \dot{-} \chi$. Hence the increasing triple $(1, 2, 3)$ is obtained internally from the K_4 ledger and no alternative strictly monotone triple of the same kind remains.

```
-- § Generation indices, named by their  $K_4$  origin
gen-index-1 :  $\mathbb{N}$ 
gen-index-1 = vertexCountK4  $\dot{-}$  degree-K4 --  $V \dot{-} d = 1$ 

gen-index-2 :  $\mathbb{N}$ 
gen-index-2 = eulerChar-computed --  $\chi = 2$ 

gen-index-3 :  $\mathbb{N}$ 
gen-index-3 = degree-K4 --  $d = 3$ 

-- § Numerical values
law16B-20-gen-index-1 : gen-index-1  $\equiv$  1
law16B-20-gen-index-1 = refl

law16B-21-gen-index-2 : gen-index-2  $\equiv$  2
law16B-21-gen-index-2 = refl

law16B-22-gen-index-3 : gen-index-3  $\equiv$  3
law16B-22-gen-index-3 = refl

-- § Strict monotonicity of the index triple
law16B-23-gen-mono-12 : suc gen-index-1  $\leq$  gen-index-2
law16B-23-gen-mono-12 = s $\leq$ s (s $\leq$ s z $\leq$ n) --  $2 \leq 2$ 

law16B-24-gen-mono-23 : suc gen-index-2  $\leq$  gen-index-3
law16B-24-gen-mono-23 = s $\leq$ s (s $\leq$ s (s $\leq$ s z $\leq$ n)) --  $3 \leq 3$ 

-- § The four atoms outside  $[1, d]$  all strictly exceed  $d$ .
law16B-25-V-above-d : suc degree-K4  $\leq$  vertexCountK4
law16B-25-V-above-d = s $\leq$ s (s $\leq$ s (s $\leq$ s (s $\leq$ s z $\leq$ n))) --  $4 \leq 4$ 

law16B-26-F-above-d : suc degree-K4  $\leq$  faceCountK4
law16B-26-F-above-d = s $\leq$ s (s $\leq$ s (s $\leq$ s (s $\leq$ s z $\leq$ n))) --  $4 \leq 4$ 

law16B-27-E-above-d : suc degree-K4  $\leq$  edgeCountK4
law16B-27-E-above-d = s $\leq$ s (s $\leq$ s (s $\leq$ s (s $\leq$ s z $\leq$ n))) --  $4 \leq 6$ 

law16B-28- $\kappa$ -above-d : suc degree-K4  $\leq$   $\kappa$ -discrete
law16B-28- $\kappa$ -above-d = s $\leq$ s (s $\leq$ s (s $\leq$ s (s $\leq$ s z $\leq$ n))) --  $4 \leq 8$ 

-- § The atomic shortfall: three independent atomic differences all collapse onto 1.
law16B-29-Vd-is-1 : vertexCountK4  $\dot{-}$  degree-K4  $\equiv$  1
law16B-29-Vd-is-1 = refl

law16B-30-Fd-is-1 : faceCountK4  $\dot{-}$  degree-K4  $\equiv$  1
law16B-30-Fd-is-1 = refl

law16B-31-d $\chi$ -is-1 : degree-K4  $\dot{-}$  eulerChar-computed  $\equiv$  1
law16B-31-d $\chi$ -is-1 = refl

-- § The exhaustion record for the three generation indices.
```

```

record GenerationExhaustion : Set where
  field
    -- indices are canonical K4 witnesses
    index-1-from-K4 : gen-index-1  $\equiv$  vertexCountK4  $\dot{-}$  degree-K4
    index-2-from-K4 : gen-index-2  $\equiv$  eulerChar-computed
    index-3-from-K4 : gen-index-3  $\equiv$  degree-K4
    -- their numerical values
    val-1 : gen-index-1  $\equiv$  1
    val-2 : gen-index-2  $\equiv$  2
    val-3 : gen-index-3  $\equiv$  3
    -- strictly monotone enumeration of [1, d]
    mono-12 : suc gen-index-1  $\leq$  gen-index-2
    mono-23 : suc gen-index-2  $\leq$  gen-index-3
    -- the four atoms outside the interval
    V-above : suc degree-K4  $\leq$  vertexCountK4
    F-above : suc degree-K4  $\leq$  faceCountK4
    E-above : suc degree-K4  $\leq$  edgeCountK4
     $\kappa$ -above : suc degree-K4  $\leq$   $\kappa$ -discrete
    -- the three coincident atomic shortfalls
    Vd-is-1 : vertexCountK4  $\dot{-}$  degree-K4  $\equiv$  1
    Fd-is-1 : faceCountK4  $\dot{-}$  degree-K4  $\equiv$  1
    d $\chi$ -is-1 : degree-K4  $\dot{-}$  eulerChar-computed  $\equiv$  1

law16B-32-generation-exhaustion : GenerationExhaustion
law16B-32-generation-exhaustion = record
{ index-1-from-K4 = refl
; index-2-from-K4 = refl
; index-3-from-K4 = refl
; val-1 = law16B-20-gen-index-1
; val-2 = law16B-21-gen-index-2
; val-3 = law16B-22-gen-index-3
; mono-12 = law16B-23-gen-mono-12
; mono-23 = law16B-24-gen-mono-23
; V-above = law16B-25-V-above-d
; F-above = law16B-26-F-above-d
; E-above = law16B-27-E-above-d
;  $\kappa$ -above = law16B-28- $\kappa$ -above-d
; Vd-is-1 = law16B-29-Vd-is-1
; Fd-is-1 = law16B-30-Fd-is-1
; d $\chi$ -is-1 = law16B-31-d $\chi$ -is-1
}

```

Both shortfall identities reduce to the same value 1, so the interval is closed internally by the tetrahedral data. That is the combinatorial conclusion needed in what follows.

This completes the combinatorial ledger needed in the next chapter. The remaining task is to show that these constants determine a unique representation record.

The computed invariants. At this stage the formal development has fixed, among other values, $V = 4$, $E = 6$, the eigenvalue ratio 3:1, and the graph evaluation

$\mathcal{C} = 137$. The next chapter uses these computed quantities only as inputs to the uniqueness argument for the representation record.

Chapter 57

K_4 Representation Theory

The aim of this chapter is to show that the invariant representation of K_4 is unique. One field is anchored directly to the vertex count, and the remaining fields are linked to it by explicit equations. The result is a rigidity statement: every admissible record collapses to the same data.

Before the richer invariant record is introduced, the representation contract itself has to be made explicit. The four endomorphism cases have already been classified; what remains is to say what it means to represent those cases as a carrier without weakening or inflating the classification. Three conditions are load-bearing.

1. *Separation.* Distinct endomorphism cases must remain distinct on the representing carrier. No identification of cases is allowed.
2. *Complete realisation.* Every endomorphism case must be represented by a carrier element. No case is left without a vertex.
3. *No surplus.* Every carrier element must arise from one of the endomorphism cases. No extra vertex is introduced beyond the classification.

These clauses do not say that K_4 has been chosen. They state what a faithful minimal representation of the four-case classifier must do. Without separation, the classifier is collapsed. Without complete realisation, a case is missing. Without no surplus, the carrier contains structure not forced by the endomorphism data. Under this contract, the vertex set is in bijection with the four-case set, and the simple graph on distinct represented cases is complete. This is the small closure that the later invariant record enriches numerically.

The composition table does not add a fifth case. For any fixed distinction, composing two interpreted cases yields another endomorphism of the same carrier, and the classifier sends that composite back to one of the four cases. Thus total composability is inherited from function composition; it is not a separate generator.

```

-- § Composition of endomorphisms stays inside the classified four-case set.
composeEndo : (d : Distinction) → (S d → S d) → (S d → S d) → S d → S d
composeEndo d f g x = f (g x)

composeCase : (d : Distinction) → EndoCase → EndoCase → EndoCase
composeCase d c e =
  K4.classify d (composeEndo d (K4.interpret d c) (K4.interpret d e))

endomorphism-cases-closed :
  (d : Distinction) →
  (c e : EndoCase) →
  ∑ EndoCase
  (λ h → K4.interpret d h ≐
    composeEndo d (K4.interpret d c) (K4.interpret d e))
endomorphism-cases-closed d c e =
  composeCase d c e ,
  K4.classify-sound d (composeEndo d (K4.interpret d c) (K4.interpret d e))

```

The following records package the representation contract. The field **closure-realize** expresses complete realisation; the field **closure-reflect-realize** expresses separation by preventing realisation from identifying cases; and **closure-no-extra** expresses no surplus by forcing every carrier element to come from a case.

```

-- § Faithful representation of the four endomorphism cases.
record FaithfulClosure : Set1 where
  field
    closure-carrier : Set
    closure-realize : EndoCase → closure-carrier
    closure-reflect : closure-carrier → EndoCase
    closure-reflect-realize :
      (c : EndoCase) → closure-reflect (closure-realize c) ≡ c

-- § Minimality: no carrier element is present unless some case realizes it.
record MinimalClosure : Set1 where
  field
    faithful-closure : FaithfulClosure
    closure-no-extra :
      (v : FaithfulClosure.closure-carrier faithful-closure) →
      ∑ EndoCase
      (λ c → FaithfulClosure.closure-realize faithful-closure c ≡ v)

CanonicalClosureVertex : Set
CanonicalClosureVertex = EndoCase

CanonicalClosureEdge : CanonicalClosureVertex → CanonicalClosureVertex → Set
CanonicalClosureEdge c e = c ≠ e

canonical-minimal-closure : MinimalClosure
canonical-minimal-closure = record
  { faithful-closure = record
    { closure-carrier = EndoCase
      ; closure-realize = λ c → c
      ; closure-reflect = λ c → c

```

```

; closure-reflect-realize = λ c → refl
}
; closure-no-extra = λ v → v , refl
}

```

The two maps in a faithful minimal closure are inverse. One direction is the field `closure-reflect-realize`; the other follows from `closure-no-extra`. Hence every carrier satisfying the three representation conditions is equivalent to the canonical four-case carrier.

```

minimal-realize-reflect :
(m : MinimalClosure) →
(v : FaithfulClosure.closure-carrier (MinimalClosure.faithful-closure m)) →
FaithfulClosure.closure-realize (MinimalClosure.faithful-closure m)
(FaithfulClosure.closure-reflect (MinimalClosure.faithful-closure m) v)
≡ v
minimal-realize-reflect m v with MinimalClosure.closure-no-extra m v
... | c , p = trans (sym (cong realize q)) p
where
  fc = MinimalClosure.faithful-closure m
  realize = FaithfulClosure.closure-realize fc
  reflect = FaithfulClosure.closure-reflect fc
  reflect-realize = FaithfulClosure.closure-reflect-realize fc

  q : c ≡ reflect v
  q = trans (sym (reflect-realize c)) (cong reflect p)

```

```

minimal-realize-injective :
(m : MinimalClosure) →
(c e : EndoCase) →
FaithfulClosure.closure-realize (MinimalClosure.faithful-closure m) c
≡ FaithfulClosure.closure-realize (MinimalClosure.faithful-closure m) e →
c ≡ e
minimal-realize-injective m c e p =
trans (sym (FaithfulClosure.closure-reflect-realize fc c))
(trans (cong (FaithfulClosure.closure-reflect fc) p)
(FaithfulClosure.closure-reflect-realize fc e))
where
  fc = MinimalClosure.faithful-closure m

```

```

case-has-closure-vertex :
(m : MinimalClosure) →
(c : EndoCase) →
Σ (FaithfulClosure.closure-carrier (MinimalClosure.faithful-closure m))
(λ v → FaithfulClosure.closure-realize (MinimalClosure.faithful-closure m) c ≡ v)
case-has-closure-vertex m c =
FaithfulClosure.closure-realize (MinimalClosure.faithful-closure m) c , refl

```

```

closure-vertex-from-case :
(m : MinimalClosure) →
(v : FaithfulClosure.closure-carrier (MinimalClosure.faithful-closure m)) →
Σ EndoCase
(λ c → FaithfulClosure.closure-realize (MinimalClosure.faithful-closure m) c ≡ v)

```

```

closure-vertex-from-case  $m\ v = \text{MinimalClosure.closure-no-extra } m\ v$ 

minimal-closure-bijection :
  ( $m : \text{MinimalClosure}$ )  $\rightarrow$ 
  SetIso
    (FaithfulClosure.closure-carrier (MinimalClosure.faithful-closure  $m$ ))
  EndoCase
minimal-closure-bijection  $m = \text{record}$ 
{ to   = FaithfulClosure.closure-reflect  $fc$ 
; from = FaithfulClosure.closure-realize  $fc$ 
; to-from = FaithfulClosure.closure-reflect-realize  $fc$ 
; from-to = minimal-realize-reflect  $m$ 
}
where
   $fc = \text{MinimalClosure.faithful-closure } m$ 

```

Dropping any one field breaks the intended meaning of representation. Dropping `closure-reflect-realize` re-admits identification of cases. Dropping the totality of `closure-realize` leaves some case without a vertex. Dropping `closure-no-extra` permits a carrier element that no endomorphism case realizes. The conditions are therefore not selected because they yield K_4 ; they are the precise contract under which representing the four-case classifier has any minimal faithful meaning. The concrete `K4Vertex` below is the explicit four-constructor carrier used by the richer invariant record.

Structural infrastructure

This section fixes the combinatorial carrier on which the representation theory acts. The vertex set, adjacency relation, basis states, and neighbor summation are given explicitly so that later arguments refer to a concrete graph rather than an abstract placeholder. The lemma `cover-necessary` records the corresponding logical fact about the eliminator schema.

```

-- §  $K_4$  vertex type
data K4Vertex : Set where
   $v_0\ v_1\ v_2\ v_3 : K4Vertex$ 

-- § Lemma: coverage is necessary for the Distinction eliminator schema.
cover-necessary :
   $\neg ((S_0 : \text{Set}) (\ell_0\ r_0 : S_0) \rightarrow \ell_0 \neq r_0 \rightarrow$ 
    ( $P : S_0 \rightarrow \text{Set}$ )  $\rightarrow P\ \ell_0 \rightarrow P\ r_0 \rightarrow (x : S_0) \rightarrow P\ x)$ 
cover-necessary elim = elim K4Vertex  $v_0\ v_1\ v_0 \neq v_1\ \text{P-vertex } tt\ tt\ v_2$ 
where
   $v_0 \neq v_1 : v_0 \neq v_1$ 
   $v_0 \neq v_1 ()$ 
  P-vertex : K4Vertex  $\rightarrow$  Set
  P-vertex  $v_0 = \top$ 

```



```

P-vertex  $v_1 = \top$ 
P-vertex  $v_2 = \perp$ 
P-vertex  $v_3 = \top$ 

-- §  $K_4$  state: assignment of  $\mathbb{N}$  to each vertex
K4State : Set
K4State = K4Vertex  $\rightarrow \mathbb{N}$ 

-- §  $K_4$  adjacency (complete graph minus diagonal)
adjacent : K4Vertex  $\rightarrow$  K4Vertex  $\rightarrow \mathbb{N}$ 
adjacent  $v_0 v_0 = 0$ 
adjacent  $v_0 \_ = 1$ 
adjacent  $v_1 v_1 = 0$ 
adjacent  $v_1 \_ = 1$ 
adjacent  $v_2 v_2 = 0$ 
adjacent  $v_2 \_ = 1$ 
adjacent  $v_3 v_3 = 0$ 
adjacent  $v_3 \_ = 1$ 

-- § Kronecker delta on  $K_4$ 
 $\delta$  : K4Vertex  $\rightarrow$  K4Vertex  $\rightarrow \mathbb{N}$ 
 $\delta v_0 v_0 = 1$ 
 $\delta v_0 \_ = 0$ 
 $\delta v_1 v_1 = 1$ 
 $\delta v_1 \_ = 0$ 
 $\delta v_2 v_2 = 1$ 
 $\delta v_2 \_ = 0$ 
 $\delta v_3 v_3 = 1$ 
 $\delta v_3 \_ = 0$ 

-- § Basis states (= Kronecker delta)
K4-basis : K4Vertex  $\rightarrow$  K4State
K4-basis =  $\delta$ 

-- § Sum of neighbor values (graph Laplacian action)
sum-neighbors : K4State  $\rightarrow$  K4Vertex  $\rightarrow \mathbb{N}$ 
sum-neighbors  $\psi v =$  adjacent  $v v_0 * \psi v_0 +$  adjacent  $v v_1 * \psi v_1$ 
+ adjacent  $v v_2 * \psi v_2 +$  adjacent  $v v_3 * \psi v_3$ 

```

Vertex-invariance laws

K_4 is vertex-transitive, so quantities defined purely from adjacency should not depend on a chosen label. The following lemmas verify this directly by case analysis on the four vertices.

```

-- § Adjacency row sum equals vertex degree
law-adjacency-degree : (v : K4Vertex)  $\rightarrow$ 
adjacent  $v v_0 +$  adjacent  $v v_1 +$  adjacent  $v v_2 +$  adjacent  $v v_3 \equiv$  degree-K4
law-adjacency-degree  $v_0 =$  refl
law-adjacency-degree  $v_1 =$  refl
law-adjacency-degree  $v_2 =$  refl

```

```

law-adjacency-degree v3 = refl

-- § Each basis state is normalized (total weight 1)
law-basis-normalized : (u : K4Vertex) →
  K4-basis u v0 + K4-basis u v1 + K4-basis u v2 + K4-basis u v3 ≡ 1
law-basis-normalized v0 = refl
law-basis-normalized v1 = refl
law-basis-normalized v2 = refl
law-basis-normalized v3 = refl

-- § Basis propagation equals adjacency (16 cases)
law-basis-spreads : (u v : K4Vertex) →
  sum-neighbors (K4-basis u) v ≡ adjacent v u
law-basis-spreads v0 v0 = refl
law-basis-spreads v0 v1 = refl
law-basis-spreads v0 v2 = refl
law-basis-spreads v0 v3 = refl
law-basis-spreads v1 v0 = refl
law-basis-spreads v1 v1 = refl
law-basis-spreads v1 v2 = refl
law-basis-spreads v1 v3 = refl
law-basis-spreads v2 v0 = refl
law-basis-spreads v2 v1 = refl
law-basis-spreads v2 v2 = refl
law-basis-spreads v2 v3 = refl
law-basis-spreads v3 v0 = refl
law-basis-spreads v3 v1 = refl
law-basis-spreads v3 v2 = refl
law-basis-spreads v3 v3 = refl

```

The representation record

The record packages the numerical quantities used later together with the equations that connect them. Its design is triangular: the anchor $\text{dim} \equiv V$ fixes the first field, and the remaining constraints determine the others successively.

```

-- § The  $K_4$  invariant representation: 17 unknowns + 1 anchor + 16 chain constraints
record K4Record : Set where
  field
    dim          : ℕ -- total simplex dimension (V)
    n-spatial    : ℕ -- codimension-1 count ( $V \stackrel{\cdot}{-} 1$ )
    n-temporal   : ℕ -- residual dimension ( $V \stackrel{\cdot}{-} \text{n-spatial}$ )
    edge-count   : ℕ -- edge count (E)
    face-count   : ℕ -- 2-simplex count (F)
    euler        : ℕ -- Euler characteristic ( $V + F \stackrel{\cdot}{-} E$ )
    graph-eval   : ℕ -- graph evaluation ( $V^d \cdot \chi + d^2$ )
    codim        : ℕ -- codimension count (= d)
    clifford-dim : ℕ -- Clifford dimension ( $2^V$ )
    hierarchy    : ℕ -- hierarchy exponent ( $V \cdot E \stackrel{\cdot}{-} \chi$ )
    auto-count   : ℕ -- automorphism count ( $|S_V|$ )
    vertex-euler-product : ℕ -- discrete spectral product  $V \cdot \chi$ 

```

```

norm-index : ℕ -- normalization index (= V)
frac-num : ℕ -- fraction numerator (= n-temporal)
frac-den : ℕ -- fraction denominator (= E)
res-index : ℕ -- residual atom count (V  $\dot{-}$  d)
min-loop : ℕ -- triangle-cycle order (V  $\dot{-}$  d)

-- The anchor: exactly ONE connection to K4
anchor : dim  $\equiv$  vertexCountK4

-- Graph structure chain (from complete graph Kdim)
cg-deg : n-spatial  $\equiv$  dim  $\dot{-}$  1
cg-temporal : n-temporal  $\equiv$  dim  $\dot{-}$  n-spatial
cg-edges : edge-count  $\equiv$  (dim * n-spatial) div ℕ 2
cg-faces : face-count  $\equiv$  dim
cg-euler : euler  $\equiv$  (dim + face-count)  $\dot{-}$  edge-count

-- Structural chain (invariant correspondences)
ph-clifford : clifford-dim  $\equiv$  2 ^ dim
ph-eval : graph-eval  $\equiv$  (dim ^ n-spatial) * euler + n-spatial * n-spatial
ph-codim : codim  $\equiv$  n-spatial
ph-hierarchy : hierarchy  $\equiv$  dim * edge-count  $\dot{-}$  euler
ph-auto : auto-count  $\equiv$  dim * n-spatial * (n-spatial  $\dot{-}$  1) * (n-spatial  $\dot{-}$  2)
ph-vertex-euler : vertex-euler-product  $\equiv$  dim * euler
ph-norm : norm-index  $\equiv$  dim
ph-res : res-index  $\equiv$  n-temporal
ph-loop : min-loop  $\equiv$  n-temporal
ph-frac-num : frac-num  $\equiv$  n-temporal
ph-frac-den : frac-den  $\equiv$  edge-count

```

Existence: the canonical witness

The first task is to exhibit an inhabitant. The canonical witness assigns to each field the value already computed in the combinatorial chapters, and every proof field closes by `refl` because those values already satisfy the defining equations.

```

-- § The canonical K4 invariant representation
canonical-rep : K4Record
canonical-rep = record
{ dim          = 4
; n-spatial    = 3
; n-temporal   = 1
; edge-count    = 6
; face-count    = 4
; euler         = 2
; graph-eval    = 137
; codim         = 3
; clifford-dim  = 16
; hierarchy     = 22
; auto-count    = 24
; vertex-euler-product = 8
; norm-index    = 4

```

```

; frac-num = 1
; frac-den  = 6
; res-index  = 1
; min-loop   = 1
; anchor     = refl -- 4  $\equiv$  vertexCountK4
; cg-deg     = refl -- 3  $\equiv$  4  $\dot{-}$  1
; cg-temporal = refl -- 1  $\equiv$  4  $\dot{-}$  3
; cg-edges   = refl -- 6  $\equiv$  (4 * 3) div $\mathbb{N}$  2
; cg-faces   = refl -- 4  $\equiv$  4
; cg-euler   = refl -- 2  $\equiv$  (4 + 4)  $\dot{-}$  6
; ph-clifford = refl -- 16  $\equiv$  24
; ph-eval    = refl -- 137  $\equiv$  43·2 + 32
; ph-codim   = refl -- 3  $\equiv$  3
; ph-hierarchy = refl -- 22  $\equiv$  4·6  $\dot{-}$  2
; ph-auto    = refl -- 24  $\equiv$  4·3·2·1
; ph-vertex-euler = refl -- 8  $\equiv$  4·2
; ph-norm    = refl -- 4  $\equiv$  4
; ph-res     = refl -- 1  $\equiv$  1
; ph-loop    = refl -- 1  $\equiv$  1
; ph-frac-num = refl -- 1  $\equiv$  1
; ph-frac-den = refl -- 6  $\equiv$  6
}

```

Determination: the forcing chain resolves every field

To prove rigidity, one starts with an arbitrary admissible record and evaluates the constraints outward from the anchor. The equations first determine $\dim$, then n -spatial and n -temporal, and from there the remaining fields follow by substitution.

```

-- § Forcing chain: every field is determined by the constraints
module ForcedValues (r : K4Record) where
open K4Record r

dim-is-4 : dim  $\equiv$  4
dim-is-4 = anchor

spatial-is-3 : n-spatial  $\equiv$  3
spatial-is-3 = trans cg-deg (cong ( $\lambda d \rightarrow d \dot{-} 1$ ) dim-is-4)

temporal-is-1 : n-temporal  $\equiv$  1
temporal-is-1 = trans cg-temporal
  (trans (cong ( $\lambda d \rightarrow d \dot{-} n\text{-spatial}$ ) dim-is-4)
    (cong ( $\lambda s \rightarrow 4 \dot{-} s$ ) spatial-is-3))

edge-is-6 : edge-count  $\equiv$  6
edge-is-6 = trans cg-edges
  (trans (cong ( $\lambda d \rightarrow (d * n\text{-spatial}) \text{div}\mathbb{N} 2$ ) dim-is-4)
    (cong ( $\lambda s \rightarrow (4 * s) \text{div}\mathbb{N} 2$ ) spatial-is-3))

faces-is-4 : face-count  $\equiv$  4
faces-is-4 = trans cg-faces dim-is-4

```

```

euler-is-2 : euler  $\equiv$  2
euler-is-2 = trans cg-euler
(trans (cong ( $\lambda d \rightarrow (d + \text{face-count}) - \text{edge-count}$ ) dim-is-4)
(trans (cong ( $\lambda f \rightarrow (4 + f) - \text{edge-count}$ ) faces-is-4)
(cong ( $\lambda g \rightarrow 8 - g$ ) edge-is-6)))

clifford-is-16 : clifford-dim  $\equiv$  16
clifford-is-16 = trans ph-clifford (cong ( $\lambda d \rightarrow 2 \wedge d$ ) dim-is-4)

eval-is-137 : graph-eval  $\equiv$  137
eval-is-137 = trans ph-eval
(trans (cong ( $\lambda d \rightarrow (d \wedge \text{n-spatial}) * \text{euler} + \text{n-spatial} * \text{n-spatial}$ ) dim-is-4)
(trans (cong ( $\lambda s \rightarrow (4 \wedge s) * \text{euler} + s * s$ ) spatial-is-3)
(cong ( $\lambda e \rightarrow (4 \wedge 3) * e + 3 * 3$ ) euler-is-2)))

codim-is-3 : codim  $\equiv$  3
codim-is-3 = trans ph-codim spatial-is-3

hierarchy-is-22 : hierarchy  $\equiv$  22
hierarchy-is-22 = trans ph-hierarchy
(trans (cong ( $\lambda d \rightarrow d * \text{edge-count} - \text{euler}$ ) dim-is-4)
(trans (cong ( $\lambda g \rightarrow 4 * g - \text{euler}$ ) edge-is-6)
(cong ( $\lambda e \rightarrow 24 - e$ ) euler-is-2)))

auto-is-24 : auto-count  $\equiv$  24
auto-is-24 = trans ph-auto
(trans (cong ( $\lambda d \rightarrow d * \text{n-spatial} * (\text{n-spatial} - 1) * (\text{n-spatial} - 2)$ ) dim-is-4)
(cong ( $\lambda s \rightarrow 4 * s * (s - 1) * (s - 2)$ ) spatial-is-3))

vertex-euler-is-8 : vertex-euler-product  $\equiv$  8
vertex-euler-is-8 = trans ph-vertex-euler
(trans (cong ( $\lambda d \rightarrow d * \text{euler}$ ) dim-is-4)
(cong ( $\lambda e \rightarrow 4 * e$ ) euler-is-2))

norm-is-4 : norm-index  $\equiv$  4
norm-is-4 = trans ph-norm dim-is-4

res-is-1 : res-index  $\equiv$  1
res-is-1 = trans ph-res temporal-is-1

loop-is-1 : min-loop  $\equiv$  1
loop-is-1 = trans ph-loop temporal-is-1

frac-num-is-1 : frac-num  $\equiv$  1
frac-num-is-1 = trans ph-frac-num temporal-is-1

frac-den-is-6 : frac-den  $\equiv$  6
frac-den-is-6 = trans ph-frac-den edge-is-6

```

Uniqueness: any two representations agree

Once the forcing calculation has been carried out for arbitrary records, fieldwise uniqueness is immediate. Any two admissible representations have corresponding fields equal to the same forced values, so they agree one component at a time.

```

-- § Uniqueness: any two K4Record agree on all 17 fields
module RepUniqueness (r1 r2 : K4Record) where
private
  module F1 = ForcedValues r1
  module F2 = ForcedValues r2
open K4Record

dim≡      : dim r1 ≡ dim r2
dim≡      = trans F1.dim-is-4 (sym F2.dim-is-4)

spatial≡  : n-spatial r1 ≡ n-spatial r2
spatial≡  = trans F1.spatial-is-3 (sym F2.spatial-is-3)

temporal≡ : n-temporal r1 ≡ n-temporal r2
temporal≡ = trans F1.temporal-is-1 (sym F2.temporal-is-1)

edge≡     : edge-count r1 ≡ edge-count r2
edge≡     = trans F1.edge-is-6 (sym F2.edge-is-6)

faces≡    : face-count r1 ≡ face-count r2
faces≡    = trans F1.faces-is-4 (sym F2.faces-is-4)

euler≡    : euler r1 ≡ euler r2
euler≡    = trans F1.euler-is-2 (sym F2.euler-is-2)

eval≡     : graph-eval r1 ≡ graph-eval r2
eval≡     = trans F1.eval-is-137 (sym F2.eval-is-137)

codim≡    : codim r1 ≡ codim r2
codim≡    = trans F1.codim-is-3 (sym F2.codim-is-3)

clifford≡ : clifford-dim r1 ≡ clifford-dim r2
clifford≡ = trans F1.clifford-is-16 (sym F2.clifford-is-16)

hierarchy≡ : hierarchy r1 ≡ hierarchy r2
hierarchy≡ = trans F1.hierarchy-is-22 (sym F2.hierarchy-is-22)

auto≡     : auto-count r1 ≡ auto-count r2
auto≡     = trans F1.auto-is-24 (sym F2.auto-is-24)

vertex-euler≡ : vertex-euler-product r1 ≡ vertex-euler-product r2
vertex-euler≡ = trans F1.vertex-euler-is-8 (sym F2.vertex-euler-is-8)

norm≡     : norm-index r1 ≡ norm-index r2
norm≡     = trans F1.norm-is-4 (sym F2.norm-is-4)

res≡      : res-index r1 ≡ res-index r2
res≡      = trans F1.res-is-1 (sym F2.res-is-1)

loop≡     : min-loop r1 ≡ min-loop r2
loop≡     = trans F1.loop-is-1 (sym F2.loop-is-1)

frac-num≡  : frac-num r1 ≡ frac-num r2
frac-num≡  = trans F1.frac-num-is-1 (sym F2.frac-num-is-1)

frac-den≡  : frac-den r1 ≡ frac-den r2
frac-den≡  = trans F1.frac-den-is-6 (sym F2.frac-den-is-6)

```

Singleton: the representation has exactly one inhabitant

Fieldwise agreement is not yet record equality because K4Record also contains proof fields. Since these proof fields are equations between natural numbers, Hedberg's theorem on decidable equality of $\mathbb{N}$ provides UIP for them. Record equality then follows from agreement of both data and proof components.

```
-- § Decidable equality on  $\mathbb{N}$ .

data DecN (A : Set) : Set where
  decided-yes : A → DecN A
  decided-no  : ¬ A → DecN A

_?N_ : (m n :  $\mathbb{N}$ ) → DecN (m  $\equiv$  n)
zero ?N zero  = decided-yes refl
zero ?N suc n = decided-no (λ ())
suc m ?N zero = decided-no (λ ())
suc m ?N suc n with m ?N n
... | decided-yes refl = decided-yes refl
... | decided-no m≠n = decided-no (λ p → m≠n (suc-injective p))

-- § Hedberg's theorem for  $\mathbb{N}$ .

-- § Canonical proof selected by decidable equality.
private
  N-canonical : {m n :  $\mathbb{N}$ } → m  $\equiv$  n → m  $\equiv$  n
  N-canonical {m} {n} p with m ?N n
  ... | decided-yes c = c
  ... | decided-no f =  $\perp$ -elim (f p)

  N-canonical-const : {m n :  $\mathbb{N}$ } (p q : m  $\equiv$  n) → N-canonical p  $\equiv$  N-canonical q
  N-canonical-const {m} {n} p q with m ?N n
  ... | decided-yes _ = refl
  ... | decided-no f =  $\perp$ -elim (f p)

-- § General cancellation: trans (sym p) p reduces to refl.
trans-sym-cancel : {m n :  $\mathbb{N}$ } (p : m  $\equiv$  n) → trans (sym p) p  $\equiv$  refl
trans-sym-cancel refl = refl

-- § Cancel: any proof p is equal to the canonical one composed with a correction.
N-cancel : {m n :  $\mathbb{N}$ } (p : m  $\equiv$  n) →
  p  $\equiv$  trans (sym (N-canonical refl)) (N-canonical p)
N-cancel refl = sym (trans-sym-cancel (N-canonical refl))

-- § Hedberg: any two proofs of m  $\equiv$  n in  $\mathbb{N}$  are equal.
N-UIP : {m n :  $\mathbb{N}$ } (p q : m  $\equiv$  n) → p  $\equiv$  q
N-UIP p q =
  trans (N-cancel p)
  (trans (cong (trans (sym (N-canonical refl))) (N-canonical-const p q)))
```

(sym (N-cancel q)))

The singleton proof decomposes an arbitrary record into its data fields and proof fields. The forced-value equations identify every data field with its canonical value; then UIP for equations in $\mathbb{N}$ collapses the remaining proof fields to the canonical witnesses. Record eta closes the final equality.

-- § K4Record singleton.

private

singleton-lemma :

$\forall d s t g f e c n g s p h a b b h b n b d u m l$
 $\rightarrow d \equiv 4 \rightarrow s \equiv 3 \rightarrow t \equiv 1 \rightarrow g \equiv 6 \rightarrow f \equiv 4 \rightarrow e \equiv 2$
 $\rightarrow c \equiv 137 \rightarrow n g \equiv 3 \rightarrow s p \equiv 16 \rightarrow h \equiv 22 \rightarrow a \equiv 24 \rightarrow b \equiv 8$
 $\rightarrow b h \equiv 4 \rightarrow b n \equiv 1 \rightarrow b d \equiv 6 \rightarrow u \equiv 1 \rightarrow m l \equiv 1$
 $\rightarrow (p a n c : d \equiv \text{vertexCountK4}) \rightarrow (p d e g : s \equiv d \dot{-} 1)$
 $\rightarrow (p t e m p : t \equiv d \dot{-} s) \rightarrow (p e d g e : g \equiv (d * s) \text{divN } 2)$
 $\rightarrow (p f a c e : f \equiv d) \rightarrow (p c h i : e \equiv (d + f) \dot{-} g)$
 $\rightarrow (p s p i n : s p \equiv 2 \wedge d) \rightarrow (p c o u p : c \equiv (d \wedge s) * e + s * s)$
 $\rightarrow (p g e n : n g \equiv s) \rightarrow (p h i e r : h \equiv d * g \dot{-} e)$
 $\rightarrow (p a u t o : a \equiv d * s * (s \dot{-} 1) * (s \dot{-} 2))$
 $\rightarrow (p b e l l : b \equiv d * e) \rightarrow (p b h : b h \equiv d)$
 $\rightarrow (p u v : u \equiv t) \rightarrow (p l o o p : m l \equiv t)$
 $\rightarrow (p b n u m : b n \equiv t) \rightarrow (p b d e n : b d \equiv g)$
 $\rightarrow \text{record}$
 $\{ \text{dim} = d ; \text{n-spatial} = s ; \text{n-temporal} = t ; \text{edge-count} = g$
 $; \text{face-count} = f ; \text{euler} = e ; \text{graph-eval} = c ; \text{codim} = n g$
 $; \text{clifford-dim} = s p ; \text{hierarchy} = h ; \text{auto-count} = a ; \text{vertex-euler-product} = b$
 $; \text{norm-index} = b h ; \text{frac-num} = b n ; \text{frac-den} = b d$
 $; \text{res-index} = u ; \text{min-loop} = m l$
 $; \text{anchor} = p a n c ; \text{cg-deg} = p d e g ; \text{cg-temporal} = p t e m p$
 $; \text{cg-edges} = p e d g e ; \text{cg-faces} = p f a c e ; \text{cg-euler} = p c h i$
 $; \text{ph-clifford} = p s p i n ; \text{ph-eval} = p c o u p ; \text{ph-codim} = p g e n$
 $; \text{ph-hierarchy} = p h i e r ; \text{ph-auto} = p a u t o ; \text{ph-vertex-euler} = p b e l l$
 $; \text{ph-norm} = p b h ; \text{ph-res} = p u v ; \text{ph-loop} = p l o o p$
 $; \text{ph-frac-num} = p b n u m ; \text{ph-frac-den} = p b d e n$
 $\} \equiv \text{canonical-rep}$

singleton-lemma .4 .3 .1 .6 .4 .2 .137 .3 .16 .22 .24 .8 .4 .1 .6 .1 .1

refl refl refl refl refl refl refl refl refl refl refl refl refl refl refl
 $p a n c \ p d e g \ p t e m p \ p e d g e \ p f a c e \ p c h i \ p s p i n \ p c o u p \ p g e n \ p h i e r \ p a u t o \ p b e l l \ p b h \ p u v \ p l o o p \ p b n u m \ p b d e n$
 $\text{rewrite N-UIP } p a n c \text{ refl} \mid \text{N-UIP } p d e g \text{ refl} \mid \text{N-UIP } p t e m p \text{ refl}$
 $\mid \text{N-UIP } p e d g e \text{ refl} \mid \text{N-UIP } p f a c e \text{ refl} \mid \text{N-UIP } p c h i \text{ refl}$
 $\mid \text{N-UIP } p s p i n \text{ refl} \mid \text{N-UIP } p c o u p \text{ refl} \mid \text{N-UIP } p g e n \text{ refl}$
 $\mid \text{N-UIP } p h i e r \text{ refl} \mid \text{N-UIP } p a u t o \text{ refl} \mid \text{N-UIP } p b e l l \text{ refl}$
 $\mid \text{N-UIP } p b h \text{ refl} \mid \text{N-UIP } p u v \text{ refl} \mid \text{N-UIP } p l o o p \text{ refl}$
 $\mid \text{N-UIP } p b n u m \text{ refl} \mid \text{N-UIP } p b d e n \text{ refl}$
 $= \text{refl}$

K4Record-is-canonical : (r : K4Record) $\rightarrow r \equiv \text{canonical-rep}$

K4Record-is-canonical r = singleton-lemma

dim n-spatial n-temporal edge-count face-count euler
graph-eval codim clifford-dim hierarchy auto-count vertex-euler-product


```

norm-index frac-num frac-den res-index min-loop
dim-is-4 spatial-is-3 temporal-is-1 edge-is-6 faces-is-4 euler-is-2
eval-is-137 codim-is-3 clifford-is-16 hierarchy-is-22 auto-is-24 vertex-euler-is-8
norm-is-4 frac-num-is-1 frac-den-is-6 res-is-1 loop-is-1
anchor cg-deg cg-temporal cg-edges cg-faces cg-euler
ph-clifford ph-eval ph-codim ph-hierarchy ph-auto ph-vertex-euler
ph-norm ph-res ph-loop ph-frac-num ph-frac-den
where
  open K4Record r
  open ForcedValues r

-- § Corollary: any two representations are propositionally equal.
K4Record-singleton : (r1 r2 : K4Record) → r1 ≡ r2
K4Record-singleton r1 r2 =
  trans (K4Record-is-canonical r1) (sym (K4Record-is-canonical r2))

```

Cross-constraints: inter-layer consistency

The same numerical quantities were obtained along several routes. The equalities collected here verify that the graph-theoretic, algebraic, spectral, and combinatorial descriptions agree on the canonical record.

```

-- § Cross-layer bridges between forced values
module CrossConstraints where
  open K4Record

-- § Spatial directions = codimension count
cross-spatial-is-gen : n-spatial canonical-rep ≡ codim canonical-rep
cross-spatial-is-gen = refl

-- § Boundary normalization index = vertex count
cross-norm-is-dim : faceCountK4 ≡ vertexCountK4
cross-norm-is-dim = refl

-- § Triangle-cycle order = residual atom count = V · d
cross-loop-is-residual : triangle-loop-order ≡ vertexCountK4 · degree-K4
cross-loop-is-residual = refl

-- § Fraction denominator = edge count
cross-frac-is-edge : frac-den canonical-rep ≡ edge-count canonical-rep
cross-frac-is-edge = refl

-- § Automorphism count = dim × edge-count (= V × E = 4 × 6 = 24)
cross-auto-is-dim-edge :
  vertexCountK4 * degree-K4 * (degree-K4 - 1) * (degree-K4 - 2) ≡ 24
cross-auto-is-dim-edge = refl

-- § Hierarchy + Euler = V · E (22 + 2 = 24)
cross-hierarchy-plus-euler :
  hierarchy-exponent + eulerChar-computed ≡
  vertexCountK4 * edgeCountK4
cross-hierarchy-plus-euler = refl

```

Constraint derivation: from named invariants to canonical witness

The canonical record can also be reconstructed directly from the named K_4 invariants. In this form no bare numeral is inserted independently; the representation is assembled entirely from previously computed quantities.

```
-- § Derivation: named invariants assemble into a representation
module ConstraintDerivation where

-- § Graph-theoretic laws ( $K_4$  as complete graph)
law-cg-deg : degree-K4  $\equiv$  vertexCountK4  $\dot{-}$  1
law-cg-deg = refl

law-cg-edges : edgeCountK4  $\equiv$  (vertexCountK4 * degree-K4)  $\text{div} \mathbb{N} 2$ 
law-cg-edges = refl

law-cg-faces : faceCountK4  $\equiv$  vertexCountK4
law-cg-faces = refl

law-cg-euler : eulerChar-computed  $\equiv$  (vertexCountK4 + faceCountK4)  $\dot{-}$  edgeCountK4
law-cg-euler = refl

-- § Algebraic laws (representation theory)
law-ph-clifford : clifford-dimension  $\equiv$  2 ^ vertexCountK4
law-ph-clifford = refl

law-ph-auto :
  vertexCountK4 * degree-K4 * (degree-K4  $\dot{-}$  1) * (degree-K4  $\dot{-}$  2)  $\equiv$  24
law-ph-auto = refl

-- § Simplex-Euler evaluation law ( $K_4$  invariant basis)
law-ph-eval :
  simplex-eval  $\equiv$ 
    (vertexCountK4 ^ degree-K4) * eulerChar-computed + degree-K4 * degree-K4
law-ph-eval = refl

-- § Combinatorial law ( $K_4$  arithmetic)
law-ph-hierarchy :
  hierarchy-exponent  $\equiv$  vertexCountK4 * edgeCountK4  $\dot{-}$  eulerChar-computed
law-ph-hierarchy = refl

-- § Structural assembly: K4Record from named invariants only
derived-rep : K4Record
derived-rep = record
  { dim          = vertexCountK4
  ; n-spatial    = degree-K4
  ; n-temporal   = vertexCountK4  $\dot{-}$  degree-K4
  ; edge-count   = edgeCountK4
  ; face-count   = faceCountK4
  ; euler        = eulerChar-computed
  ; graph-eval   = simplex-eval
```

```

; codim      = degree-K4
; clifford-dim = clifford-dimension
; hierarchy   = hierarchy-exponent
; auto-count  = vertexCountK4 * degree-K4
                * (degree-K4 - 1) * (degree-K4 - 2)
; vertex-euler-product = vertexCountK4 * eulerChar-computed
; norm-index  = vertexCountK4
; frac-num    = vertexCountK4 - degree-K4
; frac-den    = edgeCountK4
; res-index   = vertexCountK4 - degree-K4
; min-loop    = vertexCountK4 - degree-K4
; anchor      = refl
; cg-deg      = refl
; cg-temporal = refl
; cg-edges    = refl
; cg-faces    = refl
; cg-euler    = refl
; ph-clifford = refl
; ph-auto     = refl
; ph-eval     = refl
; ph-hierarchy = refl
; ph-codim    = refl
; ph-vertex-euler = refl
; ph-norm     = refl
; ph-res      = refl
; ph-loop     = refl
; ph-frac-num = refl
; ph-frac-den = refl
}

-- § derived-rep IS canonical-rep (definitional equality)
derivation-is-canonical : derived-rep  $\equiv$  canonical-rep
derivation-is-canonical = refl

```

Represented quantities as computable K_4 invariants

The record entries may now be read as explicit computable functions of the concrete K_4 carrier. This rephrasing adds no new data; it makes the relation between the representation record and graph calculations direct.

Representation Completeness

The completeness statement says that every admissible representation already has the same observable values as the canonical one. The formal bridge into the observable language is therefore independent of presentation.

```

-- § RepresentationComplete: every K4Record agrees with canonical-rep on all values.
module RepresentationComplete (r : K4Record) where
private module F = ForcedValues r

```

open K4Record

val-dim : dim r $\equiv$ dim canonical-rep
val-dim = F.dim-is-4

val-spatial : n-spatial r $\equiv$ n-spatial canonical-rep
val-spatial = F.spatial-is-3

val-temporal : n-temporal r $\equiv$ n-temporal canonical-rep
val-temporal = F.temporal-is-1

val-edge : edge-count r $\equiv$ edge-count canonical-rep
val-edge = F.edge-is-6

val-faces : face-count r $\equiv$ face-count canonical-rep
val-faces = F.faces-is-4

val-euler : euler r $\equiv$ euler canonical-rep
val-euler = F.euler-is-2

val-eval : graph-eval r $\equiv$ graph-eval canonical-rep
val-eval = F.eval-is-137

val-codim : codim r $\equiv$ codim canonical-rep
val-codim = F.codim-is-3

val-clifford : clifford-dim r $\equiv$ clifford-dim canonical-rep
val-clifford = F.clifford-is-16

val-hierarchy : hierarchy r $\equiv$ hierarchy canonical-rep
val-hierarchy = F.hierarchy-is-22

val-auto : auto-count r $\equiv$ auto-count canonical-rep
val-auto = F.auto-is-24

val-vertex-euler : vertex-euler-product r $\equiv$ vertex-euler-product canonical-rep
val-vertex-euler = F.vertex-euler-is-8

val-bh : norm-index r $\equiv$ norm-index canonical-rep
val-bh = F.norm-is-4

val-uv : res-index r $\equiv$ res-index canonical-rep
val-uv = F.res-is-1

val-loop : min-loop r $\equiv$ min-loop canonical-rep
val-loop = F.loop-is-1

val-frac-num : frac-num r $\equiv$ frac-num canonical-rep
val-frac-num = F.frac-num-is-1

val-frac-den : frac-den r $\equiv$ frac-den canonical-rep
val-frac-den = F.frac-den-is-6

Local graph observables

The first block isolates the vertex-local calculations: row sums, basis propagation, and the corresponding uniformity facts.

```
-- § Represented quantities as computable  $K_4$  functions
module K4Quantities where

-- § Adjacency row sum at each vertex
adjacency-row-sum : K4Vertex → ℕ
adjacency-row-sum v =
  adjacent v v0 + adjacent v v1 + adjacent v v2 + adjacent v v3

-- § Row sum is vertex-uniform and equals degree
adjacency-row-sum-is-degree : (v : K4Vertex) → adjacency-row-sum v ≡ degree-K4
adjacency-row-sum-is-degree = law-adjacency-degree

-- § Each row sum propagates independently (basis property)
adjacency-row-sum-independent : (u v : K4Vertex) →
  sum-neighbors (K4-basis u) v ≡ adjacent v u
adjacency-row-sum-independent = law-basis-spreads
```

Global quantities built from the local layer

Once the local quantities are fixed, the remaining global ones are short derived computations.

```
-- § Boundary count: 2-simplex count = face count
boundary-count : ℕ
boundary-count = faceCountK4

-- § Self-duality: boundary = bulk (F = V for the tetrahedron)
boundary-is-bulk : boundary-count ≡ vertexCountK4
boundary-is-bulk = refl

-- § Spectral width: largest minus smallest adjacency eigenvalue
spectral-width : ℕ
spectral-width = degree-K4 + 1

-- § Spectral width equals vertex count
spectral-width-is-V : spectral-width ≡ vertexCountK4
spectral-width-is-V = refl

-- § Spectral-Euler product: spectral width × Euler characteristic
spectral-euler-product : ℕ
spectral-euler-product = spectral-width * eulerChar-computed

-- § Spectral-Euler product equals  $\kappa$ -discrete
spectral-euler-product-is- $\kappa$  : spectral-euler-product ≡  $\kappa$ -discrete
spectral-euler-product-is- $\kappa$  = refl
```

```

-- § Complement degree:  $V \dashv \deg$ 
complement-degree :  $\mathbb{N}$ 
complement-degree = vertexCountK4  $\dashv$  degree-K4

-- § Complement degree is 1
complement-degree-is-1 : complement-degree  $\equiv$  1
complement-degree-is-1 = refl

-- § Same computation, three names (all  $V \dashv \deg$ )
complement-degree-is-max-prop : complement-degree  $\equiv$  max-propagation-per-edge
complement-degree-is-max-prop = refl

complement-degree-is-triangle : complement-degree  $\equiv$  triangle-loop-order
complement-degree-is-triangle = refl

complement-degree-is-lattice : complement-degree  $\equiv$  lattice-spacing-natural
complement-degree-is-lattice = refl

-- § Edge tally: total pairwise edges
edge-tally :  $\mathbb{N}$ 
edge-tally = edgeCountK4

```

Observable representation and closure

The representation can now be rebuilt from the observable quantities alone. The resulting record is definitionally the canonical one, so no additional assignment layer remains.

```

-- § K4Record from computed quantities (no bare numerals)
observable-rep : K4Record
observable-rep = record
{ dim           = vertexCountK4
; n-spatial    = degree-K4
; n-temporal   = complement-degree
; edge-count   = edge-tally
; face-count   = boundary-count
; euler        = eulerChar-computed
; graph-eval   = simplex-eval
; codim        = degree-K4
; clifford-dim = clifford-dimension
; hierarchy    = hierarchy-exponent
; auto-count   = vertexCountK4 * degree-K4
                * (degree-K4  $\dashv$  1) * (degree-K4  $\dashv$  2)
; vertex-euler-product = spectral-euler-product
; norm-index   = boundary-count
; frac-num     = complement-degree
; frac-den     = edge-tally
; res-index    = complement-degree
; min-loop     = complement-degree
; anchor       = refl
; cg-deg       = refl

```

```

; cg-temporal = refl
; cg-edges    = refl
; cg-faces    = refl
; cg-euler    = refl
; ph-clifford = refl
; ph-auto     = refl
; ph-eval     = refl
; ph-hierarchy = refl
; ph-codim    = refl
; ph-vertex-euler = refl
; ph-norm     = refl
; ph-res      = refl
; ph-loop     = refl
; ph-frac-num = refl
; ph-frac-den = refl
}

-- § Closure: observable-rep IS canonical-rep (definitional equality)
observable-is-canonical : observable-rep  $\equiv$  canonical-rep
observable-is-canonical = refl

```

The observable principle

To close the semantic side, observables are specified explicitly. Local observables are uniform vertex functions, and global observables are drawn from a finite canonical list.

Observable types and invariance

```

-- §  $K_4$  vertex-level invariance
IsK4Invariant : (K4Vertex  $\rightarrow$   $\mathbb{N}$ )  $\rightarrow$  Set
IsK4Invariant  $f = (\nu \ w : K4Vertex) \rightarrow f \ \nu \equiv f \ w$ 

-- § Local observable: vertex function + uniformity proof
LocalObservable : Set
LocalObservable =  $\Sigma$  (K4Vertex  $\rightarrow$   $\mathbb{N}$ ) IsK4Invariant

-- § Global admissible observables collapse to four canonical codes.
data GlobalObservable : Set where
  go-boundary go-spectral go-complement go-edges : GlobalObservable

eval-global-observable : GlobalObservable  $\rightarrow$   $\mathbb{N}$ 
eval-global-observable go-boundary = boundary-count
eval-global-observable go-spectral = spectral-euler-product
eval-global-observable go-complement = complement-degree
eval-global-observable go-edges = edge-tally

-- § Complete observable type
Observable : Set
Observable = LocalObservable  $\uplus$  GlobalObservable

```

```

-- § Every invariant function has a unique value, independent of vertex
invariant-value : (f : K4Vertex → ℕ) → IsK4Invariant f → ℕ
invariant-value f _ = f v0

invariant-any-vertex : (f : K4Vertex → ℕ) → (inv : IsK4Invariant f) →
  (v : K4Vertex) → f v ≡ invariant-value f inv
invariant-any-vertex f inv v = inv v v0

observable-value : Observable → ℕ
observable-value (inj1 f , inv) = invariant-value f inv
observable-value (inj2 g) = eval-global-observable g

```

Local observable witnesses

This block records the local pattern once: define the observable, prove uniformity, and extract its value.

```

-- § Adjacency-row-sum is a local observable
adjacency-row-sum-invariant : IsK4Invariant adjacency-row-sum
adjacency-row-sum-invariant v w =
  trans (adjacency-row-sum-is-degree v) (sym (adjacency-row-sum-is-degree w))

adjacency-row-sum-observable : LocalObservable
adjacency-row-sum-observable = adjacency-row-sum , adjacency-row-sum-invariant

-- § Observed value is degree = 3
adjacency-row-sum-observed-value : invariant-value adjacency-row-sum adjacency-row-sum-invariant
  ≡ degree-K4
adjacency-row-sum-observed-value = refl

-- § Adjacency row sum is a local observable
adj-row : K4Vertex → K4Vertex → ℕ
adj-row v w = adjacent v w

adj-row-sum-invariant : IsK4Invariant (λ v →
  adj-row v v0 + adj-row v v1 + adj-row v v2 + adj-row v v3)
adj-row-sum-invariant v w =
  trans (law-adjacency-degree v) (sym (law-adjacency-degree w))

```

Global observable witnesses

For the global layer, admissibility is encoded by a finite datatype. The type `GlobalObservable` has exactly four constructors, and the exhaustiveness lemma records that no fifth primitive class has been left open.

```

-- § Global observables (invariant by construction)

```



```

obs-boundary : Observable
obs-boundary = inj₂ go-boundary

obs-spectral : Observable
obs-spectral = inj₂ go-spectral

obs-complement : Observable
obs-complement = inj₂ go-complement

obs-edges : Observable
obs-edges = inj₂ go-edges

global-observable-exhaustive :
  (g : GlobalObservable) →
  (g ≡ go-boundary)
  ⊔ ((g ≡ go-spectral)
    ⊔ ((g ≡ go-complement) ⊔ (g ≡ go-edges)))
global-observable-exhaustive go-boundary = inj₁ refl
global-observable-exhaustive go-spectral = inj₂ (inj₁ refl)
global-observable-exhaustive go-complement = inj₂ (inj₂ (inj₁ refl))
global-observable-exhaustive go-edges = inj₂ (inj₂ (inj₂ refl))

-- § Derived ledger invariants remain observable values, but not new primitive classes.
obs-eval-value : ℕ
obs-eval-value = simplex-eval

obs-clifford-value : ℕ
obs-clifford-value = clifford-dimension

obs-hierarchy-value : ℕ
obs-hierarchy-value = hierarchy-exponent

obs-euler-value : ℕ
obs-euler-value = eulerChar-computed

```


Chapter 58

K_4 Vertex Permutations

This chapter internalizes the action of $\text{Aut}(K_4) \cong S_4$ on the vertex set. Using the permutation machinery already established on `EndoCase` and an explicit bijection with `K4Vertex`, one obtains the lifted permutations and the invariance tools needed later.

Vertex bijection

The transfer begins with an explicit bijection between `K4Vertex` and `EndoCase`. Once this identification is fixed, the existing permutation lemmas on `EndoCase` can be reused verbatim.

```
k4-to-endo : K4Vertex → EndoCase
k4-to-endo v0 = case-constL
k4-to-endo v1 = case-constR
k4-to-endo v2 = case-id
k4-to-endo v3 = case-dual

endo-to-k4 : EndoCase → K4Vertex
endo-to-k4 case-constL = v0
endo-to-k4 case-constR = v1
endo-to-k4 case-id     = v2
endo-to-k4 case-dual   = v3

k4-endo-k4 : (v : K4Vertex) → endo-to-k4 (k4-to-endo v) ≡ v
k4-endo-k4 v0 = refl
k4-endo-k4 v1 = refl
k4-endo-k4 v2 = refl
k4-endo-k4 v3 = refl

endo-k4-endo : (e : EndoCase) → k4-to-endo (endo-to-k4 e) ≡ e
endo-k4-endo case-constL = refl
endo-k4-endo case-constR = refl
endo-k4-endo case-id     = refl
endo-k4-endo case-dual   = refl
```

Permutation record and lifting

Permutations of `K4Vertex` are encoded as bijections with explicit inverses. The lifting operation `liftPerm` is conjugation through the established bijection, so bijectivity is transported automatically.

```
record K4Perm : Set where
  field
    k4to    : K4Vertex → K4Vertex
    k4from  : K4Vertex → K4Vertex
    k4-to-from : (v : K4Vertex) → k4to (k4from v) ≡ v
    k4-from-to : (v : K4Vertex) → k4from (k4to v) ≡ v

open K4Perm public

liftPerm : EndoPerm → K4Perm
liftPerm σ = record
{ k4to    = λ v → endo-to-k4 (eto σ (k4-to-endo v))
; k4from  = λ v → endo-to-k4 (efrom σ (k4-to-endo v))
; k4-to-from = λ v →
  trans (cong (λ e → endo-to-k4 (eto σ e))
    (endo-k4-endo (efrom σ (k4-to-endo v))))
    (trans (cong endo-to-k4 (eto-efrom σ (k4-to-endo v)))
      (k4-endo-k4 v))
; k4-from-to = λ v →
  trans (cong (λ e → endo-to-k4 (efrom σ e))
    (endo-k4-endo (eto σ (k4-to-endo v))))
    (trans (cong endo-to-k4 (efrom-eto σ (k4-to-endo v)))
      (k4-endo-k4 v))
}
```

Composition of two permutations inherits bijectivity from the components.

```
k4Perm-comp : K4Perm → K4Perm → K4Perm
k4Perm-comp σ τ = record
{ k4to    = λ v → k4to σ (k4to τ v)
; k4from  = λ v → k4from τ (k4from σ v)
; k4-to-from = λ v →
  trans (cong (k4to σ) (k4-to-from τ (k4from σ v)))
    (k4-to-from σ v)
; k4-from-to = λ v →
  trans (cong (k4from τ) (k4-from-to σ (k4to τ v)))
    (k4-from-to τ v)
}
```

Decidable equality and adjacency lemmas

Since `K4Vertex` has four constructors, equality is decidable by a finite case split. This allows later arguments to distinguish the diagonal from the off-diagonal exactly.

```

K4-decEq : (v w : K4Vertex) → (v ≡ w) ⊔ (v ≠ w)
K4-decEq v0 v0 = inj1 refl
K4-decEq v1 v1 = inj1 refl
K4-decEq v2 v2 = inj1 refl
K4-decEq v3 v3 = inj1 refl
K4-decEq v0 v1 = inj2 (λ ())
K4-decEq v0 v2 = inj2 (λ ())
K4-decEq v0 v3 = inj2 (λ ())
K4-decEq v1 v0 = inj2 (λ ())
K4-decEq v1 v2 = inj2 (λ ())
K4-decEq v1 v3 = inj2 (λ ())
K4-decEq v2 v0 = inj2 (λ ())
K4-decEq v2 v1 = inj2 (λ ())
K4-decEq v2 v3 = inj2 (λ ())
K4-decEq v3 v0 = inj2 (λ ())
K4-decEq v3 v1 = inj2 (λ ())
K4-decEq v3 v2 = inj2 (λ ())

```

Two basic lemmas record the shape of adjacent: it is 0 on the diagonal and 1 on every off-diagonal pair. Thus adjacency in K_4 is exactly distinctness.

```

adj-self : (v : K4Vertex) → adjacent v v ≡ 0
adj-self v0 = refl
adj-self v1 = refl
adj-self v2 = refl
adj-self v3 = refl

K4-distinct-adj : (v w : K4Vertex) → v ≠ w → adjacent v w ≡ 1
K4-distinct-adj v0 v0 ne = ⊥-elim (ne refl)
K4-distinct-adj v0 v1 _ = refl
K4-distinct-adj v0 v2 _ = refl
K4-distinct-adj v0 v3 _ = refl
K4-distinct-adj v1 v0 _ = refl
K4-distinct-adj v1 v1 ne = ⊥-elim (ne refl)
K4-distinct-adj v1 v2 _ = refl
K4-distinct-adj v1 v3 _ = refl
K4-distinct-adj v2 v0 _ = refl
K4-distinct-adj v2 v1 _ = refl
K4-distinct-adj v2 v2 ne = ⊥-elim (ne refl)
K4-distinct-adj v2 v3 _ = refl
K4-distinct-adj v3 v0 _ = refl
K4-distinct-adj v3 v1 _ = refl
K4-distinct-adj v3 v2 _ = refl
K4-distinct-adj v3 v3 ne = ⊥-elim (ne refl)

```

Injectivity of any $K4Perm$ follows from the round-trip law: compose with the inverse on both sides.

```

k4-injective : (σ : K4Perm) → (v w : K4Vertex) →
  k4to σ v ≡ k4to σ w → v ≡ w
k4-injective σ v w p =
  trans (sym (k4-from-to σ v))

```

```
(trans (cong (k4from  $\sigma$ )  $p$ )
      (k4-from-to  $\sigma$   $w$ ))
```

Transitivity and 2-transitivity

The symmetry property needed later is 2-transitivity on ordered pairs of distinct vertices. After the one-point transport lemma `k4Perm-send`, the map `edge-map` sends the reference edge (v_0, v_1) to any off-diagonal pair.

```
k4Perm-send : (a a' : K4Vertex) →  $\Sigma$  K4Perm ( $\lambda \sigma \rightarrow$  k4to  $\sigma$   $a \equiv a'$ )
k4Perm-send a a' =
  let ep = endoPerm-send (k4-to-endo a) (k4-to-endo a')
  in (liftPerm (fst ep),
      trans (cong endo-to-k4 (snd ep)) (k4-endo-k4 a'))
```

The construction is entirely explicit: each off-diagonal target is reached by at most two transpositions, and every branch closes by computation.

```
edge-map : (v w : K4Vertex) →  $v \neq w \rightarrow$ 
 $\Sigma$  K4Perm ( $\lambda \sigma \rightarrow \Sigma$  (k4to  $\sigma$   $v_0 \equiv v$ ) ( $\lambda \_ \rightarrow$  k4to  $\sigma$   $v_1 \equiv w$ ))
edge-map v0 v0 ne =  $\perp$ -elim (ne refl)
edge-map v0 v1 _ = (liftPerm (permSwap case-constL case-constL)
                    , (refl , refl))
edge-map v0 v2 _ = (liftPerm (permSwap case-constR case-id)
                    , (refl , refl))
edge-map v0 v3 _ = (liftPerm (permSwap case-constR case-dual)
                    , (refl , refl))
edge-map v1 v0 _ = (liftPerm (permSwap case-constL case-constR)
                    , (refl , refl))
edge-map v1 v1 ne =  $\perp$ -elim (ne refl)
edge-map v1 v2 _ = (k4Perm-comp (liftPerm (permSwap case-constR case-id))
                    (liftPerm (permSwap case-constL case-id))
                    , (refl , refl))
edge-map v1 v3 _ = (k4Perm-comp (liftPerm (permSwap case-constR case-dual))
                    (liftPerm (permSwap case-constL case-dual))
                    , (refl , refl))
edge-map v2 v0 _ = (k4Perm-comp (liftPerm (permSwap case-constL case-id))
                    (liftPerm (permSwap case-constR case-id))
                    , (refl , refl))
edge-map v2 v1 _ = (liftPerm (permSwap case-constL case-id)
                    , (refl , refl))
edge-map v2 v2 ne =  $\perp$ -elim (ne refl)
edge-map v2 v3 _ = (k4Perm-comp (liftPerm (permSwap case-constL case-id))
                    (liftPerm (permSwap case-constR case-dual))
                    , (refl , refl))
edge-map v3 v0 _ = (k4Perm-comp (liftPerm (permSwap case-constL case-dual))
                    (liftPerm (permSwap case-constR case-dual))
                    , (refl , refl))
edge-map v3 v1 _ = (liftPerm (permSwap case-constL case-dual)
                    , (refl , refl))
```

```

edge-map v3 v2 _ = (k4Perm-comp (liftPerm (permSwap case-constL case-dual))
                                (liftPerm (permSwap case-constR case-id))
                                , (refl , refl))
edge-map v3 v3 ne =  $\perp$ -elim (ne refl)

```

Adjacency preservation

Because adjacency in K_4 is equivalent to distinctness, every vertex permutation preserves adjacency automatically. The proof splits into the diagonal and off-diagonal cases.

```

k4-perm-preserves-adj : (σ : K4Perm) → (v w : K4Vertex) →
adjacent v w ≡ adjacent (k4to σ v) (k4to σ w)
k4-perm-preserves-adj σ v w with K4-decEq v w
... | inj1 refl = trans (adj-self v) (sym (adj-self (k4to σ v)))
... | inj2 ne = trans (K4-distinct-adj v w ne)
                    (sym (K4-distinct-adj (k4to σ v) (k4to σ w)
                    (λ p → ne (k4-injective σ v w p))))

```

K4-invariance and uniform edge weight

An invariant function on ordered vertex pairs is one that is unchanged under every K4Perm. Combined with 2-transitivity, this implies that any such function is constant on the off-diagonal.

```

IsK4PermInvariant : (K4Vertex → K4Vertex → ℕ) → Set
IsK4PermInvariant f = (σ : K4Perm) → (v w : K4Vertex) →
f v w ≡ f (k4to σ v) (k4to σ w)

```

The proof compares arbitrary edges with the reference edge (v_0, v_1) using the permutations supplied by `edge-map`.

```

invariant-uniform-edges : (f : K4Vertex → K4Vertex → ℕ) →
IsK4PermInvariant f →
(v w v' w' : K4Vertex) → v ≠ w → v' ≠ w' → f v w ≡ f v' w'
invariant-uniform-edges f inv v w v' w' ne ne' =
trans (sym (to-ref v w ne)) (to-ref v' w' ne')
where
to-ref : (a b : K4Vertex) → a ≠ b → f v0 v1 ≡ f a b
to-ref a b ne-ab =
let em = edge-map a b ne-ab
σ = fst em
p = fst (snd em)
q = snd (snd em)
in trans (inv σ v0 v1)
        (trans (cong (λ x → f x (k4to σ v1)) p)
              (cong (f a) q))

```

This reduction to a single reference edge is the symmetry principle used in the next chapter.

Chapter 59

Forced Propagation

Earlier chapters computed $\text{max-propagation-per-edge} \equiv 1$, but that numerical identity alone does not rule out competing propagation laws. This chapter formulates propagation laws explicitly and shows that the admissible constraints force the common off-diagonal value to be exactly 1.

Propagation candidates

A propagation law is taken to be an arbitrary natural-valued function on ordered vertex pairs. The definition is intentionally wide: any restriction must come from the later hypotheses.

`PropagationLaw : Set`
`PropagationLaw = K4Vertex → K4Vertex → ℕ`

The four admissibility filters

The hypotheses are separated by role. Locality fixes the diagonal at 0, while lower and upper bounds control the off-diagonal values. In the admissible record these bounds are stored only on the reference edge (v_0, v_1) ; invariance later transports them to every other off-diagonal pair.

`IsLocal : PropagationLaw → Set`
`IsLocal P = (v : K4Vertex) → P v v ≡ 0`

Edge support asks only for a lower bound of 1 on off-diagonal pairs. At this stage it does not yet identify the value.

`HasEdgeSupport : PropagationLaw → Set`
`HasEdgeSupport P = (v w : K4Vertex) → v ≠ w → 1 ≤ P v w`

```
ReferenceEdgeLower : PropagationLaw → Set
ReferenceEdgeLower P = 1 ≤ P v0 v1
```

Boundedness supplies the complementary upper bound of 1.

```
IsBounded : PropagationLaw → Set
IsBounded P = (v w : K4Vertex) → P v w ≤ 1
```

```
ReferenceEdgeUpper : PropagationLaw → Set
ReferenceEdgeUpper P = P v0 v1 ≤ 1
```

K_4 -invariance requires compatibility with every vertex permutation. By the results of the previous chapter, this is the condition that later collapses all off-diagonal values to a single common quantity.

The admissibility record packages these four conditions together. Uniqueness arises only from their combination.

```
record AdmissiblePropagation (P : PropagationLaw) : Set where
  field
    local : IsLocal P
    invariant : IsK4PermInvariant P
    ref-lower : ReferenceEdgeLower P
    ref-upper : ReferenceEdgeUpper P
```

The elimination

The elimination argument has three steps. First, invariance reduces all off-diagonal values to the reference edge value $P v_0 v_1$. Second, the lower and upper reference bounds are transported to every edge. Third, the resulting inequalities $1 \leq r \leq 1$ force $r = 1$.

Step 1: Invariance collapses off-diagonal to one value. By invariant-uniform-edges, every off-diagonal pair has the same value as the reference edge.

```
propagation-uniform : (P : PropagationLaw) → IsK4PermInvariant P →
  (v w : K4Vertex) → v ≠ w → P v w ≡ P v0 v1
propagation-uniform P inv v w ne =
  invariant-uniform-edges P inv v w v0 v1 ne (λ ())
```

Step 2: Invariance transports the brackets. The lower and upper bounds are stored only on the reference edge, but invariance carries them to every off-diagonal pair.

```
propagation-supported : (P : PropagationLaw) → AdmissiblePropagation P →
  (v w : K4Vertex) → v ≠ w → 1 ≤ P v w
```

```

propagation-supported P adm v w ne =
  subst (λ n → 1 ≤ n)
    (sym (propagation-uniform P (AdmissiblePropagation.invariant adm) v w ne))
    (AdmissiblePropagation.ref-lower adm)

propagation-bounded : (P : PropagationLaw) → AdmissiblePropagation P →
  (v w : K4Vertex) → P v w ≤ 1
propagation-bounded P adm v w with K4-decEq v w
... | inj1 refl = subst (λ n → n ≤ 1) (sym (AdmissiblePropagation.local adm v)) z ≤ n
... | inj2 ne = subst (λ n → n ≤ 1)
  (sym (propagation-uniform P (AdmissiblePropagation.invariant adm) v w ne))
  (AdmissiblePropagation.ref-upper adm)

```

Step 3: The sandwich. The transported inequalities force the common off-diagonal value to be 1.

```

v0 ≠ v1 : v0 ≠ v1
v0 ≠ v1 ()

propagation-edge-value : (P : PropagationLaw) → AdmissiblePropagation P →
  P v0 v1 ≡ 1
propagation-edge-value P adm =
  ≤-antisym
  (AdmissiblePropagation.ref-upper adm)
  (AdmissiblePropagation.ref-lower adm)

```

Step 4: Assembly. The diagonal is 0 by locality and the off-diagonal is 1 by the sandwich, so the law coincides with adjacent.

```

propagation-off-diag : (P : PropagationLaw) → AdmissiblePropagation P →
  (v w : K4Vertex) → v ≠ w → P v w ≡ 1
propagation-off-diag P adm v w ne =
  trans (propagation-uniform P (AdmissiblePropagation.invariant adm) v w ne)
    (propagation-edge-value P adm)

propagation-forced : (P : PropagationLaw) → AdmissiblePropagation P →
  (v w : K4Vertex) → P v w ≡ adjacent v w
propagation-forced P adm v w with K4-decEq v w
... | inj1 refl = trans (AdmissiblePropagation.local adm v)
  (sym (adj-self v))
... | inj2 ne = trans (propagation-off-diag P adm v w ne)
  (sym (K4-distinct-adj v w ne))

```

The theorem therefore reduces the whole comparison to the diagonal/off-diagonal split; symmetry handles the transport and the sandwich fixes the common edge value.

Existence

Existence is witnessed by the adjacency function itself. It is local by definition, invariant under every K4Perm, and satisfies both reference-edge bounds.

```

adjacent-is-local : IsLocal adjacent
adjacent-is-local v0 = refl
adjacent-is-local v1 = refl
adjacent-is-local v2 = refl
adjacent-is-local v3 = refl

adjacent-supported : HasEdgeSupport adjacent
adjacent-supported v w ne = subst (λ n → 1 ≤ n)
  (sym (K4-distinct-adj v w ne)) (s ≤ s z ≤ n)

adjacent-bounded : IsBounded adjacent
adjacent-bounded v0 v0 = z ≤ n
adjacent-bounded v0 v1 = s ≤ s z ≤ n
adjacent-bounded v0 v2 = s ≤ s z ≤ n
adjacent-bounded v0 v3 = s ≤ s z ≤ n
adjacent-bounded v1 v0 = s ≤ s z ≤ n
adjacent-bounded v1 v1 = z ≤ n
adjacent-bounded v1 v2 = s ≤ s z ≤ n
adjacent-bounded v1 v3 = s ≤ s z ≤ n
adjacent-bounded v2 v0 = s ≤ s z ≤ n
adjacent-bounded v2 v1 = s ≤ s z ≤ n
adjacent-bounded v2 v2 = z ≤ n
adjacent-bounded v2 v3 = s ≤ s z ≤ n
adjacent-bounded v3 v0 = s ≤ s z ≤ n
adjacent-bounded v3 v1 = s ≤ s z ≤ n
adjacent-bounded v3 v2 = s ≤ s z ≤ n
adjacent-bounded v3 v3 = z ≤ n

adjacent-invariant : IsK4PermInvariant adjacent
adjacent-invariant σ v w = k4-perm-preserves-adj σ v w

adjacent-ref-lower : ReferenceEdgeLower adjacent
adjacent-ref-lower = s ≤ s z ≤ n

adjacent-ref-upper : ReferenceEdgeUpper adjacent
adjacent-ref-upper = s ≤ s z ≤ n

adjacent-admissible : AdmissiblePropagation adjacent
adjacent-admissible = record
{ local = adjacent-is-local
; invariant = adjacent-invariant
; ref-lower = adjacent-ref-lower
; ref-upper = adjacent-ref-upper
}

```

Summary and the forced edge weight

The record `ForcedPropagationResult` packages the conclusion: adjacency is admissible, every admissible law coincides with it, and every admissible edge weight is therefore 1.

```

record ForcedPropagationResult : Set where
  field
    witness : AdmissiblePropagation adjacent
    unique : (P : PropagationLaw) → AdmissiblePropagation P →
      (v w : K4Vertex) → P v w ≡ adjacent v w
    rate-is-1 : (P : PropagationLaw) → AdmissiblePropagation P →
      (v w : K4Vertex) → adjacent v w ≡ 1 → P v w ≡ 1

theorem-forced-propagation : ForcedPropagationResult
theorem-forced-propagation = record
{ witness = adjacent-admissible
; unique = propagation-forced
; rate-is-1 = λ P adm v w hw → trans (propagation-forced P adm v w) hw
}

```

The constant edge-weight merely names the forced value 1. The closing lemma identifies it with the previously computed max-propagation-per-edge.

```

edge-weight : ℕ
edge-weight = 1

edge-weight-is-propagation-rate : edge-weight ≡ max-propagation-per-edge
edge-weight-is-propagation-rate = refl

```


Chapter 60

Forced Minimal Action

At the tetrahedral level there are four triangular faces, one opposite each vertex. An action law assigns a natural-number weight to each face, and the same symmetry-and-sandwich argument used for propagation applies again: invariance identifies all face values, while the reference lower and upper bounds fix the common value at 1.

The candidate space is left unrestricted. The admissible record stores only invariance together with the two bounds on a reference face, and the global consequences are recovered afterward.

Triangles and action candidates

Faces are represented by the unique vertex they omit. This makes the permutation action immediate: a vertex permutation acts on a face by permuting the omitted vertex.

The representation of faces by omitted vertices makes the symmetry transparent. Defining `K4Triangle` to be the same type as `K4Vertex` means that the permutation action on faces is simply the already available action on vertices. No separate combinatorics of three-element subsets is needed.

`K4Triangle` : Set

`K4Triangle` = `K4Vertex`

`triangle-map` : `K4Perm` $\rightarrow$ `K4Triangle` $\rightarrow$ `K4Triangle`

`triangle-map` σ t = `k4to` σ t

An action law is an arbitrary function on faces. The auxiliary predicates express the global notions of non-triviality, lower bounds, and minimality, while the admissible record itself stores only invariance together with the two bounds on the reference face `tri-012`.

```
ActionLaw : Set
ActionLaw = K4Triangle → ℕ
```

```
tri-012 : K4Triangle
tri-012 = v3
```

```
tri-013 : K4Triangle
tri-013 = v2
```

```
tri-023 : K4Triangle
tri-023 = v1
```

```
tri-123 : K4Triangle
tri-123 = v0
```

Once invariance is available, the two reference inequalities can be transported to every face, and non-triviality becomes a consequence rather than an extra axiom.

```
IsNonTrivial : ActionLaw → Set
IsNonTrivial A = (t : K4Triangle) → A t ≡ 0 → ⊥
```

```
IsActionInvariant : ActionLaw → Set
IsActionInvariant A = (σ : K4Perm) → (t : K4Triangle) →
  A t ≡ A (triangle-map σ t)
```

```
ActionBound : ActionLaw → Set
ActionBound A = (t : K4Triangle) → 1 ≤ A t
```

```
ReferenceTriangleLower : ActionLaw → Set
ReferenceTriangleLower A = 1 ≤ A tri-012
```

```
IsMinimal : ActionLaw → Set
IsMinimal A = (t : K4Triangle) → A t ≤ 1
```

```
ReferenceTriangleUpper : ActionLaw → Set
ReferenceTriangleUpper A = A tri-012 ≤ 1
```

```
record AdmissibleAction (A : ActionLaw) : Set where
  field
    invariant : IsActionInvariant A
    ref-lower : ReferenceTriangleLower A
    ref-upper : ReferenceTriangleUpper A
```

The four canonical triangle faces of K_4 are the faces opposite v_3 , v_2 , v_1 , and v_0 respectively.

Existence: the unit action law

Existence is witnessed by the constant action law with value 1. It is invariant by definition and satisfies both reference bounds immediately.

```
unit-action : ActionLaw
unit-action _ = 1
```



```

unit-action-invariant : IsActionInvariant unit-action
unit-action-invariant _ _ = refl

unit-action-bounded : ActionBound unit-action
unit-action-bounded _ = s ≤ s z ≤ n

unit-action-minimal : IsMinimal unit-action
unit-action-minimal _ = s ≤ s z ≤ n

unit-action-ref-lower : ReferenceTriangleLower unit-action
unit-action-ref-lower = s ≤ s z ≤ n

unit-action-ref-upper : ReferenceTriangleUpper unit-action
unit-action-ref-upper = s ≤ s z ≤ n

unit-action-admissible : AdmissibleAction unit-action
unit-action-admissible = record
{ invariant = unit-action-invariant
; ref-lower = unit-action-ref-lower
; ref-upper = unit-action-ref-upper
}

```

Invariance and the Sandwich Pin a Single Value

The proof has the same shape as before. The lemma `action-from-012` transports the value of the reference face to an arbitrary face, the transported bounds give $1 \leq r \leq 1$, and antisymmetry identifies the common value with 1.

```

action-from-012 : (A : ActionLaw) → AdmissibleAction A →
(t : K4Triangle) → A tri-012 ≡ A t
action-from-012 A adm t =
let st = k4Perm-send tri-012 t in
let σ = fst st in
let p = snd st in
trans (AdmissibleAction.invariant adm σ tri-012)
(cong A p)

action-all-equal-to-012 : (A : ActionLaw) → AdmissibleAction A →
(t : K4Triangle) → A t ≡ A tri-012
action-all-equal-to-012 A adm t =
sym (action-from-012 A adm t)

action-global-value : (A : ActionLaw) → AdmissibleAction A →
(t1 t2 : K4Triangle) → A t1 ≡ A t2
action-global-value A adm t1 t2 =
trans (action-all-equal-to-012 A adm t1)
(action-from-012 A adm t2)

```

Invariance transports the brackets. The lower and upper bounds are imposed only on the reference face `tri-012`. Invariance transports them to every face.

```

action-bound : (A : ActionLaw) → AdmissibleAction A → ActionBound A
action-bound A adm t =
  subst (λ n → 1 ≤ n)
    (action-from-012 A adm t)
  (AdmissibleAction.ref-lower adm)

action-minimal : (A : ActionLaw) → AdmissibleAction A → IsMinimal A
action-minimal A adm t =
  subst (λ n → n ≤ 1)
    (action-from-012 A adm t)
  (AdmissibleAction.ref-upper adm)

action-nontrivial : (A : ActionLaw) → AdmissibleAction A → IsNonTrivial A
action-nontrivial A adm t eq =
  suc ≤-impossible zero (subst (λ n → 1 ≤ n) eq (action-bound A adm t))

```

The sandwich. The two reference inequalities force $A \text{ tri-012} = 1$.

```

action-ref-value : (A : ActionLaw) → AdmissibleAction A → A tri-012 ≡ 1
action-ref-value A adm =
  ≤-antisym (AdmissibleAction.ref-upper adm)
    (AdmissibleAction.ref-lower adm)

```

Assembly. Every face has value 1: invariance reduces the problem to the reference face, and the sandwich fixes that value.

```

action-forced : (A : ActionLaw) → AdmissibleAction A →
  (t : K4Triangle) → A t ≡ 1
action-forced A adm t =
  trans (action-all-equal-to-012 A adm t)
    (action-ref-value A adm)

```

The summary record packages the existence witness, the global-value theorem, and the uniqueness statement.

```

record ForcedActionResult : Set where
  field
    witness : AdmissibleAction unit-action
    global-value : (A : ActionLaw) → AdmissibleAction A →
      (t1 t2 : K4Triangle) → A t1 ≡ A t2
    unique : (A : ActionLaw) → AdmissibleAction A →
      (t : K4Triangle) → A t ≡ 1

theorem-forced-action : ForcedActionResult
theorem-forced-action = record
  { witness = unit-action-admissible
  ; global-value = action-global-value
  ; unique = action-forced
  }

```

The forced face weight

The constant `face-weight` merely names the forced value 1. The closing equality with `triangle-loop-order` identifies it with the previously computed combinatorial length scale.

`face-weight` : $\mathbb{N}$

`face-weight` = 1

`face-weight-is-loop-order` : `face-weight` $\equiv$ `triangle-loop-order`

`face-weight-is-loop-order` = `refl`

Chapter 61

Forced Vertex Weight

The three forced constants live on the three simplicial dimensions of K_4 :

dim 1	(edges)	$\longrightarrow$	<i>edge-weight</i>
dim 2	(faces)	$\longrightarrow$	<i>face-weight</i>
dim 0	(vertices)	$\longrightarrow$	<i>vertex-weight</i>

Propagation assigns a weight to each edge and the action law assigns a weight to each face. The remaining case is the 0-dimensional one: the assignment of a weight to each vertex.

A curvature law is any function $K_4\text{Vertex} \rightarrow \mathbb{N}$, subject to four constraints—each of which admits genuine alternatives on its own:

1. *Non-triviality* — every vertex carries positive curvature.
2. *K_4 -invariance* — the law is preserved under every vertex permutation.
3. *Lower bound* — every vertex carries curvature at least 1.
4. *Minimality* — no vertex carries curvature above 1.

The vertex case is the 0-dimensional analogue of the previous two chapters. The candidate space is again unrestricted, invariance reduces all values to a reference value, and the lower and upper bounds at that reference vertex force the common value to be 1.

Curvature candidates

A curvature law is an arbitrary function $K_4\text{Vertex} \rightarrow \mathbb{N}$. If the value 1 is to emerge, it must come from the admissibility conditions and not from the choice of type.

```
CurvatureLaw : Set
CurvatureLaw = K4Vertex → ℕ
```

The auxiliary predicates mirror those of the face case. As before, the admissible record remembers only invariance and the lower and upper bounds on the reference vertex v_0 ; the corresponding global versions are derived afterward.

```
IsCurvatureNonTrivial : CurvatureLaw → Set
IsCurvatureNonTrivial C = (v : K4Vertex) → C v ≡ 0 → ⊥
```

```
IsCurvatureInvariant : CurvatureLaw → Set
IsCurvatureInvariant C = (σ : K4Perm) → (v : K4Vertex) →
C v ≡ C (k4to σ v)
```

```
CurvatureBound : CurvatureLaw → Set
CurvatureBound C = (v : K4Vertex) → 1 ≤ C v
```

```
ReferenceVertexLower : CurvatureLaw → Set
ReferenceVertexLower C = 1 ≤ C v0
```

```
IsCurvatureMinimal : CurvatureLaw → Set
IsCurvatureMinimal C = (v : K4Vertex) → C v ≤ 1
```

```
ReferenceVertexUpper : CurvatureLaw → Set
ReferenceVertexUpper C = C v0 ≤ 1
```

```
record AdmissibleCurvature (C : CurvatureLaw) : Set where
field
  invariant : IsCurvatureInvariant C
  ref-lower : ReferenceVertexLower C
  ref-upper : ReferenceVertexUpper C
```

Existence: the unit curvature law

Existence is again given by the constant law with value 1. It is invariant, and the two reference bounds hold immediately.

```
unit-curvature : CurvatureLaw
unit-curvature _ = 1

unit-curvature-invariant : IsCurvatureInvariant unit-curvature
unit-curvature-invariant _ = refl

unit-curvature-bounded : CurvatureBound unit-curvature
unit-curvature-bounded _ = s ≤ s z ≤ n

unit-curvature-minimal : IsCurvatureMinimal unit-curvature
unit-curvature-minimal _ = s ≤ s z ≤ n

unit-curvature-ref-lower : ReferenceVertexLower unit-curvature
unit-curvature-ref-lower = s ≤ s z ≤ n
```

```

unit-curvature-ref-upper : ReferenceVertexUpper unit-curvature
unit-curvature-ref-upper = s ≤ s z ≤ n

unit-curvature-admissible : AdmissibleCurvature unit-curvature
unit-curvature-admissible = record
{ invariant = unit-curvature-invariant
; ref-lower = unit-curvature-ref-lower
; ref-upper = unit-curvature-ref-upper
}

```

Invariance and the Sandwich Pin a Single Value

The proof has exactly the same structure as the previous two chapters. The lemma `curvature-from-v0` transports the value at the reference vertex to any other vertex, the transported bounds give $1 \leq r \leq 1$, and antisymmetry identifies the common value with 1.

```

curvature-from-v0 : (C : CurvatureLaw) → AdmissibleCurvature C →
(v : K4Vertex) → C v0 ≡ C v
curvature-from-v0 C adm v =
let st = k4Perm-send v0 v in
let σ = fst st in
let p = snd st in
trans (AdmissibleCurvature.invariant adm σ v0)
(cong C p)

curvature-all-equal-to-v0 : (C : CurvatureLaw) → AdmissibleCurvature C →
(v : K4Vertex) → C v ≡ C v0
curvature-all-equal-to-v0 C adm v =
sym (curvature-from-v0 C adm v)

curvature-global-value : (C : CurvatureLaw) → AdmissibleCurvature C →
(v1 v2 : K4Vertex) → C v1 ≡ C v2
curvature-global-value C adm u w =
trans (curvature-all-equal-to-v0 C adm u)
(curvature-from-v0 C adm w)

```

Invariance transports the brackets. The lower and upper brackets are imposed only on the reference vertex v_0 . Invariance transports them to every vertex.

```

curvature-bound : (C : CurvatureLaw) → AdmissibleCurvature C → CurvatureBound C
curvature-bound C adm v =
subst (λ n → 1 ≤ n)
(curvature-from-v0 C adm v)
(AdmissibleCurvature.ref-lower adm)

curvature-minimal : (C : CurvatureLaw) → AdmissibleCurvature C → IsCurvatureMinimal C
curvature-minimal C adm v =
subst (λ n → n ≤ 1)

```

```

(curvature-from-v0 C adm v)
(AdmissibleCurvature.ref-upper adm)

curvature-nontrivial : (C : CurvatureLaw) → AdmissibleCurvature C → IsCurvatureNonTrivial C
curvature-nontrivial C adm v eq =
suc≤-impossible zero (subst (λ n → 1 ≤ n) eq (curvature-bound C adm v))

```

The sandwich. The lower bound gives $1 \leq C v_0$. The upper bound gives $C v_0 \leq 1$. Antisymmetry closes to $C v_0 \equiv 1$.

```

curvature-ref-value : (C : CurvatureLaw) → AdmissibleCurvature C → C v0 ≡ 1
curvature-ref-value C adm =
≤-antisym (AdmissibleCurvature.ref-upper adm)
           (AdmissibleCurvature.ref-lower adm)

```

Assembly. Every vertex equals 1: invariance collapses to the reference value, the sandwich pins that value.

```

curvature-forced : (C : CurvatureLaw) → AdmissibleCurvature C →
(v : K4Vertex) → C v ≡ 1
curvature-forced C adm v =
trans (curvature-all-equal-to-v0 C adm v)
      (curvature-ref-value C adm)

```

The forced vertex weight

The record `ForcedCurvatureResult` has the same form as in the previous chapters: an existence witness, a global-value theorem, and a uniqueness statement. The definition of vertex-weight only names the value that has already been forced.

```

record ForcedCurvatureResult : Set where
field
  witness : AdmissibleCurvature unit-curvature
  global-value : (C : CurvatureLaw) → AdmissibleCurvature C →
    (v1 v2 : K4Vertex) → C v1 ≡ C v2
  unique : (C : CurvatureLaw) → AdmissibleCurvature C →
    (v : K4Vertex) → C v ≡ 1

theorem-forced-curvature : ForcedCurvatureResult
theorem-forced-curvature = record
{ witness = unit-curvature-admissible
; global-value = curvature-global-value
; unique = curvature-forced
}

vertex-weight : ℕ
vertex-weight = 1

vertex-weight-forced : (C : CurvatureLaw) → AdmissibleCurvature C → C v0 ≡ vertex-weight
vertex-weight-forced C adm = curvature-ref-value C adm

```


Filter audit: sufficiency versus minimality

The three preceding limit theorems secure their constants through structural orbit-transport combined with a local sandwich. A severe test remains: are these admissibility filters truly minimal, or does logical bloat exist? If a filter could be derived purely from the others, the filter set itself would not be a primitive structural law, merely an overlapping mathematical description.

The next question concerns the choice of admissibility basis itself. The lemmas in this section show why the record stores only reference bounds together with invariance. If one imposed global pointwise lower and upper bounds from the outset, then several uniqueness statements would become trivial and invariance would cease to play an essential role. The chosen formulation avoids that collapse by keeping the bounds local and letting symmetry do the transport.

propagation-forced-no-invariance :

```
(P : PropagationLaw) → IsLocal P → HasEdgeSupport P → IsBounded P →
(v w : K4Vertex) → P v w ≡ adjacent v w
propagation-forced-no-invariance P local support upper v w with K4-decEq v w
... | inj1 refl = trans (local v) (sym (adj-self v))
... | inj2 ne = trans (≤-antisym (upper v w) (support v w ne))
(sym (K4-distinct-adj v w ne))
```

```
action-nontrivial-from-bound : (A : ActionLaw) → ActionBound A → IsNonTrivial A
action-nontrivial-from-bound A lower t eq =
suc≤-impossible zero (subst (λ n → 1 ≤ n) eq (lower t))
```

action-forced-no-invariance :

```
(A : ActionLaw) → ActionBound A → IsMinimal A →
(t : K4Triangle) → A t ≡ 1
action-forced-no-invariance A lower upper t =
≤-antisym (upper t) (lower t)
```

curvature-nontrivial-from-bound :

```
(C : CurvatureLaw) → CurvatureBound C → IsCurvatureNonTrivial C
curvature-nontrivial-from-bound C lower v eq =
suc≤-impossible zero (subst (λ n → 1 ≤ n) eq (lower v))
```

curvature-forced-no-invariance :

```
(C : CurvatureLaw) → CurvatureBound C → IsCurvatureMinimal C →
(v : K4Vertex) → C v ≡ 1
curvature-forced-no-invariance C lower upper v =
≤-antisym (upper v) (lower v)
```

The force of these lemmas is conceptual, not destructive. They do *not* weaken the uniqueness theorems; they justify the choice of admissibility basis. The constant chapters prove uniqueness with a genuinely non-pointwise admissibility architecture, and these lemmas confirm that the pointwise alternative would have been redundant. The question of whether an alternative set of predicates could achieve the same singleton collapse belongs to the meta-theory of filter selection rather than to the elimination itself.

Minimal admissibility basis

While the architecture uses four foundational criteria for edge propagation and three for action and curvature natively, its philosophical minimality must be formally established computationally. Each singular primitive clause must be objectively indispensable. When exactly one is deleted, exactly one rogue geometric map structurally survives.

Minimality is proved here by explicit counterexamples. For each omitted filter a concrete rival law is exhibited that satisfies all the remaining hypotheses but fails the desired conclusion. In this way the architecture is justified not by general rhetoric about necessity but by precise witnesses showing what survives when a single condition is removed.

Propagation basis. Without locality, the everywhere-one law survives. Without invariance, a law can single out the reference edge and ignore the rest. Without the lower bracket, the zero law survives. Without the upper bracket, the doubled adjacency law survives.

propagation-all-one : PropagationLaw

propagation-all-one $_ _ = 1$

propagation-all-one-invariant : IsK4PermInvariant propagation-all-one

propagation-all-one-invariant $_ _ _ = \text{refl}$

propagation-all-one-ref-lower : ReferenceEdgeLower propagation-all-one

propagation-all-one-ref-lower $= s \leq s \ z \leq n$

propagation-all-one-ref-upper : ReferenceEdgeUpper propagation-all-one

propagation-all-one-ref-upper $= s \leq s \ z \leq n$

propagation-all-one-not-adj : propagation-all-one $v_0 \ v_0 \equiv \text{adjacent } v_0 \ v_0 \rightarrow \perp$

propagation-all-one-not-adj $()$

propagation-reference-only : PropagationLaw

propagation-reference-only $v_0 \ v_1 = 1$

propagation-reference-only $_ _ = 0$

propagation-reference-only-local : IsLocal propagation-reference-only

propagation-reference-only-local $v_0 = \text{refl}$

propagation-reference-only-local $v_1 = \text{refl}$

propagation-reference-only-local $v_2 = \text{refl}$

propagation-reference-only-local $v_3 = \text{refl}$

propagation-reference-only-ref-lower : ReferenceEdgeLower propagation-reference-only

propagation-reference-only-ref-lower $= s \leq s \ z \leq n$

propagation-reference-only-ref-upper : ReferenceEdgeUpper propagation-reference-only

propagation-reference-only-ref-upper $= s \leq s \ z \leq n$

```

propagation-reference-only-not-adj :
  propagation-reference-only  $v_0 v_2 \equiv \text{adjacent } v_0 v_2 \rightarrow \perp$ 
  propagation-reference-only-not-adj ()

propagation-zero : PropagationLaw
propagation-zero  $\_ \_ = 0$ 

propagation-zero-local : IsLocal propagation-zero
propagation-zero-local  $v_0 = \text{refl}$ 
propagation-zero-local  $v_1 = \text{refl}$ 
propagation-zero-local  $v_2 = \text{refl}$ 
propagation-zero-local  $v_3 = \text{refl}$ 

propagation-zero-invariant : IsK4PermInvariant propagation-zero
propagation-zero-invariant  $\_ \_ \_ = \text{refl}$ 

propagation-zero-ref-upper : ReferenceEdgeUpper propagation-zero
propagation-zero-ref-upper =  $z \leq n$ 

propagation-zero-not-adj : propagation-zero  $v_0 v_1 \equiv \text{adjacent } v_0 v_1 \rightarrow \perp$ 
propagation-zero-not-adj ()

propagation-double : PropagationLaw
propagation-double  $v w = \text{adjacent } v w + \mathbb{N} \text{ adjacent } v w$ 

propagation-double-local : IsLocal propagation-double
propagation-double-local  $v =$ 
  subst ( $\lambda n \rightarrow n + \mathbb{N} n \equiv 0$ )
  (sym (adj-self  $v$ ))
  refl

propagation-double-invariant : IsK4PermInvariant propagation-double
propagation-double-invariant  $\sigma v w =$ 
  cong ( $\lambda n \rightarrow n + \mathbb{N} n$ ) (k4-perm-preserves-adj  $\sigma v w$ )

propagation-double-ref-lower : ReferenceEdgeLower propagation-double
propagation-double-ref-lower =  $s \leq s z \leq n$ 

propagation-double-not-adj : propagation-double  $v_0 v_1 \equiv \text{adjacent } v_0 v_1 \rightarrow \perp$ 
propagation-double-not-adj ()

```

Action basis. Without invariance, the reference face can be privileged while the others collapse to zero. Without the lower bracket, the zero law survives. Without the upper bracket, the constant-two law survives.

```

action-reference-only : ActionLaw
action-reference-only  $v_3 = 1$ 
action-reference-only  $\_ = 0$ 

action-reference-only-ref-lower : ReferenceTriangleLower action-reference-only
action-reference-only-ref-lower =  $s \leq s z \leq n$ 

action-reference-only-ref-upper : ReferenceTriangleUpper action-reference-only

```

```

action-reference-only-ref-upper =  $s \leq s \leq n$ 

action-reference-only-not-forced : action-reference-only tri-013  $\equiv 1 \rightarrow \perp$ 
action-reference-only-not-forced ()

action-zero : ActionLaw
action-zero _ = 0

action-zero-invariant : IsActionInvariant action-zero
action-zero-invariant _ _ = refl

action-zero-ref-upper : ReferenceTriangleUpper action-zero
action-zero-ref-upper =  $z \leq n$ 

action-zero-not-forced : action-zero tri-012  $\equiv 1 \rightarrow \perp$ 
action-zero-not-forced ()

action-two : ActionLaw
action-two _ = 2

action-two-invariant : IsActionInvariant action-two
action-two-invariant _ _ = refl

action-two-ref-lower : ReferenceTriangleLower action-two
action-two-ref-lower =  $s \leq s \leq n$ 

action-two-not-forced : action-two tri-012  $\equiv 1 \rightarrow \perp$ 
action-two-not-forced ()

```

Curvature basis. The same pattern repeats on the 0-simplices. Without invariance, the reference vertex can be privileged. Without the lower bracket, the zero law survives. Without the upper bracket, the constant-two law survives.

```

curvature-reference-only : CurvatureLaw
curvature-reference-only  $v_0 = 1$ 
curvature-reference-only _ = 0

curvature-reference-only-ref-lower : ReferenceVertexLower curvature-reference-only
curvature-reference-only-ref-lower =  $s \leq s \leq n$ 

curvature-reference-only-ref-upper : ReferenceVertexUpper curvature-reference-only
curvature-reference-only-ref-upper =  $s \leq s \leq n$ 

curvature-reference-only-not-forced : curvature-reference-only  $v_1 \equiv 1 \rightarrow \perp$ 
curvature-reference-only-not-forced ()

curvature-zero : CurvatureLaw
curvature-zero _ = 0

curvature-zero-invariant : IsCurvatureInvariant curvature-zero
curvature-zero-invariant _ _ = refl

curvature-zero-ref-upper : ReferenceVertexUpper curvature-zero
curvature-zero-ref-upper =  $z \leq n$ 

```

```

curvature-zero-not-forced : curvature-zero  $v_0 \equiv 1 \rightarrow \perp$ 
curvature-zero-not-forced ()

curvature-two : CurvatureLaw
curvature-two _ = 2

curvature-two-invariant : IsCurvatureInvariant curvature-two
curvature-two-invariant _ = refl

curvature-two-ref-lower : ReferenceVertexLower curvature-two
curvature-two-ref-lower =  $s \leq z \leq n$ 

curvature-two-not-forced : curvature-two  $v_0 \equiv 1 \rightarrow \perp$ 
curvature-two-not-forced ()

```

These witnesses prove that the admissibility basis is irreducible: every primitive clause excludes a live rival that the remaining clauses cannot block. Removing any single clause breaks the singleton collapse. Nothing remaining in the basis is decorative.

Filter minimality: invariance is non-redundant. A sharper question: is the K_4 -invariance filter genuinely eliminative, or do the other three filters (locality/non-triviality, lower bound, upper bound) already force the unit constant? The answer is no—without invariance, a rival candidate survives in each simplicial dimension.

For propagation, the reference-only law $P(v_0, v_1) = 1$, $P = 0$ elsewhere, satisfies locality, the lower bracket on the reference edge, and the upper bracket—but is not K_4 -invariant. Invariance is therefore the filter that blocks this candidate. For the action and curvature, the same pattern repeats: the reference-only face law and the reference-only vertex law satisfy every filter except invariance.

These examples isolate the role of invariance very clearly. In each simplicial dimension there is a reference-only law that obeys the local conditions and the two reference bounds but fails to spread uniformly across the orbit. Invariance is exactly the hypothesis that excludes such laws.

```

-- §§ Filter minimality: invariance is the critical filter in each dimension.

-- § Propagation: without invariance, the reference-only law survives.
propagation-needs-invariance :
  IsLocal propagation-reference-only ×
  ReferenceEdgeLower propagation-reference-only ×
  ReferenceEdgeUpper propagation-reference-only ×
  (propagation-reference-only  $v_0 v_2 \equiv \text{adjacent } v_0 v_2 \rightarrow \perp$ )
propagation-needs-invariance =
  propagation-reference-only-local ,
  (propagation-reference-only-ref-lower ,
  (propagation-reference-only-ref-upper ,
  propagation-reference-only-not-adj))

```

```

-- § Action: without invariance, the reference-only face law survives.
action-needs-invariance :
  ReferenceTriangleLower action-reference-only ×
  ReferenceTriangleUpper action-reference-only ×
  (action-reference-only tri-013 ≡ 1 → ⊥)
action-needs-invariance =
  action-reference-only-ref-lower ,
  (action-reference-only-ref-upper ,
  action-reference-only-not-forced)

-- § Curvature: without invariance, the reference-only vertex law survives.
curvature-needs-invariance :
  ReferenceVertexLower curvature-reference-only ×
  ReferenceVertexUpper curvature-reference-only ×
  (curvature-reference-only v1 ≡ 1 → ⊥)
curvature-needs-invariance =
  curvature-reference-only-ref-lower ,
  (curvature-reference-only-ref-upper ,
  curvature-reference-only-not-forced)

```

Invariance blocks the reference-only rival in all three simplicial dimensions. The same argument structure extends to the remaining filters: in each case, a concrete counterexample passes all filters but one, proving that filter essential. The derivation closes when every filter is traced to a graph-theoretic property of K_4 .

Filter derivation: the lower bound is non-redundant. The zero law assigns weight 0 to every simplex. It satisfies locality (propagation: $P(v, v) = 0$ trivially), K_4 -invariance (trivially), and the upper bracket ($0 \leq 1$). It fails the lower bracket: $1 \leq 0$ is false. Without the lower bound, the zero law survives in every simplicial dimension.

The lower bound is not a design choice. K_4 is the complete graph on four vertices: every possible edge, face, and vertex exists. A weight of 0 on an existing simplex declares it structurally absent—it carries no content, contributes nothing to the simplicial complex. This contradicts completeness. Non-degeneracy (≥ 1 on the reference simplex) is forced by the fact that K_4 has no missing simplices.

Filter derivation: the upper bound is non-redundant. The doubled law (propagation: $P(v, w) = 2 \cdot \text{adjacent}(v, w)$) and the constant-2 law (action, curvature) are invariant and non-degenerate. They satisfy locality (propagation) and the lower bracket. They fail the upper bracket: $2 \leq 1$ is false. Without the upper bound, these rivals survive.

The upper bound is forced by the zero-parameter constraint. In $\mathbb{N}$, once non-degeneracy fixes a lower bound of 1, any value $n > 1$ requires an external input distinguishing it from 1. That input would be a free parameter. Since the system admits

none, the weight collapses to the minimal non-degenerate value. Minimality (≤ 1 on the reference simplex) is the arithmetical shadow of having zero free parameters.

Filter derivation: locality is non-redundant (propagation). The constant-1 law assigns weight 1 to every vertex pair, including the diagonal. It is K_4 -invariant, non-degenerate, and minimal. It fails locality: $P(v, v) = 1 \neq 0$. Without locality, this rival survives in the propagation dimension.

Locality is forced by K_4 being a simple graph. A simple graph has no self-loops: there is no edge from a vertex to itself. The propagation kernel must vanish on the diagonal because there is no simplex to propagate along. This constraint has no analogue in the action and curvature dimensions, where every simplex is a distinct combinatorial object.

The derivation theorem. Four structural properties of K_4 —transitivity of $\text{Aut}(K_4)$ on each simplicial orbit, completeness, simplicity, and the absence of free parameters—generate exactly the admissibility filters used in the three forced-constant chapters. Each property produces one filter:

K_4 property	Filter	Derived from
$\text{Aut}(K_4)$ transitivity	invariance	labeling independence
completeness (no missing simplices)	lower bound ≥ 1	non-degeneracy
zero free parameters	upper bound ≤ 1	minimality
simplicity (no self-loops)	locality ($P(v, v) = 0$)	propagation only

Completeness is proved by the three forced-constant theorems (each filter set forces a unique unit weight). Irredundancy is proved by ten essentiality witnesses: four for propagation, three each for action and curvature. Every proper subset of filters admits a concrete rival that the remaining filters cannot block.

Taken together, these results establish both sides of the claim. The three forcing theorems show completeness: once all admissibility data are present, only the unit law survives in each simplicial dimension. The ten essentiality witnesses show irredundancy: if any one condition is removed, a concrete rival remains. The admissibility basis is thus not an external layer placed over K_4 , but a minimal filter system recovered from the graph-theoretic properties listed above.

-- §§ Filter derivation: every filter is essential, every filter is derived.

-- § Propagation: without locality, the constant-1 law survives.

propagation-needs-locality :

IsK4PermInvariant propagation-all-one ×

```

ReferenceEdgeLower propagation-all-one ×
ReferenceEdgeUpper propagation-all-one ×
(propagation-all-one  $v_0 v_0 \equiv \text{adjacent } v_0 v_0 \rightarrow \perp$ )
propagation-needs-locality =
propagation-all-one-invariant ,
(propagation-all-one-ref-lower ,
(propagation-all-one-ref-upper ,
propagation-all-one-not-adj)))

-- § Propagation: without the lower bound, the zero law survives.
propagation-needs-lower :
IsLocal propagation-zero ×
IsK4PermInvariant propagation-zero ×
ReferenceEdgeUpper propagation-zero ×
(propagation-zero  $v_0 v_1 \equiv \text{adjacent } v_0 v_1 \rightarrow \perp$ )
propagation-needs-lower =
propagation-zero-local ,
(propagation-zero-invariant ,
(propagation-zero-ref-upper ,
propagation-zero-not-adj)))

-- § Propagation: without the upper bound, the doubled law survives.
propagation-needs-upper :
IsLocal propagation-double ×
IsK4PermInvariant propagation-double ×
ReferenceEdgeLower propagation-double ×
(propagation-double  $v_0 v_1 \equiv \text{adjacent } v_0 v_1 \rightarrow \perp$ )
propagation-needs-upper =
propagation-double-local ,
(propagation-double-invariant ,
(propagation-double-ref-lower ,
propagation-double-not-adj)))

-- § Action: without the lower bound, the zero action survives.
action-needs-lower :
IsActionInvariant action-zero ×
ReferenceTriangleUpper action-zero ×
(action-zero  $\text{tri-012} \equiv 1 \rightarrow \perp$ )
action-needs-lower =
action-zero-invariant ,
(action-zero-ref-upper ,
action-zero-not-forced)

-- § Action: without the upper bound, the constant-2 action survives.
action-needs-upper :
IsActionInvariant action-two ×
ReferenceTriangleLower action-two ×
(action-two  $\text{tri-012} \equiv 1 \rightarrow \perp$ )
action-needs-upper =
action-two-invariant ,
(action-two-ref-lower ,
action-two-not-forced)

-- § Curvature: without the lower bound, the zero curvature survives.

```



```

curvature-needs-lower :
  IsCurvatureInvariant curvature-zero ×
  ReferenceVertexUpper curvature-zero ×
  (curvature-zero  $v_0 \equiv 1 \rightarrow \perp$ )
curvature-needs-lower =
  curvature-zero-invariant ,
  (curvature-zero-ref-upper ,
  curvature-zero-not-forced)

-- § Curvature: without the upper bound, the constant-2 curvature survives.
curvature-needs-upper :
  IsCurvatureInvariant curvature-two ×
  ReferenceVertexLower curvature-two ×
  (curvature-two  $v_0 \equiv 1 \rightarrow \perp$ )
curvature-needs-upper =
  curvature-two-invariant ,
  (curvature-two-ref-lower ,
  curvature-two-not-forced)

```

The filter architecture is closed. Ten essentiality witnesses prove irredundancy; three forced-constant theorems prove completeness. The admissibility basis is the unique minimal complete filter set on the K_4 simplicial complex, derived from the graph-theoretic identity of K_4 as a complete, simple, vertex-transitive graph with no free parameters.

Chapter 62

Forced Edge and Matching Orbits

The previous chapter worked with ordered pairs of distinct vertices. Edges, however, carry no orientation in K_4 , because adjacency is symmetric. One therefore has to pass from ordered pairs to unordered pairs of distinct vertices and then ask how the automorphism group acts on these new objects. The same question appears one step higher for perfect matchings, which partition the four vertices into two disjoint pairs.

This chapter carries out that passage. It defines edges as unordered distinct pairs, proves that every edge is equivalent to one of the six standard ones, lifts the vertex action to edges, classifies the three perfect matchings, and shows that both edges and matchings form single $\text{Aut}(K_4)$ -orbits. The resulting orbit counts are then assembled into a single record and used in the rigidity arguments that follow.

Edges as unordered pairs

A K_4 edge is an unordered pair of distinct vertices. The distinctness witness excludes the diagonal, so the type matches the edge set of a simple graph.

The record `K4Edge` therefore stores two vertices together with a proof that they are different, and the relation `_≈E_` identifies the two possible orientations of the same geometric edge. The map `swap-edge` is the obvious involution reversing the order of the endpoints, and the basic lemmas simply record that this reversal does not change the edge class.

```
record K4Edge : Set where
  constructor edge
  field
    src : K4Vertex
    dst : K4Vertex
```

```

distinct : src ≠ dst

open K4Edge public

swap-edge : K4Edge → K4Edge
swap-edge e = record
{ src = dst e
; dst = src e
; distinct = λ p → distinct e (sym p)
}

data _≈E_ : K4Edge → K4Edge → Set where
same-orient : (e1 e2 : K4Edge) →
  src e1 ≡ src e2 → dst e1 ≡ dst e2 → e1 ≈E e2
swap-orient : (e1 e2 : K4Edge) →
  src e1 ≡ dst e2 → dst e1 ≡ src e2 → e1 ≈E e2

≈E-refl : (e : K4Edge) → e ≈E e
≈E-refl e = same-orient e e refl refl

swap-edge-≈E : (e : K4Edge) → swap-edge e ≈E e
swap-edge-≈E e = swap-orient (swap-edge e) e refl refl

swap-edge-involutive : (e : K4Edge) → swap-edge (swap-edge e) ≈E e
swap-edge-involutive e = same-orient (swap-edge (swap-edge e)) e refl refl

```

The six concrete edges of K_4 —all $\binom{4}{2}$ unordered pairs of vertices—are named once for use in the matching classification.

```

e01 e02 e03 e12 e13 e23 : K4Edge
e01 = edge v0 v1 (λ ())
e02 = edge v0 v2 (λ ())
e03 = edge v0 v3 (λ ())
e12 = edge v1 v2 (λ ())
e13 = edge v1 v3 (λ ())
e23 = edge v2 v3 (λ ())

```

These six names exhaust the edge type. The classification theorem checks all ordered vertex pairs, discards the diagonal cases, and groups the remaining pairs by orientation.

The proof of `edge-classify` is a complete case distinction on the sixteen ordered vertex pairs. Four diagonal cases are impossible by the distinctness witness, and the remaining twelve off-diagonal cases fall into six classes paired by orientation reversal. In this way the list of six edges is shown to be exhaustive rather than merely convenient.

```

edge-classify : (e : K4Edge) →
  (e ≈E e01) ⊔ (e ≈E e02) ⊔ (e ≈E e03) ⊔
  (e ≈E e12) ⊔ (e ≈E e13) ⊔ (e ≈E e23)
edge-classify (edge v0 v0 d) = ⊥-elim (d refl)
edge-classify (edge v0 v1 d) = inj1 (same-orient _ _ refl refl)

```

```

edge-classify (edge v0 v2 d) = inj2 (inj1 (same-orient __ refl refl))
edge-classify (edge v0 v3 d) = inj2 (inj2 (inj1 (same-orient __ refl refl)))
edge-classify (edge v1 v0 d) = inj1 (swap-orient __ refl refl)
edge-classify (edge v1 v1 d) =  $\perp$ -elim (d refl)
edge-classify (edge v1 v2 d) = inj2 (inj2 (inj1 (same-orient __ refl refl)))
edge-classify (edge v1 v3 d) = inj2 (inj2 (inj2 (inj1 (same-orient __ refl refl))))
edge-classify (edge v2 v0 d) = inj2 (inj1 (swap-orient __ refl refl))
edge-classify (edge v2 v1 d) = inj2 (inj2 (inj1 (swap-orient __ refl refl)))
edge-classify (edge v2 v2 d) =  $\perp$ -elim (d refl)
edge-classify (edge v2 v3 d) = inj2 (inj2 (inj2 (inj2 (same-orient __ refl refl))))
edge-classify (edge v3 v0 d) = inj2 (inj2 (inj1 (swap-orient __ refl refl)))
edge-classify (edge v3 v1 d) = inj2 (inj2 (inj2 (inj1 (swap-orient __ refl refl))))
edge-classify (edge v3 v2 d) = inj2 (inj2 (inj2 (inj2 (swap-orient __ refl refl))))
edge-classify (edge v3 v3 d) =  $\perp$ -elim (d refl)

```

The induced edge action

A permutation of vertices induces an action on edges by transporting both endpoints. Injectivity preserves distinctness, so the construction descends directly to the edge type.

The natural action on edges is obtained by transporting both endpoints. Injectivity of the underlying permutation preserves distinctness, so the result is again a valid edge. The lemma `edge-act-swap` records that this transported action is compatible with the orientation quotient, and `edge-act-id` is the expected normalization at the identity.

```

edge-act : K4Perm → K4Edge → K4Edge
edge-act σ e = record
  { src = k4to σ (src e)
  ; dst = k4to σ (dst e)
  ; distinct = λ p → distinct e (k4-injective σ (src e) (dst e) p)
  }

edge-act-swap : (σ : K4Perm) → (e : K4Edge) →
  edge-act σ (swap-edge e) ≈E swap-edge (edge-act σ e)
edge-act-swap σ e = same-orient __ refl refl

```

The identity permutation acts trivially on every edge, as expected.

```

id-perm : K4Perm
id-perm = record
  { k4to = λ v → v
  ; k4from = λ v → v
  ; k4-to-from = λ _ → refl
  ; k4-from-to = λ _ → refl
  }

edge-act-id : (e : K4Edge) → edge-act id-perm e ≈E e
edge-act-id e = same-orient __ refl refl

```

The induced action is unique among functions that transport the two endpoints of an edge, possibly with reversed orientation. The predicate `LiftsEdgeAction` isolates exactly these competitors, and the uniqueness proof is a case split on the orientation tag.

The predicate `LiftsEdgeAction` isolates exactly the admissible competitors: an action may preserve or reverse orientation, but it may not send an edge to anything outside the image of its two endpoints under the vertex permutation. Under this hypothesis, `edge-act-unique` is immediate by a two-case analysis on the orientation tag. Pointwise endpoint transport is therefore the only induced action on edges up to $\approx_E$.

```

LiftsEdgeAction : (K4Perm → K4Edge → K4Edge) → Set
LiftsEdgeAction act = (σ : K4Perm) → (e : K4Edge) →
  ((src (act σ e) ≡ k4to σ (src e)) × (dst (act σ e) ≡ k4to σ (dst e)))
  ⊔
  ((src (act σ e) ≡ k4to σ (dst e)) × (dst (act σ e) ≡ k4to σ (src e)))

edge-act-lifts : LiftsEdgeAction edge-act
edge-act-lifts σ e = inj₁ (refl , refl)

edge-act-unique : (act : K4Perm → K4Edge → K4Edge) →
  LiftsEdgeAction act →
  (σ : K4Perm) → (e : K4Edge) →
  act σ e ≈E edge-act σ e
edge-act-unique act lifts σ e with lifts σ e
... | inj₁ (ps , pd) = same-orient _ _ ps pd
... | inj₂ (ps , pd) = swap-orient _ _ ps pd

```

Every `K4Perm` has an inverse obtained by swapping its forward and backward maps.

```

k4Perm-inv : K4Perm → K4Perm
k4Perm-inv σ = record
{ k4to    = k4from σ
; k4from  = k4to σ
; k4-to-from = k4-from-to σ
; k4-from-to = k4-to-from σ
}

```

Single edge orbit

To show that edges form a single orbit, it suffices to send a fixed reference edge to the source and target and compose the resulting permutations. This is the same conjugation argument used earlier for ordered pairs.

The proof of edge transitivity is the standard conjugation argument. One chooses a permutation sending (v_0, v_1) to the endpoints of the source edge and another sending

(v_0, v_1) to the endpoints of the target edge; composing the second with the inverse of the first yields a permutation carrying one edge to the other. Hence all six edges lie in a single $\text{Aut}(K_4)$ -orbit.

```

edge-transitive : (e1 e2 : K4Edge) →
  Σ K4Perm (λ σ → edge-act σ e1 ≈E e2)
edge-transitive e1 e2 =
  let em1 = edge-map (src e1) (dst e1) (distinct e1)
      em2 = edge-map (src e2) (dst e2) (distinct e2)
      σ1 = fst em1
      p1s = fst (snd em1)
      p1d = snd (snd em1)
      σ2 = fst em2
      p2s = fst (snd em2)
      p2d = snd (snd em2)
      σ = k4Perm-comp σ2 (k4Perm-inv σ1)
      qs : k4to σ (src e1) ≡ src e2
      qs = trans (cong (k4to σ2)
                  (trans (cong (k4from σ1) (sym p1s))
                        (k4-from-to σ1 v0)))
                  p2s
      qd : k4to σ (dst e1) ≡ dst e2
      qd = trans (cong (k4to σ2)
                  (trans (cong (k4from σ1) (sym p1d))
                        (k4-from-to σ1 v1)))
                  p2d
  in (σ , same-orient (edge-act σ e1) e2 qs qd)

```

Perfect matchings of K_4

A perfect matching partitions the four vertices into two disjoint pairs. Once the partner of v_0 is fixed, the second pair is determined, so only three cases remain.

Once the partner of v_0 is chosen, the remaining edge is determined by the two leftover vertices. This leaves exactly three possibilities, recorded by the constructors of `K4Matching`. The function `matching-edges` simply unfolds each constructor into the corresponding pair of skew edges.

```

data K4Matching : Set where
  m01-23 : K4Matching
  m02-13 : K4Matching
  m03-12 : K4Matching

matching-edges : K4Matching → K4Edge × K4Edge
matching-edges m01-23 = e01 , e23
matching-edges m02-13 = e02 , e13
matching-edges m03-12 = e03 , e12

-- § Type-level closure: K4Matching has exactly three inhabitants.
K4Matching-exhaustive :

```

```

 $\forall (m : \text{K4Matching})$ 
 $\rightarrow (m \equiv \text{m01-23}) \uplus (m \equiv \text{m02-13}) \uplus (m \equiv \text{m03-12})$ 
K4Matching-exhaustive m01-23 = inj1 refl
K4Matching-exhaustive m02-13 = inj2 (inj1 refl)
K4Matching-exhaustive m03-12 = inj2 (inj2 refl)

```

The three constructors list the obvious candidates, but the classification theorem still has to show that there is no fourth matching. It does so by case analysis on the partner of v_0 .

To prove exhaustion, the development widens the search space to `MatchingCandidate`, whose only datum is the chosen partner of v_0 together with a proof that this choice is distinct from v_0 . Case distinction on that partner does the rest: the choice v_0 is impossible, while the remaining three choices correspond exactly to the three named matchings. In this sense the classification theorem is a genuine exhaustion result rather than a restatement of the data type.

```

record MatchingCandidate : Set where
  field
    partner-of-v0 : K4Vertex
    distinct0 : v0 ≠ partner-of-v0

open MatchingCandidate public

matching-classify : (c : MatchingCandidate) →
  (partner-of-v0 c ≡ v1) ⊔
  (partner-of-v0 c ≡ v2) ⊔
  (partner-of-v0 c ≡ v3)

matching-classify
  record { partner-of-v0 = v0 ; distinct0 = d } = ⊥-elim (d refl)
matching-classify
  record { partner-of-v0 = v1 ; distinct0 = _ } = inj1 refl
matching-classify
  record { partner-of-v0 = v2 ; distinct0 = _ } = inj2 (inj1 refl)
matching-classify
  record { partner-of-v0 = v3 ; distinct0 = _ } = inj2 (inj2 refl)

candidate→matching : MatchingCandidate → K4Matching
candidate→matching c with matching-classify c
... | inj1 _      = m01-23
... | inj2 (inj1 _) = m02-13
... | inj2 (inj2 _) = m03-12

matching→candidate : K4Matching → MatchingCandidate
matching→candidate m01-23 = record { partner-of-v0 = v1 ; distinct0 = λ () }
matching→candidate m02-13 = record { partner-of-v0 = v2 ; distinct0 = λ () }
matching→candidate m03-12 = record { partner-of-v0 = v3 ; distinct0 = λ () }

candidate-section : (m : K4Matching) →
  candidate→matching (matching→candidate m) ≡ m
candidate-section m01-23 = refl
candidate-section m02-13 = refl
candidate-section m03-12 = refl

```


The three transpositions

Matching transitivity is witnessed by three explicit transpositions: $(1\ 2)$, $(1\ 3)$, and $(2\ 3)$. They fix v_0 and permute the remaining labels, which is enough to move between the three matchings.

These transpositions are all that is needed for the orbit computation. Each fixes v_0 and permutes two of the remaining three vertices, so together with the identity they connect every choice of partner for v_0 with every other choice. Their definitions are completely explicit, and the round-trip laws close by inspection.

```

swap-12-perm : K4Perm
swap-12-perm = record
  { k4to    = sw
  ; k4from  = sw
  ; k4-to-from = swsw
  ; k4-from-to = swsw
  }
where
  sw : K4Vertex → K4Vertex
  sw v0 = v0
  sw v1 = v2
  sw v2 = v1
  sw v3 = v3
  swsw : (v : K4Vertex) → sw (sw v) ≡ v
  swsw v0 = refl
  swsw v1 = refl
  swsw v2 = refl
  swsw v3 = refl

```

```

swap-13-perm : K4Perm
swap-13-perm = record
  { k4to    = sw
  ; k4from  = sw
  ; k4-to-from = swsw
  ; k4-from-to = swsw
  }
where
  sw : K4Vertex → K4Vertex
  sw v0 = v0
  sw v1 = v3
  sw v2 = v2
  sw v3 = v1
  swsw : (v : K4Vertex) → sw (sw v) ≡ v
  swsw v0 = refl
  swsw v1 = refl
  swsw v2 = refl
  swsw v3 = refl

```

```

swap-23-perm : K4Perm
swap-23-perm = record
  { k4to    = sw

```

```

; k4from = sw
; k4-to-from = swsw
; k4-from-to = swsw
}
where
sw : K4Vertex → K4Vertex
sw v0 = v0
sw v1 = v1
sw v2 = v3
sw v3 = v2
swsw : (v : K4Vertex) → sw (sw v) ≡ v
swsw v0 = refl
swsw v1 = refl
swsw v2 = refl
swsw v3 = refl

```

Single matching orbit

The three explicit transpositions connect any matching with any other. Diagonal cases use the identity permutation, and off-diagonal cases use one of the three swaps.

The orbit computation is therefore finite and explicit. The diagonal cases use the identity permutation, and each off-diagonal case is handled by the appropriate transposition. Consequently the three perfect matchings form a single orbit under $\text{Aut}(K_4)$.

```

MatchingOrbitWitness : K4Matching → K4Matching → Set
MatchingOrbitWitness m1 m2 =
  ∑ K4Perm (λ σ →
    (edge-act σ (fst (matching-edges m1)) ≈E fst (matching-edges m2))
    × (edge-act σ (snd (matching-edges m1)) ≈E snd (matching-edges m2)))

matching-transitive : (m1 m2 : K4Matching) → MatchingOrbitWitness m1 m2
matching-transitive m01-23 m01-23 =
  id-perm ,
  ( same-orient __ refl refl
  , same-orient __ refl refl )
matching-transitive m01-23 m02-13 =
  swap-12-perm ,
  ( same-orient __ refl refl
  , same-orient __ refl refl )
matching-transitive m01-23 m03-12 =
  swap-13-perm ,
  ( same-orient __ refl refl
  , swap-orient __ refl refl )
matching-transitive m02-13 m01-23 =
  swap-12-perm ,
  ( same-orient __ refl refl
  , same-orient __ refl refl )
matching-transitive m02-13 m02-13 =

```

```

id-perm ,
( same-orient __ refl refl
, same-orient __ refl refl )
matching-transitive m02-13 m03-12 =
swap-23-perm ,
( same-orient __ refl refl
, same-orient __ refl refl )
matching-transitive m03-12 m01-23 =
swap-13-perm ,
( same-orient __ refl refl
, swap-orient __ refl refl )
matching-transitive m03-12 m02-13 =
swap-23-perm ,
( same-orient __ refl refl
, same-orient __ refl refl )
matching-transitive m03-12 m03-12 =
id-perm ,
( same-orient __ refl refl
, same-orient __ refl refl )

```

The orbit-count invariant

The vertex, edge, and matching transitivity results are collected into a single record. This packages the three orbit computations for later use.

The record `AutK4OrbitCounts` simply gathers the three orbit collapse theorems into one place. This is mainly an organizational step, but an important one for the later chapters: they can appeal to a single witness rather than re-quote vertex, edge, and matching transitivity separately. In particular, the orbit counts of vertices, edges, and perfect matchings are all equal to 1, while the associated cardinalities 4, 6, and 3 remain fixed combinatorial invariants of the tetrahedron.

```

record AutK4OrbitCounts : Set where
field
vertex-orbits :
(a a' : K4Vertex) →  $\Sigma$  K4Perm ( $\lambda \sigma \rightarrow \text{k4to } \sigma \ a \equiv a'$ )
edge-orbits :
(e1 e2 : K4Edge) →  $\Sigma$  K4Perm ( $\lambda \sigma \rightarrow \text{edge-act } \sigma \ e_1 \approx_E e_2$ )
matching-orbits :
(m1 m2 : K4Matching) → MatchingOrbitWitness m1 m2

theorem-aut-k4-orbits : AutK4OrbitCounts
theorem-aut-k4-orbits = record
{ vertex-orbits = k4Perm-send
; edge-orbits = edge-transitive
; matching-orbits = matching-transitive
}

```

The record states that vertices, edges, and perfect matchings each form a single orbit under $\text{Aut}(K_4)$.

The K_4 spherical cell law

The forced terminus K_4 is not merely a complete graph. Read as a 2-complex, it is the tetrahedral triangulation of a two-sphere: it carries $V = 4$ vertices, $E = 6$ edges, and $F = 4$ triangular faces, with Euler characteristic $\chi = V - E + F = 2$. The spherical topology is therefore not imposed after the fact; it is already present in the surviving finite structure.

The cell law collected here is purely combinatorial. It states four things that hold by computation: the topological identity $V + F = E + \chi$ (i.e. $4 + 4 = 6 + 2$); the self-duality $F = V$ of the tetrahedron; the handshaking identity $E \cdot \chi = V \cdot d$ (i.e. $6 \cdot 2 = 4 \cdot 3 = 12$), which rewrites to the exact ratio $E/V = d/2 = 3/2$; and the orbit-quotient ledger that records vertices, edges, and perfect matchings as single $\text{Aut}(K_4)$ -orbits.

Any spherical-cell invariant on K_4 must be a single object that bundles (i) the topological identity with $\chi(S^2)$, (ii) the edge count E , (iii) the vertex count V , and (iv) the proof that $\text{Aut}(K_4)$ has the relevant orbit transport. Without all four, the ratio E/V would be a loose number rather than a symmetry-controlled invariant.

The record `K4SphericalCellLaw` packages exactly these four facts. Each field reduces to `refl` on K_4 -invariants, except `aut-orbit-ledger`, which is `theorem-aut-k4-orbits`. No further data is needed; the cell law is obtained from the same combinatorics that already produced V, E, F, d, χ .

Every later projection out of *Void* must factor through this record if it uses the spherical K_4 cell. The ratio $E/V = 3/2$ is exact and Aut -invariant here; any extra meaning attached to that ratio belongs outside the eliminative chain.

```
record K4SphericalCellLaw : Set where
  field
    topological-sphere-cell : K4-V + K4-F ≡ K4-E + K4-chi
    self-dual-count          : K4-F ≡ K4-V
    handshaking-cell-law    : K4-E * K4-chi ≡ K4-V * K4-deg
    aut-orbit-ledger        : AutK4OrbitCounts

theorem-k4-spherical-cell-law : K4SphericalCellLaw
theorem-k4-spherical-cell-law = record
{ topological-sphere-cell = refl
; self-dual-count          = refl
; handshaking-cell-law    = refl
; aut-orbit-ledger        = theorem-aut-k4-orbits
}
```

The cell law is the formal statement that K_4 is the minimal triangulated sphere and that its ratio $E/V = 3/2$ is a forced combinatorial invariant.

2-transitive action of $\text{Aut}(K_4)$

The next step strengthens vertex transitivity to ordered pairs of distinct vertices. The same sender-and-inverse construction yields a permutation carrying one ordered pair to any other.

The same conjugation idea yields the stronger statement of 2-transitivity on ordered distinct pairs. Starting from witnesses that send (v_0, v_1) to (a, b) and (a', b') , one composes the second with the inverse of the first and reads off the resulting identities on both coordinates. Thus the 12 ordered distinct pairs also form a single orbit.

```

pair-transitive : (a b a' b' : K4Vertex) →
  a ≠ b → a' ≠ b' →
  Σ K4Perm (λ σ → (k4to σ a ≡ a') × (k4to σ b ≡ b'))
pair-transitive a b a' b' nab na'b' =
let em1 = edge-map a b nab
    em2 = edge-map a' b' na'b'
    σ1 = fst em1
    p1s = fst (snd em1)
    p1d = snd (snd em1)
    σ2 = fst em2
    p2s = fst (snd em2)
    p2d = snd (snd em2)
    σ = k4Perm-comp σ2 (k4Perm-inv σ1)
    qs : k4to σ a ≡ a'
    qs = trans (cong (k4to σ2)
      (trans (cong (k4from σ1) (sym p1s))
        (k4-from-to σ1 v0)))
    p2s
    qd : k4to σ b ≡ b'
    qd = trans (cong (k4to σ2)
      (trans (cong (k4from σ1) (sym p1d))
        (k4-from-to σ1 v1)))
    p2d
in σ, (qs, qd)

```

```

theorem-aut-k4-2-transitive :
(a b a' b' : K4Vertex) → a ≠ b → a' ≠ b' →
Σ K4Perm (λ σ → (k4to σ a ≡ a') × (k4to σ b ≡ b'))
theorem-aut-k4-2-transitive = pair-transitive

```

This is the formal content of the statement that $\text{Aut}(K_4)$ acts 2-transitively on vertices.

$\text{Aut}(K_4)$ -invariance of α, κ, d

The constants $d = 3$, $\kappa = 8$, and $\alpha\text{-denom} = 111$ are used later as global observables. To justify that role, one has to show that they are invariant under vertex relabelling.

The point is formalized by the notion of a vertex-rigid scalar: a function on vertices that is invariant under every $K4\text{Perm}$. The collapse theorem `vertex-rigid-constant` shows that such a scalar is automatically constant on the unique vertex orbit. Applying this to the constant functions with values d , κ , and α -denom produces the bundled record `AutK4Invariants`. Each quantity is an invariant under $\text{Aut}(K_4)$. Any deviation would break the symmetry chain. It is therefore a necessary observable.

```
-- § Vertex-rigid scalar.
VertexRigid : (K4Vertex → ℕ) → Set
VertexRigid f = (σ : K4Perm) (v : K4Vertex) → f (k4to σ v) ≡ f v

-- § Vertex rigidity collapses to constancy.
vertex-rigid-constant :
  (f : K4Vertex → ℕ) → VertexRigid f →
  (v w : K4Vertex) → f v ≡ f w
vertex-rigid-constant f rig v w with k4Perm-send v w
... | σ , p = trans (sym (rig σ v)) (cong f p)

-- The three discrete observables, presented as constant functions on K4Vertex.
d-vertex : K4Vertex → ℕ
d-vertex _ = degree-K4

κ-vertex : K4Vertex → ℕ
κ-vertex _ = κ-discrete

α-denom-vertex : K4Vertex → ℕ
α-denom-vertex _ = simplex-denom-K4

-- Each constant scalar is trivially K4Perm-invariant.
d-rigid : VertexRigid d-vertex
d-rigid _ _ = refl

κ-rigid : VertexRigid κ-vertex
κ-rigid _ _ = refl

α-denom-rigid : VertexRigid α-denom-vertex
α-denom-rigid _ _ = refl

-- § Aut(K4)-invariance record for the three discrete observables.
record AutK4Invariants : Set where
  field
    d-rigidity      : VertexRigid d-vertex
    κ-rigidity      : VertexRigid κ-vertex
    α-denom-rigidity : VertexRigid α-denom-vertex
    d-collapse      : (v w : K4Vertex) → d-vertex v ≡ d-vertex w
    κ-collapse      : (v w : K4Vertex) → κ-vertex v ≡ κ-vertex w
    α-denom-collapse : (v w : K4Vertex) → α-denom-vertex v ≡ α-denom-vertex w
    d-value         : degree-K4 ≡ 3
    κ-value         : κ-discrete ≡ 8
    α-denom-value   : simplex-denom-K4 ≡ 111

theorem-autk4-invariants : AutK4Invariants
theorem-autk4-invariants = record
```

```

{ d-rigidity      = d-rigid
;  $\kappa$ -rigidity    =  $\kappa$ -rigid
;  $\alpha$ -denom-rigidity =  $\alpha$ -denom-rigid
; d-collapse     = vertex-rigid-constant d-vertex d-rigid
;  $\kappa$ -collapse   = vertex-rigid-constant  $\kappa$ -vertex  $\kappa$ -rigid
;  $\alpha$ -denom-collapse = vertex-rigid-constant  $\alpha$ -denom-vertex  $\alpha$ -denom-rigid
; d-value        = law16B-2-degree
;  $\kappa$ -value      = law16B-8-kappa
;  $\alpha$ -denom-value = law16B-10-simplex-denom
}

```

The record packages the rigidity witnesses, the orbit-collapse theorems, and the numerical identities for d , κ , and α -denom.

Edge- and face-rigidity

The same rigidity argument extends to edge-indexed and face-indexed quantities. Edge transitivity handles the former, and the identification of faces with opposite vertices reduces the latter to the vertex case.

The same argument extends to the higher simplices. An edge-indexed scalar must respect both the transported edge action and the identification of opposite orientations; under these hypotheses, edge-transitivity forces it to be constant on all six edges. For faces there is even less to prove, because `K4Triangle` is identified with `K4Vertex` and the face action is simply the vertex action under that identification. Rigidity therefore propagates across all three simplicial dimensions of the tetrahedron.

```

-- § Edge rigidity: action-invariance plus orientation-invariance.
EdgeRigid : (K4Edge → ℕ) → Set
EdgeRigid f =
  ((σ : K4Perm) (e : K4Edge) → f (edge-act σ e) ≡ f e)
  ×
  ((e₁ e₂ : K4Edge) → e₁ ≈E e₂ → f e₁ ≡ f e₂)

-- Edge collapse: every edge-rigid scalar is constant on K4Edge.
edge-rigid-constant :
  (f : K4Edge → ℕ) → EdgeRigid f →
  (e₁ e₂ : K4Edge) → f e₁ ≡ f e₂
edge-rigid-constant f (rig , swp) e₁ e₂ with edge-transitive e₁ e₂
... | σ , p = trans (sym (rig σ e₁)) (swp _ _ p)

-- § Triangle rigidity under triangle-map.
TriangleRigid : (K4Triangle → ℕ) → Set
TriangleRigid f = (σ : K4Perm) (t : K4Triangle) → f (triangle-map σ t) ≡ f t

-- § Face collapse reuses vertex-rigid constancy.
triangle-rigid-constant :
  (f : K4Triangle → ℕ) → TriangleRigid f →
  (t₁ t₂ : K4Triangle) → f t₁ ≡ f t₂

```

```

triangle-rigid-constant  $f \text{ rig } t_1 \ t_2 = \text{vertex-rigid-constant } f \text{ rig } t_1 \ t_2$ 

-- The simplicial-dimension closure record.
record SimplicialRigidity : Set where
  field
    vertex-collapse :
      ( $f : \text{K4Vertex} \rightarrow \mathbb{N}$ )  $\rightarrow \text{VertexRigid } f \rightarrow (v \ w : \text{K4Vertex}) \rightarrow f \ v \equiv f \ w$ 
    edge-collapse :
      ( $f : \text{K4Edge} \rightarrow \mathbb{N}$ )  $\rightarrow \text{EdgeRigid } f \rightarrow (e_1 \ e_2 : \text{K4Edge}) \rightarrow f \ e_1 \equiv f \ e_2$ 
    face-collapse :
      ( $f : \text{K4Triangle} \rightarrow \mathbb{N}$ )  $\rightarrow \text{TriangleRigid } f \rightarrow (t_1 \ t_2 : \text{K4Triangle}) \rightarrow f \ t_1 \equiv f \ t_2$ 

theorem-simplicial-rigidity : SimplicialRigidity
theorem-simplicial-rigidity = record
{ vertex-collapse = vertex-rigid-constant
; edge-collapse = edge-rigid-constant
; face-collapse = triangle-rigid-constant
}

```

The record certifies that $\text{Aut}(K_4)$ -rigidity is uniform across the three simplicial dimensions. No dimension carries an extra parameter; the symmetry collapse is total.

The full $\text{Aut}(K_4)$ invariant ledger

The remaining discrete quantities V , E , χ , F_2 , $\mathcal{C}$, and the hierarchy exponent are handled in the same way: each is represented as a constant scalar and then bundled into a single invariance record.

Nothing essentially new is required here. Each of these quantities is represented by a constant scalar on K4Vertex , and so each is vertex-rigid by definition. The record AutK4FullLedger packages those rigidity witnesses together with the corresponding collapse theorems and the already established numerical values. In this way the ledger is sealed: every discrete constant used later in the spectral evaluation has already been certified as an $\text{Aut}(K_4)$ -invariant.

```

-- § The remaining six discrete observables as constant vertex scalars.
V-vertex : K4Vertex  $\rightarrow \mathbb{N}$ 
V-vertex _ = vertexCountK4

E-vertex : K4Vertex  $\rightarrow \mathbb{N}$ 
E-vertex _ = edgeCountK4

 $\chi$ -vertex : K4Vertex  $\rightarrow \mathbb{N}$ 
 $\chi$ -vertex _ = eulerChar-computed

F2-vertex : K4Vertex  $\rightarrow \mathbb{N}$ 
F2-vertex _ = F2

C-vertex : K4Vertex  $\rightarrow \mathbb{N}$ 
C-vertex _ = simplex-eval

```



```

hier-vertex : K4Vertex → ℕ
hier-vertex _ = hierarchy-exponent

V-rigid    : VertexRigid V-vertex
V-rigid    _ _ = refl
E-rigid    : VertexRigid E-vertex
E-rigid    _ _ = refl
χ-rigid    : VertexRigid χ-vertex
χ-rigid    _ _ = refl
F2-rigid  : VertexRigid F2-vertex
F2-rigid  _ _ = refl
C-rigid    : VertexRigid C-vertex
C-rigid    _ _ = refl
hier-rigid : VertexRigid hier-vertex
hier-rigid _ _ = refl

record AutK4FullLedger : Set where
  field
    -- Already-certified observables (Step 3)
    d-rigidity      : VertexRigid d-vertex
    κ-rigidity      : VertexRigid κ-vertex
    α-denom-rigidity : VertexRigid α-denom-vertex
    -- Full-ledger additions
    V-rigidity      : VertexRigid V-vertex
    E-rigidity      : VertexRigid E-vertex
    χ-rigidity      : VertexRigid χ-vertex
    F2-rigidity     : VertexRigid F2-vertex
    C-rigidity      : VertexRigid C-vertex
    hier-rigidity   : VertexRigid hier-vertex
    -- Single-value collapse witnesses
    V-collapse      : (v w : K4Vertex) → V-vertex  v ≡ V-vertex  w
    E-collapse      : (v w : K4Vertex) → E-vertex  v ≡ E-vertex  w
    χ-collapse      : (v w : K4Vertex) → χ-vertex  v ≡ χ-vertex  w
    F2-collapse     : (v w : K4Vertex) → F2-vertex v ≡ F2-vertex w
    C-collapse      : (v w : K4Vertex) → C-vertex  v ≡ C-vertex  w
    hier-collapse   : (v w : K4Vertex) → hier-vertex v ≡ hier-vertex w
    -- Forced numerical values
    V-value         : vertexCountK4 ≡ 4
    E-value         : edgeCountK4  ≡ 6
    χ-value         : eulerChar-computed ≡ 2
    F2-value        : F2           ≡ 17
    C-value         : simplex-eval  ≡ 137
    hier-value      : hierarchy-exponent ≡ 22

theorem-autk4-full-ledger : AutK4FullLedger
theorem-autk4-full-ledger = record
{ d-rigidity    = d-rigid
; κ-rigidity    = κ-rigid
; α-denom-rigidity = α-denom-rigid
; V-rigidity    = V-rigid
; E-rigidity    = E-rigid
; χ-rigidity    = χ-rigid
; F2-rigidity   = F2-rigid

```

```

; C-rigidity    = C-rigid
; hier-rigidity = hier-rigid
; V-collapse   = vertex-rigid-constant V-vertex  V-rigid
; E-collapse   = vertex-rigid-constant E-vertex  E-rigid
;  $\chi$ -collapse = vertex-rigid-constant  $\chi$ -vertex  $\chi$ -rigid
;  $F_2$ -collapse = vertex-rigid-constant  $F_2$ -vertex  $F_2$ -rigid
; C-collapse   = vertex-rigid-constant C-vertex  C-rigid
; hier-collapse = vertex-rigid-constant hier-vertex hier-rigid
; V-value      = refl
; E-value      = law16B-0-edges
;  $\chi$ -value    = law16B-3-euler
;  $F_2$ -value    = law16B-12-F2-is-17
; C-value      = law15A-0-simplex-eval-137
; hier-value   = law16B-9-hierarchy
}

```

The ledger packages the rigidity witnesses and numerical identities for all remaining discrete constants used later in the evaluation.

The Universal Observable Verdict

The next question is whether there is a simple criterion for deciding when a vertex observable is admissible. The answer is that rigidity is equivalent to constancy on the unique vertex orbit.

The corresponding general statement is binary. A function on `K4Vertex` is rigid exactly when it is constant on the unique vertex orbit; if it separates two vertices, then it cannot be invariant under $\text{Aut}(K_4)$. The record `RigidIffConstant` packages these two implications, while `separates-non-rigid` gives the contrapositive in the form needed for later eliminations. The sample observable `ladder-vertex` shows that non-rigid observables genuinely exist, so the dichotomy is not vacuous.

```

-- § Reverse of vertex-rigid-constant: constants are vacuously rigid.
constant-vertex-rigid :
  (f : K4Vertex → ℕ) →
  ((v w : K4Vertex) → f v ≡ f w) →
  VertexRigid f
constant-vertex-rigid f const σ v = const (k4to σ v) v

-- § The full equivalence: rigidity iff orbit-constancy.
record RigidIffConstant (f : K4Vertex → ℕ) : Set where
  field
    rigid→constant : VertexRigid f → (v w : K4Vertex) → f v ≡ f w
    constant→rigid : ((v w : K4Vertex) → f v ≡ f w) → VertexRigid f

theorem-rigid-iff-constant :
  (f : K4Vertex → ℕ) → RigidIffConstant f
theorem-rigid-iff-constant f = record
  { rigid→constant = vertex-rigid-constant f
  ; constant→rigid = constant-vertex-rigid f

```

```

}

-- § Elimination form: a single separation refutes rigidity globally.
separates-non-rigid :
  (f : K4Vertex → ℕ) (v w : K4Vertex) →
  f v ≠ f w →
  ¬ VertexRigid f
separates-non-rigid f v w sep rig =
  sep (vertex-rigid-constant f rig v w)

-- § Concrete refutation witness: a four-valued observable on K4.
ladder-vertex : K4Vertex → ℕ
ladder-vertex v0 = 0
ladder-vertex v1 = 1
ladder-vertex v2 = 2
ladder-vertex v3 = 3

ladder-separates : ladder-vertex v0 ≠ ladder-vertex v1
ladder-separates ()

ladder-not-rigid : ¬ VertexRigid ladder-vertex
ladder-not-rigid =
  separates-non-rigid ladder-vertex v0 v1 ladder-separates

-- § The universal observable verdict packaged.
record ObservableUniversal : Set1 where
  field
    rigid-iff-constant :
      (f : K4Vertex → ℕ) → RigidIffConstant f
    separation-refutes :
      (f : K4Vertex → ℕ) (v w : K4Vertex) →
      f v ≠ f w → ¬ VertexRigid f
    non-rigid-witness-exists :
      Σ (K4Vertex → ℕ) (λ f → ¬ VertexRigid f)

theorem-observable-universal : ObservableUniversal
theorem-observable-universal = record
  { rigid-iff-constant = theorem-rigid-iff-constant
  ; separation-refutes = separates-non-rigid
  ; non-rigid-witness-exists = (ladder-vertex , ladder-not-rigid)
  }

```

The Dirac Operator: Squaring to the Laplacian

The next construction asks whether the Laplacian on K_4 can be realized as the square of a first-order operator. On vertex functions alone it cannot, but on the direct sum of vertex and edge functions one can define a Dirac operator whose square is the Laplacian on each layer.

The way out is to enlarge the carrier from vertex functions to the direct sum of vertex and edge functions. On this space the exterior derivative and its adjoint exchange

the two layers, and the Dirac operator built from them becomes a genuine first-order object. Its square is then forced to act diagonally, as the vertex Laplacian on the vertex part and the edge Laplacian on the edge part. The decisive point is computational rather than metaphorical: the squaring theorems close by `refl`, so $D^2 = \Delta$ is literally the normal form of the construction.

```
-- § Six oriented edges of  $K_4$  (one chosen orientation per unordered edge).
data OrientedEdge : Set where
  oe01 oe02 oe03 oe12 oe13 oe23 : OrientedEdge

-- § Source and target maps for the chosen orientations.
oe-src : OrientedEdge → K4Vertex
oe-src oe01 = v0
oe-src oe02 = v0
oe-src oe03 = v0
oe-src oe12 = v1
oe-src oe13 = v1
oe-src oe23 = v2

oe-tgt : OrientedEdge → K4Vertex
oe-tgt oe01 = v1
oe-tgt oe02 = v2
oe-tgt oe03 = v3
oe-tgt oe12 = v2
oe-tgt oe13 = v3
oe-tgt oe23 = v3

-- § Integer subtraction.
infixl 6 _-ℤ_
_-ℤ_ : ℤ → ℤ → ℤ
x -ℤ y = x +ℤ negℤ y

-- § Vertex differential (coboundary): signed difference along each edge.
d-ve : (K4Vertex → ℤ) → (OrientedEdge → ℤ)
d-ve f oe01 = f v1 -ℤ f v0
d-ve f oe02 = f v2 -ℤ f v0
d-ve f oe03 = f v3 -ℤ f v0
d-ve f oe12 = f v2 -ℤ f v1
d-ve f oe13 = f v3 -ℤ f v1
d-ve f oe23 = f v3 -ℤ f v2

-- § Edge codifferential (adjoint): incoming minus outgoing at each vertex.
d-star : (OrientedEdge → ℤ) → (K4Vertex → ℤ)
d-star g v0 = (negℤ (g oe01) +ℤ negℤ (g oe02)) +ℤ negℤ (g oe03)
d-star g v1 = (g oe01 +ℤ negℤ (g oe12)) +ℤ negℤ (g oe13)
d-star g v2 = (g oe02 +ℤ g oe12) +ℤ negℤ (g oe23)
d-star g v3 = (g oe03 +ℤ g oe13) +ℤ g oe23

-- § The Dirac state space: vertex functions  $\oplus$  edge functions.
DiracState : Set
DiracState = (K4Vertex → ℤ) × (OrientedEdge → ℤ)

-- § The Dirac operator: exchange layers via d-ve and d-star.
```

```

dirac : DiracState → DiracState
dirac  $\phi$  = (d-star (snd  $\phi$ ), d-ve (fst  $\phi$ ))

-- § Square of the Dirac operator: Laplacian on each layer.
dirac2 : DiracState → DiracState
dirac2  $\phi$  = dirac (dirac  $\phi$ )

-- § Vertex Laplacian as d-star ∘ d-ve.
K4Lap-vert : (K4Vertex → ℤ) → (K4Vertex → ℤ)
K4Lap-vert  $f$  = d-star (d-ve  $f$ )

-- § Edge Laplacian as d-ve ∘ d-star, dually.
K4Lap-edge : (OrientedEdge → ℤ) → (OrientedEdge → ℤ)
K4Lap-edge  $g$  = d-ve (d-star  $g$ )

-- § The squaring theorems: dirac2 acts as the Laplacian on each layer.
theorem-dirac-squared-vertex :
( $\phi$  : DiracState) ( $v$  : K4Vertex) →
fst (dirac2  $\phi$ )  $v$  ≡ K4Lap-vert (fst  $\phi$ )  $v$ 
theorem-dirac-squared-vertex  $\phi$   $v_0$  = refl
theorem-dirac-squared-vertex  $\phi$   $v_1$  = refl
theorem-dirac-squared-vertex  $\phi$   $v_2$  = refl
theorem-dirac-squared-vertex  $\phi$   $v_3$  = refl

theorem-dirac-squared-edge :
( $\phi$  : DiracState) ( $e$  : OrientedEdge) →
snd (dirac2  $\phi$ )  $e$  ≡ K4Lap-edge (snd  $\phi$ )  $e$ 
theorem-dirac-squared-edge  $\phi$  oe01 = refl
theorem-dirac-squared-edge  $\phi$  oe02 = refl
theorem-dirac-squared-edge  $\phi$  oe03 = refl
theorem-dirac-squared-edge  $\phi$  oe12 = refl
theorem-dirac-squared-edge  $\phi$  oe13 = refl
theorem-dirac-squared-edge  $\phi$  oe23 = refl

-- § Algebraic identity: negation distributes over subtraction.
neg-sub : ( $a$   $b$  : ℤ) → negℤ ( $a$  -ℤ  $b$ ) ≡  $b$  -ℤ  $a$ 
neg-sub  $a$   $b$  =
trans (neg+ℤ  $a$  (negℤ  $b$ ))
      (trans (cong ( $\lambda$   $t$  → negℤ  $a$  +ℤ  $t$ ) (negℤ-involutive  $b$ ))
          (+ℤ-comm (negℤ  $a$ )  $b$ ))

-- § Expanded vertex form at  $v_0$ : K4Lap-vert  $f$   $v_0$  = sum of ( $f$   $v_0$  -  $f$  neighbor).
lap-expanded-v0 :
( $f$  : K4Vertex → ℤ) →
K4Lap-vert  $f$   $v_0$  ≡
(( $f$   $v_0$  -ℤ  $f$   $v_1$ ) +ℤ ( $f$   $v_0$  -ℤ  $f$   $v_2$ )) +ℤ ( $f$   $v_0$  -ℤ  $f$   $v_3$ )
lap-expanded-v0  $f$  =
trans
  (cong ( $\lambda$   $t$  →  $t$  +ℤ negℤ ( $f$   $v_3$  -ℤ  $f$   $v_0$ ))
    (cong2 _+ℤ_ (neg-sub ( $f$   $v_1$ ) ( $f$   $v_0$ ))
              (neg-sub ( $f$   $v_2$ ) ( $f$   $v_0$ ))))
  (cong ( $\lambda$   $t$  → (( $f$   $v_0$  -ℤ  $f$   $v_1$ ) +ℤ ( $f$   $v_0$  -ℤ  $f$   $v_2$ )) +ℤ  $t$ )
      (neg-sub ( $f$   $v_3$ ) ( $f$   $v_0$ )))

```

```

-- § The DiracOperator record bundles construction and squaring witness.
record DiracOperator : Set1 where
field
  op : DiracState → DiracState
  op-squares-to-lap-v :
    (φ : DiracState) (v : K4Vertex) →
    fst (op (op φ)) v ≡ K4Lap-vert (fst φ) v
  op-squares-to-lap-e :
    (φ : DiracState) (e : OrientedEdge) →
    snd (op (op φ)) e ≡ K4Lap-edge (snd φ) e
  expanded-vertex-form :
    (f : K4Vertex → ℤ) →
    K4Lap-vert f v0 ≡
      ((f v0 -ℤ f v1) +ℤ (f v0 -ℤ f v2)) +ℤ (f v0 -ℤ f v3)

theorem-dirac-operator : DiracOperator
theorem-dirac-operator = record
{ op = dirac
; op-squares-to-lap-v = theorem-dirac-squared-vertex
; op-squares-to-lap-e = theorem-dirac-squared-edge
; expanded-vertex-form = lap-expanded-v0
}

```

The record packages the operator, the two squaring identities, and the explicit vertex formula. In this sense the relation $D^2 = \Delta$ is part of the proved structure itself.

Chapter 63

Scale Closure

With *edge-weight* = 1, *face-weight* = 1, and *vertex-weight* = 1, no additional scale parameter remains. Any ratio built from these weights is therefore fixed, and the native FD scale reduces to the unit triple (1, 1, 1).

Simplicial scale units

The three simplicial scale units are:

$$S_1 = \sqrt{\frac{w_2 w_0}{w_1^3}}, \quad S_2 = \sqrt{\frac{w_2 w_0}{w_1^5}}, \quad S_0 = \sqrt{\frac{w_2 w_1}{w_0}},$$

where w_1, w_2, w_0 denote the forced edge, face, and vertex weights respectively. With all three weights equal to 1, every numerator and denominator is 1, so $S_1 = S_2 = S_0 = 1$ in FD units.

These expressions depend only on the previously forced weights. After substituting $w_1 = w_2 = w_0 = 1$, each unit evaluates to 1, so the three simplicial dimensions share the same native unit.

```
scale-dim1 : ℕ
scale-dim1 = (face-weight * vertex-weight) * 1
```

```
scale-dim2 : ℕ
scale-dim2 = (face-weight * vertex-weight) * 1
```

```
scale-dim0 : ℕ
scale-dim0 = (face-weight * edge-weight) * 1
```

```
scale-dim1-is-1 : scale-dim1 ≡ 1
scale-dim1-is-1 = refl
```

```
scale-dim2-is-1 : scale-dim2 ≡ 1
scale-dim2-is-1 = refl
```

```
scale-dim0-is-1 : scale-dim0  $\equiv$  1
scale-dim0-is-1 = refl
```

The closure record

The record `ScaleClosure` gathers the three weight identities and the three unit identities into a single witness.

```
record ScaleClosure : Set where
  field
    edge-forced : edge-weight  $\equiv$  1
    face-forced : face-weight  $\equiv$  1
    vertex-forced : vertex-weight  $\equiv$  1
    dim1-1 : scale-dim1  $\equiv$  1
    dim2-1 : scale-dim2  $\equiv$  1
    dim0-1 : scale-dim0  $\equiv$  1

theorem-scale-closure : ScaleClosure
theorem-scale-closure = record
{ edge-forced = refl
; face-forced = refl
; vertex-forced = refl
; dim1-1 = refl
; dim2-1 = refl
; dim0-1 = refl
}
```

No free scale

The number of forced constants matches the number of simplicial dimensions. In the arithmetic encoded below, this leaves no further independent scale parameter.

```
independent-constants :  $\mathbb{N}$ 
independent-constants = 3

independent-dimensions :  $\mathbb{N}$ 
independent-dimensions = 3

free-parameters :  $\mathbb{N}$ 
free-parameters = independent-dimensions - independent-constants

no-free-scale : free-parameters  $\equiv$  0
no-free-scale = refl
```


Normalized scale elimination

In normalized units one may ask whether any positive triple (a, b, c) besides $(1, 1, 1)$ can satisfy the bridge equations. A scale choice is then just a triple of natural-number assignments for the three simplicial dimensions, and the bridge equations become

$$b = a, \quad b = ca^2, \quad cb^2 = a^3.$$

The proof is a case split on the first coordinate. Positivity excludes $a = 0$, the arithmetic lemmas exclude $a \geq 2$, and once $a = 1$ the bridge equations force $b = 1$ and $c = 1$ as well.

```
IsScaleNonZero : ℕ → Set
```

```
IsScaleNonZero n = n ≡ 0 → ⊥
```

```
record NormalizedScaleChoice : Set where
  field
```

```
  step-dim1      : ℕ
```

```
  step-dim2      : ℕ
```

```
  step-dim0      : ℕ
```

```
  dim1-nonzero   : IsScaleNonZero step-dim1
```

```
  edge-lock      : step-dim2 ≡ step-dim1
```

```
  face-lock      : step-dim2 ≡ (step-dim0 *ℕ step-dim1) *ℕ step-dim1
```

```
  vertex-lock    : (step-dim0 *ℕ step-dim2) *ℕ step-dim2 ≡
```

```
    (step-dim1 *ℕ step-dim1) *ℕ step-dim1
```

Two small arithmetic lemmas do the work. First: if adding a tail leaves a natural unchanged, that tail must have been zero. Second: for any value $a \geq 2$, neither $a^2 = a$ nor $a^3 = a$ can hold.

```
natPlusRightLe : (a : ℕ) → (b : ℕ) → a ≤ (a +ℕ b)
```

```
natPlusRightLe zero b = z≤n
```

```
natPlusRightLe (suc a) b = s≤s (natPlusRightLe a b)
```

```
natPlusCancelSelf : (a : ℕ) → (b : ℕ) → a +ℕ b ≡ a → b ≡ 0
```

```
natPlusCancelSelf a b eq with ≤-+ℕ-cancel! a b 0
```

```
(≤-resp-≡ (subst (λ t → t ≤ a) (sym eq) (≤-refl a)) (sym (+ℕ-zero-right a)))
```

```
natPlusCancelSelf a zero eq | z≤n = refl
```

```
natPlusCancelSelf a (suc b) eq | ()
```

```
squareNotSelf : (n : ℕ) → ((suc (suc n) *ℕ suc (suc n)) ≡ suc (suc n)) → ⊥
```

```
squareNotSelf n eq =
```

```
  let
```

```
    a : ℕ
```

```
    a = suc (suc n)
```

```
  expand : a *ℕ a ≡ a +ℕ (a *ℕ suc n)
```

```
  expand = *ℕ-suc-right-+ℕ a (suc n)
```

```
  tail0 : (a *ℕ suc n) ≡ 0
```

```

tail0 = natPlusCancelSelf a (a * $\mathbb{N}$  suc n) (trans (sym expand) eq)

bad : suc n  $\equiv$  0
bad = * $\mathbb{N}$ -pos-zero $\rightarrow$ zero (suc n) (suc n) tail0
in
suc $\neq$ 0 bad
where
suc $\neq$ 0 : {k :  $\mathbb{N}$ }  $\rightarrow$  suc k  $\equiv$  0  $\rightarrow$   $\perp$ 
suc $\neq$ 0 ()

squareDominates : (n :  $\mathbb{N}$ )  $\rightarrow$  suc (suc n)  $\leq$  (suc (suc n) * $\mathbb{N}$  suc (suc n))
squareDominates n =
let
a :  $\mathbb{N}$ 
a = suc (suc n)

base : suc 0  $\leq$  a
base = s $\leq$ s z $\leq$ n

step : (suc 0 * $\mathbb{N}$  a)  $\leq$  (a * $\mathbb{N}$  a)
step =  $\leq$ -* $\mathbb{N}$ -mono $^r$  base a
in
subst ( $\lambda$  t  $\rightarrow$  t  $\leq$  (a * $\mathbb{N}$  a)) (* $\mathbb{N}$ -one-left a) step

cubeNotSelf : (n :  $\mathbb{N}$ )  $\rightarrow$  (((suc (suc n) * $\mathbb{N}$  suc (suc n)) * $\mathbb{N}$  suc (suc n))  $\equiv$  suc (suc n))  $\rightarrow$   $\perp$ 
cubeNotSelf n eq =
let
a :  $\mathbb{N}$ 
a = suc (suc n)

sq :  $\mathbb{N}$ 
sq = a * $\mathbb{N}$  a

expand : sq * $\mathbb{N}$  a  $\equiv$  sq + $\mathbb{N}$  (sq * $\mathbb{N}$  suc n)
expand = * $\mathbb{N}$ -suc-right-+ $\mathbb{N}$  sq (suc n)

sq $\leq$ a : sq  $\leq$  a
sq $\leq$ a =  $\leq$ -resp- $\equiv^r$  (natPlusRightLe sq (sq * $\mathbb{N}$  suc n)) (trans (sym expand) eq)

a $\leq$ sq : a  $\leq$  sq
a $\leq$ sq = squareDominates n

sq $\equiv$ a : sq  $\equiv$  a
sq $\equiv$ a =  $\leq$ -antisym sq $\leq$ a a $\leq$ sq
in
squareNotSelf n sq $\equiv$ a

dim0-unit-forced : (c :  $\mathbb{N}$ )  $\rightarrow$  ((c * $\mathbb{N}$  1) * $\mathbb{N}$  1)  $\equiv$  1  $\rightarrow$  c  $\equiv$  1
dim0-unit-forced zero ()
dim0-unit-forced (suc zero) refl = refl
dim0-unit-forced (suc (suc n)) ()

```

The elimination itself is direct. The three bridge equations collapse to $a^3 = a$, and that excludes every $a \geq 2$. Positivity excludes $a = 0$. So $a = 1$, hence also $b = 1$ and $c = 1$.

```

normalized-dim1-is-1 : (S : NormalizedScaleChoice) → NormalizedScaleChoice.step-dim1 S ≡ 1
normalized-dim1-is-1 S
  with NormalizedScaleChoice.step-dim1 S
  | NormalizedScaleChoice.step-dim2 S
  | NormalizedScaleChoice.step-dim0 S
  | NormalizedScaleChoice.dim1-nonzero S
  | NormalizedScaleChoice.edge-lock S
  | NormalizedScaleChoice.face-lock S
  | NormalizedScaleChoice.vertex-lock S
... | zero      | _ | _ | nz | _ | _ | _ = ⊥-elim (nz refl)
... | suc zero  | _ | _ | _ | _ | _ = refl
... | suc (suc n) | b | c | _ | cl | hl | gl =
let
  a : ℕ
  a = suc (suc n)

  G-at-a : (c *ℕ a) *ℕ a ≡ (a *ℕ a) *ℕ a
  G-at-a = trans (sym (cong (λ t → (c *ℕ t) *ℕ t) cl)) gl

  cubeEq : ((a *ℕ a) *ℕ a) ≡ a
  cubeEq = trans (sym G-at-a) (trans (sym hl) cl)
in
⊥-elim (cubeNotSelf n cubeEq)

normalized-dim2-is-1 : (S : NormalizedScaleChoice) → NormalizedScaleChoice.step-dim2 S ≡ 1
normalized-dim2-is-1 S = trans (NormalizedScaleChoice.edge-lock S) (normalized-dim1-is-1 S)

normalized-dim0-is-1 : (S : NormalizedScaleChoice) → NormalizedScaleChoice.step-dim0 S ≡ 1
normalized-dim0-is-1 S =
  dim0-unit-forced
  (NormalizedScaleChoice.step-dim0 S)
  (trans
    (cong (λ t → (NormalizedScaleChoice.step-dim0 S *ℕ t) *ℕ t)
      (sym (normalized-dim1-is-1 S))))
  (trans
    (sym (NormalizedScaleChoice.face-lock S))
    (normalized-dim2-is-1 S)))

normalized-scale-witness : NormalizedScaleChoice
normalized-scale-witness = record
{ step-dim1      = 1
; step-dim2      = 1
; step-dim0      = 1
; dim1-nonzero   = λ ()
; edge-lock      = refl
; face-lock      = refl
; vertex-lock    = refl
}
    
```

This is the normalized form of the scale-closure statement. In the regime $w_1 = w_2 = w_0 = 1$, the only admissible positive scale triple is $(1, 1, 1)$.

Distinction as measurement kernel

The next question concerns the minimal structure required for measurement. At the level needed here, a measurement scheme must at least distinguish two outcomes.

A minimal binary scheme therefore consists of an outcome type, two distinct outcomes, and an exhaustive yes/no classification of that type.

The record `BinaryMeasurement` isolates exactly these data. The map `measurement-outcome-distinction` repackages them as a `Distinction`, so the distinction kernel is recovered directly from the measurement structure.

```
record BinaryMeasurement : Set1 where
  field
    State      : Set
    Outcome    : Set
    measure    : State → Outcome
    yes        : Outcome
    no         : Outcome
    yes≠no     : yes ≠ no
    total      : (y : Outcome) → (y ≡ yes) ∨ (y ≡ no)

open BinaryMeasurement public

measurement-outcome-distinction : (M : BinaryMeasurement) → Distinction
measurement-outcome-distinction M =
  fromDistinctCovered
    (yes M)
    (no M)
    (yes≠no M)
    (total M)

measurement-outcome-nontrivial : (M : BinaryMeasurement) → NonTrivial (Outcome M)
measurement-outcome-nontrivial M =
  law0-5-distinct-pair-forces-nontrivial
    (yes M , (no M , yes≠no M))

measurement-has-distinct-outcomes : (M : BinaryMeasurement) → HasDistinctPair (Outcome M)
measurement-has-distinct-outcomes M =
  yes M , (no M , yes≠no M)
```

These lemmas establish only a threshold result: any structure encoded by `BinaryMeasurement` has distinct outcomes and therefore supports a distinction in its outcome space.

This provides a second route to the distinction package. One route begins with the type-theoretic construction itself; the present one begins with binary measurement data and extracts the same minimal structure.

For the later continuum questions, only this consequence matters: once measurement supplies a distinction kernel, the earlier discrete chain may be reused without additional assumptions.

The loop classification

The invariant ledger permits a finite loop classification. Using $E = 6$, $d = 3$, and $\chi = 2$, one obtains eight subset-sum candidates for a loop numerator.

The relevant denominators are the bulk value $72 = VEd$ and its extended version $576 = \kappa VEd$. A candidate survives only if it is irreducible against these denominators and complete with respect to the required prime strata.

The computation is finite. Six candidates fail irreducibility with 72, the remaining partial candidate $d + \chi = 5$ fails the completeness test, and the only survivor is $E + d + \chi = 11$.

```
-- § Loop Classification – the unique admissible loop numerator.

-- § The universal loop numerator: sum of all primitive loop parameters.
loop-numerator : ℕ
loop-numerator = edgeCountK4 + degree-K4 + eulerChar-computed

-- § Law: loop numerator is exactly 11.
law-loop-num-11 : loop-numerator ≡ 11
law-loop-num-11 = refl

-- § Decomposition: the three structural contributions.
law-loop-num-decomposition : loop-numerator ≡ 6 + 3 + 2
law-loop-num-decomposition = refl

-- § The handshaking identity:  $\chi \cdot d = E$ .
law-chi-times-d-is-E : eulerChar-computed * degree-K4 ≡ edgeCountK4
law-chi-times-d-is-E = refl

-- § Denominator at the bulk scale:  $V \cdot E \cdot d = 72$ .
loop-denom-bulk : ℕ
loop-denom-bulk = vertexCountK4 * edgeCountK4 * degree-K4

-- § Denominator at the extended scale:  $\kappa \cdot 72 = 576$ .
loop-denom-extended : ℕ
loop-denom-extended = loop-denom-bulk *  $\kappa$ -discrete

-- § The eight candidates and their values:
loop-cand-000 : ℕ -- empty set
loop-cand-000 = 0
loop-cand-001 : ℕ --  $\chi$  only
loop-cand-001 = eulerChar-computed
loop-cand-010 : ℕ --  $d$  only
loop-cand-010 = degree-K4
loop-cand-011 : ℕ --  $d + \chi$ 
loop-cand-011 = degree-K4 + eulerChar-computed
loop-cand-100 : ℕ --  $E$  only
loop-cand-100 = edgeCountK4
loop-cand-101 : ℕ --  $E + \chi$ 
loop-cand-101 = edgeCountK4 + eulerChar-computed
loop-cand-110 : ℕ --  $E + d$ 
loop-cand-110 = edgeCountK4 + degree-K4
```

```

loop-cand-111 :  $\mathbb{N} \dashrightarrow E + d + \chi$ 
loop-cand-111 = edgeCountK4 + degree-K4 + eulerChar-computed

-- § Values
law-cand-000 : loop-cand-000  $\equiv$  0
law-cand-000 = refl
law-cand-001 : loop-cand-001  $\equiv$  2
law-cand-001 = refl
law-cand-010 : loop-cand-010  $\equiv$  3
law-cand-010 = refl
law-cand-011 : loop-cand-011  $\equiv$  5
law-cand-011 = refl
law-cand-100 : loop-cand-100  $\equiv$  6
law-cand-100 = refl
law-cand-101 : loop-cand-101  $\equiv$  8
law-cand-101 = refl
law-cand-110 : loop-cand-110  $\equiv$  9
law-cand-110 = refl
law-cand-111 : loop-cand-111  $\equiv$  11
law-cand-111 = refl

-- § Filter 1: irreducibility with 72 eliminates 0, 2, 3, 6, 8, and 9.
law-elim-000 : gcd loop-cand-001 loop-denom-bulk  $\equiv$  2 --  $\chi=2$  shares factor 2
law-elim-000 = refl
law-elim-010 : gcd loop-cand-010 loop-denom-bulk  $\equiv$  3 --  $d=3$  shares factor 3
law-elim-010 = refl
law-elim-100 : gcd loop-cand-100 loop-denom-bulk  $\equiv$  6 --  $E=6$  shares factor 6
law-elim-100 = refl
law-elim-101 : gcd loop-cand-101 loop-denom-bulk  $\equiv$  8 --  $E+\chi=8$  shares factor 8
law-elim-101 = refl
law-elim-110 : gcd loop-cand-110 loop-denom-bulk  $\equiv$  9 --  $E+d=9$  shares factor 9
law-elim-110 = refl

-- § Pass irreducibility:  $d+\chi=5$  and  $E+d+\chi=11$ 
law-pass-011 : gcd loop-cand-011 loop-denom-bulk  $\equiv$  1 --  $d+\chi=5$  coprime ✓
law-pass-011 = refl
law-pass-111 : gcd loop-cand-111 loop-denom-bulk  $\equiv$  1 --  $E+d+\chi=11$  coprime ✓
law-pass-111 = refl

-- § Filter 2:  $d+\chi=5$  omits the edge stratum.
law-partial-omits-E : loop-cand-011 + edgeCountK4  $\equiv$  loop-cand-111
law-partial-omits-E = refl

law-full-is-partial-plus-E : loop-cand-111  $\equiv$  edgeCountK4 + loop-cand-011
law-full-is-partial-plus-E = refl

-- § Structural independence: E is multiplicatively independent of  $d+\chi$ .
law-E-structurally-independent :
  gcd edgeCountK4 (degree-K4 + eulerChar-computed)  $\equiv$  1
law-E-structurally-independent = refl -- gcd(6, 5) = 1

-- § E = 6 spans the full {2,3} prime base inside  $72 = 2^3 \cdot 3^2$ .
law-E-spans-full-base : gcd edgeCountK4 (eulerChar-computed * degree-K4)  $\equiv$  6
law-E-spans-full-base = refl -- gcd(6, 6) = 6

```

```

-- § d = 3 contributes only the 3-stratum.
law-d-in-3-stratum : gcd degree-K4 (eulerChar-computed * degree-K4) ≡ 3
law-d-in-3-stratum = refl -- gcd(3, 6) = 3

-- § χ = 2 contributes only the 2-stratum.
law-chi-in-2-stratum : gcd eulerChar-computed (eulerChar-computed * degree-K4) ≡ 2
law-chi-in-2-stratum = refl -- gcd(2, 6) = 2

-- § d+χ = 5 escapes the {2,3} stratum completely.
law-dchi-escapes-stratum :
  gcd (degree-K4 + eulerChar-computed) (eulerChar-computed * degree-K4) ≡ 1
law-dchi-escapes-stratum = refl -- gcd(5, 6) = 1

-- § Completeness derivation from χ·d = E and the volume structure.
record CompletenessDerivation : Set where
  field
    relation          : eulerChar-computed * degree-K4 ≡ edgeCountK4
    generators-coprime : gcd eulerChar-computed degree-K4 ≡ 1
    volume-via-relation : (vertexCountK4 * eulerChar-computed) * (degree-K4 * degree-K4) ≡ loop-denom-bulk
    sum-product-coprime : gcd (degree-K4 + eulerChar-computed) (eulerChar-computed * degree-K4) ≡ 1
    partial-blind-to-E : gcd loop-cand-011 edgeCountK4 ≡ 1
    full-witnesses-E : loop-cand-111 ≡ edgeCountK4 + loop-cand-011
    full-irreducible : gcd loop-cand-111 loop-denom-bulk ≡ 1

theorem-completeness-derivation : CompletenessDerivation
theorem-completeness-derivation = record
{ relation          = refl
; generators-coprime = refl
; volume-via-relation = refl
; sum-product-coprime = refl
; partial-blind-to-E = refl
; full-witnesses-E = refl
; full-irreducible = refl
}

-- § Filter 3: cross-scale consistency at extended scale.
law-partial-ext : gcd loop-cand-011 loop-denom-extended ≡ 1 -- 5 coprime to 576 too
law-partial-ext = refl
law-pass-111-ext : gcd loop-cand-111 loop-denom-extended ≡ 1 -- 11 coprime to 576 ✓
law-pass-111-ext = refl

-- § Loop classification record: unique admissible loop numerator.
record LoopClassification : Set where
  field
    forced-value      : loop-numerator ≡ 11
    forced-from-K4    : loop-numerator ≡ edgeCountK4 + degree-K4 + eulerChar-computed
    irreducible-bulk : gcd loop-numerator loop-denom-bulk ≡ 1
    irreducible-ext  : gcd loop-numerator loop-denom-extended ≡ 1
    elim-chi         : gcd eulerChar-computed loop-denom-bulk ≡ 2
    elim-d           : gcd degree-K4 loop-denom-bulk ≡ 3
    elim-E           : gcd edgeCountK4 loop-denom-bulk ≡ 6
    elim-E-chi       : gcd (edgeCountK4 + eulerChar-computed) loop-denom-bulk ≡ 8
    elim-E-d         : gcd (edgeCountK4 + degree-K4) loop-denom-bulk ≡ 9

```

```

partial-coprime      : gcd (degree-K4 + eulerChar-computed) loop-denom-bulk ≡ 1
partial-incomplete   : degree-K4 + eulerChar-computed ≡ 5
partial-omits-E      : loop-cand-011 + edgeCountK4 ≡ loop-cand-111
partial-also-ext     : gcd loop-cand-011 loop-denom-extended ≡ 1

```

```
theorem-loop-classification : LoopClassification
```

```
theorem-loop-classification = record
```

```

{ forced-value      = refl
; forced-from-K4    = refl
; irreducible-bulk  = refl
; irreducible-ext   = refl
; elim-chi          = refl
; elim-d            = refl
; elim-E            = refl
; elim-E-chi        = refl
; elim-E-d          = refl
; partial-coprime    = refl
; partial-incomplete = refl
; partial-omits-E    = refl
; partial-also-ext   = refl
}

```

Loop basis forcing

The previous classification uses the basis $\{E, d, \chi\}$. This subsection explains why that basis is determined by the prime support of $72 = 2^3 \cdot 3^2$.

The reason is arithmetic. $F_2 = 17$ lies outside the bulk prime support, while V , F , and κ remain in the 2-stratum generated by χ . The non-redundant basis that still reaches both prime strata is therefore $\{E, d, \chi\}$.

```
-- §§ Loop Basis Forcing - the basis {E, d, χ} covers the bulk prime strata.
```

```
-- §  $F_2 = 17$  escapes the bulk stratum entirely (coprime to 72).
```

```
law-F2-escapes-stratum : gcd F2 loop-denom-bulk ≡ 1
```

```
law-F2-escapes-stratum = refl
```

```
-- §  $V, F, \kappa$  are multiplicative powers of  $\chi$  inside the 2-stratum.
```

```
law-V-is-chi-squared : vertexCountK4 ≡ eulerChar-computed * eulerChar-computed
```

```
law-V-is-chi-squared = refl
```

```
law-F-is-chi-squared : faceCountK4 ≡ eulerChar-computed * eulerChar-computed
```

```
law-F-is-chi-squared = refl
```

```
law-κ-is-chi-cubed : κ-discrete ≡ eulerChar-computed * eulerChar-computed * eulerChar-computed
```

```
law-κ-is-chi-cubed = refl
```

```
-- §  $\chi$  is the minimal 2-stratum  $K_4$  invariant.
```

```
law-chi-≤-V : eulerChar-computed ≤ vertexCountK4
```

```
law-chi-≤-V = s ≤ s (s ≤ s z ≤ n)
```

```
law-chi-≤-F : eulerChar-computed ≤ faceCountK4
```

```
law-chi-≤-F = s ≤ s (s ≤ s z ≤ n)
```



```

law-chi- $\leq$ - $\kappa$  : eulerChar-computed  $\leq$   $\kappa$ -discrete
law-chi- $\leq$ - $\kappa$  = s $\leq$ s (s $\leq$ s z $\leq$ n)

-- § The bulk denominator equals 72 (refl through  $K_4$  definitions).
law-bulk-denom-72 : loop-denom-bulk  $\equiv$  72
law-bulk-denom-72 = refl

-- § Loop basis forcing record: {E, d,  $\chi$ } is the unique minimal basis.
record LoopBasisForcing : Set where
  field
    -- Stratum escape:  $F_2$  has no role in a {2,3}-coverage argument.
    F2-escapes-stratum : gcd F2 loop-denom-bulk  $\equiv$  1
    -- Multiplicative redundancy: V, F,  $\kappa$  are  $\chi$ -powers in the 2-stratum.
    V-is-chi-squared    : vertexCountK4  $\equiv$  eulerChar-computed * eulerChar-computed
    F-is-chi-squared    : faceCountK4  $\equiv$  eulerChar-computed * eulerChar-computed
     $\kappa$ -is-chi-cubed     :  $\kappa$ -discrete  $\equiv$  eulerChar-computed * eulerChar-computed * eulerChar-computed
    --  $\chi$  is the minimum of the 2-stratum cluster.
    chi- $\leq$ -V          : eulerChar-computed  $\leq$  vertexCountK4
    chi- $\leq$ -F          : eulerChar-computed  $\leq$  faceCountK4
    chi- $\leq$ - $\kappa$         : eulerChar-computed  $\leq$   $\kappa$ -discrete
    -- Stratum coverage by the surviving basis.
    chi-covers-2-stratum : gcd eulerChar-computed loop-denom-bulk  $\equiv$  2
    d-covers-3-stratum   : gcd degree-K4 loop-denom-bulk  $\equiv$  3
    E-spans-both-strata  : gcd edgeCountK4 loop-denom-bulk  $\equiv$  6
    -- Handshake: E factors as  $\chi \cdot d$ , locking the basis to three elements.
    handshake            : eulerChar-computed * degree-K4  $\equiv$  edgeCountK4
    -- Bulk denominator value.
    bulk-denom-72        : loop-denom-bulk  $\equiv$  72

theorem-loop-basis-forcing : LoopBasisForcing
theorem-loop-basis-forcing = record
{ F2-escapes-stratum = refl
; V-is-chi-squared    = refl
; F-is-chi-squared    = refl
;  $\kappa$ -is-chi-cubed     = refl
; chi- $\leq$ -V          = s $\leq$ s (s $\leq$ s z $\leq$ n)
; chi- $\leq$ -F          = s $\leq$ s (s $\leq$ s z $\leq$ n)
; chi- $\leq$ - $\kappa$         = s $\leq$ s (s $\leq$ s z $\leq$ n)
; chi-covers-2-stratum = refl
; d-covers-3-stratum   = refl
; E-spans-both-strata  = refl
; handshake            = refl
; bulk-denom-72        = refl
}

```

$\text{Aut}(K_4)$ -rigidity of the loop numerator

The loop numerator $p = 11$ can be treated exactly like the earlier ledger constants. As a constant scalar on vertices it is rigid under every automorphism and hence constant on the unique vertex orbit.

The record `LoopNumeratorRigidity` packages this rigidity statement together with the value $p = 11$.

```
-- §§ Aut(K4)-rigidity of the loop numerator p = 11.

p-vertex : K4Vertex → ℕ
p-vertex _ = loop-numerator

p-rigid : VertexRigid p-vertex
p-rigid _ _ = refl

record LoopNumeratorRigidity : Set where
  field
    p-rigidity : VertexRigid p-vertex
    p-collapse : (v w : K4Vertex) → p-vertex v ≡ p-vertex w
    p-value    : loop-numerator ≡ 11

theorem-loop-numerator-rigidity : LoopNumeratorRigidity
theorem-loop-numerator-rigidity = record
{ p-rigidity = p-rigid
; p-collapse = vertex-rigid-constant p-vertex p-rigid
; p-value = law-loop-num-11
}
```

Exponent forcing

The invariant ledger also fixes an exponent gap. If the bulk exponent is f_0 and the boundary exponent is f_1 , the relation $d^{f_1-f_0} = d^d$ forces $f_1 - f_0 = d$.

For $d = 3$, this reduces to the finite equation $3^n = 27$. The code rules out $n = 0, 1, 2, 4, 5, 6, 7, 8$, leaving $n = 3$ as the only possibility.

With the minimal positive bulk exponent fixed at 1, the boundary exponent becomes $1 + d = 4$, and the corresponding bulk and boundary volumes are determined.

```
-- §§ Exponent Forcing – the volume exponent gap is uniquely d

-- § d^d = 27: forced volume ratio.
law-dd-27 : degree-K4 ^ degree-K4 ≡ 27
law-dd-27 = refl

-- § Bulk exponent: minimal positive = 1.
bulk-exponent : ℕ
bulk-exponent = 1

-- § Boundary exponent: 1 + d = 4.
boundary-exponent : ℕ
boundary-exponent = 1 + degree-K4

law-boundary-exponent-is-4 : boundary-exponent ≡ 4
law-boundary-exponent-is-4 = refl

-- § Exponent forcing record: gap = d is unique.
record ExponentForcing : Set where
```

```

field
gap-is-d      : boundary-exponent  $\dot{-}$  bulk-exponent  $\equiv$  degree-K4
dd-is-27      : degree-K4  $\wedge$  degree-K4  $\equiv$  27
bulk-vol      : degree-K4  $\wedge$  bulk-exponent  $\equiv$  3
boundary-vol  : degree-K4  $\wedge$  boundary-exponent  $\equiv$  81
unique-gap-not-0 : degree-K4  $\wedge$  0  $\equiv$  degree-K4  $\wedge$  degree-K4  $\rightarrow \perp$ 
unique-gap-not-1 : degree-K4  $\wedge$  1  $\equiv$  degree-K4  $\wedge$  degree-K4  $\rightarrow \perp$ 
unique-gap-not-2 : degree-K4  $\wedge$  2  $\equiv$  degree-K4  $\wedge$  degree-K4  $\rightarrow \perp$ 
unique-gap-not-4 : degree-K4  $\wedge$  4  $\equiv$  degree-K4  $\wedge$  degree-K4  $\rightarrow \perp$ 
unique-gap-not-5 : degree-K4  $\wedge$  5  $\equiv$  degree-K4  $\wedge$  degree-K4  $\rightarrow \perp$ 
unique-gap-not-6 : degree-K4  $\wedge$  6  $\equiv$  degree-K4  $\wedge$  degree-K4  $\rightarrow \perp$ 
unique-gap-not-7 : degree-K4  $\wedge$  7  $\equiv$  degree-K4  $\wedge$  degree-K4  $\rightarrow \perp$ 
unique-gap-not-8 : degree-K4  $\wedge$  8  $\equiv$  degree-K4  $\wedge$  degree-K4  $\rightarrow \perp$ 

```

```
theorem-exponent-forcing : ExponentForcing
```

```
theorem-exponent-forcing = record
```

```

{ gap-is-d      = refl
; dd-is-27      = refl
; bulk-vol      = refl
; boundary-vol  = refl
; unique-gap-not-0 =  $\lambda ()$ 
; unique-gap-not-1 =  $\lambda ()$ 
; unique-gap-not-2 =  $\lambda ()$ 
; unique-gap-not-4 =  $\lambda ()$ 
; unique-gap-not-5 =  $\lambda () \text{ -- } 3^5 = 243 \neq 27$ 
; unique-gap-not-6 =  $\lambda () \text{ -- } 3^6 = 729 \neq 27$ 
; unique-gap-not-7 =  $\lambda () \text{ -- } 3^7 = 2187 \neq 27$ 
; unique-gap-not-8 =  $\lambda () \text{ -- } 3^8 = 6561 \neq 27$ 
}

```

Tree-level polynomial evaluations

The fixed ledger also determines several integer polynomial evaluations. Their role in this section is purely arithmetic: once the invariant values are fixed, the displayed monomials evaluate to definite integers.

The alternative expression for 1836 agrees with the first because of the handshake identity $\chi d = E$.

The code records these evaluations directly by computation.

```

-- §§ Tree-level polynomial evaluations – monomials in  $K_4$  invariants.

-- § First polynomial:  $\chi^2 \cdot d^3 \cdot F_2 = 4 \cdot 27 \cdot 17 = 1836$ .
tree-poly-1 :  $\mathbb{N}$ 
tree-poly-1 = (eulerChar-computed * eulerChar-computed)
              * (degree-K4 * degree-K4 * degree-K4)
              * F2

law-tree-poly-1 : tree-poly-1  $\equiv$  1836
law-tree-poly-1 = refl

-- § Alternative factorisation:  $d \cdot E^2 \cdot F_2 = 3 \cdot 36 \cdot 17 = 1836$ .

```

```

tree-poly-1-alt : ℕ
tree-poly-1-alt = degree-K4 * (edgeCountK4 * edgeCountK4) * F2

law-tree-poly-1-alt : tree-poly-1-alt ≡ 1836
law-tree-poly-1-alt = refl

-- § Both factorisations agree (because  $\chi \cdot d = E$ ).
law-tree-poly-1-agree : tree-poly-1 ≡ tree-poly-1-alt
law-tree-poly-1-agree = refl

-- § Second polynomial:  $d^2 \cdot (E + F_2) = 9 \cdot 23 = 207$ .
tree-poly-2 : ℕ
tree-poly-2 = (degree-K4 * degree-K4) * (edgeCountK4 + F2)

law-tree-poly-2 : tree-poly-2 ≡ 207
law-tree-poly-2 = refl

-- § Third polynomial:  $F_2 = 17$ .
tree-poly-3 : ℕ
tree-poly-3 = F2

law-tree-poly-3 : tree-poly-3 ≡ 17
law-tree-poly-3 = refl

-- § Degree shift:  $1836 + d = 1839$ .
tree-poly-shift : ℕ
tree-poly-shift = tree-poly-1 + degree-K4

law-tree-poly-shift : tree-poly-shift ≡ 1839
law-tree-poly-shift = refl

-- § 1836 factorisation certificate:  $4 \times 27 \times 17$ .
law-1836-factored : tree-poly-1 ≡ (eulerChar-computed * eulerChar-computed)
                                     * (degree-K4 ^ degree-K4) * F2
law-1836-factored = refl

```

Prime-stratum forcing of the proton ratio

The factorization of 1836 can be read from the prime structure of the ledger. The factor 17 localizes uniquely to F_2 , so the remaining factor 108 must be realized within the 2-3-smooth part of the ledger.

The second stage is finite: among degree- ≤ 3 atomic monomials in $\{V, E, d, \chi, \kappa\}$, exactly one has value 108, namely E^2d .

The handshake identity then rewrites the same factor as χ^2d^3 , so the two displayed forms are equivalent descriptions of the same decomposition.

```

-- §§ Prime-stratum forcing for 1836.

private
-- § Indicator: 1 if  $m \equiv n$ , 0 otherwise.
isEqℕ' : ℕ → ℕ → ℕ
isEqℕ' m n = 1 - ((m - n) + (n - m))

```

```

-- § Hit count for value 108 across the 56 degree-≤3 monomials.
hits108 : ℕ
hits108 =
  -- degree 0
  isEqN' 1 108
  -- degree 1: V, E, d, χ, κ
  + isEqN' 4 108 + isEqN' 6 108 + isEqN' 3 108 + isEqN' 2 108
  + isEqN' 8 108
  -- degree 2 (15 monomials): VV VE Vd Vχ Vκ EE Ed Eχ Eκ dd dχ dκ χχ χκ κκ
  + isEqN' 16 108 + isEqN' 24 108 + isEqN' 12 108 + isEqN' 8 108
  + isEqN' 32 108 + isEqN' 36 108 + isEqN' 18 108 + isEqN' 12 108
  + isEqN' 48 108 + isEqN' 9 108 + isEqN' 6 108 + isEqN' 24 108
  + isEqN' 4 108 + isEqN' 16 108 + isEqN' 64 108
  -- degree 3 (35 monomials): first V-led block
  + isEqN' 64 108 + isEqN' 96 108 + isEqN' 48 108 + isEqN' 32 108
  + isEqN' 128 108 + isEqN' 144 108 + isEqN' 72 108 + isEqN' 48 108
  + isEqN' 192 108 + isEqN' 36 108 + isEqN' 24 108 + isEqN' 96 108
  + isEqN' 16 108 + isEqN' 64 108 + isEqN' 256 108
  -- EEE EEd EEχ EEκ Edd Edχ Edκ Eχχ Eχκ Eκκ
  + isEqN' 216 108 + isEqN' 108 108 + isEqN' 72 108 + isEqN' 288 108
  + isEqN' 54 108 + isEqN' 36 108 + isEqN' 144 108 + isEqN' 24 108
  + isEqN' 96 108 + isEqN' 384 108
  -- ddd ddχ ddκ dχχ dχκ dκκ χχχ χχκ χκκ κκκ
  + isEqN' 27 108 + isEqN' 18 108 + isEqN' 72 108 + isEqN' 12 108
  + isEqN' 48 108 + isEqN' 192 108 + isEqN' 8 108 + isEqN' 32 108
  + isEqN' 128 108 + isEqN' 512 108

-- § Exactly one degree-≤3 monomial has value 108.
theorem-108-unique-mono : hits108 ≡ 1
theorem-108-unique-mono = refl

-- § The unique survivor: E2·d = 36·3 = 108.
unique-108-monomial : ℕ
unique-108-monomial = (edgeCountK4 * edgeCountK4) * degree-K4

law-unique-108-value : unique-108-monomial ≡ 108
law-unique-108-value = refl

-- § Closure: 1836 = (E2·d) · F2.
law-proton-as-Eed-F2 : unique-108-monomial * F2 ≡ 1836
law-proton-as-Eed-F2 = refl

-- § Prime-stratum localization at 17.
law-V-coprime-17 : gcd vertexCountK4 17 ≡ 1
law-V-coprime-17 = refl
law-E-coprime-17 : gcd edgeCountK4 17 ≡ 1
law-E-coprime-17 = refl
law-d-coprime-17 : gcd degree-K4 17 ≡ 1
law-d-coprime-17 = refl
law-χ-coprime-17 : gcd eulerChar-computed 17 ≡ 1
law-χ-coprime-17 = refl
law-F-coprime-17 : gcd faceCountK4 17 ≡ 1
law-F-coprime-17 = refl

```

```

law- $\kappa$ -coprime-17 : gcd  $\kappa$ -discrete      17  $\equiv$  1
law- $\kappa$ -coprime-17 = refl
law-F2-carries-17 : gcd F2              17  $\equiv$  17
law-F2-carries-17 = refl

-- § Handshake alias:  $\chi^2 \cdot d^3 \equiv E^2 \cdot d$  (the two written proton forms agree).
law-handshake-proton : (eulerChar-computed * eulerChar-computed)
                      * (degree-K4 * degree-K4 * degree-K4)
                       $\equiv$  unique-108-monomial
law-handshake-proton = refl

-- § Forcing record for the proton ratio.
record ProtonRatioForcing : Set where
  field
    -- § 17 is sourced uniquely from F2.
    V-coprime-17      : gcd vertexCountK4 17  $\equiv$  1
    E-coprime-17      : gcd edgeCountK4    17  $\equiv$  1
    d-coprime-17      : gcd degree-K4      17  $\equiv$  1
     $\chi$ -coprime-17     : gcd eulerChar-computed 17  $\equiv$  1
    F-coprime-17      : gcd faceCountK4    17  $\equiv$  1
     $\kappa$ -coprime-17     : gcd  $\kappa$ -discrete      17  $\equiv$  1
    F2-carries-17    : gcd F2              17  $\equiv$  17
    -- § Residue 108 has a unique degree- $\leq 3$  monomial.
    residue-unique     : hits108  $\equiv$  1
    residue-witness    : unique-108-monomial  $\equiv$  108
    -- § Closure via the handshake alias.
    proton-as-EedF2  : unique-108-monomial * F2  $\equiv$  1836
    handshake-alias    : (eulerChar-computed * eulerChar-computed)
                      * (degree-K4 * degree-K4 * degree-K4)
                       $\equiv$  unique-108-monomial

theorem-proton-ratio-forcing : ProtonRatioForcing
theorem-proton-ratio-forcing = record
{ V-coprime-17 = refl
; E-coprime-17 = refl
; d-coprime-17 = refl
;  $\chi$ -coprime-17 = refl
; F-coprime-17 = refl
;  $\kappa$ -coprime-17 = refl
; F2-carries-17 = refl
; residue-unique = refl
; residue-witness = refl
; proton-as-EedF2 = refl
; handshake-alias = refl
}

```

Correction fraction irreducibility

The ledger also determines four correction fractions. The only point needed here is that each numerator-denominator pair is already in lowest terms.

The four denominators are $\mathcal{C}^2$, $\kappa d \mathcal{C}^2$, $\kappa \mathcal{C}^2$, and $\kappa^2 \mathcal{C}^2$, and the corresponding numerators are χ , 1, d^2 , and $(p + \chi)^2$.

The gcd computations below show that each pair is coprime.

```
-- §§ Correction fraction irreducibility – gcd checks on forced numerator/denominator pairs.

-- §  $C^2 = 137^2 = 18769$ .
simplex-eval-squared :  $\mathbb{N}$ 
simplex-eval-squared = simplex-eval * simplex-eval

law-simplex-eval-sq : simplex-eval-squared  $\equiv 18769$ 
law-simplex-eval-sq = refl

-- § Denominator product  $\kappa \cdot C^2 = 150152$ .
denom-kC2 :  $\mathbb{N}$ 
denom-kC2 =  $\kappa$ -discrete * simplex-eval-squared

law-denom-kC2 : denom-kC2  $\equiv 150152$ 
law-denom-kC2 = refl

--  $\kappa \cdot d \cdot C^2 = 450456$ 
denom-kdC2 :  $\mathbb{N}$ 
denom-kdC2 =  $\kappa$ -discrete * degree-K4 * simplex-eval-squared

law-denom-kdC2 : denom-kdC2  $\equiv 450456$ 
law-denom-kdC2 = refl

--  $\kappa^2 \cdot C^2 = 1201216$ 
denom-k2C2 :  $\mathbb{N}$ 
denom-k2C2 = ( $\kappa$ -discrete *  $\kappa$ -discrete) * simplex-eval-squared

law-denom-k2C2 : denom-k2C2  $\equiv 1201216$ 
law-denom-k2C2 = refl

-- § Channel count:  $p + \chi = 11 + 2 = 13$ .
loop-plus-chi :  $\mathbb{N}$ 
loop-plus-chi = loop-numerator + eulerChar-computed

law-loop-plus-chi : loop-plus-chi  $\equiv 13$ 
law-loop-plus-chi = refl

-- § Squared channel count:  $13^2 = 169$ .
loop-plus-chi-squared :  $\mathbb{N}$ 
loop-plus-chi-squared = loop-plus-chi * loop-plus-chi

law-loop-plus-chi-sq : loop-plus-chi-squared  $\equiv 169$ 
law-loop-plus-chi-sq = refl

-- § Irreducibility:  $\gcd(\chi, C^2) = 1$ .
law-irred-chi-C2 : gcd eulerChar-computed simplex-eval-squared  $\equiv 1$ 
law-irred-chi-C2 = refl

--  $\gcd(1, \kappa d C^2) = \gcd(1, 450456) = 1$ 
law-irred-1-kdC2 : gcd 1 denom-kdC2  $\equiv 1$ 
law-irred-1-kdC2 = refl

--  $\gcd(d^2, \kappa C^2) = \gcd(9, 150152) = 1$ 
law-irred-d2-kC2 : gcd (degree-K4 * degree-K4) denom-kC2  $\equiv 1$ 
law-irred-d2-kC2 = refl

--  $\gcd((p+\chi)^2, \kappa^2 C^2) = \gcd(169, 1201216) = 1$ 
law-irred-pchi2-k2C2 : gcd loop-plus-chi-squared denom-k2C2  $\equiv 1$ 
law-irred-pchi2-k2C2 = refl
```

Boundary channel forcing

The boundary-shifted channel count is $p + \chi = 13$. In this subsection the shift is read from the two-branch structure of the distinction cover.

The cover of a distinction has exactly two branches, so its branch count is $\chi = 2$. Since the loop numerator $p = E + d + \chi$ already contains one copy of χ , one boundary traversal contributes one more and gives $p + \chi = E + d + 2\chi = 13$.

The self-coupling then squares this quantity, producing $(p + \chi)^2 = 169$. The record `BoundaryChannelForcing` packages these identities.

```
-- §§ Boundary channel forcing – the cover branch count forces  $p + \chi$ .

-- § The cover of Two-distinction has exactly 2 branches ( $\text{inj}_1$  and  $\text{inj}_2$ ).
cover-branch-count :  $\mathbb{N}$ 
cover-branch-count = 2

-- § The branch count equals the Euler characteristic.
law-cover-branches-is-chi : cover-branch-count  $\equiv$  eulerChar-computed
law-cover-branches-is-chi = refl

-- § The loop numerator  $p$  already contains one copy of  $\chi$ .
law-loop-contains-chi : loop-numerator  $\equiv$  edgeCountK4 + degree-K4 + eulerChar-computed
law-loop-contains-chi = refl

-- § One boundary traversal adds  $\chi$ : boundary-shifted channel =  $p + \chi = E + d + 2\chi$ .
law-weak-channel-double-chi :
  loop-plus-chi  $\equiv$  edgeCountK4 + degree-K4 + eulerChar-computed + eulerChar-computed
law-weak-channel-double-chi = refl

-- § The two copies of  $\chi$  in the boundary-shifted channel equal the cover branch count.
law-chi-copies-match-branches :
  eulerChar-computed + eulerChar-computed  $\equiv$  cover-branch-count + cover-branch-count
law-chi-copies-match-branches = refl

-- § Full record: boundary channel forcing.
record BoundaryChannelForcing : Set where
  field
    -- § Cover branch count.
    branches-is-chi : cover-branch-count  $\equiv$  eulerChar-computed
    -- § Loop numerator carries one  $\chi$ .
    loop-has-one-chi : loop-numerator  $\equiv$  edgeCountK4 + degree-K4 + eulerChar-computed
    -- § Boundary traversal adds one  $\chi$ .
    traversal-adds-chi : loop-plus-chi  $\equiv$  loop-numerator + eulerChar-computed
    -- § Boundary-shifted channel contains two copies of  $\chi$ .
    double-chi : loop-plus-chi  $\equiv$  edgeCountK4 + degree-K4 + eulerChar-computed + eulerChar-computed
    -- § Self-coupling squares the channel.
    coupling-squares : loop-plus-chi-squared  $\equiv$  loop-plus-chi * loop-plus-chi
    -- § Final value.
    shifted-numerator : loop-plus-chi-squared  $\equiv$  169

theorem-boundary-channel-forcing : BoundaryChannelForcing
theorem-boundary-channel-forcing = record
```



```

{ branches-is-chi    = refl
; loop-has-one-chi  = refl
; traversal-adds-chi = refl
; double-chi        = refl
; coupling-squares  = refl
; shifted-numerator = refl
}

```

Factor divisibility in the invariant ledger

The invariants also satisfy fixed divisibility relations with the bulk and extended volumes. These are straightforward arithmetic consequences of the values already established.

The gcd computations below show that V , χ , d , and E divide the bulk denominator, that κ divides the extended denominator, and that F_2 is coprime to both.

```

-- §§ Factor divisibility: which invariants divide the canonical volumes?

-- § V divides bulk volume: gcd(4, 72) = 4.
law-V-divides-bulk : gcd vertexCountK4 loop-denom-bulk ≡ vertexCountK4
law-V-divides-bulk = refl

-- §  $\chi$  divides bulk volume: gcd(2, 72) = 2.
law- $\chi$ -divides-bulk-vol : gcd eulerChar-computed loop-denom-bulk ≡ eulerChar-computed
law- $\chi$ -divides-bulk-vol = refl

-- § d divides bulk volume: gcd(3, 72) = 3.
law-d-divides-bulk : gcd degree-K4 loop-denom-bulk ≡ degree-K4
law-d-divides-bulk = refl

-- §  $\kappa$  divides extended volume: gcd(8, 576) = 8.
law- $\kappa$ -divides-ext : gcd  $\kappa$ -discrete loop-denom-extended ≡  $\kappa$ -discrete
law- $\kappa$ -divides-ext = refl

-- §  $\chi$  enters the tree-level spectral term: gcd( $\chi$ ,  $V^d \cdot \chi$ ) =  $\chi$ .
law- $\chi$ -in-tree : gcd eulerChar-computed ((vertexCountK4 ^ degree-K4) * eulerChar-computed) ≡ eulerChar-computed
law- $\chi$ -in-tree = refl

-- § E is coprime to bulk volume: gcd(6, 72) = 6. (E divides 72 – it is NOT coprime.)
law-E-divides-bulk : gcd edgeCountK4 loop-denom-bulk ≡ edgeCountK4
law-E-divides-bulk = refl

-- §  $F_2$  is coprime to bulk volume: gcd(17, 72) = 1. ( $F_2$  is genuinely coprime.)
law-F2-coprime-bulk : gcd F2 loop-denom-bulk ≡ 1
law-F2-coprime-bulk = refl

-- §  $F_2$  is coprime to extended volume: gcd(17, 576) = 1.
law-F2-coprime-ext : gcd F2 loop-denom-extended ≡ 1
law-F2-coprime-ext = refl

-- § The triple {d, E,  $F_2$ } yields a product coprime to the loop numerator.
law-dEF2-product : degree-K4 * edgeCountK4 * F2 ≡ 306
law-dEF2-product = refl

law-dEF2-coprime-11 : gcd (edgeCountK4 + degree-K4 + eulerChar-computed) (degree-K4 * edgeCountK4 * F2) ≡ 1
law-dEF2-coprime-11 = refl

```

Coupling volume derivation: the spectral filter

The tree evaluation induces a simple divisibility test against the spectral base $V^d = 64$. This partitions the six invariants into those absorbed by the spectral base and those that remain outside it.

The absorbed class is $\{V, \chi, \kappa\}$, all powers of 2. The surviving class is $\{d, E, F_2\}$, which carries the odd-prime content not dividing 64.

The product of the surviving class is $dEF_2 = 306$, and the record below packages this partition together with its basic arithmetic consequences.

```
-- §§ Coupling volume derivation: the spectral base V^d consumes {V, χ, κ}.

-- § The spectral base: V^d = 4^3 = 64.
spectral-base : ℕ
spectral-base = vertexCountK4 ^ degree-K4

law-spectral-base : spectral-base ≡ 64
law-spectral-base = refl

-- § Consumed: V divides V^d. gcd(4, 64) = 4.
law-V-consumed-by-tree : gcd vertexCountK4 spectral-base ≡ vertexCountK4
law-V-consumed-by-tree = refl

-- § Consumed: χ divides V^d. gcd(2, 64) = 2.
law-χ-consumed-by-tree : gcd eulerChar-computed spectral-base ≡ eulerChar-computed
law-χ-consumed-by-tree = refl

-- § Consumed: κ divides V^d. gcd(8, 64) = 8.
law-κ-consumed-by-tree : gcd κ-discrete spectral-base ≡ κ-discrete
law-κ-consumed-by-tree = refl

-- § Survives: d does NOT divide V^d. gcd(3, 64) = 1, but d = 3. So 1 ≡ 3 → ⊥.
law-d-survives-tree : gcd degree-K4 spectral-base ≡ degree-K4 → ⊥
law-d-survives-tree ()

-- § Survives: E does NOT divide V^d. gcd(6, 64) = 2, but E = 6. So 2 ≡ 6 → ⊥.
law-E-survives-tree : gcd edgeCountK4 spectral-base ≡ edgeCountK4 → ⊥
law-E-survives-tree ()

-- § Survives: F2 does NOT divide V^d. gcd(17, 64) = 1, but F2 = 17. So 1 ≡ 17 → ⊥.
law-F2-survives-tree : gcd F2 spectral-base ≡ F2 → ⊥
law-F2-survives-tree ()

-- § Classification record: spectral filter forces the coupling volume.
record CouplingVolumeDerivation : Set where
  field
    -- § The spectral base.
    base-value : spectral-base ≡ 64
    -- § Three invariants divide V^d (consumed by tree formula).
    V-consumed : gcd vertexCountK4 spectral-base ≡ vertexCountK4
    χ-consumed : gcd eulerChar-computed spectral-base ≡ eulerChar-computed
    κ-consumed : gcd κ-discrete spectral-base ≡ κ-discrete
    -- § Three invariants do NOT divide V^d (survive the tree filter).
```

```

d-survives : gcd degree-K4 spectral-base ≡ degree-K4 → ⊥
E-survives : gcd edgeCountK4 spectral-base ≡ edgeCountK4 → ⊥
F2-survives : gcd F2 spectral-base ≡ F2 → ⊥
-- § The survivor product.
survivor-product : degree-K4 * edgeCountK4 * F2 ≡ 306
-- § Irreducibility with loop numerator.
survivor-coprime : gcd loop-numerator (degree-K4 * edgeCountK4 * F2) ≡ 1

theorem-coupling-volume-derivation : CouplingVolumeDerivation
theorem-coupling-volume-derivation = record
{ base-value = refl
; V-consumed = refl
; χ-consumed = refl
; κ-consumed = refl
; d-survives = λ ()
; E-survives = λ ()
; F2-survives = λ ()
; survivor-product = refl
; survivor-coprime = refl
}

```

The spectral lattice

The consumed-surviving partition has an internal closure property: $V\chi\kappa = V^d = 64$. The total product of the six invariants therefore factors as spectral base times coupling volume.

Because the partition is exhaustive, the total product of all six invariants decomposes along the partition boundary without remainder:

$$V \cdot E \cdot d \cdot \chi \cdot \kappa \cdot F_2 = \underbrace{(V \cdot \chi \cdot \kappa)}_{= V^d = 64} \cdot \underbrace{(d \cdot E \cdot F_2)}_{= 306} = 19\,584.$$

The three canonical volumes are also related by pairwise gcd identities landing again on ledger invariants:

$$\begin{aligned} \gcd(V^d, V \cdot E \cdot d) &= 8 = \kappa, \\ \gcd(d \cdot E \cdot F_2, V \cdot E \cdot d) &= 18 = d \cdot E, \\ \gcd(V^d, d \cdot E \cdot F_2) &= 2 = \chi. \end{aligned}$$

The record `SpectralPartition` packages this fixed-point identity, the total factorization, and the three gcd laws.

```

-- §§ The spectral lattice: self-closure and GCD structure of canonical volumes.

-- § Self-closure: the consumed class product is the spectral base.
law-consumed-product : vertexCountK4 * eulerChar-computed * κ-discrete ≡ spectral-base
law-consumed-product = refl

```

```

-- § Total product of all six invariants.
total-invariant-product :  $\mathbb{N}$ 
total-invariant-product =
  vertexCountK4 * edgeCountK4 * degree-K4 *
  eulerChar-computed *  $\kappa$ -discrete *  $F_2$ 

law-total-invariant-product : total-invariant-product  $\equiv$  19584
law-total-invariant-product = refl

-- § The total decomposes along the partition: spectral  $\times$  coupling.
law-total-decomposition :
  total-invariant-product  $\equiv$  spectral-base * (degree-K4 * edgeCountK4 *  $F_2$ )
law-total-decomposition = refl

-- § GCD lattice: every pairwise bridge is a  $K_4$  invariant.

--  $\gcd(V^d, V \cdot E \cdot d) = 8 = \kappa$ .
law-gcd-spectral-bulk : gcd spectral-base loop-denom-bulk  $\equiv$   $\kappa$ -discrete
law-gcd-spectral-bulk = refl

--  $\gcd(d \cdot E \cdot F_2, V \cdot E \cdot d) = 18 = d \cdot E$ .
law-gcd-coupling-bulk :
  gcd (degree-K4 * edgeCountK4 *  $F_2$ ) loop-denom-bulk  $\equiv$  degree-K4 * edgeCountK4
law-gcd-coupling-bulk = refl

--  $\gcd(V^d, d \cdot E \cdot F_2) = 2 = \chi$ .
law-gcd-spectral-coupling :
  gcd spectral-base (degree-K4 * edgeCountK4 *  $F_2$ )  $\equiv$  eulerChar-computed
law-gcd-spectral-coupling = refl

-- § Classification record: the lattice is sealed.
record SpectralPartition : Set where
  field
    -- § Self-closure: consumed product = spectral base (fixed point).
    consumed-is-spectral : vertexCountK4 * eulerChar-computed *  $\kappa$ -discrete  $\equiv$  spectral-base
    -- § Total product decomposes along the partition boundary.
    total-factors       : total-invariant-product  $\equiv$  spectral-base * (degree-K4 * edgeCountK4 *  $F_2$ )
    -- § Pairwise GCDs land on  $K_4$  invariants.
    gcd-spectral-bulk   : gcd spectral-base loop-denom-bulk  $\equiv$   $\kappa$ -discrete
    gcd-coupling-bulk   : gcd (degree-K4 * edgeCountK4 *  $F_2$ ) loop-denom-bulk  $\equiv$  degree-K4 * edgeCountK4
    gcd-spectral-coupling : gcd spectral-base (degree-K4 * edgeCountK4 *  $F_2$ )  $\equiv$  eulerChar-computed

theorem-spectral-partition : SpectralPartition
theorem-spectral-partition = record
  { consumed-is-spectral = refl
  ; total-factors       = refl
  ; gcd-spectral-bulk   = refl
  ; gcd-coupling-bulk   = refl
  ; gcd-spectral-coupling = refl
  }

```

Downstream arithmetic realizers

The companion volume may interpret only what has already become a term inside this one. The following records therefore carry the arithmetic cores that later receive

interpretive names: the spectral-index quotient, the Fermat half-projection, the scaled density quotient, the two loop scales, the strong volume correction, and the internal identities of the coupling volume. No particle name, gauge-sector name, or measured quantity is used in these records.

```
-- §§ Neutral downstream realizers: arithmetic terms later interpreted by Form.
```

```
-- § The tree spectral term  $V^d \cdot \chi = 128$ .
```

```
void-spectral-topological-term : ℕ
```

```
void-spectral-topological-term = (vertexCountK4 ^ degree-K4) * eulerChar-computed
```

```
law-void-spectral-topological-term-128 : void-spectral-topological-term ≡ 128
```

```
law-void-spectral-topological-term-128 = refl
```

```
-- § Spectral quotient:  $(VE - 1)/(VE) + 1/(V^d \chi)$ .
```

```
void-ns-capacity : ℕ
```

```
void-ns-capacity = vertexCountK4 * edgeCountK4
```

```
law-void-ns-capacity-24 : void-ns-capacity ≡ 24
```

```
law-void-ns-capacity-24 = refl
```

```
void-ns-bare-numerator : ℕ
```

```
void-ns-bare-numerator = void-ns-capacity - 1
```

```
law-void-ns-bare-numerator-23 : void-ns-bare-numerator ≡ 23
```

```
law-void-ns-bare-numerator-23 = refl
```

```
void-ns-loop-denom : ℕ
```

```
void-ns-loop-denom = void-spectral-topological-term
```

```
law-void-ns-loop-denom-128 : void-ns-loop-denom ≡ 128
```

```
law-void-ns-loop-denom-128 = refl
```

```
void-ns-numerator : ℕ
```

```
void-ns-numerator = void-ns-bare-numerator * void-ns-loop-denom + void-ns-capacity
```

```
void-ns-denominator : ℕ
```

```
void-ns-denominator = void-ns-capacity * void-ns-loop-denom
```

```
law-void-ns-numerator-2968 : void-ns-numerator ≡ 2968
```

```
law-void-ns-numerator-2968 = refl
```

```
law-void-ns-denominator-3072 : void-ns-denominator ≡ 3072
```

```
law-void-ns-denominator-3072 = refl
```

```
void-ns-scaled : ℕ
```

```
void-ns-scaled = (void-ns-numerator * 10000) div void-ns-denominator
```

```
law-void-ns-scaled-9661 : void-ns-scaled ≡ 9661
```

```
law-void-ns-scaled-9661 = refl
```

```
record SpectralIndexRealizer : Set where
```

```
field
```

```
capacity-is-VE : void-ns-capacity ≡ vertexCountK4 * edgeCountK4
```

```
capacity-is-24 : void-ns-capacity ≡ 24
```

```

bare-is-23      : void-ns-bare-numerator  $\equiv$  23
loop-denom-is-128 : void-ns-loop-denom  $\equiv$  128
numerator-is-2968 : void-ns-numerator  $\equiv$  2968
denominator-is-3072 : void-ns-denominator  $\equiv$  3072
scaled-is-9661 : void-ns-scaled  $\equiv$  9661

```

```
theorem-spectral-index-realizer : SpectralIndexRealizer
```

```
theorem-spectral-index-realizer = record
```

```

{ capacity-is-VE      = refl
; capacity-is-24      = refl
; bare-is-23          = refl
; loop-denom-is-128   = refl
; numerator-is-2968   = refl
; denominator-is-3072 = refl
; scaled-is-9661      = refl
}

```

```
-- § Fermat half-projection:  $\text{floor}(F_3 / \chi) = 128$ .
```

```
void-F3-half-projection :  $\mathbb{N}$ 
```

```
void-F3-half-projection =  $F_3 \text{ div } \mathbb{N} \text{ eulerChar-computed}$ 
```

```
law-void-F3-half-projection-128 : void-F3-half-projection  $\equiv$  128
```

```
law-void-F3-half-projection-128 = refl
```

```
record FermatHalfProjectionRealizer : Set where
field
```

```
source-is-F3-257 :  $F_3 \equiv$  257
```

```
divisor-is-chi-2 : eulerChar-computed  $\equiv$  2
```

```
projection-is-128 : void-F3-half-projection  $\equiv$  128
```

```
theorem-fermat-half-projection-realizer : FermatHalfProjectionRealizer
```

```
theorem-fermat-half-projection-realizer = record
```

```

{ source-is-F3-257 = refl
; divisor-is-chi-2 = refl
; projection-is-128 = refl
}

```

```
-- § Scaled density quotient:  $V/(2\pi\chi)$ , represented as integer quotient with remainder.
```

```
void-two-pi-scaled :  $\mathbb{N}$ 
```

```
void-two-pi-scaled =  $2 * (19108 + 12308)$ 
```

```
law-void-two-pi-scaled-62832 : void-two-pi-scaled  $\equiv$  62832
```

```
law-void-two-pi-scaled-62832 = refl
```

```
void-density-chi :  $\mathbb{N}$ 
```

```
void-density-chi = eulerChar-computed
```

```
void-gauss-bonnet-curvature :  $\mathbb{N}$ 
```

```
void-gauss-bonnet-curvature = void-two-pi-scaled * void-density-chi
```

```
law-void-gauss-bonnet-curvature-125664 : void-gauss-bonnet-curvature  $\equiv$  125664
```

```
law-void-gauss-bonnet-curvature-125664 = refl
```

```
void-density-scaling :  $\mathbb{N}$ 
```

```
void-density-scaling = 10000
```

```

void-density-scaled-K4 : ℕ
void-density-scaled-K4 = 3183

void-density-remainder-K4 : ℕ
void-density-remainder-K4 = 11488

law-void-density-K4-quotient :
void-density-scaled-K4 * void-gauss-bonnet-curvature + void-density-remainder-K4
≡ vertexCountK4 * void-density-scaling * void-density-scaling
law-void-density-K4-quotient = refl

record DensityScaledRealizer : Set where
field
  numerator-is-V      : vertexCountK4 ≡ 4
  chi-is-2            : void-density-chi ≡ 2
  two-pi-is-62832     : void-two-pi-scaled ≡ 62832
  curvature-is-125664 : void-gauss-bonnet-curvature ≡ 125664
  scaled-is-3183      : void-density-scaled-K4 ≡ 3183
  quotient-with-remainder :
    void-density-scaled-K4 * void-gauss-bonnet-curvature + void-density-remainder-K4
    ≡ vertexCountK4 * void-density-scaling * void-density-scaling

theorem-density-scaled-realizer : DensityScaledRealizer
theorem-density-scaled-realizer = record
{ numerator-is-V      = refl
; chi-is-2            = refl
; two-pi-is-62832     = refl
; curvature-is-125664 = refl
; scaled-is-3183      = refl
; quotient-with-remainder = refl
}

-- § Neutral loop scales: no physical sector names are introduced here.
record LoopScaleRealizer : Set where
field
  numerator-is-11      : loop-numerator ≡ 11
  bulk-denom-is-72     : loop-denom-bulk ≡ 72
  extended-denom-is-576 : loop-denom-extended ≡ 576
  extended-is-kappa-bulk : loop-denom-extended ≡ loop-denom-bulk * κ-discrete
  numerator-bulk-coprime : gcd loop-numerator loop-denom-bulk ≡ 1
  numerator-extended-coprime : gcd loop-numerator loop-denom-extended ≡ 1

theorem-loop-scale-realizer : LoopScaleRealizer
theorem-loop-scale-realizer = record
{ numerator-is-11      = refl
; bulk-denom-is-72     = refl
; extended-denom-is-576 = refl
; extended-is-kappa-bulk = refl
; numerator-bulk-coprime = refl
; numerator-extended-coprime = refl
}

-- § Strong volume arithmetic: d^(1+bd) over κd²C².

```

```

void-strong-num-bulk : ℕ
void-strong-num-bulk = degree-K4 ^ bulk-exponent

void-strong-num-boundary : ℕ
void-strong-num-boundary = degree-K4 ^ boundary-exponent

law-void-strong-num-bulk-3 : void-strong-num-bulk ≡ 3
law-void-strong-num-bulk-3 = refl

law-void-strong-num-boundary-81 : void-strong-num-boundary ≡ 81
law-void-strong-num-boundary-81 = refl

law-void-strong-volume-factor :
  void-strong-num-boundary ≡ void-strong-num-bulk * (degree-K4 ^ degree-K4)
law-void-strong-volume-factor = refl

void-strong-common-denom : ℕ
void-strong-common-denom = κ-discrete * degree-K4 * degree-K4 * simplex-eval-squared

law-void-strong-common-denom-1351368 : void-strong-common-denom ≡ 1351368
law-void-strong-common-denom-1351368 = refl

void-strong-bulk-reduced-denom : ℕ
void-strong-bulk-reduced-denom = κ-discrete * degree-K4 * simplex-eval-squared

law-void-strong-bulk-reduced-denom-450456 : void-strong-bulk-reduced-denom ≡ 450456
law-void-strong-bulk-reduced-denom-450456 = refl

void-strong-boundary-reduced-denom : ℕ
void-strong-boundary-reduced-denom = κ-discrete * simplex-eval-squared

law-void-strong-boundary-reduced-denom-150152 : void-strong-boundary-reduced-denom ≡ 150152
law-void-strong-boundary-reduced-denom-150152 = refl

record StrongVolumeRealizer : Set where
  field
    bulk-num-is-3      : void-strong-num-bulk ≡ 3
    boundary-num-is-81 : void-strong-num-boundary ≡ 81
    volume-factor      : void-strong-num-boundary ≡ void-strong-num-bulk * (degree-K4 ^ degree-K4)
    common-denom       : void-strong-common-denom ≡ 1351368
    bulk-reduced-denom : void-strong-bulk-reduced-denom ≡ 450456
    boundary-reduced-denom : void-strong-boundary-reduced-denom ≡ 150152
    bulk-irred         : gcd 1 void-strong-bulk-reduced-denom ≡ 1
    boundary-irred     : gcd (degree-K4 * degree-K4) void-strong-boundary-reduced-denom ≡ 1

theorem-strong-volume-realizer : StrongVolumeRealizer
theorem-strong-volume-realizer = record
  { bulk-num-is-3      = refl
  ; boundary-num-is-81 = refl
  ; volume-factor      = refl
  ; common-denom       = refl
  ; bulk-reduced-denom = refl
  ; boundary-reduced-denom = refl
  ; bulk-irred         = refl
  ; boundary-irred     = refl

```



```

}

-- § Coupling-volume identities behind the product d·E·F2 = 306.
void-coupling-volume : ℕ
void-coupling-volume = degree-K4 * edgeCountK4 * F2

law-void-coupling-volume-306 : void-coupling-volume ≡ 306
law-void-coupling-volume-306 = refl

law-void-F2-from-loop : loop-numerator * degree-K4 + 1 ≡ eulerChar-computed * F2
law-void-F2-from-loop = refl

law-void-coupling-volume-cubic :
  void-coupling-volume ≡ loop-numerator * (degree-K4 * degree-K4 * degree-K4) + degree-K4 * degree-K4
law-void-coupling-volume-cubic = refl

law-void-d-cubed-bosonic :
  degree-K4 * degree-K4 * degree-K4 - 1 ≡ vertexCountK4 * edgeCountK4 + eulerChar-computed
law-void-d-cubed-bosonic = refl

law-void-coupling-complement-decomp :
  void-coupling-volume - loop-numerator
  ≡ loop-numerator * (degree-K4 * degree-K4 * degree-K4 - 1) + degree-K4 * degree-K4
law-void-coupling-complement-decomp = refl

law-void-coupling-completeness :
  (void-coupling-volume - loop-numerator) + loop-numerator ≡ void-coupling-volume
law-void-coupling-completeness = refl

record CouplingVolumeStructureRealizer : Set where
  field
    volume-is-306 : void-coupling-volume ≡ 306
    F2-derived : loop-numerator * degree-K4 + 1 ≡ eulerChar-computed * F2
    volume-cubic : void-coupling-volume ≡ loop-numerator * (degree-K4 * degree-K4 * degree-K4) + degree-K4 * degree-K4
    bosonic-dimension : degree-K4 * degree-K4 * degree-K4 - 1 ≡ vertexCountK4 * edgeCountK4 + eulerChar-computed
    complement-decomp : void-coupling-volume - loop-numerator ≡ loop-numerator * (degree-K4 * degree-K4 * degree-K4 - 1) + degree-K4 * degree-K4
    completeness : (void-coupling-volume - loop-numerator) + loop-numerator ≡ void-coupling-volume

theorem-coupling-volume-structure-realizer : CouplingVolumeStructureRealizer
theorem-coupling-volume-structure-realizer = record
  { volume-is-306 = refl
  ; F2-derived = refl
  ; volume-cubic = refl
  ; bosonic-dimension = refl
  ; complement-decomp = refl
  ; completeness = refl
  }

```

Correction tower completeness

Four correction fractions derived so far are collected here into a single record. What matters at this stage is only that their numerators, denominators, and irreducibility statements are all fixed.

This gives a closed list of the correction terms used later in the text.

```
-- §§ Correction tower completeness: four fractions, no more.

record CorrectionTowerComplete : Set where
  field
  -- § The four numerators.
  num-boundary : eulerChar-computed  $\equiv$  2
  num-first    : 1  $\equiv$  1
  num-second   : degree-K4 * degree-K4  $\equiv$  9
  num-coupling : loop-plus-chi-squared  $\equiv$  169
  -- § The four denominators.
  den-boundary : simplex-eval-squared  $\equiv$  18769
  den-first    : denom-kdC2  $\equiv$  450456
  den-second   : denom-kC2  $\equiv$  150152
  den-coupling : denom-k2C2  $\equiv$  1201216
  -- § All four fractions are irreducible.
  irred-boundary : gcd eulerChar-computed simplex-eval-squared  $\equiv$  1
  irred-first    : gcd 1 denom-kdC2  $\equiv$  1
  irred-second   : gcd (degree-K4 * degree-K4) denom-kC2  $\equiv$  1
  irred-coupling : gcd loop-plus-chi-squared denom-k2C2  $\equiv$  1

theorem-correction-tower-complete : CorrectionTowerComplete
theorem-correction-tower-complete = record
{ num-boundary = refl
; num-first    = refl
; num-second   = refl
; num-coupling = refl
; den-boundary = refl
; den-first    = refl
; den-second   = refl
; den-coupling = refl
; irred-boundary = refl
; irred-first  = refl
; irred-second = refl
; irred-coupling = refl
}
```

Sign forcing

The next step classifies the signs of the four correction terms. The classification used here depends only on whether the corresponding numerator involves the degree d .

The d -free numerators are χ and 1, while the d -bearing numerators are d^2 and $(p + \chi)^2$. This yields the sign pattern $(+1, +1, -1, -1)$.

The record `SignForcing` packages this dichotomy together with its translation into the two orientation classes.

```
-- §§ Sign forcing: numerator d-content determines correction signs.

-- § Numerator d-content classification.
data NumeratorClass : Set where
```

```

d-free : NumeratorClass -- no d-dependence in numerator  $\rightarrow \sigma = +1$ 
d-bearing : NumeratorClass -- numerator depends on d  $\rightarrow \sigma = -1$ 

-- § Type-level closure: NumeratorClass has exactly two inhabitants.
NumeratorClass-exhaustive :
   $\forall (n : \text{NumeratorClass}) \rightarrow (n \equiv \text{d-free}) \uplus (n \equiv \text{d-bearing})$ 
NumeratorClass-exhaustive d-free = inj1 refl
NumeratorClass-exhaustive d-bearing = inj2 refl

-- § The integer sign forced by d-content.
classify-sign : NumeratorClass  $\rightarrow \mathbb{Z}$ 
classify-sign d-free = +suc 0 -- +1
classify-sign d-bearing = -suc 0 -- -1

-- § Bridge: d-content determines TwoOrientation.
numerator-to-orientation : NumeratorClass  $\rightarrow$  TwoOrientation
numerator-to-orientation d-free = orient-preserve
numerator-to-orientation d-bearing = orient-swap

-- §§ Record: sign forcing for the four correction terms.
record SignForcing : Set where
  field
    -- § The four correction-term signs.
    sign-boundary : classify-sign d-free  $\equiv$  +suc 0 --  $\chi/C^2$ :  $\chi = 2$ , d-free
    sign-mass : classify-sign d-free  $\equiv$  +suc 0 --  $1/(\kappa d C^2)$ : 1, d-free
    sign-degree : classify-sign d-bearing  $\equiv$  -suc 0 --  $d^2/(\kappa C^2)$ :  $d^2$ , d-bearing
    sign-coupling : classify-sign d-bearing  $\equiv$  -suc 0 --  $(p+\chi)^2/(\kappa^2 C^2)$ :  $\ni d$ , d-bearing
    -- § Bridge to TwoOrientation.
    bridge-free : numerator-to-orientation d-free  $\equiv$  orient-preserve
    bridge-bearing : numerator-to-orientation d-bearing  $\equiv$  orient-swap

theorem-sign-forcing : SignForcing
theorem-sign-forcing = record
  { sign-boundary = refl
  ; sign-mass = refl
  ; sign-degree = refl
  ; sign-coupling = refl
  ; bridge-free = refl
  ; bridge-bearing = refl
  }

```

Threshold to the invariant record

The preceding sections leave a sealed discrete record: the graph K_4 , its invariant ledger, the unit scale, the loop numerator, and the four correction fractions with their signs and denominators.

At this stage the arithmetic is closed. What remains open is only the external interpretation of these quantities, not their formal derivation.

Accordingly, the sections that follow move from the discrete record itself to the question of what that record presupposes.

The Record Presupposes Distinction

The argument has now reached its closing question. No new invariant is introduced in this section. The ledger has already been assembled, the observable criterion has been stated, the scale has been closed, and the correction arithmetic has been reduced to fixed records. What remains is to decide whether the direction of dependence can still be reversed.

One might try to read `K4Record` as the prior object and regard distinction as only one way of presenting it. That reading would reopen the whole chain at its endpoint. It would say that the record is the ground and distinction merely an interpretation of it. The formalization does not allow this reversal.

The reason is that `K4Record` is not merely a tuple of numerical invariants. Its fields are held together by identity witnesses, anchor laws, and singleton closure. Without the logical infrastructure supplied by distinction, those fields do not form an independent record. They lose the structure that makes them a record at all.

The formal point is therefore simple. From any inhabitant of `K4Record` one extracts a `Distinction`; under $\neg \text{Distinction}$ no such record can exist. Together with `K4-is-inevitable`, this yields an isomorphism between `Distinction` and `K4Record`. The invariant record does not ground distinction. It presupposes it.

```
-- § Any admissible K4 invariant record already consumes distinction.
record-presupposes-distinction : (p : K4Record) → Distinction
record-presupposes-distinction p with K4Record.anchor p
... | refl = Two-distinction

-- § Denying distinction makes the invariant record uninhabitable.
record-without-distinction-uninhabitable : ¬ Distinction → K4Record → ⊥
record-without-distinction-uninhabitable noDistinction p with K4Record.anchor p
... | refl = noDistinction Two-distinction

-- § Every invariant record therefore implies the irrefutability of distinction.
record-presupposes-distinction-irrefutable : (p : K4Record) → ¬¬ Distinction
record-presupposes-distinction-irrefutable p noDistinction =
  record-without-distinction-uninhabitable noDistinction p

-- § Every distinction witness inhabits the admissible invariant record.
K4-is-inevitable : Distinction → K4Record
K4-is-inevitable _ = canonical-rep

-- § The closed record and the canonical distinction dependency packaged together.
record K4Record↔CanonicalDistinction : Set₁ where
  field
    forward : Distinction → K4Record
    backward : K4Record → Distinction
    round-trip-K4 : (r : K4Record) → forward (backward r) ≡ r
    round-trip-D : (d : Distinction) → backward (forward d) ≡ Two-distinction

theorem-K4-record-canonical-distinction : K4Record↔CanonicalDistinction
```

```

theorem-K4-record-canonical-distinction = record
{ forward      = K4-is-inevitable
; backward     = record-presupposes-distinction
; round-trip-K4 =  $\lambda r \rightarrow$  K4Record-singleton (K4-is-inevitable
                                (record-presupposes-distinction r)) r
; round-trip-D =  $\lambda _ \rightarrow$  refl
}

```

The code makes both directions explicit. `K4-is-inevitable` sends a distinction to the canonical record, and `record-presupposes-distinction` reads a distinction back from any admissible record. The round trip on the record side closes by the singleton theorem; the round trip on the distinction side closes definitionally.

This is the formal end of the internal loop. The chain began with a distinction, reached the canonical invariant record, and then showed that the record itself cannot be understood without the distinction it seemed to have left behind.

Pythagorean Uniqueness of K_4

After the record has closed, one final independent check remains. It does not add a new invariant and it does not change the chain. It asks whether the same value 4 can be recovered from elementary arithmetic on complete graphs alone.

For each complete graph K_n , let E_n be its edge count and consider $E_n^2 + (2n)^2$. The question is whether this number is a perfect square. The cases $n = 2$ and $n = 3$ are excluded by explicit square sandwiches, $n = 4$ gives $6^2 + 8^2 = 10^2$, and every $n \geq 5$ is excluded uniformly between consecutive squares.

Thus the theorem `pythagoras-uniqueness` reaches the same conclusion as the singleton argument by a separate arithmetic route. At the end of the book, this result functions as a check on the endpoint: the same graph is recovered without using the representation record.

```

-- § Edge count of  $K_n$  by recursion.
K-edge-count :  $\mathbb{N} \rightarrow \mathbb{N}$ 
K-edge-count zero = 0
K-edge-count (suc m) = K-edge-count m + m

-- § Half-edge count:  $2n$ .
K-kappa :  $\mathbb{N} \rightarrow \mathbb{N}$ 
K-kappa n = 2 * n

-- § The Pythagorean candidate sum  $E^2 + \kappa^2$  as a function of  $n$ .
K-pythagorean-sum :  $\mathbb{N} \rightarrow \mathbb{N}$ 
K-pythagorean-sum n = let e = K-edge-count n
                        k = K-kappa n
                        in (e * e) + (k * k)

K2-not-pythagorean : K-pythagorean-sum 2  $\equiv$  17
K2-not-pythagorean = refl

```

```

K3-not-pythagorean : K-pythagorean-sum 3  $\equiv$  45
K3-not-pythagorean = refl

K4-is-pythagorean : K-pythagorean-sum 4  $\equiv$  100
K4-is-pythagorean = refl

K5-not-pythagorean : K-pythagorean-sum 5  $\equiv$  200
K5-not-pythagorean = refl

K6-not-pythagorean : K-pythagorean-sum 6  $\equiv$  369
K6-not-pythagorean = refl

K7-not-pythagorean : K-pythagorean-sum 7  $\equiv$  637
K7-not-pythagorean = refl

K8-not-pythagorean : K-pythagorean-sum 8  $\equiv$  1040
K8-not-pythagorean = refl

K9-not-pythagorean : K-pythagorean-sum 9  $\equiv$  1620
K9-not-pythagorean = refl

K10-not-pythagorean : K-pythagorean-sum 10  $\equiv$  2425
K10-not-pythagorean = refl

K11-not-pythagorean : K-pythagorean-sum 11  $\equiv$  3509
K11-not-pythagorean = refl

K12-not-pythagorean : K-pythagorean-sum 12  $\equiv$  4932
K12-not-pythagorean = refl

-- § Perfect-square predicate.
IsPerfectSquare :  $\mathbb{N} \rightarrow \text{Set}$ 
IsPerfectSquare m =  $\Sigma \mathbb{N} (\lambda k \rightarrow m \equiv k * k)$ 

-- § K4 inhabits the predicate (witness:  $10^2 = 100$ ).
K4-pythagorean-witness : IsPerfectSquare (K-pythagorean-sum 4)
K4-pythagorean-witness = 10 , refl

-- § Strict-order shorthand, scoped to this section.
private
infix 4 _<_
_<_ :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Set}$ 
m < n = suc m  $\leq$  n

-- § Arithmetic toolkit on BUILTIN  $\mathbb{N}$ .
private
+N-zero-r : (a :  $\mathbb{N}$ )  $\rightarrow a + 0 \equiv a$ 
+N-zero-r zero = refl
+N-zero-r (suc a) = cong suc (+N-zero-r a)

+N-suc-r : (a b :  $\mathbb{N}$ )  $\rightarrow a + \text{suc } b \equiv \text{suc } (a + b)$ 
+N-suc-r zero b = refl
+N-suc-r (suc a) b = cong suc (+N-suc-r a b)

+N-comm : (a b :  $\mathbb{N}$ )  $\rightarrow a + b \equiv b + a$ 

```

```

+N-comm a zero = +N-zero-r a
+N-comm a (suc b) = trans (+N-suc-r a b) (cong suc (+N-comm a b))

+N-assoc : (a b c : ℕ) → (a + b) + c ≡ a + (b + c)
+N-assoc zero b c = refl
+N-assoc (suc a) b c = cong suc (+N-assoc a b c)

*N-zero-r : (a : ℕ) → a * 0 ≡ 0
*N-zero-r zero = refl
*N-zero-r (suc a) = *N-zero-r a

*N-suc-r : (a b : ℕ) → a * suc b ≡ a + a * b
*N-suc-r zero b = refl
*N-suc-r (suc a) b =
  cong suc
    (trans (cong (b + _) (*N-suc-r a b))
      (trans (sym (+N-assoc b a (a * b)))
        (trans (cong (_ + (a * b)) (+N-comm b a))
          (+N-assoc a b (a * b))))))

*N-comm : (a b : ℕ) → a * b ≡ b * a
*N-comm zero b = sym (*N-zero-r b)
*N-comm (suc a) b =
  trans (cong (b + _) (*N-comm a b)) (sym (*N-suc-r b a))

*N-distribr-+ : (a b c : ℕ) → (a + b) * c ≡ a * c + b * c
*N-distribr-+ zero b c = refl
*N-distribr-+ (suc a) b c =
  trans (cong (c + _) (*N-distribr-+ a b c))
    (sym (+N-assoc c (a * c) (b * c)))

*N-distribl-+ : (a b c : ℕ) → a * (b + c) ≡ a * b + a * c
*N-distribl-+ a b c =
  trans (*N-comm a (b + c))
    (trans (*N-distribr-+ b c a)
      (trans (cong (_ + (c * a)) (*N-comm b a))
        (cong ((a * b) + _) (*N-comm c a))))

*N-assoc : (a b c : ℕ) → (a * b) * c ≡ a * (b * c)
*N-assoc zero b c = refl
*N-assoc (suc a) b c =
  trans (*N-distribr-+ b (a * b) c)
    (cong (b * c + _) (*N-assoc a b c))

+N-monor-≤ : (a : ℕ) → (b c : ℕ) → b ≤ c → a + b ≤ a + c
+N-monor-≤ zero b c p = p
+N-monor-≤ (suc a) b c p = s≤s (+N-monor-≤ a b c p)

+N-monol-≤ : (a b c : ℕ) → a ≤ b → a + c ≤ b + c
+N-monol-≤ a b c p =
  subst (≤ b + c) (+N-comm c a)
    (subst ((c + a) ≤ _) (+N-comm c b) (+N-monor-≤ c a b p))

+N-mono-≤ : (a b c d : ℕ) → a ≤ b → c ≤ d → a + c ≤ b + d
+N-mono-≤ a b c d p q = ≤-trans (+N-monol-≤ a b c p) (+N-monor-≤ b c d q)

```

```

*N-monol-≤ : (a b c : ℕ) → a ≤ b → a * c ≤ b * c
*N-monol-≤ .0      b c z ≤ n = z ≤ n
*N-monol-≤ (suc a') (suc b') c (s ≤ s p) = +N-monor-≤ c (a' * c) (b' * c) (*N-monol-≤ a' b' c p)

*N-monor-≤ : (a b c : ℕ) → b ≤ c → a * b ≤ a * c
*N-monor-≤ a b c p =
  subst (≤ a * c) (*N-comm b a)
  (subst ((b * a) ≤_) (*N-comm c a) (*N-monol-≤ b c a p))

*N-mono-≤ : (a b c d : ℕ) → a ≤ b → c ≤ d → a * c ≤ b * d
*N-mono-≤ a b c d p q = ≤-trans (*N-monol-≤ a b c p) (*N-monor-≤ b c d q)

sq-mono-≤ : (a b : ℕ) → a ≤ b → a * a ≤ b * b
sq-mono-≤ a b p = *N-mono-≤ a b a b p p

+N-cancelr : (a b c : ℕ) → a + c ≤ b + c → a ≤ b
+N-cancelr a b zero p =
  subst (≤ b) (+N-zero-r a) (subst (a + 0 ≤_) (+N-zero-r b) p)
+N-cancelr a b (suc c) p =
+N-cancelr a b c (lemma p)
where
  lemma : a + suc c ≤ b + suc c → a + c ≤ b + c
  lemma q with subst (≤ b + suc c) (+N-suc-r a c) q
  ... | q1 with subst (suc (a + c) ≤_) (+N-suc-r b c) q1
  ... | s ≤ s q2 = q2

m ≤ m+k : (m k : ℕ) → m ≤ m + k
m ≤ m+k zero k = z ≤ n
m ≤ m+k (suc m) k = s ≤ s (m ≤ m+k m k)

-- § No perfect square strictly between two consecutive squares.
sandwich-no-square :
(a m : ℕ) → a * a < m → m < suc a * suc a → ¬ IsPerfectSquare m
sandwich-no-square a m lo hi (k, m ≡ kk) with ℕ-tri a k
... | lt sa ≤ k =
let sa2 ≤ kk : suc a * suc a ≤ k * k
  sa2 ≤ kk = sq-mono-≤ (suc a) k sa ≤ k
  sa2 ≤ m : suc a * suc a ≤ m
  sa2 ≤ m = subst (suc a * suc a ≤_) (sym m ≡ kk) sa2 ≤ kk
in suc ≤-impossible m (≤-trans hi sa2 ≤ m)
... | same a ≡ k =
let aa ≡ m : a * a ≡ m
  aa ≡ m = trans (cong (λ x → x * x) a ≡ k) (sym m ≡ kk)
in suc ≤-impossible (a * a) (subst (suc (a * a) ≤_) (sym aa ≡ m) lo)
... | gt sk ≤ a =
let k ≤ a : k ≤ a
  k ≤ a = ≤-trans (≤-step k) sk ≤ a
  kk ≤ aa : k * k ≤ a * a
  kk ≤ aa = sq-mono-≤ k a k ≤ a
  m ≤ aa : m ≤ a * a
  m ≤ aa = subst (≤ a * a) (sym m ≡ kk) kk ≤ aa
in suc ≤-impossible (a * a) (≤-trans lo m ≤ aa)

```



```

-- § Foundational identity:  $2 \cdot E(n) + n \equiv n \cdot n$ .
private
*2 : (x : ℕ) → 2 * x ≡ x + x
*2 x = cong (x +_) (+N-zero-r x)

pair-pair : (x y : ℕ) → (x + x) + (y + y) ≡ (x + y) + (x + y)
pair-pair x y =
  trans (+N-assoc x x (y + y))
    (trans (cong (x +_) (sym (+N-assoc x y y)))
      (trans (cong (λ z → x + (z + y)) (+N-comm x y))
        (trans (cong (x +_) (+N-assoc y x y))
          (sym (+N-assoc x y (x + y)))))))

K-edge-identity : (n : ℕ) → 2 * K-edge-count n + n ≡ n * n
K-edge-identity zero = refl
K-edge-identity (suc m) = goal
where
  E : ℕ
  E = K-edge-count m
  ih : 2 * E + m ≡ m * m
  ih = K-edge-identity m
  A : 2 * (E + m) ≡ (E + E) + (m + m)
  A = trans (*2 (E + m)) (sym (pair-pair E m))
  B : 2 * (E + m) + suc m ≡ ((E + E) + (m + m)) + suc m
  B = cong (_+ suc m) A
  C : ((E + E) + (m + m)) + suc m ≡ (E + E) + ((m + m) + suc m)
  C = +N-assoc (E + E) (m + m) (suc m)
  D : (m + m) + suc m ≡ suc (m + (m + m))
  D = trans (+N-suc-r (m + m) m)
    (cong suc (trans (+N-assoc m m m) (cong (m +_) (+N-comm m m))))
  E1 : (E + E) + suc (m + (m + m)) ≡ suc ((E + E) + (m + (m + m)))
  E1 = +N-suc-r (E + E) (m + (m + m))
  E2 : (E + E) + (m + (m + m)) ≡ ((E + E) + m) + (m + m)
  E2 = sym (+N-assoc (E + E) m (m + m))
  F : (E + E) + m ≡ m * m
  F = trans (cong (_+ m) (sym (*2 E))) ih
  G : suc (((E + E) + m) + (m + m)) ≡ suc ((m * m) + (m + m))
  G = cong (λ z → suc (z + (m + m))) F
  H : suc ((m * m) + (m + m)) ≡ (m * m) + suc (m + m)
  H = sym (+N-suc-r (m * m) (m + m))
  I : suc m * suc m ≡ (m * m) + suc (m + m)
  I = trans
    (cong suc (trans (cong (m +_) (*N-suc-r m m))
      (trans (sym (+N-assoc m m (m * m)) (+N-comm (m + m) (m * m))))
      (sym (+N-suc-r (m * m) (m + m)))))
  goal : 2 * K-edge-count (suc m) + suc m ≡ suc m * suc m
  goal =
    trans B (trans C (trans (cong ((E + E) +_) D)
      (trans E1 (trans (cong suc E2) (trans G (trans H (sym I)))))))

-- § Sandwich identities and asymptotic bounds.
private
square-2n : (n : ℕ) → (2 * n) * (2 * n) ≡ 4 * (n * n)
square-2n n =

```

```

trans (*N-assoc 2 n (2 * n))
  (trans (cong (2 * _))
    (trans (sym (*N-assoc n 2 n))
      (trans (cong (_ n) (*N-comm n 2))
        (*N-assoc 2 n n))))
    (sym (*N-assoc 2 2 (n * n))))

expand-4 : (E n : ℕ) → 4 * (2 * E + n) ≡ 8 * E + 4 * n
expand-4 E n =
  trans (*N-distribl-+ 4 (2 * E) n)
    (cong (_ + 4 * n) (sym (*N-assoc 4 2 E)))

square-E+4 : (E : ℕ) → (E + 4) * (E + 4) ≡ E * E + (8 * E + 16)
square-E+4 E =
  trans (*N-distribr-+ E 4 (E + 4))
    (trans (cong2 _+_ (*N-distribl-+ E E 4) (*N-distribl-+ 4 E 4))
      (trans (+N-assoc (E * E) (E * 4) (4 * E + 4 * 4))
        (cong (E * E +_)
          (trans (sym (+N-assoc (E * 4) (4 * E) (4 * 4)))
            (cong (_ + 4 * 4)
              (trans (cong (_ + 4 * E) (*N-comm E 4))
                (sym (*N-distribr-+ 4 4 E))))))))))

square-E+5 : (E : ℕ) → (E + 5) * (E + 5) ≡ E * E + (10 * E + 25)
square-E+5 E =
  trans (*N-distribr-+ E 5 (E + 5))
    (trans (cong2 _+_ (*N-distribl-+ E E 5) (*N-distribl-+ 5 E 5))
      (trans (+N-assoc (E * E) (E * 5) (5 * E + 5 * 5))
        (cong (E * E +_)
          (trans (sym (+N-assoc (E * 5) (5 * E) (5 * 5)))
            (cong (_ + 5 * 5)
              (trans (cong (_ + 5 * E) (*N-comm E 5))
                (sym (*N-distribr-+ 5 5 E))))))))))

lower-sandwich-id :
  (n : ℕ) → K-pythagorean-sum n + 16 ≡ (K-edge-count n + 4) * (K-edge-count n + 4) + 4 * n
lower-sandwich-id n =
  let E = K-edge-count n
  s1 : (2 * n) * (2 * n) ≡ 4 * (n * n)
  s1 = square-2n n
  s2 : 4 * (n * n) ≡ 8 * E + 4 * n
  s2 = trans (cong (4 * _) (sym (K-edge-identity n))) (expand-4 E n)
  s3 : K-pythagorean-sum n ≡ E * E + (8 * E + 4 * n)
  s3 = cong (E * E +_) (trans s1 s2)
  s4 : K-pythagorean-sum n + 16 ≡ (E * E + (8 * E + 4 * n)) + 16
  s4 = cong (_ + 16) s3
  s5 : (E * E + (8 * E + 4 * n)) + 16 ≡ E * E + ((8 * E + 16) + 4 * n)
  s5 =
    trans (+N-assoc (E * E) (8 * E + 4 * n) 16)
      (cong (E * E +_)
        (trans (+N-assoc (8 * E) (4 * n) 16)
          (trans (cong (8 * E +_) (+N-comm (4 * n) 16))
            (sym (+N-assoc (8 * E) 16 (4 * n))))))
  s6 : (E + 4) * (E + 4) ≡ E * E + (8 * E + 16)

```

```

s6 = square-E+4 E
s7 : (E + 4) * (E + 4) + 4 * n ≡ (E * E + (8 * E + 16)) + 4 * n
s7 = cong (λ + 4 * n) s6
s8 : (E * E + (8 * E + 16)) + 4 * n ≡ E * E + ((8 * E + 16) + 4 * n)
s8 = +N-assoc (E * E) (8 * E + 16) (4 * n)
in trans s4 (trans s5 (sym (trans s7 s8)))

upper-sandwich-id :
(n : ℕ) → K-pythagorean-sum n + (n * n + 25) ≡ (K-edge-count n + 5) * (K-edge-count n + 5) + 5 * n
upper-sandwich-id n =
let E = K-edge-count n
s1 : K-pythagorean-sum n ≡ E * E + (8 * E + 4 * n)
s1 = cong (E * E + _)
      (trans (square-2n n)
        (trans (cong (4 * _) (sym (K-edge-identity n))) (expand-4 E n)))
s2 : K-pythagorean-sum n + (n * n + 25) ≡ (E * E + (8 * E + 4 * n)) + (n * n + 25)
s2 = cong (λ + (n * n + 25)) s1
t≡u : (8 * E + 4 * n) + (n * n + 25) ≡ (8 * E + 4 * n) + ((2 * E + n) + 25)
t≡u = cong (λ z → (8 * E + 4 * n) + (z + 25)) (sym (K-edge-identity n))
u≡v : (8 * E + 4 * n) + ((2 * E + n) + 25) ≡ (10 * E + 5 * n) + 25
u≡v =
trans (+N-assoc (8 * E) (4 * n) ((2 * E + n) + 25))
(trans (cong (8 * E + _)
  (trans (sym (+N-assoc (4 * n) (2 * E + n) 25))
    (cong (λ + 25)
      (trans (sym (+N-assoc (4 * n) (2 * E) n))
        (trans (cong (λ + n) (+N-comm (4 * n) (2 * E)))
          (+N-assoc (2 * E) (4 * n) n))))))
  (trans (sym (+N-assoc (8 * E) (2 * E + (4 * n + n) 25))
    (cong (λ + 25)
      (trans (sym (+N-assoc (8 * E) (2 * E) (4 * n + n)))
        (cong2 _+_
          (sym (*N-distribr-+ 8 2 E))
          (+N-comm (4 * n) n))))))
v≡w : (10 * E + 5 * n) + 25 ≡ (10 * E + 25) + 5 * n
v≡w =
trans (+N-assoc (10 * E) (5 * n) 25)
(trans (cong (10 * E + _) (+N-comm (5 * n) 25))
  (sym (+N-assoc (10 * E) 25 (5 * n))))
rearr : (8 * E + 4 * n) + (n * n + 25) ≡ (10 * E + 25) + 5 * n
rearr = trans t≡u (trans u≡v v≡w)
s3 : (E * E + (8 * E + 4 * n)) + (n * n + 25) ≡ (E * E + (10 * E + 25)) + 5 * n
s3 = trans (+N-assoc (E * E) (8 * E + 4 * n) (n * n + 25))
      (trans (cong (E * E + _) rearr)
        (sym (+N-assoc (E * E) (10 * E + 25) (5 * n))))
s4 : (E * E + (10 * E + 25)) + 5 * n ≡ (E + 5) * (E + 5) + 5 * n
s4 = cong (λ + 5 * n) (sym (square-E+5 E))
in trans s2 (trans s3 s4)

lower-sandwich :
(n : ℕ) → 5 ≤ n →
suc ((K-edge-count n + 4) * (K-edge-count n + 4)) ≤ K-pythagorean-sum n
lower-sandwich n p =
let E = K-edge-count n

```

```

Sq    = (E + 4) * (E + 4)
17 ≤ 4n : 17 ≤ 4 * n
17 ≤ 4n = ≤-trans (m ≤ m+k 17 3) (*N-monor-≤ 4 5 n p)
h1 : Sq + 17 ≤ Sq + 4 * n
h1 = +N-monor-≤ Sq 17 (4 * n) 17 ≤ 4n
h2 : suc (Sq + 16) ≤ Sq + 4 * n
h2 = subst (≤ Sq + 4 * n) (+N-suc-r Sq 16) h1
h3 : suc (Sq + 16) ≤ K-pythagorean-sum n + 16
h3 = subst (suc (Sq + 16) ≤) (sym (lower-sandwich-id n)) h2
h4 : suc Sq ≤ K-pythagorean-sum n
h4 = +N-cancelr (suc Sq) (K-pythagorean-sum n) 16 h3
in h4

5n+1 ≤ nn+25 : (n : ℕ) → 5 ≤ n → suc (5 * n) ≤ n * n + 25
5n+1 ≤ nn+25 n p =
let h1 : 5 * n ≤ n * n
    h1 = *N-monol-≤ 5 n n p
    h2 : suc (5 * n) ≤ suc (n * n)
    h2 = s ≤ s h1
    nn+1 ≡ suc-nn : n * n + 1 ≡ suc (n * n)
    nn+1 ≡ suc-nn = trans (+N-suc-r (n * n) 0) (cong suc (+N-zero-r (n * n)))
    h3 : suc (n * n) ≤ n * n + 25
    h3 = subst (≤ n * n + 25) nn+1 ≡ suc-nn
          (+N-monor-≤ (n * n) 1 25 (s ≤ s z ≤ n))
in ≤-trans h2 h3

upper-sandwich :
(n : ℕ) → 5 ≤ n →
suc (K-pythagorean-sum n) ≤ (K-edge-count n + 5) * (K-edge-count n + 5)
upper-sandwich n p =
let E    = K-edge-count n
    S    = K-pythagorean-sum n
    Sq5 = (E + 5) * (E + 5)
    aux : suc (5 * n) ≤ n * n + 25
    aux = 5n+1 ≤ nn+25 n p
    h1 : S + suc (5 * n) ≤ S + (n * n + 25)
    h1 = +N-monor-≤ S (suc (5 * n)) (n * n + 25) aux
    h2 : suc (S + 5 * n) ≤ S + (n * n + 25)
    h2 = subst (≤ S + (n * n + 25)) (+N-suc-r S (5 * n)) h1
    h3 : suc (S + 5 * n) ≤ Sq5 + 5 * n
    h3 = subst (suc (S + 5 * n) ≤) (upper-sandwich-id n) h2
    h4 : suc S ≤ Sq5
    h4 = +N-cancelr (suc S) Sq5 (5 * n) h3
in h4

-- § K-pyth-sum is not a perfect square for n ∈ {2, 3} (explicit witnesses).
not-pyth-K2 : ¬ IsPerfectSquare (K-pythagorean-sum 2)
not-pyth-K2 =
sandwich-no-square 4 (K-pythagorean-sum 2)
(subst (16 <_) (sym K2-not-pythagorean) (m ≤ m+k 17 0))
(subst (< 25) (sym K2-not-pythagorean) (m ≤ m+k 18 7))

not-pyth-K3 : ¬ IsPerfectSquare (K-pythagorean-sum 3)
not-pyth-K3 =

```

```

sandwich-no-square 6 (K-pythagorean-sum 3)
(subst (36 <_) (sym K3-not-pythagorean) (m ≤ m+k 37 8))
(subst (< 49) (sym K3-not-pythagorean) (m ≤ m+k 46 3))

-- § K-pyth-sum is not a perfect square for n ≥ 5 (asymptotic sandwich).
not-pyth-≥5 : (n : ℕ) → 5 ≤ n → ¬ IsPerfectSquare (K-pythagorean-sum n)
not-pyth-≥5 n p =
  sandwich-no-square (K-edge-count n + 4) (K-pythagorean-sum n)
  (lower-sandwich n p)
  (subst (suc (K-pythagorean-sum n) ≤_) eq (upper-sandwich n p))
where
  eq : (K-edge-count n + 5) * (K-edge-count n + 5)
    ≡ suc (K-edge-count n + 4) * suc (K-edge-count n + 4)
  eq = cong2 *_ (+N-suc-r (K-edge-count n) 4) (+N-suc-r (K-edge-count n) 4)

-- § Universal Pythagorean uniqueness theorem.
pythagoras-uniqueness :
  (n : ℕ) → 2 ≤ n → IsPerfectSquare (K-pythagorean-sum n) → n ≡ 4
pythagoras-uniqueness zero () _
pythagoras-uniqueness (suc zero) (s ≤ s ()) _
pythagoras-uniqueness (suc (suc zero)) _ w = ⊥-elim (not-pyth-K2 w)
pythagoras-uniqueness (suc (suc (suc zero))) _ w = ⊥-elim (not-pyth-K3 w)
pythagoras-uniqueness (suc (suc (suc (suc zero)))) _ = refl
pythagoras-uniqueness (suc (suc (suc (suc (suc m))))) _ w =
  ⊥-elim (not-pyth-≥5 (suc (suc (suc (suc (suc m)))))
    (s ≤ s (s ≤ s (s ≤ s (s ≤ s (s ≤ s z ≤ n))))) w)

```

This arithmetic proof is independent of the singleton argument, but it reaches the same conclusion: among complete graphs, only K_4 yields the required perfect-square identity. The end of the derivation is therefore not supported by a single route only. It is reached once by the invariant record and once by a separate numerical obstruction.

The Transcendental Constraint

The last structural statement returns to the point at which a formal theory becomes observational. The measurement kernel showed that binary measurement supplies a distinction. The record theorem showed that a distinction reaches the canonical invariant record. Composing the two results gives the final implication of the text.

A binary measurement apparatus provides two distinct outcomes and an exhaustive yes/no cover. That is already a distinction in its outcome space. The earlier chain then sends that distinction to the canonical K_4 Record. The map `measurement-forces-K4` is exactly this composition.

This is the correct place for the proof to stop. The result no longer concerns a new constant, a new orbit, or a new computation. It states that the minimal form of binary observation already carries the data from which the invariant record is obtained.

```
-- §§ The Transcendental Constraint: measurement forces  $K_4$ .
```

```

-- § Binary measurement forces the invariant record.
measurement-forces-K4 : BinaryMeasurement → K4Record
measurement-forces-K4 M = K4-is-inevitable (measurement-outcome-distinction M)

-- § The forced record returns the canonical distinction.
measurement-round-trip : (M : BinaryMeasurement) →
  record-presupposes-distinction (measurement-forces-K4 M) ≡ Two-distinction
measurement-round-trip M = refl

```

The round trip states that extracting a distinction from the forced record returns the canonical witness. No extra structure is added by the composition. The endpoint is not an expansion of the premise, but a recognition of what the premise already contains.

Chapter 64

Mechanical Closure of the Forced Formula

Most of the structural identifications attached to (V, E, d, χ) were discharged in place, at the section that made the claim. Three of them belong to the kind classification and the simplex evaluation themselves: the refinement-stable closure reading (§38), the boundary term inside the simplex evaluation (§39), and the uniqueness of the canonical bulk-plus-boundary form (§38). Each of these three stands as an Agda record at the section that produced it.

Two identifications had to wait. The Extract Concept and the bulk exponent both consume the $\text{Aut}(K_4)$ -orbit theorem of Chapter 62, which is proved later in the text. This chapter discharges those two and then assembles the five identifications into a single closing record.

The Aut-Orbit Witness for Extracts

The Extract Concept of §38 rests on the claim that $\text{Aut}(K_4)$ acts transitively on each cell stratum of the closed record. The orbit theorems of Chapter 62 discharge this claim by direct construction: `k4Perm-send` witnesses vertex transitivity, `edge-transitive` witnesses edge transitivity, and `matching-transitive` witnesses matching transitivity. The unique top cell is constant by triviality. The four cell strata of the closed 3-simplex are therefore all single $\text{Aut}(K_4)$ -orbits, and the cardinality reading defining Extract is the only $\text{Aut}(K_4)$ -invariant numerical reading possible.

The Extract Concept consumes a transitivity witness on each cell stratum. The record `ExtractAutWitness` below gathers the orbit theorems as the type-level discharge. Its inhabitant is `theorem-aut-k4-orbits`; nothing further is constructed. The Extract Concept is thereby the type-level reading of `theorem-aut-k4-orbits`.

```

record ExtractAutWitness : Set where
  field
    orbit-record : AutK4OrbitCounts

extract-aut-witness : ExtractAutWitness
extract-aut-witness = record { orbit-record = theorem-aut-k4-orbits }

```

The Bulk Exponent as a Mechanical Power

The isotropy paragraph of §38 fixed the bulk exponent at d by the orbit transitivity that forbids any direction of drift from being singled out. The orbit machinery used in that argument is exactly `theorem-aut-k4-orbits` of Chapter 62. On the simplex side the bulk reading is mechanical: the bulk term is `simplex-vertices ^ simplex-degree * simplex-chi`, equal to 128 by computation. Combined with the boundary term `p-dd`, recorded earlier as `boundary-arity-witness`, it yields the simplex evaluation.

The bulk exponent must be the $\text{Aut}(K_4)$ -invariant exponent, and the bulk term must combine with the boundary term to yield `simplex-eval`. The record `BulkExponentWitness` below collects the orbit record, which is the structural source of isotropy, and the value of the bulk term. Its inhabitant is constructed from existing laws. The bulk exponent d is the unique exponent the orbit record permits, and the bulk term it produces is the one the simplex evaluation realises.

```

record BulkExponentWitness : Set where
  field
    aut-source      : AutK4OrbitCounts
    bulk-equals-128 :
      (simplex-vertices ^ simplex-degree) * simplex-chi ≡ 128

bulk-exponent-witness : BulkExponentWitness
bulk-exponent-witness = record
{ aut-source      = theorem-aut-k4-orbits
; bulk-equals-128 = refl
}

```

The Forced-Formula Witness

The five identifications — closure reading, boundary arity, form uniqueness, extract source, bulk exponent — are now all inhabited as Agda records. A single closing record collects them.

The chain of structural identifications must, at the end, inhabit a single record collecting all five. `ForcedFormulaWitness` below assembles `SelfDualClosureWitness` and `BoundaryArityWitness` (both built earlier with the simplex constants),

FormUniquenessWitness (built with the kind classification), ExtractAutWitness and BulkExponentWitness (the two records of this chapter), and the simplex evaluation itself.

Every structural attribute attached to (V, E, d, χ) in this volume is a field of one record. The canonical form $V^d \cdot \chi + d^2$ is its unique inhabitant, and its numerical value is the simplex evaluation.

```

record ForcedFormulaWitness : Set where
  field
    closure-witness      : SelfDualClosureWitness
    boundary-witness     : BoundaryArityWitness
    uniqueness-witness   : FormUniquenessWitness
    extract-witness      : ExtractAutWitness
    bulk-witness         : BulkExponentWitness
    eval-equals-137     : simplex-eval  $\equiv$  137

forced-formula-witness : ForcedFormulaWitness
forced-formula-witness = record
{ closure-witness      = self-dual-closure-witness
; boundary-witness     = boundary-arity-witness
; uniqueness-witness   = form-uniqueness-witness
; extract-witness      = extract-aut-witness
; bulk-witness         = bulk-exponent-witness
; eval-equals-137     = law15A-0-simplex-eval-137
}

-- § Audit closure: the six audit items of §sec:invariant-kinds:audit
-- as type-level fields, each inhabited by a witness already in scope.
-- No item below is closed by enumeration; each is closed by a single
-- structural witness.
record AuditClosure : Set where
  field
    item1-which-invariants :
      ExtractAutWitness
    item2-arithmetic-by-kind :
      ( $\forall \{k\} (x\ y : \text{Tagged } k) \rightarrow \text{value } (x \otimes y) \equiv \text{value } x * \mathbb{N} \text{ value } y)$ 
       $\times (\forall \{j\} (x : \text{Tagged } j) (p : \neg (j \equiv k)) (y : \text{Tagged } k)$ 
       $\rightarrow (x \oplus [p] y) \equiv \text{value } x + \mathbb{N} \text{ value } y)$ 
    item3-bulk-vs-boundary :
      BulkBoundaryIndependence
    item4-bulk-exponent :
      BulkExponentWitness
    item5-boundary-exponent :
      BoundaryArityWitness
    item6-relational-via-chi :
      simplex-chi  $\equiv$  2

audit-closure : AuditClosure
audit-closure = record
{ item1-which-invariants      = extract-aut-witness
; item2-arithmetic-by-kind   = forget- $\otimes$  , forget- $\oplus$ 
; item3-bulk-vs-boundary     = theorem-bulk-boundary-independent

```

```
; item4-bulk-exponent      = bulk-exponent-witness  
; item5-boundary-exponent = boundary-arity-witness  
; item6-relational-via-chi = law14C-3-chi  
}
```

Chapter 65

Conclusion

The Arc

A Distinction record was supplied as the formal datum. Nothing else entered. From that datum the chain unfolded in a fixed order, and the order was not a sequence of choices but a sequence of survivals.

The two-point base entered as the unique normal form for binary distinction. The endomorphism space of that base contained exactly four cases. Three vertices failed by absurd-pattern elimination; more than four vertices added structure unsupported by the four cases. The closure that survived was the complete graph on four vertices, K_4 . Its representation record was inhabited, propositionally unique, and stable under the orbit tests of Part VI. The final theorem of that part proved that any inhabitant of the record already carries the original separation witness. Distinction entered as the minimal grant; distinction re-emerged as a consequence. The arrow did not extend; it closed.

The Survivor

What survives at the endpoint is not a structure that was selected from a catalogue of possibilities. It is the only inhabitant that did not collapse under the eliminative pressure of the preceding chapters. The numerical data attached to the survivor—the vertex count, the Laplacian spectrum, the Euler characteristic, the simplex value 137, the loop numerator 11—are determined by reduction. None of them carries a free parameter, and none of them was chosen.

The Identification

The last chapters added a fact that the rest of the book had only implicitly relied on. The four basic invariants (V, E, d, χ) are not merely numbers attached to the survivor. They are identifiable *inside the system that forced them*. Each one carries a kind (configurational, relational, directional, topological), a source (an $\text{Aut}(K_4)$ -orbit class), a rule for combining with another invariant (product on shared kind, sum on disjoint kinds, both reducing by `refl` through the forgetful map), a position relative to the bulk–boundary split, and a role inside the closed record. Each of these attributes is discharged by an Agda record whose inhabitant is constructed from existing lemmas: the closure record at the close of §38, the form-uniqueness record at the close of §38, the boundary-arity record at the close of §39, and the two Aut -dependent records of Chapter 64. Any combination built from (V, E, d, χ) is therefore answerable to five structural obligations before its numerical value is even read. This is the structural acquisition that the book now carries beyond its target value.

The Boundary

The boundary of the proof is equally definite. No dynamics is derived here, no empirical constant is fixed by measurement, no continuum theory is supplied by the formal apparatus. The claim is narrower and therefore harder to attack: under the stated minimal grant, the discrete record attached to K_4 is the unique closed endpoint of the elimination chain, the chain runs from distinction back to distinction without any external addition, and the four basic invariants of that record are identified inside the system by kind, source, arithmetic, position, and role.

For that reason the conclusion is not another construction. The internal loops have closed. The two-point base is forced as the normal form; the four cases are forced by coverage; their closure is forced to K_4 ; the invariant record is forced as a singleton; the four basic invariants of that record are identified inside the system; and the singleton presupposes the original distinction. The discrete argument has reached its boundary.

This completes the proof of the discrete kernel. It does not complete the theory of formulability; it supplies the part of that theory that can be submitted to the type checker. The larger claim begins before the code, where formulability is identified with the need for distinction, and continues after the code, where the invariant ledger is read by *Form*. The present volume closes the middle term: the formal chain from represented distinction to invariant closure and back.

Outlook

What the closed kernel makes available

The discrete kernel is closed; the natural continuation does not repeat the elimination. It asks what can be carried by the kernel once it is accepted as fixed data. Any such continuation must respect the constraints that have been proved: the singleton record, the automorphism invariants, the unit scale, the measurement kernel, and the arithmetic closure of the correction ledger. With the structural identification of the four basic invariants now in place, a continuation also inherits a stricter admission test for any formula it wishes to build from them. A candidate combination is admissible only if its kind, source, operation, position, and exponent agree with the obligations the closed record carries. These constraints do not by themselves determine a theory of dynamics or an empirical interpretation. They determine the discrete ground on which such a theory would have to stand, and the structural shape of the formulas that ground would permit.

This is why the claim of the larger work is a theory of formulability rather than a completed physics. The present volume fixes the formal skeleton that any such reading must respect. The later volume may attach physical vocabulary, compare observables, and expose failure conditions; it may not retroactively change the kernel it inherits.

Two boundaries of the universal claim

Two boundaries are worth naming explicitly, because they limit how far the universal claim of this book may legitimately be carried. First, the normal-form theorem quantifies internally over every type in the universe of the formal setting, not over the family of all formal theories; the latter is a meta-theoretic object whose universal closure is excluded by Tarski's undefinability of truth and by Gödel's incompleteness. Second, the present formalisation lives at a single universe level; a routine universe-polymorphic restatement is available but adds no new mathematical content. The universal reading therefore holds inside the formal setting and stops at its boundary.

The formulability claim crosses that boundary only as a thesis about formulability, not as a theorem ranging over all possible foundations. Its force comes from the fact that every formalism capable of statement must already separate marks, rules, judgements, and failures; its formal weight comes from the Agda chain carried out here for one disciplined carrier. A reconstruction in another foundation would not weaken this claim. It would test and sharpen it.

Where the text ends

Within those boundaries, the present text ends where its own task ends: with the closed discrete record, the universal normal-form theorem at its entrance, the proof that the record cannot be detached from distinction, and the structural identification of its four basic invariants inside the system that forced them. What lies past this boundary is the subject of a separate volume; it begins only after the present chain has closed.